

AI in Finance

Finance for Engineers, Scientists, and Data Scientists

W. Suo

Draft Manuscript • October 12, 2026

© Wulin Suo

Copyright © 2026 Wulin Suo. All rights reserved.
This is a draft manuscript. No part of this work may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in a review.
Contact: Wulin.Suo@Queensu.ca

Wulin.Suo@Queensu.ca

Contents

Preface **ix**

Notation **xiii**

1 Introduction to Finance and AI **1**
1.1 Why AI in Finance . . . 1
1.2 The Financial Landscape . . . 3
1.3 What This Book Covers . . . 9
1.4 Reading Finance Beyond the Numbers . . . 11
1.5 Reasoning Under Uncertainty . . . 14
1.6 Limits, Challenges, and Practice . . . 19
1.7 Key Takeaways . . . 20
1.8 Suggested Exercises . . . 20
1.9 Further Reading . . . 21

2 Python for Financial Analysis **23**
2.1 Introduction . . . 23
2.2 The Python Language . . . 23
2.3 Numerical Computing with NumPy . . . 27
2.4 Handling Data with pandas . . . 28
2.5 Portfolio Analysis in Python . . . 33
2.6 Code That Holds Up . . . 36
2.7 Challenges in Applying Python to Financial Data . . . 40
2.8 Key Takeaways . . . 41
2.9 Suggested Exercises . . . 41
2.10 Further Reading . . . 42

3 Financial Markets, Institutions, and Instruments **43**
3.1 Introduction . . . 43
3.2 The Market and Its Participants . . . 43
3.3 Market Microstructure . . . 48
3.4 Market Infrastructure and Regulation . . . 51
3.5 Liquidity, Leverage, and Arbitrage . . . 55
3.6 Fragility and Market Design . . . 58
3.7 Challenges in Modeling Market Structure . . . 60
3.8 Key Takeaways . . . 60
3.9 Suggested Exercises . . . 61
3.10 Further Reading . . . 62

4 Financial Statements and Business Basics **63**
4.1 Introduction . . . 63
4.2 Why Statements Matter, and a Running Example . . . 63
4.3 The Three Statements . . . 64
4.4 Ratio Analysis . . . 67

Wulin.Suo@Queensu.ca

4.5 Comparison and Common-Size Analysis . . . 72
4.6 From Statements to Forecasts . . . 75
4.7 Challenges in Financial Statement Analysis . . . 78
4.8 Key Takeaways . . . 79
4.9 Suggested Exercises . . . 79
4.10 Further Reading . . . 80

5 Valuation and Asset Pricing 81
5.1 Introduction . . . 81
5.2 Discounting and Fixed Income . . . 81
5.3 Valuing Equity . . . 85
5.4 Option Pricing . . . 89
5.5 Putting It Together: Valuing Meridian Fabrication . . . 98
5.6 Challenges in Valuation and Asset Pricing . . . 100
5.7 Key Takeaways . . . 101
5.8 Suggested Exercises . . . 101
5.9 Further Reading . . . 103

6 Machine Learning Fundamentals 105
6.1 Introduction . . . 105
6.2 What Machine Learning Is . . . 105
6.3 Core Supervised Models . . . 106
6.4 Generalization and Model Selection . . . 113
6.5 Working with Financial Data . . . 117
6.6 Beyond Linear Models . . . 119
6.7 Challenges of Machine Learning in Finance . . . 123
6.8 Key Takeaways . . . 125
6.9 Suggested Exercises . . . 125
6.10 Further Reading . . . 126

7 Time Series Analysis and Forecasting 129
7.1 Introduction . . . 129
7.2 Foundations of Time Series . . . 129
7.3 Forecasting Models . . . 132
7.4 Multivariate Methods . . . 137
7.5 Volatility and State-Space Models . . . 141
7.6 Validation and Accuracy . . . 143
7.7 Challenges in Time Series Forecasting for Finance . . . 145
7.8 A Practical Workflow for Building a Time Series Model . . . 146
7.9 Key Takeaways . . . 147
7.10 Suggested Exercises . . . 147
7.11 Further Reading . . . 149

8 Market Risk Management 151
8.1 Introduction . . . 151
8.2 Measuring Market Risk . . . 151
8.3 Validating and Extending the Measures . . . 157
8.4 Stress Testing and Machine Learning . . . 164
8.5 Governance and Managing the Risk . . . 167
8.6 Case Vignette: When Value at Risk Meets a Genuine Crisis . . . 170
8.7 Challenges in Market Risk Management . . . 171
8.8 Key Takeaways . . . 172
8.9 Suggested Exercises . . . 173
8.10 Further Reading . . . 175

Wulin.Suo@Queensu.ca

9 Credit Risk Modeling 177
9.1 Introduction . . . 177
9.2 Measuring Credit Risk . . . 177
9.3 Modeling Default . . . 183
9.4 Portfolio Capital, Machine Learning, and Governance . . . 189
9.5 Case Vignette: The Gaussian Copula and the Mismodeling of CDO Credit Risk . . . 195
9.6 Challenges in Credit Risk Modeling . . . 195
9.7 Key Takeaways . . . 197
9.8 Suggested Exercises . . . 197
9.9 Further Reading . . . 200

10 Operational Risk Modeling 201
10.1 Introduction . . . 201
10.2 What Operational Risk Is, and How It Is Measured . . . 202
10.3 Where Artificial Intelligence Enters . . . 205
10.4 From Detection to Governance . . . 209
10.5 Case Vignette: The 2012 JPMorgan "London Whale" . . . 213
10.6 Challenges in Operational Risk Modeling . . . 213
10.7 Key Takeaways . . . 214
10.8 Suggested Exercises . . . 215
10.9 Further Reading . . . 216

11 Insurance and Actuarial Modeling 217
11.1 Introduction . . . 217
11.2 The Economics of Insurance . . . 218
11.3 Pricing and Claims . . . 220
11.4 Reserving, Life, Health, and Assets . . . 224
11.5 Capital, Catastrophe, and Fairness . . . 226
11.6 Case Vignette: Progressive's Snapshot and the Telematics Market . . . 229
11.7 Challenges in Insurance and Actuarial Modeling . . . 229
11.8 Key Takeaways . . . 230
11.9 Suggested Exercises . . . 231
11.10 Further Reading . . . 232

12 Portfolio Theory and Asset Allocation 233
12.1 Introduction . . . 233
12.2 Diversification and the Efficient Frontier . . . 234
12.3 Equilibrium Models and Factor Approaches . . . 236
12.4 Building and Running a Portfolio . . . 243
12.5 Challenges in Portfolio Theory and Asset Allocation . . . 251
12.6 Key Takeaways . . . 253
12.7 Suggested Exercises . . . 254
12.8 Further Reading . . . 255

13 Algorithmic Trading and Market Microstructure 257
13.1 Introduction . . . 257
13.2 Trading Signals and a First Strategy . . . 257
13.3 Execution and Market Impact . . . 259
13.4 Inventory Models and Market Making . . . 264
13.5 Arbitrage and Relative-Value Strategies . . . 265
13.6 Evaluating a Strategy . . . 267
13.7 Machine Learning and Event-Driven Signals . . . 269
13.8 Market Structure and Safeguards . . . 273
13.9 Challenges in Algorithmic Trading and Market Microstructure . . . 274
13.10 Key Takeaways . . . 276

Wulin.Suo@Queensu.ca

13.11 Suggested Exercises . . . 276
13.12 Further Reading . . . 278

14 High-Frequency Trading 279
14.1 Introduction . . . 279
14.2 The High-Frequency Landscape . . . 279
14.3 Latency Arbitrage and Informed Trading . . . 281
14.4 High-Frequency Market Making . . . 284
14.5 The Economics of Speed . . . 285
14.6 Market Structure and Market Quality . . . 288
14.7 Prediction and the Decision Pipeline . . . 290
14.8 Risk Controls and Systemic Episodes . . . 292
14.9 The Regulatory Debate: Does HFT Help or Hurt Markets? . . . 293
14.10 Challenges in High-Frequency Trading . . . 294
14.11 Key Takeaways . . . 295
14.12 Suggested Exercises . . . 295
14.13 Further Reading . . . 297

15 Fraud Detection, Compliance, and Financial Crime 299
15.1 Introduction . . . 299
15.2 Detecting Fraud . . . 299
15.3 Risk Tiering, Survival, and Money Laundering . . . 307
15.4 Economics and Governance . . . 315
15.5 Case Vignette: The Danske Bank Estonia Case . . . 317
15.6 Challenges in Fraud Detection, Compliance, and Financial Crime . . . 318
15.7 Key Takeaways . . . 319
15.8 Suggested Exercises . . . 319
15.9 Further Reading . . . 321

16 Natural Language Processing in Finance 323
16.1 Introduction . . . 323
16.2 Text as Data . . . 324
16.3 Classical Text Models . . . 327
16.4 Transformers and Large Language Models . . . 330
16.5 Generative AI in Practice . . . 337
16.6 Case Vignette: AI @ Morgan Stanley . . . 343
16.7 Challenges in Natural Language Processing in Finance . . . 343
16.8 Key Takeaways . . . 344
16.9 Suggested Exercises . . . 345
16.10 Further Reading . . . 347

17 Alternative Data and Deep Learning in Finance 349
17.1 Introduction . . . 349
17.2 The Alternative-Data Landscape . . . 350
17.3 Working with Alternative Data . . . 353
17.4 Applications and Advanced Methods . . . 361
17.5 Case Vignette: Satellite Imagery and the Rise of Alternative Data . . . 365
17.6 Challenges in Alternative Data and Deep Learning in Finance . . . 365
17.7 Key Takeaways . . . 366
17.8 Suggested Exercises . . . 367
17.9 Further Reading . . . 369

18 Explainability, Fairness, and Regulation 371
18.1 Introduction . . . 371
18.2 Why Explanation Matters . . . 372

Wulin.Suo@Queensu.ca

18.3 Explanation Methods . . . 373
18.4 Regulation, Fairness, and Robustness . . . 377
18.5 Governance and Advanced Topics . . . 381
18.6 Challenges in Explainability, Fairness, and Regulation . . . 385
18.7 Key Takeaways . . . 386
18.8 Suggested Exercises . . . 386
18.9 Further Reading . . . 388

19 Reinforcement Learning in Finance 391
19.1 Introduction . . . 391
19.2 The Reinforcement Learning Framework . . . 392
19.3 Methods . . . 397
19.4 Financial Applications . . . 401
19.5 From Simulation to Deployment . . . 408
19.6 Challenges in Reinforcement Learning for Finance . . . 411
19.7 Key Takeaways . . . 412
19.8 Suggested Exercises . . . 413
19.9 Further Reading . . . 414

20 Agent-Based Models in Finance 417
20.1 Introduction . . . 417
20.2 Why Agent-Based Models . . . 419
20.3 Emergence and Systemic Risk . . . 424
20.4 Building and Validating a Model . . . 429
20.5 Challenges in Agent-Based Modeling for Finance . . . 432
20.6 Key Takeaways . . . 433
20.7 Suggested Exercises . . . 434
20.8 Further Reading . . . 435

21 Case Studies and Capstone Projects 437
21.1 Introduction . . . 437
21.2 The Data . . . 438
21.3 Modeling and Evaluation . . . 440
21.4 From Model to Project . . . 446
21.5 Discussion Questions . . . 451
21.6 Challenges in End-to-End Financial AI Projects . . . 452
21.7 Key Takeaways . . . 453
21.8 Suggested Exercises . . . 454
21.9 Further Reading . . . 455

Appendix A Datasets and APIs for Financial AI 457
A.1 Introduction . . . 457
A.2 Market Price and Trading Data . . . 457
A.3 Options and Derivatives Data . . . 458
A.4 Fundamental and Financial Statement Data . . . 458
A.5 Macroeconomic and Interest Rate Data . . . 458
A.6 Credit, Lending, Insurance, and Consumer Finance Data . . . 458
A.7 Fraud, AML, and Regulatory Compliance Data . . . 463
A.8 Text, News, and Sentiment Data . . . 463
A.9 Alternative Data . . . 463
A.10 High-Frequency and Market Microstructure Data . . . 463
A.11 Factor, Index, and Portfolio Data . . . 463
A.12 Putting It Together: A Chapter-by-Chapter Dataset Map . . . 463
A.13 Practical Challenges in Sourcing Financial Data . . . 468
A.14 Further Reading . . . 468

Wulin.Suo@Queensu.ca

Appendix B Glossary of Finance Terms **469**

Appendix C A Career Map: Roles, Chapters, and Preparation **479**
C.1 Reading the Map 479
C.2 Preparing, Concretely 481

Bibliography **483**

Index **493**

Wulin.Suo@Queensu.ca

Preface

Artificial intelligence is now embedded in nearly every corner of the financial industry: credit decisions, fraud alerts, trade execution, risk limits, portfolio construction, and increasingly the generative tools analysts use to read a filing or draft a note. The people building and overseeing these systems are disproportionately trained in mathematics, statistics, physics, computer science, and engineering, not finance. They can derive a gradient, debug a training loop, and reason about a covariance matrix without much trouble. What they often lack, at least at first, is the financial vocabulary and institutional context that turns a model into something a bank, a fund, or a regulator can actually use, and use responsibly. This book is written to close that gap.

Who This Book Is For

I have written this book for readers who are already comfortable with quantitative reasoning and, ideally, with basic Python, but who have limited exposure to finance and economics: graduate students and advanced undergraduates in engineering, the physical sciences, statistics, and computer science; practicing data scientists and machine learning engineers moving into a financial role; and anyone building a career on the specific combination of skills this book covers. No prior finance coursework is assumed. Chapter 3 introduces financial markets and institutions from first principles, and every subsequent chapter builds forward from there, so a reader with no finance background at all should be able to start at Chapter 1 and never feel lost.

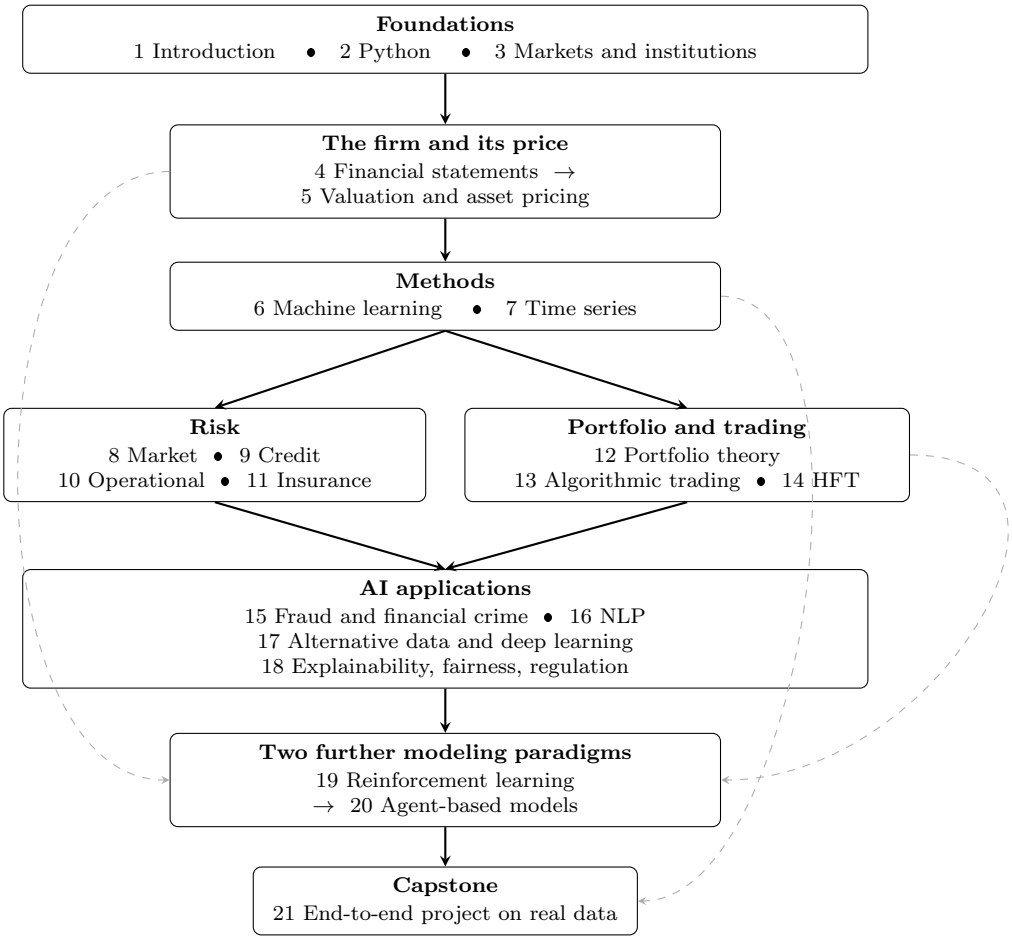
How This Book Is Organized

The twenty-one chapters are ordered by dependency, not by alphabetical convenience or topical grouping. Chapter 2 teaches the Python tools, NumPy, pandas, scikit-learn, matplotlib, that nearly every later chapter's worked examples rely on. Chapter 3's markets and institutions precede Chapter 4's financial statements, and Chapter 4's statements feed directly into Chapter 5's valuation and asset-pricing models, which in turn recur throughout the risk, portfolio, and credit chapters that follow. The four risk chapters themselves run as a deliberate sequence: market risk (Chapter 8) models how much a position's value can move, credit risk (Chapter 9) whether a counterparty will pay at all, and operational risk (Chapter 10) the losses that remain when the institution's own processes, people, or systems fail, each defined by exactly what the one before it leaves out; insurance (Chapter 11) then closes the sequence from the other side of the table, the industry whose entire product is bearing those risks for a price, and whose actuarial machinery the operational-risk chapter has already borrowed. Machine learning fundamentals (Chapter 6) are introduced once, in depth, and then reused, not re-taught, in every later chapter that needs a regression, a classifier, or a neural network: fraud scoring, credit scoring, algorithmic trading signals, natural language processing, and the explainability methods of Chapter 18 all build on that same foundation rather than each inventing their own. Chapter 19 closes the methods sequence proper with reinforcement learning, built on the trading and portfolio machinery of Chapters 12 through 14; Chapter 20 then turns to a third modeling paradigm, agent-based models of markets and financial networks, built on that same statistical and portfolio machinery together with Chapter 19's own sequential-decision framing, immediately before Chapter 21's capstone puts every method in the book to work on one real dataset. This ordering was arrived at, in part, by actually

Wulin.Suo@Queensu.ca

running the companion notebooks end to end and discovering where an earlier draft asked a chapter to use a tool it had not yet been taught. The diagram below draws that structure.

One convention holds throughout and is stated here rather than repeated in every chapter: every numerical claim in this book was computed in Python before it was written into the text, never derived by hand and checked afterward, and each is reproduced exactly in that chapter's companion notebook. Where a figure is instead quoted from published research, it is cited rather than recomputed. Readers who want to verify a number, or change an assumption and watch what happens to it, should reach for the notebook rather than re-deriving the arithmetic.



How the chapters depend on one another. Solid arrows are prerequisites. The three dashed arrows mark the places where a later group reuses an earlier one's machinery rather than rebuilding it: Chapter 5's option pricing returns in Chapters 8, 9, 12, and 19; Chapter 6's models are reused by Chapter 9 and Chapters 15 through 18 rather than retaught; and Chapter 13's execution model returns in Chapter 19. Chapter 2 underpins every companion notebook in the book and Chapter 21 draws on all of it, so neither is drawn with an arrow to each dependent chapter. An instructor short of sessions can cut most safely within a group rather than across the spine.

Beneath this chapter-by-chapter dependency, the book falls into three broader parts. Chapters 1 through 7 are preparation: the finance vocabulary, institutions, financial statements, and valuation machinery a reader needs before any model can be usefully applied, together with the Python and machine learning fundamentals every later chapter draws on. Chapters 8 through 17 cover the applications a working quantitative finance professional encounters most often: market, credit, operational, and insurance risk, portfolio construction, algorithmic and high-frequency trading, fraud detection, and the natural language processing and alternative-data methods reshaping how firms extract signal from unstructured sources. Chapters 18 through 21 turn to more advanced and specialized territory: explainability, fairness, and regulation; reinforcement learning;

Wulin.Suo@Queensu.ca

agent-based models of markets and financial networks; and a closing capstone that applies the entire toolkit to one real, publicly available dataset end to end. A reader pressed for time could stop after Part II and still have a working, practitioner-level command of AI in finance; Part III is where the book pushes into territory still actively being worked out in research and in practice.

Every chapter follows the same shape: an introduction, several sections of exposition built around at least one fully worked numerical example a reader can check by hand, a discussion of the practical challenges and limitations specific to that chapter's topic, and a closing sequence of Key Takeaways, Suggested Exercises, and Further Reading pointing to the standard textbooks and original papers behind the chapter's material. Most chapters' Suggested Exercises close with a challenge question that asks the reader to redo one of the chapter's own methods on a real, freely obtainable dataset, from sources such as the UCI Machine Learning Repository, Kaggle, SEC EDGAR and the SEC's enforcement releases, or the Consumer Financial Protection Bureau's HMDA and consumer-complaint databases, or, in the handful of cases where no clean real dataset exists (a limit order book fast enough to show latency arbitrage, say), on data generated by a documented simulation instead; the point of each is to show what a real-world version of the chapter's toy example actually looks like, messy edges included.

Each chapter is sized for a single 80-to-90-minute class session, though the chapters are not all the same length. Chapters 6, 13, 16, and 19 carry noticeably more worked material than the rest, Chapter 16 most of all, its generative-AI coverage being deliberately deeper than that of any other single topic here. An instructor who works through every worked example in those four should plan two sessions for each, or else select among their later sections. Chapter 10 is the deliberate opposite, shorter than the rest by design, and Chapter 11 is also on the short side; both leave room within a single session for exercises, discussion, or an early start on the chapter that follows. The book is designed as the backbone of a one-semester course meeting twice a week: at roughly one chapter per session, and about twenty-five sessions once those four longer chapters are given two apiece, the twenty-one chapters fit into such a semester with room left over for review, examinations, and project presentations, and an instructor pressed for sessions can be selective, most naturally among the more specialized chapters (10, 11, 14, 15, 17, 19, and 20), without breaking the dependency chain described above. A course meeting once a week can work from a core of about fourteen chapters, Chapters 1 through 9, then 12, 13, 16, 18, and 21, with the rest assigned as independent reading or term-project material. And the book serves a reader working through it independently at their own pace exactly as well. Three appendices follow the twenty-one chapters: a catalog of real financial datasets and APIs for readers who want to extend the book's examples with real data of their own, a glossary of finance and trading terms for readers who hit an unfamiliar term before the chapter that formally introduces it, and a career map connecting the industry's actual quantitative roles to the chapters that prepare for each, for the reader this book was written for in the first place.

Real, named case studies are woven throughout rather than confined to a single chapter: the insurer insolvencies after Hurricane Andrew in 1992 that created the catastrophe-modeling industry, the 1998 collapse of Long-Term Capital Management, the 2008 financial crisis and the Gaussian copula's role in mispricing CDO risk, the 2008 Société Générale rogue-trading loss, the 2010 Flash Crash, the 2012 Knight Capital trading malfunction, the 2012 JPMorgan "London Whale" loss, the Danske Bank Estonia money-laundering scandal, the 2019 Apple Card gender-bias controversy, and the SEC's 2022 settlement with Schwab over undisclosed robo-advisor cash allocations, among others. These are real events with real consequences, included because the lesson a formula teaches in the abstract is rarely the lesson a genuine failure teaches in practice. Chapter 21's capstone case study goes further still, working a real, publicly available consumer-credit dataset end to end, undocumented category codes, class imbalance, model comparison, calibration, explainability, and fairness auditing included, using nothing invented for the occasion.

The Companion Notebooks

Every worked numerical example in this book has a companion Jupyter notebook that reproduces it in Python, one notebook per chapter, using ordinary, widely available libraries rather than anything exotic. Every notebook was actually executed, and its printed output, not a hand-derived approximation of it, is what the chapter text reports; where the two disagreed during drafting, the notebook's output was treated as the source of truth and the prose was corrected to match. The companion notebooks, along with the

Wulin.Suo@Queensu.ca

LaTeX source for this book itself, are available at:

`https://github.com/wulinsuo/AI-in-Finance`

I encourage readers to run each notebook alongside the corresponding chapter, change an input or two, and see what happens; a formula becomes far more trustworthy once you have watched it respond correctly to a change you made yourself.

Technical Notes

A few of the formulas used in this book are quoted without a full derivation, since working through every one of them line by line in the main text would bury the finance and the intuition, which is this book's actual subject, under mathematics that is not. Where a derivation is long enough to be worth doing properly but not essential to following the chapter that uses the result, I have written it up separately as a standalone technical note and placed it in the `technical notes` folder of the same GitHub repository as the companion notebooks, linked above.

These notes are entirely optional. They exist for the reader who wonders where a particular formula actually comes from and wants to see the argument worked out in full; skipping them has no effect on understanding any of the approaches covered in this book, since every chapter states the results it needs and shows how to use them regardless of whether the underlying derivation has been read. More notes will be added over time as new formulas in the book warrant one.

A Note on the Draft

This is a work in progress, and the watermark on every page is there to say so honestly rather than to discourage reading. I am continuing to expand, correct, and refine the material, and I welcome corrections, questions, and suggestions from readers at the address below. If you find an error, a stale cross-reference, or a passage that would benefit from a clearer example, I would genuinely like to hear about it.

Wulin Suo
Smith School of Business
Queen's University
Wulin.Suo@Queensu.ca

Wulin.Suo@Queensu.ca

Notation

Symbols in this book follow each field's own conventions, which means a few letters carry different meanings in different chapters. The table below lists the recurring symbols and the principal uses a reader will encounter, with the chapters where each use appears; notation used only locally is defined where it appears. Typographic conventions, hats for estimates, time subscripts, units and annualization, are collected in Chapter 1's "Notation and Conventions" section.

Symbol	Principal uses
r, r_f	An interest rate or an asset's return; the risk-free rate (Chapters 3, 5, 12)
μ	Expected return or drift (Chapters 5, 8, 12)
σ, σ_t^2	Volatility of returns (Chapters 5, 8, 12); conditional variance in GARCH models (Chapter 7)
β	Regression coefficient (Chapter 6); CAPM market beta (Chapters 5, 12)
α	Significance level (Chapter 6); ARCH coefficient (Chapter 7); learning rate (Chapter 19); return in excess of a benchmark (Chapters 12, 13)
λ	Kyle's lambda price impact (Chapters 13, 14); Poisson intensity (Chapters 10, 11, 14); regularization strength (Chapter 6)
ρ	Correlation (Chapters 2, 8, 12); a convex risk measure (Chapter 19)
Δ, Γ	Option delta and gamma (Chapters 5, 8, 19); ΔX_t, a first difference (Chapters 1, 7)
q	Risk-neutral probability (Chapter 5)
$N(\cdot)$, $\phi(\cdot)$	Standard normal cumulative distribution function and density (Chapters 5, 8); ϕ alone, the Almgren-Chriss risk-aversion parameter (Chapters 13, 19)
Z, z	Standard normal variate or z-score (Chapters 6, 8, 15); Altman Z-score (Chapter 4); Bühlmann credibility weight (Chapter 11)
ε	Regression error term (Chapters 6, 7); exploration rate in ε-greedy learning (Chapter 19)
π	A decision policy (Chapter 19)
$\mathbb{E}[\cdot]$	Expectation
PD, LGD, EAD	Probability of default, loss given default, exposure at default (Chapter 9)
VaR, ES	Value at Risk and Expected Shortfall (Chapter 8)

Wulin.Suo@Queensu.ca

Wulin.Suo@Queensu.ca

Chapter 1

Introduction to Finance and AI

1.1 Why AI in Finance

Finance has always been a quantitative discipline, but the questions it asks have outgrown the tools that were built to answer them. This part sets out what changed, how the field arrived here, and who this book is written for.

1.1.1 Why Finance Needs AI

Finance is one of the most data-rich and decision-intensive domains in the modern economy. Financial institutions collect vast amounts of information every day, including prices, transactions, news, macroeconomic indicators, and firm-level reports. This information is often noisy, high-dimensional, and time-sensitive, which makes it difficult to analyze using simple rules or intuition alone.

At the same time, financial markets are highly competitive and dynamic. Firms, investors, and regulators all operate under pressure to make decisions quickly and accurately. As a result, methods from statistics, optimization, machine learning, and artificial intelligence have become increasingly important in tasks such as forecasting, risk management, trading, fraud detection, and customer service.

The central challenge in finance is not only to process data but also to make decisions under uncertainty. A portfolio manager must decide how much risk to take. A bank must estimate the probability of default. A regulator must detect suspicious behavior before losses become large. In each of these cases, the decision-maker needs a model, a framework for judgment, and an understanding of the incentives and constraints of the market.

A simple way to visualize this process is shown in Figure 1.1: raw data feeds a model, the model's output is checked before being trusted, and only then does it inform an actual decision, a step this book returns to directly when discussing why models must be interpreted rather than worshipped.

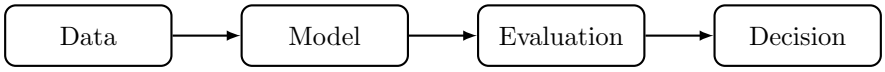


Figure 1.1: A simple pipeline from financial data to modeling, evaluation, and finally decisions

There is also a strong economic reason for the growing role of AI in finance. Many financial decisions are made under limited information, time pressure, and changing conditions. Traditional rules may be too rigid to capture subtle patterns in the data. Machine learning methods can uncover nonlinear relationships and adapt to new information, provided they are used with appropriate domain knowledge and careful validation.

This book is about the intersection of finance and artificial intelligence. Its goal is to show how modern data-driven methods can be used to solve problems in finance while also explaining the logic behind those problems.

Wulin.Suo@Queensu.ca

In practice, finance is not simply about making predictions from historical data. It is also about understanding incentives, constraints, and the consequences of decisions. A model that predicts a price movement may be useful, but it becomes much more valuable when it is linked to a trading rule, a risk limit, and an understanding of the costs of execution. Likewise, a credit model that predicts default must be interpreted in light of borrower behavior, contract terms, and broader macroeconomic conditions.

1.1.2 A Short History of Quantitative Methods and AI in Finance

The idea of using mathematical models to make financial decisions is not new; what has changed over the past seventy years is the sophistication of the models and the scale of the data available to them. Tracing this history briefly helps explain why today's AI methods are best understood as the latest step in a long-running progression, not a sudden break from everything that came before.

The modern era of quantitative finance is often dated to 1952, when Harry Markowitz formalized portfolio selection as an optimization problem over expected return and variance, work covered in depth in Chapter 12. This gave investors, for the first time, a rigorous mathematical language for the intuitive idea of diversification.

The next major milestone came in 1973, when Fischer Black, Myron Scholes, and Robert Merton derived a closed-form formula for pricing options, the model introduced in Chapter 5, which turned derivatives trading from an informal, judgment-driven activity into something that could be systematically priced and hedged. Both developments shared a common feature: they used relatively simple, closed-form mathematics to capture an economically meaningful idea, and both remain in active use today despite decades of subsequent refinement.

The 1980s and 1990s saw the rise of algorithmic and electronic trading, as exchanges automated order matching and quantitative hedge funds began systematically exploiting statistical patterns in market data, foreshadowing the market microstructure and algorithmic trading material in Chapters 13 and 14. Around the same period, banks began applying statistical scoring models to credit decisions at scale, replacing purely judgment-based underwriting with models similar in spirit to the credit models in Chapter 9.

Machine learning methods, decision trees, support vector machines, and eventually gradient boosting and random forests, entered mainstream financial use in the 2000s and 2010s as computing power grew and larger datasets became available, expanding what had been simple statistical scoring into much richer pattern recognition, the subject of Chapter 6.

Since roughly 2015, several further developments have reshaped the field. Deep learning has made it practical to learn directly from large volumes of unstructured data, images, audio, and especially text, which is the foundation for the natural language processing methods in Chapter 16 and the alternative data methods in Chapter 17. More recently, large language models have begun to be applied to tasks such as summarizing earnings calls, extracting structured information from filings, and supporting research workflows, an extension of the text-based methods in Chapter 16. In parallel, reinforcement learning has begun to be applied to genuinely sequential financial decisions, optimal trade execution, market making, and ongoing portfolio rebalancing, where an agent's own actions today shape the state it will face tomorrow, the subject of Chapter 19. A complementary, more recent strand, agent-based modeling, takes the opposite vantage point: instead of optimizing a single decision-maker, it simulates many simple, interacting participants at once to understand how market-wide phenomena such as bubbles, crashes, and contagion can emerge without any single agent intending them, the subject of Chapter 20. Table 1.1 summarizes this progression.

The throughline in this history is not that new methods replace old ones outright. Mean-variance optimization, Black-Scholes-Merton, and statistical scoring models are all still in daily use, often alongside the newer machine learning methods this book covers later. Each new wave of methods has instead expanded the range of problems that can be addressed and the amount of data that can be used, while the underlying economic questions, how to price risk, how to allocate capital, how to judge creditworthiness, have remained largely the same. This is a useful perspective to keep in mind throughout the book: a new AI technique is generally a new way of answering an old financial question, not a reason to discard the economic reasoning behind that question.

Table 1.1's AI/ML rows compress six decades of a much older, independent field into two lines, "machine learning applied to finance" and "deep learning and LLMs," which understates how much of that history had nothing to do with finance at all. AI and machine learning research has its own timeline, driven by its

Wulin.Suo@Queensu.ca

Period	Development
1950s	Markowitz mean-variance portfolio theory formalizes diversification mathematically
1970s	Black-Scholes-Merton option pricing enables systematic derivatives pricing and hedging
1980s–1990s	Electronic exchanges and algorithmic trading; statistical credit scoring at scale
2000s–2010s	Machine learning (trees, ensembles, support vector machines) applied to prediction and classification tasks across finance
2015–present	Deep learning for unstructured data; large language models applied to financial text; reinforcement learning for sequential decisions; agent-based models for market-wide simulation

Table 1.1: A brief timeline of quantitative methods in finance

own breakthroughs and its own setbacks, long before most of it was ever pointed at a financial problem; Table 1.2 traces that separate history on its own terms.

Two patterns stand out against Table 1.1. First, progress was not steady: the field's first AI winter shows that a technique's early promise (the Perceptron) can collapse under a well-founded technical critique and take over a decade to recover, a caution worth remembering whenever a new method arrives with confident claims attached. Second, essentially none of Table 1.2's milestones were built for finance, Deep Blue plays chess, AlexNet classifies photographs, AlphaGo plays Go, yet each one supplied a general-purpose capability, pattern recognition at scale, sequential decision-making, language understanding, that finance adopted only after it had already been proven elsewhere. This is why Chapters 6, 16, 17, and 19 of this book introduce these same architectures and training algorithms largely as general-purpose machine learning, before turning to what makes each one specifically useful in a financial context.

1.1.3 Who This Book Is For

This book is written for readers who are quantitatively trained but may have little or no prior background in finance or economics. The intended audience includes students and practitioners with experience in mathematics, statistics, engineering, computer science, or data science who want to apply their skills in financial settings.

The book assumes familiarity with basic probability, statistics, calculus, and programming, especially Python. It does not assume prior knowledge of financial markets, accounting, or economic theory. Instead, each financial topic is introduced from the ground up and connected to the relevant mathematical and computational tools.

This approach is deliberate. Many quantitative readers are comfortable with optimization, regression, and probabilistic modeling, but they may not yet have developed the institutional intuition that is essential in finance. For example, a predictive model may be statistically impressive but still fail if it does not respect market frictions, incentives, and regulatory constraints. The book therefore emphasizes both mathematical methods and financial interpretation.

A useful way to think about this is that finance combines three layers of reasoning: economics, statistics, and implementation. Economics provides the structure of incentives and incentives-based behavior. Statistics provides tools for estimating uncertain relationships from data. Implementation addresses how a model behaves when it is embedded in a real system with costs, delays, and operational constraints. A strong finance practitioner must be fluent in all three.

1.2 The Financial Landscape

Before any model can be useful, the terrain it operates on has to be legible: who trades, what they trade, what data the activity leaves behind, and the handful of concepts every later chapter assumes. This part

Wulin.Suo@Queensu.ca

Year	Development
1956	The Dartmouth Workshop, where John McCarthy and colleagues coin the term "artificial intelligence," is widely regarded as the field's founding event
1958	Frank Rosenblatt's Perceptron, the first trainable artificial neural network, generates enormous early excitement
1969–1986	Minsky and Papert's formal critique of the Perceptron's limitations helps trigger the first "AI winter", a prolonged pullback in AI funding and research interest
1986	Rumelhart, Hinton, and Williams popularize backpropagation, an efficient algorithm for training multi-layer neural networks (Chapter 6) that ends the first AI winter
1997	IBM's Deep Blue defeats reigning world chess champion Garry Kasparov, the first defeat of a sitting champion by a computer under tournament conditions
2012	AlexNet's decisive win in the ImageNet image-recognition competition demonstrates deep convolutional networks' advantage at scale, igniting the modern deep learning era (Chapter 17)
2016	DeepMind's AlphaGo defeats Go champion Lee Sedol using deep reinforcement learning (Chapter 19), a game long assumed too complex for a computer to master
2017	Vaswani et al.'s "Attention Is All You Need" introduces the transformer architecture, the foundation of every large language model that follows (Chapter 16)
2020	OpenAI's GPT-3, a 175-billion-parameter language model, demonstrates that scale alone unlocks broad, few-shot task performance
2022	OpenAI's ChatGPT reaches mainstream adoption faster than any consumer software product before it, bringing generative AI into everyday use

Table 1.2: A timeline of key developments in AI and machine learning

Wulin.Suo@Queensu.ca

supplies that vocabulary.

1.2.1 A Brief Introduction to Financial Markets

Consider a mismatch that shows up in every economy: a young firm has a profitable idea but no cash to build it, while a household nearby has savings sitting idle and no way to fund the idea directly, would not know how to evaluate it if it could, and has no interest in bearing all the risk alone even if it did. Multiply this mismatch across millions of firms and households, each with different amounts of capital, different risk appetites, and different time horizons, and the coordination problem becomes too large for any individual buyer and seller to solve by finding each other directly. A financial market is the mechanism that solves this coordination problem at scale: a place where financial assets, stocks, bonds, currencies, commodities, and derivatives among them, are traded, allowing individuals, businesses, and governments to transfer capital, diversify risk, and price future uncertainty without every lender having to personally track down and vet every borrower.

Financial markets do not exist only for speculation. They are a mechanism for allocating resources across time and across economic agents. A household may save for retirement. A firm may borrow to fund expansion. A government may issue debt to finance public spending. In each case, the market provides a way to connect those who have capital with those who need it.

The value of an asset is ultimately determined by the expectations of market participants about future cash flows, risk, and liquidity. For example, the price of a stock reflects the market's assessment of a firm's future profitability, while the price of a bond reflects expectations about default risk and interest rates. This makes finance a discipline that is both empirical and forward-looking.

Some of the most important financial institutions include banks, which intermediate between savers and borrowers; asset managers, which invest funds on behalf of clients; insurance companies, which protect against specific risks; exchanges, which provide platforms for trading; and regulators, which oversee market integrity and stability. Each of these institutions plays a distinct role in the functioning of the financial system, and their incentives shape the behavior of markets in subtle but important ways. Table 1.3 summarizes these roles.

Institution	Role
Banks	Intermediate between savers and borrowers
Asset managers	Invest funds on behalf of clients
Insurance companies	Protect against specific risks
Exchanges	Provide platforms for trading
Regulators	Oversee market integrity and stability

Table 1.3: Major financial institutions and their roles

Financial markets can be classified in several ways. Equity markets trade ownership claims in firms. Bond markets trade debt securities. Money markets trade short-term borrowing and lending instruments, typically with maturities under one year, and are covered in depth in Chapter 3. Derivatives markets trade contracts whose value depends on other assets. Foreign exchange markets trade currencies. Each market has its own structure, participants, and incentives. Table 1.4 summarizes these market types.

A useful way to think about markets is through the roles of participants. Investors seek returns. Borrowers seek financing. Market makers provide liquidity by standing ready to buy and sell. Arbitrageurs exploit small pricing discrepancies. Regulators enforce rules and preserve confidence. These roles interact to create the price discovery process that makes markets informative. Table 1.5 summarizes these participant roles.

This perspective also helps explain why prices can move sharply even when the underlying news is limited. A market is not a single decision-maker but a collection of heterogeneous participants with different objectives, information sets, and time horizons. As a result, prices often reflect a balance of competing forces rather than a simple average of known facts.

Wulin.Suo@Queensu.ca

Market	What is traded
Equity markets	Ownership claims in firms
Bond markets	Debt securities
Money markets	Short-term borrowing and lending instruments, such as Treasury bills, commercial paper, and repurchase agreements (Chapter 3)
Derivatives markets	Contracts whose value depends on other assets
Foreign exchange markets	Currencies

Table 1.4: Major types of financial markets

Participant	Objective or function
Investors	Seek returns
Borrowers	Seek financing
Market makers	Provide liquidity by standing ready to buy and sell
Arbitrageurs	Exploit small pricing discrepancies
Regulators	Enforce rules and preserve confidence

Table 1.5: Financial market participant roles

1.2.2 The Financial Data Ecosystem

Before turning to financial concepts, it is worth surveying the kinds of data that feed AI applications in finance, since different data types require different tools and different handling, and the rest of this book is organized in large part around them. Table 1.6 summarizes the main categories.

Each data type has its own quality issues and its own conventions, and a recurring theme of this book is that using a data type well requires understanding where it comes from, not just how to load it into a model. Market data is timestamped and voluminous but expensive at high frequency and subject to microstructure noise (Chapter 2 introduces the basic handling of price and return data; Chapters 13 and 14 return to its highest-frequency form). Fundamental data is comparatively low-frequency and subject to reporting lag and accounting discretion, as Chapter 4 discusses in detail. Macroeconomic data is often revised after initial release, so a historical dataset built from final revised figures can overstate what was actually knowable in real time, a subtle look-ahead bias. Text and alternative data are the newest additions to the financial data ecosystem and, as Chapters 16 and 17 discuss, often require substantially more preprocessing before they are usable than the structured, tabular data that dominates the earlier chapters of this book.

A practical consequence of this variety is that a single financial AI system frequently combines several of these data types at once. A credit model might combine fundamental data about a small business with transaction data from its bank accounts and even satellite imagery of its physical location; a trading strategy might combine market data with sentiment extracted from news text. Building such combined systems is usually harder than building a single-data-type model, both because the data must be aligned in time and identity (the same firm, the same reporting period) and because each data source can fail or degrade independently.

1.2.3 Core Financial Concepts

Before discussing AI applications, it is useful to understand a few central ideas in finance. These concepts provide the vocabulary and intuition needed to analyze financial problems, whether they are addressed with classical models or with modern machine learning techniques.

Risk and Return

Suppose two investments are on offer: a government bond that reliably pays 3% a year, and a new company's stock that might return 30% or might lose half its value, with no way to know in advance which it will be.

Wulin.Suo@Queensu.ca

Data type	Description and examples
Market data	Prices, quotes, trading volume, and order-book data, ranging from daily closing prices to microsecond-level tick data (Chapters 3, 13, 14)
Fundamental data	Accounting and financial statement data reported by firms, such as revenue, earnings, and balance sheet items (Chapter 4)
Macroeconomic data	Interest rates, inflation, unemployment, and other economy-wide indicators that affect asset prices broadly (Chapter 7)
Credit and transaction data	Loan repayment history, credit bureau data, and payment transaction records used in credit and fraud models (Chapters 9, 15)
Operational loss and incident data	Internal and external operational-loss event records, incident and audit narratives, and system and access logs used to model and detect operational failures (Chapter 10)
Insurance policy and claims data	Policy, exposure, claims, telematics, and catastrophe-model data used to price, reserve, and transfer insured risk (Chapter 11)
Text and news data	Earnings call transcripts, news articles, analyst reports, and social media, used for sentiment and information extraction (Chapter 16)
Alternative data	Non-traditional sources such as satellite imagery, web traffic, or shipping data that proxy for economic activity (Chapter 17)

Table 1.6: Major categories of financial data

Why would anyone hold the stock at all? If both were equally certain, no one would; the stock only attracts buyers because, on average and over time, it must offer more than the bond to compensate them for bearing the chance of a large loss. This is the risk-return tradeoff, and it is a fundamental principle in finance: higher expected return usually comes with higher risk, precisely because investors would otherwise simply hold the safer asset. Every investor must therefore decide how much uncertainty they are willing to bear in exchange for potential reward, a tradeoff central to portfolio choice, asset pricing, and corporate finance.

A simple illustration is shown in Figure 1.2. Higher expected returns are typically obtained only by accepting greater variability in outcomes.

This tradeoff is one of the most important ideas in finance because it explains why investors do not simply choose the asset with the highest expected return. They must also consider whether the additional risk is acceptable given their objectives, constraints, and time horizon. The same logic appears in corporate finance, where firms must decide how much debt to use and how much uncertainty they can tolerate in their operations.

Time Value of Money

Suppose someone offers to pay a debt either with $1,000 today or $1,000 in five years. Even setting aside inflation and the risk that the payer might not follow through, these two offers are not equivalent, and a purely quantitative reader's first instinct, that a dollar is a dollar, needs correcting before anything else in this book will make sense: $1,000 today can be invested and grow into more than $1,000 by the time five years have passed, so accepting the delayed payment means giving up that growth for nothing in return. Money available today is therefore worth more than the same nominal amount in the future, because it can be invested and earn a return in the meantime. This principle underlies discounting, present value, interest rates, and valuation. In practice, this means that future cash flows must be converted into present value terms before they can be compared fairly. This process is central to bond pricing, project valuation, and investment analysis.

Wulin.Suo@Queensu.ca

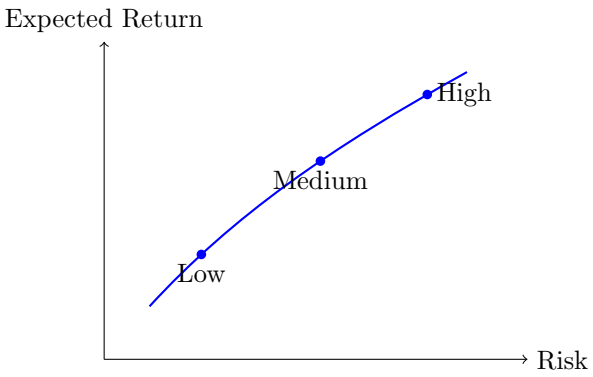


Figure 1.2: The risk-return tradeoff: higher expected returns generally require accepting greater risk

The future value of an amount PV invested today at rate r for n years is

$$FV = PV(1+r)^n,$$

and Table 1.7 applies this formula to \$1,000 invested at three different rates over three different horizons.

Annual rate	10 years	20 years	30 years
3%	\$1,343.92	\$1,806.11	\$2,427.26
6%	\$1,790.85	\$3,207.14	\$5,743.49
9%	\$2,367.36	\$5,604.41	\$13,267.68

Table 1.7: Future value of \$1,000 invested at different rates and horizons

Two patterns in Table 1.7 are worth internalizing because they recur throughout the book. First, compounding is highly nonlinear in the rate: doubling the rate from 3% to 6% more than doubles the 30-year future value, and tripling it to 9% multiplies the 30-year value by more than five. Second, the horizon matters enormously: at 9%, the difference between a 10-year and a 30-year horizon is a factor of roughly 5.6, even though the annual rate never changed. Present value, used constantly from Chapter 5 onward, is simply this same relationship read in reverse:

$$PV = \frac{FV}{(1+r)^n},$$

discounting a future amount back to today.

Diversification

Suppose an investor holds shares in a single airline. A jump in fuel prices, a pilots' strike, or a single crash investigation can each wipe out a large fraction of that airline's stock value, and all three risks are specific to that one company and its industry. Now suppose the same investor instead splits the same money across an airline, a software company, and a grocery chain, three businesses with essentially unrelated day-to-day fortunes. A fuel-price spike still hurts the airline, but does little to the grocery chain or the software company, so the same bad news that would have been devastating to an all-airline portfolio barely dents a portfolio spread across all three. This is diversification: spreading investments across different assets or risks so that no single loss can sink the whole portfolio. The intuition is simple, not all investments move in the same way at the same time, but the consequence is not: diversification is one of the few strategies in finance that reduces risk without necessarily reducing expected return, which is why Chapter 12 treats it as mathematically precise rather than merely a rule of thumb. In portfolio construction, diversification is a way to reduce idiosyncratic risk, the risk specific to one company or industry, while retaining exposure to the broad economic factors that no amount of spreading across individual stocks can eliminate.

Wulin.Suo@Queensu.ca

Market Efficiency

Suppose a quantitative analyst backtests a simple rule, buy a stock the day after it reports earnings that beat analyst expectations, and finds it would have produced a solid excess return over the past decade. Should the analyst expect this pattern to keep working once real money is committed to it? The answer turns on how much of that information was already reflected in the stock's price before the analyst could act on it: if thousands of other market participants saw the same earnings beat and traded on it within minutes, there may be little left on the table by the next morning.

The efficient market hypothesis formalizes this concern, holding that asset prices reflect available information to a large extent. This idea has important consequences for forecasting, trading, and the interpretation of abnormal returns. Although markets are not perfectly efficient at all times, this framework remains a core reference point in finance.

For quantitative readers, this idea is particularly important because it motivates the distinction between signal extraction and noise. If prices already incorporate most information quickly, then generating simple excess returns may be difficult. This does not mean that all opportunities disappear, but it does mean that the relevant question is often not whether a signal exists, but whether it survives transaction costs, model risk, and market frictions.

In this sense, market efficiency is not a statement that no one can ever beat the market. Rather, it is a reminder that profitable strategies must overcome several obstacles, including competition from other informed participants, delays in data availability, and the costs of trading.

1.3 What This Book Covers

With the landscape in place, the practical question is which financial problems machine learning genuinely improves on, and in what order they are best learned. This part maps the problems onto the chapters that treat them, and fixes the notation used throughout.

1.3.1 Financial Problems That AI Can Help Solve

AI is useful in finance because many important problems involve prediction, classification, optimization, and decision-making under uncertainty. In practice, these problems appear in forecasting asset prices or returns, predicting credit default or bankruptcy, detecting fraudulent transactions, monitoring for operational failures and misconduct, optimizing portfolios, identifying trading opportunities, and extracting information from financial text and news. In each case, the financial problem is not merely a generic data problem. The underlying financial model, market structure, and regulatory context matter. For example, a credit model must reflect the economics of default, while a trading strategy must account for transaction costs, market liquidity, and execution risk.

This is a key point for readers with a machine learning background. A model that performs well in a generic prediction task may fail in finance if it ignores institutional details. A fraud detector may need to understand customer behavior and regulatory requirements. A pricing model may need to respect no-arbitrage conditions. In other words, finance is not just a collection of datasets; it is a structured domain with economic logic and constraints.

As a simple illustration, suppose a bank wants to predict whether a borrower will default on a loan. A purely statistical model may use historical repayment behavior and demographic variables, but a useful financial model must also consider the borrower's debt burden, income stability, collateral, and macroeconomic conditions. The best solution is often a combination of economic reasoning, statistical estimation, and machine learning. The same principle applies to asset pricing, portfolio optimization, and trading. A forecasting model may perform well in sample but still fail in practice if it ignores transaction costs, liquidity constraints, or changes in market regimes. This is why finance requires both predictive accuracy and economic plausibility.

It is also important to recognize that financial data are often generated by strategic behavior. When a firm announces earnings, when a central bank changes interest rates, or when a large investor enters the market, the data reflect not only economic fundamentals but also the reactions of many agents. This makes financial modeling both statistical and behavioral in nature.

Wulin.Suo@Queensu.ca

Table 1.8 organizes the main problem types this book addresses by the kind of machine learning task involved and points to where each is covered in depth, giving a preview of the book's structure from a problem-first, rather than chapter-first, perspective.

Task type	Example financial problem	Covered in
Regression	Forecasting returns, volatility, an insurance claim cost, or a credit spread	Ch. 6, 7, 11
Classification	Predicting default, fraud, an operational event type, a claim's severity tier, or a trading signal's direction	Ch. 6, 9, 10, 11, 15
Time series forecasting	Forecasting interest rates, volatility, or macro indicators	Ch. 7
Distribution and tail estimation	Estimating a loss distribution and its far-tail quantiles: market VaR, operational-loss capital, insurance pricing and solvency	Ch. 8, 10, 11
Optimization	Constructing a portfolio subject to risk and constraint limits	Ch. 12
Clustering / dimensionality reduction	Grouping similar firms or compressing a large factor set into a few drivers	Ch. 6, 7, 12, 17
Sequential decision-making	Executing a trade over time to minimize impact, formalized as a Markov decision process	Ch. 13, 14, 19
Natural language processing	Extracting sentiment or information from text	Ch. 16
Anomaly detection	Flagging unusual transactions, account behavior, operational near-misses, or implausible insurance claims and coding patterns	Ch. 10, 11, 15, 17
Explainability and fairness auditing	Explaining an individual model decision and testing a model for disparate impact	Ch. 18, 21
Multi-agent simulation	Modeling how many simple, interacting participants jointly produce market-wide bubbles, crashes, or contagion	Ch. 20

Table 1.8: A taxonomy of AI problem types in finance

Two things stand out from this taxonomy. First, several finance problems map onto more than one machine learning task type, portfolio construction, for example, is fundamentally an optimization problem but often uses regression or dimensionality reduction as an input, which is why later chapters frequently combine methods from Chapter 6 with domain-specific structure. Second, some columns of Table 1.8 recur across many chapters (classification appears in credit, trading, and fraud contexts) precisely because the underlying statistical machinery, introduced once in Chapter 6, is reused throughout the rest of the book rather than re-derived for each application. Third, the table's last row is a genuine outlier among the others: every other row asks a model to predict, classify, or optimize on behalf of a single decision-maker, while multi-agent simulation instead asks what emerges when many simple decision-makers interact, a distinction Chapter 20 develops in full.

1.3.2 How This Book Is Organized

This book is designed to introduce finance and AI in a layered way. The early chapters focus on financial foundations so that readers without a background in the subject can build intuition. Later chapters introduce machine learning and statistical methods, followed by applications in areas such as risk management, portfolio management, trading, and compliance.

The progression is deliberate. The book begins with the basic language of finance, then moves to the mathematical tools used to analyze financial data. After that, it introduces classical financial models, such as portfolio theory, credit risk, operational-loss models, and actuarial pricing, before extending them with

Wulin.Suo@Queensu.ca

modern AI techniques. This structure is intended to help readers build intuition first and then connect that intuition to more advanced methods.

The structure of the book is intentionally progressive. It begins by introducing financial concepts in plain language, then develops the mathematical and statistical tools needed to analyze them, before turning to classical financial models and finally to AI-based extensions. This sequence allows readers to understand the economic logic of a problem before applying more advanced methods to it. Practical case studies and exercises are woven throughout so that the reader can connect theory with application and see how these ideas behave in realistic settings.

By the end of this chapter, the reader should have a broad understanding of what finance is, why it is data-rich, how markets function, and why AI methods must be interpreted in light of financial theory and institutional structure. These ideas provide the foundation for the more technical chapters that follow.

A compact summary of the book's conceptual flow is shown in Figure 1.3.

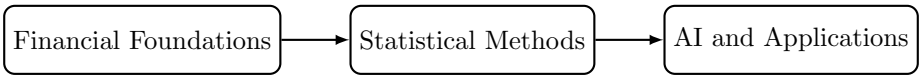


Figure 1.3: The book progresses from finance fundamentals to statistical tools and then to AI-driven applications

1.3.3 Notation and Conventions Used in This Book

Before turning to this chapter's first substantive financial topic, it is worth pausing for a short, purely reference section: a few notational conventions are used consistently across chapters, and stating them once here avoids repetition later. Random variables and their realizations are both written in italics (for example, X or x), with the distinction clear from context. A hat denotes an estimate, so $\hat{\beta}$ is an estimated coefficient and $\hat{y}$ is a predicted value, as opposed to β and y, which denote the true, generally unobserved, parameter and outcome. Vectors and matrices are written in bold only where the distinction from scalars is not otherwise clear from context, and $\|\cdot\|$ denotes the Euclidean norm. Time subscripts are written as X_t, with $t-1$ denoting the immediately preceding period, and

$$\Delta X_t = X_t - X_{t-1}$$

denoting a first difference.

Monetary amounts are given in US dollars unless stated otherwise, and percentages are used interchangeably with decimals (5% and 0.05 mean the same thing) depending on which is clearer in context. Interest rates, returns, and growth rates are annualized unless a different frequency (monthly, daily) is explicitly stated. Every chapter's worked numerical examples are also reproduced, and were originally verified, in that chapter's companion Jupyter notebook, so that a reader can check any computation in the text by running the corresponding notebook cell rather than only reading the printed result.

Each chapter follows the same template: an opening motivation, several sections of theory interleaved with at least one fully worked numerical example, a section on the practical challenges of the chapter's topic, a short list of key takeaways, a set of suggested exercises, and a further reading list of standard textbooks for readers who want to go deeper than a single chapter allows. Figures are simple schematic diagrams rather than photographs or screenshots, and are meant to organize ideas rather than to substitute for the surrounding prose.

With these conventions out of the way, the chapter turns to its first genuinely financial topic, one that reappears in some form in nearly every chapter that follows: how a firm's operations actually translate into value.

1.4 Reading Finance Beyond the Numbers

A financial statement is a set of numbers, but it is also a set of claims made by people with incentives. This part introduces the accounting that later chapters model, and the reasons a purely numerical reading of it

Wulin.Suo@Queensu.ca

will mislead.

1.4.1 Financial Statements, Cash Flows, and Valuation

Although much of modern finance is expressed through prices, rates, and stochastic models, the underlying economic reality is often much simpler: firms create value by generating cash flows over time. A company can be profitable on paper and still face financial distress if it does not generate enough cash to meet obligations. For this reason, financial analysis begins with a careful reading of financial statements, especially the balance sheet, the income statement, and the cash flow statement.

The balance sheet summarizes what a firm owns and what it owes at a point in time. Assets include cash, inventories, property, and intellectual property. Liabilities include debt, payables, and other obligations. Equity represents the residual claim of owners after liabilities are accounted for. The income statement shows revenues, expenses, and profit over a period of time. The cash flow statement, by contrast, explains how cash moved through the business and is often more informative than accounting earnings when one is trying to understand solvency and future financing needs.

This distinction is important because accounting profits and economic value are not identical. A firm may report large earnings while consuming cash through inventory accumulation, capital expenditure, or working capital needs. For investors and lenders, the question is not only whether the firm appears profitable, but whether it can sustain operations and meet future obligations. Data-driven methods in finance are often built around this type of economic interpretation. A predictive model may forecast earnings, but a financial analyst must also ask whether those earnings are backed by cash generation and durable competitive advantages.

The concept of valuation rests directly on this logic. The value of an asset is the present value of the expected future cash flows that it will generate, discounted at an appropriate rate. In symbols, a simple present value relation is

$$V = \sum_{t=1}^{T} \frac{CF_t}{(1+r)^t},$$

where CF_t denotes the cash flow in period t and r is the discount rate. This expression is one of the most important ideas in finance because it unifies bond pricing, stock valuation, project appraisal, and many forms of corporate analysis. The challenge is that future cash flows are uncertain and the discount rate must reflect both time and risk. This is where statistics and econometrics become essential.

In practice, valuation is rarely a purely mechanical exercise. Firms face changing competitive conditions, regulatory pressure, and shifts in investor sentiment. A company with strong historical performance may still lose value if its market position erodes or if interest rates rise sharply. This is why financial models are most useful when combined with qualitative judgment. A model can estimate probabilities or forecast trends, but it cannot replace the need to understand the business model, the industry structure, and the incentives of management.

This point is especially relevant for AI applications in finance. A machine learning model may forecast revenues or defaults with impressive accuracy, but it should not be deployed blindly. The output should be checked against the firm's accounting structure, its debt obligations, and the economic context in which it operates. The best systems therefore combine statistical learning with financial reasoning and careful validation.

1.4.2 Why Finance Is Not Just a Collection of Numbers

A common misconception is that finance is simply the manipulation of numerical data. In reality, finance is about incentives, contracts, and the allocation of resources under uncertainty. The numbers that appear in financial statements or market data are the outcomes of institutional processes. They emerge from decisions about borrowing, investing, pricing, regulation, and risk management.

For this reason, a quantitative analyst should not be satisfied with a high-performing prediction model unless there is also a clear economic explanation for why the model works. If a model identifies a pattern in stock returns, that pattern may reflect risk exposure, information diffusion, transaction costs, or temporary market inefficiency. If a model predicts default, it may be capturing leverage, deteriorating cash flow, or sector-level stress. In each case, the signal is meaningful only when interpreted through the lens of finance.

Wulin.Suo@Queensu.ca

This is also why financial modeling must be careful about causality. Correlation is not enough. A variable may be associated with future returns or defaults for reasons that do not persist once market conditions change. A sensible finance practitioner therefore tests assumptions, monitors model stability, and remains aware of regime shifts. AI can help discover patterns, but good financial judgment is needed to decide whether those patterns are economically plausible and practically useful.

1.4.3 A Practical Perspective on the Role of AI in Finance

The purpose of AI in finance is not to replace financial expertise. It is to augment it. AI methods can process large and complex data sources, detect nonlinear patterns, and automate tasks that would be too time-consuming for humans. However, the usefulness of these methods depends on the quality of the problem formulation. A poorly defined objective can lead to a model that looks sophisticated but produces poor decisions.

Consider the example of credit scoring. A bank may want to estimate the probability that a borrower will default over the next year. A machine learning model can use many variables, including repayment history, income, occupation, and account activity. Yet the model will be most useful when it is tied to domain knowledge about loan structure, collateral, legal obligations, and the macroeconomic environment. Otherwise, it may learn patterns that reflect temporary conditions rather than durable credit risk.

The same principle applies to portfolio construction, fraud detection, and market forecasting. The rational use of AI in finance requires an appreciation of uncertainty, costs, and incentives. It also requires humility. Even sophisticated models can fail when the environment changes, when data quality is poor, or when market participants adapt to the model itself.

This augmentation-not-replacement view also has organizational consequences that are easy to overlook in a purely technical treatment. Deploying an AI system in a financial institution typically requires sign-off from risk management, compliance, and sometimes a model validation team whose job is specifically to challenge the model before it is used with real capital or real customers, a process often called model risk management. A model that cannot be explained to these stakeholders, regardless of its statistical performance, is often not deployable at all, which is part of why Chapter 18 treats explainability as a first-class technical requirement rather than an afterthought. Readers coming from a pure machine learning background sometimes find this institutional layer frustrating, since it can slow down the deployment of a model that performs well in backtesting. It is best understood, however, as a reflection of the fact that a financial model's errors are rarely private: a mispriced loan, a bad trade, or a missed fraud case can affect customers, counterparties, and, in aggregate, the stability of the financial system as a whole, which is precisely why financial AI is regulated more heavily than AI applied to, say, recommending movies.

1.4.4 Connecting the Foundations to Later Chapters

The ideas introduced in this chapter provide the conceptual foundation for the more technical material that follows. Once the reader understands why markets exist, how assets are valued, and what kinds of institutions shape financial decisions, the later chapters on statistics, risk, trading, and machine learning will be easier to appreciate. The central theme is that financial problems are not just mathematical problems. They are problems of economic reasoning under conditions of uncertainty, constraints, and strategic behavior. This perspective is essential for anyone who wants to work at the intersection of finance and AI. A successful practitioner must be able to translate a financial question into a statistical problem, then translate the output of a model back into a financial decision. That translation requires both technical skill and financial intuition.

1.4.5 Finance as a System of Incentives and Information

Section 1.4.2 made this point at the level of a single model: a statistical pattern is only meaningful once it has an economic explanation. This section develops the same idea one level up, at the level of an entire market rather than a single model, since the incentives shaping how data are generated are themselves worth naming directly. A manager may be rewarded for short-term earnings growth even when that growth is not sustainable. A lender may extend credit to a borrower who appears safe in normal conditions but becomes

Wulin.Suo@Queensu.ca

fragile when conditions deteriorate. A regulator may face pressure to support market confidence even when this increases moral hazard. These examples show that finance is deeply intertwined with human behavior and institutional design.

The consequence is that even the best statistical model must be interpreted in context. A pattern in the data may reflect genuine economic structure, but it may also reflect the incentives of agents who produced the data. Financial data are therefore not passive observations. They are records of strategic interaction. This is a major reason why quantitative work in finance is never purely mechanical.

In many settings, information is dispersed across many participants. A single investor may know a small piece of the truth; markets aggregate these pieces through prices. This is one reason why price data are so valuable. They encode information about expectations, beliefs, and constraints. Yet they also encode noise, strategic behavior, and market frictions. The task of the analyst is therefore to separate signal from noise while remembering that the data themselves are generated by participants with incentives.

Two well-documented historical episodes illustrate how badly things can go wrong when a model is deployed without enough attention to the incentives shaping the underlying data. In the years leading up to the 2008 financial crisis, statistical models of mortgage default risk were built on historical data from a period of rising home prices and relatively loose underwriting standards. Those models implicitly assumed that the incentives generating the historical data, in particular, that mortgage originators bore meaningful consequences for making bad loans, would continue to hold. In practice, the rise of mortgage securitization changed those incentives: many originators sold loans onward quickly enough that they bore little of the eventual default risk themselves, so underwriting quality quietly deteriorated in ways the historical default models never observed. The models were not statistically wrong in a narrow sense; they were estimated correctly on the data available. They were wrong because the economic process generating the data had shifted underneath them, the non-stationarity concern flagged elsewhere in this chapter and developed further in Chapter 9.

A second, smaller-scale illustration is the phenomenon of Goodhart's Law, often summarized as "when a measure becomes a target, it ceases to be a good measure." A credit score, for example, is originally a statistical summary that correlates with creditworthiness. But once borrowers and lenders both know that a specific numerical threshold controls loan approval or pricing, some participants will alter their behavior specifically to cross that threshold, for reasons that may or may not reflect a genuine change in creditworthiness. This is a direct example of the reflexivity theme introduced in Chapter 6: unlike a model of a physical system, a financial model's targets can adapt to the existence of the model itself, which is one more reason financial AI systems require ongoing monitoring rather than a one-time validation.

1.5 Reasoning Under Uncertainty

Every financial decision is a bet made with incomplete information, which makes probability the native language of the subject. This part develops how to size such a bet, what new information is worth, and why a model's output is an argument rather than an answer.

1.5.1 Uncertainty, Probability, and Decision-Making

Finance is fundamentally about decisions under uncertainty. A lender does not know with certainty whether a borrower will repay. An investor does not know whether an asset price will rise or fall. A firm does not know whether a new project will be profitable. The relevant question is not whether uncertainty can be eliminated, because it usually cannot. The relevant question is how decision-makers should act when outcomes are uncertain and when the costs of being wrong are not symmetric.

It is worth being precise about what kind of not-knowing this actually is, since finance regularly runs into two quite different flavors. The economist Frank Knight drew this distinction sharply in 1921: *risk* describes a situation where the possible outcomes are known and their probabilities can be estimated, whether from a well-understood physical process (a fair coin) or from a long, stable history of similar past events, while *uncertainty* (sometimes called Knightian uncertainty) describes a situation where even the set of possible outcomes, let alone their probabilities, cannot be fully specified in advance. A casino game is risk in this precise sense; a genuinely novel financial crisis, a first-of-its-kind fraud scheme, or a new technology's effect

Wulin.Suo@Queensu.ca

on an entire industry is uncertainty. This distinction matters well beyond a single introductory paragraph: nearly every probabilistic model in this book, the Value at Risk models of Chapter 8, the credit-scoring models of Chapter 9, the volatility forecasts of Chapter 7, implicitly treats its subject as risk, assuming that a historical sample is enough to pin down the relevant probabilities, when the actual situation may still contain a meaningful component of genuine Knightian uncertainty that no amount of historical data can fully resolve. Keeping this distinction in mind is a useful, low-cost habit throughout the rest of this book: before trusting a model's probability estimate, it is worth asking whether the situation is closer to a repeatable game with a stable, estimable distribution, or closer to a genuinely novel situation where the model's confidence may be misplaced.

This is where probability and expected value become central. If an investor faces two possible outcomes, a gain and a loss, the expected value of the gamble may be positive even if the investor would prefer not to take the risk. This distinction motivates the idea of risk aversion. A risk-averse decision-maker usually requires a larger expected payoff to accept a gamble with greater uncertainty. This logic underlies portfolio choice, corporate financing, and insurance.

A simple expression for expected value is

$$\mathbb{E}[X] = \sum_i p_i x_i,$$

where p_i is the probability of state i and x_i is the associated payoff. This equation is simple, but it captures a profound idea: decision-making in finance requires combining possible outcomes with their probabilities. The challenge is that probabilities are rarely known with perfect precision, and the environment can change. For this reason, financial models must be updated as new information arrives and assessed against outcomes that are not fully predictable.

A small numerical example makes the idea of risk aversion concrete. Suppose an investor is offered a gamble with a 50% chance of gaining \$20,000 and a 50% chance of losing \$10,000. The expected value of the gamble is

$$\mathbb{E}[X] = 0.5(20{,}000) + 0.5(-10{,}000) = \$5{,}000,$$

which is positive, so a risk-neutral decision-maker, one who cares only about expected value, should accept it. Yet many investors would decline this gamble, because the possibility of losing \$10,000 matters more to them than the expected value calculation alone suggests. Economists capture this with a concave utility function $u(\cdot)$, so that the investor evaluates the gamble using expected utility, $\mathbb{E}[u(X)]$, rather than expected value directly. The certainty equivalent is the guaranteed amount that gives the same expected utility as the gamble; for a risk-averse investor, the certainty equivalent is always less than the \$5,000 expected value, and the gap between the two, sometimes called the risk premium, is a direct measure of how much the investor is willing to pay to avoid uncertainty.

Making this concrete requires choosing a specific utility function and applying it to the investor's total wealth, not just the gamble's payoff in isolation, since a concave function is only meaningful evaluated at actual wealth levels. Suppose the investor's wealth today is $W_0 = \$50{,}000$, and, following the square-root utility function $u(W) = \sqrt{W}$, a standard textbook example of a risk-averse (concave) utility function. The gamble leaves the investor with wealth $W_0 + 20{,}000 = \$70{,}000$ or $W_0 - 10{,}000 = \$40{,}000$, so

$$\mathbb{E}[u(W)] = 0.5\sqrt{70{,}000} + 0.5\sqrt{40{,}000} \approx 0.5(264.575) + 0.5(200.000) = 232.288.$$

The certainty equivalent wealth CE solves $u(CE) = \mathbb{E}[u(W)]$, that is, $\sqrt{CE} = 232.288$, giving $CE \approx \$53{,}957.51$, noticeably less than the expected wealth of $0.5(70{,}000) + 0.5(40{,}000) = \$55{,}000$. The risk premium is the difference,

$$\text{Risk premium} = \$55{,}000 - \$53{,}957.51 \approx \$1{,}042.49,$$

which has a direct interpretation: this investor would accept a guaranteed \$53,957.51 in place of the gamble, meaning they would pay up to \$1,042.49 to avoid the gamble's uncertainty entirely, even though the gamble's expected wealth is higher. This same logic, comparing an uncertain payoff to its certainty equivalent, underlies the pricing of insurance, the equity risk premium, and the risk-return tradeoff introduced earlier in this chapter, and it reappears in a more technical form as the reason a risk-averse portfolio manager accepts a lower expected return in exchange for lower volatility in the mean-variance framework of Chapter 12.

Wulin.Suo@Queensu.ca

The Kelly Criterion: How Much to Bet, Not Just Whether

A positive expected value answers whether a favorable gamble is worth taking at all, but a repeatable opportunity, place the same favorable bet over and over, raises a second, sharper question a single certainty-equivalent calculation does not: what *fraction* of current wealth should be wagered each time? Betting too little leaves genuine edge on the table; betting too much, this section's next result shows, can make an investor with a real, positive edge poorer with certainty over time.

The reason this question is nontrivial, rather than simply "bet as much as a positive edge allows," is that wealth wagered repeatedly compounds rather than accumulates. A single bet's expected value is an arithmetic average across outcomes, and it is the right tool for deciding whether to take that bet once. But wealth carried across many repetitions is a product of per-round multipliers, $(1+f)$ after a win and $(1-f)$ after a loss, not a sum of per-round gains, and a quantity built from repeated multiplication is governed by its *geometric* mean, not its arithmetic one. The two can disagree sharply: a single large enough loss multiplies surviving wealth by a factor close to zero, and no number of favorable multiplications afterward fully reverses that, so a bet size can raise the arithmetic average of outcomes while simultaneously driving the geometric average, and with it long-run wealth, toward zero. This asymmetry, not risk aversion in the subjective sense discussed above, is what makes betting too large a fraction ruinous even when every individual bet retains positive expected value, and it is exactly what the growth-rate result below makes precise.

The Kelly criterion answers this precisely for a repeatable bet with probability p of winning (paying b dollars per dollar wagered) and probability $q = 1 - p$ of losing the full amount wagered: the fraction of wealth that maximizes the long-run *geometric* growth rate of wealth is

$$f^* = p - \frac{q}{b}.$$

For an even-money bet ($b = 1$) with a 60% win probability, $f^* = 0.6 - 0.4 = 0.2$: wager exactly 20% of current wealth on each repetition, no more and no less. The formula is quoted here rather than derived; this book's companion technical note on the Kelly criterion works the argument through in full, showing why repeated betting is governed by a geometric rather than an arithmetic average, why f^* is a strict global maximum rather than merely a stationary point, why maximizing growth is equivalent to maximizing expected logarithmic wealth, and why the optimal fraction can rationally exceed one, recommending leverage, once a loss costs less than the entire amount staked.

The reason "no more" matters as much as "no less" is the genuinely surprising part. The long-run growth rate of wealth from repeatedly wagering a fraction f is $g(f) = p\ln(1+f) + q\ln(1-f)$; evaluating it at three candidate fractions for this same 60%-favorable bet,

$$g(0.2) \approx 0.0201, \qquad g(0.5) \approx -0.0340, \qquad g(1.0) = -\infty,$$

shows that wagering half of wealth on every repetition, despite each individual bet still having positive expected value, produces a *negative* long-run growth rate: an investor following this rule is mathematically certain to see their wealth decay toward zero over enough repetitions, not merely experience higher volatility around a still-positive trend.

Wagering everything each time ($f = 1$) is worse still, guaranteed eventual ruin, since a single loss (probability $q = 0.4$ on any given bet, and probability 1 eventually across enough repetitions) wipes out the entire stake with nothing left to bet with afterward. This is a sharper, more quantitative version of the risk-aversion lesson above: a decision-maker who evaluates only a single bet's expected value, without asking how large a fraction of wealth to commit to it when it repeats, can be led toward a sizing decision that is not just uncomfortable but ruinous, the kind of position-sizing discipline that resurfaces, in a portfolio context rather than a single repeated bet, in Chapter 12's discussion of leverage and Chapter 13's execution-sizing decisions.

The Value of Information: What Is a Perfect Signal Worth?

A closely related question asks not how much to bet, but how much a decision-maker should be willing to pay for information that resolves uncertainty *before* a decision is made. Suppose a firm is deciding whether to fund a $2 million project, an idealized version of the kind of project-investment decision that Chapter 5's

Wulin.Suo@Queensu.ca

discounted-cash-flow and real-options material treats in more detail. The project succeeds with probability 0.4, returning a net gain of \$5 million, or fails with probability 0.6, losing the full \$2 million invested. Without any additional information, the expected value of proceeding is

$$E[\text{Proceed}] = 0.4(5) + 0.6(-2) = \$0.8 \text{ million},$$

positive, and better than the \$0 from not proceeding at all, so a rational, risk-neutral firm proceeds, accepting the 60% chance of a \$2 million loss along with the 40% chance of a \$5 million gain.

Now suppose the firm could instead consult a perfectly accurate forecaster, at some price, who reveals with certainty which state (success or failure) will occur before the firm must commit. Knowing this, the firm would proceed only in the success state and decline in the failure state, earning

$$E[\text{Proceed} \mid \text{perfect information}] = 0.4(5) + 0.6(0) = \$2.0 \text{ million},$$

since the \$2 million loss is avoided entirely in the 60% of cases where failure would otherwise have occurred. The expected value of perfect information (EVPI) is the difference between these two figures,

$$\text{EVPI} = 2.0 - 0.8 = \$1.2 \text{ million},$$

the absolute maximum a rational firm should be willing to pay for a perfectly accurate signal before deciding, since paying any more would make consulting the forecaster worse than simply proceeding blind. In practice, no real forecast, credit model, or market signal is perfectly accurate, so EVPI is best understood as an upper bound: it answers "how much could better information possibly be worth here," a question worth asking explicitly before investing in additional data, a more sophisticated model, or a costly due-diligence process, the kind of cost-benefit judgment call that recurs, applied to specific data vendors and model types, throughout this book's later chapters.

1.5.2 Bayesian Updating: Learning from New Information

A closely related idea, and one that underlies how many machine learning systems in this book are best interpreted, is that beliefs about an uncertain quantity should be updated, not replaced, as new evidence arrives. Bayes' theorem formalizes this:

$$\Pr(A \mid B) = \frac{\Pr(B \mid A)\,\Pr(A)}{\Pr(B)},$$

where $\Pr(A)$ is the prior probability of some hypothesis A before seeing evidence B, $\Pr(B \mid A)$ is the likelihood of observing that evidence if A is true, and $\Pr(A \mid B)$ is the posterior probability of A after accounting for the evidence.

A concrete example, directly relevant to the fraud detection material in Chapter 15, shows why this matters. Suppose 1% of transactions at a bank are fraudulent, so $\Pr(\text{Fraud}) = 0.01$. A fraud detection model correctly flags 95% of actual fraud, $\Pr(\text{Flag} \mid \text{Fraud}) = 0.95$, but also flags 3% of legitimate transactions, $\Pr(\text{Flag} \mid \text{Not Fraud}) = 0.03$. Given that a transaction has been flagged, what is the probability it is actually fraudulent? Applying Bayes' theorem,

$$\begin{aligned}
\Pr(\text{Flag}) &= \Pr(\text{Flag} \mid \text{Fraud})\Pr(\text{Fraud}) + \Pr(\text{Flag} \mid \text{Not Fraud})\Pr(\text{Not Fraud}) \\
&= 0.95(0.01) + 0.03(0.99) \\
&= 0.0392, \\
\Pr(\text{Fraud} \mid \text{Flag}) &= \frac{0.95(0.01)}{0.0392} \approx 0.242.
\end{aligned}$$

Despite a model that sounds highly accurate (95% detection, only 3% false alarms), a flagged transaction is fraudulent only about 24% of the time, because genuine fraud is so rare that even a small false-positive rate produces many more false alarms than true detections in absolute terms. This is the same base-rate intuition behind the discussion of imbalanced classification in Chapter 6, and it is a direct, quantitative illustration of why accuracy alone, discussed qualitatively earlier in this chapter, is a poor way to evaluate a model applied to a rare event.

Wulin.Suo@Queensu.ca

Bayesian updating also describes, at a conceptual level, how a well-run financial model should be maintained over time. Rather than treating a model's parameters as fixed forever, or discarding a model entirely the first time it makes a mistake, a Bayesian perspective treats each new batch of data as an opportunity to revise a belief incrementally: the posterior from today becomes the prior for tomorrow's update. This is the same spirit behind the ongoing model monitoring developed in Chapter 18, and behind the state-space and Kalman-filtering methods introduced formally in Chapter 7, which are, in effect, Bayesian updating carried out recursively over time.

The "posterior becomes the next prior" idea is easiest to see by continuing the fraud example one step further. Suppose the flagged transaction above is now compared against the customer's typical spending location, and this second, independent signal shows the transaction occurred somewhere the customer has never transacted before, a pattern that occurs in 60% of genuine fraud cases but only 10% of legitimate ones: $\Pr(\text{Unusual Location} \mid \text{Fraud}) = 0.6$ and $\Pr(\text{Unusual Location} \mid \text{Not Fraud}) = 0.1$. Rather than starting over, a Bayesian analysis treats the first posterior, $\Pr(\text{Fraud} \mid \text{Flag}) \approx 0.242$, as the prior for this second update:

$$\begin{aligned} \Pr(\text{Location}) &= 0.6(0.242) + 0.1(1 - 0.242) = 0.221, \\ \Pr(\text{Fraud} \mid \text{Flag}, \text{Location}) &= \frac{0.6(0.242)}{0.221} \approx 0.657. \end{aligned}$$

Two independent pieces of weak-to-moderate evidence, an automated flag and an unusual location, combine to raise the probability of fraud from a 1% base rate to roughly 66%, each update using the same formula with the previous posterior in place of the original prior. Figure 1.4 lays out this chain of updates directly. This is precisely how a production fraud system in Chapter 15 accumulates evidence from several independent model outputs before deciding whether a transaction warrants manual review, and it is why an ensemble of several weak signals, evaluated one Bayesian update at a time, can be more reliable than any single signal considered alone.

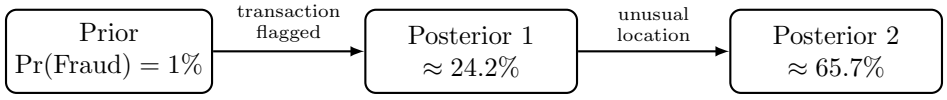


Figure 1.4: The fraud example's two-step Bayesian update chain

1.5.3 Why Models Must Be Interpreted, Not Worshipped

A common temptation in finance is to trust a model too much simply because it is sophisticated or produces attractive historical results. Such overconfidence is dangerous. A model can be mathematically elegant and still be economically misleading. A stock prediction model may perform well for a while and then fail when market conditions change. A credit model may appear accurate in a stable period and become unstable during a recession. A fraud model may detect obvious anomalies but miss subtle forms of misconduct that change over time.

Figure 1.1's pipeline already named the reason this matters: the evaluation step between a model and the decision it informs is not a formality, it is where an overconfident model gets caught, or does not. This is why model evaluation in finance is not just a statistical exercise. It is also an economic and operational exercise. The relevant questions are not only whether the model fits the data but whether its assumptions make sense, whether its outputs are usable, and whether the implementation is robust. Finance rewards humility. The best practitioners understand both the strengths and the limits of their methods.

The 1998 collapse of Long-Term Capital Management (LTCM), a hedge fund whose partners included two Nobel laureates in economics and whose models were, by the standards of the time, extremely sophisticated, is a case study in this failure mode, examined in more detail in Chapter 3. The fund's models had correctly estimated how various fixed-income spreads behaved under historically normal conditions, but had no mechanism for anticipating a regime in which several previously uncorrelated markets, including Russian sovereign debt, moved together in a global flight to quality, at the same moment the fund's own

Wulin.Suo@Queensu.ca

highly leveraged positions made it a forced seller into a market with too few buyers. No later chapter's model, however statistically sophisticated, is exempt from this same risk: a model is only ever as good as the range of conditions it was built and tested to anticipate, which is precisely why the stress-testing and scenario-analysis techniques covered in Chapter 8 exist as a check on quantitative models generally, not a specialized technique reserved for portfolio risk alone.

1.6 Limits, Challenges, and Practice

Nothing developed so far works as cleanly in production as it does on a page. This part collects the obstacles that recur throughout the book, and closes with a case in which classical finance and machine learning were combined successfully.

1.6.1 Challenges of Applying AI in Finance

Before moving into the technical chapters, it is worth naming, at a high level, the recurring challenges that will resurface throughout this book whenever AI methods are applied to financial problems. Later chapters revisit each of these in much greater depth and with concrete techniques for addressing them.

Data challenges. Financial data are noisy, unevenly sampled, subject to reporting delays, and sometimes revised after the fact. Historical datasets can also suffer from survivorship bias, in which failed firms or discontinued strategies are quietly excluded, making the past look more favorable than it actually was.

Non-stationarity. Unlike many physical systems, financial markets change their own statistical behavior over time, in response to new regulation, new technology, changing investor composition, and the actions of other market participants, including the actions of other AI systems. A model calibrated on one period can therefore fail in the next, even without any error in how it was built.

Overfitting and backtest illusions. Because financial data are limited relative to the number of strategies and features that can be tried, it is easy to find a pattern that worked well in history purely by chance. A strategy that looks excellent in a backtest can perform poorly, or lose money, once deployed with real capital.

Interpretability and accountability. Many financial decisions, particularly credit and insurance decisions, are subject to legal requirements that they be explainable and non-discriminatory. This constrains how opaque a model can be, regardless of its statistical performance, and is examined in depth in Chapter 18.

Operational and execution frictions. A model's statistical output is not the same as a financial outcome. Transaction costs, market impact, latency, and operational failures can all erode or eliminate a theoretical edge, which is why economic evaluation, not just statistical evaluation, is emphasized throughout this book.

Ethical and systemic considerations. Widely adopted AI models can create correlated behavior across many market participants, potentially amplifying booms and busts. Models used in lending, insurance, or hiring can also encode or amplify existing biases in historical data if they are not carefully audited.

These are not reasons to avoid AI in finance. They are reasons to apply it carefully, with an understanding of financial institutions, incentives, and market structure alongside statistical skill, which is precisely the combination this book aims to build.

1.6.2 Case Vignette: How Classical Finance and AI Combine in Practice

To make the abstract themes of this chapter concrete, consider a stylized (fictional) example that previews how the pieces of this book fit together. Suppose a mid-sized asset manager wants to build a systematic equity strategy that selects stocks likely to outperform over the coming quarter.

The manager does not start with machine learning. The process begins with the financial statement analysis of Chapter 4: screening a universe of firms on fundamentals such as profitability, leverage, and cash generation, discarding firms with clear signs of financial distress before any prediction model is involved. This step encodes economic reasoning, distress and financial weakness are meaningful even before considering whether a stock's price is likely to rise, that a purely data-driven model applied to raw price history would not automatically discover.

Wulin.Suo@Queensu.ca

Next, the manager builds a predictive model, using the regression and classification tools of Chapter 6, to rank the remaining firms by expected relative performance. The model's features draw on several of the data types catalogued in Table 1.6: fundamental ratios from the statement-screening step, momentum and volatility measures computed from the time series methods of Chapter 7, and, increasingly, sentiment scores extracted from earnings-call transcripts using the natural language processing methods of Chapter 16. Critically, the model is evaluated with the walk-forward methodology described later in this book, not a random train-test split, since a random split would let the model see information from the future relative to some of its training data.

The ranked predictions then become an input to portfolio construction, not the final answer. Chapter 12's mean-variance and factor-based methods convert a raw ranking into a set of position sizes that respect diversification, risk limits, and transaction cost constraints, since the highest-ranked stock in isolation might be too risky, too illiquid, or too correlated with the rest of the portfolio to hold at a large weight. Finally, before any trade is placed, the execution considerations of Chapter 13 determine how to actually buy and sell the resulting positions without moving prices adversely, and the risk models of Chapter 8 continuously monitor whether the realized portfolio is behaving as expected.

At every step in this vignette, a classical financial concept, distress screening, portfolio diversification, transaction costs, sits alongside a modern statistical or machine learning method, and neither one is sufficient on its own: the machine learning ranking without financial screening risks recommending distressed firms with temporarily attractive statistical patterns, while financial screening without machine learning would leave the manager unable to process the scale of fundamental, price, and text data available. This interleaving of classical and modern methods, not the replacement of one by the other, is the organizing principle of every applied chapter that follows.

1.7 Key Takeaways

The key lesson of this chapter is that finance is a domain in which data, uncertainty, and decision-making interact closely. Quantitative readers can make rapid progress in the subject by connecting mathematical tools with financial intuition, but they must also learn to interpret models in light of market structure, incentives, and institutional constraints. AI methods are most useful in finance when they are guided by sound financial reasoning rather than used as black-box tools. In short, the most effective applications of AI in finance combine statistical skill, economic understanding, and practical judgment.

Several more specific points from this chapter are worth carrying forward explicitly. The history in Section 1.1.2 showed that today's machine learning methods extend, rather than replace, decades of classical quantitative finance, which is why nearly every applied chapter in this book pairs a classical model with a modern one instead of presenting the modern method alone. The data ecosystem in Section 1.2.2 previewed the range of data types, market, fundamental, macroeconomic, credit, text, and alternative, that later chapters draw on, each with its own quality issues. The numerical examples in this chapter, compounding in Table 1.7, the gamble and certainty equivalent, and the fraud-detection application of Bayes' theorem, were chosen because each illustrates a principle (nonlinearity of compounding, risk aversion, and the danger of ignoring base rates) that reappears in a more technical form in a later chapter. Finally, the case vignette in Section 1.6.2 is worth revisiting after finishing the book: a reader who initially finds the asset-management example abstract should be able to return to it once Chapters 4, 6, 7, 8, 12, 13, and 16 have been covered and see exactly how each step would be implemented in practice.

1.8 Suggested Exercises

1. Define the difference between risk and uncertainty in the context of financial decision-making.
2. Explain why the time value of money matters for investment decisions.
3. List three types of financial markets and describe the assets traded in each.
4. Give one example of a financial problem where AI could be useful and one where classical statistical methods might be sufficient.

Wulin.Suo@Queensu.ca

5. Write a short paragraph explaining why financial models should still be considered when using machine learning in finance.

6. Describe one financial institution and explain how it connects savers, borrowers, or investors.

7. Discuss why a model that performs well in a generic prediction task may still fail in a financial setting.

8. A gamble offers a 60% chance of gaining \$8,000 and a 40% chance of losing \$5,000. Compute the expected value, and explain, using the concept of a certainty equivalent, why a risk-averse investor might still reject it even though the expected value is positive.

9. Pick two of the challenges listed in Section 1.6.1 ("Challenges of Applying AI in Finance") and describe a concrete financial scenario, not already used in the chapter, in which that challenge could cause an AI-based system to fail.

10. Using Table 1.7, compute the future value of \$5,000 invested for 20 years at 6%, and verify your answer is consistent with scaling the \$1,000 figure in the table.

11. A disease-screening test correctly identifies 90% of people who have a rare disease affecting 0.5% of the population, and has a 5% false-positive rate. Using Bayes' theorem as in Section 1.5.2, compute the probability that a person who tests positive actually has the disease, and explain why the result is counterintuitive.

12. Choose one entry from Table 1.8 and explain, in a short paragraph, why the financial problem you chose could plausibly be framed as more than one of the task types listed in the table (regression, classification, optimization, and so on).

13. Pick one milestone from Table 1.1 and explain, using material from later in this chapter, why the newer methods that followed it did not make it obsolete.

14. Continuing the sequential Bayesian updating example in Section 1.5.2, suppose a third, independent signal arrives: the transaction amount is a round number (e.g., exactly \$500), a pattern present in 40% of fraud cases but only 15% of legitimate ones. Using $\Pr(\text{Fraud} \mid \text{Flag}, \text{Location}) \approx 0.657$ as the new prior, compute the updated posterior probability of fraud after all three signals, and comment on how quickly the probability approaches certainty as independent evidence accumulates.

15. Using the Kelly criterion, compute the optimal betting fraction f^* for an even-money bet with a 70% win probability, and compute the long-run growth rate $g(f)$ at f^* and at $f = 0.9$, commenting on whether betting nearly everything is ever justified even when the win probability is this favorable.

16. A firm is deciding whether to enter a new market at a cost of \$3 million. Entry succeeds with probability 0.3, returning a net gain of \$12 million, or fails with probability 0.7, losing the full \$3 million. Compute the expected value of entering, the expected value under perfect information, and the resulting EVPI, and explain what the firm should conclude if a market-research report claiming near-perfect accuracy costs \$4 million.

1.9 Further Reading

For readers who want to deepen their understanding of the financial ideas introduced in this chapter, the following texts are useful starting points:

- Brealey, Myers, and Allen, *Principles of Corporate Finance*. A comprehensive introduction to corporate financial decision-making, including the time-value-of-money and valuation ideas this chapter introduces at a conceptual level and Chapter 5 develops formally.

- Hull, *Options, Futures, and Other Derivatives*. The standard reference for derivatives pricing; useful background for the brief history of Black-Scholes-Merton option pricing sketched in this chapter's timeline and covered in full in Chapter 5.

Wulin.Suo@Queensu.ca

- Bodie, Kane, and Marcus, *Investments*. Covers portfolio theory, market efficiency, and the risk-return tradeoff introduced qualitatively in this chapter, previewing the quantitative treatment in Chapter 12.
- Campbell, Lo, and MacKinlay, *The Econometrics of Financial Markets*. A rigorous treatment of the statistical methods used to test market efficiency and analyze financial time series, going well beyond this chapter's introductory discussion.
- Goodfellow, Bengio, and Courville, *Deep Learning*. The standard reference for the deep learning methods behind the large language model and unstructured-data applications previewed in this chapter's history section and covered in Chapters 16 and 17.
- Chan, *Machine Trading*. A practitioner-oriented introduction to building and testing systematic trading strategies, extending the case vignette in this chapter into the algorithmic trading material of Chapters 13 and 14.

These references cover foundational finance, asset pricing, and the use of quantitative methods in financial applications.

Wulin.Suo@Queensu.ca

Chapter 2

Python for Financial Analysis

2.1 Introduction

Python has become one of the most important tools in modern finance. It is flexible, readable, and supported by a rich ecosystem of libraries for data analysis, visualization, statistics, and machine learning. For quantitative researchers, analysts, and practitioners, Python offers a practical way to turn financial data into insight. It is also a natural gateway into the wider world of data science, because many of the techniques used in finance are identical to those used in business analytics, scientific computing, and artificial intelligence.

This chapter introduces Python from a financial perspective. Instead of treating programming as an abstract skill, it explains how Python helps with common financial tasks such as reading market data, cleaning historical prices, calculating returns, visualizing trends, and preparing data for modeling. Every concept in this chapter is tied to a single running numerical example, a two-asset portfolio of five trading days, so that the reader can check every computation by hand and reproduce it exactly in code. The emphasis is on building intuition and practical fluency rather than on purely theoretical computer science.

The chapter builds up in layers, each introduced through the same two-asset AssetA/AssetB price history so that no new example ever has to be learned from scratch. It starts with the language mechanics themselves, a first interactive session, conditionals and loops, and the core data structures (lists, dictionaries, tuples, and NumPy arrays) that everything downstream depends on.

It then turns to pandas, the library that does most of the practical work in the rest of this book: boolean indexing, merging and grouping, resampling to a coarser frequency, and cleaning a simulated missing observation. A dedicated worked example builds and evaluates the two-asset portfolio in full, computing its mean, variance, and covariance by hand and in code, then scales the same computation up to a third asset to show that none of the underlying logic changes as a portfolio grows.

The chapter closes with two practical concerns every quantitative workflow eventually meets: why vectorized NumPy code can be tens of times faster than an equivalent Python loop, and how to keep a numerical workflow reproducible enough that another person, or a future version of oneself, can rerun it and get the same answer.

This progression matters beyond the mechanics of Python itself. Sections 2.2.2 through 2.3.1 establish the vocabulary, loops, arrays, and vectorized operations, that later chapters assume without re-explaining; Section 2.5.1's two-asset portfolio recurs by name in Chapters 8 and 12, so the covariance matrix and correlation computed here are worth internalizing rather than skimming; and Sections 2.4.4 and 2.6.1 anticipate two problems, messy real-world data and slow, unvectorized code, that resurface in nearly every applied chapter that follows.

2.2 The Python Language

Everything later in this book is expressed in code, so the language itself has to come first. This part moves from why Python won the quantitative-finance argument to the syntax a reader needs before any financial idea can be written down: sessions, control flow, and the built-in data structures.

Wulin.Suo@Queensu.ca

2.2.1 Why Python Matters in Finance

A financial analyst often works with large, messy, and changing datasets. Prices arrive continuously, statements are published irregularly, and news can affect markets within seconds. This environment creates a strong need for tools that are fast, reproducible, and easy to adapt. Python excels in these areas. Scripts can be rerun, workflows can be automated, and models can be validated systematically.

Python is also attractive because it has a large scientific community. Libraries such as NumPy, pandas, Matplotlib, SciPy, and scikit-learn make it possible to perform numerical work efficiently. This ecosystem lowers the barrier to entry for researchers who want to move from a spreadsheet workflow to a more scalable and transparent pipeline. Table 2.1 summarizes the libraries used throughout this chapter and the role each one plays.

Library	Role in financial analysis
NumPy	Fast array operations and vectorized numerical computation
pandas	Tabular and time-indexed data handling, cleaning, and aggregation
Matplotlib	Static charting of prices, returns, and diagnostics
SciPy	Statistical distributions, optimization, and hypothesis testing
statsmodels	Regression, time series models (e.g., ARIMA), and statistical inference
scikit-learn	Machine learning models, preprocessing, and model evaluation

Table 2.1: Core Python libraries for financial analysis

2.2.2 A First Python Session

Suppose an analyst needs to know what a $100 investment earning 8% will be worth after one year, a calculation simple enough to do on paper once, but tedious and error-prone to redo by hand for every price, rate, and holding period a real portfolio might contain. Python solves this by turning a value into a variable, a rule into a function, and a decision into a piece of control flow, so the same calculation can be reused rather than retyped: variables store values, functions transform inputs, and control flow allows the program to make decisions. A small example computes the future value of an investment:

```
price = 100.0
return_rate = 0.08
future_value = price * (1 + return_rate)
print(future_value)
# 108.0
```

This simple snippet shows how a financial quantity can be represented in code. A price can be stored as a number, a return can be expressed as a decimal, and a future value can be computed directly. Wrapping this logic in a function makes it reusable across many holding periods:

```
def future_value(price, rate, periods=1):
    return price * (1 + rate) ** periods

print(future_value(100.0, 0.08, periods=3))
# 125.97120000000001
```

In finance, such arithmetic appears constantly. Interest rates, discount factors, portfolio weights, and transaction costs are all quantities that can be represented naturally in Python, and wrapping repeated calculations in functions avoids copying formulas by hand and reduces the chance of arithmetic mistakes.

Wulin.Suo@Queensu.ca

2.2.3 Control Flow: Conditionals and Loops

Beyond simple arithmetic, most financial calculations require making decisions and repeating operations. An `if/elif/else` block lets a program branch based on a condition, which is useful for classifying outcomes:

```
def classify_return(r):
    if r > 0.01:
        return "strong gain"
    elif r > 0:
        return "small gain"
    elif r == 0:
        return "flat"
    else:
        return "loss"

for r in [0.02, 0.004, 0.0, -0.015]:
    print(r, "->", classify_return(r))
# 0.02 -> strong gain
# 0.004 -> small gain
# 0.0 -> flat
# -0.015 -> loss
```

The `for` loop here iterates over a list of returns and applies the classification function to each one. Loops are also the natural way to express a calculation that depends on its own previous result, which arithmetic on a whole array cannot easily do. A cumulative running balance with periodic deposits is a simple example:

```
balance = 0.0
monthly_deposit = 200.0
monthly_rate = 0.004  # roughly 5% annual, compounded monthly

for month in range(1, 13):
    balance = balance * (1 + monthly_rate) + monthly_deposit

print(round(balance, 2))
# 2453.51
```

This is an inherently sequential calculation, next month's balance depends on this month's balance, so a loop (or an equivalent recursive formula) is the natural way to express it, in contrast to the vectorized return calculations in Section 2.3.1 below, which apply the same independent operation to every element of an array at once. Recognizing which category a calculation falls into, independent across observations versus sequentially dependent, is one of the most useful judgment calls a financial programmer makes, since applying a loop where vectorization would work is a common and avoidable source of slow code, discussed further in Section 2.6.1.

2.2.4 Core Data Structures

An analyst tracking even a handful of securities quickly needs more than single numbers: an ordered run of daily closing prices, a lookup from ticker symbol to sector, a fixed set of open/high/low/close values for one trading day that should never accidentally be reordered. Python provides several data structures built for these different needs. Lists store ordered collections of values. Dictionaries store mappings from keys to values. Tuples are immutable sequences, often used for fixed collections of quantities. For most financial analysis, however, the most important structure is the pandas `DataFrame`, which organizes tabular data in rows and columns.

A `DataFrame` is ideal for financial data because it resembles a spreadsheet. One column may contain dates, another may contain prices, and another may contain trading volume. The running example for this chapter is a small two-asset price history, built directly from Python lists so that it is fully self-contained and reproducible without any external file or internet connection:

Wulin.Suo@Queensu.ca

```
import numpy as np
import pandas as pd

dates = pd.date_range("2024-01-02", periods=5, freq="B")
prices = pd.DataFrame(
    {
        "AssetA": [100.00, 102.00, 101.00, 104.00, 103.00],
        "AssetB": [50.00, 50.50, 51.26, 51.00, 52.02],
    },
    index=dates,
)
print(prices)
#             AssetA  AssetB
# 2024-01-02  100.00   50.00
# 2024-01-03  102.00   50.50
# 2024-01-04  101.00   51.26
# 2024-01-05  104.00   51.00
# 2024-01-08  103.00   52.02
```

This makes it easy to filter observations, compute statistics, and merge datasets. For example, this `DataFrame` can hold daily closing prices for two assets, allowing analysts to compare their performance and, later in the chapter, to combine them into a portfolio.

2.2.5 Dictionaries, Tuples, and List Comprehensions

Suppose an analyst has just pulled together a handful of facts about one security, its ticker, sector, share count, whether it pays a dividend, long before that information is tidy enough to belong in a full `DataFrame`. Before every dataset becomes a `DataFrame`, it often starts life as a simpler Python structure. A dictionary maps descriptive keys to values and is a natural way to store metadata about a security:

```
security_info = {
    "ticker": "AAPL",
    "sector": "Technology",
    "shares_outstanding": 15_500_000_000,
    "is_dividend_payer": True,
}
print(security_info["sector"])
# Technology
```

A list of such dictionaries, one per security, is a common intermediate format before assembling a `DataFrame`, and `pandas` can convert it directly:

```
securities = [
    {"ticker": "AAPL", "sector": "Technology", "price": 190.50},
    {"ticker": "JPM", "sector": "Financials", "price": 145.20},
    {"ticker": "XOM", "sector": "Energy", "price": 102.75},
]
securities_df = pd.DataFrame(securities)
print(securities_df)
#   ticker      sector   price
# 0   AAPL  Technology  190.50
# 1    JPM  Financials  145.20
# 2    XOM      Energy  102.75
```

A tuple is similar to a list but immutable, meaning it cannot be modified after creation; this makes tuples a natural choice for representing a fixed pair of quantities that should never accidentally change, such as a (bid, ask) quote, `quote = (100.20, 100.35)`.

Wulin.Suo@Queensu.ca

List comprehensions provide a compact way to build a new list by applying an expression to each element of an existing one, and are used constantly in financial code as a more readable alternative to a short loop:

```
prices_list = [190.50, 145.20, 102.75]
price_labels = [f"${p:,.2f}" for p in prices_list]
print(price_labels)
# ['$190.50', '$145.20', '$102.75']

# a comprehension with a condition: tickers priced above $150
expensive = [s["ticker"] for s in securities if s["price"] > 150]
print(expensive)
# ['AAPL']
```

The pattern `[expression for item in iterable if condition]` reads almost like a sentence, and translating a plain-language filtering rule directly into a list comprehension is a skill that pays off constantly when preparing financial data for further analysis.

2.3 Numerical Computing with NumPy

Python's built-in containers are general-purpose and, for numerical work, slow. This part introduces the array, the data structure the rest of the book computes with, and the vectorized style of expressing conditions over it rather than looping.

2.3.1 Working with Numerical Arrays

Computing a return one day at a time in a Python loop works fine for the four prices used so far, but a real price history can span thousands of trading days across hundreds of securities, and a Python-level loop over every one of them is far slower than it needs to be. NumPy is the foundational package for numerical computation in Python. It provides arrays that are efficient and support vectorized operations. In finance, vectorization is very useful because many tasks involve applying the same operation to large collections of prices or returns, without writing explicit loops.

Given a series of daily prices $P_0, P_1, \ldots, P_T$, the simple (arithmetic) return in period t is

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}.$$

Applied to AssetA's five prices, this produces four daily returns:

$$R_1 = \frac{102.00 - 100.00}{100.00} = 2.00\%, \quad R_2 = \frac{101.00 - 102.00}{102.00} = -0.98\%,$$
$$R_3 = \frac{104.00 - 101.00}{101.00} = 2.97\%, \quad R_4 = \frac{103.00 - 104.00}{104.00} = -0.96\%.$$

The mean of these four returns is 0.76%, and their sample standard deviation (with $n - 1 = 3$ in the denominator) is 2.03%. In NumPy, this is computed directly on the underlying array without a Python-level loop:

```
asset_a = prices["AssetA"].to_numpy()
returns_a = (asset_a[1:] - asset_a[:-1]) / asset_a[:-1]
print(np.round(returns_a, 4))
# [ 0.02   -0.0098  0.0297 -0.0096]
print(round(returns_a.mean(), 4), round(returns_a.std(ddof=1), 4))
# 0.0076 0.0203
```

This is much faster and more readable than writing explicit loops for each element, and the use of arrays aligns well with the mathematical structure of quantitative finance, where calculations often operate on vectors of observations rather than single numbers.

Wulin.Suo@Queensu.ca

2.3.2 Boolean Indexing and Vectorized Conditionals

The classification function in Section 2.2.3 used an `if/elif` chain applied one return at a time, the kind of loop the previous section warned scales poorly once a price history grows large. NumPy and pandas offer a vectorized alternative that avoids the loop entirely. A boolean mask, an array of `True/False` values produced by comparing an array to a condition, can be used to select or count observations directly:

```
up_days = returns_a > 0
print(up_days)
# [ True False  True False]
print(f"Number of up days: {up_days.sum()}")
# Number of up days: 2
print(f"Average return on up days: {returns_a[up_days].mean():.4f}")
# Average return on up days: 0.0249
```

Summing a boolean array counts the number of `True` values, since Python treats `True` as 1 and `False` as 0, and using the mask to index the array selects only the matching elements. For a conditional calculation with more than two outcomes, `np.select` generalizes the earlier `if/elif` logic into a single vectorized expression:

```
conditions = [returns_a > 0.01, returns_a > 0, returns_a == 0]
choices = ["strong gain", "small gain", "flat"]
labels = np.select(conditions, choices, default="loss")
print(labels)
# ['strong gain' 'loss' 'strong gain' 'loss']
```

This produces the same classifications as the loop-based `classify_return` function in Section 2.2.3, applied to all four returns at once rather than one at a time. As datasets grow from a handful of observations to millions of rows of tick data, this difference, a single vectorized expression instead of a Python-level loop, becomes the difference between a script that runs in milliseconds and one that takes minutes, a point developed further in Section 2.6.1.

2.4 Handling Data with pandas

Financial data arrives as tables that are misaligned, differently sampled, and incomplete. This part covers the library built for exactly that: reshaping and joining tables, plotting what they contain, and cleaning what is wrong with them before any model sees it.

2.4.1 Data Manipulation with pandas

Real market data rarely arrives as clean as the toy price arrays used so far: a downloaded CSV may have gaps for holidays, dates out of order, or entire days missing for one security but not another, and none of the NumPy tools introduced so far know what to do with a date. pandas is the workhorse library for handling time series and structured data. It allows the analyst to load data from CSV files, databases, or online sources; to index by date; to sort observations; to handle missing values; and to compute rolling statistics. These capabilities are essential in finance because market data are often irregular and incomplete.

The pandas method `pct_change()` computes the same simple returns shown above, applied automatically to every column:

```
returns = prices.pct_change().dropna()
print(returns.round(4))
#             AssetA  AssetB
# 2024-01-03  0.0200  0.0100
# 2024-01-04 -0.0098  0.0150
# 2024-01-05  0.0297 -0.0051
# 2024-01-08 -0.0096  0.0200
```

Wulin.Suo@Queensu.ca

Notice that AssetB's returns (0.0100, 0.0150, −0.0051, 0.0200) move in almost the opposite direction from AssetA's returns (0.0200, −0.0098, 0.0297, −0.0096) on three of the four days. This pattern will reappear as a strong negative correlation later in the chapter, and it is the entire reason diversification will turn out to help so much in this example, although, as Section 2.5.1 will show, four observations pin that correlation down only very loosely.

This simple transformation, from a series of price levels to a series of returns, underlies much of modern empirical finance. Returns are more convenient than levels for many statistical models, especially because they are approximately stationary and easier to compare across assets of very different price scales.

2.4.2 Combining and Reshaping Data: merge, groupby, and resample

Real analyses rarely involve a single tidy table; more often, data about the same securities arrives from several sources and must be combined before it is useful. Suppose one table holds each security's sector and price, and a second holds the number of shares held in a portfolio:

```
securities_df = pd.DataFrame([
    {"ticker": "AAPL", "sector": "Technology", "price": 190.50},
    {"ticker": "JPM", "sector": "Financials", "price": 145.20},
    {"ticker": "XOM", "sector": "Energy", "price": 102.75},
    {"ticker": "MSFT", "sector": "Technology", "price": 375.10},
])
shares = pd.DataFrame({
    "ticker": ["AAPL", "JPM", "XOM", "MSFT"],
    "shares_held": [100, 50, 200, 30],
})

merged = securities_df.merge(shares, on="ticker")
merged["position_value"] = merged["price"] * merged["shares_held"]
print(merged)
#   ticker      sector   price  shares_held  position_value
# 0   AAPL  Technology  190.50          100         19050.0
# 1    JPM  Financials  145.20           50          7260.0
# 2    XOM      Energy  102.75          200         20550.0
# 3   MSFT  Technology  375.10           30         11253.0
```

The `merge` method aligns the two tables on the shared `ticker` column, the same logic as a SQL join, and is the standard way to combine data that arrives from different sources or systems. Once combined, `groupby` aggregates the result by a categorical column, here computing total position value by sector:

```
sector_summary = merged.groupby("sector")["position_value"].sum()
print(sector_summary)
# sector
# Energy        20550.0
# Financials     7260.0
# Technology    30303.0
# Name: position_value, dtype: float64
```

This immediately reveals that the portfolio's Technology exposure ($30,303, combining AAPL and MSFT) is its largest sector concentration, a question that would require considerably more manual bookkeeping without `groupby`.

A related operation, `resample`, aggregates a time-indexed series to a coarser frequency, for example converting daily prices to weekly prices by taking the last observation in each week:

```
extended_dates = pd.date_range("2024-01-02", periods=10, freq="B")
extended_prices = pd.Series(
```

Wulin.Suo@Queensu.ca

```
    [100.00, 102.00, 101.00, 104.00, 103.00, 105.00, 107.00, 106.00, 108.00, 110.00],
    index=extended_dates, name="AssetA",
)
weekly_last = extended_prices.resample("W-FRI").last()
print(weekly_last)
# 2024-01-05    104.0
# 2024-01-12    108.0
# 2024-01-19    110.0
# Freq: W-FRI, Name: AssetA, dtype: float64

weekly_return = weekly_last.pct_change().dropna()
print(weekly_return.round(4))
# 2024-01-12    0.0385
# 2024-01-19    0.0185
```

Computing weekly returns this way, from resampled weekly prices, gives exactly the same answer as compounding the underlying daily simple returns within each week, $\prod_t(1 + r_t) - 1$; for the week ending January 12, both give 0.0385. It is not the same as summing the daily returns, which gives 0.0384 here and is only an approximation for simple returns. Resampling is typically the more direct and less error-prone method when the analysis genuinely only needs the coarser frequency.

2.4.3 Visualization

A table of a hundred daily returns is hard to read at a glance; a suspicion that two assets tend to move in opposite directions, or that a return series has grown more volatile recently, is often invisible in a column of numbers but obvious the instant it is plotted. Visualization is a critical part of financial analysis. Charts help reveal patterns in returns, volatility, trends, and correlations. Matplotlib is the standard library for creating static plots.

The chapter's own `AssetA`/`AssetB` example has only four daily returns, enough to illustrate the portfolio formulas in Section 2.5.1 by hand, but too few to make a histogram or scatter plot visually informative. To show what these chart types actually look like, this section simulates a longer, one-year (252 trading day) two-asset price history with a similar negative correlation between the assets, used only for the three plots below:

```
rng = np.random.default_rng(6)
n_days = 252
mean_returns = [0.0006, 0.0004]
vols = [0.015, 0.010]
corr = -0.6
cov = [[vols[0]**2, corr*vols[0]*vols[1]],
       [corr*vols[0]*vols[1], vols[1]**2]]

daily_returns = rng.multivariate_normal(mean_returns, cov, size=n_days)
dates = pd.bdate_range("2024-01-02", periods=n_days)
returns_viz = pd.DataFrame(daily_returns, index=dates, columns=["AssetA", "AssetB"])
prices_viz = pd.DataFrame({
    "AssetA": 100 * (1 + returns_viz["AssetA"]).cumprod(),
    "AssetB": 50 * (1 + returns_viz["AssetB"]).cumprod(),
})
```

A simple line chart of the two price series and their cumulative returns can quickly reveal whether the assets move together or apart. Because AssetA and AssetB start at very different price levels ($100 versus $50), the left panel plots each on its own y-axis, using `twinx()`, so that both series' fluctuations remain visible rather than one dwarfing the other on a shared scale:

Wulin.Suo@Queensu.ca

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

ax_a = axes[0]
line_a, = ax_a.plot(prices_viz.index, prices_viz["AssetA"],
                    color="tab:blue", label="AssetA")
ax_a.set_ylabel("AssetA price", color="tab:blue")
ax_a.tick_params(axis="y", labelcolor="tab:blue")
ax_a.set_title("Prices")

ax_b = ax_a.twinx()
line_b, = ax_b.plot(prices_viz.index, prices_viz["AssetB"],
                    color="tab:orange", label="AssetB")
ax_b.set_ylabel("AssetB price", color="tab:orange")
ax_b.tick_params(axis="y", labelcolor="tab:orange")
ax_a.legend(handles=[line_a, line_b], loc="upper left")

(1 + returns_viz).cumprod().plot(ax=axes[1], title="Cumulative growth of $1")
plt.tight_layout()
plt.savefig("ch2_fig_prices_growth.png", dpi=150)
```

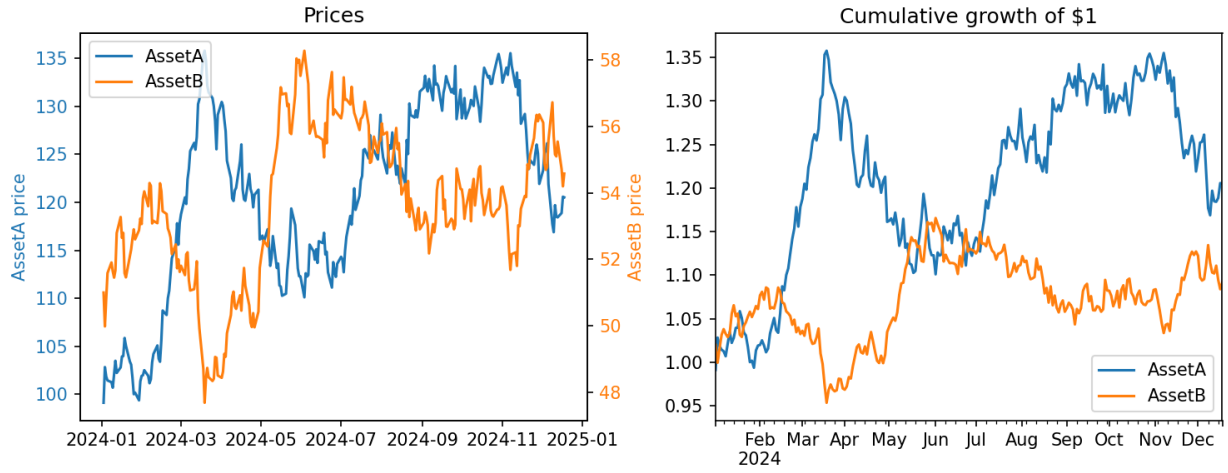


Figure 2.1: Simulated AssetA and AssetB prices, each on its own y-axis (left); cumulative growth of $1 invested in each (right), over a 252-day history

Visualization is not merely decorative. It is part of the analytical process. A model may appear strong in a statistical summary, but a plot may reveal outliers, regime shifts, or data problems that would otherwise go unnoticed. In finance, where data quality and market context matter greatly, visualization is an essential discipline, and it is often the fastest way to notice that two return series are moving in opposite directions, as AssetA and AssetB do here.

Two other chart types are used constantly in financial analysis. A histogram of returns shows the shape of the return distribution, revealing skewness or heavy tails that summary statistics alone can obscure:

```
fig, ax = plt.subplots(figsize=(6, 4))
returns_viz["AssetA"].hist(ax=ax, bins=20)
ax.set_title("Distribution of AssetA daily returns")
```

Wulin.Suo@Queensu.ca

```
ax.set_xlabel("Return")
plt.tight_layout()
```

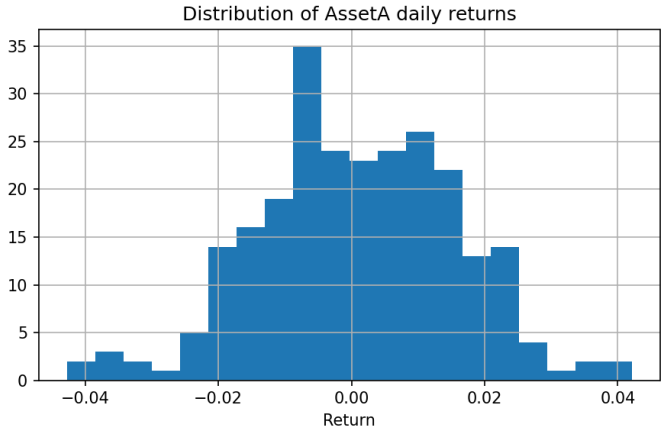


Figure 2.2: Distribution of AssetA's simulated daily returns over the 252-day history

Because the simulated returns are drawn from a normal distribution, Figure 2.2 can show neither skewness nor heavy tails, only sampling noise around a bell shape. Real daily returns typically have noticeably heavier tails than a normal distribution, which is exactly the feature a histogram of real data would expose, and the one the market-risk measures of Chapter 8 have to contend with.

A scatter plot of one asset's returns against another's is the most direct way to visualize a correlation like the one computed numerically in Section 2.5.1:

```
fig, ax = plt.subplots(figsize=(5, 5))
ax.scatter(returns_viz["AssetA"], returns_viz["AssetB"], alpha=0.6)
ax.set_xlabel("AssetA return")
ax.set_ylabel("AssetB return")
ax.set_title(f"Correlation: {returns_viz['AssetA'].corr(returns_viz['AssetB']):.3f}")
plt.tight_layout()
```

Figure 2.3 shows the cloud of 252 points sloping visibly downward, consistent with the simulated correlation of -0.641 printed in the plot title: a negative correlation means the two assets tend to move in opposite directions. With a real analysis, the same chart becomes a genuinely useful diagnostic for spotting nonlinear relationships or clusters that a single correlation number, a linear summary, cannot capture. The chapter's own four-observation example (Section 2.5.1) shows the same qualitative pattern with an even stronger correlation of -0.896, but has too few points to plot informatively.

2.4.4 Data Cleaning and Preparation

Raw financial data often contains issues: missing entries, duplicate observations, outliers, inconsistent dates, or formatting errors. Data cleaning is therefore a major part of practical work. A well-designed pipeline may involve aligning multiple time series, filling missing observations carefully, adjusting for corporate actions such as dividends and splits, and removing clearly erroneous values.

Figure 2.4 shows a simple financial data workflow, and the code below shows a typical cleaning step: forward-filling a small gap in a price series (for example, caused by a data feed outage) rather than silently dropping the observation, which would misalign the two return series.

```
raw = prices.copy()
raw.loc["2024-01-04", "AssetB"] = np.nan  # simulate a missing quote
```

Wulin.Suo@Queensu.ca

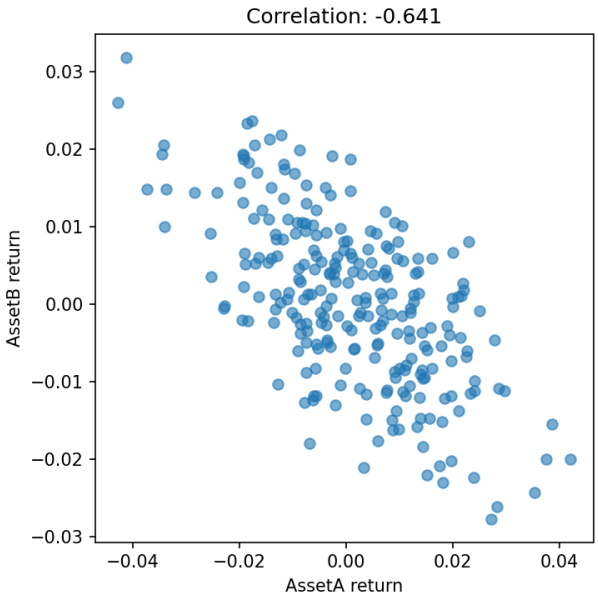


Figure 2.3: Simulated AssetA daily returns plotted against simulated AssetB daily returns

```
cleaned = raw.ffill()
print(raw["AssetB"].isna().sum(), "missing value(s) before cleaning")
print(cleaned["AssetB"].isna().sum(), "missing value(s) after cleaning")
# 1 missing value(s) before cleaning
# 0 missing value(s) after cleaning
```



Figure 2.4: A typical financial data pipeline moves from raw data to cleaned inputs and then to modeling

Forward-filling is appropriate for a short gap in an otherwise liquid series, but it is not always the right choice: forward-filling an illiquid security for many consecutive days can create the false impression of zero volatility. A price series for a stock must also often be adjusted for splits and dividends to produce a consistent total return series; otherwise, the historical performance may be distorted by mechanical price drops that have nothing to do with economic losses. This is an important lesson for quantitative finance: even a sophisticated model is only as good as the data that feed it.

2.5 Portfolio Analysis in Python

With the tools in place, this part puts them to work on the problem the rest of the book returns to repeatedly: constructing a portfolio from return data, evaluating it, and scaling the same code from two assets to many.

2.5.1 A Worked Example: Building and Evaluating a Two-Asset Portfolio

This section ties the chapter's running example together into a small but complete analysis, the kind of workflow an analyst repeats constantly, just at a larger scale. Suppose an analyst wants to evaluate an equally weighted portfolio of AssetA and AssetB.

Wulin.Suo@Queensu.ca

```
weights = np.array([0.5, 0.5])
portfolio_returns = returns.dot(weights)
print(portfolio_returns.round(4))
# 2024-01-03    0.0150
# 2024-01-04    0.0026
# 2024-01-05    0.0123
# 2024-01-08    0.0052
# dtype: float64

port_mean = portfolio_returns.mean()
port_vol = portfolio_returns.std(ddof=1)
print(round(port_mean, 4), round(port_vol, 4))
# 0.0088 0.0058
```

Compare the portfolio's volatility of 0.58% to the weighted average of the two assets' individual volatilities, $0.5 \times 2.03\% + 0.5 \times 1.08\% = 1.56\%$. The portfolio is far less volatile than either asset alone, and much less volatile than a naive average of their volatilities would suggest. This is the diversification effect introduced conceptually in Chapter 1, and it can now be made precise. The covariance between the two return series is

```
cov_matrix = returns.cov()
print(cov_matrix.round(6))
#          AssetA    AssetB
# AssetA  0.000414 -0.000198
# AssetB -0.000198  0.000118

corr = returns.corr().iloc[0, 1]
print(round(corr, 3))
# -0.896
```

so the two assets have a strong negative correlation of about -0.90. That estimate rests on only four pairs of returns, however, and a correlation estimated from so few observations is very imprecise: the standard Fisher-z 95% confidence interval runs from about -0.998 to $+0.467$, wide enough to include zero and even a moderately positive correlation. The example is chosen so that every number can be checked by hand, not as evidence about any real pair of assets, and Chapter 12 returns to the damage that this kind of estimation error does to portfolio optimization. The variance of an equally weighted two-asset portfolio is given by

$$\sigma_p^2 = w_A^2 \sigma_A^2 + w_B^2 \sigma_B^2 + 2 w_A w_B \sigma_{AB},$$

and substituting $w_A = w_B = 0.5$, $\sigma_A^2 = 0.000414$, $\sigma_B^2 = 0.000118$, and $\sigma_{AB} = -0.000198$ gives

$$\sigma_p^2 = 0.25(0.000414) + 0.25(0.000118) + 2(0.25)(-0.000198) = 0.000034,$$

so $\sigma_p = \sqrt{0.000034} \approx 0.58\%$, exactly matching the value computed directly from the portfolio return series. This equivalence, between the formula and the code, is a useful checkpoint: whenever a hand-derived formula and a direct computation on the data disagree, there is a bug somewhere, either in the formula or in the code, and reconciling the two is one of the most valuable debugging habits a quantitative analyst can build.

The ability to conduct such analyses quickly, and to extend them immediately to many more assets and much longer histories, is one reason Python is widely used in finance. It allows researchers to iterate quickly, test ideas, and communicate results in a reproducible way. In a domain where decisions are often made under time pressure, this speed matters.

2.5.2 Scaling Up: From Two Assets to a Small Portfolio Universe

The two-asset example above was deliberately small so that every number could be checked by hand. The code, however, is written in a way that scales to any number of assets without modification, which is worth demonstrating explicitly. Adding a third asset, AssetC, requires only adding a column to the original DataFrame:

Wulin.Suo@Queensu.ca

```
prices["AssetC"] = [200.00, 198.00, 202.00, 199.00, 203.00]
returns3 = prices.pct_change().dropna()
print(returns3.round(4))
#             AssetA  AssetB  AssetC
# 2024-01-03  0.0200  0.0100 -0.0100
# 2024-01-04 -0.0098  0.0150  0.0202
# 2024-01-05  0.0297 -0.0051 -0.0149
# 2024-01-08 -0.0096  0.0200  0.0201
```

Not a single line of the return, covariance, or portfolio-variance calculation needs to change; `pct_change()`, `.cov()`, and matrix multiplication all operate the same way regardless of how many columns the DataFrame has:

```
weights3 = np.array([1/3, 1/3, 1/3])
port3 = returns3.dot(weights3)
print(port3.round(4))
# 2024-01-03    0.0067
# 2024-01-04    0.0085
# 2024-01-05    0.0033
# 2024-01-08    0.0102

cov3 = returns3.cov()
port_var3 = weights3 @ cov3.values @ weights3
print(f"Equal-weighted 3-asset portfolio volatility: {np.sqrt(port_var3):.4f}")
# Equal-weighted 3-asset portfolio volatility: 0.0030
```

The expression `weights3 @ cov3.values @ weights3` is the matrix form of the portfolio variance formula, $w^\top \Sigma w$, which Section 2.6.3 restates for n assets, and it works identically whether w has 2, 3, or 200 entries. This is the practical payoff of writing analysis code in terms of vectors and matrices rather than named variables for each individual asset: a script built this way for a two-asset illustration is already, without any rewriting, the same script a researcher would use on a portfolio of hundreds of securities, which is the setting Chapter 12's portfolio optimization methods are built to handle.

Object-Oriented Programming: A Reusable `Portfolio` Class

The procedural style used throughout this chapter, a sequence of variable assignments and function calls, works well for a single, one-off analysis, but repeating the same mean, volatility, and variance calculations for every new portfolio a researcher wants to evaluate invites the kind of copy-paste duplication Section 2.6.1's performance discussion and this chapter's reproducibility section both warn against. Object-oriented programming addresses this with a `class`: a template, defined once, that bundles a set of related data (here, a portfolio's returns and weights) together with the functions, called *methods* when they belong to a class, that operate on that data, so that every new portfolio can be built from the same reusable blueprint instead of copying and re-editing the same calculations by hand each time:

```
class Portfolio:
    def __init__(self, returns, weights):
        self.returns = returns
        self.weights = weights

    @property
    def portfolio_returns(self):
        return self.returns.dot(self.weights)

    @property
    def mean(self):
```

Wulin.Suo@Queensu.ca

```
        return self.portfolio_returns.mean()

    @property
    def volatility(self):
        return self.portfolio_returns.std(ddof=1)

    def variance_from_formula(self):
        cov = self.returns.cov().values
        return self.weights @ cov @ self.weights
```

Instantiating this class on the same two-asset data used throughout the chapter reproduces every number already computed by hand,

```
portfolio = Portfolio(returns, weights)
print(round(portfolio.mean, 4), round(portfolio.volatility, 4))
# 0.0088 0.0058
print(round(portfolio.variance_from_formula(), 6))
# 3.4e-05
```

but the real payoff appears when evaluating the three-asset portfolio built above: `Portfolio(returns3, weights3)` produces the correct mean, volatility, and formula-based variance for the three-asset case using the identical class definition, with no new code written at all. This is precisely what "scaling up" means in an object-oriented codebase: the `Portfolio` class is a single, tested abstraction that every future portfolio, of any size, can reuse, rather than a script that must be copied and re-edited each time a new set of assets needs evaluating, the kind of reusable component a production research or trading system is built from.

2.5.3 Working with Time Series and Financial Data

An analyst comparing two securities that trade on different exchanges, or trying to line up daily prices with a monthly economic release, quickly runs into a problem ordinary rows and columns do not solve on their own: which date does each observation actually belong to, and what should happen when one series has an entry for a day the other is missing? Many financial datasets are time series, meaning that observations are indexed by time and ordered sequentially. Stock prices, interest rates, exchange rates, and macroeconomic indicators all arrive as time-dependent data. Working with such data requires careful handling of date indexes, alignment, and missing observations. In Python, pandas is particularly useful because it provides a time-indexed data structure that makes these operations natural, including resampling to a different frequency and computing rolling statistics:

```
rolling_vol = returns["AssetA"].rolling(window=2).std()
print(rolling_vol.round(4))
# 2024-01-03       NaN
# 2024-01-04    0.0211
# 2024-01-05    0.0279
# 2024-01-08    0.0278
# dtype: float64
```

A common workflow is to import historical prices, convert them into a time series, compute returns, and then study autocorrelation, volatility clustering, and trends. These steps are essential because many financial models depend on the idea that the future is not independent of the past. Yet the time dependence in financial data is often subtle and nonstationary, which makes careful analysis, and the topics of Chapter 7, necessary.

2.6 Code That Holds Up

Code that produces the right number once is not the same as code that can be trusted, rerun, and handed to somebody else. This part covers performance, the errors financial data reliably provokes, the summary statistics worth computing, and the practices that make a result reproducible.

Wulin.Suo@Queensu.ca

2.6.1 Why Vectorization Matters: A Performance Comparison

Section 2.3.1 claimed that vectorized array operations are faster than an equivalent Python-level loop; it is worth demonstrating this claim rather than only asserting it. Consider computing the cumulative growth of $1 across 100,000 simulated daily returns, once with an explicit loop and once with NumPy's vectorized `cumprod`:

```
import time

rng = np.random.default_rng(0)
simulated_returns = rng.normal(0.0005, 0.01, 100_000)

start = time.perf_counter()
total = 1.0
loop_values = []
for r in simulated_returns:
    total *= (1 + r)
    loop_values.append(total)
loop_time = time.perf_counter() - start

start = time.perf_counter()
vectorized_values = np.cumprod(1 + simulated_returns)
vector_time = time.perf_counter() - start

print(f"Loop: {loop_time:.4f}s, Vectorized: {vector_time:.4f}s, "
      f"speedup: {loop_time/vector_time:.0f}x")
# Loop: 0.0394s, Vectorized: 0.0010s, speedup: 39x
```

Both approaches produce identical final values, since they compute the same mathematical quantity, but the vectorized version is roughly 39 times faster on this test (the exact figure is machine-dependent and will vary, but a factor of 10 to 100 is typical for this kind of calculation). The reason is that the Python-level loop pays interpreter overhead, argument checking, dynamic type dispatch, on every single one of the 100,000 iterations, while `cumprod` executes the equivalent loop once, in compiled C code, over contiguous memory. This gap widens, not narrows, as the dataset grows, which is precisely why Section 2.7's discussion of performance at scale emphasizes vectorized NumPy and pandas operations as the default approach, reserving explicit loops for calculations, like the sequential compounding example in Section 2.2.3, that are genuinely path-dependent and cannot be vectorized.

2.6.2 Handling Errors and Common Pitfalls

Financial data pipelines fail constantly, in small, mundane ways: a data feed returns a missing value, a ticker is misspelled, a division produces infinity because a denominator was zero. Python's `try/except` block lets a program catch such errors and respond deliberately rather than crashing outright:

```
def safe_return(price_now, price_before):
    try:
        return (price_now - price_before) / price_before
    except ZeroDivisionError:
        return float("nan")

print(safe_return(105.0, 100.0))
# 0.05
print(safe_return(105.0, 0.0))
# nan
```

Wulin.Suo@Queensu.ca

Returning `nan` (not-a-number) rather than letting the program crash allows downstream code, and pandas in particular, to continue processing the rest of a dataset while clearly marking the problematic observation, exactly like the missing-value handling in Section 2.4.4. A few pitfalls recur often enough in financial Python code to be worth naming explicitly:

- **Silent `NaN` propagation.** Almost any arithmetic operation involving `NaN` produces another `NaN` rather than an error, so a single bad data point can silently spread through an entire calculation (a covariance matrix, for example) without any visible warning.
- **Comparing floating-point numbers for exact equality.** Because of how computers represent decimal numbers, a calculation that should mathematically equal `0.3` may instead equal `0.30000000000000004`; use a tolerance-based comparison (`np.isclose`) rather than `==` when checking floating-point results.
- **Mutating a DataFrame while iterating over it.** Modifying rows of a DataFrame inside a loop over that same DataFrame can produce confusing, order-dependent results; it is almost always better to build a new column or DataFrame with a vectorized expression instead.
- **Off-by-one errors in lagged calculations.** Return, rolling-window, and lag calculations are especially prone to off-by-one mistakes, using today's price where yesterday's was intended, which can silently introduce look-ahead bias into a backtest, a problem Section 2.7 takes up below.

2.6.3 Descriptive Statistics and Portfolio Analytics

A return series that has been imported, cleaned, and plotted still leaves an obvious question unanswered: how risky does this asset actually look, in numbers precise enough to compare against another asset or feed into a portfolio calculation? Once data are imported and cleaned, Python can be used to calculate descriptive statistics such as averages, standard deviations, skewness, and kurtosis. These statistics summarize the distribution of returns and provide a starting point for risk analysis:

```
summary = returns.describe().round(4)
print(summary)
#        AssetA  AssetB
# count  4.0000  4.0000
# mean   0.0076  0.0100
# std    0.0203  0.0108
# min   -0.0098 -0.0051
# 25%   -0.0097  0.0062
# 50%    0.0052  0.0125
# 75%    0.0224  0.0163
# max    0.0297  0.0200
```

For a portfolio, one may also compute correlations between assets, covariance matrices, and portfolio volatility, just as in the worked example above. These quantities are central to diversification and asset allocation, and they generalize directly to portfolios with many assets: with n assets, weights $w \in \mathbb{R}^n$, and covariance matrix Σ, portfolio variance is $\sigma_p^2 = w^\top \Sigma w$, which is precisely the two-asset formula written in matrix form.

The advantage of Python here is that the same code applies unchanged whether there are 2 assets or 200. A single script can compute rolling volatilities, drawdowns, and Sharpe-like performance measures across a whole universe of securities. This efficiency is crucial when a researcher wants to compare strategies, evaluate risk, or test whether a signal remains useful after transaction costs.

2.6.4 Reproducibility, Version Control, and Good Practice

One of the most important lessons in computational finance is that analysis must be reproducible. A model or result that cannot be rerun by another analyst, or by the same analyst six months later, is of limited value.

Wulin.Suo@Queensu.ca

Python supports reproducibility through scripts, notebooks, and package management. A clear workflow includes:

- Recording exact package versions, for example in a `requirements.txt` or `environment.yml` file, so that a colleague (or a future you) can recreate the same environment.
- Saving raw data separately from cleaned or derived data, so that a cleaning step can be re-run or audited without re-fetching from the original source.
- Documenting transformations, ideally in code and comments rather than in a separate document that can drift out of sync with the script.
- Separating analysis from presentation, so that a notebook used to explore an idea is not the same artifact used to report a result to a manager or regulator.
- Using version control (such as Git) to track changes to code over time, including the ability to identify exactly which version of the code produced a given historical result.
- Fixing random seeds (as in `np.random.default_rng(0)` in Section 2.6.1) whenever a calculation involves simulation, so that two runs of the same script on the same data produce identical results rather than only similar ones.
- Writing small checks directly into a script or notebook, such as the put-call parity check used in Chapter 5 or the accounting-identity check used in Chapter 4, so that a silent bug is caught immediately rather than discovered much later when it has already propagated into a report or a trading decision.

Unit Testing: Making Checks Automatic

The "small checks" in the list above become far more valuable once they are written as an actual, automatically run test rather than a one-off `print` statement an analyst has to remember to inspect. A unit test is simply a function that exercises a piece of code and uses Python's `assert` statement to fail loudly if the result is not what is expected. Section 2.5.1's own observation, that the portfolio-variance formula and the direct computation from the return series agree, is exactly the kind of invariant worth encoding this way:

```
def test_portfolio_variance_matches_formula():
    weights = np.array([0.5, 0.5])
    cov = returns.cov().values
    var_A, var_B, cov_AB = cov[0, 0], cov[1, 1], cov[0, 1]
    formula_variance = (
        weights[0]**2 * var_A + weights[1]**2 * var_B
        + 2 * weights[0] * weights[1] * cov_AB
    )
    direct_variance = returns.dot(weights).var(ddof=1)
    assert abs(formula_variance - direct_variance) < 1e-10, \
        "Portfolio variance formula does not match direct computation!"
    print("Test passed: formula-based and direct portfolio variance agree.")

test_portfolio_variance_matches_formula()
# Test passed: formula-based and direct portfolio variance agree.
```

The specific tolerance, 10^{-10}, matters: it is loose enough to absorb ordinary floating-point rounding but tight enough to catch a genuine error, for example an accidentally transposed covariance term or a forgotten factor of two on the cross term, the kind of subtle bug that otherwise slips through silently. Python's own `statistics.mean` offers an example of how quietly such bugs arise: handed a list of NumPy integers such as prices of 100, 101, and 103, it returns the truncated integer 101 rather than 101.3333, with no error or warning, a wrong answer that only a check against an independently computed value, here `np.mean`, would catch. A single test like this one costs a few lines to write and runs in a fraction of a second,

Wulin.Suo@Queensu.ca

but it converts a fact an analyst has verified once, by hand, into a fact that is re-verified automatically every single time the surrounding code changes, catching a regression immediately rather than only when a downstream number looks suspicious. Production financial codebases typically collect dozens or hundreds of such tests, run automatically before any code change is accepted, using a dedicated testing framework such as `pytest` rather than calling test functions manually as shown here, but the underlying principle, encode a known-correct result as an executable check rather than a comment, is the same at any scale.

Reproducibility is not a bureaucratic nicety layered on top of the real analytical work; in a financial setting it is frequently a legal and professional requirement. A regulator investigating a trading loss, an auditor reviewing a valuation, or a portfolio manager questioning a backtested strategy will all expect to be able to trace a reported number back to the exact code and data that produced it. A workflow that cannot support this kind of retrospective scrutiny is a liability regardless of how sophisticated its underlying models are.

Good practice also means writing code that is understandable. Financial projects often involve several collaborators, long time horizons, and changing requirements. A readable function, a clear variable name, and a documented transformation can prevent errors and make the analysis easier to audit. In a regulated environment, this discipline is not optional; it is a professional necessity, since a regulator or auditor may ask exactly how a number was produced months or years after the fact.

2.7 Challenges in Applying Python to Financial Data

Several practical challenges arise repeatedly when Python is used for financial analysis, and they are worth naming explicitly before moving on to statistical and machine learning methods.

Data quality and vendor discrepancies. Different data vendors can report slightly different prices, adjust for corporate actions differently, or use different conventions for holidays and time zones. A workflow that silently trusts a single source can propagate vendor-specific errors into every downstream result.

Look-ahead and survivorship bias. It is easy to accidentally use information that would not have been available at the time of a decision, for example by using a company's current list of index constituents to study its past performance, which silently excludes firms that were delisted or went bankrupt. Both issues can make a backtest look far more profitable than any strategy could actually have been in real time.

Non-stationarity and changing data schemas. Return distributions, volatility regimes, and even the format of vendor data feeds change over time. Code written for one period's data conventions may silently produce wrong answers, rather than an error, when applied to a later period with a slightly different schema.

Performance at scale. A script that works well on five days of data for two assets, as in this chapter's example, may become far too slow when applied to years of tick-by-tick data across thousands of instruments. Vectorized NumPy and pandas operations help, but some tasks genuinely require more specialized tools, such as columnar file formats, database systems, or distributed computing frameworks.

Environment and dependency drift. A notebook that runs correctly today may fail in a year if underlying libraries change their behavior or drop support for older syntax. This is why pinning dependency versions and periodically re-validating old code are standard practice in production financial systems.

Latency and real-time constraints. Some applications, particularly in trading, require code to execute within milliseconds or microseconds. Python's convenience often comes at the cost of raw execution speed, which is why performance-critical components are sometimes implemented in compiled languages or accelerated with tools such as Numba or Cython, while Python remains the layer used for research, orchestration, and analysis.

Dependency and library version conflicts. A pandas or NumPy upgrade can silently change the default behavior of a function (for example, how missing values are handled in an aggregation), which means code that has not been re-tested against a new library version should not be assumed to still behave identically, even if it runs without raising an error.

The gap between exploratory and production code. Code written quickly in a notebook to explore an idea is rarely suitable, unmodified, for a system that will run unattended in production; exploratory code typically lacks the error handling, logging, and automated testing that a production financial system requires, and treating the two as interchangeable is a common source of operational incidents.

Wulin.Suo@Queensu.ca

2.8 Key Takeaways

Python is a practical and powerful tool for financial analysis because it supports efficient computation, flexible data handling, and reproducible workflows. The libraries in the Python ecosystem make it possible to move from raw market data to descriptive analysis, forecasting, and machine learning.

The chapter's worked example showed how a handful of pandas and NumPy operations, applied to just five days of prices, can reveal a meaningful diversification effect and how a formula derived on paper should match a direct computation on the data. For readers who aim to apply AI in finance, Python is not optional; it is a core capability, and the habits introduced here, vectorized computation, careful data cleaning, and reproducible workflows, scale directly to the much larger datasets used in later chapters.

A few distinctions from this chapter are worth restating because they recur, largely unchanged, in every later chapter. First, the choice between a loop and a vectorized operation is not a stylistic preference; it is a question of whether a calculation is independent across observations (vectorize it) or genuinely sequential, like the compounding balance example (a loop, or an equivalent closed-form formula, is required). Second, combining data from multiple sources, the `merge` and `groupby` operations in Section 2.4.2, is not a peripheral skill but a central one, since almost no real financial analysis works from a single, already-tidy table. Third, every code example in this chapter was written to be checked, either by hand, as with the four AssetA and AssetB returns, or by running the companion notebook and comparing outputs line by line; this habit of verifying rather than trusting a computation is the discipline that later chapters rely on when the calculations become too complex to check by hand at all, such as the maximum-likelihood-estimated GARCH models in Chapter 7 or the gradient-boosted trees in Chapter 6.

2.9 Suggested Exercises

1. Write a Python function that computes the future value of an investment given a price, an annual rate, and a number of years, and use it to verify the two examples in Section 2.2.2.

2. Recreate the `prices` DataFrame from Section 2.2.4, add a third asset, `AssetC`, with your own five prices, and compute its returns using `pct_change()`.

3. Using the covariance formula for portfolio variance, compute the volatility of a portfolio with weights 0.7 on AssetA and 0.3 on AssetB, and check your answer against a direct computation on `portfolio_returns`.

4. Modify the data-cleaning example to simulate two consecutive missing values in AssetB, apply forward-fill, and discuss whether this is still a reasonable choice.

5. Plot the cumulative growth of \$1 invested in AssetA, AssetB, and the equally weighted portfolio on the same chart, and describe what the chart shows about diversification that the summary statistics alone do not.

6. Explain why a positive correlation between two assets would change the diversification benefit computed in Section 2.5.1, and recompute the portfolio volatility assuming a correlation of $+0.90$ instead of -0.896, holding the individual volatilities fixed.

7. Describe one concrete situation in which look-ahead bias could enter a Python backtesting script without the analyst noticing, and propose a coding practice that would prevent it.

8. Rewrite the `classify_return` function from Section 2.2.3 as a single list comprehension using a conditional expression, and verify it produces the same labels for the four AssetA returns.

9. Using `np.select` as in Section 2.3.2, classify AssetB's four returns into "strong gain" ($> 1\%$), "small gain" ($> 0\%$), "flat," or "loss," and compare the result to AssetA's classification.

10. Extend the `securities_df` and `shares` example in Section 2.4.2 with one more security of your choosing, recompute `position_value`, and recompute the sector summary.

Wulin.Suo@Queensu.ca

11. Extend the `extended_prices` series in Section 2.4.2 with five more business days of your own prices, and resample the full series to weekly frequency using both `.last()` and `.mean()`; explain when each aggregation would be the more appropriate choice.

12. Using the pattern in Section 2.6.1, describe (without necessarily running the code) why a loop that appends to a Python list one element at a time is typically slower than pre-allocating a NumPy array and filling it by index, even though both avoid calling a vectorized function.

13. Using this chapter's `Portfolio` class, instantiate it with weights 0.7 on AssetA and 0.3 on AssetB, and verify that `portfolio.volatility` matches the answer you computed by hand in Exercise 3.

14. Add a second unit test, `test_weights_sum_to_one`, that asserts a portfolio's weights sum to 1.0 within a small tolerance, and explain what kind of real coding mistake this test would catch that the variance-formula test would not.

2.10 Further Reading

- McKinney, *Python for Data Analysis*. Written by pandas' original author, this is the definitive reference for the DataFrame, groupby, merge, and resample operations introduced in this chapter, covering many more edge cases and options than a single chapter can.

- VanderPlas, *Python Data Science Handbook*. A broader tour of NumPy, pandas, and visualization that fills in the general-purpose Python and data-science background this chapter assumes, alongside its finance-specific examples.

- Hilpisch, *Python for Finance*. Extends this chapter's two-asset portfolio example toward the derivatives pricing, risk management, and algorithmic trading applications covered later in this book, using the same core pandas and NumPy toolkit.

- Hilpisch, *Python for Algorithmic Trading*. A practical, code-heavy treatment of building and backtesting trading systems in Python, a natural next step after this chapter's vectorization and performance material for readers headed toward Chapters 13 and 14.

Wulin.Suo@Queensu.ca

Chapter 3

Financial Markets, Institutions, and Instruments

3.1 Introduction

Financial markets are the institutions through which capital is allocated, risk is transferred, and prices are discovered. They allow households to save, firms to finance investment, governments to fund spending, and investors to diversify exposure. A solid understanding of these markets is essential for anyone who wants to use data science or artificial intelligence in finance, because the objects of analysis are not abstract numbers but real contracts, incentives, and trading mechanisms.

In a broad sense, modern finance is built on a simple but powerful idea: people do not all have the same preferences, opportunities, or time horizons. Some agents want to consume today, others want to save for the future, and still others are willing to accept risk in exchange for compensation. Financial markets provide the mechanisms that connect these different needs. Without such markets, savings would be less productive, firms would find it harder to grow, and risk would be harder to manage.

This chapter introduces the basic structure of financial markets and the major classes of financial instruments. The discussion begins with the roles of institutions and participants, then moves to the major types of securities traded in modern markets, and finally explains how prices are formed in real trading systems. The goal is to provide a conceptual foundation that will be useful when later chapters examine pricing, portfolio management, risk, and trading strategies.

Three worked examples anchor the more mechanical sections of the chapter: a bid-ask spread decomposed into its adverse-selection and inventory-cost components, a hand-computed walk through a small limit order book showing exactly how an incoming order gets filled at one price or several, and a no-arbitrage bond-pricing example that shows what happens, concretely, when two ways of pricing the same cash flow disagree. A case study of the 2010 Flash Crash closes the chapter, tying the abstract mechanics of order flow and liquidity to a single, well-documented afternoon when they broke down in public.

The chapter's institutional material, who trades, what they trade, and where, builds the vocabulary that every later chapter assumes: Chapters 5 and 12 price and hold the debt, equity, and derivative instruments this chapter's "Financial Instruments" section catalogues, Chapters 13 and 14 return to the limit order book and price discovery mechanics worked through in Section 3.3.2 at execution and then microsecond speed, and Section 3.5.3's no-arbitrage logic recurs, in progressively more quantitative form, in every valuation and pricing chapter that follows.

Readers already comfortable with market structure may skim the descriptive sections, but the worked examples are used again later in the book and are worth working through by hand at least once.

3.2 The Market and Its Participants

A market is an institution before it is a dataset. This part sets out what financial markets are for, who shows up in them, the firms that intermediate between those participants, and the contracts they actually

Wulin.Suo@Queensu.ca

trade.

3.2.1 Why Financial Markets Exist

A financial market exists because economic agents have different needs and different time horizons. A household may wish to save for retirement, a firm may need funding for expansion, and a government may require resources to finance public projects. These parties do not all have the same opportunities or preferences. Markets create a mechanism for matching supply and demand for capital.

The role of a financial market is therefore not limited to moving money from one person to another. Markets also allow people to shift risk across time and across states of the world. For example, a farmer who fears a decline in crop prices can use a futures contract to lock in a sale price. A pension fund that wants stable long-term income can purchase government bonds. A young entrepreneur who needs capital to build a company can issue equity to investors who are willing to share the upside and downside of the venture.

Financial markets also provide information. Prices do not only reflect current supply and demand; they also encode expectations about future cash flows, interest rates, inflation, and corporate performance. As a result, financial markets are not merely venues for exchange. They are systems for aggregating dispersed information and converting it into prices that guide decisions.

For this reason, a market becomes more useful when participants can trade quickly and with confidence. Liquidity, transparency, and regulation matter. If a market is illiquid, prices may be unstable and trading costs may be high. If it lacks transparency, participants may face information asymmetry and make poor decisions. If it lacks rules, trust can erode and market participants may withdraw from the system.

3.2.2 Who Participates in Financial Markets

Financial markets are populated by a diverse set of participants, each with different objectives and constraints. Households and retail investors often save for retirement, education, or other future needs. Firms issue securities to raise funding for investment. Governments borrow to finance public spending or manage budget imbalances. Financial institutions such as banks, mutual funds, pension funds, and insurers intermediate between those who provide capital and those who need it. Chapter 1's Table 1.5 previewed these participant roles at a glance; this section develops each one in more depth.

Market makers play a particularly important role. A market maker provides liquidity by standing ready to buy and sell securities, usually quoting both a bid price and an ask price. In return for taking on inventory risk and exposure to adverse price movements, market makers earn the spread between the two prices. This spread is a measure of trading cost, but it is also a compensation for providing immediacy and continuity to the market.

Arbitrageurs and hedge funds also influence market behavior. Arbitrageurs seek to exploit pricing discrepancies across related assets or markets. Hedge funds often pursue more complex strategies that may involve leverage, short selling, or macroeconomic views. Their activity can improve pricing efficiency, but it also can increase volatility when markets are under stress.

Finally, regulators and central banks shape the environment in which markets operate. Regulators enforce rules on disclosure, conduct, and market integrity. Central banks influence short-term interest rates and can intervene in crises to support liquidity or stabilize the financial system. For this reason, market outcomes are not determined only by private incentives; they also depend on public policy.

3.2.3 Major Financial Institutions

Financial markets are organized around institutions that perform specialized functions. Banks provide lending services, facilitate payments, and transform maturity and liquidity profiles. Asset managers collect capital from investors and allocate it across portfolios of securities. Insurance companies pool risk by charging premiums in exchange for compensation when specified losses occur. Exchanges provide venues where orders can be matched, and central clearinghouses reduce counterparty risk by interposing themselves between buyers and sellers. Chapter 1's Table 1.3 summarized these institutions' roles at a glance; this section develops each one further.

Wulin.Suo@Queensu.ca

These institutions have different business models and incentives. A commercial bank, for example, earns money by borrowing at one rate and lending at another. An asset manager earns fees by managing assets on behalf of clients. An insurance company earns premiums and invests the proceeds while managing claims. Each institution therefore faces unique risks, including credit risk, market risk, liquidity risk, and operational risk.

The financial system is fragile when institutions become highly interconnected. A bank may lend to a hedge fund, which may trade derivatives with a broker, which may depend on short-term funding from another bank. Interconnections can make the system more efficient under normal conditions, but they can also transmit shocks quickly during periods of stress. This is one reason why understanding institutions is essential for understanding financial crises.

The interaction between these institutions and the broader economy is profound. When banks tighten credit, firms may reduce investment. When insurers become more cautious, certain risks may become harder to insure. When exchanges improve market structure, trading costs may fall and liquidity may improve. These changes are not merely technical. They affect real economic activity, corporate decisions, and household welfare.

One institutional shift of the past fifteen years deserves its own mention, because it has moved a growing share of the economy's actual lending outside every venue described so far. Since the 2008 crisis, tighter bank capital regulation has pushed banks out of significant portions of corporate lending, and dedicated *private credit* funds, asset managers raising long-lockup capital specifically to make loans directly to mid-sized companies, have grown into a multi-trillion-dollar market filling that space, alongside the older private equity industry that buys entire companies rather than lending to them. These private markets differ from everything else in this chapter in one way that matters enormously for this book: they generate almost none of the data its methods assume. A private loan trades rarely or never, so there is no price history; the borrower files no public statements, so there is nothing for ratio analysis beyond what the lender privately negotiates access to; and a fund holds tens of positions, not the thousands of observations a statistical credit model trains on. AI's footprint in private markets is accordingly real but narrow, mostly natural language processing applied to deal documents and covenants, and screening tools for sourcing borrowers, while underwriting and valuation remain judgment-heavy in the way public-market lending no longer is. It is a useful standing question for the rest of this book: whenever a market's defining feature is the absence of data, how much of the toolkit still applies, and which parts fail first.

3.2.4 Financial Instruments: The Core Contracts

Financial instruments are the contracts that are traded in markets. They can be classified in several broad categories, including debt securities, equity securities, derivatives, and alternative instruments. Each category serves a different economic purpose, and each has its own pricing logic. Chapter 1's Table 1.4 previewed the markets these instruments trade in; the rest of this section develops each instrument type in depth.

Debt Securities

Suppose a government wants to build a highway that will benefit taxpayers for decades but costs more than this year's tax revenue can cover, or a corporation wants to build a factory that will not turn a profit until it is finished. Both need a way to raise a large sum of money now against a promise to repay it, with something extra, over time, and both need that promise to be tradable, so that whoever lends the money today is not stuck holding an illiquid IOU for thirty years if their own circumstances change. Debt securities are the financial technology that solves this problem: a bond is a standardized, tradable loan, where the investor buying it lends money to the issuer in exchange for a stream of future payments and the eventual repayment of principal at maturity. Bonds can be issued by governments, corporations, or other entities, and each issuer carries a different level of credit quality.

Government bonds are often viewed as relatively safe, though they still carry interest-rate risk and inflation risk. Corporate bonds generally offer higher yields because they are exposed to default risk. The pricing of bonds is closely related to the time value of money and discounting. A bond with a fixed coupon becomes more valuable when market interest rates fall and less valuable when rates rise. This is why bond prices and yields move inversely.

Wulin.Suo@Queensu.ca

The bond market is also important because it provides a benchmark for the broader economy. Long-term government bond yields are often interpreted as signals of expected future growth and inflation. In this sense, bond markets are not only mechanisms for financing; they are also tools for interpreting macroeconomic conditions.

Equity Securities

A bond is a poor fit for a very different kind of firm: one whose value lies almost entirely in an uncertain future, a startup that might be worth nothing or might be worth billions, with no stable stream of cash flow today from which to make fixed interest payments at all. Locking such a firm into a bond's rigid repayment schedule could bankrupt it during a lean year even if its long-run prospects remain excellent. Equity securities solve this by letting investors share directly in the firm's fortunes rather than lending against a fixed promise: they represent ownership claims in firms, and a common stock share gives the holder a claim on future profits and, in many cases, a vote in certain corporate matters. Equity investors are residual claimants, meaning they are paid only after other obligations, such as debt payments, have been satisfied. As a result, equities are generally riskier than bonds but offer the possibility of greater upside.

The valuation of stocks is more complicated than that of bonds because future dividends and firm value are uncertain. Investors often use dividend discount models, earnings-based valuation, or relative valuation approaches. In all cases, the underlying logic is that the value of an equity claim depends on expectations about future cash flows and the discount rate. A stock that promises strong future growth may command a high price even if current profits are modest.

Equity markets are also important because they provide a way for firms to raise permanent capital. Unlike debt, which requires contracts and scheduled payments, equity does not have a contractual maturity. This flexibility makes equity attractive for firms that expect growth, but it also means that shareholders bear more uncertainty than lenders.

Derivatives

Neither a bond nor a stock, on its own, lets a wheat farmer lock in today what price they will receive for a harvest still six months from being planted, or lets an airline protect itself against a jet-fuel price spike without actually buying and storing barrels of oil it has nowhere to put. What both need is a contract whose payoff is tied to something else's future price, without requiring either party to own that underlying asset outright in the meantime. Derivatives are this: contracts whose value depends on the performance of another asset or variable. Common examples include forwards, futures, options, and swaps. A forward contract obligates the buyer and seller to exchange an asset at a future date for a predetermined price. A futures contract performs a similar role but is standardized and traded on an exchange. An option gives the holder the right, but not the obligation, to buy or sell an asset at a specified price before or at a certain date.

Derivatives are used for hedging, speculation, and arbitrage. A producer may use futures to lock in a selling price. A portfolio manager may use options to protect against downside risk. A trader may use derivatives to express a view on volatility or market direction. Because derivatives often embed leverage, they can magnify gains and losses, which makes them important tools for both risk management and risk creation.

The economic importance of derivatives is difficult to overstate. By allowing parties to transfer risk more precisely, they expand the range of risk management strategies available to firms and investors. However, they can also create hidden exposures if participants misunderstand their payoff structure. This is one reason why derivatives are closely monitored by regulators and risk managers.

Swaps

Suppose a company has already issued floating-rate debt, so its interest payments rise and fall with market rates, but its own revenue is stable and predictable, and it would much rather know exactly what it owes each quarter than gamble on where rates go next. Reissuing the debt itself, paying off the old loan and negotiating a brand-new fixed-rate one, would be slow, expensive, and might not even be possible if the lender is unwilling to renegotiate. A swap solves this without touching the underlying loan at all: it is an

Wulin.Suo@Queensu.ca

agreement between two parties to exchange cash flows according to a pre-specified formula, and the most common type by far is the interest rate swap.

In a plain-vanilla interest rate swap, one party pays a fixed rate on a notional principal amount while the other pays a floating rate, historically tied to LIBOR, a benchmark discontinued after regulators found panel banks had manipulated it and replaced by transaction-based Alternative Reference Rates such as the Secured Overnight Financing Rate (SOFR) in the U.S., with only the net difference actually changing hands. Critically, the notional principal itself is never exchanged; it exists only to determine the size of the interest payments.

Swaps are used primarily to change the interest rate exposure of an existing position without altering the underlying asset or liability. A company that has issued floating-rate debt but prefers the predictability of fixed payments can enter a swap to pay fixed and receive floating, effectively converting its floating-rate liability into a fixed-rate one from a cash-flow perspective. A pension fund seeking to match long-dated fixed liabilities might do the reverse. Currency swaps, credit default swaps (which transfer credit risk rather than interest-rate risk, the same credit risk measured directly with the default-probability and loss models of Chapter 9), and total return swaps extend this same basic idea, exchanging one set of cash flows for another, to other kinds of risk.

Swaps trade almost entirely over-the-counter rather than on exchanges, which historically made the market for them opaque; post-2008 reforms, discussed later in this chapter, pushed much of the standardized swap market toward centralized clearing specifically to reduce the kind of counterparty risk concentration that swaps had exposed during the financial crisis.

A Worked Example: Valuing an Existing Interest Rate Swap A plain-vanilla interest rate swap can be valued as the difference between two bonds: the fixed leg is valued exactly like a coupon bond, discounting each fixed payment plus a final notional repayment, while the floating leg, because its coupon resets to the prevailing market rate at each reset date, is worth par (its notional value) at any reset date, regardless of what has happened to interest rates since the swap began. Consider a two-year swap with a \$10 million notional and a fixed rate of 5%, agreed some time ago, being valued today, immediately after a reset, when the current one-year and two-year discount rates are 4% and 4.5% respectively, lower than the 5% fixed rate locked in at initiation. The fixed leg's annual coupon is $0.05 \times 10{,}000{,}000 = \$500{,}000$, so its present value is

$$PV_{\text{fixed}} = \frac{500{,}000}{1.04} + \frac{500{,}000 + 10{,}000{,}000}{1.045^2} \approx \$10{,}095{,}933.72,$$

while the floating leg, valued at par immediately after its reset, is worth exactly $PV_{\text{floating}} = \$10{,}000{,}000$. The swap's value to the party paying fixed and receiving floating is

$$V_{\text{fixed payer}} = PV_{\text{floating}} - PV_{\text{fixed}} = 10{,}000{,}000 - 10{,}095{,}933.72 \approx -\$95{,}933.72,$$

a loss to the fixed-rate payer, and an equal and opposite gain to the floating-rate payer, since market rates have fallen since the swap was struck, leaving the fixed-rate payer locked into paying 5% when the market now says 4–4.5% is fair. This is precisely the mechanism by which an existing swap's mark-to-market value changes over its life without either counterparty doing anything at all: pure movement in market interest rates redistributes value between the two legs, the same discounting logic Chapter 5 applies to bond pricing, applied here to the difference of two cash flow streams rather than a single one.

Other Instruments

Other instruments include money market securities, preferred shares, mortgage-backed securities, and exchange-traded funds. These products illustrate the breadth of modern finance. Some are designed for short-term liquidity management, while others combine exposure to many assets in a single tradable instrument. Exchange-traded funds, in particular, have become widely used because they offer diversification and low-cost access to broad market exposure.

The growth of index funds and exchange-traded funds has changed the structure of investing. Instead of selecting individual securities, many investors now buy broad baskets of assets that track an index. This shift has lowered costs, increased market participation, and changed the nature of price formation in some

Wulin.Suo@Queensu.ca

markets. It also shows that modern finance is not only about individual securities; it is also about packaging and distributing risk efficiently.

An exchange-traded fund's price is kept close to the value of its underlying holdings through a creation-and-redemption mechanism: authorized institutional participants can exchange a basket of the underlying securities for new ETF shares, or vice versa, whenever the ETF's market price drifts far enough from its net asset value to make doing so profitable. This arbitrage-like mechanism, similar in spirit to the no-arbitrage logic in Section 3.5.3, is what keeps an ETF tracking its underlying index closely under normal conditions, though it can break down during periods of severe stress if the underlying securities themselves become difficult to trade.

3.3 Market Microstructure

Zooming in from the market to a single transaction reveals machinery that every later chapter's execution assumptions depend on. This part covers how orders meet, what a large one costs as it walks the book, and where the bid-ask spread comes from.

3.3.1 How Markets Are Organized

Markets are not all structured in the same way. Some are organized as exchanges with centralized order books, while others are over-the-counter markets where participants trade directly with one another. In exchange-traded markets, buyers and sellers submit orders that are matched according to price and time priority. In over-the-counter markets, contracts are negotiated bilaterally and may be tailored to specific needs.

The distinction matters because it affects liquidity, transparency, counterparty risk, and transaction costs. Exchange-traded products are often easier to price and monitor, while over-the-counter trades may offer flexibility but require more careful risk management. For example, an exchange-listed stock can usually be traded quickly at a visible price, whereas a bespoke derivative contract may require negotiation, credit assessment, and collateral arrangements.

The study of these rules is called market microstructure. Even small changes in execution rules, fees, or order types can have meaningful effects on the quality of price discovery. A limit order, for example, may provide price improvement but may also fail to execute. A market order may execute immediately but at the cost of adverse price impact. These tradeoffs are central to both trading and research in finance.

A useful simple measure of trading cost is the bid-ask spread:

$$\text{Spread} = P_{\text{ask}} - P_{\text{bid}}.$$

This spread captures the cost of immediacy. When the spread is narrow, trading is cheaper and liquidity is stronger. When the spread is wide, the market is less liquid and the cost of trading is higher. For quantitative researchers, such quantities are often as important as the asset's return itself because they determine the practical profitability of a strategy.

3.3.2 A Worked Example: Walking the Limit Order Book

The bid-ask spread only describes the price of trading a single share. In practice, a limit order book lists multiple price levels on both sides, and a large order can "walk" through several levels, receiving a progressively worse average price. Table 3.1 shows a simplified order book for a stock.

The best bid is \$50.05 and the best ask is \$50.10, so the quoted bid-ask spread is \$0.05, consistent with the formula above. Figure 3.1 draws the same book as a depth chart, the standard way a trading desk actually visualizes it. Now suppose a trader submits a market buy order for 250 shares, more than the 100 shares available at the best ask. The order first consumes all 100 shares at \$50.10, then the remaining 150 shares at the next level, \$50.15:

$$\text{Average execution price} = \frac{100(50.10) + 150(50.15)}{250} = \frac{5{,}010 + 7{,}522.50}{250} = \$50.13.$$

Wulin.Suo@Queensu.ca

Asks (sell orders)			Bids (buy orders)		
Level	Price	Size (shares)	Level	Price	Size (shares)
1	\$50.10	100	1	\$50.05	150
2	\$50.15	200	2	\$50.00	250
3	\$50.20	300	3	\$49.95	200

Table 3.1: A simplified limit order book

This average price is \$0.03 above the best ask of \$50.10, roughly 6 basis points of price impact, purely a consequence of order size relative to the liquidity available at each price level, with no new information entering the market at all. A trader who needs to buy or sell a large position faces this tradeoff: submitting the entire order at once guarantees immediate execution but at a worse average price, while breaking the order into smaller pieces over time (the subject of the execution algorithms in Chapter 13) can reduce price impact at the cost of exposing the trader to price movements while the order is worked.

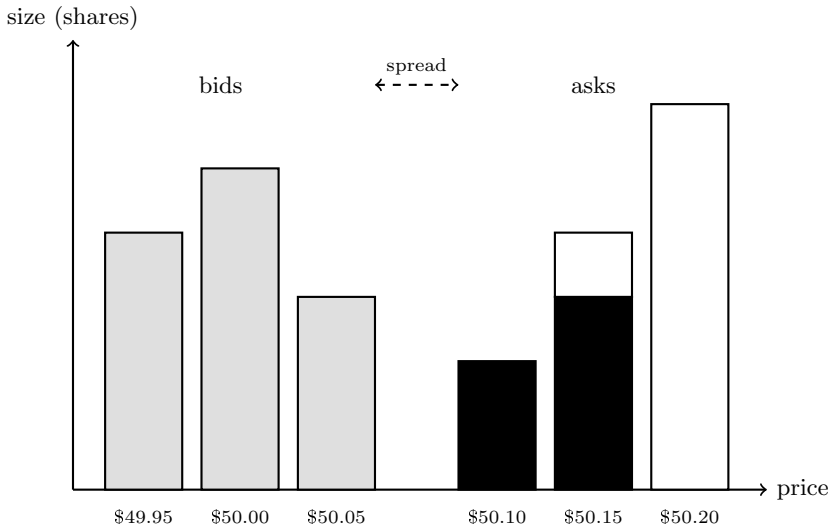


Figure 3.1: The limit order book of Table 3.1 as a depth chart. Bid sizes (light gray) sit left of the spread, ask sizes (white outline) right; the solid black portion of the two lowest ask levels marks what the 250-share market order consumes.

3.3.3 Decomposing the Bid-Ask Spread

The bid-ask spread is not pure profit for a market maker; it compensates for at least three distinct costs, and understanding the decomposition helps explain why spreads widen in some conditions and narrow in others.

- **Order-processing cost**: the operational cost of handling a trade, which is largely fixed regardless of market conditions.
- **Inventory-holding cost**: compensation for the risk a market maker bears by holding a non-zero position while waiting for an offsetting order to arrive, discussed further in Section 3.5.1.
- **Adverse-selection cost**: compensation for the risk of trading with a counterparty who has better information, since an informed trader will systematically pick off stale quotes.

Wulin.Suo@Queensu.ca

A trade price bounces back and forth between the bid and the ask as buy and sell orders arrive, even when nothing about the asset's underlying value has changed at all: a trade at the ask followed by a trade at the bid looks, in the raw price series, exactly like the price falling and then recovering, so a wider spread mechanically produces larger, more frequent back-and-forth price changes. Roll's (1984) implied spread estimator turns this bounce into a measurement tool, recovering the effective spread purely from observed price changes, without ever seeing the order book itself,

$$\text{Spread} \approx 2\sqrt{-\text{Cov}(\Delta P_t, \Delta P_{t-1})},$$

using the fact that this bid-ask bounce induces the kind of negative autocorrelation in consecutive price changes that the covariance term inside the square root captures. This is a useful reminder for the time series methods of Chapter 7: not every statistical pattern in price data reflects economically meaningful predictability, some of it is a direct, mechanical consequence of market microstructure. This book's companion technical note on Roll's estimator derives the formula rather than quoting it, showing that a constant half-spread applied according to the direction of each arriving trade makes observed price changes behave exactly like a first-order moving-average process, with an autocovariance of precisely $-s^2/4$ at lag one and exactly zero at every longer lag, and that bounce alone can never drive the first-lag autocorrelation below $-1/2$ however wide the spread.

3.3.4 The Economics of Price Discovery

Financial prices are the output of a complex process in which participants with different information and objectives update their beliefs. A stock price may rise because investors revise their expectations of future cash flows, because risk premiums change, or because a large buyer enters the market. The same price change can have multiple interpretations, which is why finance is both quantitative and interpretive.

Because prices are affected by information, incentives, and trading frictions, they do not always move smoothly. Sudden price movements can reflect news, uncertainty, algorithmic trading, or liquidity shocks. In the short run, prices may deviate from fundamental values because of temporary imbalance or limited liquidity. In the long run, however, competition and arbitrage tend to push prices toward values consistent with available information and economic fundamentals.

This is why economists and practitioners distinguish between fundamentals and short-run market dynamics. A fundamental analyst asks what an asset ought to be worth given expected cash flows and risk. A market microstructure analyst asks how trading rules and order flow shape the observed price path. Both perspectives are necessary for a complete understanding of finance.

Understanding price discovery is also important for AI applications in finance. A predictive model may identify patterns in observed prices, but without an understanding of the market mechanism behind those prices, the model may misinterpret noise as signal. The best applications of machine learning in finance combine statistical learning with economic and institutional insight. A model that predicts price movements may still fail if it ignores transaction costs, market impact, and changing liquidity conditions.

3.3.5 Why Market Structure Matters for Quantitative Finance

The practical relevance of this chapter becomes clear when one considers how financial models are used in the real world. A portfolio manager does not merely choose assets based on expected return and risk. The manager must also consider how easily those assets can be bought or sold, how much price impact trading will create, and whether the strategy can survive transaction costs.

The same logic applies to credit risk models, fraud detection systems, and forecasting algorithms. A model that performs well on historical data may still fail in real trading if it ignores the institutional setting in which decisions are implemented. This is why finance is not simply a data problem. It is a problem of data, incentives, constraints, and execution.

For quantitative readers, this point is especially important. Statistical methods can be powerful, but they are most useful when paired with an understanding of the underlying market mechanism. The most successful applications of AI in finance are not those that treat markets as a black box. They are those that combine rigorous modeling with financial intuition.

Wulin.Suo@Queensu.ca

A concrete illustration makes this abstract point specific. Suppose a machine learning model identifies a statistical pattern predicting that a small-cap stock will outperform over the next week, based purely on historical price and volume data.

Before acting on this signal, a quantitatively sophisticated practitioner asks several institutional questions that the model itself cannot answer: How wide is the bid-ask spread for this stock, and does the expected outperformance exceed the round-trip cost of trading it, using the spread decomposition introduced earlier in this chapter? How much of the stock's average daily volume would the intended position represent, and would establishing that position via the order-book mechanics described in Section 3.3.2 move the price enough to erode the expected gain? Is the stock easy to borrow if the strategy requires shorting, and are there restrictions on short selling of small-cap names during periods of stress?

None of these questions are visible in the historical price data the model was trained on, yet each one can determine whether a statistically significant signal is also an economically exploitable one. This is the precise sense in which market structure, not just statistical significance, determines whether a trading idea survives contact with a real market.

3.4 Market Infrastructure and Regulation

A trade is not finished when it is agreed. This part follows it through clearing and settlement, examines the businesses and rules that surround it, and connects that plumbing to where machine learning is actually deployed.

3.4.1 Money Markets, Capital Markets, and the Lifecycle of a Trade

Financial markets are often divided into money markets and capital markets. Money markets are used for short-term borrowing and lending, typically for maturities of less than one year. They include instruments such as Treasury bills, commercial paper, certificates of deposit, and repurchase agreements. These markets are crucial for liquidity management because banks, corporations, and governments frequently need to finance short-term obligations or invest excess cash for a brief period. Chapter 1's Table 1.4 previewed money markets alongside equity, bond, derivatives, and foreign exchange markets; this section and Chapter 5 develop each of these further.

Capital markets, by contrast, are used for long-term financing. They include equity and long-term debt markets, and they are central to funding business investment and public infrastructure. A firm that wishes to expand production may issue long-term bonds or equity. A government may issue sovereign debt to finance roads, schools, or defense spending. In both cases, the capital market allows the financing decision to be separated from the immediate cash position of the issuer.

Another useful distinction is between primary and secondary markets. In a primary market, new securities are issued for the first time. In a secondary market, existing securities are traded among investors. The primary market raises funds for the issuer, while the secondary market determines the price at which securities are traded after issuance. The existence of a liquid secondary market is important because it reduces uncertainty for investors and makes it easier for firms to issue securities in the first place.

The lifecycle of a trade illustrates the practical importance of market structure. A trader or investor first decides to buy or sell an asset. The order is routed through a broker to an exchange or an over-the-counter dealer. If the market is electronic and highly liquid, the order may be executed almost immediately. If the market is more fragmented or less liquid, the order may be partially filled or may require price concessions. Once the trade is complete, settlement occurs, which means that ownership is transferred and cash is exchanged. In many markets, settlement is not instantaneous; it may take a few days. During this period, counterparty and operational risk remain relevant.

This process is not purely mechanical. It reflects the interaction of incentives, technology, and regulation. For instance, high-frequency trading firms may compete to offer the best execution speed, while institutional investors may prioritize minimizing market impact. A retail investor may value simplicity and lower cost, whereas a hedge fund may be willing to bear more risk for a chance at a strategic advantage. All of these objectives interact within the same marketplace.

The repurchase agreement, or repo, deserves particular attention among money market instruments because of the central role it plays in the plumbing of the financial system. In a repo, one party sells a

Wulin.Suo@Queensu.ca

security, typically a government bond, to another party while simultaneously agreeing to repurchase it at a slightly higher price on a specified future date, often the very next day. Economically, this is a collateralized short-term loan: the cash borrower posts the security as collateral, and the difference between the sale and repurchase price is the effective interest rate, known as the repo rate. Banks, hedge funds, and other institutions rely on the repo market to fund large securities positions on a daily basis, which means that a disruption in repo markets, as occurred at several points during the 2008 crisis, can rapidly force institutions to sell assets they would otherwise have held, transmitting stress across the financial system in the way described in the systemic risk discussion below.

Concretely, suppose a dealer needs to borrow \$10,000,000 overnight and posts Treasury securities as collateral in exchange for cash at an annualized overnight repo rate of 5%. Using the money market convention of a 360-day year, the interest owed for one day is

$$\$10{,}000{,}000 \times 0.05 \times \frac{1}{360} = \$1{,}388.89,$$

so the dealer repurchases the collateral the next day for \$10,001,388.89. This tiny per-day interest amount, applied to an enormous notional and rolled over every business day, is why even a small rise in overnight repo rates, of the kind observed in September 2019 when repo rates briefly spiked well above the Federal Reserve's target range, can signal a meaningful shortage of cash available for this kind of short-term collateralized lending.

Settlement conventions are a second detail with outsized practical importance. Most equity markets settle trades on a T+1 or T+2 basis (one or two business days after the trade date), meaning that ownership and cash do not actually change hands at the moment a trade is agreed. This gap creates counterparty risk, the possibility that the other side of the trade fails to deliver cash or securities as promised, which is precisely the risk that the clearinghouses discussed in the next section exist to mitigate.

3.4.2 The Business of Exchanges and Market Data

It is easy to think of an exchange purely as neutral infrastructure, a venue where buyers and sellers meet, without asking how the exchange itself makes money. In practice, modern exchanges are for-profit businesses with several distinct revenue streams, and understanding them explains a number of features of market structure that would otherwise seem arbitrary.

Listing fees are charged to companies for the right to have their shares traded on a given exchange, and they give exchanges an incentive to attract high-quality, high-volume issuers. Transaction fees are charged per trade, often under a "maker-taker" model in which an exchange pays a small rebate to participants who post resting limit orders (adding, or "making," liquidity) and charges a slightly larger fee to participants whose orders execute immediately against those resting orders (removing, or "taking," liquidity). This fee structure is a direct, deliberate subsidy for market-making activity, and it materially affects the profitability calculations of the market makers discussed in Section 3.5.1.

Market data fees are a third, increasingly significant revenue source: exchanges sell real-time price and order-book data to trading firms, and the most detailed, lowest-latency feeds command substantial premiums over the free or delayed data available to retail investors. Finally, co-location fees, charging trading firms for the right to place their own servers physically inside or adjacent to the exchange's data center, directly monetize the speed advantage discussed in the technology and market design material later in this chapter; a firm that pays for co-location receives information about order flow and executes its own orders with lower latency than a firm trading from a remote location, purely as a function of the physical distance data must travel.

These revenue streams matter for a quantitative reader for two reasons. First, they explain why exchanges have strong commercial incentives to attract high-frequency trading activity specifically, since such activity generates outsized transaction and data-fee revenue relative to its economic footprint, which is part of the broader political and regulatory debate about whether current market structure appropriately balances the interests of high-frequency participants against those of long-term investors. Second, they are a reminder that the market data used throughout this book, and assumed to arrive at negligible cost in every worked example so far, is itself a commercial product whose cost, coverage, and latency vary enormously depending on how much a firm is willing to pay, a practical constraint that shapes what kind of strategy is even feasible for a given research budget.

Wulin.Suo@Queensu.ca

3.4.3 Clearing, Settlement, and Systemic Risk

Once a trade is executed, the market must ensure that the agreement is honored. Clearing refers to the process of determining obligations between counterparties, while settlement refers to the final exchange of cash and securities. In exchange-traded markets, a clearinghouse often acts as the central counterparty, guaranteeing the trade if one side defaults. This reduces bilateral credit risk, but it also creates a new concentration of risk in the clearinghouse itself.

The importance of clearing and settlement becomes most obvious during periods of stress. If many participants are forced to liquidate positions at once, the system may face liquidity shortages and margin calls. A small shock can therefore propagate through the network of institutions that depend on one another for financing and collateral. This is the core idea behind systemic risk: the failure of one institution or market segment can quickly undermine the stability of the broader financial system.

The 2008 global financial crisis showed how quickly seemingly isolated problems could become systemic. Mortgage defaults in the United States spread through securitization structures, derivatives markets, and funding markets, eventually affecting banks, insurers, and governments around the world. The lesson was not simply that risk had been mispriced, but that the financial system had become highly interconnected and insufficiently transparent. The crisis also highlighted the importance of regulations on leverage, capital, and liquidity.

A central clearinghouse manages its own concentrated risk primarily through margin requirements: each clearing member must post collateral, called initial margin, sized to cover the clearinghouse's potential loss if that member defaults under plausible adverse price moves, and must post additional variation margin daily (or, during volatile periods, intraday) to cover losses on open positions as prices move. This mechanism protects the clearinghouse, but it also means that a sharp, sudden price move can trigger simultaneous margin calls across many participants at once, forcing coordinated selling precisely when the market is least able to absorb it, a dynamic that played a visible role in various episodes of the 2008 crisis. Clearinghouses have themselves become large enough, and interconnected enough with their member banks, that regulators now treat major clearinghouses as systemically important institutions in their own right, subject to their own capital and risk-management standards.

The settlement machinery described in this section is also where one genuinely new piece of market plumbing has appeared. Asset tokenization means issuing a conventional security, a money-market fund share, a Treasury bill, a bond, as a digital token whose ownership record is a blockchain rather than an entry at a custodian or transfer agent, and the motivation is settlement economics rather than speculation: a tokenized security can settle in minutes rather than days, with delivery and payment executed atomically in a single transaction so that neither leg can fail on its own, collapsing both the intermediation cost and the counterparty-risk window this section exists to explain. Atomic settlement requires both legs to live on the same rails, which is where stablecoins enter, tokens engineered to hold a fixed value, almost always $1, against reserves of cash and short-term Treasuries. A stablecoin is economically a money-market fund share that has renounced yield in exchange for functioning as money, which means the quality of its reserves is exactly the credit question Chapter 9 would ask of any issuer, a question made vivid whenever a stablecoin has "broken the buck" and traded below its peg during a stress episode.

For a quantitative reader the consequence is concrete rather than futuristic. A settlement cycle collapsing from days to minutes changes collateral requirements, intraday liquidity management, and the practical meaning of counterparty exposure, all of which feed the risk models of Chapters 8 and 9; and the venues where tokenized instruments actually trade have market-making mechanics of their own, formula-driven rather than dealer-driven, taken up in Chapter 13.

3.4.4 Regulation, Disclosure, and Market Integrity

Markets require rules because they involve trust, information, and incentives. Without regulation, participants may exploit others through fraud, insider trading, or manipulation. Disclosure requirements help investors evaluate firms and make informed decisions. Capital and liquidity rules reduce the chance that institutions will become insolvent or unable to meet obligations during stress. Market surveillance systems detect suspicious behavior and support confidence in the fairness of the trading process.

The balance between free markets and regulation is a persistent theme in finance. Too little regulation may encourage excessive risk-taking and misconduct. Too much regulation may reduce innovation and

Wulin.Suo@Queensu.ca

impose unnecessary compliance costs. The most effective regulatory frameworks are therefore designed to protect integrity and stability while preserving the ability of markets to allocate capital efficiently.

From the perspective of AI and data science, regulation also matters because it shapes the data environment. Financial institutions must report certain information, comply with auditing standards, and provide transparent records. These requirements create rich datasets that can be used for forecasting, fraud detection, and risk analysis. At the same time, regulatory constraints limit what can be done with the data, especially when privacy, fairness, or market abuse concerns are involved.

A few specific regulatory frameworks are worth knowing by name because they recur throughout the rest of this book.

- **The Securities and Exchange Commission (SEC)** oversees U.S. securities markets and disclosure.
- **Regulation NMS** (National Market System) governs how orders are routed and executed across competing U.S. exchanges, directly shaping the market microstructure discussed throughout this chapter.
- **The Dodd-Frank Act**, passed in 2010 in response to the 2008 financial crisis, introduced the central clearing requirements for standardized swaps mentioned above, along with the Volcker Rule restricting proprietary trading by banks, and stress-testing requirements for large financial institutions that are directly relevant to the risk models in Chapters 8 and 9.
- **MiFID II** (the Markets in Financial Instruments Directive), in Europe, imposes extensive transparency and best-execution requirements on trading venues and investment firms.
- **Basel III** is an international framework setting minimum capital and liquidity requirements for banks, intended to make the banking system more resilient to the kind of interconnected shocks described in the systemic risk discussion above.

None of these frameworks is a topic this book covers in depth, but recognizing their names and general purpose is useful context for the compliance-related material in Chapter 15.

3.4.5 The Link Between Market Structure and AI

A modern AI system in finance cannot be designed as if markets were simple, frictionless, and fully observable. Real markets have heterogeneous participants, delayed information, hidden inventory, and strategic behavior. For example, an algorithm that trades based on price patterns may perform well in a calm market but fail in a stressed market where liquidity evaporates and spreads widen. A credit model that uses historical repayment data may become less reliable if underwriting standards change or if the economy enters a new regime.

This is why financial machine learning should be interpreted as an engineering problem embedded in an economic environment. The model is one component of a broader system that includes data pipelines, risk controls, execution logic, and compliance procedures. The same prediction can have very different consequences depending on how it is used. A recommendation that is harmless in a research setting may be dangerous in a live trading environment if it creates concentration risk or triggers herding behavior.

The chapter therefore closes with a practical lesson: the most valuable AI applications in finance are those that respect the structure of the market. They use data carefully, incorporate financial theory, and are evaluated not only by predictive accuracy but by their economic and operational consequences.

This connection between market structure and AI runs in both directions, and it is worth being explicit about the second direction as well. So far this section has emphasized how market structure constrains and shapes AI systems; AI systems, in turn, have themselves become part of the market structure they operate within.

The high-frequency market-making firms discussed later in this chapter increasingly rely on machine learning to set quotes and manage inventory risk in real time. Large asset managers use natural language processing, the subject of Chapter 16, to parse earnings calls and regulatory filings faster than human analysts could. Exchanges themselves use anomaly-detection systems, related to the fraud-detection methods of Chapter 15, to flag potentially manipulative trading patterns for regulatory review. This means that the "other participants" referred to throughout this chapter, whose behavior shapes liquidity, spreads, and price

Wulin.Suo@Queensu.ca

discovery, are themselves increasingly AI systems, and their collective behavior is part of what any new AI system entering the market must anticipate and adapt to.

This observation has a subtle but important consequence for model validation. A backtest evaluates a strategy against historical data generated by the market as it existed in the past, including the mix of human and automated participants active at that time. If a meaningful share of that historical liquidity and price-setting behavior came from other algorithms that have since been retired, upgraded, or replaced, then the market a new strategy will actually face going forward may behave differently from the market reflected in its backtest, not because of any change in economic fundamentals, but because the population of participants generating prices has itself changed. This is a more subtle version of the non-stationarity concern raised in Chapter 1 and developed further in Chapter 6, and it is specific to markets with a high degree of algorithmic participation.

3.5 Liquidity, Leverage, and Arbitrage

Prices hold together because someone is willing to trade against a discrepancy. This part covers who provides that liquidity and on what terms, what borrowing to trade costs when it goes wrong, and the no-arbitrage logic the rest of the book prices with.

3.5.1 Liquidity, Leverage, and the Economics of Market Making

Liquidity is one of the most important but least visible features of financial markets. A liquid market allows participants to trade quickly without moving the price too much. In practice, liquidity is a multidimensional concept:

- **Depth** is the volume that can be traded at a given price.
- **Immediacy** is the speed with which an order can be executed.
- **Resiliency** is the speed with which the market returns to a stable state after a shock.
- **Tightness** is the narrowness of the bid-ask spread.

A market with high liquidity is often easier to use for hedging, investment, and risk management, because traders can enter and exit positions more cheaply.

Market makers, introduced earlier in this chapter as participants who quote both a bid and an ask and earn the spread between them, do more than passively post two prices: they actively absorb temporary imbalances between buy and sell orders, buying when others want to sell and selling when others want to buy. This is where the inventory risk mentioned earlier actually bites. If the market maker purchases an asset and its price falls, the position loses value. If the market maker sells short and the price rises, losses can also accumulate quickly. The willingness of market makers to bear this risk is therefore central to market functioning.

Market-making decisions are shaped by information asymmetry. When informed traders possess better information than others, market makers face adverse selection. An informed trader who knows that an asset is likely to rise may be more likely to buy, while an informed trader who knows that an asset is likely to fall may be more likely to sell. The market maker, who does not know whether a trade conveys good or bad information, may respond by widening spreads and reducing participation. This mechanism is one reason why trading costs often rise in periods of uncertainty.

Leverage is another essential concept in this setting. A trader who borrows funds to increase position size can magnify gains, but can also magnify losses. Margin requirements are designed to reduce the chance that a leveraged position becomes too large relative to the trader's equity. When prices move against the position, a margin call may force liquidation. That forced selling can amplify price movements and create additional stress in the market. This is why leverage, while useful for efficient investment and speculation, is also a source of instability when risk management is weak.

Wulin.Suo@Queensu.ca

3.5.2 A Worked Example: Short Selling and Margin Calls

Leverage and margin are easiest to understand through a concrete short sale. Suppose a trader believes a stock currently priced at \$50 is overvalued and borrows 100 shares to sell short, receiving proceeds of $100 \times \$50 = \$5{,}000$. Under a typical initial margin requirement of 150% of the value of the position (the U.S. Regulation T standard), the trader must maintain total account equity of at least $1.5 \times \$5{,}000 = \$7{,}500$: the \$5,000 in short-sale proceeds plus an additional \$2,500 of the trader's own cash deposited as margin.

As the stock price P moves, the trader's liability, the cost of buying back the 100 borrowed shares, is $100P$, while total account equity remains the fixed \$7,500 credit balance minus that liability:

$$\text{Equity}(P) = 7{,}500 - 100P.$$

For short positions in stocks priced at \$5 or more, FINRA Rule 4210 requires a maintenance margin of at least 30% of the current value of the short position (many brokers require more), meaning $\text{Equity}(P)/100P \geq 0.30$. Setting this ratio equal to exactly 0.30 and solving for P gives the price at which a margin call is triggered:

$$7{,}500 - 100P = 0.30(100P) \quad \Longrightarrow \quad 7{,}500 = 130P \quad \Longrightarrow \quad P \approx \$57.69.$$

At this price (7,500/130 at full precision), equity has fallen to \$1,730.77, exactly 30% of the trader's liability, confirming the calculation. A price increase of only 15.38%, from \$50 to \$57.69, is enough to trigger a margin call on a position that started with 150% initial margin, illustrating precisely how leverage compounds a short seller's losses: the loss on the position itself is \$769.23, but it consumes 30.77% of the trader's own \$2,500 of capital, twice the adverse price move in percentage terms. If the trader cannot deposit additional funds once the call is issued, the broker will buy back some or all of the borrowed shares on the trader's behalf, often at whatever price is available, the same kind of price-insensitive trading into a moving market that this chapter's Flash Crash case study describes and, at the scale of an entire highly leveraged fund rather than a single position, in the LTCM episode discussed later in this chapter.

Variation margin, the clearinghouse mechanism introduced in Section 3.4.3, differs from the broker margin just worked through in one important respect: rather than being called only once a position breaches a maintenance threshold, it is settled to zero at the end of every trading day, marking every open position to its current market price.

Suppose a trader buys one stock-index futures contract with a notional value of \$50,000 and posts an initial margin of \$5,000 (10% of notional). If the index falls 2% that day, the trader's mark-to-market loss is $0.02 \times \$50{,}000 = \$1{,}000$, reducing the margin balance to $\$5{,}000 - \$1{,}000 = \$4{,}000$; the clearinghouse issues a variation margin call for exactly \$1,000 to restore the balance to \$5,000 before trading resumes the next day. Because this settlement happens daily rather than only at a maintenance-breach threshold, losses on a futures position never accumulate silently the way they can in the short-sale example above, but it also means that a clearing member with many leveraged futures positions faces a new, and potentially large, daily cash obligation every time the market moves sharply against it, which is precisely the liquidity-drain mechanism that the systemic-risk discussion of Section 3.4.3 refers to.

Table 3.2 summarizes the two margin mechanics side by side, since it is easy to conflate them despite their different timing and trigger conditions.

	Short-sale margin (Section 3.5.2)	Futures variation margin
Counterparty	Broker	Central clearinghouse
Settlement frequency	Only when a threshold is breached	Every trading day
Trigger	Equity falls below maintenance %	Any price move, however small
Typical use case	Individual equity short position	Standardized futures and swaps

Table 3.2: Two margin mechanisms compared: bilateral broker margin on a short sale versus centrally cleared futures margin

3.5.3 A Simple No-Arbitrage Example

A useful way to understand why prices in finance are constrained by structure is to consider a very simple arbitrage argument. Suppose an investor can buy a one-period risk-free bond that costs 100 and pays 105

Wulin.Suo@Queensu.ca

one year later. The risk-free interest rate implied by this bond is

$$r = \frac{105 - 100}{100} = 0.05.$$

Now suppose there is also a claim that pays one unit of money with certainty in one year. If the claim were priced at 0.95 today, then an investor could buy the claim for 0.95 and receive 1 next year. This is equivalent to earning a return of about 5.26%, which is higher than the risk-free rate available from the bond. In that case, rational investors would buy the claim and sell the bond, pushing prices back into line. The no-arbitrage principle says that two assets with the same payoff must have the same price, otherwise investors can create a riskless profit. This is a foundational idea behind pricing in derivatives, fixed income, and many broader valuation applications.

This example may appear simple, but it illustrates a powerful principle. Financial prices are not arbitrary; they are linked by the logic of replication and absence of arbitrage. A quantitative analyst should therefore be careful not to interpret a model output in isolation. If a model predicts a price that violates basic consistency conditions, it may be signaling an error in the model, a missing constraint, or an overlooked transaction cost. This is one reason why financial modeling is both statistical and economic. The same logic extends to more complex instruments such as options, swaps, and structured products. In these cases, the challenge is often to identify the relevant replicating portfolio and the correct discounting process. The existence of such relationships is what makes finance a systematic discipline rather than a purely descriptive one. Even when markets appear noisy, there are still strong forces that keep valuations tied to economic logic.

A second example, this time for a forward contract, makes the same point using a replicating portfolio rather than a single comparison bond. Suppose a stock trades today at $S_0 = \$100$, pays no dividends over the next year, and the risk-free rate is $r = 5\%$. A one-year forward contract obligates its holder to buy the stock in one year at a price fixed today; the no-arbitrage forward price is

$$F_0 = S_0(1 + r) = 100(1.05) = \$105,$$

the cost of replicating the forward's payoff by borrowing \$100 today at the risk-free rate, buying the stock now, and holding it for one year (a strategy known as cash-and-carry).

If the forward instead traded in the market at $F_{\text{mkt}} = \$110$, an arbitrageur could borrow \$100, buy the stock, and simultaneously sell (go short) the forward at \$110; in one year, the arbitrageur delivers the stock against the forward for \$110, repays the loan plus interest of $100(1.05) = \$105$, and keeps a riskless profit of $\$110 - \$105 = \$5$ regardless of where the stock price actually ends up, since the forward sale fixes the delivery price in advance. This is precisely the type of relationship Chapter 5's option and derivative pricing methods generalize: an option's value, unlike a forward's, depends on the underlying's volatility because the option's payoff is not a fixed, symmetric obligation, but the same replication logic, construct a portfolio of tradable assets with an identical payoff, and price the derivative to match, underlies both.

A Third Example: Triangular Arbitrage in Currency Markets

No-arbitrage pricing applies just as directly when three related prices, rather than two, must be mutually consistent, and foreign exchange markets provide the cleanest illustration. Three exchange rates, EUR/USD, GBP/USD, and EUR/GBP, are not independent: knowing any two implies the no-arbitrage value of the third, since converting EUR to USD and then USD to GBP must produce the same result as converting EUR to GBP directly. Suppose the market quotes EUR/USD = 1.10 (one euro buys \$1.10) and GBP/USD = 1.25 (one pound buys \$1.25), implying a no-arbitrage EUR/GBP rate of

$$\frac{\text{EUR/USD}}{\text{GBP/USD}} = \frac{1.10}{1.25} = 0.88,$$

one euro should be worth £0.88. If the EUR/GBP market instead directly quotes 0.87, euros are priced slightly cheaper against pounds directly than the USD route implies, and a triangular arbitrage exists: starting with £1,000,000,

$$\begin{aligned}
\text{Step 1 (GBP} \to \text{EUR at 0.87):} &\quad 1{,}000{,}000/0.87 \approx €1{,}149{,}425.29,\\
\text{Step 2 (EUR} \to \text{USD at 1.10):} &\quad 1{,}149{,}425.29 \times 1.10 \approx \$1{,}264{,}367.82,\\
\text{Step 3 (USD} \to \text{GBP at 1/1.25):} &\quad 1{,}264{,}367.82 \times 0.80 \approx £1{,}011{,}494.25,
\end{aligned}$$

Wulin.Suo@Queensu.ca

a riskless profit of about £11,494, or roughly 1.15%, on the round trip, realized in seconds and requiring no view whatsoever on whether any of the three currencies will rise or fall. Exactly as in the bond and forward examples above, this mispricing cannot persist: high-frequency and algorithmic traders (the subject of Chapters 13 and 14) monitor this kind of three-way consistency continuously, and the resulting arbitrage trading pushes the three rates back into alignment within moments of any deviation appearing, which is precisely why triangular arbitrage opportunities in liquid, major-currency pairs are typically too small and short-lived to exploit manually, existing today mainly as a textbook illustration of a real economic force rather than a realistically accessible trading strategy for anyone without a fully automated, low-latency system.

Although financial theory often uses idealized assumptions, real markets are full of frictions. These frictions include transaction costs, taxes, delays, information asymmetry, margin requirements, and limits on borrowing. A frictionless market would make it easy to buy and sell any asset at a known price, but real markets are not like that. The presence of frictions changes the economics of investment and the behavior of prices.

For example, if trading a security is expensive, an investor may need a larger expected return to justify holding it. If borrowing is restricted, a trader cannot always implement a strategy that would otherwise be profitable. If information is not shared equally, some participants may act faster or more intelligently than others. These realities matter because even small frictions can have large effects in the aggregate, particularly when many participants are trading at the same time.

This is why practitioners often distinguish between theoretical prices and realized prices. Theoretical models may suggest that two assets with identical cash flows should trade at the same value. In practice, they may not if short-selling is difficult, if settlement is delayed, or if the market is temporarily illiquid. Such deviations create opportunities for some participants, but they also create risk for others. This tension between theory and practice is one of the defining features of finance.

3.6 Fragility and Market Design

The same connections that make markets efficient also transmit failure. This part covers how stress propagates, how design and speed shape that propagation, and one afternoon in 2010 when it happened in minutes.

3.6.1 Financial Crises, Contagion, and Resilience

A complete understanding of financial markets also requires an understanding of crises. Crises are not random accidents. They often emerge when leverage is high, liquidity is fragile, and institutions are highly interconnected. In such conditions, a seemingly small shock can trigger forced selling, margin calls, and sudden changes in funding conditions. The result can be a cascade of losses that spreads across firms, asset classes, and countries. This general pattern, high leverage, fragile liquidity, and deep interconnection combining to turn a seemingly small shock into a cascade, is exactly what played out in the 2008 financial crisis, already discussed earlier in this chapter's treatment of clearing and systemic risk: mortgage defaults, amplified through securitization, derivatives, and short-term funding markets, became a systemic event precisely because institutions that seemed healthy on the surface turned out not to be resilient once funding dried up.

For AI and quantitative finance, this lesson is important. A model can be accurate under normal conditions and still fail in crisis periods if the underlying regime changes. The data may be nonstationary, the relationship between variables may weaken, and trading strategies may become far more costly to execute. This is why model stress testing and scenario analysis are essential in practice.

An earlier episode, the near-collapse of the hedge fund Long-Term Capital Management (LTCM) in 1998, illustrates a related but distinct lesson: sophisticated quantitative models can fail specifically because of leverage and crowding, not because the underlying statistical reasoning was wrong. LTCM's strategies relied on historical relationships between related securities converging back to normal levels, a reasonable premise most of the time, applied with very high leverage to turn small, reliable-looking spreads into large returns. When the 1997 Asian financial crisis and 1998 Russian debt default triggered a broad flight to safety,

Wulin.Suo@Queensu.ca

the specific historical relationships LTCM depended on moved sharply against it, and because many other market participants held similar positions, there was no way to exit without pushing prices even further in the wrong direction. The fund's near-failure was severe enough that the Federal Reserve organized a private-sector bailout to prevent broader market disruption. The lesson for quantitative finance is not that statistical models are unreliable, but that a model's estimated relationships can break down precisely when many market participants are relying on the same relationship and using leverage to size their bets on it, a crowding risk that a model built purely from historical price data will rarely reveal on its own.

3.6.2 Technology, Speed, and Market Design

The design of financial markets has changed dramatically with technology. Electronic trading, co-location, and automation have reduced costs and increased the speed at which orders can be processed. This has improved access for many participants, but it has also increased complexity. The market now resembles a digital system in which algorithms interact with each other at high speed and where small delays can be extremely valuable.

Technology therefore changes not only the efficiency of trading but also the structure of competition. A participant with faster infrastructure may gain an edge even if its economic view is not superior. This explains why market microstructure has become a central topic in modern finance. Price formation is no longer shaped only by fundamentals; it is also shaped by trading rules, network effects, and technological capabilities.

3.6.3 Case Study: The 2010 Flash Crash

On May 6, 2010, U.S. equity markets experienced one of the most dramatic intraday moves in modern history: major indices fell sharply within minutes, with some individual stocks and exchange-traded funds briefly trading at absurd prices, pennies in some cases, before the market recovered most of its losses within the same session. The episode, subsequently known as the Flash Crash, is a useful case study precisely because the subsequent joint SEC-CFTC investigation identified a chain of mechanical, structural causes rather than any single piece of adverse economic news.

The investigation traced the initial trigger to a large sell order in E-mini S&P 500 futures, executed by an automated algorithm designed to sell a fixed percentage of trading volume without regard to price or time. As this selling pressure pushed prices down, high-frequency trading firms, which had been providing a substantial share of market liquidity, began rapidly buying and reselling the same contracts to each other rather than absorbing the imbalance, a pattern sometimes described as "hot potato" trading.

Within minutes, many of these same high-frequency firms and other liquidity providers withdrew from the market entirely as their own risk controls triggered, causing depth (recall the liquidity dimensions introduced in Section 3.5.1) to evaporate almost completely on both the futures and underlying equity markets at once. With so little resting liquidity left in the order book, the arrival of ordinary sell orders was enough to walk prices down dramatically, the mechanical price-impact effect illustrated numerically in Section 3.3.2, just at a much larger and faster scale.

The Flash Crash illustrates several themes from this chapter operating simultaneously. It shows that liquidity is not a fixed, guaranteed feature of a market but a behavior that participants can withdraw suddenly and simultaneously, especially under stress, the regime-dependence of liquidity discussed in this chapter's challenges section below. It shows that automated systems interacting with other automated systems can produce dynamics, like the hot-potato pattern, that were not intended or anticipated by the designers of any individual system. And it shows the practical consequence of market fragmentation across many linked trading venues and instruments: the crash originated in a futures contract but transmitted almost instantly to the equity market, since prices in closely related instruments are tied together by the same no-arbitrage forces introduced in Section 3.5.3. In direct response, regulators introduced market-wide circuit breakers and individual stock "limit up-limit down" rules designed to pause trading automatically during extreme price moves, a structural safeguard that did not exist in May 2010.

Wulin.Suo@Queensu.ca

3.7 Challenges in Modeling Market Structure

The institutional detail covered in this chapter is not merely background; it creates specific, recurring challenges for anyone trying to build data-driven or AI-based systems on top of financial markets.

Fragmented and opaque data. A meaningful share of trading, especially in bonds, derivatives, and large institutional equity orders, occurs over-the-counter or through dark pools rather than on a single transparent exchange. Any dataset built from public exchange feeds alone is therefore an incomplete picture of the market, and models trained on it can miss where and how large trades actually occur.

Microstructure noise. At very short time horizons, prices are affected by the mechanics of trading itself, bid-ask bounce, order-book dynamics, and the strategic behavior of market makers, not just by fundamental information. A high-frequency model that treats every price tick as a fundamental signal risks fitting this microstructure noise rather than anything economically meaningful, a problem revisited in the chapters on high-frequency trading.

Regime dependence of liquidity. Liquidity, spreads, and market depth are not fixed properties of a security; they change with volatility, macroeconomic conditions, and market stress, often at the moments when a trading or risk model most needs to work reliably. A backtest run only over calm periods can therefore substantially overstate a strategy's real-world performance.

Reflexivity and adversarial dynamics. Unlike a physical system, a market contains other participants who adapt their behavior in response to observed patterns, including patterns generated by other algorithms. A signal or arbitrage relationship, once discovered and traded upon at scale, can weaken or disappear, which means static, one-time model validation is not sufficient.

Modeling systemic effects. Individual asset-level or firm-level models rarely capture the network effects, interconnections between institutions, correlated leverage, shared counterparties, that drive systemic events such as the 2008 crisis. Building models that account for these system-wide dynamics remains a much harder and less mature area than single-asset prediction.

Realistic transaction cost estimation. As emphasized throughout this chapter, the profitability of a strategy depends heavily on execution costs, market impact, and available liquidity, none of which are fully known in advance and all of which are easy to underestimate in a backtest that assumes trades execute at the observed price with no impact.

Taken together, these challenges mean that a purely statistical view of market data, one that ignores the institutional structure described in this chapter, is unlikely to produce robust financial models. This is the central justification for treating market structure as foundational material before the book turns to prediction and optimization methods in later chapters.

3.8 Key Takeaways

Financial markets exist to allocate capital, transfer risk, and reveal information. Their institutions and instruments shape the opportunities and constraints faced by investors and firms. Debt, equity, and derivatives are the main building blocks of modern finance, and the way these instruments are traded has direct consequences for liquidity, pricing, and risk management. A careful understanding of market structure is therefore necessary before one can evaluate more advanced models or trading strategies.

The order-book example in Section 3.3.2 made this concrete: even with no news and no change in fundamental value, a large order can move the average execution price simply because liquidity is finite at each price level, and the Flash Crash case study showed the same mechanism operating at a market-wide scale when liquidity providers withdrew simultaneously. The swaps and regulatory material introduced in this chapter, interest rate swaps, Dodd-Frank, MiFID II, and Basel III, will resurface directly in the risk management chapter that follows, since modern risk management is defined as much by regulatory capital requirements as by statistical modeling choices. The overall lesson to carry forward is that market structure is not a fixed backdrop against which trading happens; it actively shapes prices, costs, and the viability of any strategy built on top of it.

Wulin.Suo@Queensu.ca

3.9 Suggested Exercises

1. Explain the difference between a bond and a stock in terms of cash flow rights and risk.
2. Describe the role of a market maker in a financial market.
3. Give one example of how a derivative can be used for hedging.
4. Explain why liquidity matters for asset prices.
5. Compare exchange-traded and over-the-counter markets in terms of transparency and counterparty risk.
6. Discuss one reason why a trading strategy might fail even if it has good historical predictive performance.
7. Explain how a bid-ask spread reflects the cost of immediacy in a market.
8. Choose one challenge from Section 3.7 ("Challenges in Modeling Market Structure") and explain how it could cause a backtested trading strategy to look profitable historically but lose money in live trading.
9. Using the order book in Table 3.1, compute the average execution price and price impact (in dollars and basis points) for a market sell order of 300 shares, which will walk through the bid side of the book.
10. Explain, in your own words, the difference between the order-processing, inventory-holding, and adverse-selection components of the bid-ask spread, and give one market condition that would cause each component to widen.
11. Explain why an interest rate swap does not require the exchange of the notional principal amount, and describe a situation in which a firm would want to swap from floating-rate to fixed-rate interest payments.
12. Compare the 2008 financial crisis and the LTCM episode described in this chapter: what role did leverage play in each, and how did the two episodes differ in their trigger and their transmission mechanism?
13. Using Section 3.5.2's method, a trader shorts 200 shares at \$80 with a 150% initial margin requirement and a 30% maintenance margin. Compute the total initial account equity and the share price at which a margin call is triggered.
14. Using Section 3.5.3's cash-and-carry logic, a non-dividend-paying stock trades at \$40 and the risk-free rate is 4%. Compute the no-arbitrage one-year forward price, and describe the specific trades an arbitrageur would execute today and in one year if the forward instead traded at \$43.
15. A dealer borrows \$25 million overnight in the repo market at an annualized rate of 4.5%, using the 360-day money-market convention from this chapter's repo example. Compute the dollar interest owed for one night and the total repurchase price.
16. Explain how the Flash Crash illustrates the "regime dependence of liquidity" challenge from Section 3.7, using specific details from the case study.
17. Using this chapter's interest rate swap example, recompute the swap's value to the fixed-rate payer if the one-year and two-year discount rates were instead 5.5% and 6% (rates have risen above the 5% fixed rate rather than fallen below it), and explain in words why the sign of the swap's value to the fixed-rate payer flips relative to the original example.
18. Using this chapter's triangular arbitrage example, suppose the EUR/GBP rate is instead quoted at 0.89 rather than 0.87 (EUR/USD and GBP/USD unchanged). Determine whether the arbitrage cycle described in the text ($£ \to € \to \$ \to £$) is still profitable, and if not, describe which direction around the triangle would now be profitable instead.

Wulin.Suo@Queensu.ca

3.10 Further Reading

For readers who want to study these topics in more depth, the following books are especially useful:

- Bodie, Kane, and Marcus, *Investments*. A broad introduction to markets, instruments, and institutions that complements this chapter's overview with a fuller treatment of portfolio and asset-pricing theory.
- Hull, *Options, Futures, and Other Derivatives*. The standard reference for the derivatives instruments introduced briefly in this chapter, developing their pricing and hedging in full in Chapter 5.
- Fabozzi, *Fixed Income Analysis*. Goes well beyond this chapter's brief bond-market coverage into the pricing, risk, and institutional detail of fixed-income securities.
- Shleifer, *Inefficient Markets*. A behavioral-finance counterpoint to the efficient-market and price-discovery discussion in this chapter, examining the frictions and limits to arbitrage that keep prices from always reflecting fundamentals.
- O'Hara, *Market Microstructure Theory*. The foundational academic treatment of the order-book mechanics, bid-ask spread decomposition, and market-making economics introduced in this chapter's worked examples.

Wulin.Suo@Queensu.ca

Chapter 4

Financial Statements and Business Basics

4.1 Introduction

Financial statements are the language through which firms communicate their economic position to investors, lenders, regulators, and managers. They are not merely accounting artifacts. They are structured summaries of how a business creates value, uses capital, and manages risk. For anyone working in finance, whether as an investor, analyst, trader, or data scientist, the ability to read and interpret financial statements is indispensable. A model may forecast returns or defaults, but it is useless without an understanding of the business context behind the numbers.

This chapter introduces the three core financial statements: the balance sheet, the income statement, and the cash flow statement. It explains how they fit together, why they differ, and how they can be used to reason about profitability, liquidity, solvency, and value creation. The chapter also works through a single running numerical example, a small fictional manufacturing firm, so that every ratio and concept can be tied to actual numbers rather than abstract definitions. The goal is not to turn the reader into an accountant, but to build the practical intuition needed to analyze firms in a way that is both financially sound and quantitatively useful.

Meridian Fabrication Inc., a fictional manufacturing firm, runs through the entire chapter as its central worked example: its balance sheet, income statement, and cash flow statement are built up section by section, then mined for liquidity, leverage, profitability, and efficiency ratios (Section 4.4.5), an Altman's Z-Score distress assessment, and a common-size restatement that makes its cost structure comparable across firm sizes. A second company, Solstice Robotics Inc. (Section 4.5.1), is introduced specifically for comparison, so that a ratio's meaning becomes clear by contrast rather than by definition alone: the same debt-to-equity ratio can signal prudence at one firm and fragility at another, depending on the business built around it.

This progression matters beyond the two companies themselves. The ratio toolkit built here is the same toolkit Chapter 9's credit scoring and Chapter 6's classification models eventually formalize statistically; Section 4.6.1 makes the link to Chapter 5's valuation models explicit, since a discounted-cash-flow model is only as trustworthy as the financial-statement forecasts feeding it; and the accounting-quality cautions raised late in the chapter, that reported numbers reflect choices and estimates, not just facts, are worth carrying into every later chapter that treats a financial statement's numbers as clean model inputs rather than as the product of a real, sometimes discretionary, reporting process.

4.2 Why Statements Matter, and a Running Example

Accounting is the language a firm uses to describe itself, and like any language it can be used carefully or evasively. This part explains why the statements repay close reading, and introduces the company whose numbers the rest of the chapter works through.

Wulin.Suo@Queensu.ca

4.2.1 Why Financial Statements Matter

A firm is a system for converting resources into cash flows. It buys inputs, hires labor, invests in equipment, and sells products or services. In the process, it makes decisions about financing, operations, and strategy. Financial statements are the records that summarize these decisions. They allow stakeholders to ask essential questions: How much debt does the firm carry? Is it profitable? Is it generating enough cash to fund operations? What portion of the business is financed by owners versus lenders? Is growth coming from genuine operating improvement or from financial engineering?

These questions matter because finance is not only about price changes. It is also about the underlying economic reality of the firm. A stock can appear attractive because its price rises, but if the company is consuming cash and taking on excessive leverage, the apparent success may be fragile. Conversely, a firm with modest headline earnings may still be valuable if it produces durable cash flows and has strong competitive advantages. Financial statements help separate signal from noise.

In practice, the statements are analyzed together rather than in isolation. The balance sheet shows the firm's position at a point in time. The income statement shows performance over a period. The cash flow statement shows the movement of cash. Together, they provide a more complete picture than any single report could provide. Figure 4.1 sketches how the three statements relate to one another.

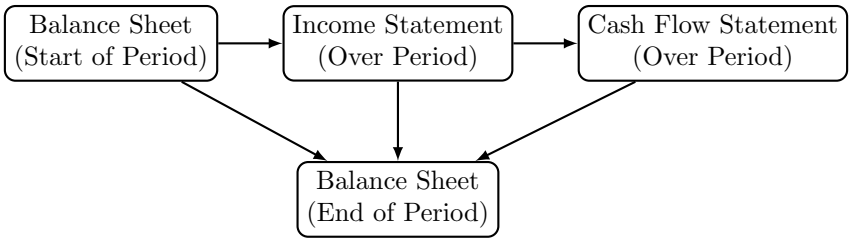


Figure 4.1: The three statements are linked: the income statement and cash flow statement explain the change in the balance sheet from the start to the end of the period

4.2.2 A Running Example: Meridian Fabrication Inc.

To keep the discussion concrete, this chapter follows a single fictional company, Meridian Fabrication Inc., a small industrial parts manufacturer. All figures are in millions of dollars. Table 4.1 shows a simplified balance sheet at the end of two consecutive fiscal years, Table 4.2 shows the income statement for the most recent year, and Table 4.3 shows the cash flow statement for the same year. These numbers are used throughout the chapter to illustrate ratios, forecasting, and the limits of reported figures.

Notice, as a first observation, that Meridian's net income of $9.0 million looks respectable, but cash flow from operations is only $6.0 million, and after $14.0 million of capital expenditures, the firm actually burned cash and had to raise $12.0 million of new debt just to keep its cash balance from falling further. This is the type of discrepancy between accounting profit and cash generation that financial statement analysis is designed to surface, and it will recur throughout the chapter.

4.3 The Three Statements

A firm reports itself through three linked documents: what it owns and owes, what it earned, and what cash actually moved. This part takes each in turn and shows how they articulate with one another.

4.3.1 The Balance Sheet

The balance sheet is a snapshot of a firm's resources and claims at a specific date. It is governed by a simple accounting identity:

Wulin.Suo@Queensu.ca

	Year 1	Year 2
Cash and equivalents	12.0	9.0
Accounts receivable	18.0	24.0
Inventory	22.0	30.0
Total current assets	**52.0**	**63.0**
Property, plant, and equipment (net)	60.0	66.0
Total assets	**112.0**	**129.0**
Accounts payable	14.0	17.0
Short-term debt	6.0	8.0
Total current liabilities	**20.0**	**25.0**
Long-term debt	40.0	50.0
Total liabilities	**60.0**	**75.0**
Shareholders' equity	52.0	54.0
Total liabilities and equity	**112.0**	**129.0**

Table 4.1: Meridian Fabrication Inc. – Balance Sheet ($ millions)

Revenue	150.0
Cost of goods sold	96.0
Gross profit	**54.0**
Selling, general, and administrative expense	30.0
Depreciation and amortization	8.0
Operating income (EBIT)	**16.0**
Interest expense	4.0
Pretax income	**12.0**
Taxes (25%)	3.0
Net income	**9.0**

Table 4.2: Meridian Fabrication Inc. – Income Statement, Year 2 ($ millions)

Net income	9.0
+ Depreciation and amortization	8.0
− Increase in accounts receivable	6.0
− Increase in inventory	8.0
+ Increase in accounts payable	3.0
Cash flow from operations	**6.0**
Capital expenditures	14.0
Cash flow from investing	**−14.0**
Net new debt issued	12.0
Dividends paid	7.0
Cash flow from financing	**5.0**
Net change in cash	**−3.0**

Table 4.3: Meridian Fabrication Inc. – Cash Flow Statement, Year 2 ($ millions)

Wulin.Suo@Queensu.ca

$$\text{Assets} = \text{Liabilities} + \text{Equity}.$$

Assets are what the firm owns or controls. They include cash, receivables, inventories, property, plant and equipment, intangible assets, and other resources expected to generate future benefit. Liabilities are obligations the firm owes to others, such as loans, accounts payable, or taxes payable. Equity is the residual interest of owners after liabilities are subtracted from assets. In Table 4.1, this identity holds in both years: \$112.0 million of assets equal \$60.0 million of liabilities plus \$52.0 million of equity in Year 1, and \$129.0 million equal \$75.0 million plus \$54.0 million in Year 2.

The balance sheet matters because it reveals the firm's financial structure. A firm with large cash reserves and modest debt is often more resilient than a firm with high leverage and weak liquidity. A firm with significant intangible assets may appear valuable in the market, but those assets may be harder to liquidate if the company encounters distress. A firm with high inventory levels may face operational risk if demand falls unexpectedly. Meridian's inventory grew from \$22.0 million to \$30.0 million, a 36% increase, and its receivables from \$18.0 million to \$24.0 million, a 33% increase, while its revenue grew only 11%, from \$135.0 million in Year 1 to the \$150.0 million of Year 2. Comparing the growth rate of inventory and receivables to the growth rate of sales is one of the simplest early-warning checks an analyst performs, and here it flags exactly the working-capital buildup that the cash flow statement shows consuming Meridian's cash.

The balance sheet can be analyzed along several dimensions. One is liquidity, which concerns the firm's ability to meet short-term obligations. Another is capital structure, which concerns the mix of debt and equity financing. A third is asset quality, which concerns whether the reported assets are likely to generate value. Financial analysts commonly compute ratios such as current assets divided by current liabilities, debt divided by equity, and long-term debt relative to total assets. These ratios provide a first pass at understanding the firm's financial position and are computed explicitly for Meridian in Section 4.4.5.

4.3.2 The Income Statement

The income statement summarizes revenues, expenses, and profit over a period such as a quarter or a year. It begins with revenue, which is the amount earned from selling goods or services. From revenue, the firm subtracts costs of goods sold and operating expenses to arrive at operating profit, often called earnings before interest and taxes (EBIT). Further deductions for interest and taxes lead to net income, which is often the headline figure quoted in the media.

For Meridian, revenue of \$150.0 million minus cost of goods sold of \$96.0 million yields gross profit of \$54.0 million, a gross margin of

$$\text{Gross margin} = \frac{54.0}{150.0} = 36.0\%.$$

After subtracting SG&A (\$30.0 million) and depreciation (\$8.0 million), operating income is \$16.0 million, an operating margin of

$$\text{Operating margin} = \frac{16.0}{150.0} = 10.7\%.$$

After \$4.0 million of interest expense and \$3.0 million of taxes (a 25% rate applied to \$12.0 million of pretax income), net income is \$9.0 million, a net margin of $9.0/150.0 = 6.0\%$.

Although net income is widely reported, it should not be interpreted mechanically. Accounting profit can be influenced by depreciation, amortization, one-time gains or losses, and estimates of reserves and provisions. These choices can affect the apparent profitability of a firm. For this reason, analysts often look beyond net income to operating income, adjusted earnings, and cash-based measures of performance.

Profitability is central to valuation. Investors care about whether the firm can earn returns above its cost of capital. A company may report growth in sales, but if margins are declining and competition is intensifying, future value may be under pressure. Profitability analysis therefore examines gross margin, operating margin, and net margin together with their trend over time, not just their level in a single year.

One further profitability measure is worth introducing alongside these three margins because of how frequently it appears in valuation practice: EBITDA, earnings before interest, taxes, depreciation, and amortization, computed by adding depreciation and amortization back to operating income. For Meridian,

Wulin.Suo@Queensu.ca

EBITDA = 16.0 + 8.0 = \$24.0 million, an EBITDA margin of 24.0/150.0 = 16.0%. EBITDA is popular specifically because it strips out depreciation and amortization, non-cash charges that can vary substantially across firms depending on their accounting choices and asset age, making it easier to compare operating performance across companies with different capital structures and depreciation schedules. It is not a substitute for free cash flow, since it ignores both capital expenditures and working-capital changes, both of which Meridian's own numbers show are large enough to turn a profitable-looking income statement into a cash-consuming year; but as a cross-sectional comparison tool, and as the denominator of the EV/EBITDA valuation multiple used throughout Chapter 5, it is worth computing routinely alongside gross, operating, and net margin.

4.3.3 The Cash Flow Statement

The cash flow statement is often the most informative statement for understanding financial health. Cash is the ultimate resource that keeps a business alive. A company can report positive profits and still face liquidity problems if cash is tied up in inventory, receivables, or capital spending. The cash flow statement shows how cash is generated and used in three major activities: operating, investing, and financing.

Operating cash flow reflects cash generated from the core business. It starts from net income and adjusts for non-cash items (such as adding back depreciation) and for changes in working-capital accounts. Investing cash flow reflects purchases and sales of long-term assets, primarily capital expenditures. Financing cash flow reflects borrowing, repayments, dividends, and equity issuance. Table 4.3 shows this decomposition for Meridian: operating cash flow of \$6.0 million, investing cash flow of −\$14.0 million, and financing cash flow of \$5.0 million, which together produce a net decrease in cash of \$3.0 million, consistent with the cash balance falling from \$12.0 million to \$9.0 million on the balance sheet.

A particularly important concept is free cash flow, typically defined as

$$\text{Free cash flow} = \text{Cash flow from operations} - \text{Capital expenditures}.$$

For Meridian, free cash flow in Year 2 is $6.0 - 14.0 = -\$8.0$ million. This single number tells a sharper story than net income alone: despite reporting a \$9.0 million profit, Meridian generated negative \$8.0 million of free cash flow and had to borrow to cover the shortfall. Free cash flow is closely related to valuation because firms that generate consistent free cash flow can support dividends, debt repayment, or reinvestment without external financing. Many valuation frameworks explicitly use free cash flows rather than accounting earnings because the latter can be distorted by non-cash charges and discretionary accounting choices.

4.4 Ratio Analysis

Raw statement figures are not comparable across firms or across years; ratios are what make them so. This part builds the standard families of ratio, from short-term liquidity through leverage to the returns that reveal whether a firm creates value at all.

4.4.1 Working Capital and Liquidity

Liquidity refers to the ability of a firm to meet short-term obligations. It is one of the clearest indicators of short-run financial health. A business that cannot pay suppliers, employees, or creditors on time may face operational disruption even if the long-term outlook is strong. Analysts therefore pay close attention to current assets, current liabilities, and the cycle of converting inventory and receivables into cash.

Working capital is defined as

$$\text{Working capital} = \text{Current assets} - \text{Current liabilities}.$$

For Meridian, working capital rose from $52.0 - 20.0 = \$32.0$ million in Year 1 to $63.0 - 25.0 = \$38.0$ million in Year 2. On its own this looks like a healthy increase in resources, but combined with the cash flow statement it is clear that most of this increase is simply cash trapped in receivables and inventory rather than cash sitting in the bank. Too little working capital can cause distress, while too much may indicate inefficiency or, as in Meridian's case, a buildup that has not yet been converted back into cash.

Wulin.Suo@Queensu.ca

In practice, working capital is actively managed, not just measured after the fact. A treasury or finance team can accelerate collections by offering early-payment discounts to customers, negotiate longer payment terms with suppliers, or use inventory management techniques such as just-in-time ordering to reduce the amount of cash tied up in stock at any given time. Each of these levers maps directly onto one of the three components of the cash conversion cycle computed below: shortening DSO (days sales outstanding) through faster collections, lengthening DPO (days payable outstanding) through supplier negotiation, or shortening DIO (days inventory outstanding) through leaner inventory management. Working capital financing, such as a revolving credit facility sized specifically to bridge the gap between paying suppliers and collecting from customers, is also common precisely because this gap, as Meridian's cash flow statement demonstrates, can consume substantial cash even for a fundamentally healthy and growing business.

The cash conversion cycle is a useful way to quantify this. It is commonly expressed as

$$\text{Cash conversion cycle} = \text{DIO} + \text{DSO} - \text{DPO},$$

where DIO measures how long inventory sits before being sold, DSO measures how long it takes to collect receivables after a sale, and DPO measures how long the firm takes to pay its own suppliers. COGS is cost of goods sold, the direct cost, materials, direct labor, and manufacturing overhead, of producing whatever a firm actually sold during the period, already introduced earlier in this chapter as the amount subtracted from revenue to arrive at gross profit.

DIO and DPO divide by COGS rather than revenue for a specific reason: inventory and accounts payable are both carried on the balance sheet at their cost to the firm, not at the higher price a customer eventually pays, so matching them against COGS, itself a cost-basis figure, keeps the ratio's numerator and denominator on the same footing. DSO, by contrast, divides accounts receivable, which is recorded at the amount actually billed to the customer, by revenue, the corresponding billed-price figure; using COGS for DSO or revenue for DIO and DPO would quietly mix cost-basis and price-basis quantities in the same ratio. Using Year 2 year-end balances and a 365-day year (analysts often average the opening and closing balances instead, which for a firm whose working capital is growing as fast as Meridian's gives a noticeably shorter cycle),

$$\begin{aligned}
\text{DIO} &= \frac{\text{Inventory}}{\text{COGS}} \times 365 = \frac{30.0}{96.0} \times 365 \approx 114 \text{ days},\\
\text{DSO} &= \frac{\text{Accounts receivable}}{\text{Revenue}} \times 365 = \frac{24.0}{150.0} \times 365 \approx 58 \text{ days},\\
\text{DPO} &= \frac{\text{Accounts payable}}{\text{COGS}} \times 365 = \frac{17.0}{96.0} \times 365 \approx 65 \text{ days}.
\end{aligned}$$

Rounding each component to the nearest day before summing gives $114 + 58 - 65 = 107$ days, but computing the cycle from the unrounded day-counts gives $114.06 + 58.40 - 64.64 \approx 107.8$, which rounds to 108 days; the one-day gap between the two is purely a rounding artifact from summing already-rounded intermediate figures, a small but instructive reminder to carry full precision through a multi-step calculation and round only the final answer. Either way, Meridian's cash is tied up in the operating cycle for roughly three and a half months before it returns to the firm. A shorter cycle is generally better because it means the firm can recycle cash more quickly and rely less on external financing. The cycle can be improved through better inventory management, faster collections, or more favorable supplier terms, and it is the kind of quantity that a data-driven monitoring system can track continuously across many firms or product lines.

4.4.2 Leverage and Solvency

Leverage refers to the extent to which a firm uses debt financing. Debt can be attractive because it is often cheaper than equity and interest expenses may be tax-deductible. However, leverage also increases financial risk. If the firm's cash flows are volatile, debt can become burdensome and may force restructuring or bankruptcy.

Solvency is the ability of a firm to meet its long-term obligations. Analysts examine leverage ratios such as debt-to-equity,

$$\text{Debt-to-equity} = \frac{\text{Total debt}}{\text{Shareholders' equity}},$$

Wulin.Suo@Queensu.ca

and interest coverage,

$$\text{Interest coverage} = \frac{\text{EBIT}}{\text{Interest expense}}.$$

For Meridian in Year 2, total debt is short-term debt plus long-term debt, $8.0 + 50.0 = \$58.0$ million, against equity of \$54.0 million, giving a debt-to-equity ratio of $58.0/54.0 \approx 1.07$: the firm is financed with slightly more debt than equity. Interest coverage is $16.0/4.0 = 4.0$, meaning operating income covers interest expense four times over. A coverage ratio of 4.0 is generally considered comfortable, but the trend matters: if debt continues to grow faster than operating income, coverage will fall and the firm's cushion against a bad year will shrink.

Leverage is not inherently good or bad. It depends on the firm's business model, cash-flow stability, and access to financing. A utility can often carry significant debt because its revenues are stable. A startup in a volatile industry may be unable to support much debt. The key is understanding whether leverage amplifies the firm's strengths or magnifies its weaknesses, and for Meridian, the fact that new debt was needed to fund a cash shortfall driven by working-capital growth is a signal worth investigating rather than ignoring.

4.4.3 Sector-Specific Considerations in Ratio Analysis

The chapter has repeatedly warned that ratios must be interpreted in context, and it is worth being specific about what that context looks like across a few contrasting industries, since a large-scale, cross-sectional financial model cannot apply a single set of thresholds to every firm it scores.

Banks and other financial institutions are the most extreme example. A bank's balance sheet consists mostly of financial assets (loans) funded mostly by financial liabilities (deposits), so a debt-to-equity ratio that would signal extreme distress for an industrial firm like Meridian, often 8 or 10 times equity, is routine and expected for a bank, whose entire business model is to hold a leveraged portfolio of loans. Standard liquidity ratios such as the current ratio are similarly not meaningful for banks, since a bank's balance sheet is not organized around a current/non-current distinction in the same way; regulators instead impose bank-specific capital and liquidity ratios, some of which were introduced in the Basel III discussion of Chapter 3.

Capital-intensive industries such as utilities, airlines, and telecommunications typically carry substantial long-term debt and correspondingly low interest coverage relative to an asset-light firm such as Solstice Robotics, the comparison company introduced in Section 4.5.1 below, precisely because their revenues are relatively stable and predictable (regulated utility rates, contracted infrastructure usage), which allows them to service more debt safely than a firm with volatile, uncertain cash flows. A debt-to-equity ratio of 1.5 might be unremarkable for a regulated utility but alarming for an unproven technology startup with the same ratio.

Asset-light service and technology businesses, more like Solstice than Meridian, often carry little debt, hold significant cash balances, and derive most of their value from intangible assets, customer relationships, brand, intellectual property, that may not appear as a large balance sheet line item at all under conventional accounting. This is one reason market-based valuation multiples (Chapter 5) are often used alongside, or instead of, book-value-based ratios for such firms: the balance sheet may substantially understate the economic assets actually driving the business.

The practical implication for any model built across many firms is that ratios should typically be compared within a sector or peer group rather than against a single universal threshold, and that sector membership itself, along with variables like capital intensity, is often a useful feature to include directly in a predictive model rather than something to control for only informally.

This has a direct, quantitative consequence for a distress score like Altman's Z': a formula whose coefficients were estimated primarily on manufacturing and industrial firms will systematically misclassify a bank, whose debt-to-equity ratio would sit far above, and whose X_4 (book equity divided by total liabilities) would therefore sit far below, any threshold intended for an industrial company, even for a perfectly healthy bank. Neither Meridian's Z'-score of 2.41 nor Solstice's 4.33, both computed later in this chapter, would be directly comparable to a Z'-score computed for a bank or a capital-intensive utility using the same coefficients, which is why credit-scoring models built for a specific sector, such as the bank-focused credit risk material in Chapter 9, are estimated on sector-specific data rather than reusing a general-purpose formula like Altman's across every industry.

Wulin.Suo@Queensu.ca

4.4.4 How Firms Create Value

Suppose a firm doubles its revenue and its balance sheet over five years, yet its shareholders end up no wealthier than if the firm had stayed the same size. Growth alone, it turns out, says nothing about whether a firm is actually creating value, because growth has to be paid for with capital, and that capital has a cost, whether it comes from lenders demanding interest or shareholders demanding a competitive return on their investment. A firm creates value only when it earns returns on invested capital that exceed the cost of that capital; if a company invests in projects that earn more than the required return, value is created, and if it invests in projects that earn less than the required return, value is destroyed no matter how much bigger the firm gets in the process. This is one of the most important ideas in corporate finance, and a common way to express it is through economic profit,

$$\text{Economic profit} = \text{Invested capital} \times (\text{ROIC} - \text{WACC}),$$

where ROIC is the return on invested capital and WACC is the weighted average cost of capital. A simple value-creation diagram is shown in Figure 4.2.



Figure 4.2: A firm generates value when its projects earn a return above the cost of capital

This idea links financial statements to valuation. A firm that generates high returns on capital and reinvests profitably can compound value over time. A firm that earns low returns and uses excessive leverage may appear large but create little wealth. Data scientists working in finance often use accounting and market data together to evaluate such patterns. A predictive model may forecast changes in revenue, but an analyst must still ask whether those changes translate into durable value creation.

The concept also explains why growth is not always beneficial. Growth can be valuable when it is tied to profitable reinvestment. Growth can be harmful when it requires large amounts of capital with weak expected returns. Meridian grew its balance sheet from \$112.0 million to \$129.0 million, a 15% increase, while free cash flow turned negative; whether this growth is value-creating depends on whether the additional invested capital will eventually generate returns above its cost, which is a question the statements alone cannot answer but strongly motivate asking.

The statements alone cannot supply a cost of capital, but they can supply everything else this calculation needs. Meridian's invested capital, the sum of its interest-bearing debt and equity, is \$58.0 + \$54.0 = \$112.0 million (Table 4.1), and its after-tax operating profit, or net operating profit after tax (NOPAT), is EBIT $\times$ (1 $-$ tax rate) = 16.0 $\times$ 0.75 = \$12.0 million. This gives

$$\text{ROIC} = \frac{\text{NOPAT}}{\text{Invested capital}} = \frac{12.0}{112.0} \approx 10.7\%.$$

If Meridian's weighted average cost of capital, estimated using the methods of Chapter 5, were 8% (close to the 8.08% that Chapter 5 estimates from Meridian's own statements), its economic profit would be $112.0 \times (0.107 - 0.08) \approx \$3.0$ million: modestly positive, meaning Meridian is creating a small amount of value on top of its cost of capital despite the cash-flow strain documented earlier in this chapter. This is an important, if initially counterintuitive, distinction: a firm can be a genuine (if marginal) value creator by this economic-profit measure while simultaneously burning cash and increasing leverage, because ROIC versus WACC answers whether capital is being deployed productively, while free cash flow answers a different question, whether the firm is generating or consuming cash in the short run. A machine learning model trained to predict "financial health" from statement data needs to be clear about which of these two, economically distinct, questions it is actually answering.

4.4.5 Common Financial Ratios and What They Reveal

Financial ratios are one of the simplest ways to convert raw statements into interpretable measures. They help analysts compare firms, track trends over time, and identify areas of concern. Ratios can be grouped

Wulin.Suo@Queensu.ca

into categories such as profitability, liquidity, efficiency, solvency, and valuation. Table 4.4 collects the main ratio families together with their formulas and Meridian's Year 2 values.

Ratio	Formula	Meridian Value
Current ratio	Current assets / Current liabilities	$63.0/25.0 = 2.52$
Quick ratio	(Current assets − Inventory) / Current liabilities	$33.0/25.0 = 1.32$
Gross margin	Gross profit / Revenue	$54.0/150.0 = 36.0\%$
Operating margin	EBIT / Revenue	$16.0/150.0 = 10.7\%$
Net margin	Net income / Revenue	$9.0/150.0 = 6.0\%$
Debt-to-equity	Total debt / Equity	$58.0/54.0 = 1.07$
Interest coverage	EBIT / Interest expense	$16.0/4.0 = 4.0$
Asset turnover	Revenue / Total assets	$150.0/129.0 = 1.16$
Return on equity (ROE)	Net income / Equity	$9.0/54.0 = 16.7\%$

Table 4.4: Summary of common financial ratios, applied to Meridian, Year 2

The current ratio of 2.52 and quick ratio of 1.32 suggest that, on paper, Meridian has ample short-term resources to cover its current liabilities. Yet this is precisely where the earlier cash flow analysis adds crucial nuance: much of the current assets are tied up in slow-moving inventory and receivables rather than cash, and the firm's actual cash balance fell during the year. A ratio computed from a single balance sheet date can look comfortable while masking a deteriorating cash trajectory, which is why ratios are always more informative when read together with the cash flow statement and their own historical trend.

A particularly useful decomposition is the DuPont identity, which breaks return on equity into three economically distinct drivers:

$$\text{ROE} = \underbrace{\frac{\text{Net income}}{\text{Revenue}}}_{\text{Net margin}} \times \underbrace{\frac{\text{Revenue}}{\text{Total assets}}}_{\text{Asset turnover}} \times \underbrace{\frac{\text{Total assets}}{\text{Equity}}}_{\text{Equity multiplier}}.$$

For Meridian, Net margin = 6.0%, Asset turnover = 1.16, and the equity multiplier is $129.0/54.0 = 2.39$. Multiplying these together at full precision, $0.060 \times 1.1628 \times 2.3889 \approx 16.67\%$, reproduces the directly computed ROE of 16.7% exactly, as it must, since revenue and total assets cancel in the product; multiplying the rounded factors instead would open a small gap that is purely a rounding artifact, the same cascade the cash conversion cycle illustrated above. The value of this decomposition is diagnostic: it shows that Meridian's ROE is being driven substantially by leverage (an equity multiplier above 2) rather than by unusually high margins or asset efficiency. Two firms can report the same ROE for very different, and not equally sustainable, reasons, and the DuPont breakdown is the standard tool for telling them apart.

The three-factor version above bundles taxes, interest, and core operating profitability together inside a single "net margin" term, which hides whether a firm's margin is being driven by its actual operations or merely by a favorable tax or financing outcome. The five-factor extended DuPont identity splits net margin further,

$$\text{ROE} = \underbrace{\frac{\text{Net income}}{\text{Pretax income}}}_{\text{Tax burden}} \times \underbrace{\frac{\text{Pretax income}}{\text{EBIT}}}_{\text{Interest burden}} \times \underbrace{\frac{\text{EBIT}}{\text{Revenue}}}_{\text{Operating margin}} \times \text{Asset turnover} \times \text{Equity multiplier},$$

isolating exactly how much of net margin survives taxation and interest expense before operating profitability is even considered. For Meridian, the tax burden is $9.0/12.0 = 0.75$ (the firm retains 75% of pretax income after taxes, consistent with the 25% tax rate used throughout this chapter), the interest burden is $12.0/16.0 = 0.75$ (the firm retains 75% of EBIT after interest expense), and the operating margin is $16.0/150.0 \approx 10.7\%$, the operating margin already computed in Section 4.4.5. Multiplying all five factors,

$$0.75 \times 0.75 \times 0.107 \times 1.163 \times 2.389 \approx 16.7\%,$$

Wulin.Suo@Queensu.ca

matches the directly computed ROE exactly. The diagnostic value of the extra split is immediate: Meridian's two burden ratios are identical at 0.75, meaning taxes and interest expense are eroding profitability by the same proportion, but a firm with, say, a much lower interest burden (heavily levered, paying substantial interest) and a high tax burden would show an identical three-factor net margin to Meridian while having a completely different underlying cost structure, a distinction only the five-factor decomposition, not the three-factor one, can actually surface.

Liquidity ratios such as the current ratio and quick ratio assess how easily a firm can meet short-term obligations. Efficiency ratios such as inventory turnover and receivables turnover reveal how quickly the firm converts resources into cash. Solvency ratios such as debt-to-equity and interest coverage reveal how vulnerable the firm is to financial distress. Valuation ratios such as price-to-earnings and price-to-book are often used by investors to compare market expectations with fundamental performance, though they require market price data that goes beyond the statements themselves.

The usefulness of ratios depends on context. A high leverage ratio may be acceptable for a stable utility but dangerous for a startup with volatile cash flows. A low inventory turnover ratio may indicate weak sales or poor inventory management, but in some sectors it may simply reflect a strategy of high service levels. Ratios are therefore best used as starting points for a conversation about the business, not as final judgments in themselves.

4.5 Comparison and Common-Size Analysis

A ratio means little in isolation. This part introduces a second company and the common-size representation that makes two firms of very different scale directly comparable.

4.5.1 A Second Company for Comparison: Solstice Robotics Inc.

Ratios are most informative in relative terms, compared across firms or across time, not in isolation. To make this concrete, consider a second fictional company, Solstice Robotics Inc., a faster-growing, less capital-intensive firm, with the Year 2 figures in Table 4.5.

Cash and equivalents	40.0
Accounts receivable	20.0
Inventory	10.0
Total current assets	**70.0**
Property, plant, and equipment (net)	30.0
Total assets	**100.0**
Accounts payable	15.0
Long-term debt	10.0
Total liabilities	**25.0**
Shareholders' equity	75.0
Revenue	120.0
Cost of goods sold	48.0
SG&A	40.0
Depreciation and amortization	5.0
Interest expense	1.0
Net income	19.5

Table 4.5: Solstice Robotics Inc. – Summary Financials, Year 2 ($ millions)

Table 4.6 places Meridian and Solstice side by side on the same ratios computed in Table 4.4.

Solstice's ROE of 26% is higher than Meridian's 16.7%, but the DuPont identity from Section 4.4.5 reveals that the two firms get there in opposite ways. Applying the same decomposition to Solstice: net margin

Wulin.Suo@Queensu.ca

Ratio	Meridian	Solstice
Current ratio	2.52	4.67
Quick ratio	1.32	4.00
Gross margin	36.0%	60.0%
Operating margin	10.7%	22.5%
Net margin	6.0%	16.25%
Debt-to-equity	1.07	0.13
Interest coverage	4.0	27.0
Return on equity	16.7%	26.0%

Table 4.6: Meridian versus Solstice, Year 2

16.25%, asset turnover $120.0/100.0 = 1.20$, and equity multiplier $100.0/75.0 = 1.33$, so $0.1625 \times 1.20 \times 1.33 \approx 26.0\%$, matching directly.

Solstice's high ROE is driven almost entirely by superior margins (16.25% net margin versus Meridian's 6.0%), with very little help from leverage (an equity multiplier of 1.33 versus Meridian's 2.39). Meridian, by contrast, achieves a respectable but lower ROE mostly *because* of leverage. An analyst, or a machine learning model trained naively on ROE alone, would rank the two firms similarly on this one statistic despite their financial structures, and therefore their risk profiles, being almost opposite; this is precisely the kind of distinction the DuPont decomposition exists to surface, and precisely the kind of nuance a single summary ratio can hide.

4.5.2 Common-Size Statements

A second way to make firms comparable, alongside ratios, is the common-size statement, in which every income statement line is expressed as a percentage of revenue and every balance sheet line as a percentage of total assets. Table 4.7 shows Meridian's and Solstice's income statements this way.

	Meridian	Solstice
Cost of goods sold	64.0%	40.0%
Gross profit	36.0%	60.0%
SG&A	20.0%	33.3%
Depreciation and amortization	5.3%	4.2%
Operating income (EBIT)	10.7%	22.5%
Interest expense	2.7%	0.8%
Net income	6.0%	16.25%

Table 4.7: Common-size income statements, Year 2 (percent of revenue)

Common-size statements make cost structure differences immediately visible in a way that raw dollar figures do not: Solstice spends proportionally much less on cost of goods sold (40% of revenue versus Meridian's 64%) but proportionally more on SG&A (33.3% versus 20%), a pattern consistent with a less capital-intensive, more sales-and-marketing-driven business model. This kind of comparison, expressing every line relative to a common base, is also the sort of feature engineering step that makes financial statement data usable as an input to the machine learning models of Chapter 6: raw dollar figures are not comparable across firms of different sizes, but common-size ratios are.

The same technique applies to the balance sheet, expressing every line as a percentage of total assets

Wulin.Suo@Queensu.ca

rather than of revenue. For Meridian's Year 2 balance sheet, this gives

$$\begin{aligned} \text{Cash} &= 6.98\%, & \text{Accounts payable} &= 13.18\%, \\ \text{Receivables} &= 18.60\%, & \text{Short-term debt} &= 6.20\%, \\ \text{Inventory} &= 23.26\%, & \text{Long-term debt} &= 38.76\%, \\ \text{Property, plant, and equipment} &= 51.16\%, & \text{Equity} &= 41.86\%, \end{aligned}$$

of total assets. Presented this way, more than half of Meridian's balance sheet, 51.16%, is tied up in fixed, illiquid property, plant, and equipment, while liabilities and equity are split nearly evenly (58.14% versus 41.86%), a considerably more leveraged structure than the split Solstice's own common-size balance sheet would show given its debt-to-equity ratio of only 0.13 from Table 4.6. Common-size balance sheets are especially useful for spotting this kind of structural difference at a glance, well before computing a single explicit ratio like debt-to-equity.

Ratios can also be combined into a single index designed to predict financial distress, rather than read one at a time the way Table 4.4 presents them. The best known example is Altman's Z-score, and before writing down its formula it is worth noticing that four of its five components are variations on ratios this chapter has already computed under other names: a liquidity measure related to the working capital of Section 4.4.1, a leverage cushion related to the debt-to-equity ratio of Section 4.4.2, and an earning-power measure and an asset-turnover measure related to the DuPont components of Section 4.4.5, combined here with a measure of cumulative retained profitability and weighted by coefficients Altman estimated statistically from historical bankruptcy data rather than derived from theory. For a private (non-publicly-traded) firm like Meridian, the Z′-score variant is

$$Z' = 0.717X_1 + 0.847X_2 + 3.107X_3 + 0.420X_4 + 0.998X_5,$$

where

$$\begin{aligned} X_1 &= \text{Working capital/Total assets}, & X_2 &= \text{Retained earnings/Total assets}, \\ X_3 &= \text{EBIT/Total assets}, & X_4 &= \text{Book value of equity/Total liabilities}, \\ X_5 &= \text{Sales/Total assets}. \end{aligned}$$

Assuming, for illustration, that all of Meridian's \$54.0 million in equity represents retained earnings, the components are

$$X_1 = \frac{38.0}{129.0} = 0.295, \quad X_2 = \frac{54.0}{129.0} = 0.419, \quad X_3 = \frac{16.0}{129.0} = 0.124,$$

$$X_4 = \frac{54.0}{75.0} = 0.720, \quad X_5 = \frac{150.0}{129.0} = 1.163,$$

giving

$$Z' = 0.717(0.295) + 0.847(0.419) + 3.107(0.124) + 0.420(0.720) + 0.998(1.163) \approx 2.41.$$

Altman's original thresholds classify $Z' > 2.9$ as a "safe" zone, $1.23 < Z' < 2.9$ as a "grey" zone of ambiguous risk, and $Z' < 1.23$ as a "distress" zone with a high historical likelihood of bankruptcy within two years. Meridian's score of 2.41 falls squarely in the grey zone, consistent with everything the chapter has found so far: healthy-looking ratios on the surface, but a deteriorating cash position and rising leverage underneath. A score like this is a useful summary statistic precisely because it compresses many of the individual signals from this chapter into one number, but, like any single ratio, it should be a starting point for further investigation rather than an automatic verdict, and it is worth noting that Altman's coefficients were themselves originally estimated using the statistical discriminant-analysis techniques that are direct ancestors of the classification methods in Chapter 6.

Computing the same score for Solstice Robotics from Section 4.5.1 makes the contrast concrete. Using Solstice's Year 2 figures, and again assuming all of Solstice's \$75.0 million in equity represents retained earnings, working capital is $70.0 - 15.0 = 55.0$ (current assets less accounts payable, its only current liability), so

$$X_1 = \frac{55.0}{100.0} = 0.550, \quad X_2 = \frac{75.0}{100.0} = 0.750, \quad X_3 = \frac{27.0}{100.0} = 0.270,$$

Wulin.Suo@Queensu.ca

$$X_4 = \frac{75.0}{25.0} = 3.000, \quad X_5 = \frac{120.0}{100.0} = 1.200,$$

where Solstice's EBIT of \$27.0 million matches the interest coverage figure in Table 4.6 ($27.0 \times \$1.0$ million interest expense = \$27.0 million), giving

$$Z' = 0.717(0.550) + 0.847(0.750) + 3.107(0.270) + 0.420(3.000) + 0.998(1.200) \approx 4.33.$$

Solstice's score of 4.33 sits comfortably in Altman's "safe" zone, in sharp contrast to Meridian's 2.41 grey-zone score, and for the reasons the rest of this chapter has already surfaced: Solstice carries far less leverage ($X_4 = 3.00$ versus Meridian's 0.72) and a far larger cushion of retained earnings and working capital relative to its asset base. The two companies' Z-scores agree with, and quantitatively summarize, everything the DuPont decomposition and ratio comparison in Table 4.6 showed qualitatively: two firms can reach similar headline profitability, or in this case very different profitability, through very different combinations of margin, efficiency, and leverage, and a single distress score is only as informative as an analyst's understanding of which of those components is driving it.

4.6 From Statements to Forecasts

Statements describe the past; valuation needs the future. This part connects reported figures to the cash flows Chapter 5 discounts, notes how far reported numbers can be trusted, and builds a forecast from them.

4.6.1 Connecting Financial Statements to Valuation

The ultimate purpose of financial statement analysis is often valuation. Investors estimate the value of a firm by examining the cash flows it can generate and discounting them appropriately. The statements help identify these cash flows. Revenues, margins, working capital needs, investment requirements, and financing choices all affect the future cash flows that matter to investors.

A simple valuation logic is that a firm's value is the present value of future free cash flows,

$$V = \sum_{t=1}^{T} \frac{FCF_t}{(1+r)^t},$$

where r reflects the riskiness of those cash flows. The statements provide the raw material for this calculation. Revenue growth and margin trends inform the forecast of future profits. Balance sheet data inform working capital and investment needs. The cash flow statement shows the historical conversion of earnings into cash. If an analyst were to project Meridian's free cash flow forward using the current negative trajectory without adjustment, the implied valuation would be far less attractive than one based on net income alone; this is why practitioners insist on free cash flow rather than accounting earnings as the input to valuation.

Free cash flow, however, comes in two distinct varieties that answer two different questions, and conflating them is a common and consequential error. Both are built from CFO, cash flow from operations, the same operating cash flow figure already introduced in Section 4.3.3. Free cash flow to the firm (FCFF) measures cash available to *all* capital providers, debt and equity holders combined, before any financing decisions,

$$\text{FCFF} = \text{CFO} + \text{Interest expense} \times (1 - \text{tax rate}) - \text{Capex},$$

adding back after-tax interest expense since that expense is a payment *to* one class of capital provider (debtholders), not a cost of the underlying business itself. Free cash flow to equity (FCFE) measures cash available to equity holders specifically, after debtholders have already been paid and any net new borrowing has been received,

$$\text{FCFE} = \text{CFO} - \text{Capex} + \text{Net borrowing}.$$

Applying both to Meridian's Year 2 statements (CFO = \$6.0 million, Capex = \$14.0 million, interest expense \$4.0 million, tax rate 25%, net new debt issued \$12.0 million),

$$\text{FCFF} = 6.0 + 4.0(1 - 0.25) - 14.0 = -\$5.0 \text{ million}, \qquad \text{FCFE} = 6.0 - 14.0 + 12.0 = \$4.0 \text{ million}.$$

Wulin.Suo@Queensu.ca

The two measures do not merely differ in magnitude, they have opposite signs: Meridian's underlying business, considered independently of how it is financed, consumed $5.0 million more cash than it generated, but issuing $12.0 million of new debt more than covered that gap, leaving $4.0 million actually available to equity holders. An analyst who computed only FCFE would see a healthy-looking positive number and could easily miss that the firm's core operations are cash-negative and that this positive figure exists only because of a financing decision, an increase in leverage, that itself carries risk this chapter has already flagged. FCFF is the natural input to an enterprise-value calculation discounted at the WACC developed in Chapter 5, since it represents cash available to every capital provider WACC blends together, while FCFE is the natural input to a direct equity-value calculation discounted at the cost of equity alone, and using the wrong discount rate with the wrong cash flow measure, WACC applied to FCFE, or the cost of equity applied to FCFF, produces a valuation error that will not announce itself as an error, only as a wrong number.

This connection is especially important for AI applications in finance. A machine learning system can identify patterns in balance sheet data or cash flow data, but those patterns must be interpreted in economic terms. A model that predicts distress should be grounded in real indicators such as falling liquidity, rising leverage, declining margins, and weak cash generation. Otherwise it may be capturing noise rather than economic reality.

4.6.2 Accounting Quality and the Limits of Reported Numbers

One of the most important lessons in financial statement analysis is that reported numbers are not always fully transparent. Accounting rules allow management some discretion, particularly in areas such as revenue recognition, depreciation, reserves, and provisioning. A firm may use aggressive accounting policies to smooth earnings or to present a stronger balance sheet than the underlying reality supports.

This is why careful analysts ask not only what the statements say, but how they were produced. They look for unusual changes in reserves, one-time charges, deferred revenue patterns, and recurring non-cash items. They also compare reported figures with cash flow trends and outside information such as industry reports, competitor behavior, or macroeconomic indicators. In Meridian's case, the widening gap between net income and operating cash flow, driven by growing receivables and inventory, is a textbook signal that an analyst would flag for further investigation: is the firm's revenue recognition aggressive, are customers slowing their payments, or is inventory building up because of weakening demand?

This gap between reported earnings and cash generation can itself be quantified. One simple accruals ratio measure is

$$\text{Accruals ratio} = \frac{\text{Net income} - \text{Cash flow from operations}}{\text{Total assets}}.$$

For Meridian, this is $(9.0-6.0)/129.0 \approx 2.3\%$: net income exceeds operating cash flow by an amount equal to 2.3% of total assets. Academic research on earnings quality has repeatedly found that firms with unusually high accruals ratios, meaning reported earnings substantially exceed cash generation, tend on average to see earnings growth disappoint and stock returns underperform in subsequent periods, precisely because a large gap between earnings and cash is a warning sign that current profitability may be inflated by aggressive accounting choices, temporary working-capital effects, or both, rather than durable operating improvement. A single year's accruals ratio, as computed here for Meridian, is only weak evidence on its own; the signal becomes more informative when tracked over several years and compared against a firm's own history and its industry peers, the sector-adjusted, longitudinal comparison a large-scale, cross-sectional machine learning model of the kind covered in Chapter 6 is well suited to automate.

Accounting quality matters for AI as well. A machine learning model trained on financial statements may learn patterns that are partly an artifact of accounting conventions rather than economic reality. The analyst must therefore understand the reporting environment and remain skeptical of overly simplistic interpretations.

4.6.3 Forecasting from Statements

Financial statement analysis becomes especially valuable when it is used to forecast future performance. Analysts often project revenues, costs, margins, capital expenditures, and working capital needs. These

Wulin.Suo@Queensu.ca

projections then feed into valuation models. A well-constructed forecast can reveal whether a firm is likely to create value, consume cash, or require external financing.

A simple forecasting approach ties each balance sheet and income statement line to revenue. For example, if COGS is historically 64% of revenue (as it is for Meridian: 96.0/150.0 = 64%), and receivables are historically 16% of revenue (24.0/150.0), then a revenue forecast for Year 3 immediately implies forecasts for COGS and receivables under a "percent of sales" model. This method is intentionally simple; it can be extended with separate growth assumptions for margins, working capital efficiency, and capital intensity as more information becomes available.

Table 4.8 carries this method through completely, projecting Meridian's full Year 3 income statement and key working-capital lines under two assumptions: 8% revenue growth, and every other line holding its historical percent-of-revenue (or, for accounts payable, percent-of-COGS) ratio constant, with interest expense held fixed at its Year 2 dollar level since it depends on the debt outstanding rather than on sales.

Line	Year 2	Year 3 (forecast)	Basis
Revenue	150.0	162.0	8% growth assumption
Cost of goods sold	96.0	103.68	64.0% of revenue
Gross profit	54.0	58.32	
SG&A	30.0	32.40	20.0% of revenue
Depreciation and amortization	8.0	8.64	5.3% of revenue
Operating income (EBIT)	16.0	17.28	
Interest expense	4.0	4.0	held constant
Pretax income	12.0	13.28	
Taxes (25%)	3.0	3.32	
Net income	9.0	9.96	
Accounts receivable	24.0	25.92	16.0% of revenue
Inventory	30.0	32.40	20.0% of revenue
Accounts payable	17.0	18.36	17.7% of COGS

Table 4.8: A percent-of-sales forecast for Meridian, Year 3 ($ millions)

Two things stand out from this forecast. First, net income is projected to grow from $9.0 million to $9.96 million, a 10.7% increase, faster than the 8% revenue growth assumption, because fixed interest expense does not grow with revenue, so a larger share of incremental operating income flows through to the bottom line, an instance of financial leverage. Second, both accounts receivable and inventory are forecast to grow in proportion to revenue, which means, following the same logic as the Year 2 cash flow statement, this growth will itself consume cash even in a scenario where the business is performing exactly as historically expected; a complete forecast would carry this projected balance sheet forward into a projected cash flow statement to check whether Meridian's growth continues to be self-funding or continues to require external financing, the free-cash-flow-based valuation exercise in Section 4.6.1. This is precisely why forecasting from statements is not a purely mechanical exercise: the percent-of-sales method makes a specific, checkable assumption (that these ratios remain constant), and the value of the exercise comes as much from questioning that assumption as from computing its consequences.

Carrying out that complete forecast is worth doing explicitly, since it resolves a question the chapter has left open since Table 4.3. Year 3 cash flow from operations is projected net income plus depreciation, less the increase in receivables and inventory, plus the increase in payables:

$$\text{CFO}_3 = 9.96 + 8.64 - (25.92 - 24.0) - (32.40 - 30.0) + (18.36 - 17.0) = 15.64,$$

more than double Year 2's $6.0 million, because the dollar growth in net income and depreciation outpaces the dollar growth in working capital even though every underlying ratio is held constant. Holding capital expenditures at their Year 2 percentage of revenue (14.0/150.0 = 9.33%) gives a Year 3 projected capex of $0.0933 \times 162.0 \approx \$15.12$ million, so

$$\text{FCF}_3 = \text{CFO}_3 - \text{Capex}_3 = 15.64 - 15.12 = \$0.52 \text{ million},$$

Wulin.Suo@Queensu.ca

compared to Year 2's free cash flow of $6.0 - 14.0 = -\$8.0$ million. Under these percent-of-sales assumptions, Meridian's free cash flow is projected to turn from sharply negative to barely positive, without any change in margins or efficiency, purely because operating cash flow scales faster with revenue than working-capital investment does once the firm is past its heaviest receivables-and-inventory buildup. This projected \$0.52 million is the Year 3 figure with which Chapter 5's discounted-cash-flow valuation of Meridian begins: a naive analyst extrapolating Year 2's deeply negative free cash flow forward would undervalue Meridian relative to what the same percent-of-sales assumptions actually imply.

Forecasting requires a balance of structure and judgment. Some assumptions are grounded in historical averages, while others depend on business strategy, industry conditions, and macroeconomic outlook. A firm may be expected to improve margins due to economies of scale, but that improvement may be delayed if competition intensifies. A firm may plan to expand capacity, but execution risk may cause delays and cost overruns. In other words, financial forecasting is both quantitative and interpretive.

The best forecasts are usually built from simple, transparent assumptions that can be tested and updated. When a model becomes too complex, it may look precise while offering little economic insight. This is a lesson that applies equally to financial analysis and machine learning.

4.7 Challenges in Financial Statement Analysis

Several practical challenges complicate the use of financial statements, especially when they are used as inputs to quantitative or machine learning models.

Comparability across firms and time. Accounting standards differ across countries (for example, the International Financial Reporting Standards, or IFRS, used across much of the world, versus U.S. Generally Accepted Accounting Principles, or GAAP), and even within a single standard, firms have discretion over methods such as inventory costing (FIFO versus LIFO) or depreciation schedules. A model trained on ratios pooled across many firms may be comparing numbers that are not truly comparable unless these differences are adjusted for.

Reporting lag and low frequency. Financial statements are typically published quarterly or annually, with a delay of weeks after the period ends. This makes them a slow-moving input compared to market prices or transaction data, and any model relying primarily on statement data will react to changes in a firm's condition only after a considerable delay.

Restatements and revisions. Reported figures are sometimes restated after initial publication, whether due to errors, changes in accounting policy, or corrections following audits. A historical dataset built from "as first reported" figures can differ meaningfully from one built with the benefit of later restatements, and using the wrong vintage of data can introduce a subtle form of look-ahead bias into a backtest.

Manipulation and earnings management. As discussed above, management has some latitude in how figures are presented. Detecting earnings management, for example through unusual accruals, is itself an active area of quantitative finance research, and a model that ignores this risk may be fooled by cosmetically strong statements.

Non-financial and off-balance-sheet items. Important sources of risk or value, such as operating leases (in some reporting regimes), pending litigation, environmental liabilities, or human capital, may not appear cleanly on the balance sheet at all. Statement-based models can therefore miss risks that are economically material but not captured in the standard line items.

Sector heterogeneity. A ratio that signals distress in one industry may be entirely normal in another; capital-intensive sectors like utilities or airlines routinely carry leverage ratios that would be alarming for a software company. Any large-scale, cross-sectional model needs to account for sector context rather than applying a single threshold universally.

Recognizing these challenges does not mean abandoning statement-based analysis. It means treating financial statements as a valuable but imperfect and delayed signal, to be combined with market data, qualitative judgment, and, where available, higher-frequency alternative data.

Wulin.Suo@Queensu.ca

4.8 Key Takeaways

Financial statements are the backbone of corporate finance. The balance sheet reveals the firm's financial structure, the income statement reveals the firm's performance, and the cash flow statement reveals the movement of cash. Together they help investors, managers, and analysts understand whether a business is profitable, liquid, solvent, and capable of creating value.

The Meridian example illustrated how a firm can report solid net income and comfortable liquidity ratios while simultaneously burning cash and increasing leverage, a discrepancy that only becomes visible when all three statements, and their ratios, are examined together. A strong understanding of these statements is essential before one can use more advanced forecasting models or valuation methods, and it is equally essential before trusting any machine learning model built on top of accounting data.

The chapter's comparison with Solstice Robotics reinforced a point that recurs throughout this book: a single summary statistic, whether a ratio, a Z'-score, or a machine learning model's output score, can mask very different underlying stories, and the DuPont identity and common-size statements exist specifically to unpack those stories rather than accept the summary number at face value.

The sector-specific discussion in this chapter is a caution against building a single cross-firm model without accounting for the fact that a bank, a utility, and a technology company operate under fundamentally different financial structures, even when they report using the same three statements.

Carried forward, the lesson for later chapters is direct: Chapter 5's valuation methods take cash flows as an input, and those cash flows come from the kind of statement analysis, and skepticism about statement quality, developed here; Chapter 9's credit models build directly on the leverage, liquidity, and distress-scoring ideas in this chapter, extended with statistical estimation in place of a fixed formula like Altman's Z'-score.

4.9 Suggested Exercises

1. Explain the difference between assets, liabilities, and equity, and verify that the accounting identity holds for both years of Meridian's balance sheet.

2. Using Table 4.1 and Table 4.2, compute Meridian's current ratio, quick ratio, and debt-to-equity ratio for Year 1, and compare them to the Year 2 values computed in the chapter.

3. Describe why cash flow can differ from accounting profit, using Meridian's Year 2 numbers as a concrete illustration.

4. Compute Meridian's free cash flow if capital expenditures had instead been \$6.0 million rather than \$14.0 million, holding all other cash flow items fixed, and discuss how this would change your assessment of the firm.

5. Using the DuPont identity, decompose Meridian's ROE for Year 1 (assume net income of \$7.5 million and revenue of \$135.0 million for Year 1) and compare the drivers of ROE across the two years.

6. Explain why a company with growing sales, like Meridian, might still face financial distress, referencing the cash conversion cycle.

7. Discuss two ways that accounting discretion could distort a machine learning model trained to predict corporate default using financial ratios.

8. Propose a percent-of-sales forecast for Meridian's Year 3 revenue, COGS, and accounts receivable, assuming 8% revenue growth and that historical ratios to revenue remain constant.

9. Using Table 4.5, verify the current ratio, quick ratio, and debt-to-equity ratio reported for Solstice in Table 4.6.

10. Explain, using the DuPont identity, why Meridian and Solstice can have different return on equity even though the gap between their net margins (6.0% versus 16.25%) is larger than the gap in their ROE (16.7% versus 26.0%).

Wulin.Suo@Queensu.ca

11. Using Table 4.5, construct a common-size balance sheet (each line as a percent of total assets) for Solstice's Year 2 balance sheet, and compare it to Meridian's common-size balance sheet computed in this chapter, identifying the single largest structural difference between the two firms.

12. Compute Meridian's accruals ratio for a hypothetical Year 3 in which net income is \$11.0 million and cash flow from operations is \$4.0 million (total assets unchanged at \$129.0 million), and explain whether this year's figure is more or less concerning than Year 2's.

13. Using the Z′-score formula, recompute Meridian's distress score assuming Year 1 figures instead of Year 2 (Year 1 EBIT can be assumed at \$13.0 million, retained earnings equal to Year 1 equity of \$52.0 million, and other Year 1 figures from Table 4.1), and discuss whether Meridian's financial health appears to be improving or deteriorating between the two years.

14. Verify the Solstice Z′-score computed in Section 4.5.1 by recomputing X_1 through X_5 from Table 4.5, and explain in one sentence which single component contributes most to the gap between Solstice's score and Meridian's.

15. Using Table 4.8's Year 3 projections, recompute Meridian's projected free cash flow if the revenue growth assumption were 12% instead of 8% (holding every ratio in Table 4.8 fixed at its stated percentage of revenue or COGS), and explain why free cash flow does not simply scale in proportion to revenue growth.

16. Compute Meridian's return on invested capital and economic profit if its weighted average cost of capital were 12% instead of 8%, holding NOPAT and invested capital at their Year 2 values, and explain what this implies about whether Meridian is creating or destroying value at that higher cost of capital.

17. **Challenge (real data).** Pick any US public company and download its most recent 10-K balance sheet and income statement figures directly from the SEC EDGAR XBRL `companyfacts` API (free, no login required). (a) Compute the same ratio suite Section 4.4.5 computed for Meridian: current ratio, quick ratio, debt-to-equity, and a three-factor DuPont ROE decomposition. (b) Using this chapter's Altman Z′-score formula, classify the firm into the safe, grey, or distress zone, and comment on whether the classification matches your own intuition about the company's financial health. (c) Real XBRL filings tag the same economic concept (for example, inventory or long-term debt) under dozens of different standardized and company-specific tags across firms and years; describe at least one tag-matching or missing-data problem you encountered while pulling the figures needed for part (a), and how you resolved it.

4.10 Further Reading

- Brealey, Myers, and Allen, *Principles of Corporate Finance*. Covers the corporate-finance logic behind cash flow, capital structure, and value creation that underlies this chapter's ratio analysis and forecasting material.

- Penman, *Financial Statement Analysis and Security Valuation*. A rigorous treatment of how ratio analysis and common-size statements, introduced in this chapter with Meridian and Solstice, connect to security valuation more broadly.

- Damodaran, *Investment Valuation*. Extends this chapter's percent-of-sales forecasting into the full range of valuation techniques developed formally in Chapter 5.

- Palepu, Healy, and Peek, *Business Analysis and Valuation*. A case-based approach to interpreting financial statements and diagnosing accounting quality issues, complementing this chapter's Altman Z-score and distress-analysis material.

Wulin.Suo@Queensu.ca

Chapter 5

Valuation and Asset Pricing

5.1 Introduction

The preceding chapters built the pieces needed to price a financial asset: Chapter 3 described the institutions and instruments that are traded, and Chapter 4 showed how a firm's financial statements reveal the cash flows it actually generates. This chapter puts those pieces together and asks the central question of asset pricing: given a stream of future cash flows, or a payoff that depends on some other asset's price, what should that claim be worth today?

The chapter answers this question with worked numerical examples for the three main instrument classes introduced in Chapter 3: bonds, equities, and derivatives. Every formula here is accompanied by a computation a reader can check by hand or reproduce in the companion notebook, in keeping with this book's view that a quantitative finance practitioner should be able to compute a price directly, not just describe the logic behind it qualitatively.

The chapter is organized to build in complexity. It begins with the simplest possible cash flow structures, a single-maturity bond and a constant-growth dividend stream, where closed-form formulas exist and every number can be verified by hand. It then introduces two ways of estimating the discount rate that all of these formulas depend on, the term structure of interest rates and the cost of capital, before turning to the more complex, nonlinear payoffs of options, which require the replication and no-arbitrage arguments central to modern derivatives pricing. A useful way to read the chapter is as a single valuation story: Meridian Fabrication, the running example introduced in Chapter 4, first illustrates how a firm's financing mix determines the discount rate, is then valued with a market-based multiple, and is finally valued from its projected cash flows, with the two values compared side by side. The goal is not to treat bonds, equities, and options as unrelated topics, but to show that the same present-value logic reappears in each context.

5.2 Discounting and Fixed Income

Every valuation in this book reduces to moving cash across time and charging for the privilege. This part develops that machinery on the instruments where it is cleanest, then asks what happens to a bond's price when the rate used to discount it moves.

5.2.1 The Time Value of Money, Revisited

Chapter 1 introduced the present value relation

$$V = \sum_{t=1}^{T} \frac{CF_t}{(1+r)^t}$$

as the unifying idea behind bond pricing, stock valuation, and project appraisal. This chapter makes that idea operational. Two quantities determine the value of any claim on future cash flows: the size and timing of the cash flows themselves, and the discount rate r, which compensates the holder for the time value of

Wulin.Suo@Queensu.ca

money and for the risk that the cash flows might not arrive as expected. A safer stream of cash flows is discounted at a lower rate and is therefore worth more, all else equal, than a riskier stream of the same expected size. Everything in this chapter is an application of this single idea to a specific type of contract.

It is worth being explicit about why the same present-value relation can describe such different instruments. A bond's cash flows are contractually fixed (barring default) and known in advance; a stock's cash flows are uncertain and expected to grow; an option's payoff is not just uncertain but depends nonlinearly on another asset's future price. What varies across the chapter is not the underlying valuation principle, only the complexity of CF_t and the sophistication required to pin down r.

5.2.2 Bond Pricing and Yield to Maturity

Bonds are the natural starting point for this chapter because their cash flows are the least ambiguous of any instrument covered here: a fixed schedule of coupons and a principal repayment, specified in the bond's contract at issuance. A bond with face value F, annual coupon C, and n years to maturity is priced by discounting each promised cash flow at the market yield y:

$$P = \sum_{t=1}^{n} \frac{C}{(1+y)^t} + \frac{F}{(1+y)^n}.$$

Consider a 3-year bond with face value $F = \$1{,}000$ and an annual coupon rate of 5% (so $C = \$50$ per year), priced at a market yield of $y = 7\%$. Table 5.1 shows each discounted cash flow.

Year t	Cash flow	Discount factor $(1+y)^{-t}$	Present value
1	\$50.00	0.9346	\$46.73
2	\$50.00	0.8734	\$43.67
3	\$1,050.00	0.8163	\$857.11
		Price	**\$947.51**

Table 5.1: Pricing a 3-year, 5% annual-coupon bond at a 7% yield

The bond trades below its \$1,000 face value because the market yield (7%) exceeds the coupon rate (5%); an investor demands compensation via a lower purchase price. The yield to maturity is, conversely, the single discount rate y that makes the present value of the promised cash flows equal to the observed market price; for $n > 1$ this equation generally has no closed-form solution and is instead solved numerically (for example with a root-finding routine such as `scipy.optimize.brentq`).

This inverse relationship between price and yield, price falls when yields rise and rises when yields fall, is one of the most important facts in fixed income and is the reason bond portfolios carry interest-rate risk. The sensitivity of price to yield is commonly summarized by duration, a weighted average of the times at which cash flows are received, weighted by the present value of each cash flow. For the bond in Table 5.1, the (Macaulay) duration is approximately 2.86 years, close to but less than the 3-year maturity, because part of the value comes from the earlier coupon payments.

Pricing Default Risk

The bond pricing formula above assumes the promised cash flows arrive with certainty, which is a reasonable approximation for a government bond but not for a corporate bond, where there is some chance of default. A simple way to incorporate this is to discount the *expected* cash flow, accounting for both the probability of default and the partial recovery a bondholder typically receives even after a default, rather than the promised cash flow.

Consider a simplified one-year, zero-coupon corporate bond with face value \$1,000, a risk-free rate of $r_f = 4\%$, an estimated one-year probability of default $p = 3\%$, and a recovery rate of 40% of face value if default occurs (that is, bondholders recover \$400 per \$1,000 of face value in default). The expected payoff at maturity is

$$\mathbb{E}[\text{Payoff}] = (1-p)(\$1{,}000) + p(0.40)(\$1{,}000) = 0.97(1{,}000) + 0.03(400) = \$982,$$

Wulin.Suo@Queensu.ca

and discounting this expected payoff at the risk-free rate gives a price of

$$P = \frac{982}{1.04} \approx \$944.23.$$

Equivalently, this can be expressed as a credit spread added to the risk-free rate: for small default probabilities, the required credit spread is approximately

$$p \times (1 - \text{recovery rate}) = 0.03 \times 0.60 = 1.8\%,$$

giving a corporate bond yield of roughly $4\% + 1.8\% = 5.8\%$ and an approximate price of $1{,}000/1.058 \approx \$945.18$, close to but not identical to the expected-cash-flow calculation, since the credit-spread shortcut is itself an approximation. Either approach illustrates the same core idea: default risk is priced by widening the effective discount rate, equivalently by shrinking the expected cash flow, relative to a risk-free bond with the same promised payments. The size of that adjustment depends directly on the market's estimate of both the probability of default and the severity of loss if default occurs, two quantities that Chapter 9's credit models are built specifically to estimate.

5.2.3 Interest Rate Risk: Using Duration to Approximate Price Changes

Duration is not just a descriptive statistic; it is the basis of a widely used first-order approximation for how a bond's price responds to a change in yield. Modified duration is defined as

$$D_{\text{mod}} = \frac{D_{\text{Mac}}}{(1+y)},$$

and the approximate percentage price change for a small yield change Δy is

$$\frac{\Delta P}{P} \approx -D_{\text{mod}}\,\Delta y.$$

For the bond in Table 5.1, $D_{\text{mod}} = 2.86/1.07 = 2.67$. If the market yield rises from 7% to 8% ($\Delta y = +0.01$), the approximation predicts

$$\Delta P \approx -2.67 \times 0.01 \times \$947.51 \approx -\$25.28, \qquad P_{\text{new}} \approx \$922.23.$$

Repricing the bond exactly at $y = 8\%$ using the original present-value formula gives $P_{\text{new}} = \$922.69$, so the duration approximation is off by about \$0.46, understating the actual price. If instead the yield falls from 7% to 5% ($\Delta y = -0.02$), the approximation gives $P_{\text{new}} \approx \$998.08$, while the exact price is $P_{\text{new}} = \$1{,}000.00$ (at $y = 5\%$, the yield equals the coupon rate, so the bond prices exactly at par). In both directions the duration approximation understates the true price, an effect known as convexity: the true price-yield relationship curves upward, so a linear approximation based on duration alone always predicts a smaller price for a yield increase and a smaller price for a yield decrease than actually occurs. For small yield changes duration is normally an adequate approximation, but for larger moves, or for portfolios where precision matters, convexity adjustments or full repricing are used instead.

Convexity can be estimated directly from three prices, the price today and the prices after a small yield increase and decrease, using

$$\text{Convexity} \approx \frac{P_+ + P_- - 2P_0}{P_0(\Delta y)^2},$$

and the second-order (duration-plus-convexity) approximation to the price change is

$$\frac{\Delta P}{P} \approx -D_{\text{mod}}\,\Delta y + \frac{1}{2}\,\text{Convexity}\,(\Delta y)^2.$$

Using $P_0 = \$947.51$ (y=7%), $P_+ = \$922.69$ (y=8%), and $P_- = \$973.27$ (y=6%),

$$\text{Convexity} \approx \frac{922.69 + 973.27 - 2(947.51)}{947.51\,(0.01)^2} \approx 9.81.$$

Wulin.Suo@Queensu.ca

Adding the convexity term to the earlier duration-only forecast for a rise to $y = 8\%$ gives

$$P_{\text{new}} \approx 947.51\left(1 - 2.67(0.01) + \tfrac{1}{2}(9.81)(0.01)^2\right) \approx \$922.69,$$

which matches the exact repriced value to the penny, a dramatic improvement over the duration-only estimate of \$922.23. This is the general pattern: duration captures the slope of the price-yield relationship, while convexity captures its curvature, and including both terms, rather than duration alone, is standard practice whenever a fixed-income portfolio's risk is being measured precisely, for example in the value-at-risk calculations of Chapter 8.

Portfolio Duration

Duration extends naturally from a single bond to a portfolio of bonds: the portfolio's duration is simply the market-value-weighted average of the durations of its individual holdings,

$$D_{\text{portfolio}} = \sum_i w_i D_i,$$

where w_i is the fraction of portfolio market value invested in bond i, the same weighted-average structure used for portfolio return in Chapter 2 and for portfolio variance in Chapter 12. Suppose a fixed-income portfolio holds 60% of its market value in the 3-year bond from Table 5.1 (duration 2.86 years when rounded, or 2.8553 years at full precision) and 40% in a 5-year zero-coupon bond priced at a 7% yield (a zero-coupon bond's duration always equals its maturity, since it has only a single cash flow, so its duration is exactly 5 years). Using the bond's full-precision duration rather than the rounded 2.86 figure, the portfolio duration is

$$D_{\text{portfolio}} = 0.6(2.8553) + 0.4(5.0) \approx 3.71 \text{ years}.$$

This single number allows a portfolio manager to approximate the percentage price impact of a parallel shift in interest rates on the entire portfolio at once, using the same modified-duration approximation introduced above, without needing to reprice every individual bond, which is especially valuable for portfolios containing hundreds of fixed-income positions. Managing a portfolio's duration, for example by adjusting the mix of short- and long-maturity bonds to hit a specific target duration, is one of the most basic and widely used fixed-income risk management techniques, and it reappears directly in the portfolio construction methods of Chapter 12 and the market risk measures of Chapter 8.

5.2.4 The Term Structure of Interest Rates

Every bond pricing example in this chapter has used a single discount rate y for all maturities, but in reality, the market yield on a bond typically depends on its maturity, a relationship called the term structure of interest rates, or the yield curve. Table 5.2 shows a stylized, upward-sloping yield curve for government bonds of different maturities.

Maturity	1 year	2 years	5 years	10 years	30 years
Yield	4.2%	4.4%	4.8%	5.2%	5.5%

Table 5.2: A stylized government bond yield curve

An upward-sloping yield curve, like the one in Table 5.2, is the most common shape historically and is usually attributed to a combination of two effects: investors typically demand a higher yield to compensate for the greater interest-rate risk of holding a longer-maturity bond (a term premium), and the market may expect short-term rates to rise in the future, which is itself reflected in higher long-term yields today. When the curve inverts, meaning short-term yields exceed long-term yields, it is often interpreted as a signal that the market expects future interest-rate cuts, typically associated with an anticipated economic slowdown, which is why yield curve inversions are closely watched as a potential recession indicator and are the kind of macroeconomic time series relationship studied further in Chapter 7.

Wulin.Suo@Queensu.ca

Once a full yield curve is available, each cash flow of a bond should, in principle, be discounted at the rate appropriate to its own maturity rather than at a single blended rate, using the spot rate for that specific maturity rather than the bond's own yield to maturity, which is really a complex weighted average of the spot rates for each of its cash flows' maturities. This distinction matters most for bonds with cash flows spread over a long horizon and becomes central when pricing and hedging any interest-rate-sensitive instrument precisely, though the single-rate approximation used throughout the rest of this chapter is adequate for building intuition about the mechanics of bond pricing.

A Worked Example: Extracting Forward Rates from the Spot Curve

The spot rates in Table 5.2 are rates for borrowing or lending starting today; a forward rate is the rate, implied by today's spot curve, for a loan that starts at some future date and ends at a later one, and no-arbitrage pins its value down exactly, using the same replication logic as Chapter 3's no-arbitrage bond example. Investing for two years at the 2-year spot rate must give the same payoff as investing for one year at the 1-year spot rate and then reinvesting for a second year at whatever the market currently expects (or has locked in via a forward contract) for that second year,

$$(1+y_2)^2 = (1+y_1)(1+f_{1,2}) \quad \Longrightarrow \quad f_{1,2} = \frac{(1+y_2)^2}{1+y_1} - 1.$$

Using Table 5.2's 1-year rate of 4.2% and 2-year rate of 4.4%,

$$f_{1,2} = \frac{1.044^2}{1.042} - 1 \approx 4.60\%,$$

higher than either the 1-year or 2-year spot rate, exactly what an upward-sloping curve implies: the market-implied rate for the second year alone must be higher than the 2-year average to pull that average up from the lower 1-year rate. The same logic extends to any pair of maturities; using the 2-year and 5-year spot rates, the implied 3-year forward rate spanning years 2 through 5 is

$$f_{2,5} = \left(\frac{(1+y_5)^5}{(1+y_2)^2}\right)^{1/3} - 1 \approx 5.07\%,$$

again above both the 2-year and 5-year spot rates, consistent with the curve's continued upward slope over that stretch. Forward rates matter well beyond this arithmetic exercise: they are the rates a company can lock in today via a forward-starting swap or a forward-rate agreement (close relatives of the interest rate swaps and forward contracts introduced in Chapter 3) to hedge future borrowing costs, and, under the pure expectations theory of the term structure, they can be interpreted (with important caveats about the term premium mentioned above) as the market's best current forecast of where future short-term rates will actually be.

5.3 Valuing Equity

An equity claim has no maturity and no promised payment, so its value depends entirely on what is expected of it and on the rate demanded in exchange. This part covers where that rate comes from and the two families of method built on it.

5.3.1 The Cost of Capital: CAPM and WACC

Every valuation formula in this chapter takes a discount rate r as an input, and it is worth pausing to show how that rate is actually estimated in practice, since the bond pricing examples so far, like the dividend discount examples that follow, have simply been given a value of r to use. Firms are financed by a mix of debt and equity, and each source of capital has its own cost.

The cost of debt is the most straightforward: it is simply the yield the firm currently pays on its debt (or would pay on new debt of similar risk), adjusted for the tax deductibility of interest payments, since interest expense reduces taxable income,

$$r_d^{\text{after-tax}} = r_d^{\text{pretax}}(1-\tau),$$

Wulin.Suo@Queensu.ca

where τ is the firm's marginal tax rate. The cost of equity is harder to observe directly, since equity has no contractual payment, and is typically estimated using the Capital Asset Pricing Model (CAPM), covered in full in Chapter 12,

$$r_e = r_f + \beta(r_m - r_f),$$

where r_f is the risk-free rate, $r_m - r_f$ is the market risk premium (the extra return investors demand for holding the overall market rather than a risk-free asset), and β measures the stock's sensitivity to market-wide movements. The weighted average cost of capital (WACC) then blends the two financing costs in proportion to their share of the firm's total capital,

$$\text{WACC} = \frac{E}{E+D}\, r_e + \frac{D}{E+D}\, r_d^{\text{after-tax}},$$

where E and D are the market (or, in the absence of market prices, book) values of equity and debt.

Using Meridian Fabrication's Year 2 figures from Chapter 4,

$$E = \$54.0 \text{ million (equity)}, \qquad D = \$58.0 \text{ million (total debt)},$$
$$r_f = 4\%, \qquad \text{Market risk premium} = 6\%,$$
$$\beta = 1.2, \qquad \text{Pretax cost of debt} = 4.0/58.0 \approx 6.90\%,$$
$$\text{Tax rate} = 25\%,$$

where the pretax cost of debt is read off Meridian's own statements rather than assumed: Year 2 interest expense of \$4.0 million on the \$58.0 million of debt outstanding at year end. A rate nearly three percentage points above the risk-free rate is what one would expect for a firm whose Altman Z′-score places it in the grey zone (Chapter 4). Then

$$r_e = 0.04 + 1.2(0.06) = 11.2\%, \qquad r_d^{\text{after-tax}} = 0.0690(1 - 0.25) \approx 5.17\%,$$

$$\text{WACC} = \frac{54.0}{112.0}(11.2\%) + \frac{58.0}{112.0}(5.17\%) \approx 0.482(11.2\%) + 0.518(5.17\%) \approx 8.08\%.$$

Book equity stands in for E here because Meridian's shares have no market price in this example; Section 5.5 returns to what changes when a market-based equity value is used instead. This WACC of 8.08%, close to the 8% Chapter 4's economic-profit example assumed, is precisely the kind of discount rate that would be used to compute Meridian's enterprise value via the free-cash-flow valuation formula in Chapter 4, and it is also the discount rate that belongs in the economic-profit formula, Invested capital × (ROIC − WACC), introduced when that chapter discussed how firms create value: a firm earning a return on invested capital below its WACC of 8.08% is destroying value even if it is reporting positive accounting profit. Notice how sensitive WACC is to its inputs: a beta of 1.5 instead of 1.2 would raise the cost of equity to $0.04 + 1.5(0.06) = 13\%$ and the WACC to roughly 8.95%, which is precisely the discount-rate uncertainty flagged as a challenge later in this chapter.

5.3.2 The Dividend Discount Model for Equities

Equity valuation follows the same discounting logic but applies it to an infinite, growing stream of dividends rather than a fixed set of coupons. The Gordon growth model prices a share as

$$P_0 = \frac{D_1}{r - g},$$

where D_1 is the dividend expected next year, r is the required rate of return on equity, and $g < r$ is the assumed constant long-run growth rate of dividends. For example, if a firm is expected to pay a dividend of $D_1 = \$3.00$ next year, investors require a return of $r = 9\%$, and dividends are expected to grow at $g = 4\%$ per year indefinitely, the model implies a fair price of

$$P_0 = \frac{3.00}{0.09 - 0.04} = \frac{3.00}{0.05} = \$60.00.$$

Wulin.Suo@Queensu.ca

The model is deliberately simple, and its simplicity is also its main weakness: the implied price is extremely sensitive to the gap $r - g$, so small changes in the assumed growth rate or discount rate can produce very large changes in the estimated value. This sensitivity, not just the formula itself, is the practical lesson: any valuation built on this model should be stress-tested against a range of plausible (r, g) pairs rather than reported as a single point estimate.

A single constant growth rate forever is a strong assumption, since young or fast-growing firms rarely sustain a high growth rate indefinitely. The two-stage dividend discount model relaxes this by assuming a high growth rate g_1 for the first n years, followed by a lower, sustainable growth rate $g_2 < r$ thereafter. The price is the present value of the high-growth dividends plus the present value of a terminal value computed with the Gordon growth formula at the end of the high-growth period:

$$P_0 = \sum_{t=1}^{n} \frac{D_0(1+g_1)^t}{(1+r)^t} + \frac{1}{(1+r)^n}\left[\frac{D_0(1+g_1)^n(1+g_2)}{r-g_2}\right].$$

For example, suppose a firm's most recent dividend was $D_0 = \$3.00$, it is expected to grow at $g_1 = 12\%$ for the next 3 years before settling into a stable long-run growth rate of $g_2 = 4\%$, and investors require $r = 9\%$. The high-growth dividends are

$$D_1 = \$3.36, \qquad D_2 = \$3.76, \qquad D_3 = \$4.21,$$

with a combined present value of \$9.50. The terminal value at the end of year 3 is

$$\frac{D_3(1+g_2)}{r-g_2} = 4.21(1.04)/0.05 \approx \$87.67,$$

which has a present value of \$67.70. Adding the two pieces gives

$$P_0 \approx \$9.50 + \$67.70 = \$77.20.$$

Notice that the terminal value, representing everything beyond year 3, accounts for roughly 88% of the total price; this is typical of growth-stage valuations and is why the assumed terminal growth rate g_2, not the near-term high growth rate, is usually the most consequential assumption in the model.

Preferred Stock: A Perpetuity Special Case

Preferred stock typically pays a fixed dividend indefinitely, with no growth at all, which makes the Gordon growth formula collapse to the special case $g = 0$:

$$P_0 = \frac{D}{r}.$$

This is simply the formula for a growing perpetuity with zero growth, sometimes called a level perpetuity. For example, a preferred share paying a fixed annual dividend of $D = \$4.50$, with investors requiring $r = 7.5\%$ given the security's risk, is worth $P_0 = 4.50/0.075 = \$60.00$. Because preferred dividends are fixed rather than growing, preferred stock valuation is, in this sense, closer in spirit to the bond pricing formula earlier in this chapter than to the growth-oriented common-stock valuation just discussed, even though preferred stock is legally a form of equity; it is a useful reminder that an instrument's economic behavior, here, a fixed, bond-like cash flow, matters more for valuation purposes than its legal label.

5.3.3 Relative Valuation: Multiples

Every valuation method in this chapter so far is an absolute valuation method: it estimates a security's intrinsic value directly from a model of its cash flows. Relative valuation takes a completely different approach: instead of modeling cash flows, it prices an asset based on how similar assets are currently priced in the market, using a valuation multiple.

The most common multiple for valuing an entire business is enterprise value to EBITDA (earnings before interest, taxes, depreciation, and amortization), since EBITDA approximates operating cash flow before

Wulin.Suo@Queensu.ca

financing and accounting choices and enterprise value (the value of both debt and equity claims combined) is capital-structure-neutral, unlike equity value alone. The method has three steps: compute the target firm's EBITDA, multiply it by a multiple observed for comparable, publicly traded firms, and then back out the implied equity value by subtracting net debt (total debt minus cash).

Applying this to Meridian Fabrication: EBITDA is EBIT plus depreciation and amortization, $16.0 + 8.0 = \$24.0$ million. Suppose comparable industrial parts manufacturers currently trade at an EV/EBITDA multiple of 6.5×. The implied enterprise value is

$$EV = \text{EBITDA} \times \text{Multiple} = 24.0 \times 6.5 = \$156.0 \text{ million}.$$

Net debt is total debt minus cash, $58.0 - 9.0 = \$49.0$ million, so the implied equity value is

$$\text{Equity value} = EV - \text{Net debt} = 156.0 - 49.0 = \$107.0 \text{ million}.$$

This is nearly double Meridian's \$54.0 million book value of equity from Chapter 4's balance sheet. That is a useful reminder that the two are conceptually different quantities which need not be close to one another: book equity is an accounting construct built from historical costs, while market, or market-implied, equity value is a forward-looking assessment of what the business is actually worth.

Relative valuation is fast and grounded in observable market prices, which is its main appeal. But it depends entirely on the comparable multiple being appropriate. If the comparable firms are systematically overvalued or undervalued, cyclically distorted, or simply not truly comparable in growth, margins, and risk to the target firm, the resulting valuation inherits that same error. Absolute (discounted-cash-flow) and relative (multiples-based) valuation are best used together rather than as substitutes: a DCF valuation grounded in explicit assumptions about growth and discount rates, checked against a multiples-based valuation grounded in current market pricing. A significant disagreement between the two is itself valuable information. It prompts the analyst to ask which set of assumptions, the DCF's or the market's, is more likely to be right.

So far, the chapter has valued claims by discounting future cash flows or by benchmarking them against similar assets in the market. Options require a different lens because their payoff is nonlinear and depends on the future price of another asset. The table below summarizes the main valuation tools introduced in the chapter and highlights when each is most useful.

Method	Best used for	Main inputs	Main caveat
Bond pricing	Fixed-income claims with known coupons	Coupon, maturity, face value, yield	Assumes the promised cash flows are paid as stated
Dividend discount model	Mature, dividend-paying equities	Dividend, growth, required return	Very sensitive to the gap $r - g$ and the terminal growth assumption
WACC-based DCF	Firms or projects with a mix of debt and equity	Free cash flow, growth, cost of debt, cost of equity, capital structure	Discount-rate uncertainty can dominate the final value
Multiples	Quick relative valuation	EBITDA or earnings, peer multiple, net debt	Depends on the comparables being truly similar
Binomial or Black-Scholes-Merton	Options and other nonlinear payoffs	Underlying price, strike, volatility, time, rates	Assumes simplifying conditions such as constant volatility

Table 5.3: A quick guide to the main valuation tools in this chapter

Wulin.Suo@Queensu.ca

5.4 Option Pricing

An option's payoff is not linear in the underlying price, which makes discounting an expected value the wrong tool. This part builds the replication argument that replaces it, works it up to Black-Scholes-Merton, and follows the consequences into risk measurement and corporate decisions.

5.4.1 No-Arbitrage Pricing and the Binomial Option Model

Every valuation formula so far in this chapter has followed the same recipe: figure out the cash flow (or expected cash flow) an instrument delivers, then discount it at a rate reflecting its risk. An option seems, at first glance, like it should work the same way: estimate the expected payoff, discount it back at some rate, done.

This natural approach turns out to fail for a specific, structural reason. A bond's coupon and a stock's expected dividend are each a single, well-defined number to discount; an option's payoff depends on where the underlying price ends up, and that dependence is not a straight line. A European call option with strike K pays $\max(S_T - K, 0)$ at maturity T, where S_T is the underlying asset price at maturity; a European put pays $\max(K - S_T, 0)$.

Figure 5.1 plots both payoffs as a function of the terminal stock price S_T for a strike of $K = \$50$. The call is worthless ("out of the money") for any $S_T \leq K$ and rises one-for-one with S_T once S_T exceeds K ("in the money"); the put shows the mirror-image pattern. This kink at $S_T = K$ is precisely what makes options nonlinear payoffs and what necessitates the replication argument developed below, rather than a simple discounted-cash-flow calculation.

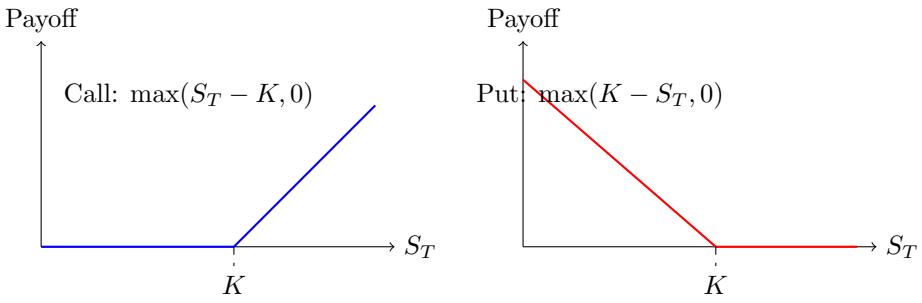


Figure 5.1: Payoff at maturity for a call option (left) and a put option (right) with strike K

Because these payoffs are nonlinear functions of the future price, options cannot be priced by simple discounting; instead, they are priced by constructing a replicating portfolio of the underlying asset and a risk-free bond that produces the same payoff in every future state, and then invoking the no-arbitrage principle introduced in Chapter 3: two portfolios with identical payoffs must have the same price today.

The simplest version of this argument is the one-step binomial model. Suppose a stock currently trades at $S_0 = \$50$ and, over one period, either rises to $S_0 u$ or falls to $S_0 d$, with $u = 1.2$ and $d = 0.9$. A risk-free investment over the same period grows by a gross factor $R = 1 + r_f = 1.04$. Consider a call option struck at $K = \$50$. The stock will be worth $S_u = 60$ or $S_d = 45$, so the call will be worth $C_u = \max(60 - 50, 0) = \$10$ or $C_d = \max(45 - 50, 0) = \$0$. No-arbitrage pricing sidesteps the need to know the stock's actual, real-world probability of rising or falling at all: it instead asks what probability of an up-move would make a portfolio of the stock and a risk-free bond, chosen to replicate the option's payoff exactly in both future states, price correctly today, and uses that probability, not any real forecast, to value the option, giving the option's value today as a discounted expectation under this risk-neutral probability

$$q = \frac{R - d}{u - d} = \frac{1.04 - 0.9}{1.2 - 0.9} = 0.467,$$

so that

$$C_0 = \frac{1}{R}[q\, C_u + (1 - q)\, C_d] = \frac{1}{1.04}[0.467(10) + 0.533(0)] \approx \$4.49.$$

Wulin.Suo@Queensu.ca

Crucially, q is not the investor's real-world belief about the probability of an up-move; it is the probability implied by the no-arbitrage condition, which is why this is called risk-neutral pricing. The same logic extends, by adding more steps and shrinking the time interval, to the continuous-time model presented next.

Extending to Two Steps

A single step is enough to introduce the mechanics, but a multi-step tree whose up- and down-moves are calibrated to the underlying's volatility is what actually converges to the Black-Scholes-Merton price, as the companion notebook demonstrates numerically for tree sizes up to 5,000 steps. Extending the same stock and option to two periods, each still with $u = 1.2$, $d = 0.9$, $R = 1.04$, produces three possible terminal prices: $S_{uu} = 50(1.2)^2 = \$72$, $S_{ud} = 50(1.2)(0.9) = \$54$, and $S_{dd} = 50(0.9)^2 = \$40.50$, with call payoffs $C_{uu} = \$22$, $C_{ud} = \$4$, and $C_{dd} = \$0$. The risk-neutral probability $q = 0.467$ is unchanged, since it depends only on u, d, and R, not on the number of steps, so the tree is solved by working backward one period at a time:

$$C_u = \frac{1}{R}[q\,C_{uu} + (1-q)\,C_{ud}] = \frac{1}{1.04}[0.467(22) + 0.533(4)] \approx \$11.92, \qquad C_d = \frac{1}{R}[q\,C_{ud} + (1-q)\,C_{dd}] \approx \$1.79,$$

$$C_0 = \frac{1}{R}[q\,C_u + (1-q)\,C_d] \approx \$6.27.$$

This two-step price of \$6.27 should not be compared with the Black-Scholes-Merton price of \$6.88 computed later in this chapter, and neither should the one-step price of \$4.49: the hand tree's u and d imply a lower volatility than the 30% that the Black-Scholes-Merton example uses, so moving from one step to two with u and d held fixed shows the mechanics of backward induction, not convergence. Convergence requires each step's up- and down-moves to be calibrated to the same annualized volatility, as the next paragraph does.

The $u = 1.2$ and $d = 0.9$ used above were chosen for hand-computability, not to represent a specific step size; a tree with an arbitrary number of steps instead derives u and d from the underlying's annualized volatility using the Cox-Ross-Rubinstein parametrization $u = e^{\sigma\sqrt{\Delta t}}$, $d = 1/u$, where Δt is the length of one step, so that every tree, regardless of how many steps it has, reflects the same annualized volatility σ. Table 5.4 prices the same at-the-money call ($S_0 = K = \$50$, $r = 4\%$, $\sigma = 30\%$, $T = 1$ year) with a CRR tree of increasing step count; the price oscillates while converging, but the error shrinks steadily, from \$1.40 at one step to under \$0.001 by 5,000 steps, confirming that the Black-Scholes-Merton formula is the limit this discrete tree approaches as the step size shrinks toward zero.

Steps	Binomial price	Error vs. Black-Scholes-Merton
1	\$8.28	+\$1.40
2	\$6.21	−\$0.67
5	\$7.16	+\$0.28
20	\$6.80	−\$0.07
100	\$6.86	−\$0.01
500	\$6.874	−\$0.003
1,000	\$6.875	−\$0.001
5,000	\$6.876	−\$0.0003
∞ (Black-Scholes-Merton)	\$6.8766	—

Table 5.4: Convergence of the CRR binomial call price to the Black-Scholes-Merton price as the number of steps increases ($S_0 = K = \$50$, $r = 4\%$, $\sigma = 30\%$, $T = 1$ year)

Figure 5.2 lays out the full tree, alongside its American put counterpart introduced below, side by side: stock prices grow forward from S_0 on the left to three recombining terminal nodes on the right, while option values are computed in the opposite direction, starting from the known terminal payoffs and working backward one period at a time using q.

Wulin.Suo@Queensu.ca

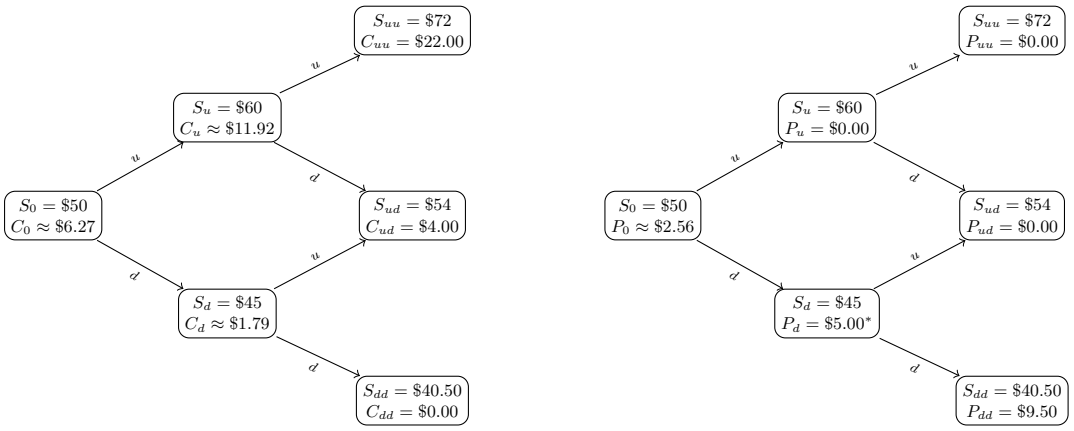
American Options and Early Exercise

Every option considered so far has been European, exercisable only at maturity. An American option can be exercised at any time up to and including maturity, and the binomial tree extends naturally to price this feature: at each node, compare the value of exercising immediately (the intrinsic value) to the value of continuing to hold the option, and take whichever is larger.

Consider an American put with the same two-period tree and strike $K = \$50$. At maturity the payoffs are unchanged: $P_{uu} = \$0$, $P_{ud} = \$0$, $P_{dd} = \max(50 - 40.50, 0) = \$9.50$. Working backward, the European continuation value at the down node ($S_d = \$45$ after one down move) is $P_d^{\text{cont}} = [q(0) + (1-q)(9.50)]/1.04 \approx \$4.87$. But immediate exercise at that node is worth $\max(50-45, 0) = \$5.00$, which exceeds the continuation value, so an optimal holder would exercise early rather than wait: $P_d^{\text{American}} = \max(5.00, 4.87) = \$5.00$. At the up node, both exercise and continuation are worth \$0 (the put is out of the money after an up move), so $P_u^{\text{American}} = \$0$. Rolling back to today, the continuation value is $[q(0) + (1-q)(5.00)]/1.04 \approx \$2.56$, and since immediate exercise today is worth $\max(50-50, 0) = \$0$, the American put is worth $P_0^{\text{American}} \approx \$2.56$.

By contrast, the European put (never allowed to exercise early) is worth only $P_0^{\text{European}} \approx \$2.50$, computed by discounting the same terminal payoffs without ever checking intrinsic value along the way. The American put's extra \$0.07 of value (the two full-precision values are \$2.5641 and \$2.4984; the gap only looks like \$0.06 if the two prices are first rounded to the nearest cent and then subtracted) is called the early-exercise premium, and it exists specifically because it was optimal to exercise early at the down node. This early-exercise feature is the reason American puts (and, when the underlying pays dividends, American calls) generally cannot be priced with a simple closed-form formula like Black-Scholes-Merton and instead require a tree, a finite-difference method, or a simulation-based approach that can check the exercise decision at every point in time.

Figure 5.2 places both trees side by side for direct comparison. The two share identical stock prices throughout, since u, d, and q never depended on which option is being priced; only the option values in the bottom line of each node differ. The asterisk at the American put's down node marks exactly where the two trees' logic diverges: every other node's value is a plain discounted expectation of its children, but $P_d = \$5.00$ replaces what would otherwise be a \$4.87 continuation value, because immediate exercise is worth more there. That single substitution is the entire early-exercise premium, and it is also why the American put's tree cannot be collapsed into a closed-form formula the way the European call's can.



European call American put

Figure 5.2: The two-step binomial tree for the \$50-strike European call (left) and American put (right). Both share the same stock-price tree, growing forward from S_0 under up- and down-moves $u = 1.2$ and $d = 0.9$ and recombining at the middle terminal node. Option values (bottom line of each node) are computed backward from the known terminal payoffs using the risk-neutral probability $q = 0.467$, except at the American put's down node, marked with an asterisk, where the \$5.00 intrinsic value of immediate exercise replaces the lower \$4.87 continuation value, producing the early-exercise premium discussed in the text.

Wulin.Suo@Queensu.ca

5.4.2 The Black-Scholes-Merton Model

The Black-Scholes-Merton model is the continuous-time limit of the binomial approach and remains the benchmark option pricing formula in both practice and further theoretical work. For a non-dividend-paying stock with current price S_0, strike K, risk-free rate r (continuously compounded), volatility σ, and time to maturity T (in years), the price of a European call is

$$C = S_0 N(d_1) - K e^{-rT} N(d_2), \qquad d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T},$$

where $N(\cdot)$ is the standard normal cumulative distribution function. Using the same at-the-money strike as the binomial example, $S_0 = K = \$50$, with $r = 4\%$, $\sigma = 30\%$, and $T = 1$ year,

$$d_1 = \frac{0 + (0.04 + 0.045)(1)}{0.30} = 0.283, \qquad d_2 = 0.283 - 0.30 = -0.017,$$

and looking up $N(0.283) = 0.6115$ and $N(-0.017) = 0.4934$ gives

$$C = 50(0.6115) - 50\, e^{-0.04}(0.4934) \approx \$6.88.$$

This is the price that the CRR tree of Table 5.4, calibrated to the same $\sigma = 30\%$, approaches as its step size shrinks. It is not directly comparable with the hand-computed one- and two-step prices of \$4.49 and \$6.27, whose $u = 1.2$ and $d = 0.9$ were chosen for arithmetic convenience and imply a lower volatility. The corresponding put price is

$$P = K e^{-rT} N(-d_2) - S_0 N(-d_1) \approx \$4.92,$$

and the two prices must satisfy put-call parity,

$$C + K e^{-rT} = P + S_0,$$

which can be checked directly: $6.88 + 50\, e^{-0.04} = 6.88 + 48.04 = \$54.92$, and $4.92 + 50 = \$54.92$, confirming consistency. Put-call parity holds regardless of any particular pricing model, since it follows from no-arbitrage alone; it is therefore a useful sanity check on any option pricing implementation, including a machine learning model trained to approximate option prices.

The quantity $N(d_1) = 0.6115$ in this example is also the option's delta, the sensitivity of the option price to a one-dollar change in the stock price, and is the first of several "Greeks" used to measure and hedge an option position's risk exposures.

5.4.3 The Greeks: Measuring an Option's Risk Exposures

A market maker who has just sold the call option priced above is exposed to several distinct sources of risk, and each of the Greeks quantifies one of them. Four of the five Greeks below share a single building block: $\phi(\cdot)$, the standard normal probability density function,

$$\phi(x) = \frac{1}{\sqrt{2\pi}}\, e^{-x^2/2},$$

the height of the standard normal bell curve at x, distinct from $N(x)$, the cumulative area under that curve already used to price the option itself. Evaluated at the same d_1 computed above, but at full precision ($d_1 = 0.28333$, not the rounded 0.283 displayed there, since rounding before this step would cascade into every Greek that uses it),

$$\phi(d_1) = \frac{1}{\sqrt{2\pi}}\, e^{-0.28333^2/2} \approx 0.3832.$$

Wulin.Suo@Queensu.ca

For a European call under Black-Scholes-Merton,

$$\Delta = N(d_1), \qquad \text{(sensitivity to the stock price)}$$

$$\Gamma = \frac{\phi(d_1)}{S_0\sigma\sqrt{T}}, \qquad \text{(sensitivity of delta itself to the stock price)}$$

$$\text{Vega} = S_0\phi(d_1)\sqrt{T}, \qquad \text{(sensitivity to volatility)}$$

$$\Theta = -\frac{S_0\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}N(d_2), \qquad \text{(sensitivity to the passage of time)}$$

$$\rho = KTe^{-rT}N(d_2). \qquad \text{(sensitivity to the interest rate)}$$

Evaluating each at the same inputs as before ($S_0 = K = \$50$, $r = 4\%$, $\sigma = 30\%$, $T = 1$) gives $\Delta = 0.6115$ (already computed), $\Gamma = \phi(d_1)/(50 \times 0.30 \times 1) \approx 0.0255$, Vega $= 50 \times \phi(d_1) \times 1 \approx 19.16$, $\Theta \approx -3.82$ per year (about $-\$0.0105$ per calendar day), and $\rho = 23.70$ (per unit, i.e., per 100 percentage points, change in the interest rate). The shared $\phi(d_1) \approx 0.3832$ term is why gamma, vega, and theta rise and fall together: whatever pushes d_1 closer to zero, the peak of the bell curve, an at-the-money option, or higher volatility relative to how far the stock sits from the strike, simultaneously raises gamma, vega, and the magnitude of theta, while delta and rho depend only on the cumulative-probability terms $N(d_1)$ and $N(d_2)$ and do not share this common driver.

These numbers have direct trading interpretations. Delta of 0.6115 means the option's price moves about \$0.61 for every \$1 move in the stock, which is why a market maker who sold this call would typically hedge by buying 0.6115 shares per option sold, a strategy called delta hedging. Gamma of 0.0255 measures how quickly that hedge ratio itself changes as the stock moves, so a market maker must rebalance the hedge more frequently when gamma is high. Vega of 19.16 (often quoted per one percentage point of volatility, so $19.16/100 \approx \$0.19$) shows the option loses or gains meaningful value purely from changes in the market's volatility expectations, entirely independent of the stock price itself, which is the channel through which the volatility clustering discussed in Chapter 7 affects option values. Theta of $-\$3.82$ per year reflects time decay: holding everything else fixed, the option loses value simply as it approaches expiration, since less time remains for the stock to move in the holder's favor.

A Worked Example: Delta Hedging and Its Limits

Suppose a market maker sells 1,000 of these call options and immediately delta-hedges by buying $1{,}000 \times 0.6115 = 611.5$ shares, funded however the market maker chooses. If the stock rises by \$1, from \$50 to \$51, the hedge's stock position gains $611.5 \times \$1 = \$611.50$. Recomputing the exact Black-Scholes-Merton price at $S = \$51$ (holding every other input fixed) gives a new call price of \$7.5008, up from \$6.8766, so the short option position loses $1{,}000 \times (7.5008 - 6.8766) = \$624.15$. The hedge does not offset this loss exactly: the net result is a loss of $611.50 - 624.15 = -\$12.65$, small relative to the \$1,000-contract position but not zero, purely because delta is only a first-order (linear) approximation of how the option's price actually responds to the stock price, the same limitation this chapter's modified-duration approximation for a bond's price-yield relationship has. Just as convexity corrected the bond-duration approximation, gamma corrects the delta approximation here: the delta-only prediction for the new option price is $C_0 + \Delta(S_1 - S_0) = 6.8766 + 0.6115(1) = \$7.4882$, while adding the gamma correction, $C_0 + \Delta(S_1 - S_0) + \frac{1}{2}\Gamma(S_1 - S_0)^2 = 7.4882 + 0.5(0.0255)(1)^2 \approx \$7.5009$, lands almost exactly on the true recomputed price of \$7.5008.

A market maker who rebalances a delta hedge only occasionally, rather than continuously, is therefore always exposed to some residual gamma risk between rebalances, larger for bigger stock moves (since the gamma correction term grows with the square of the price change) and larger for options with higher gamma, the at-the-money, short-time-to-expiration options this section already flagged as having the largest gamma. This is precisely why professional options market makers monitor and actively manage gamma, not just delta, and why a large, sudden price move is disproportionately costly to a delta-hedged options book relative to a series of small moves of the same total size.

Wulin.Suo@Queensu.ca

5.4.4 Implied Volatility

The Black-Scholes-Merton formula takes σ as an input and produces a price as output. In practice, market participants often run this relationship in reverse: given an observed market price for an option, they solve for the volatility that the Black-Scholes-Merton formula would need in order to reproduce that price, called the implied volatility. Because the formula is not easily invertible algebraically, implied volatility is typically found numerically, using the same kind of root-finding procedure introduced for bond yield to maturity in Section 5.2.2.

Suppose the same call option (the one priced at \$6.88 under an assumed volatility of 30%) is instead observed trading in the market at \$7.50. The implied volatility $\hat{\sigma}$ solves $C_{BS}(\hat{\sigma}) = 7.50$, which a numerical solver finds to be $\hat{\sigma} \approx 33.25\%$, noticeably higher than the 30% originally assumed. This gap is informative: it means the market is pricing in more uncertainty about the stock's future price than the 30% historical-volatility assumption used earlier in this chapter would suggest, whether because of an upcoming earnings announcement, macroeconomic uncertainty, or simply a different market view. Because implied volatility is forward-looking, reflecting the market's current expectation rather than a backward-looking historical average, it is often treated as a more useful volatility estimate for pricing other, related options than historical volatility is.

If the Black-Scholes-Merton model's constant-volatility assumption held exactly, every option on the same underlying and the same maturity, regardless of strike, would imply the same $\hat{\sigma}$: a call at $K = \$40$ and a call at $K = \$60$ would both back out the same number, because the model treats σ as a property of the underlying stock, not of the individual option contract. In practice this is not what happens. Plotting implied volatility against strike, for options on the same underlying and the same maturity, typically produces a curve, not a flat line, and market practitioners distinguish two common shapes. A **volatility smile** is roughly symmetric: implied volatility is lowest for at-the-money options and rises for both deep in-the-money and deep out-of-the-money strikes, a pattern most associated with currency options, where a large move in either direction, a currency crisis or a currency surge, is plausible. A **volatility skew**, the shape actually observed for equity and equity-index options, is instead mostly downward sloping: implied volatility is highest for low strikes (deep in-the-money calls, equivalently far out-of-the-money puts) and falls as the strike rises, so out-of-the-money puts are systematically priced as if the stock were more volatile than an at-the-money or out-of-the-money call would suggest.

Two economic explanations are usually offered for the equity skew specifically, and both remain active areas of empirical research rather than settled fact: the *leverage effect* (Black, 1976), whereby a falling stock price mechanically raises a levered firm's debt-to-equity ratio and therefore its equity volatility, so low-strike, low-stock-price states are genuinely more volatile ones; and *crash risk*, sometimes called "crashophobia" (Rubinstein, 1994), the observation that the pronounced equity skew now taken for granted was far less visible in listed U.S. index option prices before the October 1987 crash, and that persistent demand for downside protection, buying out-of-the-money puts as portfolio insurance, has kept those puts priced at a premium (equivalently, a higher implied volatility) ever since.

A Worked Example: Computing an Implied Volatility Skew

The single implied-volatility computation above generalizes directly: solving $C_{BS}(\hat{\sigma}) =$ market price separately at each strike, holding $S_0 = \$50$, $r = 4\%$, and $T = 1$ year fixed throughout, traces out the skew itself rather than only describing it in the abstract. Table 5.5 does this for five hypothetical market prices spanning strikes of \$40 to \$60, including the \$50 at-the-money option already solved above.

The pattern in Table 5.5 is the equity skew described above, not a symmetric smile: implied volatility falls by roughly 16 percentage points, from 40.02% down to 24.00%, as the strike rises from \$40 to \$60, with the single largest drop, more than six points, occurring between the \$50 and \$55 strikes. A trader reading this table would conclude that the market is pricing meaningfully more downside risk into this stock than a single, constant-volatility Black-Scholes-Merton price would imply: the far-out-of-the-money \$60 call is priced as if volatility were only 24%, while the deep-in-the-money \$40 call, equivalent by put-call parity to a far-out-of-the-money put, is priced as if volatility were 40%, sixteen points higher.

Table 5.5 fixes maturity at one year and varies only the strike, a single one-dimensional slice through what is, in general, a genuinely two-dimensional object. Repeating the same exercise with $T = 0.25$ (three

Wulin.Suo@Queensu.ca

Strike (K)	Market price	d (moneyness, $S_0 - K$)	Implied volatility
\$40	\$14.23	+\$10 (in-the-money)	40.02%
\$45	\$10.57	+\$5	36.00%
\$50	\$7.50	\$0 (at-the-money)	33.25%
\$55	\$4.20	−\$5	26.92%
\$60	\$2.19	−\$10 (out-of-the-money)	24.00%

Table 5.5: Implied volatilities solved from five one-year call option market prices on the same underlying ($S_0 = \$50$, $r = 4\%$)

months) instead of one year, holding S_0, r, and the same five strikes fixed, produces a second slice:

$$K = \$40 : 47.03\%, \qquad K = \$45 : 40.04\%, \qquad K = \$50 : 33.55\%,$$
$$K = \$55 : 27.04\%, \qquad K = \$60 : 24.48\%.$$

Figure 5.3 plots both slices together. Two things stand out. First, both curves slope downward, confirming the skew shape is not specific to a single maturity. Second, and more informative, the two curves are not simply parallel: the three-month curve sits noticeably above the one-year curve at low strikes (47.03% versus 40.02% at $K = \$40$, a gap of nearly seven points) but the two curves nearly converge at high strikes (24.48% versus 24.00% at $K = \$60$, under half a point apart). The three-month skew is therefore steeper than the one-year skew, a well-documented empirical pattern: near-term crash protection is priced at a particularly large premium, since a crash is by definition a short, sudden event, while longer-dated options have more time for volatility to mean-revert toward its long-run average, flattening the skew somewhat. This joint variation across both strike and maturity, not strike alone, is precisely what the **volatility surface** refers to: a full grid of implied volatilities, one for every combination of strike and maturity actively traded, of which Table 5.5 and its three-month counterpart are two single-maturity cross-sections.

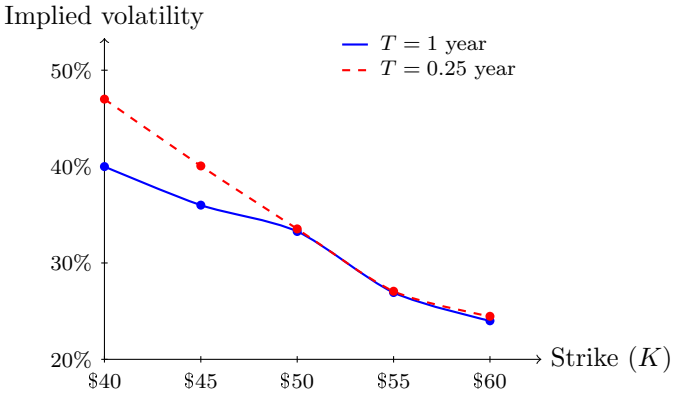


Figure 5.3: The implied volatility skew for one-year (solid) and three-month (dashed) call options on the same underlying ($S_0 = \$50$, $r = 4\%$), from Table 5.5 and its three-month counterpart

Why the Volatility Smile Matters for AI and Machine Learning

The skew and surface just computed are not just a curiosity for option traders; they are one of the most important places where modern finance meets machine learning. A simple Black-Scholes-Merton model assigns one volatility to every option on the same underlying, but the market does not behave that way, as Table 5.5 and Figure 5.3 showed directly. Options with different strikes and maturities imply systematically different volatilities, differences structured enough to form a rich pattern that can be learned from data. That is why researchers and practitioners use machine learning to model implied-volatility surfaces, to estimate

Wulin.Suo@Queensu.ca

the residual error between theoretical prices and observed market prices, and to build fast pricing or hedging tools for large option books.

The opportunity is attractive because the data are high-dimensional and highly structured: strike, maturity, underlying price, interest rates, and recent market regimes all matter. A machine learning model can learn which combinations of these features predict unusually high or low implied volatility, but it must still respect economic constraints. In particular, the fitted surface should remain economically sensible, avoid producing arbitrage opportunities, and be stable out of sample rather than simply memorizing recent market moves. For that reason, AI and machine learning are best viewed here as tools for capturing patterns in the volatility surface, not as replacements for the no-arbitrage logic that makes option pricing coherent in the first place.

A single option's implied volatility is a useful estimate for that contract, but it can be distorted by events affecting one company, and no single option summarizes the market's overall volatility expectations. The CBOE Volatility Index (VIX), often called the market's "fear gauge," aggregates implied volatilities across a wide range of strikes and maturities on S&P 500 index options into one number, using a model-free methodology (following Britten-Jones and Neuberger, 2000) that never inverts the Black-Scholes-Merton formula for any single option: the volatility-surface aggregation just described, condensed into one tradable index. Quoted as an annualized percentage covering the next 30 days, VIX typically trades between 12% and 20% in calm markets. Because VIX futures, options, and exchange-traded products make it directly tradable, it recurs throughout this book as a market-wide risk signal distinct from any single asset's own volatility: as a regime filter for trading strategies in Chapter 13, and as a price-based complement to the text-based sentiment measures of Chapter 16.

Figure 5.4 shows VIX's daily closing history since its 1990 inception, downloaded from Yahoo Finance, with its largest spikes labeled: the 1998 collapse of Long-Term Capital Management amid the Russian sovereign default (Chapter 3; 45.74), the September 2001 terrorist attacks (43.74), the mid-2002 WorldCom bankruptcy and accounting-scandal selloff that closed out the dot-com bear market (45.08), the 2008 global financial crisis (80.86), the May 2010 Flash Crash (Chapter 3's own case study; 40.95), the August 2011 U.S. sovereign credit downgrade (48.00), the August 2015 selloff triggered by a surprise Chinese currency devaluation (40.74), the February 2018 "Volmageddon" collapse of short-volatility exchange-traded products (37.32), the March 2020 COVID-19 selloff (the all-time closing high of 82.69), the August 2024 yen carry-trade unwind, when a surprise Bank of Japan rate hike met a weak U.S. jobs report (65.73 intraday, 38.57 at the close), and the April 2025 tariff war (52.33, the third-highest peak of any VIX episode on record, behind only 2008 and 2020, although dozens of individual daily closes during those two crises were higher). Every spike shares the same shape: VIX drifts in a narrow band, then jumps to 40%-80% or more within days when a shock forces an abrupt repricing of risk, then subsides once the acute phase passes, the volatility clustering and mean reversion that Chapter 7's GARCH(1,1) recursion models directly.

Not every geopolitical shock qualifies: the June 2025 Israel-Iran conflict and the accompanying U.S. strikes on Iranian nuclear facilities left VIX only in the low twenties, a reminder that VIX prices *equity* volatility specifically, so a shock must threaten broad equity cash flows or risk appetite, not merely be geopolitically significant, to move it sharply.

That spike-and-decay pattern is what makes VIX a forecasting input rather than a description of the present, and the mean reversion after every labeled spike is part of why: an unusually high VIX is informative precisely because it is unlikely to persist. Being backed out of option prices, VIX is the market's own forecast of the coming thirty days, and the standard empirical finding is that implied volatility carries information about future realized volatility beyond what past returns alone provide, which is why the volatility models of Chapter 7 and the risk models of Chapter 8 often carry an implied-volatility term alongside their GARCH-style recursions.

Two cautions come with that. VIX forecasts volatility, not direction: it says the market expects large moves, not which way they will go, so it belongs in a risk or position-sizing model rather than a return forecast. And it is a biased forecast of realized volatility, sitting above it on average because index options embed compensation for bearing crash risk, the same downside-protection demand that produces the downward-sloping skew documented earlier in this section. A model that treats VIX as unbiased will systematically over-forecast volatility, and the gap between the two, the volatility risk premium, is itself a traded quantity rather than a modeling error to be removed.

Wulin.Suo@Queensu.ca

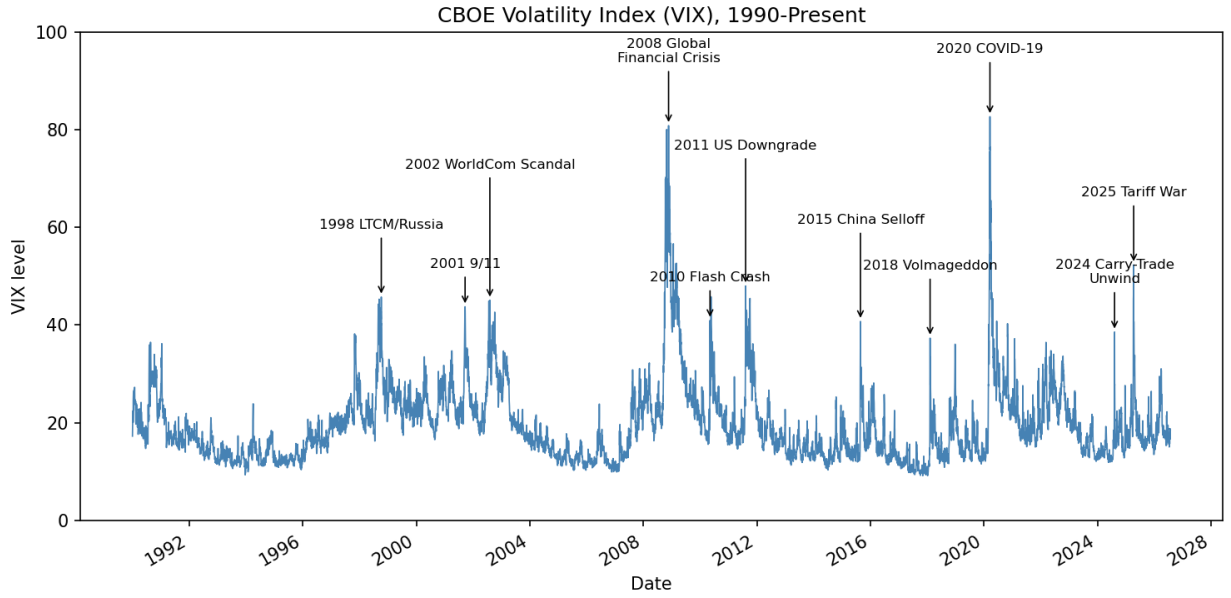


Figure 5.4: The CBOE Volatility Index (VIX), daily closes from January 1990 through the most recent available date, downloaded via the `yfinance` library. Labeled spikes correspond to the crises discussed in the text.

5.4.5 Real Options: Applying Option Pricing to Corporate Decisions

The option pricing machinery developed in this chapter for financial derivatives applies, with some adaptation, to certain corporate investment decisions as well, an application known as real options analysis. The insight is that many corporate decisions have an option-like structure: a firm has the right, but not the obligation, to take some action, and can choose whether and when to exercise that right based on how conditions evolve.

Consider a mining company deciding whether to develop a mineral deposit. A standard net-present-value calculation would estimate a single expected value for the project and approve it only if that value is positive. But the firm actually holds something closer to a call option: it can wait, observe how commodity prices evolve, and develop the deposit only if prices rise enough to make it profitable, walking away (at the cost of whatever it spent on exploration and permitting) if prices fall instead. This option to wait has value, exactly as the extra flexibility embedded in an American option (Section 5.4.1) has value beyond its European counterpart, and a standard net-present-value analysis that ignores this flexibility can systematically undervalue the opportunity.

Other common examples include the option to expand a successful pilot project, the option to abandon a failing project and salvage its assets, and the option to switch between input sources or output products as relative prices change. In each case, the underlying "asset" is the value of the future cash flows the decision affects, the "strike price" is the cost of taking the action, and volatility in the underlying business or commodity price plays the same role it plays in the Black-Scholes-Merton formula: more uncertainty makes the option to wait, expand, or abandon more valuable, not less, since flexibility is most useful precisely when the future is most uncertain. This is a genuinely counterintuitive result relative to naive intuition (which often treats uncertainty as purely a negative), and it is one of the most important practical lessons real options analysis contributes to corporate finance. Real options are typically harder to value precisely than financial options, since the "underlying asset" rarely trades in a liquid market with an observable volatility, but the qualitative logic, that flexibility has value and that value increases with uncertainty, remains a useful lens for evaluating investment decisions even when a precise Black-Scholes-Merton-style number is not available.

A simplified numerical illustration makes the option-to-wait logic concrete, even without a fully specified

Wulin.Suo@Queensu.ca

binomial tree of traded prices. Suppose the mining company's deposit costs $K = \$50$ million to develop, and, if developed today, is worth exactly \$50 million, a marginal project with NPV $= 50.0 - 50.0 = \$0$ that a naive analysis would treat as a coin flip, worth doing but not clearly attractive. Suppose that if the firm waits one year, the deposit's value will have moved to either \$70 million (if commodity prices rise) or \$35 million (if they fall), each with probability 0.5, and the firm develops the deposit only if doing so is still profitable at that point. The expected payoff from waiting is

$$E[\text{Payoff}] = 0.5 \times \max(70.0 - 50.0,\ 0) + 0.5 \times \max(35.0 - 50.0,\ 0) = 0.5 \times 20.0 + 0.5 \times 0 = \$10.0 \text{ million},$$

which, discounted one year at a risk-free rate of 5%, has a present value of $10.0/1.05 \approx \$9.52$ million, substantially higher than the \$0 NPV of developing immediately. The probability 0.5 is not an arbitrary guess, and discounting at the risk-free rate is legitimate only because of it: it is exactly the risk-neutral probability of Section 5.4.1 implied by the deposit's current value, $q = (1.05 \times 50 - 35)/(70 - 35) = 0.5$, the value of q that makes today's \$50 million equal the discounted risk-neutral expectation of the two end-of-year outcomes. A real-world probability of 0.5 would instead call for a risk-adjusted discount rate, and changing either end-of-year value while holding today's value fixed changes q as well (Exercise 17). Figure 5.5 lays out this comparison as a one-step decision tree: developing immediately is a single certain outcome, while waiting branches into the two possible commodity-price states, each carrying its own payoff.

The value of waiting comes entirely from the asymmetry built into the option to walk away: the firm captures the full upside if prices rise but avoids the downside entirely if they fall, an asymmetry a simple point-estimate NPV calculation, which implicitly commits to developing the project regardless of how prices move, cannot capture at all. This is why a corporate finance team that evaluates a marginal project using only a single expected NPV, without asking whether waiting for more information is itself a valuable and available choice, can systematically pass over projects, or approve them prematurely, in ways that ignore real, quantifiable option value. Chapter 19 revisits the closely related American-option early-exercise decision from Section 5.4.1 using a model-free reinforcement-learning algorithm, Q-learning, and shows that it recovers the exact same optimal exercise policy without ever being told the tree's transition probabilities or payoff structure in advance, an instructive machine-learning counterpart to the closed-form, model-based reasoning used throughout this section.

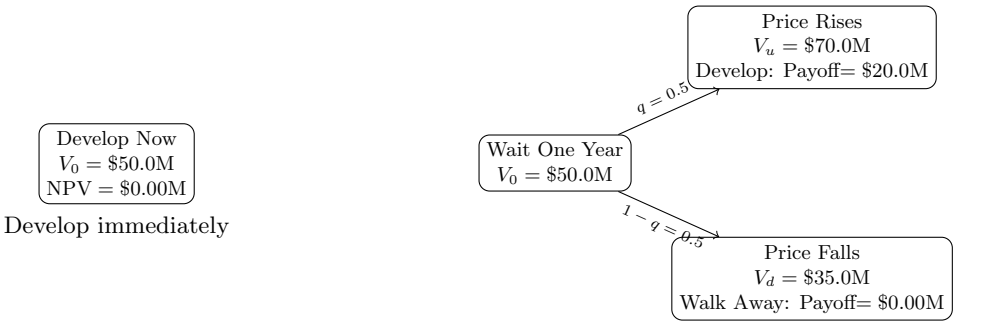


Figure 5.5: A one-step real-options decision tree for the mining deposit example: developing immediately (left) versus waiting one year to observe the commodity-price state (right)

5.5 Putting It Together: Valuing Meridian Fabrication

This chapter has now developed both major approaches to valuing a business, discounted cash flow and relative valuation multiples, and it is worth applying both to Meridian Fabrication side by side, using the WACC computed earlier in this chapter, to see how differently they can answer the same question.

For the DCF approach, the valuation date is the end of Year 2, the date of Meridian's latest balance sheet, so Year 3's free cash flow arrives one period from now and Year 5's three periods from now (Figure 5.6). Chapter 4's percent-of-sales forecast supplies the first of these: free cash flow recovers from −\$8.0 million

Wulin.Suo@Queensu.ca

in Year 2 to \$0.52 million in Year 3, as working capital absorbs less cash. Beyond that one-year forecast, suppose the recovery continues as Meridian's working-capital investment matures, to \$3.0 million in Year 4 and \$6.0 million in Year 5, after which free cash flow grows at a stable long-run rate of $g = 3\%$ forever. Discounting at the full-precision WACC of 8.079% computed earlier in this chapter (before rounding to 8.08% for display),

$$PV(\text{Years 3-5 FCF}) = \frac{0.52}{1.08079} + \frac{3.0}{1.08079^2} + \frac{6.0}{1.08079^3} \approx \$7.80 \text{ million},$$

and the terminal value at the end of Year 5, using the Gordon growth formula on Year 6's free cash flow, is

$$TV_5 = \frac{6.0(1.03)}{0.08079 - 0.03} \approx \$121.69 \text{ million}, \qquad PV(TV_5) = \frac{121.69}{1.08079^3} \approx \$96.39 \text{ million},$$

discounted three periods, not five, because TV_5 sits at the end of Year 5, exactly as far from the valuation date as Year 5's own free cash flow. The DCF enterprise value is $7.80 + 96.39 \approx \$104.19$ million, and subtracting net debt of $\$58.0 - \$9.0 = \$49.0$ million gives an implied equity value of approximately \$55.19 million.

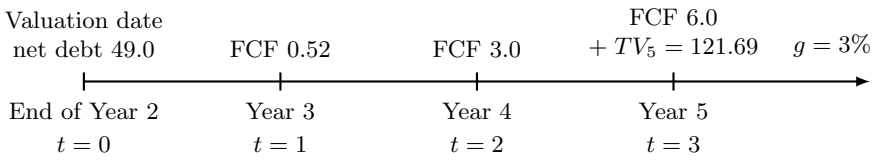


Figure 5.6: Cash-flow timeline for the Meridian DCF (\$ millions). The valuation date, $t = 0$, is the end of Year 2, so Year 3's free cash flow is discounted one period, and Year 5's free cash flow and the terminal value TV_5 three periods.

Compare this to the multiples-based equity value computed in Section 5.3.3: \$107.0 million, nearly twice as large. Both are legitimate valuation methods applied to the same company; the large gap between them reflects a real economic disagreement, not a computational error. The DCF value is driven by Meridian's own, currently weak, cash generation and its slow projected recovery, while the multiples value assumes Meridian deserves to trade at the same EBITDA multiple as healthier comparable firms, effectively assuming its cash flow problems are temporary and will not persist.

The two methods are not even fully independent. The WACC used book equity of \$54.0 million as E, which the formula permits only when no market value is available; substituting the multiples-implied \$107.0 million would raise the equity weight from 48.2% to 64.8% and the WACC from 8.08% to 9.08%, which would lower the DCF value further and widen the gap. The choice of capital-structure weights is therefore itself a judgment about which of the two values is closer to the truth.

Interestingly, the DCF-implied equity value of \$55.19 million is close to Meridian's \$54.0 million book value of equity, while the multiples-based value is far above it. That proximity should not be over-read, since book value is a historical-cost figure and nothing forces a DCF to land near it, but it is consistent with the DCF assumptions being the more conservative starting point until there is concrete evidence, such as an improving cash conversion cycle or margin expansion, that Meridian's operating performance is converging toward that of its healthier peers.

This kind of triangulation across methods, and an honest accounting of why they disagree, is standard practice in real valuation work and is a more useful output than any single number produced by either method alone. In practice, the analyst would not stop at the two values; the next step would be to test how much each estimate changes under alternative market multiples, discount rates, and cash-flow scenarios. That sensitivity analysis is often more informative than the point estimate itself.

Wulin.Suo@Queensu.ca

5.6 Challenges in Valuation and Asset Pricing

The models in this chapter are elegant and, within their assumptions, exact. Applying them to real markets introduces several recurring challenges.

Discount-rate uncertainty. Every model in this chapter takes r as given, but in practice the discount rate itself must be estimated, typically from an asset pricing model such as the CAPM or a multi-factor model (covered in Chapter 12), and estimates of the required return can vary substantially across reasonable methodologies. Since the Gordon growth model shows that price is extremely sensitive to $r - g$, uncertainty in r translates directly into wide uncertainty in fair value.

Volatility is not observed, only estimated. The Black-Scholes-Merton formula requires σ, the volatility of the underlying asset, but σ cannot be observed directly; it must be estimated from historical returns or backed out from other option prices as an "implied volatility." Different estimation windows and methods can produce meaningfully different prices for the same option, and volatility itself changes over time, which is precisely the motivation for the GARCH-family models introduced in Chapter 7.

Model risk and the limits of closed-form pricing. Every closed-form pricing model rests on simplifying assumptions, constant volatility and continuous trading for Black-Scholes-Merton, a flat and unchanging growth rate for the Gordon growth model, no default risk in the basic bond pricing formula. Real markets violate all of these to varying degrees, and a model that is used outside the range of conditions it was built for can silently produce a confidently wrong price, a risk usually called model risk.

Credit risk complicates bond pricing. The bond pricing formula in this chapter assumes the promised cash flows are paid with certainty. A corporate bond carries default risk, so its price reflects both discounting and the probability-weighted possibility of missed payments, which requires the credit models developed in Chapter 9.

Calibration versus estimation at scale. A trading desk must reprice large portfolios of derivatives continuously as market conditions change, which requires the underlying models to be calibrated quickly and consistently across thousands of instruments. This operational requirement, not just theoretical accuracy, is a major reason simpler closed-form models such as Black-Scholes-Merton remain in wide use even though more elaborate stochastic volatility and jump-diffusion models are known to fit historical data better.

Where machine learning fits in. Because closed-form models are known to deviate from observed market prices in systematic ways (the volatility smile is a direct symptom), researchers and practitioners have used machine learning both to learn the deviation directly (predicting the gap between a model price and the observed market price) and to learn pricing functions or implied volatility surfaces end to end from data. These approaches can capture patterns a fixed formula cannot, but they inherit the same risk-management burden as any other statistical model, including the overfitting and non-stationarity concerns raised in Chapter 6: a pricing model trained on historical option data can fail exactly when market conditions shift away from the training regime, which is usually when accurate pricing matters most.

The term structure adds a further layer of estimation risk. Section 5.2.4's yield curve discussion showed that a single discount rate is itself a simplification; a full analysis requires an entire curve of spot rates, and that curve must itself be estimated (typically by "bootstrapping" it from a set of observed bond prices), which introduces additional model choices and potential errors before any single bond or derivative is even priced.

Real options are hard to value precisely. The real options perspective introduced in Section 5.4.5 is conceptually powerful but practically difficult to apply with the same precision as a financial option, since the underlying asset (the value of a future business opportunity) rarely has an observable market price or a clean, constant volatility estimate, which means real options analysis is often used qualitatively, to check whether a standard net-present-value calculation is missing significant flexibility value, rather than to produce a single definitive price.

Relative valuation inherits the market's own mistakes. The multiples-based approach in Section 5.3.3 depends entirely on comparable companies being reasonably priced; if an entire sector is experiencing a speculative bubble or an unwarranted sell-off, a multiples-based valuation will simply reproduce that mispricing for the target firm rather than correct for it, since the method is explicitly designed to price an asset relative to its peers, not relative to a first-principles estimate of intrinsic value. This is precisely why the DCF-versus-multiples comparison in Section 5.5 is best treated as informative even, and perhaps especially, when the two methods disagree substantially.

Wulin.Suo@Queensu.ca

Illiquidity complicates every valuation approach. All of the methods in this chapter implicitly assume that once a fair value is computed, an investor could transact at or near that value. For a private company, a thinly traded bond, or a large block of stock relative to average trading volume, this assumption can fail badly: the price actually realizable in a sale may differ meaningfully from any of the theoretical values computed in this chapter, simply because there are too few willing counterparties, echoing the liquidity and market-impact themes of Chapter 3.

5.7 Key Takeaways

Every valuation problem in this chapter reduces to the same idea: discount the relevant future cash flows, or replicate the relevant future payoff, at a rate or under a probability measure consistent with no arbitrage. Bond pricing, the dividend discount model, and option pricing look like different subjects, but they are the same present-value logic applied to progressively more complex cash flow structures, from a fixed schedule of coupons, to a growing perpetuity of dividends, to a payoff that depends nonlinearly on another asset's future price. The chapter's worked examples showed both how to compute these prices and where the underlying models are known to break down, since a practitioner who can only compute a textbook price without understanding its assumptions is exposed to the kind of model risk described above.

The chapter also connected valuation directly to the statements and institutions covered earlier in the book. The WACC example in Section 5.3.1 discounted Meridian Fabrication's own capital structure from Chapter 4, showing exactly how a firm's balance sheet feeds into the rate used to value it. The credit-risk-adjusted bond price in Section 5.2.2 previewed the probability-of-default and recovery-rate concepts that Chapter 9's credit models estimate statistically rather than assume. And the Greeks and implied volatility material showed that even a formula as elegant as Black-Scholes-Merton is best understood as a lens for organizing risk exposures and market expectations, rather than as a black box that simply outputs a single correct price. Carrying this perspective, elegant models as organizing frameworks, not oracles, forward is the mindset the rest of this book tries to instill.

The closing Meridian valuation in Section 5.5 is perhaps the single most important exercise in the chapter to internalize, precisely because it did not produce a single answer. A DCF value of \$55.19 million and a multiples-based value of \$107.0 million are both defensible, both computed correctly from the formulas in this chapter, and yet they disagree by nearly a factor of two.

Two further points follow from this. First, the disagreement itself is informative: it isolates exactly which assumption, Meridian's own projected cash-flow recovery versus the market's willingness to price it like a healthier peer, is driving the difference, which is a far more useful diagnostic than either number alone. Second, this is the situation a machine learning model trained to predict "fair value" from historical data would face silently and without comment: absent an explicit decomposition like the one performed here, a model blending signals from both approaches could produce a single confident-looking number while obscuring a genuine, economically meaningful disagreement about which valuation philosophy is more appropriate for this specific company at this specific time. Preserving, rather than collapsing, this kind of interpretability is a theme this book returns to directly in Chapter 18.

5.8 Suggested Exercises

The exercises below draw on every major tool developed in this chapter, bond pricing and duration, the dividend discount model, binomial and Black-Scholes-Merton option pricing, the cost of capital, and relative valuation, and several build directly on the Meridian Fabrication and Solstice Robotics examples carried over from Chapter 4, so that the numbers you compute here can be checked against, and interpreted alongside, that earlier analysis.

1. A 2-year bond with face value \$1,000 and an annual coupon rate of 6% is priced at a market yield of 8%. Compute its price using the discounting formula from Section 5.2.2.

2. Using the Gordon growth model, find the fair price of a stock expected to pay a dividend of \$2.50 next year, assuming a required return of 10% and a long-run dividend growth rate of 6%. Then

Wulin.Suo@Queensu.ca

recompute the price assuming growth of 8% instead, and comment on how sensitive the price is to this two-percentage-point change.

3. Using the one-step binomial model with $S_0 = \$40$, $u = 1.25$, $d = 0.8$, $R = 1.05$, and strike $K = \$40$, compute the risk-neutral probability q and the fair price of a call option today.

4. Verify put-call parity using the Black-Scholes-Merton call and put prices computed in Section 5.4.2, and explain in your own words why put-call parity must hold regardless of which option pricing model is used.

5. Using the two-stage dividend discount model from Section 5.3.2, compute the fair price of a stock with $D_0 = \$2.00$, a high-growth rate of $g_1 = 15\%$ for the first 4 years, a terminal growth rate of $g_2 = 5\%$, and a required return of $r = 11\%$. What fraction of the total price comes from the terminal value?

6. A 3-year bond with a Macaulay duration of 2.86 years is priced at \$947.51 when $y = 7\%$. Use the modified-duration approximation to estimate its price if the yield falls to 6%, then explain, using the concept of convexity, whether you would expect the approximation to overstate or understate the exact repriced value.

7. Explain why the Gordon growth model is more sensitive to estimation error than the bond pricing formula in this chapter, referencing the discount-rate uncertainty challenge in Section 5.6.

8. Describe one concrete way a Black-Scholes-Merton-based pricing system could fail silently (produce a confident but wrong price) if market conditions moved outside the model's assumptions.

9. Compute Meridian's WACC using Section 5.3.1's formulas if its equity beta were 0.9 instead of 1.2, holding all other inputs (risk-free rate, market risk premium, cost of debt, tax rate, and capital structure) fixed, and explain the direction of the change.

10. Using the Greek formulas in Section 5.4.3, explain why gamma is largest for an option that is close to at-the-money ($S_0 \approx K$) and smaller for options that are deep in- or out-of-the-money, without necessarily recomputing the formula numerically.

11. A sixth call option on the same stock (still $S_0 = \$50$, $r = 4\%$, $T = 1$), with a strike of $K = \$65$, trades in the market at \$1.00. Using the same numerical root-finding approach used to build Table 5.5, solve for its implied volatility, and explain whether the result is consistent with the skew pattern already observed across the \$40–\$60 strikes in Section 5.4.4.

12. Using the portfolio duration formula in Section 5.2.3, compute the duration of a bond portfolio holding 30% in a bond with duration 1.5 years, 50% in a bond with duration 4.0 years, and 20% in a bond with duration 8.0 years.

13. Using Table 5.2, explain in your own words why a corporate bond's yield should be even higher than the government yield at the same maturity, referencing the credit-risk-adjustment concept from Section 5.2.2.

14. Using the multiples method in Section 5.3.3 and Solstice Robotics' Chapter 4 figures (EBIT of \$27.0 million, depreciation and amortization of \$5.0 million, total debt of \$10.0 million, and cash of \$40.0 million), compute its implied enterprise and equity value assuming comparable technology firms trade at an EV/EBITDA multiple of 12×. Compare the implied equity value to its book equity of \$75.0 million.

15. Explain, using the DCF-versus-multiples comparison in Section 5.5, one reason a DCF valuation and a multiples-based valuation of the same company might disagree substantially, and describe what an analyst should do when they do.

16. Describe a real option (using the framework in Section 5.4.5) that a pharmaceutical company holds when deciding whether to continue funding a drug at each stage of clinical trials, and explain why higher uncertainty about the drug's eventual efficacy makes this option more, not less, valuable.

Wulin.Suo@Queensu.ca

17. Using Section 5.4.5's option-to-wait example, recompute the value of waiting one year if the deposit's up-state value were \$90 million instead of \$70 million (down-state value, development cost, today's \$50 million value, and the risk-free rate unchanged). Do it twice: first holding the probability at 0.5, then using the risk-neutral probability q implied by the new up-state value. Show that the first calculation raises the option's present value by less than half of the \$20 million increase in the up-state value, and explain why only the second calculation is consistent with no arbitrage.

18. Using Section 5.5's Meridian DCF as a template, recompute the enterprise and equity value if the terminal growth rate were $g = 2\%$ instead of 3%, holding the WACC and the Year 3-5 free cash flow forecasts fixed, and explain why the terminal growth assumption has such a large effect on the total valuation.

19. A junior analyst argues that because the DCF and multiples valuations for Meridian disagree so much, one of the two methods must simply be "wrong" and should be discarded. Write a short paragraph, drawing on Section 5.5, explaining why this conclusion does not necessarily follow.

20. Using Table 5.2, compute the implied 1-year forward rate spanning years 4 to 5 using the 5-year and (assumed) 4-year spot rates of 4.8% and 4.7% respectively, and compare it to the two forward rates already computed in the text.

21. Using this chapter's delta-hedging example, recompute the hedge's net P&L if the stock instead falls from \$50 to \$49 (holding the number of shares hedged and all other option inputs fixed), and explain why the sign of the residual hedging error is the same as in the original up-move example even though the stock moved in the opposite direction.

5.9 Further Reading

- Hull, *Options, Futures, and Other Derivatives*. The standard reference for derivatives pricing, covering the binomial model, Black-Scholes-Merton, and the Greeks in far greater depth and generality than this chapter.
- McDonald, *Derivatives Markets*. A complementary treatment of options and forwards with a strong emphasis on the no-arbitrage replication arguments introduced in this chapter.
- Damodaran, *Investment Valuation*. An extensive treatment of both discounted cash flow and relative valuation, including practical guidance on estimating discount rates, growth rates, and comparable-company multiples.
- Bodie, Kane, and Marcus, *Investments*. Covers the CAPM and cost-of-capital material in this chapter within the broader context of portfolio theory and asset pricing, previewing Chapter 12.
- Fabozzi, *Fixed Income Analysis*. A thorough treatment of bond pricing, duration, convexity, and the term structure of interest rates for readers who want to go well beyond this chapter's introductory treatment.
- Brealey, Myers, and Allen, *Principles of Corporate Finance*. Covers real options and the cost-of-capital material from a corporate finance perspective, complementing the more market-oriented treatment in this chapter.
- Black, F., "Studies of Stock Price Volatility Changes," *Proceedings of the 1976 Meetings of the Business and Economics Statistics Section, American Statistical Association*. The original leverage-effect explanation for the equity volatility skew discussed in this chapter's implied-volatility section.
- Rubinstein, M., "Implied Binomial Trees," *Journal of Finance*. The "crashophobia" explanation for the post-1987 equity skew discussed in this chapter's implied-volatility section.
- Britten-Jones, M., and A. Neuberger, "Option Prices, Implied Price Processes, and Stochastic Volatility," *Journal of Finance*. The model-free methodology underlying the VIX index discussed in this chapter's volatility-surface section.

Wulin.Suo@Queensu.ca

Wulin.Suo@Queensu.ca

Chapter 6

Machine Learning Fundamentals

6.1 Introduction

Machine learning has become a central part of the quantitative toolkit in finance. It provides methods for discovering patterns in data, forecasting outcomes, reducing dimensionality, and classifying observations. In principle, its function is simple: use historical data to learn relationships that can generalize to new situations. In practice, this task is not trivial, especially in finance, where data are noisy, relationships shift over time, and the cost of mistakes can be large.

This chapter introduces the main ideas behind machine learning in a way that is accessible to readers who are mathematically trained but may be new to the subject. The focus is on understanding the core concepts, together with the underlying formulas, rather than on memorizing library calls. Two small worked examples run through the chapter: a regression problem relating a firm's leverage to its credit spread, and a classification problem predicting loan default. The discussion covers supervised and unsupervised learning, prediction and classification, overfitting, regularization, evaluation strategies, and the bias-variance tradeoff.

By the end of the chapter, the reader will have seen how a model's parameters are actually found (via a closed-form solution or gradient descent), how several qualitatively different algorithms, distance-based, margin-based, probabilistic, and tree-based, all address the same underlying prediction problem from different angles, and how to select among them and their hyperparameters without silently overfitting the selection process itself. These foundations are used directly and repeatedly in every applied chapter that follows: the credit models of Chapter 9, the portfolio methods of Chapter 12, the trading strategies of Chapters 13 and 14, the fraud detection systems of Chapter 15, the text-based methods of Chapter 16, the deep-learning extensions of Chapter 17, the explainability tools of Chapter 18, and the end-to-end case study of Chapter 21 all build on the vocabulary and techniques introduced here, rather than re-deriving them from scratch.

6.2 What Machine Learning Is

Two questions come before any algorithm: what a learning system actually does, and what kind of problem is being posed to it.

6.2.1 What Machine Learning Is

Suppose an analyst has a decade of loan-application data and wants a system that predicts default risk more accurately than a fixed set of hand-written underwriting rules, one that gets better as more data arrives rather than needing to be manually rewritten every time economic conditions shift. Machine learning is the branch of statistics and computer science built to do this: building models that improve with experience, rather than encoding a fixed decision procedure by hand. The basic setup is straightforward: a model is given data, it learns parameters from those data by minimizing some measure of error, and it is then used to make predictions or decisions on new data. The model may be used for regression, classification, clustering, dimensionality reduction, or anomaly detection.

Wulin.Suo@Queensu.ca

This basic setup is, in an important sense, not new. Fitting a linear regression by least squares, a technique dating to Gauss and Legendre in the early 1800s, is itself a machine learning algorithm by this definition: it minimizes an error measure (squared error) over a set of parameters using data. What distinguishes the modern field of machine learning is less the underlying mathematical principle, minimizing a loss function over parameters, than the scale of data and computation now available, and the resulting emphasis on flexible, high-capacity models (trees, ensembles, neural networks) whose parameters number in the thousands, millions, or more, and whose behavior is correspondingly harder to interpret than that of the small, transparent linear models classical statistics was built around.

In finance, machine learning can be used for several purposes. It can forecast returns or volatility, classify whether a firm is likely to default, detect fraudulent transactions, or extract sentiment from news articles. The common thread is that the model must learn from observed patterns and then generalize to future cases, which is a fundamentally harder task than simply describing the past.

The challenge is that finance is not a static environment. Relationships can change because of policy shifts, market regimes, investor behavior, or macroeconomic conditions. A model that performs well in one regime may fail in another. This is why machine learning in finance must be approached with care and tested under realistic conditions, a theme that runs throughout this chapter and resurfaces in every applied chapter that follows.

6.2.2 Supervised and Unsupervised Learning

Machine learning problems are often divided into supervised and unsupervised learning. In supervised learning, the outcome of interest, called the label or target, is observed in the training data. For example, one may have historical data on firm characteristics and a label indicating whether default occurred. The task is to learn a function $\hat{f}$ mapping features x to a target y, so that $\hat{f}(x) \approx y$ on new, unseen data. Common examples include linear regression, logistic regression, decision trees, and neural networks.

In unsupervised learning, no explicit target is available. The goal is to discover structure in the data. Clustering methods group similar observations together, while dimensionality reduction methods compress the data into lower-dimensional representations. In finance, unsupervised learning can help identify groups of similar assets, detect unusual trading patterns, or reduce the dimensionality of large datasets.

A third category, semi-supervised learning, sits between the two: it uses a small amount of labeled data together with a much larger amount of unlabeled data, which is a common situation in finance where obtaining a reliable label (for example, confirming that a transaction was genuinely fraudulent, which may require a lengthy investigation) is expensive relative to collecting the raw data itself. A related distinction is between discriminative models, which learn the boundary or function that separates or predicts labels directly (linear regression, logistic regression, and most models in this chapter), and generative models, which instead learn the full probability distribution of the data and can be used to generate new, synthetic examples resembling the training data. Naive Bayes, introduced later in this chapter, is a simple generative model; large language models, discussed in Chapter 16, are far more elaborate examples of the same generative principle applied to text.

The distinction is important because the choice of method depends on the question being asked. If one wants to predict default, supervised learning is appropriate. If one wants to understand hidden structure in a large set of securities, unsupervised learning may be more appropriate. The remainder of this chapter proceeds roughly in that order: supervised regression and classification first, since they are the most immediately useful for the applied chapters that follow, then unsupervised clustering and dimensionality reduction, and finally neural networks, which can be applied to either supervised or unsupervised problems depending on how they are trained.

6.3 Core Supervised Models

This part builds the standard supervised toolkit from the ground up, each model worked numerically before the next is introduced.

Wulin.Suo@Queensu.ca

6.3.1 Regression: A Worked Example

Suppose a credit analyst wants to relate a firm's leverage, measured as debt-to-EBITDA, to the credit spread the market demands on its bonds, measured in basis points. Table 6.1 shows five observations.

Debt/EBITDA (x)	2	3	4	5	6
Credit spread, bps (y)	120	150	210	260	300

Table 6.1: Leverage and credit spread for five hypothetical issuers

Linear regression models the target as a linear function of the feature plus noise, $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$. The ordinary least squares (OLS) estimator chooses β_0, β_1 to minimize the sum of squared residuals,

$$(\hat{\beta}_0, \hat{\beta}_1) = \arg\min_{\beta_0,\beta_1} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2 ,$$

which has the closed-form solution

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

With $\bar{x} = 4$ and $\bar{y} = 208$, the data in Table 6.1 give $\sum_i (x_i - \bar{x})(y_i - \bar{y}) = 470$ and $\sum_i (x_i - \bar{x})^2 = 10$, so

$$\hat{\beta}_1 = \frac{470}{10} = 47, \qquad \hat{\beta}_0 = 208 - 47(4) = 20.$$

The fitted model is $\widehat{\text{spread}} = 20 + 47 \times \text{leverage}$: each additional turn of debt-to-EBITDA is associated with roughly 47 additional basis points of credit spread. The fitted values are 114, 161, 208, 255, and 302 bps, close to the observed 120, 150, 210, 260, and 300. Figure 6.1 plots the five issuers alongside this fitted line, making the tightness of the fit, and the size of each point's residual, visually immediate.

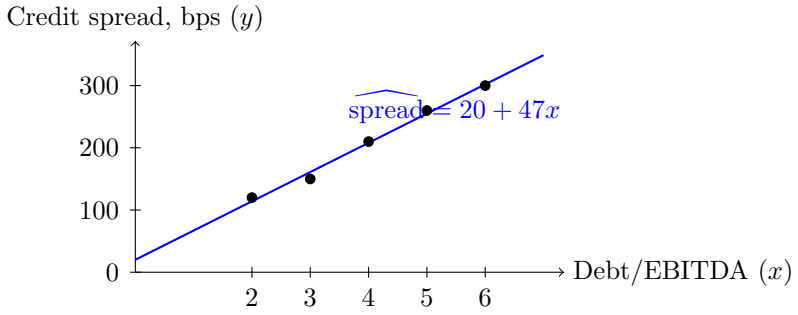


Figure 6.1: Debt-to-EBITDA leverage and credit spread for the five hypothetical issuers in Table 6.1 (black dots), with the OLS-fitted line $\widehat{\text{spread}} = 20 + 47x$ overlaid (blue)

The residual sum of squares is

$$\text{SSE} = \sum_i (y_i - \hat{y}_i)^2 = 190,$$

and the total sum of squares around the mean is

$$\text{SST} = \sum_i (y_i - \bar{y})^2 = 22{,}280,$$

giving

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{190}{22{,}280} \approx 0.991,$$

Wulin.Suo@Queensu.ca

so leverage explains about 99% of the variation in credit spread in this small, deliberately clean example. In sklearn, the same fit is obtained with

```
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([2, 3, 4, 5, 6]).reshape(-1, 1)
y = np.array([120, 150, 210, 260, 300])
model = LinearRegression().fit(X, y)
print(model.intercept_, model.coef_)   # 20.0 [47.]
print(model.score(X, y))               # 0.9915...
```

Real credit spread data would show far more scatter than this toy example; the point is to make the mechanics of fitting and evaluating a linear model completely transparent before adding real-world noise and complexity.

Standard Errors, Confidence Intervals, and p-Values for Coefficients

The point estimate $\hat{\beta}_1 = 47$ says nothing on its own about how confidently the analyst should treat that number, and a machine learning workflow that reports only a point estimate and an R^2 is leaving useful information on the table. With $n = 5$ observations and two estimated parameters $(\hat{\beta}_0, \hat{\beta}_1)$, there are $n-2 = 3$ residual degrees of freedom, and the estimated noise variance is

$$\hat{\sigma}^2 = \text{SSE}/(n-2) = 190/3 \approx 63.33.$$

The standard error of the slope, which measures how much $\hat{\beta}_1$ would be expected to vary across hypothetical repeated samples of five issuers drawn from the same underlying relationship, is

$$SE(\hat{\beta}_1) = \sqrt{\frac{\hat{\sigma}^2}{\sum_i (x_i - \bar{x})^2}} = \sqrt{\frac{63.33}{10}} \approx 2.52.$$

Dividing the estimate by its standard error gives the t-statistic, $t = 47/2.52 \approx 18.68$, which under the null hypothesis $H_0 : \beta_1 = 0$ (leverage carries no information about credit spread at all) follows a t-distribution with 3 degrees of freedom. The associated p-value, the probability of observing a t-statistic this extreme purely by chance if H_0 were true, is $p \approx 0.0003$, far below the conventional 5% threshold; this is strong evidence, within this small and deliberately clean sample, that the slope is genuinely nonzero. A 95% confidence interval for the slope is

$$\hat{\beta}_1 \pm t^*_{0.975,3} \cdot SE(\hat{\beta}_1) = 47 \pm (3.182)(2.52) \approx (39.0,\ 55.0),$$

using the t-distribution critical value $t^*_{0.975,3} \approx 3.182$. The correct reading is that if this sampling and estimation procedure were repeated many times, about 95% of the resulting intervals would contain the true relationship between leverage and spread, not that there is a 95% chance the true β_1 falls in this specific interval. Applying the same steps to the intercept gives $SE(\hat{\beta}_0) \approx 10.68$, $t \approx 1.87$, and $p \approx 0.158$: the intercept is not statistically distinguishable from zero at the 5% level, which is unsurprising given that only one of the five leverage values ($x = 2$) is anywhere near $x = 0$, so the data are not very informative about the spread a zero-leverage issuer would command.

This distinction between the slope's overwhelming significance and the intercept's weak significance illustrates a point that generalizes well beyond this example: statistical significance is coefficient-specific, and a model can pin down some relationships in the data far more precisely than others. It is also worth flagging the same overfitting connection raised throughout this chapter: fitting a regression with many candidate features and reporting only the ones whose p-values happen to fall below 0.05 overstates the evidence, since testing enough coefficients against noise will eventually produce a spuriously "significant" one by chance alone, the multiple-testing concern revisited in Chapter 7's discussion of pairs selection for cointegration trading. A single small p-value is far more persuasive when it was the only hypothesis tested than when it is the best of dozens.

Wulin.Suo@Queensu.ca

6.3.2 How Models Learn: Gradient Descent

The closed-form OLS solution above exists because linear regression's loss function has a special structure: it is quadratic in the parameters, so its minimum can be found in one step by solving a system of linear equations. Most machine learning models, including logistic regression, neural networks, and many others introduced later in this chapter, do not have this convenient property, and their parameters must instead be found iteratively using an optimization algorithm called gradient descent. Figure 6.2 shows the idea for a single generic parameter, written θ there and standing for whichever parameters a given model has to fit.

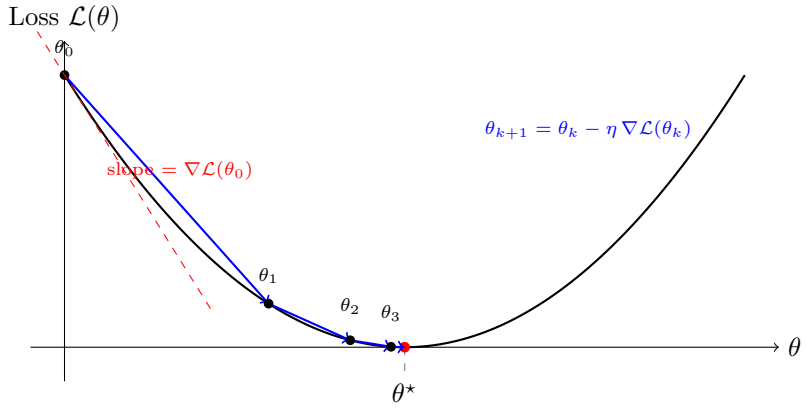


Figure 6.2: The idea of gradient descent. Starting from an initial guess θ_0, each step moves against the local slope of the loss curve, the negative gradient, illustrated here by the tangent line at θ_0, producing a sequence $\theta_0, \theta_1, \theta_2, \ldots$ that descends toward the minimizer $\theta^\star$. Steps are large where the curve is steep and shrink automatically as the curve flattens near the minimum, the pattern seen numerically in Table 6.2 below.

The idea is simple: starting from some initial guess for the parameters, repeatedly take a small step in the direction that most reduces the loss, the negative of the gradient (the vector of partial derivatives) of the loss with respect to each parameter, and stop once further steps no longer improve the loss. For the mean squared error loss

$$\mathcal{L}(\beta_0, \beta_1) = \frac{1}{n}\sum_i (\beta_0 + \beta_1 x_i - y_i)^2,$$

the gradients are

$$\frac{\partial \mathcal{L}}{\partial \beta_0} = \frac{2}{n}\sum_i (\hat{y}_i - y_i), \qquad \frac{\partial \mathcal{L}}{\partial \beta_1} = \frac{2}{n}\sum_i (\hat{y}_i - y_i)\, x_i,$$

and each iteration updates the parameters by

$$\beta_0 \leftarrow \beta_0 - \eta \frac{\partial \mathcal{L}}{\partial \beta_0}, \qquad \beta_1 \leftarrow \beta_1 - \eta \frac{\partial \mathcal{L}}{\partial \beta_1},$$

where η is the learning rate, a small positive number controlling the step size. Applying this to the same credit-spread data as Section 6.3.1, starting from $\beta_0 = \beta_1 = 0$ with a learning rate of $\eta = 0.01$, produces the trajectory in Table 6.2.

After 10,000 iterations, gradient descent has converged to $\beta_0 \approx 20.00$ and $\beta_1 \approx 47.00$, matching the closed-form OLS solution computed directly in Section 6.3.1 up to rounding. This convergence is not a coincidence: for a quadratic loss function like this one, gradient descent is mathematically guaranteed to converge to the same global minimum that the closed-form formula finds directly, provided the learning rate is small enough to avoid overshooting. The learning rate itself involves a tradeoff that recurs throughout machine learning: too small, and convergence (as seen here) can require many thousands of iterations; too large, and

Wulin.Suo@Queensu.ca

Iteration	β_0	β_1
1	4.16	18.52
2	6.76	30.04
3	8.38	37.21
10	11.05	48.55
1,000	18.91	47.24
10,000	20.00	47.00

Table 6.2: Gradient descent converging toward the closed-form OLS solution

the updates can overshoot the minimum and diverge rather than converge at all. In practice, η is one of the most important hyperparameters to tune when training a model with gradient descent, including the neural networks discussed later in this chapter, and it is rarely fixed at a single value throughout training; adaptive variants that shrink the learning rate over time, or that use different effective rates for different parameters, are standard in modern deep learning software. The value of understanding gradient descent, even for a problem like linear regression where a closed-form solution already exists, is that the same algorithm scales to problems with no closed-form solution at all: logistic regression's log-loss, a neural network's loss function with millions of parameters, and many other models throughout this book are all fit using gradient descent or a close variant of it, making it arguably the single most important algorithm in modern machine learning.

6.3.3 Classification: A Worked Example

Regression predicts a continuous target; classification predicts a discrete one, such as whether a borrower defaults. Logistic regression models the probability of the positive class as

$$p(x) = \Pr(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}},$$

so that $p(x)$ is always between 0 and 1, unlike a linear model's unbounded output. The parameters are typically chosen to minimize the log-loss (negative log-likelihood),

$$\mathcal{L}(\beta) = -\frac{1}{n} \sum_{i=1}^{n} \Big[y_i \log p(x_i) + (1 - y_i) \log \big(1 - p(x_i)\big) \Big].$$

A remark on terminology worth one sentence, because the reader will meet it in industry: linear and logistic regression are both members of the generalized linear model (GLM) family, which pairs a linear combination of features with a link function and an outcome distribution suited to the target, identity link with a normal outcome for ordinary regression, log-odds link with a binary outcome here, and, in the family's other widely-used members, a log link with Poisson-distributed counts or Gamma-distributed positive amounts; GLMs remain the regulatory workhorse of insurance pricing precisely because their fitted coefficients read as multiplicative rating factors.

Suppose a lender scores 10 loan applicants and, after choosing a probability threshold, classifies each as predicted default (1) or predicted no default (0). Table 6.3 shows the actual and predicted outcomes.

Applicant	1	2	3	4	5	6	7	8	9	10
Actual default	1	0	1	0	0	1	0	0	1	0
Predicted default	1	0	1	1	0	1	0	0	0	0

Table 6.3: Actual versus predicted default for 10 loan applicants

Comparing the two rows produces a confusion matrix: 3 true positives (TP; applicants 1, 3, 6), 1 false negative (FN; applicant 9, an actual default the model missed), 1 false positive (FP; applicant 4, a good

Wulin.Suo@Queensu.ca

	Actual	
Predicted	Default (1)	No Default (0)
Default (1)	$TP = 3$	$FP = 1$
No Default (0)	$FN = 1$	$TN = 5$

Table 6.4: Confusion matrix for the 10 loan applicants in Table 6.3

loan flagged as risky), and 5 true negatives (TN). Table 6.4 arranges these four counts into the standard 2×2 layout, with the model's predictions as rows and the actual outcomes as columns.

From these four counts, the standard classification metrics follow directly:

$$\begin{aligned}
\text{Accuracy} &= \frac{TP + TN}{n} = \frac{3 + 5}{10} = 0.80, \\
\text{Precision} &= \frac{TP}{TP + FP} = \frac{3}{4} = 0.75, \\
\text{Recall (TPR)} &= \frac{TP}{TP + FN} = \frac{3}{4} = 0.75, \\
\text{F1} &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = 0.75, \\
\text{FPR} &= \frac{FP}{FP + TN} = \frac{1}{6} \approx 0.167.
\end{aligned}$$

Each metric answers a different question. Accuracy answers "what fraction of predictions were correct overall," which can be misleading when defaults are rare, since always predicting "no default" would score highly on accuracy while being useless. Precision answers "of the loans flagged as risky, how many actually defaulted," which matters when false positives (turning away good customers) are costly. Recall answers "of the loans that actually defaulted, how many did the model catch," which matters when false negatives (missed defaults) are costly. A receiver operating characteristic (ROC) curve plots the true positive rate against the false positive rate as the classification threshold varies, and the area under this curve (AUC) summarizes performance across all thresholds at once, rather than at the single threshold used in Table 6.3.

6.3.4 K-Nearest Neighbors: A Non-Parametric Classifier

Logistic regression, like linear regression, is a parametric model: it assumes a specific functional form (a linear combination of features passed through a sigmoid) and estimates a fixed, finite set of parameters. K-nearest neighbors (KNN) takes a completely different, non-parametric approach: it makes no assumption about the functional form of the relationship between features and target, and instead classifies a new observation by looking directly at the k most similar observations in the training data and taking a majority vote of their labels.

Consider a small loan-underwriting dataset with two features, debt-to-income ratio (%) and years of credit history, shown in Table 6.5.

Applicant	Debt/income (%)	Credit history (yrs)	Outcome
A	35	2	Default
B	20	8	No default
C	40	1	Default
D	15	10	No default
E	30	3	Default
F	18	7	No default

Table 6.5: A small loan dataset for K-nearest neighbors classification

Wulin.Suo@Queensu.ca

To classify a new applicant with debt-to-income of 28% and 4 years of credit history, KNN computes the Euclidean distance from this query point to every training point,

$$d = \sqrt{(28 - x_1)^2 + (4 - x_2)^2},$$

giving

$$d_E = 2.24, \quad d_A = 7.28, \quad d_B = 8.94,$$
$$d_F = 10.44, \quad d_C = 12.37, \quad d_D = 14.32,$$

sorted from nearest to farthest. With $k = 3$, the three nearest neighbors are E (Default), A (Default), and B (No default), so the majority vote (2 Default, 1 No default) classifies the new applicant as a predicted default. Figure 6.3 shows this geometrically: the query point sits closest to a mix of Default and No-default points, and only the identity of its k nearest neighbors, not any global decision boundary, determines the prediction.

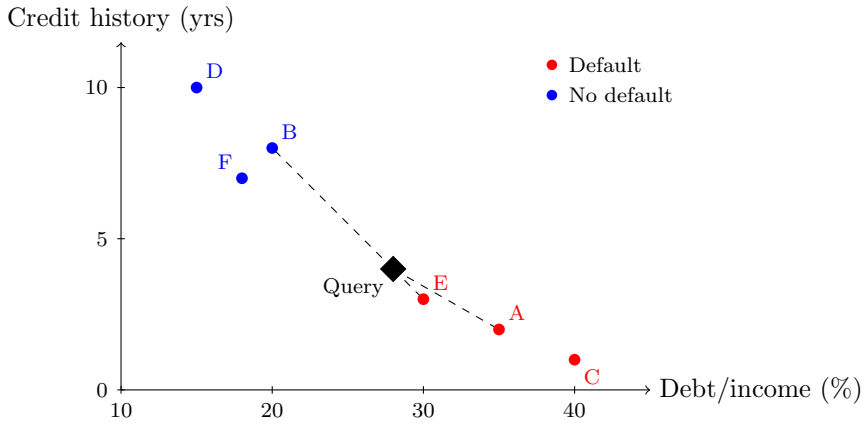


Figure 6.3: The K-nearest neighbors example of Table 6.5: six labeled training points and the query applicant (debt/income 28%, 4 years of credit history, black diamond), with dashed lines to its three nearest neighbors under $k = 3$

The choice of k controls a bias-variance tradeoff directly analogous to the one in Section 6.4.1: a small k (even $k = 1$) makes the prediction highly sensitive to individual nearby points, including noisy or mislabeled ones (high variance), while a large k smooths over local structure and can underfit, in the extreme case where k equals the entire training set, always predicting the overall majority class regardless of the query point (high bias). KNN also has a practical limitation that becomes severe in high dimensions: as the number of features grows, all points tend to become roughly equidistant from one another, a phenomenon known as the curse of dimensionality, which is one reason KNN is more commonly used with a small number of carefully chosen features than as a general-purpose model for wide financial datasets.

6.3.5 Support Vector Machines

Suppose a credit analyst has a single, standardized measure of financial distress for a handful of firms and wants to draw the clearest possible dividing line between the ones that later defaulted and the ones that did not, not just any line that happens to separate the two groups in the historical data, but the one least likely to misclassify a new, slightly different firm sitting close to the boundary. A support vector machine (SVM) approaches classification geometrically: rather than modeling a probability (logistic regression) or voting among neighbors (KNN), it searches directly for the hyperplane (a line, in two dimensions) that separates the two classes with the widest possible margin, the distance from the separating boundary to the nearest training point of either class. Intuitively, a wider margin means the decision boundary is less sensitive to small perturbations in the data, which is one reason SVMs often generalize well.

Wulin.Suo@Queensu.ca

The training points that lie exactly on the margin, the closest points to the boundary, are called support vectors, since they alone determine the position of the separating hyperplane; every other training point could, in principle, be moved or removed without changing the fitted boundary at all, in contrast to a linear regression, where every point contributes to the fitted line. When the classes cannot be separated perfectly by a straight line, a soft-margin SVM allows some points to fall on the wrong side of the margin, controlled by a penalty parameter that trades off margin width against the number of misclassified training points, directly analogous to the regularization strength λ in ridge and lasso regression.

SVMs can also be extended to nonlinear decision boundaries using the "kernel trick," which implicitly maps the original features into a higher-dimensional space where a linear separator corresponds to a nonlinear boundary in the original feature space, without ever explicitly computing the higher-dimensional coordinates. In finance, SVMs have historically been used for classification tasks such as bankruptcy prediction and directional price forecasting, though tree-based ensembles (Section 6.6.1) and neural networks have become more common choices for large, tabular financial datasets in recent years, in part because SVMs scale less gracefully to very large training sets and because tree-based models typically require less feature preprocessing.

A deliberately simple example makes the maximum-margin idea concrete without requiring the full quadratic-programming machinery SVMs use in practice. Suppose four training points are perfectly separable along a single feature (say, a standardized measure of financial distress): two "high-risk" points at $x = 1$ and $x = 2$, and two "low-risk" points at $x = 4$ and $x = 5$. Because the classes are already perfectly separated along this one dimension, the maximum-margin hyperplane is simply the point exactly halfway between the two closest points of opposite classes, $x = 2$ and $x = 4$, namely $x = 3$.

Writing the decision function as $wx + b$ with $w = 1$, the boundary condition $wx + b = 0$ at $x = 3$ requires $b = -3$, and checking the two nearest points confirms $w(2) + b = 2 - 3 = -1$ and $w(4) + b = 4 - 3 = 1$, the ± 1 margin boundaries the SVM optimization enforces. The points at $x = 2$ and $x = 4$ are the support vectors: they alone determine w and b, and the geometric margin, the distance from the boundary to either margin boundary, is $1/|w| = 1$, for a total margin width of 2, the full gap between the nearest opposite-class points.

The two more extreme points, $x = 1$ and $x = 5$, play no role at all in determining the boundary, illustrating concretely the earlier claim that only support vectors matter: moving the point at $x = 1$ to $x = -100$ would not change the fitted hyperplane in the slightest, whereas moving the support vector at $x = 2$ even slightly would immediately shift the boundary. Figure 6.4 shows this geometry directly.

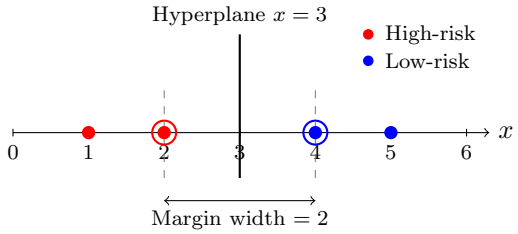


Figure 6.4: The maximum-margin hyperplane for the one-dimensional example: two high-risk points ($x = 1, 2$) and two low-risk points ($x = 4, 5$), with the support vectors circled

6.4 Generalization and Model Selection

A model that fits its training data perfectly has usually learned the noise; this part covers how that failure is detected, measured, and controlled.

6.4.1 Overfitting and Generalization

A central problem in machine learning is overfitting. A complex model can fit the training data very well yet fail when applied to new data. This occurs because the model has learned noise, quirks, or idiosyncrasies

Wulin.Suo@Queensu.ca

that are not repeated outside the sample. In finance, this is especially important because historical data can be highly noisy and because market regimes shift over time. Table 6.6 shows a stylized pattern that recurs constantly in practice: as a model (here, a polynomial regression) is allowed to grow more flexible, training error keeps falling, but test error falls only up to a point and then rises again.

Polynomial degree	1	2	3	5	9
Training error (MSE)	8.4	6.1	4.9	3.0	0.4
Test error (MSE)	9.0	6.5	5.4	6.8	15.2

Table 6.6: Stylized training and test error as model complexity increases

The minimum test error in Table 6.6 occurs at degree 3, not at the most flexible model, degree 9, even though degree 9 fits the training data almost perfectly. This gap between training and test performance is the signature of overfitting, and it is the reason a model should never be selected purely on training performance. Figure 6.5 plots the same two rows of Table 6.6 as curves rather than numbers, making the characteristic divergence easier to see at a glance.

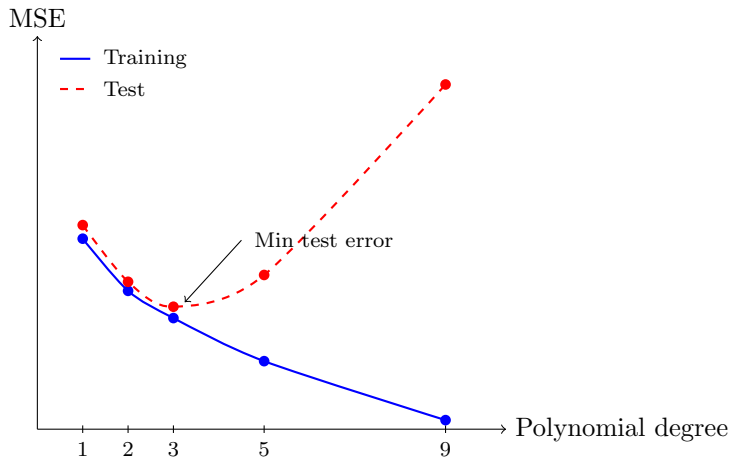


Figure 6.5: Training and test error (MSE) from Table 6.6 as polynomial degree increases

Generalization refers to the ability of a model to perform on unseen data. The goal of model selection is not merely to maximize performance on the training set, but to choose a model that will perform well out of sample. k-fold cross-validation formalizes this: the data are split into k roughly equal parts, and the model is trained on $k-1$ of them and evaluated on the remaining part, repeating k times so that every observation is used for validation exactly once. Figure 6.6 illustrates a single fold of 5-fold cross-validation.



Fold 3 held out for validation; Folds 1, 2, 4, 5 used for training

Figure 6.6: One iteration of 5-fold cross-validation. The procedure repeats five times, holding out a different fold each time, and the five validation scores are averaged.

For financial time series, ordinary k-fold cross-validation can be misleading because it may train on future data and validate on the past, which is not possible in real deployment. Walk-forward validation, in

Wulin.Suo@Queensu.ca

which the training window always precedes the validation window in time, is the standard alternative and is discussed further in Chapter 7.

Regularization is a technique designed to reduce overfitting by penalizing model complexity directly in the optimization objective. Ridge regression adds a penalty proportional to the sum of squared coefficients,

$$\hat{\beta}^{\text{ridge}} = \arg\min_{\beta} \sum_{i=1}^{n} (y_i - x_i^\top \beta)^2 + \lambda \sum_{j=1}^{p} \beta_j^2,$$

while lasso regression uses the sum of absolute values,

$$\hat{\beta}^{\text{lasso}} = \arg\min_{\beta} \sum_{i=1}^{n} (y_i - x_i^\top \beta)^2 + \lambda \sum_{j=1}^{p} |\beta_j|.$$

In both cases, $\lambda \geq 0$ controls the strength of the penalty: $\lambda = 0$ recovers ordinary least squares, and larger λ shrinks coefficients toward zero, trading a small increase in bias for a potentially large reduction in variance. Lasso has the additional property that it can shrink coefficients exactly to zero, which makes it useful for feature selection when many candidate predictors are available, a common situation in finance.

A Worked Example: Ridge Versus Lasso on a Genuine and a Spurious Feature

The sparsity property is easiest to see with two features, one genuinely predictive and one pure noise, fit with both penalties at the identical λ. Simulating 200 observations with $y = 2.0\,x_1 + \varepsilon$, where x_1 and x_2 are independent standard-normal features and x_2 has no relationship to y at all, ordinary least squares correctly identifies both roles, $\hat{\beta}_1^{\text{OLS}} \approx 1.949$ and $\hat{\beta}_2^{\text{OLS}} \approx 0.031$, the latter small and attributable to sampling noise alone.

Before fitting, one practical trap. The library used in the companion notebook, scikit-learn, does not scale the two objectives the same way. Its `Ridge` minimizes exactly the ridge objective above, but its `Lasso` minimizes

$$\frac{1}{2n} \sum_{i=1}^{n} (y_i - x_i^\top \beta)^2 + \alpha \sum_{j=1}^{p} |\beta_j|,$$

so `Lasso(alpha=1.0)` is the lasso above with $\lambda = 2n\alpha = 400$, not $\lambda = 1$. Calling both with the default `alpha=1.0` therefore compares a lasso penalty 400 times stronger than the ridge one. Under the single objective written above, the comparison is

$$\begin{aligned} \lambda = 1: &\quad \hat{\beta}^{\text{ridge}} = (1.936,\ 0.030), \quad \hat{\beta}^{\text{lasso}} = (1.945,\ 0.028), \\ \lambda = 400: &\quad \hat{\beta}^{\text{ridge}} = (0.543,\ -0.019), \quad \hat{\beta}^{\text{lasso}} = (0.654,\ 0.000). \end{aligned}$$

At $\lambda = 1$ neither penalty does much, since λ is tiny next to a residual sum of squares built from 200 observations. At $\lambda = 400$ the contrast is striking: ridge shrinks both coefficients smoothly, the genuine one to 0.543, but leaves the spurious one nonzero, while lasso drives the spurious feature's coefficient to *exactly* zero (not merely small) and actually keeps more of the genuine feature's coefficient (0.654). This illustrates lasso's defining behavior directly: rather than shrinking every coefficient smoothly and proportionally the way ridge does, lasso's penalty has a kink at zero that makes it optimal, past a certain threshold, to set a weak coefficient to precisely zero rather than merely small, effectively performing feature selection as a side effect of fitting the model rather than as a separate step. The scaling trap is itself a practical lesson: libraries scale penalties differently (by $1/n$, $1/(2n)$, or not at all), so a nominal penalty value means nothing until you have read the objective the library actually minimizes, and in any case λ should be tuned by cross-validation (introduced earlier in this section) rather than copied from a default.

6.4.2 Bias, Variance, and the Tradeoff

Suppose two analysts fit different models to the exact same noisy training sample: one fits a single straight line, the other fits a high-degree polynomial that passes through every training point exactly, the degree-9 fit in Table 6.6 among them. Which one should be trusted on new data the model has not yet seen, and

Wulin.Suo@Queensu.ca

why? The bias-variance tradeoff is the foundational concept in machine learning that answers this precisely. For a model $\hat{f}$ trained on random training data and evaluated at a point x with true relationship

$$y = f(x) + \varepsilon,$$

where ε has mean zero and variance σ^2, the expected squared prediction error decomposes as

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \underbrace{\Big(f(x) - \mathbb{E}[\hat{f}(x)]\Big)^2}_{\text{Bias}^2} + \underbrace{\text{Var}\big(\hat{f}(x)\big)}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible error}}.$$

A model with high bias is too simple and systematically misses the true relationship, regardless of how much data it sees; a straight line fit to a strongly curved relationship is a typical example. A model with high variance is too flexible and reacts strongly to the noise in whatever particular training sample it happened to see; the degree-9 polynomial in Table 6.6 is a typical example. A good model strikes a balance between the two, and this is exactly what the minimum of the test error curve in Table 6.6 represents: the complexity level at which bias and variance are jointly minimized, not the complexity level that minimizes bias alone.

This tradeoff is especially relevant in finance. A very simple model may be stable but fail to capture subtle structure. A very complex model may fit the sample well but fail in real deployment because it does not generalize. With limited data and unstable relationships, simpler, higher-bias models can be surprisingly effective, because their lower variance makes them more robust across changing regimes. With large, high-quality datasets and a stable signal, more complex models may perform better. The key is to evaluate models honestly, using out-of-sample or walk-forward testing, and not assume that complexity automatically improves results.

6.4.3 Hyperparameter Tuning and Model Selection

Every model introduced in this chapter has at least one hyperparameter, a setting chosen before training rather than learned from the data, that controls its position on the bias-variance spectrum: the ridge penalty λ, the number of neighbors k in KNN, the maximum depth of a decision tree, or the number of trees and learning rate in a gradient-boosted ensemble. Choosing these hyperparameters well is often as consequential for a model's real-world performance as choosing the model family itself, and it must be done without ever looking at the final test set, or the resulting performance estimate is no longer a trustworthy measure of generalization.

The standard workflow, called grid search combined with cross-validation, proceeds as follows: specify a grid of candidate hyperparameter values (for example, $\lambda \in \{0.01, 0.1, 1, 10, 100\}$), and for each candidate value, run k-fold cross-validation (or, for time series data, the walk-forward validation introduced in Chapter 7) entirely within the training set, recording the average validation performance. The hyperparameter value with the best average cross-validated performance is then selected, and only at this point is the model retrained on the full training set and evaluated once on the held-out test set, which was never used during the search. This nested structure, tuning within cross-validation, and only then checking the final test set, is essential: using the test set to choose hyperparameters, even informally by trying a few values and picking the one that looks best on the test set, quietly reintroduces the same overfitting problem that cross-validation was designed to prevent, and inflates the reported performance in a way that will not hold up in production.

Financial applications typically add two further complications to this standard workflow. First, because financial time series exhibit the non-stationarity discussed throughout this book, the hyperparameters selected on one period's data are not guaranteed to remain optimal in a later period, which argues for periodically re-running the tuning process rather than fixing hyperparameters permanently. Second, the grid search itself, trying many hyperparameter combinations, is a form of multiple testing exactly analogous to the backtest-overfitting concern discussed in Chapter 7: searching over a very large grid increases the chance of finding a hyperparameter setting that performs well on the validation folds purely by chance, which is why practitioners often prefer a modest, economically motivated grid over an exhaustive search across hundreds of candidate values.

Wulin.Suo@Queensu.ca

6.4.4 Model Evaluation

Evaluating a machine learning model requires more than looking at a single number. Different tasks require different metrics. For regression, common metrics include mean squared error (MSE), mean absolute error (MAE), and R^2, all illustrated in Section 6.3.1. For classification, common metrics include accuracy, precision, recall, F1, and AUC, all illustrated in Section 6.3.3. In finance, one also cares about calibration (do predicted probabilities match observed frequencies), stability (does performance hold up across different time periods), and economic usefulness (does the model's output translate into a profitable or risk-reducing decision after costs).

A model that predicts default with high accuracy may still be impractical if it is poorly calibrated, produces too many false positives, or fails to distinguish risk levels in a useful way. Likewise, a trading model may have good statistical performance but poor economic performance once transaction costs and execution limits are considered. This is why evaluation in finance must be tied to the actual decision problem, not just to a generic statistical scorecard.

Calibration deserves a brief numerical illustration since it is easy to confuse with accuracy. Suppose a credit model assigns a predicted default probability of 20% to a group of 50 borrowers. The model is well calibrated for this group if, among those 50 borrowers, close to 10 (that is, $20\% \times 50$) actually default; if instead only 2 default, the model's probabilities are too high (poorly calibrated) even if the model still ranks borrowers correctly from most to least risky, which is the kind of distinction accuracy and even AUC can miss, since both depend only on the ranking of predictions, not on whether the predicted probabilities themselves are numerically meaningful. Calibration matters enormously in finance because predicted probabilities are frequently used directly in downstream calculations, such as the expected-loss and reserve calculations in Chapter 9, where a systematically overstated or understated default probability translates directly into an overstated or understated dollar reserve, not just a misranked list of borrowers.

6.5 Working with Financial Data

Financial data breaks several assumptions the previous parts made quietly, and the features handed to a model matter more than the model.

6.5.1 Practical Considerations in Financial ML

The practical deployment of machine learning in finance requires attention to data leakage, sample selection, and concept drift. Data leakage occurs when information that would not be available at prediction time is used in the model, for example including a variable that is only recorded after the outcome is known. This can produce optimistic in-sample or backtest results that vanish once the model is used in real time. Sample selection issues arise when the training data are not representative of the future environment, for instance training a credit model only on approved loans, which excludes information about applicants who were rejected. Concept drift occurs when the relationship between features and outcomes changes over time, for example when a recession changes which borrower characteristics matter most for default.

A compact view of the modeling lifecycle is shown in Figure 6.7.

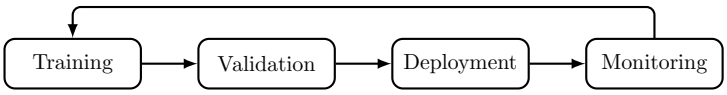


Figure 6.7: A machine learning workflow in finance should include training, validation, deployment, and continuous monitoring, with findings from monitoring feeding back into retraining

These issues are not minor technicalities. They often determine whether a model survives in production. Strong financial modeling involves not only fitting a statistical relationship, but also ensuring that the relationship is stable enough to remain useful, and building the monitoring infrastructure to detect when it stops being useful.

Wulin.Suo@Queensu.ca

6.5.2 Feature Engineering for Financial Data

Every model in this chapter takes a fixed set of numerical features as input, but raw financial data, prices, financial statement line items, transaction records, are rarely usable directly, and the process of transforming raw data into informative model inputs, called feature engineering, is often as important to a model's performance as the choice of algorithm itself. Several patterns recur constantly when engineering features from financial data. Ratios, such as the debt-to-equity and margin ratios computed throughout Chapter 4, normalize raw dollar figures so that firms of very different sizes become comparable, exactly as the common-size statements in that chapter did.

Lagged and rolling features, such as a 30-day rolling volatility or a 12-month trailing revenue growth rate, summarize recent history into a single number a model can consume, echoing the rolling-window calculations introduced in Chapter 2. Interaction features, such as leverage multiplied by an indicator for a recessionary period, allow a linear model to capture that the effect of one variable depends on the level of another, a relationship a plain linear model cannot represent on its own. Categorical features, such as industry sector, are typically converted into numerical form via one-hot encoding (a separate binary column for each category) before being used in most models, since the sector-specific behavior discussed in Chapter 4 needs to be represented numerically for a model to use it as a signal rather than an arbitrary label. Table 6.7 and Table 6.8 make these transformations concrete for a small cross-section of three firms.

Firm	Sector	Debt	Equity	Rev.$_t$	Rev.$_{t-4}$	Recession	Mkt Cap
1	Tech	20	80	150	120	0	45.0
2	Energy	90	60	210	230	1	8.0
3	Retail	40	40	95	90	0	1.2

Table 6.7: Raw firm-level inputs for a small cross-section (Debt, Equity, and Revenue in $ millions; Market Cap in $ billions; Recession is 1 if the firm's primary sector was in a broad-based downturn during the period)

Table 6.8 shows the engineered features these raw inputs produce. The leverage ratio and year-over-year revenue growth follow directly from the raw dollar figures; the interaction term is zero for both non-recession firms regardless of their leverage, but equals 1.50 for Firm 2, flagging that its already-elevated leverage coincides with a sector-wide downturn, exactly the kind of conditional relationship a plain linear model cannot represent on its own. One-hot encoding converts the raw Sector column into three binary indicators: Firm 1 becomes (Technology=1, Energy=0, Retail=0), Firm 2 becomes (Technology=0, Energy=1, Retail=0), and Firm 3 becomes (Technology=0, Energy=0, Retail=1).

Firm	Leverage	Rev. growth (YoY)	Leverage×Recession	$\log_{10}$(Mkt Cap, $B)	Mkt Cap pctile
1	0.25	25.0%	0.00	1.65	100%
2	1.50	−8.7%	1.50	0.90	50%
3	1.00	5.6%	0.00	0.08	0%

Table 6.8: Engineered features derived from Table 6.7

Two additional considerations are specific to finance. First, every engineered feature must respect the same point-in-time discipline emphasized throughout this book: a feature computed using data that would not actually have been available at the time of prediction introduces the kind of data leakage described above, even if the underlying computation, such as a rolling average, seems innocuous. Second, financial features are frequently skewed or heavy-tailed, and Table 6.8's market-capitalization columns make this concrete: the three firms' raw market caps span more than an order of magnitude ($1.2 billion to $45.0 billion, a ratio of 37.5×), while $\log_{10}$ of the same figures compresses this to a range of about 1.57 (0.08 to 1.65), and the cross-sectional percentile rank compresses it further still, to a bounded $[0, 100\%]$ scale regardless of the firms' raw dollar magnitudes. Transformations such as these are common preprocessing steps that make skewed or heavy-tailed features better behaved as inputs to the linear and distance-based models discussed in this chapter.

Wulin.Suo@Queensu.ca

6.6 Beyond Linear Models

This part covers the model families that dominate practice when the relationship is not linear and the data are not tidy.

6.6.1 Tree-Based Models and Ensembles

Tree-based methods are among the most useful models in applied finance because they can capture nonlinear relationships and interactions without requiring the analyst to specify them in advance. A decision tree splits the data recursively according to feature thresholds, choosing at each step the split that most reduces impurity in the resulting groups. For classification, a common impurity measure is the Gini impurity,

$$G = 1 - \sum_{k=1}^{K} p_k^2,$$

where p_k is the proportion of observations in a node belonging to class k; $G = 0$ when a node is pure (all one class) and is largest when classes are evenly mixed. An alternative is entropy, $H = -\sum_k p_k \log_2 p_k$, which behaves similarly.

A single tree tends to have high variance: small changes in the training data can produce very different splits. Random forests reduce this variance by averaging many trees, each trained on a bootstrap sample of the data and a random subset of features at each split, so that the trees are decorrelated from one another. Gradient boosting takes a different approach: it builds trees sequentially, each one correcting the errors of the ensemble built so far. Write $F_m(x)$ for the ensemble's prediction after m trees have been added; boosting initializes $F_0(x)$ to a simple constant (typically the mean of the target for regression, or the log-odds of the positive class for classification) and then grows the ensemble one shallow tree at a time,

$$F_m(x) = F_{m-1}(x) + \nu\, h_m(x), \qquad m = 1, \ldots, M,$$

where h_m is a new regression tree, not fit to the original target y, but fit to the current pseudo-residuals, $y_i - F_{m-1}(x_i)$ for squared-error loss (more generally, the negative gradient of the loss function with respect to the ensemble's current predictions $F_{m-1}(x_i)$, which is where the name "gradient" boosting comes from), so that each new tree specifically targets whatever the ensemble so far is still getting wrong.

The learning rate $\nu \in (0, 1]$, typically set well below 1 (commonly 0.01 to 0.3 in practice), then shrinks each tree's contribution before it is added: a smaller ν requires more trees M to reach a given level of fit, but tends to generalize better, the same regularization-driven bias-variance tradeoff the ridge and lasso penalties illustrated earlier in this chapter. Figure 6.8 shows this sequential structure schematically, in direct contrast to a random forest's independently-grown, averaged trees: each boosting stage cannot be fit until the previous stage's pseudo-residuals are known, an inherently sequential dependency that random forests do not have, which is also why random forests parallelize trivially across trees while gradient boosting does not. Rather than reducing variance through averaging, boosting reduces bias by iteratively correcting the errors of the current model, and is typically combined with a small learning rate and a limited number of trees to control overfitting.

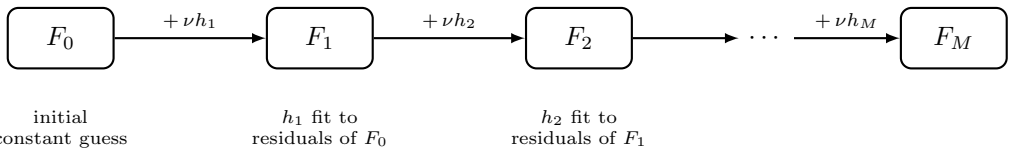


Figure 6.8: Gradient boosting as a sequential chain of shrunken trees, each fit to correct the previous stage's residuals

In finance, such models are widely used for default prediction, credit scoring, and fraud detection. They can also be used to estimate the importance of variables, which can be useful for understanding which features drive a prediction. However, they are not automatically more reliable than simpler models. A

Wulin.Suo@Queensu.ca

tree-based model can still overfit, especially when the data are noisy or when the time structure is ignored during cross-validation. Careful validation, along the lines discussed in Section 6.4.1 and using walk-forward schemes rather than random splits, remains essential.

Table 6.9 summarizes the key differences between the two ensembling strategies introduced above, since they are easily confused despite solving different problems.

	Bagging (e.g., random forest)	Boosting (e.g., gradient boosting)
Trees built	In parallel, independently	Sequentially, each correcting the last
Primarily reduces	Variance	Bias
Base learner	Deep, low-bias trees	Shallow, high-bias trees
Overfitting risk	Lower, more robust to noise	Higher, requires careful tuning
Typical use case	Robust baseline with minimal tuning	Squeezing out maximum predictive accuracy

Table 6.9: Bagging (random forests) versus boosting (gradient-boosted trees)

Neither approach dominates the other in every setting, but the difference in overfitting risk is a genuinely important practical consideration in finance: a gradient-boosted model with many trees and a small learning rate can achieve very low training error, and following the walk-forward validation discipline of Chapter 7 rather than a random split is especially important for catching overfitting before it reaches production.

6.6.2 Naive Bayes: A Probabilistic Classifier

Chapter 1 introduced Bayes' theorem using a fraud-detection example; naive Bayes turns that same idea into a full classification algorithm. Given features $x_1, \ldots, x_p$, it estimates the probability of each class k using Bayes' theorem,

$$\Pr(y = k \mid x_1, \ldots, x_p) \propto \Pr(y = k) \prod_{j=1}^{p} \Pr(x_j \mid y = k),$$

and predicts whichever class has the highest resulting probability. The "naive" assumption is that the features are conditionally independent given the class, which is rarely exactly true (a firm's leverage and interest coverage, for example, are clearly related) but often works well enough in practice to be a fast, effective baseline, particularly for text classification tasks such as the sentiment analysis introduced in Chapter 16, where the features are word counts or word presence indicators and the independence assumption, while still technically false, distorts the result less than it might for tightly coupled financial ratios. Naive Bayes trains almost instantly even on large datasets, since fitting it only requires estimating simple frequency-based probabilities rather than solving an optimization problem like gradient descent, which makes it a useful quick baseline to compare against before investing in a more sophisticated model.

For text classification specifically, where each feature x_j is a count of how often a particular word appears, the individual probabilities $\Pr(x_j \mid y = k)$ are most naturally estimated directly from training-data word frequencies, the fraction of class k's total word count occupied by word x_j. This raw frequency estimate has a serious practical flaw: any word that simply never happens to appear in class k's training documents receives an estimated probability of exactly zero, and because the naive Bayes score is a *product* across all features, a single zero-probability word forces the entire product to zero regardless of how strongly every other word favors that class, an artifact of a small training sample rather than genuine evidence the class is impossible. Laplace smoothing (also called add-one smoothing) fixes this by adding a small constant to every count before normalizing,

$$\Pr(x_j \mid y = k) = \frac{\text{count}(x_j, k) + 1}{\text{count}(k) + V},$$

where $\text{count}(x_j, k)$ is how many times word x_j appeared across class k's training documents, $\text{count}(k)$ is the total word count across those same documents, and V is the vocabulary size, the number of distinct words

Wulin.Suo@Queensu.ca

being counted. Adding one to the numerator guarantees every word receives a strictly positive probability even if it was never observed in that class's training data, while adding V to the denominator keeps the resulting probabilities properly normalized, summing to one across the vocabulary. Chapter 16's 10-K risk-factor classifier applies this correction.

6.6.3 Clustering and Dimensionality Reduction

Not all machine learning tasks involve predicting a known outcome. Sometimes the goal is to understand structure in a large set of assets or transactions. Suppose a portfolio manager has a spreadsheet of a few hundred holdings and no label telling her which ones behave similarly; she would like an algorithm that groups them automatically, purely from their measured characteristics, rather than sorting them by eye. K-means clustering partitions observations into K groups by minimizing the within-cluster sum of squared distances to each group's center,

$$\arg\min_{\{S_k\}} \sum_{k=1}^{K} \sum_{x_i \in S_k} \|x_i - \mu_k\|^2, \qquad \mu_k = \frac{1}{|S_k|} \sum_{x_i \in S_k} x_i,$$

where S_k denotes the set of observations assigned to cluster k and μ_k is that cluster's center, the mean of its assigned points. This groups similar observations together, for example firms with similar financial profiles or trading accounts with similar behavior. Principal component analysis (PCA) is the standard dimensionality-reduction method: it finds the direction w (with $\|w\| = 1$) that maximizes the variance of the projected data, $\arg\max_w \mathrm{Var}(Xw)$, then finds the next such direction orthogonal to the first, and so on. In finance, PCA applied to a panel of asset returns often reveals that a small number of components, frequently interpretable as a broad market factor followed by sector or style factors, explain the large majority of the total variance.

These approaches are especially useful when the analyst faces high-dimensional data and wants to uncover latent structure. For instance, a portfolio manager may want to understand whether returns are driven by a few common factors or by many idiosyncratic sources, a question addressed more fully in Chapter 12. A fraud analyst may want to identify clusters of unusual behavior that deserve closer investigation. In both cases, unsupervised learning can provide a valuable first step before more targeted modeling.

To make K-means concrete, consider four firms described by two features, leverage and profit margin (both scaled to lie between 0 and 1):

$$F_1 = (0.20, 0.30), \qquad F_2 = (0.30, 0.25),$$
$$F_3 = (0.80, 0.05), \qquad F_4 = (0.90, 0.10).$$

Initializing two cluster centers at F_1 and F_4, the algorithm alternates between two steps: assign each point to its nearest center, then recompute each center as the mean of its assigned points. In this example, F_1 and F_2 (both low leverage, higher margin) are closest to the first center, while F_3 and F_4 (both high leverage, low margin) are closest to the second, giving updated centers $\mu_1 = (0.25, 0.275)$ and $\mu_2 = (0.85, 0.075)$. Recomputing distances with these updated centers does not change any point's assignment, so the algorithm has converged after a single update: the data naturally separates into a "low-leverage, higher-margin" group and a "high-leverage, low-margin" group, without ever being told what leverage or margin mean, purely from the geometry of the two features. Figure 6.9 shows the four firms, the initial centers, and the updated centers μ_1 and μ_2 together.

6.6.4 Neural Networks and Deep Learning in Finance

Suppose an analyst suspects that default risk depends on leverage and profitability not through any simple additive combination, but through some more intricate interaction between the two that is too complex to write down as a specific nonlinear formula in advance, and wants a model flexible enough to learn whatever that relationship turns out to be directly from data, rather than committing in advance to a particular functional form the way linear or logistic regression do. Neural networks represent another major class of

Wulin.Suo@Queensu.ca

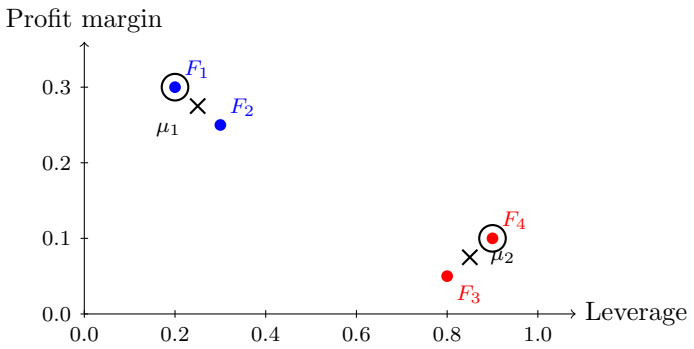


Figure 6.9: The K-means example: four firms in leverage/profit-margin space, with the initial centers (circled) and the updated centers μ_1 and μ_2

machine learning methods, built for this kind of unspecified nonlinearity. A simple feedforward network with one hidden layer computes

$$\hat{y} = \sigma_2\big(W_2\,\sigma_1(W_1 x + b_1) + b_2\big),$$

where W_1, b_1, W_2, b_2 are learned weight matrices and bias vectors, and σ_1, σ_2 are nonlinear activation functions, such as the rectified linear unit $\mathrm{ReLU}(z) = \max(0, z)$ for hidden layers or, for a binary output layer, the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

which maps any real number into the interval $(0, 1)$ so that the output can be read as a probability, and which is the same function logistic regression applied in Section 6.3.3 above. The subscripted σ_1 and σ_2 stand for whichever activation a given layer happens to use; a bare $\sigma(\cdot)$, here and in every later chapter, means this sigmoid specifically. Stacking more such layers produces a deep network. The weights are learned by minimizing a loss function (mean squared error for regression, log-loss for classification) using gradient descent, with gradients computed efficiently via backpropagation, an efficient algorithm for computing the gradient of the loss with respect to every weight in the network by applying the chain rule layer by layer from the output back to the input.

A small numerical example makes the forward pass (computing a prediction from an input) concrete. Suppose a network has two inputs, a hidden layer with two ReLU units, and a single sigmoid output, with weights

$$W_1 = \begin{pmatrix} 0.4 & -0.2 \\ 0.1 & 0.3 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0.05 \\ -0.1 \end{pmatrix}, \quad W_2 = \begin{pmatrix} 0.6 & -0.5 \end{pmatrix}, \quad b_2 = 0.1.$$

For an input $x = (1.0, 0.5)$, the hidden layer's pre-activation values are $W_1 x + b_1 = (0.35, 0.15)$, and since both are already positive, the ReLU activation leaves them unchanged: $h = (0.35, 0.15)$. The output layer's pre-activation is $W_2 h + b_2 = 0.6(0.35) - 0.5(0.15) + 0.1 = 0.235$, and applying the sigmoid function gives the final prediction $\hat{y} = 1/(1 + e^{-0.235}) \approx 0.558$.

Every one of the millions of parameters in a modern deep network is used in this way, matrix multiplications and elementwise nonlinearities, repeated layer after layer; what makes deep learning powerful is not any single computation's complexity but the ability to learn, via backpropagation and gradient descent, weight values across many layers that jointly capture useful nonlinear structure in the data. Figure 6.10 lays out this exact network: two input neurons feed forward into two ReLU hidden neurons, which feed forward into a single sigmoid output neuron, with every edge and bias labeled with the weight values above and every neuron labeled with the value it computes.

A Worked Example: Backpropagation, One Layer at a Time

Section 6.3.2 introduced gradient descent for a single-layer linear regression, where the gradient of the loss with respect to each parameter could be written down directly. A multi-layer network needs the chain rule

Wulin.Suo@Queensu.ca

to propagate a gradient backward through each layer in turn, backpropagation, and the same toy network above makes every step concrete. Suppose this network is being trained as a classifier and the true label for input $x = (1.0, 0.5)$ is $y = 1$ (a positive case, using the log-loss $L = -[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})]$ standard for classification). A well-known simplification, for this pairing of sigmoid output and log-loss, is that the gradient of the loss with respect to the output layer's pre-activation z_2 collapses to simply $\hat{y} - y$:

$$\frac{\partial L}{\partial z_2} = \hat{y} - y = 0.5585 - 1 = -0.4415.$$

Backpropagation now works backward one layer at a time, applying the chain rule at each step. The output layer's weights and bias receive

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial z_2} h^\top = -0.4415\,(0.35,\ 0.15) = (-0.1545,\ -0.0662), \qquad \frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z_2} = -0.4415.$$

To continue backward into the hidden layer, the gradient must first pass through W_2 to reach h, then through the ReLU activation to reach the hidden layer's pre-activations z_1:

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial z_2} W_2 = -0.4415\,(0.6,\ -0.5) = (-0.2649,\ 0.2208), \qquad \frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial h} \odot \mathrm{ReLU}'(z_1),$$

where $\mathrm{ReLU}'(z) = 1$ if $z > 0$ and 0 otherwise, and $\odot$ denotes elementwise multiplication; since both hidden pre-activations (0.35 and 0.15) are positive, $\mathrm{ReLU}'(z_1) = (1, 1)$ here and $\partial L/\partial z_1$ passes through unchanged. Finally, the first layer's weights and bias receive

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} x^\top = \begin{pmatrix} -0.2649 \\ 0.2208 \end{pmatrix} (1.0,\ 0.5) = \begin{pmatrix} -0.2649 & -0.1325 \\ 0.2208 & 0.1104 \end{pmatrix}, \qquad \frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z_1} = (-0.2649,\ 0.2208).$$

Every one of these four gradients was obtained purely by repeated application of the chain rule, working from the loss backward to the output layer, then backward again through the hidden layer, never once needing to re-derive the forward pass from scratch; this reuse of intermediate quantities computed once, moving backward, is precisely what makes backpropagation efficient enough to train networks with millions of parameters, rather than the (computationally infeasible) alternative of perturbing each weight individually and re-running the entire forward pass to estimate its gradient numerically.

A single gradient descent step with learning rate $\eta = 0.1$, the update rule Section 6.3.2 introduced, moves every parameter a small step opposite its gradient, $W_2 \leftarrow W_2 - \eta\, \partial L/\partial W_2$ and likewise for b_2, W_1, and b_1, nudging the network's prediction for this particular example fractionally closer to the target $y = 1$, and repeating this update across many training examples and many iterations is, in its entirety, how a deep network's weights are actually learned.

In finance, neural networks have been used for forecasting, option pricing, time series modeling, and text analysis. The appeal of deep learning is that it can automatically learn useful nonlinear representations from raw data, reducing the need for hand-crafted features. However, neural networks are not a magic solution. They often require substantial data, careful tuning of architecture and learning rate, and strong regularization (such as dropout or weight decay, which is mathematically equivalent to the ridge penalty introduced earlier) to avoid overfitting. They may also be more difficult to interpret than a linear model or a single decision tree, which matters when the model influences real-world, and sometimes regulated, decisions. For this reason, deep learning is often most useful when paired with domain knowledge and robust validation practices. A neural network that performs well in a laboratory setting may still fail in production if the data environment shifts or if the model is not monitored properly.

6.7 Challenges of Machine Learning in Finance

Several challenges make financial applications of machine learning harder than many textbook demonstrations suggest.

Low signal-to-noise ratio. Financial returns are dominated by noise relative to the size of any genuine predictive signal, unlike many image or language tasks where the target is essentially deterministic given the

Wulin.Suo@Queensu.ca

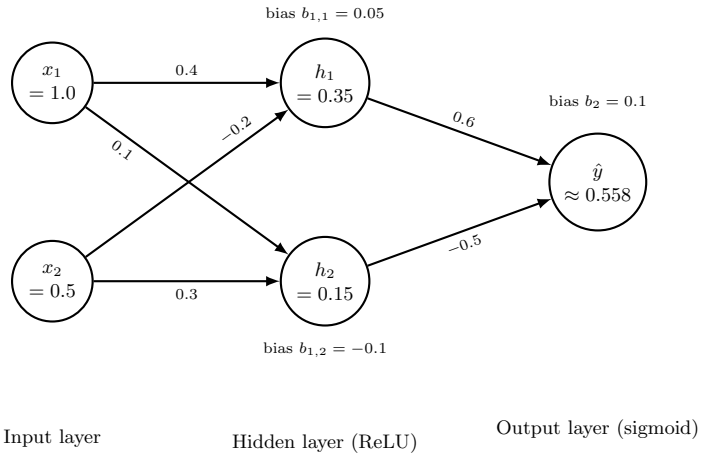


Figure 6.10: The toy feedforward network worked through above. Chapter 18 revisits it to verify LIME and integrated-gradients explanations against its exact analytical gradient.

input. An R^2 of a few percent can be economically meaningful for return prediction, whereas the same R^2 would be considered a failure in the clean regression example of Section 6.3.1.

Non-stationarity. The statistical relationships a model learns can change as markets evolve, as regulation changes, or as other participants adapt their own strategies. A model is effectively trained on the past while being asked to perform in a future that may not resemble it, which is a much harder problem than the standard machine learning assumption that training and test data are drawn from the same distribution.

Multiple testing and backtest overfitting. When researchers try many features, model specifications, or trading rules and report only the best-performing one, the reported performance is biased upward, sometimes dramatically. This is closely related to the overfitting problem in Section 6.4.1, but it operates across an entire research process rather than within a single model fit, and it is one of the most persistent sources of disappointing live performance after a promising backtest.

Imbalanced outcomes. Events such as default, fraud, or market crashes are rare relative to normal outcomes. A classifier that simply predicts "no event" can achieve high accuracy while being useless, which is why precision, recall, and related metrics from Section 6.3.3 matter more than accuracy in these settings. Practical remedies include resampling the training data (oversampling the rare class or undersampling the common one), assigning a higher misclassification cost to the rare class during training, and choosing a classification threshold explicitly based on the relative costs of false positives and false negatives for the specific decision at hand, rather than defaulting to the conventional 0.5 cutoff, which implicitly assumes the two types of error are equally costly.

Interpretability and regulatory scrutiny. Many financial applications, particularly credit decisions, are subject to regulation requiring that decisions be explainable to the affected individual and auditable by a regulator. This constrains the use of some highly flexible models, such as a large ensemble or a deep neural network, or requires supplementing them with explanation methods, a topic developed further in Chapter 18.

Feedback and reflexivity. Unlike physical systems, financial markets can react to the presence of a successful model: if a profitable pattern becomes known and widely traded, the trading activity itself can erode or eliminate the pattern. This means past predictive power is not simply a static property of the data, but can be altered by the deployment of the model itself.

Hyperparameter and model-selection overfitting. Section 6.4.3's grid-search workflow, if applied carelessly, is itself a source of overfitting: searching over many hyperparameter combinations or model families and reporting only the best cross-validated result inflates the reported performance in the same way as the multiple-testing problem above, just one level removed from the original features and trading rules.

Computational cost at scale. Some of the most powerful techniques in this chapter, an exhaustive hyperparameter grid search, a large gradient-boosted ensemble, a deep neural network, can be computation-

Wulin.Suo@Queensu.ca

ally expensive to train and retrain, which matters in finance because models often need to be refit frequently (daily or even intraday) to stay current with a non-stationary market, creating a real engineering constraint on top of the statistical ones.

These challenges do not make machine learning inapplicable to finance; they make careful validation, economically grounded evaluation, and ongoing monitoring non-negotiable parts of any deployment, rather than optional extras.

6.8 Key Takeaways

Machine learning offers valuable tools for prediction, classification, and pattern discovery in finance, but its power must be balanced with caution. The regression and classification examples in this chapter showed exactly how the standard formulas, ordinary least squares, logistic regression, the confusion matrix, and R^2, are computed, so that later chapters can build on precise foundations rather than vague intuition. Overfitting, unstable relationships, and shifting regimes all threaten performance in real-world settings, and the bias-variance decomposition gives a precise vocabulary for understanding why. The most successful applications combine statistical learning with domain knowledge, careful validation, and a clear understanding of the financial decision being made.

This chapter also introduced a genuine diversity of algorithmic approaches, parametric models fit by a closed-form solution (linear regression) or by gradient descent (logistic regression and, later, neural networks), non-parametric models that reason directly from stored examples (KNN), margin-based geometric classifiers (SVM), probabilistic classifiers built directly on Bayes' theorem (naive Bayes), and ensembles of simple trees combined either in parallel to reduce variance (random forests) or sequentially to reduce bias (gradient boosting), and it is worth remembering that no single one of these is universally best. The right choice depends on the size and structure of the data, the importance of interpretability (Chapter 18 returns to this directly), and the computational constraints of the deployment setting, which is why Section 6.4.3's disciplined, cross-validated approach to model and hyperparameter selection matters as much as knowing the algorithms themselves.

6.9 Suggested Exercises

1. Using the data in Table 6.1, verify by hand that $\hat{\beta}_1 = 47$ and $\hat{\beta}_0 = 20$, and compute the fitted spread for a firm with debt/EBITDA of 4.5.

2. Add a sixth observation, debt/EBITDA of 7 and a credit spread of 500 bps, to Table 6.1, refit the regression, and discuss how much the slope and R^2 change. What does this reveal about the sensitivity of OLS to outliers?

3. Using Table 6.3, suppose the lender lowers its classification threshold so that applicant 5 is now also predicted to default. Recompute the confusion matrix and all five metrics, and discuss how precision and recall trade off against each other.

4. Explain the difference between supervised and unsupervised learning, giving one financial example of each that is not already used in the chapter.

5. Derive, in your own words, why a ridge penalty with a very large λ drives all coefficients toward zero, and explain what this implies about the bias and variance of the resulting model.

6. Describe the bias-variance tradeoff using the formal decomposition in Section 6.4.2, and relate each of the three terms to a specific row of Table 6.6.

7. Why is walk-forward validation generally preferred over random k-fold cross-validation for financial time series?

8. Discuss one reason a model that performs well in-sample or in a backtest may fail out of sample or in live trading, drawing on at least two of the challenges described in Section 6.7 ("Challenges of Machine Learning in Finance").

Wulin.Suo@Queensu.ca

9. Starting from $\beta_0 = \beta_1 = 0$, manually compute the first gradient descent update (Section 6.3.2) for the credit-spread data using a learning rate of $\eta = 0.02$ instead of 0.01, and compare the resulting (β_0, β_1) to the first iteration shown in Table 6.2.

10. Using Table 6.5, classify a new applicant with a debt-to-income ratio of 22% and 6 years of credit history using $k = 1$ and again using $k = 5$, and explain why the two predictions differ.

11. Explain, in your own words, why a wider margin in a support vector machine is generally associated with better generalization to new data, drawing an analogy to the bias-variance tradeoff.

12. Choose one hyperparameter from any model in this chapter (for example, λ, k, or the number of boosting trees) and describe a grid of candidate values you would try, along with why searching too wide a grid could itself introduce overfitting risk, using Section 6.4.3 and Section 6.7's discussion of model-selection overfitting.

13. Explain why the naive independence assumption in naive Bayes is more clearly violated for the financial-ratio features used elsewhere in this chapter than for the word-count features used in text classification, and discuss whether you would still expect naive Bayes to be a reasonable baseline for a financial application.

14. Using this chapter's maximum-margin example, suppose a fifth point is added at $x = 3.5$ belonging to the "low-risk" class. Compute its functional margin under the original hyperplane ($w = 1$, $b = -3$), explain why this point must become a new support vector, and recompute the resulting hyperplane and margin width.

15. Using this chapter's ridge-versus-lasso worked example, explain in your own words why lasso's exact zero for the spurious feature's coefficient is a direct consequence of the penalty term's shape (a kink at zero), rather than simply a coincidence of this particular dataset, and describe what would need to be true of ridge's penalty for it to ever produce an exactly zero coefficient.

16. Using this chapter's backpropagation worked example, recompute $\partial L/\partial z_2$, $\partial L/\partial W_2$, and $\partial L/\partial b_2$ if the true label were $y = 0$ instead of $y = 1$ (all forward-pass values unchanged), and explain in words why the sign of every downstream gradient flips as a result.

17. **Challenge (real data).** Download the UCI Bank Marketing dataset (a Portuguese bank's telemarketing campaign, roughly 45,000 client contacts, with features such as age, job, account balance, and prior campaign outcomes, and a binary label for whether the client subscribed to a term deposit). (a) Fit both a logistic regression and a k-nearest-neighbors classifier (trying at least three values of k) to predict subscription on a held-out test split, and compare their accuracy, precision, and recall. (b) Using Section 6.4.2's bias-variance framework, explain which of your k values appears to underfit and which appears to overfit, based on how training and test accuracy diverge. (c) This dataset's subscription rate is only around 11%, far from the roughly balanced toy examples used elsewhere in this chapter; discuss, referencing this chapter's overfitting and validation material, why a model that reports 89% accuracy on this dataset could still be practically useless to the bank's marketing team.

6.10 Further Reading

- Hastie, Tibshirani, and Friedman, *The Elements of Statistical Learning*. The standard advanced reference for the statistical theory behind essentially every model in this chapter, including a much more rigorous treatment of the bias-variance decomposition and regularization.
- Murphy, *Machine Learning: A Probabilistic Perspective*. A thorough, probability-first treatment that develops naive Bayes, logistic regression, and neural networks within a unified Bayesian framework, extending the Bayes' theorem introduction from Chapter 1.
- James et al., *An Introduction to Statistical Learning*. A more accessible companion to Hastie, Tibshirani, and Friedman, with the same authors and worked examples in R and Python; a good starting point before attempting the more mathematically demanding treatment.

Wulin.Suo@Queensu.ca

- Hull et al., *Machine Learning in Business: An Introduction to the World of Data Science*. A business- and finance-oriented introduction to many of this chapter's models, by the same Hull whose derivatives and risk-management texts are cited in Chapters 1, 3, 5, 8, and 9, with less formal derivation and more emphasis on practical business use cases than the theory-first references above.
- de Prado, *Advances in Financial Machine Learning*. Focuses specifically on the financial-data pitfalls emphasized throughout this chapter, backtest overfitting, walk-forward validation, and feature construction for financial time series, in far greater depth than a single chapter allows.
- Cristianini and Shawe-Taylor, *An Introduction to Support Vector Machines*. A focused treatment of the margin-maximization and kernel-trick ideas introduced only briefly in this chapter.

Wulin.Suo@Queensu.ca

Wulin.Suo@Queensu.ca

Chapter 7

Time Series Analysis and Forecasting

7.1 Introduction

Almost every dataset encountered so far in this book, prices in Chapter 2, financial statement line items in Chapter 4, credit spreads in Chapter 5, has an implicit time dimension: observations arrive in a sequence, and that sequence usually carries information that a model ignoring order would throw away.

This chapter makes time itself part of the model. It introduces the statistical vocabulary for describing dependence across time, stationarity, autocorrelation, and volatility clustering, and the classical models built on that vocabulary: autoregressive and moving-average models, ARIMA, GARCH, and state-space models.

Two worked examples run through the chapter: a short, hand-computable interest-rate-deviation series used to fit and forecast an autoregressive model, and the AssetA return series first introduced in Chapter 2, reused here to illustrate how GARCH captures volatility clustering. Beyond these two running examples, the chapter also introduces formal stationarity testing, seasonal models, and two ways of extending single-series analysis to several related series at once, cointegration and vector autoregression.

Before closing, it returns to a promise made in Chapter 6: why cross-validation, as normally practiced in machine learning, must be modified for time series data.

The chapter follows the same order an analyst would actually work through a new series: first establish whether it is stationary (Section 7.2.3) and how strongly it depends on its own past (Section 7.2.4), then fit and forecast a model on that basis, quantifying the uncertainty around that forecast rather than reporting a single number (Section 7.3.1). The chapter then widens the lens from one series to several at once, cointegration (Section 7.4.1) for pairs that move together, and the Kalman filter (Section 7.5.2) for a state that evolves and is only noisily observed, before closing with the walk-forward validation discipline that Chapters 8 and 13 both depend on directly.

Nearly every later chapter in this book that touches a time-ordered quantity, GARCH-based volatility in Chapter 8, momentum and mean-reversion trading signals in Chapter 13, cites this chapter's vocabulary rather than re-deriving it.

7.2 Foundations of Time Series

Financial series violate the independence every earlier chapter assumed, and this part develops the vocabulary for saying how.

7.2.1 Why Time Series Analysis Matters in Finance

Financial data are overwhelmingly time series: daily prices, quarterly earnings, monthly inflation, tick-by-tick trades. Three features of financial time series make them harder to work with than the typical machine learning dataset. First, observations are not independent; today's return is statistically related to yesterday's, even if only weakly, which violates the independent-and-identically-distributed assumption behind many standard statistical procedures. Second, the variance of returns is itself time-varying, quiet

Wulin.Suo@Queensu.ca

periods are followed by quiet periods and volatile periods by volatile periods, a phenomenon called volatility clustering. Third, the underlying process generating the data can change over time (Chapter 6's non-stationarity concern), so a model fit on one period may not describe the next.

Time series methods exist precisely to handle these features rather than assume them away. They also underpin much of the rest of this book: the risk models in Chapter 8 need a volatility forecast, the portfolio methods in Chapter 12 need a covariance forecast, and the trading strategies in Chapter 13 need to know whether a signal is transient noise or a persistent pattern.

It is worth being explicit about what makes this chapter's models different from the regression and classification models of Chapter 6, even though both ultimately involve fitting parameters to minimize a loss function. A standard machine learning model assumes each training observation is drawn independently from the same underlying distribution as every other observation, which is precisely the assumption a time-ordered dataset violates: today's return is not independent of yesterday's, and a model that ignores this dependence, for example by randomly shuffling a time series before splitting it into training and test sets, will produce badly misleading performance estimates.

Every technique in this chapter, from the correlogram to GARCH to walk-forward validation, exists specifically to model, exploit, or correctly evaluate a model in the presence of this dependence, rather than assuming it away.

7.2.2 Stationarity, White Noise, and Random Walks

Suppose an analyst fits a seemingly solid model relating an interest rate series to its own past values, only to find the fitted relationship falls apart the moment new data arrives, not because the estimation was done incorrectly, but because the series itself was never behaving consistently enough to model in the first place. This section makes precise exactly what kind of consistency a time series needs for a fitted model to have any hope of generalizing. A time series $\{X_t\}$ is weakly stationary if its mean and variance do not change over time and the covariance between X_t and X_{t-k} depends only on the lag k, not on t itself:

$$\mathbb{E}[X_t] = \mu, \qquad \mathrm{Var}(X_t) = \sigma^2, \qquad \mathrm{Cov}(X_t, X_{t-k}) = \gamma_k \text{ for all } t.$$

Stationarity matters because most time series models, including every model in this chapter, are built on the assumption that the statistical relationships estimated from the past will continue to hold; without it, there is no reason to expect a fitted model to generalize at all.

White noise is the simplest stationary process: a sequence ε_t with mean zero, constant variance σ^2, and zero covariance at every nonzero lag, $\mathrm{Cov}(\varepsilon_t, \varepsilon_{t-k}) = 0$ for $k \neq 0$. White noise is unpredictable by construction and serves as the building block, the "noise," in every model that follows. A slightly stronger notion, strict stationarity, requires that the entire joint probability distribution of any collection of observations be unaffected by shifting them in time, not just their first two moments (mean and variance); weak stationarity, defined above, is what almost all of the practical models in this chapter actually rely on, since it is both easier to verify from data and sufficient for the estimation methods used throughout.

A random walk is $P_t = P_{t-1} + \varepsilon_t$, and it is not stationary: its variance grows with t (specifically $\mathrm{Var}(P_t) = t\sigma^2$ if P_0 is fixed), so its statistical behavior genuinely changes over time. Asset prices behave approximately like random walks, which is why Chapter 2 worked with returns, $R_t = (P_t - P_{t-1})/P_{t-1}$, rather than price levels: returns are much closer to stationary, since they do not inherit the ever-growing variance of the price level. This is not a minor technical preference; fitting a stationary-series model such as AR or ARMA directly to a non-stationary price level produces meaningless, unstable estimates, which is why the first step in almost any time series analysis is to check whether the series needs to be differenced (transformed to $\Delta X_t = X_t - X_{t-1}$) or converted to returns before modeling.

7.2.3 Testing for Stationarity: The Augmented Dickey-Fuller Test

Deciding whether a series is stationary should not rest on visual inspection alone; the standard formal test is the (augmented) Dickey-Fuller test. The test rewrites the AR(1) model

$$X_t = c + \phi X_{t-1} + \varepsilon_t$$

Wulin.Suo@Queensu.ca

in first-difference form,

$$\Delta X_t = \alpha + \gamma X_{t-1} + \varepsilon_t, \qquad \gamma = \phi - 1,$$

and tests the null hypothesis $H_0 : \gamma = 0$ (equivalent to $\phi = 1$, a unit root, meaning the series is a non-stationary random walk) against the alternative $H_1 : \gamma < 0$ (meaning $\phi < 1$, so the series is stationary and mean-reverting). The "augmented" version adds lagged difference terms

$$\Delta X_{t-1}, \Delta X_{t-2}, \ldots$$

to the right-hand side to account for additional autocorrelation, but the core logic, testing whether γ is significantly negative, is unchanged.

This section and the two that follow all draw on a single running example, given in Table 7.1: eight monthly observations of a short-term interest-rate deviation from its policy target, measured in basis points. The series is deliberately tiny, short enough that every calculation in it can be checked by hand; Section 7.2.4 computes its correlogram and Section 7.3.1 fits an AR(1) model to it.

Month t	1	2	3	4	5	6	7	8
Deviation X_t	9.0	8.0	7.0	7.0	5.0	6.0	4.0	4.0

Table 7.1: A short interest-rate deviation series (basis points)

Applying the (non-augmented) test to this series means regressing ΔX_t on X_{t-1}:

$$\hat{\gamma} = -0.290, \qquad SE(\hat{\gamma}) = 0.259, \qquad t\text{-statistic} = \frac{\hat{\gamma}}{SE(\hat{\gamma})} \approx -1.12.$$

Notice that $\hat{\gamma} = -0.290$ is consistent with the AR(1) coefficient fitted in Section 7.3.1, since

$$\hat{\phi} - 1 = 0.710 - 1 = -0.290$$

exactly. Critically, the Dickey-Fuller test does not use standard normal or t-distribution critical values, because under the null hypothesis of a unit root the test statistic has a nonstandard distribution; the relevant critical value at the 5% level (with a constant term and no trend) is approximately -2.89, not the familiar -1.96.

Since $|-1.12| < 2.89$, this test fails to reject the null hypothesis of a unit root, even though the AR(1) coefficient of 0.71 and the visibly decaying correlogram in Figure 7.1 both suggest genuine mean reversion. This apparent contradiction is not a flaw in the test; it is a direct consequence of the sample size: with only 8 observations, the Dickey-Fuller test has very little statistical power to distinguish a stationary process from a unit-root process, the "limited effective sample size" challenge flagged later in this chapter. In practice, the ADF test is typically applied to series with many more observations than this illustrative example, where its power to detect stationarity is much greater.

7.2.4 Autocorrelation and the Correlogram

Suppose an analyst has just concluded, from the ADF test above, that an interest-rate series is likely mean-reverting rather than a random walk, and now wants to know exactly how far back its memory extends: does today's value mainly reflect yesterday's, or does a shock from several periods ago still leave a detectable trace? The autocorrelation function (ACF) answers this directly by measuring how strongly a series is related to its own past values. At lag k, it is defined as

$$\rho_k = \frac{\gamma_k}{\gamma_0} = \frac{\mathrm{Cov}(X_t, X_{t-k})}{\mathrm{Var}(X_t)},$$

and is estimated from a sample of T observations with mean $\bar{X}$ as

$$\hat{\rho}_k = \frac{\sum_{t=k+1}^{T}(X_t - \bar{X})(X_{t-k} - \bar{X})}{\sum_{t=1}^{T}(X_t - \bar{X})^2}.$$

Wulin.Suo@Queensu.ca

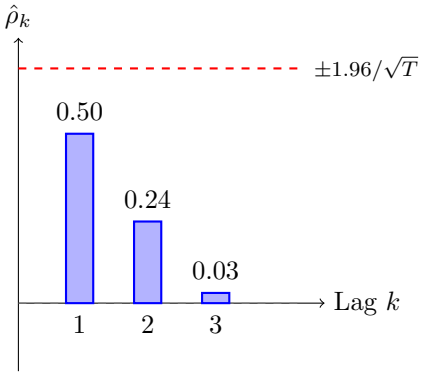


Figure 7.1: Sample correlogram for the interest-rate deviation series in Table 7.1, with the approximate 95% significance band $\pm 1.96/\sqrt{T}$ (dashed red; symmetric about zero, so only its positive side is drawn)

A plot of $\hat{\rho}_k$ against k is called a correlogram, and it is one of the most commonly used diagnostic tools in time series analysis: white noise has a correlogram that is statistically indistinguishable from zero at every lag, while a genuinely autocorrelated series shows a clear pattern, decaying gradually for an autoregressive process, cutting off sharply for a pure moving-average process. Figure 7.1 shows the correlogram computed from the worked example in the next section.

Because $\hat{\rho}_k$ is estimated from a finite sample, it is subject to sampling error, and a common rule of thumb treats $\hat{\rho}_k$ as statistically indistinguishable from zero if it falls within an approximate 95% confidence band of $\pm 1.96/\sqrt{T}$, where T is the number of observations used to compute the ACF. For the 8-observation series analyzed in the next section, this band is $\pm 1.96/\sqrt{8} \approx \pm 0.693$, which is larger in magnitude than the sample autocorrelation $\hat{\rho}_1 = 0.497$ computed there; Figure 7.1 plots this band directly alongside the correlogram.

In other words, even the largest, most visually striking autocorrelation in this chapter's own worked example is not, strictly speaking, statistically distinguishable from zero at conventional confidence levels, purely because the sample is so small. This is not a contradiction of the chapter's earlier claim that the correlogram shows a clear AR(1) pattern; rather, it is a reminder, consistent with the ADF test's failure to reject a unit root in Section 7.2.3, that with only 8 observations essentially no time series diagnostic can be fully trusted, and that the qualitative pattern (geometric decay) is more informative here than any individual significance test.

7.3 Forecasting Models

This part builds the classical forecasting families in order, from a single autoregressive term to the full ARIMA specification.

7.3.1 A Worked Example: Fitting and Forecasting an AR(1) Model

This section works through the full lifecycle of a small time series model: fitting it to data, checking that its correlogram behaves as theory predicts, and using it to produce both a point forecast and, later in the chapter, a sense of how that forecast's uncertainty grows with the horizon. An autoregressive model of order 1, AR(1), expresses the current value as a linear function of the immediately preceding value plus noise:

$$X_t = c + \phi X_{t-1} + \varepsilon_t, \qquad \varepsilon_t \sim \text{white noise}(0, \sigma^2).$$

The process is stationary if and only if $|\phi| < 1$, in which case its theoretical autocorrelation decays geometrically, $\rho_k = \phi^k$, and its long-run (unconditional) mean is $c/(1-\phi)$.

The data are the running series introduced in Section 7.2.3, the eight monthly interest-rate deviations of Table 7.1.

Wulin.Suo@Queensu.ca

The sample autocorrelations, computed directly from the formula above using the full-sample mean $\bar{X} = 6.25$, are $\hat{\rho}_1 = 0.50$, $\hat{\rho}_2 = 0.24$, and $\hat{\rho}_3 = 0.03$, plotted in Figure 7.1. The roughly geometric decay, each lag's autocorrelation is a bit less than half the previous one, is the signature an AR(1) process should produce.

The AR(1) coefficients are estimated the same way any linear regression is estimated, by regressing X_t on X_{t-1} using the OLS formulas from Chapter 6:

$$\hat{\phi} = \frac{\sum_t (X_{t-1} - \bar{X}_{\text{lag}})(X_t - \bar{X}_{\text{cur}})}{\sum_t (X_{t-1} - \bar{X}_{\text{lag}})^2}, \qquad \hat{c} = \bar{X}_{\text{cur}} - \hat{\phi}\,\bar{X}_{\text{lag}},$$

where $\bar{X}_{\text{lag}}$ and $\bar{X}_{\text{cur}}$ are the means of the lagged and current values, respectively (each computed over the 7 available pairs, so they differ slightly from the full-sample mean $\bar{X} = 6.25$ used for the correlogram). Fitting this to Table 7.1 gives

$$\hat{\phi} = 0.71, \qquad \hat{c} = 1.19, \qquad R^2 = 0.60.$$

Since $|\hat{\phi}| = 0.71 < 1$, the fitted process is stationary, with an implied long-run mean of $\hat{c}/(1 - \hat{\phi}) = 1.19/0.29 \approx 4.11$ basis points. Notice that this is noticeably different from the raw sample mean of 6.25: with only 8 observations, the AR(1) coefficient estimate carries substantial small-sample uncertainty, and the model-implied long-run mean should be treated as no more precise than the data that produced it. The one-step-ahead forecast, given the final observation $X_8 = 4.0$, is

$$\hat{X}_9 = \hat{c} + \hat{\phi}\,X_8 = 1.19 + 0.71(4.0) \approx 4.03 \text{ basis points.}$$

This is the essential logic of every autoregressive forecast: today's fitted relationship, applied to the most recent observation, produces tomorrow's point forecast.

Confidence Intervals and p-Values for Model Parameters

A point estimate such as $\hat{\phi} = 0.71$ is only half the story; without some sense of how precisely $\hat{\phi}$ is known, it is impossible to say whether the fitted mean reversion is a genuine feature of the process or an artifact of a short, noisy sample.

Chapter 6's Section 6.3.1 built the standard-error, t-statistic, p-value, and confidence-interval machinery for this purpose, including the correct (and the common incorrect) ways to state what a confidence interval means; that machinery applies here unchanged, since $\hat{\phi}$ is itself just an OLS slope, of X_t on X_{t-1}, so this section applies the method rather than re-deriving it. For the AR(1) regression fitted above, with $n = 7$ usable pairs and 2 estimated parameters (so $n - 2 = 5$ residual degrees of freedom),

$$SE(\hat{\phi}) = \sqrt{\frac{\hat{\sigma}^2}{\sum_t (X_{t-1} - \bar{X}_{\text{lag}})^2}} \approx 0.259, \qquad \hat{\sigma}^2 = \frac{\text{SSR}}{n-2} \approx 1.187, \qquad t = \frac{\hat{\phi}}{SE(\hat{\phi})} = \frac{0.71}{0.259} \approx 2.74,$$

where SSR is the sum of squared residuals from the fitted line. Under the null hypothesis $H_0 : \phi = 0$ (no mean reversion at all, this month's deviation carries no information about next month's), this t-statistic follows a t-distribution with 5 degrees of freedom and gives $p \approx 0.041$: by the conventional 5% threshold, $\hat{\phi} = 0.71$ is (barely) significant, so the data offer modest evidence of genuine mean reversion, but the margin is thin. The corresponding 95% confidence interval,

$$\hat{\phi} \pm t^*_{0.975,\,5} \cdot SE(\hat{\phi}) = 0.71 \pm (2.571)(0.259) \approx (0.04,\ 1.38),$$

is wide, spanning nearly the entire plausible range from very weak to very strong mean reversion, and includes values close to 1 (the unit-root boundary).

This is the same small-sample fragility flagged when the Dickey-Fuller test in Section 7.2.3 failed to reject a unit root despite $\hat{\phi} = 0.71$ visually suggesting real mean reversion: a t-statistic, a p-value, and a confidence interval are three views of the same underlying estimation uncertainty, and with only 8 observations, that uncertainty is simply large. Applying the identical method to the intercept gives $SE(\hat{c}) \approx 1.750$, $t \approx 0.68$, and $p \approx 0.526$, so the intercept is not statistically distinguishable from zero at conventional significance levels, a much weaker result than the slope's.

Wulin.Suo@Queensu.ca

Chapter 6's Section 6.3.1 already flagged the two practical warnings that apply here without modification: statistical significance is not the same as economic or financial significance, and testing many coefficients or candidate models against the same data inflates the chance that at least one spuriously crosses the 5% threshold by pure luck. That second warning is the multiple-testing problem Section 7.4.1's discussion of pairs selection revisits concretely, fulfilling the forward pointer Chapter 6 itself made to this section: a single p-value below 0.05 is far less persuasive when it is the best of twenty candidate pairs tried rather than the only pair considered.

7.3.2 Multi-Step Forecasting and the Growth of Uncertainty

The one-step-ahead forecast above answers "what happens next period," but many financial decisions require forecasting several periods ahead. For an AR(1) model, this is done recursively: use the one-step forecast $\hat{X}_9$ as if it were the actual next observation to produce $\hat{X}_{10} = \hat{c} + \hat{\phi}\hat{X}_9$, and continue in the same way for as many steps as needed. Applying this to the fitted model ($\hat{\phi} = 0.71$, $\hat{c} = 1.19$) starting from $X_8 = 4.0$ gives the forecasts in Table 7.2.

Horizon h	1	2	3	5	10
Forecast $\hat{X}_{8+h}$	4.03	4.06	4.07	4.09	4.11

Table 7.2: Multi-step-ahead AR(1) forecasts from month 8

Two features of Table 7.2 illustrate general properties of stationary autoregressive forecasts. First, the forecasts converge smoothly toward the model's long-run mean of $\hat{c}/(1-\hat{\phi}) \approx 4.11$ as the horizon grows, rather than continuing to track short-run fluctuations; a stationary model's best guess about the distant future is essentially just its unconditional mean, since any information in today's observation about deviations from that mean fades geometrically at rate $\hat{\phi}$ per period.

Second, the uncertainty around these forecasts also grows with the horizon but, unlike a random walk, does not grow without bound: the variance of the h-step forecast is proportional to $\sum_{j=0}^{h-1} \hat{\phi}^{2j}$, which converges to $1/(1-\hat{\phi}^2) \approx 2.01$ times the one-step forecast variance as $h \to \infty$, rather than growing linearly with h as it would for a non-stationary series. This is a direct, quantitative version of the qualitative warning in this chapter's challenges section that longer-horizon forecasts are less reliable than one-step forecasts, and it shows precisely how much less reliable, and why that unreliability eventually levels off, for a genuinely mean-reverting series.

Diagnosing the Fit: The Ljung-Box Test and Choosing Between AR(1) and AR(2)

Fitting a model is not the end of the process; the residuals should be checked for leftover structure the model failed to capture, the "did this actually work" question the Box-Jenkins methodology introduced later in this chapter is built around. The Ljung-Box test formalizes this check: it tests the joint null hypothesis that the first h residual autocorrelations are all zero, meaning an adequately specified model should leave behind nothing but white noise, using the statistic

$$Q = n(n+2)\sum_{k=1}^{h} \frac{\hat{\rho}_k^2}{n-k},$$

where $\hat{\rho}_k$ is the residuals' own sample autocorrelation at lag k (computed just as in Section 7.2.4, but applied to the residuals rather than the raw series) and n is the number of residuals. Under the null hypothesis of a correctly specified model, Q follows a chi-squared distribution with $h - p - q$ degrees of freedom, where p and q are the number of AR and MA parameters already estimated, subtracted because fitting those parameters already accounts for some of the dependence the test would otherwise attribute to unmodeled autocorrelation.

Applying this to the AR(1) model fitted above ($\hat{\phi} = 0.71$, $\hat{c} = 1.19$), the seven residuals are

$$(0.42, 0.13, 0.84, -1.16, 1.26, -1.45, -0.03),$$

Wulin.Suo@Queensu.ca

giving residual autocorrelations $\hat{\rho}_1 = -0.68$ and $\hat{\rho}_2 = 0.49$, both considerably larger in magnitude than a well-fitting model's residuals should show. With $h = 2$ lags and $p = 1$ AR parameter, $Q = 7.91$ against a chi-squared distribution with $2 - 1 = 1$ degree of freedom, giving $p \approx 0.005$, comfortably below the conventional 5% threshold. The AR(1) model's residuals, in other words, are not well described as white noise: some genuine structure remains unmodeled.

This motivates asking whether a higher-order model captures more of that structure. Fitting an AR(2) model,

$$X_t = c + \phi_1 X_{t-1} + \phi_2 X_{t-2} + \varepsilon_t,$$

to the same six usable observations ($t = 3, \ldots, 8$; the sample must shrink by one further observation to accommodate the second lag, and the AR(1) model is refit on this identical six-observation sample so the two models' fit statistics are directly comparable) gives $\hat{\phi}_1 = 0.03$, $\hat{\phi}_2 = 0.88$, and a residual sum of squares of only 1.39, versus 5.60 for the AR(1) model refit on the same six observations, a dramatic improvement. Comparing the two models with the information criteria introduced in the ARIMA section below,

$$\text{AIC} = n \ln(\text{SSR}/n) + 2k, \qquad \text{BIC} = n \ln(\text{SSR}/n) + k \ln n,$$

with $n = 6$ throughout ($k = 2$ and $k = 3$ parameters for AR(1) and AR(2), respectively), gives

$$\begin{aligned} \text{AIC}_{\text{AR(1)}} &= 3.59, & \text{AIC}_{\text{AR(2)}} &= -2.76, \\ \text{BIC}_{\text{AR(1)}} &= 3.17, & \text{BIC}_{\text{AR(2)}} &= -3.39 : \end{aligned}$$

both criteria favor AR(2) by a wide margin, even after the extra parameter's complexity penalty. Yet this conclusion should be read with real caution rather than taken as a settled upgrade: an AR(2) model on only six observations is already using half of the sample's data points as estimated parameters ($k = 3$ against $n = 6$), precisely the "limited effective sample size" and "model selection versus overfitting" concerns raised later in this chapter's challenges section, so a dramatically lower AIC/BIC here is at least as likely to reflect a small sample's capacity to fit its own noise as it is a genuinely better-specified model. In a longer series, this same Ljung-Box-then-AIC/BIC workflow, fit a candidate model, test whether its residuals are white noise, and if not, compare a higher-order alternative using an information criterion, remains standard practice; it is only the tiny sample size here that makes the specific numerical conclusion (prefer AR(2)) less trustworthy than the procedure itself.

7.3.3 Moving Averages and Exponential Smoothing

Before turning to ARIMA and GARCH, it is worth introducing two much simpler forecasting tools that remain widely used because of their transparency. A simple moving average forecast uses the average of the most recent m observations as the forecast for the next period,

$$\hat{X}_{t+1} = \frac{1}{m} \sum_{i=0}^{m-1} X_{t-i}.$$

It is easy to compute and understand, but it weights all m past observations equally and drops old observations abruptly once they fall outside the window.

Exponential smoothing addresses both issues by weighting all past observations, but with exponentially decaying weight on older data:

$$\hat{X}_{t+1} = \alpha X_t + (1 - \alpha)\hat{X}_t, \qquad 0 < \alpha < 1,$$

which can be expanded to show that $\hat{X}_{t+1}$ is a weighted average of every past observation, with weight $\alpha(1 - \alpha)^i$ on the observation i periods back. A larger α makes the forecast react quickly to new information at the cost of more noise; a smaller α produces a smoother but slower-reacting forecast. Applied to Table 7.1 with $\alpha = 0.4$ and an initial forecast $\hat{X}_1 = X_1 = 9.0$, the recursion produces a smoothed forecast that tracks the series' gradual decline without reacting sharply to any single observation, falling from 9.0 to a forecast for the next month of $\hat{X}_9 = 0.4(4.0) + 0.6(5.396) \approx 4.84$, in contrast to the AR(1) model, which explicitly ties the forecast to the estimated mean-reversion coefficient $\hat{\phi}$ rather than only to recent history.

Wulin.Suo@Queensu.ca

Simple exponential smoothing has an important limitation: because it forecasts using only a smoothed level, it always predicts a flat, unchanging line into the future, which performs poorly for a series with a clear trend, the kind of gradual decline visible in Table 7.1. Holt's linear trend method addresses this by smoothing two components separately, a level L_t and a trend T_t:

$$L_t = \alpha X_t + (1-\alpha)(L_{t-1} + T_{t-1}),$$
$$T_t = \beta(L_t - L_{t-1}) + (1-\beta)T_{t-1},$$

and forecasts h steps ahead as $\hat{X}_{t+h} = L_t + hT_t$, explicitly projecting the estimated trend forward rather than assuming it disappears. Applying this to Table 7.1 with $\alpha = 0.4$, $\beta = 0.3$, an initial level $L_1 = 9.0$, and an initial trend $T_1 = X_2 - X_1 = -1.0$, the recursion, updated once for each of X_2 through X_8, produces a final level of $L_8 \approx 3.631$ and trend of $T_8 \approx -0.706$, giving a one-step-ahead forecast of

$$\hat{X}_9 = L_8 + T_8 \approx 3.631 - 0.706 \approx 2.93.$$

This is noticeably lower than both the simple exponential smoothing forecast (about 4.84, computed above) and the AR(1) forecast (4.03, computed in Section 7.3.1), because Holt's method explicitly extrapolates the series' downward trend rather than assuming the series will revert toward a stable mean, as the AR(1) model does, or stay flat, as simple exponential smoothing does. Which forecast is more appropriate depends entirely on whether the analyst believes the series is genuinely trending or is instead a mean-reverting process passing through a temporarily low patch, a judgment call that no single mechanical formula can resolve and that connects directly back to the stationarity testing in Section 7.2.3: a trending (non-stationary) series is better suited to Holt's method, while a mean-reverting (stationary) series is better suited to the AR(1) approach. A further extension, Holt-Winters smoothing, adds a third seasonal component to handle the kind of periodic patterns discussed in the SARIMA section below.

7.3.4 ARIMA Models

Suppose a shock, a one-time data-reporting glitch, say, appears to influence a series for exactly one additional period before vanishing completely, rather than decaying away gradually forever the way the AR(1) model above implies. The AR(1) model in Section 7.3.1 is a special case of a much broader family built to capture this kind of finite-memory dependence, among others. A moving-average model of order q, MA(q), expresses the current value as a function of recent noise terms rather than recent levels,

$$X_t = \mu + \varepsilon_t + \theta_1\varepsilon_{t-1} + \cdots + \theta_q\varepsilon_{t-q},$$

and combining autoregressive and moving-average terms gives an ARMA(p, q) model. Because many financial series (prices, but also many macroeconomic series) are non-stationary in levels but stationary after differencing, the ARIMA(p, d, q) model applies an ARMA(p, q) model to the series after differencing d times: $d = 1$ removes a linear trend, $d = 2$ removes a quadratic trend, and so on. In practice, d is rarely more than 1 or 2 for financial data, since returns (a $d = 1$ difference of log prices) are already close to stationary.

The MA(q) component has a theoretical autocorrelation structure that is the mirror image of the AR(p) process introduced earlier in this chapter, and this contrast is precisely what makes the ACF and PACF useful diagnostic tools together. For an MA(1) process $X_t = \mu + \varepsilon_t + \theta\varepsilon_{t-1}$, the theoretical autocorrelation is $\rho_1 = \theta/(1+\theta^2)$ and $\rho_k = 0$ for every lag $k > 1$; for $\theta = 0.6$, this gives $\rho_1 = 0.6/1.36 \approx 0.441$ and then an abrupt drop to zero at lag 2, in sharp contrast to the AR(1) correlogram in Figure 7.1, whose autocorrelations decay gradually across every lag rather than cutting off sharply. This is the basis of the Box-Jenkins identification rule: a correlogram (ACF) that cuts off sharply after lag q suggests an MA(q) model, while a partial autocorrelation function that cuts off sharply after lag p suggests an AR(p) model; a series where both decay gradually suggests a mixed ARMA(p, q) model, for which the specific orders are typically chosen by comparing information criteria across a small number of candidates rather than by reading the correlogram alone.

Choosing the orders p, d, and q is traditionally done with the Box-Jenkins methodology: examine the ACF and its partial-autocorrelation counterpart (the PACF, which measures the correlation between X_t and X_{t-k} after removing the influence of the intervening lags) to identify candidate orders, fit several candidate models,

Wulin.Suo@Queensu.ca

and compare them using an information criterion such as AIC or BIC, which penalize model complexity to avoid simply picking the model with the most parameters. This is the time series analogue of the model-selection problem in Chapter 6: more lags will always fit the training data at least as well, but a model with too many lags overfits sample noise exactly as a high-degree polynomial does.

Many financial and economic series also exhibit seasonality, a repeating pattern at a fixed period, such as retail sales spiking every December or trading volume dipping every summer. SARIMA (Seasonal ARIMA) extends ARIMA by adding a second set of autoregressive, differencing, and moving-average terms operating at the seasonal period s, denoted SARIMA$(p,d,q)(P,D,Q)_s$: the non-seasonal part (p,d,q) captures short-run dynamics exactly as before, while the seasonal part $(P,D,Q)_s$ captures dependence at lags $s, 2s, 3s, \ldots$. For example, a monthly retail sales series might be modeled as SARIMA$(1,1,1)(1,1,1)_{12}$, where the seasonal terms specifically model the relationship between each month and the same month one year ($s = 12$ months) earlier, on top of the ordinary month-to-month dynamics the non-seasonal terms capture.

Seasonal differencing, $\Delta_s X_t = X_t - X_{t-s}$, is the seasonal analogue of the ordinary differencing introduced above, and it is applied when a series exhibits a seasonal unit root, an extension of the stationarity testing in Section 7.2.3 to the seasonal frequency. Most asset return series in finance show little seasonality at the daily frequency, which is why the GARCH and ARIMA examples elsewhere in this chapter do not require a seasonal extension, but seasonality is common and important in macroeconomic series (retail sales, employment, seasonally-adjusted GDP) that feed into the fundamental analysis of Chapter 4 and the macroeconomic risk factors of Chapters 8 and 9.

7.4 Multivariate Methods

One series is rarely the whole story; this part models several jointly, and extracts the few factors that drive them.

7.4.1 Cointegration and Pairs Trading

Section 7.2.3 showed that individual price series are typically non-stationary. A remarkable exception arises when two non-stationary series move together closely enough that a specific linear combination of them is stationary, a relationship called cointegration. Two stocks in the same industry, for example, might each individually wander like a random walk, yet the spread between their prices could still revert reliably to a stable long-run relationship, because both are driven by the same industry-wide fundamentals.

Table 7.3 shows six days of prices for two hypothetical bank stocks.

Day	1	2	3	4	5	6
Stock A	50	51	53	52	54	55
Stock B	48	49	50	50	51	52
Spread $(A - B)$	2	2	3	2	3	3

Table 7.3: Prices for two hypothetical cointegrated bank stocks

Both individual price series trend upward and would likely fail a stationarity test on their own, exactly like the random walk introduced earlier in this chapter. The spread, however, fluctuates narrowly around a mean of $\bar{s} = 2.5$ with a sample standard deviation of only 0.55, far more stable than either underlying price series. The standard procedure for testing this formally, the Engle-Granger two-step method, first regresses one price on the other to estimate the cointegrating relationship,

$$A_t = \alpha + \beta B_t + u_t,$$

which for this data gives $\hat{\alpha} = -12.5$ and $\hat{\beta} = 1.3$, and then applies the Dickey-Fuller test from Section 7.2.3 to the regression residuals $\hat{u}_t$: if the residuals reject the unit-root null (that is, they are stationary), the two series are cointegrated. One detail matters here: the test statistic must be compared with Engle-Granger critical values (tabulated by MacKinnon), not with the -2.89 used for a single observed series. Because $\hat{\beta}$ was chosen by least squares to make the residuals as small and stable as possible, the residuals look more

Wulin.Suo@Queensu.ca

stationary than an arbitrary series would, so the bar is higher: about -3.34 at the 5% level for two series in large samples, and more negative still in short ones.

Note that this regression-based approach generalizes the simple $A_t - B_t$ spread used above (which implicitly assumes $\beta = 1$) to an estimated hedge ratio $\hat{\beta}$, the number of shares of B needed to offset one share of A; the simple difference is a reasonable approximation only when the two series move roughly one-for-one. Here $\hat{\beta} = 1.3$ is close enough to 1 for the raw spread in Table 7.3 to look stable, but, as the z-score below shows, not close enough to trade on.

This relationship is the foundation of pairs trading, a strategy covered in more depth in Chapter 13. In practice, the spread is standardized into a z-score, $z_t = (s_t - \bar{s})/\hat{\sigma}_s$, so that trading rules can be stated in terms of standard deviations rather than the spread's raw units, which vary from one pair to the next. Having estimated the hedge ratio, the spread to standardize is the residual $\hat{u}_t = A_t - \hat{\alpha} - \hat{\beta}B_t$, not the $\beta = 1$ difference $A_t - B_t$. For the data in Table 7.3 the residuals are $0.1, -0.2, 0.5, -0.5, 0.2, -0.1$, with mean zero (as least-squares residuals always have) and sample standard deviation 0.346, so the final residual has a z-score of $z_6 = (-0.1 - 0)/0.346 \approx -0.29$. (Standardizing the raw difference instead would give $(3 - 2.5)/0.548 \approx 0.91$, even the opposite sign, which shows how much the choice of hedge ratio matters.) A common rule of thumb opens a trade when $|z_t|$ exceeds roughly 2 (signaling an unusually wide or narrow spread) and closes it once z_t reverts close to zero, so this spread, less than a third of a standard deviation from its equilibrium, would not trigger a new trade under such a rule.

When the spread is unusually wide relative to its historical mean and standard deviation, a trader shorts the relatively expensive stock (here, A) and buys $\hat{\beta}$ shares of the relatively cheap one (B) for every share of A, betting that the spread will revert toward its mean; when the spread is unusually narrow, the trader takes the opposite position. Because the strategy trades the spread rather than the direction of either stock, it is designed to profit regardless of whether the overall market rises or falls, a property called market neutrality, provided the cointegrating relationship itself remains stable, which is the kind of assumption that can silently break down, a risk explored further when Chapter 13 discusses the practical risks of statistical arbitrage strategies.

Pairs selection is also where the multiple-testing problem flagged in Section 7.3.1 bites hardest. A trader rarely tests one pair; a more typical screen runs the Engle-Granger test on every pair in a universe of, say, 20 bank stocks, which is $\binom{20}{2} = 190$ pairs. If not a single pair were truly cointegrated, a 5% test would still flag about $190 \times 0.05 = 9.5$ of them by chance, and those false positives are exactly the pairs whose spreads will stop mean-reverting once real money is on them. Two standard defenses are the Bonferroni correction, which requires each individual test to clear $0.05/190 \approx 0.00026$ so that the chance of even one false positive stays below 5%, and the less conservative false discovery rate procedure of Benjamini and Hochberg, which instead controls the expected fraction of flagged pairs that are false. Neither replaces an economic reason to expect the relationship (same industry, same funding model, same regulator) and a test on data the screen never saw.

7.4.2 Vector Autoregression: Modeling Several Series Jointly

Every model so far in this chapter describes a single series in terms of its own past. A vector autoregression (VAR) extends the AR(p) framework to several series simultaneously, allowing each variable to depend on its own past values and the past values of every other variable in the system. A VAR(1) model for two series X_t and Y_t is

$$X_t = c_1 + a_{11}X_{t-1} + a_{12}Y_{t-1} + \varepsilon_{1,t},$$
$$Y_t = c_2 + a_{21}X_{t-1} + a_{22}Y_{t-1} + \varepsilon_{2,t},$$

where the off-diagonal coefficients a_{12} and a_{21} capture cross-series dependence, for example how last month's interest rate might help predict this month's exchange rate, and vice versa. This is a natural way to model interconnected macroeconomic and financial series, such as interest rates, exchange rates, and inflation, together, rather than modeling each with a separate, univariate ARIMA model that ignores their interdependence.

A practically convenient feature of the VAR(1) system above is that, conditional on the lag order, each equation can be estimated separately by ordinary least squares, the same OLS machinery introduced in

Wulin.Suo@Queensu.ca

Chapter 6, simply regressing X_t on X_{t-1} and Y_{t-1}, and separately regressing Y_t on the same two lagged variables. Choosing the number of lags to include (extending the model to VAR(p) for $p > 1$) follows the same information-criterion logic, AIC or BIC, used to select ARIMA orders earlier in this chapter, since a VAR with too many lags relative to the amount of data will overfit just as readily as an over-parameterized univariate model.

VAR models are also the standard tool for testing Granger causality: variable X is said to "Granger-cause" variable Y if past values of X improve the prediction of Y beyond what past values of Y alone provide, formally tested by checking whether the coefficients on lagged X in the Y equation (here, a_{21}) are statistically significant.

Granger causality is a statement about predictive usefulness, not causality in the deeper structural sense introduced in Chapter 1's discussion of correlation versus causation; a variable can Granger-cause another simply because it reflects the same information sooner, without one variable structurally driving the other at all.

Once a VAR is fitted, an impulse response function traces out how a one-time shock to one variable propagates through the system over subsequent periods: starting from the estimated coefficients, a hypothetical one-unit shock is applied to $\varepsilon_{1,t}$ (holding $\varepsilon_{2,t} = 0$), and the model is iterated forward to see how both $X_{t+1}, X_{t+2}, \ldots$ and $Y_{t+1}, Y_{t+2}, \ldots$ respond over time, tracing the shock's full dynamic effect on both series rather than just its immediate, one-period impact.

This is a standard tool in macroeconomic policy analysis, for example tracing how a shock to short-term interest rates propagates to exchange rates and, eventually, inflation, and it is the multivariate analogue of the multi-step forecasting exercise carried out for the single-series AR(1) model earlier in this chapter. VAR models are widely used in macroeconomic forecasting and in studying how shocks propagate between markets, a natural multivariate extension of the single-series volatility and mean-reversion models developed throughout the rest of this chapter.

A concrete impulse response makes this propagation mechanism tangible. Suppose a VAR(1) has already been fitted (via the equation-by-equation OLS procedure above) to changes in a short-term interest rate X_t and an exchange rate Y_t, with estimated coefficients $a_{11} = 0.6$, $a_{12} = 0.2$, $a_{21} = 0.1$, and $a_{22} = 0.7$ (intercepts omitted, since an impulse response tracks the deviation from baseline rather than the level). A one-unit shock to the interest rate at time 0, with the exchange rate unshocked ($X_0 = 1$, $Y_0 = 0$), propagates as

$$X_1 = 0.6(1) + 0.2(0) = 0.6, \qquad Y_1 = 0.1(1) + 0.7(0) = 0.1,$$

$$X_2 = 0.6(0.6) + 0.2(0.1) = 0.38, \qquad Y_2 = 0.1(0.6) + 0.7(0.1) = 0.13,$$

and continuing this recursion gives

$$X_3 = 0.254, \qquad Y_3 = 0.129, \qquad X_4 = 0.178, \qquad Y_4 = 0.116.$$

Two features of this impulse response are worth noting. First, even though the exchange rate was not shocked directly, it responds immediately, rising from 0 to 0.1 in a single period, purely through the cross-series coefficient $a_{21} = 0.1$; this is the kind of spillover a collection of separate, univariate ARIMA models, each blind to the other series, could never capture. Second, the exchange rate's response does not simply fade monotonically alongside the interest rate's own decay: it initially rises further, from 0.1 to 0.13, before declining, because it continues to absorb the (still-elevated) interest rate for a second period even as the direct shock itself has already started to fade, a lagged-peak pattern that is a hallmark signature of genuine cross-series transmission rather than each series simply reverting independently.

Both series in this example do eventually decay back toward zero, which is not guaranteed by the individual coefficients alone; it depends on the stability of the whole coefficient matrix

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

jointly. The multivariate generalization of the univariate AR(1) stationarity condition $|\phi| < 1$ is that every eigenvalue of A must lie strictly inside the unit circle. For this system, A's eigenvalues are exactly 0.5 and 0.8, both comfortably below 1 in absolute value, confirming that the impulse response above must decay to zero eventually rather than explode, exactly as the numerical values $0.6, 0.38, 0.254, 0.178$ for X and $0.1, 0.13, 0.129, 0.116$ for Y already suggested.

Wulin.Suo@Queensu.ca

A VAR fitted to real, non-stationary financial series (interest rate levels rather than changes, for example) can easily produce a coefficient matrix with an eigenvalue at or above 1, in which case the whole system is unstable and any impulse response or multi-step forecast built from it, however precisely computed, describes an explosive process rather than a meaningful economic dynamic. This is the direct multivariate analogue of the unit-root problem the Dickey-Fuller test in Section 7.2.3 is designed to detect in a single series, and it is why a VAR, like an ARIMA model, is normally fit to differenced or otherwise stationary series rather than to raw levels.

7.4.3 A Worked Example: PCA and Forecasting the Term Structure of Interest Rates

The VAR framework above handles two or three interconnected series comfortably, but Chapter 5's yield curve is not two series, it is a whole curve, one yield for every maturity a government issues debt at. Modeling eight or more maturities as a single high-dimensional VAR is possible in principle but wasteful in practice, because adjacent points on a yield curve are extremely highly correlated: the 2-year and 3-year yields, for instance, almost always move together.

Chapter 6 introduced principal component analysis (PCA) as the standard dimensionality-reduction method, finding the direction w that maximizes $\mathrm{Var}(Xw)$; applied to a panel of yield curve maturities rather than a panel of asset returns, PCA turns this redundancy into a genuine forecasting advantage, since a small number of extracted factors can carry nearly all of the curve's variation.

This worked example uses real data: the U.S. Treasury's daily par yield curve rates from January 2, 1990 through December 29, 2023, restricted to the eight maturities with essentially complete history over that period (3-month, 6-month, 1-year, 2-year, 3-year, 5-year, 7-year, and 10-year), giving 8,503 daily observations. Applying PCA to this panel, mean-centered but not rescaled to unit variance (so that maturities with naturally larger yield swings are not artificially shrunk to match calmer ones), gives the explained variance ratios in Table 7.4.

Component	PC1	PC2	PC3	PC4	PC5
Variance explained	95.95%	3.78%	0.22%	0.03%	0.01%

Table 7.4: Explained variance ratio by principal component, 1990–2023 Treasury yield curve panel

The first three components alone explain 99.96% of the total variation across 34 years of daily yield curve moves, a striking result: an eight-dimensional forecasting problem collapses, with almost no loss of information, to a three-dimensional one. This is not a coincidence specific to this dataset; it is one of the most reproduced empirical results in fixed income, first documented by Litterman and Scheinkman (1991), and the three components have well-known, economically interpretable shapes once their loadings, the weight each maturity contributes to that component, are plotted against maturity.

Figure 7.2 shows this level-slope-curvature structure. PC1's loadings range only from 0.30 to 0.38 across all eight maturities, all positive and all close together: a positive PC1 shock raises every yield on the curve by roughly the same amount, a parallel shift, which is why PC1 is called the *level* factor. PC2's loadings move monotonically from -0.42 at the 3-month maturity to $+0.53$ at the 10-year maturity, so a positive PC2 shock lowers short-term yields while raising long-term ones, steepening the curve; this is the *slope* factor.

PC3's loadings are positive at both ends of the curve (0.52 at 3 months, 0.45 at 10 years) and negative in the middle (-0.46 at both 2 and 3 years), a hump-shaped pattern called the *curvature* factor, since a positive PC3 shock bends the belly of the curve down relative to both ends.

Because level, slope, and curvature are themselves just time series, each extracted by projecting every day's curve onto its corresponding loading vector, they can be forecast with the same univariate tools developed earlier in this chapter, then mapped back to a curve-wide forecast through the loadings. Consider the dominant factor, PC1. Testing its level directly with the Augmented Dickey-Fuller test of Section 7.2.3 gives a test statistic of -2.295 ($p = 0.174$), failing to reject the unit-root null: the level factor, like the individual stock prices of Section 7.4.1 and the raw interest-rate VAR discussed above, is itself non-stationary and cannot be fed directly into an AR model. Differencing it once and re-testing gives a test statistic of

Wulin.Suo@Queensu.ca

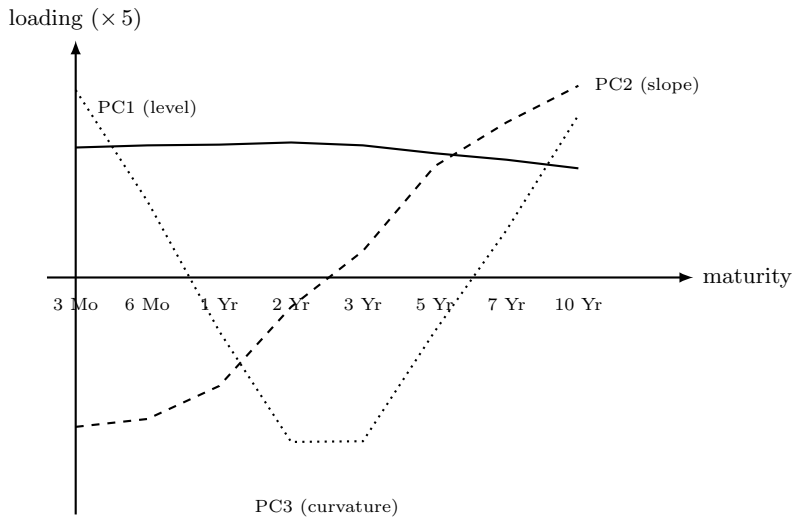


Figure 7.2: PCA loadings by maturity for the first three principal components of the 1990–2023 Treasury yield curve panel

-16.474 ($p < 0.0001$), decisively stationary, the same diagnose-then-difference discipline this chapter has applied to every other series.

Fitting an AR(1) model to the differenced level factor ΔPC1_t by OLS gives

$$\Delta\text{PC1}_t = -0.0011 + 0.0411\,\Delta\text{PC1}_{t-1} + \varepsilon_t,$$

with the AR(1) coefficient statistically significant ($p = 0.0002$) thanks to the enormous sample size, 8,502 daily changes, but economically tiny: $R^2 = 0.0017$, meaning the previous day's move in the level factor explains under one-fifth of one percent of the next day's move. On the last trading day of the sample (December 29, 2023), the level factor stood at 3.063 after a daily change of -0.023; the fitted model forecasts the next change as $-0.0011 + 0.0411(-0.023) \approx -0.002$, an essentially negligible parallel shift. Mapped back through the PC1 loadings in Figure 7.2, this forecasts each of the eight maturities to move by only about -0.0007 to -0.0008 percentage points the next day, an order of magnitude smaller than a single basis point.

This is an honest result, not a disappointing one. A statistically significant but practically negligible R^2 is precisely what an efficient, highly liquid market should produce: if daily changes in the level of the Treasury curve were meaningfully predictable from their own recent history, that predictability would itself be a trading opportunity large market participants would compete away, the same random-walk logic introduced for individual asset prices in Section 7.2.2 now confirmed directly on 34 years of real term-structure data. The genuine payoff of the PCA step is not that it makes the level factor easy to predict, nothing makes daily interest-rate changes easy to predict, but that it replaces an unwieldy eight-maturity forecasting problem with a three-factor one, and that whatever structure a more sophisticated model (a GARCH specification on the level factor's volatility, for instance, using the tools of Section 7.5.1 directly on ΔPC1_t) does find can be mapped back to every maturity on the curve at once through the same fixed loadings, rather than requiring eight separate models to stay mutually consistent by construction.

7.5 Volatility and State-Space Models

Volatility is itself predictable in a way the level is not, and some of what matters is never observed directly.

7.5.1 Volatility Clustering and the GARCH Family

A defining stylized fact of financial returns is that their variance is not constant: large price moves tend to be followed by more large moves (of either sign), and calm periods tend to persist, a pattern called volatility

Wulin.Suo@Queensu.ca

clustering. This directly affects the option pricing models in Chapter 5, which assumed a single constant volatility σ; GARCH models exist to make that assumption realistic by letting volatility evolve with the data.

The ARCH(1) model (autoregressive conditional heteroskedasticity) makes today's conditional variance a function of yesterday's squared shock,

$$\sigma_t^2 = \omega + \alpha\,\varepsilon_{t-1}^2, \qquad \omega > 0,\ \alpha \geq 0.$$

The GARCH(1,1) model, by far the most widely used volatility model in practice, adds a dependence on yesterday's own variance forecast,

$$\sigma_t^2 = \omega + \alpha\,\varepsilon_{t-1}^2 + \beta\,\sigma_{t-1}^2, \qquad \omega > 0,\ \alpha, \beta \geq 0,\ \alpha + \beta < 1.$$

The condition $\alpha+\beta < 1$ ensures the process is stationary, with a well-defined long-run variance $\omega/(1-\alpha-\beta)$ that the forecast reverts to over time; $\alpha + \beta$ close to 1 (a common finding in estimated financial GARCH models) implies that shocks to volatility die out only slowly, which is precisely what volatility clustering looks like in the data.

To make the recursion concrete, reuse the four AssetA returns from Chapter 2,

$$r = (0.0200, -0.0098, 0.0297, -0.0096),$$

whose sample variance was 0.000414. Take this as the seed σ_1^2, and suppose (for illustration, not estimated from only four data points) that $\omega = 0.00002$, $\alpha = 0.10$, and $\beta = 0.85$, so that $\alpha + \beta = 0.95$ and the long-run variance is $\omega/(1 - 0.95) = 0.0004$, a long-run volatility of 2.00%. The recursion needs shocks rather than returns, and the two are not quite the same thing: a GARCH model sits on top of a mean equation $r_t = \mu + \varepsilon_t$, so the shock is properly the deviation of the return from its expected value, $\varepsilon_t = r_t - \mu$. Setting $\mu = 0$ lets the returns themselves stand in for the shocks, which is the conventional treatment for daily data, where the average drift is smaller than a typical daily move by two orders of magnitude and makes almost no difference to the fitted variance path. Applying the recursion with $\varepsilon_{t-1} = r_{t-1}$ on that basis gives Table 7.5.

t	Shock used, r_{t-1}	r_{t-1}^2	σ_t^2	σ_t (volatility)
1 (seed)	–	–	0.000414	2.03%
2	0.0200	0.00040	0.000412	2.03%
3	−0.0098	0.00010	0.000380	1.95%
4	0.0297	0.00088	0.000431	2.08%
5	−0.0096	0.00009	0.000396	1.99%

Table 7.5: GARCH(1,1) variance recursion using Chapter 2's AssetA returns

The forecast volatility rises from 1.95% to 2.08% exactly after the large return of 2.97% at $t = 3$, and falls back toward the long-run level of 2.00% once smaller returns follow, precisely the volatility-clustering behavior GARCH is designed to capture. In practice, ω, α, and β are estimated from a much longer return history by maximum likelihood rather than assumed, but the recursion, and the forward-looking risk forecast it produces, works identically once fitted.

The GARCH(1,1) model treats a positive and a negative return of the same magnitude as equally informative about future volatility, since the recursion depends only on r_{t-1}^2. This is a well-documented empirical shortcoming: in equity markets especially, a large negative return (a crash) is typically followed by higher volatility than an equally large positive return, an effect sometimes called the leverage effect.

Extensions such as the EGARCH (exponential GARCH) and GJR-GARCH models address this by allowing the volatility recursion to respond asymmetrically to the sign of the previous shock, not just its magnitude, at the cost of additional parameters to estimate. These extensions are not developed in detail here, but recognizing that the basic GARCH(1,1) model implicitly assumes symmetry is important context for interpreting its forecasts, particularly around market downturns, when this symmetry assumption is most likely to be violated.

Wulin.Suo@Queensu.ca

7.5.2 State-Space Models and the Kalman Filter

Suppose a fund re-estimates a stock's market beta from a rolling regression each quarter and finds the estimate bouncing around noticeably from one quarter to the next. Some of that bounce is pure estimation noise, sampling variation that would appear even if the true beta never changed at all, but part of it might be the true beta itself genuinely drifting over time as the company's business evolves, and a single quarter's estimate cannot tell the two apart.

A different, more general way to model this kind of time-varying behavior is the state-space representation, which separates an unobserved ("latent") state that evolves over time from the observed data that provides noisy information about it:

$$\begin{aligned} \text{State equation:} \quad & \beta_t = \beta_{t-1} + \eta_t, \\ \text{Observation equation:} \quad & y_t = x_t^\top \beta_t + \varepsilon_t, \end{aligned}$$

where β_t is an unobserved parameter that is allowed to drift slowly over time (for example, a stock's market beta, or the level of an interest rate), and y_t is what is actually observed. The Kalman filter is the standard algorithm for recursively estimating β_t as new observations y_t arrive, updating a running estimate and its uncertainty at every step rather than re-estimating from scratch.

Each step of the filter has two parts: a prediction, using the state equation alone, followed by an update, which corrects that prediction using the new observation, weighted by how relatively reliable the prediction and the observation are. Concretely, given a current estimate $\hat{\beta}_{t-1}$ with variance P_{t-1}, a process (state) noise variance Q, and an observation noise variance R,

$$\begin{aligned} \text{Predict:} \quad & P_{t|t-1} = P_{t-1} + Q, \\ \text{Kalman gain:} \quad & K_t = \frac{P_{t|t-1}}{P_{t|t-1} + R}, \\ \text{Update:} \quad & \hat{\beta}_t = \hat{\beta}_{t-1} + K_t(y_t - \hat{\beta}_{t-1}), \qquad P_t = (1 - K_t)P_{t|t-1}. \end{aligned}$$

Suppose a fund's current estimate of a stock's market beta is $\hat{\beta}_0 = 1.0$ with variance $P_0 = 0.04$, the true beta is believed to drift slowly with process variance $Q = 0.001$ per period, and a single period's regression-based beta estimate is noisy with observation variance $R = 0.09$. If the next two periods' noisy beta estimates are $y_1 = 1.30$ and $y_2 = 1.25$, the filter updates as follows: at $t = 1$, $P_{1|0} = 0.041$, $K_1 = 0.041/0.131 \approx 0.313$, giving $\hat{\beta}_1 = 1.0 + 0.313(1.30 - 1.0) \approx 1.094$ and $P_1 \approx 0.028$; at $t = 2$, $K_2 \approx 0.245$, giving $\hat{\beta}_2 = 1.094 + 0.245(1.25 - 1.094) \approx 1.132$. Notice that the estimate moves toward each new noisy observation but does not jump all the way to it, the smoothing behavior exponential smoothing achieves with a fixed weight α; the Kalman filter can be seen as an adaptive generalization of exponential smoothing, where the effective weight K_t adjusts automatically based on how much uncertainty remains in the current estimate versus how noisy new observations are.

Deriving the filter's update equations from first principles is beyond the scope of a single chapter, but the core idea, that a model's parameters need not be fixed forever and can be tracked as they evolve, is directly useful in finance: a fund's exposure to a risk factor, or the volatility of an asset (an alternative to GARCH known as stochastic volatility modeling), can both be framed this way.

Notice also that the Kalman gain K_t shrinks slightly across the two updates in this example, from 0.313 to 0.245, reflecting the filter's growing confidence in its running estimate as more observations accumulate, even though the process noise Q continually reintroduces some uncertainty at every step to prevent the estimate from becoming overconfident and failing to adapt to genuine future changes in the underlying beta.

7.6 Validation and Accuracy

Time order makes the standard validation split invalid, so this part replaces it and asks what a forecast should be scored against.

Wulin.Suo@Queensu.ca

7.6.1 Walk-Forward Validation for Time Series

Chapter 6 introduced k-fold cross-validation and noted that it can be misleading for time series, because a random split can train a model on data from after the validation period, information that would never be available at the time a real forecast was made. Walk-forward validation avoids this by always keeping the training window entirely before the validation window in time, then rolling both windows forward, as shown in Figure 7.3.

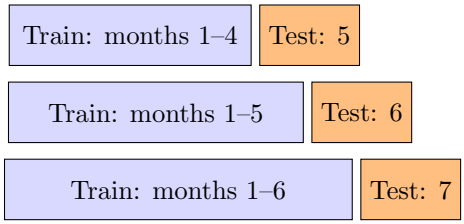


Figure 7.3: Walk-forward validation across three successive train/test splits

At each step, the model is refit (or updated) using only data available up to that point, produces a forecast for the next period, and that forecast is scored against the realized outcome once it arrives. The process repeats, rolling forward through the entire sample, and the reported performance is the average across all these genuinely out-of-sample forecasts. This is more computationally expensive than a single train-test split, since the model may be refit many times, but it is the only evaluation procedure that mimics how a forecasting model would actually be used in practice, one period at a time, without ever peeking ahead.

7.6.2 Evaluating Forecast Accuracy

Once walk-forward validation has produced a sequence of genuinely out-of-sample forecasts, they must be scored using a metric appropriate to the forecasting problem. Beyond the mean squared error used elsewhere in this book, two metrics are especially common in time series work. The root mean squared error (RMSE),

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(\hat{X}_t - X_t)^2},$$

returns the error to the original units of the series (unlike MSE, which is in squared units), making it directly interpretable, for example, as an average error of "2.1 basis points" rather than "4.4 squared basis points." The mean absolute percentage error (MAPE),

$$\text{MAPE} = \frac{100\%}{n}\sum_{t=1}^{n}\left|\frac{X_t - \hat{X}_t}{X_t}\right|,$$

expresses errors as a percentage of the actual value, which makes it easier to compare forecast accuracy across series with very different scales (comparing forecast accuracy for a \$10 stock and a \$400 stock in dollar terms is not very meaningful, but a percentage error is). MAPE has a well-known weakness, however: it is undefined when $X_t = 0$ and becomes extremely large and unstable when X_t is close to zero, which is a real practical concern for series such as interest rates or spreads that can pass through or near zero, unlike strictly positive prices.

Beyond a single summary statistic, it is standard practice to compare a candidate model's walk-forward RMSE or MAPE against at least one naive benchmark, such as a forecast that simply repeats the last observed value ($\hat{X}_{t+1} = X_t$) or the simple moving average from Section 7.3.3. A sophisticated model that cannot outperform this kind of naive benchmark on a walk-forward basis is not adding predictive value, regardless of how compelling its in-sample fit looked, which is the discipline that separates a genuinely useful forecasting model from an elaborate but ultimately unhelpful one.

Wulin.Suo@Queensu.ca

A small worked comparison shows why this benchmark matters. Suppose a walk-forward evaluation produces five one-step-ahead forecasts for a price series with actual values $100, 102, 98, 105, 101$. A fitted model forecasts $99, 103, 97, 104, 102$, while the naive "repeat the last value" benchmark forecasts $98, 100, 102, 98, 105$ (each simply the prior period's actual value). Computing both metrics for each,

$$\text{RMSE}_{\text{model}} = 1.00, \qquad \text{RMSE}_{\text{naive}} = 4.22, \qquad \text{MAPE}_{\text{model}} = 0.99\%, \qquad \text{MAPE}_{\text{naive}} = 3.73\%,$$

the fitted model beats the naive benchmark by a wide margin on both metrics, roughly a fourfold reduction in RMSE and MAPE alike. That is encouraging, but five forecasts are far too few to prove that the model captures genuine, exploitable structure rather than having been lucky. Whether one forecast's loss is significantly smaller than another's is what the Diebold-Mariano test formally assesses: it treats the period-by-period difference between the two models' losses as a series and tests whether its mean is zero, and with five such differences it has very little power. A long walk-forward history is what turns a gap like this into evidence. Had the two RMSE values instead been close, say 4.22 versus 4.05, the appropriate conclusion would be the opposite: the model's added complexity is buying almost nothing over simply extrapolating the last observed value, and a practitioner should think hard about whether that complexity is worth the extra estimation risk and maintenance cost.

7.7 Challenges in Time Series Forecasting for Finance

Several challenges recur across almost every time series application in finance.

Regime changes and structural breaks. A model estimated over a calm period, including the AR(1) and GARCH parameters estimated above, can become badly miscalibrated once the underlying regime shifts, for example when a central bank changes policy or a market enters a crisis. Unlike the smooth, gradual change assumed by a stationary model, a structural break is a sudden shift in the data-generating process itself. Formal tests exist for detecting a structural break at a known or unknown date (the Chow test and the CUSUM test are two classical examples), but in practice, a combination of periodic re-estimation and the walk-forward validation discipline introduced above is usually a more robust defense than trying to detect every break in real time, since a break severe enough to matter economically is typically also severe enough to show up as a sudden deterioration in walk-forward forecast accuracy.

Limited effective sample size. Financial time series often look large (years of daily data) but contain far less independent information than the raw observation count suggests, because nearby observations are highly correlated. This limits how many parameters can be reliably estimated and is part of why simple, low-order models (AR(1), GARCH(1,1)) so often perform competitively against more elaborate alternatives.

Model selection versus overfitting. As in Chapter 6, adding more autoregressive or moving-average terms will always improve the in-sample fit. Coupled with walk-forward validation, information criteria such as AIC and BIC help guard against selecting an overly complex model that fits historical noise rather than a genuine, persistent pattern.

Fat tails and non-normality. Financial returns have more extreme observations than a normal distribution predicts. GARCH models capture time-varying volatility but are often paired with a fatter-tailed error distribution (such as the Student's t) to capture this feature, since even a correctly time-varying variance does not by itself guarantee well-calibrated tail forecasts.

Multi-step forecasting compounds uncertainty. A one-step-ahead forecast, like the AR(1) example in this chapter, is usually far more reliable than the same model's five- or twenty-step-ahead forecast, since forecast uncertainty compounds with the horizon. Any decision that depends on a longer-horizon time series forecast should treat that forecast as considerably less certain than a one-step forecast, even when both come from the same fitted model.

Spurious cointegration and relationship breakdown. The pairs-trading strategy in Section 7.4.1 depends on a cointegrating relationship remaining stable, but with enough series to search over, some pairs will appear cointegrated in a historical sample purely by chance, an instance of the same multiple-testing problem raised in Chapter 6. Even a genuine, economically sound cointegrating relationship can break down going forward, for example if a merger, a change in business strategy, or a regulatory shift changes one company's fundamentals relative to its former peer.

Wulin.Suo@Queensu.ca

Model misspecification in multivariate systems. A VAR model's conclusions, including any Granger-causality test, are sensitive to which variables are included and how many lags are used; omitting a relevant variable that actually drives both series in the system can produce a spurious Granger-causality finding, incorrectly suggesting one included variable predicts another when both are really being driven by the omitted one.

Classical models versus neural sequence models. Every model in this chapter, AR(1), ARIMA, GARCH, and VAR, is a classical statistical model with a small number of interpretable parameters, estimated by OLS or maximum likelihood. A recurrent neural network can, in principle, learn a considerably more flexible mapping from a series' own history to its next value than any fixed linear, fixed-lag-order model this chapter introduced, and Chapter 17 works through exactly such a model, a hand-computed RNN forward pass, in detail. That added flexibility is not a free upgrade, however: a neural sequence model has many more parameters to fit from data that, as this section's limited-effective-sample-size challenge already noted, contains far less independent information than its raw observation count suggests, so the added flexibility often costs more in overfitting risk than it returns in genuine forecast accuracy unless it is disciplined with the walk-forward validation this chapter introduced. The practical choice between a classical time series model and a neural sequence model is therefore rarely settled by which is more sophisticated in the abstract, but by which one a limited, noisy financial dataset can actually support without simply memorizing its own history.

The newest entrant in this debate sidesteps the limited-sample objection in an interesting way. **Time-series foundation models**, large neural sequence models pretrained once on enormous corpora of diverse time series (retail demand, energy load, traffic, weather, and much else) and then applied to a new series with little or no additional fitting, arrived in the mid-2020s under names like Chronos and TimesFM, borrowing directly from the pretrain-then-adapt recipe Chapter 16 describes for language models. Because the heavy parameter fitting happened on the pretraining corpus rather than on the forecaster's own short series, the overfitting arithmetic above changes: the model brings pattern knowledge in rather than extracting it all from the sample at hand, and on generic forecasting benchmarks these models produce respectable zero-shot forecasts of series they have never seen. What has not changed is anything specific to finance: published evaluations to date show them competitive with, not dominant over, well-tuned classical baselines on financial series, whose low signal-to-noise ratio and regime shifts are the features a pretraining corpus of mostly non-financial series does little to prepare a model for. A practitioner should treat such a model as one more candidate forecaster to be judged inside the same walk-forward, benchmark-against-naive discipline this chapter built, not as an exemption from it.

7.8 A Practical Workflow for Building a Time Series Model

With this many tools introduced across the chapter, it is useful to close with a single, concrete workflow that ties them together in the order they would typically be applied in practice, summarized in Figure 7.4.

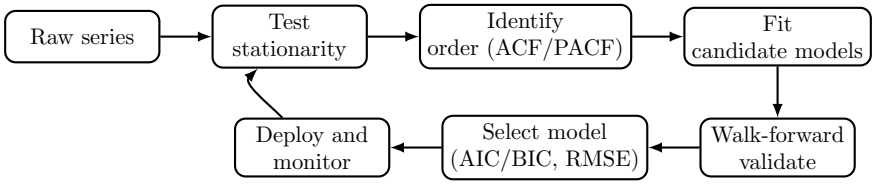


Figure 7.4: A practical time series modeling workflow

Starting from a raw series, the first step is always to test stationarity (Section 7.2.3), since every model in this chapter, from AR(1) to GARCH to VAR, assumes some form of stationarity either in levels or after transformation. If the series is non-stationary, difference it (or work with returns, as in Chapter 2) and re-test before proceeding.

Next, the ACF and PACF guide the choice of candidate model orders, following the Box-Jenkins logic introduced in the ARIMA section, rather than guessing arbitrarily or defaulting to the most complex model

Wulin.Suo@Queensu.ca

available. Several candidate specifications are then fit and compared using information criteria (in-sample) and, more importantly, walk-forward validation (out-of-sample), scored with the RMSE or MAPE metrics introduced above and always benchmarked against a naive forecast.

Only once a model has demonstrated genuine walk-forward skill is it deployed, and even then, monitoring continues: the feedback arrow in Figure 7.4 back to the stationarity test reflects the reality, emphasized throughout this chapter's challenges section, that financial time series are subject to regime changes and structural breaks, so a model that was well-specified last year may need to be re-identified, not just re-fit, as new data arrives.

This workflow applies equally to the single-series models (AR, ARIMA, GARCH) and the multi-series extensions (cointegration testing, VAR) introduced in this chapter; the only difference is that the multivariate case requires checking stationarity and identifying appropriate lag orders for a whole system of series jointly, rather than one series at a time.

7.9 Key Takeaways

Time series methods extend the statistical toolkit of Chapter 6 to data where order matters. Stationarity is the property that makes a fitted model meaningful for the future, and checking for it, formally with the Dickey-Fuller test or informally via the correlogram, is usually the first step in any analysis. The AR(1) example showed how a model's coefficients are estimated by the same OLS logic as any regression, while the GARCH example showed how volatility itself can be modeled and forecast, directly feeding the option pricing and risk models in the chapters that follow.

The cointegration and VAR material extended these ideas from a single series to relationships between series, previewing the pairs-trading strategies of Chapter 13 and the multivariate risk factor models of Chapter 8.

The chapter's closing lesson, walk-forward validation, scored with metrics like RMSE and MAPE and always benchmarked against a naive forecast, is a direct consequence of a single principle repeated throughout this book: a model should never be evaluated using information it would not have had in real time, and it should never be trusted simply because it outperforms a strawman rather than a genuinely competitive baseline.

Several smaller but recurring lessons are worth restating explicitly. The Dickey-Fuller test and the ACF confidence-band calculation both showed, using the chapter's own 8-observation running example, that formal statistical significance is much harder to achieve with small samples than a purely visual reading of a chart or a plausible-looking coefficient might suggest; a quantity that looks convincingly non-zero can still fail to clear a formal significance threshold, and this gap should make an analyst more cautious, not less, about small-sample conclusions.

The comparison between the AR(1) forecast, the simple exponential smoothing forecast, and the Holt's-method forecast for the same series illustrated that different, individually reasonable modeling choices (mean-reversion versus trend extrapolation) can produce meaningfully different point forecasts from identical data, and that choosing between them requires economic judgment about the series' likely future behavior, not just a mechanical best fit to the past.

Finally, the progression from a single series (AR, ARIMA, GARCH) to relationships between series (cointegration, VAR) mirrors a pattern that recurs throughout this book: understanding one object in isolation is usually a prerequisite for, but not a substitute for, understanding how it interacts with the other objects around it, whether those are two cointegrated stock prices here or the many assets in the portfolios of Chapter 12.

7.10 Suggested Exercises

The exercises below span the full range of tools introduced in this chapter, from the basic mechanics of fitting and forecasting a single AR(1) series to testing formally for stationarity, comparing competing forecasting methods, and extending the analysis to relationships between two or more series. Several exercises build directly on the chapter's own worked examples, so your answers can be checked against, or contrasted with, the numbers already computed in the text.

Wulin.Suo@Queensu.ca

1. Explain, in your own words, why fitting an AR(1) model directly to a price series rather than a return series is likely to produce an unreliable model.

2. Using the data in Table 7.1, verify by hand that the sample mean is $\bar{X} = 6.25$ and that $\hat{\rho}_1 = 0.50$.

3. Add a ninth observation, $X_9 = 3.0$, to Table 7.1, refit the AR(1) model, and compare the new $\hat{\phi}$ and one-step-ahead forecast $\hat{X}_{10}$ to the values computed in the chapter.

4. Using the GARCH(1,1) recursion in Table 7.5, compute σ_6^2 if a fifth AssetA return of $r_5 = 0.035$ were observed.

5. Explain why $\alpha + \beta < 1$ is required for a GARCH(1,1) process to be stationary, and describe what would happen to the variance forecast if $\alpha + \beta = 1$ exactly.

6. Describe a concrete scenario in which k-fold cross-validation, as used in Chapter 6, would give a misleadingly optimistic evaluation of a time series forecasting model.

7. Discuss one reason a GARCH(1,1) model fit on five years of daily returns during a calm market might forecast volatility poorly at the onset of a financial crisis.

8. Using the Dickey-Fuller regression in Section 7.2.3, explain why $\hat{\gamma} = \hat{\phi} - 1$, and verify this relationship numerically using the AR(1) coefficient computed in Section 7.3.1.

9. Explain why the Dickey-Fuller test failed to reject the unit-root null hypothesis in Section 7.2.3 even though the fitted AR(1) coefficient clearly indicated mean reversion, and describe what you would expect to happen to the test's conclusion if the same series had 200 observations instead of 8.

10. Using Table 7.3, compute the spread on a hypothetical Day 7 where Stock A trades at 56 and Stock B trades at 51, and explain, using the mean and standard deviation of the historical spread, whether a pairs trader would be more likely to open a new position or close an existing one on that day.

11. Explain, in your own words, the difference between Granger causality and true causal influence, and describe one financial scenario in which two series could Granger-cause each other without either one truly driving the other.

12. A retailer's monthly sales data shows a strong, repeating spike every December. Explain why an ordinary ARIMA model without a seasonal component would likely underperform a SARIMA model on this series, and describe what seasonal period s you would use.

13. Using the multi-step forecasting formulas in Section 7.3.2, compute the 4-step-ahead AR(1) forecast $\hat{X}_{12}$ from $X_8 = 4.0$, and verify it lies between the 3-step and 5-step forecasts already reported in Table 7.2.

14. Using Holt's linear trend method from Section 7.3.3 with $\alpha = 0.5$ and $\beta = 0.2$ (instead of $\alpha = 0.4$, $\beta = 0.3$), recompute the level and trend after processing $X_2 = 8.0$ through $X_4 = 7.0$ (three updates starting from $L_1 = 9.0$, $T_1 = -1.0$), and compare your intermediate values to those reported in the chapter.

15. Using the Kalman filter recursion in Section 7.5.2, compute a third update step assuming a new observation $y_3 = 1.20$ arrives, continuing from $\hat{\beta}_2 \approx 1.132$ and $P_2 \approx 0.022$ with the same $Q = 0.001$ and $R = 0.09$.

16. Explain, using the workflow in Figure 7.4, what step or steps you would need to repeat if a monitoring system detected that a deployed time series model's walk-forward forecast errors had grown substantially larger over the past quarter.

17. Using the confidence-band rule in Section 7.2.4 ($\pm 1.96/\sqrt{T}$), compute how many observations T would be needed for a sample autocorrelation of $\hat{\rho}_1 = 0.50$ to be considered statistically significant at the 95% level, and comment on how this compares to the 8 observations actually available in Table 7.1.

Wulin.Suo@Queensu.ca

18. Using this chapter's VAR(1) impulse response example, compute X_1 and Y_1 following a one-unit shock to the exchange rate instead of the interest rate (that is, $X_0 = 0$, $Y_0 = 1$), and explain, in terms of the individual coefficients a_{12} and a_{21}, why the interest rate's one-period response to an exchange-rate shock turns out to be larger than the exchange rate's one-period response to an interest-rate shock was.

19. Using this chapter's RMSE/MAPE comparison, recompute both metrics for the naive benchmark if its forecasts were instead $99, 101, 99, 104, 100$ (closer to the actual values than the original naive forecasts), and discuss whether the fitted model would still show a clear advantage over this improved benchmark.

20. Using Section 7.3.2's Ljung-Box calculation, explain in your own words why a large Q statistic (and correspondingly small p-value) is evidence that a model's residuals still contain autocorrelation, and why this counts against the model even though the model's own R^2 or fitted coefficients might look reasonable on their own.

21. Section 7.3.2 found that AIC and BIC both favor the AR(2) model over the AR(1) model, yet also cautioned that this conclusion may simply reflect overfitting on only six observations. Describe a concrete piece of additional evidence (not just a lower in-sample AIC or BIC) that would make you more confident the AR(2) model is genuinely better specified, not just better fitting.

22. **Challenge (real data).** Download at least five years of daily closing prices for a real, liquid stock (for example, via the `yfinance` library, as Chapter 2 and Chapter 5 already do) and compute its daily log returns. (a) Test the return series for a unit root using the augmented Dickey-Fuller test, as in Section 7.2.3, and contrast the statistical power of this test on your (likely thousands of) real observations with this chapter's own 8-observation toy example, where the test failed to reject a unit root despite a clearly mean-reverting fitted coefficient. (b) Fit a GARCH(1,1) model to the return series and plot the fitted conditional volatility alongside the raw returns, identifying at least one real historical event (an earnings surprise, a market-wide shock) that produced a visible volatility spike. (c) Compare your fitted $\alpha + \beta$ to the stationarity condition discussed in this chapter, and comment on whether real daily equity returns typically sit close to the $\alpha + \beta = 1$ boundary this chapter warns is a sign of near-nonstationary volatility.

7.11 Further Reading

- Tsay, *Analysis of Financial Time Series*. Written specifically for financial applications, with extensive coverage of ARIMA, GARCH, and multivariate time series methods including cointegration and VAR models.

- Hamilton, *Time Series Analysis*. The classic, mathematically rigorous graduate-level reference for the state-space and Kalman filtering material introduced only briefly in this chapter, alongside a thorough treatment of stationarity and unit-root testing.

- Engle, "Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation," *Econometrica*. The original 1982 paper introducing ARCH models, for readers interested in the primary source behind the GARCH family developed in this chapter.

- Hyndman and Athanasopoulos, *Forecasting: Principles and Practice*. A highly practical, freely available online text covering exponential smoothing, ARIMA, and forecast evaluation with an emphasis on applied forecasting workflows like the one in Figure 7.4.

- Engle and Granger, "Co-integration and Error Correction: Representation, Estimation, and Testing," *Econometrica*. The original paper introducing the cointegration concept and the Engle-Granger two-step testing procedure used in Section 7.4.1.

- Sims, "Macroeconomics and Reality," *Econometrica*. The influential 1980 paper introducing vector autoregression as an alternative to large-scale structural macroeconomic models, foundational to the VAR and Granger-causality material in this chapter.

Wulin.Suo@Queensu.ca

- Ansari et al., "Chronos: Learning the Language of Time Series," *Transactions on Machine Learning Research*. The paper introducing one of the pretrained time-series foundation models discussed in this chapter's closing challenges, applying the language-model pretraining recipe to zero-shot forecasting.
- Litterman and Scheinkman, "Common Factors Affecting Bond Returns," *Journal of Fixed Income*. The original paper documenting the level, slope, and curvature factor structure of the yield curve reproduced in Section 7.4.3.

Wulin.Suo@Queensu.ca

Chapter 8

Market Risk Management

8.1 Introduction

Every model in this book so far has focused on estimating a quantity of interest: a price, a return, a probability of default, a forecast. Risk management asks a different, complementary question: not "what do we expect to happen," but "how bad could it get, and how often." A portfolio manager who only forecasts expected returns and ignores the distribution of possible losses is flying blind, since the same expected return can come from a smooth, low-volatility strategy or from a strategy that is usually flat but occasionally loses everything. This chapter introduces the standard toolkit for quantifying and managing that downside for market risk specifically: Value at Risk and Expected Shortfall, maximum drawdown, volatility forecasting, backtesting, liquidity risk, and the model governance and capital adequacy concepts that let a firm and its regulators judge whether it can survive its own worst days. Chapter 9 develops the parallel toolkit for credit risk, probability of default, loss given default, and credit scoring, since the two risk types, while related, call for genuinely different tools and are large enough topics to deserve their own dedicated treatment.

One worked example runs through most of the chapter: a twenty-day return history for a $10 million equity portfolio, used to compute and compare parametric, historical, Monte Carlo, and GARCH-based risk measures, backtest them, and stress-test the resulting portfolio.

The chapter builds its toolkit in the order a risk desk would actually deploy it. First a taxonomy of risk types (Section 8.2.1), to establish what is and is not in scope. Then Value at Risk itself (Section 8.2.2), computed three different ways and then extended to a full portfolio and to a volatility-adjusted version that updates with Chapter 7's GARCH forecasts, and the backtesting discipline (Section 8.3.1) that checks whether any of this actually worked out of sample. Later sections extend the same VaR machinery in three directions: to options positions via the Greeks (the delta-normal approximation, then the sensitivity-based limits of Section 8.3.3), to hypothetical crises the historical sample never contained (Section 8.4.1), and, in Section 8.4.2, to a direct and honest test of whether a more flexible machine learning model can out-forecast GARCH at its own game. Chapter 9 develops the analogous toolkit for credit risk using the same governance and validation philosophy introduced here.

8.2 Measuring Market Risk

This part builds the standard measures in order of increasing honesty about the tail, each worked numerically on the same portfolio.

8.2.1 Types of Financial Risk

Suppose a risk manager reports simply that the fund "lost money because of risk." The statement is true but useless: it does not say whether the loss came from prices moving against a held position, a counterparty failing to pay what it owed, or a position becoming impossible to sell without moving its own price further down, three genuinely different problems that call for three different responses. Financial risk is not a single

Wulin.Suo@Queensu.ca

quantity but a family of related concerns, and confusing one type for another is a common and costly mistake. Market risk is the risk of loss from movements in prices, rates, or volatilities that a firm is exposed to through the positions it holds; it is the subject of this chapter. Credit risk is the risk that a counterparty, whether a bond issuer, a loan borrower, or a derivatives counterparty, fails to meet a contractual obligation; Chapter 5 already introduced the pricing consequence of credit risk, and Chapter 9 develops the risk-measurement side of the same problem. Liquidity risk comes in two related forms: funding liquidity risk, the risk that a firm cannot meet its own cash obligations as they come due, and market liquidity risk, the risk that a position cannot be sold quickly without moving its price significantly against the seller, directly connected to the bid-ask spread and market depth concepts of Chapter 3. Operational risk covers losses from failed internal processes, systems, human error, or external events, including the kind of model risk flagged throughout this book whenever a model's assumptions are quietly violated by a changing environment; Chapter 9 covers a standard capital treatment for it, and Chapter 10 develops the AI methods for detecting and managing it. Underwriting risk, the risk that claims on policies a firm has written exceed the premiums it collected for them, is the insurer's counterpart to all of the above, and Chapter 11 develops it alongside the reserving and ratemaking machinery it requires. A single adverse event often touches more than one category at once: a large, sudden market move (market risk) can trigger a wave of defaults among leveraged counterparties (credit risk) precisely when many firms are simultaneously forced to sell into an illiquid market (liquidity risk), which is the contagion dynamic behind the financial crises discussed in Chapter 3.

Table 8.1 summarizes each risk type alongside its standard measurement tool and where in this book that tool is developed, giving a preview of this chapter's structure organized by risk type rather than by technique.

Risk type	Standard measurement tool	Where developed
Market risk	Value at Risk, Expected Shortfall, maximum drawdown	Sections 8.2.2–8.3.1 of this chapter
Credit risk	Probability of default, loss given default, expected loss	Chapter 9
Liquidity risk	Bid-ask spread, liquidity-adjusted VaR	Chapter 3 and later in this chapter
Operational risk	Loss distribution approach, scenario analysis, incident-text and anomaly models	Chapter 10, with the regulatory capital charge in Chapter 9
Underwriting risk	Expected loss ratio, combined ratio, chain-ladder reserves	Chapter 11
Systemic risk	Stress testing, contagion modeling	Chapter 3's financial crises discussion, this chapter's stress-testing section, and Chapter 20's network contagion models

Table 8.1: A taxonomy of financial risk types and their standard measurement tools

8.2.2 Value at Risk: Parametric and Historical Methods

Value at Risk (VaR) answers a specific, deliberately narrow question: over a given holding period and at a given confidence level, what is the loss that will not be exceeded except with some small, specified probability? Formally, the α-level VaR of a portfolio is the smallest loss L such that

$$\Pr(\text{Loss} > L) \leq 1 - \alpha.$$

A 1-day 95% VaR of \$1 million means that, under the model's assumptions, the portfolio is expected to lose more than \$1 million on only about 5% of trading days, or roughly one day in twenty.

Table 8.2 shows twenty days of returns for a hypothetical \$10 million equity portfolio managed by a firm we will call Meridian Capital Partners.

Wulin.Suo@Queensu.ca

Day	1	2	3	4	5	6	7	8	9	10
Return	0.06%	0.42%	-0.27%	-1.01%	-0.49%	-1.13%	0.13%	1.67%	-0.53%	-0.68%
Day	11	12	13	14	15	16	17	18	19	20
Return	0.65%	0.49%	0.19%	-1.06%	0.02%	0.89%	-1.55%	-0.49%	-2.22%	-1.49%

Table 8.2: Twenty days of returns for a $10 million equity portfolio

The sample mean return is $\bar{r} = -0.32\%$ and the sample standard deviation is $\hat{\sigma} = 0.937\%$ (with $n-1 = 19$ in the denominator, as in every standard-deviation calculation in this book). The parametric (variance-covariance) method assumes returns are normally distributed and computes VaR directly from the mean, standard deviation, and the appropriate normal quantile z_α:

$$\text{VaR}_\alpha = -(\bar{r} - z_\alpha \hat{\sigma}) \times V,$$

where V is the portfolio's dollar value. Using $z_{0.95} = 1.645$ and $z_{0.99} = 2.326$ and $V = \$10{,}000{,}000$,

$$\text{VaR}_{95\%} = -(-0.0032 - 1.645(0.00937)) \times 10{,}000{,}000 \approx \$186{,}195,$$

$$\text{VaR}_{99\%} = -(-0.0032 - 2.326(0.00937)) \times 10{,}000{,}000 \approx \$250{,}081.$$

The historical method instead makes no distributional assumption at all: it simply sorts the observed returns and reads the appropriate percentile directly from the data. Sorting the twenty returns in Table 8.2 from worst to best, the second-worst return (-2.22% and -1.55% are the two worst) corresponds to the 5th percentile of a 20-observation sample, giving

$$\text{VaR}_{95\%}^{\text{historical}} = -(-0.0155) \times 10{,}000{,}000 = \$155{,}000.$$

The historical and parametric VaR estimates disagree by more than $30,000 here, which is not a contradiction but a direct illustration of a persistent tension in risk measurement: the parametric method borrows statistical strength from the entire sample and produces a smooth estimate, but it assumes normality, which can be a poor description of returns with heavier tails than a normal distribution predicts, echoing the fat-tails caveat raised in Chapter 7's discussion of GARCH models. The historical method makes no such assumption, but with only twenty observations, its estimate depends heavily on a single data point (here, the -1.55% return) and would be considerably more stable with a longer history, typically one to several years of daily data in practice.

Portfolio VaR with the Variance-Covariance Matrix

The single-return-series version of parametric VaR above is a special case of a more general matrix formula that extends naturally to a portfolio of several assets. Recall the two-asset covariance matrix from Chapter 2's worked AssetA/AssetB example,

$$\Sigma = \begin{pmatrix} 0.000414 & -0.000198 \\ -0.000198 & 0.000118 \end{pmatrix},$$

computed there from the two assets' daily returns, and the equal weights $w = (0.5, 0.5)$ used throughout that chapter. The portfolio's return variance is $w^\top \Sigma w$, the formula Chapter 2 verified numerically, and the parametric VaR of a portfolio worth V dollars, assuming a zero mean return (a common simplification in practice, since a one-day expected return is typically negligible relative to one-day volatility), is

$$\text{VaR}_\alpha \approx z_\alpha \sqrt{w^\top \Sigma w}\, V.$$

Applying this to a hypothetical $2 million position split evenly between AssetA and AssetB, $\sqrt{w^\top \Sigma w} \approx 0.583\%$ (matching the portfolio volatility computed in Chapter 2), so

$$\text{VaR}_{95\%} \approx 1.645(0.00583)(2{,}000{,}000) \approx \$19{,}182,$$

Wulin.Suo@Queensu.ca

$$\text{VaR}_{99\%} \approx 2.326(0.00583)(2{,}000{,}000) \approx \$27{,}130.$$

This matrix formulation is what actually scales to real trading books, which routinely hold hundreds or thousands of positions: rather than tracking a single blended portfolio-return series, a risk system maintains the full covariance matrix Σ across every position and recomputes $w^\top \Sigma w$ whenever positions change, the same computational pattern Chapter 2 introduced for a portfolio of arbitrary size using `weights @ cov_matrix.values @ weights`.

Component VaR: Decomposing Portfolio Risk by Position

A portfolio-level VaR figure answers "how much risk does the whole book have," but a risk manager allocating limits across desks, or a portfolio manager deciding which position to trim first, needs the complementary question: how much of that risk does each individual position actually contribute? Component VaR answers this by decomposing total portfolio variance into a sum of per-asset contributions, using the same covariance matrix already in hand,

$$\text{Component Var}_i = w_i(\Sigma w)_i, \qquad \sum_i \text{Component Var}_i = w^\top \Sigma w,$$

so that each asset's fractional share of total portfolio variance, $\text{Component Var}_i/(w^\top \Sigma w)$, scales the total portfolio VaR into a per-asset Component VaR that sums exactly back to the whole. Applying this to Section 8.2.2's two-asset position, $\Sigma w = (0.000108, -0.00004)$, so

$$\begin{aligned}\text{Component Var}_A &= 0.5(0.000108) = 0.000054,\\ \text{Component Var}_B &= 0.5(-0.00004) = -0.00002,\end{aligned}$$

summing to the portfolio variance of 0.000034 exactly as required. Scaling these fractions (1.588 and −0.588) by the total $\text{VaR}_{95\%}$ of \$19,182 gives

$$\text{Component VaR}_A \approx \$30{,}468, \qquad \text{Component VaR}_B \approx -\$11{,}285,$$

which sum back to the \$19,182 total, exactly as the decomposition guarantees. AssetB's component is *negative*, even though AssetB has positive standalone volatility on its own. Its strongly negative correlation with AssetA (Section 8.4.1 computes it as $\rho \approx -0.896$) makes it a diversifier within this specific portfolio: adding more of AssetB, holding AssetA fixed, would actually *reduce* total portfolio risk, not add to it. That is a fact the standalone volatility figures for AssetA and AssetB alone could never reveal. This is precisely why risk limits at a real trading firm are typically set and monitored using component or marginal VaR rather than standalone position size or standalone volatility: a position's true risk contribution depends on how it interacts with everything else already in the book, not on how risky it looks in isolation.

8.2.3 Expected Shortfall: Going Beyond VaR

VaR has a well-known conceptual weakness: it answers "how much can I lose with 95% confidence," but says nothing about how bad the remaining 5% of outcomes actually are. Two portfolios can have identical VaR while one has a slightly worse tail and the other has a catastrophically worse one. Expected Shortfall (ES), also called Conditional VaR, addresses this directly by averaging the losses in the tail beyond VaR rather than simply reporting its boundary:

$$\text{ES}_\alpha = \mathbb{E}[\text{Loss} \mid \text{Loss} > \text{VaR}_\alpha].$$

For a continuous loss distribution it makes no difference whether this inequality is strict ($>$) or not ($\geq$). In a finite sample it does, since the VaR is itself one of the observed losses, so a discrete convention is needed: this book counts the VaR observation itself as part of the tail, averaging every loss at or beyond the VaR (other texts average only the $n(1-\alpha)$ losses strictly beyond it, which with twenty observations at 95% would leave just the single worst day). For the historical method applied to Table 8.2, the two worst returns (−2.22% and −1.55%) are the ones at or beyond the 95% VaR threshold, so

$$\text{ES}_{95\%}^{\text{historical}} = -\frac{(-0.0222) + (-0.0155)}{2} \times 10{,}000{,}000 = \$188{,}500.$$

Wulin.Suo@Queensu.ca

The parametric Expected Shortfall under normality has a closed-form expression involving the standard normal density $\phi(\cdot)$,

$$\mathrm{ES}_\alpha = \left(\hat{\sigma} \frac{\phi(z_\alpha)}{1-\alpha} - \bar{r} \right) \times V,$$

which for $\alpha = 95\%$ gives $\mathrm{ES}_{95\%} \approx \$225{,}366$, noticeably larger than the parametric $\mathrm{VaR}_{95\%}$ of \$186,195, exactly as it should be, since ES averages over the full tail rather than reporting only its edge. Because ES is a coherent risk measure in a precise mathematical sense that VaR is not (it satisfies a subadditivity property guaranteeing that diversification can never increase measured risk, which VaR can occasionally violate in pathological cases), banking regulators have shifted market-risk capital requirements from VaR toward Expected Shortfall in recent years, a change explored further in this chapter's discussion of regulatory capital. Figure 8.1 shows why the two measures differ: VaR marks only the boundary of the shaded loss tail, while ES averages over the entire tail beyond it.

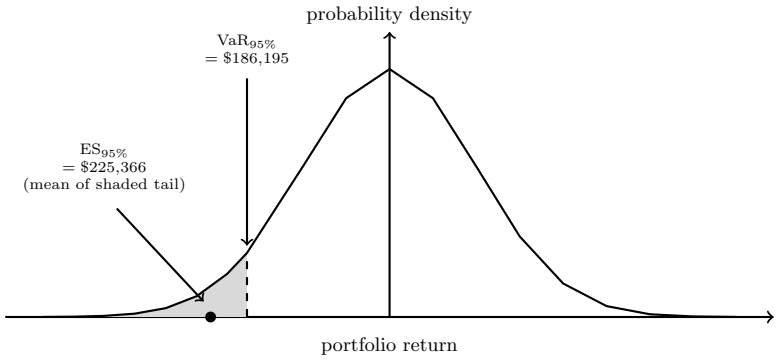


Figure 8.1: The parametric VaR and Expected Shortfall of Table 8.2's portfolio, illustrated against the assumed normal return distribution

8.2.4 Maximum Drawdown: A Path-Dependent Risk Measure

Value at Risk and Expected Shortfall both summarize the distribution of possible one-day losses, but neither says anything about a different, equally practical question: how much value would an investor holding this portfolio actually have seen erode from its own peak over an extended period, not just on any single day? Maximum drawdown answers this. Given a portfolio value path $V_0, V_1, \ldots, V_T$, the drawdown at time t is the percentage decline from the running peak up to and including that point,

$$\mathrm{Drawdown}_t = \frac{\max_{0 \le s \le t} V_s - V_t}{\max_{0 \le s \le t} V_s} \ge 0,$$

and the maximum drawdown over the full path is simply the worst (largest) drawdown observed anywhere along it, $\mathrm{MDD} = \max_{0 \le t \le T} \mathrm{Drawdown}_t$. Unlike VaR and Expected Shortfall, which depend only on the distribution of individual returns and not on their order, maximum drawdown is inherently path-dependent. Shuffling the same twenty daily returns into a different sequence would generally change the maximum drawdown, even though every VaR and Expected Shortfall figure computed from them would be identical. Both the peak against which decline is measured and the trough that realizes it depend on exactly *when*, not just how large, the losses occur.

Applying this to the cumulative value path implied by Table 8.2's twenty daily returns, starting from the portfolio's initial \$10,000,000, the running peak of \$10,048,025 is reached after Day 2, following two small positive returns, and the portfolio's value never again exceeds that peak for the remainder of the sample; its lowest point, \$9,371,210, occurs at the very end, Day 20, after four consecutive negative returns on Days 17 through 20. The maximum drawdown is therefore

$$\mathrm{MDD} = \frac{10{,}048{,}025 - 9{,}371{,}210}{10{,}048{,}025} \approx 6.74\%, \qquad \text{a peak-to-trough decline of } \$676{,}815.$$

Wulin.Suo@Queensu.ca

This is a materially different, and arguably more decision-relevant, number than the portfolio's cumulative return over the full twenty days, only −6.29%. The extra 0.45 percentage points of drawdown reflect the fact that the portfolio was briefly *up* \$48,025 before its losses began. An investor who happened to hold the position from Day 2 onward therefore experienced a larger peak-to-trough decline than a naive start-to-end comparison alone would reveal. This is the sense in which maximum drawdown captures something the chapter's daily VaR figures cannot. A portfolio whose losses on every individual day comfortably clear a well-calibrated 95% VaR threshold can still suffer a large, sustained drawdown if enough of those daily losses occur consecutively rather than being offset by intervening gains, the pattern in this example's final four days.

Maximum drawdown is used throughout this book wherever a full return path, not just a single day's distribution, is the natural object of interest. Chapter 13 uses it directly, alongside the Sharpe ratio and hit rate, to evaluate a backtested trading strategy's worst peak-to-trough decline. Asset managers and hedge funds routinely report it as a standard risk statistic, precisely because, unlike VaR, it requires no distributional assumption at all, only a value path. It is also immediately interpretable to an investor asking, in plain language, "how much could I have lost had I been unlucky enough to enter at the worst possible time." Its main limitation mirrors that same simplicity. A single realized maximum drawdown is one historical observation of a path-dependent statistic, not an estimate of a full distribution the way VaR or Expected Shortfall are. Comparing two strategies' historical maximum drawdowns therefore says less about their future worst-case risk than comparing their VaR or Expected Shortfall figures would.

8.2.5 Monte Carlo Simulation for Risk Measurement

A third approach, Monte Carlo simulation, sits between the parametric and historical methods: it assumes a distributional model, as the parametric method does, but instead of relying on a closed-form formula, it simulates a very large number of hypothetical return scenarios from that model and reads VaR and ES directly off the simulated distribution, exactly as the historical method reads them off the observed one. Simulating 100,000 draws from a normal distribution with the same mean and standard deviation as Table 8.2 ($\bar{r} = -0.32\%$, $\hat{\sigma} = 0.937\%$) gives $\text{VaR}_{95\%} \approx \$187{,}460$, $\text{VaR}_{99\%} \approx \$251{,}690$, and $\text{ES}_{95\%} \approx \$226{,}378$. All are close to the parametric formulas above, as they should be: with enough simulated draws, Monte Carlo VaR converges to the exact parametric answer whenever the underlying model is one for which a closed form exists at all.

The method's real value emerges precisely when no such closed form is available, for example a portfolio containing options (whose payoffs are nonlinear in the underlying price, a case the delta-normal discussion later in this chapter returns to directly) or a model with fat tails, jumps, or time-varying volatility of the kind introduced in Chapter 7's GARCH models. In those cases, Monte Carlo simulation remains straightforward, simulate many paths under the more realistic model and read off the empirical percentile, while a closed-form parametric formula either does not exist or requires a much more complex derivation.

Volatility-Adjusted VaR: Connecting GARCH Forecasts to Risk Measurement

Every VaR method above shares a hidden assumption: each uses a single volatility estimate, computed once from a fixed historical window, and holds it fixed. Chapter 7's GARCH(1,1) model suggests a more responsive alternative, since volatility itself clusters and evolves: rather than a static historical standard deviation, a VaR estimate can be updated every day using the GARCH model's own one-step-ahead variance forecast, an approach widely used in practice and sometimes called volatility-adjusted or "GARCH VaR."

Recall Chapter 7's GARCH(1,1) recursion applied to Chapter 2's four AssetA returns ($\omega = 0.00002$, $\alpha = 0.10$, $\beta = 0.85$): the one-step-ahead forecast volatility was 1.95% immediately before the large +2.97% return at $t = 3$, and rose to 2.08% immediately after it. Applying the same parametric VaR formula as Section 8.2.2, but with each of these two GARCH-forecast volatilities in place of a static historical standard deviation, to a hypothetical \$10 million position gives

$$\text{VaR}_{95\%}^{\text{pre-shock}} = 1.645(0.0195)(10{,}000{,}000) \approx \$320{,}775,$$
$$\text{VaR}_{95\%}^{\text{post-shock}} = 1.645(0.0208)(10{,}000{,}000) \approx \$342{,}160,$$

Wulin.Suo@Queensu.ca

an increase of about 6.7% in a single day, purely from the GARCH model's own updated variance forecast, with no new position taken and no change to the portfolio itself. A VaR model relying on a rolling historical-window standard deviation instead would still be averaging in several of the calmer, pre-shock returns and would understate the position's risk for as long as those calmer observations remain in the averaging window, precisely the lag GARCH's recursive weighting is designed to avoid. This is the volatility-clustering-aware risk measurement Chapter 7 flagged GARCH as being useful for, applied directly to the VaR machinery of this chapter.

Filtered Historical Simulation: Combining the Two Methods

The historical method (Section 8.2.2) and the GARCH-based method just introduced solve two different problems well: the historical method captures the actual shape of the return distribution, its skew and fat tails, without imposing a normal-distribution assumption, while GARCH captures how much that distribution's width is currently elevated or depressed relative to its long-run average. Filtered historical simulation combines both strengths in a single procedure. Each historical return r_t is first standardized by the GARCH volatility estimate prevailing on that same day, $z_t = r_t/\sigma_t$, producing a series of standardized residuals that should be roughly free of volatility clustering if the GARCH model is well specified; VaR is then read off the appropriate percentile of this standardized series, and rescaled by *today's* GARCH forecast volatility rather than whatever volatility happened to prevail on the day each historical return was originally observed.

Applying this logic to the two GARCH forecasts already computed, $\sigma_3 = 1.95\%$ (pre-shock) and $\sigma_4 = 2.08\%$ (post-shock): a large historical return observed during the calmer $t = 3$ regime, divided by that day's own small σ_t, contributes a large standardized residual, exactly as it should, since that return was unusually large *relative to its own volatility regime*, not just in absolute terms.

Chapter 2's AssetA series supplies exactly such an observation. The +2.97% return arrived while $\sigma_3 = 1.95\%$ prevailed, the calmest forecast in the whole recursion, so its standardized residual is

$$z_3 = \frac{0.0297}{0.0195} = 1.523,$$

and rescaling that residual by the post-shock forecast $\sigma_4 = 2.08\%$ gives a hypothetical return for today's conditions of $1.523 \times 0.0208 = 3.17\%$. Plain historical simulation would have reused the 2.97% figure unchanged, understating the move by about twenty basis points for no reason other than that the return happened to be observed in a calmer volatility regime than the one now prevailing.

That is the general procedure: rescaling a standardized residual by today's forecast volatility, whatever it currently is, produces a properly scaled hypothetical return for today's actual conditions, avoiding both the plain historical method's problem (treating every day's return as if it occurred under today's volatility) and the plain parametric method's problem (imposing normality on a shape that raw historical data might contradict). Filtered historical simulation is the kind of practical middle ground large trading desks use in production VaR systems, requiring the same GARCH machinery already developed above, applied across an entire historical return series rather than to a single day's forecast.

8.3 Validating and Extending the Measures

A risk number nobody has tested is a decoration; this part checks the measures against realized losses and extends them where the simple versions break.

8.3.1 Backtesting Value at Risk Models

A VaR model is only useful if it is actually calibrated correctly, and the standard way to check this is backtesting: comparing how often realized losses actually exceeded the model's VaR threshold (an "exception") against how often the model itself predicted an exception should occur. A well-calibrated 95% VaR model should produce an exception on roughly 5% of days, no more and, just as importantly, no fewer, since a model that is never exceeded is not necessarily good news; it may simply be too conservative, tying up capital that could otherwise be deployed productively.

Wulin.Suo@Queensu.ca

Applying the parametric 95% VaR threshold from Section 8.2.2 (a daily return of -1.862%, equivalent to the \$186,195 VaR figure) to the twenty returns in Table 8.2, exactly one return, the -2.22% observation, falls below the threshold, for an observed exception rate of $1/20 = 5\%$, matching the model's target almost exactly. Applying the parametric 99% threshold (a return of -2.501%) produces zero exceptions, consistent with an expected count of $0.01 \times 20 = 0.2$ exceptions, since zero is the natural outcome when the expected count is already well below one.

Kupiec's proportion-of-failures (POF) test formalizes this comparison, testing the null hypothesis that the model's true exception rate equals its target p against a likelihood-ratio statistic,

$$LR_{\text{POF}} = -2 \ln \left[\frac{(1-p)^{n-x} p^x}{(1-x/n)^{n-x} (x/n)^x} \right],$$

which is compared against a chi-squared distribution with one degree of freedom (critical value 3.841 at the conventional 95% test level) under the null. For the twenty-day sample above, with $p = 0.05$, $n = 20$, and $x = 1$ exception, the observed rate $x/n = 1/20 = 0.05$ exactly equals the target p, so the numerator and denominator are identical and $LR_{\text{POF}} = 0$, nowhere close to rejecting the null.

This looks like a perfect endorsement of the model, but it is really a demonstration of the same small-sample caution raised throughout this book: a single additional bad day would move the observed rate to $2/20 = 10\%$, and even that doubled exception rate would not, by itself, generate enough evidence at only twenty observations to reliably distinguish a well-calibrated model from a miscalibrated one. Kupiec's test, and the Basel traffic-light framework built on the same underlying logic, require a considerably longer history, typically the full one-year, 250-observation window described below, before they have any real statistical power, exactly as Chapter 7's Dickey-Fuller test did: a test can be constructed and interpreted correctly and still lack the power to detect a real problem on a short sample.

Backtesting failures are taken seriously by regulators: under the Basel framework, a bank whose internal VaR model produces too many exceptions in a rolling one-year window can be required to hold additional regulatory capital, using a multiplier applied directly to the reported VaR figure, which creates a direct financial incentive for banks to calibrate their risk models honestly rather than reporting an artificially low VaR to reduce capital requirements.

The Basel Traffic-Light Backtesting Framework

Basel formalizes this backtesting logic into a three-zone "traffic light" system, based on the number of VaR exceptions observed over a rolling 250-trading-day window at the 99% confidence level, where $0.01 \times 250 = 2.5$ exceptions are expected on average if the model is correctly calibrated. Zero to four exceptions fall in the green zone, requiring no action beyond continued monitoring and leaving the baseline capital multiplier of 3 unchanged. Five to nine exceptions fall in the yellow zone, triggering closer supervisory scrutiny and an increased capital multiplier that scales up within the zone; and ten or more exceptions fall in the red zone, where the model is presumed flawed and the multiplier rises to its maximum regulatory level of 4.

This framework only works as intended over its full 250-day design window, which is worth underscoring using this chapter's own toy example: Table 8.2's twenty-day, one-exception sample cannot be classified into a zone at all, since the traffic-light thresholds are calibrated to 250 observations, not twenty. Naively extrapolating the observed rate ($1/20 = 5\%$) to a full year would suggest roughly twelve or thirteen exceptions, which would fall well into the red zone. But doing so would repeat the small-sample overreach Section 8.3.1 just cautioned against. Twenty days is nowhere near enough data to support that extrapolation, and the traffic-light system is explicitly designed to be read only once a full 250-day window has actually been observed, not inferred from a short sample no matter how suggestive it looks.

Backtesting Expected Shortfall

The regulatory shift from VaR to Expected Shortfall raises a genuine methodological complication: VaR backtesting reduces to counting exceptions, a simple binary event (did the loss exceed the threshold or not) that Kupiec's test and the traffic-light framework both build on directly, but ES is an average of losses *within* the tail, so a single exceedance day does not, by itself, say whether the model's ES forecast was too high, too low, or about right.

Wulin.Suo@Queensu.ca

A common practical approach compares the *realized* loss on each exception day to the model's ES forecast for that same day, averaged across however many exceptions have occurred. Table 8.2's single 95% exception, the -2.22% return, realizes a loss of $0.0222 \times 10{,}000{,}000 = \$222{,}000$ against the parametric $\text{ES}_{95\%}$ forecast of \$225,366 computed earlier, a ratio of about 0.985, suggesting the forecast was, if anything, very slightly conservative on this one occurrence.

A single exception is obviously far too little evidence to validate an ES model on its own, but the underlying comparison, average realized loss on exception days against the model's own ES forecast for those days, is how a real trading desk accumulates the longer track record needed to judge whether its ES model is well calibrated, not merely whether its VaR threshold is crossed the right number of times.

From Daily VaR to Economic Capital: Time Scaling and Its Limits

Every VaR figure computed above answers a specific one-day question, but two further, related questions arise directly from it in practice: how much capital should be held against a longer regulatory horizon, and how much capital should the institution as a whole hold, over a full year, to survive a genuinely severe loss with high confidence? Both questions build on the same daily VaR already computed, using a time-scaling shortcut and a much higher confidence level, respectively.

Basel's market-risk capital framework historically scaled a bank's daily VaR up to a 10-day regulatory horizon using the square-root-of-time rule,

$$\text{VaR}_T \approx \text{VaR}_1 \sqrt{T},$$

which holds exactly when daily returns are independent and identically distributed, since the variance of a sum of T independent, identically distributed daily returns is exactly T times the one-day variance. Applying this to the parametric 99% VaR of \$250,081 computed in Section 8.2.2 gives a 10-day regulatory VaR of $250{,}081\sqrt{10} \approx \$790{,}824$, computed once from the same daily figure rather than requiring an entirely separate 10-day model.

The independence assumption behind this shortcut, however, is the assumption Chapter 7 spent an entire chapter warning against: real daily returns are rarely perfectly independent, and even a modest positive autocorrelation meaningfully changes how fast multi-day risk actually grows. If daily returns instead followed an AR(1) process with a modest autocorrelation of $\phi = 0.10$ (much weaker than the $\phi = 0.71$ Chapter 7 estimated for interest-rate deviations, but still a realistic magnitude for daily equity returns), the variance of a T-day cumulative return is

$$\text{Var}\left(\sum_{t=1}^{T} r_t\right) = T\sigma^2 + 2\sigma^2 \sum_{k=1}^{T-1} (T-k)\phi^k,$$

which for $T = 10$ gives a true 10-day volatility about 9.4% higher than the square-root-of-time rule assumes, and correspondingly a true 10-day 99% VaR of about \$865,413 rather than \$790,824: the square-root-of-time shortcut, applied to autocorrelated returns, quietly understates the regulatory capital a bank should actually hold, in the direction that matters (understating risk, not overstating it).

Economic capital extends this same time-scaling logic one step further, from a 10-day regulatory horizon to a full one-year horizon calibrated to a much higher confidence level, typically 99.9% or higher, chosen to be consistent with the default probability of the credit rating (commonly single-A or double-A) a bank wants its own solvency to reflect; Chapter 9 develops the analogous calculation directly for credit portfolios using the Vasicek model. Unlike the regulatory VaR multiplier in Section 8.3.1, which is a supervisory capital charge computed from a specific, prescribed methodology, economic capital is a bank's own internal estimate of how much capital its entire risk-taking business genuinely needs to survive a severe, low-probability year. It is used as much for internal capital allocation and performance measurement as for regulatory compliance: a desk generating a given return on a smaller economic-capital charge is, all else equal, using the firm's scarce capital more efficiently.

8.3.2 Liquidity Risk and Liquidity-Adjusted VaR

Every VaR calculation so far has implicitly assumed that a position can be liquidated at the current market price, but Chapter 3 already established that this is not quite right: selling a position, especially a large or

Wulin.Suo@Queensu.ca

illiquid one, costs at least half the bid-ask spread relative to the mid price, and a large enough order can walk through several levels of the order book at a progressively worse price, just as in Chapter 3's limit-order-book example. Liquidity-adjusted VaR (LVaR) incorporates this directly by adding an estimated liquidation cost to the standard VaR figure,

$$\text{LVaR}_\alpha = \text{VaR}_\alpha + \frac{1}{2}\,(\text{spread}) \times V,$$

where the spread is expressed as a fraction of position value and the factor of one-half reflects the cost of crossing from the mid price to the relevant side of the quote, the same logic used to compute the round-trip trading cost in Chapter 3. Applying this to the \$2 million two-asset position from Section 8.2.2, assuming an average bid-ask spread of 30 basis points, the liquidity cost adjustment is

$$\frac{1}{2}(0.0030)(2{,}000{,}000) = \$3{,}000, \qquad \text{LVaR}_{95\%} = 19{,}182 + 3{,}000 = \$22{,}182,$$

an increase of roughly 16% over the standard parametric VaR of \$19,182. This adjustment matters most precisely in the conditions where it is most often ignored. During a market-wide liquidity crunch, bid-ask spreads themselves widen sharply; recall from Chapter 3 that spreads compensate market makers for inventory risk and adverse selection, both of which increase during stressed markets. The true cost of liquidating a position exactly when a firm most needs to reduce risk can therefore be far higher than a liquidity adjustment calibrated on calm-market spreads would suggest, another instance of the procyclicality problem discussed later in this chapter.

Market liquidity risk, the focus above, is distinct from funding liquidity risk, the risk that a firm cannot meet its own near-term cash obligations, and Basel III addresses the latter directly with the Liquidity Coverage Ratio (LCR),

$$\text{LCR} = \frac{\text{High-Quality Liquid Assets}}{\text{Total Net Cash Outflows over 30 days}} \geq 100\%,$$

which requires a bank to hold enough genuinely liquid assets (cash and high-grade government bonds, broadly) to survive 30 days of a specified stress scenario without needing to raise new funding. A bank with \$50 million in high-quality liquid assets and projected net cash outflows of \$40 million over a 30-day stress scenario has $\text{LCR} = 50/40 = 125\%$, comfortably above the regulatory minimum. The LCR is best understood as a funding-side complement to the market-side liquidity-adjusted VaR above. Liquidity-adjusted VaR asks how much a specific position would cost to sell; the LCR asks whether the firm as a whole has enough liquid resources on hand to avoid being a forced seller of anything at all during a period of stress. That is precisely the scenario, forced selling into an already-stressed and illiquid market, that turned market liquidity risk into a systemic problem during the 2008 crisis discussed later in this chapter.

Basel III pairs the LCR with a second, longer-horizon ratio, the Net Stable Funding Ratio (NSFR),

$$\text{NSFR} = \frac{\text{Available Stable Funding}}{\text{Required Stable Funding}} \geq 100\%,$$

which asks essentially the same question as the LCR, does the bank have enough stable funding on hand, but over a one-year horizon rather than 30 days, and weighted toward the structural mismatch between how long a bank's assets take to mature (loans, often years) and how quickly its liabilities could be withdrawn (deposits, often on demand). Available stable funding weights a bank's capital and liabilities by how reliably they will still be there in a year (retail deposits are weighted as more stable than short-term wholesale borrowing, for instance), while required stable funding weights its assets by how long they are locked up (a long-dated loan requires more stable funding backing it than an overnight reverse repo). Where the LCR addresses a sudden, short-term liquidity run, the NSFR addresses the slower-moving risk of a bank funding long-term assets with short-term, flighty liabilities, the maturity-transformation mismatch that turned a funding problem into a solvency problem for several institutions during the 2008 crisis once wholesale funding markets froze.

VaR for Nonlinear Portfolios: The Delta-Normal Approximation and Its Limits

Every VaR calculation so far has assumed a linear portfolio: one whose value moves in direct, constant proportion to the underlying risk factor. A portfolio containing options violates this assumption, and Chapter 5's Greeks give the vocabulary to see exactly how.

Wulin.Suo@Queensu.ca

Consider a market maker short 1,000 of the one-year, \$50-strike calls priced in Chapter 5 ($S_0 = K = \$50$, $r = 4\%$, $\sigma = 30\%$, $C_0 \approx \$6.88$, $\Delta = 0.6115$, $\Gamma = 0.0255$). The delta-normal method approximates the position's one-day loss using only delta, treating the option price as locally linear in the stock price. Converting the annual volatility to a daily figure the standard way, $\sigma_{\text{daily}} = 0.30/\sqrt{252} \approx 1.90\%$, the position's delta-implied one-day dollar standard deviation is

$$\Delta \times S_0 \times \sigma_{\text{daily}} \times 1{,}000 = 0.6115 \times 50 \times 0.0190 \times 1{,}000 \approx \$577.85,$$

giving $\text{VaR}_{95\%}^{\text{delta-normal}} \approx 1.645(577.85) \approx \$950.56$ and $\text{VaR}_{99\%}^{\text{delta-normal}} \approx 2.326(577.85) \approx \$1{,}344.08$.

Full revaluation instead reprices the same option with the same Black-Scholes-Merton formula at a stressed stock price, holding strike, rate, volatility, and time to maturity fixed (a simplification that ignores one day of time decay, isolating the delta and gamma effect this example is meant to illustrate). The worst case for a short call is the stock rising, so at the 95% stressed price $S_0(1 + z_{0.95}\sigma_{\text{daily}}) \approx \$51.55$, the same option reprices to \$7.86, a position loss of \$980.79, about 3.2% *more* than the delta-normal estimate. At the more extreme 99% stressed price of \$52.20, the option reprices to \$8.28, a loss of \$1,403.99, now about 4.5% more than delta-normal predicted.

A delta-gamma approximation closes most of this gap without requiring a full revaluation, by adding a quadratic correction term built directly from the same gamma already computed in Chapter 5,

$$\Delta P \approx \left(\Delta\,\Delta S + \tfrac{1}{2}\Gamma\,\Delta S^2\right) \times n_{\text{contracts}}.$$

With $\Delta S_{95} = S_0 z_{0.95}\sigma_{\text{daily}} \approx \$1.554$ and $\Delta S_{99} \approx \$2.198$, this gives delta-gamma losses of

$$1{,}000\left(0.6115(1.554) + \tfrac{1}{2}(0.0255)(1.554)^2\right) \approx \$981.43,$$
$$1{,}000\left(0.6115(2.198) + \tfrac{1}{2}(0.0255)(2.198)^2\right) \approx \$1{,}405.79,$$

within a few cents of the full-revaluation figures of \$980.79 and \$1,403.99, compared to delta-normal's \$950.56 and \$1,344.08. The quadratic term costs only one extra number, gamma, already available from Chapter 5's Greeks, and recovers nearly all of the accuracy a full repricing would have provided, which is why delta-gamma VaR, rather than delta-normal alone or full revaluation of every position, is the standard compromise used in practice for portfolios with too many options to reprice individually at every stressed scenario. This growing gap is exactly gamma's effect: delta describes the option's slope at the current stock price, but a genuinely large move travels far enough along the price-value curve that the slope itself has changed by the time the move is complete, and gamma measures precisely that curvature. The delta-normal method is a reasonable, cheap approximation for small moves or a portfolio with little gamma exposure relative to its size, but it systematically understates risk for a portfolio with meaningful gamma, understating it more severely the larger and more extreme the move being measured, exactly why the 99% gap above is proportionally wider than the 95% gap. A trading desk with substantial options exposure therefore typically supplements delta-normal VaR with full revaluation, or with a second-order delta-gamma approximation that adds a quadratic correction term, precisely to correct for this shortfall.

8.3.3 Sensitivity-Based Risk Limits

A desk can sit comfortably inside every VaR limit it has been given and still be carrying the position that ends its year. The reason is structural rather than a failure of calibration: VaR compresses an entire book into a single number, and that compression is exactly what hides where the risk actually sits. Two desks reporting the same \$300,000 daily VaR can hold portfolios with nothing in common, one of them perfectly hedged against the move that is about to happen and the other pointed directly into it.

Practitioners therefore run sensitivities alongside VaR, never instead of it. A sensitivity answers a different and much narrower question than VaR does: not "how bad could tomorrow be, all things considered," but "if this one risk factor moves by one unit, and nothing else changes, how much money do we make or lose?" The answers are unglamorous, linear, and immediately actionable, which is precisely why the trading floor lives on them.

Wulin.Suo@Queensu.ca

The Greeks as Limits in Their Own Right

Chapter 5 computed delta, gamma, vega, and theta for the option this chapter has been using, and the delta-normal approximation earlier in this chapter then drew on the first two of them, but only as ingredients in a VaR approximation. On a real derivatives desk each is a limit in its own right, monitored and enforced independently of any VaR number. Return to that section's position, a market maker short 1,000 of Chapter 5's one-year \$50-strike calls ($S_0 = K = \$50$, $r = 4\%$, $\sigma = 30\%$), and read the whole set as a risk report rather than as pricing output. Table 8.3 collects them, scaled to the position and joined by rho, the one Greek Chapter 5 did not compute.

Greek	Measures exposure to	Position	Limit	Utilization
Delta	a \$1 move in the stock	−611.5	1,500	40.8%
Gamma	the change in delta per \$1 move	−25.55	50	51.1%
Vega	a one-percentage-point move in implied volatility	−\$191.62	\$250	76.6%
Theta	the passage of one day	+\$10.47	—	—
Rho	a one-percentage-point move in interest rates	−\$237.00	\$400	59.3%

Table 8.3: The full Greek profile of a short position in 1,000 one-year \$50-strike calls, with representative desk limits

Read the table as a risk manager would, one line at a time. The position is short delta, so it loses money if the stock rallies. It is short gamma, meaning that delta itself moves against the position in both directions, the effect the delta-normal example isolated numerically. It is short vega, so it loses if implied volatility rises. It is long theta, collecting \$10.47 a day in time decay for as long as nothing happens, which is the market maker's entire business model and the compensation for being short gamma and vega. Note the change of units from Chapter 5, which quoted this same option's theta as −\$3.82 per year: a risk report is read every morning, so a desk quotes theta per day, and dividing the annual figure by 365 recovers exactly the number above. And it is short rho, though at this maturity that is the least of its concerns.

The critical observation is the one the VaR machinery of this chapter cannot make. Every VaR figure computed for this option position earlier in the chapter, whether delta-normal, delta-gamma, or full revaluation, shocks the *stock price* and holds implied volatility fixed at 30%. The model is therefore structurally blind to vega: a change in implied volatility is not among the risk factors it perturbs, so no amount of recalibration or backtesting will make that risk appear in the number. Yet a five-percentage-point rise in implied volatility, from 30% to 35%, is an entirely ordinary single-day move in a stressed market, and it would cost this position

$$1{,}000 \times \$0.1916 \times 5 = \$958,$$

against a 99% full-revaluation VaR of \$1,403.99 attributed to price moves alone. A routine volatility move, invisible to the risk report, is worth 68% of the entire price-driven 99% VaR. This is not an argument against VaR. It is an argument for reading a vega limit next to it.

Bucketed Sensitivities: The DV01 Ladder

The same logic applies with even more force to interest-rate books, where the risk factor is not a single price but an entire yield curve. The standard sensitivity there is DV01, the dollar value of a basis point: the profit or loss produced by a one-basis-point move in yields. A single aggregate DV01 for the whole book is the rates equivalent of a single VaR number, and it conceals the same thing.

Consider a desk whose positions are reported in the four standard maturity buckets of Table 8.4, with each figure giving the position's profit or loss from a one-basis-point *rise* in that bucket's yield, so that a long bond position appears as a negative number.

Wulin.Suo@Queensu.ca

Bucket	DV01 (per bp)	Parallel +10bp	Flattening scenario
2-year	−$25,000	−$250,000	−$500,000
5-year	−$40,000	−$400,000	−$400,000
10-year	+$38,000	+$380,000	−$190,000
30-year	+$27,000	+$270,000	−$270,000
Net	$0	$0	−$1,360,000

Table 8.4: A bucketed DV01 ladder whose net exposure is exactly zero

The net DV01 of this book is exactly zero. A risk report carrying only that aggregate would describe the desk as flat, immune to interest-rate risk, and under a parallel shift it genuinely is: adding ten basis points to every yield produces gains and losses that cancel to the cent, as the middle column shows.

But the yield curve does not only move in parallel, and this desk is not flat at all. Its *gross* DV01, the sum of the absolute bucket exposures, is $130,000 per basis point. It is long the short end and short the long end, a position that profits if the curve steepens and loses if it flattens. Applying a flattening scenario in which two-year yields rise 20 basis points, five-year yields rise 10, ten-year yields fall 5, and thirty-year yields fall 10 gives

$$(-25{,}000)(20) + (-40{,}000)(10) + (38{,}000)(-5) + (27{,}000)(-10) = -\$1{,}360{,}000,$$

a loss of $1.36 million from a curve move that the net-DV01 report scores as zero exposure. The bucketed ladder is what makes this visible, and it is why rate desks are given limits per bucket rather than one limit on the total. The general principle transfers directly to the equity and credit books discussed elsewhere in this chapter: aggregate where you must, but never let the aggregate be the only thing anyone looks at.

This is not merely a practitioner convention. It is also the direction regulation has taken, as the sensitivities-based method at the heart of the Fundamental Review of the Trading Book computes capital from these bucketed delta, vega, and curvature sensitivities rather than from a single portfolio-level loss distribution.

There is also a direct link back to Chapter 7's principal component analysis of the yield curve, which is worth making explicit because it explains *why* the ladder decomposes risk the way it does. That analysis found that essentially all of the curve's daily variation is captured by three factors: a level factor that shifts all maturities together, a slope factor that tilts short against long, and a curvature factor. Projecting this desk's ladder onto those factors restates the whole example in one line. A net DV01 of zero is exactly zero exposure to the *level* factor, which is why the parallel shift produces nothing; the desk's real position is a large exposure to the *slope* factor, and the flattening scenario is simply a slope-factor move. Bucketing by maturity and then reasoning about the buckets is the practitioner's version of a factor decomposition, and a desk that has already run the PCA can set limits on the three factor exposures directly, which is both more compact and better aligned with how the curve actually moves than limits on four maturity buckets treated as if independent.

Computing Sensitivities at Scale: Adjoint Differentiation

The two preceding subsections asked for five Greeks and four DV01 buckets, nine numbers in total. A real trading book asks for thousands, across every instrument, every risk factor, and every maturity bucket, recomputed at least daily. How those numbers are actually produced turns out to be a computational problem with a direct and rather elegant answer from machine learning.

The obvious method is to bump the input and reprice: value the book, add a basis point to one risk factor, value it again, and difference the two. This is easy to implement and completely correct, and it does not scale, because each sensitivity costs one additional full revaluation. A book exposed to n risk factors therefore costs $n + 1$ valuations, and when the instruments involved are themselves priced by Monte Carlo simulation, so that a single valuation takes minutes rather than milliseconds, and n runs into the thousands, this is precisely how risk reporting turned into an overnight batch process that a trader cannot interrogate during the day.

Wulin.Suo@Queensu.ca

Adjoint algorithmic differentiation removes the dependence on n altogether. Its cost is bounded by a small constant multiple, typically three to five times, of a *single* valuation, no matter how many sensitivities are requested. A book that took an hour to produce one set of Greeks by bumping produces the entire set in minutes.

The mechanism will be immediately familiar to any reader who has trained a neural network, because it is the same mechanism. A pricing formula, exactly like a network's forward pass, is a composition of elementary operations, each of whose local derivative is known analytically. Evaluating the formula once while recording that composition builds a computational graph. Seeding the output with a derivative of one and applying the chain rule backward through the graph then accumulates the derivative of the output with respect to every input simultaneously, in a single reverse sweep. This is reverse-mode automatic differentiation, and backpropagation is nothing other than its application to a loss function instead of a price. The reader who understands why training a network with a million parameters costs only a constant multiple of a forward pass already understands why a bank can compute a thousand Greeks for the price of a few valuations: it is the same theorem.

Applying this directly to the Black-Scholes price of Table 8.3's option makes the point concrete. One forward evaluation records a graph of roughly twenty elementary nodes; one reverse sweep then returns $\partial C/\partial S$, $\partial C/\partial \sigma$, $\partial C/\partial T$, and $\partial C/\partial r$ at once, giving a delta of 0.611539, a vega of 0.191623 per percentage point of volatility, a theta of -0.010472 per day, and a rho of 0.237003 per percentage point of rates. Every one of these agrees with the closed-form expression to nine decimal places, which is the other practical advantage of the method: unlike a bumped finite difference, an adjoint derivative is exact up to floating-point rounding, with no step size to choose and no cancellation error to trade off against truncation error.

Gamma is the instructive exception. It is a second derivative, and a single reverse sweep returns only first derivatives, so obtaining it requires differentiating the derivative, that is, applying the machinery to its own output. This is the distinction between a gradient and a Hessian in machine learning, and it carries the same cost implication: first-order sensitivities are nearly free, and second-order ones are not. Production systems handle this with nested automatic differentiation, the same construction used for Hessian-vector products when training a model with a second-order optimizer.

The technique entered finance through Giles and Glasserman's 2006 work on Monte Carlo Greeks and is now standard across derivatives desks and in the valuation adjustment calculations that dominate modern derivatives infrastructure. It is worth pausing on what this says about the relationship between the two fields this book spans. Adjoint differentiation is not an application of machine learning to finance in the usual sense, since no model is trained and nothing is learned from data. It is something arguably more interesting: a single algorithm, developed and popularized in one field, turning out to be the answer to a computational problem in the other, for reasons that have nothing to do with the subject matter and everything to do with the structure of differentiation itself.

8.4 Stress Testing and Machine Learning

Historical data cannot describe an event that has not happened yet, which is where scenarios and learned models both enter.

8.4.1 Stress Testing and Scenario Analysis

VaR and Expected Shortfall are inherently backward-looking in an important sense: even the parametric and Monte Carlo versions are calibrated to a historical mean and standard deviation, so they implicitly assume that the future will statistically resemble the recent past. Stress testing addresses this by asking a different question entirely: not "how bad are the worst outcomes the historical distribution suggests," but "what would happen to this portfolio under a specific, named adverse scenario," whether historical (such as replaying the return pattern from the 2008 financial crisis or the 2020 COVID-19 market shock) or hypothetical (such as a sudden 200-basis-point rate shock or a 30% equity market decline).

Applying a historical stress scenario, a single-day return of -7%, comparable to some of the worst individual trading days during the 2008 crisis, to the \$10 million portfolio in Table 8.2 gives a stressed loss of

$$\text{Stressed Loss} = 0.07 \times \$10{,}000{,}000 = \$700{,}000,$$

Wulin.Suo@Queensu.ca

nearly three times the parametric 99% VaR of \$250,081 computed in Section 8.2.2.

This gap is not a sign that the VaR calculation was wrong; it is the entire point of stress testing. VaR, by construction, describes a specific quantile of a specific assumed distribution, while a stress scenario deliberately explores an outcome that a short, calm historical sample may never have contained at all. Regulators require large financial institutions to run this kind of exercise (in the United States, the Federal Reserve's Comprehensive Capital Analysis and Review, or CCAR, is a prominent example) precisely because a risk model calibrated only on recent, relatively calm data can systematically understate the likelihood and severity of a genuine crisis, the same non-stationarity concern raised about financial time series models throughout Chapter 7.

Standard stress testing, as above, starts from a scenario and asks how much the firm would lose; reverse stress testing runs the same logic backward, starting from a specific outcome, such as the firm's capital falling below the regulatory minimum or the firm becoming insolvent entirely, and asking what combination of market moves, defaults, or funding shocks would be required to produce it. This reframing is deliberately designed to counteract a subtle bias in forward stress testing: scenario designers, drawing on recent history and their own intuition about what a bad year looks like, tend to generate scenarios that resemble past crises rather than genuinely novel ones, while reverse stress testing forces an explicit search over the space of outcomes the firm cannot survive, regardless of whether any such scenario has ever actually occurred, and then asks the risk committee to judge how plausible that outcome really is.

Concretely, suppose the desk running the \$10 million portfolio in Table 8.2 has been allocated a risk capital cushion of \$500,000 specifically to absorb this book's trading losses before it would need additional capital from elsewhere in the firm. Reverse stress testing asks directly: what single-day return would exhaust this cushion? Since Loss $= |r| \times \$10{,}000{,}000$, solving $500{,}000 = |r| \times 10{,}000{,}000$ gives $|r| = 5\%$, a single-day move of -5% or worse. Comparing this to the -7% historical stress scenario used above, the reverse stress test reveals something the forward scenario alone did not make explicit: the historical stress scenario, if it recurred, would not merely breach this cushion but exceed it by $\$700{,}000 - \$500{,}000 = \$200{,}000$, a genuine capital shortfall rather than a mere limit breach, information a risk committee needs well before such a day actually arrives rather than discovering it only after the fact.

Correlation Stress Testing: Quantifying "Correlations Go to One"

The case vignette below describes correlations rising sharply during a genuine crisis, but it is worth making that claim numerically concrete using the same two-asset position from Section 8.2.2. Chapter 2's AssetA/AssetB covariance matrix implies a calm-period correlation of

$$\rho_{\text{calm}} = \frac{-0.000198}{\sqrt{(0.000414)(0.000118)}} \approx -0.896,$$

a strongly diversifying, negative relationship, which is why the equally weighted portfolio's volatility, 0.583%, is so much lower than either asset's own volatility ($\sigma_A \approx 2.03\%$, $\sigma_B \approx 1.09\%$) individually. Stress the correlation toward the crisis-period pattern described qualitatively in the vignette, holding each asset's own volatility fixed but replacing ρ_{calm} with a stressed $\rho_{\text{stress}} = 0.5$: the stressed covariance becomes $\rho_{\text{stress}}\sigma_A\sigma_B \approx 0.0001105$, and the resulting portfolio volatility is

$$\sigma_p^{\text{stress}} = \sqrt{w^\top \Sigma_{\text{stress}} w} \approx 1.372\%,$$

more than double the calm-period figure of 0.583%, so that $\text{VaR}_{95\%}^{\text{stress}} \approx 1.645(0.01372)(2{,}000{,}000) \approx \$45{,}141$, roughly 2.35 times the calm-period parametric VaR of \$19,182 computed earlier, with neither asset's own volatility having changed at all. This is the quantitative content behind the vignette's qualitative warning: a diversification benefit that looks substantial under calm-period correlations can shrink dramatically, and portfolio risk can more than double, purely because assets that used to move independently, or even oppositely, start moving together, the mechanism that made 2008 more dangerous for diversified portfolios than their pre-crisis risk models suggested.

Wulin.Suo@Queensu.ca

8.4.2 Machine Learning Applications in Market Risk

Every technique in this chapter so far is a classical statistical model, chosen deliberately because a risk number that feeds directly into regulatory capital and executive decision-making needs to be as transparent and auditable as possible. Machine learning nonetheless enters market risk management in several concrete, increasingly standard ways, each extending rather than replacing the tools already developed above.

Volatility forecasting is the most direct extension: a recurrent neural network or gradient-boosted tree ensemble can, in principle, capture nonlinear volatility patterns that a fixed GARCH(1,1) specification cannot, feeding the same volatility-adjusted VaR machinery introduced earlier in this chapter with a more flexible forecast, at the cost of the overfitting and interpretability concerns Chapter 6 raised and this chapter's own challenges section returns to below.

A Head-to-Head Test: GARCH(1,1) versus Gradient Boosting on Asymmetric Volatility

Chapter 7 flagged a specific, well-documented shortcoming of GARCH(1,1): because its recursion depends only on ε_{t-1}^2, it treats a positive and a negative shock of the same size as equally informative about tomorrow's variance, when in practice negative shocks (the leverage effect) tend to raise volatility more. A gradient-boosted tree, given the lagged return's sign as an explicit input feature rather than only its square, can in principle learn this asymmetry, something a plain GARCH(1,1) specification structurally cannot represent at all.

The companion notebook tests this directly: it simulates 750 days of returns from a known, asymmetric GJR-GARCH-style process ($\omega = 0.000005$, $\alpha = 0.05$, $\beta = 0.85$, with an extra variance term $\gamma = 0.12$ triggered only by negative shocks), fits a plain symmetric GARCH(1,1) by maximum likelihood on the first 600 days, and separately fits a gradient-boosted regressor on the same 600 days using lagged squared *and* signed returns as features, with hyperparameters chosen by time-series-aware cross-validation rather than an arbitrary guess.

Both models then forecast one step ahead across the remaining 150 days, and because the true simulated variance path is known exactly (it is not observable in real markets, which is precisely why this test requires simulated rather than real data), each forecast can be scored directly against it.

The result is a genuine surprise only if one assumes flexibility always wins: the misspecified, symmetric GARCH(1,1) achieves a forecast-variance RMSE of 2.416×10^{-5}, beating the sign-aware, cross-validated gradient-boosting model's RMSE of 3.830×10^{-5} by 36.9%, even though the gradient-boosting model had direct access to the information (the shock's sign) that GARCH(1,1) cannot use. The gradient-boosting model does still beat a naive constant-variance benchmark (4.202×10^{-5} RMSE) by 8.8%, so it is not learning nothing, but a correctly-structured recursive model, propagating variance forward one term at a time exactly as the true data-generating process does, retains a large practical advantage over a generic tree ensemble learning from a lagged-feature window, even when that ensemble is given the one piece of information it needs and is tuned carefully rather than left at default settings. This is a genuinely useful, sobering result, not a failure of the exercise: it is why volatility forecasting in practice tends to augment GARCH-family models with richer inputs (implied volatility, order flow, cross-asset signals) rather than replace the recursion outright with an off-the-shelf ML model trained on lagged returns alone, and it is a concrete illustration of the overfitting and interpretability tradeoff this section already warned a more flexible model carries.

A second application addresses a purely computational bottleneck. Full revaluation of a large options book, of the kind the delta-normal example performed for a single position, becomes prohibitively slow once thousands of positions must be repriced at thousands of simulated Monte Carlo scenarios. Some trading desks therefore train a neural network as a fast pricing surrogate, learned once from a pricing model like Black-Scholes-Merton or a full Monte Carlo engine, then queried repeatedly at simulation speed rather than re-solving the original pricing model at every scenario.

A third, newer application uses generative models, the GANs and diffusion models Chapter 16 introduces, to synthesize plausible stress scenarios beyond anything a firm's own historical window happens to contain, directly addressing the limitation Section 8.4.1 flagged: a historical sample that never included a severe crisis cannot, by construction, suggest one as a scenario, but a generative model trained to capture the statistical structure of extreme moves can still propose a novel, structurally plausible one.

Wulin.Suo@Queensu.ca

In every one of these applications, the underlying discipline is unchanged from the rest of this chapter: a more flexible model is validated, backtested, and governed exactly as rigorously as a classical one, and typically more so, precisely because its added flexibility makes an undetected flaw both easier to introduce and harder to catch.

8.5 Governance and Managing the Risk

Measuring a risk is not the same as owning it; this part covers who validates the numbers and what is actually done about them.

8.5.1 Model Validation and Model Risk Management

Every quantitative method in this chapter, VaR, Expected Shortfall, GARCH-based volatility forecasting, liquidity adjustment, is a model in the specific sense introduced in Chapter 1: a simplified representation of reality whose outputs are only as trustworthy as its assumptions. Model risk management is the discipline of formally governing this fact, treating every model used to make a material financial decision as an object requiring independent scrutiny before deployment and ongoing monitoring afterward, rather than trusting a model simply because it was built by a competent team.

A mature model risk management framework, closely following supervisory guidance such as the U.S. Federal Reserve and OCC's SR 11-7, organizes this scrutiny around three activities. Independent validation requires that a model be reviewed by staff who did not build it, checking the conceptual soundness of its assumptions (does a normal-distribution assumption make sense for this portfolio's return history), the correctness of its implementation (does the code actually compute what the mathematical specification says it should), and its outcomes (does backtesting, as in Section 8.3.1, confirm the model performs as claimed). Ongoing monitoring tracks a model's performance after deployment, since a model validated as sound at launch can degrade as market conditions change, the non-stationarity concern raised throughout this book, and a monitoring program exists specifically to catch that degradation before it causes a material loss rather than after. Model inventory and documentation maintain a complete, current record of every model in use, its owner, its validation status, and its known limitations, since a firm that does not know how many models it has, or where they are used, cannot possibly govern the risk they collectively pose.

In practice, a firm with hundreds of models cannot validate all of them with equal intensity, so most model risk management frameworks tier models by materiality. A model whose output could plausibly affect more than $50 million of capital or reported risk might be classified Tier 1, requiring full independent validation and annual revalidation, while a narrowly scoped model with less than $5 million of potential impact might be Tier 3, revalidated only every two to three years. The portfolio VaR model developed in this chapter, given its direct role in regulatory capital calculations, would almost certainly sit in the highest tier at any bank of meaningful size.

This formal governance layer, and the same three-part structure, applies equally to the credit risk models Chapter 9 develops. It is what separates a well-run quantitative risk function from a collection of individually clever models. The methods in this chapter are necessary but not sufficient for sound risk management, since even a mathematically correct model can produce dangerously misleading output if it is applied outside the conditions under which it was validated, the failure mode illustrated by this chapter's 2008 case vignette.

8.5.2 Managing the Risk, Not Just Measuring It

Every tool built so far in this chapter answers a question of the form "how much could we lose?" None of them answers the question a desk head actually has to act on, which is "and what are we going to do about it?" A risk report that never changes a position is an expensive newsletter. Measurement earns its keep only at the point where a number crosses a threshold and someone trades differently as a result.

There are only four things a firm can do with a risk it has measured. It can **accept** the risk, which is a legitimate decision when the exposure is small relative to capital and is being paid for. It can **reduce** it by cutting the position. It can **hedge** it by taking an offsetting position in a related instrument, keeping the exposure it wants and shedding the exposure it does not. Or it can **transfer** it to someone else through an insurance contract or a derivative, which Chapter 11 develops from the perspective of the party accepting

Wulin.Suo@Queensu.ca

such risks for a living. The rest of this section works through the mechanics of the two middle options, since those are the ones that require calculation.

Hedging: Removing an Exposure You Do Not Want

Return once more to the short call position of Section 8.3.3. Its delta of -611.5 says that the position loses \$611.50 for every \$1 the stock rises. A market maker who wants to earn the option's time decay without taking a directional view on the stock can eliminate that exposure directly, by buying 611.5 shares of the underlying. The hedged book's delta is then zero by construction, and the first-order price risk is gone.

What is left over is the interesting part. Delta hedging removes the linear exposure, but the position is still short 25.55 units of gamma, and gamma is precisely the rate at which the hedge goes stale as the stock moves. At the 99% one-day move of \$2.198 used in the delta-normal example, the delta-hedged position still loses

$$\tfrac{1}{2} \times 25.55 \times (2.198)^2 = \$61.72,$$

compared with the \$1,403.99 full-revaluation loss unhedged. The hedge removes 95.6% of the 99% loss, which is a large improvement and not a complete one. Two features of that residual matter more than its size. First, it is quadratic, so it grows with the square of the move: doubling the shock roughly quadruples the leftover loss. Second, and less obviously, it is *one-directional*. A short-gamma position loses on the residual whether the stock rises or falls, because the sign of ΔS disappears when it is squared. The delta hedge has converted a directional bet into a bet that the stock will not move much, which is a genuinely different risk and not an absence of one.

This is the honest general shape of hedging. A hedge does not remove risk; it exchanges a risk you have decided you do not want for a smaller, differently-shaped one you are willing to hold, and the residual is what the risk report must then track.

That residual is also the opening for a machine learning treatment of the same problem, and Chapter 19 takes it up directly on this very position. The calculation above quietly assumes the hedge can be rebalanced continuously and for free. It cannot: every adjustment to the share position pays a bid-ask spread, so a hedger who rebalances often enough to keep delta near zero pays away the option premium in transaction costs, while one who rebalances rarely enough to keep costs down carries the residual gamma risk just computed. There is no closed-form answer to where the optimum sits, because it depends on the cost structure, the volatility, and how much tail risk the desk is willing to tolerate. Chapter 19 solves it as a sequential decision problem, finding a rebalancing band that beats daily delta hedging on both average cost and tail risk simultaneously, and then generalizes the band to a neural hedging policy trained directly on the distribution of outcomes. The classical delta hedge computed here is the baseline that work is measured against.

Position Sizing: Controlling Risk Before It Exists

Hedging acts on a position that already exists. The cheaper control acts before the position is put on at all, by choosing its size so that the resulting risk is the risk the firm wanted in the first place.

Volatility targeting is the standard mechanical version. Take this chapter's \$10 million portfolio, whose daily return standard deviation of 0.937% was computed in Section 8.2.2. Scaling to an annual figure in the conventional way gives

$$\sigma_{\text{annual}} = 0.937\% \times \sqrt{252} = 14.87\%.$$

Suppose the mandate this portfolio is run under specifies a 10% annualized volatility target. The portfolio as it stands is running roughly half again as much risk as it was authorized to, and the fix requires no view on any individual holding: scale the whole book by the ratio of target to realized volatility,

$$\frac{10\%}{14.87\%} = 0.672,$$

holding \$6.72 million in the risky portfolio and the remaining \$3.28 million in cash. Because volatility scales linearly with position size while the portfolio's composition does not change at all, this hits the target exactly without any of the estimation difficulties that Chapter 12's mean-variance optimizer runs into.

Wulin.Suo@Queensu.ca

The catch is the one the whole chapter has been circling, and it is what makes position sizing a forecasting problem rather than an arithmetic one. The scaling factor is a ratio of a target to an *estimate*, so the entire control is only as good as that estimate, and volatility is not constant, which is precisely what Chapter 7's GARCH models exist to capture. This is not a peripheral concern: the difference between sizing on a trailing sample standard deviation and sizing on a conditional forecast that responds to yesterday's shock is the difference between de-risking after a turbulent week and de-risking during it. Section 8.4.2 tests this quantity, putting GARCH(1,1) head to head against a gradient-boosted tree on a series with a known asymmetric volatility response, and whichever model wins that comparison is the one that should be driving the scaling factor here. Volatility targeting is where a better volatility forecast turns directly into a better-sized position, which is a more concrete payoff than forecast accuracy statistics usually offer.

A volatility-targeted portfolio therefore rescales continually, cutting exposure as markets get turbulent and adding it back as they calm. That is the intended behavior, but it also means such a strategy sells into falling markets mechanically, alongside every other volatility-targeted fund doing the same arithmetic on the same day, which is a mechanism for the kind of correlated forced selling that Section 8.4.1 stress-tests for.

Stop-Losses and Drawdown Triggers, and Why They Disappoint

The last common control is the crudest: set a threshold, and when losses reach it, cut the position regardless of what any model says. Stop-losses have an obvious appeal, since they require no distributional assumption whatsoever and cannot be gamed by a mis-specified covariance matrix.

They also underdeliver, and this chapter's own data shows why. Section 8.2.4 computed a maximum drawdown of 6.74% for the twenty-day portfolio, reached on the final day. Suppose the desk had operated a rule liquidating the portfolio whenever its drawdown from the running peak reached 5%. Tracking the drawdown day by day, the rule would not have fired until day 19, and liquidating at that point would have avoided only the final day's -1.49% return. Out of a 6.74% peak-to-trough decline, the stop-loss rescues about 149 basis points.

The reason is generic rather than an artifact of this particular series. A drawdown trigger is a lagging indicator by construction: it cannot fire until the loss has already been taken, so it necessarily concedes the entire distance to the threshold plus whatever happens between the trigger and the fill. Worse, in a series that recovers rather than continuing to fall, the same rule crystallizes a loss that would have reversed, and does so precisely at the point of maximum pessimism. Stop-losses are best understood as a last-resort control against catastrophic tail behavior and operational failure, not as a risk management technique that improves expected outcomes. The tools that actually reduce risk in a systematic way are the ones this section opened with, sizing the position correctly before the fact, and hedging the exposures that are genuinely unwanted.

8.5.3 Risk Governance: The Three Lines of Defense

Every quantitative method in this chapter, VaR, Expected Shortfall, economic capital, exists inside an organizational structure that determines whether it is actually used correctly, and financial institutions typically organize risk oversight using a "three lines of defense" model, illustrated in Figure 8.2.

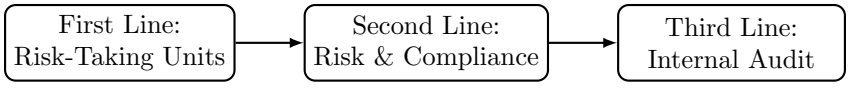


Figure 8.2: The three lines of defense in risk governance

The first line of defense is the business unit that actually takes the risk, a trading desk or a lending team, and is responsible for managing it day to day within approved limits. The second line is an independent risk management and compliance function, organizationally separate from the business units it oversees, that builds and maintains models like the VaR and economic capital methods introduced in this chapter and the credit-scoring methods of Chapter 9, sets risk limits, and monitors whether the first line is operating within them. The third line, internal audit, periodically and independently verifies that the first two lines are functioning as intended, checking not just whether a number was computed correctly but whether the

Wulin.Suo@Queensu.ca

entire risk management process was followed. This structure exists specifically to prevent the single point of failure that arises when the people generating revenue from a risk position are also the people responsible for measuring and reporting that risk, a conflict of interest that has been implicated in numerous historical risk management failures.

A model like the ones developed in this chapter is only as trustworthy as the governance structure that surrounds it: a perfectly specified VaR model computed by a risk function with no real independence from the trading desk it monitors provides little genuine protection. The second line of defense is typically led by a Chief Risk Officer (CRO), a senior executive with direct reporting access to the board of directors' risk committee, independent of the chief executive and business-line management, specifically so that a risk concern can reach the board without first being filtered by the same management chain whose performance the concern might reflect poorly on. This reporting-line independence is not a bureaucratic formality. Several well-documented historical risk management failures trace back to a risk function that identified a genuine problem internally but lacked the organizational standing or independence to have that concern acted on before it caused a material loss. That is precisely why modern governance codes and regulatory guidance treat CRO independence, board-level risk reporting, and the second line's authority to veto or unwind a position as essential features of a credible risk management function rather than optional best practices.

A concrete limit example shows how these three lines actually interact day to day. Suppose the second line has approved a daily 99% VaR limit of \$300,000 for the desk running the \$10 million portfolio from Table 8.2. The parametric 99% VaR of \$250,081 computed in Section 8.2.2 represents a limit utilization of $250{,}081/300{,}000 \approx 83.4\%$, comfortably within the approved limit, so the first line continues trading within its normal risk-taking mandate and the second line's monitoring is routine. Now suppose realized volatility rises sharply enough, following the volatility-adjusted VaR logic introduced earlier in this chapter, that the desk's VaR the next day increases to \$340,000, a breach of \$40,000, or roughly 13.3% over the approved limit. This is precisely the event the three-lines structure exists to catch. The breach must be reported to the second line the same day, which independently assesses whether the increase reflects a temporary, defensible spike in market volatility or a genuine change in the desk's risk profile that warrants an immediate position reduction. A limit breach that recurs or is not promptly remediated is the kind of finding the third line's periodic audit is designed to flag, even if the second line's day-to-day response was otherwise adequate.

8.6 Case Vignette: When Value at Risk Meets a Genuine Crisis

The 2008 financial crisis is the standard cautionary tale about the limits of the methods developed in this chapter, and it is worth walking through precisely why, since the failure was not a matter of arithmetic errors. Many large banks entered 2008 with VaR models built and validated exactly as this chapter describes: parametric or historical methods, calibrated on several years of historical returns, backtested against the kind of exception-counting procedure in Section 8.3.1, and generally passing those backtests during the calm years leading up to the crisis. The most famous symptom came early, in the August 2007 "quant quake" that preceded the main crisis. Explaining heavy losses at Goldman Sachs's own quantitative hedge funds, the firm's chief financial officer, David Viniar, told the *Financial Times*: "We were seeing things that were 25-standard deviation moves, several days in a row." A normal distribution assigns a single 25-standard-deviation move a probability of essentially zero, so small that it should not occur even once across the entire history of the universe under the model's own assumptions, let alone on several days in a row.

This did not mean the underlying arithmetic was wrong; it meant the normality assumption, the same parametric assumption used throughout Section 8.2.2 of this chapter, was a poor description of returns during a genuine crisis, the fat-tails caveat flagged repeatedly throughout this book. A model calibrated on a multi-year window that happened not to include a severe crisis will, by construction, underestimate the frequency of extreme moves, since it has literally never observed one in its training data, the same finite-sample problem behind this chapter's caution about short backtesting windows. Compounding this, correlations between asset classes that had appeared low or stable during calm periods rose sharply during the crisis, a pattern sometimes summarized as "correlations go to one in a crisis," which meant that diversification benefits assumed by portfolio-level VaR calculations, including the variance-covariance approach in Section 8.2.2, provided far less protection than historical correlations had suggested they would.

The lesson this vignette is meant to leave is not that VaR or Expected Shortfall are poor tools; both

Wulin.Suo@Queensu.ca

remain standard, required practice at every major financial institution today. It is that every method in this chapter answers the question it was designed to answer only as well as its underlying assumptions hold. A serious risk management practice therefore treats stress testing, backtesting, and the qualitative judgment of an independent second line of defense as necessary complements to any single statistical model, not optional extras.

The regulatory response to this episode is worth naming explicitly, since it shaped the risk management landscape this chapter describes. The Basel Committee's Fundamental Review of the Trading Book (FRTB), finalized in the years following the crisis and phased in at internationally active banks through the early 2020s, replaced VaR with Expected Shortfall as the basis for market-risk capital calculations. The reason is that ES, unlike VaR, is sensitive to the severity of losses beyond the threshold, not merely their frequency. That addresses the "25-standard-deviation event" problem described above directly: a VaR-based capital charge is blind to how much worse the tail beyond the VaR threshold actually is, while an ES-based charge is not. FRTB also requires banks to calibrate their models using a period of significant historical stress, rather than only recent, potentially calm data, a direct regulatory response to the finite-sample, calm-period-calibration problem this vignette illustrates, and introduces a more granular desk-level model approval process so that a single bank-wide model validation can no longer mask weaknesses in how any one specific trading desk's risk is measured. None of this makes the statistical methods in this chapter obsolete; it makes them subject to a considerably more demanding, and more explicitly crisis-aware, regulatory standard than the pre-2008 VaR framework was.

8.7 Challenges in Market Risk Management

Several challenges recur across nearly every market risk management application, and Chapter 9 revisits several of them in a credit-risk setting.

Model risk in the risk model itself. Every method in this chapter, parametric VaR's normality assumption, the GARCH recursion's own parametric structure, the delta-normal method's linearity assumption, is itself a model with assumptions that can fail exactly when they matter most, during a crisis. A risk model that works well in calm markets can dramatically understate risk in a crisis precisely because crises are, by definition, unusual, low-probability events that a model calibrated on typical conditions was never designed to capture. The gap between a linear approximation and a full, nonlinear revaluation, small for a modest move, can widen substantially exactly when the move being measured is largest.

Correlation and tail dependence. Individual risk measures often look reasonable in isolation, but a real portfolio becomes dangerous when many positions move together at the worst possible time, a pattern called tail dependence. This chapter's own case vignette showed a concrete, historically documented instance: correlations that looked low and stable during calm markets rose sharply during the 2008 crisis, eroding the diversification benefit the variance-covariance VaR of Section 8.2.2 assumed. Estimating tail dependence directly, rather than assuming a single, stable correlation matrix, is both statistically difficult and highly consequential for the resulting risk estimate.

Procyclicality. Risk measures calibrated on recent data tend to show low risk during calm periods (encouraging more risk-taking and leverage) and high risk immediately after a shock has already occurred (forcing deleveraging precisely when the system can least afford it), an amplifying feedback loop that has been implicated in several historical financial crises and that no purely statistical risk measure resolves on its own.

Data quality and limited loss history. Severe market crashes are, fortunately, rare, which means the historical data available to estimate their probability and severity is inherently limited, the small-sample problem flagged repeatedly throughout this book's time series and statistical inference material, and illustrated directly by this chapter's own twenty-observation VaR and backtesting examples. A VaR model estimated from a calm multi-year window that happened not to include a severe crisis will systematically understate tail risk.

Regulatory and business tension. Risk models exist within an organization that also wants to grow revenue and take on positions, and a risk function that is too conservative can be overridden or underresourced, while one that is too permissive can miss a crisis entirely; getting this balance right is as much an organizational and governance challenge as a statistical one.

Wulin.Suo@Queensu.ca

Regulatory arbitrage. Whenever regulatory capital is calculated from a bank's own reported VaR figure, as under the Basel backtesting multiplier described above, an institution has some incentive to structure its risk reporting specifically to minimize the resulting capital charge rather than its actual underlying risk. That might mean choosing a favorable historical window, a favorable confidence level, or a model specification that happens to backtest well. The resulting gap between measured and true risk can widen quietly over time, even when every individual calculation is technically correct.

Limit-setting on a diversification-sensitive measure. Component VaR's dependence on the full correlation structure of the book, not just an individual position's own volatility, cuts both ways. It correctly rewards genuinely diversifying positions with a smaller risk charge. But a desk operating close to its limit has some incentive to seek out positions that look diversifying under the currently estimated, calm-period correlation matrix, the positions Section 8.4.1's correlation stress test warns can stop diversifying, and start actively compounding losses, the moment correlations move against the book during a genuine crisis.

Machine learning adds a further layer of model risk. As Chapter 6 discussed, a neural network or gradient-boosted model can, in principle, forecast volatility or tail risk more flexibly than GARCH or a simple historical window, capturing nonlinear patterns a fixed parametric form cannot. But it inherits that chapter's overfitting and interpretability concerns in a setting where the consequences of an undetected error are unusually severe. An opaque VaR model that a regulator or independent validator cannot fully interrogate is a much harder model to trust with a bank's reported capital requirement, regardless of its backtested accuracy.

8.8 Key Takeaways

Market risk management complements the prediction-focused methods of earlier chapters by asking how bad outcomes can get, not just what is expected to happen. Value at Risk and Expected Shortfall summarize the downside of a return distribution using parametric, historical, or Monte Carlo methods; each involves a real tradeoff between distributional assumptions and finite-sample noise, and Expected Shortfall is generally preferred to VaR precisely because it accounts for the severity of losses beyond the VaR threshold rather than only their frequency, even though backtesting that severity directly is a genuinely harder problem than counting VaR exceptions.

Volatility itself need not be treated as fixed: Chapter 7's GARCH model lets a VaR estimate update immediately after a large move, rather than waiting for a slow-moving historical average to catch up, filtered historical simulation combines that responsiveness with the historical method's distribution-free flexibility, and the delta-normal approximation, while a convenient linearization for options portfolios, understates risk in proportion to the gamma exposure it ignores, a gap a second-order delta-gamma correction closes almost entirely.

Component VaR carries this same portfolio-level thinking one step further, decomposing total risk into per-position contributions that depend on the whole book's correlation structure rather than any position's standalone volatility, which is why a stress test that pushes correlations toward the crisis-period pattern this chapter's case vignette describes can more than double measured portfolio risk without any individual position's own volatility changing at all.

No aggregate should ever be the only thing anyone looks at. Sensitivities sit alongside VaR rather than beneath it, because a single portfolio-level number necessarily discards the information about *where* the risk is: a rate book can report a net DV01 of exactly zero while carrying a $1.36 million exposure to a curve flattening, and an options position's VaR can be computed correctly to the cent while remaining structurally blind to a routine move in implied volatility. Producing those sensitivities at the scale a real book requires is itself solved by reverse-mode automatic differentiation, the identical algorithm that trains neural networks.

Measurement is also not the same thing as management. Hedging exchanges an unwanted exposure for a smaller, differently-shaped residual rather than eliminating risk; position sizing controls risk before it is taken, which makes it only as good as the volatility forecast behind it; and stop-losses, the crudest control of the three, are lagging by construction and rescue far less of a drawdown than intuition suggests. None of these methods is self-policing either: backtesting (including Basel's own traffic-light zones), independent model validation, and a governance structure like the three lines of defense are what actually catch a miscalibrated model before it causes a loss, rather than after.

Wulin.Suo@Queensu.ca

Across every method in this chapter, the central lesson is the same one that closed Chapter 5's discussion of valuation: a risk number is only as trustworthy as the assumptions behind it, and a serious risk management practice combines several methods, and an honest accounting of where they disagree, rather than relying on any single measure alone. Chapter 9 now turns to the analogous toolkit for credit risk.

8.9 Suggested Exercises

1. Explain, in your own words, the difference between Value at Risk and Expected Shortfall, and describe a situation in which two portfolios could have identical VaR but very different Expected Shortfall.

2. Using the returns in Table 8.2, compute the parametric 90% VaR (using $z_{0.90} = 1.282$) for the \$10 million portfolio, and compare it to the 95% and 99% figures computed in the chapter.

3. Using the historical method, compute the 90% VaR and Expected Shortfall from Table 8.2 (following the convention of Section 8.2.2, which reads the 95% VaR from the second-worst return, leaving one observation beyond it, the 90% VaR leaves two observations beyond it and so corresponds to the third-worst return out of twenty).

4. Using Table 8.2's cumulative value path and the definition in Section 8.2.4, compute the drawdown at Day 6 (relative to the running peak established by that point) and compare it to the chapter's maximum drawdown of 6.74%, realized only at Day 20. Explain why a portfolio can pass through a large intermediate drawdown that is later exceeded by an even larger one, rather than the path recovering.

5. Using Chapter 7's GARCH(1,1) recursion values at $t = 2$ ($\sigma = 2.03\%$) and $t = 5$ ($\sigma = 1.99\%$), compute the parametric 95% VaR of a \$5 million position at each date, and explain why the two figures are so much closer together than the $t = 3$-versus-$t = 4$ comparison in the text.

6. Explain why a portfolio's 99% VaR can be smaller than its loss under a stress-test scenario, and why this is not evidence that the VaR calculation is wrong.

7. A bank's VaR model produces 7 exceptions over a rolling 250-day window at the 99% confidence level. Using the Basel traffic-light framework, classify this outcome and explain what supervisory consequence it would typically trigger.

8. Using this chapter's delta-normal example, recompute the 95% and 99% delta-normal and full-revaluation VaR for a position that is instead *long* 1,000 calls, and explain which direction of stock-price move now represents the position's worst case.

9. Pick two of the challenges listed in this chapter and describe a concrete scenario, not already used in the text, in which that challenge could cause a risk management system to understate a firm's true risk.

10. Using Section 8.2.2's two-asset covariance matrix, recompute the portfolio's volatility and 95% VaR under a stressed correlation of $\rho = 0.8$ instead of the $\rho = 0.5$ used in the text, and compare the resulting increase to the one computed in Section 8.4.1.

11. A bank has \$60 million in high-quality liquid assets and projected net cash outflows of \$55 million over a 30-day stress scenario. Compute its LCR, and explain why a bank could satisfy the LCR comfortably while still having an inadequate NSFR.

12. A third asset, AssetC, is added to Section 8.2.2's two-asset position with a Component VaR of \$4,000. Using the fact that Component VaRs must sum to the portfolio total, explain what this implies about how AssetC is correlated with the existing AssetA/AssetB position, without needing to know AssetC's own standalone volatility.

Wulin.Suo@Queensu.ca

13. Using this chapter's three-lines-of-defense limit example, compute the limit utilization if the approved daily VaR limit were instead \$275,000 (holding the parametric 99% VaR of \$250,081 fixed), and explain whether a subsequent rise in VaR to \$290,000 would count as a limit breach under this tighter limit.

14. Using this chapter's reverse-stress-testing example, compute the single-day return that would exhaust a risk capital cushion of \$800,000 instead of \$500,000 for the same \$10 million portfolio, and explain whether the -7% historical stress scenario from Section 8.4.1 would still exceed this larger cushion.

15. Explain, using this chapter's FRTB discussion, why a capital framework based on Expected Shortfall is less vulnerable than a VaR-based framework to a repeat of the "25-standard-deviation event" problem described in the 2008 case vignette.

16. Using the square-root-of-time rule from this chapter's economic-capital discussion, compute the 5-day regulatory VaR implied by the parametric 99% VaR of \$250,081, and explain why this figure is smaller than the 10-day figure computed in the chapter even though both are scaled from the same 1-day VaR.

17. This chapter's economic-capital discussion showed that a modest positive autocorrelation of $\phi = 0.10$ makes the square-root-of-time rule understate 10-day VaR by about 9.4%. Explain, in your own words, why a *negative* autocorrelation (mean reversion) in daily returns would instead make the square-root-of-time rule *overstate* multi-day risk, and describe a market condition in which returns might plausibly exhibit this pattern.

18. Using the vega in Table 8.3, compute the profit or loss on the short 1,000-call position if implied volatility rises by three percentage points while the stock price does not move at all, and express it as a percentage of the position's 95% full-revaluation VaR of \$980.79 from the delta-normal example. Explain why no amount of backtesting the VaR model would have revealed this exposure.

19. Apply a *steepening* scenario to the DV01 ladder of Table 8.4, in which two-year yields fall 10 basis points, five-year yields fall 5, ten-year yields rise 10, and thirty-year yields rise 15. Compute the desk's profit or loss, and explain why the sign of the result is the opposite of the flattening scenario worked in the text even though the net DV01 is zero in both cases.

20. A trading book is exposed to 500 distinct risk factors. (a) How many full valuations are required to produce a complete set of first-order sensitivities by bump-and-revalue? (b) Roughly how many are required by adjoint algorithmic differentiation, and why does this figure not depend on the number 500? (c) Explain why gamma, unlike delta, is not obtained for free in the same reverse sweep, and name the machine learning computation that faces the same first-order-versus-second-order cost distinction.

21. Using Section 8.5.2's volatility-targeting calculation, recompute the dollar amount to hold in the risky portfolio (a) if the mandate's target volatility were 15% rather than 10%, and (b) if the daily volatility estimate were instead Chapter 7's GARCH post-shock forecast of 2.08%, holding the 10% target. Comment on the size of the position change in (b) and on what it implies about how often a volatility-targeted portfolio must trade.

22. Show that the residual loss on a delta-hedged short-gamma position grows quadratically, by recomputing Section 8.5.2's figure of \$61.72 for a stock move twice as large as the \$2.198 used there. Explain why the residual is a loss for *both* a rise and a fall in the stock price, and what this implies about the kind of bet a delta-hedged option seller is really making.

23. **Challenge (real data).** Download at least two years of daily prices for a small portfolio of real stocks (for example, three to five components of a major index, via `yfinance`) and compute the portfolio's daily returns using fixed weights of your choosing. (a) Compute the portfolio's parametric and historical 99% one-day VaR, as in Section 8.2.2, and compare the two estimates. (b) Backtest your parametric VaR estimate over the full sample using Kupiec's test, referenced in Section 8.3.1: count the number of days on which the realized loss exceeded the VaR estimate, and determine whether the exception rate falls inside the Basel traffic-light green zone for your sample size. (c) Real portfolio returns are typically both fat-tailed and volatility-clustered, unlike the twenty-observation toy example this chapter builds on; discuss whether your backtest results suggest the parametric (normal-distribution) VaR model over-

Wulin.Suo@Queensu.ca

or understates risk relative to the historical method, and connect this to this chapter's own discussion of GARCH-based, volatility-adjusted VaR.

8.10 Further Reading

- Jorion, *Value at Risk: The New Benchmark for Managing Financial Risk*. The standard reference for VaR methodology, covering the parametric, historical, and Monte Carlo approaches introduced in this chapter in much greater depth and rigor.
- Hull, *Risk Management and Financial Institutions*. A broad treatment of market, credit, and operational risk within financial institutions, extending this chapter's introductory coverage of each risk type.
- McNeil, Frey, and Embrechts, *Quantitative Risk Management*. A mathematically rigorous treatment of coherent risk measures, Expected Shortfall, and the tail-dependence and correlation issues raised in this chapter's challenges section.
- Basel Committee on Banking Supervision, *Basel III: A Global Regulatory Framework for More Resilient Banks and Banking Systems*. The primary regulatory source for the capital adequacy, backtesting, and liquidity-coverage concepts introduced in this chapter. Its market-risk successor, *Minimum Capital Requirements for Market Risk* (January 2016, rev. February 2019), is the standard produced by the Fundamental Review of the Trading Book: it replaced Value at Risk with Expected Shortfall as the regulatory measure and introduced the sensitivities-based capital charge that this chapter's treatment of sensitivity-based risk limits develops.
- Giles, M. and P. Glasserman, "Smoking Adjoints: Fast Monte Carlo Greeks," *Risk*, January 2006. The paper that introduced adjoint differentiation to derivatives risk management, and the direct source for Section 8.3.3's claim that a full set of sensitivities costs a small constant multiple of a single valuation.
- Baydin, A. G., B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic Differentiation in Machine Learning: A Survey," *Journal of Machine Learning Research*, vol. 18, 2018. A clear and self-contained account of forward- and reverse-mode automatic differentiation, written for the machine learning audience, making explicit the equivalence with backpropagation that Section 8.3.3 relies on.
- Crouhy, Galai, and Mark, *The Essentials of Risk Management*. A practitioner-oriented synthesis spanning market, credit, and operational risk, model validation, and risk governance, tying together the full breadth of topics covered in this chapter and Chapter 9 in a single volume.

Wulin.Suo@Queensu.ca

Wulin.Suo@Queensu.ca

Chapter 9

Credit Risk Modeling

9.1 Introduction

Chapter 8 developed the standard toolkit for market risk, Value at Risk, Expected Shortfall, and the volatility and correlation machinery behind them. This chapter turns to a related but genuinely distinct problem: credit risk, the risk that a counterparty, whether a bond issuer, a loan borrower, or a derivatives counterparty, fails to meet a contractual obligation. Where market risk asks how much a position's *value* could move, credit risk asks whether a counterparty will *pay at all*, a binary event with its own probability, its own severity once it occurs, and its own regulatory and modeling apparatus, probability of default, loss given default, credit scoring, structural models, and rating systems, developed in full below.

The chapter proceeds from the single-borrower case to the whole-portfolio case, mirroring the logic Chapter 8 used for individual positions versus an entire book. It begins with the three-component decomposition of expected loss and what drives each component, moves through three genuinely distinct ways of estimating a borrower's probability of default, a statistical model fit to historical data, a structural model that reads default risk directly out of market prices, and a market-implied approach that reads it out of credit derivative spreads, and then extends single-borrower ratings into multi-year transition matrices. The chapter closes by moving from a single loan to an entire portfolio, where the correlation between borrowers' defaults, not just their individual probabilities, becomes the dominant driver of extreme loss, a theme illustrated concretely both by a worked single-factor model and by a real historical episode in which underestimating exactly that correlation contributed to a systemic financial crisis.

One worked example runs through much of the chapter: a small loan portfolio that reuses the credit-risk-adjusted bond from Chapter 5 and the loan dataset from Chapter 6, extending both into a full treatment of expected loss, credit scoring, and portfolio credit risk.

9.2 Measuring Credit Risk

Credit risk decomposes into a probability, an exposure, and a loss given that it happens; this part builds each and scores a borrower.

9.2.1 Credit Risk Fundamentals: Expected Loss

Suppose a bank holds a bond and wants a single number summarizing how much of that exposure it should expect to lose to default, on average across many similar bonds, well before knowing whether this particular one actually defaults. Credit risk is conventionally decomposed into three components whose product answers this, giving the expected loss on a credit exposure:

$$\text{Expected Loss} = \text{PD} \times \text{LGD} \times \text{EAD},$$

where PD is the probability of default over the relevant horizon (typically one year), LGD is the loss given default, the fraction of the exposure not recovered after default, and EAD is the exposure at default, the

Wulin.Suo@Queensu.ca

dollar amount outstanding at the time of default (which can differ from the current balance for instruments like credit lines that can be drawn down further before default).

Recall the corporate bond from Chapter 5's discussion of pricing default risk: face value \$1,000, one-year default probability PD = 3%, and a recovery rate of 40% of face value, so LGD = 1 − 0.40 = 60%. Treating the full face value as the exposure at default, EAD = \$1,000, the expected loss is

$$\text{Expected Loss} = 0.03 \times 0.60 \times 1{,}000 = \$18.$$

Figure 9.1 lays out this same decomposition as a pipeline, since the rest of this chapter is organized around estimating each of these three inputs in turn, first PD (via logistic regression and, later, the Merton model), then LGD, then EAD.

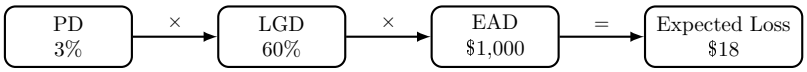


Figure 9.1: The expected-loss decomposition, applied to the Chapter 5 bond example

This is not a coincidence relative to Chapter 5's earlier calculation: that chapter found the bond's expected payoff to be \$982 against a promised \$1,000, and 1,000 − 982 = 18 exactly, confirming that discounting the expected payoff and separately computing an expected loss are two equivalent ways of describing the same underlying credit risk.

The practical distinction between the two framings is one of purpose rather than substance. The pricing framing of Chapter 5 asks "what should this bond be worth today." The expected-loss framing here asks "how much of this exposure should a lender expect to lose, on average, and how much capital or loan-loss reserve should be held against that possibility," which is the accounting and regulatory question addressed later in this chapter.

EAD is the least discussed of the three components in most introductory treatments, but it deserves its own worked example, since it is not simply "the current balance" whenever a borrower has undrawn credit available. Consider a committed revolving credit line with a \$500,000 total commitment, currently drawn to \$200,000, leaving \$300,000 undrawn. A defaulting borrower does not necessarily default with only the currently drawn amount outstanding; many borrowers draw down a credit line further as their financial condition deteriorates, often specifically because they are approaching default and want to secure cash while they still can, so exposure at default must account for some fraction of the undrawn commitment being drawn down before default actually occurs.

Basel's standardized approach captures this with a credit conversion factor (CCF), the assumed fraction of the undrawn commitment that converts to drawn exposure by the time of default; for an unconditionally cancellable commercial credit line, a CCF of 50% is a representative assumption, giving

$$\text{EAD} = \text{Drawn} + \text{CCF} \times \text{Undrawn} = 200{,}000 + 0.50(300{,}000) = \$350{,}000,$$

75% higher than the \$200,000 currently drawn balance, a difference the expected-loss formula above would otherwise have ignored entirely. This is precisely why credit lines and other commitment-based lending products carry more expected loss, for a given PD and LGD, than a fully drawn term loan of the same current balance. It is also why EAD, though it receives less modeling attention than PD or LGD in most textbook treatments, is estimated from its own dedicated historical dataset of drawn-versus-committed balances at the time of past defaults, in any bank sophisticated enough to model it directly rather than relying on a regulatory CCF table.

What Drives Loss Given Default

PD tends to receive the most modeling attention, since it is the component most directly tied to the statistical and machine learning methods of Chapter 6, but LGD is just as consequential for expected loss and considerably harder to model well, since recovery outcomes depend on factors that are only observed after a default has already occurred.

Wulin.Suo@Queensu.ca

Seniority and collateral are the two most important drivers: a secured loan backed by specific collateral (real estate, equipment, inventory) typically recovers far more in default than an unsecured loan to the same borrower, since the lender has a direct claim on identifiable assets rather than competing with every other unsecured creditor for whatever remains. Seniority within the capital structure matters similarly: senior debt is repaid before subordinated debt in a bankruptcy, so a subordinated bondholder can face a substantially lower recovery rate than a senior bondholder to the very same defaulted issuer, even though both bondholders faced the identical probability of default. Recovery rates are also cyclical in a way that compounds portfolio credit risk rather than diversifying it. During a recession, defaults become more frequent (PD rises) at precisely the same time that collateral values fall and distressed-asset markets become crowded with sellers, so LGD rises too, since collateral fetches less in a forced sale. PD and LGD are therefore positively correlated across the credit cycle rather than independent, as this chapter's simplified expected-loss and economic-capital formulas implicitly assumed for tractability. A lender that estimates LGD from an average computed across a full credit cycle, rather than conditioning it on current or forecast economic conditions, will therefore tend to understate expected and unexpected losses specifically during the downturns when accurate loss estimates matter most, the same procyclicality theme raised elsewhere in this chapter.

Collateral coverage, the ratio of collateral value to loan balance, is directly estimable from a lender's own historical default data, and a simple linear regression already illustrates the qualitative pattern above numerically. Consider four hypothetical defaulted loans:

$$\begin{aligned}
&\text{Coverage} = 0.40, \quad \text{Recovery} = 25\%,\\
&\text{Coverage} = 0.60, \quad \text{Recovery} = 45\%,\\
&\text{Coverage} = 0.80, \quad \text{Recovery} = 55\%,\\
&\text{Coverage} = 1.20, \quad \text{Recovery} = 75\%.
\end{aligned}$$

Fitting Recovery $= a + b\,(\text{Coverage})$ by ordinary least squares, the estimation method Chapter 6 introduced,

$$\hat{b} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = 0.60, \qquad \hat{a} = \bar{y} - \hat{b}\bar{x} = 0.05,$$

so $\widehat{\text{Recovery}} = 0.05 + 0.60\,(\text{Coverage})$, with $R^2 \approx 0.969$, a strong linear fit given only four points.

A loan with coverage exactly at 1.0 (collateral value equal to the outstanding balance) has a predicted recovery of $0.05 + 0.60(1.0) = 65\%$, implying LGD $= 35\%$. That is meaningfully lower than the 60% LGD assumed for the unsecured bond example running through this chapter, and it is the seniority-and-collateral effect described above made numerically concrete: a well-collateralized loan can have a substantially lower LGD than an otherwise identical unsecured exposure. A lender with enough historical default data can estimate that relationship directly, using the same regression logic Chapter 6 developed for very different financial applications, applied here to the loss side of credit risk rather than the default-probability side.

The high R^2 is worth treating with the caution Chapter 6 raised about small samples, and computing the slope's standard error, following the same procedure applied to the credit-spread regression in that chapter, makes this concrete rather than merely qualitative. With only two degrees of freedom (four observations, two estimated parameters), $\text{SE}(\hat{b}) \approx 0.0756$, giving a t-statistic of $0.60/0.0756 \approx 7.94$ and a 95% confidence interval for the slope of roughly $(0.27, 0.93)$. The slope is still statistically significant at conventional levels ($p \approx 0.016$) despite the small sample, but the confidence interval itself is enormous relative to the point estimate. A true slope anywhere from 0.27 to 0.93 remains consistent with this data. Propagating that parameter uncertainty through, the predicted recovery rate at a coverage ratio of 1.0 could plausibly range from roughly 32% to 98%, an enormous practical range for a single reported LGD figure. This is precisely why a lender relying on a collateral-recovery regression estimated from only a handful of historical defaults should treat the fitted line as a useful starting point rather than a precise input, and should prioritize collecting more default history, or borrowing estimates from a broader industry dataset, before relying on it for capital or pricing decisions.

Credit Stress Testing: Stressed PD and LGD

The procyclicality just described, PD and LGD both rising together in a downturn, is exactly what credit stress testing makes concrete. Rather than assuming the base-case PD and LGD estimated from calm,

Wulin.Suo@Queensu.ca

average conditions, a stress test recomputes expected loss under a specified adverse scenario, a recession, a sector-specific downturn, a sharp decline in collateral values, and asks how much larger expected loss becomes.

Applying a stress scenario to the same bond from Section 9.2.1, raising the default probability from 3% to a stressed PD = 8% (about 2.7 times its base-case value) and raising loss given default to a stressed LGD = 70% (reflecting weaker collateral values in a downturn, the mechanism Section 9.2.1 described), the stressed expected loss is

$$\text{Expected Loss}^{\text{stress}} = 0.08 \times 0.70 \times 1{,}000 = \$56,$$

more than triple the base-case expected loss of \$18, even though neither input moved by an implausible amount on its own. This tripling is precisely why treating PD and LGD as independent, average-cycle constants, as this chapter's simpler formulas do for tractability, can be dangerously misleading for capital planning: a lender that reserves only against the calm-period expected loss will find its actual losses considerably larger than reserved for in the scenario, a recession, when the shortfall matters most. Regulatory stress-testing exercises such as the Federal Reserve's CCAR, introduced from the market-risk side in Chapter 8, apply this same stressed-PD-and-LGD logic across a bank's entire loan book, not just a single bond, projecting credit losses under a specified adverse macroeconomic scenario rather than the calm historical averages ordinary credit models are typically calibrated on.

9.2.2 Credit Scoring with Logistic Regression

Expected loss requires an estimate of PD for each borrower, and Chapter 6 already introduced the standard statistical tool for producing one: logistic regression, which models the probability of a binary outcome, here, default within the horizon, as a function of borrower characteristics. Recall the small loan dataset from Chapter 6's K-nearest neighbors example, with six applicants described by debt-to-income ratio and years of credit history. Fitting a logistic regression to that same dataset,

$$\Pr(\text{Default}) = \frac{1}{1+e^{-z}}, \qquad z = -12.152 + 0.530\,(\text{Debt/Income}) - 0.231\,(\text{Credit History}),$$

gives coefficients that make economic sense: higher leverage increases the log-odds of default (positive coefficient), while a longer credit history reduces it (negative coefficient). These coefficients come from scikit-learn's default fit, which adds an L2 (ridge) penalty with $C = 1$, and on this dataset the penalty is not a detail. Debt-to-income alone separates the two classes perfectly (the three defaulters sit at 30–40%, the three non-defaulters at 15–20%), and under such *complete separation* the unpenalized maximum-likelihood estimate does not exist: the likelihood keeps rising as the coefficients grow without bound, so a plain fit (statsmodels, for example) either fails with a perfect-separation error or returns arbitrarily large coefficients. The quoted values are therefore a product of the penalty as much as of the data, which is why real scorecards on small or cleanly split samples use penalized or Firth-corrected logistic regression, and why the perfect in-sample AUC this sample produces later in the chapter is a warning sign rather than an achievement. Applying this to the same new applicant considered in Chapter 6 (debt-to-income of 28%, 4 years of credit history),

$$z = -12.152 + 0.530(28) - 0.231(4) \approx 1.761, \qquad \Pr(\text{Default}) = \frac{1}{1+e^{-1.761}} \approx 85.3\%.$$

This is consistent with, and considerably more precise than, the KNN classification from Chapter 6, which only produced a discrete "predicted default" label from a 2-to-1 majority vote among the three nearest neighbors; logistic regression instead reports a continuous probability, which is the PD input Section 9.2.1's expected-loss formula requires and which a simple majority-vote classifier cannot supply.

Lenders frequently rescale a model's output probability into a more interpretable credit score using a simple linear transformation, for example

$$\text{Score} = 850 - 550 \times \Pr(\text{Default}),$$

chosen so that a default probability near 0 produces a score near 850 (the top of a typical consumer credit-score scale) and a default probability near 1 produces a score near 300 (the bottom). For the applicant

Wulin.Suo@Queensu.ca

above, Score $= 850 - 550(0.853) \approx 381$, a score that would place this applicant in a high-risk tier under most lenders' underwriting policies. The specific scaling constants are a business choice, not a statistical one, but the underlying logic, mapping a well-calibrated probability onto a scale that loan officers and borrowers can interpret intuitively, is universal, and connects directly back to the calibration discussion in Chapter 6: a credit score is only meaningful to the extent that the probability underlying it is itself accurately calibrated, not just useful for ranking applicants from safest to riskiest.

A subtler difficulty affects every credit scoring model built this way, regardless of how well it is fit or calibrated: a lender only observes loan performance for applicants it actually approved, since a rejected applicant never receives a loan and therefore never generates a default-or-no-default outcome to learn from. Training a model exclusively on approved applicants, the six-applicant sample used throughout this section, means the model never sees how a rejected applicant, someone the previous underwriting policy already judged too risky, would actually have performed, a selection bias that can silently distort the fitted relationship between borrower characteristics and true default risk. Reject inference is the general name for techniques that attempt to correct for this, ranging from simple approaches (assigning rejected applicants an assumed outcome based on their characteristics) to more sophisticated statistical corrections that model the approval decision itself alongside the default outcome. None of these fixes is fully satisfying, since the true counterfactual, what would have happened had a rejected applicant been approved, is fundamentally unobservable, which is why reject inference remains an acknowledged, imperfect compromise in credit scoring practice rather than a solved problem, and why a credit model's performance on the population it was trained on can differ meaningfully from its performance on the full population of applicants it is actually used to screen. This is a credit-specific instance of a more general lesson from Chapter 6: a model is only as good as the data it learns from, and data generated by a decision process, here, a prior lending policy's approvals and rejections, is never a neutral, representative sample of the population that process was applied to.

Validating a Credit Scoring Model: The Gini Coefficient and AUC

Ranking applicants correctly from safest to riskiest is a distinct property from calibration, and it has its own standard validation metric in credit scoring: the area under the ROC curve (AUC), or equivalently the Gini coefficient, $\text{Gini} = 2\,\text{AUC} - 1$, which rescales AUC so that a purely random ranking scores 0 and a perfect ranking scores 1. Both measure the same underlying question introduced more generally in Chapter 6: across every possible pair consisting of one defaulter and one non-defaulter, what fraction of the time does the model assign the defaulter a strictly higher predicted default probability?

Applying the fitted logistic regression above to all six applicants in Chapter 6's Table 6.4, sorted by predicted default probability, gives the values in Table 9.1.

Applicant	Debt/income (%)	Pr(Default)	Actual outcome
C	40	0.9998	Default
A	35	0.9974	Default
E	30	0.9550	Default
B	20	0.0323	No default
F	18	0.0144	No default
D	15	0.0015	No default

Table 9.1: Predicted default probabilities for all six training applicants, sorted from highest to lowest risk

Every one of the nine possible (defaulter, non-defaulter) pairs, three defaulters times three non-defaulters, is correctly ordered: each of C, A, and E's predicted probabilities exceeds each of B, F, and D's, so AUC $= 9/9 = 1.0$ and Gini $= 2(1.0) - 1 = 1.0$, a perfect ranking. This is a genuine red flag rather than a cause for celebration, and for the reason Chapter 6 raised repeatedly: this AUC is computed on the same six observations the model was fit to, with two features and six data points, so a perfect in-sample ranking is close to the least surprising outcome imaginable, not evidence the model would rank a new, out-of-sample applicant with anything like this accuracy. A real credit scoring model's AUC and Gini coefficient are only

Wulin.Suo@Queensu.ca

meaningful when computed on a genuinely held-out validation sample, following the train-test discipline Chapter 6 developed at length, and an AUC this close to a perfect score on such a small training sample is itself a warning sign of overfitting to memorize, rather than a property to report proudly in a model validation document.

9.2.3 Scorecards: Weight of Evidence, Points, and Why They Are a GAM

The regression above is the right model but not the form a lender deploys. Almost every consumer credit model in production is a *scorecard*: the same logistic regression, but with each characteristic first cut into bins, each bin replaced by a number summarizing its risk, and the result expressed as whole points. Binning lets a linear model capture a non-linear relationship, makes the model robust to outliers and data errors, and leaves every part of it inspectable row by row.

The number replacing each bin is its *weight of evidence*, the log ratio of the share of non-defaulters to the share of defaulters falling in it,

$$\mathrm{WOE}_i = \ln\!\left(\frac{g_i/G}{b_i/B}\right),$$

with g_i, b_i the good and bad counts in bin i and G, B the portfolio totals. Table 9.2 works this for a portfolio of 1,000 loans, 900 performing and 100 defaulted.

Debt/income	Good	Bad	g_i/G	b_i/B	WOE	IV contribution
$< 15\%$	270	10	0.300	0.100	$+1.099$	0.220
15–25%	315	20	0.350	0.200	$+0.560$	0.084
25–35%	225	30	0.250	0.300	-0.182	0.009
$\geq 35\%$	90	40	0.100	0.400	-1.386	0.416
Total	900	100	1.000	1.000	—	0.729

Table 9.2: Weight of evidence and information value for a debt-to-income characteristic

The column is *monotone*, falling as leverage rises; a reversal is read as evidence that the bins are wrong or the sample too thin, and bins are merged until it disappears. The whole characteristic then summarizes into its *information value*, $\mathrm{IV} = \sum_i (g_i/G - b_i/B)\,\mathrm{WOE}_i = 0.729$, which by convention (below 0.02 useless, 0.1–0.3 medium, 0.3–0.5 strong) makes this a strong predictor; values far above 0.5 usually mean the characteristic partly restates the outcome, the credit version of the leakage problem Chapter 6 warns about.

The fitted log-odds are then rescaled so that a fixed number of points always means the same thing. Choosing PDO $= 20$ *points to double the odds* fixes the slope at $20/\ln 2 = 28.854$, and the gap between the safest and riskiest bands becomes $28.854(1.099 + 1.386) = 71.7$ points, or $71.7/20 = 3.59$ doublings, an odds ratio of $2^{3.59} = 12.0$. That checks straight off the table: the odds of being good are $270/10 = 27$ in the lowest band and $90/40 = 2.25$ in the highest, and $27/2.25 = 12.0$ exactly. The points scale adds no information; it repackages the log-odds into units a human can add up.

The Scorecard Is a Generalized Additive Model

It is tempting to file the scorecard under "the classical method machine learning came to replace." That reading is wrong twice, and correcting it locates where machine learning genuinely earns its keep in credit.

Start with the bins. Choosing cut points by hand is possible but is not what anyone does; the standard tool is *supervised discretization*, fitting a decision tree to the single characteristic and taking its splits as bin edges. On a simulated 20,000-loan portfolio, a depth-limited tree with a floor of 800 accounts per leaf selects cuts at 17.1%, 32.7%, 41.3%, 44.5%, and 47.3%, giving six bins whose weights of evidence run $+1.054$ down to -1.433, monotone with no manual merging and an information value of 0.620. The scorecard's most hand-crafted-looking step is done by a tree learner.

Wulin.Suo@Queensu.ca

Now the functional form. Writing the scorecard out,

$$\ln \frac{p}{1-p} = \beta_0 + \sum_j \beta_j \,\mathrm{WOE}_j(x_j) = \beta_0 + \sum_j f_j(x_j),$$

each characteristic contributes an arbitrary piecewise-constant function of its own value, and the contributions add. That is the definition of a *generalized additive model*. A credit scorecard is a GAM fitted by hand, and has been since the 1950s, decades before the term existed.

This matters because the current state of the art in interpretable machine learning, the *Explainable Boosting Machine*, fits this object automatically: it boosts shallow trees on one feature at a time, round-robin, so no interaction can form, and the accumulated trees for feature j become f_j. The companion notebook implements this in about twenty lines and fits it to the same portfolio. Comparing its learned shape function for debt-to-income against the hand-built WOE curve over a 150-point grid gives a correlation of 0.977. They are not analogous; they are two estimates of the same function.

The accuracy comparison is then informative rather than a horse race. On held-out data the scorecard scores an ROC-AUC of 0.8402, the boosted additive model 0.8296, and an unrestricted gradient-boosted model allowed to learn interactions 0.8328, against a ceiling of 0.8429 set by the true data-generating process, which does contain a genuine interaction between leverage and file length. The scorecard lands within 0.003 of the ceiling and nothing beats it. When marginal effects are monotone, bins are well chosen, and interactions are weak, a scorecard is close to optimal and a more flexible model has little left to find. That, rather than institutional inertia, is why it remains the incumbent in regulated lending.

The corollary is the useful part, and the rest of this chapter's machine learning rests on it: ML earns its keep in credit not by fitting a better function of the same characteristics, but by doing what a scorecard cannot do at all, building features from data that has no natural bins, producing a term structure rather than a single probability (as the lifetime expected-loss discussion later in this chapter does), and searching a model space rather than fitting one model.

9.3 Modeling Default

This part moves from scoring an applicant to modeling default itself, from a firm's balance sheet, from ratings, and over a loan's whole life.

Structural Credit Models: The Merton Model

Logistic regression estimates PD statistically, from historical data on borrower characteristics and past defaults. An entirely different approach, due to Merton (1974), estimates PD directly from market prices using an insight that connects directly back to the option pricing machinery of Chapter 5: a firm's equity can be modeled as a call option on the firm's assets, with the face value of its debt playing the role of the strike price. The intuition is precise, not merely metaphorical. Shareholders have limited liability: if a firm's assets are worth more than its debt obligations at maturity, shareholders receive the difference (assets minus debt); if assets are worth less than the debt, shareholders receive nothing and the firm defaults, with bondholders taking control of the (insufficient) assets instead. This is the payoff structure $\max(V_T - D, 0)$ of a European call option with the firm's asset value V_T playing the role of the underlying price and the debt's face value D playing the role of the strike, so the Black-Scholes-Merton formula from Chapter 5 applies directly, with the firm's asset value in place of a stock price and its asset volatility in place of σ:

$$E_0 = V_0 N(d_1) - D e^{-rT} N(d_2), \qquad d_1 = \frac{\ln(V_0/D) + (r + \sigma_V^2/2)T}{\sigma_V \sqrt{T}}, \qquad d_2 = d_1 - \sigma_V \sqrt{T},$$

where E_0 is the market value of equity, V_0 is the (unobserved) market value of the firm's assets, and σ_V is asset volatility. The probability of default, the probability that assets fall below the debt's face value at maturity, is the risk-neutral probability that the option expires out of the money,

$$\Pr(\text{Default}) = N(-d_2),$$

Wulin.Suo@Queensu.ca

the same $N(-d_2)$ term that appeared when Chapter 5 priced a European put. Figure 9.2 draws this payoff structure directly: equity holders receive nothing if the firm's assets are worth less than its debt at maturity, and the excess above debt otherwise, exactly a call option's hockey-stick payoff with the debt's face value playing the role of the strike price.

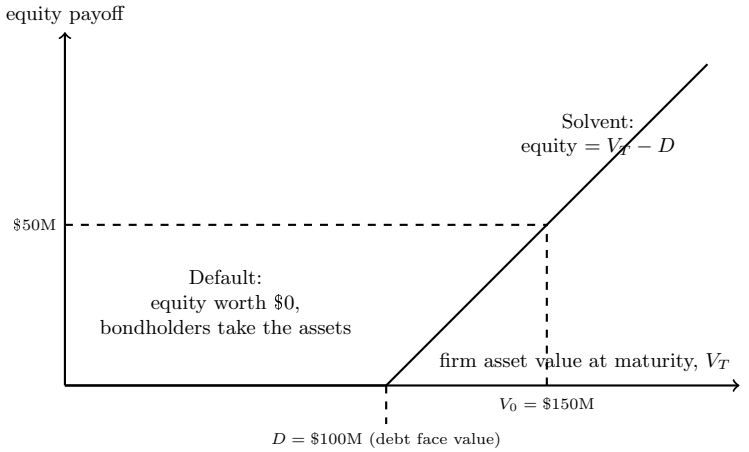


Figure 9.2: The Merton model's equity payoff at maturity, $\max(V_T - D, 0)$, for the worked example's $D = \$100$ million

Consider a firm with

$$V_0 = \$150 \text{ million (estimated asset value)}, \qquad D = \$100 \text{ million (debt due in one year)},$$
$$\sigma_V = 25\% \text{ (asset volatility)}, \qquad r = 4\% \text{ (risk-free rate)}.$$

Then

$$d_1 = \frac{\ln(150/100) + (0.04 + 0.03125)(1)}{0.25} \approx 1.907, \qquad d_2 = 1.907 - 0.25 \approx 1.657,$$

giving an implied equity value of $E_0 = 150\,N(1.907) - 100\,e^{-0.04}N(1.657) \approx \$54.37$ million and a one-year default probability of

$$\Pr(\text{Default}) = N(-1.657) \approx 4.88\%.$$

Unlike logistic regression, which requires a historical dataset of past defaults to estimate, the Merton model requires no default history at all. It needs only an estimate of asset value and volatility, in practice usually inferred from the firm's observable equity value and equity volatility by inverting the same two equations. That is why structural models are especially useful for large, publicly traded firms with liquid equity and few or no historical defaults to learn from, the firms for which a purely statistical model like logistic regression would have the least historical data to work with. The commercial descendant of this approach, Moody's KMV model, is widely used in practice for this reason, converting the model's "distance to default" (d_2 in the notation above) into a market-implied probability of default that updates in real time as equity prices move, without waiting for a rating agency's periodic review.

9.3.1 Inverting the Model: Recovering Asset Value and Volatility from Equity

The worked example above ran the Merton model forward: given the firm's asset value and asset volatility, both computed the implied equity value and the default probability. In practice, neither V_0 nor σ_V is ever directly observable, no market quotes a firm's total asset value the way one quotes its stock price, so a real application must run the model in reverse, starting from what the market actually prices, the firm's equity, and backing out the unobserved asset value and volatility underneath it. This is precisely the KMV approach mentioned above, and it is worth making the inversion concrete rather than leaving it as a one-sentence aside.

The equity value equation from above, $E_0 = V_0 N(d_1) - De^{-rT}N(d_2)$, is one equation in the two unknowns V_0 and σ_V; a second equation comes from recognizing that equity, as a call option on the firm's assets, has

Wulin.Suo@Queensu.ca

its own volatility linked to the asset volatility through the option's delta, $N(d_1)$ (the same Greek introduced in Chapter 5):

$$\sigma_E E_0 = N(d_1)\,\sigma_V V_0.$$

Given a firm's genuinely observable market equity value E_0 and equity volatility σ_E (estimated from historical or option-implied stock returns, just as in Chapter 5 and Chapter 7), these two equations can be solved simultaneously for the two unknowns V_0 and σ_V, typically using a numerical root-finder rather than an algebraic closed form, since d_1 and d_2 both depend on V_0 and σ_V nonlinearly.

As a consistency check, apply this inversion to the same firm from the forward example above. Its Merton-model equity value there was $E_0 \approx \$54.37$ million; its implied equity volatility, computed from the same d_1 using the formula above, is $\sigma_E = N(1.907)(0.25)(150)/54.37 \approx 67.03\%$, considerably higher than the 25% asset volatility underneath it, since equity absorbs the firm's business risk in a leveraged, option-like way, an equity holder's stake is far more volatile than the assets that stake is a claim on. Feeding only these two observable quantities, together with the known debt, rate, and maturity,

$$E_0 = \$54.37 \text{ million}, \qquad \sigma_E = 67.03\%,$$
$$D = \$100 \text{ million}, \qquad r = 4\%, \qquad T = 1 \text{ year},$$

into a numerical solver for the two equations above recovers $\hat{V}_0 = \$150.00$ million and $\hat{\sigma}_V = 25.00\%$ exactly, and therefore the identical default probability of 4.88% computed directly from the (in this illustration, already known) asset value and volatility.

This exact recovery is expected here, since the "observed" equity inputs were themselves generated from the true V_0 and σ_V rather than taken from noisy real market data, but it confirms that the inversion is well posed. A real application feeds in a firm's actual market equity value and estimated equity volatility, with no knowledge of the true V_0 or σ_V at all, and the same two-equation solve recovers a market-implied estimate of both. That estimate updates automatically every time the firm's stock price moves, the real-time property that makes the KMV approach practically useful for surveillance of large public borrowers between a rating agency's periodic reviews.

9.3.2 Market-Implied Default Probabilities: Credit Default Swap Spreads

The Merton model backs out a default probability from a firm's *equity* price; a credit default swap (CDS), introduced in Chapter 3 as a contract that transfers credit risk in exchange for a periodic premium, offers a second, more direct market-implied route, since a CDS spread is, by construction, the market's own price for insuring against the default event this chapter has been modeling. A standard simplification, sometimes called the credit triangle, relates the CDS spread s directly to a constant hazard rate (default intensity) h and the recovery rate R assumed on the underlying debt,

$$h \approx \frac{s}{1-R},$$

with the cumulative default probability over a horizon T given by $\Pr(\text{Default by } T) = 1 - e^{-hT}$, treating default as a Poisson-arrival process with constant intensity h, the same modeling device behind the survival-analysis machinery Chapter 15 used to characterize how long flagged fraud accounts remain active. A one-year CDS trading at a spread of $s = 200$ basis points (2%) on debt with an assumed recovery rate of $R = 40\%$, the same recovery assumption used throughout this chapter's bond example, implies

$$h \approx \frac{0.02}{1-0.40} \approx 3.33\%, \qquad \Pr(\text{Default within one year}) = 1 - e^{-0.0333} \approx 3.28\%,$$

reassuringly close to, though not identical to, the PD = 3% assumed directly for this chapter's bond example, a useful cross-check that a market-implied and a directly assumed default probability broadly agree.

The same constant-hazard-rate assumption extends immediately to any horizon. The cumulative five-year default probability implied by the identical $h \approx 3.33\%$ is $1 - e^{-0.0333(5)} \approx 15.35\%$, only about 4.7 times the one-year figure of 3.28% even though the horizon is five times longer. Cumulative survival probability compounds multiplicatively rather than the default probability simply scaling linearly with time, so each

Wulin.Suo@Queensu.ca

later year's default risk applies only to the borrowers still surviving, the same compounding logic behind the multi-year transition-matrix default probabilities computed later in this chapter. Unlike the Merton model, which requires an estimate of the firm's asset volatility, the CDS-implied approach requires no volatility input at all, only an observed market spread and an assumed recovery rate, but it depends entirely on that market actually existing and trading with reasonable liquidity. This is true for large, actively traded corporate and sovereign names but not for the vast majority of small and mid-sized borrowers this chapter's logistic regression and Merton approaches are built to handle instead.

This chapter now has three genuinely distinct ways of arriving at a probability of default, and it is worth being explicit about when each is the right tool rather than treating them as interchangeable. Logistic regression, Section 9.2.2's approach, requires a historical dataset of past borrowers and their outcomes, and it is the natural choice for the population that dataset represents, retail and small-business borrowers with no traded equity or debt at all. Its accuracy, though, is bounded by how representative and how recent that historical sample is, the reject-inference and limited-loss-history concerns raised elsewhere in this chapter. The Merton model requires no default history, but it does require a firm to have liquid, exchange-traded equity, since its entire logic depends on inferring asset value and volatility from an observable stock price. That makes it well suited to large public companies and poorly suited to a privately held small business, the opposite population from where logistic regression is most naturally applied. The CDS-implied approach requires neither a default history nor an equity price, only a liquid credit derivatives market, narrowing its applicability further still to the relatively small set of large, actively traded corporate and sovereign names with a meaningfully liquid CDS market at all. A large bank's credit risk function typically uses all three where each is available, logistic regression or a machine-learned variant for its retail and small-business book, Merton-style structural models for large public corporate borrowers, and CDS-implied spreads as a real-time market check on the largest, most liquid names, precisely because no single approach's data requirements are met across a bank's entire, heterogeneous loan and bond portfolio.

Rating Models and Transition Matrices

Suppose a bond portfolio manager holds a bond currently rated investment grade and wants to know not just its one-year default probability but how likely it is to be downgraded well before that, a genuinely different risk, since a downgrade alone can force some funds to sell and can lower the bond's market price long before any payment is actually missed.

Beyond scoring individual loans, institutional credit risk (corporate bonds, sovereign debt) is often managed through a rating system, a small number of discrete risk categories (such as AAA, AA, A, BBB, down to Default) that summarize a borrower's creditworthiness. Ratings are not static: a borrower can be upgraded, downgraded, or default over time, and a rating transition matrix records the probability of moving from each rating to every other rating over a fixed horizon, typically one year. Table 9.3 shows a simplified three-state transition matrix with ratings High and Medium plus an absorbing Default state (once a borrower defaults, it remains in that state, so the Default row is $[0, 0, 1]$).

From \ To	High	Medium	Default
High	0.90	0.08	0.02
Medium	0.10	0.80	0.10
Default	0.00	0.00	1.00

Table 9.3: A simplified one-year rating transition matrix

The one-year default probability is read directly off the matrix: 2% for a High-rated borrower, 10% for a Medium-rated one. Multi-year default probabilities require treating the transition matrix as a Markov chain and computing its matrix power: the two-year transition matrix is $P^2 = P \times P$, and the entry in row i, column Default gives the cumulative two-year default probability starting from rating i. Carrying out this multiplication,

$$P^2_{\text{High}\to\text{Default}} = 0.90(0.02) + 0.08(0.10) + 0.02(1.00) = 0.046,$$

so a High-rated borrower's cumulative two-year default probability is 4.6%, more than double the one-year figure of 2%, since the borrower can now default either directly or after first migrating to the riskier Medium

Wulin.Suo@Queensu.ca

rating. The same calculation for a Medium-rated borrower gives a two-year default probability of 18.2%, and extending to three years ($P^3 = P^2 \times P$) gives 7.596% for a High-rated borrower. This compounding behavior, cumulative default risk growing faster than linearly with horizon because of the possibility of downgrade-then-default, is a direct credit-risk analogue of the growing forecast uncertainty over longer horizons discussed for time series models in Chapter 7, and it is why long-dated credit exposures (a 10-year bond, say) carry meaningfully more cumulative default risk than the one-year transition probabilities alone would suggest.

Transition matrices also make credit risk visible before an actual default occurs, a genuinely useful property none of this chapter's other tools directly supply. A bondholder whose issuer is downgraded from High to Medium has suffered a real economic loss well before that issuer ever misses a payment, since a lower-rated bond generally trades at a lower price to compensate for its higher default risk. Marking a credit portfolio to its current ratings, rather than waiting for realized defaults, is the logic behind mark-to-market credit risk models such as CreditMetrics. These combine a transition matrix like Table 9.3 with bond-pricing formulas from Chapter 5 to estimate a portfolio's value distribution one year forward, including the possibility of a downgrade that never becomes an outright default at all. This is a meaningfully richer picture than the default-only economic capital calculation in Section 9.4.1 below, which treats every outcome short of default as a full recovery of value; a mark-to-market approach instead recognizes that a downgrade from High to Medium, even without a default, is itself a loss event worth capital against.

A concrete example makes this mark-to-market logic precise. Consider a currently High-rated, 3-year, \$1,000 face-value bond paying a 5% annual coupon, exactly at the yield High-rated bonds currently command, so it trades at par today. One year from now, the bond has two years of remaining cash flows, and its value depends on which rating state it has migrated to: if it remains High-rated, it is still priced at the 5% yield, giving, using Chapter 5's bond-pricing formula,

$$P_{\text{High}} = \frac{50}{1.05} + \frac{1{,}050}{1.05^2} = \$1{,}000.00,$$

unchanged, since coupon and yield still coincide; if downgraded to Medium, the market now demands a higher yield of, say, 7% to compensate for the greater default risk,

$$P_{\text{Medium}} = \frac{50}{1.07} + \frac{1{,}050}{1.07^2} = \$963.84,$$

a loss of \$36.16 in value with no default having actually occurred; and if the bond defaults, the bondholder recovers 40% of face value immediately, \$400.00. Weighting each outcome by Table 9.3's one-year transition probabilities for a High-rated borrower (90%, 8%, 2%) gives an expected one-year-forward value of

$$E[V] = 0.90(1{,}000.00) + 0.08(963.84) + 0.02(400.00) = \$985.11,$$

with a standard deviation of \$84.16 across the three possible outcomes. Notice that this expected value already sits below today's \$1,000 par price purely from downgrade and default risk, entirely separate from any change in the risk-free discount rate, and that the bulk of the \$84.16 of dispersion in this simple three-state example comes from the small but severe default outcome, not from the much more likely but modest Medium-rating markdown, the kind of asymmetric, fat-tailed risk that a full CreditMetrics-style analysis (extended to many rating states and many bonds simultaneously) is designed to capture at portfolio scale.

Lifetime Expected Loss: IFRS 9 and CECL

Everything so far computes expected loss over one year. Since 2018 that has not been what a bank reports: IFRS 9, the ninth International Financial Reporting Standard, and its U.S. equivalent CECL, the Current Expected Credit Loss standard, require provisioning against losses expected over an exposure's *remaining life*, which is what makes credit models the largest modeling estate in most banks. The change answers a crisis-era failure of incurred-loss accounting, under which a bank could not recognize a loss until impairment was objectively evident, so provisions stayed low through the boom and arrived in a rush when capital was scarcest.

IFRS 9 sorts exposures into stages: **Stage 1**, performing, at 12-month expected credit loss; **Stage 2**, having suffered a *significant increase in credit risk*, at *lifetime* loss; **Stage 3**, credit-impaired. CECL drops

Wulin.Suo@Queensu.ca

the staging and requires lifetime loss for everything from day one. Either way a one-year PD no longer suffices: what is needed is a *PD term structure*, the marginal probability of default in each future year.

The transition matrix introduced above already contains one. Take its Medium-rated borrower and Section 9.2.1's bond, EAD = \$1,000 and LGD = 60%, discounting at 5%. Raising the matrix to the nth power and reading the Medium-to-Default entry gives cumulative default probability by year n; differencing gives the marginals.

Year	Cumulative PD	Marginal PD	Undiscounted loss	Discounted loss
1	0.1000	0.1000	\$60.00	\$57.14
2	0.1820	0.0820	\$49.20	\$44.63
3	0.2502	0.0682	\$40.92	\$35.35
4	0.3078	0.0576	\$34.53	\$28.41
5	0.3570	0.0493	\$29.57	\$23.17
			Lifetime ECL	\$188.70

Table 9.4: Lifetime expected credit loss for a five-year Medium-rated exposure

Year two is checkable by hand: a Medium borrower defaults then either by staying Medium and defaulting, $0.80 \times 0.10 = 0.080$, or by being upgraded first, $0.10 \times 0.02 = 0.002$, giving 0.082 marginal and 0.182 cumulative. The marginals decline not because the borrower improves but because the survivors are a shrinking pool. In Stage 1 the bank provisions $0.10 \times 0.60 \times \$1{,}000/1.05 = \$57.14$; in Stage 2, the full \$188.70. Nothing about the borrower changes at transfer, yet $\$188.70/\$57.14 = 3.30$, and that cliff is what makes the significant-increase test one of the highest-stakes classification problems in the bank.

Learning the Term Structure: A Discrete-Time Hazard Model

That calculation assumes the transition matrix is constant for five years. It is not, and the standard requires a forward-looking, scenario-weighted estimate, which a fixed matrix cannot express. Here the term structure stops being arithmetic and becomes a fitted model. The standard formulation is a *discrete-time hazard*: for each loan, in each year it is still alive, model the conditional probability of defaulting during that year,

$$h(a, m) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 a + \beta_2 a^2 + \beta_3 m)}},$$

with a the loan's age and m a macroeconomic index. The quadratic in age captures *seasoning*, the regularity that defaults are rare just after origination, peak a few years in, then decline. Fitted this way it is an ordinary logistic regression on a loan-year panel, which is why it is the workhorse: every tool from Chapter 6 applies unchanged. It is also the discrete-time counterpart of the survival analysis Chapter 15 develops for flagged accounts, and this book's companion technical note on the Kaplan-Meier estimator derives the continuous-time version of the same survival function from the censored likelihood directly.

One subtlety decides whether it is estimable at all. If every loan originates in the same year then age and calendar year are the same variable, and no quantity of data can separate seasoning from the economy. Identification requires *staggered origination*, several cohorts observed over overlapping years, so that loans of the same age meet different economies. This is the credit instance of the age-period-cohort problem, and it constrains how the sample must be assembled rather than how the model is fitted.

Simulating six cohorts over ten calendar years, about 75,000 loan-years, recovers the data-generating parameters closely: the fitted hazard is $-3.138 + 0.600a - 0.099a^2 + 0.455m$ against a true $-3.100 + 0.550a - 0.090a^2 + 0.450m$. Evaluating it under three macroeconomic paths gives lifetime ECL of \$125.49 (benign, $m = -1.0$), \$181.79 (base, $m = 0$), and \$292.82 (adverse, $m = +1.5$); weighting these 30/50/20 gives \$187.11.

Two results deserve attention. The weighted figure lands within a few dollars of the flat matrix's \$188.70, so the two approaches agree about the central case, but only the hazard model produces the other two columns, and the \$125-to-\$293 spread is the entire content of a forward-looking provision. Second, expected loss is *convex* in the macroeconomic factor, so the average of the scenario losses exceeds the loss at the

Wulin.Suo@Queensu.ca

average scenario: the weighted \$187.11 exceeds the base-only \$181.79 by 2.9%. A bank modeling only its central forecast under-provides by that margin in calm conditions and by far more when the scenario spread widens, which is exactly when it matters. This is Jensen's inequality doing supervisory work, and it is why the standard insists on probability-weighting rather than a single best estimate.

Two extensions complete the picture. Borrowers prepay, and a loan that prepays generates no credit loss, so lifetime loss is properly a *competing risks* problem against prepayment; ignoring it overstates provisions on the exposures most likely to disappear before defaulting. And the significant-increase test is deliberately left undefined by the standard, making it a threshold-selection problem under the asymmetric costs Chapter 21 works through explicitly.

9.4 Portfolio Capital, Machine Learning, and Governance

A loan book is not the sum of its loans; this part covers correlated losses, the models that price them, and who checks the result.

9.4.1 Portfolio Credit Risk and Economic Capital

Expected loss, as computed in Section 9.2.1, is the amount a lender should expect to lose on average and should therefore already be reflected in loan pricing and loan-loss reserves; it is not, by itself, a risk measure, since an average loss carries no information about how much losses could vary around that average in an unusually bad year. Economic capital is the buffer a financial institution holds specifically to absorb unexpected losses beyond the expected level, so that it remains solvent even in a bad, though not average, year.

Consider a simplified portfolio of 100 loans, each identical to the Chapter 5 bond example (EAD of \$1,000, PD of 3%, LGD of 60%), with defaults treated, for illustration, as independent across borrowers. The portfolio's expected loss is simply $100 \times \$18 = \$1{,}800$. Treating the number of defaults as a binomial random variable with $n = 100$ and $p = 0.03$, its standard deviation is $\sqrt{np(1-p)} \approx 1.706$ defaults, so the standard deviation of the dollar loss is

$$\hat{\sigma}_{\text{Loss}} = 1.706 \times 0.60 \times 1{,}000 \approx \$1{,}023.52.$$

Approximating the loss distribution as normal (a simplification discussed further below), the 99% Value at Risk of portfolio losses is $\text{EL} + z_{0.99}\,\hat{\sigma}_{\text{Loss}} \approx 1{,}800 + 2.326(1{,}023.52) \approx \$4{,}181$, and economic capital is defined as the unexpected loss beyond the expected level,

$$\text{Economic Capital} = \text{VaR}_{99\%}(\text{Loss}) - \text{EL} \approx 4{,}181 - 1{,}800 \approx \$2{,}381,$$

roughly 1.32 times the portfolio's expected loss. This independent-default assumption is a significant simplification made purely for tractability. In reality defaults are correlated, since borrowers are jointly exposed to the same macroeconomic conditions, and a recession raises every borrower's default probability simultaneously. That correlation makes the true loss distribution's tail considerably fatter than the independent-default binomial approximation suggests, which is precisely why real-world regulatory capital models, such as the Basel Committee's asset-correlation-adjusted framework, explicitly build in a default-correlation parameter rather than assuming independence. Regulatory capital, the minimum capital a bank is legally required to hold under frameworks such as Basel III, is calculated similarly in spirit but is set by regulators using standardized formulas and correlation assumptions rather than a bank's own internal risk estimates, so that banks cannot lower their measured risk simply by choosing favorable internal assumptions.

Correlation between borrowers is not the only way a portfolio's true risk can exceed what its expected loss alone suggests; concentration, how evenly total exposure is spread across borrowers, matters even before accounting for correlation at all. The Herfindahl-Hirschman Index (HHI), $\text{HHI} = \sum_i s_i^2$ where s_i is borrower i's share of total exposure, quantifies this directly. The 100-loan portfolio above, with exposure spread evenly ($s_i = 1/100$ for every loan), has $\text{HHI} = 100 \times (0.01)^2 = 0.01$. Now compare a differently structured portfolio with the same \$100,000 total exposure and the same 3% average PD, but concentrated in only 10 borrowers of \$10,000 each: $\text{HHI} = 10 \times (0.10)^2 = 0.10$, ten times higher. Both portfolios have an identical expected loss of \$1,800, since expected loss depends only on total exposure and average PD, not

Wulin.Suo@Queensu.ca

on how that exposure is distributed across borrowers, but the concentrated portfolio is meaningfully riskier: a single default now removes 10% of the portfolio's exposure rather than 1%, so an unlucky draw of even one or two defaults among ten large, weakly diversified borrowers can produce a loss far outside what the binomial approximation above, implicitly built for a portfolio of many small, similarly sized exposures, would suggest. This is why bank regulators and internal risk models impose single-borrower and single-industry concentration limits as a complement to, not a replacement for, the correlation-adjusted capital calculation of the Vasicek model below: a portfolio can satisfy every correlation assumption in a capital model and still be dangerously undiversified in a way that same model, built around a representative average borrower, never directly observes.

Correlated Defaults: The Single-Factor (Vasicek) Model

The independent-default assumption above is the simplification this chapter's case vignette shows going badly wrong at scale, and the standard way to relax it, the same one underlying the Basel IRB capital formulas, is the single-factor (Vasicek) model. Each borrower's default is driven by an unobserved asset value that depends on a common systematic factor Z, shared by every borrower in the economy, and an idiosyncratic factor unique to that borrower, with asset correlation ρ controlling how much of each borrower's fortunes are tied to the shared factor versus their own. Conditional on the realized systematic factor, defaults become independent, and standard results give the α-quantile of the portfolio's default rate directly as a closed-form worst-case default rate (WCDR),

$$\mathrm{WCDR}(\alpha) = N\left(\frac{N^{-1}(\mathrm{PD}) + \sqrt{\rho}\,N^{-1}(\alpha)}{\sqrt{1-\rho}}\right).$$

The formula is quoted here rather than proved; this book's companion technical note on the single-factor model derives it in full, showing why conditioning on the systematic factor makes defaults independent, why the law of large numbers then collapses an infinitely granular portfolio's loss rate to a deterministic and strictly monotone function of that one factor, and why that monotonicity is exactly what allows the loss distribution to be inverted in closed form. The note also verifies that the resulting distribution has mean exactly PD while being strongly right-skewed, which is the precise sense in which correlation moves risk into the tail without changing the average loss at all. Applying this to the same 100-loan portfolio above (PD = 3%) with an illustrative asset correlation of $\rho = 0.20$, typical of the range Basel's own corporate risk-weight formulas use, at the 99% confidence level,

$$\mathrm{WCDR}(0.99) = N\left(\frac{N^{-1}(0.03) + \sqrt{0.20}\,N^{-1}(0.99)}{\sqrt{0.80}}\right) = N\left(\frac{-1.881 + 0.447(2.326)}{0.894}\right) \approx 17.37\%,$$

more than double the 6.97% figure the independent-default binomial approximation implied at the same 99% confidence level. Translating this into dollar terms, the correlated model's worst-case portfolio loss is $100 \times 0.1737 \times 0.60 \times 1{,}000 \approx \$10{,}422$, against the same \$1,800 expected loss as before, giving economic capital of $10{,}422 - 1{,}800 \approx \$8{,}622$, roughly 3.6 times the \$2,381 figure the independent-default assumption produced. This gap, not a modest correction but more than a tripling of required capital, is why regulators do not allow banks to assume independent defaults, and it is the mechanism this chapter's case vignette describes: a shared systematic factor, whether a national housing downturn or a broader recession, pushes every borrower's default probability in the same direction at once, fattening the portfolio loss distribution's tail far beyond what a model treating each loan as an isolated coin flip would suggest, even when every individual loan's own PD is estimated correctly.

Correlation and stress compound rather than simply add, and it is worth combining them explicitly to see by how much. Applying Section 9.2.1's stressed PD = 8% to this same portfolio, still holding $\rho = 0.20$, gives

$$\begin{aligned}
\text{Stressed expected loss} &= \$4{,}800,\\
\text{Stressed WCDR}(0.99) &\approx 34.17\%,\\
\text{Worst-case loss} &= 100 \times 0.3417 \times 0.60 \times 1{,}000 \approx \$20{,}504,\\
\text{Economic capital} &= 20{,}504 - 4{,}800 \approx \$15{,}704,
\end{aligned}$$

Wulin.Suo@Queensu.ca

roughly 6.6 times the base-case, independent-default economic capital of $2,381 computed at the start of this section. None of the three ingredients behind that 6.6× figure, the stressed PD, the stressed LGD embedded in expected loss, and the asset correlation, is individually extreme. But a bank that only stresses PD and LGD while continuing to assume independent defaults, or that only accounts for correlation under calm-period PD, would each capture a fraction of the true combined effect. That is precisely the compounding risk regulators require banks to model jointly rather than one factor at a time.

The single-factor model above carries one further simplification worth flagging explicitly: the closed-form WCDR formula assumes an infinitely granular portfolio, one made up of a very large number of small, roughly equal-sized exposures, so that no single borrower's own default can materially move the portfolio's total loss on its own. A real loan book is never quite this diversified. A portfolio with a handful of unusually large exposures relative to the rest carries additional name concentration risk that the single-factor formula, by construction, cannot see. If one of this chapter's hypothetical 100 loans were instead ten times its stated size, its individual default would move total portfolio losses far more than the granular formula assumes, even holding every borrower's PD, LGD, and asset correlation fixed. Basel's own capital framework addresses this with a separate granularity adjustment, an add-on capital charge that grows as a portfolio's exposures become more concentrated among a few large borrowers. The elegant closed-form single-factor result above is only as trustworthy as the granularity assumption it depends on. That is one more instance of the general theme running through this entire chapter: every formula here is exact given its stated assumptions, and the discipline of credit risk management lies substantially in knowing which assumption is closest to breaking for a specific, real portfolio.

9.4.2 The Basel Framework: A Brief History

The specific numerical requirements behind regulatory capital did not emerge all at once; they are the product of a decades-long regulatory response to successive financial crises, summarized in Table 9.5.

Accord	Key development
Basel I (1988)	Introduced a simple, standardized minimum capital ratio (8% of risk-weighted assets) applied uniformly across large internationally active banks
Basel II (2004)	Allowed large banks to use internal PD, LGD, and EAD models (much like Sections 9.2.1 and 9.2.2 of this chapter) to compute risk-weighted assets, rather than the fixed weights of Basel I
Basel III (2010–2017, phased in)	Raised capital quality and quantity requirements substantially in response to the 2008 financial crisis, added new liquidity requirements (Chapter 8's LCR and NSFR), and introduced the shift from VaR to Expected Shortfall for market-risk capital discussed in Chapter 8
Basel III finalization (2017; implemented from 2023 onwards)	Limited how far internal models can lower capital, through an output floor phasing in to 72.5% of standardized risk-weighted assets by 2028, and replaced all of Basel II's operational-risk approaches with a single Standardized Measurement Approach

Table 9.5: A brief history of the Basel bank capital accords

Each accord was, in a specific sense, a reaction to the previous one's demonstrated weaknesses. Basel I's uniform risk weights did not distinguish a genuinely safer borrower from a riskier one, which Basel II addressed by permitting internal models. But the 2008 crisis then revealed that those internal models, calibrated on the calm pre-crisis period, had allowed some banks to hold far too little capital against risks that materialized suddenly and severely. Those are the model risk and procyclicality concerns discussed later in this chapter's challenges section, and Basel III's higher and higher-quality capital requirements were designed to address them directly. Basel II's internal-ratings-based approach, in particular, is built directly on this chapter's own machinery: a bank estimates its own PD (Section 9.2.2), LGD (Section 9.2.1), and EAD

Wulin.Suo@Queensu.ca

for each exposure, and feeds them into a regulatory risk-weight formula derived from the single-factor Vasicek model of Section 9.4.1, with the asset correlation itself specified by the regulator rather than estimated by the bank, precisely so that a bank cannot lower its own capital requirement simply by assuming a lower correlation than its true portfolio exhibits, the regulatory-arbitrage concern this chapter's challenges section raises explicitly.

9.4.3 Operational Risk Capital: The Basic Indicator Approach

Operational risk, introduced from the market-risk side in Chapter 8, is harder to quantify than market or credit risk, since there is no equivalent of a return series or a default history that applies uniformly across firms. A rogue trader's unauthorized position, a failed core banking system upgrade, and a mis-sold financial product are all operational risk events, but they share little statistical structure in common.

Basel II's simplest capital treatment for operational risk, the Basic Indicator Approach, now of historical interest but still the clearest illustration of the idea, sidestepped this difficulty by tying capital directly to firm size rather than to any specific operational loss model,

$$K = \alpha \times \overline{\mathrm{GI}},$$

where $\overline{\mathrm{GI}}$ is the bank's average annual gross income over the past three years (a simple proxy for the overall scale of its operations) and $\alpha = 15\%$ is a fixed regulatory parameter. Under that approach, a bank with average annual gross income of \$200 million would hold

$$K = 0.15 \times 200{,}000{,}000 = \$30{,}000{,}000$$

in operational risk capital, regardless of how many operational loss events it has actually experienced.

Basel II also offered a slightly finer standardized variant and an internally modeled Advanced Measurement Approach, but the Basel III finalization of 2017 retired all three in favor of a single Standardized Measurement Approach, which scales capital with a size-based business indicator adjusted by the bank's own historical loss experience; no bank may now set operational-risk capital with its own internal model, a reversal Chapter 10 examines. The Basic Indicator Approach's crude size-based proxy remains a useful illustration of a broader principle in bank capital regulation: when a risk is too difficult to model with precision, credibly, and consistently across institutions, regulators often fall back on a simple, conservative, and hard-to-manipulate formula rather than an elaborate model whose inputs a bank could shade in its own favor, the same tension between model sophistication and gameability raised throughout this chapter's discussion of internal ratings-based approaches to credit risk. The contrast with the rest of this chapter is instructive: PD, LGD, and asset correlation are all difficult to estimate precisely, but a bank at least has a coherent statistical framework, logistic regression, the Merton model, the Vasicek single-factor model, for estimating each one from data that plausibly exists. Operational risk events, a rogue trader, a failed software deployment, a mis-sold product, share so little common structure that no single statistical framework fits all of them well, which is why regulators settled for a crude, size-based proxy rather than insisting on a unified operational-risk analogue of the PD/LGD/EAD decomposition that organizes the rest of this chapter. Chapter 10 takes up operational risk in its own right, developing both the loss-distribution machinery behind that capital charge and the AI methods, incident-text classification, anomaly detection, surveillance, and early warning, that a bank actually uses to manage it.

Machine Learning Applications in Credit Risk

Every technique in this chapter so far, logistic regression, the Merton model, transition matrices, the Vasicek single-factor model, is a classical statistical or structural model, chosen deliberately because a credit decision that affects a real borrower needs to be explainable to that borrower, to a regulator, and to an internal model validator. Machine learning nonetheless enters credit risk modeling in several concrete ways that extend, rather than replace, the tools developed above, several of which this book develops in full elsewhere.

Tree ensembles and gradient boosting, introduced in Chapter 6, routinely outperform logistic regression on raw discriminatory power, capturing nonlinear interactions between features, a debt-to-income ratio that matters far more once credit history falls below some threshold, say, that a linear model like Section 9.2.2's

Wulin.Suo@Queensu.ca

logistic regression cannot represent at all; Chapter 21's capstone case study works through this comparison on a real consumer credit dataset, finding a several-point ROC-AUC improvement from tree ensembles over logistic regression, at the cost of the closed-form Shapley decomposition Chapter 18 showed is only available for a linear-in-the-log-odds model like this chapter's own credit scorer.

A Head-to-Head Test: Gradient Boosting versus Logistic Regression

Section 9.2.2's six-applicant sample is too small, and its two classes too cleanly separated, to demonstrate a genuine tree-ensemble advantage; logistic regression already reaches a perfect in-sample AUC of 1.0 on it, leaving no room for any model to improve on it. To test the claim above properly, the companion notebook instead simulates a larger, more realistic population of 10,000 borrowers, with a genuine threshold interaction built into the true default-generating process: debt-to-income drives default risk, but far more steeply once credit history falls below four years, the kind of nonlinearity a logistic regression linear in both features cannot represent, however it is fit.

Splitting into 8,000 training and 2,000 genuinely held-out borrowers, a logistic regression fit on debt-to-income and credit history alone achieves a held-out AUC of 0.8494; a gradient-boosted tree ensemble fit on the identical two features, with no additional information, reaches a held-out AUC of 0.8634, an improvement of 1.40 percentage points, consistent in direction (if more modest in size, given the smaller and more controlled setting here) with the several-point improvement Chapter 21's real-dataset capstone finds. The gradient-boosting model's own feature importances confirm why: it assigns 83% of its total importance to credit history and only 17% to debt-to-income, reflecting the fact that credit history is doing double duty in the true model, both as a direct risk factor and as the threshold variable that determines how much debt-to-income matters, a role a linear logistic regression's single coefficient on credit history cannot capture.

This result sits directly alongside Section 9.2.3's, where a scorecard was found to beat a gradient-boosted model, and the two are worth reconciling explicitly because the apparent disagreement is the whole argument for binning. The logistic regression beaten here is *linear in both features*, so a threshold effect is precisely what it cannot represent, however it is fit. The scorecard that wins there is not: weight-of-evidence binning gives each band its own value, so the same threshold is absorbed without difficulty. What the tree ensemble buys in this subsection is therefore not flexibility over a scorecard but flexibility over a straight line, and the gap closes once the straight line is replaced by the form lenders actually deploy. The residual advantage a tree retains, once binning is done, is only whatever genuine interactions remain, which in credit data are usually modest. As Section 9.2.2 already emphasized, this improvement is only meaningful because it is measured on genuinely held-out data; had it been measured on the same 8,000 borrowers the tree ensemble was fit to, its greater flexibility alone would inflate the apparent gap regardless of whether the underlying interaction were real.

Alternative data sources, satellite imagery, transaction and cash-flow data, and web activity, developed in Chapter 17, extend credit scoring to borrowers with too little conventional credit-bureau history for Section 9.2.2's approach to work well, a genuinely distinct capability rather than simply a more flexible functional form. Cash-flow underwriting makes the distinction concrete: its raw material is a stream of semi-structured merchant strings, so there are no bins to choose until a text classifier of the kind Chapter 16 develops has decided which transactions are rent, and the machine learning sits at the front of the pipeline rather than at the modeling step. Graph neural networks, introduced in Chapter 17 for a shell-company layering network, apply naturally to counterparty and supply-chain credit risk as well, since a firm's true default risk depends partly on the financial health of its major customers and suppliers, a network effect no borrower-level feature table can represent on its own. In every one of these applications, the same discipline from Section 9.2.2 applies with undiminished force: discriminatory power and calibration must both be validated out of sample, and a model's added flexibility is only worth its added opacity if independent validation and ongoing monitoring can actually keep pace with it.

Monotonic Constraints: Making a Tree Ensemble Defensible

When a tree ensemble is the right choice, it still needs the property Section 9.2.3's scorecard gets for free. A boosted tree fitted to credit data will somewhere do something indefensible: a finite sample always contains regions where noise makes a few high-leverage borrowers look safer than those just below them, and the model learns the wiggle. If predicted default probability falls as debt-to-income rises, then somewhere in the

Wulin.Suo@Queensu.ca

book is an applicant who would be approved by borrowing *more*, and a declined applicant whose adverse action notice cannot honestly be written.

Monotonic constraints tell each tree that splits on a feature may move the prediction in only one direction, so the fitted function is monotone by construction rather than by inspection. Refitting the simulation above with the constraint, the unconstrained model reverses direction at 53 of 199 grid steps, and at its worst its fitted probability *falls* by more than twenty percentage points, from 62.0% to 39.6%, across a single grid step in which debt-to-income rises from 46.8% to 47.1%; the constrained model reverses nowhere. The expected accuracy cost does not appear: ROC-AUC is 0.8757 constrained against 0.8597 unconstrained, both below the 0.8812 ceiling. The constraint *improved* discrimination, because the monotone relationship is genuinely present and forbidding reversals removes only overfitting. That is not a general law; the honest statement is that a constraint encoding something true costs nothing and regularizes, while one encoding something false will cost accuracy and deserves questioning rather than silent removal.

A monotonic constraint is, more generally, an *inductive bias*, prior knowledge about the shape of the solution built into the model class rather than learned from data, in the same family as the translation equivariance a convolutional network gets from weight sharing in Chapter 17.

Fairness in lending is a stronger obligation than the auditing Chapter 18 develops, and one where machine learning is the tool rather than the problem. Under disparate-impact doctrine, showing that a disparity is justified by business necessity does not end the analysis: if a *less discriminatory alternative* exists, a model serving the lender's purpose comparably well with a smaller disparity, then using the original is not defensible. This converts fairness from a test into a *search* over the model space, conducted by retraining under fairness regularization, transforming features that carry disparity without much signal, and varying thresholds and segmentations to trace the achievable frontier of performance against disparity. It is the third item in Section 9.2.3's corollary, and the one setting in credit where fitting many models rather than one is the compliant behavior rather than the suspicious one.

Model Validation and Governance for Credit Models

Chapter 8 developed the general model risk management framework, independent validation, ongoing monitoring, and model inventory and documentation, that applies to every quantitative model in this book, and the same three-lines-of-defense governance structure applies without modification to the credit models developed in this chapter: a lending or trading desk originates the risk (first line), an independent credit risk function builds and monitors the PD, LGD, and rating models this chapter developed (second line), and internal audit periodically verifies the whole process is actually functioning as designed (third line).

Credit models carry two validation concerns beyond the general framework, however. First, discriminatory power, Section 9.2.2's AUC and Gini coefficient, must be checked on genuinely out-of-sample data, not the training sample a model was fit to, the overfitting caution that section raised concretely. Second, calibration, whether a model's predicted probabilities actually match observed default rates, is a distinct property from discrimination that a high AUC does not guarantee: a model can rank borrowers correctly from safest to riskiest while still being systematically over- or under-confident about the actual probability level, precisely the calibration warning Chapter 6 raised and this chapter's credit-scoring section returns to directly. A model risk function validating a credit model must check both properties separately, since a model that fails calibration while passing discrimination can still produce a badly miscalibrated dollar reserve even though it ranks borrowers in the right order.

A mature credit model validation function also revisits, on a regular cycle, every one of this chapter's other structural assumptions, not just the PD model's own discrimination and calibration. Rating transition matrices, Section 9.4.1's independent-default assumption, and Section 9.4.1's asset-correlation parameter are all themselves estimated quantities, and each can drift silently as economic conditions change: a transition matrix estimated from a benign decade will understate downgrade risk once conditions deteriorate, the procyclicality concern raised throughout this chapter, and an asset-correlation assumption that looked reasonable in a calm period is the kind of assumption this chapter's case vignette showed failing at the worst possible moment. Independent validation of a credit model, in other words, is not a one-time check performed at launch but an ongoing discipline applied to every input the model depends on, not merely to the headline PD or LGD figure it ultimately reports.

Wulin.Suo@Queensu.ca

9.5 Case Vignette: The Gaussian Copula and the Mismodeling of CDO Credit Risk

Chapter 8's case vignette described how correlations that looked stable during calm markets rose sharply during the 2008 crisis, eroding the diversification benefit assumed by market-risk models. The credit-risk analogue of that same story concerns collateralized debt obligations (CDOs), securities that pool many individual loans, mortgages, or bonds and slice the resulting cash flows into tranches with different seniority, and the specific correlation model Wall Street and rating agencies used to price and rate them.

In 2000, the quantitative analyst David X. Li published a widely influential paper proposing the Gaussian copula as a tractable way to model default correlation across many borrowers at once, the correlated-default complication Section 9.4.1 flagged as a simplification in this chapter's own economic-capital example. The copula approach let a single correlation parameter summarize how likely two borrowers were to default together, and it was quickly adopted throughout the industry to price and rate mortgage-backed CDO tranches, since it turned an otherwise intractable joint-default problem into a single, easily calibrated number.

The difficulty, with the benefit of hindsight, was the same one this chapter's economic-capital example flags explicitly: the correlation parameter was typically calibrated from a historical period of relatively calm, rising home prices, and it assumed that mortgage defaults across geographically diverse loan pools would remain only modestly correlated, largely independent local events. When the nationwide housing downturn arrived, mortgage defaults across the country rose together far more than the calibrated correlation assumed, the same "correlations go to one in a crisis" pattern Chapter 8's own vignette described for asset returns, and AAA-rated CDO tranches, built on the assumption that a nationwide wave of correlated mortgage defaults was a near-impossibility, suffered mass downgrades and defaults during 2007 and 2008. The episode was popularized in the financial press, most notably a widely read 2009 account describing the Gaussian copula as "the formula that killed Wall Street," not because the mathematics was wrong, a copula is a perfectly legitimate way to model dependence, but because the single correlation number it required was calibrated on the kind of calm, pre-crisis data this chapter has repeatedly warned understates true tail dependence.

It is worth being precise about what, exactly, went wrong, since the lesson is easy to over-simplify into "correlation models are untrustworthy" rather than the more specific and more useful diagnosis this chapter has been building toward. The Gaussian copula and Section 9.4.1's single-factor Vasicek model are close mathematical relatives, both reduce a complicated joint-default problem to a single correlation parameter acting through a shared systematic factor, and neither is wrong in the sense of being mathematically inconsistent.

What failed was an estimation choice made once, on a historical sample that never contained a nationwide housing downturn, and then treated as a fixed input rather than something to be stress-tested the way Section 9.2.1 stress-tested PD and LGD directly. A correlation parameter stress-tested toward a more severe, more correlated scenario, the exercise Section 9.4.1 performs numerically for this chapter's own loan portfolio, would have revealed a dramatically larger tail loss well before 2007, using the same underlying model, not a different one. The failure, in other words, was organizational and procedural as much as mathematical: nothing in the copula framework itself prevented stress-testing the correlation assumption, but the incentive structure across originators, rating agencies, and investors, each benefiting in the short run from a favorable, low-correlation rating, meant that nobody was rewarded for asking the question.

The lesson generalizes directly from Chapter 8's own vignette: a correlation assumption calibrated on historical, relatively calm data is a recurring point of failure across both market and credit risk, and a model's mathematical sophistication, the Gaussian copula is a legitimate, widely used statistical tool in many other contexts, is no substitute for stress-testing the specific assumption, here, default correlation, that turns out to matter most exactly when conditions are worst.

9.6 Challenges in Credit Risk Modeling

Several challenges recur across nearly every credit risk modeling application, extending the market-risk challenges Chapter 8 raised into a credit-specific setting.

Correlated defaults and tail dependence. The independent-default simplification in Section 9.4.1's economic capital example understates portfolio credit risk deliberately, for pedagogical clarity, but a real

Wulin.Suo@Queensu.ca

credit portfolio model must estimate default correlation directly, the difficulty this chapter's case vignette showed going badly wrong at scale. Section 9.4.1's single-factor model is the standard way to bring correlation back in, but it still assumes a single shared systematic factor driving every borrower at once; a real portfolio spanning several industries and geographies is more accurately described by several correlated factors, and misspecifying how many systematic factors actually exist, or how strongly a given borrower loads on each, is both statistically difficult and highly consequential for the resulting capital estimate, recurring in a different guise in Chapter 8's own correlation-stress discussion for market risk.

Procyclicality. PD and LGD estimates calibrated on recent, calm data tend to understate risk during good times and can spike sharply once a downturn is already underway, an amplifying feedback loop, since understated capital requirements encourage more lending precisely when underwriting standards are already loosening, and Section 9.2.1's stressed-PD-and-LGD exercise exists specifically to counteract this by testing the scenario before it arrives rather than reacting once it has. The combined stress-and-correlation calculation in Section 9.4.1, a 6.6× increase in economic capital from three individually modest-looking assumptions, is a direct illustration of just how severely procyclicality can compound once every input moves in the same direction simultaneously, the scenario a bank's capital planning must survive rather than merely a calm-period average.

Data quality and limited loss history. Credit data is plentiful in rows and thin in the dimension that actually constrains a model. A lender may hold records on a million loans, yet the number of complete economic cycles those loans have lived through is one, or zero, and default is a slow event: an origination cohort's true loss rate is not known until years after the underwriting decision it would judge. A decade of records can therefore amount to a single draw from the distribution a PD model is meant to describe, which is why a PD estimated from a decade that happened not to include a recession will systematically understate true credit risk. The sample is also selected rather than merely observed, since outcomes exist only for the applicants the lender chose to approve. That is Section 9.2.2's reject-inference problem restated as a data limitation rather than a modeling one, and no amount of additional rows fixes it, because the missing borrowers are missing by construction. Section 9.2.2's perfect in-sample AUC is the same danger in miniature: with only six training observations, a model's apparent discriminatory power says almost nothing about how it would actually perform on new applicants.

Regulatory and business tension. What makes this tension specific to credit is its timing. A market-risk limit breach announces itself within days, whereas a loosened underwriting standard books revenue immediately and delivers its losses two or three years later, so for several reporting periods a credit function that is being overruled looks indistinguishable from one that was simply too cautious. That asymmetry is why lending standards reliably loosen late in an expansion, when competition for volume is fiercest and the last downturn is furthest away, and it is what the three-lines-of-defense structure introduced in Chapter 8 is meant to counteract here, by separating the unit that originates loans from the unit that independently validates the models pricing and reserving against them. A second constraint has no market-risk analogue at all: a consumer credit model must be explainable to the applicant it declines, under the adverse-action rules Chapter 18 develops, so the business's pressure toward raw accuracy and the regulator's toward transparency can pull the model's functional form in opposite directions, which is the tradeoff the monotonic constraints discussed above exist to negotiate.

Regulatory arbitrage. Whenever a regulatory capital requirement is calculated using a specific formula, whether the Basic Indicator Approach's gross-income proxy or an internal-ratings-based credit model, an institution has some incentive to structure its activities specifically to minimize the resulting capital charge rather than to minimize its actual underlying risk, a gap between measured and true risk that can widen quietly over time even when every individual calculation is technically correct. An institution permitted to choose its own asset-correlation assumption in Section 9.4.1's single-factor framework, for instance, has an obvious incentive to select a lower ρ than its true portfolio exhibits, understating required capital while remaining, on paper, fully compliant with the letter of the regulatory formula.

Machine learning adds a further layer of model risk. As Chapter 6 discussed, tree ensembles and neural networks can capture nonlinear default and loss patterns that logistic regression and the Merton model, both of which impose a specific functional form, cannot, but they also inherit that chapter's overfitting and interpretability concerns in a setting where the consequences of an undetected error are unusually severe. An opaque credit model that a regulator or an internal validator cannot fully interrogate is a much harder model to trust with a bank's actual capital, regardless of its backtested accuracy, which is why Section 9.2.2's

Wulin.Suo@Queensu.ca

discriminatory-power and calibration checks apply with undiminished force to a gradient-boosted credit model, not just to the linear logistic regression this chapter develops by hand.

9.7 Key Takeaways

Credit risk is decomposed into probability of default, loss given default, and exposure at default, whose product gives expected loss. Logistic regression, introduced in Chapter 6, is the standard statistical tool for estimating PD from borrower characteristics, evaluated using the Gini coefficient and AUC for ranking power and separately for calibration. The Merton model and CDS-implied hazard rates show that market prices, equity or credit-default-swap spreads, can estimate PD directly, without a historical default dataset at all.

LGD is not a fixed constant either: the collateral-coverage regression in Section 9.2.1 shows that recovery, and therefore LGD, can itself be estimated from a lender's own historical data using the OLS machinery Chapter 6 introduced for very different problems.

Rating transition matrices extend single-period default probabilities into multi-year cumulative estimates via simple matrix algebra, and credit stress testing makes concrete what happens to expected loss when PD and LGD are pushed toward recession-like values simultaneously, rather than assumed fixed at their calm-period averages. That same matrix supplies the PD term structure IFRS 9 and CECL require, turning a one-year expected loss into a lifetime one: for the chapter's Medium-rated exposure the provision rises from \$57.14 to \$188.70, a 3.30× cliff crossed by reclassification alone, and a fitted discrete-time hazard model conditions that figure on the economy, where the convexity of loss in the macroeconomic factor is why the standard demands probability-weighted scenarios rather than a single central forecast.

The chapter's most useful negative result concerns machine learning itself. A credit scorecard, weight-of-evidence bins fed to a logistic regression, is a generalized additive model fitted by hand, the same object an Explainable Boosting Machine fits automatically, and on simulated data containing a genuine interaction it lands within 0.003 AUC of the theoretical ceiling with nothing beating it. Machine learning earns its keep in credit not by fitting a better function of the same characteristics but by doing what a scorecard cannot: building features from data with no natural bins, producing a term structure rather than a probability, and searching a model space rather than fitting one model. Where a tree ensemble is genuinely the right tool, monotonic constraints buy back the defensibility the scorecard had for free, and here cost no accuracy at all.

Economic capital translates the difference between expected and unexpected loss into a concrete capital buffer, and the single-factor Vasicek model in Section 9.4.1 shows precisely how much larger that buffer must be once correlated, rather than independent, defaults are taken seriously. That combined stress-and-correlation effect multiplied required capital nearly sevenfold in this chapter's own worked example, and this chapter's case vignette showed it contributing to a genuine, systemic mispricing of mortgage-backed credit risk when it was neglected in practice.

None of these methods is self-policing: independent model validation, checking both discriminatory power and calibration separately, and a governance structure like the three lines of defense Chapter 8 introduced are what actually catch a miscalibrated model before it causes a loss, rather than after.

Across every method in this chapter, the central lesson is the same one that closed Chapter 8's discussion of market risk: a risk number, whether a VaR figure or a credit score, is only as trustworthy as the assumptions behind it. A serious risk management practice therefore combines several methods, together with an honest accounting of where they disagree, rather than relying on any single measure alone.

9.8 Suggested Exercises

1. A loan has EAD of \$500,000, PD of 5%, and LGD of 50%. Compute its expected loss, and explain how each of the three components would need to change to cut the expected loss in half.

2. Using the collateral-coverage regression in Section 9.2.1, compute the predicted recovery rate and implied LGD for a loan with a collateral coverage ratio of 1.50, and explain why extrapolating the fitted line this far beyond the observed data (which ranged from 0.40 to 1.20) should be treated cautiously.

Wulin.Suo@Queensu.ca

3. Using the logistic regression coefficients in Section 9.2.2, compute the predicted probability of default and the corresponding credit score for an applicant with a debt-to-income ratio of 22% and 6 years of credit history.

4. Using the same bond as Section 9.2.1, recompute the stressed expected loss under a milder recession scenario with PD = 6% and LGD = 65%, and compare the resulting multiple of the base-case expected loss to the 3.11× figure computed in the text.

5. Using Table 9.1, verify by hand that the Gini coefficient is 1.0, and explain what would happen to the computed AUC if applicant E's true outcome had instead been "No default" rather than "Default."

6. Using the transition matrix in Table 9.3, compute the four-year cumulative default probability for a Medium-rated borrower.

7. Using the Merton model from this chapter, compute the one-year default probability for a firm with asset value $V_0 = \$200$ million, debt of $D = \$120$ million due in one year, asset volatility $\sigma_V = 30\%$, and a risk-free rate of $r = 5\%$.

8. A portfolio of 200 identical loans each has EAD of \$2,000, PD of 4%, and LGD of 45%, with defaults assumed independent. Compute the portfolio's expected loss, the standard deviation of portfolio losses, and its economic capital at the 99% level.

9. A bank's average annual gross income over the past three years is \$350 million. Using the Basic Indicator Approach, compute its operational risk capital requirement, and explain one advantage and one disadvantage of tying operational risk capital to gross income rather than to a bank's actual operational loss history.

10. Explain why a credit model can have a high Gini coefficient while still being poorly calibrated, and describe a concrete lending decision that would go wrong as a result even though the model ranks applicants correctly.

11. Explain, using the Gaussian copula case vignette, why a mathematically sound correlation model can still contribute to a systemic mispricing of risk, and describe what a stress test of the correlation assumption itself, rather than of PD or LGD individually, might have revealed before 2008.

12. Explain why reject inference is fundamentally an incomplete fix for the selection bias described in Section 9.2.2, and describe a concrete way a lender's approval policy could change over time such that a credit model validated on last year's approved applicants performs noticeably worse on this year's.

13. Using the transition matrix in Table 9.3, explain in your own words why a mark-to-market credit risk model would assign a Medium-rated bond a lower value than an identically priced High-rated bond, even in a year when neither bond actually defaults.

14. Pick two of the challenges listed in this chapter and describe a concrete scenario, not already used in the text, in which that challenge could cause a credit risk management system to understate a firm's true risk.

15. Using Section 9.4.1's single-factor formula, recompute the 99% worst-case default rate for the 100-loan portfolio under an asset correlation of $\rho = 0.12$ instead of $\rho = 0.20$, and explain in your own words why a lower asset correlation produces a lower worst-case default rate for the same average PD.

16. Using this chapter's mark-to-market bond example, recompute $E[V]$ and its standard deviation if the Medium-rated yield were 9% instead of 7% (holding the transition probabilities, coupon, and default recovery fixed), and explain why the expected value falls by less than the price gap between the two Medium-rated scenarios might suggest at first glance.

17. Using this chapter's Herfindahl-Hirschman Index example, compute the HHI for a portfolio with the same \$100,000 total exposure concentrated in just 4 borrowers of \$25,000 each, and explain why this portfolio's expected loss is identical to, but its concentration risk is much higher than, the 100-loan portfolio's.

Wulin.Suo@Queensu.ca

18. Using this chapter's EAD example, compute the exposure at default for a \$1,000,000 committed credit line currently drawn to \$400,000, assuming a CCF of 60% instead of 50%, and explain why a higher CCF assumption increases a portfolio's expected loss even if PD and LGD are unchanged.

19. A one-year CDS on a different issuer trades at a spread of $s = 350$ basis points, and the assumed recovery rate is $R = 35\%$. Using Section 9.3.2's credit-triangle approximation, compute the implied hazard rate and the one-year default probability, and compare the result to a Merton-model estimate for the same firm if one were available.

20. Combine a stressed PD = 6% with an asset correlation of $\rho = 0.15$ for the 100-loan portfolio, and compute the resulting economic capital, comparing it to both the base-case (\$2,381) and the fully stressed (\$15,704) figures computed in the text.

21. Explain why the CDS-implied default probability approach requires no estimate of asset volatility, unlike the Merton model, and describe the specific circumstance under which a CDS-implied estimate would not be available for a given borrower.

22. Using this chapter's Merton-model inversion example, explain in your own words why the firm's implied equity volatility (67.03%) is so much higher than its assumed asset volatility (25%), and describe what would happen to that gap, wider or narrower, if the firm instead had much less debt outstanding relative to its asset value.

23. Using the firm in Exercise 7 ($V_0 =$ \$200 million, $D =$ \$120 million, $\sigma_V = 30\%$, $r = 5\%$), compute its Merton-model equity value E_0 and implied equity volatility σ_E, and explain what two market-observable quantities a real analyst would feed into a numerical solver, without knowing V_0 or σ_V in advance, to recover this same result in practice.

24. Using Table 9.2, merge the two lowest debt-to-income bands into a single band covering everything below 25%, recompute that band's weight of evidence and the characteristic's information value, and explain why merging bins can never increase information value and should therefore be done only when monotonicity or bin size demands it.

25. Recompute Section 9.2.3's points spread between the safest and riskiest debt-to-income bands using PDO = 25 instead of 20, and confirm that although the spread in points changes, the implied odds ratio does not. Explain what this says about which parts of a scorecard's scale are substantive and which are presentational.

26. Repeat Table 9.4's lifetime expected credit loss calculation for a *High*-rated borrower instead of a Medium-rated one, using the same transition matrix, LGD, exposure, and discount rate. Verify the second year's marginal probability by hand, report the resulting Stage 1 and Stage 2 provisions, and explain why the ratio between them is *larger* for the higher-quality borrower even though every provision figure is smaller.

27. The chapter's discrete-time hazard example weights the benign, base, and adverse scenarios 30/50/20. Recompute the probability-weighted lifetime ECL under weights of 20/50/30 and 10/50/40, and express each as an excess over the base-scenario-only figure of \$181.79. Explain why that excess grows as weight shifts toward the adverse scenario, referring to the convexity of expected loss in the macroeconomic factor.

28. **Challenge (real data).** Download the UCI Statlog (German Credit Data) dataset (1,000 loan applicants, 20 features, a binary good/bad credit-risk label) directly from the UCI Machine Learning Repository. (a) Fit a logistic regression predicting default and report its AUC and Gini coefficient on a held-out test split, the same validation performed on the toy six-applicant model in Section 9.2.2. (b) Treating the fitted probabilities as an estimated PD and assuming a flat 60% LGD, compute the portfolio's expected loss and compare it to the realized default rate in your test split. (c) Before fitting anything, write a short data-cleaning note on at least two features whose categorical codes (for example, checking-account status) are not self-explanatory from the column names alone, the same undocumented-category-code problem Chapter 21's capstone case study encounters on a different real dataset entirely.

Wulin.Suo@Queensu.ca

9.9 Further Reading

- Saunders and Allen, *Credit Risk Management In and Out of the Financial Crisis*. Covers credit scoring, rating models, and portfolio credit risk with a strong emphasis on the lessons of the 2008 crisis referenced in this chapter's case vignette.

- Merton, "On the Pricing of Corporate Debt: The Risk Structure of Interest Rates," *Journal of Finance*. The original 1974 paper establishing the structural credit model developed in Section 9.3.

- Gupton, Finger, and Bhatia, *CreditMetrics—Technical Document*. J.P. Morgan's original technical document introducing the ratings-migration, mark-to-market portfolio credit risk framework discussed in this chapter's treatment of rating transition matrices.

- Basel Committee on Banking Supervision, *Basel III: A Global Regulatory Framework for More Resilient Banks and Banking Systems*. The primary regulatory source for the capital adequacy concepts introduced in this chapter's discussion of economic and regulatory capital.

- Li, "On Default Correlation: A Copula Function Approach," *Journal of Fixed Income*. The original paper proposing the Gaussian copula model discussed in this chapter's case vignette.

- Salmon, "Recipe for Disaster: The Formula That Killed Wall Street," *Wired*. A widely read journalistic account of the Gaussian copula's role in the 2008 financial crisis, providing helpful context for this chapter's case vignette.

- Vasicek, "The Distribution of Loan Portfolio Value," *Risk*. The original source for the single-factor correlated-default model developed in Section 9.4.1, underlying the Basel IRB capital formulas.

- Hull, *Options, Futures, and Other Derivatives*. Covers credit default swap pricing and the hazard-rate approximation used in Section 9.3.2's market-implied default probability discussion in considerably greater depth.

- Hand and Henley, "Statistical Classification Methods in Consumer Credit Scoring: A Review," *Journal of the Royal Statistical Society, Series A*. The standard survey of the scorecard tradition developed in Section 9.2.3, written before machine learning entered the field and therefore unusually clear about what the classical method was actually optimizing.

- Lou, Caruana, and Gehrke, "Intelligible Models for Classification and Regression," *Proceedings of KDD*. The paper behind the Explainable Boosting Machine, which fits by boosting the same additive form Section 9.2.3 shows a credit scorecard has always had.

- Basel Committee on Banking Supervision, *Guidance on Credit Risk and Accounting for Expected Credit Losses*. The supervisory expectations governing the lifetime expected-loss models discussed in this chapter, including the scenario-weighting requirement whose convexity consequence the chapter computes.

- Bellotti and Crook, "Credit Scoring with Macroeconomic Variables Using Survival Analysis," *Journal of the Operational Research Society*. Develops the discrete-time hazard approach used in this chapter on real consumer credit data, including the macroeconomic conditioning that a fixed transition matrix cannot express.

- Crouhy, Galai, and Mark, *The Essentials of Risk Management*. A practitioner-oriented synthesis spanning market, credit, and operational risk, model validation, and risk governance, tying together the full breadth of topics covered in this chapter and Chapter 8 in a single volume.

Wulin.Suo@Queensu.ca

Chapter 10

Operational Risk Modeling

10.1 Introduction

In January 2008, Société Générale disclosed a loss of roughly 4.9 billion euros. No market view had been wrong, no borrower had defaulted, and no model had mispriced a derivative. A single trader on the bank's arbitrage desk, Jérôme Kerviel, had built enormous unauthorized directional positions and concealed them behind offsetting fictitious trades that the bank's own controls failed to catch until the positions had to be unwound into a falling market. The loss was not a market-risk loss or a credit-risk loss in the sense of the previous two chapters. It was an *operational* loss: a failure of the bank's internal processes, systems, and people.

Chapter 8 developed the machinery for market risk, how much a position's value could move, and Chapter 9 developed the machinery for credit risk, whether a counterparty will pay at all. This chapter turns to the risk that is left over once those two are accounted for, and that neither of their toolkits handles well: the risk of loss from failed internal processes, people, systems, or external events. Operational risk is at once the most intuitive of the three, everyone understands a rogue trader or a systems outage, and the hardest to model, because its events share almost none of the statistical regularity that makes market and credit risk tractable.

It would be a mistake to treat operational risk as a minor residual. For many large banks it consumes a comparable share of regulatory capital to market risk, and its worst realizations, the multi-billion-dollar rogue-trading losses, the mis-selling scandals that end in nine- and ten-figure settlements, the systems failures that halt a payment network, have brought down or nearly brought down major institutions in a way a bad week of trading rarely does. What makes it distinctive is not its size but its character: where market and credit losses arrive through prices and defaults that the previous two chapters could model with dense data, operational losses arrive through the failure of the institution's own machinery, and the data on those failures is scarce, messy, and mostly unstructured.

That difficulty is why this chapter is worth a careful treatment in a book about artificial intelligence in finance, and why its central message is a nuanced one rather than a triumphant one. Artificial intelligence does not hand operational risk the kind of clean predictive model it hands credit scoring or volatility forecasting. What it does instead is asymmetric: it is genuinely powerful in the layers of the problem that are made of unstructured data and rare-pattern detection, reading incident reports, flagging anomalous behavior, surveilling for misconduct, warning early from leading indicators, and genuinely limited in the layer that regulators care most about, putting a credible number on the far tail. Keeping that asymmetry in view is the discipline this chapter is built around.

The chapter proceeds in three movements. It first establishes what operational risk is and why it resists a unified statistical model, then develops the classical actuarial baseline, the loss distribution approach, that any AI method must be measured against. It then works through the four layers where machine learning genuinely contributes, text, anomaly detection, conduct surveillance, and predictive early warning, each reusing methods this book has already developed, before turning reflexively to the ways deploying AI itself *creates* new operational risk, and closing with the regulatory reason AI's natural home here is risk management rather than regulatory capital.

Wulin.Suo@Queensu.ca

10.2 What Operational Risk Is, and How It Is Measured

Operational risk resists a single model, and this part explains why, then builds the loss distribution anyway.

10.2.1 What Operational Risk Is, and Why It Resists a Single Model

The Basel Committee defines operational risk as the risk of loss resulting from inadequate or failed internal processes, people, and systems, or from external events. The definition is deliberately broad, and it is conventionally organized into seven event-type categories, shown in Table 10.1, that span everything from a rogue trader to a flooded data center.

Event type	Representative event
Internal fraud	A trader conceals unauthorized positions
External fraud	A third party breaches systems and steals funds
Employment practices & workplace safety	A discrimination or wrongful-termination settlement
Clients, products & business practices	A mis-sold product or a market-manipulation fine
Damage to physical assets	A fire or natural disaster destroys a facility
Business disruption & system failures	A core banking outage halts settlement
Execution, delivery & process management	A failed trade settlement or data-entry error

Table 10.1: The seven Basel operational-risk event types, with representative losses

What makes this list so awkward to model is precisely that its rows share so little in common. A market-risk desk has a long, dense history of daily returns; a credit portfolio has thousands of loans with observable defaults. Operational risk has neither. Its defining events are *rare*, so any single institution observes only a handful of large losses in its entire history; *heavy-tailed*, so the few large events dominate the total and the average loss is a poor guide to the extreme one; *heterogeneous*, so a rogue-trading loss and a mis-selling fine and a data-center fire have no common driver a single model could exploit; and subject to severe *reporting and survivorship bias*, since near-misses go unrecorded, small losses are absorbed silently, and the institutions that suffer the very largest losses sometimes do not survive to contribute them to any database.

A further complication, invisible in the flat list of Table 10.1, is that the event types differ enormously in their frequency-severity profile, and in opposite directions. Execution, delivery, and process-management errors are the most *frequent*, a mistyped settlement, a duplicated payment, a reconciliation break happens daily at a large bank, but each is individually small. Clients, products, and business-practices events, mis-selling, market-conduct violations, large regulatory settlements, are comparatively *rare* but individually enormous, and at most large banks they dominate total operational losses by value despite being a small fraction of events by count. A model or a control program calibrated to the frequent-but-small failures will therefore miss most of the actual loss, which lives in the rare-but-huge category, one more reason the tail, not the average, is where operational-risk attention belongs.

This is the same conclusion Chapter 9 reached when it introduced operational-risk capital from the credit side: a rogue trader's unauthorized position, a failed core-banking upgrade, and a mis-sold product "share so little common structure that no single statistical framework fits all of them well." That is not a gap this book is about to close with a neural network. It is the fixed constraint within which every method in this chapter, classical or AI, has to operate.

10.2.2 The Four Data Elements of Operational Risk

Because no single data source captures operational risk well, the discipline conventionally draws on four distinct elements, and understanding them explains both where the loss model of the next section gets its inputs and where artificial intelligence has the most to add.

Internal loss data is a bank's own history of realized operational losses, the primary input to the frequency and severity estimates of the loss distribution approach described below, but it is sparse, especially in the

Wulin.Suo@Queensu.ca

tail, and skewed toward the small, frequent losses that are least relevant to capital. *External loss data* is drawn from consortium databases of losses suffered by peer institutions, supplementing the tail with events a single firm has been fortunate enough not to experience, at the cost of relevance, since another firm's loss may reflect a control environment nothing like one's own. *Scenario analysis* fills the remaining gap with structured expert judgment: workshops in which experienced staff estimate the frequency and severity of severe hypothetical events that have not occurred but plausibly could, the only element that can speak to genuinely unprecedented tail risk. *Business environment and internal control factors* are the forward-looking indicators, the key risk indicators of Section 10.3.5 among them, that describe the current control environment rather than past losses.

Three of these four elements are, in large part, text and expert judgment rather than clean numbers, which is precisely why the natural-language and classification methods of this chapter matter so much: the incident narratives behind internal loss data, the free-text descriptions in external databases, and the qualitative reasoning of scenario workshops are all unstructured, and turning them into model inputs is the AI-heavy part of operational-risk practice. The actuarial machinery of the next section is only as good as the data these four elements feed it, and its most common failure in practice is not a flaw in the mathematics but a thin, biased, or miscategorized loss dataset underneath it. Figure 10.1 shows how the four elements combine to feed both the quantitative loss model and the qualitative control assessment.

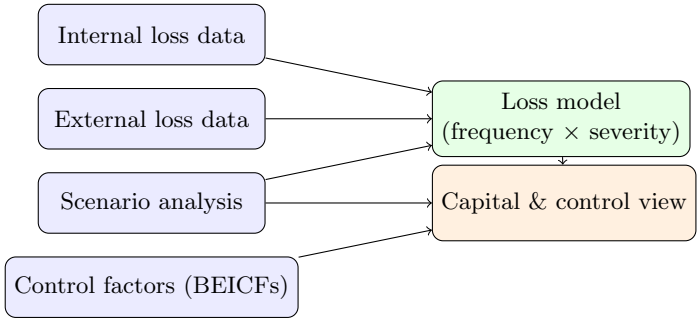


Figure 10.1: The four operational-risk data elements. Three of them are largely unstructured text and judgment, which is where AI enters.

10.2.3 Combining the Elements: Credibility and Bayesian Updating

Figure 10.1 draws three arrows into the loss model without saying what happens where they meet, and that junction is a real modeling decision rather than a bookkeeping step. A business line's own five-year history might imply eight or nine events a year; the consortium database of comparable peers, and the scenario workshop asked to imagine what has not yet happened, might imply twice that. Averaging the two is arbitrary, and picking whichever is preferred is worse. What is needed is a rule that leans on internal data in proportion to how much of it there is, and falls back on the external and scenario view when it is thin, which is precisely the problem credibility theory solves and which Chapter 11 develops in the actuarial setting of pricing an individual policyholder against a book-wide rate.

The operational-risk version has an especially clean form. Treat the annual event count as Poisson with an unknown rate λ, and encode the external-plus-scenario view as a Gamma prior on λ with mean λ_{prior} and a strength parameter β measured in *years of equivalent experience*, so that asserting $\beta = 3$ says the outside view is worth about three years of the bank's own history. The Gamma is conjugate to the Poisson, so after observing $\sum n_i$ events over n years the posterior mean is available in closed form, and it rearranges into the credibility blend of Chapter 11:

$$\hat{\lambda} \;=\; \frac{\alpha + \sum_i n_i}{\beta + n} \;=\; Z\,\hat{\lambda}_{\text{internal}} + (1 - Z)\,\lambda_{\text{prior}}, \qquad Z = \frac{n}{n + \beta},$$

with $\alpha = \beta\,\lambda_{\text{prior}}$. The credibility factor Z carries all the judgment: it rises toward one as the internal history lengthens, and the whole disagreement between the data elements is resolved by a single, auditable statement about how many years of internal experience the outside view is worth.

Wulin.Suo@Queensu.ca

Work it through for the business line of Section 10.2.4. Internal loss data records 42 events over $n = 5$ years, so $\hat{\lambda}_{\text{internal}} = 8.4$. External data and the scenario workshop together point to $\lambda_{\text{prior}} = 18$, held with strength $\beta = 3$ years, so $\alpha = 54$. Then

$$Z = \frac{5}{5+3} = 0.625, \qquad \hat{\lambda} = 0.625(8.4) + 0.375(18) = 5.25 + 6.75 = 12.0,$$

which is also $(54 + 42)/(3 + 5) = 96/8 = 12.0$ read straight off the posterior, and it is the frequency Section 10.2.4 uses. That example's $\lambda = 12$ is not an assumption pulled from the air; it is what the four data elements produce once they are combined properly. The posterior additionally reports its own uncertainty, $\sqrt{\alpha + \sum n_i}/(\beta + n) = \sqrt{96}/8 \approx 1.22$ events per year, which the point estimate alone conceals.

The stakes are visible in the capital number. Running the loss-distribution Monte Carlo described next at each candidate frequency, holding the severity distribution fixed, gives a 99.9% quantile of about $7.19 million on internal data alone, $8.45 million on the blend, and $10.26 million on the outside view alone. A bank that quietly used only its own loss history, the most defensible-looking choice and the one its data best supports, would understate its capital by roughly 15%, and would do so precisely because it had been fortunate enough not to experience the events its peers had. This is the one place where a statistical method reaches inside the loss-modeling core rather than sitting around it, and it is worth being clear that it is Bayesian statistics of the kind Chapter 1 introduced rather than machine learning; what the AI layers below contribute is the raw material this blend consumes, since the incident narratives, external database descriptions, and scenario write-ups all have to be classified and cleaned before any of these counts mean anything.

The Loss Distribution Approach: Frequency Times Severity

Suppose a bank wants a single number summarizing how large its operational losses could be over the coming year, for one business line and one event type, well before knowing which specific events will actually occur. The classical actuarial answer, and still the workhorse of quantitative operational-risk modeling, is the loss distribution approach, which builds the annual loss out of two separately estimated pieces: how *many* loss events occur (frequency), and how *large* each one is (severity).

Frequency is typically modeled as a Poisson random variable with rate λ, the expected number of events per year. Severity is modeled as a positive, right-skewed distribution fit to the sizes of past losses; the lognormal is the standard first choice. The total annual loss S is then the sum of a random number of severity draws,

$$S = \sum_{i=1}^{N} X_i, \qquad N \sim \text{Poisson}(\lambda), \qquad X_i \sim \text{Lognormal}(\mu, \sigma),$$

a construction known as a compound Poisson distribution. Because this sum has no convenient closed form, its distribution is obtained by Monte Carlo: simulate a year by drawing N from the Poisson, then drawing N severities and adding them; repeat for hundreds of thousands of simulated years; and read the required quantiles off the resulting histogram of annual losses. This is the same Monte-Carlo-of-a-portfolio logic Chapter 8 used for market-risk Value at Risk, applied now to a compound frequency-severity sum rather than to correlated asset returns.

10.2.4 A Worked Example

Consider a single business line with a Poisson frequency of $\lambda = 12$ loss events per year, the credibility blend of internal, external, and scenario data computed in Section 10.2.3, and lognormal severities with $\mu = 10.5$ and $\sigma = 1.4$ (in natural logs of dollars). Those severity parameters correspond to a median loss of about $36,000 and, because the lognormal is right-skewed, a larger mean loss of about $97,000, the gap between the two already signaling a skewed distribution in which the average is pulled up by occasional large events. Simulating 500,000 years gives the annual-loss distribution summarized in Table 10.2.

Two features of Table 10.2 carry the entire lesson of operational-risk quantification. First, the expected annual loss, about $1.16 million, is the number a business line would budget for as a routine cost of doing business, and it is almost irrelevant to the capital question. Second, the 99.9% quantile, the loss exceeded only once in a thousand years and the level at which regulatory operational-risk capital is conventionally set,

Wulin.Suo@Queensu.ca

Quantity	Annual loss
Expected annual loss	\$1,160,000
99% Value at Risk	\$4,340,000
99% Expected Shortfall	\$6,110,000
99.9% Value at Risk (regulatory capital quantile)	\$8,450,000

Table 10.2: Loss distribution approach: simulated annual operational loss for $\lambda = 12$, Lognormal$(10.5, 1.4)$ severities, 500,000 simulated years

is about \$8.45 million, roughly *seven times* the expected loss. Capital for operational risk is overwhelmingly about the tail, not the average, and a model that fits the body of the loss distribution beautifully while getting the tail slightly wrong can misstate the capital number by millions. Modeling that tail well, typically with the heavy-tailed and extreme-value methods that are a specialized study in their own right, is the central technical difficulty of operational-risk capital, and precisely where the sparse, heavy-tailed loss data is least forgiving.

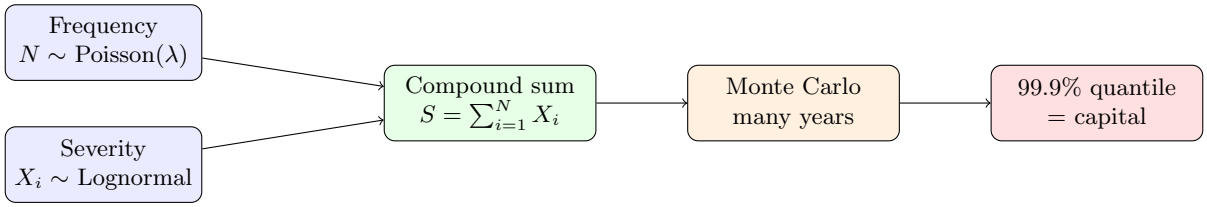


Figure 10.2: The loss distribution approach: Monte Carlo convolves frequency and severity into an annual-loss distribution, and capital is read off its far tail

10.3 Where Artificial Intelligence Enters

Four distinct layers of machine learning sit on top of operational-risk data, each answering a different question.

10.3.1 Where Artificial Intelligence Enters: Four Layers

Nothing in the loss-distribution machinery above used machine learning, and that is deliberate: the frequency-severity core of operational-risk quantification is largely a small-data, distributional problem to which modern AI adds little. Artificial intelligence contributes elsewhere, in four layers that sit around the loss-modeling core rather than inside it, shown in Figure 10.3. Each corresponds to a different point on the timeline of an operational-risk event, from the leading indicators that precede it, through the near-misses and anomalies that presage it, to the incident record it leaves behind, and each reuses a family of methods this book has already developed.

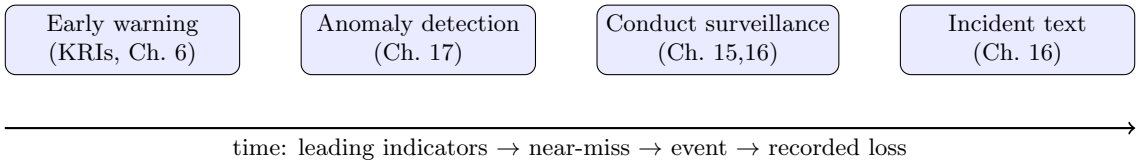


Figure 10.3: Four layers where machine learning contributes, along the timeline of a loss event

Wulin.Suo@Queensu.ca

10.3.2 AI Layer 1: Text as the Primary Operational-Risk Data Source

The single most abundant operational-risk data source inside a bank is not a table of losses; it is text. Incident reports, internal-audit findings, risk-and-control self-assessment narratives, customer complaints, and help-desk tickets accumulate continuously and describe operational events in far richer detail than any structured loss database captures. The problem is that this text is unstructured, and turning it into something a loss model can use, above all, sorting each incident into a Basel event type so that frequencies and severities can be estimated per category, is a text-classification task of the kind Chapter 16 developed.

Consider a minimal illustration in the spirit of Chapter 16's Naive Bayes classifier, distinguishing two event types from short incident descriptions: internal fraud (IF) and business disruption or system failure (SF). Table 10.3 gives a tiny training set.

Event type	Incident descriptions (training)
Internal fraud (IF)	"trader hid unauthorized position"; "employee falsified approval to hide loss"; "unauthorized trade concealed by staff"
System failure (SF)	"batch system failed overnight settlement"; "software deployment caused outage"; "system outage delayed settlement"

Table 10.3: Training incidents for a two-class Naive Bayes event-type classifier

With equal class priors ($P(\text{IF}) = P(\text{SF}) = 0.5$), a vocabulary of $V = 24$ distinct words, and Laplace (add-one) smoothing, the classifier scores a new incident by summing log-probabilities word by word, just as in Chapter 16. For the smoothed likelihood of word w in class c,

$$P(w \mid c) = \frac{\text{count}(w, c) + 1}{\text{count}(c) + V},$$

where $\text{count}(c)$ is the total word count in class c. Classifying the new report "unauthorized trade hidden by trader" gives log-scores of about -15.83 for internal fraud and -18.75 for system failure; exponentiating and normalizing turns these into a posterior probability of **0.949** for internal fraud. The words *unauthorized*, *trade*, and *trader* all appear in the fraud training incidents and none in the system-failure ones, so even though *hidden* was never seen in training (and contributes only through smoothing), the classifier is decisive and correct.

At production scale the same idea, moved from a hand-checkable Naive Bayes to the TF-IDF representations and contextual embeddings of Chapter 16, does three jobs at once: it routes each incident to the right Basel category so the loss distribution approach's frequency-severity model is fed clean per-category data; it clusters free-text root-cause narratives to surface recurring control weaknesses no single report would reveal; and it flags, from complaint text alone, the mis-selling and conduct issues that make up the "clients, products and business practices" event type, often the single largest source of operational losses at large banks.

The clustering job deserves emphasis, because it is where text analytics does something the structured loss database cannot. A bank's incidents are logged by whoever wrote them up, in inconsistent language, and the Basel category assigned at logging is often a coarse after-the-fact guess. Unsupervised topic modeling over the incident corpus, the same dimensionality-reduction impulse as Chapter 6's clustering, groups incidents by what actually happened rather than by how they were filed, and it routinely reveals that a scatter of separately logged "minor" incidents, a repeated reconciliation break, a recurring manual workaround, share a single underlying control failure that, uncorrected, is one bad day away from a large loss. Extracting a structured severity and root cause from each narrative, increasingly with the large language models of Chapter 16 rather than keyword rules, is what turns a pile of free text into the clean per-category frequency and severity data the loss distribution approach silently assumes it was handed.

10.3.3 AI Layer 2: Anomaly Detection for Near-Misses and Control Breaks

Waiting for a recorded loss is waiting too long. Many operational failures announce themselves first as *anomalies*, a settlement process that suddenly takes an unusual path, a user account that touches systems

Wulin.Suo@Queensu.ca

it never normally touches, a trade booked outside its desk's usual pattern, well before they crystallize into a reportable loss. Because these near-misses are, by definition, deviations from normal behavior rather than examples of a labeled class, they are a natural fit for the unsupervised anomaly-detection methods of Chapter 17, above all the autoencoder, which learns to reconstruct normal activity and flags whatever it reconstructs poorly.

The mechanism is the one Chapter 17 developed: train a model on a large sample of ordinary daily control-log feature vectors, then score each new vector by its reconstruction error, the squared distance between the vector and the model's attempt to reproduce it, and flag any observation whose error exceeds a high percentile of the normal distribution. In a stylized five-feature illustration where normal reconstruction errors follow the expected chi-square pattern, the 99th-percentile threshold sits at about 14.9 (close to the theoretical chi-square value of 15.1), and an injected anomalous vector representing a process break produces a reconstruction error of 35.5, comfortably above the threshold and therefore flagged.

The single-vector result flatters the method, though, and a fuller test shows why. Applying the same 99th-percentile threshold (about 15.2 in this run) to a test set of 500 normal observations and 50 genuinely anomalous ones, whose feature means are shifted only moderately rather than dramatically, gives the confusion matrix in Table 10.4: the detector catches 28 of the 50 anomalies (recall 0.56) at a precision of 0.85. The precision is respectable, but the recall is the sobering number: at a threshold strict enough to keep false alarms low, the detector misses nearly half of the moderate anomalies entirely. Loosening the threshold to catch more of them would flood the investigators with false positives, the same precision-recall trade-off Chapter 15 made central to fraud detection, and it is why anomaly detection is a triage tool that narrows where humans look rather than a complete substitute for their judgment.

	Flagged	Not flagged
Truly anomalous (50)	28 (true positive)	22 (false negative)
Truly normal (500)	5 (false positive)	495 (true negative)

Table 10.4: Anomaly detector at the 99th-percentile reconstruction-error threshold, on a test set of 500 normal and 50 anomalous observations

The same logic, at production scale and on real transaction, access, and trade logs, is what lets an operational-risk team catch a control failure while it is still a near-miss, the one point on the event timeline where intervention is cheapest, provided the team reads the recall number honestly and does not mistake a quiet alert queue for an absence of problems.

10.3.4 AI Layer 3: Conduct and Rogue-Trading Surveillance

The Société Générale loss that opened this chapter belongs to a long line, Barings and Nick Leeson in 1995, Société Générale and Jérôme Kerviel in 2008, UBS and Kweku Adoboli in 2011, in which a single trader's concealed positions, undetected until far too late, produced a loss large enough to threaten the institution. Each is, in retrospect, a surveillance failure: the warning signs, unusual trade cancellations, positions inconsistent with stated limits, communications hinting at concealment, existed in the bank's own data and were simply not connected in time.

Conduct surveillance is where two of this book's threads converge on operational risk. The trade-pattern side is anomaly detection applied to a trader's own activity, the methods of Chapter 15's financial-crime models and Section 10.3.3's reconstruction-error approach, now hunting for a book that no longer looks like its own history. The communications side is natural language processing applied to electronic communications, the e-comms surveillance systems that scan chat and email for the linguistic markers of collusion, concealment, or market manipulation, using the sentiment and classification machinery of Chapter 16. The 2012–2015 LIBOR and foreign-exchange manipulation cases, in which the incriminating evidence was largely trader chat, are the reason every major bank now runs NLP-based communications surveillance as a first-class operational-risk control rather than an afterthought.

A minimal illustration shows how the communications side works, using the same Naive Bayes machinery as Section 10.3.2. Suppose a surveillance model is trained on two classes of short chat messages: ordinary

Wulin.Suo@Queensu.ca

business talk ("please confirm the trade details," "the settlement date is tomorrow") and messages exhibiting concealment markers ("keep this off the record," "do not put this in writing," "delete this message after reading"). Scoring the new message "do not leave a trail on this," with a vocabulary of 36 words and equal priors, gives log-scores of about -23.86 for the suspicious class and -28.99 for the clean class, a posterior probability of **0.994** that the message warrants review. The words *do*, *not*, *leave*, and *trail* echo the concealment training examples closely enough that the classifier is nearly certain, the kind of first-pass triage that lets a human surveillance analyst focus on the small fraction of communications most likely to matter, from a daily volume no team could read in full.

The caveat parallels the fraud-detection lesson of Chapter 15: the base rate of genuine misconduct in trader communications is extremely low, so even a highly accurate classifier flags far more innocent messages than guilty ones, and a surveillance program's usefulness depends as much on ranking and prioritizing its alerts, and on the human review that follows them, as on the raw accuracy of the classifier itself.

The trade-pattern side of surveillance is where operational risk connects most directly to the market-microstructure material of Chapters 13 and 14. A rogue trader's book, by construction, stops looking like its own history: positions grow beyond stated limits, hedges that should offset a position are fictitious or absent, and the pattern of trade cancellations and amendments used to conceal the position departs from the desk's normal behavior. Each of these is detectable as an anomaly against the trader's own historical baseline, and the most effective surveillance systems fuse the two channels, a communications flag and a trading-pattern flag arriving together is far more informative than either alone, an application of the Bayesian-updating logic Chapter 1 introduced, in which two independent weak signals combine into one strong one. The persistent difficulty, common to every rogue-trading case in the historical record, is that the concealment is deliberate and adaptive: the trader knows the controls and works around them, which is why surveillance is an adversarial problem in which yesterday's detection rule is precisely what tomorrow's evasion is designed to defeat.

10.3.5 AI Layer 4: Predictive Early Warning from Key Risk Indicators

The earliest point on the event timeline is before any anomaly at all, when only the *conditions* that make a loss more likely are present. Operational-risk teams track these conditions as key risk indicators: measurable quantities such as staff overtime hours, system downtime, transaction volume spikes, and employee turnover that plausibly precede loss events. The predictive question, does this business unit's current indicator profile signal elevated loss risk next quarter, is a supervised classification problem of the kind Chapter 6 developed, and a logistic regression is the natural first model.

Consider the eight business units in Table 10.5, each described by three indicators, overtime hours (in hundreds), system downtime hours, and staff turnover percentage, and labeled by whether a major loss event followed in the next quarter. A logistic regression fit to this data (after standardizing the indicators so their coefficients are comparable) gives positive standardized coefficients on all three (about 0.82 on overtime, 0.69 on downtime, and 0.77 on turnover), confirming that each is associated with higher loss probability, with overtime the strongest single signal. Those magnitudes come from scikit-learn's default fit, which adds an L2 penalty with $C = 1$, and here the penalty is doing real work: overtime alone separates the two groups perfectly (at most 22 for the quiet units, at least 40 for the loss units), as do downtime and turnover, and under such *complete separation* the unpenalized maximum-likelihood estimate does not exist, since the likelihood keeps rising as the coefficients grow without bound. The signs are trustworthy; the exact sizes, and the ranking among three nearly equal coefficients, reflect the penalty as much as the data, which is a reason to treat any eight-observation model as an illustration rather than an estimate.

The fitted model then scores new units: a stressed unit running high on all three indicators receives a predicted loss probability of about **0.90**, while a calm unit low on all three receives about **0.07**, a spread wide enough to be genuinely useful for prioritizing where limited control-testing attention should go.

The caveat is one this book has raised before and that applies with particular force here. The moment a key risk indicator becomes a management target, staff have an incentive to manage the indicator rather than the underlying risk, and it ceases to measure what it once did, the Goodhart's Law dynamic introduced in Chapter 1. A unit told that high overtime flags it for scrutiny can simply stop recording overtime. Predictive early-warning models are therefore best treated as one input to human judgment, monitored for the kind of gaming that would quietly hollow out their signal, rather than as automated verdicts.

Wulin.Suo@Queensu.ca

Unit	Overtime (00s hrs)	Downtime (hrs)	Turnover (%)	Loss event?
1	12	2	8	No
2	45	9	22	Yes
3	18	3	10	No
4	50	12	30	Yes
5	8	1	5	No
6	40	8	25	Yes
7	22	4	12	No
8	55	15	35	Yes

Table 10.5: Eight business units: key risk indicators and subsequent major-loss outcome

10.4 From Detection to Governance

Detecting a control breach is only useful if something changes; this part covers prevention, the risks AI itself introduces, and validation.

10.4.1 From Detection to Prevention: Process Mining

The four detection layers above catch operational failures as or after they happen; a fifth, more preventive use of data aims to remove the conditions that let them happen at all. Process mining reconstructs how a business process actually ran, step by step, from the timestamped event logs that back-office systems generate automatically, and compares that reconstructed reality against how the process was designed to run. The operational-risk value is that the gaps between the two, a control step routinely skipped, an approval granted before its prerequisite, a transaction path that bypasses a four-eyes check, are the latent control weaknesses that eventually produce a loss, and they are visible in the logs long before any loss occurs.

Process mining is less glamorous than the machine-learning layers and uses simpler techniques, graph reconstruction and conformance checking rather than deep networks, but it targets the cheapest possible point of intervention: fixing a broken control before it is ever exploited. It pairs naturally with the anomaly detection of Section 10.3.3, which flags individual unusual events, by instead flagging systematic deviations in the process itself, and with robotic process automation, which, by executing routine steps deterministically, removes the human error that the "execution, delivery and process management" event type of Table 10.1 is built around, while introducing its own new automation-failure risk in exchange.

This preventive posture also reframes where the four AI layers deliver their value. Detection reduces the *severity* of a loss by shortening the time between a failure and its discovery; prevention reduces the *frequency* of loss events outright by removing the conditions that cause them. In the frequency-severity language of the loss distribution approach, the surveillance and anomaly layers act mostly on severity, catching an unfolding event sooner, while process mining, robotic automation, and the leading-indicator layer act on frequency, and a mature operational-risk program deliberately invests across both so that the loss distribution is pulled inward from both ends rather than only truncated at the tail.

10.4.2 Entitlement Graphs and Segregation of Duties

The oldest operational-risk control of all is *segregation of duties*: no one person should be able to both create a payee and pay it, or both enter a trade and confirm it, because a single individual holding both halves can complete a fraud without needing anyone's cooperation. Every rogue-trading case in Section 10.3.4's list involved this control failing. Testing it sounds trivial, a matter of checking whether any user holds two conflicting permissions, and in a small system it is. What makes it hard at scale is that permissions are rarely granted to people directly. They are granted to roles, roles are nested inside other roles for administrative convenience, and a user inherits the transitive closure. The conflict is then a *path* through the access graph, not an edge, and a check written over directly-granted permissions cannot see it.

Wulin.Suo@Queensu.ca

Table 10.6 makes the gap concrete on a deliberately small system: eight users, six granting roles, two composite roles that merely nest others (`AP_MANAGER` contains the accounts-payable clerk and supervisor roles; `OPS_ONCALL` contains treasury operations and static data), and four declared toxic permission pairs. Checking each user's directly-granted permissions flags exactly one user, `u3`, who visibly holds both halves of a conflict. Resolving the nested roles first and then applying the identical four-pair test flags four users. The three additional violations, `u4`, `u6`, and `u8`, are invisible to the direct check not because the test is wrong but because each user's conflicting permission arrives one hop away, through a composite role granted for an unrelated reason.

User	Conflicting pair held	Visible to a direct check?
`u3`	create vendor + approve payment	Yes
`u4`	create vendor + approve payment	No, via `AP_MANAGER`
`u6`	modify static data + release payment	No, via `OPS_ONCALL`
`u8`	create vendor + release payment	No, via `OPS_ONCALL`

Table 10.6: Segregation-of-duties conflicts in an eight-user system. A pairwise check on directly-granted permissions finds one; following paths through nested roles finds four.

A ratio of four to one on eight users is not itself the finding; the point is that the missed fraction grows with the depth of role nesting, and real entitlement systems at large banks carry tens of thousands of users, thousands of roles, and nesting several levels deep, accumulated over decades of joiners, movers, and leavers whose permissions were added on each move and rarely removed. At that scale the same question becomes a genuine graph problem, and the network methods this book develops elsewhere apply directly: reachability queries of the kind Chapter 15's layering-network analysis uses to trace funds through intermediaries, now tracing authority through roles, and, where the toxic combinations are not all known in advance, the graph neural networks of Chapter 17 learning which access patterns resemble those that preceded past incidents. This is the cheapest intervention point in the entire chapter, since a conflict removed from the entitlement graph is a loss event that cannot occur through that route at all, as against the detection layers, which at best notice it sooner.

10.4.3 Predictive Process Monitoring

Conformance checking compares a process against its design after the fact. The natural extension is to ask, of a case still running, what is likely to happen to it next, which turns the event log into a sequence-prediction problem of the kind Chapters 7 and 16 develop. Given the prefix of activities a payment or a trade settlement has completed so far, together with their timestamps and attributes, a model predicts the next activity, the remaining time to completion, or the probability that the case ends in a breach, an exception, or a manual write-off. Recurrent and attention-based architectures are the standard tools, since the informative signal is usually the *order* of steps and the gaps between them rather than any single step in isolation, which is precisely the structure Chapter 16's self-attention mechanism was built to exploit.

The operational-risk value is that it converts process data from a diagnostic record into a leading indicator at the level of the individual case, complementing the business-unit-level indicators of Section 10.3.5: a settlement predicted to breach its deadline can be escalated while there is still time to act on it. The caveats are the familiar ones and apply with full force. The training labels come from historical cases whose outcomes were themselves shaped by whatever interventions the firm was already making, so a model that learns from them is learning the process as operated rather than the process as designed; and a prediction that triggers an intervention destroys the very outcome it predicted, making honest evaluation harder than a backtest suggests, which is the same reflexivity Section 10.3.5 raised about key risk indicators and which the next section takes up as a general problem.

Wulin.Suo@Queensu.ca

The Reflexive Turn: Artificial Intelligence as a Source of Operational Risk

Every method in the four layers above uses AI to *manage* operational risk. But deploying those same models is itself an operational activity, with its own failure modes, and so artificial intelligence is not only a tool against operational risk but increasingly a source of it. This reflexive point is not a rhetorical flourish; it is now an explicit category in most banks' operational-risk taxonomies.

The oldest form is model risk, the risk that a model is simply wrong, or is used outside the conditions it was built for, which this book has flagged from Chapter 6 onward and which regulators treat as an operational-risk discipline in its own right. A credit or trading model whose assumptions are quietly violated by a changing environment produces losses that are, in the end, operational: a failure of an internal system. The monitoring apparatus of Chapter 18, population-stability indices, drift detection, and the explainability tools that let a human challenge a model's outputs, is best understood as an operational-risk control for models, the mechanism by which model risk is detected before it becomes a loss.

Generative AI adds newer failure modes that did not exist when the Basel event types were written. A large language model that fabricates a plausible but false answer (hallucination), a system manipulated into ignoring its instructions through adversarial input (prompt injection), and a critical dependence on a single external model vendor whose service can change or fail (third-party and concentration risk) are all operational risks in the strict Basel sense, failures of systems, processes, or external parties, even though no traditional operational-risk framework anticipated them.

A sharper version of the same concern arrives once the model stops merely producing text and starts *taking actions*. The LLM agents with tool use that Chapter 16 describes do not draft an answer for a person to read and check; they call functions, and those functions can submit an order, release a payment, open a ticket, or change a configuration. That difference is not one of degree. Every control in the four detection layers above assumes a window between a failure and its consequences, the window in which an anomaly is a near-miss rather than a loss, and an agent that acts closes that window by construction: the first observable evidence is an executed transaction, not a proposal anyone declined. Two further properties compound it. An agent granted the entitlements it needs to be useful across several systems is a segregation-of-duties breach of the kind Section 10.4.2 exists to prevent, since it holds both halves of a conflicting pair and, unlike a human, will never hesitate at the second half; and because it acts at machine speed and without fatigue, a single flawed instruction propagates across thousands of cases before the daily control report that would have caught it is even generated. The controls that follow are correspondingly conventional and worth stating plainly, because they are the ones the operational-risk framework already knows how to write: scope each agent's permissions to the minimum its task requires rather than to the maximum its convenience suggests, keep a human approval step on any action that moves money or commits the firm, log every tool call as a first-class auditable event, and impose hard rate and value limits so that a failure is bounded before anyone notices it. None of this is novel risk management. It is the same segregation, authorization, and limit discipline banks have applied to human traders since Barings, applied to a non-human actor that the existing control framework does not yet recognize as one.

A concrete example makes the mapping vivid. Suppose a bank deploys a retrieval-augmented assistant, of the kind Chapter 16 described, to draft responses to client queries about their products. If the model confidently states an incorrect fee or an inaccurate contractual term, and a client acts on it, the resulting loss is not a market or credit loss; it is a "clients, products and business practices" operational loss, indistinguishable in its consequences from a human adviser's mis-statement, but arriving at a scale and speed no human adviser could match, since the same flawed model answers thousands of queries before anyone notices. The controls are correspondingly operational: retrieval grounding to reduce fabrication, human review of high-stakes outputs, logging for after-the-fact audit, and the drift monitoring of Chapter 18 to catch a model whose behavior degrades after a vendor update. The lesson is that embedding AI does not create a new *kind* of risk so much as a new, faster *channel* for the operational risks the framework already names. As institutions embed the generative methods of Chapter 16 into research, client service, and compliance workflows, mapping these new failure modes onto the operational-risk framework, and building controls for them, becomes part of the operational-risk function's job.

Wulin.Suo@Queensu.ca

Regulation: Why Artificial Intelligence Lives in Management, Not Capital

There is a striking regulatory irony at the center of this chapter. Just as machine learning was maturing into a serious tool for operational risk, the Basel Committee moved decisively *away* from letting banks use their own internal models to set operational-risk *capital*. The Advanced Measurement Approach, which had permitted sophisticated internal loss-distribution models of the kind this chapter's loss distribution approach describes, was retired in the Basel III finalization in favor of the Standardized Measurement Approach, which ties capital mechanically to a size-based "business indicator" adjusted by the bank's own historical loss experience, the same philosophy as the Basic Indicator Approach Chapter 9 described, chosen for the same reason: the events are too heterogeneous and too sparse for regulators to trust any single institution's internal model of the tail.

The consequence is clarifying rather than discouraging. Because regulatory operational-risk capital is now formulaic, the value of the machinery in this chapter, the loss-distribution model and, above all, the four AI layers, lies in *managing* operational risk rather than in setting the regulatory number: in reducing the frequency and severity of loss events through earlier detection, better surveillance, and sharper prioritization, and in informing a bank's own internal economic-capital view even where it no longer drives the regulatory one. Artificial intelligence's home in operational risk is the risk-management function, not the capital calculation, and that is exactly where its comparative advantage, unstructured data and rare-pattern detection, actually lies. Model risk itself remains squarely within supervisory scope through model-governance expectations such as the U.S. Federal Reserve's SR 11-7 guidance, which is why the reflexive controls described above are not optional.

Validating and Backtesting Operational-Risk Models

Every model in this chapter must itself be validated, and operational risk makes validation unusually hard for the same reason it makes modeling hard: the events are too rare to backtest in the ordinary sense. A market-risk Value at Risk model can be backtested against roughly 250 daily outcomes a year, as Chapter 8 described, so a 99% model should be breached about two or three times annually and a wildly miscalibrated one is caught quickly. A 99.9% operational-risk capital model, by contrast, makes a prediction about a once-in-a-thousand-year loss that no institution will ever observe often enough to test directly. Validation therefore leans on indirect checks: goodness-of-fit tests on the severity distribution, sensitivity analysis to the assumed severity and frequency distributions, benchmarking against external loss data, and structured challenge of the scenario-analysis judgments that drive the tail.

The AI layers, by contrast, are far more testable, and should be held to the same standards as any other model in this book. An incident classifier has a confusion matrix (Chapter 6); an anomaly detector has a precision-recall trade-off measured against injected or historical anomalies (Chapter 15); an early-warning model has an out-of-sample discriminatory-power measure such as the AUC of Chapter 9. Holding these models to that standard, rather than trusting them because they happen to address an important problem, is itself part of the model-risk discipline of the reflexive turn above: an unvalidated surveillance model that quietly stops working is not a safeguard against operational risk but a new instance of it.

That said, "more testable" should not be read as "straightforwardly testable," because for the surveillance layer in particular the labels needed to compute any of those metrics barely exist, and the ones that do exist are selected in a way that flatters the model. Confirmed misconduct is not merely rare; it is rare *and* observed only where some control already caught it. A recall figure computed against a bank's history of confirmed rogue-trading cases is therefore recall against the events the current surveillance system, or an auditor, or a market move, happened to expose, and it is silent about the ones that were concealed successfully, which are precisely the population the model exists to find. A model can post an excellent recall on that sample while being systematically blind to the concealment techniques that work. Chapter 9 meets the identical structure in credit under the name reject inference, where a scorecard can only be validated on applicants it approved and never on the ones it turned away, and the response here borrows the same instincts: evaluate against deliberately injected synthetic scenarios that encode known evasion patterns as well as against historical cases, treat the alert queue's precision, which *is* cleanly measurable from investigator dispositions, as the more trustworthy of the two numbers, and read any reported recall as an upper bound rather than an estimate. The honest summary is that the detection layers can be validated well against false positives and only weakly against false negatives, which is a further reason, alongside

Wulin.Suo@Queensu.ca

the base-rate problem of Section 10.3.4, to treat a quiet alert queue as an ambiguous signal rather than a reassuring one.

10.5 Case Vignette: The 2012 JPMorgan "London Whale"

In 2012 JPMorgan Chase lost more than $6 billion in its Chief Investment Office through a portfolio of credit derivatives that grew so large a single trader earned the nickname "the London Whale." The loss is usually told as a risk-management story, and it was, but at its core it was operational. A spreadsheet-based Value at Risk model used to monitor the position contained a formula error that understated its risk; the position had grown beyond what its stated strategy could justify; and internal warnings were not escalated in time. Every element, a flawed model, a control that failed to bind, an escalation process that did not function, is an operational-risk failure in the Basel sense, not a market-risk one, even though the loss ultimately materialized through market prices.

The episode is a compact illustration of this chapter's whole argument. The model error is model risk (the reflexive turn discussed earlier in this chapter), of the kind Chapter 18's validation and monitoring tools exist to catch. The unnoticed drift of the position away from its mandate is a near-miss anomaly (Section 10.3.3) that trade-pattern surveillance is designed to surface. The failure to escalate is the kind of process breakdown that leading indicators (Section 10.3.5) and incident analysis (Section 10.3.2) are meant to make visible. No single AI system would have prevented the London Whale, but each layer of this chapter maps directly onto a point at which the failure could have been caught earlier, which is precisely why operational-risk management is best understood as a portfolio of overlapping detection layers rather than a single model.

The aftermath reinforces the chapter's regulatory theme. JPMorgan ultimately paid roughly $920 million in coordinated regulatory settlements over the episode, an amount that dwarfs the capital any internal model would plausibly have held against this specific risk, and the enforcement findings centered not on the size of the market bet but on the control and governance failures around it: risk limits that were breached and then re-calibrated rather than enforced, a valuation process that masked losses, and a model change that was implemented without adequate validation. That emphasis is why, as the regulation discussion earlier in this chapter argued, the supervisory response to operational risk runs through governance expectations and model-risk discipline rather than through a capital formula, and why the value of this chapter's tools lies in strengthening the controls whose failure, not the trade itself, turned a large loss into a far larger one.

10.6 Challenges in Operational Risk Modeling

The difficulties that make operational risk awkward are not incidental; they are structural, and honest practice acknowledges them rather than modeling past them.

Data scarcity and the unquantifiable tail. The largest operational losses are, by definition, the rarest, so the very events that dominate the capital number are the ones about which there is least data. No amount of machine learning manufactures tail observations that do not exist, and a genuinely unprecedented event, a first-of-its-kind cyberattack, a novel misconduct scheme, is closer to the Knightian uncertainty of Chapter 1 than to the estimable risk any model assumes.

Reporting and survivorship bias. Loss databases record what was noticed and reported; near-misses, silently absorbed small losses, and the losses of firms that did not survive are all systematically missing, biasing any fitted frequency and severity downward in ways no purely statistical correction can fully repair.

Alert fatigue. Every detection layer in this chapter produces false positives, and the base-rate reasoning of Chapter 15 applies in full: when true events are rare, even an accurate detector generates far more false alarms than genuine catches, and a surveillance system that overwhelms its investigators is, in practice, no surveillance at all.

Model-on-model risk. Using AI to manage operational risk adds models whose own failures are operational risks, so the reflexive controls described earlier in this chapter must extend to the very models built to detect operational failures, a regress that is manageable but never fully closed.

Gaming and reflexivity. As the Goodhart dynamic of Section 10.3.5 shows, indicators and controls that become targets can lose their meaning, so operational-risk models require ongoing revalidation against

Wulin.Suo@Queensu.ca

the possibility that the behavior they measure has adapted to their existence.

The pace of new failure modes. The event types in Table 10.1 were fixed years before generative AI, cloud concentration, or large-scale ransomware became material, and the pace at which technology creates genuinely new operational-risk channels now outruns the frameworks meant to catalog them. A framework that can only manage the risks it has already named is structurally behind, which is why the reflexive, continuously-monitored posture described earlier in this chapter matters more in operational risk than the one-time model sign-off that suffices elsewhere.

10.7 Key Takeaways

- Operational risk is the risk of loss from failed internal processes, people, systems, or external events, distinct from the market risk of Chapter 8 and the credit risk of Chapter 9, and structurally harder to model because its events are rare, heavy-tailed, heterogeneous, and under-reported.

- The four data elements are combined, not pooled: a Gamma-Poisson credibility blend weights internal history against external and scenario data through a single factor $Z = n/(n + \beta)$, and in the worked example it turned a thin internal estimate of 8.4 events a year into the $\lambda = 12$ the loss model uses, a bank relying on internal data alone understating its capital by about 15%.

- The classical loss distribution approach builds annual loss from a Poisson frequency and a lognormal severity by Monte Carlo; in the worked example, the 99.9% capital quantile was about seven times the expected loss, so operational-risk capital is overwhelmingly a tail problem.

- Operational-risk capital is dominated by the severity tail rather than the event frequency or the average loss, so modeling that tail well, with heavy-tailed and extreme-value methods, is the central actuarial difficulty and the place sparse data hurts most.

- Artificial intelligence contributes in four layers around the loss-modeling core, incident-text classification (Chapter 16), anomaly detection for near-misses (Chapter 17), conduct and rogue-trading surveillance (Chapters 15–16), and predictive early warning from key risk indicators (Chapter 6), not by replacing the actuarial loss model.

- Deploying AI is itself an operational activity: model risk, drift, and generative-AI failure modes such as hallucination and prompt injection are operational risks in their own right, and the monitoring tools of Chapter 18 are their controls.

- Because Basel retired internal-model operational-risk capital in favor of a size-based formula, AI's home in operational risk is management and detection, exactly where its strengths lie, rather than the regulatory capital calculation.

- Prevention beats detection: process mining, entitlement-graph analysis, and predictive process monitoring remove the conditions for a loss rather than shortening the time to notice one, and because permissions are inherited through nested roles, a segregation-of-duties conflict is a path through the access graph rather than an edge, invisible to a pairwise check.

- An AI system that *acts* rather than merely drafts is a different operational risk from one that generates text: it closes the detection window the four layers depend on, and holds conflicting entitlements by construction, so it needs the scoping, authorization, logging, and limit controls banks have applied to human traders since Barings.

- The AI layers must themselves be validated with the same tools as any other model, confusion matrices, precision-recall curves, and out-of-sample discrimination, because an unmonitored surveillance or early-warning model that silently stops working is not a control against operational risk but a fresh source of it.

Wulin.Suo@Queensu.ca

- Those validations are far stronger against false positives than false negatives: confirmed-misconduct labels exist only where a control already caught something, the same reject-inference structure Chapter 9 meets in credit, so a reported surveillance recall is an upper bound and a quiet alert queue is ambiguous rather than reassuring.

10.8 Suggested Exercises

1. Using the loss distribution approach with $\lambda = 8$ events per year and Lognormal($\mu = 11, \sigma = 1.2$) severities, simulate the annual loss distribution and estimate the expected loss and the 99.9% capital quantile. By what multiple does the capital exceed the expected loss, and how does that multiple change if you raise σ to 1.6?

2. Explain, in terms of the frequency and severity components, why two business lines with the same expected annual operational loss can require very different amounts of capital. Which component does raising the severity σ affect, and why does it move the 99.9% quantile more than the mean?

3. Extend the two-class Naive Bayes classifier of Section 10.3.2 to a third event type (for example, external fraud) with three new training incidents, and classify a new report of your choosing. Show the effect of Laplace smoothing on a word that appears in only one class.

4. Using the key-risk-indicator logistic regression of Section 10.3.5, explain why standardizing the indicators before fitting makes the coefficients comparable across indicators, and describe one way a business unit could game each of the three indicators without reducing its actual operational risk.

5. Explain why an anomaly detector with a precision of 0.85 but a recall of only 0.56, as in Table 10.4, can still be operationally valuable, and describe the concrete consequence, in analyst workload and in missed events, of lowering its threshold to raise recall to 0.90.

6. Using Section 10.2.3's credibility blend, recompute $\hat{\lambda}$ when the scenario workshop's view is held with strength $\beta = 8$ years rather than 3, holding the internal 42 events over 5 years and $\lambda_{\text{prior}} = 18$ fixed. Report the new credibility factor Z and blended frequency, re-run the loss simulation at that frequency, and explain why a supervisor might be uncomfortable letting a bank choose β for itself.

7. Section 10.2.3 showed that using internal loss data alone understates the 99.9% capital quantile by about 15% for this business line. Explain why the understatement is smaller in percentage terms than the corresponding gap in *expected* loss, referring to how frequency and severity each enter the compound sum, and say what this implies for a business line whose severity σ is larger.

8. Add a ninth user to Table 10.6's system who holds the `TRADE_ENTRY` role plus a new composite role containing `TRADE_CONFIRM`, and confirm that the direct check misses the resulting conflict while the path-based check finds it. Then describe one realistic way an entitlement graph accumulates such conflicts without anyone making an obviously wrong decision.

9. This chapter's discussion of validating and backtesting operational-risk models argues that a surveillance model's reported recall should be read as an upper bound. Construct a small numerical illustration: assume 10 genuine misconduct episodes occurred, the existing controls exposed 4 of them, and a new model flags 3 of those 4. State the recall the bank would report, the recall against all 10, and explain which of the two a model validator can actually observe.

10. **Challenge (real data).** Public enforcement actions are a rich, freely available source of operational-loss narratives. Download a sample of settlement or enforcement-action texts (for example, from the U.S. Securities and Exchange Commission's litigation releases or the Consumer Financial Protection Bureau's enforcement database), label each by Basel event type, and train a text classifier (TF-IDF plus logistic regression, or the embedding methods of Chapter 16) to predict the event type from the narrative. Report its confusion matrix, and discuss which event types are hardest to separate and why.

Wulin.Suo@Queensu.ca

10.9 Further Reading

- Hull, *Risk Management and Financial Institutions*. Includes a clear introductory treatment of operational risk and its regulatory capital, complementing the market- and credit-risk chapters cited in Chapters 8 and 9.
- Cruz, Peters, and Shevchenko, *Fundamental Aspects of Operational Risk and Insurance Analytics*. A thorough treatment of the loss distribution approach, frequency-severity modeling, and the extreme-value methods for the severity tail.
- McNeil, Frey, and Embrechts, *Quantitative Risk Management: Concepts, Techniques and Tools*. A standard graduate reference spanning market, credit, and operational risk, and the extreme-value methods for heavy-tailed losses.
- Board of Governors of the Federal Reserve System, *Supervisory Guidance on Model Risk Management* (SR 11-7). The foundational supervisory statement treating model risk as an operational-risk discipline, underlying this chapter's discussions of the reflexive turn and of regulation.
- Shevchenko, *Modelling Operational Risk Using Bayesian Inference*. A book-length treatment of the problem Section 10.2.3 works in miniature, combining internal loss data, external data, and expert scenario judgment into a single frequency and severity estimate.
- van der Aalst, *Process Mining: Data Science in Action*. The standard reference for the event-log reconstruction and conformance checking of Section 10.4.1, and for the predictive extensions of Section 10.4.3.

Wulin.Suo@Queensu.ca

Chapter 11

Insurance and Actuarial Modeling

11.1 Introduction

In August 1992, Hurricane Andrew crossed southern Florida and destroyed, alongside tens of thousands of homes, the insurance industry's confidence in its own loss estimates. Insurers had priced Florida windstorm coverage from a few decades of loss experience that happened to contain no direct hit on a major metropolitan area; Andrew produced insured losses several times larger than the industry's models had contemplated, and roughly a dozen insurers were rendered insolvent by a single storm. The failure was not one of underwriting discipline but of method: historical claims experience, the industry's traditional basis for pricing, is structurally silent about events larger than any it happens to contain. Out of that failure grew the modern catastrophe-modeling industry, simulation-based, physics-informed, and data-hungry, and with it the broader lesson this chapter develops: insurance is, more than any other business in this book, a business of estimating loss distributions, and every advance in how loss distributions are estimated eventually becomes an advance in how insurance is priced, reserved, and supervised.

Insurance deserves a chapter of its own in a book about artificial intelligence in finance for two reasons. First, it is enormous, by assets, by premium volume, and by employment of the quantitatively-trained people this book addresses, and its analytical core, actuarial science, is the oldest data science in finance: actuaries were fitting frequency and severity distributions to loss data a century before the term "machine learning" existed. Second, its structure makes the AI questions unusually crisp. An insurer prices a contract before knowing what it will cost, observes rich behavioral data about its customers, adjudicates millions of claims a year, and does all of this inside one of the most consumer-protective regulatory perimeters in finance, so every theme this book has developed, predictive modeling, text and image data, fraud detection, explainability, and fairness, arrives here simultaneously and with unusually high stakes.

The chapter's central claim, the same asymmetry Chapter 10 found in operational risk, is worth stating up front. Machine learning does not replace the actuarial core; it feeds and sharpens it at every point where the data is *granular* or *unstructured*: driver-level sensor streams instead of rating-cell averages, claim-level features instead of aggregate triangles, document and image inputs instead of questionnaires, and interactions no multiplicative rating plan anticipates. The classical structure, frequency times severity, credibility, the development triangle, the filed rating plan, remains the layer that regulators audit and balance sheets are built on. Each section below therefore pairs the two explicitly: the classical calculation first, then a worked example of the machine-learning layer that now sits underneath it, each small enough to check by hand.

The chapter proceeds from economics to methods to frontiers, and Figure 11.1 maps where AI enters along the way. It first establishes what makes insurance structurally different and how an insurer actually makes money, then develops the classical quantitative core, frequency-severity pricing, the one-way-rating failure that motivated generalized linear models, credibility theory, and chain-ladder reserving, each with a worked example a reader can check by hand. It then works through where machine learning genuinely changes each function, pricing, telematics, claims, reserving, underwriting, before widening to the balance-sheet topics, life and health insurance, asset-liability management, catastrophe risk and parametric transfer, and solvency capital, that connect insurance to the valuation, portfolio, and risk chapters earlier in this book. It closes where insurance regulation now concentrates: fairness, price optimization, and proxy discrimination

Wulin.Suo@Queensu.ca

in AI-driven pricing.

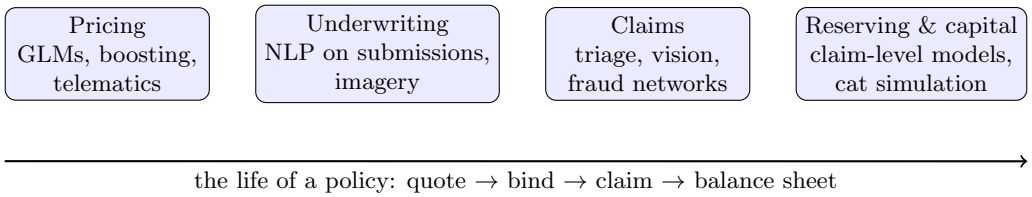


Figure 11.1: AI across the insurance value chain. Every stage feeds a filed rate or a regulated capital number, which is what makes governance structural here.

11.2 The Economics of Insurance

An insurer sells a promise and invests the proceeds, which makes its economics unlike any other financial firm's.

11.2.1 What Makes Insurance Different

Insurance inverts the production cycle of almost every other business: the price is collected first and the cost of goods sold, the claims, is discovered afterward, sometimes decades afterward. Everything distinctive about the industry follows from this inversion. Profitability cannot be read off current revenue; it must be *estimated*, by forecasting the ultimate cost of claims already incurred (the reserving problem of Section 11.4.1) and of contracts still in force. The product is a promise, so the industry is regulated primarily for solvency, an insurer must demonstrably be able to pay claims years away, and secondarily for fairness in pricing, since coverage is often practically or legally mandatory.

The economic engine is risk pooling, and it is worth seeing the engine's mathematics once, because a single calculation explains why the industry exists. Suppose each of n independent policies produces an annual loss with mean $\mu = \$720$ and standard deviation $\sigma = \$3{,}000$, a violently uncertain individual prospect, with a coefficient of variation of $3{,}000/720 \approx 417\%$. The pooled loss across $n = 10{,}000$ such policies has mean $n\mu = \$7.2$ million but standard deviation only $\sigma\sqrt{n} = \$300{,}000$, a coefficient of variation of 4.17%: aggregation has shrunk relative uncertainty by a factor of $\sqrt{n} = 100$. A loss more than 10% above expectation is a $z = 2.4$ event, roughly a 0.8% probability under a normal approximation. Individually unpredictable losses have become collectively predictable, and *that predictability, not risk-bearing appetite, is the product*: the insurer sells the law of large numbers, retail. The calculation also displays its own failure mode, which Section 11.5.1 takes up: everything above assumed independence, and a hurricane, correlating a hundred thousand claims into one event, deletes the $\sqrt{n}$.

Two classic frictions limit the engine. Adverse selection is the tendency of higher-risk customers to seek more coverage, which the insurer combats with underwriting and risk-based pricing; moral hazard is the tendency of insured parties to take more risk once covered, combated with deductibles and coverage limits. Both frictions are, at bottom, information problems, which is precisely why an industry built on them has always been an early adopter of any technology that reduces information asymmetry, from the mortality table to the telematics device of Section 11.3.4.

The reader will recognize the analytical machinery immediately. An insurance portfolio's annual loss is a compound frequency-severity sum, exactly the loss distribution approach Chapter 10 developed for operational risk, and for the same reason: both fields face losses that arrive as a random number of events of random size. The historical direction of travel, worth appreciating, is that operational risk borrowed this machinery *from* actuarial science, not the other way around.

Wulin.Suo@Queensu.ca

11.2.2 How an Insurer Makes Money: Float and the Combined Ratio

Before pricing a single policy, it pays to understand the business model the prices feed. An insurer's operating result has two engines, illustrated in Figure 11.2. The underwriting engine is summarized by the combined ratio: losses plus expenses, divided by premiums. A combined ratio below 100% means the insurance operation is profitable in its own right. The investment engine runs on float: because premiums arrive before claims are paid, the insurer continuously holds a large pool of policyholder money, the loss reserves of Section 11.4.1 plus unearned premiums, which it invests for its own account.

A worked example shows how the engines combine. An insurer writes $100 million of premium, incurs $65 million of losses (a 65% loss ratio) and $30 million of expenses (a 30% expense ratio), for a combined ratio of 95% and an underwriting profit of $5 million. It also holds $150 million of float invested at a 4% yield, earning $6 million, so total pre-tax operating income is $11 million, a 22% return on its $50 million of supporting capital. Now rerun the year with a 103% combined ratio: underwriting loses $3 million, yet the insurer still earns $3 million overall, because the $3 million underwriting loss is simply the *cost* of holding $6 million worth of investable float, cheaper than any other funding the insurer could raise. This arithmetic, made famous by Berkshire Hathaway's shareholder letters, explains a great deal of observed industry behavior: why insurers will write at small underwriting losses in competitive markets, why reserving accuracy (which determines how much float there really is, and for how long) is a first-order economic question rather than an accounting formality, and why the asset-liability topics of Section 11.4.4 belong in the same chapter as pricing.

It also locates, before any model appears, exactly where machine learning can move the profit needle, because the combined ratio has only two components to move. Every AI application in this chapter attacks one of them: better risk selection and pricing (Sections 11.3.3–11.3.4) and earlier fraud detection (Section 11.3.5) lower the *loss* ratio, while automated claims handling, straight-through processing, and NLP-driven underwriting (Sections 11.3.5 and 11.5.3) lower the *expense* ratio, and on the worked book each point of combined ratio is worth $1 million of premium, pre-tax, every year. That is the business case that funds the industry's AI investment, and it disciplines the evaluation standard used throughout this chapter: an insurance model is judged by the ratio points it moves, not by its accuracy in the abstract.

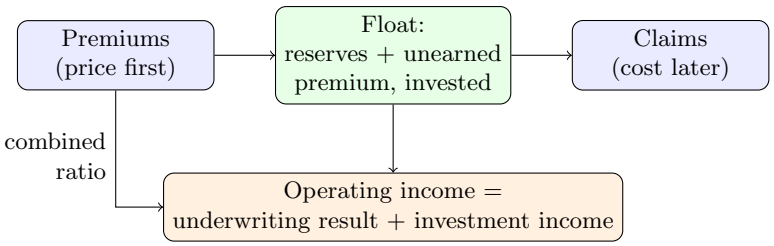


Figure 11.2: The inverted production cycle: premiums arrive before claims are paid, and the pool of policyholder money in between, the float, is invested for the insurer's account. Underwriting profitability (the combined ratio) and investment income on float are the business's two engines.

11.2.3 The Classical Pricing Baseline: Frequency Times Severity

Suppose an auto insurer must set next year's premium for a policyholder it has never observed have an accident. The classical answer prices the *expected* loss and then loads it for expenses and profit. The pure premium is

$$\text{pure premium} \;=\; \mathbb{E}[N] \times \mathbb{E}[X],$$

expected claim frequency (N) times expected claim severity (X), and the art of ratemaking is estimating how each component varies with observable characteristics of the risk.

A worked example fixes the mechanics. Suppose the book-wide base rates are a claim frequency of 0.08 claims per policy-year and a mean severity of $9,000, giving a base pure premium of $0.08 \times 9{,}000 = \$720$.

Wulin.Suo@Queensu.ca

Rating factors enter multiplicatively: suppose young drivers exhibit 1.8 times the base frequency, urban territories 1.25 times the base frequency, and urban claims run 1.10 times the base severity (denser traffic produces more collisions; urban repair and injury costs run higher). A young urban driver's expected frequency is then $0.08 \times 1.8 \times 1.25 = 0.18$ and expected severity $9{,}000 \times 1.10 = \$9{,}900$, for a pure premium of $0.18 \times 9{,}900 = \$1{,}782$, about 2.5 times the base risk. Loading for a 25% expense ratio and a 5% profit provision, the gross premium solves premium $\times (1 - 0.25 - 0.05) =$ pure premium, giving $1{,}782/0.70 \approx \$2{,}545.71$.

This arithmetic is the skeleton; the instrument that estimates the relativities filling it in is the generalized linear model of the next section, and it is important to be clear about vocabulary before the machine-learning comparison arrives: when the insurance industry says "traditional pricing," it means *GLM-based* pricing. The GLM is not a stepping stone on the way to modern methods; it has been the established standard for decades, and it is the incumbent that machine learning must beat.

11.3 Pricing and Claims

This part covers how a premium is actually set, from the generalized linear model through credibility to telematics, and what happens when a claim arrives.

11.3.1 GLM Ratemaking: The Traditional Approach

Where do relativities like the 1.8 and 1.25 above come from? The tempting answer, compute each factor's relativity from its own marginal experience, one factor at a time, contains a trap important enough to work through numerically, because escaping the trap is precisely what made the generalized linear model the industry's standard.

Consider a portfolio with two rating factors, age (young/experienced) and territory (urban/rural), with exposures and claim counts by cell:

Cell	Exposure	Claims	Frequency
Young, urban	200	54	0.270
Young, rural	50	9	0.180
Experienced, urban	300	45	0.150
Experienced, rural	450	45	0.100

Table 11.1: Exposures (policy-years), claim counts, and claim frequencies by rating cell

The table was built with an exactly multiplicative structure: a base frequency of 0.100, a young relativity of 1.8, and an urban relativity of 1.5 (check: $0.100 \times 1.8 \times 1.5 = 0.270$). Now compute the relativities the one-way approach recovers. Young drivers overall: $(54 + 9)/(200 + 50) = 0.252$; experienced overall: $(45 + 45)/(300 + 450) = 0.120$; one-way age relativity $0.252/0.120 = 2.10$, not 1.8. Urban overall: $(54 + 45)/(200 + 300) = 0.198$; rural: $(9 + 45)/(50 + 450) = 0.108$; one-way territory relativity 1.833, not 1.5. Both are inflated, and for the same reason: young drivers in this portfolio cluster in urban territory (200 of 250 exposures), so the age marginal absorbs part of the territory effect and vice versa, double-counting their overlap. A rating plan built from the one-way factors would charge the young urban cell $2.10 \times 1.833 = 3.85$ times base, against a true relativity of 2.70, a 43% overcharge, and would symmetrically undercharge the experienced rural cell, with the mispricings then feeding adverse selection as overcharged customers defect to better-priced competitors.

The remedy is simultaneous estimation, and that is precisely what the generalized linear model (GLM) does: a Poisson GLM with a log link fit to claim counts (with exposure as an offset), and a Gamma GLM fit to severities, estimate all relativities *jointly*, each factor's effect measured holding the others fixed, recovering the true 1.8 and 1.5 from Table 11.1 exactly. GLMs extended the ordinary regression of Chapter 6 to the skewed, non-negative distributions insurance data actually exhibit, and their adoption is itself a piece of history worth knowing: spreading from British motor insurance in the 1980s and 1990s, GLM ratemaking

Wulin.Suo@Queensu.ca

displaced the earlier one-way and minimum-bias methods to become the industry's standard classification-ratemaking approach worldwide, the method the Casualty Actuarial Society's own rating monograph presents as the established baseline, so that the phrase "traditional pricing" in today's industry debates refers to a method that was, in its time, the data-science modernization. The multiplicative-relativity output is also *filed*: in most U.S. states an insurer must submit its rating plan to a regulator, justify each factor, and use what it filed, a constraint with deep consequences for what follows.

11.3.2 Credibility: The Actuary's Bayesian Shrinkage

Between the book-wide rate and the individual risk sits a question the GLM alone does not answer: how much should one policyholder's *own* experience move their price? A commercial fleet with three years of history and one claim exhibits an observed frequency of $1/3 \approx 0.333$ against a class rate of 0.100; charging either number is plainly wrong, the first because three years of one fleet is noise, the second because it ignores real information. Credibility theory, actuarial science's oldest continuously-used tool, prescribes the blend

$$\hat{f} \;=\; Z \times \bar{f}_{\text{own}} + (1 - Z) \times f_{\text{class}}, \qquad Z = \frac{n}{n + k},$$

where n is the volume of the risk's own experience and k measures how noisy individual experience is relative to how much risks genuinely differ (formally, the ratio of expected process variance to the variance of true means across risks, the Bühlmann formulation). With $n = 3$ years and $k = 12$, the credibility weight is $Z = 3/15 = 0.20$ and the blended frequency $0.20 \times 0.333 + 0.80 \times 0.100 \approx 0.147$: the fleet's bad year moves its rate meaningfully but not catastrophically, and each additional year of clean experience will pull it back.

The reader who has followed this book will recognize the formula three times over. It is Chapter 1's Bayesian updating, the class rate as prior, own experience as likelihood, with Z the posterior weight on the data; it is the shrinkage of Chapter 6's ridge regularization, pulling noisy individual estimates toward a grand mean by an amount that declines as evidence accumulates; and it is structurally the partial pooling that modern hierarchical models perform. Actuaries derived it in the 1910s–1960s because rate regulation forced them to price small risks honestly; machine learning rederived it as regularization half a century later. Credibility is this chapter's cleanest evidence for a claim made in the introduction: actuarial science is not a field awaiting rescue by data science; it is a parallel, older branch of the same subject.

11.3.3 From GLMs to Machine Learning in Pricing

The machine-learning question in pricing is therefore precisely posed: the incumbent is not a naive method but the GLM, a decades-refined statistical model with a regulatory apparatus built around it, and "machine learning versus traditional pricing" means, concretely, boosted trees versus the GLM. Everything Chapter 6 established about flexible models applies, with one structural twist. Gradient-boosted trees routinely beat a GLM's predictive accuracy on large personal-lines datasets, for the familiar reason: real loss processes contain interactions (age with vehicle power, territory with annual mileage) that a multiplicative model captures only if an actuary thinks to specify them, while a tree ensemble finds them automatically, the same lesson as Section 11.3.1's one-way trap, one level up.

A Worked Example: The Interaction a GLM Cannot Price

The claim deserves numbers, and Table 11.1's portfolio provides them with one realistic change. Suppose inexperience is genuinely more dangerous in dense traffic, so the young-urban cell's true frequency is 0.32 (64 claims on 200 exposures) rather than the multiplicative 0.27, while the other three cells keep their multiplicative values. No multiplicative model can fit these four cells: multiplicativity forces the young-urban frequency to equal base times young times urban, and the data now violate that. Refitting the Poisson GLM to the modified portfolio shows how the failure actually manifests, which is subtler than simply mispricing the interaction cell. The fitted parameters come back as a base frequency of 0.0977 with relativities of 2.05 (young) and 1.57 (urban): the interaction has *leaked into the marginals*, inflating both, because the young-urban cell dominates the exposure behind each. The fitted model then prices the young-urban cell at 0.315 against a true 0.320, only 1.6% light, but overprices young-rural at 0.201 against a true 0.180, an

Wulin.Suo@Queensu.ca

11.4% overcharge on the cell least responsible for the signal, with the experienced cells absorbing smaller distortions. The mispricing lands on the *other* cells, exactly where nobody is looking.

A depth-two decision tree, Chapter 6's simplest nonlinear model, prices all four cells exactly: its first split on age and second on territory partition the portfolio into the four cells themselves, each predicted at its own observed frequency, and a gradient-boosted ensemble reaches the same fit within a few trees. This is the entire GLM-versus-machine-learning pricing debate in a 2×2 table: the flexible model wins not by magic but by declining to impose a structure the data contradict. On the standard academic benchmark for this comparison, a French motor liability dataset of nearly seven hundred thousand policies (this chapter's closing challenge exercise), the accuracy improvement is consistent and material, and the same result reproduces across insurers' internal books.

Where Machine Learning Improves on the Traditional Approach, and Where It Does Not

The interaction is one instance of a general pattern, and it is worth stating precisely where the improvement beyond the traditional approach comes from, because each channel corresponds to a specific limitation of the traditional toolkit, the GLM in pricing, and its counterparts elsewhere in the chapter, rather than to generic "more power." First, *interaction discovery at scale*: an actuary can hand-specify the young-urban term once told about it, but cannot hand-specify the thousands of candidate interactions among dozens of rating variables that a boosted ensemble searches automatically; the ensemble's advantage grows with the width of the rating plan. Second, *nonlinear continuous effects without hand-chosen bins*: a GLM represents driver age through bands or polynomial terms the actuary selects in advance, and the true age curve, steeply falling through the twenties, flat in mid-life, rising again past seventy, is captured only as well as those choices allow, while a tree ensemble learns the curve's shape, including its non-monotonicity, directly from data. Third, *high-cardinality categorical variables*: vehicle model and postcode take thousands of levels that a GLM must manually group (or shrink with Section 11.3.2's credibility machinery, applied level by level), while tree ensembles and learned embeddings, Chapter 16's representation idea applied to categories, handle them natively. Fourth, and most consequentially, *features the GLM could never build*: a GLM can only consume a tabular feature someone constructed, while the deep-learning methods of Chapters 16 and 17 construct the features themselves, the braking signature inside a telematics stream, the roof condition inside an aerial image, the hazard buried in a broker narrative, which is why the largest practical gains have come less from swapping the GLM out of the filed plan than from feeding it richer inputs it could not have created.

The same four channels recur, in different clothes, wherever this chapter meets a traditional method: claim-level reserving improves on the aggregate triangle by using features the triangle discards (Section 11.4.1); dynamic-lapse models improve on fixed liability durations by learning behavior the bond arithmetic assumes away (Section 11.4.4); and cause-enriched mortality models improve on a single index for the same reason. In each case the traditional method is not wrong but *coarse*: it summarizes at a level of aggregation chosen for an era of scarce data and hand computation, and machine learning's contribution is to price, reserve, and hedge at the granularity the data now supports.

Honesty requires the other column. Where the book of business is small, the traditional method's low variance and the credibility machinery win, exactly Chapter 6's bias-variance logic; where the rating plan is narrow and well-understood, there are few interactions left to find; and at the regulatory interface, the GLM's auditability is not a limitation but the point. The mature practice is therefore not replacement but a loop: fit the boosted challenger, let it reveal which interactions and curve shapes the filed GLM is missing, and *distill* those findings back into added GLM terms, so that the filed model captures most of the challenger's lift while remaining a document a regulator can read, with the residual gap between the two serving as a running measure of how much accuracy the filing constraint currently costs.

The structural twist is that the filed-rating-plan constraint converts Chapter 18's explainability discussion from good practice into a binding production requirement. A rate filing must state what the model charges and why, an adverse underwriting decision must often be explained to the applicant under the same family of laws as Chapter 18's adverse-action notices, and several U.S. states now require insurers to demonstrate that models built on nontraditional data do not proxy for prohibited characteristics (Section 11.5.4). The practical industry pattern is therefore a hybrid: machine learning models are used freely where they do not directly set the filed rate, in underwriting triage, retention and demand modeling, claims analytics, and as a challenger benchmark that identifies which interactions the filed GLM should add, while the filed rating

Wulin.Suo@Queensu.ca

plan itself evolves more slowly, with the GLM (or a constrained, explainable boosted model) remaining the regulatory interface. It is this book's recurring accuracy-versus-accountability trade-off, institutionalized more explicitly than anywhere else in finance.

11.3.4 Telematics and Usage-Based Insurance

Traditional rating variables, age, territory, vehicle, are all *proxies* for how a person actually drives. Telematics replaces the proxy with the behavior itself: a device or phone app records mileage, speed, time of day, and the harsh-braking and cornering events that predict collisions, and usage-based insurance prices on that record. This is the cleanest example in this book of AI shrinking an information asymmetry at the individual level: the insurer no longer infers risk from group membership but observes each driver's own revealed behavior.

Continuing the worked example, and making the model explicit rather than waving at it: the insurer's telematics layer is itself a GLM on behavioral features, a log-link frequency model in which the fitted coefficient on harsh-braking events is +0.163 per event per hundred miles relative to the rating cell's average (more harsh braking, more claims). Six months of data show the young urban driver braking harshly two fewer times per hundred miles than their cell's average, so the telematics multiplier is $e^{0.163\times(-2)} = e^{-0.326} \approx 0.722$, converting the age-based relativity of 1.8 into a personal one of $1.8 \times 0.722 \approx 1.30$. The pipeline is worth seeing whole: raw sensor stream → engineered behavioral features (Chapter 17's feature-extraction discipline applied to accelerometer data) → a GLM coefficient → a rating relativity a regulator can read, machine learning at the granular layer feeding the classical structure at the filed layer, the chapter's central asymmetry in one line of arithmetic. The revised expected frequency is $0.08 \times 1.30 \times 1.25 = 0.13$, the pure premium falls to $0.13 \times 9{,}900 = \$1{,}287$, and the gross premium to $1{,}287/0.70 \approx \$1{,}838.57$, a saving of about \$707, or 27.8%, earned by demonstrated behavior rather than demographic reclassification. The same calculation run in reverse explains the business model's controversy: a driver whose data reads badly faces a surcharge no traditional variable would have imposed, and the data itself, continuous location and motion records, raises privacy questions the industry's regulators are still working through. There is also a selection subtlety worth noticing: enrollment is voluntary, and drivers who expect their data to read well are the ones who opt in, so the observed telematics book is not a random sample of the portfolio, a live example of the selection effects Chapter 1 warned inhabit every observational dataset.

11.3.5 Claims: Triage, Automation, and Fraud

Claims is where an insurer's promises are kept, and it is operationally the most AI-saturated function in the industry. Three applications dominate. *Severity triage* predicts, from the first notice of loss, which claims will become expensive, the injury claim that will develop into litigation, the water leak that will become a mold claim, so that experienced adjusters are assigned early where they change outcomes; this is a regression problem in Chapter 6's sense, with the twist that the target is long-tailed and partially censored (open claims' ultimate costs are not yet known, the same censoring structure as Chapter 15's survival analysis).

A minimal triage model shows the mechanics, in the spirit of Chapter 10's key-risk-indicator example. Suppose a logistic model scores the probability that a new auto claim ultimately exceeds \$50,000, from three first-notice features: an attorney is already involved (+1.6 on the log-odds), an injury is reported (+1.3), and the claim was reported more than a week after the accident (+0.8), against an intercept of −2.2. Here σ is Chapter 6's sigmoid, $\sigma(z) = 1/(1+e^{-z})$, not the standard deviation the same letter denoted in the pooling calculation earlier in this chapter; the two meanings are both standard, and which one is meant is always fixed by whether the symbol is applied to an argument. A claim arriving with attorney and injury flags but promptly reported scores $\sigma(-2.2 + 1.6 + 1.3) = \sigma(0.7) \approx 0.668$; a promptly-adjusted no-injury claim reported late scores $\sigma(-2.2 + 0.8) = \sigma(-1.4) \approx 0.198$. The first is routed to a senior adjuster on day one, the second to straight-through processing, and the model's value is measured not by its accuracy in the abstract but by the outcome difference early senior attention makes on the claims it flags, the same decision-focused evaluation standard this book has applied since Chapter 1. *Straight-through processing* pays simple, well-documented claims automatically, using computer vision on damage photos to estimate repair costs and NLP on claim descriptions to verify coverage, with the operational-risk lesson of Chapter 10 attached: an automated adjudication pipeline is itself a system whose failures (overpayment at scale, systematic wrongful denial) are operational events. *Claims fraud detection* is Chapter 15's machinery

Wulin.Suo@Queensu.ca

transplanted: rare positives, adversarial adaptation, network structure (staged-accident rings share repair shops, clinics, and phone numbers exactly as laundering networks share addresses), and the same precision-recall economics, since every investigated honest claim delays a legitimate payment to a customer who has just suffered a loss.

11.4 Reserving, Life, Health, and Assets

Beyond pricing, an insurer must estimate what it already owes, and manage the assets held against it.

11.4.1 Reserving: The Chain Ladder and Its Successors

An insurer's largest liability is typically not claims it knows about but claims it must *estimate*: losses incurred but not yet reported or not yet fully paid. Reserving estimates this liability, and the classical tool, the chain ladder, is worth working once by hand because it is the industry's balance-sheet backbone and a genuinely elegant use of minimal data.

Cumulative paid claims (in \$000s) are arranged by accident year and development age:

Accident year	12 months	24 months	36 months
Year 1	400	600	660
Year 2	450	675	?
Year 3	500	?	?

Table 11.2: A cumulative paid-claims development triangle (\$000s)

The chain ladder assumes each accident year develops proportionally the way its predecessors did. The 12-to-24-month development factor, estimated from all years observed at both ages, is $(600+675)/(400+450) = 1{,}275/850 = 1.50$; the 24-to-36-month factor is $660/600 = 1.10$. Year 2's ultimate loss is then projected as $675 \times 1.10 = 742.5$ and Year 3's as $500 \times 1.50 \times 1.10 = 825$, so the required reserve, ultimate losses minus amounts already paid, is $(742.5 - 675) + (825 - 500) = 67.5 + 325 = \$392.5$ thousand.

Machine learning enters reserving along two paths. Individual-claim reserving replaces the aggregated triangle with claim-level prediction, typically a gradient-boosted or neural regression of each open claim's remaining cost on the same first-notice features as the triage model above (injury type, attorney involvement, jurisdiction) plus its paid-to-date trajectory, which both improves accuracy and, more importantly, explains *why* reserves move, information a triangle mathematically cannot contain; neural extensions of the chain ladder itself, embedding the development-factor structure inside a network that lets the factors vary with claim characteristics, are the subject of an active actuarial literature (see Wüthrich and Merz in Further Reading). And the chain ladder's own assumptions, that development patterns are stable across accident years, are exactly what breaks when claims inflation shifts or a new claims process changes settlement speed; the monitoring mindset of Chapter 18, watching for the distribution drift that silently invalidates a fitted model, applies to reserving models with an auditor and a regulator as the interested parties. The stakes are the float arithmetic of Section 11.2.2: reserves *are* the float, so systematic under-reserving manufactures phantom profits today at the cost of real insolvency risk later, which is why reserve opinions are among the few statistical estimates in finance that a named, professionally-liable individual must sign.

11.4.2 Life Insurance, Mortality, and Longevity

Everything so far has been property-casualty insurance, short contracts, uncertain frequency and severity. Life insurance inverts the difficulty: the event is certain, only its timing is not, and contracts run for decades, which moves the modeling weight from loss distributions to mortality curves and discounting, the machinery of Chapter 5 applied to human lifetimes.

The atomic calculation is worth seeing once. A one-year term policy pays \$500,000 if a policyholder with annual death probability $q = 0.002$ dies during the year. Its expected present value, the net premium, is

$$q \times B \times v \;=\; 0.002 \times 500{,}000 \times \frac{1}{1.04} \;\approx\; \$961.54,$$

Wulin.Suo@Queensu.ca

mortality probability times benefit times a one-year discount factor at 4%. Every life product, term, whole life, annuities, is an expected-present-value sum of exactly such terms across future years, with level-premium products deliberately overcharging early years to prefund the later ones, creating the enormous, long-duration liabilities that drive Section 11.4.4.

The modeling frontier is the mortality curve itself. Mortality improves over time, unevenly by age and cohort, and the standard stochastic model, the Lee-Carter model, is a pleasing reunion of two of this book's tools: it decomposes the age-by-year mortality surface into age profiles and a single time-varying mortality index, a factor extraction in the spirit of Chapter 7's PCA term-structure example, and then forecasts that index as a random walk with drift, Chapter 7's simplest time-series model, applied to human longevity. The symmetric risk is longevity risk: an annuity or pension book loses when people live *longer* than projected, a slow-moving, undiversifiable trend risk that no pooling can remove, transferred in practice through longevity swaps between pension funds and reinsurers. AI's contributions run along both fronts: mortality models enriched with cause-of-death, socioeconomic, and medical-advance information, and accelerated underwriting, replacing the traditional medical exam with predictive models on application data, prescription histories, and, controversially, wearable-device data, moving a weeks-long process toward minutes for low-risk applicants while concentrating the fairness questions of Section 11.5.4 on whoever the model routes into full underwriting instead.

11.4.3 Health Insurance and Risk Adjustment

Health insurance runs on different economics from everything above: contracts reprice annually, so the long-tail reserving problem largely disappears, and the dominant friction is not catastrophe but selection, since individuals know far more about their expected medical costs than any application reveals. Modern health-insurance markets manage this with a mechanism that deserves a quantitative reader's attention because it is one of the largest predictive-modeling systems in the world with money attached: risk adjustment. Regulators pay (or transfer between) insurers according to a model-predicted cost score for each enrollee, so that an insurer attracting sicker members is compensated rather than bankrupted, deliberately removing the incentive to select healthy customers.

The mechanics are a regression with a budget attached, and the risk score is worth computing once, because it *is* a predictive model, an additive regression of next-year cost on demographics and diagnosis indicators, whose fitted coefficients are published as score weights. In the U.S. Medicare Advantage program's HCC framework, a 76-year-old female enrollee might carry a demographic base of 0.45, plus 0.30 for diabetes with complications, 0.33 for heart failure, and 0.22 for COPD: a risk score of 1.30, which against a baseline payment of $10,000 brings the insurer $13,000. The incentive problem follows immediately, and it is Goodhart's law, Chapter 10's recurring warning, at industrial scale: the insurer controls the diagnostic coding that feeds the model, so "upcoding," documenting members as sicker than clinically warranted, converts directly into revenue. Adding one unsupported vascular-disease code at 0.13 lifts the score to 1.43, worth an extra $1,300 per member per year, or $1 billion a year across a million-member book for every tenth of a point, and both academic estimates and government audits have found aggregate coding-intensity effects of roughly this magnitude, making risk-score integrity one of the largest financial-modeling enforcement areas in the United States.

The AI story is symmetrical, and both sides can be made concrete. On the revenue side, insurers deploy NLP on clinical charts, Chapter 16's machinery, to find *legitimately supportable* diagnoses the treating physician documented but never coded, a use that is lawful and, from the model's perspective, accuracy-improving. On the audit side, the simplest anomaly test is already decisive at scale: if an insurer codes vascular disease in 9% of 10,000 enrollees against a clinically-expected regional prevalence of 6%, the excess is $z = (0.09 - 0.06)/\sqrt{0.06 \times 0.94/10{,}000} \approx 12.6$ standard errors, a coding pattern that no plausible casemix difference explains and that flags the book for chart-level review, with Chapter 15's full anomaly-detection machinery taking over from there. The equilibrium is an adversarial modeling contest with a public budget as the stake, and the honest caveat belongs alongside it: a large z proves the coding is anomalous, not that it is fraudulent, and distinguishing genuine casemix from manufactured casemix is the human-review step the statistics only prioritize.

Wulin.Suo@Queensu.ca

11.4.4 The Investment Side: Asset-Liability Management

An insurer's balance sheet is a leveraged bond portfolio wrapped around a promise, and managing it is a direct application of Chapter 5's duration machinery with one twist: both sides of the balance sheet move with interest rates. Consider a life insurer with \$110 million of assets of duration 5 against \$100 million of liabilities of duration 12 (long-duration promises are the nature of the product), for a surplus of \$10 million. A one-percentage-point *rise* in rates moves assets by $-110 \times 5 \times 1\% = -\$5.5$ million but liabilities by $-100 \times 12 \times 1\% = -\$12$ million, so surplus *rises* to \$16.5 million; a one-point *fall* symmetrically compresses surplus to \$3.5 million. The sign is the point: with liabilities longer than assets, the classic life-insurance posture, it is *falling* rates that quietly impair the industry, which is exactly what a decade of near-zero rates did to life insurers and pension funds worldwide, and why duration-matching, immunization, and the long-dated hedging that Chapter 5's swap and duration tools support are a core insurance discipline rather than an optimization nicety.

The machine-learning layer enters through a fact the calculation above quietly assumed away: a liability's duration is not bond mathematics, because the "bond" can walk. Policyholders hold options, to lapse a policy, surrender it for cash, or pour new money into a guarantee, and they exercise them in response to rates: when market rates rise well above the credited rate, surrender becomes attractive; when guarantees are above market, policyholders cling to them. Insurers model this with dynamic lapse functions fitted to policyholder-level experience data, a Chapter 6 supervised-learning problem in which the features are the rate gap, policy age, tax penalties, and distribution channel, and the fitted behavior feeds directly back into the liability's *effective* duration. Rerunning the worked example with behavior in it: if a one-point rate rise accelerates lapses enough to shorten effective liability duration from 12 to 9, surplus rises only to $110 - 5.5 - (100 - 9) = \$13.5$ million rather than \$16.5 million; if a one-point fall slows lapses and extends duration to 14, surplus compresses to $110 + 5.5 - (100 + 14) = \$1.5$ million rather than \$3.5 million. Behavior has clipped the good outcome *and* deepened the bad one, the signature of a sold option, Chapter 5's negative convexity with a human exercising it, and the quality of the behavioral model, not the bond arithmetic, is what determines whether the hedge book is sized correctly. The 2022–2023 rate spike gave these models their first genuine out-of-sample test in a generation, the regime-shift risk Chapter 7 warned that models fitted on a calm decade must face.

Two further machine-learning deployments round out the ALM picture. Valuing blocks with embedded guarantees (variable annuities above all) requires nested simulation, economic scenarios outside, valuation runs inside each scenario, whose cost grows multiplicatively; insurers increasingly train neural-network surrogates (Chapter 6's function approximators, in the least-squares Monte Carlo tradition) to replace the inner valuation loop, making hedging programs computable overnight rather than over weeks, with the surrogate's approximation error itself a model risk to be validated per Chapter 10. And the portfolio problem that remains, investing float and surplus against regulatory constraints on asset risk, is Chapter 12's mean-variance machinery under a solvency constraint, which is why insurers became early, large buyers of the private credit assets Chapter 3 described: long, illiquid, yield-bearing assets are a natural match for long, illiquid, *behaviorally-modeled* liabilities.

11.5 Capital, Catastrophe, and Fairness

The tail is where insurance is decided; this part covers catastrophe capital, solvency regulation, and the fairness questions new data raises.

11.5.1 Catastrophe Risk, Reinsurance, and Alternative Capital

Pooling fails exactly where insurance is needed most: catastrophe losses are not independent across policyholders, a hurricane correlates a hundred thousand claims into one event, so Section 11.2.1's $\sqrt{n}$ arithmetic collapses and the tail must be either modeled and capitalized or transferred. Transfer happens through reinsurance, insurance for insurers, typically written in layers stacked into a tower, as in Figure 11.3. A worked illustration: a reinsurer sells a \$100 million layer attaching at \$50 million (it pays losses between \$50M and \$150M from a single event). If catastrophe models put the annual probability of an event reaching the layer at 0.04 and the expected loss within the layer, given such an event, at \$60 million, the layer's expected annual

Wulin.Suo@Queensu.ca

loss is $0.04 \times 60 = \$2.4$ million, a "rate on line" of 2.4% of the limit, before the substantial risk load such correlated, capital-intensive exposure commands. The same exposure increasingly reaches capital markets directly through catastrophe bonds, whose investors receive a spread, typically a multiple of the modeled expected loss, for bearing a tail risk largely uncorrelated with the market risk of Chapter 8's portfolios, the diversification argument of Chapter 12 applied to hurricanes.

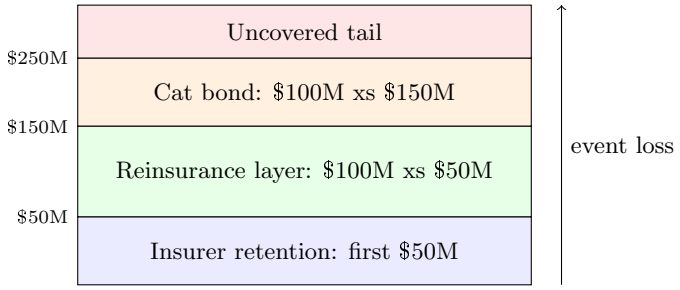


Figure 11.3: A catastrophe program as a tower: the insurer retains the first losses, traditional reinsurance covers the middle layers, and a catastrophe bond passes a higher layer to capital-market investors. Each layer's price is quoted as a rate on line: expected annual loss to the layer, plus a risk load, divided by the layer's limit.

The models behind these numbers are Hurricane Andrew's institutional legacy: simulation engines that generate tens of thousands of synthetic hurricane seasons or earthquake catalogs, propagate each event's physics over a portfolio of insured locations, and apply engineering damage functions, Monte Carlo over a compound process, once again Chapter 10's architecture at continental scale. AI's contribution is concentrated in the inputs: computer vision on aerial and satellite imagery (Chapter 17's methods) now determines roof geometry, construction type, and post-event damage at the individual-building level, replacing what was once a coarse, self-reported exposure database, and climate-conditioned event generation, which draws synthetic seasons from warming scenarios rather than from the historical record alone, extends these models toward the forward-looking scenario analysis that climate stress testing requires, the same scenario-based logic as the stress tests of Chapter 8.

Parametric Insurance and Basis Risk

A structural innovation rounds out the catastrophe toolkit. Parametric insurance pays a fixed amount when a measured *index* crosses a trigger, wind speed at a reference station, rainfall over a growing season, ground acceleration at a site, rather than indemnifying assessed damage. The payout requires no adjuster, so it arrives in days rather than months, and the product can be sold where claims adjustment is impractical, which is why parametric structures dominate sovereign disaster pools, index-based crop insurance across the developing world, and consumer novelties like flight-delay cover, and why they are a natural match for the satellite and sensor data of Chapter 17.

The price of simplicity is basis risk: the index and the loss are correlated, not identical. A worked example quantifies it. A parametric hurricane policy pays $200,000 if measured wind at the reference station reaches the trigger. Suppose the modeled annual probability of the policyholder suffering material damage is 0.05; given damage, the trigger fires with probability 0.80; in the 0.95 of years without damage, it false-fires with probability 0.01. Then the policyholder faces a $0.05 \times 0.20 = 1.0\%$ annual probability of the worst outcome, real damage and no payout, while the insurer faces a $0.95 \times 0.01 = 0.95\%$ probability of paying without damage; the overall payout probability is $0.05 \times 0.80 + 0.0095 = 0.0495$, for a pure premium of $0.0495 \times 200{,}000 = \$9{,}900$. Both mismatch probabilities are design parameters: a denser sensor network or a model-based index (a "cat-in-a-grid" trigger computed from the physics models above) tightens the loss-index correlation and shrinks basis risk, which is precisely where machine learning earns its keep in parametric design, and the trade-off between trigger fidelity and trigger verifiability, an index must be objective and manipulation-proof exactly because no adjuster will ever look at the actual loss, is Chapter 10's Goodhart discipline applied to contract design.

Wulin.Suo@Queensu.ca

11.5.2 Solvency: Capital Regulation as a Value at Risk

The regulatory capstone ties insurance directly back to Chapter 8. Modern insurance solvency regimes, Solvency II in Europe most explicitly, define required capital as a Value at Risk: the Solvency Capital Requirement is calibrated so that the insurer can absorb a 1-in-200 one-year loss, a 99.5% one-year VaR on the change in the insurer's own net asset value, aggregated across underwriting, market, credit, and operational risk modules with prescribed correlations. The calculation is Chapter 8's machinery pointed at a balance sheet instead of a trading book: if an insurer's one-year net-asset-value change has standard deviation \$40 million, a normal approximation puts the 99.5% requirement at $2.576 \times 40 \approx \$103$ million, and everything Chapter 8 said about the normal approximation understating fat tails applies with force, since underwriting results are compound-Poisson skewed, not Gaussian. U.S. risk-based capital reaches a similar structure by formula rather than internal model, and the internal-model-versus-standard-formula tension, sophistication and risk-sensitivity against comparability and gameability, is the same debate Chapter 10 described operational-risk capital resolving in favor of the standard formula.

Machine learning enters the capital calculation itself through the computational bottleneck at its center. The 99.5% one-year VaR requires knowing the *distribution* of the insurer's net asset value a year ahead, and net asset value at year one is itself a valuation, of every embedded guarantee and option on the book, under each scenario, the same nested-simulation explosion as Section 11.4.4's hedging problem, now at whole-balance-sheet scale. Internal-model insurers therefore lean on proxy models: least-squares Monte Carlo regressions and, increasingly, neural-network approximators trained to reproduce the full valuation as a fast function of the scenario variables, so that a million-scenario capital distribution becomes computable at all. A supervisor approving such an internal model is thereby approving a machine-learning approximation of a balance sheet, which is why European supervisory bodies have begun publishing guidance specifically on ML use inside internal models, and why the surrogate's fitting error is examined in the far tail, the 0.5% that sets the capital number, where any regression approximates worst. For the modeler, the practical consequence is that every model in this chapter, pricing, reserving, catastrophe, mortality, behavior, and the proxy models stacked on top of them, feeds a regulated capital number, and therefore lives inside the model-governance perimeter of Chapters 10 and 18, with validation, documentation, and independent challenge as legal requirements rather than good practice.

11.5.3 Underwriting with New Data and Text

Commercial insurance underwriting is a document business: a submission arrives as a broker's narrative, loss runs from prior insurers, financial statements, and schedules of locations or vehicles, and an underwriter's first hours are spent extracting structure from paper. That is Chapter 16's problem statement almost verbatim, and NLP-driven submission ingestion, extracting exposures, flagging omissions, triaging which submissions deserve an underwriter's scarce attention, has become the standard first deployment of language models in commercial carriers. Property underwriting increasingly begins from imagery rather than questionnaires, the same building-level computer vision as Section 11.5.1, applied one policy at a time, and cyber underwriting, one of the industry's fastest-growing and hardest lines, prices from external scans of an applicant's security posture, an alternative-data exercise in Chapter 17's exact sense, applied to a peril whose correlated, catastrophe-like tail (one vulnerability, thousands of simultaneous claims) the industry is still learning to model.

11.5.4 Fairness and Proxy Discrimination

Insurance pricing discriminates by design, in the actuarial sense of charging different expected costs differently, and the law's task has always been separating legitimate risk-based distinctions from prohibited ones. AI sharpens this old problem into Chapter 18's proxy problem: a model barred from using a protected characteristic can reconstruct it from correlated inputs, and the richer the feature set, the better the reconstruction. Insurance is where regulators have moved on this most concretely. Colorado's SB 21-169 requires insurers to test models and external data sources for unfairly discriminatory outcomes and to govern them accordingly; the European Union's gender ruling prohibits gender-based pricing outright even where it is actuarially predictive, a legislature deliberately overriding statistical validity with a social judgment; and the U.S. state-by-state debate over credit-based insurance scores, predictive of claims but correlated with

Wulin.Suo@Queensu.ca

protected characteristics, rehearsed every argument in Chapter 18's fairness section decades before machine learning raised the stakes. A second, subtler fairness frontier is price optimization: using demand models, each customer's estimated willingness to pay and propensity to shop, to set premiums *above* risk-based cost for the customers least likely to leave. The practice is standard revenue management in airlines and retail, but insurance regulators in many U.S. states have explicitly banned it, on the ground that an insurance price must reflect expected cost and expense, not exploitable inertia, and the customers who shop least, often older and lower-income policyholders, are precisely those the loyalty penalty falls on hardest; the United Kingdom reached the same conclusion in 2021, prohibiting renewal prices above the equivalent new-customer price. For the modeling reader the episode is instructive because the objectionable model is *perfectly accurate*: demand estimation is legitimate science, and the prohibition is aimed not at the model's errors but at its use, the clearest illustration in this book that model governance is about deployment decisions, not only statistical quality. The chapter-scale lesson travels in both directions: insurance supplies the financial industry's most developed case law on what "fair pricing" can be made to mean operationally, and Chapter 18's technical apparatus, fairness metrics, proxy detection, counterfactual testing, supplies insurance with the tools its newest statutes effectively mandate.

11.6 Case Vignette: Progressive's Snapshot and the Telematics Market

Usage-based insurance moved from concept to mass market when Progressive, after more than a decade of pilots dating to the late 1990s, rolled out its Snapshot program nationally in the early 2010s: a plug-in device, later a phone app, recording mileage, time of day, and hard-braking events, with discounts tied to observed behavior. Two features of the episode reward attention. First, the earliest and largest benefit was not pricing precision but *selection*: the mere offer of a monitoring discount attracted drivers who believed they drove well and repelled those who knew they did not, so the program improved the insurer's book before any telematics rating factor was filed, a live demonstration of adverse selection running in reverse. Second, the program's evolution traces this chapter's regulatory arc in miniature: early versions surcharged nothing and only discounted, precisely because filing a telematics *surcharge* required convincing regulators of the model's fairness and accuracy, and the industry's subsequent decade of negotiating what behavioral data may be used, retained, and shared previews the governance questions every AI pricing innovation now faces.

11.7 Challenges in Insurance and Actuarial Modeling

The tail is uninsurable by experience. Hurricane Andrew's lesson recurs: the losses that matter most are those history has not yet produced, so catastrophe, cyber, and pandemic risk must be modeled from structure rather than experience, with all of the model risk Chapter 10 attaches to exactly that move.

Feedback and selection are everywhere. Prices change who buys, monitoring changes who enrolls, and claims automation changes what is claimed; almost no insurance dataset is a random sample of the risk it is used to price, and Chapter 1's selection warnings bind with unusual force.

Long feedback loops. A mispriced auto book reveals itself in a year; a mispriced liability or life book can take a decade, so model validation must lean on the indirect methods of Chapter 10's validation discussion rather than rapid outcome feedback, and the float arithmetic of Section 11.2.2 means the errors compound financially while they remain statistically invisible.

Correlated tails defeat the product's own logic. The industry sells the law of large numbers, but its gravest exposures, hurricanes, earthquakes, cyber events, pandemics, are precisely the ones where independence fails; managing the gap between the diversifiable book and the undiversifiable tail is the industry's permanent structural problem.

Accuracy versus accountability, institutionalized. The filed-rating-plan regime makes the interpretability constraint structural: the industry's central modeling question is rarely "what is the most accurate model" but "what is the most accurate model that can be filed, explained, and audited."

Wulin.Suo@Queensu.ca

Privacy as a pricing input. Telematics, wearables, and property imagery make the underwriting data itself the sensitive object, so data governance is not compliance overhead but part of product design.

11.8 Key Takeaways

- Insurance inverts the production cycle, price first, costs later, and sells the law of large numbers retail: pooling 10,000 independent risks cut relative loss uncertainty from 417% to 4.17% in the worked example, a $\sqrt{n}$ improvement that correlated catastrophe losses delete.

- An insurer earns from two engines, the combined ratio and investment income on float; in the worked example a 95% combined ratio and $150M of float at 4% produced a 22% return on capital, and even a 103% combined ratio remained profitable because underwriting losses are the cost of cheap float.

- Classical pricing is frequency times severity with multiplicative relativities; one-way estimation double-counts correlated factors (a 43% overcharge in the worked 2×2 example), which is why the jointly-estimated GLM displaced it. "Traditional pricing" in today's industry debates means *GLM-based* pricing: the GLM is the decades-established, filed incumbent that machine learning must beat, not a stepping stone toward it.

- Machine learning's pricing advantage is the interaction: in the worked example, a true young-urban interaction that no multiplicative model can represent leaked into the GLM's fitted marginals and overcharged the young-rural cell by 11.4%, while a depth-two tree priced all four cells exactly; telematics runs the same logic at driver level, a fitted coefficient of +0.163 per harsh-braking event converting sensor data into a filed relativity (two fewer events than the cell average give $1.8 \times e^{0.163 \times (-2)} \approx 1.30$).

- Credibility theory, $Z = n/(n + k)$, is Bayesian shrinkage derived by actuaries decades before machine learning rederived it as regularization: the worked fleet's 0.333 observed frequency was blended to 0.147 against a 0.100 class rate.

- The chain ladder projects ultimate losses from development factors (1.50 and 1.10 in the worked triangle, a $392.5 thousand reserve); reserves are the float, so reserving error is an economic event, not an accounting one.

- Life insurance moves the weight to mortality and discounting (a $961.54 net premium in the worked term-life example), with Lee-Carter forecasting mortality as factor extraction plus a random walk. Asset-liability management is Chapter 5's duration machinery on both sides of the balance sheet, where falling rates hurt the long-liability insurer, and its ML layer is behavioral: dynamic-lapse models make liability duration a fitted model output, and in the worked example behavior clipped the favorable surplus outcome (16.5 → 13.5) while deepening the unfavorable one (3.5 → 1.5), a sold option with a human exercising it.

- Catastrophe risk is transferred through reinsurance towers and cat bonds priced from simulation models (2.4% rate on line in the worked layer), and solvency regulation caps the chapter as a 99.5% one-year VaR on the insurer's own net assets, Chapter 8's machinery pointed at a balance sheet.

- Health insurance replaces long-tail reserving with a selection problem managed by risk adjustment, a regression with a budget attached, whose coding incentives (an extra 0.13 of risk score was worth $1,300 per enrollee per year in the worked example) make model integrity an adversarial, audited contest.

- Parametric insurance trades adjusters for an index, paying instantly at the cost of basis risk: in the worked example, a 1.0% annual chance of damage without payout against a $9,900 pure premium, with index design, and its manipulation-proofness, the modeling problem.

- Insurance fairness law, from credit-based insurance scores to Colorado's SB 21-169, the EU gender ruling, and the price-optimization bans, is the financial industry's most developed treatment of proxy discrimination and model *use*, and it now effectively mandates Chapter 18's technical toolkit.

Wulin.Suo@Queensu.ca

11.9 Suggested Exercises

1. Recompute Section 11.2.1's pooling example for $n = 100$ and $n = 1{,}000{,}000$ policies. At what n does the probability of the portfolio's loss exceeding 110% of its mean fall below 0.1% (using the normal approximation), and why should you distrust that approximation in the far tail?

2. Recompute the worked pricing example for an experienced rural driver with frequency relativities 0.9 (age) and 0.8 (territory) and a severity relativity of 0.95. What gross premium results, and what fraction of the young urban driver's premium is it?

3. Using Table 11.1, verify by direct computation that the one-way relativities are 2.10 and 1.833, and then show that if the exposures had been balanced (250 in every cell, claims scaled to preserve each cell's frequency), the one-way method would have recovered the true relativities. What general condition on the exposure distribution makes one-way estimation unbiased?

4. In Section 11.3.3's interaction example, the GLM overcharged the young-rural cell by 11.4%. Explain the mechanism in your own words, why does an interaction concentrated in one cell distort the price of a *different* cell?, and then describe how an actuary could keep the filed model multiplicative yet still fix the mispricing (hint: an interaction term is itself just another rating variable). What does your fix imply about the practical role of a boosted challenger model in a GLM-filing regime?

5. A fleet has 5 years of experience with credibility constant $k = 12$. Compute its credibility weight, and the blended frequency if it observed 0.30 against a class rate of 0.10. How many years of experience would the fleet need for its own data to receive 75% weight?

6. Extend the chain-ladder triangle with a fourth accident year showing 12-month cumulative paid claims of 550. Compute its projected ultimate loss and the new total reserve, and state precisely which assumption you invoked about Year 4's future development.

7. Recompute Section 11.4.4's ALM example for an insurer that has hedged its liability duration down to 8 using swaps. What is the surplus after a one-point fall in rates, and what has the hedge cost the insurer if rates instead rise by one point?

8. Suppose a dynamic-lapse model of the kind Section 11.4.4 describes, fitted to policyholder experience, implies a lapse rate of $5\% + 4 \times \max(0, r - 3\%)$ per year against a 3% credited rate, where r is the market rate. Compute the lapse rate at market rates of 3%, 5%, and 6%; explain in option language (Chapter 5) why this behavior leaves the insurer short an option, the policyholder's right to surrender at book value when rates rise, which is a put on the policy's bond-like value or, equivalently, a call on rates; and describe what data a Chapter 6 classifier would need, and what drift monitoring it would require, before you would trust it to set the hedge for a one-in-a-generation rate spike.

9. A reinsurance layer of \$50 million attaching at \$150 million has a modeled 0.02 annual probability of attachment and an expected in-layer loss of \$30 million given attachment. Compute its expected annual loss and rate on line, and explain why its risk load per dollar of expected loss should exceed that of Section 11.5.1's lower layer.

10. In the parametric example of Section 11.5.1, suppose a better index raises the trigger's hit rate given damage from 0.80 to 0.95 while leaving the false-trigger rate at 0.01. Recompute both basis-risk probabilities and the pure premium, and explain why the policyholder should be willing to pay *more* than the pure-premium increase for this redesign.

11. An auditor observes that an insurer's average Medicare Advantage risk score rose from 1.05 to 1.20 over five years while its members' hospitalization and mortality rates were flat. Using Section 11.4.3, lay out the innocent and the upcoding explanations for this pattern; recompute the section's prevalence z-test for an insurer coding the same condition in 7% of 2,500 enrollees against the 6% benchmark, and explain why this smaller book's identical-direction deviation is far weaker evidence; then propose one *longitudinal* test (using the flat hospitalization data) that the cross-sectional prevalence test cannot provide.

Wulin.Suo@Queensu.ca

12. A regulator bars a rating variable that is predictive but correlates with a protected characteristic. Using Chapter 18's distinction between statistical validity and disparate impact, argue both sides: the actuarial case for the variable and the fairness case against it, and identify what empirical test would inform (though not settle) the dispute.

13. **Challenge (real data).** The French motor third-party liability dataset (`freMTPL2freq`, available through OpenML and the `CASdatasets` project) contains claim counts and rating features for roughly 678,000 real policies. Fit a Poisson GLM for claim frequency on driver age, vehicle age, and bonus-malus level, then fit a gradient-boosted model on the same features. Compare out-of-sample Poisson deviance, plot each model's implied age relativity, and identify one interaction the boosted model found that the GLM missed. Discuss which model you could defend in a rate filing, and what you would change about the boosted model to make it filable.

11.10 Further Reading

- Goldburd, Khare, Tebaldi, and Modlin, *Generalized Linear Models for Insurance Rating* (CAS Monograph No. 5). The Casualty Actuarial Society's freely available practitioner monograph on GLM ratemaking, the document that defines what "traditional pricing" concretely means in Section 11.3.1.

- Ohlsson and Johansson, *Non-Life Insurance Pricing with Generalized Linear Models*. The standard compact academic treatment of the same GLM machinery.

- Wüthrich and Merz, *Statistical Foundations of Actuarial Learning and its Applications*. A rigorous, freely available treatment of machine learning in actuarial work, including the GLM-versus-boosting comparison and neural reserving.

- Bühlmann and Gisler, *A Course in Credibility Theory and its Applications*. The definitive treatment of Section 11.3.2's credibility machinery by its principal architect.

- Denuit, Hainaut, and Trufin, *Effective Statistical Learning Methods for Actuaries* (three volumes). GLMs, tree-based methods, and neural networks developed specifically for insurance data.

- Dickson, Hardy, and Waters, *Actuarial Mathematics for Life Contingent Risks*. The standard modern text behind Section 11.4.2's life-contingency calculations.

- Grossi and Kunreuther, *Catastrophe Modeling: A New Approach to Managing Risk*. The standard introduction to the simulation-based catastrophe models of Section 11.5.1 and their post-Andrew history.

- Frees, *Regression Modeling with Actuarial and Financial Applications*. A bridge between this book's regression toolkit and actuarial applications, with real insurance datasets.

Wulin.Suo@Queensu.ca

Chapter 12

Portfolio Theory and Asset Allocation

12.1 Introduction

Chapter 2 introduced a two-asset portfolio to teach the mechanics of Python, and found, almost as an aside, that combining two negatively correlated assets produced a portfolio considerably less volatile than either asset held alone. This chapter returns to that observation and treats it as the central question of a whole field: given a set of available assets, their expected returns, and how they move together, what is the best way to combine them?

Portfolio theory answers this question with a specific, quantitative definition of "best": the highest expected return for a given level of risk. This chapter develops the classical tools built on that definition, namely mean-variance optimization, the Capital Asset Pricing Model, and multi-factor models, along with the more robust, practically motivated methods portfolio managers use when the classical theory's assumptions do not hold up well in real data.

Four worked examples run through the chapter, each reusing data already established earlier in this book. The two-asset AssetA/AssetB portfolio from Chapter 2 is extended into a full efficient frontier and a closed-form minimum-variance portfolio. The three-asset AssetA/AssetB/AssetC extension from Chapter 2 is reused to illustrate mean-variance optimization in matrix form, and to expose a genuine practical hazard, estimation error, that the classical theory alone does not warn against.

A small market-model regression revisits the equity beta of 1.2 assumed without derivation in Chapter 5's WACC calculation, this time estimating it directly from data. And a stylized equity/bond pair, with volatilities and expected returns chosen to be economically realistic rather than drawn from the toy price series, carries the tangency, Black-Litterman, and robo-advisor examples, where the four-day sample behind the other three is too short and too oddly correlated to illustrate the point.

A Brief History of Portfolio Theory

The ideas in this chapter did not arrive all at once, and knowing their sequence clarifies which problem each was designed to solve. Table 12.1 summarizes the main milestones.

Each development responded to a specific limitation of what came before: Markowitz's framework, while mathematically elegant, requires estimating a full covariance matrix and is highly sensitive to estimation error, which motivated both the single-factor simplification of CAPM and, decades later, Black-Litterman's more stable blending approach; and CAPM's single market factor, in turn, was found to leave systematic patterns in average returns unexplained, which Fama and French's additional factors were built specifically to capture.

This chapter follows roughly the same progression, moving from the classical Markowitz and CAPM foundations to the multi-factor and robust methods that address their practical shortcomings.

Wulin.Suo@Queensu.ca

Year	Development
1952	Markowitz formalizes mean-variance optimization, giving diversification, already intuitive to practitioners, a precise mathematical foundation
1964	Sharpe (building on related work by Lintner and Treynor) introduces the Capital Asset Pricing Model, connecting an asset's expected return to a single market-wide risk factor
1990	Black and Litterman propose a method for blending an investor's own views with market-implied equilibrium returns, addressing a practical instability in naive mean-variance optimization
1992–1993	Fama and French introduce size and value factors alongside the market factor, launching multi-factor asset pricing models

Table 12.1: A brief history of portfolio theory

12.2 Diversification and the Efficient Frontier

Combining assets changes the risk of the whole in a way that does not follow from the parts, and that observation is the foundation of the chapter.

12.2.1 Diversification and the Power of Combining Assets

Chapter 2 computed the covariance matrix for AssetA and AssetB directly from their return histories,

$$\Sigma = \begin{pmatrix} 0.000414 & -0.000198 \\ -0.000198 & 0.000118 \end{pmatrix},$$

implying individual volatilities of $\sigma_A = 2.03\%$ and $\sigma_B = 1.09\%$ and a correlation of $\rho_{AB} = -0.896$. For a two-asset portfolio with weight w on Asset A and $1 - w$ on Asset B, portfolio variance is

$$\sigma_p^2(w) = w^2\sigma_A^2 + (1-w)^2\sigma_B^2 + 2w(1-w)\rho_{AB}\sigma_A\sigma_B.$$

Chapter 2 evaluated this formula only at the single equal-weighted point $w = 0.5$, finding $\sigma_p = 0.58\%$, considerably below the weighted average of the two individual volatilities.

This chapter asks a more ambitious question: among every possible value of w, which one minimizes $\sigma_p^2(w)$? Differentiating with respect to w and setting the result to zero gives a closed-form solution,

$$w_A^{\text{MV}} = \frac{\sigma_B^2 - \sigma_{AB}}{\sigma_A^2 + \sigma_B^2 - 2\sigma_{AB}},$$

which, applied to the covariance matrix above, gives $w_A^{\text{MV}} = 34.05\%$ and $w_B^{\text{MV}} = 65.95\%$, a noticeably different allocation from the equal weighting Chapter 2 used purely for illustration. This minimum-variance portfolio has volatility $\sigma_p^{\text{MV}} = 0.322\%$, nearly half the equal-weighted portfolio's 0.583%, purely from choosing weights more carefully; no new asset, no new information about expected returns, only a smarter combination of the same two assets already available.

12.2.2 The Efficient Frontier and the Global Minimum-Variance Portfolio

Sweeping w across its full range, not just the two points considered so far, traces out every possible risk-return combination achievable from AssetA and AssetB, illustrated schematically in Figure 12.1. Because $\rho_{AB} < 0$, the resulting curve bows sharply to the left of a straight line connecting the two individual assets, the geometric signature of diversification: some combination of the two assets is less risky than either asset held alone, which is only possible because the assets do not move in lockstep.

Wulin.Suo@Queensu.ca

In this particular sample, Asset B happens to have *both* lower risk and higher return than Asset A ($\sigma_B = 1.09\%$ versus $\sigma_A = 2.03\%$, and $\bar{r}_B = 1.00\%$ versus $\bar{r}_A = 0.76\%$), so Asset A is not itself mean-variance efficient; the figure's upper arm, running from the leftmost point of the curve to Asset B, is the efficient portion, while the lower arm, running from that same leftmost point down to Asset A, is feasible but dominated. That leftmost point is the *global* minimum-variance portfolio, and the qualifier matters: every portfolio on the frontier is a minimum-variance portfolio for its own target expected return, whereas this one minimizes variance with no return constraint imposed at all, which is what makes it the single leftmost point rather than one of many.

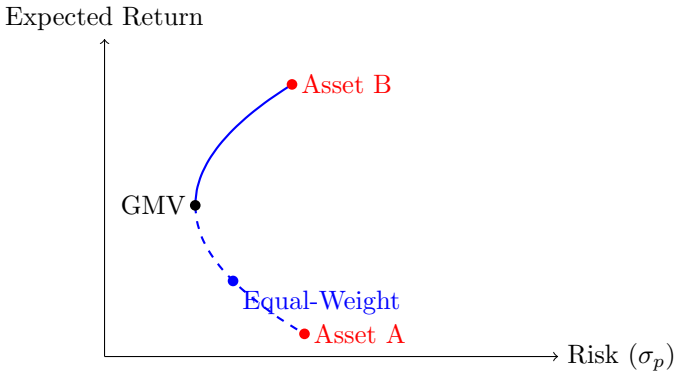


Figure 12.1: A schematic efficient frontier for the two-asset example (solid: efficient; dashed: feasible but dominated). GMV marks the global minimum-variance portfolio, the leftmost attainable point. An investor selects somewhere along the solid segment according to their risk tolerance; GMV is that menu's left endpoint, not its recommended item.

Only the upper portion of this curve, starting at the global minimum-variance portfolio and moving toward the higher-returning asset, is efficient in Markowitz's specific sense: every portfolio on this segment offers the highest possible expected return for its level of risk. The lower portion (the dashed segment in Figure 12.1) is feasible, an investor could hold it, but never optimal, since a portfolio directly above it on the efficient segment offers a strictly higher return at the same risk.

It is worth being clear about what the theory does and does not recommend here. The efficient frontier is a menu rather than an answer: the framework narrows the whole feasible region down to that solid upper segment, and the investor then picks a point on it according to how much variance they are willing to bear, a choice the mathematics deliberately leaves open. The global minimum-variance portfolio is simply that menu's left endpoint, the point an investor would choose only if they were unwilling to accept any additional variance whatsoever in exchange for return, so it is better understood as one extreme of the efficient set than as the portfolio the analysis is pointing toward. It nonetheless earns a practical role for a reason that has nothing to do with risk appetite: it is the only point on the frontier whose weights require no expected-return estimates at all, needing only the covariance matrix, which matters a great deal given how badly the estimation-error problem of Section 12.2.3 affects estimated means in particular.

This is the precise, quantitative content behind the informal advice to diversify: it is not merely that diversification feels prudent, but that every portfolio not on the efficient frontier is demonstrably dominated by one that is.

12.2.3 Mean-Variance Optimization in Matrix Form

The two-asset formulas above generalize to any number of assets using matrix notation. For n assets with covariance matrix Σ and a vector of ones $\mathbf{1}$, the (unconstrained) global minimum-variance portfolio weights are

$$w^{\mathrm{MV}} = \frac{\Sigma^{-1}\mathbf{1}}{\mathbf{1}^{\top}\Sigma^{-1}\mathbf{1}},$$

Wulin.Suo@Queensu.ca

the direct matrix generalization of the closed-form two-asset formula in Section 12.2.1. Applying this to the three-asset covariance matrix from Chapter 2's AssetA/AssetB/AssetC extension,

$$\Sigma_3 = \begin{pmatrix} 0.000414 & -0.000198 & -0.000383 \\ -0.000198 & 0.000118 & 0.000176 \\ -0.000383 & 0.000176 & 0.000358 \end{pmatrix},$$

gives minimum-variance weights of

$$w_A \approx 44.8\%, \qquad w_B \approx 14.2\%, \qquad w_C \approx 41.0\%,$$

with a portfolio standard deviation of only about 0.050%, roughly six times lower than the equal-weighted three-asset portfolio's 0.297% computed in Chapter 2.

This result should be viewed with considerable suspicion rather than celebrated, and the reason is a genuinely important practical lesson. Chapter 2's three-asset dataset contains only four daily observations, far too few to estimate a 3×3 covariance matrix (six distinct entries) reliably. The matrix-form optimizer above found weights that happen to nearly cancel out the sample's realized co-movements almost perfectly, but it is optimizing against estimation noise as much as against genuine risk, the overfitting phenomenon Chapter 6 illustrated with a high-degree polynomial: with enough free parameters relative to the amount of data, a model (here, a portfolio optimizer) can achieve an excellent-looking in-sample result that will not repeat out of sample. This is widely recognized in practice as Markowitz optimization's central weakness: it is, in Michaud's memorable phrase, an "estimation-error maximizer," since the optimizer actively seeks out and overweights the asset pairs whose historical correlation was most favorable, which are disproportionately likely to be pairs whose favorable correlation was partly a statistical accident of the specific sample rather than a stable underlying relationship. Sections 12.3.4 and 12.3.5 introduce two of the standard practical responses to this problem.

Shrinkage Estimation of the Covariance Matrix

A direct statistical response to the estimation-error problem above, distinct from the risk parity and Black-Litterman approaches developed later in this chapter, is to shrink the noisy sample covariance matrix itself toward a simpler, lower-variance target before ever handing it to the optimizer. A standard target is a diagonal matrix built from the average sample variance, and the shrunk covariance matrix is a weighted average of this target and the raw sample estimate,

$$\Sigma^{\text{shrunk}} = \delta F + (1 - \delta)\Sigma, \qquad F = \bar{\sigma}^2 I,$$

where $\delta \in [0, 1]$ is the shrinkage intensity (often chosen to minimize expected estimation error using a formula due to Ledoit and Wolf, though a fixed value illustrates the idea just as well) and $\bar{\sigma}^2$ is the average of the sample variances on the diagonal of Σ. Applying $\delta = 0.5$ to the three-asset covariance matrix from Section 12.2.3, whose average variance is $\bar{\sigma}^2 \approx 0.000297$, shrinks every off-diagonal covariance term roughly in half and pulls each diagonal variance partway toward the common average. Recomputing the minimum-variance weights using this shrunk covariance matrix gives

$$w_A^{\text{shrunk}} \approx 39.2\%, \qquad w_B^{\text{shrunk}} \approx 30.5\%, \qquad w_C^{\text{shrunk}} \approx 30.3\%,$$

a far more balanced and intuitively reasonable allocation than the extreme, nearly-single-pair-canceling weights the raw sample covariance matrix produced. The shrunk portfolio's true (sample) variance is necessarily somewhat higher than the unshrunk optimizer's in-sample-minimizing variance, exactly as it must be, since the unshrunk optimizer is defined to be the in-sample minimum by construction; the entire justification for shrinkage is that this modest in-sample cost buys a large improvement in expected out-of-sample reliability, a bias-variance tradeoff directly analogous to the ridge regression penalty Chapter 6 applied to an overfit polynomial.

12.3 Equilibrium Models and Factor Approaches

If everyone optimizes, prices must satisfy conditions of their own; this part develops those conditions and the factor models that relax them.

Wulin.Suo@Queensu.ca

12.3.1 The Capital Asset Pricing Model and Beta Estimation

Chapter 5 introduced the Capital Asset Pricing Model, $r_e = r_f + \beta(r_m - r_f)$, to estimate Meridian Fabrication's cost of equity, using an equity beta of 1.2 that was simply assumed rather than derived. This section closes that gap: beta is, in fact, a regression coefficient, estimated exactly like the OLS regressions of Chapter 6, by regressing an asset's excess returns on the market's excess returns,

$$r_{i,t} - r_f = \alpha_i + \beta_i(r_{m,t} - r_f) + \varepsilon_t.$$

Table 12.2 shows six months of hypothetical excess returns for a stock and the overall market.

Month	1	2	3	4	5	6
Market excess return	−2.0%	−1.0%	0.0%	1.0%	2.0%	3.0%
Stock excess return	−2.5%	−1.3%	0.2%	1.1%	2.6%	3.3%

Table 12.2: Six months of excess returns for a stock and the market

Applying the same OLS formulas used throughout Chapter 6,

$$\hat{\beta} = \frac{\sum_t (r_{m,t} - \bar{r}_m)(r_{i,t} - \bar{r}_i)}{\sum_t (r_{m,t} - \bar{r}_m)^2} \approx 1.19, \qquad \hat{\alpha} \approx -0.03\%, \qquad R^2 \approx 0.992.$$

The estimated beta of 1.19 closely matches the value of 1.2 that Chapter 5 simply assumed, which is a reassuring, if constructed, confirmation that the earlier chapter's assumption was reasonable, and it illustrates concretely where a CAPM beta actually comes from in practice: not from theory alone, but from a regression identical in form to the ones Chapter 6 introduced for entirely different applications. The near-zero estimated alpha is also economically meaningful: CAPM predicts that, after accounting for market exposure, no asset should offer a systematically positive or negative excess return, so a statistically significant nonzero alpha (checked using the t-statistic and p-value machinery introduced in Chapter 6) is often interpreted either as evidence against CAPM or as a genuine mispricing an active manager might try to exploit.

Systematic versus Idiosyncratic Risk

The $R^2 \approx 0.992$ from the regression above has a direct risk-decomposition interpretation, not just a goodness-of-fit one. Total return variance splits into a systematic (market-driven) component and an idiosyncratic (asset-specific) component,

$$\mathrm{Var}(r_i) = \underbrace{\beta_i^2 \,\mathrm{Var}(r_m)}_{\text{systematic}} + \underbrace{\mathrm{Var}(\varepsilon_i)}_{\text{idiosyncratic}},$$

and R^2 is the fraction of total variance that is systematic. For the stock in Table 12.2, with returns expressed as decimals, $\mathrm{Var}(r_m) \approx 0.00035$ and systematic variance is $\hat{\beta}^2 \,\mathrm{Var}(r_m) \approx (1.19)^2(0.00035) \approx 0.000494$, against total variance of about 0.000498, leaving an idiosyncratic remainder of only about 0.0000038, consistent with the regression's near-perfect fit.

CAPM's central economic claim rests directly on this decomposition: idiosyncratic risk can be diversified away by holding many assets whose asset-specific shocks are independent of one another, the diversification mechanism of Section 12.2.1, so a well-diversified investor should not expect to earn any additional compensation for bearing it. Systematic risk, by contrast, affects every asset simultaneously through the shared market factor and cannot be diversified away no matter how many assets are added to the portfolio. That is precisely why CAPM's expected-return formula compensates an asset only for its beta (systematic risk) and not for its total volatility. Two assets with identical total volatility but different betas should, under CAPM, offer different expected returns, while two assets with identical betas but very different idiosyncratic volatility should offer the same expected return, a testable and occasionally violated prediction that motivated the multi-factor extensions discussed later in this chapter.

Wulin.Suo@Queensu.ca

The Security Market Line

This relationship between beta and expected return has a natural graphical representation, the Security Market Line, plotting CAPM's expected return, $r_f + \beta(r_m - r_f)$, as a straight line against beta itself, shown in Figure 12.2. Unlike the Capital Market Line of Section 12.3.2 below, which plots return against total risk (σ_p) and is populated only by efficient portfolios, the Security Market Line plots return against systematic risk (beta) alone and, under CAPM, every correctly priced individual asset or portfolio, efficient or not, should lie exactly on it.

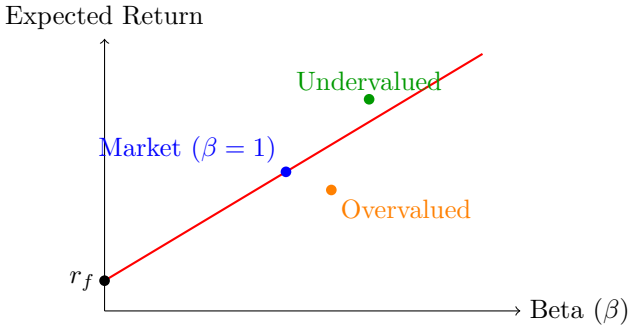


Figure 12.2: The Security Market Line plots CAPM's expected return against beta, with an undervalued (green) and an overvalued (orange) asset marked relative to it

An asset whose actual expected return plots above the Security Market Line is, under CAPM, undervalued: it offers more expected return than its systematic risk alone would justify, and the nonzero alpha from Section 12.3.1's regression is the vertical distance between an asset's actual position and the line. An asset plotting below the line is correspondingly overvalued. This is precisely the graphical version of the active-management logic introduced later in this chapter's discussion of the information ratio: an active manager's entire task can be described as searching for assets that plot persistently above the Security Market Line, and a manager who succeeds systematically is finding genuine mispricing rather than simply taking on more systematic risk than their benchmark, which the Security Market Line makes visually explicit in a way that a raw return comparison does not.

12.3.2 The Capital Market Line and the Sharpe Ratio

CAPM's beta measures only one dimension of an asset's risk, its co-movement with the market; the Sharpe ratio provides a complementary, portfolio-level summary of risk-adjusted performance,

$$\text{Sharpe Ratio} = \frac{\bar{r}_p - r_f}{\sigma_p},$$

the excess return earned per unit of total risk taken. Comparing the equal-weighted and minimum-variance two-asset portfolios from Section 12.2.1, using their sample mean returns of 0.88% and 0.92% respectively and treating the daily risk-free rate as approximately zero for this short-horizon illustration,

$$\text{Sharpe}_{\text{equal}} = \frac{0.88\%}{0.583\%} \approx 1.51, \qquad \text{Sharpe}_{\text{MV}} = \frac{0.92\%}{0.322\%} \approx 2.85.$$

The minimum-variance portfolio not only has lower risk, as expected by construction, but also a substantially higher Sharpe ratio, since it happens to also have a slightly higher mean return in this particular sample; this is not a general guarantee (the minimum-variance portfolio does not target return at all) but is a useful reminder that lower risk and competitive returns are not mutually exclusive.

Plotting every portfolio's Sharpe ratio against a combination of the risk-free asset and the tangency portfolio (the portfolio of risky assets with the single highest Sharpe ratio) produces the Capital Market Line, illustrated in Figure 12.3, the set of risk-return combinations achievable by mixing the risk-free asset with that one specific risky portfolio.

Wulin.Suo@Queensu.ca

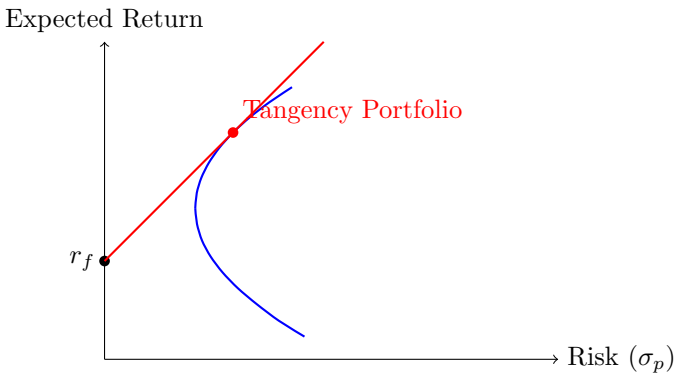


Figure 12.3: The Capital Market Line (red) is tangent to the efficient frontier of risky assets (blue) at the tangency portfolio

A central result of modern portfolio theory (the two-fund separation theorem) is that every investor, regardless of their individual risk tolerance, should hold some combination of only the risk-free asset and this single tangency portfolio, differing only in how much of each they hold, not in the composition of the risky portion itself. A more risk-averse investor simply holds more of the risk-free asset and less of the tangency portfolio, while a less risk-averse investor may even borrow at the risk-free rate to hold more than 100% of their wealth in the tangency portfolio, moving further out along the Capital Market Line in either direction without ever needing to change which risky assets they hold or in what proportion.

A Worked Example: Computing the Tangency Portfolio

The tangency portfolio pictured in Figure 12.3 has a closed-form solution directly analogous to the minimum-variance formula of Section 12.2.1, maximizing the Sharpe ratio rather than minimizing variance alone,

$$w^{\tan} = \frac{\Sigma^{-1}(\boldsymbol{\mu} - r_f \mathbf{1})}{\mathbf{1}^\top \Sigma^{-1}(\boldsymbol{\mu} - r_f \mathbf{1})},$$

the matrix generalization of maximizing $(\bar{r}_p - r_f)/\sigma_p$ directly rather than minimizing σ_p^2 alone. Applying this to AssetA and AssetB, with mean daily returns $\mu_A = 0.76\%$, $\mu_B = 1.00\%$ and, as in Section 12.3.2's Sharpe ratio calculation above, a risk-free rate of approximately zero for this short-horizon illustration, gives

$$w_A^{\tan} = 33.76\%, \qquad w_B^{\tan} = 66.24\%,$$

strikingly close to the minimum-variance weights of 34.05% and 65.95% computed in Section 12.2.1, with a Sharpe ratio of 2.849, barely above the minimum-variance portfolio's own 2.848.

This near-coincidence is specific to this dataset, not a general property of the tangency portfolio, and it is worth seeing a case where the two portfolios genuinely differ to understand why. With $r_f \approx 0$ and both assets' mean returns small relative to their risk, the excess-return vector $\boldsymbol{\mu} - r_f \mathbf{1}$ that drives the tangency formula is nearly proportional to a constant vector, which makes $w^{\tan}$ collapse close to the minimum-variance weights (driven purely by $\Sigma^{-1}\mathbf{1}$) almost by coincidence. Applying the identical formula to the equity/bond assumptions of the robo-advisor discussion later in this chapter instead ($\sigma_e = 16\%$, $\sigma_b = 5\%$, $\rho_{eb} = -0.20$, $r_e = 8\%$, $r_b = 3\%$), with a more economically meaningful risk-free rate of $r_f = 2\%$, gives a tangency portfolio of

$$w_e^{\tan} = 32.0\% \text{ equities}, \qquad w_b^{\tan} = 68.0\% \text{ bonds}, \qquad \text{Sharpe} = 0.468,$$

meaningfully different from that section's minimum-variance weights of 13.1% equities and 86.9% bonds (Sharpe ratio 0.374 at the same risk-free rate). The lesson generalizes: the tangency portfolio and the minimum-variance portfolio coincide only when expected returns happen not to matter much for distinguishing one asset from another relative to their risk; whenever expected returns differ meaningfully across

Wulin.Suo@Queensu.ca

assets, as equities and bonds do here, the two portfolios can diverge substantially, and only the tangency portfolio is actually the one the two-fund separation theorem recommends every investor hold in combination with the risk-free asset.

12.3.3 Multi-Factor Models: Beyond a Single Market Beta

CAPM's single market factor, while foundational, leaves systematic patterns in average returns unexplained; empirically, smaller companies and companies with high book-to-market ratios have historically earned higher average returns than CAPM's single beta predicts. Fama and French's three-factor model extends CAPM by adding two additional factors, SMB (Small Minus Big, the return of small-capitalization stocks minus large-capitalization stocks) and HML (High Minus Low, the return of high book-to-market "value" stocks minus low book-to-market "growth" stocks), on top of the original market factor:

$$r_{i,t} - r_f = \alpha_i + \beta_{i,\mathrm{MKT}}(r_{m,t} - r_f) + \beta_{i,\mathrm{SMB}}\,\mathrm{SMB}_t + \beta_{i,\mathrm{HML}}\,\mathrm{HML}_t + \varepsilon_t.$$

Estimating a two-factor version of this model (market and SMB only, for a compact illustration) requires the matrix form of OLS introduced conceptually in Chapter 6, $\hat{\beta} = (X^\top X)^{-1} X^\top y$, since more than one explanatory variable no longer admits the simple single-variable formulas used earlier in this chapter. Fitting this to six months of hypothetical market, SMB, and asset return data gives

$$\hat{\alpha} \approx -0.6\%, \qquad \hat{\beta}_{\mathrm{MKT}} \approx 1.19, \qquad \hat{\beta}_{\mathrm{SMB}} \approx 0.35, \qquad R^2 \approx 0.999;$$

the positive SMB loading indicates the asset behaves somewhat like a small-cap stock, earning a return premium associated with that factor beyond what its market beta alone would predict.

Each additional factor in a model like this is, in effect, an additional explanatory variable in a multiple regression, so the model-selection and overfitting cautions from Chapter 6 apply directly. Adding enough factors will always improve in-sample R^2. That is why serious factor-model research relies on economically motivated factors validated out of sample, rather than a large, undisciplined search over every conceivable candidate factor, a practice sometimes derided in the finance literature as data mining for a "factor zoo."

Multi-factor models have moved from academic research into everyday investment products through so-called "smart beta" or factor-based exchange-traded funds. These construct a portfolio designed to have deliberately high exposure to a specific factor, such as value, size, momentum, or low volatility, rather than simply weighting holdings by market capitalization as a traditional index fund does. The appeal of these products is direct: if a factor like SMB or HML has historically earned a return premium, an investor may prefer to capture that premium systematically and at low cost through a rules-based fund rather than paying an active manager to attempt the same thing through discretionary stock selection. The same overfitting caution above applies with particular force here, since a factor's historical premium, discovered through the kind of statistical search this section warns against, can weaken or disappear once it becomes widely known and systematically invested in, as more capital chasing the same factor exposure competes away the very mispricing the premium was compensating for, a dynamic with a direct analogue in Chapter 3's discussion of market efficiency: a genuinely exploitable pattern tends not to remain exploitable once it is discovered and traded on at scale.

12.3.4 Naive Risk Parity: A Correlation-Light Alternative

Section 12.2.3 showed that full mean-variance optimization can be dangerously sensitive to estimation error in the covariance matrix, particularly its off-diagonal correlation terms, which are typically the hardest entries to estimate precisely. The simplest form of risk parity, naive risk parity (inverse-volatility weighting), sidesteps much of this fragility by allocating weight inversely to volatility alone, ignoring correlation entirely,

$$w_i^{\mathrm{RP}} = \frac{1/\sigma_i}{\sum_j 1/\sigma_j}.$$

Applied to AssetA and AssetB,

$$w_A^{\mathrm{RP}} = 34.8\%, \qquad w_B^{\mathrm{RP}} = 65.2\%, \qquad \text{Sharpe} \approx 2.84,$$

Wulin.Suo@Queensu.ca

remarkably close to the true minimum-variance weights of 34.1% and 65.9% (Sharpe ratio 2.85) computed in Section 12.2.1 despite using far less information (no correlation estimate at all). That closeness, however, is a feature of this dataset rather than a general property. For two assets, the minimum-variance weights coincide exactly with the inverse-volatility weights when the correlation is exactly -1, and AssetA and AssetB's sample correlation is -0.896, close to that limit. Holding the two volatilities fixed and raising the correlation shows how quickly the agreement breaks down: the minimum-variance weight on AssetA falls to 30.35% at $\rho = -0.5$, 22.18% at $\rho = 0$, and 2.41% at $\rho = 0.5$, while the inverse-volatility weight stays at 34.81% throughout. The case for inverse-volatility weighting is therefore not that it approximates the mean-variance optimum but that it is robust: it requires estimating only n volatilities rather than the full $n(n-1)/2$ correlation pairs, each of which is an additional source of estimation error of the kind that made the three-asset optimization in Section 12.2.3 so unreliable. Risk parity as practiced in industry, often called equal risk contribution, goes one step further and does use correlations: it chooses weights so that every asset contributes the same amount, $w_i(\Sigma w)_i$, to total portfolio variance, the same risk-contribution decomposition that underlies Chapter 8's Component VaR. Inverse-volatility weighting is the special case in which all pairwise correlations are equal; with only two assets the two methods give identical weights whatever the correlation, so the distinction first matters with three or more. This tradeoff, accepting a method that is provably suboptimal under the classical theory's own assumptions in exchange for far greater robustness to violations of those assumptions, is a recurring and important theme in practical portfolio management.

12.3.5 Black-Litterman: Blending Views with Market Equilibrium

Suppose you manage the 60/40 portfolio of Section 12.4.6 and you come to work with exactly one opinion: equities look cheap relative to bonds. This is the ordinary situation of a working investor, and it is one the optimizer of Section 12.2.3 simply cannot accept. That machinery demands a complete vector of expected returns, one number for every asset in the portfolio, so before it will run at all you must also hand it an expected return for bonds, about which you have no opinion whatsoever. Whatever number you invent, the optimizer treats it as a firmly held belief and acts on it exactly as decisively as on the view you actually hold.

There is a second problem, and it is just as practical. The formula has nowhere to record how sure you are. An opinion you would stake your reputation on and one you formed over coffee enter the calculation identically and produce identical portfolios, even though the difference between them is precisely what ought to determine how far the portfolio moves away from where it started.

Both problems land on the input that can least afford them. Chopra and Ziemba quantified this directly: for a typical investor's risk tolerance, errors in estimated *means* are roughly an order of magnitude more costly, in terms of the utility actually lost, than errors of the same relative size in variances, which are in turn about twice as costly as errors in covariances. The expected-return vector is the most fragile input in the entire apparatus, and the naïve procedure requires you to fabricate most of it in order to express one genuine view. That same ordering is why the global minimum-variance portfolio of Section 12.2.2 is more robust than its position at one extreme of the efficient frontier would suggest: needing no expected returns at all, it is immune to the largest error source by construction.

The Black-Litterman model answers both objections at once, and it does so without touching the covariance matrix. It supplies a default answer to "what do I think about bonds?" so that holding no opinion becomes a position the mathematics can actually represent, and it lets every opinion you do hold carry an explicit statement of how confident you are in it.

The default comes from the market itself. Other investors' collective holdings are observable, and if those holdings are broadly sensible then they already encode a set of expected returns: whatever numbers would have made that particular portfolio the optimal one to hold. Recovering them means running Section 12.2.3's machinery backward, a step called reverse optimization, which deduces the excess returns $\mathbf{\Pi}$ that make the observed market weights w_{mkt} optimal instead of deducing weights from returns,

$$\mathbf{\Pi} = \delta \, \Sigma \, w_{\text{mkt}},$$

where δ is the market's coefficient of risk aversion. This is the answer to the bond question: not a number you made up, but the one implied by what everyone else is already doing.

Wulin.Suo@Queensu.ca

Your own opinions then enter through three objects, each corresponding to something an investor can actually say. A matrix P records which assets each view is about, so that "equities relative to bonds" can be expressed without saying anything about either one on its own. A vector Q gives the magnitudes claimed. And a matrix Ω states, on its diagonal, how uncertain each view is, which is where "I would stake my reputation on this" and "this is a hunch" finally become different inputs. The blend of prior and views is then

$$E[R] = \left[(\tau\Sigma)^{-1} + P^\top \Omega^{-1} P\right]^{-1} \left[(\tau\Sigma)^{-1}\mathbf{\Pi} + P^\top \Omega^{-1} Q\right],$$

the scalar τ setting how tightly the equilibrium prior itself is held. Intimidating as the notation looks, it is doing something familiar: this is Chapter 1's Bayesian update written in matrix form, with $\mathbf{\Pi}$ as the prior, Q as the evidence, and Ω^{-1} as the weight that evidence earns. A confident view (small Ω) pulls the answer toward Q; a tentative one leaves it near $\mathbf{\Pi}$.

The two consequences are worth stating plainly, because they are exactly what the opening two problems asked for. An investor with no views at all gets the market weights back, unchanged, so "I have no opinion" now produces a sensible portfolio rather than an arbitrary one. And an investor with a view, such as "I believe equities will outperform bonds by 9% over the coming year," gets a portfolio tilted away from the market by an amount that depends on how firmly the view was asserted, rather than the unconstrained lurch a purely historical-return optimizer would deliver. This is what makes Black-Litterman well suited to combining a systematic, data-driven baseline with the qualitative, analyst-driven judgment that a purely statistical model like those in Chapter 6 cannot generate on its own. The worked example that follows shows both consequences as numbers.

12.3.6 A Worked Example: Blending a View into a 60/40 Portfolio

Take the equity and bond assumptions Section 12.3.2 already used ($\sigma_e = 16\%$, $\sigma_b = 5\%$, $\rho_{eb} = -0.20$, $r_f = 2\%$) and treat Section 12.4.6's classic 60/40 split as the market portfolio. That portfolio's volatility is 9.41% and, under the same 8% and 3% expected returns, its excess return is 4.00%, which pins down the risk-aversion coefficient as $\delta = 0.0400/0.0941^2 = 4.52$. Reverse optimization then implies equilibrium excess returns of

$$\Pi_e = 6.6546\%, \qquad \Pi_b = 0.0181\%.$$

Two features of this prior deserve attention before any view is added. These are not the 8% and 3% assumed a moment ago, and they were never meant to be: they are the returns that would make a 60/40 investor's weights optimal, so the implied equity-minus-bond spread, $6.6546\% - 0.0181\% = 6.6365\%$, is simply wider than the 5.00% those assumptions embed. Bonds, meanwhile, earn almost no equilibrium premium at all, which is not a claim that bonds are unattractive but a consequence of their low volatility and negative correlation contributing almost nothing to the 60/40 portfolio's total risk, precisely what a risk-based equilibrium ought to conclude.

Now suppose the investor holds the single view quoted above, that equities will beat bonds by 9%, against the 6.6365% equilibrium already implies. Setting $P = (1, -1)$, $Q = 9\%$, $\tau = 0.05$, and $\Omega = P(\tau\Sigma)P^\top$, which calibrates the view's uncertainty to the prior's own uncertainty about that same spread, the posterior expected excess returns become 7.68% for equities and -0.14% for bonds, and the resulting optimal weights are

$$w_e = 68.4\% \text{ equities}, \qquad w_b = 31.6\% \text{ bonds}.$$

The view moved the portfolio 8.4 percentage points toward equities rather than to a corner, and the size of that move tracks the confidence attached to it: halving Ω, which asserts the view more firmly, produces 71.1/28.9, while tripling it produces only 64.2/35.8. That bounded, monotone response is the entire point of the construction. The same 9% opinion yields a tilt the investor can dial rather than the violent swing a purely historical-return optimizer would produce from identical information, and setting the view aside altogether returns exactly 60/40, the equilibrium the model began from.

Wulin.Suo@Queensu.ca

12.4 Building and Running a Portfolio

A portfolio on paper becomes a portfolio in practice only after constraints, costs, currencies, and mandates are imposed on it.

Practical Allocation Decisions: Constraints, Rebalancing, and Transaction Costs

The mathematical portfolios developed so far in this chapter ignore several frictions that matter enormously in practice. Real portfolios are usually subject to constraints beyond the mathematics of the optimization itself: a long-only mandate prohibits negative weights (short positions), which the unconstrained formulas in Sections 12.2.1 and 12.2.3 do not rule out on their own, and position limits or sector caps may further restrict how concentrated any single holding or sector can become, both of which typically require solving the optimization as a constrained quadratic program rather than using the closed-form matrix formulas presented above.

A concrete example shows exactly when this constraint binds. Consider two assets with $\sigma_X = 8\%$, $\sigma_Y = 20\%$, and an unusually high correlation of $\rho_{XY} = 0.9$. Applying the closed-form minimum-variance formula from Section 12.2.1 gives

$$w_X^{\text{MV}} \approx 145.5\%, \qquad w_Y^{\text{MV}} \approx -45.5\%, \qquad \sigma_p \approx 5.26\%,$$

a substantial short position in Asset Y financing a leveraged long position in Asset X, achieving an unconstrained portfolio volatility below either individual asset's own volatility. A long-only mandate rules this out entirely, and because portfolio variance is monotonically decreasing in w_X across the entire feasible range $w_X \in [0, 1]$ on the way toward the (infeasible) unconstrained optimum, the best achievable long-only portfolio is the corner solution $w_X = 100\%$, $w_Y = 0\%$, with volatility equal to Asset X's own 8%. In this particular case, the long-only constraint does not merely reduce the benefit of diversification; it eliminates it completely, since every feasible long-only combination of the two assets is riskier than holding Asset X alone. This is a direct illustration of why the unconstrained formulas developed earlier in this chapter are a starting point for intuition rather than a literal prescription: real portfolio construction almost always requires solving a constrained optimization problem numerically, and the constraints themselves can qualitatively change the character of the optimal solution, not just its exact numerical weights.

Even a well-constructed portfolio does not stay at its target weights on its own, since different assets earn different returns over time and weights drift accordingly, so rebalancing back to target weights is periodically necessary. Every rebalancing trade, however, incurs the transaction costs introduced in Chapter 3, including at least half the bid-ask spread and any market impact from the trade's size, so frequent rebalancing that chases small deviations from target weights can destroy more value in trading costs than it preserves in risk control.

Consider a \$1,000,000 portfolio that has drifted 5 percentage points from its target weights, requiring a \$50,000 trade in each direction (selling the overweight asset, buying the underweight one) to restore them. At a one-way transaction cost of 15 basis points, roughly consistent with the bid-ask-spread-based costs introduced in Chapter 3, each \$50,000 trade costs $\$50{,}000 \times 0.0015 = \$75$, for a total round-trip rebalancing cost of \$150, or 0.015% of the portfolio's value. This may look negligible for a single rebalancing event. But a manager who rebalances daily in response to every small drift compounds this cost across hundreds of trading days per year. That is precisely why practitioners typically rebalance on a fixed calendar schedule, quarterly for example, or only once a weight has drifted beyond some tolerance band, rather than continuously. The choice explicitly trades off tracking the target allocation precisely against the cumulative transaction-cost drag of doing so too often.

Taxable investors face a further consideration entirely absent from the mathematics above: selling an appreciated position to rebalance can trigger a capital gains tax liability, so a tax-aware rebalancing strategy may deliberately tolerate more allocation drift than the pure risk-minimization mathematics would recommend, in exchange for deferring a tax cost.

12.4.1 Solving the Constrained Problem Numerically

Once inequality constraints such as a long-only mandate or a position cap are added, the elegant closed-form matrix formulas of Sections 12.2.1 and 12.2.3 no longer apply directly, since those formulas were derived

Wulin.Suo@Queensu.ca

by setting a gradient to zero, a technique that only characterizes an unconstrained (or purely equality-constrained) optimum. The general constrained minimum-variance problem is instead stated and solved as

$$\min_{\mathbf{w}} \ \mathbf{w}^\top \Sigma \mathbf{w} \quad \text{subject to} \quad \mathbf{w}^\top \mathbf{1} = 1, \quad \mathbf{w}^\top \boldsymbol{\mu} = r_0 \text{ (optional)}, \quad w_i^{\text{low}} \le w_i \le w_i^{\text{high}} \ \ \forall i,$$

where the budget constraint ($\mathbf{w}^\top \mathbf{1} = 1$) is always present, the target-return constraint is included only when tracing a specific point on the efficient frontier rather than the global minimum-variance portfolio itself, and the box constraint $w_i^{\text{low}} = 0$, $w_i^{\text{high}} = 1$ recovers a long-only mandate, a tighter w_i^{high} recovers a position or sector cap, and $w_i^{\text{low}} < 0$ permits limited short selling.

This is a quadratic objective with linear constraints, solved in practice with a numerical constrained optimizer, most commonly sequential quadratic programming (SLSQP), available directly as `scipy.optimize.minimize(..., method="SLSQP")` in Python, which iterates toward a solution rather than computing one in a single matrix step the way Sections 12.2.1 and 12.2.3's formulas do.

Applying this numerical approach to the AssetX/AssetY example above, minimizing variance subject to the budget constraint and long-only bounds $0 \le w_i \le 1$, converges in a handful of iterations to $w_X = 100\%$, $w_Y = 0\%$, the corner solution derived by hand above, a useful confidence check that the numerical method and the closed-form reasoning agree exactly where both are available.

One practical wrinkle is worth flagging explicitly, since it is easy to miss: a generic optimizer's default convergence tolerance is tuned for objective values of a typical order of magnitude, and the raw portfolio variances in this chapter, on the order of 0.0001 or smaller for daily returns, are small enough that a naive call can terminate after a single iteration at whatever starting point was supplied, silently reporting success despite having barely moved. Rescaling the objective, for instance by working in squared-percent units rather than raw squared decimals, or tightening the optimizer's tolerance explicitly, avoids this failure mode; the companion notebook shows both the failure and the fix side by side.

This machinery also makes it possible to revisit Section 12.2.3's three-asset minimum-variance portfolio (weights of 44.8%, 14.2%, and 41.0% in AssetA, AssetB, and AssetC, with a volatility of about 0.050%) under a position cap no single asset may exceed, a common institutional constraint that the closed-form formula cannot express at all. Capping every weight at 40% forces AssetA's allocation down from its unconstrained 44.8% to the 40% cap, with the freed-up weight redistributed to the other two assets:

$$w_A^{\text{cap}} = 40.0\%, \qquad w_B^{\text{cap}} = 28.4\%, \qquad w_C^{\text{cap}} = 31.6\%, \qquad \sigma_p^{\text{cap}} \approx 0.113\%,$$

more than double the uncapped minimum-variance portfolio's 0.050%, a concrete price paid for the diversification-forcing discipline the cap imposes.

Tracing out a full constrained efficient frontier under the same 40% cap follows the same recipe with the optional target-return constraint included: solving for the minimum-variance portfolio achieving a 0.75% expected return, for instance, gives

$$w_A = 40.0\%, \qquad w_B = 35.1\%, \qquad w_C = 24.9\%, \qquad \sigma_p \approx 0.136\%,$$

one point on a constrained frontier that a reader can trace fully by sweeping the target return r_0 across a grid and re-solving at each point, exactly as the companion notebook does.

12.4.2 Tracking Error and the Information Ratio

Many portfolio managers are evaluated not on absolute return alone but relative to a specified benchmark, and the Sharpe ratio of Section 12.3.2 has a direct benchmark-relative analogue. The active return in any period is simply the portfolio's return minus the benchmark's return, $r_t^{\text{active}} = r_t^p - r_t^b$, and tracking error is the standard deviation of this active return series,

$$\text{Tracking Error} = \text{std}(r^{\text{active}}).$$

Using Chapter 2's equal-weighted AssetA/AssetB portfolio returns as the "portfolio" and AssetB's own returns as a hypothetical benchmark, the active returns across the four days are

$$r_1^{\text{active}} = 0.50\%, \qquad r_2^{\text{active}} = -1.24\%, \qquad r_3^{\text{active}} = 1.74\%, \qquad r_4^{\text{active}} = -1.48\%,$$

Wulin.Suo@Queensu.ca

giving a tracking error of 1.52% and a mean active return of −0.12%. The information ratio, the active-return analogue of the Sharpe ratio, divides the two,

$$\text{Information Ratio} = \frac{\text{mean active return}}{\text{Tracking Error}} \approx \frac{-0.12\%}{1.52\%} \approx -0.08,$$

a small negative value indicating that, over this short and deliberately illustrative sample, the portfolio slightly underperformed its benchmark per unit of active risk taken. This is the kind of number a manager and their client would examine closely. A persistently negative information ratio suggests a manager is not adding value relative to simply holding the benchmark, net of whatever fees the active management arrangement costs. A positive and statistically significant one (again checked with the same t-statistic machinery from Chapter 6) is one standard piece of evidence that a manager's active decisions are genuinely adding value rather than reflecting noise.

Machine Learning in Portfolio Construction

The classical methods in this chapter estimate their inputs, expected returns, variances, and covariances, using simple sample statistics, but every one of those inputs is itself a prediction problem that the machine learning methods of Chapter 6 can potentially improve. Regularized regression (ridge or LASSO, introduced in Chapter 6's discussion of overfitting) can estimate expected returns or factor exposures more stably than raw historical averages, shrinking noisy coefficient estimates toward zero in a manner directly analogous to the covariance shrinkage of Section 12.2.3.

12.4.3 Can Machine Learning Forecast Expected Returns?

Section 12.3.5 established that expected returns are the input whose estimation errors cost the most, so the obvious question is whether the methods of Chapter 6 can predict them better than a historical average can. This has been tested about as thoroughly as any question in empirical finance. Gu, Kelly, and Xiu ran the full comparison, putting ordinary least squares, penalized linear models, random forests, gradient-boosted trees, and neural networks on the same footing across roughly thirty thousand US stocks and ninety-four firm characteristics over six decades. Neural networks and tree ensembles won clearly, for the same reason they won in Section 12.4.4's much smaller example, namely their ability to capture interactions and nonlinearities that a linear specification cannot represent, and the linear benchmarks were close to useless out of sample.

The headline result is a monthly out-of-sample R^2 of roughly 0.40%, and a reader arriving from Chapter 6 will read that as a catastrophic failure. It is not, and understanding why is the most transferable lesson in this section. Monthly equity volatility runs around 5% against an expected return near 0.7%, so noise genuinely dominates signal by construction, and an R^2 near half a percent sits close to what is attainable rather than far below it. Campbell and Thompson made the argument precise: the right yardstick for a return forecast is not the fraction of variance it explains but the squared Sharpe ratio of the portfolio it funds, and by that standard a monthly R^2 of half a percent is economically substantial. Gu, Kelly, and Xiu's long-short portfolios formed on the neural-network forecasts duly earned Sharpe ratios far above what their linear competitors managed.

Three features of the problem keep it hard, and none of them will be familiar from a standard machine learning course. The signal-to-noise ratio, as just described, is a property of the phenomenon rather than of the estimator. The target is adaptive and, in effect, adversarial: McLean and Pontiff found that documented anomaly returns shrink by roughly a quarter between a study's sample period and the period immediately after it, and by well over half once the result is published, which is to say that a pattern degrades precisely because someone found it. And the data are scarcer than they look, since sixty years of monthly observations amount to only a few hundred points per series, a thin diet for a method designed for millions of images.

Putting this beside Section 12.3.5's ordering yields the uncomfortable summary of this whole chapter's estimation problem: predictability runs in the opposite direction from importance. Variances and covariances, whose errors cost the least, are the most forecastable, which is why the volatility models of Chapter 7 work as well as they do. The level of expected returns, whose errors cost the most, is the least forecastable. In between sits cross-sectional prediction, deciding which assets will outperform which, which is materially more tractable than predicting the market's direction and is where nearly all of the successful applied work concentrates.

Wulin.Suo@Queensu.ca

This also settles where an ML forecast should enter a portfolio process. Feeding it directly into Section 12.2.3's optimizer as $\boldsymbol{\mu}$ discards the one thing that makes it usable, which is that the model reports not only a prediction but the accuracy of its own past predictions. A model that emits a forecast together with an out-of-sample error is producing the pair Black-Litterman asks for, a view Q and its uncertainty Ω, and routing it that way makes the resulting tilt automatically proportional to how much the model has actually earned. A predictor with a 0.40% R^2 receives a correspondingly large Ω and moves the portfolio only slightly, which is the correct response to a weak but genuine signal rather than a disappointment.

12.4.4 A Head-to-Head Test: Ridge Regression versus OLS on Collinear Factors

Section 12.3.3's two-factor example uses only six months of data and two clearly distinct factors, too clean a setting to show a genuine estimation-error problem. A real factor library is rarely this well-behaved: candidate factors are often correlated with one another, momentum and a "quality" factor both partly reflect recent price strength, say, and a short history makes that collinearity far more damaging. The companion notebook extends the factor regression to twelve months and five candidate factors (market, SMB, HML, momentum, and quality), with quality *constructed* as a near-linear combination of HML and momentum (0.61 and 0.89 correlation respectively) to mimic this real-world redundancy, and with known true exposures $\beta = (1.10, 0.30, 0.20, -0.10, 0.15)$ so that each estimator's error can be measured directly against ground truth rather than only against a single sample's noisy realized returns.

Ordinary least squares, fit on all twelve months, badly overfits this short, collinear history:

$$\hat{\beta}_{\text{OLS}} = (1.078,\ -0.467,\ 0.979,\ 1.435,\ -2.322), \qquad \text{RMSE} = 1.390,$$

getting the market exposure roughly right but the SMB sign backward, wildly overstating HML and momentum, and estimating a quality exposure of -2.322 against a true value of $+0.15$, an error of more than two and a half units on a coefficient that should be modest. A ridge regression fit on the identical data, with a shrinkage penalty chosen to be small but nonzero, instead recovers

$$\hat{\beta}_{\text{ridge}} = (0.957,\ 0.124,\ 0.027,\ -0.011,\ -0.075), \qquad \text{RMSE} = 0.167,$$

still imperfect, but with every coefficient pulled back toward a plausible order of magnitude and no wild sign reversal on a factor that should matter, an 88% RMSE reduction against the five true exposures. This mirrors almost exactly the covariance-shrinkage story of Section 12.2.3. The raw, unregularized estimator is not simply less precise but actively misleading, fitting the specific collinear noise pattern this one short sample happened to contain, while a modest shrinkage penalty trades a small amount of bias for a large reduction in this kind of estimation-error blowup.

Clustering methods, such as the K-means algorithm Chapter 6 applied to group firms by leverage and profit margin, can be applied instead to group assets by return co-movement before optimizing, an approach known as hierarchical risk parity: rather than inverting a single, potentially ill-conditioned covariance matrix across all assets simultaneously (the source of the estimation-error fragility in Section 12.2.3), the portfolio is built recursively, first allocating risk across clusters of similar assets and then within each cluster. This tends to produce allocations that are more stable and diversified than naïve full-matrix mean-variance optimization when the number of assets is large relative to the length of the available return history.

More recent research has extended tree-based ensembles and neural networks (both introduced in Chapter 6) directly to return prediction and portfolio weight generation, training a model to output portfolio weights that maximize a risk-adjusted objective like the Sharpe ratio directly, rather than first predicting returns and then feeding those predictions into a separate mean-variance optimization step. This end-to-end approach can, in principle, avoid compounding two separate sources of error (prediction error and optimization error), but it inherits every overfitting and interpretability concern Chapter 6 raised, in a setting where an opaque, poorly validated model directly controls real capital allocation, the model risk management concerns Chapter 8 discussed at length. As in every other application of machine learning to finance covered in this book, the message is not that these methods should be avoided, but that they must be validated, monitored, and understood by the people ultimately responsible for the capital they allocate, not deployed simply because they are available.

A further, more experimental line of research frames the entire allocation and rebalancing decision as a sequential decision-making problem and applies reinforcement learning, an approach in which an algorithm

Wulin.Suo@Queensu.ca

learns a trading or allocation policy directly through simulated trial and error against a reward signal such as risk-adjusted return net of transaction costs, rather than through a single-shot prediction and optimization step. This framing is naturally suited to the rebalancing tradeoffs discussed earlier in this chapter, since a reinforcement learning agent can, in principle, learn to weigh the benefit of tracking target weights closely against the transaction-cost drag of trading too frequently within a single unified objective, rather than treating rebalancing policy and portfolio construction as two separate decisions. The same caution applies here with even greater force: a reinforcement learning agent trained in a simulated market environment inherits every assumption baked into that simulation, and a policy that performs well in simulation can fail unpredictably when deployed against real markets whose dynamics differ from the training environment in ways that were never anticipated, an especially acute version of the non-stationarity concern raised throughout this book.

International Diversification and Currency Risk

Every example so far in this chapter has implicitly assumed a single currency, but a portfolio that holds foreign assets introduces an additional risk factor absent from the domestic case: currency risk, the risk that movements in exchange rates change the domestic-currency value of a foreign holding independently of how that asset performs in its own local market. A US investor holding a European stock earns a return that combines the stock's local-currency performance with the euro's movement against the dollar over the same period, and these two components can partially offset or reinforce each other, so the same underlying stock can look like a very different investment once currency effects are included.

International diversification is, in one sense, simply diversification as developed throughout this chapter applied across a wider opportunity set: an investor holding only domestic equities is implicitly betting that domestic economic conditions are the only source of risk worth diversifying, when in fact returns across countries are typically far from perfectly correlated, since different economies are exposed to different industry mixes, monetary policies, and business cycles. This is the same mechanism behind Section 12.2.1's two-asset example, applied at the level of entire national markets rather than individual securities, and it is why most institutional portfolios hold meaningful allocations to international equities and bonds despite the added complexity of currency risk.

A small numerical example shows how large the currency component can be. Suppose a US investor holds a European stock that returns 5% in euro terms over the period, while the euro simultaneously depreciates 3% against the US dollar. The unhedged US-dollar return combines both effects multiplicatively,

$$r_{\text{USD}}^{\text{unhedged}} = (1 + r_{\text{local}})(1 + r_{\text{FX}}) - 1 = (1.05)(0.97) - 1 \approx 1.85\%,$$

more than three percentage points below the stock's own local-currency performance, purely because of currency movement. A fully currency-hedged position, using the forward contracts introduced in Chapter 3 to lock in today's exchange rate for the investment's future conversion back to dollars, would instead earn approximately the full 5% local-currency return (less the hedge's own cost), isolating the stock-picking decision from the currency decision entirely. Whether to hedge is itself a portfolio decision, not a free choice: currency-hedged international exposure isolates the diversification benefit of holding foreign assets while removing the currency bet, at the cost of the hedge's own transaction costs, while unhedged exposure retains the currency bet as an additional, sometimes uncompensated, source of risk and occasionally return, and different institutional investors reach different conclusions about which side of that tradeoff suits their objectives.

Robo-Advisors: Automating Asset Allocation

A robo-advisor packages nearly every tool this chapter has developed, a risk-based target allocation, periodic rebalancing, tax awareness, into a single automated, low-cost retail product, delivered through a web or mobile interface rather than a human advisor's office. Firms such as Betterment and Wealthfront, both founded in 2008, pioneered the category; traditional incumbents including Vanguard and Schwab entered a few years later, typically pairing the same automated core with an option to add a human advisor for a higher fee, a hybrid model this section returns to below.

The appeal is straightforward: the classical machinery developed above, from diversification in Section 12.2.1 through international diversification and currency risk, does not, in principle, require a highly

Wulin.Suo@Queensu.ca

paid professional to execute once it has been encoded correctly, so automating it can extend a reasonably disciplined version of institutional-grade asset allocation to investors with far smaller account balances than a traditional advisory relationship has historically served.

From a Risk Questionnaire to a Target Allocation

A new client typically completes a short questionnaire covering age, time horizon, income stability, and stated comfort with market declines, which the platform converts into a target stock/bond split. A common industry heuristic sets the equity share at 110 minus the investor's age, so a 35-year-old investor receives a suggested allocation of 75% equities and 25% bonds.

It is worth asking, using this chapter's own machinery, exactly what this heuristic is and is not doing. Treating "equities" and "bonds" as two broad asset classes with illustrative volatilities $\sigma_e = 16\%$, $\sigma_b = 5\%$, and correlation $\rho_{eb} = -0.20$, Section 12.2.1's closed-form minimum-variance formula gives

$$\begin{aligned} \text{Minimum-variance:} &\quad 13.1\% \text{ equities},\ 86.9\% \text{ bonds},\ \sigma_p = 4.43\%, \\ \text{Questionnaire-based 75/25:} &\quad 75\% \text{ equities},\ 25\% \text{ bonds},\ \sigma_p = 11.81\%, \end{aligned}$$

the questionnaire split roughly two and a half times riskier than the minimum-variance point.

This is not a mistake in either number; it reflects two different objectives. Using illustrative expected returns of 8% for equities and 3% for bonds, and the same 2% risk-free rate as the tangency and Black-Litterman examples,

$$\begin{aligned} \text{Minimum-variance:} &\quad E[r_p] = 3.65\%,\ \text{Sharpe} = 0.374, \\ \text{Heuristic 75/25:} &\quad E[r_p] = 6.75\%,\ \text{Sharpe} = 0.402, \end{aligned}$$

so the heuristic does not sacrifice risk-adjusted return relative to the minimum-variance point; it improves on it. The minimum-variance portfolio ignores expected returns entirely and holds far less equity than the 32.0%/68.0% tangency portfolio, whose Sharpe ratio of 0.468 is the highest these two assets can deliver. The 75/25 split overshoots the tangency weights in the other direction, holding more than twice the tangency portfolio's equity share, so it does give up some Sharpe ratio relative to the tangency portfolio in exchange for substantially higher expected growth. A young investor with a long horizon and years of future contributions ahead of them is being pointed toward a point further out on the efficient frontier of Figure 12.1, beyond the tangency point, a trade a long-horizon investor is usually willing to make.

This is worth flagging as a genuine, practical departure from Section 12.3.2's two-fund separation theorem, which says every investor should hold the very same maximum-Sharpe tangency portfolio and differ only in how much of it they hold, using leverage if necessary to hold more than 100% of it. A robo-advisor does not, in practice, offer a young client a leveraged position in a bond-heavy tangency portfolio; it instead shifts the mix of risky assets themselves toward more equity, a pragmatic response to the fact that most retail investors neither can nor will borrow at the risk-free rate to lever up a conservative portfolio, whatever the two-fund separation theorem recommends in principle. As a client approaches a stated goal, typically retirement, the target allocation is walked back toward bonds along a predetermined glide path, the same lifecycle logic behind a conventional target-date fund.

Rebalancing and Tax-Loss Harvesting at Scale

The discussion of rebalancing mechanics under "Practical Allocation Decisions" earlier in this part of the chapter is worth re-reading with a robo-advisor specifically in mind. The \$1,000,000 portfolio, 5-percentage-point drift, 15-basis-point transaction cost example from that discussion is not merely an illustration. It is the calculation a robo-advisor's rebalancing engine performs continuously and automatically across every client account simultaneously, executing a trade only once a client's own drift tolerance band is breached rather than on a fixed calendar schedule a human advisor might follow more loosely.

Tax-loss harvesting, a hallmark feature of the taxable (non-retirement) accounts these platforms offer, extends the same automation to a client's tax bill directly. When a held position falls in value, the platform sells it to realize the loss and immediately buys a similar, but not "substantially identical," security, avoiding the wash-sale rule that would otherwise disallow the loss, while keeping the client's market exposure essentially unchanged.

Wulin.Suo@Queensu.ca

Suppose a client holds a $50,000 equity-fund position that has fallen 10%, a $5,000 realized loss. Under U.S. tax rules, up to $3,000 of a net capital loss can offset ordinary income in the year it is realized, and any remainder offsets capital gains. At a 32% marginal income-tax rate and a 15% long-term capital-gains rate, this single harvesting event saves $\$3{,}000 \times 32\% = \$960$ against ordinary income and $\$2{,}000 \times 15\% = \$300$ against capital gains, a first-year tax saving of $1,260 on this one position alone. It is generated automatically and, at the scale of hundreds of thousands of accounts, far more consistently than a human advisor manually reviewing each client's holdings once or twice a year could realistically match. Some platforms extend this idea to direct indexing, holding the individual stocks that make up an index directly rather than a single index fund. The point is to multiply the number of individual tax lots available to harvest a loss from at any given time.

12.4.5 Fee Structure and Long-Horizon Impact

A typical robo-advisor charges roughly 0.25% of assets annually, against roughly 1% for a traditional human advisory relationship. Compounded over a long horizon, this gap is far larger than it first appears. Starting from $100,000 and assuming a 7% gross annual return sustained for 30 years,

$$\begin{aligned} \text{0.25\% fee (robo-advisor):} &\quad \$709{,}637, \\ \text{1\% fee (traditional advisor):} &\quad \$574{,}349, \\ \text{0\% fee (hypothetical benchmark):} &\quad \$761{,}226. \end{aligned}$$

The lower-cost automated option preserves $135,288 more than the traditional-fee alternative over three decades, purely from a 75-basis-point difference in annual cost. That is a concrete illustration of the fee-drag-through-compounding mechanism Chapter 5's time-value-of-money material makes possible to quantify precisely, here applied to a management fee rather than a bond coupon or a discount rate.

None of this makes a robo-advisor strictly superior to a human one. The risk-tolerance questionnaire is a considerably blunter instrument than the individualized judgment a skilled human advisor can bring to a client's full financial picture. Tax situations, estate planning, and major life events are not things a standardized set of multiple-choice questions can capture. Self-reported risk tolerance is itself known to be unstable, shifting with recent market performance and how a question happens to be framed rather than reflecting a fixed underlying preference. That is why some platforms now supplement the initial questionnaire with an ongoing, behavior-informed refinement of a client's risk profile, rather than treating the onboarding survey as a one-time, permanent input. This is precisely where machine learning, beyond the return- and covariance-estimation applications already discussed above, does genuinely distinct work in a robo-advisor. A classifier trained on a client's actual observed behavior, whether they reduced contributions or attempted to sell equities during a past downturn, can produce a revealed-preference risk profile that predicts future behavior better than the stated questionnaire answer alone. A similar optimization layer decides, lot by lot and continuously rather than on a fixed schedule, exactly which tax lots to harvest and which substitute security to buy. That is a genuinely large-scale constrained optimization problem, well beyond what a human advisor reviewing client accounts manually could replicate at the same frequency. This is also why the industry has moved toward hybrid models, such as Vanguard's Personal Advisor Services and Betterment's premium tier. These layer a human advisor on top of the same automated core, specifically for the situations the algorithm alone handles least well.

Case Vignette: Schwab Intelligent Portfolios and the Cost of Undisclosed Cash Drag

In June 2022, the U.S. Securities and Exchange Commission announced that Charles Schwab & Co. and an affiliated adviser would pay $187 million, $52 million in disgorgement and prejudgment interest and a $135 million civil penalty, to settle charges concerning its robo-advisor, Schwab Intelligent Portfolios. From March 2015 through November 2018, the SEC found, Schwab's algorithm allocated a materially large share of client portfolios to cash, averaging about 12.5% across accounts and ranging from roughly 6% for the most aggressive risk profiles to nearly 24.9% for the most conservative ones, without disclosing that Schwab's own internal analysis showed this allocation would very likely underperform a portfolio holding less cash under most market conditions. Schwab profited directly from the arrangement, earning an estimated $46 million from the spread between what it paid depositors on that cash and what it earned investing it, income the

Wulin.Suo@Queensu.ca

cash allocation's own disclosed marketing materials did not adequately explain was a live consideration in how the portfolios were actually built.

The episode is a direct illustration of a point Chapter 8's model risk governance material and Chapter 18's explainability material both made in different contexts: an algorithmic asset allocation model is still a model, and a cash allocation is still an asset allocation decision, one this chapter's own mean-variance and risk-parity machinery would need to justify on genuine risk-return grounds, not accept simply because the firm operating the algorithm has a separate balance-sheet incentive to hold more of it. A robo-advisor's principal selling point is that automation removes the conflicts of interest and behavioral biases a commission-driven human advisor might introduce. That does not automatically hold if the automation itself is built, silently, around the platform operator's own incentives rather than the stated investment strategy alone. Which is why the same governance discipline Chapter 8 and Chapter 15 developed for other financial models, independent validation, clear disclosure, and ongoing monitoring against the model's own stated objective, applies to a robo-advisor's allocation engine with no less force than to a bank's credit-scoring or trading model.

12.4.6 Case Vignette: From the 60/40 Portfolio to the Endowment Model

The practical history of asset allocation illustrates several of this chapter's themes concretely. For decades, a simple 60% equities / 40% bonds allocation was the default institutional and individual portfolio, an intuitive, if informal, application of the diversification logic in Section 12.2.1: equities and bonds have historically been imperfectly correlated (bonds have often, though not always, performed relatively well during equity downturns), so combining them in a fixed ratio captured much of diversification's benefit without requiring any of the matrix algebra developed in this chapter.

Beginning in the 1990s, several large university endowments, most prominently Yale's under David Swensen, moved deliberately away from this simple split toward what became known as the endowment model: substantial allocations to less liquid alternative assets, including private equity, venture capital, hedge funds, and real assets, alongside a reduced allocation to traditional public equities and bonds. The economic logic behind this shift is a direct, practical application of the diversification and factor-model ideas developed earlier in this chapter: alternative assets often have return drivers that are only partially explained by the same market factor that drives public equities, so adding them can shift a portfolio's position on the efficient frontier of Figure 12.1 in a genuinely favorable direction, not merely diversify within the existing risk factors.

This benefit, however, comes with a cost that a purely mean-variance framework can understate: illiquidity. An endowment with a long investment horizon and predictable, modest annual spending needs can tolerate holding assets that cannot easily be sold for years at a time. It can be compensated for that illiquidity with an illiquidity premium, a higher expected return demanded specifically for accepting the inability to trade freely. A pension fund or individual investor with nearer-term or less predictable liquidity needs faces a fundamentally different tradeoff, and cannot simply copy the endowment model's specific allocations without also matching its underlying liquidity profile. That mismatch has, in practice, caused real difficulty for institutions that adopted illiquid strategies without adequate attention to their own funding needs.

ESG Constraints and the Cost of a Tilt

More recently, many institutional investors have layered environmental, social, and governance (ESG) considerations onto their asset allocation process, either by excluding certain sectors or companies entirely (negative screening) or by explicitly tilting portfolio weights toward higher-scoring companies within a sector (positive tilting), sometimes formalized as an additional factor alongside the market, size, and value factors of this chapter's multi-factor model.

From a purely mean-variance perspective, any constraint or tilt not directly motivated by risk and expected return, including an ESG constraint, can only leave a portfolio on or below the unconstrained efficient frontier of Figure 12.1, never above it.

Whether ESG investing is nonetheless value-enhancing in practice is genuinely disputed in the empirical literature, and depends on unresolved questions this chapter's framework cannot settle on its own. One is whether ESG characteristics themselves proxy for otherwise-unpriced risks, in which case ESG exposure is really a disguised risk factor, capturable within the CAPM and multi-factor machinery already developed

Wulin.Suo@Queensu.ca

in this chapter. Another is whether investors are willing to accept a lower expected return in exchange for values-alignment as a distinct, non-financial objective, a preference the classical mean-variance framework was never built to represent.

A Worked Example: The Cost of an ESG Tilt

Section 12.4.1's numerical machinery makes the "on or below the efficient frontier" claim above precise rather than qualitative. Extend that section's three-asset AssetA/AssetB/AssetC covariance matrix with an illustrative ESG score for each asset, on the familiar 0-to-100 composite scale real rating providers use: AssetA (a carbon-intensive legacy business) scores 40, AssetB (a clean-technology firm) scores 85, and AssetC (a diversified industrial) scores 60.

The unconstrained long-only minimum-variance portfolio, unchanged from Section 12.2.3, weights these 44.8%, 14.2%, and 41.0%, giving a portfolio ESG score of only $0.448(40) + 0.142(85) + 0.410(60) \approx 54.6$, dragged down by AssetA's large weight despite its low score.

Adding a minimum-portfolio-ESG-score constraint to the identical SLSQP optimizer, $\mathbf{w}^\top \mathbf{esg} \geq \text{ESG}_{\min}$, alongside the same budget and long-only constraints, reallocates weight away from AssetA and toward AssetB as the floor rises:

$$\begin{aligned}
\text{ESG} \geq 65: &\quad w = (36.0\%,\ 48.8\%,\ 15.2\%), &\quad \sigma_p \approx 0.216\%,\\
\text{ESG} \geq 70: &\quad w = (31.8\%,\ 65.4\%,\ 2.8\%), &\quad \sigma_p \approx 0.315\%,\\
\text{ESG} \geq 75: &\quad w = (22.2\%,\ 77.8\%,\ 0.0\%), &\quad \sigma_p \approx 0.484\%,
\end{aligned}$$

volatility rising from the unconstrained 0.050% to more than four times that at the mildest floor tested, and nearly ten times at the strictest, a concrete illustration of the frontier-shifting cost the qualitative argument above describes: every additional point of required portfolio ESG score is bought with additional variance, since the optimizer is, by construction, no longer free to choose the covariance-minimizing weights alone.

This worked example's ESG scores were assigned by hand specifically to make the tradeoff visible; a genuinely difficult practical problem lies one level upstream of it, in choosing which score to use at all. Berg, Kölbel, and Rigobon (2022) compare ESG ratings for the same companies across six major providers and find an average pairwise correlation of only 38% to 71%, far lower than the near-perfect agreement credit rating agencies typically show for the same bond; the divergence traces mostly to disagreement about which underlying metrics to measure and how to weight them, not to noisy measurement of an otherwise agreed-upon concept.

A portfolio tilted toward "high ESG" using one provider's scores can therefore look meaningfully different, in both composition and the volatility cost computed above, from a portfolio built on a different provider's scores for the identical universe of assets, a model-risk concern squarely inside the governance discipline Chapter 8 and Chapter 18 developed for every other model in this book, now applied to a data input rather than a fitted model.

12.5 Challenges in Portfolio Theory and Asset Allocation

Several challenges recur across nearly every practical application of the methods in this chapter, and later chapters revisit some of them, particularly the machine learning and execution-related ones, in greater depth.

Estimation error in expected returns. Section 12.2.3 demonstrated estimation error in the covariance matrix, but expected returns are, in practice, considerably harder to estimate precisely than volatilities or correlations, since return estimates are far noisier relative to their own magnitude over any realistic sample length; a mean-variance optimizer fed noisy expected-return estimates alongside a noisy covariance matrix compounds both sources of error simultaneously, which is the specific practical problem Black-Litterman in Section 12.3.5 was designed to address.

Non-stationarity of correlations. Chapter 8's discussion of the 2008 financial crisis noted that correlations between assets can rise sharply during a crisis, exactly when diversification is needed most; a portfolio built on calm-period correlation estimates can therefore provide considerably less protection during a genuine crisis than the mean-variance mathematics, calibrated on that calm-period data, would suggest.

Wulin.Suo@Queensu.ca

Factor model instability. The factors that explain returns well in one historical period do not necessarily continue to do so in the next, and the multi-factor models of this chapter, like every statistical model in this book, are estimated on historical data and implicitly assume that the estimated relationships will continue to hold, an assumption that can and does fail as market structure and investor behavior evolve.

Crowding. When many investors independently adopt similar factor tilts or risk models, whether through smart-beta products, similar academic research, or convergent proprietary models, their positions can become highly correlated without any of them realizing it, so that a shock affecting one crowded strategy can force simultaneous, mutually reinforcing selling across many ostensibly independent portfolios; this is a portfolio-construction-level instance of the same correlation-in-a-crisis phenomenon Chapter 8 described for market risk more broadly.

Liquidity and capacity constraints. An optimization's mathematically optimal weights may not be achievable in practice for a large portfolio, since buying or selling a large position in a less liquid asset can move its price adversely, the market-impact concern introduced in Chapter 3; a strategy that looks optimal on paper can underperform substantially once realistic trading costs and capacity limits are taken into account.

Behavioral and institutional deviations from the theory. Real investors and institutions frequently deviate from the mathematically optimal portfolio for reasons the classical theory does not capture. These include career risk, since a manager who deviates too far from a benchmark and is wrong faces disproportionate consequences compared to one who hugs the benchmark and is wrong, along with the regulatory capital treatment of specific asset classes and behavioral biases such as overconfidence or loss aversion. All of them mean observed portfolio choices in practice often differ systematically from what Sections 12.2.1 through 12.3.5 would recommend.

Time-varying risk and return. Every formula in this chapter, from the two-asset minimum-variance weight to the multi-factor regression, implicitly treats volatilities, correlations, and expected returns as constants to be estimated once from a historical sample. Chapter 7 showed at length that volatility, at minimum, is not constant but clusters and evolves over time, and there is good reason to believe correlations and factor exposures shift as well; an allocation framework that re-estimates its inputs only infrequently can therefore be working from stale, and sometimes badly wrong, assumptions about the very risk and return relationships it is trying to optimize over.

Survivorship bias in factor research. A factor's historical premium is typically estimated from a database of companies that includes only firms that survived to be recorded, quietly excluding firms that went bankrupt or were delisted, echoing the survivorship bias Chapter 1 flagged as a general data challenge; a factor that appears to have earned a strong historical premium may look considerably less attractive once the return-dragging failures it would have included, had they been part of the historical dataset, are properly accounted for.

Multiple testing across the factor literature. Academic and practitioner research has, collectively, tested an enormous number of candidate factors against the same limited history of market data. The multiple-testing concern raised in Chapters 6 and 7 therefore applies at the level of the entire published literature, not just within a single study. With enough factors tested against the same data by enough independent researchers, some will appear statistically significant by chance alone. A published factor with an impressive backtested premium is not automatically a genuine, exploitable phenomenon simply because it cleared a conventional significance threshold in one study.

Model risk in portfolio construction itself. Every optimizer, shrinkage estimator, and factor model in this chapter is, in the sense developed at length in Chapter 8, itself a model requiring the same governance discipline, independent validation, ongoing monitoring, and honest documentation of known limitations, as a bank's VaR or credit-scoring model. A portfolio construction process that treats its own optimizer as an unquestionable black box, rather than as one input to be checked against simpler benchmarks, economic intuition, and the specific estimation-error and non-stationarity concerns raised throughout this chapter, is exposed to the same kind of undetected model risk that Chapter 8's case vignette described in the context of Value at Risk.

Liquidity mismatches. This chapter's case vignette on the endowment model illustrated a specific version of a general hazard: any allocation to illiquid assets must be sized against the investor's own liquidity needs, not just against the assets' expected diversification benefit, and an institution that mismatches these two can find itself forced to sell its most liquid remaining holdings at the worst possible time precisely because

Wulin.Suo@Queensu.ca

its illiquid holdings cannot be sold at all, a dynamic closely related to the funding-liquidity risk discussed in Chapter 8.

Benchmark and objective misspecification. The tracking error and information ratio of Section 12.4.2 are only as meaningful as the benchmark chosen to compute them; a manager evaluated against a benchmark that does not actually reflect their investment mandate can appear to add or destroy value purely as an artifact of a poorly chosen comparison, a subtle but consequential measurement problem that has no purely statistical fix, since the right benchmark is a judgment call about what the portfolio is actually trying to achieve.

None of these challenges argues against using the classical or robust methods developed earlier in this chapter; each is instead a reason to apply them with the same combination of statistical rigor and institutional judgment this book has emphasized from Chapter 1 onward, treating a mean-variance optimizer's output as a well-informed starting point for discussion rather than as a final, unquestionable answer.

12.6 Key Takeaways

Portfolio theory formalizes an intuition Chapter 2 first illustrated numerically: combining imperfectly correlated assets can reduce risk without sacrificing return, and mean-variance optimization identifies the specific combination that does so best, given accurate inputs. The efficient frontier traces out every such optimal combination, the Capital Market Line and two-fund separation theorem show that combining the risk-free asset with a single tangency portfolio dominates every other combination, CAPM connects an asset's expected return to a single estimable market beta (which Section 12.3.1 showed matches Chapter 5's WACC assumption reasonably well) and decomposes total risk into systematic and idiosyncratic components, and multi-factor models extend this single-factor logic to capture additional systematic patterns in returns.

The chapter's central practical warning, however, is that the classical theory's mathematical elegance masks a serious real-world fragility: full mean-variance optimization is acutely sensitive to estimation error, especially with limited data, which is why covariance shrinkage, risk parity, hierarchical clustering, and Black-Litterman all exist as more robust, if theoretically suboptimal, alternatives that practitioners actually use. The sharpest form of that fragility is an inversion worth remembering: the three inputs rank in the opposite order on how much their errors cost and on how well they can be forecast. Covariances are the most predictable and the least damaging to get wrong, expected returns the least predictable and by an order of magnitude the most damaging, which is why so much of the practical machinery in this chapter consists of ways to avoid having to estimate expected returns at all.

Beyond the core optimization machinery, real portfolio management layers on long-only and other constraints that can qualitatively change optimal weights, periodic rebalancing disciplined by transaction costs, tracking error and information ratios for benchmark-relative evaluation, and currency considerations for internationally diversified portfolios, all of which the classical unconstrained, single-currency mathematics sets aside for clarity but which a real allocation decision cannot ignore.

Every method in this chapter, like every model in this book, is only as good as the assumptions and data behind it, and a serious asset allocation practice combines the classical framework's discipline with an explicit accounting for estimation error, transaction costs, liquidity, and the practical constraints that the underlying mathematics does not include on its own.

Robo-advisors show what happens when this whole toolkit is automated for a retail audience: a risk questionnaire maps to a point on the efficient frontier, not necessarily the minimum-variance point, rebalancing and tax-loss harvesting apply the chapter's practical-allocation and tax logic continuously and at scale, and machine learning contributes a genuinely distinct capability beyond the return- and covariance-estimation role described earlier in the chapter, learning a client's revealed risk tolerance from behavior rather than a one-time questionnaire answer.

The Schwab Intelligent Portfolios case showed directly that automating an allocation decision does not exempt it from the same governance and disclosure discipline Chapter 8 and Chapter 15 require of every other financial model.

Wulin.Suo@Queensu.ca

12.7 Suggested Exercises

1. Using the two-asset formula in Section 12.2.1, verify by hand that $w_A^{\text{MV}} = 34.05\%$ minimizes portfolio variance, and compute the resulting portfolio's expected return.

2. Two assets have $\sigma_X = 15\%$, $\sigma_Y = 25\%$, and $\rho_{XY} = 0.20$. Compute the minimum-variance portfolio weights and the resulting portfolio volatility.

3. Explain, using Figure 12.1, why a rational investor would never hold a portfolio on the dashed (inefficient) segment of the frontier, even if they were extremely risk-averse.

4. Using the data in Table 12.2, verify by hand that the estimated beta is approximately 1.19, and compute the CAPM-implied cost of equity assuming $r_f = 3\%$ and a market risk premium of 7%.

5. A portfolio has an average daily return of 0.05% and a daily standard deviation of 1.1%. Assuming a daily risk-free rate of approximately zero, compute its Sharpe ratio, and compare it to the Sharpe ratios computed in Section 12.2.1.

6. Three assets have volatilities of 10%, 20%, and 30%. Compute their risk parity (inverse-volatility) weights.

7. Explain, in your own words, why Michaud describes mean-variance optimization as an "estimation-error maximizer," referencing the three-asset example in Section 12.2.3.

8. Using Section 12.2.3's shrinkage formula and the three-asset covariance matrix of Section 12.2.3, recompute the minimum-variance weights at a shrinkage intensity of $\delta = 0.25$ rather than the $\delta = 0.5$ used in the text, and state whether they fall between the unshrunk weights and the $\delta = 0.5$ weights. Then explain, without computing anything further, what the weights must become as $\delta \to 1$, and why.

9. Describe a specific scenario in which an investor using Black-Litterman with no strong views would end up holding a different portfolio than one using naïve historical-mean mean-variance optimization, and explain why.

10. Using Section 12.3.6's equity/bond setup, recompute the Black-Litterman weights for an investor who believes equities will beat bonds by 7% rather than 9%, holding τ and Ω unchanged. Compare the resulting tilt away from 60/40 with the text's, and explain why a view only slightly above the equilibrium spread of 6.6365% moves the portfolio so little.

11. A portfolio manager rebalances a two-asset portfolio back to its 60/40 target weights every time the actual weight drifts by more than 5 percentage points. Explain, referencing Chapter 3's discussion of transaction costs, why this rule might outperform both a policy of rebalancing daily and a policy of never rebalancing at all.

12. Using Section 12.4.1's method, recompute the three-asset minimum-variance portfolio under a 50% (rather than 40%) position cap, and state whether the cap still binds on AssetA and, if so, by how much the resulting volatility differs from both the uncapped and the 40%-capped cases.

13. Explain, in your own words, why a numerical optimizer can terminate after a single iteration and report "success" on a badly scaled objective function, referencing Section 12.4.1's naive-versus-rescaled comparison, and describe one way to detect this failure mode besides comparing against a known closed-form answer.

14. Pick two of the challenges listed in this chapter and describe a concrete scenario, not already used in the text, in which that challenge could cause a mean-variance-optimized portfolio to perform poorly out of sample.

15. A portfolio earns returns of 1.5%, −0.5%, 2.0%, and 0.5% over four periods, while its benchmark earns 1.0%, 0.5%, 1.0%, and 0.5% over the same periods. Compute the tracking error and information ratio, and interpret the sign of the information ratio.

Wulin.Suo@Queensu.ca

16. A US investor holds a Japanese bond that returns 2% in yen terms while the yen appreciates 4% against the US dollar over the same period. Compute the unhedged US-dollar return, and explain why it differs from the bond's local-currency return.

17. Explain, referencing the hierarchical risk parity idea in this chapter, why clustering assets before optimizing might produce more stable portfolio weights than applying full mean-variance optimization to all assets simultaneously.

18. Using the Security Market Line in Figure 12.2, explain what it means for an asset to plot above the line, and describe how this relates to the nonzero alpha estimated in Section 12.3.1.

19. Explain why a factor's historically estimated return premium might weaken after it becomes the basis of a widely traded smart-beta product, referencing the market-efficiency discussion in Chapter 3.

20. Using the robo-advisor discussion's method, compute the 110-minus-age heuristic equity weight for a 50-year-old investor, and, using the same illustrative $\sigma_e = 16\%$, $\sigma_b = 5\%$, $\rho_{eb} = -0.20$, $r_e = 8\%$, $r_b = 3\%$ assumptions with $r_f = 2\%$, compute that portfolio's volatility, expected return, and Sharpe ratio.

21. A client holds a \$30,000 position that has fallen 8%. Using the robo-advisor discussion's tax-loss-harvesting method and a 24% marginal income-tax rate and 15% long-term capital-gains rate, compute the first-year tax savings from harvesting this loss.

22. Using the robo-advisor discussion's fee-drag example, recompute the 30-year future value of a \$100,000 initial investment at a 6% (rather than 7%) gross annual return for both the 0.25% and 1% fee cases, and state whether the dollar gap between them grows or shrinks relative to the text's example.

23. Explain, referencing Section 12.3.2's two-fund separation theorem, why a robo-advisor's practice of shifting a client's equity/bond mix based on risk tolerance is a departure from what that theorem, taken literally, would recommend.

24. Two assets have $\sigma_X = 12\%$, $\sigma_Y = 18\%$, $\rho_{XY} = 0.10$, $\mu_X = 9\%$, $\mu_Y = 6\%$, and $r_f = 1\%$. Using Section 12.3.2's formula, compute the tangency portfolio weights, and explain whether you would expect them to be closer to or further from the minimum-variance weights than the AssetA/AssetB example in the text, before actually computing the minimum-variance weights to check.

25. Explain, in your own words, why the tangency portfolio and the minimum-variance portfolio coincided so closely for AssetA and AssetB but not for the equity/bond example in Section 12.3.2, referencing the role r_f and the spread in expected returns each play in the tangency formula.

26. **Challenge (real data).** Download at least five years of monthly returns for five to ten real stocks (via `yfinance`) together with the Fama-French three-factor data for the same period (freely available from the Ken French Data Library). (a) Estimate each stock's CAPM beta by regressing its excess returns on the market factor, as in Section 12.3.1, and compare the range of estimated betas to the single illustrative beta of 1.19 used in this chapter. (b) Construct the minimum-variance and tangency portfolios from your assets' sample covariance matrix, as in Sections 12.2.1 and 12.3.2, and comment on whether the resulting weights look reasonable or exhibit the extreme, concentrated positions Michaud's "estimation-error maximizer" critique (Section 12.2.3) warns about. (c) Regress each stock's excess return on all three Fama-French factors (market, size, value) rather than the market alone, and discuss whether the market-only CAPM alpha from part (a) shrinks once size and value exposures are accounted for.

12.8 Further Reading

- Markowitz, "Portfolio Selection," *Journal of Finance*. The original 1952 paper introducing mean-variance optimization, foundational to nearly every method in this chapter.

Wulin.Suo@Queensu.ca

- Sharpe, "Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk," *Journal of Finance*. The original derivation of the Capital Asset Pricing Model used throughout Section 12.3.1.
- Fama and French, "Common Risk Factors in the Returns on Stocks and Bonds," *Journal of Financial Economics*. The original paper introducing the SMB and HML factors used in this chapter's multi-factor model example.
- Black and Litterman, "Global Portfolio Optimization," *Financial Analysts Journal*. The original treatment of the Bayesian view-blending approach summarized in Section 12.3.5.
- Grinold and Kahn, *Active Portfolio Management*. A practitioner-oriented treatment connecting mean-variance theory to real active-management decisions, including risk parity and other robust allocation methods introduced in this chapter.
- Bodie, Kane, and Marcus, *Investments*. A comprehensive textbook treatment of portfolio theory, CAPM, and factor models, extending this chapter's introductory coverage of each topic.
- Ang, *Asset Management: A Systematic Approach to Factor Investing*. A modern, practitioner-oriented synthesis of factor models, smart beta, and the practical allocation considerations, including illiquidity and rebalancing, discussed later in this chapter.
- Michaud and Michaud, *Efficient Asset Management*. The original source for the "estimation-error maximizer" critique of mean-variance optimization discussed in Section 12.2.3, along with resampling-based methods for producing more robust portfolios.
- Gu, Kelly, and Xiu, "Empirical Asset Pricing via Machine Learning," *The Review of Financial Studies*. The comprehensive machine learning bake-off behind Section 12.4.3, including the neural-network and tree results and the long-short portfolios built on them.
- Campbell and Thompson, "Predicting Excess Stock Returns Out of Sample: Can Anything Beat the Historical Average?" *The Review of Financial Studies*. The argument that a return forecast should be judged against the Sharpe ratio it produces rather than the variance it explains, which is what makes Section 12.4.3's tiny R^2 figures economically meaningful.
- McLean and Pontiff, "Does Academic Research Destroy Stock Return Predictability?" *The Journal of Finance*. The source of Section 12.4.3's post-publication decay estimates, and the clearest evidence that a return-forecasting target adapts to being modeled.
- Chopra and Ziemba, "The Effect of Errors in Means, Variances, and Covariances on Optimal Portfolio Choice," *Journal of Portfolio Management*. The source of Section 12.3.5's ordering of the three input-estimation errors by how much utility each destroys.
- Ledoit and Wolf, "Honey, I Shrunk the Sample Covariance Matrix," *Journal of Portfolio Management*. The standard practitioner reference for Section 12.2.3's shrinkage estimator, including the formula for choosing the shrinkage intensity δ optimally rather than fixing it by hand as the text's example does.
- D'Acunto, Prabhala, and Rossi, "The Promises and Pitfalls of Robo-Advising," *The Review of Financial Studies*. An empirical study of robo-advised accounts documenting the gap between clients' stated and revealed risk tolerance discussed in this chapter's treatment of robo-advisors.
- U.S. Securities and Exchange Commission, "Schwab Subsidiaries Misled Robo-Adviser Clients about Absence of Hidden Fees," press release 2022-104. The regulatory announcement underlying this chapter's Schwab Intelligent Portfolios case vignette.
- Berg, Kölbel, and Rigobon, "Aggregate Confusion: The Divergence of ESG Ratings," *Review of Finance*. The empirical study of cross-provider ESG rating disagreement underlying this chapter's ESG-tilt worked example.

Wulin.Suo@Queensu.ca

Chapter 13

Algorithmic Trading and Market Microstructure

13.1 Introduction

Every strategy considered so far in this book, from the DuPont-decomposed valuation of Chapter 4 to the mean-variance portfolios of Chapter 12, has treated buying and selling an asset as instantaneous and free, an assumption made deliberately so that the underlying financial or statistical idea could be presented without an additional layer of execution detail. Chapter 3 already showed this is not quite right: every trade costs at least half the bid-ask spread, and a large enough order walks through several levels of the order book at a progressively worse price. This chapter treats execution itself as a subject worth studying carefully, not a frictionless afterthought: how a trading signal is generated, how an order is actually worked in the market, how a strategy's historical performance should be evaluated, and how Chapter 3's market microstructure concepts shape every one of those decisions.

Several worked examples run through the chapter, each grounded in material introduced earlier in this book. A moving-average crossover strategy, backtested on a short synthetic price series, illustrates how a trading signal becomes a sequence of positions and, ultimately, a profit-and-loss series. The limit-order-book walk from Chapter 3 is extended into a comparison between aggressive, single-shot execution and a patient, multi-slice alternative, motivating the standard TWAP and VWAP execution benchmarks, and into a regression-based estimate of Kyle's lambda that gives price impact a formal economic interpretation.

The market-making intuition of Chapter 3 is developed into a fully worked Avellaneda-Stoikov inventory example, showing exactly how and why a market maker's quotes should skew as its position grows.

The pairs-trading setup introduced conceptually in Chapter 7 is extended into an actual backtested trade, entered and exited using the same z-score rule that chapter only described qualitatively, and then generalized into a multi-stock statistical arbitrage example built on Chapter 12's factor-regression machinery.

This chapter sits at a specific point in the book's overall structure that is worth making explicit. Chapters 6 and 7 supplied the statistical and machine learning tools capable of generating a trading signal in the first place; Chapter 12 supplied the portfolio-construction logic for combining many assets and signals into a coherent overall position; and this chapter supplies the missing link between the two, everything that happens after a model or a portfolio optimizer has decided what it wants to hold, and before that decision becomes a completed, real-world trade at a real, and imperfect, price.

Skipping this link, treating execution as instantaneous and free, is precisely the simplifying assumption every earlier chapter in this book has made for clarity, and this chapter exists specifically to show what that assumption costs when it is finally relaxed.

13.2 Trading Signals and a First Strategy

Every algorithmic strategy begins with a signal, a rule that turns observable data into a view about where a price is going. This part classifies the signals the industry actually trades and works one of the simplest

Wulin.Suo@Queensu.ca

all the way through, from raw prices to a realized return.

13.2.1 A Taxonomy of Trading Strategies and Signals

Suppose two traders both notice the exact same pattern in a stock's recent price history but reach opposite conclusions about what to do with it: one buys, betting a recent rise will continue; the other sells short, betting the same rise has already gone too far and must reverse. Both can be running a perfectly coherent strategy; they simply disagree about the economic source of the pattern each believes is in front of them.

Strategies are usually organized by this distinction, the economic source of the pattern they exploit. Momentum strategies bet that a recent price trend will continue, on the theory that information diffuses gradually through a market rather than being reflected in prices instantly. Mean-reversion strategies bet the opposite: that a price, or a relationship between two prices as in the pairs trade developed later in this chapter, has moved too far from some fundamental or statistical anchor and will revert toward it, an application of Chapter 7's stationarity concepts. Market-making strategies, discussed further in Section 13.4, take no directional view on price at all, profiting from repeatedly capturing the bid-ask spread while managing the resulting inventory risk. Statistical arbitrage generalizes mean reversion to many assets simultaneously, often using Chapter 12's factor models to identify a stable relationship worth betting will hold.

Every one of these strategy types ultimately produces the same basic output: a signal, a number (or simply a direction) indicating whether and how much to buy or sell at a given point in time. Converting that signal into an actual position, and that position into actual executed trades, is the specific concern of most of this chapter, and it is a step with its own economics, costs, and risks that are entirely separate from whether the underlying signal is any good in the first place.

13.2.2 A Worked Example: A Moving-Average Crossover Strategy

A simple moving-average crossover strategy, one of the oldest and most transparent momentum strategies, holds a long position whenever a short-window moving average is above a longer-window moving average, and is flat otherwise, on the theory that the short average crossing above the long average signals a strengthening upward trend. Table 13.1 applies a 3-day short average and a 6-day long average to twelve days of hypothetical prices.

Day	Price	MA(3)	MA(6)	Signal	Strategy Return
1	100	–	–	0	–
2	101	–	–	0	0.00%
3	103	101.33	–	0	0.00%
4	106	103.33	–	0	0.00%
5	110	106.33	–	0	0.00%
6	115	110.33	105.83	1	0.00%
7	119	114.67	109.00	1	3.48%
8	118	117.33	111.83	1	−0.84%
9	114	117.00	113.67	1	−3.39%
10	109	113.67	114.17	0	−4.39%
11	104	109.00	113.17	0	0.00%
12	100	104.33	110.67	0	0.00%

Table 13.1: A 3-day/6-day moving-average crossover strategy

The signal turns on (long) at day 6, when the 3-day average first exceeds the 6-day average, and turns off (flat) at day 10, when it first falls back below. Crucially, the strategy return in each row uses the *previous* day's signal applied to the *current* day's price return, since a signal computed from today's closing price cannot actually be traded until at least the next available price, which is the point-in-time discipline Chapter 6 emphasized when warning against data leakage; using today's signal against today's own return would silently look ahead.

Wulin.Suo@Queensu.ca

Compounding the strategy returns in the table gives a cumulative return of -5.22% over the twelve days, while simply buying and holding the asset from day 1 to day 12 (which starts and ends at a price of 100) returns exactly 0%. In this particular, deliberately illustrative sample, the crossover strategy actually underperforms a naive buy-and-hold position: it captures part of the days-6-through-8 uptrend but stays long one day too long into the days-9-and-10 reversal, precisely the whipsaw risk that afflicts every momentum strategy during a trend reversal. This is an intentionally honest example, not a cherry-picked success story: a real backtest evaluation, developed further in Section 13.6.1, must be prepared to find that a plausible-sounding strategy simply does not work on the available data.

13.3 Execution and Market Impact

A signal is only half a strategy. Turning a desired position into an actual one means sending orders into a market that moves in response to them, so this part develops the mechanics of execution, the cost of demanding liquidity too quickly, and the models that balance that cost against the risk of trading too slowly.

13.3.1 Order Types and the Mechanics of Execution

Chapter 3 introduced the market order (executed immediately at the best available price, walking through the book if necessary) and the limit order (executed only at a specified price or better, potentially never filling at all). Converting a trading signal like the one in Section 13.2.2 into an actual position requires choosing among these order types, and the choice involves a direct tradeoff: a market order guarantees execution but accepts whatever price impact results, while a limit order controls the execution price but risks not being filled at all if the market moves away before the order reaches the front of the queue. A stop order, a third common type, becomes a market order automatically once a specified trigger price is reached, commonly used to implement a predetermined exit rule (a stop-loss) without requiring continuous manual monitoring of the position.

Beyond the order type itself, an algorithmic execution system must decide how to route an order, since modern equity markets consist of many competing exchanges and alternative trading venues rather than a single centralized order book, a fragmentation Chapter 3 touched on in its discussion of Regulation NMS. A smart order router evaluates the available liquidity and price across every venue simultaneously and splits or directs an order to capture the best aggregate execution, a considerably more complex problem than the single-order-book walk introduced in Chapter 3, but built on the same underlying economics of price and available size at each level.

A large order also raises a genuine tension that the order types above do not fully resolve: posting the full order as a visible limit order reveals the trader's intentions to every other market participant, who can then adjust their own behavior in response, for example by front-running the anticipated remaining demand. Iceberg orders address this by displaying only a small portion of the total order size at any given time, automatically revealing more once the visible portion is filled, so that the order book shows only a fraction of the true remaining size. Dark pools take this further still, entirely private trading venues that match buy and sell orders without displaying any pre-trade price or size information to the broader market, only reporting completed trades after the fact.

Both mechanisms trade away pre-trade transparency, one of the market-quality benefits Chapter 3 attributed to well-functioning exchanges, in exchange for reduced information leakage and, in principle, lower market impact for large orders; this is a genuine tradeoff rather than a free improvement, since a market with a large fraction of its volume trading in dark, non-displayed venues can make the lit, publicly visible order book a less complete and less reliable picture of overall supply and demand, a concern regulators have studied closely as dark-pool trading volume has grown.

Figure 13.1 places these decisions in sequence. A signal is generated, an order type is chosen, a venue is selected, and the resulting execution is measured, with that measurement feeding back into how the next signal is traded. That loop is what the rest of this chapter works through one stage at a time, and every stage in it introduces costs that the signal itself never sees.

Wulin.Suo@Queensu.ca

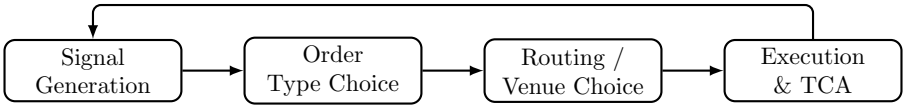


Figure 13.1: The path from a trading signal to a completed, evaluated trade: each stage introduces its own decisions and costs, and transaction cost analysis at the end feeds back into how future signals and execution choices are made

13.3.2 Market Impact and the Cost of Trading Fast

Chapter 3's limit-order-book example showed that a 250-share market order, larger than the 100 shares available at the best ask of \$50.10, walked through a second price level and executed at an average price of \$50.13, a price impact of about 6 basis points relative to the best ask. This cost arises entirely from the size of the order relative to the liquidity available at the time; it is not a fee charged by anyone, but the direct consequence of consuming the available supply at each successive price level.

An alternative to executing the entire order at once is to break it into smaller pieces and space them out over time, allowing the order book to replenish between slices. Suppose the same 250-share order is instead split into five 50-share slices, each small enough to execute entirely within the 100 shares available at the best ask in Chapter 3's example, and suppose the best ask remains at \$50.10 between slices (a simplifying assumption; in reality, the book may or may not fully replenish, and the price may drift for reasons entirely unrelated to this trader's own activity).

This patient approach achieves an average execution price of exactly \$50.10, eliminating the price impact entirely, but at a cost the single-shot execution does not bear: over the time it takes to work five separate slices, the underlying price can move for entirely unrelated reasons, a risk called timing risk or execution risk. The aggressive, single-shot strategy pays a known, certain price-impact cost immediately; the patient, multi-slice strategy pays an expected impact cost of zero but accepts an uncertain, and potentially much larger or smaller, timing-risk cost. Neither strategy is universally superior; which one a trader should prefer depends on the size of the order relative to available liquidity, the asset's volatility, and how urgently the position needs to be established, which is the tradeoff formalized in Section 13.3.5.

13.3.3 The Kyle Model: A Formal Model of Price Impact

Chapter 3's order-book walk demonstrates price impact mechanically, one order consuming successive price levels, but does not explain *why* a market maker sets prices that respond to order flow in the first place. Kyle's (1985) classic model supplies the economic explanation: a market maker cannot distinguish an order coming from an informed trader (who trades because they know something about the asset's true value) from one coming from a liquidity trader (who trades for reasons unrelated to information, such as needing cash), and must set a single price schedule that, on average, breaks even against this mixture. The market maker's rational response is to move the price in proportion to the net signed order flow it observes,

$$\Delta P = \lambda \times (\text{Net Order Flow}) + \varepsilon,$$

where net order flow is buy volume minus sell volume over some interval and λ (Kyle's lambda) measures the price impact per unit of net order flow, the same linear-regression structure used throughout this book, here estimated by regressing observed price changes on observed net order flow.

Table 13.2 shows six intervals of net order flow (in thousands of shares) and the resulting price change (in cents).

Applying the same OLS formula used since Chapter 6,

$$\hat{\lambda} = \frac{\sum_t (\text{Flow}_t - \overline{\text{Flow}})(\Delta P_t - \overline{\Delta P})}{\sum_t (\text{Flow}_t - \overline{\text{Flow}})^2} \approx 1.95 \text{ cents per thousand shares}, \qquad R^2 \approx 0.99.$$

A larger estimated λ indicates a market in which prices move more sharply for a given amount of net buying or selling pressure, which Kyle's model interprets as a market in which informed trading is a larger fraction of

Wulin.Suo@Queensu.ca

Interval	1	2	3	4	5	6
Net order flow (000s)	5	−3	8	−6	2	4
Price change (cents)	10	−7	15	−11	3	9

Table 13.2: Net order flow and price change for estimating Kyle's lambda

total order flow, or in which liquidity (the depth available to absorb order flow without moving the price, the order-book depth introduced in Chapter 3) is thinner. This gives price impact, treated so far in this chapter as a mechanical consequence of a specific order book, a deeper economic interpretation: it is compensation the market maker requires for the risk of unknowingly trading against better-informed counterparties, the same adverse-selection component of the spread Chapter 3 first introduced, now expressed as a single estimable slope coefficient rather than only a qualitative concept.

One caution about that slope is worth stating early, because everything built on it inherits the problem: $\hat{\lambda}$ is estimated here as a single constant, and it is not one. Impact per unit of order flow varies with the time of day, widest around the open and narrowest in the quiet middle of the afternoon; with the prevailing volatility regime; and with order size itself, since a larger order walks into progressively thinner depth. A production pre-trade cost model therefore fits a conditional impact surface, regressing realized impact on order size, participation rate, quoted spread, volatility, and time of day, which is Chapter 6's supervised-learning setup pointed at execution rather than at a trading signal. Kyle's constant-λ result makes the economics legible; the fitted surface is what a desk actually budgets against.

13.3.4 TWAP and VWAP: Basic Execution Algorithms

The patient, multi-slice approach in Section 13.3.2 is a simple version of the time-weighted average price (TWAP) algorithm: split an order into equal-sized slices spread evenly across a time horizon, without reference to how trading volume is actually distributed over that horizon. Table 13.3 shows five hypothetical intervals with market prices and total trading volume.

Interval	1	2	3	4	5
Price	\$50.00	\$50.05	\$50.10	\$50.08	\$50.12
Market volume	10,000	12,000	8,000	15,000	9,000

Table 13.3: Prices and market volume over five execution intervals

A trader executing a 1,000-share order via TWAP trades 200 shares in each interval regardless of volume, achieving an average execution price equal to the simple average of the five prices,

$$\text{TWAP execution price} = \frac{50.00 + 50.05 + 50.10 + 50.08 + 50.12}{5} = \$50.07.$$

The volume-weighted average price (VWAP) benchmark, by contrast, weights each interval's price by that interval's share of total market volume,

$$\text{VWAP} = \frac{\sum_t P_t V_t}{\sum_t V_t} = \frac{50.00(10{,}000) + \cdots + 50.12(9{,}000)}{10{,}000 + 12{,}000 + 8{,}000 + 15{,}000 + 9{,}000} \approx \$50.068,$$

close to, but not identical to, the naive TWAP execution price, a difference of about 0.4 basis points in this example. The gap turns on which intervals carried the volume, not on which carried the most: the two most expensive intervals, the third at \$50.10 and the fifth at \$50.12, drew the least volume of the five, 8,000 and 9,000 shares, so volume weighting puts the least weight exactly where the price was worst for a buyer. Interval 4 carries the largest volume of all, 15,000 shares, but at \$50.08 it sits a cent *above* the TWAP average and so nudges the weighted price the other way. A VWAP-following execution algorithm would therefore have traded proportionally less in those two expensive intervals than the equally weighted TWAP schedule did, capturing a slightly better average price. In practice, a VWAP execution algorithm

Wulin.Suo@Queensu.ca

typically trades according to a historical intraday volume profile, since the actual volume for the current day is not known in advance. It attempts to match the market's own volume distribution rather than the simpler, uniform TWAP schedule. The theory is that trading in proportion to when the market is naturally most liquid minimizes the executing trader's own footprint and price impact.

That last sentence quietly contains a forecasting problem, and it is the hard part of the algorithm rather than the arithmetic above. Trading to VWAP requires knowing how the day's volume will be distributed before the day is over, and a historical average profile is the crudest possible forecast of it. Volume departs from that profile for reasons partly predictable in advance: an index rebalance, a scheduled earnings release, an options expiry, or simply an unusually heavy opening auction all bend the curve in ways a static profile cannot anticipate. Predicting the intraday volume curve is therefore a supervised regression problem in exactly Chapter 7's sense, with the day's own realized volume so far as the single most informative feature, and it demands the same walk-forward evaluation. An execution desk treats this forecast, not the weighting formula, as where a VWAP algorithm is actually won or lost.

13.3.5 Optimal Execution: Balancing Impact and Timing Risk

Section 13.3.2's comparison between an aggressive, single-shot execution and a patient, multi-slice alternative is a simplified version of the optimal execution problem formalized by Almgren and Chriss. Given an order of a fixed size that must be completed by a fixed deadline, how should it be split over time to minimize the combination of expected market impact cost and the variance introduced by timing risk? Market impact cost is typically modeled as increasing in the trading rate, since trading faster within any given interval consumes more of the available liquidity at each price level, just as in Chapter 3's order-book walk. Timing risk, by contrast, is modeled as increasing in the total time the order remains unexecuted, since a longer execution horizon exposes the remaining unexecuted quantity to more periods of potentially adverse price movement, using the same variance-grows-with-horizon logic Chapter 7 developed for multi-step forecasting.

The key output of this framework is not a single number but an efficient frontier directly analogous to the risk-return efficient frontier of Chapter 12. For a given level of acceptable timing risk, there is a specific optimal trading trajectory that minimizes expected execution cost, typically front-loaded, trading more aggressively early and tapering off, rather than either fully aggressive or perfectly uniform. A more risk-averse trader, one who weights the variance of the outcome more heavily relative to its expected cost, will choose a faster, more front-loaded execution than a less risk-averse trader facing the identical order and market conditions.

This formalizes, with actual mathematical structure, the intuitive tradeoff Section 13.3.2 described only qualitatively: neither the purely aggressive nor the purely patient extreme is generally optimal, and the right answer depends on quantities, market impact sensitivity and price volatility, that must themselves be estimated from data, with all the estimation-error caveats Chapter 12 raised about any input estimated from a limited historical sample.

13.3.6 A Worked Example: The Almgren-Chriss Optimal Trajectory

Almgren and Chriss formalize the tradeoff above by minimizing a risk-adjusted total cost, expected temporary-impact cost plus ϕ times its variance from timing risk, where $\phi \geq 0$ is the trader's risk-aversion parameter (written ϕ here rather than the λ of Almgren and Chriss's own paper, to keep it distinct from Kyle's lambda above),

$$\min_{x_1,\ldots,x_{N-1}} \underbrace{\eta \sum_{j=1}^{N} \frac{(x_{j-1}-x_j)^2}{\tau}}_{\text{expected impact cost}} + \phi \underbrace{\sigma^2 \tau \sum_{j=1}^{N} x_j^2}_{\text{timing-risk variance}},$$

where x_j is the number of shares still unexecuted after interval j (so $x_0 = X$, the full order, and $x_N = 0$, fully executed), τ is the length of each interval, η is the temporary-impact cost coefficient, and σ^2 is the per-interval return variance. The closed-form solution to this minimization is

$$x_j = X \frac{\sinh(\kappa(T - t_j))}{\sinh(\kappa T)}, \qquad \kappa = \sqrt{\frac{\phi \sigma^2}{\eta}},$$

Wulin.Suo@Queensu.ca

with $T = N\tau$ the total execution horizon; κ is the single parameter that governs how front-loaded the optimal trajectory is, growing with risk aversion ϕ and volatility σ and shrinking with the impact cost η. The trajectory is stated here without proof; this book's companion technical note on the Almgren-Chriss model derives it from the discrete first-order condition, which turns out to be a linear second-order difference equation, and verifies the two limiting cases that make the result trustworthy: uniform, TWAP-style trading as risk aversion vanishes, and immediate liquidation as it grows without bound.

Applying this to Section 13.3.2's 250-share order split across the same $N = 5$ intervals, with illustrative parameters $\sigma = 2\%$ per interval, $\eta = 0.01$, and a modest risk aversion of $\phi = 0.5$ (so $\kappa \approx 0.1414$), the optimal trajectory leaves

$$\begin{aligned} \text{Unexecuted shares:} \quad & x = (250,\ 194.2,\ 142.4,\ 93.4,\ 46.2,\ 0), \\ \text{Traded per interval:} \quad & (55.8,\ 51.9,\ 49.0,\ 47.1,\ 46.2), \end{aligned}$$

mildly front-loaded relative to the perfectly uniform 50-share-per-interval TWAP schedule of Section 13.3.2. Raising risk aversion tenfold to $\phi = 5$ (so $\kappa \approx 0.4472$) sharpens this front-loading considerably,

$$\text{Traded per interval:} \quad (92.8,\ 60.9,\ 41.3,\ 30.1,\ 25.0),$$

nearly double the first-interval quantity of the low-risk-aversion trajectory: a more risk-averse trader accepts a higher expected impact cost from trading more aggressively up front specifically to shrink the variance contributed by however much of the order remains exposed to further price moves. Figure 13.2 plots both optimal trajectories against the uniform TWAP baseline, making the front-loading pattern, and how much more pronounced it becomes as ϕ rises, directly visible.

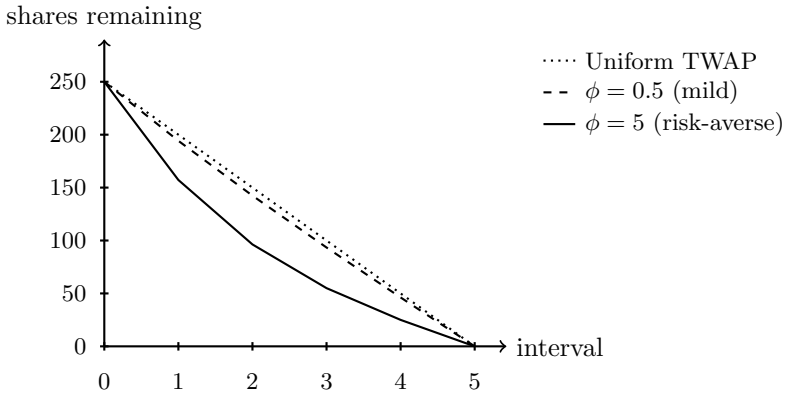


Figure 13.2: Almgren-Chriss optimal unexecuted-share trajectories at two risk-aversion levels, against the uniform TWAP baseline

Scoring all three strategies, this front-loaded optimal trajectory, the uniform TWAP schedule, and the fully aggressive single-shot execution of Section 13.3.2, on the identical risk-adjusted objective at $\phi = 0.5$ makes the tradeoff numerically concrete:

$$\begin{aligned} \text{Optimal trajectory:} \quad & \text{cost} = 139.4, \\ \text{Uniform TWAP:} \quad & \text{cost} = 140.0, \\ \text{Fully aggressive single-shot:} \quad & \text{cost} = 625.0, \end{aligned}$$

the TWAP schedule barely worse since $\phi = 0.5$ is only mildly risk-averse here, and the fully aggressive execution over four times worse, since it eliminates timing-risk variance entirely but pays a quadratically larger impact cost for trading the entire order in one interval. This is the efficient-frontier logic described qualitatively above, made concrete: the fully aggressive and perfectly uniform schedules are both feasible points, but neither is optimal once risk aversion is weighed formally against expected cost, and the specific front-loaded shape of the true optimum, more pronounced the more risk-averse the trader, is not a generic property of "spreading an order out" but the precise output of this closed-form solution.

Wulin.Suo@Queensu.ca

It is worth being explicit about what that closed form costs, because the assumptions are stronger than the elegance of the result suggests. The trajectory above is optimal only if impact is linear in the trading rate, volatility is constant across the horizon, and the trader has no view whatever on where the price is heading. All three are false in ways any execution desk would recognize, and relaxing any single one destroys the closed form entirely. What survives is the problem rather than the solution: choosing a sequence of trade sizes to minimize expected cost plus a penalty on risk is a sequential decision problem, solvable by learning a policy from simulated episodes instead of by solving a difference equation. Chapter 19 does precisely this, recovering the trajectory above for these same parameters using tabular Q-learning, then pushing the machinery into frictions the closed form has no way to express.

13.4 Inventory Models and Market Making

Chapter 3 introduced market making conceptually: a market maker profits from the bid-ask spread by standing ready to both buy and sell, but bears inventory risk from holding a non-zero position while waiting for an offsetting order to arrive. The Avellaneda-Stoikov model formalizes this by having the market maker continuously adjust both quotes based on current inventory, not just on the underlying asset's fair value. A market maker who has accumulated a large long position, from selling less than buying, should skew both the bid and ask downward: reducing the bid makes it relatively less attractive for other traders to sell to the market maker, and lowering the ask makes it relatively more attractive to buy from it. That actively works the existing inventory back toward zero rather than passively waiting for random order flow to do so.

The model formalizes this skew through a reservation price, the price at which the market maker would be indifferent to a marginal trade given its current inventory, which sits below the true mid price when the market maker is long and above it when short,

$$r = s - q\,\gamma\,\sigma^2\,(T - t),$$

where s is the current mid price, q is the market maker's current inventory (positive if long, negative if short), γ is a risk-aversion parameter, σ^2 is the asset's return variance, and $T - t$ is the time remaining in the market maker's trading horizon. The optimal bid and ask are then centered on this reservation price, not on the mid price itself, and separated by an optimal spread

$$\delta = \gamma\sigma^2(T - t) + \frac{2}{\gamma}\ln\left(1 + \frac{\gamma}{\kappa}\right),$$

where κ governs how sensitively order arrival rates respond to how competitively the market maker is quoting. Consider a market maker with mid price $s = \$100$, currently long $q = 5$ units, $\gamma = 0.1$, $\sigma = 2$, one full unit of time remaining ($T - t = 1$), and $\kappa = 1.5$. The reservation price is $r = 100 - 5(0.1)(2^2)(1) = \$98.00$, two dollars below the mid price, reflecting the market maker's desire to reduce its long position. Both halves of the spread formula above are quoted rather than derived; this book's companion technical note on the Avellaneda-Stoikov model derives each of them separately from first principles, the reservation price from exponential (CARA) utility applied to a normally distributed terminal mark-to-market value, and the per-side markup from the Poisson arrival intensity, which is what makes the total spread decompose into an inventory-risk term that vanishes as the horizon closes and an order-arrival term that does not. The optimal spread works out to $\delta \approx \$1.69$, giving

$$\begin{aligned} \text{Inventory-aware quotes:} &\quad \text{bid} = r - \delta/2 \approx \$97.15,\ \text{ask} = r + \delta/2 \approx \$98.85, \\ \text{Naive quotes (ignoring inventory):} &\quad \text{bid} = \$99.15,\ \text{ask} = \$100.85, \end{aligned}$$

the inventory-aware quotes both exactly \$2.00 lower, the reservation-price adjustment carried straight through to each side, making the market maker's ask more attractive to a potential buyer (encouraging trades that reduce the long position) and its bid less attractive to a potential seller (discouraging trades that would increase it further), all while preserving the same \$1.69 spread width and therefore the same expected compensation per completed round-trip trade.

This inventory-skewing behavior connects directly to the economic decomposition of the bid-ask spread introduced in Chapter 3: the inventory-holding cost component of the spread is the compensation a market

Wulin.Suo@Queensu.ca

maker requires for the risk that its current inventory, positive or negative, moves against it before an offsetting trade arrives, and the Avellaneda-Stoikov framework makes that intuition mathematically precise by deriving the specific optimal quote adjustment as a function of current inventory, remaining time horizon, and the asset's volatility. Notice, too, that the skew grows with γ, σ^2, and the size of the inventory position q itself: a more risk-averse market maker, a more volatile asset, or a larger accumulated position all call for a more aggressive skew, which is just the intuition that inventory risk becomes more urgent to offload when it is larger or more dangerous to hold. A market maker who ignores inventory risk entirely, quoting a fixed spread around fair value regardless of current position, is exposed to the kind of accumulating, unhedged directional risk that a well-designed inventory model exists to control, an application-specific instance of the broader risk management discipline developed in Chapter 8.

The caveat attached to Section 13.3.5 applies here too, and for the same underlying reason. Avellaneda-Stoikov is solvable in closed form because order arrivals are Poisson with a fixed intensity, volatility is constant, and the only state that matters is current inventory together with time remaining. A market maker in a real book faces arrival rates that cluster rather than arrive independently, counterparties whose order flow carries information that the model's symmetric arrivals cannot represent at all, and a choice of venue layered on top of the choice of quote. Learned quoting policies exist to handle exactly these, which is why Chapter 19 rebuilds this model as a reinforcement learning problem rather than treating it as finished. The closed form is still the right thing to meet first: it explains *why* quotes should skew toward inventory and by how much, which is a question no learned policy will answer for you.

13.5 Arbitrage and Relative-Value Strategies

Momentum and market making both take a view on a single instrument. A third family instead trades the relationship between instruments, holding offsetting positions so that the strategy profits from relative mispricing rather than from market direction.

13.5.1 A Worked Example: A Pairs-Trading Backtest

Chapter 7 introduced pairs trading conceptually using six days of prices for two hypothetical bank stocks, finding a spread mean of $\bar{s} = 2.5$, a standard deviation of $\hat{\sigma}_s = 0.548$, and a final z-score of 0.91, too small to trigger a trade under the common rule of entering when $|z| > 2$. Extending that same series four more days, with new prices for Stock A of \$58, \$56.50, \$54.80, and \$54.50, and for Stock B of \$54, \$53, \$52, and \$52, produces the z-scores in Table 13.4, computed using the mean and standard deviation estimated from the original six days (an out-of-sample application of parameters estimated in-sample, the walk-forward discipline of Chapter 7).

Day	7	8	9	10
Spread $(A - B)$	4.00	3.50	2.80	2.50
z-score	2.74	1.83	0.55	0.00

Table 13.4: Out-of-sample spread and z-score for the extended pairs-trading example

Day 7's z-score of 2.74 exceeds the entry threshold of 2, triggering a trade: short 1 share of Stock A and long 1 share of Stock B (the $\beta \approx 1$ simple-spread convention Chapter 7 used), betting that the unusually wide spread will revert toward its historical mean. By day 10, the spread has fully reverted to 2.50, almost exactly the in-sample mean, triggering an exit.

The trade's profit is simply the decline in the spread over the holding period,

$$\text{P\&L} = \text{Spread}_{\text{entry}} - \text{Spread}_{\text{exit}} = 4.00 - 2.50 = \$1.50 \text{ per share-pair},$$

since shorting the spread profits exactly in proportion to how much it narrows. Relative to the capital required to establish the position (the entry prices of the two legs, $58 + 54 = \$112$), this represents a return

Wulin.Suo@Queensu.ca

of approximately 1.34% over the three-day holding period. This single trade illustrates the complete pairs-trading lifecycle introduced only conceptually in Chapter 7: estimating the relationship in-sample, monitoring it out-of-sample, entering when the spread deviates unusually far from its estimated mean, and exiting once it reverts. One caveat, raised in Chapter 7 and worth repeating here, is that this entire strategy depends on the cointegrating relationship remaining stable. A single successful trade in an illustrative four-day extension is not evidence that the relationship will continue to hold indefinitely.

13.5.2 Statistical Arbitrage Beyond Pairs

The two-stock pairs trade of Section 13.5.1 generalizes naturally once more than two assets are involved, using the factor-model machinery of Chapter 12 rather than a simple two-asset regression. Instead of finding a single hedge ratio β between two specific stocks, a statistical arbitrage strategy regresses each stock in a universe on a common set of factors, the market, size, and value factors of Chapter 12's multi-factor model, or a larger set statistically extracted by principal component analysis, and treats the regression residual, the return unexplained by any common factor, as the object to trade. A stock whose residual has drifted unusually far from its own historical mean is a candidate to trade against a basket of peers, in the same mean-reversion spirit as the two-stock trade but diversified across many such residual bets simultaneously.

A concrete example makes this precise. Consider three stocks, W, X, and Y, in the same industry sector, each regressed on the same six months of market excess returns used for the CAPM beta regression in Chapter 12. Using the first five months to estimate each stock's beta and residual volatility, the CAPM regression approach Chapter 12 introduced, Table 13.5 shows each stock's estimated beta and its month-6, out-of-sample residual z-score.

Stock	Estimated β	Month-6 residual	z-score
W	0.98	+1.26%	+13.3
X	1.21	−0.15%	−1.8
Y	0.81	+0.09%	+1.1

Table 13.5: Estimated betas and out-of-sample residual z-scores for three sector peers

Stocks X and Y behave as their historical betas would predict, with residuals well within the range of ordinary in-sample noise. Stock W, by contrast, returned far more than its beta of 0.98 and the month's 3% market move would explain, a residual z-score of over 13, an extraordinarily large deviation that strongly suggests either genuine, stock-specific news (in which case the move may be permanent and should not be traded against) or a temporary, unexplained dislocation (in which case a statistical arbitrage desk would short Stock W against a long position in a market-neutral basket of its sector peers, betting the residual reverts).

This is precisely the multi-asset generalization of the two-stock pairs trade in Section 13.5.1: rather than trading one stock's price against one other specific stock's price, the position is effectively long the sector and short this one stock's excess idiosyncratic move relative to it, diversifiable across many such residual signals computed the same way across an entire universe of stocks simultaneously.

This generalization changes the risk profile of the strategy in an important way. A single pairs trade is exposed to the idiosyncratic risk that this particular cointegrating relationship breaks down, the risk flagged in Chapter 7 and reiterated above; a statistical arbitrage book trading hundreds of such residual relationships simultaneously diversifies away much of that single-relationship risk, in the same portfolio-level sense Chapter 12 developed for ordinary asset returns, provided the individual residual bets are not all driven by some common, unmodeled factor that would make them break down together.

This last caveat is not hypothetical. Exactly this kind of correlated breakdown across many simultaneously held statistical arbitrage positions was a central feature of the August 2007 "quant quake", when several large, otherwise unrelated quantitative equity funds experienced simultaneous, severe losses over the course of a few days. It was widely attributed to many funds holding similar statistical arbitrage positions and being forced to unwind them at the same time, causing the kind of crowding-driven, mutually reinforcing

Wulin.Suo@Queensu.ca

price move that Chapter 12's discussion of factor crowding described in the context of long-term portfolio allocation rather than short-term trading.

13.6 Evaluating a Strategy

A strategy that looks profitable on paper has cleared a much lower bar than one that is genuinely tradable. This part develops the metrics that summarize a backtest, the traps that make a backtest lie, and the post-trade accounting that reveals what the strategy actually cost to run.

13.6.1 Performance Metrics

Evaluating a backtested strategy, whether the moving-average crossover of Section 13.2.2 or the pairs trade of Section 13.5.1, requires more than a single cumulative return figure. The Sharpe ratio, introduced in Chapter 12, applies directly to a strategy's own return series, dividing its mean return by its return volatility; for the moving-average strategy's eleven return periods (mostly zeros, since the strategy is flat for seven of them), the mean return is -0.47% against a sample standard deviation of 2.03%, giving a Sharpe ratio of approximately -0.23 and reflecting the strategy's negative average return over this sample.

Maximum drawdown, introduced in Chapter 8, measures the largest peak-to-trough decline in cumulative value over the backtest period, a risk measure that a return-only or even a Sharpe-ratio summary can miss entirely. The moving-average strategy's cumulative return path peaks at a 3.48% gain (after day 7) before falling to a cumulative loss of 8.41% (by day 10), so its maximum drawdown is 8.41%. That is a materially different, and arguably more decision-relevant, number than the final -5.22% cumulative return alone, since it reveals how much value an investor holding the strategy would have seen erode from the strategy's own peak, not just where the strategy ultimately ended up.

The hit rate, the fraction of active (non-flat) trading days that were profitable, is 25% for the moving-average strategy, one profitable day out of four active days. It is a useful diagnostic precisely because a strategy can have a positive overall Sharpe ratio with a hit rate below 50% if its winning trades are, on average, sufficiently larger than its losing ones, or vice versa. Hit rate alone, like accuracy alone in Chapter 6's classification metrics, can be a misleading summary of a strategy's economic value when considered in isolation from the size of the gains and losses it produces.

13.6.2 Common Backtesting Pitfalls

Beyond choosing the right performance metrics, a credible backtest must also avoid several data-related traps that can make a strategy look far better than it would have actually performed in real time. Survivorship bias, introduced in Chapter 1 as a general data-quality concern, is especially dangerous in strategy backtesting: testing a strategy only on stocks that are still trading today silently excludes every company that went bankrupt or was delisted, which is precisely the population a well-designed strategy most needs to be tested against, since avoiding future disasters is at least as economically valuable as picking future winners.

Look-ahead bias occurs whenever a backtest uses information that would not actually have been available at the time a trading decision was made, for example using a company's finalized, restated financial statements rather than the preliminary figures that were actually public at the time, or, in this chapter's own moving-average example, accidentally using today's signal against today's own return rather than lagging it by one period as Section 13.2.2 was careful to do.

Both biases share the same root cause as the data leakage problem Chapter 6 warned about in a machine learning context: information from outside the genuinely available information set silently improves backtested performance in a way no real-time trading process could ever have replicated, and both require the same discipline to avoid, careful, skeptical attention to exactly what data would have been known, and in what form, at each historical point in time the backtest simulates a decision.

13.6.3 A Worked Example: The Cost of Searching

Both pitfalls above are properties of the data. The third, and in practice the most damaging, is a property of the researcher. This section opened by computing a Sharpe ratio of -0.23 for the 3-day/6-day crossover

Wulin.Suo@Queensu.ca

built in Section 13.2.2, and almost nobody stops there: the natural next step is a 5-day/20-day pair, then a 10-day/50-day pair, then a stop-loss rule, reporting whichever combination came out best. That is not a backtest of one strategy. It is a search across many, and the figure it reports is the maximum of many noisy estimates rather than any single strategy's edge.

How damaging that is turns on a prior question: how far from zero can a worthless strategy's Sharpe ratio wander on luck alone? Any estimate from a finite sample differs from what it estimates, and the typical size of that gap is its *standard error*, not a mistake but the price of having watched a random process for a finite time. For a strategy with no edge over T periods, the measured Sharpe ratio is centered on zero with a standard error of $1/\sqrt{T}$ per period, which on annualizing becomes

$$\text{standard error of the annualized Sharpe ratio} \approx \frac{1}{\sqrt{\text{years of data}}}.$$

The observation count cancels out entirely, leaving only calendar span: re-running a five-year backtest on five-minute bars multiplies the observations by about seventy-eight and buys no extra ability to tell an edge from luck, because it shrinks the per-period Sharpe ratio by that same factor. Only a longer history helps. This book's companion technical note on the sampling distribution of the Sharpe ratio derives both results from the fact that a measured Sharpe ratio is the ordinary one-sample t-statistic divided by $\sqrt{T}$.

Five years of daily data therefore gives a standard error of 0.447. Testing one strategy against it is Chapter 6's hypothesis test, and at the conventional one-sided 5% level the figure must clear 1.645 standard errors, the distance beyond which only 5% of a bell curve lies, or $1.645 \times 0.447 = 0.74$; one worthless strategy in twenty does. Search N configurations, though, and the reported number is no longer one draw but the largest of N, which lands further out the more draws are taken, without ceiling. Table 13.6 gives that expected best in standard errors, the same figure as a Sharpe ratio, and the *honest* hurdle the winner must beat for the search *as a whole* to be significant. The naive 0.74 controls the chance that one pre-specified strategy looks good by luck; the honest hurdle controls the chance that *any* of the N does.

Configurations N	Expected best, in standard errors	Expected best Sharpe	Honest 5% hurdle
1	0.00	0.00	0.74
10	1.54	0.69	1.15
100	2.51	1.12	1.47
1,000	3.24	1.45	1.74

Table 13.6: Best-of-N in-sample Sharpe ratios when every strategy searched is worthless, five years of daily data

Take $N = 100$: five short windows, five long windows, and four exit rules, a search most researchers would call modest. The expected best in-sample annualized Sharpe ratio is 1.12 though every one of the hundred is worthless by construction, and a Monte Carlo of ten thousand such searches averages 1.118 against the analytic 1.121. That clears the naive 0.74 hurdle comfortably and would be presented in most settings as a strong result, whereas the honest hurdle for the best of a hundred is 1.47, almost exactly twice. Out of sample the question is settled: the winner earns a mean Sharpe ratio of +0.004, loses money outright 49.6% of the time, and the in-sample to out-of-sample rank correlation across the hundred is +0.001. The winner of a search is not better than what it beat. It is luckier, and luck does not persist into the next sample.

Three responses follow, and none of them is to stop searching. Report N, since a backtest presented without the number of configurations tried cannot be evaluated at all. Compare the reported figure against the best-of-N hurdle rather than against zero, which is what the deflated Sharpe ratio of Bailey and López de Prado does, discounting a reported figure by however much a search of that size would be expected to inflate it and adjusting for the non-normality of real strategy returns. And recognize that neighboring grid points are highly correlated, since a 49-day and a 50-day moving average produce nearly identical positions, so the number of *genuinely independent* trials sits well below the raw count: correlating every pair at 0.5 lowers the mean best Sharpe ratio to 0.80, what about eighteen independent trials would give rather than a hundred.

What none of it can manufacture is data: for a strategy whose *true* annualized Sharpe ratio is 0.5, a genuine and respectable edge, separating it from zero in a single pre-specified test takes about eleven years

Wulin.Suo@Queensu.ca

of daily returns, and clearing a hundred-configuration search takes about forty-three. Very little strategy research runs on anything approaching that history, which is the strongest argument available for pre-specifying a few economically motivated candidates, as Section 13.2.1 framed them, rather than letting a grid search decide what the hypothesis should have been.

13.6.4 Implementation Shortfall and Transaction Cost Analysis

The gap between a strategy's theoretical performance, computed from the prices a signal was based on, and its actual realized performance, computed from the prices at which trades were genuinely executed, is called implementation shortfall, Execution Price − Decision Price. For the TWAP execution in Section 13.3.4, if a portfolio manager's decision to buy was made when the price stood at \$50.00 (the decision price) but the resulting order was ultimately executed via TWAP at \$50.07, the implementation shortfall is \$50.07 − \$50.00 = \$0.07 per share, 14 basis points, or \$70 in total on the 1,000-share order. This single number bundles together every friction discussed in this chapter: the market impact of Section 13.3.2, the timing risk of Section 13.3.5, and any pure price drift between the decision and its execution unrelated to the trade itself. It is the standard metric used in transaction cost analysis (TCA) to judge whether a trading desk's execution is adding or subtracting value relative to the investment decisions it implements. A strategy that looks profitable on decision prices alone can prove unprofitable once realistic implementation shortfall is subtracted, which is why serious backtesting, extending the performance metrics of Section 13.6.1, should model transaction costs explicitly rather than assume execution at the exact prices a signal was computed from.

13.7 Machine Learning and Event-Driven Signals

Every signal in this chapter so far, the moving-average crossover and the pairs-trading z-score, is a simple, fully interpretable rule with only a handful of parameters. The classification and regression methods of Chapter 6 can generate considerably richer ones: a gradient-boosted tree or neural network trained on a large set of engineered features, the lagged returns, rolling volatilities, and volume-based features of Chapter 6's feature engineering section, can capture nonlinear patterns that a fixed-form rule like a moving-average crossover cannot represent at all. This part develops the features such a model is actually given and the discipline that keeps a flexible model honest, then turns to signals that arrive as events rather than as continuous series.

13.7.1 A Head-to-Head Test: Gradient Boosting versus Logistic Regression on a Regime Interaction

The twelve-day price series used throughout the feature-engineering catalog below is far too short to fit any model; testing the claim above requires a much longer history. The companion notebook builds 5,000 days of returns with volatility clustering, using the GARCH-style recursion Chapter 7 introduced, and layers a genuine regime interaction on top: three-day momentum continues during calm, low-volatility stretches but reverses into mean-reversion during turbulent ones. That is exactly the kind of interaction a logistic regression's additive coefficients cannot represent, since a single coefficient on momentum cannot be positive in one regime and negative in another. Both a logistic regression and a gradient-boosted classifier are trained on the same three engineered features from Table 13.7, on the first 3,500 simulated days and evaluated on the final 1,496, consistent with the walk-forward discipline Chapter 7 developed because time-ordered data cannot be split randomly without leaking future information.

Logistic regression reaches a held-out AUC of 0.5130, barely above the 0.50 a coin flip achieves, precisely because its additive structure cannot capture a momentum effect that flips sign with the volatility regime. Gradient boosting reaches 0.5328, a modest but genuine improvement holding up across repeated simulations with different random seeds, with feature importances of 0.423 (momentum), 0.388 (volatility), and 0.189 (RSI) showing it draws on both jointly rather than either alone. Both AUCs are modest deliberately: financial return data have a genuinely low signal-to-noise ratio, the caution Chapter 6 raised repeatedly, so an AUC in the low 0.50s is a realistic stand-in for the small, hard-won edge a real trading signal offers.

Wulin.Suo@Queensu.ca

Gradient boosting captures a nonlinear interaction a linear-in-its-features model structurally cannot, even though the resulting edge, like most genuine ones, is small rather than dramatic.

13.7.2 A Feature-Engineering Catalog for Trading Signals

Chapter 6 developed the general financial-ML feature patterns, ratios, lagged and rolling windows, interaction terms, categorical encodings, that apply across credit scoring, fraud detection, and every other application in this book. Trading signals draw on a narrower catalog built almost entirely from price, volume, and order-book history, organized here into four families. **Price-based** features summarize recent direction and magnitude: momentum, the trailing return over a fixed lookback window, and realized volatility, the rolling standard deviation of returns over the same window, with the moving averages of Section 13.2.2 a third. **Technical indicators** repackage price history into a bounded, more directly interpretable scale: the Relative Strength Index (RSI), the Absolute Price Oscillator (APO), the difference between a fast and a slow exponential moving average, and Bollinger Bands, which wrap a moving average in a pair of volatility-scaled upper and lower bands, all three worked through numerically below.

Volume-based features, raw volume, dollar volume (price times volume), and volume relative to its own recent average, capture how much conviction accompanies a price move, a dimension none of the price-based features above can see on their own. **Microstructure-based** features draw directly on Chapter 3 and this chapter's own market-quality material: the bid-ask spread, and the order-flow imbalance that motivated Kyle's model in Section 13.3.3, both change on a timescale of individual trades rather than daily closes and can supply information a purely price-based feature never sees at all.

Momentum, Realized Volatility, and RSI

Table 13.7 computes three of these features, reusing the identical twelve-day price series from Section 13.2.2's moving-average crossover example: a 3-day momentum ($\text{Mom3}_t = P_t/P_{t-3} - 1$), a 3-day realized volatility (the rolling standard deviation of the three most recent daily returns), and a 3-period RSI, $\text{RSI} = 100 - 100/(1 + RS)$ where RS is the average of the window's positive price changes (gains) divided by the average magnitude of its negative changes (losses).

Day	Price	Mom3 (%)	Vol3 (%)	RSI3
4	106	6.00	0.781	100.00
5	110	8.91	0.732	100.00
6	115	11.65	0.667	100.00
7	119	12.26	0.450	100.00
8	118	7.27	2.328	90.00
9	114	−0.87	2.835	44.44
10	109	−8.40	1.493	0.00
11	104	−11.86	0.523	0.00
12	100	−12.28	0.313	0.00

Table 13.7: Three engineered trading-signal features, computed from Section 13.2.2's twelve-day price series

The three columns tell a genuinely complementary story about the same twelve days Section 13.2.2 already walked through. Momentum simply tracks the trend, rising through day 7 and falling steadily thereafter, mirroring the price series it is computed from.

Realized volatility does something a momentum feature cannot. It stays low and calm throughout the smooth days-4-through-7 uptrend (0.45% to 0.78%) and then spikes sharply, to 2.3% and 2.8%, right at the days 8 and 9 trend reversal Section 13.2.2 identified as the crossover strategy's whipsaw. That is a concrete instance of the volatility clustering Chapter 7's GARCH material developed at length: volatility rises around genuine regime changes, not uniformly through time. It is precisely why realized volatility is itself a useful predictive feature and not merely a risk-management input.

Wulin.Suo@Queensu.ca

RSI illustrates a real limitation alongside its usefulness: because every change in the days-4-through-7 window is positive, RSI saturates at exactly 100 for four consecutive days, the indicator's conventional "overbought" signal, but it cannot distinguish a mild uptrend from an explosive one once every recent change has the same sign, and it falls just as quickly to a saturated 0 ("oversold") once the reversal takes hold and every recent change turns negative. A model trained only on RSI would have no way to tell days 4 through 7 apart from each other, despite their visibly different momentum, which is why practitioners typically feed a model both a bounded indicator like RSI and the underlying continuous features, momentum and realized volatility among them, side by side, letting the model itself learn how much weight each deserves rather than discarding the information a saturated indicator throws away.

Two Further Technical Indicators

Two further technical indicators are common enough in practice to be worth working through numerically alongside RSI, both computed from an exponentially weighted moving average (EMA) rather than the simple moving average of Section 13.2.2: an EMA weights recent prices more heavily than older ones, updating as $\mathrm{EMA}_t = \lambda P_t + (1-\lambda)\mathrm{EMA}_{t-1}$ with smoothing factor $\lambda = 2/(N+1)$ for an N-period window, seeded at the first available price.

The **Absolute Price Oscillator (APO)** is simply the difference between a fast and a slow EMA, $\mathrm{APO}_t = \mathrm{EMA}_t^{\mathrm{fast}} - \mathrm{EMA}_t^{\mathrm{slow}}$; industry practice typically uses a 10-day fast and 40-day slow window (or, in the closely related MACD indicator, a 12-day and 26-day pair smoothed further by a signal line), but Table 13.8, which also carries the Bollinger Bands introduced next, instead uses a 3-day and 6-day pair, scaled down to fit this chapter's twelve-day illustrative series, the same way Section 13.2.2 scaled its own moving-average windows down from industry-standard lengths.

Day	Price	EMA3	EMA6	APO	SMA5	Upper	Lower
1	100	100.000	100.000	0.000	100.000	100.000	100.000
2	101	100.500	100.286	0.214	100.500	101.500	99.500
3	103	101.750	101.061	0.689	101.333	103.828	98.839
4	106	103.875	102.472	1.403	102.500	107.083	97.917
5	110	106.938	104.623	2.314	104.000	111.266	96.734
6	115	110.969	107.588	3.381	107.000	117.040	96.960
7	119	114.984	110.849	4.136	110.600	122.234	98.966
8	118	116.492	112.892	3.600	113.600	123.447	103.753
9	114	115.246	113.208	2.038	115.200	121.575	108.825
10	109	112.123	112.006	0.117	115.000	122.043	107.957
11	104	108.062	109.719	−1.657	112.800	124.071	101.529
12	100	104.031	106.942	−2.911	109.000	122.023	95.977

Table 13.8: Two further technical indicators on the same twelve-day price series: the Absolute Price Oscillator (3-day and 6-day EMA) and Bollinger Bands (5-day SMA, $k = 2$)

The APO column tells the same trend story as momentum but with an EMA's smoother, weighted-average lens: it is positive and climbing throughout the days-1-through-7 uptrend, peaking at 4.136 exactly at day 7's local high, then falls steadily and turns negative by day 11 once the fast EMA has crossed back below the slow one, a lagging but directly interpretable confirmation of the same trend reversal Section 13.2.2's crossover strategy was built to detect.

Bollinger Bands take a different approach to the same underlying price series, wrapping a simple moving average in bands set a fixed number of standard deviations above and below it: $\mathrm{Middle}_t = \mathrm{SMA}_t$, $\mathrm{Upper}_t = \mathrm{SMA}_t + k \cdot \sigma_t$, and $\mathrm{Lower}_t = \mathrm{SMA}_t - k \cdot \sigma_t$, where σ_t is the rolling standard deviation of price (not return) over the same window as the moving average and k is conventionally set to 2. Industry practice typically uses a 20-day window; the table instead uses a 5-day window, for the same reason it shortened the EMA windows above.

Wulin.Suo@Queensu.ca

The band width (upper minus lower) is itself a volatility indicator, telling the same story as Table 13.7's Vol3 column: the bands are tight while days 1 through 5 climb smoothly, then widen sharply from day 6 onward and stay wide through day 12, tracking the same days-8-and-9 volatility spike Vol3 flagged and the same GARCH-style clustering Chapter 7 introduced, expressed geometrically around the price chart rather than as a single rolling standard deviation. Both indicators are further transformations of the same rolling mean and dispersion already summarized by momentum and realized volatility, so neither supplies information a model could not in principle extract from those two directly. Their real value is the packaging: a chart-ready representation a human trader can read at a glance, which is why both remain popular in practitioner tools even though a machine-learning pipeline is free to work from the underlying features instead.

A Market-Wide Feature: The VIX

Every indicator in this catalog so far, momentum, realized volatility, RSI, APO, Bollinger Bands, is computed from a single asset's own price history and looks backward at what has already happened. The CBOE Volatility Index (VIX), introduced in Chapter 5 as the market-implied volatility aggregated across S&P 500 index options, supplies a genuinely different kind of feature: a market-wide, forward-looking measure of the entire market's expected volatility over the next 30 days, rather than any single stock's own recent price behavior.

Trading strategies routinely use the VIX level, or its rate of change, as a regime filter layered on top of the asset-specific signals developed above. A momentum strategy's whipsaw risk, of which Section 13.2.2's own days-8-and-9 reversal is a small-scale illustration, tends to be worse when VIX is elevated and the broad market is already unsettled. A strategy might therefore reduce position sizes, widen its stop-loss thresholds, or stand aside entirely once VIX crosses a specified threshold, rather than treating every trading day as statistically identical regardless of the prevailing volatility regime. Because VIX is itself directly tradable, through VIX futures, options, and exchange-traded products, some strategies also trade VIX-linked instruments directly as a hedge against the same broad market volatility that erodes the returns of the asset-specific strategies developed throughout this chapter.

Two Disciplines: Point-in-Time Data and Stationarity

Two disciplines from earlier chapters apply to every feature in Table 13.7 with particular force. First, the same point-in-time rule Section 13.2.2 enforced for the moving-average signal applies to every engineered feature here: a momentum, volatility, or RSI value computed from prices through day t can only be used to trade day $t+1$'s return, never day t's own, and a backtest that lines up a feature with the same day's return it was partly computed from is committing the data leakage Chapter 6 warned against, only easier to miss because the feature looks like an input rather than an answer.

Second, Chapter 7's stationarity material explains why every feature in the table is a *transformation* of price rather than the raw price level itself. A stock's price can trend arbitrarily far from its starting value over the life of a dataset, the non-stationarity Chapter 7's unit-root discussion described, so a model trained on raw price levels from one historical period would need to extrapolate to price ranges it never saw in training. Momentum, RSI, and volume ratios are all normalized to a roughly stationary, comparable scale regardless of the underlying stock's price level or the historical period examined. That is why they, rather than price itself, are the features actually handed to a model.

From Features to Portfolio Construction

A quantitative equity manager applying these tools typically does not predict an absolute return target directly but instead trains a model to produce a cross-sectional ranking, a relative ordering of an entire universe of stocks from most to least attractive over the next period, since ranking is often both easier to learn reliably and more directly useful for portfolio construction than a precise point forecast of each stock's return. This ranked output feeds naturally into the mean-variance and risk-parity machinery of Chapter 12: rather than optimizing over raw historical returns, the portfolio optimizer takes the model's ranked or scored predictions as its expected-return input, combining a machine-learning-generated forecast with the

Wulin.Suo@Queensu.ca

risk-management discipline developed for classical portfolio construction, the kind of layered, classical-plus-modern pipeline emphasized throughout this book.

Beyond price and volume history, an increasingly important feature source is order-book-derived microstructure data itself, such as the order-flow imbalance (the net difference between resting buy and sell volume at the top of the book) that motivated Kyle's model in Section 13.3.3, and alternative data sources such as those introduced in Chapter 17, satellite imagery, web traffic, or transaction data, which can supply a genuinely independent source of predictive information uncorrelated with the price and volume history every other market participant already observes.

Overfitting and Walk-Forward Validation

The overfitting concern of Section 13.6.3 applies with particular force in this setting, and for an additional reason: a flexible model does not merely search a grid of parameters, it searches an enormous implicit space of functional forms, so its effective N is larger than any explicit configuration count would suggest and correspondingly harder to deflate for. Walk-forward validation, introduced in Chapter 7 specifically because a random train-test split leaks future information into the training set for time-ordered data, is therefore not optional but essential for any machine-learning-based trading signal, and even a walk-forward-validated signal must still be evaluated net of the implementation shortfall and market-impact costs developed earlier in this chapter before its statistical performance can be translated into a genuine, tradable economic edge.

The methods developed throughout this chapter apply across a wide range of trading horizons, from a multi-day pairs trade to an execution algorithm working an order over a single afternoon. At the shortest end of that spectrum, high-frequency trading pushes every one of this chapter's ideas, order types, market impact, inventory management, down to time horizons of milliseconds or microseconds, where the physical latency of transmitting information and orders becomes a first-order concern rather than a rounding error. High-frequency trading is substantial enough a topic, with its own distinctive technology, market-making techniques, and regulatory debates, that it receives its own dedicated treatment in Chapter 14 rather than being folded into this chapter's more general execution and microstructure framework.

13.7.3 News and Event-Driven Trading

A distinct signal category, related to but separate from the momentum and mean-reversion signals introduced earlier in this chapter, trades directly on the arrival of new information: an earnings announcement, a central bank rate decision, a merger announcement, or a news headline. Because such events are, by construction, largely unpredictable in their timing (a firm's exact earnings-release date is known in advance, but the content of the announcement is not), event-driven strategies typically focus less on predicting when a signal will arrive and more on reacting to it faster and more accurately than other market participants once it does.

This is precisely the domain where the natural language processing methods covered in Chapter 16 intersect directly with the execution machinery of this chapter: extracting a sentiment score or a structured summary from an earnings call transcript or a news wire, as Chapter 16 develops in detail, is only half of an event-driven trading strategy. The other half is this chapter's concern, converting that extracted signal into an actual position quickly enough to matter (since an interpretable but slowly generated signal may arrive well after the price has already moved to reflect the same information) and executing the resulting trade while managing the market impact and implementation shortfall costs developed in Sections 13.3.2 and 13.3.5, which are often especially severe immediately after a major news event, when liquidity is thin and many other participants are trying to trade on the same information simultaneously.

13.8 Market Structure and Safeguards

Automated trading interacts with other automated trading, sometimes in ways nobody designed. This part covers the safeguards the market imposes on that interaction, and one episode in which a firm's own missing safeguards destroyed it in forty-five minutes.

Wulin.Suo@Queensu.ca

13.8.1 Circuit Breakers and Trading Halts

Chapter 3 described the 2010 Flash Crash, in which the Dow Jones Industrial Average fell and then recovered nearly 1,000 points within minutes, driven in significant part by algorithmic trading systems interacting in ways no individual participant had designed or anticipated. One direct regulatory response to that episode, and to the general risk that automated trading can amplify a price move far beyond what changed fundamentals would justify, is the circuit breaker: an automatic, exchange-wide (or single-stock) trading halt triggered once price moves beyond a specified threshold within a specified time window, giving market participants, human and algorithmic alike, a pause to assess whether a price move reflects genuine new information or a mechanical, self-reinforcing feedback loop of the kind that characterized the Flash Crash.

Circuit breakers interact directly with the execution and market-making concepts developed earlier in this chapter. A market maker managing inventory under the Avellaneda-Stoikov framework of Section 13.4, for example, must decide how to handle a position when trading in a security is suddenly halted, since the model's continuous quote-adjustment logic assumes the ability to trade continuously, an assumption a halt violates by design. Similarly, an execution algorithm using a TWAP or VWAP schedule spread across a trading day, as introduced earlier in this chapter, must have explicit logic for what to do if trading is halted partway through the schedule, either pausing and resuming once trading reopens or reassessing the remaining execution plan entirely, since simply proceeding as though nothing happened once trading resumes could execute the remaining slices at prices that no longer reflect the market conditions the original schedule was designed around.

13.8.2 Case Vignette: The Knight Capital Trading Glitch

On August 1, 2012, Knight Capital Group, then one of the largest market makers in US equities, deployed new trading software to its production servers. A deployment error left obsolete code active on one of eight servers, code that, combined with the new software, caused the firm's systems to send a rapid, unintended stream of erroneous orders into the market. Within approximately 45 minutes, before the problem was identified and the system halted, Knight Capital had accumulated large, unintended positions across dozens of stocks, and unwinding them resulted in a loss of about $440 million, more than the firm's capital cushion could absorb; its share price fell by roughly three-quarters within two days, wiping out most of its equity value, and the loss forced Knight to accept an emergency capital infusion and ultimately a rescue merger with a competitor.

This episode is a direct, real-world illustration of the technology and latency risk flagged later in this chapter's challenges section, and it is worth being specific about why it happened, since the failure was not a flawed trading strategy in the sense this chapter has otherwise discussed. Knight's algorithms were not making a bad prediction or a poorly calibrated risk-reward tradeoff; a software deployment error caused the firm's own execution infrastructure, the very machinery this chapter has treated as neutral plumbing for implementing a trading decision, to behave in a way no human ever intended or approved.

The speed advantage that makes algorithmic and high-frequency trading profitable, the same speed this chapter's discussion of latency emphasized, is precisely what turned a deployment bug into a $440 million loss in well under an hour: a human-paced trading operation making the same kind of error would very likely have noticed and intervened long before losses accumulated to anything close to that scale. The lesson generalizes directly to the model risk management framework introduced in Chapter 8: an algorithmic trading system requires the same independent testing, staged deployment, and real-time monitoring and kill-switch capability that a bank's VaR or credit model requires, precisely because the speed that makes automation valuable is the same speed that makes an undetected error catastrophic.

13.9 Challenges in Algorithmic Trading and Market Microstructure

Several challenges recur across nearly every algorithmic trading application.

Backtest overfitting. With enough historical data and enough candidate strategies, some rule will appear profitable purely by chance, the same multiple-testing concern raised in Chapters 6, 7, and 12.

Wulin.Suo@Queensu.ca

Section 13.6.3 put a number on it: searching a hundred worthless configurations yields an expected best in-sample Sharpe ratio of 1.12, and the strategy that wins that search has no out-of-sample edge whatsoever. A strategy discovered by searching over many parameter combinations therefore needs to be validated out of sample, and deflated for the size of the search, considerably more skeptically than one derived from a specific, pre-specified economic hypothesis.

Market impact is itself uncertain and strategy-dependent. The clean market-impact numbers in Section 13.3.2 come from a simplified, static order book; real market impact depends on prevailing volatility, on whether other market participants can detect and react to a large order being worked, and on overall market liquidity conditions that themselves vary over time, all of which make market impact considerably harder to estimate precisely than the other inputs to a trading strategy.

Latency and technology risk. Execution algorithms and market-making strategies alike depend on fast, reliable technology, and a software bug, a connectivity failure, or a delay in receiving market data can turn a theoretically sound strategy into a source of real, sometimes rapid, losses; this is the specific operational risk category from Chapter 10 applied to a domain where a failure can compound within seconds rather than days.

Adverse selection from faster participants. A trading algorithm that posts a limit order risks that order being filled specifically by a faster or better-informed participant at the moment when doing so is disadvantageous, the same adverse-selection component of the bid-ask spread introduced in Chapter 3; the more of this risk a strategy is exposed to, the wider the effective spread it must earn just to break even.

Regulatory and market-structure change. Algorithmic and high-frequency trading operate within a regulatory framework (Regulation NMS and its analogues, introduced in Chapter 3) that itself evolves, and a strategy whose profitability depends on a specific market-structure feature, a particular exchange's fee structure or order-matching rule, for example, can see that edge disappear or reverse if the underlying rules change.

Crowding in statistical arbitrage. Section 13.5.2's discussion of the 2007 quant quake illustrated a specific, historically documented version of a general hazard. When many trading firms independently discover and trade similar statistical relationships, their positions become correlated in a way none of them can observe directly from their own book alone. A shock or forced unwind at one large fund can then trigger simultaneous, mutually reinforcing losses across many ostensibly unrelated strategies, precisely the crowding risk Chapter 12 raised in the context of long-horizon factor investing, but which can unfold over hours rather than months in a statistical arbitrage context.

Exchange fee structures and rebates. Many exchanges charge a fee to remove liquidity (executing against a resting order) while paying a rebate to add it (posting a resting limit order that another participant later executes against). This maker-taker pricing model creates its own subtle incentives independent of the underlying trading signal. A strategy's profitability can depend materially on whether it typically posts passive limit orders or crosses the spread with aggressive market orders, and two strategies with identical raw trading signals can have very different net profitability purely because of how each interacts with a given exchange's specific fee schedule, an execution-level detail entirely separate from whether the underlying signal is economically sound. A realistic backtest, following the transaction-cost discipline of Section 13.6.1, should account for these fees and rebates explicitly rather than assuming every trade is executed at a single, cost-free price, since even a few basis points of fee difference, compounded across thousands of trades, can be the difference between a strategy that is genuinely profitable after costs and one that only appears to be. This is a further, concrete illustration of the implementation-shortfall discipline introduced earlier in this chapter: the fee schedule itself is one more component of the gap between a strategy's theoretical and realized performance, no less real for being easy to overlook in an initial backtest.

Capacity constraints on strategy scale. A strategy backtested at a small notional size can look highly attractive on a risk-adjusted basis. But the market impact costs developed in Section 13.3.2 grow, roughly speaking, faster than linearly with order size. The same strategy run at ten or a hundred times the capital can therefore see its net-of-cost returns deteriorate sharply, or even turn negative, well before it exhausts the raw statistical edge its backtest identified. This capacity limit is a genuine economic constraint on how much capital any single trading idea, however sound, can actually absorb profitably.

Wulin.Suo@Queensu.ca

13.10 Key Takeaways

Converting a trading signal into a real position and a real position into an actual profit-and-loss series requires confronting the same market microstructure realities Chapter 3 introduced: the bid-ask spread, the limited depth available at each price level, and the price impact of consuming that depth too quickly. The Kyle model gives that price impact a formal economic interpretation, as compensation for the risk of trading against better-informed counterparties, and TWAP and VWAP provide simple, standard benchmarks for execution quality, while the Almgren-Chriss framework formalizes the tradeoff between market impact and timing risk that this chapter's simpler comparisons illustrated concretely.

Market making extends the spread-capturing intuition of Chapter 3 into an explicit inventory-management problem, and pairs trading, introduced only conceptually in Chapter 7, becomes a complete, backtestable strategy once entry and exit rules are applied to real, out-of-sample data, a logic that extends naturally to statistical arbitrage across many assets using the factor models of Chapter 12. Both the execution and the market-making models are closed-form answers bought with strong assumptions, and Chapter 19 revisits each one as a sequential decision problem solved by learning rather than by algebra.

Evaluating any such strategy requires more than a single return figure: the Sharpe ratio, maximum drawdown, and hit rate each reveal a different dimension of performance, and every one of them must be computed net of realistic transaction costs and implementation shortfall, not against the frictionless prices a signal was originally computed from, since the gap between theoretical and realized performance is often the difference between a strategy that looks good on paper and one that is actually worth trading.

Beyond the mechanics of any single strategy, this chapter's case vignette and its discussion of circuit breakers underscore a broader point that recurs throughout this book: the technology and market structure surrounding a trading decision are not neutral plumbing but active sources of risk in their own right, deserving the same governance discipline Chapter 8 developed for financial risk models generally.

Finally, this chapter's challenges section is a reminder that a statistically sound backtest is necessary but never sufficient. Fees, capacity limits, crowding, and the everyday risk of a software or operational failure all sit between a promising historical result and a genuinely profitable, sustainable trading operation. The machine learning signals of Chapter 6 and the natural-language and alternative-data sources of Chapters 16 and 17 must clear these same practical hurdles before a sophisticated model becomes a trade that actually makes money, net of the frictions this chapter has taken care to make explicit.

The next chapter narrows the focus to the extreme, millisecond-scale end of this same spectrum, where every idea introduced here, order types, market impact, inventory management, backtesting discipline, still applies but is compressed into a timescale where speed itself becomes a primary source of competitive advantage.

13.11 Suggested Exercises

1. Recompute the moving-average crossover strategy's cumulative return using a 2-day short average and a 4-day long average on the same twelve-day price series, and compare the resulting signal timing to the 3-day/6-day version in the text.

2. A 500-share market order walks through an order book with 200 shares available at \$30.00 and additional shares available at \$30.05. Compute the average execution price and the price impact in basis points relative to the best price.

3. Using the extended pairs-trading data in Table 13.4, compute the trade's profit and loss if the position were instead closed at day 9 rather than day 10, and compare it to the day-10 exit computed in the text.

4. A strategy has a Sharpe ratio of 1.2 and a maximum drawdown of 18%, while a second strategy has a Sharpe ratio of 0.8 and a maximum drawdown of 5%. Discuss which strategy an investor with a low tolerance for large losses might prefer, and why Sharpe ratio alone would not reveal this consideration.

5. A portfolio manager's decision price is \$100.00 and the resulting order is executed at an average price of \$100.12 on a 5,000-share order. Compute the implementation shortfall in basis points and in total dollars.

Wulin.Suo@Queensu.ca

6. Explain, referencing Table 13.7's RSI column, why an indicator that saturates at 0 or 100 can lose information a model might otherwise use, and describe why feeding a model both RSI and the underlying momentum feature side by side addresses this limitation.

7. Using Table 13.8, verify by hand that day 8's APO is 3.600, and explain why APO does not turn negative until day 11, four days after day 7's price high, rather than exactly at it.

8. Using Table 13.8, explain why the band width at day 7 (Upper minus Lower) is so much larger than at day 2, referencing this section's discussion of Table 13.7's Vol3 column.

9. Explain why walk-forward validation, introduced in Chapter 7, is essential when evaluating a machine-learning-based trading signal, and describe a specific way that a random train-test split could produce a misleadingly optimistic backtest.

10. Pick two of the challenges listed in this chapter and describe a concrete scenario, not already used in the text, in which that challenge could cause an algorithmic trading strategy to lose money despite a statistically sound backtest.

11. Using Table 13.2, verify by hand that Kyle's lambda is approximately 1.95 cents per thousand shares, and interpret what a much larger estimated lambda (say, 10 cents per thousand shares) would suggest about that market compared to the one in the text.

12. Explain the difference between an iceberg order and trading in a dark pool, and describe one advantage and one disadvantage each has relative to posting a single, fully visible limit order.

13. Describe how a market maker using the inventory-skewing logic of Section 13.4 should adjust its behavior if trading in the underlying stock is suddenly halted by a circuit breaker, and explain what risk this halt introduces that continuous trading does not.

14. Using the Avellaneda-Stoikov formulas in Section 13.4, recompute the reservation price, optimal spread, bid, and ask for a market maker with the same parameters as the worked example except that it is short $q = -8$ units instead of long 5, and explain why the resulting skew now points in the opposite direction.

15. Explain why the Knight Capital case vignette is better understood as a technology and governance failure than as a bad trading strategy, and describe one specific model risk management practice from Chapter 8 that, if followed, might plausibly have prevented or limited the loss.

16. A statistical arbitrage fund holds 200 similar mean-reversion positions across a sector. Explain, referencing the 2007 quant quake, why this diversification might provide less protection than the number of positions alone would suggest.

17. Using Table 13.5, explain why Stock W's large residual z-score alone does not tell a trader whether to actually put on the suggested trade, and describe what additional information (beyond the regression output itself) an analyst would want before shorting Stock W against its sector peers.

18. A backtest of a value strategy is run only on stocks currently listed on a major exchange, using their full, publicly available price history. Identify the specific bias this introduces, referencing this chapter's discussion of backtesting pitfalls, and describe one concrete step a researcher could take to correct it.

19. Using the Almgren-Chriss worked example, compute κ and the resulting optimal trajectory x_j for the same 250-share order and $N = 5$ intervals if risk aversion were $\phi = 2$ instead of 0.5 or 5, and state whether the resulting schedule is more or less front-loaded than both trajectories computed in the text.

20. **VWAP under a mistaken volume forecast.** Section 13.3.4 argues that a VWAP algorithm is only as good as its forecast of the day's volume curve. Using Table 13.3's prices and realized volumes, suppose an algorithm works a 1,000-share buy order according to a forecast volume profile of 15%, 20%, 25%, 15%, and 25% across the five intervals. (a) Compute the realized-volume VWAP benchmark and the average price the algorithm actually achieves. (b) Report the shortfall against that benchmark in

Wulin.Suo@Queensu.ca

cents per share, in basis points, and in dollars on the full order. (c) Identify which intervals' forecast errors caused the shortfall and which one partially offset it, and explain why an error in an interval whose price sits close to the VWAP costs almost nothing.

21. **Challenge (real data).** Download at least two years of daily prices for a real, liquid stock (via `yfinance`) and implement this chapter's moving-average crossover strategy (Table 13.1) using a 20-day short window and a 50-day long window. (a) Backtest the strategy's cumulative return against a simple buy-and-hold benchmark over the full sample, and compute both strategies' Sharpe ratios. (b) Using walk-forward validation rather than a single in-sample backtest, as this chapter's own discussion (building on Chapter 7) recommends, split your sample into several sequential training/testing windows and report whether the strategy's out-of-sample Sharpe ratio holds up as well as its in-sample Sharpe ratio. (c) Compute the feature set in Table 13.7 (momentum, realized volatility, RSI) for your real price series, and discuss whether the same volatility-clustering pattern this chapter's toy twelve-day example showed around its crossover reversal appears at your strategy's actual entry and exit points.

13.12 Further Reading

- Almgren and Chriss, "Optimal Execution of Portfolio Transactions," *Journal of Risk*. The original formalization of the market-impact-versus-timing-risk tradeoff developed in Section 13.3.5.
- Appel, *Technical Analysis: Power Tools for Active Investors*. Written by the creator of the MACD indicator, the closely related, signal-line-smoothed cousin of the Absolute Price Oscillator used in Section 13.7.2's feature-engineering worked example.
- Avellaneda and Stoikov, "High-Frequency Trading in a Limit Order Book," *Quantitative Finance*. The original inventory-based market-making model summarized in Section 13.4.
- Bailey and López de Prado, "The Deflated Sharpe Ratio," *Journal of Portfolio Management*. The source of the deflation adjustment recommended in Section 13.6.3, and the standard reference on backtest overfitting.
- Bollinger, *Bollinger on Bollinger Bands*. Written by the creator of the Bollinger Bands indicator used in Section 13.7.2's feature-engineering worked example.
- Chan, *Algorithmic Trading: Winning Strategies and Their Rationale*. A practitioner-oriented treatment of strategy development and backtesting, including the pairs-trading and mean-reversion strategies extended in this chapter.
- Harris, *Trading and Exchanges: Market Microstructure for Practitioners*. A comprehensive treatment of order types, market structure, and execution that extends this chapter's introductory coverage and connects directly back to Chapter 3.
- Kissell, *The Science of Algorithmic Trading and Portfolio Management*. Covers transaction cost analysis and implementation shortfall, introduced in this chapter, in considerably greater quantitative depth.
- Kyle, "Continuous Auctions and Insider Trading," *Econometrica*. The original 1985 paper formalizing the price-impact model summarized in Section 13.3.3.
- Wilder, *New Concepts in Technical Trading Systems*. The original source introducing the Relative Strength Index used in Section 13.7.2's feature-engineering worked example.
- Guéant, Lehalle, and Fernandez-Tapia, "Dealing with the Inventory Risk: A Solution to the Market Making Problem," *Mathematics and Financial Economics*. A rigorous, closed-form treatment extending the Avellaneda-Stoikov inventory model of Section 13.4 to more realistic order-arrival assumptions.
- Patterson, *Dark Pools: The Rise of the Machine Traders and the Rigging of the U.S. Stock Market*. A journalistic but well-researched account of the rise of dark pools, algorithmic trading, and the market-structure debates touched on in this chapter's discussion of order routing and venue choice.

Wulin.Suo@Queensu.ca

Chapter 14

High-Frequency Trading

14.1 Introduction

Chapter 13 developed execution algorithms, market impact, and inventory-based market making at the pace of orders worked over minutes or hours. This chapter pushes every one of those same ideas down to the shortest end of the trading-horizon spectrum: milliseconds and microseconds, where the physical time it takes information and orders to travel between computers becomes as important as the economic logic governing what those orders should say. High-frequency trading (HFT) is not a separate branch of finance with its own theory; it is Chapter 13's market making, statistical arbitrage, and order-routing machinery operating at a timescale where speed itself becomes a source of competitive advantage, and where the operational and technology risks introduced by the Knight Capital vignette become the central, not peripheral, concern.

Several worked examples run through the chapter. A latency-arbitrage calculation quantifies, in dollars, exactly what a faster market participant can extract from a slower one during the brief window created by a physical speed advantage. A Glosten-Milgrom sequential-trade model and its probability-of-informed-trading (PIN) statistic extend Chapter 1's Bayes'-theorem introduction to a market maker updating its beliefs trade by trade. A high-frequency market-making profitability example extends Chapter 13's Avellaneda-Stoikov inventory model to a full trading day of quote refreshes, showing concretely how thin per-trade spread capture becomes a meaningful daily profit once repeated enough times, and how adverse selection erodes it. Further examples cover ETF-basket and cross-market statistical arbitrage, a co-location break-even calculation, and an order-flow-imbalance regression that extends Chapter 13's Kyle's-lambda framework to a predictive setting.

The chapter's sections follow the physical and economic logic of speed itself: the race to zero latency (Section 14.2.2) explains why microseconds are worth paying for at all, the latency-arbitrage and Glosten-Milgrom examples (Sections 14.3.1 and 14.3.2) quantify what that speed is worth to whoever has it, and the high-frequency inventory and daily P&L sections (Sections 14.4.1 and 14.4.2) turn a single quote refresh into a full day's economics. Later sections turn from strategy to market plumbing and its regulatory consequences: queue position and price-time priority (Section 14.5.5), the order-flow-imbalance regression that a real trading desk would use to anticipate the next tick (Section 14.7.1), and the risk controls, flash-crash case study, and regulatory debate that close the chapter, asking whether the speed this chapter spends most of its pages quantifying makes markets better or merely faster.

14.2 The High-Frequency Landscape

High-frequency trading is less a single strategy than a family of them sharing one constraint: they must act before anyone else can. This part establishes what the label actually covers, and why the pursuit of speed became an arms race with a physical floor.

Wulin.Suo@Queensu.ca

14.2.1 What High-Frequency Trading Is, and Why It Matters

Suppose two firms are both trying to trade the same stock a fraction of a second after the same piece of market-moving news breaks. One firm's edge is not a better forecast of where the stock is headed next quarter, it has no such forecast, it is simply that its servers sit physically closer to the exchange's own and its software reacts in microseconds rather than milliseconds. High-frequency trading describes a cluster of related practices built almost entirely around winning this kind of race, not a single strategy: extremely short holding periods (often seconds or less), very high order-to-trade ratios (many quotes posted and canceled for every one that executes), and a reliance on speed, low-latency infrastructure, and often co-location at exchange data centers, as the primary source of competitive advantage rather than superior information about fundamental value. HFT firms are typically not trying to predict where a stock's price will be next quarter, or even next minute; they are trying to react to information, an order-book update, a trade print, a price change on a correlated instrument, faster than other market participants can.

This distinction matters because it reframes what HFT actually contributes to and extracts from a market. A large share of HFT activity is market making in the sense Chapter 3 and Chapter 13 already introduced: continuously posting both a bid and an ask, earning the spread, and managing the resulting inventory risk, the Avellaneda-Stoikov problem from Chapter 13, just executed at a much higher frequency and with much shorter holding periods per unit of inventory. A second major category is latency arbitrage, detecting a price change on one venue or instrument and trading on a related, slower-to-update venue or instrument before its quotes catch up, developed in the next section. A third, more controversial category includes strategies whose profitability depends specifically on other participants' reaction times or order-handling logic, sometimes shading into manipulative practices such as spoofing (posting orders with no intention of executing them, to influence other participants' behavior) that are illegal in most jurisdictions and are discussed further in this chapter's regulatory section.

Table 14.1 organizes the main strategy categories together with the economic mechanism each exploits and where this chapter develops it further.

Strategy type	Economic mechanism	Developed in
Market making	Earn the spread; manage inventory risk with continuously updated quotes	Sections 14.4.1, 14.3.2
Latency arbitrage	Trade against stale quotes before they update, exploiting a pure speed advantage	Section 14.3.1
Statistical arbitrage	Trade a fast, model-implied relationship between related instruments before it reverts	Chapter 13's Section on statistical arbitrage, extended to microsecond horizons
Manipulative practices	Create a false impression of supply or demand to move prices, then trade on the resulting move	This chapter's regulatory case vignette

Table 14.1: A taxonomy of high-frequency trading strategies

Every category in Table 14.1 except the last shares a common structure with the classical models developed throughout this book: a well-specified economic mechanism, spread capture, speed-based adverse selection, or statistical mean reversion, that can be formalized, measured, and reasoned about using the tools this book has developed. The last category is different in kind, not degree: it is not a trading strategy that happens to be aggressive, but conduct that securities and commodities law define as manipulative regardless of how it is implemented technologically, a distinction developed further in this chapter's regulatory discussion.

Wulin.Suo@Queensu.ca

14.2.2 The Race to Zero: Latency, Co-location, and the Physics of Speed

At millisecond and microsecond timescales, the speed of light itself becomes a binding, physical constraint on trading, not merely a metaphor. Two consequences follow directly. First, HFT firms pay substantial fees for co-location, placing their own servers physically inside or immediately adjacent to an exchange's data center, since every additional meter of cable adds a small but, at this timescale, meaningful transmission delay. Second, firms competing to connect two distant trading hubs have historically raced to use whatever physical medium transmits data fastest, since a firm whose data arrives even a few milliseconds sooner can react to a price-moving event before a slower competitor's systems have even received the same information.

A concrete calculation shows why this matters enough to spend real money on. Signals sent through fiber-optic cable travel at only about 68% of the speed of light in vacuum, because the glass core has a refractive index of about 1.47, which slows light to roughly $c/1.47$; a microwave signal transmitted through open air, by contrast, travels at very close to the full vacuum speed of light, about 99.97% of c. Over a corridor of roughly 1,200 kilometers (comparable to the distance between major financial hubs such as New York and Chicago), the round-trip transmission times are

$$t_{\text{fiber}} = \frac{2 \times 1{,}200 \text{ km}}{0.68c} \approx 11.77 \text{ ms}, \qquad t_{\text{microwave}} = \frac{2 \times 1{,}200 \text{ km}}{0.9997c} \approx 8.01 \text{ ms},$$

a difference of roughly 3.8 milliseconds. This is precisely why, historically, firms have invested heavily in microwave and even millimeter-wave relay networks between major trading hubs despite their weather sensitivity and lower bandwidth relative to fiber: a persistent few-millisecond speed advantage, available to whoever built or leases the fastest link, translates directly into being able to act on new information before competitors using slower infrastructure, the kind of advantage the latency-arbitrage example below quantifies in dollar terms.

14.3 Latency Arbitrage and Informed Trading

The simplest way to profit from speed is to trade against a price somebody else has not yet updated. This part works that mechanism through numerically, then asks what a market maker facing possibly-informed counterparties should rationally do about it.

14.3.1 A Worked Example: Latency Arbitrage and Stale Quotes

Consider a stock quoted at a bid of \$99.95 and an ask of \$100.05 by two market makers, Firm S (slow, with a data-to-order latency of 500 microseconds) and Firm F (fast, with a latency of 50 microseconds). Suppose a piece of public news causes the stock's fundamental value to jump instantly to \$100.10. Firm F's systems detect this and cancel or update its stale quotes within 50 microseconds; Firm S takes 500 microseconds to do the same, leaving its old ask of \$100.05 executable for an extra $500 - 50 = 450$ microseconds after the price has already moved.

During that window, Firm F can buy against Firm S's stale \$100.05 ask, immediately holding shares now worth \$100.10, a risk-free profit of \$0.05 per share, purely a function of the 450-microsecond latency gap and unrelated to any view about where the stock is headed next. Figure 14.1 lays out this sequence of events on a timeline. If Firm F can execute 5,000 shares against Firm S's stale quote before it updates, and this kind of event happens roughly 20 times in a trading day across F's traded universe,

$$\text{Profit per event} = 5{,}000 \times \$0.05 = \$250, \qquad \text{Daily profit} = 20 \times \$250 = \$5{,}000.$$

From Firm S's perspective, this is not a mysterious loss but the direct, quantifiable cost of adverse selection introduced in Chapter 13's discussion of Kyle's model. Firm S's quotes are, in effect, being picked off specifically by counterparties with better information, here purely faster rather than more informed in the traditional sense. The \$0.05 per-share loss is the kind of cost the inventory-holding and adverse-selection components of the bid-ask spread, introduced in Chapter 3, exist to compensate for. A slower market maker that ignores this risk will find its realized spread capture persistently lower than its quoted spread would suggest. A predictable fraction of its fills are, by construction, the ones a faster competitor chose to make happen.

Wulin.Suo@Queensu.ca

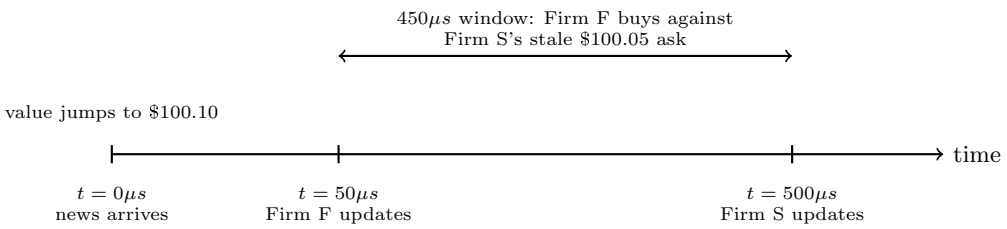


Figure 14.1: The sequence of events behind the latency-arbitrage worked example (not drawn to scale)

14.3.2 A More Advanced Model: Sequential Trade and Bayesian Learning

Chapter 13's Kyle model treats informed and uninformed order flow as arriving in a single batch and derives a single price-impact coefficient, λ, from it. The Glosten-Milgrom model takes a complementary approach better suited to how a high-frequency market maker actually operates: it treats trades as arriving one at a time and has the market maker update its beliefs about the asset's true value after every single trade, using the Bayesian updating rule Chapter 1 introduced for the fraud-detection example, now applied to the sequence of buy and sell orders a market maker observes in real time.

Suppose an asset's true value V is equally likely, before any trading, to be $V_H = \$51$ or $V_L = \$49$, so the unconditional expected value is \$50. A fraction $\pi = 10\%$ of order flow comes from informed traders, who know V exactly and buy when $V = V_H$ and sell when $V = V_L$. The remaining 90% comes from uninformed traders, who buy or sell with equal probability regardless of V.

A rational, competitive market maker sets its ask equal to the asset's expected value *conditional on the next order being a buy*, and its bid equal to the expected value conditional on the next order being a sell, since either quote must break even on average against a mix of informed and uninformed counterparties. Using Bayes' theorem just as in Chapter 1,

$$\Pr(V_H \mid \text{Buy}) = \frac{\Pr(\text{Buy} \mid V_H)\ \Pr(V_H)}{\Pr(\text{Buy})} = \frac{[\pi(1) + (1-\pi)(0.5)](0.5)}{0.5} = 0.55,$$

so observing a buy order raises the market maker's belief that $V = V_H$ from the 50% prior to a 55% posterior, and the resulting break-even quotes are

$$\text{Ask} = 0.55(51) + 0.45(49) = \$50.10, \qquad \text{Bid} = 0.45(51) + 0.55(49) = \$49.90,$$

a \$0.20 spread. This spread is not an arbitrary markup; it is, in this simple two-value setup, exactly $\pi(V_H - V_L) = 0.10 \times \$2.00 = \$0.20$, showing directly that the entire spread here compensates for adverse selection risk, and that a higher fraction of informed traders π mechanically widens it, the same qualitative conclusion Kyle's λ produces via a completely different mathematical route.

The model's real power is in what happens next: today's posterior becomes tomorrow's prior, the same recursive Bayesian-updating logic behind the Kalman filter Chapter 7 introduced for state-space models. If a second order also arrives as a buy, the market maker updates again, this time starting from the 55% prior rather than 50%,

$$\Pr(V_H \mid \text{Buy}, \text{Buy}) = \frac{[\pi(1) + (1-\pi)(0.5)](0.55)}{0.505} \approx 0.599,$$

where the denominator $0.55(0.55) + 0.45(0.45) = 0.505$ is the unconditional probability of a second buy under the updated prior, no longer the exactly one-half that the symmetric first round produced. The revised posterior pushes the next ask up to approximately $0.599(51) + 0.401(49) \approx \$50.20$ and the next bid up to exactly \$50.00.

Two consecutive buy orders, with no other information at all, have moved the market maker's entire quote range up by about ten cents, the kind of momentum a sequence of same-direction trades should rationally create in a market maker who cannot distinguish informed from uninformed flow trade by trade: each additional buy makes it modestly more likely that the order flow is dominated by informed buyers who

Wulin.Suo@Queensu.ca

know $V = V_H$, and the quotes drift toward V_H accordingly. This is precisely the mechanism, formalized with actual numbers, behind the intuitive market observation that a market maker (or an algorithm) "leans into" a sequence of one-directional trades, and it is a direct, practical illustration of why high-frequency market-making systems continuously re-estimate fair value from recent order flow rather than quoting a single static price throughout the day.

The Probability of Informed Trading (PIN)

Glosten-Milgrom updates beliefs trade by trade; a related model, developed by Easley, Kiefer, O'Hara, and Paperman, summarizes the same underlying idea into a single daily statistic, the probability of informed trading. The model assumes that on any given day, an information event occurs with probability α, in which case informed traders arrive at rate μ; uninformed buyers and sellers arrive continuously at rate ε each, information event or not. Averaging over these arrival rates gives

$$\text{PIN} = \frac{\alpha\mu}{\alpha\mu + 2\varepsilon}.$$

For a stock with $\alpha = 30\%$, $\mu = 150$ informed trades per day on event days, and $\varepsilon = 100$ uninformed buy (and separately, sell) arrivals per day,

$$\text{PIN} = \frac{0.30(150)}{0.30(150) + 2(100)} = \frac{45}{245} \approx 18.4\%,$$

meaning roughly 18% of this stock's trades are estimated to come from informed traders, a single, tradeable-frequency-independent summary of the same adverse-selection risk Section 14.3.1 and this section's Glosten-Milgrom example quantified trade by trade. A high-frequency market maker uses a statistic like PIN somewhat differently than the per-trade updating of Glosten-Milgrom: rather than updating quotes after every individual trade, PIN (typically re-estimated periodically from recent trading data, since it requires enough trades to estimate reliably) informs how aggressively to widen quotes and how much inventory risk to tolerate in a given name overall, a slower-moving risk parameter layered on top of the trade-by-trade quote adjustments the rest of this chapter develops.

14.3.3 Statistical Arbitrage at High Frequency: ETF-Basket Arbitrage

Chapter 13 introduced statistical arbitrage as trading a model-implied relationship between related instruments, illustrated there with a multi-day pairs trade. At high-frequency timescales, the most common version of this strategy exploits an even more mechanical relationship: an exchange-traded fund (ETF) is, by construction, a claim on a specified basket of underlying securities, and a mechanism called creation and redemption allows authorized participants to exchange ETF shares for the underlying basket (or vice versa) at the fund's official net asset value, which keeps the ETF's market price closely tethered to the real-time fair value of its underlying holdings. This is the same no-arbitrage logic Chapter 3 introduced for a risk-free bond and a certain-payoff claim.

Suppose an ETF trades at \$150.05 while the real-time, weighted fair value of its underlying basket of stocks is \$150.00, a \$0.05 premium (0.033%). An HFT firm can sell the (relatively expensive) ETF and simultaneously buy the (relatively cheap) underlying basket, or a close proxy such as index futures, capturing the premium as the two prices converge, whether through the creation-redemption mechanism directly or simply through ordinary trading pressure. On a \$10 million position, this captures

$$\$10{,}000{,}000 \times 0.033\% \approx \$3{,}333,$$

and if smaller versions of this same mispricing recur roughly 100 times in a trading day, at an average position of \$200,000 each,

$$\text{Daily profit} \approx 100 \times (\$200{,}000 \times 0.033\%) \approx \$6{,}667.$$

This strategy shares its underlying economic logic directly with Chapter 3's no-arbitrage bond example: two claims on the same underlying cash flows must trade at the same price. It requires HFT-level speed specifically because the mispricing here is both small in percentage terms and short-lived. Any participant

Wulin.Suo@Queensu.ca

slower than the fastest competitors capturing the same discrepancy will find the gap has already closed by the time their order arrives, the stale-quote dynamic of Section 14.3.1 applied to a relationship between two instruments rather than a single stock's own quotes.

14.4 High-Frequency Market Making

Chapter 13's inventory model assumed a dealer with time to think. This part pushes the same machinery down to horizons where quotes are revised thousands of times a day, and works a full day's profit and loss for a high-frequency market maker.

14.4.1 Market Making at High Frequency: Extending the Inventory Model

Chapter 13's Avellaneda-Stoikov example computed a market maker's optimal quote skew using a time horizon $T - t$ of a full unit (one day, in that illustration). High-frequency market making applies the same formula,

$$r = s - q\,\gamma\,\sigma^2\,(T - t),$$

but with $T-t$ measured in a tiny fraction of that horizon, since an HFT market maker refreshes its assessment of fair value and its quotes continuously rather than once per day. Holding inventory q, risk aversion γ, and volatility σ fixed at Chapter 13's values ($q = 5$, $\gamma = 0.1$, $\sigma = 2$), shrinking the horizon from $T - t = 1$ (one full day) to $T - t = 0.001$ (roughly one-thousandth of a day, on the order of a minute) shrinks the reservation-price skew proportionally,

$$\text{skew}(T-t = 1) = 5(0.1)(2^2)(1) = 2.00, \qquad \text{skew}(T-t = 0.001) = 5(0.1)(2^2)(0.001) = 0.002,$$

a full 1,000-fold reduction. This is the precise mathematical reason high-frequency market making requires holding very little inventory for very little time: since the model's inventory-risk penalty scales linearly with the remaining time horizon, a market maker who refreshes quotes (and, ideally, flattens inventory) thousands of times per day can profitably run with much higher per-trade risk aversion or much larger momentary inventory than one refreshing only once per day, simply because each individual exposure is held for a vastly shorter time. This is also precisely why the technology investments described in Section 14.2.2 matter economically: a firm that can update its quotes and manage inventory a few hundred microseconds faster than a competitor is, in effect, operating with a shorter $T - t$ and therefore a smaller optimal skew for the same underlying risk, allowing it to quote more competitively (a tighter effective spread) while bearing the same true risk per unit of time.

It is worth checking whether Chapter 13's *total* optimal spread, not just the inventory skew, shrinks by the same dramatic factor, since the two are easy to conflate. Recall Chapter 13's optimal spread formula,

$$\delta = \gamma\sigma^2(T - t) + \left(\frac{2}{\gamma}\right)\ln\left(1 + \frac{\gamma}{\kappa}\right),$$

has two genuinely different terms: the first, an inventory-risk term, scales directly with $T - t$ exactly like the skew above; the second, an order-arrival-elasticity term governed by κ, does not depend on $T - t$ at all. Holding $\kappa = 1.5$ fixed alongside the same $\gamma = 0.1$ and $\sigma = 2$, the total spread at $T - t = 1$ is

$$\delta = 0.1(4)(1) + \left(\frac{2}{0.1}\right)\ln\left(1 + \frac{0.1}{1.5}\right) \approx 0.400 + 1.291 = 1.691,$$

matching Chapter 13's \$1.69 figure, while at $T - t = 0.001$ it is $\delta \approx 0.0004 + 1.291 = 1.291$, a reduction of only about 23.6%, not the 99.9% reduction the skew alone showed. The elasticity term, 1.291, is a floor the total spread cannot shrink below regardless of how short the inventory-holding horizon becomes. It compensates the market maker for a genuinely different risk, how sensitively order arrivals respond to quote competitiveness, which has nothing to do with how long any single unit of inventory is held.

This is a concrete, quantitative correction to an easy overgeneralization. Refreshing quotes thousands of times per day all but eliminates the inventory-driven component of a market maker's spread, but it does not eliminate the spread itself. A genuinely competitive HFT quote must still be wide enough to compensate for adverse selection from faster or better-informed counterparties, the risk Section 14.3.2's Glosten-Milgrom model and Section 14.3.1's stale-quote example quantify from two different angles.

Wulin.Suo@Queensu.ca

14.4.2 A Worked Example: Daily Profit and Loss from High-Frequency Market Making

Extending Section 14.4.1's logic to an actual trading day, suppose a high-frequency market maker in a liquid stock completes 50,000 fills over the course of one day, each fill one side of a round trip (a buy soon offset by a sell, or vice versa), so 25,000 round trips in all, at an average size of 100 shares. A round trip earns the full spread of \$0.01 per share, i.e. a half-spread of \$0.005 per share on each side, so

$$\text{Gross spread capture} = 50{,}000 \times 100 \times \$0.005 = \$25{,}000.$$

Not every fill is profitable, however: suppose 5% of these fills are with informed ("toxic") counterparties, of the kind the latency-arbitrage example in Section 14.3.1 illustrated from the other side, and that these trades lose an average of \$0.02 per share rather than earning the half-spread,

$$\text{Adverse selection loss} = (50{,}000 \times 0.05) \times 100 \times \$0.02 = \$5{,}000,$$

leaving a net daily profit of \$25,000 − \$5,000 = \$20,000, with adverse selection consuming exactly 20% of gross spread capture. This decomposition is the practical, day-to-day version of the same tradeoff Chapter 13's Kyle model formalized. A market maker's realized profitability depends not just on the quoted spread and trading volume, but on the fraction of that volume coming from counterparties who are, in effect, faster or better-informed. That is precisely why market makers invest heavily in the technology described in Section 14.2.2. Reducing one's own latency does not just create the offensive latency-arbitrage opportunities of Section 14.3.1; it directly reduces how often one's own quotes are the stale ones being picked off.

14.5 The Economics of Speed

Speed costs money, and how much it is worth has a numerical answer. This part prices the infrastructure, the message traffic, and the queue position that high-frequency strategies depend on, alongside the one case in which the pursuit crossed into outright manipulation.

14.5.1 Cross-Market Latency Arbitrage: Futures Leading the Cash Market

Section 14.3.1 illustrated latency arbitrage within a single stock's own quotes, and the ETF-basket example extended the same logic to an ETF and its underlying basket. A third, equally common form exploits the well-documented empirical fact that broad-market index futures, such as the E-mini S&P 500 contract, typically incorporate new macroeconomic information a few milliseconds before the corresponding cash-market instruments, individual index-member stocks and index ETFs such as SPY, because futures markets are highly liquid, centrally traded in one venue, and heavily used by macro-sensitive participants reacting to news in real time.

Suppose index futures jump by 2.00 index points on a surprise economic release, from 5,000.00 to 5,002.00. Since an index ETF such as SPY is constructed to track roughly one-tenth of the underlying index level, this implies a fair-value move in the ETF of approximately $2.00/10 = \$0.20$. If SPY's own quotes take, say, 300 microseconds longer to reflect this than the futures market already has, the same stale-quote window as Section 14.3.1, a firm monitoring futures directly can buy SPY at its still-stale price and capture the difference once SPY's quotes catch up.

On a 1,000-share trade,

$$\text{Profit per event} = 1{,}000 \times \$0.20 = \$200, \qquad \text{Daily profit at 30 events/day} = 30 \times \$200 = \$6{,}000.$$

This mechanism is one of the single most economically important reasons HFT firms monitor futures order books as a leading indicator for equity and ETF trading decisions. It also explains why, during major scheduled economic releases, equity index ETFs' quoted spreads often widen noticeably in the seconds immediately following the announcement. Slower market makers in the cash market, aware that faster participants are watching futures and may already know the "correct" new price, rationally protect themselves by quoting wider until their own systems catch up. That is the same adverse-selection-driven spread widening Chapter 3 and this chapter's Glosten-Milgrom model both predict following any event that increases the perceived fraction of informed order flow.

Wulin.Suo@Queensu.ca

14.5.2 The Economics of Co-location: A Break-Even Calculation

Section 14.2.2 described co-location and low-latency infrastructure as a necessary investment without quantifying when that investment actually pays for itself. Suppose an exchange charges $25,000 per month for a co-located server rack and $10,000 per month for a direct, low-latency market data feed (as opposed to the slower, free consolidated feed most retail-facing brokers use), a total fixed cost of $35,000 per month. Using the $250-per-event profit figure from Section 14.3.1, the firm needs only

$$\frac{\$35{,}000}{\$250} = 140 \text{ profitable latency events per month, or about 6.7 per trading day,}$$

to break even on the infrastructure cost.

If the firm instead captures latency-arbitrage opportunities at the rate assumed earlier in this chapter, 20 events per day across a 21-trading-day month, its monthly profit from this single mechanism is $20 \times 21 \times \$250 = \$105{,}000$, three times the infrastructure cost. This is precisely why co-location and direct data feeds, despite their substantial recurring cost, are considered essential infrastructure rather than a discretionary expense for any firm pursuing latency-sensitive strategies: the break-even bar is low relative to the scale of opportunity a persistent speed advantage can realistically capture, which is also exactly why exchanges have a direct commercial incentive to keep offering, and continually upgrading, these paid speed advantages. The direct data feed's own $10,000-per-month cost is, in fact, doing double duty: it is also exactly what makes the SIP-versus-direct-feed arbitrage of Section 14.6.1 possible at all, since a firm relying on the free, slower consolidated feed cannot see the stale consolidated-quote window that mechanism exploits in the first place. Adding that mechanism's $3,000-per-day profit (over 21 trading days, $63,000 per month) to the $105,000 monthly profit from latency arbitrage alone brings the total monthly return on the same $35,000 infrastructure investment to $168,000, meaning the two mechanisms together, both made possible by the identical direct-feed subscription, pay for the infrastructure nearly five times over. This is a useful illustration of a broader pattern in high-frequency trading economics: a single fixed infrastructure investment, once made, typically unlocks several related, individually smaller sources of latency-based profit simultaneously, rather than paying for exactly one strategy in isolation, which is part of why the fixed costs of competing at this level, though large in absolute terms, are easier to justify than they might first appear.

14.5.3 Case Vignette: Prosecuting Spoofing

Not every controversial high-frequency practice is merely aggressive; some are illegal outright, and it is worth being precise about where that legal line falls. Spoofing, placing a large, visible order with no genuine intention of letting it execute, specifically to create a false impression of supply or demand that other participants (increasingly, other algorithms) react to, before canceling it moments later, was made explicitly illegal in U.S. futures markets under the Dodd-Frank Act of 2010, and the first criminal conviction under that provision came in 2015, against Michael Coscia, a trader who had used this technique in commodity futures markets in 2011: placing large orders on one side of the book to move the price, executing a smaller genuine order on the other side at the resulting favorable price, and canceling the large orders before they could be filled.

This case is instructive precisely because it clarifies the boundary this chapter has drawn throughout. Legitimate high-frequency market making, the Avellaneda-Stoikov and Glosten-Milgrom logic of Sections 14.4.1 and 14.3.2, involves posting genuine quotes that the firm is willing to have executed, even if many of them are canceled and replaced as fair-value estimates update. Spoofing involves posting orders specifically *because* the firm does not want them executed, using the order book's role as a public signal of supply and demand against the very participants who rely on that signal being genuine. The high cancellation rates and large order-to-trade ratios discussed in this chapter's earlier section are not, by themselves, evidence of spoofing, since legitimate market making also produces them; what distinguishes spoofing legally is the trader's intent, evidenced in practice by patterns such as consistently canceling large orders moments before they would otherwise be filled, which is precisely why regulators and exchanges monitor order-cancellation patterns as closely as executed trades.

Wulin.Suo@Queensu.ca

14.5.4 Order-to-Trade Ratios and the Economics of Message Traffic

An HFT market maker's quoting activity generates far more messages, new orders, cancellations, and modifications, than it generates executed trades, since a firm continuously updating its quotes as fair value estimates change will cancel and repost far more often than it actually transacts. Suppose a firm sends 2,000,000 order messages in a trading day and executes 40,000 trades,

$$\text{Order-to-trade ratio} = \frac{2{,}000{,}000}{40{,}000} = 50:1.$$

Many exchanges impose a fee once this ratio exceeds a specified threshold, intended to discourage excessive message traffic that consumes shared exchange infrastructure without contributing proportionally to executed volume.

At an illustrative allowed ratio of 20:1 and a fee of \$0.01 per message above that threshold,

$$\text{Allowed messages} = 40{,}000 \times 20 = 800{,}000, \qquad \text{Excess messages} = 2{,}000{,}000 - 800{,}000 = 1{,}200{,}000,$$

$$\text{Total fee} = 1{,}200{,}000 \times \$0.01 = \$12{,}000.$$

This fee structure is a direct, market-based response to a genuine externality. Each additional message a firm sends imposes a small processing cost on the exchange's shared matching infrastructure regardless of whether it results in a trade. A firm whose strategy involves posting and canceling extremely large numbers of orders relative to its executed volume therefore imposes costs on the system that a simple per-trade commission does not capture, whether its motive is legitimate quote management or the specific purpose of influencing other participants' perception of supply and demand, a practice discussed further in this chapter's regulatory section.

14.5.5 Queue Position and the Economics of Price-Time Priority

Suppose two traders each place a limit order to buy at the exact same price, at nearly the exact same moment, yet one order fills within seconds while the other waits nearly a minute, with nothing about the two orders themselves different except where each happened to land in a queue neither trader can see directly. Most equity and futures exchanges allocate fills at a given price level using strict price-time priority: orders at a better price execute first, and among orders at the same price, whichever arrived earliest is filled first. This single rule has a large, underappreciated effect on high-frequency strategy design, because it means an order's expected time to execution depends not only on how much volume trades at its price, but on exactly how many shares are queued ahead of it, a quantity a trader can observe directly from the order book's depth data.

Suppose a trader submits a limit buy order at the best bid, where 400 shares from other orders are already queued ahead of it, and incoming marketable sell orders trade through that price level at an average rate of 600 shares per minute (10 shares per second). Treating this rate as roughly constant over a short interval, the expected time until the queue ahead of the trader's order is fully executed is

$$\text{Expected wait} = \frac{400 \text{ shares}}{10 \text{ shares/second}} = 40 \text{ seconds},$$

and if the trader's own order is for 100 shares, the expected time until it is completely filled is $(400+100)/10 = 50$ seconds. This is a direct, quantifiable reason why two economically identical limit orders at the same price can have very different expected execution times, and why sophisticated market participants track queue position, not just price, as a first-class input to their trading decisions.

This mechanic also reveals a cost of canceling and replacing an order that the order-to-trade fee discussion in the previous section did not capture: a canceled order that is resubmitted, even at the exact same price, loses its accumulated time priority entirely and joins the back of a new queue. If the book has moved on by the time of resubmission such that only 250 shares are now queued ahead at that price, the trader's expected wait falls to $250/10 = 25$ seconds, which sounds like an improvement, but only because the trader gave up a queue position it had already waited to earn; had it simply held the original order, its expected remaining wait, partway through the original 40-second queue, would typically have been shorter still. This is

Wulin.Suo@Queensu.ca

precisely why HFT market-making systems are engineered to modify order size in place or adjust only when the improvement in price clearly outweighs the lost queue position, and why exchanges' matching-engine designs, whether an order retains time priority after a partial cancellation or a price-preserving modification, are themselves a significant strategic detail that HFT firms study closely alongside the message-fee schedules of Section 14.5.4.

14.6 Market Structure and Market Quality

Whether high-frequency trading improves markets depends on how the market is wired, and on what improvement is measured against. This part covers the data-feed architecture that creates the fastest participants' advantage, and the metrics by which the result is judged.

14.6.1 Direct Data Feeds, the Consolidated Tape, and Reg NMS

Section 14.2.2 explained why firms pay for co-location and low-latency transmission, but did not explain why a data feed's speed matters independently of an exchange's own matching latency: in U.S. equity markets, every exchange publishes its own direct data feed in addition to contributing its quotes to a consolidated feed, the Securities Information Processor (SIP), that aggregates quotes from all exchanges into the single National Best Bid and Offer (NBBO) that Regulation NMS (National Market System) requires brokers to respect when routing and executing retail and institutional orders.

Because the SIP must collect, synchronize, and republish quotes from every exchange, it is structurally slower than any single exchange's own direct feed, historically by low single-digit milliseconds, a gap of the same order of magnitude as the fiber-versus-microwave difference computed in Section 14.2.2. A firm subscribing to direct feeds from every exchange, rather than relying on the SIP, can therefore observe the true, up-to-the-microsecond NBBO before it is reflected in the publicly disseminated consolidated quote, and can react to a price change on one exchange before a participant relying solely on the SIP even sees that the price has moved. This is not a separate strategy from the latency arbitrage of Section 14.3.1, it is the identical stale-quote mechanism, but arising from a difference in data infrastructure and regulatory design rather than pure physical transmission distance.

Reg NMS's order protection rule, which prohibits trading through a protected quote displayed on another exchange, was intended to guarantee execution quality across a fragmented market of dozens of trading venues. But it simultaneously created a durable structural incentive for the direct-feed investment this section describes. A broker or venue that must check the NBBO to comply with the rule needs that information as quickly as possible, and a firm that already has it fractionally sooner than everyone else can trade profitably on the resulting gap. This is a clear illustration of a recurring theme in this book: a well-intentioned regulation, protecting investors from trade-throughs, can create new, unanticipated economic incentives, here for a latency arms race in market data infrastructure, that the rule's designers did not set out to create.

A numerical illustration, structured identically to Section 14.3.1's stale-quote example but operating at the millisecond, rather than microsecond, scale of the SIP-versus-direct-feed gap, shows the same mechanism at work. Suppose a stock's true NBBO moves the moment a large order executes on one exchange, visible instantly on that exchange's own direct feed, but the SIP-based consolidated NBBO, and therefore every market participant relying on it rather than paying for direct feeds, does not reflect the new price for 3 milliseconds.

A firm subscribed to direct feeds can trade against counterparties still quoting or routing off the stale SIP-based NBBO throughout that window; if this firm captures \$0.02 per share on 10,000 shares each time this occurs, and the pattern recurs roughly 15 times per trading day across its traded universe,

$$\text{Profit per event} = 10{,}000 \times \$0.02 = \$200, \qquad \text{Daily profit} = 15 \times \$200 = \$3{,}000,$$

a meaningfully smaller daily figure than Section 14.3.1's microsecond-scale example, since a millisecond-wide window based on a known, structural feed delay is rarer and more heavily competed away than a genuinely novel news event, but it recurs with much greater regularity and predictability, since the SIP-versus-direct-feed gap exists on every single trading day rather than only when unpredictable news arrives. This regularity

Wulin.Suo@Queensu.ca

is precisely why subscribing to direct feeds from every exchange is considered close to a baseline requirement for any serious HFT operation, rather than merely one latency-reduction technique among several: unlike a genuinely novel news event, the SIP's structural latency disadvantage is a known, exploitable constant that any firm without direct feeds pays every single day, whether or not it ever experiences a single dramatic latency-arbitrage event of the kind described earlier in this chapter.

14.6.2 Measuring Market Quality: Effective Spreads and Price Discovery

The quoted bid-ask spread, introduced in Chapter 3, is not always what a trader actually pays, since orders can execute inside the spread, an aggressive order can receive price improvement over the posted quote, or a large order can walk through several price levels as in Chapter 3's order-book example. The effective spread corrects for this by comparing the execution price directly to the prevailing midpoint at the moment the order arrived,

$$\text{Effective Spread} = 2 \times |P_{\text{execution}} - P_{\text{midpoint}}|.$$

If the midpoint at order arrival is $(\$50.07 + \$50.08)/2 = \$50.075$ and a buy order executes at the full quoted ask of \$50.08,

$$\text{Effective spread} = 2 \times |50.08 - 50.075| = \$0.01,$$

identical to the quoted spread, meaning the trade received no price improvement.

If instead the same order executes at \$50.078, perhaps because a competing market maker's hidden or "odd-lot" liquidity intervened,

$$\text{Effective spread} = 2 \times |50.078 - 50.075| = \$0.006,$$

narrower than the \$0.01 quoted spread. Academic studies of market quality frequently find that effective spreads have narrowed considerably as HFT market making has grown. Proponents cite this as direct evidence that HFT liquidity provision benefits ordinary investors through tighter realized trading costs. Critics respond that narrower average effective spreads coexist with the periodic liquidity withdrawal and adverse-selection costs discussed in the rest of this chapter, so a single average metric can mask what happens specifically during the moments of stress when execution quality matters most.

A Worked Example: Roll's Implied Spread from Bid-Ask Bounce

Both measures above need quote data, the prevailing bid and ask, to compute. Chapter 3 introduced a related idea in passing: bid-ask bounce, prices mechanically alternating between the bid and the ask as buy and sell orders arrive, induces negative autocorrelation in observed price changes even when nothing about the underlying fundamental value has moved. Roll's model (Roll, 1984) turns this same mechanical artifact from a nuisance into a measurement tool: if bounce is the only thing driving that negative autocorrelation, the spread itself can be recovered directly from a sequence of transaction prices alone, with no quote data at all.

The model assumes each observed trade price is the efficient price m_t plus or minus half the spread, $P_t = m_t + c\,I_t$, where $c = s/2$ is half the spread and $I_t \in \{+1, -1\}$ is the trade-direction indicator (buy or sell), drawn independently and with equal probability from one trade to the next, and independent of the efficient price's own random-walk innovations. Under these assumptions the first-order autocovariance of price changes reduces to

$$\text{Cov}(\Delta P_t,\, \Delta P_{t-1}) = -c^2 = -\frac{s^2}{4},$$

so that, whenever this sample covariance is negative, as bid-ask bounce predicts, the spread can be recovered by inverting the relationship,

$$\hat{s} = 2\sqrt{-\widehat{\text{Cov}}(\Delta P_t,\, \Delta P_{t-1})}.$$

Suppose a stock's true, constant bid-ask spread is \$0.02 (bid \$50.00, ask \$50.02), and 20 consecutive trades arrive, each independently and equally likely to hit the bid or the ask. The resulting price changes bounce between $-\$0.02$, \$0, and $+\$0.02$ depending on whether consecutive trades land on the same side or opposite sides of the spread. Computing the sample covariance between each price change and the

Wulin.Suo@Queensu.ca

one immediately before it, averaged across the 18 consecutive pairs available from 19 price changes, gives $\widehat{\mathrm{Cov}}(\Delta P_t, \Delta P_{t-1}) \approx -0.0000889$, and therefore

$$\hat{s} = 2\sqrt{0.0000889} \approx \$0.0189,$$

within about 6% of the true \$0.02 spread, recovered entirely from trade prices, without ever observing a single quote. This gap is not a computational error. Roll's estimator is unbiased only in expectation over infinitely many i.i.d. trades. Any single finite sample will typically land somewhat above or below the true value purely from sampling noise, and twenty trades is a realistic size for measuring the spread of a single quiet stock over a short window. That is the same finite-sample theme that has recurred throughout this book's other estimators. A larger sample of trades shrinks this gap, exactly as a larger sample shrinks the standard error of any statistical estimate.

Roll's model has a well-known practical limitation directly visible in its own formula: the estimator is undefined whenever the sample covariance comes out non-negative. That happens with some regularity in short or quiet real samples. Genuine information arrival, momentum, and other economically meaningful sources of serial correlation compete with, and can overwhelm, the comparatively weak bounce signal the model relies on exclusively. This is precisely why practitioners typically prefer the effective-spread and quote-based measures developed earlier in this section, which use richer data and do not depend on this one specific, occasionally-violated statistical assumption. Roll's estimator is reserved for settings where quote data genuinely is not available.

14.7 Prediction and the Decision Pipeline

Everything so far has reacted to observable state. This part turns to forecasting the next price move from order flow, and to the uncomfortable trade-off between a model's accuracy and the microseconds it costs to evaluate.

14.7.1 Predictive Modeling: Order Flow Imbalance

Not every HFT model is a formal microstructure model like Kyle's or Glosten-Milgrom; many production trading signals are simple statistical predictive models, in the same regression spirit as Chapter 6, fit directly to order-book data. The most widely used such signal is order flow imbalance (OFI), the change in resting buy-side depth minus the change in resting sell-side depth at the top of the book between two snapshots, on the theory that a book getting more (relatively) bid-heavy predicts a price increase over the next few ticks, and vice versa. Table 14.2 shows six order-book snapshots with the resulting OFI and the price change over the subsequent short interval.

Snapshot	1	2	3	4	5	6
OFI (contracts)	50	−30	80	−60	20	40
Next price change (cents)	2	−1	3	−2	1	2

Table 14.2: Order flow imbalance and the subsequent price change

Fitting the same simple OLS regression used since Chapter 6, $\Delta P = \alpha + \beta\,\mathrm{OFI} + \varepsilon$, gives

$$\hat{\alpha} \approx 0.22 \text{ cents}, \qquad \hat{\beta} \approx 0.0369 \text{ cents per contract of imbalance}, \qquad R^2 \approx 0.99,$$

an unusually tight fit for illustration; real order-book data produces a much noisier, though still typically statistically significant, relationship. For a new snapshot with OFI = 65, the model predicts a price move of $0.22 + 0.0369(65) \approx 2.62$ cents over the next short interval. Following the same standard-error procedure introduced in Chapter 6, with only $6 - 2 = 4$ degrees of freedom, $\mathrm{SE}(\hat{\beta}) \approx 0.0016$, giving $t \approx 23.0$ and a 95% confidence interval for $\hat{\beta}$ of roughly $(0.032, 0.041)$. The coefficient is estimated unusually precisely here, but only because this illustrative dataset was deliberately constructed to fit almost perfectly. A genuine production OFI model, fit to thousands of noisier real snapshots rather than six clean illustrative

Wulin.Suo@Queensu.ca

ones, should be expected to show a visibly wider confidence interval alongside a lower, though still often statistically significant, R^2.

This is, in effect, the simplest possible member of the machine learning model family Chapter 6 introduced, and it illustrates precisely the latency-versus-complexity tradeoff developed next. A single-feature linear regression like this one can be evaluated in a few machine instructions. A richer model incorporating dozens of order-book and trade-history features, of the kind a gradient-boosted tree or neural network from Chapter 6 could in principle learn, may predict more accurately but takes measurably longer to compute. That cost matters enormously at HFT timescales and much less at the daily or weekly horizons most of Chapter 6's examples were evaluated at.

14.7.2 The Decision Pipeline, and Machine Learning's Latency Cost

Every high-frequency trading decision, whether market making, latency arbitrage, order-flow-imbalance prediction, or the sequential-trade updating of Section 14.3.2, passes through the same basic pipeline, illustrated in Figure 14.2: market data arrives, a fair-value or signal model consumes it, a risk check validates the resulting order, and the order is transmitted to the exchange. Every stage adds latency, and at HFT timescales, the choice of model at the second stage is itself a latency decision, not merely an accuracy decision.

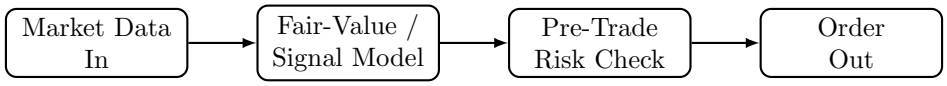


Figure 14.2: The high-frequency trading decision pipeline

Chapter 6 developed machine learning models ranging from a simple linear regression to deep neural networks, evaluating them primarily on predictive accuracy. At HFT timescales, a model's inference latency, how long it takes to actually produce a prediction from new data, becomes a competing objective of comparable importance. A simple threshold rule or a small logistic regression can typically produce a prediction in low single-digit microseconds, and a shallow decision tree or small gradient-boosted ensemble in perhaps ten to a few tens of microseconds. A deep neural network of even moderate size can take from the high tens of microseconds into the low milliseconds, depending heavily on its architecture and the hardware it runs on. These are illustrative, order-of-magnitude figures rather than precise benchmarks, since actual latency depends heavily on implementation and hardware. If a firm's total available decision budget, the time between receiving new market data and needing to act on it before a competitor does, is itself only tens of microseconds, a more accurate but slower model can be strictly worse in practice than a less accurate but faster one, since arriving at the correct decision after a competitor has already traded captures none of the opportunity at all. This is precisely why the simplest models in Chapter 6, not the most sophisticated ones, dominate the most latency-sensitive layers of high-frequency trading, with more complex machine learning models, including the deep learning methods introduced in Chapter 6, reserved for slower-moving decisions, such as which symbols to focus market-making capital on today, where microseconds do not matter but predictive accuracy does.

Concept Drift and Model Retraining at High Frequency

A model's inference speed is only half of the latency-sensitive machine learning problem; the other half is how quickly the relationship the model has learned goes stale. Chapter 7 introduced the idea that a time series' statistical properties can change over time. The same concept, there called non-stationarity, appears here as concept drift. The true relationship between order flow imbalance and subsequent price changes in Section 14.7.1's example is not a fixed physical law but an empirical regularity that depends on the current mix of market participants, prevailing volatility, and even the specific stock's tick size regime, all of which change over horizons far shorter than the daily or monthly retraining cycles common in Chapter 6's slower applications.

A high-frequency predictive model like the OFI regression is therefore typically retrained far more frequently than a model built on Chapter 6's daily or lower-frequency data, sometimes multiple times within a single trading day, using only a recent rolling window of observations rather than the firm's entire trading

Wulin.Suo@Queensu.ca

history, precisely so that the fitted coefficients $\hat{\alpha}$ and $\hat{\beta}$ continue to reflect current market conditions rather than a stale regime. This creates a direct, quantifiable tradeoff of its own: a very short rolling window adapts quickly to genuine regime change but produces noisier coefficient estimates from fewer observations, the bias-variance tradeoff Chapter 6 introduced in a different context, while a longer window produces more stable estimates that may lag a genuine shift in market structure. Firms address this in practice with automated monitoring of a live model's out-of-sample predictive accuracy, triggering an automatic retrain, or in the worst case an automatic shutdown of that particular signal via the kill-switch mechanism described next, the moment realized accuracy degrades below a pre-specified threshold, since at HFT timescales there is no practical opportunity for a human to notice a model's slow decay and intervene manually before meaningful losses accumulate.

14.8 Risk Controls and Systemic Episodes

A system that can trade thousands of times a second can also lose money thousands of times a second. This part covers the controls that stop it, and the afternoon on which the interaction of many such systems briefly emptied the market.

14.8.1 Risk Controls: Kill Switches and Pre-Trade Risk Checks

Chapter 13's Knight Capital vignette showed how a firm can lose an enormous amount of money in well under an hour when automated trading infrastructure malfunctions, precisely because the same speed that makes automated strategies profitable also makes an undetected error catastrophic before a human can intervene. High-frequency trading firms and the exchanges they trade on have developed several layers of technical risk control specifically in response to this asymmetry. Pre-trade risk checks validate every outgoing order against limits, maximum order size, maximum position, a fat-finger price collar around the current market price, before it ever reaches the exchange, rejecting anything that would violate those limits regardless of what the trading logic upstream intended to do. A kill switch is a mechanism, ideally independent of the trading logic itself, that can immediately halt all order flow from a firm or a specific strategy, either automatically (triggered by a breach of a risk limit, such as an unusually large accumulated position or an abnormal rate of message traffic) or manually by a human supervisor.

A concrete price-collar example shows how a pre-trade check actually intervenes. Suppose a stock's last traded price is \$50.00, and a firm's pre-trade risk check enforces a fat-finger collar of $\pm 2\%$ around that reference price, rejecting any outgoing order priced outside $[\$49.00, \$51.00]$. If a software defect causes the trading algorithm to submit a buy order at \$45.00, perhaps from a decimal-place error in an upstream price feed, the pre-trade check rejects it before it ever reaches the exchange, entirely independent of whatever flawed logic generated that price in the first place. Without this check, an order at \$45.00 against a genuine \$50.00 market would either execute at a wildly favorable price for a counterparty willing to sell (an immediate, avoidable loss) or simply fail to fill at all while consuming exchange bandwidth and risking a cascading message-traffic problem of its own; the collar makes the failure mode a rejected message logged for review rather than an executed trade at an obviously erroneous price, the kind of independent, code-level safeguard the Knight Capital incident lacked.

The Knight Capital failure is instructive here precisely because it was not a failure of trading strategy but a failure of deployment and risk-control discipline: obsolete code was left active on production servers, and no automated control caught the resulting runaway order flow before it had generated hundreds of millions of dollars in unintended positions. This connects directly to the model risk management framework Chapter 8 introduced for financial risk models generally: an algorithmic trading system, like a VaR model or a credit-scoring model, requires independent testing before deployment, staged rollout rather than immediate full-scale activation, and continuous, automated monitoring with the authority to intervene, precisely because the speed advantage that makes high-frequency trading commercially viable is the same speed that makes a human-in-the-loop safeguard, by itself, too slow to prevent a catastrophic loss.

Wulin.Suo@Queensu.ca

14.8.2 The 2010 Flash Crash Revisited: The Hot-Potato Effect

Chapter 3 introduced the May 2010 Flash Crash, in which the Dow Jones Industrial Average fell and then recovered nearly 1,000 points within minutes. The subsequent regulatory investigation identified a specific mechanism, sometimes called the "hot-potato effect," through which high-frequency trading amplified the initial shock: as a large sell order began moving prices down, several high-frequency firms, each individually managing inventory risk exactly as Section 14.4.1's model prescribes, rapidly bought and resold the same underlying shares among themselves, passing accumulating inventory from one firm to another within fractions of a second rather than absorbing it, without any single firm's net inventory position changing very much. This rapid intra-HFT passing inflated trading volume dramatically without providing the genuine risk-absorbing liquidity that volume figures alone would suggest was available, and as more firms simultaneously hit the same inventory and risk limits that Section 14.4.1's model would recommend, many withdrew from market making entirely within moments of each other, removing liquidity from the market at precisely the moment it was most needed. This episode illustrates a systemic version of a risk that Chapter 8 discussed at the level of an individual VaR model. A risk-management rule that is individually sensible, reduce inventory and widen quotes when risk rises, can produce a collectively destabilizing outcome when many participants, largely using similar models and facing similar conditions, follow that rule simultaneously. That is the same crowding dynamic Chapter 12 raised in the context of long-horizon factor investing and Chapter 13 raised in the context of statistical arbitrage, here compressed into a timescale of seconds rather than months. Circuit breakers, introduced in Chapter 13 as a response to this kind of event, exist specifically to interrupt this feedback loop by forcing a pause before a rapid, mechanically self-reinforcing price move can fully unwind.

14.9 The Regulatory Debate: Does HFT Help or Hurt Markets?

High-frequency trading remains genuinely contested among regulators, academics, and market participants, and it is worth stating both sides of the debate fairly rather than presenting only one. Proponents point to the narrower effective spreads documented in this chapter's market-quality discussion, faster and more efficient price discovery (new information gets reflected in prices more quickly when fast participants are constantly monitoring and reacting to it), and the sheer increase in continuously available liquidity relative to the slower, more manual market-making arrangements HFT has largely displaced. Critics make three arguments. The Flash Crash's hot-potato effect is evidence that HFT liquidity can be illusory precisely when it is needed most. The adverse-selection costs quantified in Sections 14.3.1 and 14.4.2 are ultimately borne by slower participants, including many ordinary investors' pension and mutual funds. And manipulative practices such as spoofing, prosecuted under securities law in numerous cases, involve a trader posting large orders with no genuine intention of executing them, specifically to create a false impression of supply or demand that other participants, often other algorithms, react to.

Both perspectives can be correct simultaneously about different aspects of the same activity: HFT market making of the kind formalized in this chapter's inventory model plausibly does tighten effective spreads and improve liquidity most of the time, while HFT's aggregate behavior during genuine stress events, and a minority of participants' use of manipulative rather than genuinely liquidity-providing strategies, plausibly does create real, periodic costs and risks that average market-quality statistics understate. This is precisely why regulatory responses have been targeted rather than blanket: circuit breakers address the hot-potato and flash-crash risk specifically, order-to-trade fees address excessive message traffic specifically, and spoofing prosecutions address manipulative intent specifically, rather than any single rule attempting to address "high-frequency trading" as if it were one undifferentiated activity.

Regulatory approaches also differ meaningfully across jurisdictions, reflecting different judgments about which of these targeted interventions matter most. The European Union's MiFID II framework takes a more structural approach than the largely case-by-case U.S. enforcement model this chapter has described, requiring firms engaged in algorithmic trading to register specifically as such, to test their algorithms before deployment in a manner regulators can audit directly, a formalized, externally verifiable version of the internal model risk management discipline described in this chapter's risk-controls section, and to maintain minimum resting times for certain order types specifically to discourage the very high cancellation rates this chapter's message-traffic discussion analyzed.

Wulin.Suo@Queensu.ca

The SEC's own tick-size pilot program, which temporarily widened the minimum price increment for a sample of smaller-capitalization stocks, was designed to test one of this chapter's own structural claims directly. Would a wider tick size, by making price-time priority (Section 14.5.5) a less finely graded advantage, reduce HFT-driven quote competition and thicken displayed liquidity, or would it simply widen effective spreads without a corresponding liquidity benefit? The pilot's mixed results are a useful reminder that even a well-designed natural experiment does not always resolve a genuinely contested empirical question. Concretely, the SEC's Tick Size Pilot, which ran from 2016 to 2018, widened the minimum price increment from the standard \$0.01 to \$0.05 for roughly 1,200 small-capitalization stocks split across several test groups with different additional rules (some also required a "trade-at" rule forcing certain orders to route to the quoting exchange rather than internalizing them). The pilot's published results found that quoted and effective spreads widened materially for pilot stocks relative to a comparable control group, exactly as a wider mandatory tick size would mechanically predict, but found no correspondingly clear improvement in displayed depth or overall market quality large enough to offset that widened spread, a result closer to critics' prediction (wider spreads with limited offsetting benefit) than to proponents' hoped-for outcome (thicker, more stable liquidity). The SEC ultimately did not extend the pilot's tick-size widening as a permanent, market-wide rule, though the episode continues to inform ongoing tick-size and market-structure policy discussions, a reminder that a real, carefully designed regulatory experiment can still produce a result too ambiguous, or too specific to the particular stocks and rules tested, to settle a structural question as broad as "does high-frequency trading, on net, help or hurt ordinary investors."

14.10 Challenges in High-Frequency Trading

Several challenges recur across nearly every high-frequency trading application, and many connect directly to themes developed elsewhere in this book.

The arms race has diminishing, and unevenly distributed, returns. Every dollar spent on shaving another few microseconds off latency, whether through co-location, microwave links, or specialized hardware, raises the cost of competing at the top of the speed distribution without necessarily increasing the total liquidity or price-discovery benefit to the market as a whole. Critics argue this is a socially wasteful arms race. Participants argue that whichever firm is fastest captures a disproportionate share of the available latency-arbitrage profits quantified in Section 14.3.1, making the investment individually rational even if its aggregate social value is genuinely debatable.

Technology and operational risk dominate strategy risk. As the Knight Capital vignette in Chapter 13 and this chapter's discussion of kill switches emphasized, the dominant risk in high-frequency trading is often not that a strategy's economic logic is wrong, but that its implementation fails in an unanticipated way at a speed that outpaces human intervention, a risk category that requires engineering and governance discipline as much as financial modeling skill.

Backtesting at microsecond resolution is extraordinarily data- and infrastructure-intensive. The walk-forward validation discipline introduced in Chapter 7 and reiterated throughout Chapter 13 still applies, but testing a microsecond-sensitive strategy requires tick-by-tick, timestamp-accurate historical data and a simulation environment that faithfully reproduces queue position and latency, a considerably higher bar than the daily or minute-level data sufficient for the slower strategies of earlier chapters.

Regulatory and market-structure change can eliminate an edge overnight. A strategy whose profitability depends on a specific exchange fee schedule, order-to-trade ratio rule, or co-location arrangement, the market-structure dependence flagged in Chapter 13's challenges section, is especially exposed in high-frequency trading, where such rules are frequently and deliberately adjusted by regulators and exchanges specifically in response to the practices this chapter describes.

Crowding and correlated risk management compound during stress. The hot-potato mechanism behind the 2010 Flash Crash is a standing risk whenever many participants run similar inventory-management logic simultaneously; a firm's own risk model can be individually well-calibrated and still contribute to a collectively destabilizing outcome during a genuine shock, the systemic risk theme Chapter 8 raised in the context of correlated VaR models across many institutions.

High fixed costs create barriers to entry and market concentration. The co-location, direct data feed, and specialized-hardware investments described throughout this chapter are largely fixed costs that do

Wulin.Suo@Queensu.ca

not scale down for a smaller firm, which is why high-frequency market making and latency arbitrage are, in practice, dominated by a comparatively small number of well-capitalized specialist firms rather than open to any participant with a good trading idea. This concentration raises its own policy question, distinct from the liquidity and manipulation concerns discussed elsewhere in this chapter: whether a market structure that rewards infrastructure spending this heavily is producing the most efficient possible level of price discovery, or merely the most efficient level attainable given a technology arms race that smaller, potentially equally skilled participants cannot afford to enter.

A predictive signal's own success can erode it. An order-flow-imbalance model, or any other short-horizon predictive signal, is discovered being priced into the market by every other sophisticated participant who independently arrives at a similar signal. That is a specific instance of the concept-drift problem this chapter raised earlier, but with a distinctive twist. The drift here is caused not by an exogenous change in market conditions but endogenously, by the very act of the signal being profitably traded on at scale by many participants simultaneously. This is a faster, more severe version of the crowded-factor concern Chapter 12 raised for slower, cross-sectional trading signals: at HFT timescales, a genuinely novel predictive relationship can be discovered, exploited, and substantially arbitraged away within weeks or months rather than years, which is why continuous signal research and turnover is itself an operating cost of doing business in this space, not a one-time model-building exercise.

14.11 Key Takeaways

High-frequency trading applies the market-making, execution, and inventory-management ideas of Chapter 13 at a timescale where physical latency becomes a first-order economic variable, and where the operational risks introduced by the Knight Capital vignette move from a cautionary footnote to the central concern. Latency arbitrage translates a pure speed advantage, whether from co-location, a faster physical transmission medium, or a faster data feed under Reg NMS's market-data structure, directly into dollar profits extracted from slower participants' stale quotes, whether within a single stock, between an ETF and its basket, or between index futures and the cash market, and Chapter 13's Avellaneda-Stoikov inventory model shows precisely why operating at a much shorter time horizon allows a market maker to run profitably with a much smaller inventory-risk penalty.

Price-time priority means that queue position, not just price, is a first-class economic variable in HFT strategy design, and order-to-trade ratio fees and effective-spread measurement are two of the concrete tools regulators and researchers use to evaluate whether a given firm's or the market's aggregate high-frequency activity is contributing genuine liquidity or primarily consuming shared infrastructure.

The 2010 Flash Crash's hot-potato effect shows how individually rational, Chapter-13-style inventory management can still produce a collectively destabilizing outcome when many participants follow similar rules simultaneously.

As with every method in this book, the appropriate response to these risks is not to avoid the technology but to govern it carefully: independent testing, staged deployment, automated kill switches, and continuous monitoring for concept drift are not optional extras for a high-frequency trading operation but the direct, speed-adjusted analogue of the model risk management discipline Chapter 8 developed for financial models generally.

14.12 Suggested Exercises

1. Using Section 14.3.1, recompute the daily latency-arbitrage profit if Firm F can execute against 800 shares per event and such events occur 35 times per day, holding the \$0.05 price jump fixed.

2. Explain, in your own words, why a persistent millisecond-scale latency advantage translates into a repeatable dollar profit rather than a one-time gain, referencing the stale-quote mechanism in Section 14.3.1.

3. Using the reservation-price formula in Section 14.4.1, compute the skew for $q = 8$, $\gamma = 0.15$, $\sigma = 1.5$, and $T - t = 0.01$, and compare it to the chapter's example.

Wulin.Suo@Queensu.ca

4. A high-frequency market maker completes 80,000 fills (40,000 round trips) per day at an average size of 150 shares, capturing a half-spread of \$0.004 per share on each fill (a full spread of \$0.008 per round trip), with 4% of fills toxic at an average loss of \$0.03 per share. Compute gross spread capture, adverse selection loss, and net daily P&L.

5. A firm sends 3,500,000 order messages and executes 50,000 trades in a day. Compute its order-to-trade ratio, and its fee at an allowed ratio of 25:1 and a per-excess-message fee of \$0.008.

6. An order executes at \$25.02 when the prevailing midpoint was \$25.015. Compute the effective spread, and explain whether this trade received price improvement relative to a \$0.02 quoted spread.

7. Explain, using the hot-potato mechanism in this chapter, why high trading volume during a market stress event does not necessarily indicate that genuine, risk-absorbing liquidity was available.

8. Describe two specific technical risk controls a high-frequency trading firm should have in place to prevent a Knight-Capital-style incident, and explain what each one would have needed to do differently on August 1, 2012 to prevent the loss described in Chapter 13.

9. Discuss, in a short paragraph, why the same empirical finding (narrower average effective spreads) can be cited by both proponents and critics of high-frequency trading as evidence for their position, referencing this chapter's regulatory debate section.

10. Pick two of the challenges listed in this chapter and describe a concrete scenario, not already used in the text, in which that challenge could cause a high-frequency trading strategy to lose money or draw regulatory scrutiny despite a sound underlying economic rationale.

11. Using Section 14.5.5, compute the expected wait time for a 200-share limit order joining a queue of 900 shares at the best bid, given an average execution rate of 15 shares per second, and explain why canceling and resubmitting that order at the same price could increase its expected time to fill.

12. Explain, using Section 14.7.1's concept-drift discussion, why a predictive model like the order-flow-imbalance regression needs to be retrained far more frequently than the credit-scoring model in Chapter 6, and describe one practical way a firm could detect that retraining is needed without human intervention.

13. Index futures jump by 3.50 points and the corresponding ETF tracks one-tenth of the index level. Using the cross-market latency arbitrage logic in this chapter, compute the implied ETF fair-value move, the profit on a 2,000-share trade, and the daily profit at 25 such events per day.

14. Using this chapter's SIP-versus-direct-feed example, recompute the daily profit if the per-share capture were \$0.015 instead of \$0.02 but the number of daily events rose to 22 (reflecting a more liquid, more frequently mispriced stock), and compare the result to the original \$3,000 figure.

15. Using this chapter's order-flow-imbalance regression, compute the width of the 95% confidence interval for $\hat{\beta}$ (the upper bound minus the lower bound), and explain why this width would be expected to grow substantially if the same regression were refit on a much noisier, more realistic sample of order-book snapshots.

16. Using this chapter's fat-finger price-collar example, determine whether an outgoing order at \$48.50 would be accepted or rejected under the same \$50.00 reference price and $\pm 2\%$ collar, and recompute the acceptable price range if the collar were widened to $\pm 3\%$.

17. Using this chapter's co-location break-even calculation, recompute the combined monthly profit from latency arbitrage and the SIP-versus-direct-feed mechanism if the SIP-based arbitrage instead occurred only 10 times per day rather than 15, and state the resulting ratio of total monthly profit to the \$35,000 monthly infrastructure cost.

Wulin.Suo@Queensu.ca

18. Using Section 14.4.1's spread-floor calculation, recompute the total optimal spread δ and the percentage reduction from its $T - t = 1$ value if κ were instead 3.0 (a market where order arrivals respond less sensitively to quote competitiveness), holding $\gamma = 0.1$ and $\sigma = 2$ fixed, and explain whether the spread floor rises or falls relative to the $\kappa = 1.5$ case in the text.

19. Explain, in your own words, why the inventory-risk term of the optimal spread shrinks to nearly zero as $T - t$ shrinks while the order-arrival-elasticity term does not, and describe what kind of market-making cost the elasticity term is actually compensating the market maker for.

20. Using Section 14.6.2's Roll's-model example, explain what it would mean, in terms of the underlying assumptions of the model, if the sample covariance of price changes for a different stock's trade sequence came out positive rather than negative, and why the formula $\hat{s} = 2\sqrt{-\widehat{\mathrm{Cov}}(\Delta P_t, \Delta P_{t-1})}$ would fail in that case.

21. **Challenge (synthetic data).** Real, order-by-order limit-order-book data at the time resolution this chapter's examples assume (LOBSTER, Databento, WRDS NYSE TAQ) is paid or license-restricted, so generate a synthetic session instead: simulate 30 minutes of trading with Poisson order arrivals, limit orders arriving at $\lambda_L = 20$ per second spread uniformly across 10 price levels on each side, and market orders arriving at $\lambda_M = 2$ per second. (a) Reconstruct the resulting limit order book at 1-second snapshots and compute the realized bid-ask spread and quoted depth over the session, comparing your synthetic session's average spread to the 1-2 cent NBBO spread this chapter assumes for a liquid large-cap stock. (b) Inject a single large order arriving 50 milliseconds ahead of the rest of the market reacting to a simulated price-moving news event, and measure the profit it could extract using this chapter's co-location break-even calculation. (c) Discuss which of your simulation's design choices, arrival rates, order-size distribution, the absence of any strategic order cancellation, are least realistic relative to how this chapter describes real high-frequency venues behaving, and what real dataset, even a paid one, would be needed to validate or replace each assumption.

14.13 Further Reading

- Aldridge, *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*. A comprehensive practitioner-oriented treatment of the strategies, technology, and risk management introduced in this chapter.

- Lewis, *Flash Boys: A Wall Street Revolt*. A widely read journalistic account of the latency arms race and co-location practices discussed in Section 14.2.2, useful context despite its explicitly critical perspective on the practices it describes.

- Budish, Cramton, and Shim, "The High-Frequency Trading Arms Race: Frequent Batch Auctions as a Market Design Response," *Quarterly Journal of Economics*. An influential academic analysis of the latency-arbitrage mechanism formalized in Section 14.3.1 and a proposed market-design alternative to continuous trading.

- U.S. Securities and Exchange Commission and Commodity Futures Trading Commission, *Findings Regarding the Market Events of May 6, 2010*. The official joint regulatory report documenting the hot-potato effect and other mechanisms behind the Flash Crash discussed in this chapter.

- Menkveld, "High-Frequency Trading and the New Market Makers," *Journal of Financial Markets*. An empirical study of high-frequency market making and its effect on market quality, directly relevant to this chapter's discussion of effective spreads.

- Roll, "A Simple Implicit Measure of the Effective Bid-Ask Spread in an Efficient Market," *Journal of Finance*. The original derivation of the bid-ask-bounce spread estimator worked through in Section 14.6.2.

Wulin.Suo@Queensu.ca

- U.S. Securities and Exchange Commission, *Regulation NMS Adopting Release*. The primary regulatory source for the order protection rule and consolidated-tape (SIP) requirements discussed in this chapter's treatment of direct data feeds.
- O'Hara, "High Frequency Market Microstructure," *Journal of Financial Economics*. A survey of how high-frequency trading has reshaped classical market microstructure theory, including the Glosten-Milgrom and PIN models developed earlier in this chapter.

Wulin.Suo@Queensu.ca

Chapter 15

Fraud Detection, Compliance, and Financial Crime

15.1 Introduction

Chapter 1 previewed this chapter with a single number: a fraud detection model that correctly flags 95% of actual fraud and mistakenly flags only 3% of legitimate transactions still produces a flagged transaction that is fraudulent only about 24% of the time, because fraud is rare enough that even a small false-positive rate generates many more false alarms than true detections. That number is not a curiosity; it is the central operating constraint of every fraud detection and anti-money-laundering (AML) system in the financial industry, and this chapter is built around understanding and managing it. Chapter 6 developed classification models and the confusion matrix in the abstract, using loan default as the running example; Chapter 9 applied a closely related toolkit, logistic regression, scoring, and rating models, to credit risk. This chapter applies the same statistical machinery to a different, related family of problems: detecting fraud, laundered money, and other financial crime, where the base rate of the event being detected is typically far lower than a loan portfolio's default rate, and where the operational and legal consequences of a false positive or a missed detection are correspondingly distinct.

Four worked examples run through the chapter. The Bayes' theorem calculation from Chapter 1 is scaled up to a full day of transaction volume, converting an abstract posterior probability into a concrete daily confusion matrix and an alert queue a compliance team actually has to work through. A small fraud-scoring logistic regression, in the same spirit as Chapter 6's confusion matrix example and Chapter 9's credit-scoring model, illustrates how a handful of transaction features combine into a fraud probability, and where such a model can still miss a genuine case. A Kaplan-Meier survival analysis, a technique new to this book, estimates how long flagged accounts typically remain active before a confirmed fraud closes them, introducing censored observations, data this book has not previously had to handle. Finally, a structuring example shows how AML monitoring rules are built around a specific, legally defined reporting threshold, and how criminals attempt to evade it.

15.2 Detecting Fraud

Fraud is a rare-event classification problem, and this part builds a detector while confronting exactly how rare the positives are.

15.2.1 The Landscape of Financial Crime and Where AI Fits

Suppose a bank builds an excellent fraud-scoring model, one that catches nearly every stolen-card transaction, and considers its detection obligations largely satisfied, only to later face a regulatory fine for failing to flag an unrelated money-laundering scheme that a fraud model was never designed to catch in the first place. "Fraud detection" understates the breadth of what financial institutions are legally required to monitor for.

Wulin.Suo@Queensu.ca

Table 15.1 organizes the main categories of financial crime this book touches on, the detection approach each typically relies on, and where it is developed.

Crime type	Typical detection approach	Developed in
Card and payment fraud	Real-time transaction scoring, anomaly detection	Sections 15.2.2, 15.2.3
Account takeover	Behavioral biometrics, device and login anomaly detection	Section 15.3.1
Synthetic identity fraud	Cultivation-pattern and shared-attribute linkage; unsupervised detection where labels are absent	Section 15.2.5
Authorized push payment (scam) fraud	Deviation from the customer's own payment history; mule detection on the receiving side	Section 15.2.5
Money laundering	Rule-based transaction monitoring, network analysis, structuring detection	Section 15.3.3
Market manipulation (spoofing, layering)	Order-book pattern surveillance	Chapter 14's spoofing case vignette
Insider trading	Anomaly detection linking trading activity to material news events	Chapter 1's Bayesian updating, extended here

Table 15.1: A taxonomy of financial crime and detection approaches

Every row in Table 15.1 shares the same underlying statistical structure: a rare, adversarial event must be distinguished from an overwhelming volume of legitimate activity, and the adversary actively adapts once a detection method becomes known, a dynamic this book has not emphasized as strongly elsewhere. A credit-scoring model's population of borrowers, Chapter 9's subject, is not trying to defeat the model; a money launderer designing a transaction pattern specifically to stay under a reporting threshold is. This adversarial quality is why financial crime detection combines the statistical tools of Chapters 6 and 9 with rule-based systems, network analysis, and a regulatory reporting apparatus that has no real counterpart in the credit-scoring literature, all developed in this chapter.

That shared structure conceals a second difference between the rows, one that decides more about which methods are available than the crime type itself does: what *labels* each problem produces as a by-product of its own operation. Card fraud sits at one extreme. A cardholder disputes a charge within days, the dispute is ground truth rather than an opinion, and millions arrive automatically, so a supervised model can be retrained continuously against genuine outcomes. Money laundering sits at the other. The only label a bank ordinarily obtains is that one of its own analysts filed a Suspicious Activity Report, which records a suspicion rather than a confirmed crime, and law enforcement almost never reports back what came of it, so the loop never closes. A model fitted to those filings learns to predict its own institution's alerting behavior rather than criminality, and the true prevalence of laundering, the base rate on which every calculation in the next section depends, is not merely unmeasured but unmeasurable.

This asymmetry, rather than any judgment about which techniques are fashionable, is why the methods in this chapter are distributed as they are. Supervised learning dominates where outcomes are observed; unsupervised detection, rules, and network structure carry the weight where they are not; and machine learning enters the anti-money-laundering side largely through alert triage, ranking work the rules have already produced, because analyst dispositions are the one label that problem does generate in quantity. It is worth asking of each method below what it is being trained on, and whether that thing is an outcome or an opinion.

Wulin.Suo@Queensu.ca

15.2.2 Anomaly Detection in Transactions

Suppose a bank wants to stop a stolen credit card from being drained before the legitimate cardholder even notices it is missing, deciding, transaction by transaction and in real time, whether each new charge looks like the actual cardholder or a thief. The simplest fraud detection systems built for this decision are rule-based: a transaction is flagged if it exceeds a fixed dollar threshold, occurs in a country the cardholder has never transacted in, or arrives within seconds of another transaction from the same account in a physically distant location, a velocity check.

Rule-based systems are transparent and easy to audit, which matters enormously for the regulatory reasons discussed later in this chapter, but they are also brittle: a threshold set to catch genuine fraud will also catch a legitimate customer's unusual but innocent purchase. A threshold known publicly (a $10,000 reporting trigger, discussed in Section 15.3.3, is a matter of public record) can simply be engineered around.

Statistical anomaly detection improves on a fixed rule by asking whether a transaction is unusual relative to a customer's own historical behavior rather than relative to a fixed, population-wide threshold, computing something like a z-score for transaction amount, location, or timing against that customer's own rolling mean and standard deviation, or applying the unsupervised clustering methods Chapter 6 introduced to flag transactions that do not resemble any of a customer's usual behavioral clusters. Both approaches, rule-based and statistical, ultimately produce the same kind of output as Chapter 6's classifiers: a binary flag, or a continuous risk score thresholded into one, and are therefore subject to the same trade-off Chapter 1's Bayes' theorem example quantified.

A Worked Example: The Bayes' Theorem Result at Full Transaction Volume

Chapter 1's example assumed a bank where 1% of transactions are fraudulent, a model correctly flags 95% of actual fraud, and incorrectly flags 3% of legitimate transactions, and found that a flagged transaction is fraudulent only about 24.2% of the time. That posterior probability is easy to underestimate in its practical significance until it is scaled up to the volume a real institution actually processes.

Suppose the bank handles 500,000 transactions in a day. At a 1% fraud rate, this implies 5,000 fraudulent and 495,000 legitimate transactions, and applying the model's 95% true-positive rate and 3% false-positive rate gives

$$TP = 5{,}000 \times 0.95 = 4{,}750, \qquad FN = 5{,}000 \times 0.05 = 250,$$

$$FP = 495{,}000 \times 0.03 = 14{,}850, \qquad TN = 495{,}000 \times 0.97 = 480{,}150.$$

The model flags $4{,}750 + 14{,}850 = 19{,}600$ transactions in a single day, of which only 4,750 are genuine fraud, giving

$$\text{Precision} = \frac{4{,}750}{19{,}600} \approx 0.2423, \qquad \text{Recall} = \frac{4{,}750}{5{,}000} = 0.95,$$

exactly matching, up to rounding, Chapter 1's 24.2% posterior probability, since precision and Pr(Fraud | Flag) are the same quantity computed two different ways. The F_1 score, $2 \times 0.2423 \times 0.95/(0.2423 + 0.95) \approx 0.386$, is a useful single-number summary for comparing models, but it is the raw count, 19,600 flagged transactions in one day, that a compliance team must actually staff and review, and Section 15.4.1 returns to this operational cost.

This scaled-up version of Chapter 1's example makes concrete why fraud detection systems almost never rely on a single model or a single threshold in isolation. A bank that simply lowered its false-positive rate to improve precision would, by the same arithmetic, also lower recall and let more genuine fraud through, unless it simultaneously improved the model's underlying discriminative power. That is precisely the motivation for combining several distinct signals, transaction-level scoring, behavioral risk tiering, and time-based survival modeling, developed in the rest of this chapter, rather than relying on any one of them alone.

A Second Application: Insider Trading Surveillance

The same Bayesian structure applies well beyond transaction fraud. Market surveillance teams monitor for insider trading, trading ahead of material non-public information, largely by flagging statistically abnormal trading volume or price movement in the days immediately preceding a public announcement (an earnings

Wulin.Suo@Queensu.ca

release, a merger, a regulatory decision), the same kind of anomaly-detection logic Chapter 14's order-flow-imbalance model applied to microstructure data, now applied to a much lower-frequency, event-driven setting.

Suppose that, across all announcements a surveillance team reviews in a year, genuine insider trading actually precedes about 2% of them, a volume-anomaly detector correctly flags 75% of these genuine cases, and it also flags 8% of announcements with no insider trading at all, a somewhat higher false-positive rate than Section 15.2.2's transaction-fraud example, reflecting how much noisier pre-announcement trading volume is compared to a single card transaction. Applying Bayes' theorem just as in Chapter 1,

$$\Pr(\text{Flag}) = 0.75(0.02) + 0.08(0.98) = 0.0934, \qquad \Pr(\text{Insider} \mid \text{Flag}) = \frac{0.75(0.02)}{0.0934} \approx 0.1606,$$

so a flagged announcement reflects genuine insider trading only about 16.1% of the time, even lower than Section 15.2.2's 24.2% figure. The interesting comparison is *why* it is lower: this example's true-positive rate (75%) is actually worse than the transaction-fraud example's (95%), which on its own would push precision down only modestly, but its false-positive rate (8%) is more than double the transaction-fraud example's (3%), and it is this difference that dominates the result.

At a low base rate, precision is far more sensitive to the false-positive rate than to the true-positive rate, since the false-positive rate is multiplied by the (much larger) population of non-events; a surveillance system's design effort is therefore almost always better spent tightening false-positive rates than chasing marginal improvements in true-positive rates, a general lesson that applies equally to the transaction-fraud and AML alert systems developed elsewhere in this chapter.

15.2.3 A Worked Example: Fraud Scoring with Logistic Regression

Section 15.2.2's example treated the detection model as a black box with a fixed true- and false-positive rate; this section builds one explicitly, using the logistic regression Chapter 6 introduced and Chapter 9 applied to credit scoring. Table 15.2 records ten transactions, each with three features: the transaction amount expressed as a z-score relative to the cardholder's typical spending, the distance from the cardholder's home address in hundreds of kilometers, and a late-night indicator (1 if the transaction occurred between midnight and 5 a.m. local time), together with whether the transaction was actually fraudulent.

Transaction	Amount z-score	Distance (100s km)	Late night	Actual fraud	Predicted
1	3.1	8.0	1	1	1
2	0.2	0.1	0	0	0
3	2.8	6.5	1	1	1
4	1.9	6.0	0	0	0
5	0.4	0.2	0	0	0
6	3.4	9.2	1	1	1
7	0.1	0.4	0	0	0
8	0.6	0.1	1	0	0
9	1.2	0.3	1	1	0
10	0.3	0.2	0	0	0

Table 15.2: Ten transactions used to fit a fraud-scoring logistic regression

Fitting a logistic regression

$$\Pr(\text{Fraud}) = \sigma(\beta_0 + \beta_1 z + \beta_2 d + \beta_3 n)$$

using the sigmoid function introduced in Chapter 6, to these ten observations gives coefficients $\hat{\beta}_0 \approx -2.57$, $\hat{\beta}_1 \approx 0.83$ (amount z-score), $\hat{\beta}_2 \approx 0.15$ (distance), and $\hat{\beta}_3 \approx 0.82$ (late night), all with the intuitively correct sign: an unusually large purchase, made far from home, late at night, all independently raise the predicted fraud probability.

Wulin.Suo@Queensu.ca

These coefficients carry one caveat that is worth seeing early. The ten transactions are linearly separable: a straight boundary in the three features classifies all ten correctly, as a linear support vector machine confirms. On separable data, plain maximum likelihood has no finite solution, because scaling up any separating coefficient vector always raises the likelihood a little further. The fit above is therefore not unpenalized. It uses scikit-learn's default L2 (ridge) penalty with $C = 1$, which shrinks the coefficients toward zero exactly as Chapter 6's ridge regression shrinks its slopes, and acts like a prior that keeps a ten-point model from claiming certainty it has not earned. Weakening the penalty shows what it is holding back: at $C = 10^6$ the coefficients run to $(27.08, -4.15, 12.70)$ with an intercept of -36.10, every transaction is classified correctly, and the distance coefficient even turns negative. The penalized fit is the more sensible model, but transaction 9's miss below is partly a product of the penalty, not of the data alone.

At a 0.5 classification threshold, the model correctly flags transactions 1, 3, and 6 (true positives) and correctly clears transactions 2, 4, 5, 7, 8, and 10 (true negatives), but assigns transaction 9 a predicted probability of only about 33%, below the threshold, missing a genuine fraud case (a false negative) despite the late-night flag being present, because the transaction's amount and distance from home were both unremarkable. There are no false positives in this small sample, giving

$$\text{Precision} = \frac{3}{3+0} = 1.00, \qquad \text{Recall} = \frac{3}{3+1} = 0.75,$$

$$F_1 = \frac{2(1.00)(0.75)}{1.00 + 0.75} \approx 0.857,$$

a recall figure that, by design, exactly mirrors Chapter 6's own loan-default confusion matrix example, three true positives, one false negative, out of four actual positive cases.

Transaction 9 is instructive precisely because it illustrates a structural limitation of any single-transaction scoring model: a fraudulent transaction that looks unremarkable in isolation, an ordinary amount at a nearby location, will not be caught by a model that only ever looks at one transaction at a time, no matter how well it is fit, which is why Section 15.3.1 layers a behavioral, account-level risk score on top of transaction-level scoring, and why Section 15.3.2 tracks how account risk accumulates over time rather than resetting with every new transaction.

The Threshold Choice: A Precision-Recall Trade-off

The 0.5 threshold used above is a modeling choice, not a mathematical requirement, and moving it trades precision against recall in the way Chapter 6 described in the abstract. Table 15.3 recomputes the same ten-transaction confusion matrix at two alternative thresholds.

Threshold	Precision	Recall	F_1
0.30	0.800	1.000	0.889
0.50	1.000	0.750	0.857
0.85	1.000	0.500	0.667

Table 15.3: Precision-recall trade-off at three classification thresholds

Lowering the threshold to 0.30 now also flags transaction 4 (predicted probability 0.470, actually legitimate), turning it into a false positive, but it catches transaction 9 as well, raising recall to a perfect 100% at the cost of precision falling from 1.00 to 0.80. Raising the threshold to 0.85 moves in the opposite direction: transaction 3 (predicted probability 0.822) now also falls below the bar and becomes a second false negative, so recall drops to 50% while precision stays perfect, since the model is now flagging only the two most unambiguous cases. Neither threshold is objectively correct; the right choice depends on the same alert-economics trade-off Section 15.4.1 quantifies at full scale, a lower threshold means more alerts and more investigator hours per genuine fraud caught, while a higher threshold means fewer alerts but more fraud slipping through entirely, and a bank sets its operating threshold by weighing the dollar cost of a missed fraud against the staffing cost of an additional false-positive investigation, not by any property of the model itself.

That choice becomes determinate as soon as a cost is attached to each kind of error. Suppose reviewing a flagged transaction costs \$50 of investigator time and a missed fraud costs \$200 on average, so that operating

Wulin.Suo@Queensu.ca

at threshold τ costs

$$C(\tau) = \$50 \times (\text{transactions flagged}) + \$200 \times (\text{frauds missed}).$$

On the ten fitted probabilities this gives \$350 at the default $\tau = 0.5$ (three reviews and one missed fraud) and \$500 at $\tau = 0.85$, against a minimum of \$250 at $\tau = 0.25$, where five transactions are flagged and none is missed. The default threshold is 29% more expensive than the best one available, and the optimum is no accident of this sample: flagging pays whenever the expected loss avoided exceeds the review cost, $p \times \$200 > \$50$, so the cost-minimizing threshold is just the ratio $\$50/\$200 = 0.25$. A threshold of 0.5 is correct only when a missed fraud costs exactly twice a review, which no one has claimed here. Section 15.4.1 runs the same calculation at the scale of a real alert queue.

15.2.4 Advanced Models and Techniques for Class Imbalance

Every example so far has used ordinary logistic regression at a default decision threshold to make the underlying statistical issue, extreme class imbalance, as transparent as possible. Production fraud and AML systems typically go further, using both more expressive models and dedicated techniques for the imbalance problem itself, and it is worth being precise about what each one actually does, since they solve different parts of the problem and are frequently combined rather than used as substitutes for one another.

Cost-Sensitive Learning: Class Weighting

The most direct fix reweights the training loss itself so that a missed fraud case (a false negative) counts for more than a missed legitimate transaction (a false positive) during model fitting, rather than treating every observation equally. Refitting Section 15.2.3's logistic regression on the same ten transactions in Table 15.2, but with each class weighted inversely to its frequency,

$$\text{Weight (legitimate)} = 10/(2 \times 6) \approx 0.83, \qquad \text{Weight (fraud)} = 10/(2 \times 4) = 1.25,$$

shifts the fitted coefficients to

$$\hat{\beta}_0 \approx -2.25, \qquad \hat{\beta}_1 \approx 0.86, \qquad \hat{\beta}_2 \approx 0.13, \qquad \hat{\beta}_3 \approx 0.85,$$

and raises transaction 9's predicted probability from 33% to about 41.7%, still short of the default 0.5 threshold. At the unchanged 0.5 threshold, class weighting alone actually makes this small sample's confusion matrix worse, transaction 4 now also crosses into a false positive, giving precision 0.75, recall unchanged at 0.75, and $F_1 \approx 0.75$, below the original model's 0.857. This is an important, honest limitation to see clearly: class weighting reshapes the decision boundary, but it does not guarantee that every previously missed case crosses whatever threshold is already in place, and on a sample this small its effect is noisy.

Combined with the threshold adjustment from Section 15.2.3's Table 15.3, lowering the threshold to 0.40 alongside the reweighted coefficients does catch transaction 9 (now at 41.7%) while still flagging transaction 4, giving precision 0.80, recall 1.00, and $F_1 \approx 0.889$, the same operating point Table 15.3 reached by lowering the unweighted model's threshold to 0.30. The practical lesson is that class weighting and threshold selection are not independent choices, for a linear model like logistic regression they move the decision boundary in closely related ways, and a compliance team tuning one should generally revisit the other rather than treating either as a complete solution on its own.

Resampling: Undersampling and SMOTE

A second family of techniques changes the training data rather than the loss function. Random undersampling simply discards a random subset of majority-class (legitimate) observations until the training set is more balanced, cheap and simple, but wasteful, since it throws away real information about legitimate behavior that later helps the model avoid false positives. The Synthetic Minority Oversampling Technique (SMOTE) instead grows the minority class by generating synthetic fraud examples: for each real fraud case, it finds its nearest neighbors among other fraud cases (exactly Chapter 6's k-nearest-neighbors distance calculation, applied here to construct new training points rather than to classify a new one) and creates new synthetic

Wulin.Suo@Queensu.ca

points along the line segments connecting them. This gives a model more minority-class examples to learn from without discarding legitimate-transaction data, but it carries its own risk. A synthetic point interpolated between two real fraud cases is not a real transaction. If the two real cases it was interpolated between do not actually represent a coherent fraud pattern, SMOTE can teach a model to recognize a combination of features that does not correspond to any real fraud behavior at all, a subtler version of the overfitting risk Chapter 6 introduced for an overly flexible model fit to too little data.

Applying SMOTE directly to Table 15.2's four real fraud cases (transactions 1, 3, 6, and 9), asking for two additional synthetic points using $k = 3$ neighbors, produces this interpolation mechanically:

$$\begin{aligned}
&\text{Synthetic point 1:} \quad (1.836,\ 2.763,\ 1), \quad 39.7\% \text{ from transaction 9's } (1.2, 0.3, 1) \\
&\qquad\qquad\qquad\qquad\quad \text{toward transaction 3's } (2.8, 6.5, 1), \\
&\text{Synthetic point 2:} \quad (3.263,\ 8.654,\ 1), \quad 54.5\% \text{ from transaction 1's } (3.1, 8.0, 1) \\
&\qquad\qquad\qquad\qquad\quad \text{toward transaction 6's } (3.4, 9.2, 1),
\end{aligned}$$

the linear interpolation the algorithm is defined to perform, verified here by solving for each point's interpolation fraction λ along the amount-z-score axis and confirming it reproduces the distance coordinate exactly. Neither synthetic point corresponds to an actual observed transaction, which is precisely the risk described above: both happen to fall between genuinely similar fraud cases here, but SMOTE has no way to know whether that will hold for every pair it selects on a larger, messier dataset.

More Expressive Models: Ensembles and Unsupervised Anomaly Detection

Logistic regression's score is additive in its features, amount, distance, and lateness each contribute independently, which is part of why Section 15.2.3's coefficients are easy to interpret, but real fraud often depends on *combinations* of features interacting in ways an additive model can only approximate. Chapter 6's tree-based ensembles, Random Forest and gradient boosting, split the feature space recursively and can represent a rule like "flag this transaction only if the amount is unusual *and* it occurs at an unusual location *and* at an unusual time, but not if only one of these is true," a genuinely nonlinear interaction a single logistic regression coefficient vector cannot express directly. This added flexibility does not, on its own, solve class imbalance, a gradient-boosted model trained on the same skewed data will show the same base-rate problem Section 15.2.2 quantified unless it is also combined with class weighting or resampling, and it comes at the cost of interpretability, which Section 15.4.2 returns to as a regulatory constraint on model choice.

The companion notebook tests this claim directly, since the ten-transaction sample is too small to show a genuine ensemble advantage. It simulates 5,000 transactions with the kind of three-way interaction described above built into the true fraud-generating process, amount, distance, and lateness each contribute a modest amount on their own, but fraud probability jumps sharply only when all three are simultaneously unusual, and evaluates a logistic regression and a gradient-boosted ensemble, both given matched class weighting and neither given any information about the true interaction structure, on a genuinely held-out test set using precision-recall AUC, the metric argued for below.

Logistic regression, unable to represent the three-way interaction no matter how its single coefficient vector is fit, reaches a PR-AUC of 0.2847; gradient boosting, which can split on combinations of features, reaches 0.3795, a 33.3% relative improvement, with feature importances of 0.408 (amount), 0.490 (distance), and 0.102 (lateness) confirming it is using all three features jointly rather than leaning on any single one. This is a genuine, honest improvement, not the trivial in-sample perfection Chapter 9's Gini-coefficient/AUC validation example (using its own six-applicant credit dataset) warned against, precisely because it is measured on data the ensemble never saw during training.

A fundamentally different approach abandons supervised labels altogether. Isolation Forest and autoencoder-based anomaly detection instead learn what *normal* transaction or account behavior looks like from the (overwhelmingly legitimate) data itself, without ever being told which historical cases were confirmed fraud, and flag new observations that are statistically isolated or poorly reconstructed relative to that learned notion of normal. This matters for a reason specific to this chapter: the Kaplan-Meier analysis of Section 15.3.2 will show that confirmed-fraud labels can take weeks to arrive even after an account is flagged, and the adversarial adaptation described in this chapter's challenges means a labeled training set is always describing yesterday's fraud patterns. An unsupervised anomaly detector does not need a confirmed label to flag a

Wulin.Suo@Queensu.ca

new, never-before-seen pattern as unusual, at the cost of a real trade-off: an unsupervised flag only says "this looks statistically unusual," not "this is fraud," so it typically produces an even lower precision than the supervised models in Section 15.2.3, feeding directly into the same alert queue whose cost Section 15.4.1 quantifies, and is best used as one additional signal alongside, not a replacement for, the supervised and behavioral scoring developed earlier in this chapter.

Fitting an Isolation Forest to the same simulated training transactions, with the fraud labels never shown to it at all, and scoring the held-out test transactions confirms this is more than a theoretical possibility: the 34 held-out fraud cases receive a mean anomaly score of 0.5525 versus 0.4658 for legitimate transactions, and the unsupervised score alone achieves a ROC-AUC of 0.7428 against the labels it never saw during fitting, genuine (if modest, exactly as expected) discriminatory power obtained without ever training on a single confirmed fraud label.

Evaluating Models Under Imbalance: Precision-Recall AUC, Not ROC-AUC

One evaluation pitfall deserves its own mention. The receiver operating characteristic (ROC) curve plots the true-positive rate against the false-positive rate across every possible threshold, and its area under the curve (ROC-AUC) is a common single-number summary of a classifier's overall ranking ability. Under the severe class imbalance this entire chapter has emphasized, ROC-AUC can look deceptively good: because the false-positive rate is computed as a share of an enormous legitimate population, even a model that performs only modestly well at actually ranking fraud can still show a low false-positive rate and a high ROC-AUC, precisely the same base-rate distortion Section 15.2.2's Bayes' theorem example exposed for a fixed confusion matrix. The precision-recall (PR) curve, plotting precision against recall across thresholds, and its area (PR-AUC), does not have this blind spot, since precision is computed directly against the number of *predicted* positives rather than against the (huge) total negative population, making it the more informative single-number summary whenever the event being detected is rare, the situation this chapter's fraud, insider-trading, and AML examples all share.

15.2.5 Two Fraud Types That Defeat Transaction-Level Scoring

Everything in this part has assumed a fraudulent transaction is, in principle, distinguishable from a legitimate one given enough features and a good enough model. Two of the fastest-growing categories of fraud violate that assumption outright, each in a different way, and both are worth understanding precisely because they show where the statistical machinery developed above stops being sufficient.

Synthetic Identity Fraud: The Case With No Labels

A synthetic identity is not a stolen one. The fraudster assembles a person who does not exist, typically pairing a real but unused identity number, often belonging to a child, a recent immigrant, or a deceased person whose records are not actively monitored, with a fabricated name, date of birth, and address. Because no real person owns the composite, no real person ever notices it, disputes a charge, or files a report.

That single fact changes the detection problem. The account is then *cultivated*: it takes a small credit line, pays every bill on time, and is rewarded with progressively higher limits, exactly as a genuinely creditworthy new customer would be. Suppose four tradelines are grown from a \$500 to a \$10,000 limit over eighteen months, with about \$120 a month paid across all of them,

$$\text{Paid in} = 18 \times \$120 = \$2{,}160, \qquad \text{Exposure at bust-out} = 4 \times \$10{,}000 = \$40{,}000,$$

so the "bust-out," drawing every line to its limit and disappearing, nets roughly \$37,840 on an outlay of \$2,160, a return of about eighteen times. Note what the behavioral risk score of Section 15.3.1 would have said throughout: transaction velocity normal, geography stable, activity consistent with the profile declared at onboarding. Every input this chapter has built points the wrong way, because during cultivation the account genuinely is a model customer. The score spikes only at the bust-out, when the money is already gone.

The deeper problem is the labels. A bust-out is usually written off as ordinary credit loss, since there is no victim to reclassify it and no obvious moment of theft. If 5% of a lender's 100,000 annual charge-offs are

Wulin.Suo@Queensu.ca

in fact synthetic identities, then 5,000 fraud cases a year are sitting in the training data labeled "default," and a supervised fraud model never learns them at all. This is the same structural blindness Chapter 9's reject inference describes, in a more damaging form: the measured fraud base rate is understated, so every precision figure computed against those labels is optimistic by an unknown margin.

It is also the strongest argument in this chapter for the unsupervised methods of Section 15.2.4. An Isolation Forest or autoencoder never needed a confirmed fraud label in the first place, and the cultivation pattern, several thin-file accounts sharing an address, a device fingerprint, or a phone number, and all maturing on similar trajectories, is precisely the kind of structure an anomaly detector or the network analysis of Section 15.3.3 can surface without one.

Authorized Push Payment Fraud: The Case Where Every Feature Is Legitimate

In an authorized push payment scam the customer is deceived, by a fake invoice, an impersonated bank official, or a romance or investment pretext, into sending the money themselves. The credentials are correct, the device is the customer's usual one, the authentication succeeds, and the instruction is genuine. This is transaction 9 of Table 15.2 carried to its limit: not merely a fraudulent payment that looks unremarkable, but one that is, at the moment of authorization, indistinguishable from a legitimate payment on every feature a transaction-level model observes, because it *is* a legitimate instruction from the account holder.

For a long time this was treated as a customer-education problem rather than a modeling one, on the reasoning that a bank that correctly executed an authorized instruction had done nothing wrong. Mandatory reimbursement regimes, most prominently the United Kingdom's, which since 2024 has required the sending and receiving payment firms to split reimbursement of qualifying scam losses between them, moved the loss onto the bank's own books and converted it into an ordinary expected-cost calculation.

The economics then become favorable very quickly. Suppose a bank processes 500,000 faster payments a month, 0.02% of which are scams, at an average loss of \$4,200, and that it bears half of the reimbursement,

$$\text{Scams} = 0.0002 \times 500{,}000 = 100, \qquad \text{Bank's exposure} = 100 \times \$4{,}200 \times 0.5 = \$210{,}000 \text{ per month.}$$

A model operating at 60% recall and only 5% precision, poor by any of this chapter's earlier standards, catches 60 scams while raising $60/0.05 = 1{,}200$ alerts. At 15 minutes per intervention, a call to the customer before the payment is released, and a fully loaded analyst cost of \$45 an hour (the industry rule of thumb that Section 15.4.1 also uses to price an alert queue),

$$\text{Cost} = \frac{1{,}200 \times 15}{60} \times \$45 = \$13{,}500, \qquad \text{Loss prevented} = 60 \times \$4{,}200 \times 0.5 = \$126{,}000,$$

a net benefit of \$112,500 a month, or \$1.35 million a year. Solving for the recall at which the intervention cost is exactly recovered gives 6.43%: the model pays for itself once it stops between six and seven of the hundred monthly scams. That is a far more forgiving operating point than anything in Section 15.4.1, and it is the direct consequence of a single missed case costing the bank \$2,100 rather than nothing.

Because the payment instruction itself carries no signal, detection has to look elsewhere: at deviation from the customer's own payment history rather than from any population norm (a first-time payee, an unusual amount, an account that has never before sent a faster payment at all), at contextual signals such as a payment initiated during or immediately after a long inbound call, and above all at the *receiving* side, where mule accounts show the short-dwell, high-pass-through pattern that Section 15.3.3's layering example already taught us to recognize. The victim's transaction looks normal; the beneficiary's does not.

15.3 Risk Tiering, Survival, and Money Laundering

Beyond a single score, this part ranks customers by behavior, models time-to-detection, and turns to laundering typologies.

15.3.1 Behavioral Risk Frameworks and Customer Risk Tiering

Chapter 9's credit-scoring model mapped a continuous predicted default probability into discrete letter-grade ratings for portfolio and pricing purposes; financial institutions apply an analogous mapping in the fraud

Wulin.Suo@Queensu.ca

and AML context, called customer risk tiering, but built from behavioral features accumulated over an entire account relationship rather than a single transaction.

Typical inputs include the account's transaction velocity (how the frequency and size of transactions this month compares to its own trailing average, directly analogous to Section 15.2.3's amount z-score, but computed account-wide rather than transaction-by-transaction), geographic diversity of counterparties, the presence of newly added payees or beneficiaries, and, in the anti-money-laundering context developed in Section 15.3.3, whether the customer's declared occupation and expected transaction volume, gathered during onboarding under Know-Your-Customer (KYC) requirements, are consistent with actual observed activity.

Table 15.4 shows an illustrative risk-tiering scheme built from a composite behavioral score on a 0-100 scale, combining several such features with weights an institution calibrates from its own historical fraud and AML case data, in the same spirit as Chapter 9's letter-grade rating buckets.

Composite score	Risk tier	Typical monitoring response
0–29	Low	Standard, periodic account review
30–59	Medium	Increased transaction monitoring sensitivity
60–84	High	Manual review of new activity; tighter velocity limits
85–100	Enhanced due diligence	Senior compliance review; may restrict or close account

Table 15.4: An illustrative customer risk-tiering scheme

A concrete composite score makes Table 15.4 usable rather than purely illustrative. Suppose an institution combines three behavioral sub-scores, each already normalized to a 0-100 scale, using weights calibrated from its own historical case data: a transaction-velocity sub-score (how far this month's activity deviates from the account's own trailing average), a geographic-diversity sub-score (how unusual the mix of counterparty locations is relative to the account's history), and a KYC-consistency sub-score (how far observed activity deviates from what the customer declared at onboarding), weighted 40%, 35%, and 25% respectively based on the institution's own back-tested case outcomes.

For an account with a velocity sub-score of 70 (transaction volume well above its own recent average), a geographic-diversity sub-score of 50 (a moderate increase in counterparty locations), and a KYC-consistency sub-score of 90 (activity far exceeding what the customer declared at onboarding), the composite score is

$$0.40(70) + 0.35(50) + 0.25(90) = 28.0 + 17.5 + 22.5 = 68.0,$$

placing the account in the High tier of Table 15.4, triggering manual review of new activity and tighter velocity limits, even though no single sub-score alone (velocity at 70, geographic diversity at only 50) would necessarily have triggered that response in isolation.

This is the point of a composite, weighted score: the KYC-consistency mismatch, the largest single contributor here at 22.5 of the 68.0 total, combines with two more moderate signals to cross a review threshold that no individual behavioral signal alone was large enough to reach, precisely the kind of accumulating, multi-signal evidence the sequential Bayesian updating example in Chapter 1 introduced in the context of combining several independent fraud signals.

The weights are written here as though a committee chose them, and in many institutions one did. But "calibrated from the institution's own historical case data" is a description of supervised learning rather than an alternative to it: given a labeled history of which accounts eventually warranted a report, the weights are fitted coefficients from exactly the logistic regression of Section 15.2.3, and the four tiers are a discretization of its fitted score, in the same way Chapter 9's letter grades discretize a predicted default probability.

This tiering scheme changes what a fraud or AML system optimizes for compared to Section 15.2.3's per-transaction model: rather than asking whether *this* transaction looks fraudulent, it asks whether *this account*, taken as a whole, has become risky enough to warrant tighter monitoring of everything it does going forward, which is precisely the right framework for catching a transaction like Table 15.2's transaction 9, unremarkable on its own but potentially the first sign of a compromised account whose behavioral score has already begun drifting upward.

Regulators, discussed further in Section 15.3.3, generally require this kind of ongoing, risk-based customer due diligence rather than a one-time onboarding check precisely because a customer's true risk, like a firm's

Wulin.Suo@Queensu.ca

credit risk in Chapter 9's rating-transition framework, evolves over time rather than being fixed at account opening.

15.3.2 Survival Models: Time to Detection and Account Risk Over Time

Every classification model in this book, including Section 15.2.3's logistic regression, answers a yes-or-no question at a single point in time. A genuinely new question, and one financial institutions care about operationally, is how long a flagged or compromised account survives before fraud is actually confirmed and the account is closed, information that helps a compliance team prioritize which alerts to investigate first and helps a fraud team estimate how much exposure a typical compromised account accumulates before detection.

Survival analysis, developed originally in biostatistics to study time until death or disease recurrence, answers this kind of question, and this chapter introduces it because no earlier chapter has needed a method for data in which the outcome of interest, here fraud confirmation, has not yet happened for every observation by the time the data are analyzed.

The key complication survival analysis handles, and that ordinary regression cannot, is censoring: some accounts in a monitoring sample are closed for unrelated reasons (voluntary closure, non-fraud-related risk exit) or are simply still open and unresolved at the time the data are pulled, meaning the true time-to-fraud-confirmation for those accounts is only known to be *at least* as long as the observed time, not exactly equal to it.

Simply dropping censored observations, or treating them as if fraud were confirmed at the moment of censoring, both bias the resulting estimate; the Kaplan-Meier estimator instead uses every observation, censored or not, up until the point it is censored. The estimator computes the survival function, the probability an account remains unconfirmed (open, or closed for reasons unrelated to fraud) beyond time t, as a product of conditional survival probabilities at each observed event time,

$$\hat{S}(t) = \prod_{t_i \le t} \left(1 - \frac{d_i}{n_i}\right),$$

where $\hat{S}(t)$ is the estimated probability that an account survives, meaning it remains unconfirmed as fraud, beyond time t, n_i is the number of accounts still under observation (not yet closed or censored) just before time t_i, and d_i is the number of confirmed-fraud closures exactly at time t_i. This book's companion technical note on the Kaplan-Meier estimator shows that this product-limit formula is not an ad hoc device for coping with censored data but the exact nonparametric maximum-likelihood estimator of the survival function once right censoring is written into the likelihood correctly, which is also why the curve steps down only at observed event times and never at censoring times.

A Worked Example: Kaplan-Meier Survival of Flagged Accounts

Suppose a bank's fraud team tracks 12 accounts flagged by Section 15.3.1's risk-tiering system, recording, for each, the number of days until either a confirmed fraud closure (an event) or an unrelated account closure or the end of the observation window (a censored observation, marked with a +): 5, 8, 8^+, 12, 15, 15^+, 20, 24^+, 30, 30^+, 35, 40^+. Table 15.5 works through the Kaplan-Meier calculation at each day on which a confirmed fraud closure actually occurred.

Two features of Table 15.5 are worth pausing on. First, the number at risk, n_i, drops not only when a fraud closure occurs but also whenever a censored observation passes that day, which is why n_i falls from 12 to 9 between day 5 and day 12 even though only two confirmed closures occurred in between: the account censored on day 8 also leaves the risk set, accounting for the third of those three departures without contributing a d_i term.

This is precisely the mechanism by which censored observations still contribute information, they remain in the risk set for as long as they are actually observed, without being incorrectly counted as either a confirmed event or an assumed survivor beyond their last observed day.

Second, the estimated probability that a flagged account survives (remains unconfirmed) beyond 35 days is only about 20.3%. Roughly 80% of accounts in this sample that are eventually confirmed as fraud are

Wulin.Suo@Queensu.ca

Day t_i	At risk n_i	Fraud closures d_i	Factor $(1 - d_i/n_i)$	$\hat{S}(t_i)$
5	12	1	0.9167	0.9167
8	11	1	0.9091	0.8333
12	9	1	0.8889	0.7407
15	8	1	0.8750	0.6481
20	6	1	0.8333	0.5401
30	4	1	0.7500	0.4051
35	2	1	0.5000	0.2025

Table 15.5: Kaplan-Meier survival calculation for 12 flagged accounts

therefore confirmed within a little over a month of being flagged. That is a concrete, evidence-based number a compliance team can use to set service-level targets for how quickly a flagged account's investigation should be completed. It also lets the team recognize that an account still open and unresolved well beyond that horizon is behaving atypically relative to the rest of the flagged population, an idea developed further next.

How Precise Is $\hat{S}(t)$? Greenwood's Formula

A survival estimate like $\hat{S}(35) \approx 0.2025$ is, exactly like every other point estimate in this book, only half the story without some sense of how precisely it is known from only 12 accounts.

Greenwood's formula gives the standard error of the Kaplan-Meier estimator at any time t,

$$\widehat{\mathrm{Var}}[\hat{S}(t)] = \hat{S}(t)^2 \sum_{t_i \le t} \frac{d_i}{n_i(n_i - d_i)},$$

built from the same at-risk counts n_i and event counts d_i already tabulated in Table 15.5, so no new data are required to compute it. Applying this at $t = 20$, where $\hat{S}(20) \approx 0.5401$, the summed term $\sum d_i/(n_i(n_i - d_i))$ across the five event times up to and including day 20 is approximately 0.0818, giving $\widehat{\mathrm{Var}}[\hat{S}(20)] \approx 0.5401^2(0.0818) \approx 0.02385$ and a standard error of SE ≈ 0.154, so a normal-approximation 95% confidence interval is $0.540 \pm 1.96(0.154) \approx (0.237, 0.843)$, already wide enough to span nearly the entire plausible range of survival probabilities.

At $t = 35$, the final event time, $\hat{S}(35) \approx 0.2025$ with SE ≈ 0.165, giving a 95% interval of approximately (0.000, 0.526) once the lower bound is truncated at zero (a known limitation of the normal approximation applied to a probability, since the true interval cannot extend below zero or above one): the point estimate of roughly 20% is consistent with a true 35-day survival probability anywhere from essentially zero to over 50%.

This is precisely the small-sample caution raised throughout this book, at the AR(1) confidence interval in Chapter 7 and the collateral-coverage regression in Chapter 9, applied here to survival analysis: with only 12 flagged accounts and just 7 confirmed fraud closures, the Kaplan-Meier curve's later points, computed from a shrinking risk set as accounts are censored or resolved, carry genuinely large uncertainty even though the point estimate itself looks perfectly precise to four decimal places.

A compliance team setting the service-level target mentioned above should treat the estimated 80%-confirmed-within-35-days figure as a useful planning number derived from a small pilot sample, not a precise population parameter, and should expect that figure to firm up considerably, its confidence interval narrowing, as more flagged accounts accumulate observed outcomes over time.

This is also where the survival curve earns the prioritization role claimed for it above, since what orders an alert queue is not $\hat{S}(t)$ itself but a *conditional* probability read off it. A freshly flagged account is confirmed within its first fifteen days with probability $1 - \hat{S}(15) = 0.352$, whereas an account still unresolved at day 20 is confirmed by day 35 with probability $1 - \hat{S}(35)/\hat{S}(20) = 1 - 0.2025/0.5401 = 0.625$, nearly twice as likely to convert over the coming fortnight as the fresh flag is over its first. An investigator choosing what to open next should therefore reach for the aging case rather than the newest one, the opposite of the instinct to work a queue newest-first, and it follows from Table 15.5 without collecting any additional data.

Kaplan-Meier is deliberately nonparametric and uses no covariates at all, which is why it can say *when* flagged accounts are confirmed but not *which* ones. The learning extensions supply the missing half. Cox

Wulin.Suo@Queensu.ca

proportional hazards multiplies a baseline hazard by a function of an account's features, and survival forests and gradient-boosted survival models drop the proportionality assumption entirely, so the same censored data behind Table 15.5 yields a per-account predicted time to confirmation, which is what an alert-ranking system actually consumes. Chapter 9's discrete-time hazard formulation is the credit-risk counterpart of the same idea.

15.3.3 Anti-Money Laundering: Structuring, Monitoring, and Regulation

Money laundering, disguising the origin of funds obtained from criminal activity so that they appear to come from a legitimate source, is a distinct problem from transaction fraud, since the funds involved are often not stolen from the institution itself, and the crime being concealed frequently occurred entirely outside the financial system (drug trafficking, corruption, sanctions evasion) before the money ever reached a bank.

In the United States, the Bank Secrecy Act requires banks to file a Currency Transaction Report (CTR) for any cash transaction exceeding $10,000, and a Suspicious Activity Report (SAR) for any transaction pattern, regardless of size, that appears designed to evade reporting requirements or otherwise suggests criminal activity; equivalent frameworks exist in most jurisdictions under the international standards set by the Financial Action Task Force (FATF).

A Worked Example: Detecting Structuring

Structuring (sometimes called "smurfing") is the practice of deliberately breaking a large cash transaction into several smaller ones, each below the CTR reporting threshold, specifically to avoid triggering a report. Suppose a customer makes five cash deposits of $9,500 each over three days, each individually $500 below the $10,000 CTR threshold,

$$\text{Total deposited} = 5 \times \$9{,}500 = \$47{,}500,$$

an amount that, deposited as a single transaction, would unambiguously trigger a CTR, and that, spread across five transactions each safely under the threshold, triggers none individually. This is why AML monitoring systems cannot rely solely on Section 15.2.2's single-transaction threshold rules: a rule of the form "flag any transaction over $10,000" is by construction blind to a pattern engineered specifically to stay under it.

Effective structuring detection instead applies a rule across a rolling window of activity, for example, flagging any customer with three or more cash deposits within a seven-day period each falling within $1,000 of the reporting threshold, which the five $9,500 deposits above would trigger even though no single deposit would. This is a direct illustration of the behavioral, account-level monitoring philosophy introduced in Section 15.3.1: the relevant signal is a pattern across transactions and across time, not any single transaction viewed in isolation.

Placement, Layering, and Integration: A Network Example

Structuring is the classic technique for the first of three stages regulators and investigators use to describe how illicit funds are laundered: *placement*, introducing cash into the financial system in the first place, the deposit pattern in the worked example above. *Layering* follows placement: moving the placed funds through a chain of transactions and accounts, often across several institutions and jurisdictions, specifically to obscure the connection between the funds and their criminal origin.

Integration is the final stage, reintroducing the now-layered funds into the legitimate economy, for example through a real estate purchase or an investment in a legitimate business, at which point the funds are, for most practical purposes, indistinguishable from legitimately earned money.

Layering is where the network-based detection approach flagged in Table 15.1 becomes necessary, because no single account's activity, viewed in isolation, looks obviously suspicious. Suppose $500,000 in placed funds moves through four shell entities, Alpha, Beta, Gamma, and Delta, each receiving the funds and forwarding 98% of what it received onward within 24 hours (retaining 2%, nominally a service or consulting fee), a repeating pattern that Table 15.6 traces hop by hop.

After four hops, $461,184.08, over 92% of the original placed amount, reaches its ultimate destination, with only $38,815.92 lost to the network's own retained "fees" along the way, having passed through four

Wulin.Suo@Queensu.ca

Entity	Received	Forwarded (98%)	Retained (2%)
Alpha	$500,000.00	$490,000.00	$10,000.00
Beta	$490,000.00	$480,200.00	$9,800.00
Gamma	$480,200.00	$470,596.00	$9,604.00
Delta	$470,596.00	$461,184.08	$9,411.92

Table 15.6: A four-hop layering chain: funds received, forwarded, and retained at each shell entity

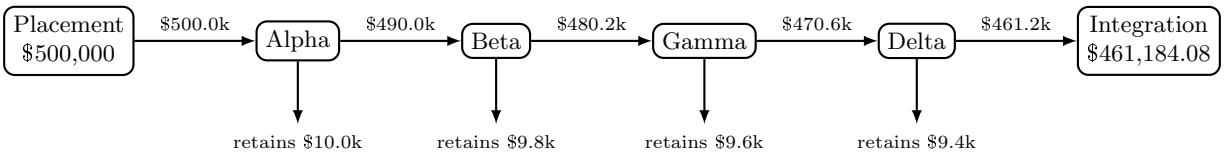


Figure 15.1: The four-hop layering chain of Table 15.6 as a network

legally separate entities in under a week. Figure 15.1 draws this chain as the network it actually is, rather than as four separate, individually unremarkable rows in a table.

No individual hop in Table 15.6 looks obviously wrong; each is a plausible-looking business payment of a plausible size. What is anomalous is the pattern viewed as a network, and each of its three features is a number rather than an impression: a pass-through ratio of exactly 0.98 at every hop, where a real operating business retains and redeploys most of what it receives; a dwell time under 24 hours, against the weeks a genuine payables cycle takes; and, at every interior node, exactly one party paying in and exactly one party paid out, an in-degree and out-degree of one in the vocabulary of graphs, where a real supplier has many customers and many suppliers. Each has innocent explanations on its own. All three at once, repeated four hops in a row, do not.

That conjunction is a reason to look, not a conclusion. A factoring company, an escrow agent, or a freight forwarder shows high pass-through and short dwell as a matter of ordinary business, and a sole-source subcontractor genuinely does have one customer and one supplier; what the shape of the network cannot supply on its own is a reason to believe the money is criminal. Three further tests do that work. The first is *commercial substance*: a real intermediary has employees, premises, and counterparties outside this one chain, whereas Alpha has a bank account, an incorporation date shortly before the flows began, and a registered address it shares with dozens of unrelated entities. The second is *common control*: shared directors or beneficial owners, a shared address, telephone number, or device, or four accounts opened at one branch within days of each other, which turns four legally separate companies into one hand moving its own money and has no innocent reading at all. The third is the fee. Two percent of principal, identical at every hop, is a cut rather than a price, because a genuine service is charged for work performed and does not scale exactly with the sum passing through. Only once such corroboration is in hand does an alert amount to the suspicion of criminal origin that a Suspicious Activity Report requires; the graph by itself reports, exactly as Section 15.2.4 said of any unsupervised flag, that something is unusual rather than that it is a crime.

This is precisely why anti-money-laundering systems increasingly supplement Section 15.3.1's per-account behavioral scoring with network or graph-based analysis, examining the structure of relationships between accounts rather than any single account's activity alone, since a layering scheme is, by design, constructed so that every individual node in the chain looks unremarkable.

Stated as a learning problem, this is node and subgraph classification on the transaction graph. Community detection isolates tightly connected clusters that ordinary commercial relationships do not produce, and graph neural networks, which Chapter 17 develops on this chapter's own layering network, learn a representation of each account from its neighbors' features as well as its own, so that an account can be scored on the company it keeps rather than only on what it does. The supervision is weak, since confirmed laundering is rare and arrives late, which is why in practice these models rank leads for investigators instead of deciding anything.

Wulin.Suo@Queensu.ca

Figure 15.2 places this rule-based structuring and layering detection alongside the statistical and behavioral tools developed earlier in the chapter, showing how a real institution's monitoring system typically combines several distinct signals before a case ever reaches a human investigator.

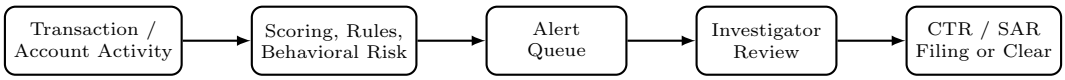


Figure 15.2: The fraud and AML monitoring pipeline

Beyond structuring, AML regulation imposes an ongoing due-diligence obligation that goes beyond fraud detection's largely reactive posture: Customer Due Diligence (CDD) at onboarding establishes a customer's expected activity profile (Section 15.3.1's KYC inputs), Enhanced Due Diligence (EDD) applies heightened scrutiny to higher-risk customers such as politically exposed persons or those in high-risk jurisdictions, and ongoing monitoring compares actual behavior against that expected profile continuously, the account-level, time-varying risk view Section 15.3.2's survival framework formalizes.

Two of these obligations are increasingly discharged with the language models Chapter 16 develops. Adverse-media screening, checking whether a customer appears in news reporting tied to fraud, corruption, or sanctions evasion, is a document-classification problem of exactly the kind that chapter's Naive Bayes and embedding methods address, and it inherits this chapter's precision problem in acute form, since most name matches in news text are simply the wrong person. Drafting the suspicious-activity report itself, a structured account of what was observed and why it appears suspicious, is now commonly assisted by a large language model working from the alert's own records. Both uses are governed rather than autonomous: a filed SAR is a legal statement by the institution, so the model proposes and an investigator signs, and the model is itself subject to the validation regime of Section 15.4.2.

15.3.4 Sanctions Screening: A Matching Problem, Not a Lookup

A bank must not move money for a party on a government sanctions list, and unlike almost every other obligation in this chapter, this one admits no risk appetite: the prohibition is absolute, and in the United States liability under the Office of Foreign Assets Control (OFAC) regime is strict, meaning a bank that processes a prohibited payment has violated the law whether or not it knew, intended, or profited.

It is tempting to conclude from this that sanctions screening is a database lookup rather than a modeling problem. The legal *consequence* of a match is indeed binary, but the *match itself* is not, and conflating the two is the most common way to misunderstand where the difficulty lies. A watchlist records a name in one particular romanization; a payment message arrives carrying whatever spelling, word order, punctuation, and truncation the originating bank's system happened to produce. Nothing guarantees the two strings agree character for character even when they name the same person, and nothing prevents two genuinely different people from having names that differ by a single letter.

The standard tool for comparing two strings that should be the same but are not is the edit distance, or Levenshtein distance, $d(a, b)$: the minimum number of single-character insertions, deletions, or substitutions needed to turn one string into the other, normalized into a similarity score bounded in $[0, 1]$,

$$\mathrm{sim}(a, b) = 1 - \frac{d(a, b)}{\max(|a|, |b|)}.$$

Two short surnames make the mechanics concrete. Turning `SMITH` into `SMYTH` requires one substitution, so $d = 1$ and $\mathrm{sim} = 1 - 1/5 = 0.800$; turning `SMITH` into `SCHMIDT` requires four edits against a longer target, giving $\mathrm{sim} = 1 - 4/7 \approx 0.429$. A screening engine flags any pair scoring above some threshold and sends it to an analyst.

Table 15.7 applies this to a watchlist entry, `Mohammed Al-Rashid`, against seven incoming counterparty names, four of which are the same individual and three of which are not. Scores are computed twice: once after simple normalization (uppercase, punctuation stripped), and once after additionally sorting the name's tokens alphabetically, a standard preprocessing step meant to defeat surname-first ordering.

Token sorting earns its place immediately: `Al-Rashid, Mohammed` is the same name written surname-first, and plain edit distance scores it 0.2941, below every non-match in the table, because reversing two

Wulin.Suo@Queensu.ca

Counterparty name	Same person?	Normalized	Token-sorted
Mohammad Al-Rashid	yes	0.9412	0.9412
Mohamed Alrashid	yes	0.9412	0.9412
Muhammad Al-Rasheed	yes	0.7778	0.7778
Al-Rashid, Mohammed	yes	0.2941	1.0000
Mohammed Al-Rashad	no	0.9412	0.9412
Ahmed Al-Rashid	no	0.7647	0.2941
Mohammed Al-Karim	no	0.7647	0.7647

Table 15.7: Name-similarity scores against the watchlist entry `Mohammed Al-Rashid`

words costs almost as many edits as rewriting the string. Sorting the tokens first raises it to a perfect 1.0000. The same step also pushes `Ahmed Al-Rashid`, a different person, down from 0.7647 to 0.2941, improving separation in both directions at once.

The result that matters, though, is the one no amount of preprocessing fixes. `Mohammed Al-Rashad`, a different individual whose family name differs from the target's by a single vowel, scores 0.9412, tying *exactly* with two of the four genuine matches, while the weakest genuine match, `Muhammad Al-Rasheed`, scores only 0.7778. The highest-scoring non-match therefore outranks a true match, and no threshold whatsoever separates the two classes: set the bar at 0.90 and the engine clears a sanctioned party while flagging an innocent one; set it at 0.75 and it catches everything at the cost of flagging almost everyone.

This is not a defect in the Levenshtein metric that a better string distance would repair. It is a statement about the information content of a name: a name alone does not identify a person, and two of these strings are simply too close for any function of the characters to tell apart. Production screening systems accordingly treat the string score as a first-stage filter and resolve the remainder with secondary identifiers, date of birth, nationality, passport or national identity number, and address, which is precisely why the Know-Your-Customer records collected at onboarding (Section 15.3.1) do double duty as screening inputs rather than sitting inertly in a compliance file.

The strict-liability point now bites. Section 15.2.3's threshold choice was an economic trade-off, and Section 15.4.1 prices it in analyst hours; here one missed true match is a legal violation regardless of how favorable the arithmetic looks, so a bank cannot buy precision by accepting a little less recall. It sets the threshold low enough to make a miss nearly impossible and absorbs whatever false-positive volume results. That inverted operating point is what makes screening expensive.

The scale follows directly. Suppose a mid-sized bank screens 1,200,000 payment messages a month against consolidated watchlists, that 15 of them genuinely involve a sanctioned party, and that a threshold set for near-certain recall produces a 0.5% false-positive rate against the remaining traffic. Then

$$\text{Alerts} = 0.99(15) + 0.005(1{,}199{,}985) \approx 6{,}015, \qquad \text{Precision} = \frac{14.85}{6{,}015} \approx 0.00247,$$

so roughly 0.25% of screening alerts are genuine, against the 3.0% alert-to-SAR conversion that Section 15.4.1 computes for transaction monitoring: a precision twelve times worse. At 10 minutes of analyst time per screening alert, half what an AML alert takes in that section because most screening alerts are cleared on sight, and the same fully loaded \$45 hourly cost used for authorized push payment interventions above,

$$\text{Analyst hours} = \frac{6{,}015 \times 10}{60} \approx 1{,}002, \qquad \text{Monthly cost} \approx 1{,}002 \times \$45 \approx \$45{,}100,$$

or roughly \$541,000 a year. This is a second alert queue, three-quarters the size of the 8,000-alert transaction-monitoring queue analyzed in Section 15.4.1, running at one-twelfth its precision, and it exists at every institution that moves money across borders. Supervised learning enters here for a reason worth stating, because labels are abundant in screening where they are scarce almost everywhere else in this chapter: every alert is adjudicated by an analyst, so a bank accumulates hundreds of thousands of match and no-match decisions a year as a by-product of doing the work. A classifier trained on those dispositions scores a candidate pair on far more than string distance, on how common the name is in the relevant population,

Wulin.Suo@Queensu.ca

whether date of birth, nationality, and document number agree, whether the two spellings belong to the same transliteration family, and which kind of list entry was matched; and because it *ranks* rather than thresholds, it can hold recall fixed while pushing most false positives below the line at which the queue is worked. That is the concrete form of the improvement this section has been describing.

A reader who takes only one operational fact from this chapter should take this one: in most banks, name screening, not transaction monitoring, is the largest single source of compliance alerts, and reducing its false-positive rate without touching recall is among the highest-value applications of machine learning anywhere in financial crime compliance.

15.3.5 Privacy-Preserving Collaboration: Federated Learning Across Institutions

The layering networks above expose a structural weakness in single-institution detection: a laundering scheme deliberately spreads its transactions across many banks precisely so that no single bank sees enough of the pattern to recognize it. The obvious remedy, pooling transaction data across institutions, is largely prohibited: bank-secrecy and data-protection law restricts sharing customer data, and no bank wants its transaction records in a competitor's hands. Federated learning is the machine-learning answer to this impasse: institutions jointly train one shared model *without any of them ever transmitting its underlying data*, by exchanging only model updates. Each bank fits the current shared model to its own private transactions, sends back only the resulting parameter estimates, and a coordinator combines them, most simply by federated averaging, a sample-size-weighted mean of the local estimates, into the next version of the shared model.

A hand-checkable round makes the mechanics concrete. Suppose three banks jointly train the fraud-scoring logistic regression of Section 15.2.3 on a shared structuring-intensity feature, holding 2,000, 5,000, and 3,000 labeled transactions respectively. Their local coefficient estimates come back as 0.90, 1.30, and 1.10: each bank's view is real but partial, and the smallest bank's estimate is the noisiest. Federated averaging weights each estimate by its sample share,

$$\hat{\beta}_{\text{fed}} \;=\; \frac{2{,}000 \times 0.90 + 5{,}000 \times 1.30 + 3{,}000 \times 1.10}{10{,}000} \;=\; \frac{11{,}600}{10{,}000} \;=\; 1.16,$$

against a naive unweighted mean of 1.10; the weighting matters because it lets the estimate lean toward the banks with the most evidence. The payoff is easiest to see from the smallest bank's seat: scoring a transaction with structuring intensity $x = 2$ (with intercept -3), its own local model flags it with probability $\sigma(-3 + 0.90 \times 2) \approx 0.2315$, while the federated model assigns $\sigma(-3 + 1.16 \times 2) \approx 0.3363$, a meaningfully sharper detector obtained without a single customer record leaving any bank.

Model updates are not automatically harmless, though: a parameter estimate computed from a small dataset can itself leak information about the records behind it, so production systems add differential privacy, calibrated random noise added to each institution's update before transmission, sized so that no observer of the update can reliably infer whether any single transaction was present in the underlying data. The aggregation step is what makes this affordable: perturbing the three local estimates above by illustrative noise draws of $+0.04$, -0.02, and $+0.01$ changes the federated coefficient only from 1.16 to 1.161, because independent noise largely cancels in a weighted average even while masking each individual contribution. The privacy-utility trade-off does bind, more noise means more privacy and a worse model, and federated systems inherit new operational questions of their own (a malicious participant could poison the shared model through its updates, the adversarial dynamic Chapter 18's robustness discussion examines, transplanted into the training loop), but the core bargain, most of the pooled-data detection gain at none of the pooled-data legal exposure, is why cross-institution federated AML pilots have moved from research into supervised production trials.

15.4 Economics and Governance

A detection model's value is decided by the cost of its false positives and by who is accountable for it. This part is deliberately the shortest in the chapter, and the imbalance is itself the point: detection methods earn

Wulin.Suo@Queensu.ca

their length because they can be taught with worked examples, whereas the failures that actually produce enforcement actions are organizational, and are better shown once and concretely, by the case vignette that follows, than developed at method length.

15.4.1 Alert Economics and the Cost of Low Precision

Section 15.2.2 showed that a single bank's fraud model can generate 19,600 flagged transactions in a single day; AML monitoring systems, layering rule-based structuring checks, behavioral risk scores, and sanctions screening on top of transaction scoring, typically generate a comparably large alert volume relative to the number that ultimately warrant a filed report.

Suppose a bank's combined monitoring system generates 8,000 alerts in a typical month, of which 240 are ultimately investigated and filed as SARs,

$$\text{Alert-to-SAR conversion rate} = \frac{240}{8{,}000} = 3.0\%,$$

a precision figure of the same qualitative magnitude as Section 15.2.2's 24.2% fraud-flag precision, and considerably lower once every layer of monitoring is combined, since each additional rule or model adds its own false-positive rate. If each alert requires an average of 20 minutes of an investigator's time to review, even the 97% of alerts that are ultimately cleared as false positives,

$$\text{Analyst hours per month} = \frac{8{,}000 \times 20 \text{ minutes}}{60} \approx 2{,}667 \text{ hours},$$

equivalent to roughly 16 to 17 full-time analysts working a standard month purely to clear the alert queue this single monitoring system generates. At a fully loaded compliance-analyst cost of \$45 per hour, a common industry rule of thumb once salary, benefits, training, and overhead are included, this translates directly into a monthly labor cost of

$$2{,}666.67 \times \$45 \approx \$120{,}000,$$

or roughly \$1.44 million annually, purely to staff alert review for one monitoring system at one institution, before accounting for the cost of the underlying data infrastructure, the models themselves, or the SAR-filing process that follows a confirmed case. This dollar figure is what makes precision improvements a direct cost-reduction lever rather than merely a statistical nicety.

If a model improvement doubled precision from 3.0% to 6.0% while holding recall (and therefore the 240 SARs actually filed) fixed, the same 240 confirmed cases would require only $240/0.06 = 4{,}000$ alerts to surface, exactly half of the original 8,000, cutting analyst hours to $4{,}000 \times 20/60 \approx 1{,}333$ hours and monthly labor cost to $1{,}333 \times \$45 \approx \$60{,}000$, precisely half of the original \$120,000, since cost scales linearly with alert count once recall (and therefore the fixed number of investigations that must occur regardless of precision) is held constant.

This is why a bank's compliance technology budget and its model-development priorities are, in practice, set jointly rather than by the compliance and modeling functions independently: a data-science team's precision improvement translates directly, dollar for dollar, into the compliance department's staffing budget in a way few other model performance metrics do. This is precisely why the layered approach developed in this chapter, transaction scoring (Section 15.2.3), behavioral risk tiering (Section 15.3.1), and survival-informed prioritization (Section 15.3.2), matters as much for its operational cost efficiency as for its detection accuracy. A monitoring system that better ranks alerts by true risk, even without changing its overall recall, lets a fixed investigative staff spend its limited hours on the alerts most likely to be genuine. That is the same precision-recall trade-off Chapter 6 introduced, now expressed directly in analyst-hours and dollars rather than as an abstract statistical metric.

15.4.2 Governance: Model Risk Management and the Three Lines of Defense

Chapters 8 and 9 introduced model risk management, independent validation, ongoing performance monitoring, and clear accountability for a model's use, in the context of market and credit risk models; Chapter 14 revisited the same discipline for algorithmic trading systems, building on Chapter 13's Knight Capital

Wulin.Suo@Queensu.ca

vignette. Fraud and AML models carry an additional governance obligation beyond the purely financial risk those earlier chapters addressed: a poorly governed AML program exposes the institution itself to direct regulatory and criminal liability, not merely financial loss, since failing to detect and report money laundering is, in most jurisdictions, itself a legal violation regardless of whether the institution profited from it.

Banks and regulators structure this accountability using the three lines of defense framework that Chapter 8's risk-governance section develops in full, and the structure itself carries over unchanged: business units own the first line, an independent compliance and risk function the second, internal audit the third. What differs in the financial-crime setting is what each line is accountable *for*. The second line here builds and validates the scoring and monitoring models this chapter has developed rather than the VaR models of Chapter 8, and the third line's independent testing is itself subject to regulatory examination, so a weak audit function is not merely an internal control gap but a finding a supervisor can act on directly.

This structure is a direct, institutionalized answer to a failure mode this chapter's case vignette illustrates concretely: a monitoring system can exist on paper, and even generate the alert volumes quantified in Section 15.4.1, while still failing in practice if the first line lacks the incentive or staffing to act on alerts, if the second line's independent challenge is weak or under-resourced, or if the third line's audits are not, in fact, independent of the business they are reviewing.

Model Monitoring: Precision Drift and Adversarial Adaptation

Chapter 14 introduced concept drift, a predictive model's fitted relationship growing stale as market conditions change, as a reason high-frequency trading models require frequent retraining. Fraud and AML models face a related but distinct version of the same problem: because Section 15.2.1 identified financial crime as adversarial, fraudsters and launderers who learn (directly or by trial and error) which transaction patterns a bank's model tends to flag will deliberately shift their behavior away from those patterns, degrading the model's precision even if the underlying population of legitimate customers has not changed at all. Table 15.8 illustrates a stylized six-month trajectory for the fraud-scoring model of Section 15.2.3, holding its classification threshold fixed.

Month	1	2	3	4	5	6
Precision	24.2%	22.5%	20.8%	19.6%	18.3%	17.1%

Table 15.8: An illustrative six-month precision drift at a fixed classification threshold. The trajectory is constructed to show the shape, not measured from a real portfolio.

If the second line has set a minimum acceptable precision floor of 20% below which the alert-economics burden of Section 15.4.1 is judged too costly relative to the fraud actually being caught, Table 15.8 crosses that floor between month 3 and month 4, which should trigger a specific, pre-defined governance response, refitting the model on recent data, adding new features that capture the fraudsters' apparent behavioral shift, or tightening the classification threshold, rather than allowing the model to keep running unchanged simply because no single month's decline looks alarming on its own. This is precisely the kind of ongoing performance monitoring Chapter 8's model risk management framework requires, and it is the second line's specific responsibility under the three-lines-of-defense structure above to define the precision floor in advance and act automatically once it is breached, rather than leaving the decision to the first-line staff who are, understandably, focused on clearing today's alert queue rather than questioning whether the underlying model still works.

15.5 Case Vignette: The Danske Bank Estonia Case

Between roughly 2007 and 2015, the Estonian branch of Danske Bank, Denmark's largest bank, processed an estimated 200 billion euros in payments through a portfolio of non-resident customers, many with connections to Russia and other former Soviet states, that was later found to include large volumes of transactions with no plausible legitimate economic purpose. The branch had inherited this non-resident customer portfolio through an earlier acquisition and, rather than winding it down as a high-risk legacy business, continued

Wulin.Suo@Queensu.ca

serving and even expanding it, drawn by the fee income a small branch's outsized transaction volume generated. An internal whistleblower first raised concerns in 2013; an external investigation the bank itself later commissioned found extensive, longstanding failures in customer due diligence, ongoing monitoring, and the bank's own internal escalation of red flags, precisely the first- and second-line failures Section 15.4.2's three-lines-of-defense framework is designed to prevent. Estonian authorities ultimately required the branch's closure in 2019, and in 2022 Danske Bank pleaded guilty in the United States to fraud charges connected to the case, agreeing to forfeit and pay roughly $2 billion.

The case is instructive for this chapter specifically because it was not, primarily, a failure of statistical modeling. Rule-based transaction monitoring, of the kind Section 15.3.3 describes, was reportedly in place at the branch. But it generated so many alerts, echoing Section 15.4.1's alert-economics discussion, relative to the compliance staff available to review them, that a large fraction went uninvestigated or were cleared without adequate scrutiny. It is also a case study in correspondent banking risk specifically. Much of the flagged activity moved through the branch's relationships with other banks acting as intermediaries for the ultimate, often difficult-to-identify beneficial owners of the funds. That structural feature of international payments makes a single bank's own customer due diligence, however well executed, an incomplete safeguard on its own. The case's lasting regulatory legacy has been a much closer supervisory focus, across the European Union and beyond, on the governance question Section 15.4.2 formalizes. The question is not merely whether a monitoring system technically exists. It is whether an institution can demonstrate, to an independent third line and ultimately to a regulator, that alerts are being investigated and reported at a standard consistent with the risk the underlying customer base actually presents.

15.6 Challenges in Fraud Detection, Compliance, and Financial Crime

Several challenges recur across nearly every application in this chapter, and many connect directly to themes developed elsewhere in this book.

Extreme class imbalance makes accuracy a misleading metric. As Section 15.2.2's scaled Bayes' theorem example showed concretely, even a highly accurate-sounding model produces a large number of false positives in absolute terms whenever the event being detected is rare, which is precisely why this chapter, like Chapter 6's imbalanced-classification discussion, insists on precision, recall, and alert-volume economics rather than raw accuracy.

The adversary adapts, which most of this book's other models do not have to contend with. A credit-scoring model's population of borrowers (Chapter 9) is not actively trying to defeat the model; a launderer engaged in structuring (Section 15.3.3) is specifically designing behavior to evade a known detection rule, meaning a fraud or AML model's effectiveness can decay not just through the ordinary concept drift Chapter 14 discussed for order-flow models, but through deliberate, targeted evasion once a detection threshold becomes known or guessable.

Data across institutions and jurisdictions is fragmented. The correspondent-banking structure behind this chapter's case vignette illustrates a recurring practical limit: any single institution's monitoring system, however well built, only observes its own slice of a transaction chain that may span several banks and countries, which is why network-level and cross-institutional information sharing, within the legal limits privacy and banking secrecy law impose, materially improves detection relative to any one institution acting alone, and why the federated learning of Section 15.3.5 is worth the operational complexity it adds.

False positives carry a real customer and reputational cost, not just an operational one. Every incorrectly frozen account or declined transaction identified in Section 15.4.1's alert queue is also a legitimate customer temporarily denied normal use of their own funds, a cost that does not show up in a precision or recall statistic but matters directly to customer relationships and, in aggregate, to regulatory scrutiny of whether a program's false-positive rate is itself proportionate.

Explainability requirements constrain model choice. As Chapter 18 develops further, a suspicious activity report or an account closure decision frequently must be explained to a regulator or, in some jurisdictions, to the customer, which limits how much a fraud or AML program can rely on the least interpretable models Chapter 6 introduced, in tension with those models' potential accuracy advantage.

Governance failures, not modeling failures, are the more common root cause of large-scale

Wulin.Suo@Queensu.ca

enforcement actions. The Danske Bank Estonia case, like the Knight Capital vignette in Chapter 13, illustrates that the binding constraint is frequently not the sophistication of the detection model but whether the organization around it, staffing, escalation, independent challenge, and audit, actually functions as the three-lines-of-defense framework in Section 15.4.2 assumes it does on paper.

15.7 Key Takeaways

Fraud detection and anti-money-laundering compliance apply the classification tools Chapter 6 developed and Chapter 9 specialized to credit risk to a domain defined by extreme class imbalance and an adversary that actively adapts to known detection rules. Beyond a single logistic regression, production systems draw on a broader toolkit developed in Section 15.2.4: class weighting and resampling techniques such as SMOTE address the imbalance directly, tree-based ensembles capture nonlinear feature interactions a purely additive model misses, unsupervised anomaly detection catches patterns no confirmed label yet describes, and precision-recall AUC, not ROC-AUC, is the evaluation metric suited to a rare-event problem like this one.

Chapter 1's Bayes' theorem fraud example, scaled to a full day of transaction volume and re-applied to insider trading surveillance, shows concretely why even an apparently accurate model generates a large false-positive alert queue, and that precision at a low base rate is far more sensitive to the false-positive rate than to the true-positive rate, so alert economics, not just statistical accuracy, must guide how a fraud or AML program is designed and staffed.

A single-transaction scoring model, however well fit, structurally cannot catch a fraud that looks unremarkable in isolation, which motivates account-level behavioral risk tiering and, this chapter's methodologically new contribution, survival analysis, the Kaplan-Meier estimator, to characterize how account risk accumulates and resolves over time using both confirmed events and censored observations.

Two growing fraud types defeat transaction-level scoring outright rather than merely straining it. A synthetic identity has no victim to report it, so its losses are booked as ordinary credit loss and never reach a supervised model's training labels at all, which is the strongest case in this chapter for unsupervised detection and for linkage analysis across accounts. An authorized push payment scam is genuinely authorized by the customer, so no feature of the payment itself carries signal, and detection must move to the customer's own baseline and to the beneficiary account; mandatory reimbursement is what turned that from a customer-education problem into an expected-cost calculation a bank can solve.

Sanctions screening looks like a lookup and is in fact a matching problem: Section 15.3.4's worked example shows a non-match tying exactly with two genuine matches on name similarity alone, so no threshold separates them and secondary identifiers must finish the job. Because liability for a missed match is strict, recall cannot be traded for precision the way it can everywhere else in this chapter, which is why name screening is typically a bank's largest single alert queue and its least precise one.

Anti-money-laundering regulation adds a reporting and due-diligence apparatus, CTRs, SARs, KYC, and sanctions screening, with no direct counterpart in ordinary credit or fraud risk modeling, built specifically around legally defined thresholds that launderers attempt to evade through the placement-layering-integration sequence, structuring at the placement stage and shell-entity network chains at the layering stage. The Danske Bank Estonia case shows that the binding constraint on a compliance program is at least as often organizational, whether the three lines of defense actually function independently, as it is statistical, and that governance discipline, not just modeling sophistication, is what separates a monitoring system that works on paper from one that works in practice.

15.8 Suggested Exercises

1. A different fraud model achieves a 90% true-positive rate and a 1% false-positive rate at the same 1% fraud prevalence Chapter 1 assumed. Using Bayes' theorem, compute the resulting precision, and explain why a lower false-positive rate improved precision more than the higher true-positive rate in Chapter 1's original model.

2. Using the insider trading surveillance example in Section 15.2.2, recompute Pr(Insider | Flag) if the

Wulin.Suo@Queensu.ca

false-positive rate is reduced from 8% to 4%, holding the 2% base rate and 75% true-positive rate fixed, and compare the size of this improvement to what a similarly sized improvement in the true-positive rate alone would produce.

3. Using Table 15.2, suppose an eleventh transaction has an amount z-score of 2.2, a distance of 4.0 (hundreds of km), and occurs during the day (late-night flag 0). Using the fitted coefficients in Section 15.2.3, compute the transaction's predicted fraud probability and classification at a 0.5 threshold.

4. Explain, referencing transaction 9 in Table 15.2, why a transaction-level model can have perfect precision and still leave a bank exposed to fraud, and describe how Section 15.3.1's behavioral risk tiering could catch the case the transaction-level model missed.

5. Using Section 15.2.4's class-weighting example, explain why raising transaction 9's predicted probability from 33% to 41.7% was not, by itself, enough to fix the false negative at the original 0.5 threshold, and why the combination of class weighting and a lower threshold reached the same operating point as simply lowering the unweighted model's threshold to 0.30.

6. Explain, in a short paragraph, why ROC-AUC can overstate a fraud model's practical usefulness under the kind of extreme class imbalance this chapter has emphasized, and why precision-recall AUC does not share this weakness.

7. Using Table 15.5's method, compute the Kaplan-Meier survival estimate for a new sample of 8 flagged accounts with observed times (days, + denotes censored) of 4, 6^+, 10, 14, 14^+, 22, 28^+, 33, showing your calculation at each confirmed-fraud event time.

8. A customer makes four cash deposits of \$9,200 each within five days. Compute the total deposited, and explain, using Section 15.3.3's structuring discussion, why no individual deposit triggers a Currency Transaction Report even though the total clearly would if deposited at once.

9. Using Table 15.6's method, compute a five-hop layering chain starting from \$300,000, with each entity retaining 3% and forwarding the rest, and report the amount reaching the final destination and the total retained across all five hops.

10. A bank's monitoring system generates 12,000 alerts per month, of which 180 become filed SARs, and each alert takes an average of 25 minutes to review. Compute the alert-to-SAR conversion rate and the total analyst-hours required per month, and compare both figures to Section 15.4.1's worked example.

11. Using Section 15.4.2's three-lines-of-defense framework, identify which line (first, second, or third) each of the following failures most directly represents, and explain why: (a) a branch employee ignores an internal escalation policy for a suspicious customer; (b) an independent audit team reports directly to the same regional manager whose unit it is auditing; (c) the compliance function never independently tests whether the transaction-monitoring model's alert thresholds are still appropriate.

12. Pick two of the challenges listed in this chapter and describe a concrete scenario, not already used in the text, in which that challenge could cause a fraud or AML program to fail despite a well-designed underlying statistical model, and explain briefly why adding more computing power or a more flexible model would not, by itself, resolve either scenario you describe.

13. Explain, using the Danske Bank Estonia case vignette, why correspondent banking relationships make financial crime detection harder for any single institution acting alone, and describe one mechanism (discussed in this chapter or otherwise), such as cross-institution information sharing or a shared utility for sanctions and beneficial-ownership screening, that could mitigate this limitation.

14. Compare the class-imbalance problem in this chapter's fraud examples to the loan-default classification example in Chapter 6: using Chapter 6's confusion matrix (a 75% true-positive rate and, among the six actual non-defaulters, one false positive), compute what the loan portfolio's default rate would need to be, holding the same true-positive and false-positive rates constant, for precision to fall to the same 24.2% level as this chapter's fraud example.

Wulin.Suo@Queensu.ca

15. Using Greenwood's formula, compute $\widehat{\mathrm{Var}}[\hat{S}(12)]$ and the resulting 95% confidence interval for $\hat{S}(12) \approx 0.7407$, and compare its width to the width of the interval already computed at $t = 20$ in the text, explaining why the earlier time point's interval is narrower even though both are computed from the same 12 accounts.

16. Using Section 15.3.4's similarity definition, compute sim for the pairs `MOHAMMED ALRASHID`/`MUHAMMED ALRASHID` and `MOHAMMED ALRASHID`/`MOHAMMED ALRASHEED`, and explain why the second pair, which involves a genuinely different family name, nonetheless scores higher than `Muhammad Al-Rasheed` does in Table 15.7.

17. Using Section 15.3.4's screening-volume example, recompute the monthly alert count, precision, analyst hours, and dollar cost if a better matching engine cut the false-positive rate from 0.5% to 0.2% while holding recall at 99%, and express the annual saving relative to the $541,000 figure in the text. Then compute what the same 60% reduction in false positives would save in Section 15.4.1's transaction-monitoring queue, and explain why a bank might nonetheless prioritize the screening engine even though the transaction-monitoring saving turns out to be the larger of the two.

18. Explain why a bank cannot manage its sanctions-screening alert volume the way Section 15.2.3 manages its fraud-scoring alert volume, by raising the classification threshold, and describe what a screening program can change instead in order to reduce false positives without reducing recall.

19. Using Section 15.2.5's cultivation arithmetic, recompute the bust-out for a synthetic identity that grows *six* tradelines to a $7,500 limit each over 24 months while paying $150 a month across all of them. Report the amount paid in, the exposure at bust-out, the net loss, and the return on the fraudster's outlay, and explain why the return multiple is lower here than the text's despite the larger absolute loss.

20. Suppose the reimbursement rules of Section 15.2.5 were tightened so the sending bank bore 100% of authorized push payment losses rather than half. Holding the 500,000 monthly payments, 0.02% scam rate, $4,200 average loss, 60% recall, 5% precision, 15-minute intervention and $45 hourly cost fixed, recompute the loss prevented, the net monthly benefit, and the break-even recall, and comment on what the change in break-even recall implies about how much detection effort a bank would rationally fund under each regime.

21. **Challenge (real data).** Download the Kaggle "Credit Card Fraud Detection" dataset (Université Libre de Bruxelles; roughly 285,000 European cardholder transactions over two days, with 492 confirmed frauds and features already anonymized via PCA). Its 0.172% fraud rate is far more extreme than Table 15.2's ten-transaction toy example. (a) Fit a logistic regression with and without class weighting, as in Section 15.2.4, and report precision, recall, F_1, and precision-recall AUC for both, since accuracy alone is uninformative at this imbalance. (b) Using Section 15.4.1's cost-based framework and assuming a $50 review cost per flagged transaction and a $200 average loss per missed fraud, find the threshold that minimizes total expected cost, and compare its alert volume to what a naive 0.5 threshold would produce. (c) Since the features are already anonymized PCA components rather than raw transaction attributes, this dataset cannot support the same kind of domain-driven feature engineering (for example, distance from home) used in this chapter's own toy example; discuss what information a real bank's fraud team has access to that this public, privacy-scrubbed dataset deliberately withholds.

15.9 Further Reading

- Bolton and Hand, "Statistical Fraud Detection: A Review," *Statistical Science*. A foundational survey of statistical approaches to fraud detection, including the class-imbalance and base-rate issues central to Section 15.2.2.
- Chawla, Bowyer, Hall, and Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*. The original paper introducing the resampling technique discussed in Section 15.2.4.

Wulin.Suo@Queensu.ca

- Klein, *Survival Analysis: Techniques for Censored and Truncated Data*. A standard, thorough reference on the Kaplan-Meier estimator and censoring introduced in Section 15.3.2.
- Financial Action Task Force (FATF), *International Standards on Combating Money Laundering and the Financing of Terrorism and Proliferation*. The primary international regulatory framework underlying the KYC, CDD, and reporting obligations discussed in Section 15.3.3.
- U.S. Department of the Treasury, Office of Foreign Assets Control, *A Framework for OFAC Compliance Commitments*. The regulator's own statement of what a sanctions compliance program must include, and the primary source for the strict-liability standard underlying Section 15.3.4.
- U.S. Financial Crimes Enforcement Network (FinCEN), *Bank Secrecy Act Regulations and Guidance*. The primary U.S. regulatory source for the CTR and SAR reporting requirements discussed in this chapter.
- Bruun & Hjejle, *Report on the Non-Resident Portfolio at Danske Bank's Estonian Branch*. The external investigation commissioned by Danske Bank itself, the primary source for the factual account in this chapter's case vignette.
- Institute of Internal Auditors, *The Three Lines Model: An Update of the Three Lines of Defense*. The current authoritative statement of the governance framework developed in Section 15.4.2.

Wulin.Suo@Queensu.ca

Chapter 16

Natural Language Processing in Finance

16.1 Introduction

Every earlier chapter in this book has worked with numbers that arrive already structured: prices, returns, ratios, transaction amounts. Yet a large share of the information that moves financial markets never starts out as a number at all. It starts as an earnings-call transcript, a paragraph in a 10-K's risk-factors section, a news wire, or a social-media post, and it must be converted into features a model can use before any of the tools developed in Chapters 6 through 15 apply to it. This chapter develops that conversion, and the family of techniques, collectively natural language processing (NLP), built on top of it.

Several forward references from earlier chapters converge here. Chapter 1 noted that deep learning's ability to learn directly from unstructured text is the foundation for this chapter, and that large language models (LLMs) extend those text-based methods to summarizing earnings calls and extracting structured information from filings. Chapter 3 observed that large asset managers use NLP to parse earnings calls and filings faster than human analysts. Chapter 6 introduced Naive Bayes as a simple generative classifier and previewed its use for text classification, where features are word counts, and noted that LLMs are far more elaborate examples of the same generative principle applied to text. Chapter 13 flagged that extracting a sentiment score from an earnings call is only half of an event-driven trading strategy, converting that signal into an executed trade is the other half, and belongs to the execution machinery that chapter developed. This chapter is where all of those threads are made concrete.

Eighteen worked examples run through the chapter. The first few build representations: a bag-of-words and TF-IDF representation of a small set of earnings-call sentences, and a financial sentiment score built from a subset of the Loughran-McDonald dictionary, the sentiment lexicon actually used in finance research and practice. Those feed an event-driven trading signal, evaluated the way Chapter 13 evaluates any trading strategy, and a Naive Bayes classifier that sorts 10-K risk-factor sentences into categories, extending Chapter 6's generative-model discussion. The chapter then turns to modern architectures: a small worked illustration of word-embedding geometry, including a vector-arithmetic analogy, and a fully hand-computed scaled dot-product self-attention calculation, the core mechanism inside transformer models, extended to show how a causal mask changes a token's representation across encoder-only, decoder-only, and encoder-decoder architectures. A retrieval-augmented generation example that grounds a generated answer in a specific retrieved passage, extended with precision@k and faithfulness-score evaluation metrics; a forward diffusion step illustrating how generative models manufacture synthetic stress-test scenarios; a Gunning Fog readability comparison motivating disclosure analysis; a low-rank adaptation (LoRA) parameter-reduction calculation for efficient fine-tuning; a three-step LLM agent that computes and then independently cross-checks a financial ratio; and an API-versus-self-hosting cost comparison quantifying when quantization's memory savings translate into real deployment savings.

Because generative AI, large language models, retrieval-augmented generation, prompting, and synthetic-data generation via GANs and diffusion models, is developed here in more depth than any single topic in

Wulin.Suo@Queensu.ca

earlier chapters, this chapter runs longer than the rest of the book.

16.2 Text as Data

Most financial information arrives as prose, and this part turns that prose into numbers a model can use.

16.2.1 Text as Data in Finance

Financial text differs from the numerical data this book has used so far in three ways that matter for modeling. First, it is unstructured: a sentence does not arrive with predefined fields the way a balance sheet line item does, so the first modeling step is always to impose some structure on it. Second, it is highly domain-specific: ordinary sentiment analysis trained on product reviews or social media routinely misreads financial language, the word "liability" is neutral accounting terminology, not a negative sentiment word, and a sentence like "losses were smaller than expected" is good news phrased with two words, losses and smaller, that a generic sentiment model would likely both score as negative. Third, financial text is produced under legal and strategic incentives that shape its content: a firm's management chooses what to disclose, how to phrase it, and how much hedging language to use, so the text itself is a strategic object, not a neutral report of underlying reality.

Table 16.1 catalogues the main sources of financial text this chapter and the rest of the book draw on, and where each is developed further.

Source	Typical use	Developed in
Earnings call transcripts	Management tone and forward guidance, extracted as a sentiment score	Sections 16.2.3, 16.3.1
Regulatory filings (10-K, 10-Q)	Risk-factor classification, disclosure readability	Sections 16.3.2, 16.5.4
News wires and press releases	Event detection, sentiment scoring for event-driven strategies	Section 16.3.1
Social media and analyst reports	Aggregate sentiment signals, higher noise, higher volume	Section 16.4.2

Table 16.1: Sources of financial text and where they are used in this chapter

Figure 16.1 sketches the pipeline this chapter follows, from raw text to a model-ready representation to an application. Every technique in this chapter is a way of filling in the middle step.

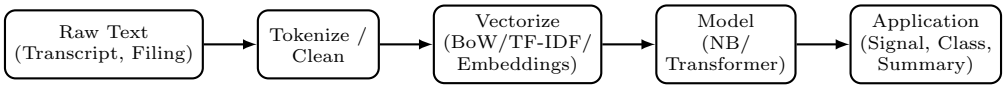


Figure 16.1: The text-to-application pipeline this chapter develops

16.2.2 From Text to Numbers: Bag-of-Words and TF-IDF

Suppose an analyst has thousands of earnings-call transcripts and wants to feed them into one of Chapter 6's classifiers, every one of which expects a fixed-length numeric vector as input, not a paragraph of English text. The simplest way to turn text into a feature vector is the bag-of-words representation: fix a vocabulary of terms, and represent each document as the count of each vocabulary term it contains, discarding word order entirely. This is a drastic simplification, "growth despite headwinds" and "headwinds despite growth" produce identical vectors, but it is a surprisingly effective baseline, and it is the "word counts" feature representation Chapter 6 anticipated for Naive Bayes text classification.

Wulin.Suo@Queensu.ca

A raw count, however, overweights common words that appear in almost every document and carry little discriminating information. Term frequency-inverse document frequency (TF-IDF) corrects for this. For term t in document d, drawn from a collection of N documents,

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \ln\left(\frac{N}{\text{df}(t)}\right),$$

where $\text{tf}(t, d)$ is the raw count of t in d and $\text{df}(t)$ is the number of documents containing t at least once. A term that appears in every document has $\text{df}(t) = N$ and therefore an idf of $\ln(1) = 0$, its tf-idf weight vanishes regardless of how often it appears, while a term confined to a single document receives the largest possible idf weight.

A Worked Example: Bag-of-Words and TF-IDF for Four Earnings-Call Sentences

Consider four short earnings-call sentences, treated as four documents ($N = 4$):

D_1: "revenue growth exceeded expectations this quarter"
D_2: "management is cautious about revenue growth given the headwinds"
D_3: "management remains confident despite the headwinds this quarter"
D_4: "expectations were not met and revenue declined"

Restricting attention to six vocabulary terms gives the term-frequency matrix in Table 16.2, alongside each term's document frequency, idf, and resulting tf-idf weight in D_1.

Term	D_1	D_2	D_3	D_4	df	idf = ln(4/df)	tf-idf in D_1
revenue	1	1	0	1	3	0.2877	0.2877
growth	1	1	0	0	2	0.6931	0.6931
headwinds	0	1	1	0	2	0.6931	0.0000
expectations	1	0	0	1	2	0.6931	0.6931
quarter	1	0	1	0	2	0.6931	0.6931
management	0	1	1	0	2	0.6931	0.0000

Table 16.2: Term frequency, document frequency, idf, and TF-IDF weight in D_1

The first four columns of Table 16.2 are themselves the bag-of-words representation. Reading down them over the vocabulary (revenue, growth, headwinds, expectations, quarter, management) gives $D_1 = (1, 1, 0, 1, 1, 0)$ and $D_2 = (1, 1, 1, 0, 0, 1)$, and these are the vectors a classifier would receive if no weighting were applied at all. They also make the discarding of word order concrete: any rearrangement of D_1's words, grammatical or not, produces exactly this same vector, and D_1 and D_4 are recorded as sharing two terms even though one sentence reports growth and the other a decline.

The word "revenue" appears in three of the four documents, so it earns a modest idf of $\ln(4/3) \approx 0.2877$; every other listed term appears in exactly two documents, giving a larger idf of $\ln(4/2) = \ln 2 \approx 0.6931$. "Headwinds" and "management" each appear in D_1 zero times, so their tf-idf contribution to D_1's vector is zero regardless of their idf weight, while "growth," "expectations," and "quarter" each contribute their full idf value of 0.6931 since each occurs exactly once in D_1. This is the representation a downstream classifier, such as the Naive Bayes model in Section 16.3.2, would receive as input in place of raw counts.

16.2.3 Financial Sentiment Analysis and the Loughran-McDonald Dictionary

Suppose a trader wants to know, within seconds of an earnings call ending, whether management sounded upbeat or defensive, without waiting for an analyst to read the transcript and write a summary. The most direct way to score sentiment is a lexicon-based approach: maintain a list of positive and negative financial words, count how many of each appear in a document, and combine the counts into a tone score,

$$\text{tone} = \frac{\#\text{positive words} - \#\text{negative words}}{\#\text{total words}}.$$

Wulin.Suo@Queensu.ca

Generic sentiment lexicons, built from product reviews or general text, perform poorly in finance for the reason Section 16.2.1 raised: words like "tax," "cost," or "liability" read as negative in everyday English but are routine, neutral vocabulary in a financial filing. The Loughran-McDonald dictionary, developed specifically from 10-K filings, addresses this by classifying words into finance-specific categories (positive, negative, uncertainty, litigious, and others) based on how they actually function in financial disclosure.

A Worked Example: Scoring Two Earnings-Call Transcripts

Using a small subset of real Loughran-McDonald positive words ({strong, growth, exceeded, confident, improved}) and negative words ({cautious, headwinds, decline, declined, weak, concern, concerns}), consider two short transcripts:

> Transcript A: "revenue growth was strong this quarter and results exceeded expectations across every segment we remain confident heading into next year"
> Transcript B: "management is cautious about headwinds and results declined modestly we have some concerns about demand next quarter"

Transcript A contains 20 words, of which 4 (growth, strong, exceeded, confident) are positive and none are negative, giving a tone score of $(4-0)/20 = 0.20$. Transcript B contains 17 words, of which none are positive and 4 (cautious, headwinds, declined, concerns) are negative, giving a tone score of $(0-4)/17 \approx -0.2353$. These two scores, one clearly positive and one clearly negative, are the kind of feature that feeds directly into the event-driven trading signal developed next.

A Second Worked Example: Scoring Central-Bank Communication

Company disclosures are not the only text markets parse by machine. The Federal Open Market Committee's post-meeting statement is arguably the single most market-moving recurring document in finance, and it is scored with the machinery above, with one twist: the relevant axis is not positive versus negative but *hawkish* versus *dovish*, language signaling tighter future monetary policy versus easier. The distinction matters because the sign convention depends on the asset: a "strong" labor market is good news about the economy but hawkish news for bond prices, so a general-purpose sentiment dictionary, and even the Loughran-McDonald dictionary itself, misreads central-bank language for the same reason generic lexicons misread 10-Ks, and practitioners maintain purpose-built hawkish and dovish word lists instead.

Using the hawkish list {strong, elevated, increases, tightening, restrictive} and the dovish list {moderated, eased, easing, patient, softening}, consider two consecutive stylized statements:

> January: "inflation remains elevated and the committee anticipates that ongoing increases in the target range will be appropriate the labor market remains strong"
> March: "inflation has moderated in recent months and the committee will be patient as it assesses incoming data the labor market has shown signs of softening"

The January statement contains 22 words, of which 3 (elevated, increases, strong) are hawkish and none are dovish, giving a net-hawkishness score of $(3-0)/22 \approx +0.1364$. The March statement contains 25 words, of which none are hawkish and 3 (moderated, patient, softening) are dovish, scoring $(0-3)/25 = -0.1200$. The number a rates trader actually cares about, though, is neither level but the *change* between them: a drift of $-0.1200 - 0.1364 \approx -0.2564$, a decisive swing from hawkish to dovish. Because every market participant has already read the previous statement, the information in a new one lies almost entirely in what changed, which is why professional coverage of an FOMC release is dominated by redline comparisons against the prior statement, and why the tone *difference*, not the tone itself, is the natural trading signal, the event-driven logic of the next section applied to the policy calendar instead of the earnings calendar. Research versions of this idea, scoring decades of central-bank communication and measuring the market reaction to the component of each statement that could not have been anticipated, are by now a standard tool in empirical monetary economics (see Further Reading).

Wulin.Suo@Queensu.ca

16.3 Classical Text Models

Before transformers, text was classified and traded on with methods that remain fast, auditable, and often sufficient.

16.3.1 A Worked Example: An Event-Driven Sentiment Trading Signal

Chapter 13 emphasized that extracting a sentiment score from an earnings call is only half of an event-driven strategy: the signal must still be evaluated the way any trading strategy is evaluated, and eventually converted into an executed position, which is Chapter 13's concern, not this chapter's. This section stops at the first half, generating and evaluating the signal itself.

Suppose eight earnings-call events produce the tone scores and next-day stock returns in Table 16.3. A simple trading rule takes a long position when tone is positive and a short position when tone is negative, so the strategy's return on each event is sign(tone) $\times$ return.

Event	Tone score	Next-day return (%)	Signal	Strategy return (%)	Correct?
1	+0.15	+1.2	Long	+1.2	Yes
2	−0.10	−0.8	Short	+0.8	Yes
3	+0.08	+0.3	Long	+0.3	Yes
4	−0.05	+0.4	Short	−0.4	No
5	+0.20	+2.1	Long	+2.1	Yes
6	−0.18	−1.5	Short	+1.5	Yes
7	+0.02	−0.2	Long	−0.2	No
8	−0.12	−0.9	Short	+0.9	Yes

Table 16.3: Eight earnings-call events: tone score, next-day return, signal, and strategy return

The strategy is correct, meaning the sign of its position matches the sign of the subsequent return, in 6 of 8 events, a 75% hit rate. Events 4 and 7 are misses: in event 4, mildly negative tone was followed by a small positive return, and in event 7, mildly positive tone was followed by a small negative return, both events where the tone score was close to zero and therefore only weakly informative, a reminder that a sentiment signal is genuinely noisy, not a deterministic predictor. Comparing the two strategies (Sharpe-like ratio: mean divided by standard deviation, with no annualization applied here),

$$\begin{aligned} \text{Signal-driven strategy:} &\quad \text{mean} = 0.775\%,\ \text{std} = 0.8481\%,\ \text{Sharpe-like} = 0.9138, \\ \text{Buy-and-hold:} &\quad \text{mean} = 0.075\%,\ \text{std} = 1.1829\%,\ \text{Sharpe-like} = 0.0634. \end{aligned}$$

The signal adds real value on this small sample, both a higher average return and a more consistent one, even though it is wrong a quarter of the time. Turning this signal into an actual executed position, at the right size and without excessive market impact, is the execution problem Chapter 13 developed in detail.

The tone score above is a text-based sentiment measure, extracted from language rather than prices, which is worth placing alongside a second, price-based measure this book has already introduced: the CBOE Volatility Index (VIX) from Chapter 5, often called the market's "fear gauge," which aggregates option prices into a single, forward-looking measure of expected market-wide volatility without reading a single word of text. That nickname deserves some care before the two are treated as the same kind of quantity: VIX is mostly a forecast of how much the index will move, and only partly a reading of how investors feel about it, because an option's price compensates both for the volatility expected and for the discomfort of bearing it. It is that second, risk-compensation component that behaves like a sentiment score, while the headline index blends the two together. The two measures are complementary rather than redundant: a text-based tone score like the one above can capture sentiment specific to a single company's earnings call, information a broad market index like VIX cannot see at all, while VIX can move sharply in response to macroeconomic or geopolitical developments that never mention any single company by name and that a firm-specific sentiment model was never trained to detect. A more sophisticated event-driven strategy often conditions on both at

Wulin.Suo@Queensu.ca

once: using VIX as a market-wide risk-regime signal, exactly as Chapter 13 describes, alongside a firm-specific, text-derived tone score like the one developed in this section, since a positive earnings-call tone score is a genuinely different trading proposition depending on whether it arrives during a calm market or one already gripped by elevated, VIX-signaled uncertainty.

16.3.2 Naive Bayes and Document Classification

Suppose a compliance team needs to sort thousands of sentences drawn from a 10-K's risk-factors section into "Credit Risk" or "Market Risk" categories, a task too large to do by hand every filing season but too nuanced for a simple keyword rule to get right on its own. Chapter 6 introduced Naive Bayes as a generative classifier that estimates

$$\Pr(y = k \mid x_1, \ldots, x_p) \propto \Pr(y = k) \prod_{j=1}^{p} \Pr(x_j \mid y = k)$$

and noted its particular suitability for text, where features are word counts and the conditional-independence assumption, while not literally true, distorts the result less than it might for correlated financial ratios. This section applies that model to this document classification task, using only the counts of six vocabulary words (default, borrower, counterparty, volatility, rate, currency).

Table 16.4 shows the six training sentences verbatim, three representative of each class, each written in the same style as an actual 10-K risk-factors disclosure.

Sentence	Label
"A default by a major borrower or counterparty could trigger a chain of default across our loan portfolio."	Credit Risk
"Our borrower base includes several highly leveraged parties, and a default by any significant borrower could impair our credit portfolio."	Credit Risk
"We are exposed to counterparty risk if a significant counterparty were to default on its obligations under our agreements."	Credit Risk
"Significant volatility in interest rate markets, combined with broader market volatility, could adversely affect the value of our trading positions."	Market Risk
"Changes in the benchmark rate or the market rate environment, combined with volatility in trading conditions, could affect our net interest income."	Market Risk
"Fluctuations in the currency exchange market and adverse currency movements, together with changes in the base lending rate, could reduce our reported earnings."	Market Risk

Table 16.4: Training sentences for the 10-K risk-factor Naive Bayes classifier

Counting runs over the six vocabulary terms only, not over every word in the sentences, so the class total below is a vocabulary-token count rather than a sentence length. Credit Risk contributes `default` four times, `borrower` three, and `counterparty` three, with the three market terms absent, giving $N_{\text{Credit}} = 10$; Market Risk contributes `rate` four times, `volatility` three, and `currency` twice, with the three credit terms absent, giving $N_{\text{Market}} = 9$. Chapter 6's Section 6.6.2 Laplace (add-one) smoothing correction then divides by $N_c + V$ with vocabulary size $V = 6$, that is by 16 for Credit Risk and 15 for Market Risk, giving the

Wulin.Suo@Queensu.ca

estimated word probabilities

$$\begin{aligned}
\Pr(\text{default} \mid \text{Credit}) &= 0.3125,\\
\Pr(\text{borrower} \mid \text{Credit}) = \Pr(\text{counterparty} \mid \text{Credit}) &= 0.25,\\
\Pr(\text{volatility} \mid \text{Credit}) = \Pr(\text{rate} \mid \text{Credit}) = \Pr(\text{currency} \mid \text{Credit}) &= 0.0625,\\
\Pr(\text{rate} \mid \text{Market}) &= 0.3333,\\
\Pr(\text{volatility} \mid \text{Market}) &= 0.2667,\\
\Pr(\text{currency} \mid \text{Market}) &= 0.20,\\
\Pr(\text{default} \mid \text{Market}) = \Pr(\text{borrower} \mid \text{Market}) = \Pr(\text{counterparty} \mid \text{Market}) &= 0.0667,
\end{aligned}$$

reflecting each class's training vocabulary.

A Worked Example: Classifying an Ambiguous Sentence by Hand

Consider the held-out test sentence "A default by a key counterparty during a period of heightened market volatility could be exacerbated by continued volatility in broader financial markets," with word counts (default: 1, counterparty: 1, volatility: 2, all others: 0), a genuinely ambiguous case that mentions both a credit-risk term and, twice, a market-risk term. With equal class priors $\Pr(\text{Credit}) = \Pr(\text{Market}) = 0.5$, the (unnormalized) class scores are

$$\begin{aligned}
\text{score}(\text{Credit}) &= 0.5 \times 0.3125^1 \times 0.25^1 \times 0.0625^2 = 0.00015259,\\
\text{score}(\text{Market}) &= 0.5 \times 0.0667^1 \times 0.0667^1 \times 0.2667^2 = 0.00015802.
\end{aligned}$$

Normalizing gives $\Pr(\text{Credit} \mid \text{doc}) \approx 0.4912$ and $\Pr(\text{Market} \mid \text{doc}) \approx 0.5088$: the model predicts Market Risk, narrowly and incorrectly, since this sentence's true label is Credit Risk. This is the kind of near-toss-up case where the independence assumption's imprecision matters most, the two repeated "volatility" mentions tip a genuinely mixed-signal sentence to the wrong side by less than two percentage points of posterior probability.

Applying the same model to a test set of six sentences (three of each true class) produces the confusion matrix in Table 16.5.

	Predicted Credit	Predicted Market
Actual Credit	2 (TP)	1 (FN)
Actual Market	0 (FP)	3 (TN)

Table 16.5: Confusion matrix for the 10-K risk-factor Naive Bayes classifier (positive class: Credit Risk)

Precision is $2/2 = 1.00$, recall is $2/3 \approx 0.6667$, and $F_1 \approx 0.80$: every sentence the model labeled Credit Risk really was Credit Risk, but it missed one genuine Credit Risk sentence, the ambiguous one worked through above, by classifying it as Market Risk instead.

16.3.3 Word Embeddings: From Sparse Counts to Dense Vectors

Bag-of-words and TF-IDF vectors are sparse and high-dimensional: with a vocabulary of tens of thousands of words, most entries in any document's vector are zero, and the representation has no notion that "stock" and "equity" are related concepts while "stock" and "bond" are not, every distinct word is just a separate, unrelated dimension. An embedding is this: a fixed-length, continuous-valued vector standing in for a discrete object, here a word, chosen so that geometric closeness in the vector space reflects semantic closeness in meaning. Word embedding methods (Word2Vec and GloVe are the classical examples) learn such a dense, low-dimensional vector for each word from patterns of co-occurrence in a large text corpus, such that words used in similar contexts end up with similar vectors. Cosine similarity,

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \, \|v\|}$$

Wulin.Suo@Queensu.ca

is the standard way to measure how similar two embedding vectors are.

A Worked Example: Cosine Similarity of Toy Embeddings

To illustrate the geometric idea without training an actual embedding model, assign six financial terms small hand-chosen two-dimensional vectors:

$$\begin{aligned} \text{stock} &= (0.90, 0.20), & \text{equity} &= (0.85, 0.25), \\ \text{bond} &= (0.10, 0.90), & \text{debt} &= (0.15, 0.85), \\ \text{bullish} &= (0.80, -0.30), & \text{bearish} &= (-0.75, -0.35). \end{aligned}$$

Computing cosine similarity between related pairs gives $\cos(\text{stock}, \text{equity}) \approx 0.9977$ and $\cos(\text{bond}, \text{debt}) \approx 0.9980$, both close to 1, reflecting that these vectors point in nearly the same direction. Unrelated or opposed pairs behave very differently: $\cos(\text{stock}, \text{bond}) \approx 0.3234$ is only weakly similar, and $\cos(\text{bullish}, \text{bearish}) \approx -0.7000$ is strongly negative, capturing that the two terms are near-opposites. In a real embedding trained on financial text, this same geometric structure, synonyms and related concepts clustering together, opposites pointing in different directions, emerges from word co-occurrence patterns alone, without ever being told what any word means.

A second, even more striking property of well-trained embeddings is that vector *arithmetic* on these representations can capture relationships between words, not just similarity, and this toy example was constructed so that the classic "analogy" relation holds exactly. Computing stock − equity + bond,

$$(0.90, 0.20) - (0.85, 0.25) + (0.10, 0.90) = (0.15, 0.85) = \text{debt},$$

reproduces the debt vector exactly, because stock and equity, and separately bond and debt, were placed as near-synonym pairs displaced from one another by the identical offset, $(0.05, -0.05)$. In a real, trained embedding space, this same kind of arithmetic famously (if imperfectly) captures relationships like "king − man + woman ≈ queen," and in a financial embedding space, an analogous relationship might hold between a company and its industry-standard debt instrument, or between a country and its currency, allowing an analyst to query the embedding space with relationships rather than only single-word similarity. Real embeddings only approximate this property, and the approximation is noticeably weaker for rarer or more abstract financial terms than for common, concrete ones, but the toy example here isolates the geometric idea cleanly before it gets obscured by the noise of a real, high-dimensional trained model.

16.4 Transformers and Large Language Models

Attention changed what a language model can represent, and this part develops it from the mechanism up to retrieval and prompting.

16.4.1 Attention and the Transformer Architecture

Embeddings give each word a fixed vector regardless of context, but the same word can mean different things in different sentences ("the Fed will likely raise rates" versus "the flood raised water levels"). Transformer models address this with a self-attention mechanism. Each token's embedding x is linearly projected, using three weight matrices W^Q, W^K, W^V that are learned during training and shared across every token in the sequence, into a query vector $Q = xW^Q$, a key vector $K = xW^K$, and a value vector $V = xW^V$; a token does not "produce" these vectors from nowhere, it is simply the same embedding viewed through three different learned lenses. Intuitively, a token's query asks "what am I looking for," its key advertises "what I have to offer if some other token is looking for this," and its value is the actual information it contributes once attention has decided to focus on it, a search query, an index entry, and a payload, respectively.

A token's contextual representation is then a weighted combination of every token's value vector, with weights determined by how well each token's key matches the query,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \qquad \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}},$$

Wulin.Suo@Queensu.ca

where d_k is the dimension of the key vectors, included to keep the dot products from growing too large as d_k increases, and softmax rescales a vector of raw scores into positive weights that sum to exactly one, so the weighted combination of value vectors that follows is a genuine, interpretable average rather than an arbitrary weighted sum.

This single mechanism, applied across many layers and attention "heads," is the core computational building block inside BERT, GPT, and every other modern transformer-based language model. Figure 16.2 lays out this computation as a diagram, the same three-projection, one-combination structure every attention layer repeats.

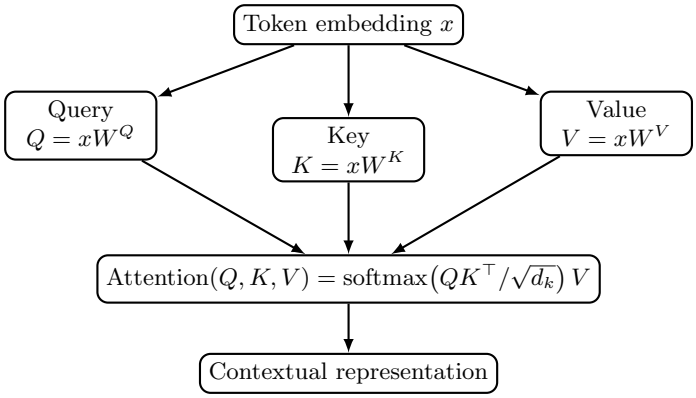


Figure 16.2: The self-attention computation as a diagram

A Worked Example: Self-Attention for a Three-Token Sentence

Consider the toy sentence "stocks rallied today," with $d_k = 2$. Rather than training actual projection matrices W^Q, W^K, W^V, this example hand-chooses the resulting key and value vectors directly, standing in for whatever a trained model would have produced: $K_{\text{stocks}} = (1, 0)$, $K_{\text{rallied}} = (0, 1)$, $K_{\text{today}} = (1, 1)$, and $V_{\text{stocks}} = (2, 0)$, $V_{\text{rallied}} = (0, 2)$, $V_{\text{today}} = (1, 1)$. "stocks" and "today" were deliberately given the same first key component so that any query matching strongly on that component attends to both of them equally, exactly the effect this example is built to illustrate.

Computing the contextual representation of "stocks" uses its query $Q_{\text{stocks}} = (1, 0)$. The raw scores are $Q \cdot K_i/\sqrt{2}$: 0.7071 against "stocks," 0 against "rallied," and 0.7071 against "today," since "stocks" and "today" share a key component in the first dimension while "rallied"s key is orthogonal to the query. Applying the softmax formula above to these three scores gives attention weights of 0.4011, 0.1978, and 0.4011 respectively, which do sum to 1 exactly as softmax guarantees, and the weighted sum of value vectors is

$$0.4011\,(2, 0) + 0.1978\,(0, 2) + 0.4011\,(1, 1) = (1.2033,\ 0.7967).$$

Figure 16.3 draws these three attention weights directly, with edge thickness proportional to how much each token contributes. The token "today," despite being a different word, contributes exactly as much to "stocks"'s new representation as "stocks" contributes to itself, because their key vectors happen to align with the query, while "rallied" contributes far less, its value $(0, 2)$ pulled in only weakly.

This is the mechanism that lets a transformer build a contextual representation of "raise" that differs depending on whether nearby tokens are "rates" or "water levels": a trained W^K would give "rates" and "raise" well-aligned keys in a financial context and "water" and "raise" well-aligned keys in an environmental one, precisely the way this toy example's keys were aligned by construction rather than learned from data.

Multi-Head Attention and Positional Encoding

A single attention calculation forces every token to distribute its focus according to one shared notion of relevance, but relevance in a real sentence is not one-dimensional: local grammatical structure (which word modifies which noun) and long-range dependency (which earlier passage a later reference actually points

Wulin.Suo@Queensu.ca

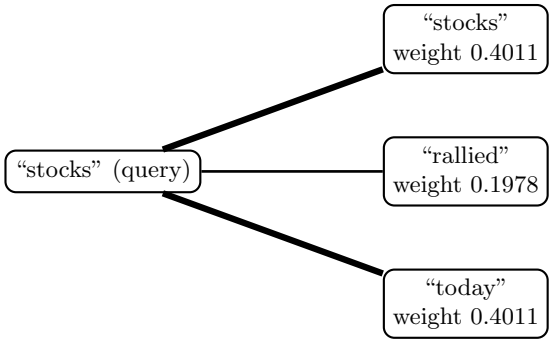


Figure 16.3: Self-attention weights for the query token "stocks" over the three-token sentence "stocks rallied today," with edge thickness proportional to attention weight

back to) are two genuinely different kinds of relationship, and a single attention pattern has no way to represent both well at once. Two refinements turn this single calculation into the mechanism actually used in practice. First, a real transformer layer computes several attention calculations in parallel, each with its own learned Q, K, V projections, called attention heads, then concatenates their outputs. One head might learn to attend primarily to nearby tokens (useful for local syntax), another to the subject of a distant verb (useful for long-range dependencies such as connecting a hedge's description early in a paragraph to a number reported much later), and the model combines all of these perspectives rather than relying on a single attention pattern. Second, the attention calculation above treats the input as an unordered set: nothing in $Q \cdot K$ depends on token position, so "the bank raised rates" and "rates raised the bank" would receive identical attention weights if given identical token embeddings. Positional encoding (PE) fixes this by adding a position-dependent vector to each token's embedding before attention is computed, most commonly

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

and

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

for position pos and embedding dimension index i, so that word order, essential to meaning in any language, survives into a mechanism that would otherwise discard it entirely.

Encoder-Only, Decoder-Only, and Encoder-Decoder Architectures

The self-attention mechanism above is the shared building block behind every transformer-based model, but different financial NLP tasks call for arranging that block differently, and the resulting three architecture families are not interchangeable. An encoder-only model (BERT and its financial variant FinBERT are the standard examples) lets every token attend bidirectionally to every other token, before and after it in the sequence, the unmasked computation in the worked example above, and is trained with a masked-language-modeling objective (predicting a randomly hidden word from its full surrounding context in both directions). This bidirectional view makes encoder-only models well suited to classification and embedding tasks, such as the sentiment scoring of Section 16.2.3 and the document classification of Section 16.3.2, where the full context of a passage is available at once and the task is to produce a single label or vector, not to generate new text. A decoder-only model (the GPT family, and the LLMs described throughout the rest of this chapter) instead restricts each token to attending only to itself and earlier tokens, a causal mask, since it is trained to predict each next token from only what precedes it, the generative, one-token-at-a-time process Section 16.4.2 describes. An encoder-decoder model (T5-style architectures) uses both: a bidirectional encoder processes the full input, and a causal decoder generates the output one token at a time while cross-attending to the encoder's output, a natural fit for tasks with a clear input-to-output transformation, such as translating a filing into another language or condensing a full earnings call into a short summary.

Wulin.Suo@Queensu.ca

A Worked Example: How a Causal Mask Changes a Token's Representation

Return to the "stocks rallied today" example, now computing the contextual representation of "rallied" using its own query $Q_{\text{rallied}} = (0, 1)$, first with full bidirectional attention (as an encoder-only model would compute it) and then with a causal mask that hides "today" from "rallied," since "today" comes after it in the sequence (as a decoder-only model would compute it). Bidirectionally, the raw scores $Q_{\text{rallied}} \cdot K_i/\sqrt{2}$ are 0 against "stocks," 0.7071 against "rallied," and 0.7071 against "today," giving softmax weights of 0.1978, 0.4011, and 0.4011 and an output of

$$0.1978\,(2, 0) + 0.4011\,(0, 2) + 0.4011\,(1, 1) = (0.7967,\ 1.2033).$$

Under a causal mask, "today" is simply removed from consideration entirely, leaving only the scores against "stocks" (0) and "rallied" (0.7071); renormalizing these two alone via softmax gives weights of 0.3302 and 0.6698, and an output of

$$0.3302\,(2, 0) + 0.6698\,(0, 2) = (0.6605,\ 1.3395).$$

The two representations of the identical word "rallied," in the identical sentence, differ, $(0.7967, 1.2033)$ versus $(0.6605, 1.3395)$, purely as a consequence of architecture choice: an encoder-only model's representation of "rallied" is allowed to reflect that "today" follows it, while a decoder-only model's representation, used to predict what comes *after* "rallied," cannot be allowed to peek at "today" at all without trivializing its own training objective.

This is precisely why swapping a decoder-only LLM in for a task an encoder-only model was designed for, or vice versa, is not merely a matter of using "a big enough model": the two produce systematically different token representations by construction, not merely by training data.

How These Models Are Trained, and Why They Eventually Need Retraining

No one hand-writes the rule that lets a model correctly guess which distant word a pronoun refers to, or which number a sentence's context makes most plausible; a model this large has to learn it, and the only training signal available at the necessary scale is the text itself, with no human labels attached. None of the weight matrices in Figure 16.2, W^Q, W^K, W^V, or the many more inside a full transformer's feedforward and output layers, are hand-specified; every one of them is learned by ordinary gradient descent (Chapter 6's Section 6.3.2), applied not to a small labeled dataset but to a self-supervised objective computed directly from raw, unlabeled text at enormous scale.

An encoder-only model is pretrained by masking a random fraction of tokens in each training sentence and learning to predict the missing token from its surrounding bidirectional context; a decoder-only model is pretrained on the simpler, single objective of predicting each next token from everything before it, applied repeatedly across a very large corpus of general text, the causal-masked computation the worked example above just carried out by hand. This is what "training" actually means for a language model: no human labels the correct output token by token, the text itself supplies the training signal, which is precisely why these models can be pretrained on a vastly larger volume of data than any supervised model in this book has needed.

Pretraining is not, however, a one-time event a financial institution can complete and then ignore indefinitely. Financial language itself drifts: new instruments, regulations, tickers, and company names appear continuously, established terms occasionally acquire new meanings (a "pandemic bond" meant nothing before 2020), and the vocabulary a sentiment or classification model was pretrained on can gradually stop matching the vocabulary a live system actually encounters, a text-domain analogue of the concept drift Chapter 14's Section 14.7.2 and Chapter 18's population-stability-index discussion already raised for numeric models. Three responses are common in practice, roughly in increasing order of cost: prompting or retrieval (Section 16.4.3) can supply current information to an otherwise-frozen model without touching its weights at all; parameter-efficient fine-tuning updates a small number of additional parameters on a modest, domain-specific dataset without repeating full pretraining; and continued pretraining resumes the original self-supervised objective on fresh text, updating the model's weights more thoroughly at proportionally higher computational cost. Which response is appropriate depends on whether the problem is a knowledge gap (the model simply lacks a recent fact, best fixed by retrieval) or a genuine shift in how language itself is used (better fixed by continued pretraining or fine-tuning), a diagnosis that itself requires the same kind of

Wulin.Suo@Queensu.ca

ongoing monitoring Chapter 18's model-governance discussion already established for every other model in this book.

16.4.2 Large Language Models in Finance: Capabilities, Generative Modeling, and Limits

Suppose an analyst is handed a hundred-page merger filing an hour before a decision is due, or a portfolio manager wants a plain-English answer to a specific question buried somewhere inside it, tasks none of this chapter's classification or scoring tools, built to output a single label or score, can actually perform. Chapter 6 noted that large language models are far more elaborate examples of the same generative-modeling principle behind Naive Bayes: both learn a probability distribution, Naive Bayes over word counts given a class, an LLM over the next token given all preceding tokens, and both can be used generatively, Naive Bayes to characterize what a document of a given class looks like, an LLM to produce entire new documents. An LLM is, mechanically, a very large stack of the self-attention layers developed in Section 16.4.1, trained on enormous volumes of text to predict the next token, then further adapted (through instruction tuning and related techniques) to follow natural-language prompts.

In finance, this generative capability supports tasks well beyond the classification and scoring problems earlier sections addressed: summarizing a lengthy earnings call into a short brief, extracting a structured table of figures from an unstructured filing, drafting a first pass at a research note, or answering a natural-language question about a specific document. These are qualitatively different tasks from the fixed-output classification and scoring problems this book has otherwise emphasized, and they come with a correspondingly different failure mode: hallucination, an LLM's tendency to generate fluent, confident-sounding text that is factually wrong, a specific number invented rather than retrieved, a filing detail misattributed, a summary that omits a covenant violation buried in a footnote.

A classification model's failure mode is a wrong label with a computable error rate; an LLM's characteristic failure mode is a wrong but plausible-sounding answer, which is harder to detect automatically and more dangerous in a domain where a single invented figure in a research note or a summarized covenant can be individually consequential. The next four sections develop, in turn, the leading mitigation for hallucination (retrieval-augmented generation), how to instruct a generative model effectively (prompting), a second, distinct use of generative modeling that has nothing to do with text (synthetic data generation), and how a financial institution governs all of this in practice.

Beyond Text: Code Generation and Multimodal Applications

Two further capabilities extend an LLM's usefulness beyond the purely text-in, text-out tasks described so far. Code generation applies the same next-token prediction principle to programming languages rather than natural language. An analyst can describe a desired calculation in plain English, for example "load this CSV of daily returns and compute the annualized Sharpe ratio and maximum drawdown," and receive working Python code implementing it, directly extending the pandas- and NumPy-based analysis this book has built since Chapter 2. The generated code still needs the same scrutiny any code does before it runs against real portfolio data. A subtly wrong annualization factor or an off-by-one window is a bug like any other, not a hallucination in the text sense, but no less consequential.

Multimodal models extend the same architecture to inputs beyond plain text, reading a scanned PDF page, a chart embedded in an investor presentation, or a table image directly, and answering questions or extracting figures from it without a separate, brittle optical-character-recognition step, which matters in practice since a great deal of financial disclosure (annual report exhibits, scanned older filings, chart-heavy investor decks) is not available as clean machine-readable text to begin with.

16.4.3 Retrieval-Augmented Generation: A Worked Example

Retrieval-augmented generation (RAG) is the most widely deployed mitigation for hallucination in financial applications. Rather than relying solely on facts an LLM absorbed during training, potentially outdated, generic, or simply never seen, a RAG system first retrieves the most relevant passages from a trusted, current

Wulin.Suo@Queensu.ca

document collection (a bank's own filings, policy manuals, or licensed research), then conditions the model's generated answer on those specific retrieved passages.

This converts an open-ended generation task into something closer to an open-book comprehension task: the model is asked to answer using the retrieved text, not to recall facts unaided, and its output can be traced back to a specific source passage for verification, exactly what an auditor or compliance reviewer needs. The retrieval step is a direct application of Section 16.3.3's embedding geometry: embed the user's query and every candidate document in the same vector space, then rank documents by cosine similarity to the query and pass the top-ranked passages to the generative model.

A Worked Example: Retrieving the Right Passage from a Toy Knowledge Base

Consider a knowledge base of four short passages about two companies, embedded using the same hand-illustrative three-dimensional scheme as before, dimensions loosely representing revenue-topic, leverage-topic, and other-company-topic, shown in Table 16.6.

Document	Passage	Embedding
Doc1	"Company X reported Q3 revenue of $420 million, up 8% year over year."	$(0.90, 0.10, 0.00)$
Doc2	"Company X's debt-to-equity ratio increased to 1.8 in Q3."	$(0.10, 0.90, 0.00)$
Doc3	"Company Y announced a new product launch in Q4."	$(0.00, 0.00, 0.90)$
Doc4	"Company X's Q3 net margin improved to 12.5% from 10.1%."	$(0.60, 0.50, 0.00)$

Table 16.6: Toy knowledge base for the retrieval-augmented generation worked example

A user asks, "What was Company X's revenue in Q3?", embedded as $(0.95, 0.05, 0.00)$. Computing cosine similarity between the query and each document gives

$$\begin{aligned}\cos(\text{query}, \text{Doc1}) &\approx 0.9983,\\ \cos(\text{query}, \text{Doc4}) &\approx 0.8008,\\ \cos(\text{query}, \text{Doc2}) &\approx 0.1625,\\ \cos(\text{query}, \text{Doc3}) &= 0.0000.\end{aligned}$$

Doc1 is retrieved as the top match by a wide margin, and only Doc1 (or, at a higher retrieval count, Doc1 and Doc4) would be passed to the generative model. The model then answers using Doc1's actual text, producing "Company X's Q3 revenue was $420 million, up 8% year over year," grounded in a retrieved, verifiable source, rather than a plausible-sounding figure invented from the model's training data alone. Note that Doc2 and Doc3, despite both mentioning Company X or a similar business topic, are correctly ranked far below Doc1 and Doc4, since neither actually answers a revenue question, the discrimination a well-behaved retrieval step needs to make.

Evaluating a RAG System: Precision@k and a Faithfulness Score

Retrieving *a* document is not the same as retrieving the *right* document, and a production RAG system needs a quantitative way to check both the retrieval step and the generation step separately, rather than only spot-checking final answers by hand. Precision@k measures the retrieval step alone: of the top k retrieved documents, what fraction are actually relevant to the query? For the revenue question above, only Doc1 genuinely answers it; Doc4 (net margin) and Doc2 (leverage) are topically related to Company X but do not answer *this* question. Ranking documents by the cosine similarities already computed (Doc1, then Doc4, then Doc2, then Doc3),

$$\text{Precision@1} = \frac{1}{1} = 100\%, \qquad \text{Precision@2} = \frac{1}{2} = 50\%, \qquad \text{Precision@3} = \frac{1}{3} \approx 33.3\%,$$

Wulin.Suo@Queensu.ca

since only one of the top-1, one of the top-2, and one of the top-3 retrieved documents is actually relevant. This is a direct, quantitative reason to keep k small and the similarity threshold strict: retrieving more documents does not just add noise, it mechanically dilutes precision, which matters because every additional, non-relevant passage passed to the generative model is an additional opportunity for it to draw on irrelevant context when composing its answer.

A second, distinct check addresses the generation step: given that the right passage was retrieved, did the model's generated answer actually stick to it? A faithfulness score compares the generated answer's own embedding to the retrieved source passage's embedding, using the same cosine-similarity geometry as retrieval itself. The grounded answer from above, "Company X's Q3 revenue was $420 million, up 8% year over year," embeds essentially identically to Doc1, at $(0.89, 0.12, 0.00)$, giving

$$\text{Faithfulness} = \cos\big((0.89, 0.12, 0.00),\ \text{Doc1}\big) \approx 0.9997,$$

close to a perfect match, since the answer is simply restating Doc1's content. Contrast this with a hallucinated alternative answer that invents an unrelated leverage-related claim never actually stated in Doc1, embedding instead at $(0.30, 0.85, 0.00)$, closer to Doc2's topic than Doc1's:

$$\text{Faithfulness} = \cos\big((0.30, 0.85, 0.00),\ \text{Doc1}\big) \approx 0.4349,$$

far below the grounded answer's score. A production system can use this kind of threshold, flagging any generated answer whose score against its retrieved source falls below, say, 0.85, for automatic rejection or human review. That makes the score a useful *topical-drift* check: it catches an answer that has wandered off to a different subject.

It does not catch the error that matters most in finance, a wrong number on the right topic. Suppose the model answers "Company X's Q3 revenue was $240 million, down 8% year over year," transposing two digits and reversing the direction. The answer is about exactly the same subject as Doc1, so in this chapter's topic-based embedding it sits exactly where the grounded answer sits and scores the same 0.9997, sailing through the 0.85 threshold. Real embedding models behave the same way in kind, because they encode what a sentence is about far more strongly than the particular digits it contains. Even plain word-count vectors built from the actual sentences, the bag-of-words representation of Section 16.2.2, give the wrong answer a cosine of 0.7333 against Doc1, against 0.8667 for the grounded one: lower, but nowhere near as low as the leverage answer's 0.4349, and a threshold strict enough to reject it would also start rejecting correct answers that are merely phrased differently.

Faithfulness therefore needs a second, *claim-level* check. Extract every verifiable claim from the answer, its numbers, entities, dates, and directions of change, and require each one to appear in, or be computable from, the retrieved source. On this example the check is mechanical: the grounded answer's three claims (420, 8, "up") are all found in Doc1, while the wrong answer's (240, 8, "down") pass on only one of three and the answer is rejected. Production systems extend the same idea beyond exact matching, by asking a natural-language-inference (entailment) model whether the source entails each extracted claim, or by asking a second LLM to judge each claim against the source ("LLM-as-judge"), and evaluation frameworks such as RAGAS package these claim-level faithfulness scores alongside retrieval metrics like precision@k. Together, the two checks give the "sample many outputs and check them against source documents for factual grounding" validation practice Section 16.5.2 calls for a concrete, computable form rather than only a qualitative principle.

16.4.4 Prompting Strategies for Financial Tasks

How a task is phrased to a generative model, its prompt, materially affects output quality, and three prompting strategies recur across financial applications. Zero-shot prompting simply states the task: "Classify the sentiment of the following earnings-call excerpt as positive, negative, or neutral: *[excerpt]*." This works reasonably well for common, well-defined tasks but can be inconsistent for tasks with domain-specific conventions the model has not reliably learned, the kind of finance-specific vocabulary issue Section 16.2.1 raised.

Few-shot prompting adds a small number of worked examples directly in the prompt before the actual task, showing the model the specific labeling convention wanted: two or three (excerpt, label) pairs, drawn

Wulin.Suo@Queensu.ca

perhaps from Section 16.2.3's transcripts, followed by a new, unlabeled excerpt to classify. This is analogous to Chapter 6's supervised learning setup, except the "training" happens entirely within a single prompt, with no weight updates to the model at all, and the "training set" can be as small as two or three examples.

Chain-of-thought prompting asks the model to reason step by step before producing a final answer, rather than jumping directly to a conclusion, for example: "First identify the reported revenue figure and the prior-year figure, then compute the percentage change, then state whether this represents an acceleration or deceleration relative to the previous quarter's growth rate, showing each step." For multi-step numerical tasks, computing a financial ratio from figures buried in a paragraph, or reconciling two differently defined metrics, this measurably reduces errors relative to asking for the final number directly. It forces the model's intermediate steps into the output where they can be checked. That is much the same spirit as this book's own insistence, established in Chapter 7 and followed throughout, on showing every intermediate step of a calculation rather than only its final result.

16.5 Generative AI in Practice

Generation raises problems consumption does not, from synthetic data to governance to domain adaptation.

16.5.1 Generative Models for Synthetic Financial Data

A second, quite different application of generative modeling has nothing to do with text: generating entirely new, synthetic numerical data, additional stress-test scenarios, synthetic order-book sequences, or extra examples of a rare event class, that resembles real data closely enough to be useful for training or testing other models. Two architectures dominate this use case.

A generative adversarial network (GAN) pits two neural networks against each other: a generator G that maps random noise z, drawn from a simple fixed distribution p_z such as a standard normal, to synthetic samples $G(z)$, and a discriminator D that tries to distinguish real samples x, drawn from the true data distribution p_{data} the GAN is learning to imitate, from generated ones, trained jointly via

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}\left[\log\left(1 - D(G(z))\right)\right].$$

Each term rewards D for one half of its job: the first for assigning a high probability to samples that are genuinely real, the second for assigning a low one to the generator's fakes. Only the second term involves G at all, which is why the generator's own objective is to drive $D(G(z))$ upward against a discriminator simultaneously trying to push it down. As training proceeds, the generator improves at fooling the discriminator and the discriminator improves at catching it, and at convergence the generator produces samples the discriminator can no longer reliably distinguish from real data. In finance, this has been applied to augmenting a fraud detection model's minority class, extending Chapter 15's discussion of resampling techniques such as SMOTE with entirely synthetic, rather than interpolated, minority-class examples, and to generating synthetic limit-order-book sequences for backtesting execution algorithms without exposing real client order flow.

The companion notebook evaluates this minimax game concretely, reusing Chapter 15's fraud-scoring data: a fixed, illustrative discriminator $D(x) = \sigma(2.0x - 5.0)$ assigns transaction 1's real fraud amount, standardized to 3.1, a probability of being real of $D(x_{\text{real}}) = 0.7685$, fairly confident. (That standardized amount is the same feature Chapter 15 scores, named there with the conventional z of a standard score; here z is reserved for the generator's noise input, so this section calls the feature by its role rather than by that letter.) An early-training generator, still producing unrealistic samples close to a typical legitimate transaction's scale, might output $G(z) = 1.0$, which the discriminator correctly flags as almost certainly fake, $D(G(z)) = 0.0474$, a wide gap of 0.7211 from $D(x_{\text{real}})$.

A later-training generator that has learned to produce a more realistic, fraud-scale sample, $G(z) = 2.9$, is assigned $D(G(z)) = 0.6900$, closing the gap to just 0.0786, the "generator improves at fooling the discriminator" dynamic described above, made concrete: the generator's loss, $-\log D(G(z))$, falls from 3.0486 at the early stage to 0.3711 at the later stage, a direct numerical measure of how much harder the generator has become to catch.

Wulin.Suo@Queensu.ca

GANs are also well known for being difficult to train in practice. Because the generator and discriminator are optimized simultaneously against each other rather than toward a single, shared objective, training can fail to converge, oscillating rather than settling into an equilibrium, or collapse into mode collapse, where the generator discovers a small number of samples that reliably fool the discriminator and stops exploring the rest of the data distribution, producing synthetic fraud examples or stress scenarios that are realistic but insufficiently diverse to be useful for the very purpose, augmenting a thin dataset, they were generated for. This instability, not a limitation shared by every generative architecture, is one reason diffusion models, described next, have become the more common choice for new synthetic-data applications despite being more computationally expensive to sample from.

Diffusion models take a different approach, motivated by physical diffusion processes: a forward process gradually adds noise to real data over many steps until it becomes indistinguishable from pure noise, $x_t = \sqrt{\alpha_t}\, x_0 + \sqrt{1-\alpha_t}\, \epsilon$, where x_0 is the original data point, ϵ is standard-normal noise, and α_t decreases from near 1 (step 0, almost no noise) toward 0 (fully noised) as t increases. A neural network is then trained to reverse this process, predicting and removing noise one step at a time, so that starting from pure random noise and running the learned reverse process produces an entirely new, synthetic sample resembling the original data distribution.

A Worked Example: One Forward Diffusion Step

Consider a toy stress-test scenario variable, a multiplier applied to a baseline loss, with baseline (undiffused) value $x_0 = 1.00$ (no stress) and a noise draw $\epsilon = -0.50$. At an early diffusion step with $\alpha_t = 0.80$,

$$x_t = \sqrt{0.80}\,(1.00) + \sqrt{0.20}\,(-0.50) = 0.6708.$$

At a later, noisier step with $\alpha_t = 0.55$,

$$x_t = \sqrt{0.55}\,(1.00) + \sqrt{0.45}\,(-0.50) = 0.4062.$$

As α_t falls from 0.80 to 0.55, the scenario value moves from 0.6708 down to 0.4062, drifting away from the deterministic baseline of 1.00 and toward the random noise draw of -0.50; continuing this process to $\alpha_t \to 0$ would carry x_t all the way to ϵ itself, pure noise with no remaining trace of the original scenario. Note that the two coefficients are square roots chosen so that $\left(\sqrt{\alpha_t}\right)^2 + \left(\sqrt{1-\alpha_t}\right)^2 = 1$: they hold the *variance* of x_t constant as noise replaces signal, and they are deliberately not a weighted average of x_0 and ϵ, since they sum to more than one. A same-signed noise draw can therefore push x_t temporarily outside the range spanned by x_0 and ϵ before it settles toward ϵ. A trained diffusion model learns to run this process in reverse, starting from noise and reconstructing a plausible, never-before-observed stress scenario, one that Chapter 8's stress-testing discussion could use to probe portfolio losses under conditions beyond anything in the historical sample.

16.5.2 Governance and Risk Management for Generative AI

Chapter 15 developed a three-lines-of-defense governance framework for fraud and compliance models; generative AI systems need the same discipline, applied to a set of risks classical, deterministic models did not raise. Three deserve particular attention.

First, non-determinism: the same prompt submitted twice can produce two different outputs, which complicates traditional model validation approaches that assume a fixed model produces a fixed, reproducible output for a fixed input, and pushes validation toward statistical sampling of many outputs rather than checking a single deterministic result. Second, prompt injection: adversarial text embedded in a retrieved document, an uploaded file, or even ordinary user input can attempt to hijack a generative system's behavior, for example a malicious instruction hidden inside a retrieved filing that reads, to the model, as an instruction to ignore its original task, an attack surface with no real counterpart in the classification models earlier chapters developed. Third, data leakage: a RAG system with access to sensitive internal documents can, if not carefully scoped, surface information to a user who should not see it, turning a helpful retrieval feature into an inadvertent access-control failure.

These risks change how the third line's independent validation function, introduced in Chapter 15, must operate: rather than checking a model's predictions against a fixed, labeled test set, validating a generative

Wulin.Suo@Queensu.ca

system requires sampling many outputs, checking them against source documents for factual grounding (the kind of check Section 16.4.3's retrieval step is meant to support), and explicitly testing for prompt-injection and data-leakage vulnerabilities before deployment, not just at initial launch but on an ongoing basis as the underlying model, retrieved document collection, or user population changes. Regulators have generally extended existing model risk management guidance, rather than writing entirely new rules, to cover these systems, which means a generative AI deployment must satisfy the same governance expectations, documented validation, ongoing monitoring, and a clear escalation path, developed for the classical models earlier chapters covered, even though the tools available to satisfy them look different in practice.

16.5.3 Advanced Topics: Fine-Tuning, Agents, and Domain Adaptation

The base LLM described in Section 16.4.2, trained only to predict the next token, is not yet the helpful, instruction-following assistant Section 16.4.3 and this chapter's case vignette describe. Four further ideas, increasingly used in financial NLP systems, close that gap.

Fine-Tuning and Reinforcement Learning from Human Feedback

A model pretrained purely to predict the next token is, left as is, an unreliable assistant: asked a direct question, it may just as easily continue writing more text in the same style as actually answer it, since nothing in next-token prediction alone teaches it to follow an instruction or to prefer a helpful, honest response over a fluent but unhelpful one. After next-token pretraining, a model is typically further trained in two stages: supervised fine-tuning on curated (prompt, ideal response) pairs, teaching it to follow instructions rather than merely continue text, and reinforcement learning from human feedback (RLHF), which uses human preference judgments between pairs of candidate outputs to train a reward model, then adjusts the LLM to produce outputs that reward model scores highly. The reward model itself is typically trained as a preference classifier: given two candidate outputs with reward scores r_A and r_B, the Bradley-Terry model estimates the probability that a human would prefer output A as

$$\Pr(A \succ B) = \text{sigmoid}(r_A - r_B) = \frac{1}{1 + e^{-(r_A - r_B)}}.$$

A Worked Example: Reward-Model Preference Probability

Suppose a reward model scores a well-grounded, source-accurate earnings-call summary at $r_A = 2.1$ and a fluent but subtly inaccurate alternative summary at $r_B = 0.8$. The estimated preference probability is $\text{sigmoid}(2.1 - 0.8) = \text{sigmoid}(1.3) \approx 0.7858$: the reward model favors the accurate summary about 78.6% of the time it is compared against the inaccurate one, and RLHF training nudges the underlying LLM's outputs toward whichever behavior the reward model scores this way, in aggregate across many such comparisons.

Parameter-Efficient Fine-Tuning: Low-Rank Adaptation (LoRA)

RLHF and supervised fine-tuning both describe *what* a model is trained to do after pretraining; a separate, practical question is *how* that further training is actually carried out, since updating every one of a large model's weights, full fine-tuning, requires storing a full second copy of the model's parameters (the update itself) alongside the original, and repeating this for every downstream task or client is prohibitively expensive at the scale of a modern LLM.

Low-Rank Adaptation (LoRA) is the most widely used solution: instead of learning a full, dense weight update ΔW for a $d \times d$ weight matrix, LoRA constrains the update to a low-rank decomposition, $\Delta W = BA$, where B is $d \times r$, A is $r \times d$, and the rank r is chosen to be far smaller than d. Only B and A are trained; the original weight matrix W is frozen entirely, and the adapted model's forward pass simply uses $W + BA$ in place of W.

A Worked Example: Parameter Reduction from Low-Rank Adaptation

Consider a single $d = 4{,}096$ weight matrix, a realistic dimension for one attention or feedforward projection inside a modern LLM. Full fine-tuning of this one matrix requires learning all $d^2 = 4{,}096^2 = 16{,}777{,}216$

Wulin.Suo@Queensu.ca

entries of ΔW. Using LoRA with rank $r = 8$, the trainable parameters are instead only those of B $(d \times r)$ and A $(r \times d)$,

$$2 \times d \times r = 2 \times 4{,}096 \times 8 = 65{,}536,$$

a reduction factor of $16{,}777{,}216/65{,}536 = 256\times$ for this single matrix, and a comparable reduction applies across every weight matrix LoRA is attached to throughout the model.

This is precisely why LoRA has become the default way to adapt a large financial-domain model, whether fine-tuning a base LLM on a specific bank's internal document style or on a specific asset class's terminology. It avoids both the storage cost of a full duplicate model per use case and the computational cost of updating billions of frozen parameters that full fine-tuning would otherwise touch. The frozen base model, potentially already quantized using this section's own quantization technique described below, can be shared across many clients or tasks, with only a small, task-specific B and A pair, a few tens of thousands of parameters rather than billions, stored and swapped in per use case.

LLM Agents and Tool Use

Computing and cross-checking a company's P/E ratio requires looking up its current price, pulling its latest reported earnings from a filing, dividing the two, and comparing the result against a sector benchmark, a sequence of lookups and arithmetic no single LLM call, which only ever produces text in one pass, can carry out on its own. An agent extends a single-turn LLM call into a multi-step loop: the model can decide to call an external tool, a retrieval system, a calculator, a database query, or a trading API, observe the tool's result, and decide on a next action, repeating until it produces a final answer. This directly composes the retrieval mechanism of Section 16.4.3 with ordinary computation.

A Worked Example: A Three-Step Agent Computing and Cross-Checking a P/E Ratio

Suppose an agent is asked, "What is Company X's price-to-earnings ratio using its Q3 diluted EPS and quarter-end share price?" The agent first calls a retrieval tool (exactly Section 16.4.3's mechanism) against the same kind of knowledge base, retrieving two facts: diluted EPS of \$3.15 and a quarter-end closing price of \$57.87. It then calls a calculator tool rather than attempting the arithmetic itself, computing $\text{P/E} = 57.87/3.15 \approx 18.37$. Delegating the arithmetic to a calculator tool, rather than having the LLM compute it directly, removes an entire category of arithmetic-hallucination risk, since a calculator's division is exact by construction, while an LLM asked to divide two numbers directly can still make an outright arithmetic error despite retrieving perfectly correct inputs.

A more robust agent does not stop once it has an answer; it takes a third step, calling a second, independent tool to cross-check the result before presenting it. Suppose the agent additionally queries a market-data vendor's own reported P/E for Company X, retrieving 19.10. Comparing this against its own computed value,

$$\text{Discrepancy} = \frac{19.10 - 18.37}{18.37} \approx 3.97\%,$$

exceeds a pre-set tolerance of, say, 2%, so rather than presenting either number as a confident final answer, the agent flags the disagreement explicitly, for example reporting both figures along with a note that they differ by more than the tolerance and that the discrepancy may reflect a timing difference (the vendor's figure may use trailing rather than diluted EPS, or a different price snapshot) worth investigating before either number is used in a client-facing report.

This cross-check step costs one additional tool call but converts a single, silently-possibly-wrong number into either a confidently validated one or an explicitly flagged discrepancy, the kind of independent-verification discipline Chapter 8's model-validation framework requires of a classical risk model, now applied to a single agentic tool-use chain.

Mixture-of-Experts: Scaling Without Proportional Compute

Making a language model more capable usually means making it larger, but a larger dense feedforward network makes every single token more expensive to process, whether or not that token's context actually needed the model's full capacity. A mixture-of-experts (MoE) architecture replaces a single large feedforward

Wulin.Suo@Queensu.ca

network inside each transformer layer with several smaller "expert" networks and a router that sends each token to only a handful of them, letting total model capacity grow without a proportional increase in the compute needed per token.

A Worked Example: Sparse Activation Fraction

Consider a MoE layer with 8 experts of 6 billion parameters each (48 billion total expert parameters) that routes each token to only its top 2 experts. Each token activates $2 \times 6 = 12$ billion of the 48 billion expert parameters, an active fraction of $12/48 = 0.25$, or 25%: the model has the representational capacity of a 48-billion-parameter feedforward layer while each token's forward pass costs only what a 12-billion-parameter layer would.

Domain-Adapted Financial Language Models: BloombergGPT

Section 16.2.3's Loughran-McDonald dictionary made a simple, lexicon-based version of a broader point: general-purpose tools built on general-purpose text underperform on financial language. BloombergGPT, announced by Bloomberg in 2023, applies the same principle to a full-scale LLM rather than a hand-built word list: a 50-billion-parameter model trained from scratch on a mixed corpus of over 700 billion tokens, roughly 363 billion tokens of Bloomberg's own financial data (news, filings, and other proprietary financial text spanning decades) combined with roughly 345 billion tokens of general-purpose text, with 569 billion tokens actually sampled for training. The model outperformed similarly sized general-purpose models on financial-specific benchmarks (sentiment analysis, financial named-entity recognition, and financial question answering) while remaining competitive with them on general-purpose language benchmarks, empirical confirmation, at LLM scale, of the same domain-specificity argument Section 16.2.1 made about financial vocabulary at the very start of this chapter.

Quantization: Deploying a Large Model at Lower Cost

A financial institution that wants to run a model like BloombergGPT on its own infrastructure, rather than sending every query to an external provider's API, an important consideration given Section 16.5.2's data-leakage concern, faces a direct memory and compute cost. Quantization reduces this cost by storing each parameter with fewer bits than the precision used during training, trading a small amount of accuracy for a large reduction in memory footprint.

A Worked Example: Memory Footprint Under Quantization

A 50-billion-parameter model held at 16-bit precision (2 bytes per parameter) requires $50 \times 10^9 \times 2 = 100 \times 10^9$ bytes, 100 gigabytes, just to store its weights, before accounting for activations or the memory a long conversation's key-value cache consumes. Quantizing to 8-bit precision (1 byte per parameter) halves this to 50 gigabytes, and quantizing further to 4-bit precision (0.5 bytes per parameter) brings it to 25 gigabytes, a fourfold reduction from the original 16-bit footprint. This is the kind of trade-off, some accuracy degradation in exchange for a model that fits on cheaper hardware and answers faster, that determines whether an institution can feasibly run a domain-adapted model like this in-house at all, rather than as a purely theoretical exercise.

A Worked Example: API Cost Versus Self-Hosting at Scale

Quantization's payoff is easiest to see by putting a dollar figure on the alternative, and the right way to frame that figure is as a break-even volume, because API prices move far faster than this book's print cycle. The two cost structures differ in kind. An API bills per token, so its cost is purely variable, and providers bill output tokens at a higher rate than input tokens, typically four to six times as much, since generating a token costs more compute than reading one. Self-hosting is mostly a fixed cost: suppose the 4-bit-quantized 50-billion-parameter model runs on one dedicated cloud GPU node reserved around the clock at $20 an hour, or $480 a day ($14,400 a month), whether it is busy or idle, with capacity for 4,000 queries an hour, 96,000 a day.

Wulin.Suo@Queensu.ca

Let a query carry n_{in} input tokens (the prompt plus any retrieved context) and n_{out} output tokens, with API prices p_{in} and p_{out} per million tokens. Self-hosting wins once daily query volume Q exceeds

$$Q^* = \frac{\$480}{(n_{\text{in}}\, p_{\text{in}} + n_{\text{out}}\, p_{\text{out}})/10^6}.$$

Take an institution's financial-research assistant handling 40,000 queries a day at 800 input and 200 output tokens each. The prices are the parameter that dates. At the 2023 list price of the original GPT-4, \$30 per million input tokens and \$60 per million output tokens, a query cost \$0.036, the API bill came to \$1,440 a day, three times the self-hosted node, and the break-even was only 13,333 queries a day. As of 2026, published list prices span roughly \$0.05 to \$10 per million input tokens and \$0.50 to \$50 per million output tokens, from the smallest models to the frontier ones. At an illustrative frontier-tier price of \$5 in and \$25 out, a query costs \$0.009, the same workload costs \$360 a day through the API, cheaper than the \$480 node, and self-hosting wins only above 53,333 queries a day. At an illustrative mid-tier price of \$2 in and \$10 out, the break-even rises to 133,333 queries a day, more than the node can serve, and at a small model's \$0.10 in and \$0.50 out, about \$7 a day buys the whole workload. Input and output asymmetry matters as much as the headline rate: tilt the same 1,000 tokens toward long generated answers, 200 in and 800 out, and the frontier-tier cost per query rises to \$0.021, cutting the break-even to 22,857 queries a day.

The conclusion that survives price changes is the structure, not any single number. Falling per-token prices have pushed the break-even volume up by roughly a factor of four or more since 2023, so a pure cost case for self-hosting now requires either very high volume, an output-heavy workload, or a premium model whose API price is high. The memory-footprint calculation is still what makes the self-hosted option feasible at all, since a model that fits on one accessible GPU only at 4-bit precision is the model that keeps the fixed cost at one node rather than a multi-GPU cluster. And cost is rarely the whole decision: Section 16.5.2's data-leakage concern can favor self-hosting even below the break-even, because it keeps every query's content inside the institution's own infrastructure rather than sending it to an external provider.

16.5.4 Disclosure Analysis: Readability and the Gunning Fog Index

Regulators require that public disclosures be understandable to ordinary investors (the U.S. Securities and Exchange Commission's "Plain English" rule is one such requirement), and readability metrics give a simple, quantitative way to assess whether a given passage meets that bar. The Gunning Fog index estimates the years of formal education a reader needs to understand a passage on a first reading,

$$\text{Fog} = 0.4\left[\frac{\text{words}}{\text{sentences}} + 100 \times \frac{\text{complex words}}{\text{words}}\right],$$

where a complex word is (approximately) one with three or more syllables.

A Worked Example: Two Disclosure Paragraphs Compared

Consider two short passages. Paragraph A: "The firm's sales grew this year. Profits also rose. We expect this trend to grow next year." This has 17 words across 3 sentences and zero complex words (every word has one or two syllables), giving $\text{Fog} = 0.4\,[(17/3) + 100(0/17)] = 2.27$.

Paragraph B: "The Company's operations are subject to numerous regulatory, competitive, and macroeconomic uncertainties. These uncertainties could materially and adversely affect anticipated profitability and operational continuity." This has 24 words across 2 sentences, of which 14 are complex (numerous, regulatory, competitive, macroeconomic, uncertainties, materially, adversely, anticipated, profitability, operational, continuity, and others), giving $\text{Fog} = 0.4\,[(24/2) + 100(14/24)] = 28.13$.

A Fog index above roughly 17 corresponds to a post-graduate reading level; Paragraph B's score of 28.13 is far beyond that, and while these two toy paragraphs are deliberately constructed to contrast sharply, the direction of the result matches a well-documented empirical finding: 10-K risk-factor sections and other mandatory disclosures routinely score in this same elevated range in practice, dense, technical, and long-sentence.

Readability metrics like this one give compliance and research teams a simple, automatable way to flag disclosures that may not be meeting the plain-English standard regulators intend, or, for a research

Wulin.Suo@Queensu.ca

application, to test whether unusually low readability in a given filing is itself a predictive signal (some published research finds that harder-to-read disclosures are associated with worse subsequent firm performance, consistent with obfuscation being used strategically).

The Fog index is also useful as a diagnostic during the drafting process itself, not only after the fact. Rewriting Paragraph B in plainer language, "Our business faces many risks from rules, competition, and the economy. These risks could hurt our profits and how well we operate," gives 22 words across 2 sentences with only 3 complex words (competition, economy, operate), so

$$\text{Fog} = 0.4\,[(22/2) + 100(3/22)] \approx 9.85,$$

close to Paragraph A's toy score and nowhere near Paragraph B's original 28.13, despite covering essentially the same substantive content, the same categories of risk and the same warning about their potential effect on profitability. This is precisely the practical use case regulators intend a Plain English rule to encourage: the underlying legal and business meaning did not need to change at all to bring the Fog score down by nearly two-thirds, only the sentence structure and word choice, which is why an automated readability check integrated directly into a disclosure-drafting workflow can flag an overly complex passage for simplification before it is ever filed, rather than only after a regulator or researcher measures it externally.

16.6 Case Vignette: AI @ Morgan Stanley

In September 2023, Morgan Stanley announced that AI @ Morgan Stanley Assistant, built together with OpenAI on GPT-4, was fully live for its wealth management financial advisors and support staff. The assistant is, in this chapter's terms, a retrieval-augmented generation system: it gives advisors natural-language access to the firm's "intellectual capital," a collection of roughly 100,000 research reports and internal documents, retrieving the passages relevant to an advisor's question and generating an answer grounded in them, the mechanism Section 16.4.3 worked through on a toy four-document example. Morgan Stanley later reported that the tool reached over 98% adoption among advisor teams, and that the share of the firm's research content advisors could effectively find and use rose from roughly 20% to roughly 80%, a direct measure of the retrieval problem Section 16.4.3 illustrated at genuinely large scale rather than four toy documents.

The firm subsequently extended the same underlying models to a second, structurally different application: AI @ Morgan Stanley Debrief, which uses OpenAI's Whisper speech-to-text model together with GPT-4 to turn a client meeting recording, made only with the client's consent, into a structured summary, client notes filed automatically into the firm's customer relationship management system, and a draft follow-up email an advisor reviews and sends. This is squarely the summarization capability Section 16.4.2 described, deployed on genuinely sensitive input (a recorded client conversation) rather than a public filing, which is precisely why consent, access control, and human review before anything reaches a client are treated as first-class design requirements rather than afterthoughts, the same governance discipline Section 16.5.2 argued a generative AI deployment needs from the outset.

A third tool, AskResearchGPT, later extended the same retrieval-augmented pattern from wealth management into the firm's institutional research business, illustrating that a well-built RAG system, once validated for one internal use case, tends to generalize to others faster than building a comparable system from scratch each time. The case is instructive for this chapter specifically because its two headline results, near-total adoption and a fourfold increase in effective document accessibility, came not from a more powerful underlying model than any other firm could license, but from the unglamorous, chapter-spanning work this chapter has emphasized throughout: a well-curated retrieval corpus, a retrieval mechanism grounded in the same cosine-similarity geometry Section 16.3.3 introduced, and a governance structure, consent for Debrief's recordings, human review of drafted outputs, that made deployment at scale defensible.

16.7 Challenges in Natural Language Processing in Finance

Several challenges recur across the techniques this chapter developed, and many connect to themes from earlier chapters.

Wulin.Suo@Queensu.ca

Domain-specific vocabulary defeats generic tools. As Section 16.2.1 emphasized, sentiment lexicons and language models trained on general text routinely misread financial language, which is why finance-specific resources like the Loughran-McDonald dictionary exist and why even a large general-purpose language model benefits from finance-specific fine-tuning or careful prompting.

A weak signal is still noisy, not deterministic. Section 16.3.1's trading signal added real value on average, a higher mean return and a better Sharpe-like ratio than buy-and-hold, while still being wrong on two of eight events, a reminder that an NLP-derived signal is a probabilistic input to a strategy, not a certainty, the same lesson Chapter 6 emphasized about model evaluation generally.

Hallucination is a qualitatively different failure mode than misclassification. A classifier's error rate is directly computable from a confusion matrix, as Section 16.3.2 demonstrated; an LLM's tendency to generate fluent, wrong answers, discussed in Section 16.4.2, has no equally clean error rate, since detecting a hallucination often requires the same domain expertise the tool was meant to save. Retrieval-augmented generation (Section 16.4.3) mitigates this, but does not eliminate it: a model can still misread or misquote a correctly retrieved passage.

Generative AI introduces risks classical models never raised. Non-deterministic outputs, prompt injection, and RAG-related data leakage, all discussed in Section 16.5.2, have no real counterpart in the deterministic classification and scoring models Chapters 6 through 15 developed, and require validation approaches, statistical sampling of outputs rather than checking a single fixed prediction, that those earlier chapters' models did not need.

Explainability and audit requirements constrain deployment. As Chapter 18 develops further, a trading decision or disclosure classification influenced by an NLP model may need to be explained after the fact, which is more difficult for the deep, attention-based architectures of Section 16.4.1 than for the transparent, inspectable probabilities of Section 16.3.2's Naive Bayes classifier.

Data licensing and evaluation are both harder than they look. High-quality financial text, especially real-time news and licensed data feeds, is expensive and contractually restricted, and evaluating an NLP system's real-world performance is harder than evaluating a numerical model, since two humans often disagree about the "correct" sentiment or summary for a given passage, so even the ground truth used to measure accuracy is itself somewhat subjective.

16.8 Key Takeaways

Financial text must be converted into numerical features before any of this book's earlier tools apply to it, and this chapter developed that conversion end to end: bag-of-words and TF-IDF (Section 16.2.2) turn text into sparse count vectors, the Loughran-McDonald lexicon (Section 16.2.3) turns those vectors into a finance-specific sentiment score, and word embeddings (Section 16.3.3) replace sparse counts with dense vectors that capture semantic similarity geometrically.

A lexicon-based sentiment score is a genuinely useful but noisy trading signal, as Section 16.3.1 showed with a 75% hit rate that still outperformed buy-and-hold on both average return and consistency, and generating that signal is only half the task Chapter 13 develops in full. Naive Bayes, introduced as an abstract generative classifier in Chapter 6, applies directly to document classification (Section 16.3.2), including the genuinely ambiguous cases where its conditional-independence assumption's imprecision is most visible.

Self-attention (Section 16.4.1) is the mechanism that lets transformer-based models, including the large language models of Section 16.4.2, build context-dependent representations of text, and those same models' generative capability supports summarization and information extraction well beyond classification, at the cost of a qualitatively new failure mode, hallucination, that a computable error rate cannot fully capture. Retrieval-augmented generation (Section 16.4.3) is the leading mitigation, grounding generated answers in specific, verifiable source passages, while prompting strategy (Section 16.4.4), zero-shot, few-shot, or chain-of-thought, shapes how reliably a generative model performs a given task without any change to the model itself.

Generative modeling extends well beyond text: GANs and diffusion models (Section 16.5.1) generate entirely synthetic numerical data, additional minority-class fraud examples or stress-test scenarios beyond the historical sample, by learning to reverse a noising process or to fool an adversarially trained discriminator. All of this requires governance (Section 16.5.2) that extends Chapter 15's three-lines-of-defense framework to

Wulin.Suo@Queensu.ca

risks, non-determinism, prompt injection, data leakage, that classical, deterministic models never presented.

Section 16.5.3 went one level deeper still: RLHF's reward modeling turns human preferences into a trainable signal, agents compose retrieval with actual computation (and should delegate arithmetic to a calculator tool rather than trust an LLM's own division), mixture-of-experts architectures decouple model capacity from per-token compute cost, and BloombergGPT demonstrates the Loughran-McDonald dictionary's domain-specificity argument holding at full LLM scale. Finally, readability metrics like the Gunning Fog index (Section 16.5.4) turn a regulatory concern, whether disclosure is genuinely understandable, into a simple, automatable measurement.

16.9 Suggested Exercises

1. Using Table 16.2's method, compute the idf and tf-idf weight (in D_2) for a seventh term, "confident," which appears once in D_3 only, and explain why its idf is the largest of any term considered so far.

2. Using Section 16.2.3's method and word lists, compute the tone score for the sentence "results were weak and management remains cautious about the coming quarter despite improved margins."

3. Using Table 16.3's method, add a ninth event with tone score $+0.11$ and next-day return -0.6%, and recompute the strategy's hit rate, mean return, and Sharpe-like ratio.

4. Explain, using events 4 and 7 in Table 16.3, why a tone score close to zero is a weaker signal than one far from zero, and propose a modification to the trading rule that would account for this (for example, only trading when |tone| exceeds some threshold).

5. Using Section 16.3.2's fitted word probabilities, classify a new sentence with word counts (default: 0, borrower: 1, counterparty: 1, volatility: 0, rate: 1, currency: 0), showing the unnormalized class scores and the resulting posterior probabilities.

6. Using Table 16.5, explain in words why this classifier's precision is perfect (1.00) while its recall is not, and describe a business setting in which that trade-off (never a false alarm, but occasionally missing a true case) would be preferable to the reverse.

7. Using Section 16.3.3's toy embeddings, add a seventh term, "dividend" $= (0.70, 0.40)$, compute its cosine similarity to "stock" and to "bond," and explain which existing term it should be geometrically closest to and why.

8. Using Section 16.3.3's vector-arithmetic example, compute bearish $-$ bullish $+$ stock using the chapter's toy embeddings, and explain in words what analogy this computation is attempting to capture, and why, unlike the stock $-$ equity $+$ bond example, the result does not land exactly on any of the chapter's six labeled terms.

9. Using Section 16.4.1's worked example, recompute the self-attention output for the token "rallied" (using query $Q_{\text{rallied}} = (0, 1)$ instead of Q_{stocks}), and explain which token's value vector now receives the most weight.

10. Using Section 16.4.1's causal-versus-bidirectional worked example, compute "today"'s own contextual representation (using query $Q_{\text{today}} = (1, 1)$) under a causal mask, noting that a token at the last position of a sequence can attend to every earlier token, and explain why, for the very last token in a sentence, a causal mask produces the same output a fully bidirectional model would.

11. Explain, in a short paragraph, why an LLM's hallucination failure mode is harder to catch automatically than a classification model's error rate, referencing Section 16.4.2.

12. Using Section 16.5.4's method, compute the Gunning Fog index for a passage of your own construction with 30 words, 2 sentences, and 6 complex words, and compare it to both worked paragraphs' scores.

Wulin.Suo@Queensu.ca

13. Compare this chapter's Naive Bayes document classifier (Section 16.3.2) to Chapter 6's original loan-default Naive Bayes discussion: explain, in your own words, why the conditional-independence assumption is more defensible for word-count features in a document than it would be for a firm's financial ratios.

14. Using Section 16.4.3's worked example, add a fifth knowledge-base document, Doc5, "Company X's Q3 operating expenses rose 5% to $310 million," with embedding $(0.75, 0.30, 0.00)$, compute its cosine similarity to the same query, and state where it ranks among the five documents.

15. Using Section 16.4.3's precision@k example, recompute Precision@4 if Doc5 from the previous exercise (also not relevant to a revenue question) is retrieved fourth, and explain in words why adding a fifth, similarly irrelevant document could never increase precision at any k, only decrease it or leave it unchanged.

16. Using Section 16.4.3's faithfulness-score example, an answer embeds at $(0.80, 0.25, 0.00)$. Compute its faithfulness score against Doc1, and state whether it would pass a review threshold of 0.90.

17. Write a zero-shot prompt, a two-example few-shot prompt, and a chain-of-thought prompt, following Section 16.4.4's patterns, for the task of classifying whether a sentence from a 10-K risk-factors section describes credit risk or market risk (the task Section 16.3.2 solved with Naive Bayes instead).

18. Using Section 16.5.1's worked example, compute x_t for a third diffusion step with $\alpha_t = 0.30$, holding $x_0 = 1.00$ and $\epsilon = -0.50$ fixed, and confirm that x_t has moved even closer to ϵ than the two steps already computed.

19. Explain, in a short paragraph, why validating a generative AI system requires sampling many outputs rather than checking a single prediction against a labeled test set, referencing Section 16.5.2 and contrasting it with how Chapter 6 evaluates a classification model.

20. Pick two of the challenges listed in this chapter and describe a concrete scenario, not already used in the text, in which that challenge could cause an NLP-based financial application to fail despite technically correct code.

21. Using this chapter's case vignette, explain why AI @ Morgan Stanley Debrief's requirement of client consent before recording a meeting is best understood as a governance control (Section 16.5.2) rather than a purely legal formality, and describe one way the tool's design (human review before sending a draft follow-up) further limits the hallucination risk Section 16.4.2 described.

22. Using Section 16.5.3's Bradley-Terry worked example, recompute the preference probability if the inaccurate summary's reward score r_B rises from 0.8 to 1.7 (holding $r_A = 2.1$ fixed), and explain in words why the preference probability moves toward 0.5 rather than away from it.

23. Using Section 16.5.3's agent example, explain why the agent calling a calculator tool to compute 57.87/3.15 is preferable to the LLM performing that division itself, and describe a different two-step financial task (not P/E) that would similarly benefit from separating retrieval from computation.

24. Using Section 16.5.3's mixture-of-experts example, recompute the active parameter fraction for a layer with 16 experts of 4 billion parameters each, routing each token to its top 4 experts, and compare the resulting total capacity and active-compute cost to the chapter's original 8-expert example.

25. Using Section 16.5.3's quantization worked example, compute the memory footprint of a 175-billion-parameter model at FP16, INT8, and INT4 precision, and state the reduction factor from FP16 to INT4.

26. Using Section 16.5.3's LoRA worked example, recompute the number of trainable parameters and the reduction factor relative to full fine-tuning if the rank r is increased from 8 to 32 (holding $d = 4{,}096$ fixed), and explain in words the tradeoff a practitioner faces when choosing a larger rank.

Wulin.Suo@Queensu.ca

27. **Challenge (real data).** Download the risk-factors (Item 1A) section of a real company's 10-K filing directly from SEC EDGAR's full-text search (free, no login required), and download the Loughran-McDonald sentiment word list (freely available from the University of Notre Dame's Software Repository for Accounting and Finance). (a) Apply this chapter's tone-scoring method (Section 16.2.3) to the real risk-factors text, reporting the resulting tone score and the specific positive and negative words the dictionary matched. (b) Compute the Gunning Fog index (Section 16.5.4) for the same passage, and compare it to both of this chapter's own worked-example scores. (c) Real 10-K risk-factor sections are written by legal and investor-relations teams who have every incentive to sound appropriately cautious regardless of the firm's actual condition; discuss whether a low (negative) tone score on this kind of boilerplate legal language should be interpreted the same way as a low tone score on the earnings-call language this chapter's own worked examples use.

16.10 Further Reading

- Loughran and McDonald, "When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks," *Journal of Finance*. The original paper introducing the finance-specific sentiment dictionary used in Section 16.2.3.
- Hansen and McMahon, "Shocking Language: Understanding the Macroeconomic Effects of Central Bank Communication," *Journal of International Economics*. A research-grade version of Section 16.2.3's central-bank worked example, using text methods to separate what the Federal Reserve says from what markets expected it to say.
- Jurafsky and Martin, *Speech and Language Processing*. A comprehensive, freely available reference covering bag-of-words, TF-IDF, and embedding methods (Sections 16.2.2 and 16.3.3) in far greater depth.
- Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems*. The original transformer paper introducing the self-attention mechanism worked through by hand in Section 16.4.1.
- Mikolov et al., "Efficient Estimation of Word Representations in Vector Space," *arXiv*. The original Word2Vec paper underlying the embedding geometry illustrated in Section 16.3.3.
- Li, "The Information Content of Forward-Looking Statements in Corporate Filings: A Naive Bayesian Machine Learning Approach," *Journal of Accounting Research*. An application of the Naive Bayes text classification approach used in Section 16.3.2 to real corporate filings.
- Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*. The original paper introducing the retrieval-augmented generation approach worked through in Section 16.4.3.
- Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Advances in Neural Information Processing Systems*. The original paper on the chain-of-thought prompting strategy discussed in Section 16.4.4.
- Goodfellow et al., "Generative Adversarial Networks," *Communications of the ACM*. The original paper introducing the generator-discriminator framework discussed in Section 16.5.1.
- Ho, Jain, and Abbeel, "Denoising Diffusion Probabilistic Models," *Advances in Neural Information Processing Systems*. The paper underlying the forward and reverse diffusion process worked through by hand in Section 16.5.1.
- Board of Governors of the Federal Reserve System and Office of the Comptroller of the Currency, *Supervisory Guidance on Model Risk Management (SR 11-7)*. The foundational U.S. model risk management guidance being extended by supervisors to cover the generative AI governance questions raised in Section 16.5.2.

Wulin.Suo@Queensu.ca

- Morgan Stanley, "Key Milestone in Innovation Journey with OpenAI" and "Launch of AI @ Morgan Stanley Debrief" (press releases). The primary source for the retrieval-augmented generation and meeting-summarization deployment described in this chapter's case vignette.
- Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," *Advances in Neural Information Processing Systems*. The paper introducing the RLHF pipeline and reward modeling worked through by hand in Section 16.5.3.
- Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," *International Conference on Learning Representations*. A foundational paper on the agent tool-use loop illustrated in Section 16.5.3's worked example.
- Shazeer et al., "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," *International Conference on Learning Representations*. The original paper introducing the sparse routing architecture discussed in Section 16.5.3.
- Wu et al., "BloombergGPT: A Large Language Model for Finance," *arXiv*. The paper describing the domain-adapted financial language model discussed in Section 16.5.3.
- U.S. Securities and Exchange Commission, *A Plain English Handbook*. The SEC's own guidance underlying the readability and disclosure-analysis discussion in Section 16.5.4.

Wulin.Suo@Queensu.ca

Chapter 17

Alternative Data and Deep Learning in Finance

17.1 Introduction

Chapter 1 introduced a table of the major categories of financial data this book draws on, and set aside one row, alternative data, non-traditional sources such as satellite imagery, web traffic, or shipping data that proxy for economic activity, for this chapter. Chapter 3 noted that large asset managers already parse filings and calls faster than human analysts; this chapter turns to data sources that did not exist, in usable form, a generation ago, and to the deep learning architectures built to extract signal from them. Chapter 4 observed that alternative data can supplement the delayed, discretionary numbers in financial statements; Chapter 6 introduced the feedforward neural network and its forward pass; Chapter 16 developed the transformer architecture for text. This chapter extends the neural network toolkit to two more data shapes, grid-structured images and ordered sequences, and works through the full pipeline from raw alternative data to a usable financial signal, while treating the biases specific to this kind of data as a first-class topic rather than an afterthought.

Thirteen worked examples run through the chapter: a subscription cost-benefit calculation weighing a data vendor's licensing cost against the expected trading edge it buys; a simple regression of a retailer's revenue growth on a satellite-derived parking-lot car-count proxy; a multi-source regression that combines that satellite proxy with web-traffic and card-transaction growth into an earnings-surprise signal, evaluated the way Chapter 13 evaluates any trading strategy; a direct comparison of that signal built the naive way versus the way that respects real-world data-delivery lag; a peer-relative normalization of the same satellite proxy against a sector benchmark. A hand-computed two-dimensional convolution over a stylized satellite image patch; a hand-computed recurrent neural network forward pass over a short sequence of web-traffic readings; a hand-computed autoencoder reconstruction used to flag a data-quality anomaly; a Loughran-McDonald tone score built directly from a company's own risk-factor disclosures, illustrating both the appeal and the limits of building an ESG-style signal in-house rather than buying one; and, going a level deeper still, a transfer-learning fine-tuning step, a graph neural network update over a shell-company network, a cross-attention fusion of three data modalities into one representation, and a contrastive-pretraining probability calculation showing how a labeled-data-free encoder learns to tell same-image crops apart from different-image crops.

This chapter's structure mirrors the arc of a real alternative-data project, and is worth previewing before working through the details. It begins where any such project begins, deciding what alternative data even is and how a fund acquires and evaluates it (Sections 17.2.1 and 17.2.2), before turning to the analytics itself: a single-source regression, a multi-source signal, the point-in-time discipline that keeps that signal honest, and the seasonal and peer-relative adjustments that make raw readings usable in the first place (Sections 17.2.3 through 17.3.2).

It then turns to the deep learning architectures that extract features from raw images and sequences rather than from a vendor's already-processed number (Section 17.3.3), the data-quality and governance

Wulin.Suo@Queensu.ca

problems specific to this kind of data (Section 17.3.4), a worked example on building an ESG-style text signal in-house as a partial, auditable response to those same governance problems (Section 17.4.1), and a set of more advanced architectures, transfer learning, graph neural networks, cross-attention fusion, and contrastive pretraining, that represent where the field is moving beyond the basics (Section 17.4.2). A real-world case vignette and a closing set of challenges tie the chapter's techniques back to the genuine economic and regulatory stakes of using data that, in most cases, did not exist in usable form before this decade.

17.2 The Alternative-Data Landscape

This part establishes what alternative data is, who sells it, and works two signals end to end.

17.2.1 What Counts as Alternative Data

Suppose a hedge fund wants to know how a retailer's holiday-quarter sales are trending three months before the retailer itself reports earnings, information that would be extremely valuable if the fund could act on it early and worthless the moment everyone else has it too. Traditional market, fundamental, and macroeconomic data (Chapter 1's Table 1.6) cannot answer this: market data only reflects what the market already knows, and fundamental data about this quarter will not exist until the retailer files it.

Alternative data exists to fill this gap. In financial industry usage, it means any data source used to inform an investment or credit decision that falls outside the traditional trio of market data, fundamental data, and macroeconomic data. The term is a moving target: yesterday's alternative data becomes today's mainstream input once enough market participants adopt it, so the boundary is defined by current practice rather than any fixed technical property. Table 17.1 catalogues the categories most widely used in practice.

Category	Examples	Typical financial use
Satellite and aerial imagery	Parking-lot car counts, crop health indices, oil-storage tank levels, construction progress	Retail sales nowcasting, commodity supply estimation, real-estate tracking
Web and app data	Website traffic, app downloads and reviews, job postings, e-commerce pricing	Revenue nowcasting, hiring-trend and competitive signals
Transaction and card-panel data	Aggregated, anonymized consumer card-spending panels	Same-store sales nowcasting ahead of earnings
Geolocation and foot traffic	Mobile-device location pings at store or venue locations	Foot-traffic proxies for retail, dining, and travel
Shipping and logistics	Vessel-tracking (AIS) data, port congestion, rail and truck telemetry	Commodity flow and trade-volume forecasting
Social and search data	Social-media posts, review platforms, search-trend indices	Consumer-sentiment and brand-momentum signals
On-chain (blockchain) data	Public transaction ledgers, wallet flows, exchange inflows and outflows, stablecoin issuance	Crypto asset-flow signals, exchange-risk monitoring, AML and fraud forensics

Table 17.1: Major categories of alternative data

Two properties distinguish alternative data from the data types earlier chapters used. First, it usually arrives already partially processed by a vendor, a satellite imagery provider sells car counts, not raw pixels, so the modeler inherits whatever assumptions and errors are baked into that proprietary processing step, typically without visibility into it. Second, it is available at a frequency and granularity, individual stores, individual weeks, that traditional fundamental data (quarterly, firm-level) simply does not offer, which is precisely why it is valuable: it lets an analyst estimate a firm's performance before the firm itself reports it.

The table's last category deserves its own note, because it inverts both of those properties. **On-chain data**, the complete public transaction ledger a blockchain such as Bitcoin's or Ethereum's maintains by construction, records every transfer, wallet balance, and smart-contract interaction, free of charge and with

Wulin.Suo@Queensu.ca

no vendor standing between the analyst and the raw record. Funds trading crypto assets themselves (Chapter 13's tokenization discussion introduces the instruments) read signals directly from it, net flows into and out of exchange-controlled wallets as a proxy for imminent selling or accumulation, stablecoin issuance as a proxy for capital entering the ecosystem, while risk and compliance teams watch the same ledger for a different reason, monitoring exchange solvency and tracing stolen or laundered funds. What the analyst inherits instead of a vendor's opaque preprocessing is pseudonymity: every party is an unlabeled address, so the commercial value-add in this category is entity labeling and address clustering, the same graph techniques Chapter 15 applies to shell-company layering chains, sold by the same analytics vendors compliance teams already rely on. That makes on-chain data the rare category where the raw data is free and complete while the interpretive layer is the scarce, priced asset, the exact reverse of a satellite feed, whose pixels are expensive and whose car counts are the deliverable.

17.2.2 The Alternative-Data Vendor Ecosystem

Unlike the market and fundamental data of Chapters 2 through 5, which arrive through well-established exchange feeds and regulatory filings, alternative data is procured through a comparatively young and heterogeneous vendor market, and how a fund evaluates and integrates a vendor matters almost as much as the underlying data source itself. A typical procurement process runs through several stages.

First, a fund identifies a candidate hypothesis, does satellite-derived parking-lot data actually predict Cascade's revenue, and requests a trial dataset covering a historical period the vendor already has on file. Second, the fund backtests that trial data against known, already-reported outcomes, the exercise Section 17.2.3 works through by hand, before paying for a live subscription; a vendor unwilling to provide a meaningful historical trial, or one whose trial-period performance does not hold up when checked, is a warning sign rather than a mere inconvenience. Third, if the trial is convincing, the fund negotiates a licensing agreement that specifies not just price but exclusivity (is this data available to other subscribers, and how many), delivery latency (Section 17.3.4 returns to why this matters enormously), and historical backfill (how far back the vendor can provide data, which determines how much history is available for the fund's own model validation).

This procurement cycle is itself a source of adverse selection worth naming directly: a vendor has every incentive to showcase the historical period and the subset of covered firms where its data performed best, since that is what sells subscriptions, so a trial dataset's strong backtested performance is not automatically evidence that the data will perform as well in the future or across firms the trial period did not happen to include. This is a version of the multiple-testing concern Chapter 6 raised about a researcher's own model search, now applied to a vendor's marketing rather than a researcher's specification search.

A subscription decision ultimately comes down to a cost-benefit calculation, whether the expected trading edge, scaled to the capital actually deployed on it, exceeds the data's licensing cost. Suppose a fund would deploy a \$50 million position whenever Section 17.2.4's signal fires long, which happened in five of the eight quarters tabulated in Table 17.3 below. Suppose too that the signal's incremental edge over an unconditional baseline is the 0.5425 percentage points computed there, the 0.88% average return on signaled quarters minus the 0.3375% average return across all quarters. With four earnings releases per year and a 5/8 signal frequency, the fund expects roughly $4 \times 5/8 = 2.5$ signaled events annually, for an expected incremental profit of $\$50\text{M} \times 0.54\% \times 2.5 \approx \$678{,}000$ per year. Against Section 17.5's subscription-cost range, a satellite feed at \$250,000 per year plus a smaller web and card-panel bundle at \$150,000 per year, total data costs come to \$400,000. The net expected benefit is therefore roughly \$278,000 per year: a genuine but modest edge once data costs are netted out, and one that Section 17.3.4 warns will shrink further as more funds license the same feeds and compete the edge away. Even the gross figure flatters the signal. The 0.5425-point edge is measured on the same eight quarters the regression was fitted to, so it is an in-sample number, and \$678,000 is best read as an upper bound on what the signal would earn on quarters it has not seen.

This same arithmetic underlies a build-versus-buy decision many larger funds eventually face. Once expected trading volume across many similar signals is large enough, building an in-house satellite-analytics or web-scraping team can be cheaper in the long run than paying a vendor's margin on top of the same underlying imagery or logs. The cost is taking on the computer-vision and data-engineering work Section 17.3.3 describes directly, rather than buying an already-processed feature. Smaller funds typically lack the scale to justify this fixed cost and remain vendor customers indefinitely. That is one more way, alongside

Wulin.Suo@Queensu.ca

Section 17.5's subscription-cost figures, in which alternative data's advantages concentrate among larger, better-resourced market participants.

17.2.3 A Worked Example: Satellite Imagery and Retailer Revenue

Suppose a hypothetical home-improvement retailer, Cascade Home & Garden, is followed by a data vendor that counts cars in a representative sample of its store parking lots from satellite imagery, in the manner Section 17.5 describes being done for real retailers since 2010. Table 17.2 lists eight quarters of year-over-year car-count growth alongside Cascade's subsequently reported year-over-year revenue growth.

Quarter	Car-count growth (%)	Revenue growth (%)
Q1	2.1	1.5
Q2	−1.5	−0.3
Q3	4.8	4.5
Q4	0.6	1.0
Q5	6.2	4.0
Q6	−3.0	−1.0
Q7	3.5	3.8
Q8	1.0	−0.2

Table 17.2: Satellite car-count growth and Cascade's reported revenue growth, by quarter

Fitting the simple regression Revenue growth $= \alpha + \beta\,$(Car-count growth) by ordinary least squares, following Chapter 6's Section 6.3.1 method, gives $\bar{x} = 1.7125$, $\bar{y} = 1.6625$, $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y}) = 43.92375$, $S_{xx} = \sum(x_i - \bar{x})^2 = 67.28875$, so $\hat{\beta} = S_{xy}/S_{xx} = 0.6528$ and $\hat{\alpha} = \bar{y} - \hat{\beta}\bar{x} = 0.5446$.

The fitted line, $\widehat{\text{Revenue growth}} = 0.5446 + 0.6528\,$(Car-count growth), achieves $R^2 = 0.870$, car-count growth alone explains 87% of the quarter-to-quarter variation in Cascade's revenue growth in this sample, genuinely strong for a single proxy variable, but, consistent with this book's practice of not overstating a model's fit, still leaves 13% of the variation unexplained.

Suppose satellite data for a ninth quarter arrives showing 5.0% car-count growth, three weeks before Cascade's earnings release. The regression predicts $\widehat{\text{Revenue growth}} = 0.5446 + 0.6528(5.0) = 3.809\%$, a concrete, tradeable estimate of a number the market will not see officially for three more weeks.

17.2.4 Combining Alternative Data Sources: A Multi-Source Earnings Signal

A single alternative data source is rarely used alone in practice; a real alternative-data desk combines several sources into one model, both to average out any single source's idiosyncratic noise and to capture aspects of a firm's business a single proxy misses. Figure 17.1 sketches the resulting pipeline.

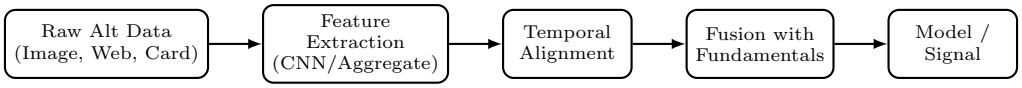


Figure 17.1: The alternative-data pipeline this chapter develops

Extending Section 17.2.3's example, suppose Cascade is also tracked by a web-traffic vendor (year-over-year growth in visits to Cascade's e-commerce site) and a card-panel vendor (year-over-year growth in aggregated card spending at Cascade's stores). Table 17.3 lists all three alternative-data growth rates alongside the resulting earnings surprise, actual revenue growth minus the analyst consensus estimate available at the time, and Cascade's stock return on the trading day after the earnings announcement.

Regressing surprise on the three alternative-data growth rates by ordinary least squares, using the normal equations

$$\hat{\beta} = (X^\top X)^{-1} X^\top y,$$

Wulin.Suo@Queensu.ca

Quarter	Car (%)	Web (%)	Card (%)	Surprise (%)	Fitted (%)	Signal	Return (%)
Q1	2.1	3.0	1.5	0.5	0.48	Long	1.1
Q2	−1.5	−0.5	−1.0	−1.3	−1.08	Flat	−0.6
Q3	4.8	5.5	3.0	2.5	1.95	Long	2.0
Q4	0.6	1.5	0.8	0.0	−0.44	Flat	0.3
Q5	6.2	4.0	4.5	2.0	2.46	Long	−0.4
Q6	−3.0	−2.0	−1.8	−1.5	−1.92	Flat	−1.4
Q7	3.5	2.5	2.0	2.3	1.66	Long	1.5
Q8	1.0	2.0	0.5	−1.2	0.18	Long	0.2

Table 17.3: Multi-source alternative data, earnings surprise, and next-day return, by quarter

with X's columns an intercept, car-count growth, web-traffic growth, and card growth, gives

$$\hat{\beta} = (-0.291,\ 1.092,\ -0.113,\ -0.792), \qquad R^2 = 0.837.$$

The negative coefficient on card growth is not a mistake: with car-count and web-traffic growth already in the regression, card growth's own explanatory power is largely redundant with theirs, and its small residual coefficient partly offsets collinear noise the other two variables share, a common occurrence when several correlated proxies for the same underlying quantity are entered into one regression together, and a reminder that individual coefficients in a multi-source model should not be read as clean, isolated effects the way Chapter 6's Section 6.3.1 single-variable coefficient can be.

Column "Fitted" reports each quarter's fitted surprise from this regression, and column "Signal" converts a positive fitted surprise into a long position taken ahead of the earnings release (flat otherwise), the same logic Chapter 16's Section 16.3.1 applied to a sentiment score. Five of the eight quarters signal long; of those, four (Q1, Q3, Q7, Q8) are followed by a positive next-day return and one (Q5) by a negative return despite a strongly positive fitted surprise, an 80% hit rate.

The signaled quarters average a 0.88% next-day return versus 0.34% across all eight quarters, a real edge within this sample (it is measured on the same quarters the regression was fitted to, so it is in-sample), but, exactly as Chapter 6 and Chapter 16's Section 16.3.1 both emphasize, a probabilistic one: Q5's miss shows that even a well-fitted, multi-source alternative-data model does not eliminate the risk that a single quarter's price reaction depends on something the data could not capture, guidance issued alongside the earnings release, for instance. A ninth quarter with car, web, and card growth of 5.0%, 4.5%, and 3.5% respectively yields a fitted surprise of

$$-0.291 + 1.092(5.0) - 0.113(4.5) - 0.792(3.5) = 1.888\%,$$

again a tradeable estimate available before the official release.

17.3 Working with Alternative Data

Alternative data breaks alignment, scale, and quality assumptions that price data never did.

17.3.1 Point-in-Time Alignment and the Cost of Look-Ahead Bias

Section 17.2.4's regression silently assumed that every growth rate in Table 17.3 is the figure that was genuinely in hand before that quarter's earnings release. Card-panel data shows why that assumption fails even when delivery is fast. Commercial card panels typically arrive within days or weeks of the period they cover, which is their selling point, but the history a vendor sells today is not the history a subscriber would have seen in real time. Late-arriving transactions are added after the first release, the panel's composition shifts as card issuers join or leave it, and vendors routinely revise and re-weight past figures, backfilling the entire history with today's panel and today's methodology. A backtest that treats today's revised card series as if each quarter's figure had been known in that form at the time is committing look-ahead bias, using information the backtest could not actually have had.

Wulin.Suo@Queensu.ca

To see how much this matters, refit Section 17.2.4's regression on quarters Q2 through Q8 (seven quarters, dropping Q1 since it has no prior quarter to lag from) two ways. The naive version uses each quarter's own, contemporaneous card growth, exactly as before:

$$\hat{\beta}_{\text{naive}} = (-0.290,\ 1.102,\ -0.119,\ -0.802), \qquad R^2 = 0.837,$$

with four quarters signaling long (Q3, Q5, Q7, Q8) and a 75% hit rate. The ideal fix is to use each quarter's first-release vintage, the figure as the vendor originally delivered it, but a backtest often lacks those vintages. A common conservative substitute, used here, is the card growth reported one quarter earlier, a figure certainly in hand before the quarter's earnings release:

$$\hat{\beta}_{\text{realistic}} = (0.251,\ 0.424,\ -0.100,\ -0.286), \qquad R^2 = 0.873,$$

but Q8 no longer signals long once the lag is respected, leaving only three quarters (Q3, Q5, Q7) with a 66.7% hit rate, and a higher average return (1.03% versus 0.83%) on that smaller, more selective set of trades.

The lesson is not that the lagged, realistic version is simply worse, its R^2 is actually higher and its average signaled return is larger, but that it is a *different* model making *different* trading decisions than the naive version, on the very same underlying quarters. A backtest run on the naive, contemporaneous-card version would report performance for trades the fund could never actually have placed, since the card figures those trades depend on were not yet in hand in that form, regardless of whether that misspecified backtest happens to look better or worse than the honest one. This is why Section 17.3.4 treats delivery timing and revisions as data-quality issues rather than mere modeling nuances: the direction of the resulting bias is not predictable in general, but the fact that ignoring real-world delivery timing and data vintages invalidates the backtest as a measure of tradeable performance is completely general, and point-in-time data discipline, using, for every historical date, only the data vintage that would genuinely have been available on that date, is the standard defense against it.

17.3.2 Feature Engineering for Alternative Data

Raw alternative-data readings, like the raw market and fundamental data of Chapters 2 through 5, are rarely used exactly as delivered; Section 17.3.1 addressed when a reading becomes usable, and this section addresses what should be done to it before it is usable at all.

Seasonal adjustment. Web traffic, foot traffic, and card-transaction volumes all carry strong calendar effects, day-of-week patterns, holiday spikes, and back-to-school or year-end shopping seasons, that have nothing to do with a firm's underlying performance. Comparing a raw reading against the same period one year earlier, exactly what year-over-year growth rates such as Section 17.2.3's car-count growth already do, removes most of this seasonality automatically, provided the calendar aligns cleanly (a retailer's Thanksgiving week, for instance, falls on a different date each year, which can distort a naive year-over-year comparison unless the alignment is done by trading day or holiday-relative position rather than by calendar date).

Peer-relative normalization. A retailer's car-count or web-traffic growth reflects both firm-specific performance and sector-wide conditions, gasoline prices, general consumer spending, weather, that affect every retailer in the sector simultaneously. Subtracting a sector-average benchmark from a firm's own reading isolates the firm-specific component a stock-picking strategy actually cares about.

Suppose sector-average car-count growth for Cascade's peer group across the same eight quarters of Table 17.2 is $(1.0, -0.5, 2.0, 0.3, 2.5, -1.0, 1.5, 0.5)$; subtracting it from Cascade's own car-count growth gives a peer-adjusted series $(1.1, -1.0, 2.8, 0.3, 3.7, -2.0, 2.0, 0.5)$. Refitting Section 17.2.3's regression with this peer-adjusted series as the predictor gives $\hat{\beta} = 1.064$, $\hat{\alpha} = 0.678$, and $R^2 = 0.867$, essentially unchanged from the raw series' $R^2 = 0.870$. In this particular sample sector-wide movements were fairly small relative to Cascade's own idiosyncratic variation, so peer-adjustment does not change the fit much here, but the adjustment matters far more in a period when the whole sector moves together, a broad consumer-spending downturn, a fuel-price spike depressing driving-based foot traffic sector-wide, where an un-adjusted signal would mistake a sector-wide move for firm-specific news.

Wulin.Suo@Queensu.ca

17.3.3 Neural Networks for Structured and Unstructured Data

Section 17.2.3 treated the vendor's car count as a finished number, but the vendor itself had to produce that count from raw satellite pixels, and Section 17.2.4's web-traffic and card figures likewise start from raw logs or transaction records. Chapter 6 introduced the feedforward network, in which every input feeds every hidden unit with its own independent weight; that architecture ignores two kinds of structure alternative data routinely has, the two-dimensional grid structure of an image and the ordered, sequential structure of a time-stamped log, and specialized architectures were developed to exploit each, alongside a third architecture that needs no labeled target at all.

Convolutional Neural Networks for Grid-Structured Data

Suppose a vendor wants to teach a model to count cars automatically in a million-pixel satellite tile rather than paying an analyst to do it by eye. A feedforward network, of the kind Chapter 6 covers, could in principle treat every one of those million pixels as an independent input feature, but it would need a separate learned weight connecting each pixel to each hidden unit, and it would have to relearn from scratch that a cluster of bright pixels in the top-left corner and the identical cluster in the bottom-right corner both mean the same thing, a car, since nothing in a feedforward network's structure tells it that a pattern's meaning does not depend on where in the image it sits.

A convolutional neural network (CNN) exploits this redundancy: a useful pattern in an image, a car's outline, an edge, a texture, can appear anywhere in the frame, so instead of learning a separate weight for every pixel-hidden-unit pair, a CNN learns a small filter (or kernel) and slides it across the entire image, reusing the same weights at every position. This weight sharing is what makes CNNs practical for images: a 1000×1000-pixel satellite tile has a million pixels, far too many for a feedforward network's per-pixel weights to be learned from any realistic training set, but a 2×2 filter has only four weights to learn regardless of image size.

Consider a stylized 4×4 satellite image patch of a single parking lot, with pixel intensities between 0 (dark asphalt) and 1 (a car's bright roof), containing two clusters of parked cars in opposite corners:

$$I = \begin{pmatrix} 0.1 & 0.1 & 0.8 & 0.9 \\ 0.1 & 0.1 & 0.9 & 0.8 \\ 0.7 & 0.8 & 0.1 & 0.1 \\ 0.8 & 0.9 & 0.1 & 0.1 \end{pmatrix}.$$

Convolving this image with a 2×2 averaging filter

$$K = \begin{pmatrix} 0.25 & 0.25 \\ 0.25 & 0.25 \end{pmatrix}$$

at stride 1 (sliding the filter one pixel at a time, computing the elementwise product with each 2×2 patch and summing) produces a 3×3 feature map. The top-left entry, for instance, averages the patch

$$\begin{pmatrix} 0.1 & 0.1 \\ 0.1 & 0.1 \end{pmatrix}$$

to 0.1; the entry one column to the right averages

$$\begin{pmatrix} 0.1 & 0.8 \\ 0.1 & 0.9 \end{pmatrix}$$

to $(0.1 + 0.8 + 0.1 + 0.9)/4 = 0.475$. Carrying this out for all nine positions gives

$$\text{Feature map} = \begin{pmatrix} 0.100 & 0.475 & 0.850 \\ 0.425 & 0.475 & 0.475 \\ 0.800 & 0.475 & 0.100 \end{pmatrix}.$$

The two corner entries where a car cluster sits, 0.850 (top right) and 0.800 (bottom left), stand out clearly above the two empty corners, both 0.100, and above the mixed-boundary middle entries, which is exactly what this filter is designed to do: highlight regions of concentrated brightness.

Wulin.Suo@Queensu.ca

A real, trained CNN learns many such filters simultaneously (edge detectors, corner detectors, texture detectors), stacks several convolutional layers so that later layers respond to increasingly complex combinations of earlier layers' features, and typically applies a ReLU activation and a pooling step (taking, say, the maximum of each 2×2 block of the feature map) after each convolution, reducing the map's size while keeping its strongest responses, before finally feeding the result into feedforward layers of the kind Chapter 6 described to produce a single number, an estimated car count.

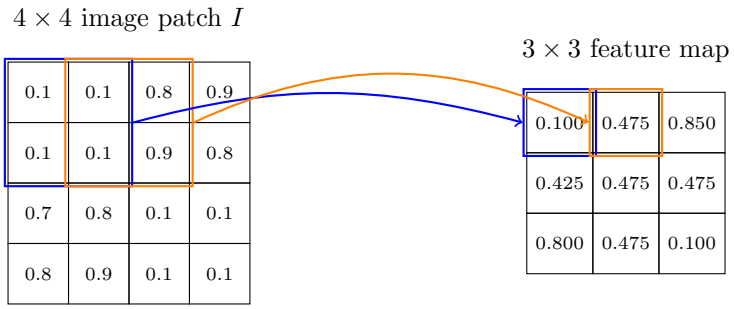


Figure 17.2: Sliding a 2×2 averaging filter across the 4×4 satellite patch I at stride 1

Stacking convolutional layers naively, however, runs into a practical training problem once a network gets deep: gradients computed by backpropagation (Chapter 6) must flow back through every layer, and in a sufficiently deep plain CNN they can shrink toward zero long before reaching the earliest layers, leaving those layers essentially untrained, the vanishing-gradient problem.

The architecture that made very deep CNNs practical, the residual network (ResNet), addresses this with a simple structural change: each block adds a shortcut connection carrying the block's input directly to its output, so the block only has to learn a residual adjustment on top of what already passed through unchanged, rather than having to learn the entire transformation from scratch, letting gradients flow through the shortcut even when a particular block's own learned transformation contributes little. This is the same practical instinct as Chapter 12's shrinkage-based portfolio methods, do not force a model to learn from nothing what could instead be inherited or only mildly adjusted, applied to network depth rather than portfolio weights.

Recurrent Neural Networks for Sequential Data

Suppose an analyst wants to judge whether Cascade's web traffic is accelerating going into its next earnings release using the last several weeks of readings rather than just the most recent one, since a single week's number cannot distinguish a steady climb from a spike that is about to reverse. A feedforward network could take all several weeks as separate input features, but it would treat them as an unordered bag, with nothing in its structure telling it that the reading from three weeks ago came before the reading from last week.

A recurrent neural network (RNN) addresses this: an ordered sequence, such as several successive weeks of web-traffic readings, in which the order itself carries information a feedforward network, which treats its inputs as an unordered set of features, would discard. An RNN processes a sequence one step at a time, maintaining a hidden state h_t that is passed forward and updated at every step,

$$h_t = \tanh(w_x x_t + w_h h_{t-1} + b),$$

so that h_t is, in principle, a function of every input the network has seen so far, $x_1, \ldots, x_t$, not just the current one, giving the network a form of memory that Chapter 7's autoregressive and GARCH recursions share in spirit, each also defines a current value as a function of a prior state plus new information, but that Chapter 6's feedforward network has no analogue for.

Suppose Cascade's e-commerce site shows three successive weekly readings of normalized web-traffic growth in the final three weeks before its next earnings release, $x_1 = 0.5$, $x_2 = -0.2$, $x_3 = 0.9$, and a single-hidden-unit RNN with weights $w_x = 0.6$, $w_h = 0.4$, $b = 0.1$, and initial hidden state $h_0 = 0$. Step by

Wulin.Suo@Queensu.ca

step:

$$h_1 = \tanh(0.6 \times 0.5 + 0.4 \times 0 + 0.1) = \tanh(0.400) = 0.3799,$$
$$h_2 = \tanh(0.6 \times (-0.2) + 0.4 \times 0.3799 + 0.1) = \tanh(0.13196) = 0.1312,$$
$$h_3 = \tanh(0.6 \times 0.9 + 0.4 \times 0.1312 + 0.1) = \tanh(0.69248) = 0.5996.$$

Notice that h_2 is small, close to zero, not because x_2 was extreme (it was mildly negative) but because it also carries a damped memory of h_1; the hidden state genuinely blends past and present rather than reacting to the latest input alone.

Passing the final hidden state through an output layer, $\hat{y} = \sigma(w_y h_3 + b_y)$, where $\sigma(z) = 1/(1 + e^{-z})$ is Chapter 6's sigmoid, with $w_y = 1.2$ and $b_y = -0.3$, gives $\hat{y} = \sigma(1.2 \times 0.5996 - 0.3) = \sigma(0.4195) = 0.6034$, which might, for instance, be interpreted as a 60.3% estimated probability that Cascade's upcoming quarter shows accelerating (rather than decelerating) revenue growth. Figure 17.3 unrolls this same computation across its three time steps, making explicit how each hidden state depends on both the current input and the previous hidden state.

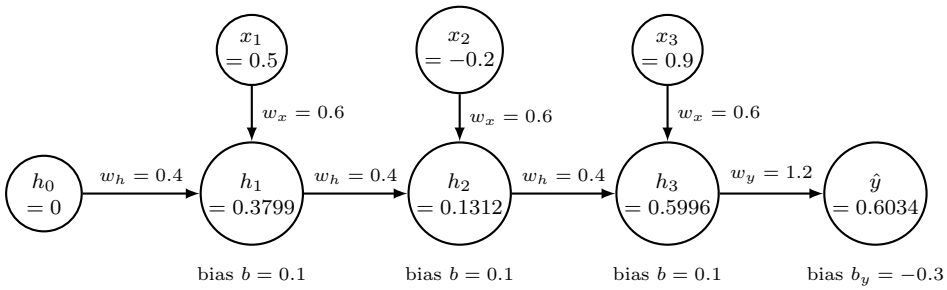


Figure 17.3: The single-hidden-unit RNN unrolled across its three time steps

In practice, a plain RNN's memory decays quickly over long sequences, since each step multiplies the running influence of an early input by another factor less than one, a limitation addressed by the long short-term memory (LSTM) architecture. An LSTM maintains a separate cell state c_t, alongside the hidden state, updated as

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,$$

where

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

is a forget gate controlling how much of the previous cell state to retain,

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

is an input gate controlling how much of a new candidate value

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

to admit, and $\odot$ denotes elementwise multiplication; a third, output gate

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

then controls how much of c_t is exposed as the hidden state

$$h_t = o_t \odot \tanh(c_t).$$

The three gates use σ, whose output lies in $(0, 1)$ and so reads as a fraction retained or admitted, while the candidate uses tanh, whose output lies in $(-1, 1)$ and so carries a signed value to be written into the cell state. The worked example below turns on precisely that difference: from identical arguments it computes

Wulin.Suo@Queensu.ca

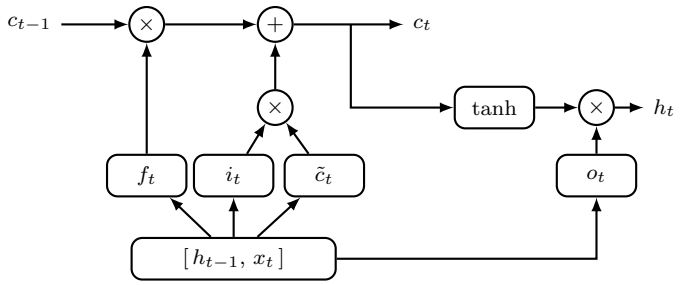


Figure 17.4: The LSTM cell. The cell state crosses the top of the diagram scaled once and added to once, never passed through a squashing function, which is why information survives along it; the three gates below all read the same concatenated $[h_{t-1}, x_t]$.

$i_1 = \sigma(0.25) = 0.5622$ but $\tilde{c}_1 = \tanh(0.25) = 0.2449$. Figure 17.4 puts these six equations in one picture, with the cell state running along the top and the gates feeding it from below.

Because the cell state's update is additive rather than the plain RNN's fully multiplicative recursion, a forget gate near 1 lets information pass across many more time steps largely undamped, directly addressing the memory-decay limitation the single-unit example above illustrates, at the cost of four times as many weights to learn as the plain recurrent unit, one full set each for the forget, input, and output gates and the candidate.

A worked numerical example makes this concrete rather than leaving it as a qualitative claim. Applying a single-unit LSTM to the identical three-week input sequence used for the plain RNN above ($x_1 = 0.5$, $x_2 = -0.2$, $x_3 = 0.9$, $h_0 = 0$, $c_0 = 0$), with illustrative gate weights $w_{f,h} = 0.5$, $w_{f,x} = 0.3$, $b_f = 2.0$ (a deliberately large bias, chosen so the forget gate stays close to 1 and the memory-retention effect is visible) and $w_{i,h} = w_{i,x} = w_{c,h} = w_{c,x} = w_{o,h} = w_{o,x} = 0.5$ with all other biases at 0, the first step gives

$$f_1 = \sigma(2.15) = 0.8957, \qquad i_1 = \sigma(0.25) = 0.5622, \qquad \tilde{c}_1 = \tanh(0.25) = 0.2449,$$
$$c_1 = f_1 c_0 + i_1 \tilde{c}_1 = 0.1377, \qquad o_1 = \sigma(0.25) = 0.5622, \qquad h_1 = o_1 \tanh(c_1) = 0.0769.$$

Continuing this recursion through the second and third steps gives

$$f_2 = 0.8785, \qquad c_2 = 0.0912, \qquad h_2 = 0.0441,$$
$$f_3 = 0.9082, \qquad c_3 = 0.3537, \qquad h_3 = 0.2092;$$

passing h_3 through the same output layer as the plain RNN example ($w_y = 1.2$, $b_y = -0.3$) gives $\hat{y} = \sigma(1.2 \times 0.2092 - 0.3) = 0.4878$.

The forget gates $f_2 = 0.8785$ and $f_3 = 0.9082$ are the mechanism worth isolating: their product, $0.8785 \times 0.9082 \approx 0.7979$, is the fraction of the original cell state c_1 that survives, purely through the forget gate's own multiplicative decay, two additional time steps later, versus only $w_h^2 = 0.4^2 = 0.16$ for the plain RNN's equivalent two-step decay of its hidden state's direct dependence on h_1.

Roughly 80% of c_1's information survives to c_3 in the LSTM, against roughly 16% of the plain RNN's analogous quantity, a concrete, order-of-magnitude illustration of the memory-decay fix the forget gate is designed to provide: not that information never decays in an LSTM, the forget gate is still strictly less than 1, but that a well-trained forget gate can be pushed close enough to 1 that memory persists over many more steps than a plain RNN's fixed recurrent weight allows. Figure 17.5 draws the first step of this same computation as a cell diagram, showing all four gates computed from the identical inputs before they combine into the new cell and hidden states.

Autoencoders for Unsupervised Anomaly Detection

Suppose a data-quality analyst wants to flag whenever a quarter's alternative-data readings look unusual, without knowing in advance what "unusual" means and without a single labeled historical example of a bad reading to train against, an unsupervised problem the CNN and RNN examples above cannot address, since

Wulin.Suo@Queensu.ca

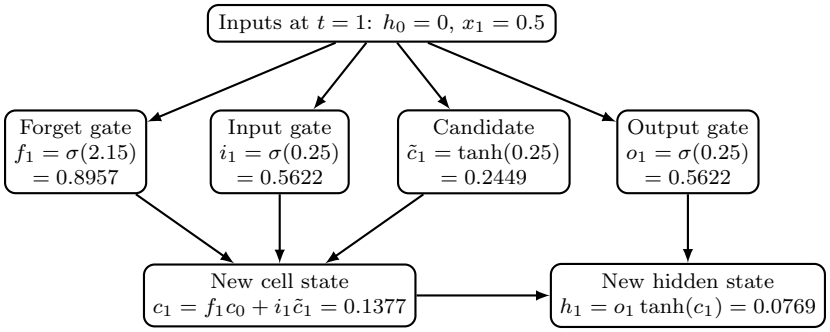


Figure 17.5: The LSTM cell's first time step, computed from the worked example in the text

both are supervised, trained toward a known target, an estimated car count, a probability of accelerating growth.

An autoencoder is trained without any label at all: it learns an encoder that compresses an input down to a small bottleneck representation and a decoder that reconstructs the original input from that bottleneck, with the reconstruction error itself as the only training signal. Once trained on typical, well-behaved data, an autoencoder reconstructs anomalous inputs poorly, since its bottleneck was never shaped to represent whatever made them unusual, which makes reconstruction error a natural anomaly score, extending Chapter 15's classification-based anomaly detection to a setting with no labeled examples of what an anomaly looks like.

Consider a simple linear autoencoder, encoder weights $w = (0.5, 0.3, 0.2)$ applied to Table 17.3's three alternative-data growth rates $x = (\text{car}, \text{web}, \text{card})$ for each quarter, producing a single bottleneck code $z = w \cdot x$, and a tied-weight decoder that reconstructs $\hat{x} = z\,w$. For Q4, $x = (0.6, 1.5, 0.8)$, so $z = 0.5(0.6)+0.3(1.5)+0.2(0.8) = 0.91$, and $\hat{x} = 0.91(0.5, 0.3, 0.2) = (0.455, 0.273, 0.182)$, giving a reconstruction error $\sum(x_i - \hat{x}_i)^2 = (0.6 - 0.455)^2 + (1.5 - 0.273)^2 + (0.8 - 0.182)^2 = 1.909$.

Carrying this out for all eight quarters and expressing each quarter's error relative to its own total energy $\sum x_i^2$ (so that quarters with naturally larger growth rates are not automatically flagged just for being larger) gives relative errors ranging from 39.4% (Q7) to 58.7% (Q4). Q4 stands out as the quarter whose alternative-data pattern the autoencoder reconstructs worst, not because its numbers are the largest in absolute terms, Q5's raw error of 30.9 is far larger, but because Q4's web-traffic growth (1.5%) is unusually large relative to its car-count and card growth (0.6% and 0.8%), a pattern this particular bottleneck direction does not represent well.

One caution on these numbers: the weights $w = (0.5, 0.3, 0.2)$ are illustrative, chosen for easy hand arithmetic, and have not been trained. A trained tied-weight linear autoencoder learns a unit-length weight vector (it recovers the data's first principal direction), and only a unit-length w makes $\hat{x} = z\,w$ a genuine projection. This w has length 0.6164, so every reconstruction above is shrunk toward zero and every error is inflated. Rescaling w to unit length, $(0.8111, 0.4867, 0.3244)$, the minimal change that makes it a proper projection, cuts the relative errors to between 1.6% (Q7) and 33.0% (Q4); the levels fall sharply, but Q4 still stands out as the worst-reconstructed quarter, which is the conclusion the example exists to show.

In practice, this would be the kind of quarter a data-quality analyst should investigate first, is the web-traffic reading itself an error, a one-time promotional spike, or a genuine early signal the other two sources have not yet picked up, before that quarter's data is trusted in a downstream model.

Transformers as a Unifying Architecture

Chapter 16 developed the transformer architecture and its self-attention mechanism (Section 16.4.1) for text, where each token attends to every other token in a sequence. The same mechanism generalizes beyond text: a vision transformer (ViT) divides an image into fixed-size patches, treats each patch the way a transformer treats a word token, and applies self-attention across patches instead of convolution across pixels, letting a satellite image classifier weigh a distant part of the image against the current patch directly, something a

Wulin.Suo@Queensu.ca

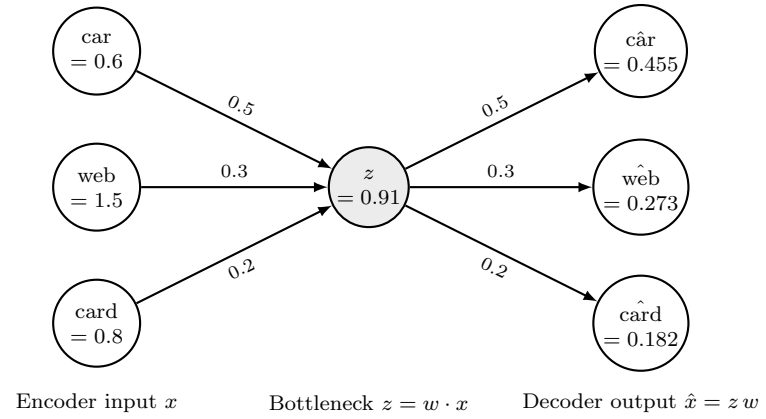


Figure 17.6: The linear autoencoder applied to Q4's three alternative-data growth rates

CNN's local filters only do indirectly, through many stacked layers.

The companion notebook makes this concrete by splitting the same 4×4 satellite image the CNN example convolved into four non-overlapping 2×2 patches, treating each as a token exactly as Chapter 16 treated a word, and computing scaled dot-product self-attention across them. Querying from patch 2 (the top-right car cluster, mean brightness 0.85) against keys built from each patch's own brightness, patch 2 attends to itself with weight 0.6174 and to patch 3 (the bottom-left car cluster, mean brightness 0.80, diagonally the farthest patch in the image) with weight 0.3817, while the two empty patches 1 and 4 (mean brightness 0.10 each) receive a combined weight of only 0.001.

This is the claim above made numerical: patch 2's most relevant neighbor is patch 3, a content-similar patch on the opposite corner of the image, not either of the two empty patches nearer to it, and self-attention identifies that relevance in a single step rather than requiring several stacked convolutional layers to propagate information across that same distance. In current practice, CNNs remain common for smaller or highly structured image tasks (including much car-counting and object-detection work), while transformer-based architectures increasingly dominate both large-scale image classification and the web-traffic and card-transaction sequence modeling that once used RNNs, so that a single architectural family, self-attention, now spans text (Chapter 16), images, and sequences alike.

17.3.4 Data Quality, Bias, and Integration Challenges

Alternative data's advantages, frequency, granularity, and a genuine informational edge before official reporting, come with distinctive quality problems that do not have close analogues in the market and fundamental data earlier chapters used.

Coverage bias. A satellite vendor can only cover stores it has usable imagery for, typically larger, free-standing locations with visible parking lots, so its car-count sample systematically underrepresents smaller-format or mall-based stores, and a model trained only on the covered subset may not generalize to a retailer's business as a whole.

Vendor black-box processing. As Section 17.2.1 noted, a vendor typically sells an already-processed feature, a car count, a traffic index, not raw pixels or logs, so an analyst inherits whatever computer-vision or aggregation errors are baked into that pipeline without being able to inspect or correct them directly; a vendor's undisclosed methodology change between periods can look, in the data, indistinguishable from a genuine change in the underlying business.

Delivery lag and look-ahead bias. A backtest that assumes alternative data was available on the date it describes, rather than the (often later) date it was actually delivered to subscribers, silently uses information that would not have been tradeable in real time; Section 17.2.4's multi-source signal, for instance, is only genuinely useful if all three data sources reach a trading desk before the earnings release they are meant to anticipate, not merely before the quarter ends, and only in the vintage that actually arrived then, not as later revised and backfilled.

Wulin.Suo@Queensu.ca

Licensing cost and alpha decay. High-quality alternative data is expensive, often licensed exclusively or semi-exclusively, and Section 17.5's case documents a genuine 4-5% informational edge around earnings from satellite car counts; as more funds license the same feed and trade on the same signal, that edge is competed away over time in the way Chapter 6 described a known profitable pattern eroding once it becomes widely traded, so an alternative-data edge, like any other, has a shelf life.

Privacy and legal exposure. Card-panel and geolocation data are aggregated and anonymized before sale, but reconstructing individual behavior from supposedly anonymized data has repeatedly proven possible in other domains, so a vendor's privacy practices, and a buyer's own legal exposure, are due-diligence questions that market and fundamental data never raised.

The line between alternative data and material non-public information is not always bright. Card-panel and web-traffic vendors aggregate across many individual cardholders or visitors specifically so that no single company's non-public information is being sold, but the boundary is a matter of legal judgment rather than a fixed technical threshold, and a narrowly enough scoped feed, transaction data from only a handful of a firm's largest customers, say, can approach material non-public information even though it never touches the firm's own systems and no insider ever leaks anything, a genuinely new compliance risk that Chapter 15's insider-trading discussion, framed entirely around information leaking from inside a firm, did not anticipate.

17.4 Applications and Advanced Methods

This part builds a signal from disclosure text, then surveys the deep-learning methods the data invites.

17.4.1 Building an ESG Signal Directly from Disclosure Text

Chapter 12 showed that six major ESG rating providers agree with one another only weakly, an average pairwise correlation of just 38% to 71%, and traced most of that disagreement to providers measuring different underlying facts about a company and weighting them differently, not to noisy measurement of an otherwise agreed-upon concept. Section 17.2.2's build-versus-buy logic applies to this problem exactly as it applies to satellite imagery: rather than purchasing a vendor's already-computed ESG score and inheriting whatever undisclosed methodology produced it, a fund can build its own narrow, transparent environmental-tone signal directly from a company's public disclosures, using the same text-analysis tools Chapter 16 developed for a different purpose.

Suppose Meridian Fabrication, the manufacturer whose ratio analysis Chapter 4 worked through in detail, files a 10-K whose risk-factors section runs to several pages. Chapter 16's Naive Bayes classifier (Section 16.3.2) can be extended with a third training class, Environmental Risk, built from a small set of sentences using vocabulary (emissions, environmental, climate, remediation) that neither the original Credit Risk nor Market Risk class's training sentences ever mention. The retrained classifier then flags four sentences in Meridian's filing as Environmental Risk with high confidence, each keying on at least one environment-specific term the other two classes' training vocabulary lacks entirely:

> S1: "Management remains confident that planned facility upgrades will reduce our emissions, but continued regulatory headwinds could still increase compliance costs."
> S2: "We remain cautious about the pace of emissions reduction given persistent supply chain headwinds affecting our environmental capital projects."
> S3: "Emissions declined for the third consecutive year, reflecting strong progress on our environmental targets despite a challenging regulatory backdrop."
> S4: "Physical risks from climate change, including potential flooding at our coastal facility, remain a weak point in our disaster preparedness planning and a continuing concern for the board."

Applying Chapter 16's Loughran-McDonald dictionary tone score (Section 16.2.3) to each of these four flagged sentences, using the same small positive word set {strong, growth, exceeded, confident, improved} and negative word set {cautious, headwinds, decline, declined, weak, concern, concerns} Chapter 16 used to

Wulin.Suo@Queensu.ca

score its two earnings-call transcripts, gives

$$\begin{aligned}
&\text{S1: } (1-1)/20 = 0.0,\\
&\text{S2: } (0-2)/19 \approx -0.1053,\\
&\text{S3: } (1-1)/19 = 0.0,\\
&\text{S4: } (0-2)/28 \approx -0.0714,
\end{aligned}$$

an average environmental tone score of -0.0442 across the four sentences, a mildly cautious reading of Meridian's own environmental disclosure.

This exercise surfaces a genuinely new data-quality problem, distinct from anything Section 17.3.4 catalogued, that is specific to building ESG signals from generic financial-sentiment tools: the Loughran-McDonald lexicon was built to score revenue and earnings language, where a word like "declined" is reliably bad news, but the same word describes unambiguously good environmental news in sentence S3, emissions falling for a third consecutive year, and the lexicon cannot tell the difference. S3's tone score of 0.0 is, if anything, understating how positive that sentence actually is; a lexicon purpose-built for environmental disclosure, distinguishing a decline in emissions from a decline in revenue, would be needed to fix this, and no such standardized, publicly available dictionary exists yet the way Loughran-McDonald's does for general financial tone.

Comparing this section's result to Chapter 12's worked example makes the deeper point. Chapter 12's three fictional assets were scored on a common, if arbitrary, 0-to-100 composite scale, and its worked example varied only the *weight* providers place on the same three E, S, and G sub-scores, one of three sources of divergence Berg, Kölbel, and Rigobon identify. This section's -0.0442 tone score is not on that scale at all, and is not directly comparable to it: it measures something narrower (the tone of language in one filing's risk-factors section) using an entirely different method (word counting rather than a proprietary composite index), which is the *scope* and *measurement* divergence the same paper identifies as the larger share of why ESG ratings disagree. Building an in-house, transparent signal like this one does not resolve the disagreement documented in Chapter 12; it adds a fourth, differently-scoped measurement to a field that, per that same evidence, already has too many mutually inconsistent ones.

What it buys instead is auditability: unlike a vendor's black-box score, every step from raw sentence to final number in this section can be inspected, recomputed, and defended to a model validator in the way Chapter 8's model-risk-management framework requires.

17.4.2 Advanced Topics: Transfer Learning, Graphs, and Fusion

Three further techniques extend the neural network toolkit of Section 17.3.3 in directions that matter increasingly in real alternative-data practice: reusing an already-trained model instead of training from scratch, exploiting network structure between entities rather than treating each firm in isolation, and combining several data modalities into one representation.

Transfer Learning: Fine-Tuning a Pretrained Vision Backbone

Section 17.3.4 noted that a newly available alternative-data feed typically has only a short history, too little, on its own, to train a large CNN's millions of weights from scratch. Transfer learning sidesteps this: a CNN already trained on a large, general image dataset (which has already learned generic filters for edges, textures, and shapes) is reused as a frozen feature extractor, and only a small final layer, mapping its output features to the task at hand, car count, cloud cover, construction status, is trained on the limited task-specific data available. Because the frozen backbone contributes no gradient updates, this final-layer training is a direct application of Chapter 6's gradient descent to far fewer parameters than training a full network would require.

Suppose a pretrained backbone, frozen, produces a two-dimensional feature vector $f = (0.7, 0.3)$ for a new satellite image patch, and the task is to fine-tune a final logistic layer, currently $w = (0.4, 0.4)$, $b = -0.1$, to predict whether the patch contains a car cluster (label $y = 1$ for this patch). The current prediction is

$$\hat{y} = \sigma(w \cdot f + b) = \sigma(0.4 \times 0.7 + 0.4 \times 0.3 - 0.1) = \sigma(0.30) = 0.5744.$$

Wulin.Suo@Queensu.ca

Following Chapter 6's gradient descent update with learning rate $\eta = 0.5$, the gradient of the log-loss with respect to w is

$$(\hat{y} - y)f = (0.5744 - 1)(0.7, 0.3) = (-0.298, -0.128),$$

and with respect to b is $\hat{y} - y = -0.426$, giving updated weights

$$w \leftarrow (0.4, 0.4) - 0.5(-0.298, -0.128) = (0.549, 0.464)$$

and

$$b \leftarrow -0.1 - 0.5(-0.426) = 0.113.$$

Recomputing the prediction with these updated weights gives

$$\hat{y} = \sigma(0.549 \times 0.7 + 0.464 \times 0.3 + 0.113) = \sigma(0.6365) = 0.6540,$$

closer to the true label of 1 after a single gradient step on a handful of parameters, illustrating why transfer learning lets a fund with only a few hundred labeled satellite patches still fine-tune a usable classifier, rather than needing the millions of labeled images a full CNN would require if trained from scratch.

Graph Neural Networks for Ownership and Counterparty Networks

Chapter 15 modeled a shell-company layering network as a static sequence of hops, one entity moving funds to the next. A graph neural network (GNN) treats such a network as a live computational structure: each node's representation is updated by aggregating its neighbors' representations, so that risk (or any other property) genuinely propagates through the network rather than being assessed for each entity in isolation, the structure alternative data increasingly captures, common addresses, shared directors, correlated transaction timing, that a purely tabular model discards. A single GNN layer updates node v's representation as

$$h_v = \sigma(w_1 x_v + w_2 \cdot \text{mean}_{u \in N(v)}(x_u) + b),$$

where $N(v)$ is the set of v's neighbors in the graph.

Consider a four-node layering chain, $A \to B \to C \to D$, extending Chapter 15's shell-company example, with initial suspicion scores $x_A = 0.9$ (a flagged placement account), $x_B = 0.2$, $x_C = 0.1$, and $x_D = 0.05$ (an apparently unremarkable integration account), and neighbor sets

$$N(A) = \{B\},\ N(B) = \{A, C\},\ N(C) = \{B, D\},\ N(D) = \{C\}.$$

With weights $w_1 = 0.6$, $w_2 = 0.8$, $b = -0.05$, one round of mean-aggregation gives

$$h_B = \sigma\big(0.6 \times 0.2 + 0.8 \times \tfrac{0.9+0.1}{2} - 0.05\big) = \sigma(0.47) = 0.6154$$

and

$$h_C = \sigma\big(0.6 \times 0.1 + 0.8 \times \tfrac{0.2+0.05}{2} - 0.05\big) = \sigma(0.11) = 0.5275,$$

compared to $h_A = \sigma(0.65) = 0.6570$ and $h_D = \sigma(0.06) = 0.5150$. Node B's updated score, 0.615, is far higher than its own raw feature of 0.2 would suggest, purely because it is one hop from the flagged account A, while node C, two hops away, reaches only 0.528, barely above D's 0.515: after a single round, A's signal has not yet arrived there, and a second round of aggregation is needed to carry it that far; a model that scored each account independently, the way Chapter 15's original logistic regression did, would miss this kind of network-propagated risk.

Multi-Modal Fusion via Cross-Attention

A single financial decision often draws on several alternative-data modalities at once, an image feature from Section 17.3.3's CNN, a sentiment embedding from Chapter 16's text pipeline, and a tabular alternative-data summary like Section 17.2.4's regression inputs, and combining them well matters as much as extracting each one individually. Cross-attention, a direct extension of Chapter 16's self-attention mechanism (Section

Wulin.Suo@Queensu.ca

16.4.1), treats one modality's representation as the query and the other modalities as key-value pairs to attend over:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top/\sqrt{d_k})V,$$

exactly as before, but now fusing modalities instead of fusing word tokens.

Suppose a tabular query vector $Q = (0.6, 0.8)$ (summarizing Cascade's alternative-data growth direction) attends over an image modality with key $K_{\text{img}} = (0.9, 0.1)$ and value $V_{\text{img}} = (0.7, 0.2)$, and a text-sentiment modality with key $K_{\text{txt}} = (0.2, 0.9)$ and value $V_{\text{txt}} = (-0.3, 0.6)$ (a mildly cautious sentiment reading). With $d_k = 2$, the scaled dot-product scores are

$$Q \cdot K_{\text{img}}/\sqrt{2} = 0.62/1.4142 = 0.4384$$

and

$$Q \cdot K_{\text{txt}}/\sqrt{2} = 0.84/1.4142 = 0.5940;$$

applying softmax gives attention weights 0.4612 on the image modality and 0.5388 on the text modality. The fused representation is

$$0.4612(0.7, 0.2) + 0.5388(-0.3, 0.6) = (0.161, 0.416),$$

a single vector that leans slightly more on the cautious text signal than on the image signal, and that a downstream model, a trading rule in the spirit of Section 17.2.4, a credit decision, a fraud flag, can consume in place of either modality alone, without the analyst having to hand-specify how much weight each modality deserves.

Self-Supervised Contrastive Pretraining

Section 17.4.2's transfer-learning example assumed a pretrained backbone was already available; contrastive pretraining is one common way such a backbone gets trained in the first place, without any human-provided labels at all, which matters enormously for satellite imagery, where labeled examples (this patch definitely contains three cars) are far scarcer than the raw imagery itself. The idea is to take two randomly augmented views of the same image, a small crop and rotation, say, and train the encoder so that these two views' embeddings are pulled close together (a positive pair), while embeddings of a different image are pushed apart (a negative pair), using only this relative structure, same image versus different image, as the training signal.

Suppose an encoder produces embedding $a = (0.6, 0.8)$ for one crop of a satellite patch, $p = (0.55, 0.83)$ for a second, differently augmented crop of the *same* patch, and $n = (0.1, 0.9)$ for a crop of a genuinely different patch. Using Chapter 16's cosine similarity (Section 16.3.3), $\cos(a, p) = 0.9983$ and $\cos(a, n) = 0.8614$, the same view of the same underlying scene is, as intended, geometrically closer than a different scene.

A contrastive loss converts these similarities into a probability that the positive pair is correctly identified among the candidates,

$$\Pr(\text{positive}) = \frac{e^{\cos(a,p)/\tau}}{e^{\cos(a,p)/\tau} + e^{\cos(a,n)/\tau}},$$

a softmax over similarities exactly analogous to Section 17.4.2's cross-attention softmax, with a temperature parameter τ controlling how sharply the model is pushed to separate positive from negative. With $\tau = 0.1$, $\Pr(\text{positive}) = e^{9.983}/(e^{9.983} + e^{8.614}) = 0.797$, already favoring the true positive pair well above chance (50%) even before any training, and training drives this probability further toward 1 by adjusting the encoder's weights so that same-image crops end up even closer together, and different-image crops even farther apart, than they start.

Once trained this way on a large, unlabeled archive of satellite imagery, the resulting encoder is the frozen backbone Section 17.4.2's transfer-learning example fine-tuned with only a handful of labeled examples, closing the loop between this chapter's two ways of coping with scarce alternative-data labels.

Wulin.Suo@Queensu.ca

17.5 Case Vignette: Satellite Imagery and the Rise of Alternative Data

The parking-lot car-count example this chapter has used throughout is not hypothetical in its origins. In 2009, brothers Tom and Alex Diamond conceived the idea of counting cars in retailer parking lots from satellite imagery to anticipate revenue, and founded RS Metrics to sell the resulting data, beginning with car counts at McDonald's locations and later covering Lowe's, Home Depot, and dozens of other retailers. The approach reached wider attention in 2010, when UBS analyst Neil Currie purchased parking-lot counts for 100 Walmart store locations and published them ahead of Walmart's earnings release, an early, public demonstration that satellite-derived alternative data could anticipate a reported number before the company itself disclosed it. A later academic study using RS Metrics imagery from 2011 to 2017, covering 44 major U.S. retailers including Walmart, Target, Costco, and Whole Foods, confirmed that year-over-year change in parking-lot car counts reliably predicts quarterly sales, and quantified the resulting informational edge at 4% to 5% in the three trading days surrounding a quarterly earnings announcement, a substantial return for such a short window, corroborating this chapter's own worked example (Section 17.2.3) at genuinely large scale.

A separate study of the same satellite coverage's introduction across retailers found that its capital-market effects went beyond simply rewarding the funds that subscribed to it: retailers who gained satellite coverage subsequently saw more informed short-selling activity, less-informed individual investor buying, and lower stock liquidity around their earnings reports, evidence that unequal access to alternative data can widen the information gap between sophisticated and ordinary investors without necessarily making prices more accurate any faster, precisely the access-asymmetry concern Section 17.3.4 raises about licensing cost and exclusivity.

Satellite-derived alternative data extends well beyond retail. Orbital Insight built a comparable business around commodity markets: by measuring the shadows cast by the floating roofs used on many crude-oil storage tanks, a floating roof sits directly atop the stored oil and its height reveals how full the tank is, the firm estimates inventory levels across more than 27,000 tanks worldwide, work that, during the demand shock of the early 2020s pandemic period, revealed global storage utilization running at roughly 55% of an estimated six-billion-barrel total capacity, a supply-side data point no single government agency could have produced as quickly. Today, satellite-based financial data services are estimated to generate over $500 million in annual revenue, within a broader alternative-data market Wall Street is estimated to spend roughly $3 billion on annually, and a single institutional satellite subscription can run from $50,000 to $500,000 per year, underscoring Section 17.3.4's point that this edge is available primarily to well-capitalized participants able to pay for exclusive or near-exclusive access.

17.6 Challenges in Alternative Data and Deep Learning in Finance

Several challenges recur across this chapter's techniques and connect directly to themes from earlier chapters.

A short history limits statistical confidence. Section 17.2.3 and Section 17.2.4 fit regressions on only eight quarters, two years of data, because that is genuinely all the history a newly available alternative-data feed typically has; Chapter 6's warnings about overfitting with few observations relative to model complexity apply with particular force here, since a vendor's dataset is often too new to have lived through a full economic cycle.

Non-stationarity compounds with vendor changes. Chapter 6 noted that financial relationships can shift as markets evolve; alternative data adds a second source of instability on top of that, a vendor can silently change its imagery source, its computer-vision model, or its store coverage, so a model's apparent degradation may reflect a data-generating-process change at the vendor rather than a change in the underlying financial relationship.

Deep architectures trade interpretability for representational power. A CNN's learned filters and an RNN's hidden state are far less directly interpretable than the single regression coefficient of Section 17.2.3, a tension Chapter 18 develops in depth, and one that matters more, not less, once a model's input is something as opaque to manual review as a stack of satellite pixels.

Wulin.Suo@Queensu.ca

Integration is usually the hard part, not modeling. As Section 17.3.4 emphasized, aligning several data sources in time and by firm identity, and confirming each was genuinely available before, not after, the date being modeled, typically consumes more effort in a real alternative-data project than fitting the model itself, the reverse of the balance in the market-data-only settings of Chapters 12 and 13.

Cost and exclusivity concentrate the advantage. Section 17.5's subscription costs mean alternative data's edge accrues disproportionately to well-resourced funds able to license multiple exclusive feeds, an access asymmetry with no real counterpart in the publicly available market and fundamental data most of this book has used.

Model risk governance now has to reach the vendor, not just the model. Chapter 15's three-lines-of-defense framework was built to validate a single model's inputs, outputs, and ongoing performance; an alternative-data pipeline like Section 17.2.4's adds an entire additional layer that framework must now cover, the vendor relationship itself, since a governed model's validation is only as good as the vendor's coverage, delivery timing, and processing methodology sitting upstream of it, none of which the fund directly controls or, in most cases, can fully observe.

17.7 Key Takeaways

Alternative data, satellite imagery, web traffic, card-transaction panels, and the like, fills the row Chapter 1's data-ecosystem table reserved for it: non-traditional sources that proxy for economic activity ahead of its official reporting. Section 17.2.3 showed that even a single such proxy, satellite-derived parking-lot car counts, can explain a large share (87% in the chapter's worked example) of a retailer's quarterly revenue variation, and Section 17.2.4 showed that combining several sources into one regression produces a genuinely useful, if imperfect (an 80% hit rate, not 100%), earnings-surprise signal in the same trading-signal-evaluation framework Chapter 13 established. Before any of this modeling happens, Section 17.3.2 showed that raw alternative-data readings typically need seasonal adjustment and peer-relative normalization to isolate firm-specific signal from calendar effects and sector-wide movements.

Extracting such signals from raw alternative data, rather than from a vendor's already-processed number, requires neural network architectures beyond Chapter 6's plain feedforward network: convolutional neural networks (Section 17.3.3) exploit an image's grid structure through weight-shared filters, recurrent neural networks exploit a sequence's ordering through a hidden state carried from step to step, and transformer-based self-attention, introduced for text in Chapter 16, increasingly unifies both roles. None of this changes the fact, developed in Section 17.3.4 and the chapter's closing challenges, that alternative data carries its own distinctive quality problems, coverage bias, vendor black-box processing, look-ahead bias from delivery lag, and cost-driven alpha decay, that a modeler must treat as seriously as the modeling itself, a lesson this chapter's case vignette on satellite-derived retail and commodity data illustrated at real-world scale.

Section 17.3.1 showed concretely that respecting real-world data-delivery lag changes which trades a model actually recommends, not merely how good its backtest looks, and Section 17.3.3's autoencoder showed that the same deep learning toolkit that extracts a signal can also be turned, unsupervised, on the data itself to flag a quarter worth double-checking before it is trusted. Section 17.4.1 applied this same lesson to a specific and increasingly important alternative-data category: rather than buying a vendor's already-computed ESG score, whose disagreement with competing providers Chapter 12 documented at 38% to 71% pairwise correlation, a fund can build a narrow, auditable environmental-tone signal directly from a company's own disclosures using Chapter 16's Naive Bayes classifier and Loughran-McDonald tone score, at the cost of adding one more, differently-scoped measurement to a field that already has too many mutually inconsistent ones rather than resolving the disagreement outright.

Section 17.4.2 went one level deeper still: transfer learning lets a CNN's frozen, already-learned features be fine-tuned on a small alternative-data sample rather than requiring millions of labeled images, contrastive pretraining shows how that frozen backbone can itself be trained with no labels at all, graph neural networks let risk propagate across a network of related entities the way Chapter 15's static layering chain could only describe, not compute, and cross-attention, the same mechanism Chapter 16 used to fuse word tokens, fuses image, text, and tabular alternative-data modalities into one representation without an analyst having to hand-specify how much each deserves.

Wulin.Suo@Queensu.ca

17.8 Suggested Exercises

1. Using Section 17.2.3's method, refit the regression after adding a ninth quarter with car-count growth of -2.0% and revenue growth of -1.5%, and report the new slope, intercept, and R^2.

2. Explain, using Table 17.2, why an R^2 of 0.870 from a single alternative-data proxy is considered strong in this context, contrasting it with Chapter 6's discussion of low signal-to-noise ratios in financial prediction generally.

3. Using Section 17.2.4's fitted coefficients, compute the predicted surprise for a tenth quarter with car, web, and card growth of -2.0%, -1.0%, and -1.5% respectively, and state whether the resulting signal is long or flat.

4. Explain, using Q5 in Table 17.3, why a strongly positive fitted surprise (2.46%) can still be followed by a negative next-day return (-0.4%), and describe one piece of earnings-day information the alternative-data model in this chapter cannot capture.

5. Using Section 17.3.3's convolution example, recompute the 3×3 feature map using a 2×2 filter of all 0.5s instead of all 0.25s, and explain how the result relates to the original feature map.

6. Using the same convolution example, apply 2×2 max pooling (taking the maximum of each non-overlapping 2×2 block) to the original 3×3 feature map's top-left 2×2 block, and state the resulting pooled value.

7. Using Section 17.3.3's RNN example, recompute h_1, h_2, and h_3 if w_h is increased from 0.4 to 0.8 (holding w_x, b, and the inputs fixed), and explain in words how a larger w_h changes how much the hidden state is influenced by earlier time steps versus the current input.

8. Explain, in a short paragraph, why a convolutional neural network's weight sharing makes it far more practical than a plain feedforward network for a 1000×1000-pixel satellite image, referencing the parameter-count argument in Section 17.3.3.

9. Using this chapter's case vignette, explain why the 2010 UBS Walmart parking-lot report is a natural precedent for Section 17.2.3's worked example, and describe, in your own words, why the same informational edge tends to shrink as more funds license the same data source.

10. Pick two of the data-quality issues in Section 17.3.4 and describe a concrete scenario, not already used in the text, in which that issue could cause an alternative-data-based trading or credit model to fail despite technically correct code.

11. Compare this chapter's satellite car-count regression (Section 17.2.3) to Chapter 5's binomial and Black-Scholes-Merton option-pricing models in terms of what each treats as the source of uncertainty, and explain why alternative-data models are typically evaluated by out-of-sample predictive accuracy rather than by no-arbitrage argument.

12. Using Section 17.3.1's method, explain in your own words why the realistic, lagged-card model's higher R^2 (0.873 versus 0.837) does not mean it is simply the better trading model, referencing the difference in which quarters each version actually signals.

13. Using Section 17.3.2's method, recompute the peer-adjusted regression's slope and R^2 if the sector-average car-count growth for Q5 is revised from 2.5% to 5.0% (a sector-wide surge that quarter), holding all other quarters fixed, and explain why the peer-adjusted model's R^2 falls even though Cascade's own reported revenue growth has not changed.

14. A vendor offers a trial dataset covering only the three quarters (of Table 17.3) in which its data predicted the following quarter's earnings surprise correctly. Using Section 17.2.2's discussion, explain what is wrong with evaluating the vendor's data quality using only this trial.

Wulin.Suo@Queensu.ca

15. Using Section 17.2.2's cost-benefit method, recompute the expected net annual benefit if the fund can only deploy a \$20 million position per signaled event (instead of \$50 million), holding the signal frequency, edge, and data costs fixed, and state whether the subscription is still worth its \$400,000 annual cost.

16. Using Section 17.3.3's autoencoder example, compute the reconstruction error and relative error for Q2 (car $= -1.5$, web $= -0.5$, card $= -1.0$) using the same encoder/decoder weights $w = (0.5, 0.3, 0.2)$, and state whether Q2 would be flagged ahead of Q7 (relative error 39.4%) for data-quality review.

17. Using Section 17.4.2's transfer-learning example, perform a second gradient-descent step starting from the updated weights $w = (0.549, 0.464)$, $b = 0.113$, and report the new prediction, confirming that it continues to move toward the true label of 1.

18. Using Section 17.4.2's graph neural network example, compute h_A and h_D's updated scores using the same weights, and explain, using the network's chain structure $A \to B \to C \to D$, why h_D's score barely moves from its raw feature while h_B's moves substantially.

19. Using Section 17.4.2's cross-attention example, recompute the fused output if the text modality's key becomes $K_{\text{txt}} = (0.8, 0.2)$ (a much less distinctive key vector, more similar to K_{img}), and explain in words why the two attention weights move closer to 50/50.

20. Explain, in a short paragraph, why a graph neural network's mean-aggregation step (Section 17.4.2) is a more direct computational analogue of "guilt by association" than Chapter 15's original, per-account fraud-scoring logistic regression.

21. Using Section 17.4.2's contrastive-pretraining example, recompute Pr(positive) if the temperature τ is raised from 0.1 to 1.0 (holding the two cosine similarities fixed), and explain in words why a higher temperature makes the model's implied preference for the true positive pair less extreme.

22. Using Section 17.3.3's LSTM worked example, recompute the forget gate f_1 and the resulting fraction of c_1 surviving to c_3 (that is, $f_2 \times f_3$) if the forget-gate bias b_f were 0.5 instead of 2.0, holding every other weight fixed, and explain why a smaller bias makes the LSTM behave more like the plain RNN example earlier in this section.

23. Explain, in your own words, why the plain RNN's memory-decay problem is a consequence of its hidden state being fully overwritten by a multiplicative recursion at every step, while the LSTM's cell state update is additive, and describe what would happen to the LSTM's own memory retention if its forget gate were trained to a value close to 0 instead of close to 1.

24. Using Section 17.4.1's method, recompute the tone score for a fifth sentence, "The board remains confident that emissions will decline further even as regulatory headwinds intensify," using the same positive and negative word lists, and explain why this sentence's mixed vocabulary makes its tone score a poor summary of whether the sentence is good or bad news.

25. Explain, in your own words, why Section 17.4.1's in-house tone score and Chapter 12's vendor-style ESG scores are not directly comparable even though both are commonly described as "an ESG measurement," and describe which one of Berg, Kölbel, and Rigobon's three sources of rating divergence, scope, measurement, or weight, this incomparability illustrates.

26. **Challenge (real data).** Download at least two years of weekly Google Trends search-interest data (via `pytrends` or manually from trends.google.com, no API key required) for a public retailer's brand name, and quarterly reported revenue for the same company from the SEC EDGAR XBRL `companyfacts` API (as in Chapter 4's challenge exercise). (a) Following Section 17.2.3's method, regress the company's quarter-over-quarter revenue growth on the average quarter-over-quarter growth in search interest, and report the fitted slope, intercept, and R^2. (b) Using Section 17.3.1's distinction between a realistic, lagged signal and a look-ahead-biased one, identify what reporting lag your search-interest data is actually available at relative to the revenue figure it is meant to predict, and explain whether your regression in (a) would still be usable in real time. (c) Following Section 17.2.2's cost-benefit framework,

Wulin.Suo@Queensu.ca

discuss one alternative-data quality issue from Section 17.3.4 other than look-ahead bias that is specific to Google Trends data, such as index normalization or query ambiguity, and explain how it could produce a spuriously strong or weak regression fit.

17.9 Further Reading

- Berkeley Haas School of Business, "How Hedge Funds Use Satellite Images to Beat Wall Street, and Main Street." A research summary of the RS Metrics parking-lot car-count studies underlying Section 17.2.3 and this chapter's case vignette.
- Katona, Painter, Patatoukas, and Zeng, "On the Capital Market Consequences of Big Data: Evidence from Outer Space," *Journal of Financial and Quantitative Analysis*. The academic study of RS Metrics satellite imagery (2011-2017, 44 U.S. retailers) quantifying the earnings-window informational edge discussed in Section 17.5.
- Feng and Fay, "An Empirical Investigation of Forward-Looking Retailer Performance Using Parking Lot Traffic Data Derived from Satellite Imagery," *Journal of Retailing*. A complementary study of satellite-derived parking-lot data's power to predict retailer sales, underlying Section 17.2.3's worked example.
- LeCun, Bengio, and Hinton, "Deep Learning," *Nature*. A survey covering convolutional and recurrent neural network architectures underlying Section 17.3.3.
- Goodfellow, Bengio, and Courville, *Deep Learning*. A comprehensive textbook treatment of convolution, recurrence, and the attention mechanisms Chapter 16 and Section 17.3.3 both draw on.
- Hochreiter and Schmidhuber, "Long Short-Term Memory," *Neural Computation*. The original paper introducing the LSTM architecture mentioned as an extension of Section 17.3.3's plain recurrent network.
- Dosovitskiy et al., "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale," *International Conference on Learning Representations*. The original vision transformer paper underlying Section 17.3.3's discussion of transformers as a unifying architecture.
- Kolanovic and Krishnamachari, "Big Data and AI Strategies," J.P. Morgan. An industry survey of alternative data sources and their use in systematic investing, covering the categories in Table 17.1.
- Berg, Kölbel, and Rigobon, "Aggregate Confusion: The Divergence of ESG Ratings," *Review of Finance*. The empirical study of cross-provider ESG rating disagreement discussed in Chapter 12 and revisited in Section 17.4.1 from the standpoint of building an in-house text-derived signal rather than buying a vendor's score.
- Katona, Painter, Patatoukas, and Zeng, "On the Capital Market Consequences of Big Data: Evidence from Outer Space" (same source as above). Also documents the short-selling, liquidity, and information-asymmetry effects of satellite coverage discussed in this chapter's expanded case vignette.
- Hinton and Salakhutdinov, "Reducing the Dimensionality of Data with Neural Networks," *Science*. A foundational paper on autoencoders, underlying Section 17.3.3's reconstruction-error anomaly-detection example.
- Pan and Yang, "A Survey on Transfer Learning," *IEEE Transactions on Knowledge and Data Engineering*. A comprehensive survey of the pretrained-backbone fine-tuning approach worked through in Section 17.4.2.
- Kipf and Welling, "Semi-Supervised Classification with Graph Convolutional Networks," *International Conference on Learning Representations*. A foundational paper on the graph neural network mean-aggregation update used in Section 17.4.2's shell-company network example.

Wulin.Suo@Queensu.ca

- Chen, Kornblith, Norouzi, and Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," *International Conference on Machine Learning*. The SimCLR paper underlying Section 17.4.2's contrastive-pretraining worked example.
- He, Zhang, Ren, and Sun, "Deep Residual Learning for Image Recognition," *IEEE Conference on Computer Vision and Pattern Recognition*. The original ResNet paper underlying Section 17.3.3's discussion of shortcut connections and the vanishing-gradient problem in deep CNNs.

Wulin.Suo@Queensu.ca

Chapter 18

Explainability, Fairness, and Regulation

18.1 Introduction

Every applied chapter so far has, at some point, flagged the same unresolved obligation and deferred it here. Chapter 1 warned that a model too opaque to explain to a risk committee is often not deployable at all, whatever its statistical performance. Chapter 5 showed that collapsing two defensible valuations into one confident-looking number can hide a genuine, economically meaningful disagreement, and named preserving that kind of interpretability as a theme the book would return to directly. Chapter 6 noted that regulation constrains how opaque a credit or insurance model can be, and that the importance of interpretability is itself one of the criteria for choosing among competing algorithms.

Chapter 15 observed that a suspicious activity report or an account closure decision frequently must be explained to a regulator or a customer, limiting how much a fraud or anti-money-laundering program can lean on its least interpretable models. Chapter 16 made the same point about a trading decision or disclosure classification driven by an attention-based language model. Chapter 17 showed that a convolutional or recurrent network's learned filters and hidden states are far less directly inspectable than a single regression coefficient, a tension that only sharpens as a model's input becomes something as opaque to manual review as a stack of satellite pixels. This chapter is where all of these deferred obligations are finally paid off, not as an afterthought bolted onto a finished model, but as a technical and regulatory discipline with its own methods, worked examples, and open problems.

The chapter is organized around three questions a financial institution must answer before, during, and after deploying a model, mirrored in its three planned subsections: what can be said about *why* a model produced a given output (interpretable and explainable AI), whether the model treats people and groups *fairly* under one or more competing legal and ethical definitions of fairness (ethical and regulatory considerations), and whether the model keeps working as intended once it is deployed, monitored, and eventually attacked or gamed (robustness, governance, and responsible AI).

Twelve worked examples run throughout, each reusing a model or dataset from an earlier chapter rather than inventing an isolated one, because explainability and fairness are properties of a model's actual behavior on real cases, not abstract add-ons. Permutation feature importance and partial dependence work on Chapter 15's fraud-scoring logistic regression; an exact Shapley-value decomposition and a counterfactual explanation address Chapter 9's declined loan applicant; a locally interpretable surrogate model is built around Chapter 6's toy feedforward network. A two-group fairness audit, a calibration-by-group check, and adverse-action reason codes rest on the same credit-scoring model; an adversarial "structuring" perturbation against the fraud model extends Chapter 15's money-laundering vocabulary to model evasion directly; and a population stability index quantifies score drift as a complement to Chapter 15's precision-drift monitoring. Going a level deeper still, a cost-weighted algorithmic-recourse comparison and an integrated-gradients attribution revisit Chapter 6's network a second time, to show exactly where a naive local-gradient shortcut fails and a proper path integral does not.

Wulin.Suo@Queensu.ca

A real-world case vignette, the 2019 Apple Card gender-bias controversy and its regulatory aftermath, ties the chapter's methods back to a dispute that was, at its core, the kind of explainability and fairness question this chapter formalizes, and a second vignette on Upstart Network's regulatory engagement with the CFPB shows the same underlying concerns handled proactively rather than after the fact.

18.2 Why Explanation Matters

An unexplained model is not merely opaque; in lending and insurance it is often illegal.

18.2.1 Why Financial AI Needs a Theory of Explanation

Suppose a bank's model validation team asks a data scientist why the credit model just denied a particular applicant, and the honest answer is "because that is what several hundred decision trees, averaged together, produced." That answer is true, but it satisfies no one: not the applicant, who is legally entitled under fair-lending rules to specific reasons for the denial; not the regulator, who needs some way to verify the model is not proxying for a protected characteristic; and not the risk committee, who cannot sign off on a decision it cannot itself understand.

Two related but distinct words recur throughout this chapter in response to this problem, and are worth fixing precisely before using them interchangeably, as casual usage often does. **Interpretability** is a property of a model itself: a model is interpretable to the degree that a human can understand, without any additional tool, how its inputs map to its outputs, simply by inspecting its structure. A linear regression's coefficients, Chapter 6's credit-spread regression among them, are interpretable in this direct sense: the coefficient itself is the answer to "how much does the prediction change per unit change in this feature."

Explainability, by contrast, is a property of a method applied *after* a model is built, regardless of the model's own structure: an explanation method takes a model, interpretable or not, and produces a human-comprehensible account of a specific prediction or of the model's behavior in general. A deep neural network is not interpretable in the first sense, its millions of weights admit no direct reading, but it can still be explained, approximately, using the post-hoc methods this chapter develops. Every explanation method in this chapter falls into one of four cells formed by two independent distinctions, illustrated in Figure 18.1: whether it explains the model's behavior in general (*global*) or a single prediction (*local*), and whether it is built into the model's own structure (*intrinsic*) or applied afterward to an already-fit model (*post-hoc*).

	Intrinsic	**Post-hoc**
Global	Linear/logistic regression coefficients (Ch. 6, 8); a single decision tree	Permutation importance; partial dependence (Section 18.3.1)
Local	One applicant's own linear score decomposition (additive by construction)	Shapley values; LIME (Sections 18.3.2, 18.3.3)

Figure 18.1: A taxonomy of explanation methods along two axes: global versus local, and intrinsic versus post-hoc

The trade-off this taxonomy makes visible is not a minor implementation detail; it is the central tension of this entire chapter. Chapter 6 already observed that the most flexible, highest-capacity models, large ensembles, deep neural networks, and transformers (Chapter 16) among them, are typically also the least directly interpretable, while the most interpretable models, a linear regression, a shallow decision tree, are typically less flexible and can fit a smaller class of relationships. Post-hoc explanation methods do not

Wulin.Suo@Queensu.ca

eliminate this trade-off; they manage it, by producing an approximate, human-readable account of a black-box model's behavior without changing the underlying model at all. That approximation is genuinely useful, a bank can keep the gradient-boosted model that best predicts default while still producing an adverse-action notice for a declined applicant, but it is an approximation, and Section 18.3.3's worked example shows concretely both how well such an approximation can work near the point it is built around and how it can degrade away from that point.

18.2.2 Intrinsically Interpretable Models Revisited

Before turning to the post-hoc methods that occupy the rest of this chapter, it is worth taking seriously the option of simply choosing a model that needs no post-hoc explanation at all, the left column of Figure 18.1. Chapter 6 already introduced the two workhorses of this column: linear and logistic regression, whose coefficients are themselves the explanation, and a single decision tree, whose sequence of if-then splits can be read directly as a chain of human-comprehensible rules ("if debt-to-income exceeds 30% and credit history is under two years, predict default"). Chapter 9's credit-scoring model and Chapter 15's fraud-scoring model, both logistic regressions, were deliberately built this way, and Section 18.3.2's exact, closed-form Shapley decomposition was only possible because of it; a bank that can accept a small accuracy cost in exchange for this kind of direct interpretability avoids the entire post-hoc apparatus this chapter otherwise requires.

Two refinements extend this intrinsic column considerably further than a single linear model or a single tree, without leaving it. A **generalized additive model (GAM)** replaces logistic regression's linear combination $\beta_0 + \sum_j \beta_j x_j$ with a sum of flexible, individually plotted functions, $\beta_0 + \sum_j f_j(x_j)$. This lets each feature's relationship to the outcome be nonlinear, a U-shaped effect of age on default risk, say, rather than a single coefficient forcing a straight line, while keeping the model additive across features. Each f_j can therefore still be inspected and explained on its own, in the same global, intrinsic sense as a linear coefficient. A **credit scorecard**, the industry-standard format underlying Chapter 9's 850-to-300 score transformation, takes a related approach in the opposite direction. It bins each feature into a small number of ranges (debt-to-income under 20%, 20-30%, over 30%, for instance), assigns each bin a small integer number of points from a table, and sums the points across features to produce the final score. That is a format a loan officer, an auditor, or a regulator can verify by hand, addition and table lookups only, with no matrix algebra or floating-point evaluation at all. Both formats trade away some of a fully flexible model's predictive power, the trade-off Section 18.2.1 named directly, in exchange for landing entirely in the intrinsic column and needing none of Sections 18.3.1 through 18.3.3's machinery, which is precisely why scorecards remain the dominant format for consumer credit underwriting in practice even though tree ensembles and neural networks routinely outperform them on held-out accuracy alone.

18.3 Explanation Methods

This part works the three standard families numerically on models built earlier in the book.

18.3.1 Global Explanations: Permutation Importance and Partial Dependence

Suppose a risk manager wants a one-sentence answer to which of the fraud model's three features it actually relies on, before deciding whether it is safe to keep purchasing all three data feeds going forward, without asking a data scientist to walk through a single coefficient. A global explanation answers this by describing a model's behavior across its entire input space, or at least across a representative dataset, rather than for one specific case.

Permutation feature importance asks a direct empirical question: how much does the model's performance degrade if a single feature's values are shuffled (permuted) across the dataset, breaking whatever real relationship that feature had with the outcome while leaving every other feature, and the true labels, untouched? A feature whose permutation causes a large performance drop is one the model relies on heavily; a feature whose permutation changes nothing is one the model could apparently do without, at least on this data.

Wulin.Suo@Queensu.ca

Recall Chapter 15's fraud-scoring logistic regression, fit to ten transactions with three features (amount z-score, distance from home in hundreds of kilometers, and a late-night indicator) and coefficients $\hat{\beta}_0 \approx -2.57$, $\hat{\beta}_1 \approx 0.83$, $\hat{\beta}_2 \approx 0.15$, $\hat{\beta}_3 \approx 0.82$, achieving 90% accuracy and an F_1 of 0.857 at its 0.5 threshold.

Permuting a feature means reassigning its ten values to the ten transactions in a different order; the clearest way to do this by hand is simply to reverse the column, so that transaction 1 receives transaction 10's value for that feature, transaction 2 receives transaction 9's value, and so on, while every other feature and every actual-fraud label stays exactly where it was. Table 18.1 reports the resulting accuracy and F_1 for each of the three features permuted this way, one at a time.

Feature permuted	Accuracy	Precision	F_1
(baseline, no permutation)	0.900	1.000	0.857
Amount z-score	0.300	0.000	undefined
Distance from home	0.900	1.000	0.857
Late-night indicator	0.900	1.000	0.857

Table 18.1: Permutation feature importance for the fraud-scoring model

Reversing the amount z-score column is catastrophic: accuracy collapses from 90% to 30%, and the model's three flagged transactions after permutation (5, 8, and 10) are all false positives, none of the four genuine frauds is caught, so precision and recall both fall to zero and F_1 is undefined (conventionally reported as zero). Reversing either the distance column or the late-night column alone, by contrast, changes not a single one of the ten predictions: accuracy and F_1 are identical to the unpermuted baseline.

The conclusion is not that distance and late-night timing are irrelevant to fraud in general, Chapter 15's own coefficients show both raise predicted fraud probability, but that for this specific ten-transaction sample, the amount z-score so dominates the linear combination that no rearrangement of the other two features' values happens to push any transaction across the 0.5 threshold in either direction. This is the caveat permutation importance always carries: it measures a feature's contribution to performance *on the data at hand*, not some feature's context-free importance, and a feature that appears unimportant here could easily prove decisive on a different, larger sample where its values interact more consequentially with the others.

A complementary global tool, **partial dependence**, asks a different question: holding the joint distribution of every other feature fixed at its empirical values, how does the model's average prediction change as one feature of interest is swept across a range of values? Formally, the partial dependence of the fraud model on the amount z-score z is $\mathrm{PD}(z) = \frac{1}{n}\sum_{i=1}^{n} \sigma\big(\hat{\beta}_0 + \hat{\beta}_1 z + \hat{\beta}_2 d_i + \hat{\beta}_3 n_i\big)$, averaging over the ten transactions' actual distance and late-night values while only z is varied. Table 18.2 evaluates this at six values of z.

Amount z-score	-1	0	1	2	3	4
Average predicted probability	0.095	0.185	0.320	0.489	0.663	0.807

Table 18.2: Partial dependence of predicted fraud probability on amount z-score

The curve crosses the 0.5 classification threshold between $z = 2$ and $z = 3$, at approximately $z \approx 2.07$, meaning that an otherwise-typical transaction (average distance and late-night status across the sample) is predicted fraudulent once its amount is roughly two standard deviations above the cardholder's usual spending. Partial dependence and permutation importance answer different questions and can disagree: a feature can have a smooth, informative partial-dependence curve while still showing zero permutation importance on a specific sample, exactly as distance did above, since partial dependence describes the model's functional form directly while permutation importance describes how much a specific dataset's performance metric happens to depend on that feature.

Wulin.Suo@Queensu.ca

18.3.2 Local Explanations: Shapley Values for a Single Decision

A global explanation tells a modeler or regulator how a model behaves in general; it does not answer the question an individual declined applicant actually asks, which is why *my* application was denied. A **Shapley value**, borrowed from cooperative game theory (Shapley, 1953) and popularized in machine learning as SHAP (SHapley Additive exPlanations), answers this question by fairly dividing credit for a single prediction's deviation from some baseline among the features that produced it. For a model that is linear in its inputs, the case for Chapter 9's credit-scoring logistic regression's log-odds, the Shapley value has a particularly clean closed form. The contribution of feature j is simply $\hat{\beta}_j(x_j - \bar{x}_j)$, the feature's coefficient times its deviation from a reference baseline. These contributions sum *exactly* to the difference between the individual's predicted log-odds and the baseline's, a property called efficiency that any valid additive attribution must satisfy.

Recall Chapter 9's credit-scoring model,

$$z = -12.152 + 0.530\,(\text{Debt/Income}) - 0.231\,(\text{Credit History})$$

(coefficients shown rounded; the fitted model itself carries full floating-point precision), applied to an applicant with a 28% debt-to-income ratio and 4 years of credit history. To decompose this prediction, a baseline is needed: a natural, already-established choice is the mean applicant in Chapter 6's six-applicant reference population (debt-to-income $\bar{x}_1 = 26.33\%$, credit history $\bar{x}_2 = 5.17$ years). Comparing the two,

$$\begin{aligned} \text{Applicant:} \quad & z \approx 1.761,\ \Pr(\text{Default}) \approx 85.3\%,\ \text{score} \approx 381, \\ \text{Baseline:} \quad & z_{\text{baseline}} \approx 0.608,\ \Pr(\text{Default})_{\text{baseline}} \approx 64.7\%,\ \text{score} \approx 494, \end{aligned}$$

the applicant's score comfortably below most lenders' approval cutoffs, the baseline itself a subprime score but far less severe.

The Shapley decomposition of the gap between them is

$$\underbrace{1.761}_{z_{\text{applicant}}} = \underbrace{0.608}_{z_{\text{baseline}}} + \underbrace{0.530\,(28 - 26.33)}_{\text{debt-to-income contribution} = 0.883} + \underbrace{(-0.231)\,(4 - 5.17)}_{\text{credit-history contribution} = 0.270},$$

and indeed $0.608 + 0.883 + 0.270 = 1.761$ exactly, up to rounding, verifying the efficiency property directly. Both features push the applicant's predicted risk above the baseline: the debt-to-income ratio is higher than the reference population's average (more leverage, more risk) and the credit history is shorter than average (less seasoning, more risk). But debt-to-income's contribution (0.883) is more than three times credit history's (0.270). That identifies debt-to-income as the dominant driver of this applicant's elevated predicted default risk, not merely a contributing factor tied for first place. Figure 18.2 draws this decomposition as a waterfall chart, the standard way a SHAP explanation is presented in practice.

From Shapley Values to Adverse Action Reason Codes

This decomposition is not merely an academic exercise: it is, in substance, exactly what U.S. law already requires a lender to produce. The Equal Credit Opportunity Act and its implementing Regulation B require that a lender who denies credit furnish the applicant with the specific, principal reasons for the denial, an *adverse action notice*, rather than a bare rejection. Ranking the Shapley contributions above by magnitude produces precisely this: "high debt-to-income ratio relative to similar applicants" as the principal reason (contribution 0.883), and "limited length of credit history" as a secondary, contributing reason (contribution 0.270). This works cleanly here because the underlying model is linear in the log-odds, so the Shapley value has an exact closed form; for a nonlinear model such as a gradient-boosted ensemble, the same additive decomposition can still be computed, using the general (and considerably more expensive) Shapley formula that averages a feature's marginal contribution over every possible ordering of the other features, which is precisely why fast approximations to full Shapley computation, and simpler surrogate methods like Section 18.3.3's, remain an active area of applied research.

Wulin.Suo@Queensu.ca

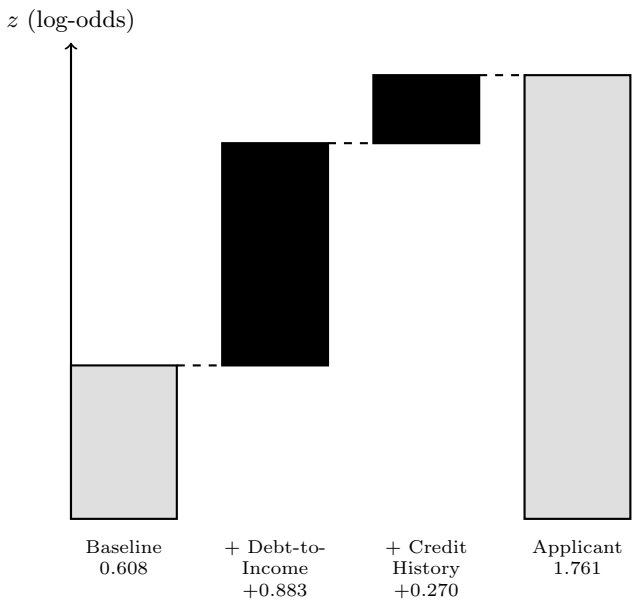


Figure 18.2: The Shapley decomposition as a waterfall chart

Counterfactual Explanations: How Much Would Need to Change?

A closely related, and often more actionable, form of explanation answers a different question: not why was I denied, but what is the smallest change that would have gotten me approved? Suppose this lender approves any applicant with a credit score of at least 620, equivalent to

$$\Pr(\text{Default}) \leq \frac{850 - 620}{550} \approx 41.8\%,$$

or $z \leq -0.330$. Holding the applicant's 4 years of credit history fixed and solving

$$-12.152 + 0.530\,(\text{DTI}) - 0.231(4) = -0.330$$

for debt-to-income gives DTI $\approx 24.05\%$, a reduction of about 3.95 percentage points from the applicant's actual 28%. This *counterfactual explanation*, reduce your debt-to-income ratio by roughly four percentage points, holding everything else fixed, and this same model would approve you, is often more useful to a declined applicant than a bare coefficient or a Shapley value. It states a concrete, achievable target rather than merely a ranked list of contributing factors. It does share the same important caveat as Section 18.3.3's local surrogate: it describes what would happen if only debt-to-income changed and nothing else about the applicant's profile shifted along with it, an assumption that is convenient for exposition but not guaranteed to hold in reality.

18.3.3 Local Surrogate Models: LIME for a Genuinely Nonlinear Model

Shapley values have an exact closed form for Chapter 9's credit-scoring model only because that model is linear in the log-odds; most of the flexible models this book has introduced, Chapter 6's ensembles, Chapter 17's CNNs and RNNs, are not. **LIME** (Local Interpretable Model-agnostic Explanations) handles this case differently: rather than working out an exact decomposition analytically, it treats the model as a black box, samples a handful of points in the neighborhood of the specific input to be explained, records the black box's predictions at each sampled point, and fits a simple, interpretable model, ordinarily a weighted linear regression, to those sampled input-output pairs. The fitted local model's coefficients are then reported as the explanation, valid, by construction, only in the neighborhood the samples were drawn from.

Wulin.Suo@Queensu.ca

Recall Chapter 6's small feedforward network, two inputs, a hidden layer of two ReLU units, and a sigmoid output, with weights

$$W_1 = \begin{pmatrix} 0.4 & -0.2 \\ 0.1 & 0.3 \end{pmatrix},\ b_1 = \begin{pmatrix} 0.05 \\ -0.1 \end{pmatrix},\ W_2 = \begin{pmatrix} 0.6 & -0.5 \end{pmatrix},\ b_2 = 0.1,$$

which at $x_0 = (1.0, 0.5)$ produces hidden activations $h_0 = (0.35, 0.15)$ and a prediction $\hat{y}_0 \approx 0.558$. Because both ReLU units are active (their pre-activations are positive) at x_0, the network is, in a small enough neighborhood, exactly linear there, so this example doubles as a check on LIME's method: the fitted local surrogate should recover the true local behavior almost exactly, not merely approximately. Table 18.3 lists eight perturbed points around x_0, each within ± 0.1 of x_0 in each coordinate, and the network's exact prediction at each.

Perturbed input (x_1, x_2)	Hidden activations	Prediction $\hat{y}$
$(1.1, 0.5)$	$(0.39, 0.16)$	0.5632
$(0.9, 0.5)$	$(0.31, 0.14)$	0.5538
$(1.0, 0.6)$	$(0.33, 0.18)$	0.5518
$(1.0, 0.4)$	$(0.37, 0.12)$	0.5651
$(1.1, 0.6)$	$(0.37, 0.19)$	0.5565
$(0.9, 0.4)$	$(0.33, 0.11)$	0.5605
$(1.1, 0.4)$	$(0.41, 0.13)$	0.5698
$(0.9, 0.6)$	$(0.29, 0.17)$	0.5471

Table 18.3: Perturbed samples around $x_0 = (1.0, 0.5)$ and the network's exact prediction

Fitting an ordinary least-squares surrogate

$$\hat{y} \approx a + b_1 x_1 + b_2 x_2$$

to these eight samples gives $a \approx 0.545$, $b_1 \approx 0.0468$, $b_2 \approx -0.0666$. The analytical local gradient, obtained by differentiating the network directly (valid precisely because both ReLU units stay active throughout this neighborhood), is $\frac{\partial \hat{y}}{\partial x} = \hat{y}_0(1 - \hat{y}_0)\, W_2 W_1 \approx (0.0469, -0.0666)$, matching the LIME-fitted slope to three decimal places. Evaluating the fitted surrogate at x_0 itself gives $\hat{y}_{\text{surrogate}}(x_0) \approx 0.558$, matching the network's true output 0.558 almost exactly, exactly the local-fidelity property LIME is designed to deliver. This agreement is not a coincidence of this particular network; it is a direct consequence of the neighborhood chosen being small enough that no ReLU unit crosses its activation boundary.

Perturbing further away breaks this: at $x = (1.0, -0.1)$, the second hidden unit's pre-activation is exactly zero and the ReLU clips it, giving a true prediction of $\hat{y} \approx 0.594$, while the local surrogate fitted above, extrapolated to this same point, predicts approximately 0.598, a small but growing discrepancy that would widen further still at points beyond it. LIME's neighborhood radius is therefore not a mathematical given but a modeling choice with real consequences. Too narrow, and the explanation is trivially accurate but uninformative about the model's broader behavior. Too wide, and it silently averages over a region where the black box's true behavior is no longer linear at all, understating the kind of kinked, nonlinear behavior that motivated using a deep model in the first place.

18.4 Regulation, Fairness, and Robustness

Explanation is a legal requirement before it is a technical one, and fairness is measured, not asserted.

18.4.1 International Regulatory Approaches to Explainability

Section 18.3.2 described the U.S. approach to mandated explanation, the Equal Credit Opportunity Act's adverse-action notice requirement, which is principles-based: it specifies the outcome a lender must produce

Wulin.Suo@Queensu.ca

(specific, principal reasons for a denial) without prescribing which technical method must be used to produce it, which is why Section 18.3.2's exact Shapley decomposition and a scorecard's summed points from Section 18.2.2 are equally acceptable ways of satisfying it. The European Union's approach is both broader in scope and, in places, more prescriptive. The General Data Protection Regulation (GDPR)'s Article 22 grants individuals a right not to be subject to a decision based solely on automated processing that produces legal or similarly significant effects, which credit and insurance underwriting plainly are. Articles 13–15, read alongside it, are generally understood to require that the individual be given at least meaningful information about the logic involved, sometimes summarized as a "right to explanation." The precise technical content this requires remains genuinely contested among legal scholars: a global description of the model's logic, as most regulators and commentators read it, versus an individualized explanation of the specific decision, a stronger reading some have argued for. This is a rare case in this book where the technical machinery, Section 18.3.2 and 18.3.3's methods, is more settled than the legal question of exactly how much of it the law actually demands.

The EU's more recent Artificial Intelligence Act goes further still, moving from a data-protection framing to a product-safety one: it classifies creditworthiness assessment and credit-scoring systems explicitly as "high-risk AI" (Annex III), a category that triggers concrete, ex-ante obligations before deployment, a documented risk-management system, technical documentation sufficient for a regulator to audit, human oversight capable of overriding the system, and demonstrated accuracy, robustness, and cybersecurity, rather than the U.S. framework's more reactive, per-decision adverse-action obligation. A financial institution operating in both jurisdictions therefore faces two different compliance postures on the same underlying model: the U.S. framework asks it to explain individual outcomes after the fact and to avoid disparate impact in aggregate (Section 18.4.2), while the EU framework asks it to certify the system's design, documentation, and human oversight before the model is ever used to score a single real applicant, a genuine example of the same technical toolkit, Sections 18.3.1 through 18.3.3, being pressed into service to satisfy two structurally different regulatory philosophies at once.

One further development closes the loop: the regulators themselves now run models. Supervisory technology, SupTech, is the umbrella term for AI deployed by the supervisor rather than the supervised: securities regulators mine trading and filing data for the manipulation patterns Chapter 14 described, financial-intelligence units apply network analytics to the suspicious-activity reports Chapter 15's institutions file, and central banks apply text methods to their own supervisory correspondence to triage which firms warrant attention. The practical consequence for a model builder is symmetry: the examination of a bank's AI system is increasingly conducted *by* an AI system, and every governance artifact this chapter has argued for, documentation, monitoring records, explanation apparatus, is exactly what that examination consumes. It also imports every problem in this chapter to the other side of the table: a supervisor's model can be biased, opaque, or wrong with consequences at system-wide scale, which is why supervisory bodies have begun publishing model-governance standards for their own tools that look strikingly like the SR 11-7 discipline they impose on the firms they oversee.

18.4.2 Fairness in Financial AI

Explainability answers what drove a specific decision; fairness asks a related but logically separate question, whether a model's decisions, however well explained, treat different groups equitably. Financial regulators have historically operationalized this through two, not entirely compatible, statistical definitions, and a substantial body of theoretical work has shown that this incompatibility is not a matter of insufficiently clever engineering but a mathematical impossibility in general.

Disparate impact and the four-fifths rule. A lending, hiring, or underwriting practice exhibits disparate impact if it produces substantially different outcome rates across groups, even when no feature encodes group membership directly and no discriminatory intent can be shown. U.S. regulatory guidance operationalizes "substantially different" using the *four-fifths rule*: if the group with the lower favorable-outcome rate receives that outcome less than 80% as often as the group with the higher rate, the *adverse impact ratio*,

$$\mathrm{AIR} = \frac{\min(\mathrm{rate}_A, \mathrm{rate}_B)}{\max(\mathrm{rate}_A, \mathrm{rate}_B)},$$

falls below 0.80, disparate impact is presumed and the practice invites regulatory scrutiny regardless of

Wulin.Suo@Queensu.ca

intent.

Equalized odds and error-rate parity. A second family of definitions asks not whether groups receive the favorable outcome at similar rates, but whether the model is equally *accurate* for each group, specifically, whether its false-negative rate (missing an applicant who will actually default, in this chapter's running example) and false-positive rate are similar across groups. A model can satisfy the four-fifths rule while its false-negative rate is dramatically higher for one group than another, or vice versa. A substantial theoretical literature (Chouldechova, 2017; Kleinberg, Mullainathan, and Raghavan, 2017, developed originally in the context of criminal-justice risk scores but directly applicable here) proves something stronger. Except in the special case where the two groups have identical underlying default rates, no model can simultaneously satisfy calibration, equal false-negative rates, and equal false-positive rates across groups. A regulator or an internal model-risk function must therefore choose, explicitly, which fairness definition a given model is being held to, rather than assuming that satisfying one automatically satisfies the others.

Table 18.4 makes this tension concrete with a small, stylized fairness audit of Chapter 9's credit-scoring model applied to two hypothetical four-applicant groups, Alpha and Beta, at the same 620-score approval threshold used in Section 18.3.2 (Pr(Default) $\leq 41.8\%$).

Applicant	Debt/income	Credit history	Pr(Default)	Decision	Will actually default
Alpha-1	15%	9 yrs	0.2%	Approve	No
Alpha-2	22%	6 yrs	13.2%	Approve	**Yes**
Alpha-3	30%	3 yrs	95.5%	Deny	Yes
Alpha-4	18%	8 yrs	1.1%	Approve	No
Beta-1	25%	4 yrs	54.3%	Deny	No
Beta-2	33%	2 yrs	99.2%	Deny	Yes
Beta-3	20%	5 yrs	6.2%	Approve	No
Beta-4	38%	1 yr	>99.9%	Deny	Yes

Table 18.4: A fairness audit of the credit-scoring model on two synthetic four-applicant groups

Group Alpha's approval rate is $3/4 = 75\%$; Group Beta's is $1/4 = 25\%$, giving an adverse impact ratio of $0.25/0.75 = 0.333$, far below the 0.80 threshold and, on this evidence alone, a clear four-fifths-rule violation calling for further fair-lending investigation. Yet the error-rate picture is not simply the mirror image of this disparity. Among Group Alpha's two applicants who will actually default (Alpha-2 and Alpha-3), the model approves one of them, Alpha-2, whose moderate debt-to-income and credit history keep her predicted probability just under the threshold, a false-negative rate of 50%. Among Group Beta's two future defaulters (Beta-2 and Beta-4), the model correctly denies both, a false-negative rate of 0%. The group that is approved far more often (Alpha) is thus also the group whose true risk the model is worse at catching, while the group facing the harsher approval rate (Beta) is, on this small sample, screened perfectly. Neither statistic alone tells the whole story. A regulator focused only on approval-rate parity would flag this model for disadvantaging Beta, while a regulator focused only on error-rate parity would note that Alpha's approved population is quietly carrying more missed risk. Reconciling the two findings, rather than optimizing either one in isolation, is the judgment call the impossibility result above shows cannot be avoided by a purely statistical fix.

The impossibility result cited above names a third statistic, calibration, worth checking directly rather than only invoking by name. Both groups happen to share an identical actual default rate here, $2/4 = 50\%$ for Alpha and $2/4 = 50\%$ for Beta, the special case the impossibility theorem carves out as the one setting where all three properties, calibration, equal false-negative rates, and equal false-positive rates, could in principle coexist.

Yet a simple calibration-in-the-large check, comparing each group's average *predicted* default probability to its actual default rate, shows the model is not equally calibrated across them even so:

$$\begin{aligned} \text{Group Alpha predictions:} &\quad 0.19\%,\ 13.26\%,\ 95.50\%,\ 1.14\% \quad (\text{mean } 27.5\%,\ \text{actual } 50\%), \\ \text{Group Beta predictions:} &\quad 54.3\%,\ 99.2\%,\ 6.2\%,\ 99.96\% \quad (\text{mean } 64.9\%,\ \text{actual } 50\%), \end{aligned}$$

the model systematically under-predicts risk for Alpha and over-predicts it for Beta, even though both

Wulin.Suo@Queensu.ca

groups' realized outcomes are identical. This is not a violation of the impossibility theorem, which is a statement about idealized population distributions, not a guarantee about any one small finite sample, but it is a concrete illustration of exactly how the tension it describes actually shows up in practice: equal base rates do not, by themselves, force equal calibration, equal error rates, or equal approval rates in a real, finite population, and a fairness audit that checked only the adverse impact ratio, only the error-rate gap, or only calibration would each tell a different, individually incomplete part of this same story.

Individual Fairness: A Third Paradigm

Both fairness definitions above are *group* fairness notions: they compare an aggregate statistic, an approval rate or an error rate, across two or more predefined groups. A third paradigm, **individual fairness** (Dwork, Hardt, Pitassi, Reingold, and Zemel, 2012), asks a different question entirely: are two individuals who are similar with respect to the task at hand treated similarly by the model, regardless of which group either belongs to? Formally, this requires a task-specific similarity metric, a notion of distance between applicants that captures what "similar with respect to creditworthiness" actually means, and a model satisfies individual fairness if it maps similar inputs to similar outputs under that metric.

Table 18.4's Alpha-2 and Beta-3 illustrate why this third lens matters: their predicted default probabilities are 13.2% and 6.2% respectively, not identical but broadly comparable, despite belonging to different groups, and both receive the same final decision (approval, in this case, though a stricter similarity threshold could flag even this small gap in predicted probability as an individual-fairness violation worth investigating). The practical difficulty individual fairness faces is the difficulty this parenthetical hints at. Choosing the similarity metric itself requires the same kind of value judgment that group fairness definitions push onto the choice of protected groups and outcome statistics: which features count as legitimately relevant to creditworthiness and which do not. Individual fairness therefore does not so much resolve the value judgment Section 18.4.2's impossibility result identifies as relocate it, from "which group statistic to equalize" to "which similarity metric to adopt."

18.4.3 Case Vignette: The Apple Card Gender-Bias Controversy

In November 2019, software entrepreneur David Heinemeier Hansson posted publicly that the Apple Card, issued by Goldman Sachs, had granted him a credit limit twenty times higher than his wife's, despite the couple filing joint tax returns and his wife having a longer credit history and a comparable or better credit score. Apple co-founder Steve Wozniak reported a similar disparity with his own spouse. The claims spread rapidly and prompted the New York State Department of Financial Services to open a formal inquiry into whether Goldman Sachs's credit-limit algorithm violated fair-lending law.

The regulator's eventual findings, published in 2021, illustrate the distinction this chapter has drawn throughout with unusual clarity. The investigation did not find evidence that the algorithm used sex or gender as an input, nor evidence of intentional discrimination under the Equal Credit Opportunity Act, the specific claim the initial controversy had focused on.

It did, however, find that Goldman Sachs's underwriting process lacked the kind of testing and monitoring this chapter's Section 18.4.2 formalizes. The bank could not adequately demonstrate, after the fact, that its model did not produce disparate outcomes correlated with sex, because it had not systematically tested for this in the way a four-fifths-rule audit like Table 18.4's would. And its customer service representatives, when asked to explain a specific credit-limit decision, could offer no principled, model-derived reason of the kind Section 18.3.2's adverse-action reason codes are designed to produce, only that "the algorithm decided." The regulator's ultimate recommendation was not a fine but a call for stronger fairness testing and clearer consumer-facing explanations of algorithmic credit decisions, precisely the two capabilities this chapter has built by hand. The episode is a useful corrective to a common misreading of fair-lending law. The absence of a protected characteristic as a model input, sometimes called "fairness through unawareness," is neither necessary nor sufficient to avoid disparate impact. Facially neutral inputs correlated with a protected characteristic, such as a joint account structure that reflects gendered patterns in whose name assets are held, can reproduce the same disparity a cruder model would have produced by using the protected characteristic directly.

Wulin.Suo@Queensu.ca

18.4.4 Adversarial Robustness in Financial Models

A model's outputs can be manipulated deliberately, not just misunderstood accidentally, and Chapter 15's fraud and money-laundering vocabulary already has a name for this behavior in a different context: *structuring*, deliberately splitting a transaction to fall just under a reporting or detection threshold. The same idea applies directly to evading a fraud-scoring model rather than a currency-reporting requirement.

Recall Chapter 15's transaction 1, an amount z-score of 3.1, a distance of 800 kilometers from the cardholder's home, and a late-night flag, correctly flagged as fraud with predicted probability 88.3%. Holding distance and late-night status fixed and solving $-2.57+0.83\,z+0.15(8.0)+0.82(1) = 0$ for z gives $z \approx 0.663$: a fraudster who could somehow disguise this same transaction's amount so that it appears only 0.663 standard deviations above the cardholder's typical spending, for instance by splitting it into several smaller purchases at the same distant, late-night location, would push the model's prediction just under the 0.5 threshold and evade detection entirely. This is a substantial required change, a reduction of 2.44 in the amount z-score, not a barely perceptible perturbation, precisely because the transaction's distance and timing already carry so much of the model's suspicion that amount alone cannot be adjusted by a small amount to escape it. This is a meaningfully different, and more reassuring, finding than the classic adversarial-example results in image classification, where an imperceptibly small pixel-level perturbation (Goodfellow, Shlens, and Szegedy, 2015) can flip a deep network's prediction entirely. A low-dimensional, linear-in-the-log-odds model with several independently suspicious features built in, like this fraud-scoring model, is considerably harder to game with a change to any single feature than a high-dimensional deep network is. It remains vulnerable, though, to an adversary who can jointly manipulate several features at once, reducing amount, choosing a nearby location, and transacting during business hours simultaneously, a joint attack this single-feature analysis does not rule out.

This is precisely why Chapter 17's discussion of alternative-data vendor black-box processing and Chapter 15's own point about adversarial adaptation both matter here. Any model whose scoring logic can be inferred or guessed by the population it scores will eventually see its inputs shift in response, whether the inference comes from a sophisticated fraud ring or simply from market participants learning which alternative-data patterns a fund's model rewards. That is the concept-drift dynamic Chapter 14 introduced for high-frequency trading models and Chapter 15 quantified for fraud-model precision.

Adversarial robustness means an adversary deliberately crafting an input to fool the model. It is a distinct concept from the stress testing Chapters 8 and 9 introduced for market and credit risk, which subjects a model or portfolio to an extreme but plausible scenario, a severe recession or a market crash, to see how it behaves. Both nonetheless ask a version of "how does this model behave outside the data it was ordinarily trained on." Stress testing probes the model's behavior under a shift in the entire population's characteristics, unemployment rising across the board, say, while adversarial robustness probes its behavior under a targeted, deliberate manipulation of one individual case; a model can be robust to one and vulnerable to the other. A fraud-scoring model retrained frequently enough to handle a genuine, economy-wide shift in spending patterns can still be gamed one transaction at a time by a single sophisticated fraudster, exactly as Section 18.4.4 showed directly. And a stress-tested credit model that behaves sensibly in a simulated recession offers no particular guarantee about the counterfactual-manipulation risk Section 18.3.2 raised: an applicant who learns the model's threshold and adjusts a self-reported input specifically to cross it, rather than through any genuine change in creditworthiness.

18.5 Governance and Advanced Topics

A model in production drifts, and this part covers what is monitored and what is done about it.

18.5.1 Model Governance and Monitoring in the AI Era

Chapters 8 and 9 introduced model risk management, independent validation and ongoing performance monitoring for market and credit risk models, and Chapter 15 formalized the three-lines-of-defense structure (business unit, independent compliance and risk function, internal audit) for fraud and anti-money-laundering programs. U.S. bank regulators codify this discipline for essentially every model a bank relies on in a single supervisory guidance document, SR 11-7 (Board of Governors of the Federal Reserve System and Office of the

Wulin.Suo@Queensu.ca

Comptroller of the Currency, 2011), which explicitly names conceptual soundness, ongoing monitoring, and outcomes analysis as the three pillars of sound model risk management, the three capabilities, understanding why a model produces a given output, tracking whether it keeps working, and checking its actual outcomes for fairness, this chapter has built tools for directly.

Chapter 15's precision-drift table tracked a fraud model's precision declining from 24.2% to 17.1% over six months as fraudsters adapted their behavior, but precision is a *lagging* indicator: computing it requires investigators to eventually confirm which flagged transactions were genuinely fraudulent, a process that itself takes time.

The **population stability index (PSI)** offers a complementary, *leading* indicator that needs no labels at all, only the model's own score outputs, comparing the distribution of scores in a current period against a baseline distribution established at the time the model was validated. Formally, $\text{PSI} = \sum_i (c_i - b_i) \ln(c_i/b_i)$, summed over score bins i with baseline proportion b_i and current proportion c_i. Table 18.5 illustrates this for the fraud-scoring model, comparing its validation-time score distribution across four bins against a later snapshot.

Score bin	$[0, 0.25)$	$[0.25, 0.50)$	$[0.50, 0.75)$	$[0.75, 1.00]$	Total PSI
Baseline proportion	0.50	0.25	0.15	0.10	
Current proportion	0.34	0.29	0.22	0.15	
Bin contribution	0.062	0.006	0.027	0.020	0.115

Table 18.5: Population stability index for the fraud-scoring model's score distribution

A total PSI of 0.115 falls into the conventional "moderate shift, worth investigating" band used widely in credit-risk monitoring practice (below 0.10 is treated as no material change; above 0.25 as a major shift typically requiring retraining or revalidation before the model continues to be used), well before precision itself has moved enough to be conclusively distinguishable from noise. A larger share of transactions now score in the lower bins and a smaller share in the top bin, consistent with the same adaptive evasion Section 18.4.4 demonstrated directly: as more transactions structure themselves to sit just under the model's threshold, the population of scores shifts downward even before enough investigated cases accumulate to move the precision figure meaningfully. This is the kind of early warning SR 11-7's ongoing-monitoring pillar is meant to produce, and exactly why a mature model governance program tracks both a leading, label-free statistic like PSI and a lagging, outcome-based statistic like precision, rather than either alone.

The National Institute of Standards and Technology's AI Risk Management Framework (NIST AI RMF 1.0, 2023) organizes this entire chapter's concerns, explainability, fairness, robustness, and governance, into a single voluntary framework built around four functions: *govern* (establishing accountability and policy, this chapter's Section 18.5.1 and Chapter 15's three lines of defense), *map* (understanding a model's context and likely impact, including on different demographic groups, Section 18.4.2), *measure* (the technical methods of Sections 18.3.1 through 18.3.3 and 18.4.4), and *manage* (acting on what measurement reveals, from adverse-action notices to model retirement). The framework is deliberately not finance-specific, and deliberately not a checklist to complete once. It is best read as an organizing structure for a discipline that, as Section 18.4.3's case vignette showed directly, tends to fail through gaps between these four functions: an algorithm that was measured for accuracy but never mapped for demographic impact, or governed on paper without anyone able to produce an actual explanation when a regulator finally asked for one.

18.5.2 Advanced Topics

Five further topics extend this chapter's core toolkit in directions that matter increasingly in real practice. They are: making a counterfactual explanation genuinely actionable rather than merely mathematically valid; a causal rather than purely statistical definition of fairness; extending explanation methods to the deep architectures Chapters 16 and 17 introduced; attaching a finite-sample uncertainty guarantee to a model's individual predictions; and a real example of a regulator engaging with an AI-based lender proactively rather than after a public controversy.

Wulin.Suo@Queensu.ca

Algorithmic Recourse: Making Counterfactuals Actionable

Section 18.3.2's counterfactual explanation, reduce debt-to-income by about four percentage points, is only one of several ways to cross the same approval threshold; holding debt-to-income fixed at the applicant's actual 28% instead and solving for the required credit history gives a counterfactual of approximately 13.0 years, an increase of about 9.0 years from the applicant's actual 4. Both counterfactuals are equally valid solutions to the same equation, $z \leq -0.330$, but they are not remotely equally achievable: a bank could reasonably expect an applicant to pay down debt over a matter of months, while no applicant can shorten the wait for nine additional years of credit history to elapse.

Algorithmic recourse (Ustun, Spangher, and Liu, 2019) formalizes this distinction by attaching an explicit cost to changing each feature, rather than treating every unit of every feature as equally easy to change, and recommending the counterfactual with the lowest total cost instead of merely the nearest one in raw feature space. Assigning debt-to-income a cost of 1 unit per percentage point (roughly, the difficulty of paying down that much debt) and credit history a cost of 5 units per year (reflecting that this dimension can only be waited out, never accelerated) gives a total cost of 3.95 for the debt-to-income path and $9.04 \times 5 = 45.19$ for the credit-history path, making explicit, and quantifying, what is already obvious once the two raw numbers are compared side by side: recourse through debt-to-income is the only one of the two counterfactuals a lender should actually recommend to this applicant. The general lesson extends well beyond this one example. Any counterfactual-explanation system deployed in production should distinguish mutable features an individual can realistically act on, such as debt-to-income, savings, or a co-signer, from effectively immutable ones: age, the literal passage of time, and, importantly, any feature correlated with a protected characteristic. Recommending recourse through an immutable feature is not merely unhelpful; it can itself raise the same fair-lending concerns Section 18.4.2 developed directly.

Counterfactual Fairness: A Causal Perspective

Section 18.4.2's disparate-impact and equalized-odds definitions are both purely statistical: they compare outcome or error rates across groups without any model of *why* those rates differ. **Counterfactual fairness** (Kusner, Loftus, Russell, and Silva, 2017) asks a causal question instead: would this specific individual's decision have been different had their protected attribute (sex, race) been different, holding fixed everything about them that is not itself causally downstream of that attribute?

This reframing matters exactly where Section 18.4.3's case vignette left off. A joint account's credit history can be, causally, downstream of a couple's gender-influenced decision about whose name assets are held under, so a model that uses joint-account history as a feature is not necessarily counterfactually fair even though it never reads sex or gender directly. Holding the individual's actual, gender-influenced account history fixed while imagining a different gender is precisely the counterfactual comparison this definition asks for, and the correlation-only reasoning of Section 18.4.2 cannot express it. Operationalizing this requires a causal graph specifying which features are downstream of the protected attribute and which are not, a substantially harder modeling commitment than any purely statistical fairness metric, which is why counterfactual fairness remains more common as an analytical lens for diagnosing cases like Apple Card's after the fact than as a routinely deployed pre-launch test.

Explaining Deep Models: Saliency, Integrated Gradients, and the Faithfulness Problem

Section 18.3.3's LIME surrogate treats the network as a black box; when the model's own gradients are available, as they are for any network trained by backpropagation, a more direct family of methods computes an attribution straight from those gradients rather than from a sampled local regression. The simplest version, a *saliency map*, is just the gradient of the output with respect to each input, the analytical local gradient Section 18.3.3 already computed, $(0.0469, -0.0666)$ at x_0.

Multiplying this gradient by the full distance to a baseline, $x_0 - x' = (1.0, 0.5)$ from $x' = (0, 0)$, gives a naive attribution of $(0.047, -0.033)$, summing to 0.014, but the network's true output actually changes by $0.558 - 0.532 = 0.026$ between these two points, nearly double the naive estimate. The discrepancy is not a rounding error: the straight-line path from x' to x_0 crosses a genuine kink in the network, the second hidden unit's ReLU is inactive at x' (pre-activation -0.1) and switches on at exactly $t = 0.4$ of the way along the

Wulin.Suo@Queensu.ca

path, so a single gradient evaluated only at the endpoint x_0 simply cannot see the nonlinearity the path passes through earlier on.

Integrated gradients (Sundararajan, Taly, and Yan, 2017) fixes this by integrating the gradient along the entire path rather than evaluating it once,

$$\mathrm{IG}_i(x) = (x_i - x_i') \int_0^1 \frac{\partial F}{\partial x_i}\big(x' + t(x - x')\big)\, dt.$$

Because this network's pre-activations are piecewise linear in t along the straight path, the integral has an exact closed form on each of the two segments (before and after the ReLU switches on at $t = 0.4$), using the fact that $\sigma'(a + bt)$ integrates to $\sigma(a + bt)/b$. Carrying this out gives $\mathrm{IG}_1 = 0.0520$ and $\mathrm{IG}_2 = -0.0260$, summing to 0.0260, matching the true output change exactly (up to rounding), the *completeness* axiom integrated gradients is specifically designed to satisfy and that the naive endpoint-gradient shortcut above visibly failed. This matters directly for Chapters 16 and 17's deep architectures: a CNN's convolutional filters and an attention-based transformer both admit the same gradient-based treatment, and integrated gradients (or a close variant) is the standard way to produce a saliency map for an image classifier or a token-level attribution for a language model in practice.

A further, more subtle warning concerns attention weights specifically, since Chapter 16 introduced self-attention and it is tempting to read a high attention weight as "the model is explaining that it relied on this token." Empirical work (Jain and Wallace, 2019) has shown that attention weights frequently fail a basic test any genuine explanation should pass, alternative attention distributions can be found that leave a transformer's output essentially unchanged, meaning the specific weights actually observed are not uniquely responsible for the prediction in the way an explanation implies.

The same faithfulness concern extends to a large language model's own natural-language, chain-of-thought explanation of its reasoning. Research on LLM-generated rationales (Turpin, Michael, Perez, and Bowman, 2023) has found cases where a model's stated reasoning does not match the actual factor that changed its answer. Such an explanation is fluent and plausible but not, in the technical sense this chapter has used throughout, faithful to the computation that actually produced the output. This is precisely why gradient-based methods like integrated gradients, which are computed directly from the model's own mathematics rather than from a second, separately generated explanation, remain the more trustworthy default for a genuinely high-stakes financial deployment of Chapter 16 or Chapter 17's architectures, notwithstanding how much more fluent a model's own self-reported explanation may read.

Conformal Prediction: Honest Uncertainty for Individual Decisions

Every explanation method in this chapter says why the model predicted what it did; none says how *sure* the model is entitled to be about it. **Conformal prediction** converts any trained model's scores into set-valued predictions carrying a finite-sample coverage guarantee. Rank a held-out calibration set's nonconformity scores, for a classifier one minus the probability assigned to the true label, take the appropriate quantile, and for each new case output every label whose score clears it. The resulting set is guaranteed to contain the true label at a chosen rate, say 90%, with no assumptions about the model or the data beyond exchangeability. For a lender the operational appeal is concrete: a borderline applicant whose prediction set is {repay, default} is a case the model genuinely cannot resolve, a natural trigger for the human review Section 18.5.1's governance framework already provides for, while a confident singleton set needs no escalation. Two caveats keep it inside this chapter's discipline: the guarantee is marginal, holding on average rather than within each subgroup, so a validator should still check coverage group by group in the spirit of Section 18.4.2's audits; and it is only as good as the calibration data's continued relevance, since under the drift Section 18.5.1's population-stability index monitors, yesterday's calibration quantile silently loses today's coverage.

Case Vignette: Regulatory Sandboxes and the Upstart No-Action Letter

Section 18.4.3's case vignette showed a regulator responding to a public controversy after deployment; the CFPB's engagement with Upstart Network shows the same underlying concerns, whether an AI-based underwriting model produces disparate impact and can be adequately explained, addressed proactively instead. In 2017, the Consumer Financial Protection Bureau granted Upstart a no-action letter, a formal assurance

Wulin.Suo@Queensu.ca

that the agency would not bring an enforcement action against specific practices. Upstart is an online lender that uses machine learning models for credit underwriting, including alternative data such as educational attainment. The letter was conditioned on Upstart regularly reporting detailed fair-lending testing results to the CFPB, including the kind of disparate-impact statistics Section 18.4.2's adverse impact ratio formalizes, broken out by race, ethnicity, and sex.

The CFPB issued a second, renewed letter in November 2020, which also required Upstart to notify the agency of significant changes to its model before making them. That condition is what ended the arrangement. In April 2022 Upstart told the CFPB it intended to add a significant number of new variables to its underwriting model; the Bureau asked for more time to evaluate the changes; and Upstart, citing the need to keep its models accurate and updated during a period of rapid economic change, asked for the letter to be terminated, which the CFPB did by order in June 2022. The ending is itself a lesson in this chapter's monitoring theme: a supervisory assurance covers the model as reviewed, and a model that must be retrained quickly sits uneasily inside it. The contrast with Section 18.4.3 is instructive: Upstart's own reporting reportedly showed its model approving a meaningfully higher share of applicants, at similar or lower average interest rates, than a traditional credit-score-only underwriting model would have, evidence used to argue that a more flexible model can *expand* credit access rather than merely risk disparate impact, but that evidence was only credible, and only available to the regulator at all, because Upstart had committed in advance to the ongoing fairness testing and reporting infrastructure this chapter's Section 18.4.2 and Section 18.5.1 have developed by hand. A regulatory sandbox or no-action letter is not a waiver of the underlying obligations, disparate-impact testing, adverse-action explanation, ongoing monitoring, still apply in full; it is instead a structured, supervised space for a firm to demonstrate those obligations are being met by a genuinely novel model before, rather than only after, a wide public rollout.

18.6 Challenges in Explainability, Fairness, and Regulation

Several challenges recur across this chapter's methods and connect directly to themes from earlier chapters.

Explanation methods are themselves approximations, not ground truth. Section 18.3.3's worked example showed a case where a local surrogate matched the true model almost exactly. The same example also showed exactly how that agreement degrades once a perturbation crosses a genuine kink in the underlying function. A post-hoc explanation is only as trustworthy as the neighborhood, or the reference baseline, it was built around, a modeling choice with no universally correct answer.

Multiple fairness definitions cannot generally be satisfied simultaneously. Section 18.4.2's worked example, and the impossibility results underlying it, mean that a bank's choice of which fairness criterion to optimize for is an explicit policy decision with real distributional consequences, not a detail that a sufficiently sophisticated model can sidestep.

Proxy discrimination survives the removal of protected characteristics. Section 18.4.3's case vignette showed directly that omitting a protected characteristic from a model's inputs does not guarantee the model's outcomes are free of disparate impact along that same characteristic, since other, facially neutral features can correlate with it closely enough to reproduce the same pattern.

Explainability and model capability can be genuinely in tension. Chapter 6, Chapter 16, and Chapter 17 each observed that some of the most predictively powerful models, deep ensembles, attention-based transformers, convolutional and recurrent networks, are also among the least directly interpretable. This chapter's post-hoc methods manage that tension rather than eliminate it. That is why some regulated use cases, consumer credit decisions in particular, still favor genuinely interpretable models over marginally more accurate black-box alternatives.

Monitoring must anticipate adaptive, not just random, drift. Section 18.4.4 and Section 18.5.1 together show that in an adversarial financial setting, unlike a physical or purely statistical forecasting one, the population being scored can learn from and react to the model itself, meaning drift detection must be treated as an ongoing arms race rather than a one-time validation exercise.

Governance frameworks require organizational capability, not just documentation. Section 18.4.3 showed that a bank can have a governance framework on paper and still fail the underlying obligation, producing no usable explanation when a specific customer or regulator asked for one, which is why SR 11-7 and the NIST AI RMF both emphasize ongoing organizational practice over a one-time

Wulin.Suo@Queensu.ca

compliance checklist.

18.7 Key Takeaways

Interpretability is a property of a model's own structure, while explainability is a property of a method applied to a model, a distinction that matters because most of this book's more powerful models, Chapter 6's ensembles, Chapter 16's transformers, Chapter 17's CNNs and RNNs, are not interpretable in the first sense and therefore depend entirely on the post-hoc methods of Section 18.2.1's taxonomy. Global methods, permutation importance and partial dependence (Section 18.3.1), describe a model's behavior in general, using Chapter 15's fraud-scoring model to show that a feature can matter enormously in principle while showing zero measured importance on a specific sample. Local methods answer a single case's question directly. An exact Shapley decomposition (Section 18.3.2) for a linear-in-the-log-odds model like Chapter 9's credit scorer produces adverse-action reason codes and a counterfactual explanation almost for free. LIME (Section 18.3.3) extends the same idea to genuinely nonlinear models like Chapter 6's toy neural network, at the cost of validity that is guaranteed only in a local neighborhood whose size is itself a modeling choice.

Fairness (Section 18.4.2) is not a single statistical target: the four-fifths rule and error-rate parity can point in different directions on the very same model and data, as this chapter's synthetic two-group audit showed concretely, and a substantial theoretical literature proves this tension cannot generally be resolved away. The 2019 Apple Card controversy (Section 18.4.3) showed these are not academic concerns: a real, widely used consumer credit product failed the fairness-testing and explanation obligations this chapter formalizes, without ever using a protected characteristic as a model input.

Section 18.4.4 extended Chapter 15's structuring vocabulary from evading a reporting threshold to evading a model directly. Section 18.5.1 then showed that a leading indicator like the population stability index can flag that evasion before enough labeled outcomes accumulate for a lagging indicator like precision to catch up. Together they tie explainability, fairness, robustness, and governance into the single ongoing discipline that SR 11-7 and the NIST AI Risk Management Framework both formalize.

Section 18.5.2 pushed each of these threads one level deeper: a cost-weighted view of Section 18.3.2's counterfactual showed that not every mathematically valid recourse path is actually achievable, integrated gradients showed exactly where a naive gradient-based shortcut understates a deep model's true attribution and why attention weights and chain-of-thought rationales cannot always be trusted as faithful explanations of Chapter 16's own models, and the CFPB's Upstart no-action letter showed the same fairness and explainability obligations this chapter formalizes being met proactively, through structured regulatory engagement, rather than only after a public controversy like Apple Card's.

18.8 Suggested Exercises

1. Using Table 18.1's method, permute the late-night indicator using a different fixed rearrangement (a cyclic shift by one position rather than a full reversal) and recompute the confusion matrix. Does the conclusion that late-night timing shows zero permutation importance on this sample still hold?

2. Explain, in your own words, why a feature can have a strong, monotonic partial-dependence curve (Table 18.2) while simultaneously showing zero permutation importance (Table 18.1) on the same dataset.

3. Using Section 18.3.2's method, compute the exact Shapley decomposition for a second applicant with a 32% debt-to-income ratio and 7 years of credit history, using the same reference baseline, and verify the efficiency property (that the two contributions plus the baseline log-odds sum exactly to this applicant's own log-odds).

4. Rank the two contributions from Exercise 3 and state which reason code would appear first on this applicant's adverse action notice.

5. Using Section 18.3.2's method, compute the counterfactual credit history (holding debt-to-income fixed at 28%) that would have gotten Chapter 9's original applicant approved at the 620-score threshold, and compare the required change to the required debt-to-income change computed in the text.

Wulin.Suo@Queensu.ca

6. Using Table 18.3's network, compute the network's exact prediction at $x = (1.05, 0.55)$ and compare it to the LIME surrogate's prediction at the same point, reporting the discrepancy.

7. Explain why the LIME surrogate fitted in Section 18.3.3 recovers the analytical local gradient almost exactly, referencing which structural feature of the network (the sign of each hidden unit's pre-activation) makes this possible.

8. Using Section 18.4.2's formula, compute the adverse impact ratio if Group Beta's approval rate in Table 18.4 were instead $2/4 = 50\%$ (holding Group Alpha's rate fixed), and state whether the four-fifths rule would still be violated.

9. Using Table 18.4, compute each group's false-positive rate (the fraction of applicants who will not actually default but are nonetheless denied), and discuss whether this statistic points in the same direction as the false-negative-rate comparison in the text.

10. Explain, using the impossibility result cited in Section 18.4.2, why a lender cannot generally fix Table 18.4's adverse impact ratio and its false-negative-rate gap with a single adjustment to the model's classification threshold alone.

11. Using Section 18.4.3's case vignette, explain in a short paragraph why removing sex or gender as a model input does not, by itself, guarantee that a credit-limit algorithm's outcomes are free of gender-correlated disparate impact.

12. Using Section 18.4.4's method, compute the amount z-score reduction needed to flip Chapter 15's transaction 6 (amount z-score 3.4, distance 9.2, late-night flag 1) from fraud to not-fraud, holding distance and late-night status fixed, and compare the required reduction to transaction 1's.

13. Explain why the fraud-scoring model in Section 18.4.4 is harder to game with a single-feature perturbation than a deep image classifier is with a pixel-level perturbation, referencing the number of independently suspicious features each transaction carries.

14. Using Table 18.5's formula, compute the total PSI if the current-period distribution instead matched the baseline distribution exactly, and explain what this result confirms about the PSI formula's behavior when there is no drift at all.

15. Using the same PSI formula, compute the total PSI for a more extreme drift scenario in which the current distribution is $(0.20, 0.25, 0.30, 0.25)$ (holding the baseline fixed), and state whether this would fall in the "no material change," "moderate shift," or "major shift" band described in the text.

16. Explain, in a short paragraph, why the population stability index is described as a leading indicator and Chapter 15's precision-drift table as a lagging indicator, and describe one practical consequence of a model governance program monitoring only the lagging one.

17. Using Section 18.5.1's description of SR 11-7 and the NIST AI RMF, map each of this chapter's twelve worked examples to the framework function (govern, map, measure, manage) it most directly supports, and justify any example you assign to more than one function.

18. Compare Section 18.3.2's exact, closed-form Shapley decomposition to Section 18.3.3's sampled, approximate LIME surrogate, and explain the specific structural property of the credit-scoring model (as opposed to the toy neural network) that makes the closed form available in the first place.

19. Using Section 18.5.2's method, compute the algorithmic-recourse cost of a third counterfactual path that changes both debt-to-income and credit history by equal fractions of their individually required changes (that is, half of the debt-to-income path and half of the credit-history path combined), and compare its total weighted cost to the two single-feature paths in the text.

20. Using Section 18.5.2's integrated-gradients derivation, recompute IG_1 and IG_2 using a baseline of $x' = (0.5, 0.25)$ instead of $(0, 0)$, and verify that the two attributions still sum to $\hat{y}(x_0) - \hat{y}(x')$ at this new baseline, illustrating that a Shapley-style or integrated-gradients attribution is only ever defined relative to a chosen reference point, not in some absolute sense.

Wulin.Suo@Queensu.ca

21. Using Section 18.4.2's calibration-in-the-large check, recompute each group's average predicted default probability if Alpha-3's predicted probability were instead 60% rather than 95.5% (holding every other applicant's prediction fixed), and state whether Group Alpha's calibration gap (average predicted probability versus its 50% actual default rate) widens or narrows as a result.

22. Explain, in your own words, why the fact that Group Alpha and Group Beta share an identical 50% actual default rate in Table 18.4 does not, by itself, guarantee that a model calibrated on the combined population will also be equally calibrated within each group separately.

23. **Challenge (real data).** Download a recent year of HMDA (Home Mortgage Disclosure Act) loan application register data for a single large lender (freely available from the Consumer Financial Protection Bureau's HMDA data browser, no login required). (a) Using Section 18.4.2's adverse impact ratio formula, compute the approval-rate ratio between two applicant groups of your choosing (for example, by race or ethnicity, both reported in the data), and state whether the four-fifths rule is violated, exactly as it was for Table 18.4's toy two-group example. (b) Fit a simple logistic regression predicting loan approval from the legitimate underwriting variables HMDA reports (income, loan amount, debt-to-income category), and recompute the adverse impact ratio using the model's predictions instead of the raw historical approval decisions, discussing whether the disparity narrows, widens, or stays about the same. (c) HMDA data reports the reason lenders gave for each denial; compare the stated denial reasons for the two groups you chose, and discuss whether this real, lender-reported information changes your interpretation of any disparity found in part (a), relative to the toy example in the text, where no such reason codes were available to explain the underlying gap.

18.9 Further Reading

- Lundberg and Lee, "A Unified Approach to Interpreting Model Predictions," *Advances in Neural Information Processing Systems*. The original SHAP paper unifying Shapley-value-based feature attribution, underlying Section 18.3.2.

- Ribeiro, Singh, and Guestrin, "'Why Should I Trust You?' Explaining the Predictions of Any Classifier," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. The original LIME paper underlying Section 18.3.3.

- Shapley, "A Value for n-Person Games," in *Contributions to the Theory of Games*. The original cooperative-game-theory paper defining the Shapley value that Section 18.3.2 applies to a single credit decision.

- Molnar, *Interpretable Machine Learning*. A comprehensive, freely available treatment of permutation importance, partial dependence, Shapley values, and LIME, covering the full taxonomy in Figure 18.1.

- Doshi-Velez and Kim, "Towards a Rigorous Science of Interpretable Machine Learning," arXiv. A foundational discussion of what interpretability means and how to evaluate it rigorously, underlying the distinction drawn in Section 18.2.1.

- Barocas, Hardt, and Narayanan, *Fairness and Machine Learning: Limitations and Opportunities*. A comprehensive treatment of statistical fairness definitions, including disparate impact and equalized odds, underlying Section 18.4.2.

- Chouldechova, "Fair Prediction with Disparate Impact: A Study of Bias in Recidivism Prediction Instruments," *Big Data*. One of the two papers establishing the impossibility of simultaneously satisfying calibration and error-rate parity across groups, cited in Section 18.4.2.

- Kleinberg, Mullainathan, and Raghavan, "Inherent Trade-Offs in the Fair Determination of Risk Scores," *Proceedings of Innovations in Theoretical Computer Science*. The companion impossibility result to Chouldechova's, cited in Section 18.4.2.

Wulin.Suo@Queensu.ca

- Dwork, Hardt, Pitassi, Reingold, and Zemel, "Fairness Through Awareness," *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*. The original individual-fairness definition discussed as a third fairness paradigm in Section 18.4.2.
- Board of Governors of the Federal Reserve System and Office of the Comptroller of the Currency, "Supervisory Guidance on Model Risk Management," SR 11-7. The foundational U.S. bank regulatory guidance on model risk management underlying Section 18.5.1.
- National Institute of Standards and Technology, "AI Risk Management Framework (AI RMF 1.0)." The govern/map/measure/manage framework discussed at the close of Section 18.5.1.
- Goodfellow, Shlens, and Szegedy, "Explaining and Harnessing Adversarial Examples," *International Conference on Learning Representations*. The foundational paper on adversarial perturbations in deep learning, contrasted with the fraud-model perturbation in Section 18.4.4.
- New York State Department of Financial Services, "Report on Apple Card Investigation." The regulatory report underlying the case vignette in Section 18.4.3.
- Ustun, Spangher, and Liu, "Actionable Recourse in Linear Classification," *Proceedings of the Conference on Fairness, Accountability, and Transparency*. The foundational algorithmic-recourse paper underlying the cost-weighted counterfactual comparison in Section 18.5.2.
- Kusner, Loftus, Russell, and Silva, "Counterfactual Fairness," *Advances in Neural Information Processing Systems*. The original causal counterfactual-fairness definition discussed in Section 18.5.2.
- Sundararajan, Taly, and Yan, "Axiomatic Attribution for Deep Networks," *Proceedings of the International Conference on Machine Learning*. The original integrated-gradients paper underlying Section 18.5.2's worked example on Chapter 6's network.
- Jain and Wallace, "Attention Is Not Explanation," *Proceedings of the North American Chapter of the Association for Computational Linguistics*. The empirical case against reading Chapter 16's attention weights directly as explanations, discussed in Section 18.5.2.
- Turpin, Michael, Perez, and Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting," *Advances in Neural Information Processing Systems*. The empirical study of unfaithful LLM-generated rationales discussed in Section 18.5.2.
- Consumer Financial Protection Bureau, "CFPB Grants First No-Action Letter to Upstart Network." The regulatory announcement underlying the case vignette in Section 18.5.2.

Wulin.Suo@Queensu.ca

Wulin.Suo@Queensu.ca

Chapter 19

Reinforcement Learning in Finance

19.1 Introduction

Every model built so far in this book, however sophisticated, has answered a single question once. Chapter 6's classifiers predict a label from a feature vector; Chapter 9's credit models predict a default probability; Chapter 13's Almgren-Chriss trajectory prescribes an entire trading schedule, but it does so by solving one optimization problem in advance and then executing the resulting plan. None of these approaches change their answer in response to how the world actually responds to the decisions already made.

A portfolio manager, a market maker, and a trading desk do not operate this way: they observe an outcome, update their beliefs, and choose their next action accordingly, over and over, for as long as the position stays open. Reinforcement learning is the branch of machine learning built specifically for this setting, sequential decision-making under uncertainty, where an agent's own actions influence the future states it will face, and where the right action today depends on the consequences it sets up for tomorrow, not only on today's immediate reward.

This chapter formalizes that setting, the Markov decision process, and develops the two families of algorithms built to solve it: value-based methods, culminating in Q-learning and its neural-network extension, and policy-based methods, culminating in the policy-gradient algorithms that power most modern applications. Rather than introducing these ideas in the abstract and only later connecting them to finance, this chapter builds directly on machinery this book has already developed: Chapter 5's binomial-tree early-exercise decision, Chapter 12's portfolio rebalancing problem, and Chapter 13's optimal-execution and market-making models are all, on inspection, special cases of the general sequential-decision problem this chapter formalizes, ones simple enough that a closed-form or backward-induction solution exists. Reinforcement learning is what a practitioner reaches for once the problem is still sequential but no longer simple enough for a closed form, a genuinely common situation once market impact, transaction costs, and a client's actual risk preferences no longer fit the clean functional forms those earlier chapters assumed.

The chapter's central worked example makes this connection concrete rather than asserting it. It reframes Chapter 5's American put early-exercise decision as a three-state Markov decision process, solves it with model-free Q-learning instead of backward induction, and shows the learned answer converges to the same value the tree already established. That is a small, fully verifiable proof that reinforcement learning recovers a known-correct answer, before this chapter asks the reader to trust it on problems that have no closed form at all.

The chapter proceeds in three stages. Sections 19.2.1 through 19.2.5 build the general Markov decision process framework, the Bellman equation, and the vocabulary, model-based versus model-free, on-policy versus off-policy, needed to place every algorithm that follows. Sections 19.3.1 through 19.3.3 develop the algorithms themselves, starting with the fully verifiable tabular Q-learning worked example and building up to deep Q-networks and policy-gradient methods for problems too large for a table. The remaining sections turn to application and honest assessment: optimal execution, market making, and portfolio management (Sections 19.4.1 through 19.4.4), a real industry case study (Section 19.5.1), and the evaluation methodology and risks (Sections 19.5.2 and 19.6) that separate a reinforcement-learning system that is merely impressive in a backtest from one that is actually safe to deploy with real capital.

Wulin.Suo@Queensu.ca

19.2 The Reinforcement Learning Framework

This part sets up the sequential decision problem and the value functions that make it tractable.

19.2.1 The Reinforcement Learning Framework: States, Actions, Rewards, and Policies

Consider a market maker deciding, moment by moment, how to skew its bid and ask quotes as its inventory evolves, or a trader deciding how many shares of a large order to release this minute versus holding back for a better price later: in both cases, today's decision changes tomorrow's starting position, so the two decisions cannot be solved separately, one at a time, the way most of the models in Chapters 12 and 13 do. A reinforcement learning problem is formalized as a Markov decision process, a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, precisely to capture this kind of interlocking, path-dependent decision problem.

At each discrete time step t, an **agent** observes a **state** $s_t \in \mathcal{S}$, a summary of everything about the environment relevant to the decision at hand (a market maker's current inventory and the prevailing bid-ask spread; a trading agent's remaining order size and elapsed time; a portfolio manager's current holdings and estimated risk factors). The agent selects an **action** $a_t \in \mathcal{A}$ (post a quote at a given skew; sell a given number of shares this period; rebalance toward a given target weight) according to a **policy** $\pi(a \mid s)$, which gives the probability of taking action a in state s and so maps each state to a probability distribution over the available actions, with $\sum_a \pi(a \mid s) = 1$. A policy that places all of its probability on one action per state is deterministic, the special case where the agent always does the same thing from the same state.

The environment then transitions to a new state s_{t+1} according to a transition function $P(s_{t+1} \mid s_t, a_t)$ and returns a scalar **reward** r_{t+1} (realized profit and loss net of costs; a small quadratic risk penalty; an exercise payoff), governed by a reward function $R(s_t, a_t, s_{t+1})$. Figure 19.1 depicts this loop; it repeats until the episode ends, either at a fixed horizon (the market closes, the order is fully executed) or when the agent reaches an absorbing state (a position is liquidated, an option is exercised).

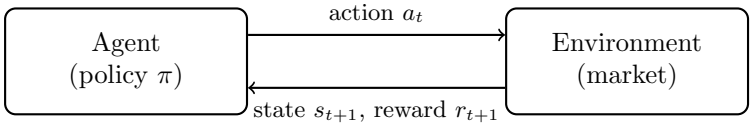


Figure 19.1: The reinforcement learning loop

The word "Markov" in Markov decision process is a precise mathematical assumption, not a figure of speech: it requires that $P(s_{t+1} \mid s_t, a_t)$ and $R(s_t, a_t, s_{t+1})$ depend only on the current state and action, not on the full history that led there. This is the assumption behind every stochastic process this book has already used to model prices, from the binomial tree's independent up-and-down moves in Chapter 5 to the random-walk and GARCH innovations of Chapter 7, so it is not a new leap of faith for a reader of this book, only a new name for a familiar structural assumption, now applied to an agent's decisions rather than only to prices themselves. In practice, whether a proposed state representation is genuinely Markovian is itself a design choice: a market maker's state that includes only the current mid-price, and not recent order flow, discards information a real market maker would use, which is why Chapter 14's order-flow-imbalance regression and Chapter 13's Kyle's-lambda framework are natural sources of state features for a genuinely useful trading agent, not merely of standalone predictive signals.

The agent's goal is to choose a policy that maximizes the expected **return**, the discounted sum of future rewards,

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1},$$

where the **discount factor** $\gamma \in [0, 1]$ plays a role directly analogous to the discount factor $1/(1+r)$ used everywhere in Chapter 5's valuation formulas: a reward received later is worth less today, both because of the time value of money and, in the reinforcement learning setting, because a reward far in the future is a noisier, harder-to-credit consequence of the current action. A myopic agent with $\gamma = 0$ maximizes only the immediate reward, the one-shot optimization every earlier chapter's models perform; the closer γ is to

Wulin.Suo@Queensu.ca

1, the more the agent's current choice is shaped by consequences many steps away, which is precisely the additional structure a sequential problem like optimal execution or ongoing portfolio management requires and a one-shot model cannot represent.

19.2.2 From One-Shot Optimization to Sequential Decisions

It is worth being explicit about why the models developed in Chapters 12 and 13 did not need this machinery, since the answer clarifies exactly what reinforcement learning adds. The Almgren-Chriss optimal-execution model (Chapter 13's Section 13.3.5) assumes a known, fixed functional form for market impact, quadratic in the trade rate, and a known, fixed risk-aversion parameter ϕ; given those assumptions, the entire optimal trajectory can be solved in closed form in one step, with no need to learn anything from experience, because the model already specifies exactly how the environment will respond to every possible action. Chapter 13's Avellaneda-Stoikov inventory model (Section 13.4) is more genuinely dynamic, the market maker's quotes are recomputed at every instant as inventory and time-to-close evolve, but the update rule is itself a closed-form formula derived from an assumed order-arrival process, not a policy learned from trial and error. Chapter 12's Section 12.4 practical-allocation discussion goes furthest toward acknowledging the sequential nature of real portfolio management, noting that rebalancing decisions recur over time and that transaction costs make each rebalancing decision depend on the portfolio's current position, not only on today's estimated optimal weights, but it stops short of solving that sequential problem directly, instead treating each rebalancing decision as a separate, myopic optimization. Each of these models is exactly right within its assumptions, and each remains the correct tool to reach for whenever those assumptions hold: a known, well-estimated impact function and a well-specified risk-aversion parameter make a closed-form solution both more accurate and vastly cheaper to compute than a learned one.

Reinforcement learning becomes the more appropriate tool precisely when that closed form is unavailable, because the true market-impact function is not well approximated by any simple parametric form, because transaction costs interact with the portfolio's history in ways too complex to solve analytically, or because the reward the agent actually cares about, a client's realized utility net of taxes, fees, and behavioral tolerance for drawdowns, does not reduce cleanly to a quadratic cost function at all. In every one of these harder cases, the underlying problem is still a sequential decision problem exactly as formalized in Section 19.2.1; what changes is only that no one can write down P and R in closed form, which rules out the direct optimization Chapters 12 and 13 rely on and motivates learning a good policy from simulated or historical experience instead.

19.2.3 The Simplest Case: Multi-Armed Bandits

A trading desk sending orders to market has a choice most descriptions of execution quietly skip past: which venue to send each order to. A large-cap U.S. equity trades on a dozen or more venues, and the same order routed to two of them will come back with different realized prices, because each venue has its own mix of resting liquidity, its own fee schedule, and its own population of counterparties. A desk that knew each venue's average price improvement would simply route everything to the best one. No desk knows this. Worse, the only way to find out what a venue delivers is to send it orders, which means the desk is paying, in worse fills, for every order it spends learning rather than earning.

This is the tension that separates reinforcement learning from everything in Chapters 6 and 7. A supervised model is handed a fixed dataset and never influences what it gets to see next; here, the agent's own choices determine which data it ever observes, so an action that looks bad early may simply never be tried again, and its true value never discovered. The problem in its purest form is the **multi-armed bandit**, named for a row of slot machines with unknown payout rates: at each step the agent picks one of K actions, called **arms**, and receives a noisy reward drawn from that arm's own fixed but unknown distribution.

What makes the bandit the right place to start is everything it leaves out. There is one state, which never changes; the reward depends only on the action just taken, never on history; and no action moves the agent anywhere, so there is no next state to look ahead to. The entire apparatus of Section 19.2.4's Bellman equation, discounting, bootstrapping, and credit assignment across a trajectory, is unnecessary here, because $Q(a)$ is nothing more elaborate than the mean reward of arm a, estimable by averaging. The only genuine

Wulin.Suo@Queensu.ca

difficulty left is the one this chapter cannot avoid anywhere: the **exploration-exploitation trade-off**, choosing between the arm that currently looks best and one whose value is still uncertain.

A Worked Example: Routing Orders Across Three Venues

A desk routes each arriving order to one of three venues. Measured in basis points of price improvement against the arrival mid-price, venue A delivers a mean of 1.2, venue B a mean of 2.0, and venue C a mean of 0.5, each with a standard deviation of 2.0 basis points. Those means are stated here so the example can be checked; the desk does not know them, and must infer them from the fills it receives. Since the standard deviation of a single fill is 2.5 times the 0.8 basis-point gap between the best venue and the second best, one order tells the desk almost nothing.

The desk keeps an estimate $Q(a)$ of each venue's mean and a count $N(a)$ of orders sent there, updating after each fill by the running-average rule

$$Q(a) \leftarrow Q(a) + \frac{1}{N(a)}\big[r - Q(a)\big],$$

which is exactly the sample mean of the rewards seen from a, written so it can be updated one fill at a time without storing them. After sending one order to each venue to get started, it follows an ε-greedy policy with $\varepsilon = 0.2$: with probability 0.2 route to a uniformly random venue, otherwise route to whichever venue currently has the highest Q. Table 19.1 traces the first twelve orders.

Order	Rule	Venue	Fill	$Q(A)$	$Q(B)$	$Q(C)$
1	warm-up	A	1.45	1.450	—	—
2	warm-up	B	1.74	1.450	1.740	—
3	warm-up	C	1.78	1.450	1.740	1.780
4	explore	A	1.92	1.685	1.740	1.780
5	exploit	C	2.39	1.685	1.740	2.085
6	exploit	C	−2.03	1.685	1.740	0.713
7	exploit	B	2.08	1.685	1.910	0.713
8	exploit	B	1.56	1.685	1.793	0.713
9	exploit	B	0.54	1.685	1.480	0.713
10	exploit	A	0.57	1.313	1.480	0.713
11	exploit	B	4.09	1.313	2.002	0.713
12	explore	C	3.23	1.313	2.002	1.342

Table 19.1: The first twelve orders of the venue-routing bandit. Fills and estimates are in basis points of price improvement; a dash marks a venue not yet tried. Every estimate follows from the running-average rule applied to the fills above it, so each row can be checked by hand.

The opening rounds show precisely the failure mode that makes exploration necessary. The single warm-up order to venue C returned 1.78 basis points, a good fill drawn by chance from the *worst* of the three venues, and that one lucky draw was enough to make C the greedy choice. Orders 5 and 6 duly went there. What rescued the desk was not a clever exploration rule but simple repetition: order 6 came back at -2.03, dragging $Q(C)$ from 2.085 down to 0.713 and demoting C below both rivals. Exploiting a wrong estimate long enough tends to correct it, because every exploit is also a sample. By order 12 the desk has $Q(B)$ highest and is routing to the genuinely best venue, though $Q(A) = 1.313$ and $Q(C) = 1.342$ remain badly estimated off three and four orders respectively.

Run the same policy for 5,000 orders and the estimates sharpen to $Q(A) = 1.25$, $Q(B) = 1.99$, and $Q(C) = 0.40$, against true means of 1.2, 2.0, and 0.5. The desk routes 86.2% of its orders to B and realizes an average of 1.826 basis points per order.

That average is the point worth dwelling on, because it is visibly short of the 2.0 basis points a desk that simply knew the answer would earn. The shortfall, 868 basis points in total across the 5,000 orders, is the **regret**: what learning cost, measured against an oracle that never had to learn. Most of it is not the early

Wulin.Suo@Queensu.ca

mistakes, which cost little and are over quickly. It is the ε-greedy rule itself, which keeps routing one order in five at random forever, long after the answer is no longer in doubt. That permanent tax is exactly

$$\varepsilon \times \big(2.0 - \tfrac{1}{3}(1.2 + 2.0 + 0.5)\big) = 0.2 \times 0.7667 = 0.1533 \text{ basis points per order,}$$

which is why the asymptotic average is $0.8 \times 2.0 + 0.2 \times 1.2333 = 1.8467$ rather than 2.0, and why the realized 1.826 sits just below it. The remedy is to explore hard early and taper off: decaying ε over the same 5,000 orders lifts the share routed to B from 86.2% to 95.2% and the average fill from 1.826 to 1.943 basis points, without giving up the early exploration that found B in the first place. Every worked example later in this chapter uses a decayed exploration schedule for this reason.

Two threads run forward from here. Add a next state, so that today's action changes tomorrow's situation, and averaging is no longer enough: the agent must reason about consequences that arrive later, which is what Section 19.2.4's Bellman equation supplies and what Section 19.3.1's Q-learning example puts to work. Keep the single state but shrink the gap between arms relative to the noise, and even this simplest of problems becomes genuinely hard: Section 19.4.2 returns to a bandit whose best and second-best arms differ by so little against so much noise that 200,000 rounds still leave the agent on the wrong one.

19.2.4 Value Functions and the Bellman Equation

Suppose two agents follow the exact same policy but happen to start an episode from different states, one with a small, comfortable inventory and one already deep in an overextended position: the policy alone cannot say how much trouble each agent is actually in, only what each plans to do next from wherever it stands. Two functions summarize how good a given state, or state-action pair, is under a policy π, answering this question: the **state-value function** $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$, the expected return starting from state s and following π thereafter, and the **action-value function** $Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$, the expected return from taking action a in state s and following π afterward.

Because the return decomposes into an immediate reward plus a discounted continuation, both functions satisfy a recursive relationship, the **Bellman equation**, named for Richard Bellman's original work on dynamic programming:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a)\Big[R(s,a,s') + \gamma V^\pi(s')\Big].$$

The **Bellman optimality equation** specializes this to the best possible policy, replacing the expectation over actions with a maximization:

$$Q^*(s,a) = \sum_{s'} P(s' \mid s, a)\Big[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\Big].$$

A reader who worked through Chapter 5's two-step binomial tree (Section 5.4.1) has already solved a Bellman optimality equation by hand without that name attached to it. At each node, the tree's backward-induction step compares the value of exercising immediately against the discounted expected value of continuing, and takes whichever is larger. That is the $\max_{a'}$ in the equation above, applied to the two actions "exercise" and "hold." This is not a loose analogy; Section 19.3.1 makes the correspondence exact by reformulating that identical tree as a Markov decision process and re-deriving the same option value from it. Dynamic programming methods, value iteration and policy iteration, solve the Bellman equation directly whenever P and R are fully known, exactly as Chapter 5's tree does. Reinforcement learning's model-free methods, developed next, instead estimate Q^* purely from sampled transitions and rewards, without ever needing P and R written down explicitly. That property is what makes them applicable to problems, like most real trading and portfolio problems, where no one can specify P and R in closed form to begin with.

19.2.5 A Taxonomy of Reinforcement Learning Methods

Two further distinctions organize the wide range of reinforcement learning algorithms in use today, and fixing both precisely now makes it easier to place every method the rest of this chapter introduces. Figure 19.2

Wulin.Suo@Queensu.ca

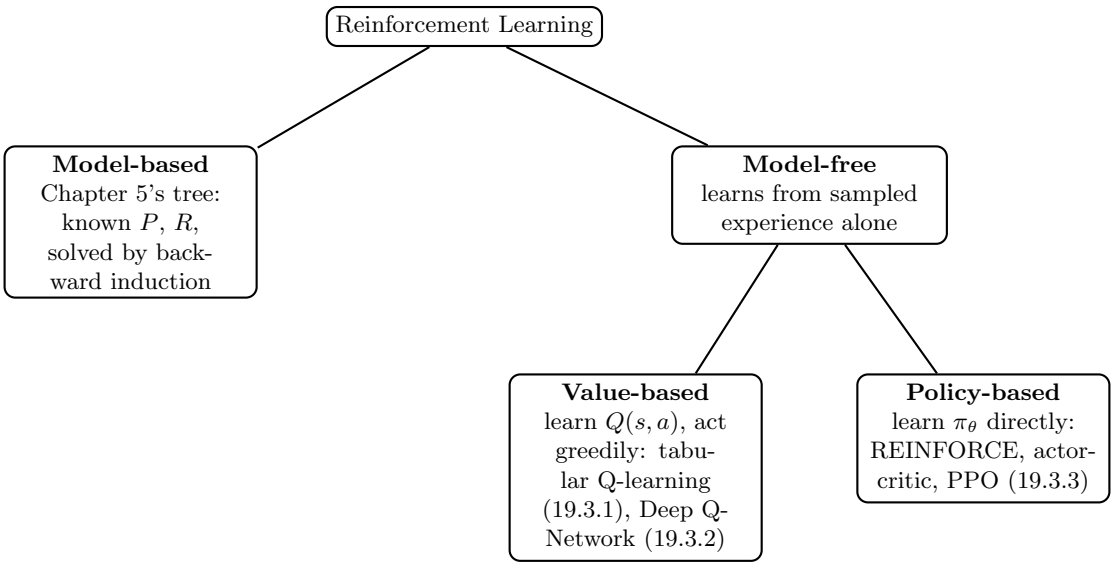


Figure 19.2: A map of the reinforcement learning methods this chapter develops

previews where each of them fits before the text below explains why; π_θ there denotes a policy carrying tunable parameters θ, the form Section 19.3.3 develops.

Model-based methods have access to, or explicitly learn, the transition function P and reward function R themselves, and use them to plan. Chapter 5's binomial tree is a model-based method: q, u, d, and every terminal payoff are known in advance, so backward induction can compute the exact optimal value with no simulated experience required at all. **Model-free methods** never estimate P or R directly; they learn a value function or a policy purely from sampled transitions and rewards, treating the environment as a black box that can be queried (by simulating an episode) but not inspected. Every method developed in the remainder of this chapter, tabular Q-learning, deep Q-networks, and the policy-gradient family, is model-free, which is precisely why each remains usable in the realistic settings Section 19.2.2 described, where no closed-form P and R exist for a real limit order book or a real client's multi-period utility.

A third, hybrid category, sometimes called **learned-model-based methods**, trains an approximate model of P and R directly from data (rather than assuming it, as Chapter 5's tree does) and then plans against that learned model, gaining some of model-based planning's sample efficiency without requiring the transition and reward functions to be known in closed form in advance. This hybrid approach underlies some of reinforcement learning's most prominent successes outside finance, including the game-playing systems that combine a learned model with tree search, but it introduces its own risk, the sim-to-real gap Section 19.5.2 discusses, since a plan is only as good as the learned model it is built against.

Off-policy methods learn about a policy different from the one actually generating their training data. Q-learning's update rule is a clear example: its bootstrapped target uses $\max_{a'} Q(s', a')$, the value of the best available action, regardless of which action the ε-greedy behavior policy, one that takes a random action with probability ε and the currently best-valued action otherwise, actually selected next. This lets a single stream of exploratory, partly random experience be reused to learn about the greedy-optimal policy even while the agent is still busy exploring, exactly why Section 19.3.1's worked example converges to the correct answer despite spending a meaningful fraction of its 100,000 episodes taking deliberately random actions. Off-policy methods have a clear data-efficiency advantage, one batch of experience can be reused across many rounds of learning, exactly why DQN's replay buffer (Section 19.3.2) works at all.

On-policy methods instead evaluate and improve the policy currently being followed. The classic on-policy analogue of Q-learning, SARSA (State-Action-Reward-State-Action), differs only by replacing $\max_{a'} Q(s', a')$ with $Q(s', a')$ for the action a' the policy actually selects next, so its learned values reflect the actual, still-exploring policy's behavior rather than the ultimate greedy target. Policy-gradient methods (Section 19.3.3) are typically on-policy as well, since their gradient estimator is only valid for returns actually

Wulin.Suo@Queensu.ca

experienced under the current policy π_θ, not under some other, older policy. On-policy methods are generally simpler to reason about and more stable to train than off-policy methods, since they never have to correct for a mismatch between the policy that generated the data and the policy currently being evaluated.

19.3 Methods

From a lookup table to a neural policy, this part builds the algorithms in order of the problems that forced each one.

19.3.1 A Worked Example: Q-Learning Recovers Chapter 5's Early-Exercise Decision

Setting Up the Markov Decision Process

Chapter 5's American put example used a two-step binomial tree with $S_0 = K = \$50$, up and down factors $u = 1.2$ and $d = 0.9$, and a gross risk-free return $R = 1.04$ per period, giving a risk-neutral up-probability

$$q = \frac{R - d}{u - d} = \frac{0.14}{0.30} \approx 0.4667$$

and a per-step discount factor $\gamma = 1/R \approx 0.9615$.

That tree has exactly three decision points: today, at $S_0 = \$50$; after one up move, at $S_u = \$60$; and after one down move, at $S_d = \$45$. At each of these three states the holder chooses between two actions: **exercise**, receive the intrinsic value $\max(K - S, 0)$ immediately and end the episode, or **hold**, receive nothing now and let the tree evolve one more step.

The two terminal nodes reachable from the up state, $S_{uu} = \$72$ and $S_{ud} = \$54$, both pay $\max(50-S, 0) = \$0$; of the two reachable from the down state, S_{ud} again pays \$0, while $S_{dd} = \$40.50$ pays $\max(50-40.50, 0) = \$9.50$.

Putting these pieces together, this is precisely a Markov decision process with states $\mathcal{S} = \{S_0, S_u, S_d\}$ and actions $\mathcal{A} = \{\text{exercise}, \text{hold}\}$. Transition probabilities are given by q and $1-q$, rewards by the intrinsic-value and terminal-payoff structure above, and $\gamma = 1/R$ discounts every step exactly as Chapter 5's tree already did.

Figure 19.3 redraws Chapter 5's tree in this chapter's language, labeling each decision node with its two available actions rather than with a single option value.

Chapter 5 solved this exact problem by backward induction, working from the known terminal payoffs back to today. Restated in this chapter's notation, that solution is

$$\begin{aligned}
Q^*(S_u, \text{exercise}) &= \$0.00, & Q^*(S_u, \text{hold}) &= \gamma\big[q(0) + (1-q)(0)\big] = \$0.00,\\
Q^*(S_d, \text{exercise}) &= \$5.00, & Q^*(S_d, \text{hold}) &= \gamma\big[q(0) + (1-q)(9.50)\big] \approx \$4.8718,\\
Q^*(S_0, \text{exercise}) &= \$0.00, & Q^*(S_0, \text{hold}) &= \gamma\big[q\,V^*(S_u) + (1-q)\,V^*(S_d)\big] \approx \$2.5641,
\end{aligned}$$

where $V^*(S_u) = \max(0.00, 0.00) = \$0.00$ and $V^*(S_d) = \max(5.00, 4.8718) = \$5.00$, since exercising is optimal at the down node. The optimal policy is therefore to hold at S_0 and S_u and exercise at S_d, giving an American put value of $V^*(S_0) \approx \$2.5641$, the value Chapter 5 reports.

Tabular Q-Learning

Backward induction requires knowing q, γ, and every terminal payoff in advance, the closed-form knowledge Section 19.2.2 argued is unavailable in most real trading and portfolio problems. **Q-learning** instead estimates Q^* from simulated experience alone, never using q directly in its update rule.

Starting from an arbitrary initial table $Q(s, a)$ (here, all zeros), the agent repeatedly simulates an episode: at each state it chooses an action using an ε-greedy policy (with probability ε, act randomly to keep exploring; otherwise, act greedily with respect to the current Q-table), observes the resulting reward r and next state s', and updates

$$Q(s, a) \leftarrow Q(s, a) + \alpha\Big[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\Big],$$

Wulin.Suo@Queensu.ca

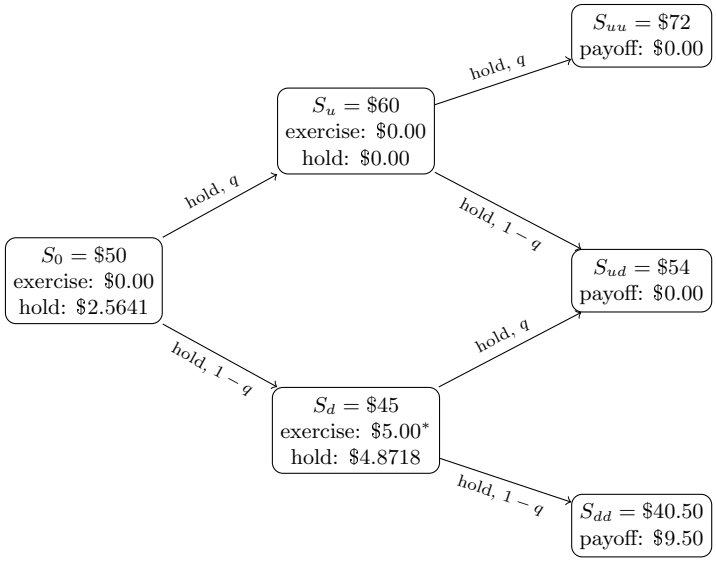


Figure 19.3: The early-exercise decision of Chapter 5's American put, redrawn as a three-state Markov decision process, with the optimal action at each node marked by an asterisk

where α is a learning rate. This update rule, known as **temporal-difference learning**, nudges the current estimate $Q(s, a)$ toward a bootstrapped target, the observed reward plus the discounted value of the best action available in the next state, without ever requiring a model of the transition probabilities.

The agent learns q's effect on the optimal decision purely by observing many simulated up and down moves and seeing which actions paid off, in the same statistical spirit as every other machine learning method in this book learning a relationship from data rather than being told it directly.

A Worked Example: The First Two Learning Steps

Starting from a Q-table initialized entirely to zero, with a learning rate of $\alpha = 0.1$, suppose the very first simulated episode reaches S_d, chooses **hold** (all actions still look identically worthless at this point, so the choice is effectively random), and the simulated down-move occurs, landing on the terminal payoff of \$9.50 at S_{dd}. Since this transition ends the episode, the bootstrapped target's continuation term is zero, and the update is

$$Q(S_d, \text{hold}) \leftarrow 0 + 0.1\big[9.50 + \gamma(0) - 0\big] = 0.1(9.50) = \$0.9500,$$

the first nonzero entry the table ever acquires. Now suppose a later episode reaches S_0, chooses **hold**, and the simulated move is down, transitioning to S_d with an immediate reward of \$0 (holding earns nothing until a terminal payoff is reached). The bootstrapped target now uses the current, already partially learned table, $\max_{a'} Q(S_d, a') = \max(0, 0.9500) = 0.9500$, giving

$$Q(S_0, \text{hold}) \leftarrow 0 + 0.1\big[0 + \gamma(0.9500) - 0\big] \approx 0.1(0.9615)(0.9500) \approx \$0.0913.$$

Two updates in, the table already shows the qualitative shape of the true solution, $Q(S_d, \text{hold})$ noticeably positive and $Q(S_0, \text{hold})$ smaller but nonzero, even though both values are still far from their closed-form targets of \$4.8718 and \$2.5641.

This is the mechanism the 100,000-episode simulation below repeats: each terminal payoff nudges the value of the state that led to it, and that nudge is itself propagated one more step backward the next time an earlier state is visited, gradually diffusing the terminal reward's information all the way back to S_0 without ever consulting q directly.

The companion notebook simulates this process for 100,000 episodes, using a learning rate that decays with each state-action pair's visit count and an exploration rate ε that decays from 0.30 to 0.02 over training, standard choices that guarantee enough early exploration to discover the exercise decision at S_d while still

Wulin.Suo@Queensu.ca

converging to a stable, greedy policy by the end of training. Table 19.2 compares the learned Q-table to the closed-form values derived above.

State-action	Closed-form Q^*	Learned Q (Q-learning)	Relative gap
$(S_0, \text{exercise})$	\$0.0000	\$0.0000	—
(S_0, hold)	\$2.5641	\$2.5328	1.22%
$(S_u, \text{exercise})$	\$0.0000	\$0.0000	—
(S_u, hold)	\$0.0000	\$0.0000	—
$(S_d, \text{exercise})$	\$5.0000	\$5.0000	0.00%
(S_d, hold)	\$4.8718	\$4.9234	1.06%

Table 19.2: Closed-form Bellman-optimal Q-values (from backward induction) versus tabular Q-learning's learned estimates after 100,000 simulated episodes

Comparing the Learned and Closed-Form Solutions

The learned policy matches the closed-form optimal policy exactly: hold at S_0 (\$2.5328 > \$0.0000), exercise at S_d (\$5.0000 > \$4.9234), and, at S_u, both actions are learned as worth \$0.0000, an exact tie that matches Chapter 5's own observation that "both exercise and continuation are worth \$0" at that node, so which action the tie-break happens to favor is economically inconsequential.

Every "exercise" value converges exactly, with no residual error, since exercising ends the episode immediately and its reward is observed without any noise; every "hold" value converges to within roughly 1% of the true value but not exactly, since a "hold" estimate is a bootstrapped average built from a finite number of noisy sampled continuations rather than an exact expectation computed from a known q.

This gap is not a bug to be engineered away; it is the fundamental price of model-free learning, and it is the single most important lesson this worked example offers: when the transition probabilities and payoffs are genuinely known in closed form, as they are in Chapter 5's tree, backward induction is strictly better, exact, instantaneous, and requiring no simulated data at all, and reinforcement learning should not be used. Reinforcement learning earns its keep only in the more realistic setting, developed throughout the rest of this chapter, where no such closed form exists and simulated or historical experience is the only source of information available.

19.3.2 From Tables to Function Approximation: Deep Q-Networks

Tabular Q-learning stores one number for every state-action pair, which works only because Section 19.3.1's example has just three states and two actions. A realistic trading state, some combination of inventory, elapsed time, recent order-flow imbalance (Chapter 14), and a handful of price-based features, takes values from a continuous or extremely large discrete space, making an explicit table impossible to store or to visit often enough to fill in.

The standard fix, introduced by Mnih et al. (2015) and known as the **Deep Q-Network** (DQN), replaces the table with a neural network $Q(s, a; \theta)$, the feedforward architecture Chapter 6's Section 6.6.4 introduced, trained to approximate the same Bellman-optimal Q^* function. The network takes a state (or a fixed-size feature vector describing one) as input and outputs one value per possible action; training minimizes the squared temporal-difference error,

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\big)^2\Big],$$

using the same gradient-descent machinery Chapter 6's Section 6.3.2 already developed for supervised learning, with θ^- a periodically-updated copy of the network's own parameters (a "target network") used to keep the bootstrapped target from chasing a constantly-moving estimate, one of two stabilization tricks, alongside storing and randomly resampling past transitions from a large "replay buffer" rather than training only on the most recent experience, that made deep Q-learning reliable enough to train at all.

Wulin.Suo@Queensu.ca

The underlying update rule is otherwise identical to tabular Q-learning: the network simply generalizes across similar states instead of memorizing each one independently, letting the agent act sensibly in a state it has never visited before, provided that state resembles ones it has seen, the same generalization argument that justifies every other neural network in this book.

Deep Q-networks are also considerably more finicky to train than tabular Q-learning, in a way worth naming explicitly given this book's repeated warnings, in Chapter 12's badly-scaled optimizer example among others, that a plausible-looking numerical method can silently fail to converge without an obvious error message.

A DQN's training loss can decrease steadily while the actual learned policy remains poor, since the loss measures how well the network fits its own, still-inaccurate bootstrapped targets rather than how good the resulting policy actually is. The only reliable check is to evaluate the policy's real performance directly. That is the discipline Section 19.3.1's worked example applied, comparing learned Q-values against an independently known closed-form answer, and the discipline Section 19.5.2 generalizes into a full evaluation methodology for problems where no closed-form answer exists to check against.

19.3.3 Policy Gradient Methods and Continuous Actions

Q-learning and its deep variant both learn a value function and derive a policy from it implicitly, by acting greedily. This works cleanly when the action set is small and discrete, exercise or hold, buy or sell, but breaks down when the natural action is continuous, exactly how many shares to trade this period, or exactly what target weight to assign an asset, since $\max_{a'} Q(s', a')$ has no simple closed form once a' ranges over a continuum.

Policy gradient methods sidestep this by parameterizing the policy directly, $\pi_\theta(a \mid s)$, typically as a neural network outputting the parameters of a continuous distribution (a mean and standard deviation, for instance), and adjusting θ to directly increase the expected return, using the policy gradient theorem,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\Big[\nabla_\theta \log \pi_\theta(a \mid s)\, G_t\Big],$$

which says, informally, that actions followed by a higher-than-expected return should be made more likely, and actions followed by a lower-than-expected return should be made less likely, with the size of the nudge proportional to how surprising the return was.

Actor-critic methods reduce the noise in this estimate by learning a value function (the "critic") alongside the policy (the "actor") and using it to judge whether a given return was better or worse than expected from that state, rather than judging it against the raw, highly variable return alone.

Proximal Policy Optimization (PPO), introduced by Schulman et al. (2017), is the most widely used modern policy-gradient algorithm. It constrains each policy update to stay close to the previous policy, preventing the large, destabilizing updates that plagued earlier policy-gradient methods, and its combination of reasonable sample efficiency and training stability has made it the default starting point for continuous-action problems across robotics, game-playing, and, increasingly, the trading and portfolio applications developed next.

Table 19.3 summarizes the four families of methods this chapter has introduced, alongside the setting each is best suited to and the concrete finance application, developed in the sections that follow, each is most naturally applied to.

19.3.4 Learning from Expert Demonstrations: Imitation and Inverse Reinforcement Learning

Every method developed so far learns purely from trial and error, an agent that starts knowing nothing about which actions are good must discover this for itself through the ε-greedy exploration of Section 19.3.1 or its neural-network analogues.

When a record of a skilled human decision-maker's past choices already exists, an experienced trader's execution decisions, say, a faster alternative is available: **behavioral cloning** treats the expert's historical state-action pairs as an ordinary supervised training set and fits a policy $\pi_\theta(a \mid s)$ to predict the expert's

Wulin.Suo@Queensu.ca

Method	Learns	Best suited to	Natural finance application
Tabular Q-learning	Action-value function $Q(s, a)$	Small, discrete state and action spaces	Section 19.3.1's early-exercise decision; small discretized inventory or execution problems
Deep Q-Network (DQN)	Neural-network approximation of $Q(s, a; \theta)$	Large or continuous state spaces, discrete actions	Execution and market-making agents (Section 19.4.1) with rich order-book state features
Policy gradient (REINFORCE, actor-critic)	A parameterized policy $\pi_\theta(a \mid s)$ directly	Continuous action spaces	Trade-size or quote-skew decisions that vary continuously rather than over a small discrete menu
Proximal Policy Optimization (PPO)	A parameterized policy, with a stabilized update rule	Continuous actions, need for reliable, stable training	Portfolio-rebalancing agents (Section 19.4.2) and production execution systems such as Section 19.5.1's LOXM

Table 19.3: A summary of the reinforcement learning methods introduced in this chapter

action from the state, using the same classification machinery Chapter 6 developed, sidestepping reinforcement learning's exploration problem entirely by never requiring the agent to try an action the expert did not.

Behavioral cloning has a well-known weakness: it is only ever as good as the expert it imitates, and it can fail badly in states the expert rarely visited, since a supervised classifier trained only on an expert's typical behavior has no guidance for how to recover once the agent's own small errors push it into an unfamiliar state the expert's demonstrations never covered, compounding over a long trajectory in a way a single supervised prediction never does.

Inverse reinforcement learning takes a different approach to the same underlying idea. Rather than directly imitating the expert's actions, it infers the reward function the expert appears to be optimizing, then solves the resulting Markov decision process using this chapter's own methods. The resulting policy can, in principle, generalize sensibly to states the expert never actually visited. The cost is a harder and less well-posed estimation problem, since many different reward functions can often rationalize the same observed expert behavior equally well.

Both techniques are directly relevant to the applications developed next: a system like Section 19.5.1's LOXM, reported to be trained on "a large history of past trades together with simulated trading scenarios," plausibly combines this kind of demonstration-based learning, bootstrapping an initial policy from historical human and algorithmic execution decisions, with further trial-and-error refinement in simulation, rather than learning a trading policy from pure, unguided exploration alone.

This hybrid approach is common in practice precisely because it addresses reinforcement learning's sample-inefficiency problem directly: starting from a reasonable, expert-informed policy rather than from nothing gives the trial-and-error refinement stage a far smaller gap to close, the practical advantage that makes imitation-based pretraining worth the additional complexity in a genuinely high-stakes application.

19.4 Financial Applications

This part points the machinery at execution, allocation, and hedging, recovering earlier chapters' answers where they exist.

Wulin.Suo@Queensu.ca

19.4.1 Reinforcement Learning for Optimal Execution and Market Making

Optimal execution was among the earliest documented applications of reinforcement learning in finance. Nevmyvaka, Feng, and Kearns (2006) applied tabular Q-learning, structurally similar to Section 19.3.1's worked example but with a far larger state space built from order-book features, to real historical limit order book data. They found that a learned execution policy could meaningfully reduce trading costs relative to the simple TWAP and VWAP benchmarks Chapter 13's Section 13.3.4 introduced, without ever being given a closed-form market-impact model to optimize against.

The state in a modern version of this problem typically includes remaining inventory, time remaining in the execution window, the current bid-ask spread, and recent order-flow imbalance; the action is how many shares to submit, and at what limit price, this period; and the reward is the negative implementation shortfall (Chapter 13's Section 13.6.4 transaction-cost-analysis measure) realized on that period's fill.

Unlike Almgren-Chriss, which commits in advance to a fixed schedule regardless of how the market subsequently behaves, a reinforcement-learned execution policy can condition its trade size on the order book it actually observes, trading faster when liquidity is unusually deep and slowing down when the book thins out, a genuinely adaptive behavior no closed-form schedule can replicate.

A Worked Example: Q-Learning Recovers the Almgren-Chriss Optimal Trajectory

Section 19.3.1's worked example verified Q-learning against a known answer on a small, three-state problem. It is worth doing the same on a problem closer to genuine trading practice: Chapter 13's own Almgren-Chriss worked example, the 250-share order split across $N = 5$ intervals with impact coefficient $\eta = 0.01$, per-interval volatility $\sigma = 2\%$, and risk aversion $\phi = 0.5$, whose closed-form optimal trajectory left unexecuted shares (250, 194.2, 142.4, 93.4, 46.2, 0) at a total risk-adjusted cost of 139.4.

Reformulated as a Markov decision process, the state at interval j is the inventory x_{j-1} still unexecuted, discretized to the nearest 5 shares (a grid of 51 possible inventory levels), and the action is how many of those shares, again in multiples of 5, to sell this interval.

The reward for selling a_j shares and leaving $x_j = x_{j-1} - a_j$ unexecuted is the negative of Chapter 13's own per-interval cost,

$$r_j = -\left[\eta \frac{a_j^2}{\tau} + \phi\sigma^2\tau\, x_j^2\right],$$

with $\tau = 1$ matching Chapter 13's own normalization, and the last interval ($j = 5$) forces full liquidation, $x_5 = 0$, exactly as the closed-form problem requires. Unlike Section 19.3.1's tree, this environment is entirely **deterministic**: choosing an action leaves no residual uncertainty about the resulting state or reward, since Almgren-Chriss prices timing risk analytically through the variance term $\phi\sigma^2\tau x_j^2$ rather than by exposing the agent to a randomly sampled price path.

Reinforcement learning is still the right tool here for a different reason than in Section 19.3.1: not to cope with stochastic transitions, but to search a state-action space of roughly 5,300 discretized possibilities, far too many to enumerate by hand, without ever being given the closed-form $\sinh(\cdot)$ trajectory formula Chapter 13 derived analytically.

Exact backward-induction dynamic programming over this same discretized grid, computable directly since only 51 inventory levels and 4 real decision points are involved, gives an optimal discretized trajectory of (250, 195, 145, 95, 45, 0) shares at a total cost of 139.52, close to but not identical to the continuous closed-form values, an expected consequence of rounding a continuous optimum onto a 5-share grid rather than any error in the discretization itself.

Tabular Q-learning, trained model-free for 600,000 episodes with an ε-greedy behavior policy decaying from $\varepsilon = 0.5$ to 0.05 and a visit-count-decayed learning rate, **converges to this same trajectory and this same cost**, 139.52, not merely close to it.

This is a meaningfully different outcome from Section 19.3.1's worked example, where the learned "hold" values settled within roughly one percent of their closed-form targets rather than matching them exactly: because this environment is deterministic, every reward Q-learning observes is noise-free, so once training has explored a state-action pair enough times to identify its best continuation, the learned value converges to it exactly, with no residual estimation error left to average away.

Wulin.Suo@Queensu.ca

Figure 19.4 makes the backward-induction computation itself concrete. Dynamic programming does not work through the five intervals in the order a trader would experience them; it starts from the one stage with no real decision left, the forced liquidation at x_4, and solves backward, one stage at a time, using each already-solved stage's value function to solve the one before it, until $V_1(x_0)$ finally gives the answer for the full, five-interval problem.

Only after this entire backward pass is complete can the optimal trajectory be traced forward, choosing at each state whichever action attains the value just computed for it, the bottom row of the figure.

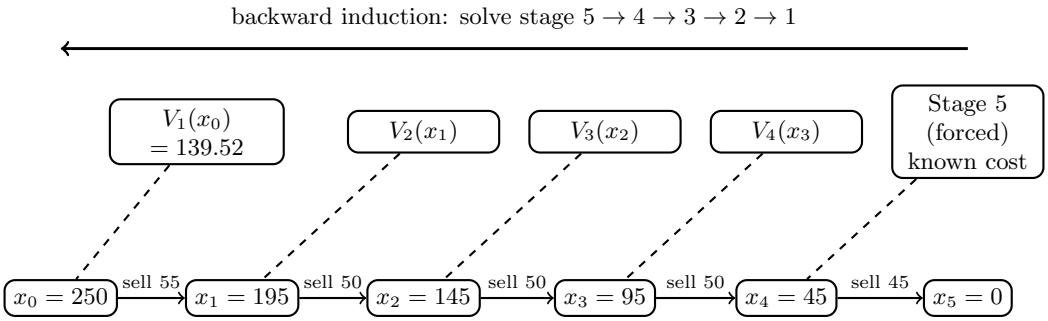


Figure 19.4: Backward-induction dynamic programming for the execution problem

Table 19.4 places the Q-learning-recovered trajectory alongside the uniform TWAP schedule and the fully aggressive single-shot execution Chapter 13 also scored, confirming the same efficient-frontier ordering Chapter 13 established analytically now emerges from an agent that was never given the underlying cost formula at all.

Strategy	Unexecuted-share trajectory	Total cost
Fully aggressive (single shot)	$(250, 0, 0, 0, 0, 0)$	625.00
Uniform TWAP	$(250, 200, 150, 100, 50, 0)$	140.00
Q-learning (learned, discretized)	$(250, 195, 145, 95, 45, 0)$	139.52

Table 19.4: Total risk-adjusted execution cost under three strategies for the same 250-share, $N = 5$-interval order ($\eta = 0.01$, $\sigma = 2\%$, $\phi = 0.5$)

Figure 19.5 plots the same three trajectories, making the front-loading pattern immediately visible: the fully aggressive strategy liquidates instantly, the uniform TWAP schedule is a straight line, and the Q-learning trajectory sits just below TWAP from the first interval on, with fewer shares left unexecuted, trading a little faster than uniform while inventory is largest and tapering to match TWAP by the final interval, exactly the mild front-loading Chapter 13's continuous closed-form solution also produced.

Market making extends naturally to the same framework. Chapter 13's Avellaneda-Stoikov model derives a closed-form optimal quote skew from an assumed order-arrival intensity function and a known risk-aversion parameter.

A reinforcement-learning market maker instead learns a quoting policy directly from simulated or historical order flow, with state features again drawn from current inventory, time-to-close, and order-book depth, action given by the bid and ask quotes (or, in a simplified discrete-action version, a small set of candidate skews), and reward given by realized spread capture net of inventory risk and adverse selection, the same adverse-selection concern Chapter 14's Glosten-Milgrom model formalizes. The appeal is identical to the execution case: a learned policy can adapt to order-arrival patterns that deviate from Avellaneda-Stoikov's assumed functional form, at the cost of requiring a large amount of representative training data and offering none of the closed-form model's transparency about why a given quote was chosen, a tradeoff this chapter's closing sections return to directly.

Wulin.Suo@Queensu.ca

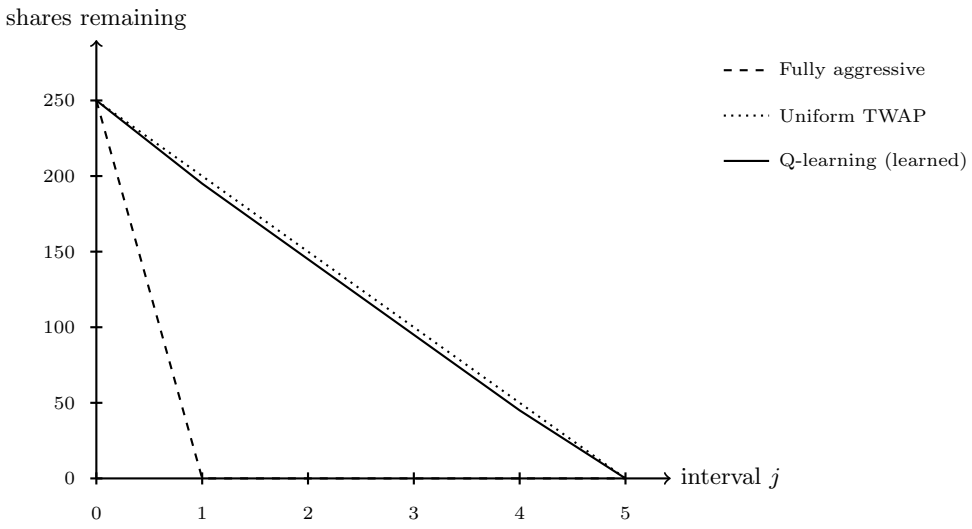


Figure 19.5: Unexecuted-share trajectories under the three strategies of Table 19.4

Designing States, Actions, and Rewards in Practice

Neither problem above specifies its Markov decision process uniquely, and the specific design choices a practitioner makes matter enormously to whether the resulting agent actually learns something useful. A state representation that omits recent order-flow imbalance discards the predictive signal Chapter 14's order-flow-imbalance regression showed carries real information about near-term price movement, weakening the agent below what a well-informed human trader, or a simpler rule-based system that explicitly conditions on that signal, could achieve. A state that includes too many redundant or noisy features, conversely, slows learning by forcing the agent to discover on its own which features actually matter, the feature-engineering discipline Chapter 6's Section 6.5.2 already developed for supervised models, now facing the added difficulty that a reinforcement-learning agent must learn this from its own trial and error rather than from a fixed, already-labeled training set.

Reward design carries the same tension in sharper form, because Section 19.6 shows that an imperfect reward does not merely produce a slightly worse agent, it produces an agent that is often precisely, confidently wrong about what it is optimizing. A common practical compromise is to shape the reward with an intermediate, densely observed signal, a small penalty proportional to inventory held at each step, say, in addition to the sparse, terminal profit-and-loss outcome. This speeds up learning considerably, since the agent gets useful feedback every step rather than only at the end of a long execution episode. But it introduces its own risk: if the shaping term is not carefully scaled relative to the true objective, the agent can learn to over-optimize the intermediate proxy, holding needlessly little inventory, say, at the expense of the actual goal it was meant merely to support learning toward. There is no universal recipe for getting this balance right; in practice it is validated the same way any other model choice in this book is validated, by comparing the resulting policy's realized, out-of-sample performance against the closed-form or rule-based benchmark it is meant to improve on, exactly as Section 19.5.2 recommends.

19.4.2 Reinforcement Learning for Portfolio Management

Chapter 12's mean-variance framework solves a single-period allocation problem: given today's estimated expected returns and covariance matrix, find the weights that optimize a risk-return tradeoff today. Section 12.4's practical-allocation discussion acknowledges that real portfolios are rebalanced repeatedly and that transaction costs make each rebalancing decision depend on the portfolio's current position, but it stops at flagging this complexity rather than solving the resulting multi-period problem directly.

Framed as a Markov decision process, the state is the portfolio's current holdings together with estimated risk-factor exposures (Chapter 12's Section 12.3.3 multi-factor model is a natural source of these features);

Wulin.Suo@Queensu.ca

the action is a new target weight vector, or equivalently the trades needed to reach it. The reward combines the period's realized return with an explicit penalty for transaction costs and, if desired, for realized volatility or drawdown, letting the agent learn a rebalancing policy that trades off today's allocation quality against the cost of getting there, rather than treating every rebalancing decision as a fresh, myopic optimization the way Chapter 12 does.

This framing is genuinely more general than Chapter 12's static optimization, and that generality is exactly its cost. A reinforcement-learning portfolio agent requires either a large amount of representative historical data or a realistic market simulator to train in, since a poorly specified reward (one that fails to penalize drawdown, say, or that rewards raw return without any risk adjustment) will produce a policy that optimizes the wrong objective with no warning that anything has gone wrong, a version of the "reward hacking" problem Section 19.6 discusses in more depth.

In practice, most institutional applications of reinforcement learning to portfolio management remain closer to research than to production, used to explore whether a learned, adaptive rebalancing policy can outperform Chapter 12's static mean-variance benchmark net of realistic transaction costs, rather than to replace that benchmark outright. The honest current state of the field is that reinforcement learning has demonstrated clearer, more reproducible gains in the execution and market-making applications of Section 19.4.1, where the reward (realized trading cost) is comparatively unambiguous to specify, than in full portfolio management, where the "right" reward is itself a genuinely open question tied to a client's actual, often unstated, risk preferences.

Multi-period portfolio choice has a classical closed-form precedent worth naming, since it clarifies exactly what a learned policy would need to rediscover on its own. Chapter 1's Section 1.5.1 introduced the **Kelly criterion** for a single repeated bet; the same result, maximizing the expected *logarithm* of long-run wealth rather than expected wealth itself, extends naturally to repeated portfolio rebalancing under a power-utility objective, producing an allocation fraction that avoids both the ruin risk of over-betting and the excessive caution of under-betting. This is itself a sequential decision problem with a known closed-form answer under simplifying assumptions, constant, known return and variance parameters, the "Chapter 12 could already have gone further" observation that opened Section 19.2.2. It gives a useful sanity check for a learned rebalancing agent: on a simplified simulated market where the Kelly fraction is known exactly, a correctly implemented reinforcement-learning portfolio agent should converge toward that same fraction, an internal-consistency check with the same purpose as Section 19.3.1's comparison against Chapter 5's closed-form option value, before that agent is ever trusted on a real, non-simplified market where no such closed form is available to check against.

A Worked Example: Why Model-Free Search Struggles Here

Running that sanity check honestly, rather than only asserting it would work, is instructive precisely because it does not go as cleanly as Sections 19.3.1 and 19.4.1. Chapter 1's repeated bet, an even-money wager ($b = 1$) won with probability $p = 0.6$, has closed-form optimal fraction $f^* = p - q/b = 0.20$ and long-run growth rate $g(f^*) \approx 0.0201$. Framed as a Markov decision process, this is a single, repeated state (the optimal fraction does not depend on current wealth, so no wealth variable needs to be tracked) with action f discretized to a grid of 5% increments, $f \in \{0.00, 0.05, \ldots, 0.95\}$, and an immediate, noisy reward $r = \ln(1 + f)$ with probability p or $r = \ln(1 - f)$ with probability q, drawn independently every period, no bootstrapping required since there is no next state to look ahead to. In other words this is a multi-armed bandit, structurally identical to Section 19.2.3's venue-routing problem with twenty arms in place of three. What it does not share with that problem is any hope of an easy answer, and the reason is worth stating before the numbers arrive.

What makes this problem genuinely hard for a model-free method is the relationship between the reward's noise and the size of what there is to learn: the gap between the optimal action's growth rate and its nearest neighbors on the grid, $g(0.20) - g(0.15) \approx 0.0013$ and $g(0.20) - g(0.25) \approx 0.0013$, is tiny, while a single bet's outcome swings between $\ln(1.20) \approx 0.182$ and $\ln(0.80) \approx -0.223$, a standard deviation roughly 150 times larger than the gap Q-learning needs to resolve.

Trained for 200,000 simulated bets with the same ε-greedy, visit-count-decayed setup used throughout this chapter, tabular Q-learning settles on $f = 0.15$, not $f^* = 0.20$, leaving $g(0.20) - g(0.15) \approx 0.0013$ of achievable growth rate on the table, about 6.4% of the optimal rate. This is a real, if modest, miss, not the

Wulin.Suo@Queensu.ca

exact or near-exact recovery Sections 19.3.1 and 19.4.1 demonstrated. Figure 19.6 shows why: the growth-rate curve is nearly flat across the entire 0.10-to-0.30 range, so the two marked points, the true optimum and where Q-learning actually converges, sit almost on top of each other.

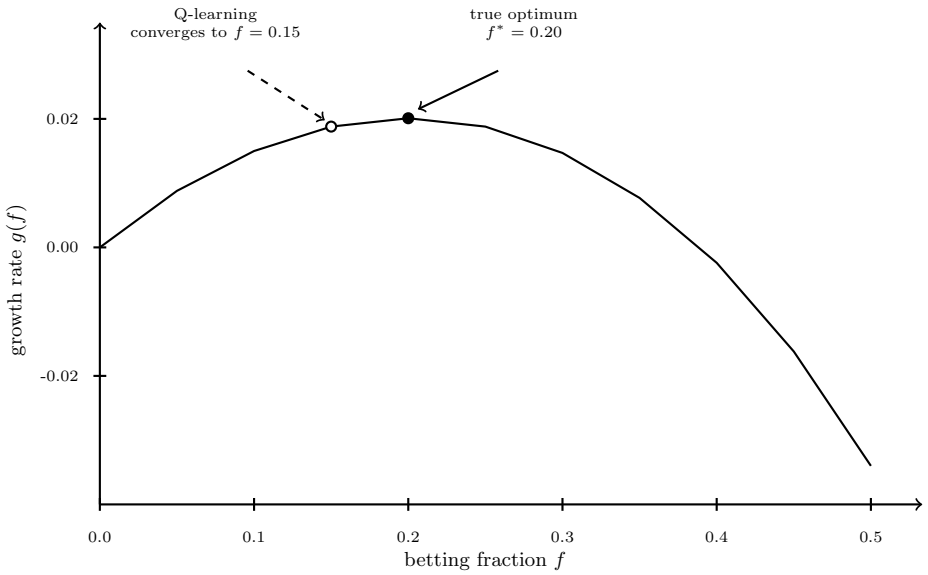


Figure 19.6: The Kelly growth-rate curve $g(f) = p\ln(1+f) + q\ln(1-f)$ for $p = 0.6$, $b = 1$

The contrast worth drawing out is what a **model-based** approach, Section 19.2.5's other category, does with the identical 200,000 observations. Rather than treating each bet's outcome as an opaque reward to average over a large discretized action grid, it exploits the fact that this particular problem has a compact underlying model: a single unknown parameter, the win probability p, plugged directly into Chapter 1's closed form.

Simply counting wins and losses across the same 200,000 simulated bets gives an observed win frequency $\hat{p} = 0.6003$, and substituting into $\hat{f}^* = \hat{p} - (1-\hat{p})/b$ gives $\hat{f}^* \approx 0.2006$, accurate to within 0.0006 of the true optimum, an order of magnitude closer than Q-learning's answer from the exact same data. This is Section 19.2.5's model-based-versus-model-free distinction made numerically concrete rather than only asserted, and it sharpens Section 19.2.2's opening argument: reinforcement learning is the right tool once no compact model is available, but when one is, as it is here, a single Bernoulli parameter standing in for the entire problem, exploiting that structure directly can be dramatically more sample-efficient than a black-box search over a discretized action grid, the gap this worked example just measured.

Not every sequential decision problem that can be written as a Markov decision process is equally well suited to being solved by treating it as one; recognizing when a problem's structure should be modeled directly rather than learned model-free is itself part of the practitioner's job, not something reinforcement learning relieves them of.

19.4.3 Deep Hedging: Learning to Hedge Under Frictions

Chapter 5's replication argument prices an option by continuously holding exactly Δ shares of the underlying, and Chapter 8's delta-normal example measures the risk of an option book through that same delta. Both rest on an assumption no trading desk actually enjoys: that the hedge can be rebalanced continuously and for free. A real desk pays a bid-ask spread and fees on every rebalancing trade, so it faces a genuine dilemma the frictionless theory simply defines away. Rebalance after every tick and transaction costs devour the option premium; rebalance rarely and the position runs unhedged in between, exposed to the gap risk Chapter 8 quantified. Deciding *when* and *by how much* to adjust the hedge, trade by trade, as the underlying moves

Wulin.Suo@Queensu.ca

and expiry approaches, is a sequential decision problem under uncertainty, which is to say it is this chapter's subject.

Cast in Section 19.2.1's terms, the state is what the hedger observes, time to maturity τ, the underlying price S_t, and the position h_t currently held; the action is the new position h_{t+1} to hold until the next decision point; and the objective is defined on the terminal outcome: let W_T be the hedger's final wealth after the option is settled and the hedge unwound, premium, trading gains, and all transaction costs included. **Deep hedging** chooses the policy to minimize a convex risk measure of the shortfall,

$$\min_{\pi} \ \rho(-W_T^{\pi}), \qquad \text{for example } \rho = \mathrm{ES}_{95\%},$$

where the risk measure is the Expected Shortfall Chapter 8 introduced, now serving not to measure a fixed position's risk but as the training objective a policy is optimized against.

A small worked example makes the trade-off concrete, and ties it back to numbers this book has already computed. Take Chapter 8's market maker, short the one-year \$50-strike call priced in Chapter 5 ($S_0 = K = \$50$, $r = 4\%$, $\sigma = 30\%$, $C_0 \approx \$6.88$, $\Delta_0 = 0.6115$), and have them hedge it over the option's life on 10,000 simulated price paths (geometric Brownian motion with real-world drift $\mu = 8\%$, 252 daily steps, seed 7 in the companion notebook), paying a proportional transaction cost of 50 basis points of traded value on every trade, the kind of friction an illiquid underlying imposes. The policy class is deliberately simple, a one-parameter family this chapter's earlier examples have prepared: rebalance to the current Black-Scholes-Merton delta only when the held position has drifted more than a *band* b away from it, so $b = 0$ is the textbook daily delta hedge and larger b trades less often. The very first trade is hand-checkable: the hedger buys $\Delta_0 = 0.6115$ shares at \$50, paying $0.005 \times 0.6115 \times \$50 \approx \$0.15$ in costs. Table 19.5 reports, across the 10,000 paths, the mean terminal profit and loss, its standard deviation, the mean total transaction cost, and the 95% Expected Shortfall of the terminal loss.

Policy	Mean P&L	Std. dev.	Mean cost	$\mathrm{ES}_{95\%}$
No hedge	−\$1.32	\$12.54	\$0.00	\$39.30
Daily delta ($b = 0$)	−\$1.60	\$0.59	\$1.56	\$3.01
Band $b = 0.02$	−\$1.37	\$0.61	\$1.34	\$2.82
Band $b = 0.06$	−\$0.95	\$0.70	\$0.93	\$2.56
Band $b = 0.20$	−\$0.60	\$1.92	\$0.53	\$5.76

Table 19.5: Hedging a short one-year \$50-strike call under 50-basis-point proportional transaction costs, across 10,000 simulated paths. The band policy with $b = 0.06$, found by searching the band grid for the lowest $\mathrm{ES}_{95\%}$, beats the textbook daily delta hedge on mean cost and tail risk simultaneously.

The table repays a careful read. Leaving the position unhedged is catastrophic, a \$12.54 standard deviation and a \$39.30 Expected Shortfall against a \$6.88 premium collected. The textbook daily delta hedge crushes that risk twenty-fold, but at a price the frictionless theory never mentions: \$1.56 of expected transaction costs, nearly a quarter of the premium. The band policy at $b = 0.06$, selected by searching the grid for the lowest $\mathrm{ES}_{95\%}$, is the striking row: it cuts mean costs by 40% *and* carries less tail risk than hedging every day, because each rebalance it skips saves a cost that would otherwise be paid with certainty, while the extra hedging error it accepts is small and diversifies across the path. Push the band too wide, though, and unhedged risk floods back in, as the $b = 0.20$ row shows. And the friction level, not taste, dictates the answer: rerun with a 5-basis-point cost and the ES-optimal band shrinks to 0.01, essentially the textbook hedge, confirming that Black-Scholes-Merton delta hedging is not wrong but is the frictionless limit of a more general policy the data-driven search recovers.

Deep hedging proper, introduced by Buehler, Gonon, Teichmann, and Wood in 2019, replaces this one-parameter band with a neural network policy $\pi_\theta(\tau, S_t, h_t)$ trained by stochastic gradient descent over simulated paths to minimize the same kind of convex risk measure, the step from a hand-designed policy family to a learned one that Section 19.3.2 already made for value functions. The gain is generality rather than magic. The network is never shown a delta, a gamma, or any other Greek, yet in the frictionless limit it

Wulin.Suo@Queensu.ca

rediscovers the Black-Scholes-Merton hedge on its own, and away from that limit it can absorb features no closed-form hedge accommodates gracefully: several hedging instruments at once, transaction costs of realistic shape, discrete trading times, position limits, and payoffs with no replicating formula at all. Reports of production use at major derivatives desks followed within a few years of the paper, making hedging, alongside Section 19.5.1's optimal execution, one of the two applications in this chapter where learned policies have genuinely reached practice. The caveats are the ones this chapter has already developed, and they bind here with full force: every path the policy learns from comes out of a simulator, so the quality of the learned hedge is bounded by the realism of that simulator, exactly Section 19.5.2's sim-to-real gap, and a neural hedging policy inherits the governance and explainability burden Chapter 18 attaches to any model whose decisions cannot be read off a formula.

19.4.4 Multi-Agent Dynamics and the Limits of a Single-Agent Framework

Every Markov decision process formalized in Section 19.2.1 assumes a single agent facing a fixed, if stochastic, environment, an assumption real financial markets violate in an important way: the "environment" a trading or market-making agent faces is itself made up, in large part, of other agents, some human, increasingly many themselves algorithmic, each adapting to the same price and order-flow signals the agent under study is adapting to. Chapter 14's hot-potato effect during the 2010 Flash Crash is a vivid, already-documented illustration of this dynamic: individually reasonable inventory-management rules, followed by many participants simultaneously, produced a collectively destabilizing outcome that no single participant's rule, viewed on its own, would have predicted. A reinforcement-learning agent trained against a static, historical simulator implicitly assumes other participants' behavior stays fixed, an assumption that becomes less accurate the larger and more influential the trained agent's own trading becomes, since a strategy profitable at small scale can behave very differently once it is large enough to move the same order flow it was trained to react to.

Multi-agent reinforcement learning extends the single-agent framework of this chapter to model several learning agents simultaneously, each adapting to the others' evolving behavior, and is an active area of research specifically for the market-making and execution problems Section 19.4.1 developed, since real limit order books are genuinely populated by multiple competing, adapting participants. The honest state of this research, as of this book's writing, is that multi-agent methods are considerably harder to train reliably than their single-agent counterparts, convergence guarantees that hold for tabular Q-learning in Section 19.3.1's simple setting generally do not extend to a setting where the environment itself is changing because other agents are learning too. Most production trading systems, including Section 19.5.1's LOXM, are best understood as single-agent policies trained against a historical or simulated environment rather than as genuine multi-agent systems, with the gap between that simplifying assumption and the true, multi-participant market being the sim-to-real and non-stationarity concerns Section 19.6 raises.

A useful, more tractable middle ground treats other participants' aggregate behavior as part of the state rather than as separate learning agents to be modeled explicitly. A single-agent method from Sections 19.3.1 through 19.3.3 can then respond to features like recent order-flow imbalance or realized volatility that summarize what other participants are collectively doing, without attempting the considerably harder problem of modeling each of them individually. That pragmatic compromise is what most of this chapter's real applications, execution and market making among them, actually rely on in practice.

19.5 From Simulation to Deployment

A policy trained in simulation has proved nothing yet, and this part covers what would.

19.5.1 Case Vignette: JPMorgan's LOXM

In 2017, JPMorgan disclosed LOXM, an equity trade execution system trained using reinforcement learning on a large history of past trades together with simulated trading scenarios, initially deployed on the bank's European cash equities desk.

Structurally, LOXM addresses the problem formalized in Section 19.4.1. At each point during an order's execution, the system decides how much of the remaining order to place, at what price, and for how long

Wulin.Suo@Queensu.ca

to wait before reassessing. It balances the risk of trading too aggressively, incurring unnecessary market impact, against the risk of trading too passively, missing the current opportunity and having to execute later at a worse price. That is the same impact-versus-timing-risk tradeoff Almgren-Chriss solves in closed form in Chapter 13, but learned here directly from data rather than derived from an assumed cost function. The system was reported to have been trained on millions of historical and simulated trading scenarios specifically so that it could adapt its behavior to prevailing market conditions rather than following one fixed schedule regardless of context, the central advantage Section 19.4.1 attributes to a learned execution policy over Almgren-Chriss's closed-form alternative.

LOXM is a useful case precisely because it illustrates both sides of this chapter's central tension in a single, real deployment. On one hand, it demonstrates that reinforcement learning for execution is not merely an academic exercise, a major bank judged a learned policy reliable enough to deploy on live client order flow, the kind of practical validation the closed-form models developed earlier in this book have had for decades and that any newer machine-learning-based alternative must eventually earn. On the other hand, a deep reinforcement-learning execution policy is far harder to explain to a regulator or an internal model validator than an Almgren-Chriss schedule, whose entire logic reduces to a handful of interpretable parameters, a governance burden Section 19.6 and Chapter 18's explainability toolkit both address directly, and one that any institution deploying a system like LOXM must take on explicitly rather than treat as an afterthought.

LOXM was also an early, rather than an isolated, example. In the years since 2017, industry reporting has described similar reinforcement-learning-based execution research and deployments at several other major banks and trading firms. This is consistent with this chapter's broader claim that optimal execution, alongside Section 19.4.3's hedging application, has the clearest track record of reinforcement learning actually reaching production, precisely because its reward, realized execution cost relative to a well-defined benchmark, is comparatively unambiguous to specify correctly, the property Section 19.4.2 identified as still generally missing from full portfolio management.

19.5.2 Simulation, Backtesting, and Off-Policy Evaluation

Every reinforcement learning agent in this chapter is trained by trial and error, which raises an operational question none of this book's earlier, purely supervised models faced: where does the trial and error actually happen? Training directly on live markets is rarely acceptable, since an untrained or partially trained policy will, by construction, take some genuinely bad actions while exploring, and a bad trading action costs real money in a way a bad prediction from a still-training classifier does not. The standard alternative is to train in a **simulator**, a model of the market environment built from historical data, that can be reset and replayed as many times as needed without financial consequence; Chapter 13's own backtesting framework (Section 13.6) is a close cousin of this idea. It inherits the same core limitation, a simulator built from historical data reflects how the market behaved historically, including in response to other participants' historical order flow, not necessarily how it would respond to a new agent's own actions once deployed at scale, a gap sometimes called the **sim-to-real gap** that is at least as serious a concern for a reinforcement-learning trading agent as ordinary backtest overfitting is for the supervised models Chapter 13 already warned about.

A related and more subtle problem is **off-policy evaluation**: given historical data generated by one policy (a human trader's decisions, say, or an older execution algorithm), how good would a different, newly trained policy have performed, without ever actually running it? This matters because historical market data reflects only the actions the deployed policy actually took, not the counterfactual outcomes of actions it did not take. That is the missing-counterfactual problem Chapter 18's algorithmic-recourse discussion raised for credit decisions, now recurring in a sequential setting where a single wrong counterfactual estimate can compound over many steps. Techniques such as importance sampling reweight historical outcomes to estimate what a new policy would have achieved, but their reliability degrades quickly the more the new policy's behavior diverges from the one that generated the historical data. That is why a genuinely novel learned policy is typically rolled out cautiously: first in simulation, then on a small fraction of live order flow, with continuous monitoring for the kind of performance drift Chapter 15's precision-drift and Chapter 18's population-stability-index discussions already developed for other models. It is not deployed at full scale on the strength of a backtest or an off-policy estimate alone.

This staged approach, simulator, then a small live allocation, then a gradually increasing one, is sometimes

Wulin.Suo@Queensu.ca

called **curriculum deployment**, and it mirrors a discipline this book has already recommended elsewhere for a different reason: Chapter 8's three-lines-of-defense model and Chapter 18's staged-deployment recommendation for high-stakes machine learning models both argue for this kind of graduated rollout, independent of whether the model in question is a reinforcement-learning policy or a static classifier. What is genuinely new to the reinforcement-learning setting is that the policy itself may continue to update during this staged rollout: a live, still-learning system rather than a fixed, already-frozen model. The monitoring in place must therefore detect not only a fixed policy's performance drifting away from its backtested expectation, but a still-adapting policy drifting toward behavior nobody has explicitly validated at all. That is a strictly harder monitoring problem than the ones Chapters 15 and 18 originally posed.

19.5.3 Evaluating Trained Policies: Metrics and Variance Across Training Runs

Chapter 13's Section 13.6.1 developed a standard toolkit, Sharpe ratio, maximum drawdown, hit rate, for evaluating a trading strategy's backtested performance, and every one of those metrics applies unchanged to a reinforcement-learning policy's simulated or live performance. What reinforcement learning adds is a genuinely new source of variability that a closed-form model like Almgren-Chriss simply does not have: since every method in this chapter is trained via a stochastic process, random initialization, ε-greedy exploration, and, for the neural-network methods of Sections 19.3.2 and 19.3.3, stochastic gradient descent itself, two training runs of the identical algorithm on identical data, differing only in their random seed, can converge to meaningfully different policies with meaningfully different realized performance.

Section 19.3.1's worked example makes this concrete in miniature. It was trained with a fixed seed (seed = 7) specifically so that its reported numbers, \$2.5328 and \$4.9234 among them, would be exactly reproducible for a reader rerunning the companion notebook. A different seed would converge to slightly different learned values, still close to the closed-form \$2.5641 and \$4.8718, since the underlying algorithm and its guarantees do not depend on the seed, but not numerically identical to the values reported in Table 19.2. In a small, three-state problem with a known correct answer to check against, this seed sensitivity is a minor curiosity; in a large, real trading application with no closed-form answer available, it is a serious practical concern, since a single training run's backtested performance could simply be a favorable draw rather than a property of the algorithm the institution intends to rely on.

The standard remedy is to train multiple independent runs from different seeds and report performance as a distribution rather than a single number. It is standard in machine learning generally, but has not been needed anywhere else in this book, whose supervised and closed-form models are deterministic given their input data. It is the statistical caution Chapter 6's confidence intervals and Chapter 7's out-of-sample RMSE comparisons already applied to estimation uncertainty, now applied to the additional uncertainty introduced by the training process itself, not just by the data.

19.5.4 Practical Considerations: What Deployment Actually Requires

The methods and applications developed above can leave an unrealistic impression that adopting reinforcement learning is primarily an algorithmic choice, DQN or PPO, Q-learning or actor-critic, when in practice the harder and more expensive decisions are organizational and infrastructural, and worth naming explicitly before this chapter closes.

A trustworthy simulator is usually the binding constraint, not the algorithm. Every method in this chapter needs a large volume of training experience, and Section 19.5.2 already established why that experience typically cannot come from live markets. Building a market simulator accurate enough to train a genuinely useful policy, one that reproduces realistic order-book dynamics, price impact, and other participants' plausible reactions, is a substantial engineering undertaking in its own right, often larger than the effort spent on the learning algorithm itself. Few institutions can justify that project unless the underlying decision problem is important and recurring enough to be worth the fixed investment. That is why optimal execution at a large bank, a genuinely high-volume, high-value recurring decision, has been an early adopter while more bespoke, lower-frequency problems largely have not.

The required skill set differs from, and must complement rather than replace, this book's earlier chapters. A team building a reinforcement-learning trading system needs the market-microstructure fluency Chapters 13 and 14 developed, the statistical and machine-learning foundations of Chapter 6, and,

Wulin.Suo@Queensu.ca

on top of both, genuine expertise in the distinct engineering discipline of reinforcement learning itself: stable training, careful hyperparameter selection, and the evaluation methodology of Section 19.5.2. That combination is still comparatively rare, and most institutions build it gradually rather than hire for all of it at once. This is one more reason closed-form models like Almgren-Chriss and Avellaneda-Stoikov remain the right default choice for most applications: they require the market-microstructure expertise this book has developed throughout, but not the additional, specialized reinforcement-learning engineering capability on top of it.

The honest cost-benefit comparison is against the closed-form alternative, not against doing nothing. A closed-form model, once specified, costs almost nothing to run and is immediately auditable by the model-risk-management process Chapter 8 established; a reinforcement-learning policy costs substantially more to build, validate, and monitor on an ongoing basis, and that additional cost is only justified when the closed-form model's simplifying assumptions are demonstrably costing real money, the market-impact and reward-specification gaps Section 19.2.2 identified. A useful discipline, consistent with this book's compute-first approach throughout, is to build and validate the closed-form baseline first, know precisely what it costs in realized performance, and only then invest in a learned alternative with a clear, pre-specified bar for how much improvement would justify the added complexity and ongoing governance burden.

Compute cost is a real, ongoing line item, not a one-time expense. Training a deep Q-network or a PPO policy over the hundreds of thousands or millions of simulated episodes a realistic problem requires consumes meaningfully more computing resources than fitting any single supervised model elsewhere in this book. That cost recurs every time the policy is retrained, whether on a fixed schedule to keep pace with the non-stationarity Section 19.6 describes, or in response to a detected drift in live performance. This ongoing retraining cost, together with the ongoing monitoring cost Section 19.5.2 requires, belongs in the same total-cost-of-ownership comparison as the upfront simulator investment discussed above, not treated as a one-time project expense that ends once a policy is first deployed.

19.6 Challenges in Reinforcement Learning for Finance

Reinforcement learning inherits every model-risk concern raised elsewhere in this book and adds several that are specific to its sequential, trial-and-error structure.

Reward specification is the entire problem in disguise. An agent optimizes the reward function it is given, not the objective its designer actually had in mind, and the gap between the two is usually invisible until the agent finds and exploits it, a failure mode generally called **reward hacking**. An execution agent rewarded purely for beating a benchmark price, with no penalty for market impact left behind for the next order, can learn to trade in ways that look good on that one metric while quietly making the market worse for the desk's future orders; a portfolio agent rewarded for raw return with no risk adjustment can learn to take on the tail risk a real client would never accept. Specifying a reward that genuinely captures an institution's objective, including the parts that are hard to quantify, is consistently harder in practice than designing the learning algorithm itself.

Non-stationarity compounds ordinary concept drift. Chapter 15 and Chapter 18 both warned that a supervised model's accuracy can degrade as the world changes. A reinforcement-learning trading agent faces the same risk with an added twist, since other market participants adapt to *its* behavior too, particularly once a learned strategy is large enough to influence prices. The environment itself becomes something that changes partly *because of* the agent's own past actions, a genuinely adversarial dynamic no supervised classifier in this book has had to contend with.

Reinforcement learning is dramatically more sample-hungry than the supervised methods elsewhere in this book. Section 19.3.1's worked example needed 100,000 simulated episodes to estimate just six numbers to within roughly one percent, and that problem has only three states and two actions. Chapter 6's supervised classifiers, by contrast, routinely reach useful accuracy from a training set several orders of magnitude smaller relative to their problem's apparent complexity, because a supervised label provides direct, unambiguous feedback on every single example, while a reinforcement-learning reward, especially a delayed, terminal one, provides comparatively little information per episode about which of many earlier actions actually deserves credit for it, the credit-assignment difficulty this section returns to below. This sample inefficiency is precisely why Section 19.5.4's simulator investment is not optional infrastructure

Wulin.Suo@Queensu.ca

but the central prerequisite for any of this chapter's methods to work at all outside of small, hand-verifiable settings like this chapter's own worked example.

Exploration is expensive and risky in a way it is not in a game. The ε-greedy exploration in Section 19.3.1's worked example costs nothing beyond simulated time; the same exploration on a live trading desk means deliberately taking some actions believed to be suboptimal, in real markets, with real capital, specifically to learn whether they might be better than currently believed. This is a fundamentally different risk profile from a self-driving research agent exploring a video game, and it is the central reason simulation-based and off-policy training, Section 19.5.2's subject, are not merely convenient but close to mandatory for any serious financial application.

Explainability and governance do not disappear just because a policy performs well. A deep reinforcement-learning policy inherits all the opacity concerns Chapter 18 raised for deep neural networks generally, compounded by the fact that its behavior is defined over an entire trajectory of decisions rather than a single prediction, making a SHAP-style, single-decision explanation only a partial account of what the policy is actually doing. A model validator or regulator evaluating a system like Section 19.5.1's LOXM needs some account of why the policy behaves as it does across a representative range of market conditions, not just a performance summary, the model-risk-management discipline Chapter 8 established for market-risk models and Chapter 18 extended to explainability generally, now applied to a policy that acts rather than merely predicts.

Credit assignment gets harder the longer the horizon. A supervised classifier receives immediate, unambiguous feedback on every prediction it makes, Chapter 6's confusion matrix scores a decision the moment it is made. A reinforcement-learning agent, by contrast, often does not learn whether an early action was good or bad until many steps later, when the resulting reward finally arrives, the **credit assignment problem**. Section 19.3.1's worked example sidesteps this almost entirely, since its longest episode is only two decisions deep, but a full trading day's worth of execution decisions, or a multi-quarter portfolio-rebalancing episode, can make it genuinely difficult to tell which of many earlier actions actually caused a later gain or loss, slowing learning and demanding far more training data than a supervised problem of comparable apparent complexity.

Simulators are themselves models, with their own model risk. Section 19.5.2's sim-to-real gap is not merely a data-availability inconvenience. A market simulator built from historical data encodes its own assumptions about how prices and order flow behave, and an agent trained exclusively against a flawed simulator will learn a policy that is optimal for that simulator's biases rather than for the real market. That is no different in kind from Chapter 8's warning that a risk model is only as trustworthy as the assumptions behind it. Validating a trading simulator against out-of-sample market behavior, not just validating the agent trained inside it, is therefore a necessary part of the same model-risk-management discipline this chapter has invoked throughout.

The regulatory and legal framework for autonomous trading policies is still catching up. Chapter 8's model-risk-management guidance and Chapter 18's explainability requirements were largely written with static, single-prediction models in mind. A policy that adapts its own behavior over time, potentially continuing to learn after deployment, raises validation questions those frameworks do not yet answer: how often must a live, still-adapting policy be re-approved, and what constitutes an adequate explanation of a policy rather than a prediction? Existing supervisory guidance does not address either with the specificity it applies to a fixed logistic regression or gradient-boosted tree. Institutions deploying reinforcement learning in a regulated trading or advisory context should expect to extend, and in places invent, their own governance answers rather than assume existing frameworks already cover this case completely.

19.7 Key Takeaways

Reinforcement learning formalizes sequential decision-making as a Markov decision process, with states, actions, rewards, and a discount factor, and its objective, maximizing expected discounted return, generalizes the one-shot optimizations this book has relied on since Chapter 5.

The Bellman equation expresses a state's or action's value recursively in terms of immediate reward plus discounted continuation value, and Chapter 5's binomial-tree backward induction already solves a Bellman optimality equation by hand. This chapter's central worked example made that fact exact, reformulating the

Wulin.Suo@Queensu.ca

same tree as a three-state MDP and recovering its known value, \$2.5641, using model-free tabular Q-learning instead. The learned and closed-form solutions agree to within roughly one percent after 100,000 simulated episodes.

Deep Q-networks and policy-gradient methods, including actor-critic algorithms and PPO, extend the same value-based and policy-based logic respectively to large or continuous state and action spaces where a table is impossible to maintain.

Optimal execution and market making, Chapter 13's closed-form models, generalize naturally into reinforcement-learning problems once market impact and order-arrival behavior are too complex for a clean parametric form. That generalization has already been validated in research (Nevmyvaka, Feng, and Kearns, 2006) and in industry practice (JPMorgan's LOXM). Portfolio management admits the same framing but remains a harder, more open problem, chiefly because its reward is harder to specify unambiguously than an execution cost.

Every one of these applications inherits reinforcement learning's distinctive risks: reward misspecification, non-stationarity driven by other agents' adaptation, the expense and danger of live exploration, severe sample inefficiency relative to this book's earlier supervised methods, and an explainability burden that compounds Chapter 18's concerns rather than resolving them. All of these argue for simulation-based training, cautious staged deployment, and continuous monitoring, rather than treating a well-trained policy as a finished, self-sufficient product.

Markets are also not single-agent environments: other participants adapt to a learned policy's own behavior just as it adapts to theirs. That is why most production systems remain single-agent approximations of a genuinely multi-agent problem. It is also why the practical bottleneck to adopting any of this chapter's methods is usually a trustworthy simulator and the specialized engineering skill to train against it reliably, not the choice of algorithm itself.

19.8 Suggested Exercises

1. Using Section 19.3.1's three-state Markov decision process, verify by hand that $Q^*(S_d, \text{hold}) = \gamma[q(0) + (1-q)(9.50)] \approx \$4.8718$, and explain in your own words why immediate exercise is optimal at that state despite continuation having a positive expected value.

2. Suppose the down-move factor in Section 19.3.1's tree were $d = 0.8$ instead of 0.9, with $u = 1.2$ and $R = 1.04$ unchanged. Recompute q, the terminal payoff at the down-down node, and the closed-form value $V^*(S_0)$, and state whether the optimal policy at S_0 changes.

3. Explain why a discount factor of $\gamma = 1$ would be inappropriate for an infinite-horizon trading agent that earns a small positive expected reward every period, and describe what specifically goes wrong with the return G_t in that case.

4. Using this chapter's Q-learning update rule, compute by hand the new value of $Q(S_0, \text{hold})$ after a single simulated transition from S_0 to S_d with reward 0, given a current table of $Q(S_0, \text{hold}) = 2.00$, $Q(S_d, \text{exercise}) = 5.00$, $Q(S_d, \text{hold}) = 4.50$, a learning rate $\alpha = 0.1$, and $\gamma \approx 0.9615$.

5. Compare Almgren-Chriss (Chapter 13, Section 13.3.5) and a reinforcement-learning execution agent (Section 19.4.1) along three dimensions: the data required to use each, the interpretability of the resulting trading schedule, and each approach's ability to adapt mid-execution to an order book that behaves differently than assumed.

6. Describe a reward function for a portfolio-rebalancing agent (Section 19.4.2) that would encourage the exact kind of reward hacking Section 19.6 warns about, and propose a specific, concrete change to that reward function that would discourage it.

7. Explain why off-policy evaluation (Section 19.5.2) becomes less reliable the more a newly trained policy's behavior diverges from the policy that generated the available historical data, and describe one practical consequence this has for how a bank should stage the rollout of a new trading agent.

Wulin.Suo@Queensu.ca

8. A market-making desk considers replacing its Avellaneda-Stoikov quoting model (Chapter 13, Section 13.4) with a learned reinforcement-learning policy. List three pieces of evidence a model validator should require before approving this replacement, drawing on both this chapter's discussion and Chapter 8's model-risk-management framework.

9. Explain, in your own words, why maximizing expected log-wealth (the Kelly criterion, Section 19.4.2) produces a different, generally more conservative allocation than maximizing expected wealth directly, and describe why this makes the Kelly fraction a useful sanity check for a reinforcement-learning portfolio agent trained on a simplified simulated market.

10. Using Section 19.4.4's discussion, explain why a reinforcement-learning execution agent trained against a fixed historical simulator might perform worse than expected once deployed at large enough scale to influence the very order flow it was trained on, and describe one practical step a firm could take to detect this problem before it causes significant losses.

11. Explain why behavioral cloning (Section 19.3.4) can fail badly in a state the expert trader rarely visited, even though it performs well on states similar to the expert's typical behavior, and describe how inverse reinforcement learning's approach, inferring a reward function rather than directly imitating actions, is intended to address this specific weakness.

12. Section 19.2.3's desk has sent four orders to venue A, giving $Q(A) = 1.60$, and six to venue B, giving $Q(B) = 1.85$. (a) A fifth order to A fills at 3.00 basis points; apply the running-average rule to compute the updated $Q(A)$, and state which venue the greedy policy now selects. (b) A single fill has a standard deviation of 2.0 basis points. Compute the standard error of each of the two estimates and explain why the switch in part (a) is far weaker evidence than it looks. (c) A fourth venue D, whose true mean of 2.4 basis points beats every existing venue, is now connected, and the desk keeps $\varepsilon = 0.2$ with exploration spread uniformly over all four. Compute the new permanent shortfall $\varepsilon(\mu^* - \bar{\mu})$, compare it to the three-venue figure, and explain why adding a venue better than all the others nonetheless raises the per-order cost of a fixed exploration rate.

13. **Challenge (real data).** Download at least three years of daily prices for a real, liquid stock (via `yfinance`). (a) Discretize the state space into a small number of buckets based on the sign and magnitude of the trailing five-day return, in the spirit of Section 19.3.1's three-state example, and train a tabular Q-learning agent with actions {buy, hold, sell} and a reward equal to the next day's realized return (long) or its negative (short), using this chapter's Q-learning update rule. (b) Compare the trained agent's cumulative return over the full sample to a simple buy-and-hold benchmark, then re-run training on only the first half of the sample and evaluate the resulting policy on the second half, and report whether the agent's apparent advantage survives this split. (c) Using Section 19.5.2's discussion of off-policy evaluation, explain why backtesting a single historical price path is an especially weak form of policy evaluation for a trading agent, and describe one practical safeguard, drawing on Section 19.4.4 or Chapter 8's model-risk-management framework, a firm should require before deploying such an agent with real capital.

19.9 Further Reading

- Sutton, R. S., and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, Cambridge, MA, 2018. The standard textbook treatment of the Markov decision process framework, the Bellman equation, and value-based and policy-based methods developed throughout this chapter.

- Mnih, V., et al., "Human-Level Control through Deep Reinforcement Learning," *Nature*, vol. 518, 2015, pp. 529–533. The original Deep Q-Network paper underlying Section 19.3.2's function-approximation extension of tabular Q-learning.

- Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv*, 2017 (arXiv:1707.06347). The algorithm underlying the modern policy-gradient methods discussed in Section 19.3.3.

Wulin.Suo@Queensu.ca

- Nevmyvaka, Y., Y. Feng, and M. Kearns, "Reinforcement Learning for Optimized Trade Execution," *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 673–680. An early and influential application of tabular Q-learning to real limit-order-book execution data, underlying Section 19.4.1's discussion.

- Almgren, R., and N. Chriss, "Optimal Execution of Portfolio Transactions," *Journal of Risk*, vol. 3, no. 2, 2001, pp. 5–39. The closed-form optimal-execution model this chapter's Section 19.2.2 contrasts with a learned, model-free alternative.

- Avellaneda, M., and S. Stoikov, "High-Frequency Trading in a Limit Order Book," *Quantitative Finance*, vol. 8, no. 3, 2008, pp. 217–224. The closed-form inventory-based market-making model Section 19.4.1 generalizes into a learned quoting policy.

- Buehler, H., L. Gonon, J. Teichmann, and B. Wood, "Deep Hedging," *Quantitative Finance*, vol. 19, no. 8, 2019, pp. 1271–1291. The paper that introduced Section 19.4.3's risk-measure-based, neural-network approach to hedging under transaction costs and other frictions.

- Abbeel, P., and A. Y. Ng, "Apprenticeship Learning via Inverse Reinforcement Learning," *Proceedings of the Twenty-First International Conference on Machine Learning*, 2004, pp. 1–8. The foundational paper on inverse reinforcement learning discussed in Section 19.3.4.

Wulin.Suo@Queensu.ca

Wulin.Suo@Queensu.ca

Chapter 20

Agent-Based Models in Finance

20.1 Introduction

Every model in this book so far has taken one of two forms. Chapter 6's machine learning models estimate a function from a fixed dataset, implicitly assuming the data-generating process itself does not change while the model learns from it. Chapter 12's portfolio theory and Chapter 19's reinforcement-learning agents each reason about a single decision-maker, or a small number of them, optimizing against an environment that is either fixed or, at most, adapts slowly.

Neither approach is built to answer a question that turns out to matter a great deal in practice: what happens when a large number of participants, each following a simple, individually reasonable rule, interact repeatedly in the same market? Agent-based models (ABMs) answer exactly that question. Rather than solving for an equilibrium in closed form or estimating a statistical relationship from historical data, an ABM specifies a population of heterogeneous agents, each governed by a simple behavioral rule, sets them loose interacting in a simulated market, and treats whatever aggregate pattern emerges, a stable price, a bubble, a crash, a cascade of defaults, as the object of study.

This chapter's central claim is that emergence is the point, not a side effect. Chapter 14's account of the 2010 Flash Crash already described a version of this: no single participant's inventory-management rule was unreasonable in isolation, yet many such rules operating simultaneously produced a collectively destabilizing hot-potato effect that no single participant intended or could have predicted from their own rule alone. An ABM makes it possible to reproduce that kind of story directly, agent by agent, rather than only describing it after the fact.

Several worked examples run through this chapter, every one of them reproduced in the companion notebook, following the same compute-first discipline as every other chapter in this book. A zero-intelligence double-auction example shows that a market mechanism can produce an efficient outcome even when every participant in it behaves close to randomly.

A fundamentalist-chartist price-dynamics example shows how the same market can either converge smoothly toward fair value or spiral into a self-reinforcing bubble, depending only on the mix of trading strategies present, with no change to the market mechanism itself, and a 500-period extension of the same model shows fat tails and volatility clustering emerging from nothing more than two simple, fixed behavioral rules switching in and out of dominance. A financial-contagion example, built on the same clearing-vector logic underlying real central-bank stress-testing models, shows a shock to one institution's balance sheet cascading into the default of a second institution that was never directly exposed to the original shock at all, and a second network topology, compared directly against the first under the identical shock, shows that the same total loss can be concentrated into a small number of outright defaults or spread thinly across many surviving-but-weakened institutions, depending only on how the network is shaped.

A final pair of examples previews the AI-driven alternative to this chapter's fixed-rule agents, developed in full in Section 20.3.4. The first is a seller who learns which of two ask prices is more profitable through repeated trial and error, using Chapter 19's own Q-learning machinery, showing the honest limits of trial-and-error learning when an unlucky early experience locks in the wrong lesson. The second is a fundamentalist-chartist market in which the population mix itself is no longer switched by a threshold the modeler imposes,

Wulin.Suo@Queensu.ca

but by agents comparing each strategy's own recently realized profit through a discrete-choice rule, showing the same bubble-triggering threshold derived earlier in the chapter crossed endogenously rather than by assumption.

It is worth being explicit about where this leaves agent-based models among the three modeling paradigms this book has now covered. Chapter 6's supervised and unsupervised learning models answer the question "given this data, what function best describes it," and are validated by checking predictions against data the model never saw. Chapter 19's reinforcement-learning agents answer "given this environment, what sequence of actions maximizes reward," and are validated by their realized performance, in a backtest or in production.

Agent-based models answer a third, different question: "given these individual rules, what pattern emerges when many such agents interact," and, as Section 20.4.1 develops at length, are validated far more weakly than either of the other two, by whether a simulated pattern resembles an observed one, not by a held-out prediction or a realized return. None of the three approaches is more fundamentally correct than the others; they answer different questions, and knowing which question a given financial problem actually poses, a prediction problem, a sequential-decision problem, or an emergence problem, is itself one of this book's recurring practical lessons. Table 20.1 lays the three side by side.

	Supervised/unsupervised ML (Ch. 6)	Reinforcement learning (Ch. 19)	Agent-based models (this chapter)
Core question	What function best fits this data?	What action sequence maximizes reward?	What pattern emerges from many interacting rules?
Input	A fixed, historical dataset	An environment to interact with	A set of behavioral rules and a market mechanism
Validation	Held-out prediction accuracy	Realized backtest or live performance	Resemblance to observed stylized facts (weakest of the three)
Typical output	A trained predictive model	A trained policy	A simulated history and its emergent statistics

Table 20.1: Three modeling paradigms used in this book

A reader who has followed this book's growing emphasis on machines that learn might reasonably object that every rule described above, zero-intelligence, fundamentalist, chartist, is instead fixed by the modeler in advance, which sits oddly in a chapter that is supposed to belong in a book about AI in finance. That fixed-rule design is a deliberate methodological choice, not a limitation of agent-based modeling as such, and it is one this chapter makes on purpose so that Table 20.1's third column stays honest: isolating the question of what pattern emerges from a given set of interaction rules from the separate question of how an individual agent's own rule might itself be learned keeps each worked example small enough to hand-check, the same discipline Chapters 1 through 19 already apply everywhere else in this book.

That second question, learning the rule itself, is not foreign to this chapter; it is Chapter 19's question, and Section 20.3.4 connects the two directly. It argues that agent-based models and multi-agent reinforcement learning (MARL) are best read as two different responses to the same limitation Chapter 19 identifies in its own single-agent framing, that training a policy against a fixed historical simulator implicitly assumes every other participant's behavior stays fixed, and it makes the connection concrete through the learning-seller example previewed above, in which one agent in an otherwise fixed-rule market instead learns its own quoting rule through Chapter 19's Q-learning machinery. Every fixed-rule agent elsewhere in this chapter is best read as a transparent, tractable special case of that more general, and considerably harder to train and validate, world in which agents learn rather than follow a rule handed to them, not as a rejection of it.

The chapter's sections follow the same escalating structure in every application: state the mechanism and a small, hand-checkable illustration first, then show what happens when that mechanism is scaled up or repeated many times. Sections 20.2.2 and 20.2.3 build up market-level price dynamics from individual trading rules, Section 20.3.1 scales the latter to hundreds of periods to show real statistical patterns emerging, Sections 20.3.2 and 20.4.2 apply the identical logic to a network of banks rather than a market of traders. Section 20.3.4 steps back to compare this chapter's fixed-rule agents directly against Chapter 19's learned

Wulin.Suo@Queensu.ca

alternative, and Section 20.4.3 distills the common workflow underlying all of it into an explicit, reusable template before the chapter closes with the practical limitations, summarized in Section 20.4.1 and the Challenges section that follows it, that keep agent-based models a specialized complement to this book's other methods rather than a wholesale replacement for them.

20.2 Why Agent-Based Models

Replacing the representative agent with many heterogeneous ones changes what a model can produce.

20.2.1 Why Agent-Based Models? From Representative Agents to Heterogeneous, Interacting Ones

Much of classical finance, and much of this book, reasons in terms of a representative agent: Chapter 12's CAPM derives an equilibrium as though a single, rational, mean-variance investor stood in for the whole market; Chapter 7's GARCH models fit a single statistical process to an entire return series without asking which market participants' behavior produced that process. These simplifications are enormously useful, and this book relies on them throughout, but they are also, by construction, unable to answer certain questions.

A representative-agent model cannot easily explain why a market that is efficient on an average day can experience a Flash Crash on a particular afternoon, because the average behavior baked into the model has already smoothed away the heterogeneity that made the crash possible in the first place. Agent-based modeling takes the opposite starting point: instead of solving for what a single rational agent would do, it specifies what a whole population of different, typically boundedly rational, agents actually do, and asks what pattern emerges from their interaction.

This bottom-up approach is well suited to the phenomena that are hardest to explain with a representative agent: the fat tails and volatility clustering Chapter 7's GARCH models describe statistically but do not explain mechanistically; asset-price bubbles and crashes that unfold over time rather than instantaneously; and the kind of systemic, contagious risk Chapter 8's stress-testing framework treats largely at the level of a single institution's portfolio rather than as a property of an interconnected system.

Agent-based modeling did not originate in finance. Its most famous early instance is Schelling's (1971) segregation model, built with nothing more sophisticated than a checkerboard and coins of two colors: each agent has a mild preference to have at least some same-type neighbors, far short of wanting to live in a segregated neighborhood, and moves to a nearby empty square only if that mild preference is unmet. Simulating this simple, individually reasonable rule across many agents produces, essentially without exception, a starkly segregated checkerboard, a population-level outcome none of the individual agents' rules asked for or intended. The financial applications in this chapter are the same logic applied to markets rather than neighborhoods: individually reasonable rules, simulated across many interacting agents, producing an aggregate outcome, efficiency, a bubble, a segregated checkerboard, a cascade of defaults, that is a property of the interaction itself rather than of any one agent's intentions.

The idea reached finance specifically through the Santa Fe Institute's Artificial Stock Market, built in the early 1990s by Arthur, Holland, LeBaron, Palmer, and Tayler, widely regarded as the first serious agent-based model of a financial market. Rather than the two fixed strategy types of Section 20.2.3's fundamentalist-chartist model, the Santa Fe market populated its agents with evolving collections of predictive rules, more like a population of simple, competing forecasting models than a fixed behavioral type, and let poorly performing rules die out and successful ones proliferate, an early instance of the same evolutionary, performance-based selection logic Brock and Hommes later formalized more tractably. Its central finding, that this market could switch between a calm, efficient-market-like regime and a volatile, trend-chasing regime depending on how quickly agents adapted their rules, foreshadows the regime-switching mechanism behind Section 20.3.1's worked example, and the project is generally credited with establishing agent-based computational finance as a distinct research area in its own right, separate from the checkerboard-and-social-science tradition Schelling's model belongs to.

Beyond these historical origins, agent-based models occupy a specific, recurring role in the finance industry today, one this chapter's later sections examine in turn rather than merely assert. Central banks and other macroprudential regulators run network-based models with behavioral feedbacks, such as the Bank of

Wulin.Suo@Queensu.ca

England's RAMSI discussed in the case vignette below, to stress-test an entire banking system's interconnected exposures rather than one institution's portfolio at a time, now a standard part of the macroprudential toolkit. Exchanges and market-microstructure researchers use zero-intelligence and heterogeneous-agent markets, Sections 20.2.2 and 20.2.3's own subject matter, to understand how much of observed market efficiency, or its breakdown during an event like the Flash Crash, comes from the trading mechanism itself rather than from any participant's sophistication. Quantitative trading desks build synthetic, agent-populated limit-order-book markets to test how a new execution algorithm, of the kind Chapter 13 develops, is likely to move prices and interact with other participants' behavior before ever risking capital on it live, a cheaper and safer proving ground than production trading. And a growing body of academic and industry research combines agent-based markets with the multi-agent reinforcement learning of Section 20.3.4, training an execution or market-making policy inside a simulated population of other agents rather than only against static historical data.

Figure 20.1 sketches the basic ABM simulation loop that every example in this chapter follows: a population of agents, each with its own rule, observes the current market state, each agent acts (submits an order, updates a belief, decides whether to pay a debt), those actions are aggregated by a market mechanism into a new market state, and the loop repeats. Nothing here is estimated from historical data the way Chapter 6's models are; everything is specified as a rule and then simulated forward.

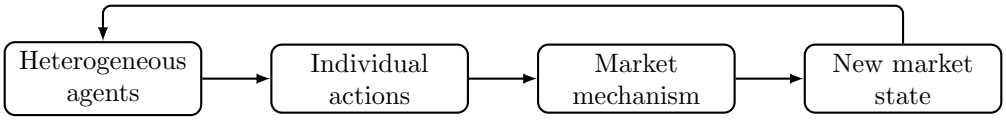


Figure 20.1: The basic agent-based simulation loop

20.2.2 Zero-Intelligence Traders and the Double Auction

The most striking early result in agent-based market modeling, due to Gode and Sunder (1993), is that a double auction, the same continuous matching of bids against asks behind the limit order book Chapter 3 introduced, can produce a highly efficient outcome even when every trader in it is, in a precise sense, not intelligent at all: a "zero-intelligence" trader submits a bid or ask drawn at random from a budget-feasible range, with no attempt to reason strategically about price or about other traders. Consider two buyers with private valuations $V_{B1} = \$52$ and $V_{B2} = \$48$, and two sellers with private costs $C_{S1} = \$45$ and $C_{S2} = \$50$, for one unit of the same asset each. The efficient allocation, the one a fully rational market would produce, is easy to find directly: buyer 1 and seller 1 should trade, since $V_{B1} > C_{S1}$ generates a surplus of $52 - 45 = \$7$, while buyer 2 and seller 2 should not trade, since $V_{B2} < C_{S2}$ would destroy value. Total efficient surplus is therefore exactly \$7.

Now suppose each buyer i submits a random bid drawn from $[0, V_{Bi}]$ and each seller j a random ask drawn from $[C_{Sj}, 100]$, each range budget-feasible for that trader alone, a buyer never bids above their own valuation and a seller never asks below their own cost, with \$100 an arbitrary price ceiling. Nobody has any knowledge of anyone else's valuation. Suppose that in one simulated round these random draws happen to be $b_1 = \$50$ (buyer 1), $b_2 = \$30$ (buyer 2), $a_1 = \$47$ (seller 1), and $a_2 = \$65$ (seller 2). The standard double-auction clearing rule sorts bids from highest to lowest and asks from lowest to highest, then matches and trades every pair for which the bid meets or exceeds the ask:

$$\begin{aligned} \text{Bids (descending):} &\quad 50,\ 30, \\ \text{Asks (ascending):} &\quad 47,\ 65. \end{aligned}$$

The top bid, \$50, meets the top ask, \$47 ($50 \geq 47$), so buyer 1 and seller 1 trade; the second bid, \$30, does not meet the second ask, \$65 ($30 < 65$), so buyer 2 and seller 2 do not trade. The clearing mechanism has, purely as a byproduct of matching random bids against random asks, selected the efficient pair, buyer 1 with seller 1, and rejected the inefficient one. Whatever transaction price rule is used, for instance the midpoint of the matched bid and ask, $(50 + 47)/2 = \$48.5$, the realized surplus, computed from the traders' true valuations

Wulin.Suo@Queensu.ca

rather than their randomly drawn bids, is $V_{B1} - C_{S1} = 52 - 45 = \$7$, the full efficient surplus, captured even though neither trader bid or asked anything close to their true valuation. The lesson, confirmed far more systematically in Gode and Sunder's original experiments across many traders and many rounds, is that a market's efficiency can come substantially from the structure of the trading mechanism itself, rather than from the sophistication of the individual traders using it, a genuinely counterintuitive result relative to the fully rational representative agent Chapter 12 builds its equilibrium models around. It is also worth being precise about what this single hand-worked round does and does not establish. It shows the mechanism *can* select the efficient pair from random bids, not that it *always* will. As the companion notebook's own supplementary experiments with this same tiny two-buyer, two-seller market confirm, a single random draw from a wide bidding range is in fact a noisy way to reach that outcome. Gode and Sunder's actual, far more robust result relies on many traders and many independent trading opportunities per period, not on any one exchange like the one worked by hand above.

Building a Simple ABM in Python

Every agent-based model in this chapter reduces to the same two ingredients Chapter 2 already teaches how to code: a data structure representing each agent's state, and a function that updates the market given every agent's action. The zero-intelligence double auction above is only a few lines:

```
import random

def zero_intelligence_round(buyers, sellers, seed=None):
    rng = random.Random(seed)
    bids = [(rng.uniform(0, v), v) for v in buyers]      # random bid, true value
    asks = [(rng.uniform(c, 100), c) for c in sellers]    # random ask, true cost
    bids.sort(key=lambda x: -x[0])
    asks.sort(key=lambda x: x[0])
    trades = []
    for (bid, val), (ask, cost) in zip(bids, asks):
        if bid >= ask:
            trades.append({'price': (bid + ask) / 2, 'surplus': val - cost})
    return trades

trades = zero_intelligence_round(buyers=[52.0, 48.0], sellers=[45.0, 50.0], seed=2)
print(trades)   # [{'price': 48.91, 'surplus': 7.0}]
```

Every other example in this chapter follows the identical pattern: Section 20.2.3's price dynamics are a single-line update rule called in a loop; Section 20.3.2's clearing vector is a short function computing each bank's payment given what it has received; and Section 20.3.1's 500-period simulation is that same fundamentalist-chartist loop with an `if` statement selecting which regime's parameters to use each period. None of the mechanics here require any tool beyond what Chapter 2 already covers; what is different about agent-based modeling is not the code, but the modeling habit of specifying individual rules and letting the population-level pattern emerge from running them, rather than solving for that pattern in closed form. That brevity is not incidental: it is the concrete substance of the claim that a fixed-rule agent is fully interpretable, every one of these short functions is a rule a reader can audit line by line, in a way a Chapter 19 policy's learned weights, once training is done, simply are not, a contrast Section 20.3.4 sets out in full once both approaches are on the table.

This also explains why the companion notebook, rather than the chapter text, is the right place to actually explore an ABM's behavior. Every worked example above reports one particular set of parameter values and, where relevant, one particular random seed. Changing any of them, the switching threshold, the population shares, the interbank network's shape, the buyer valuation distribution the learning seller faces, produces a different simulated history. The only way to build real intuition for how sensitive a given result is to a given assumption is to make that change and rerun the simulation, the kind of hands-on exploration this book's preface encourages for every chapter's companion notebook.

Wulin.Suo@Queensu.ca

20.2.3 Heterogeneous Agents, Bubbles, and Crashes: Fundamentalists versus Chartists

Zero-intelligence traders illustrate that a market mechanism can be efficient despite unsophisticated participants; they do not, on their own, explain bubbles and crashes, since random bidding has no memory and no tendency to extrapolate a trend. The next step in the agent-based tradition, associated with Brock and Hommes and with Lux and Marchesi, introduces two distinct, purposeful agent types whose relative population shares can shift over time. In plain terms, before the equations below express it precisely, a fundamentalist behaves like a value investor: the further the price sits below what the asset is really worth, the more eagerly the fundamentalist buys, and the further above, the more eagerly they sell, a demand rule that always leans toward pulling price back to fundamental value. A chartist behaves like a momentum trader instead: the faster the price has just been rising, the more eagerly the chartist buys, betting that a recent trend continues, a demand rule with no opinion at all about what the asset is actually worth. Formally, fundamentalists believe the price will revert toward a known fundamental value p_f and trade accordingly, submitting demand proportional to the mispricing,

$$d_t^f = \alpha(p_f - p_t),$$

while chartists (trend followers) extrapolate the most recent price change forward, submitting demand proportional to recent momentum,

$$d_t^c = \beta(p_t - p_{t-1}).$$

A simple market-impact rule aggregates the two populations' demand, weighted by their respective population shares n_f and $n_c = 1 - n_f$, into a single aggregate demand,

$$D_t = n_f\, d_t^f + n_c\, d_t^c,$$

which a linear price-impact equation converts into a price update,

$$p_{t+1} = p_t + \lambda D_t,$$

where λ controls how strongly aggregate demand moves the price. Table 20.2's "Aggregate demand" column below reports this quantity, D_t, at each step. Everything about this market, the fundamental value, the two behavioral rules, the price-impact equation, is held fixed across the two scenarios below; only the population mix (n_f, n_c) differs.

A Worked Example: Convergence versus a Self-Reinforcing Bubble

Let $p_f = 100$, $\alpha = 0.3$, $\beta = 1.0$, $\lambda = 1.5$, and start both scenarios from the same two initial prices, $p_{-1} = \$93$ and $p_0 = \$95$ (a modest existing uptrend). In Scenario A, fundamentalists are the large majority, $n_f = 0.8$, $n_c = 0.2$. Table 20.2 works the first three steps by hand.

t	p_t	d_t^f	d_t^c	Aggregate demand D_t	p_{t+1}
0	95.000	1.500	2.000	1.600	97.400
1	97.400	0.780	2.400	1.104	99.056
2	99.056	0.283	1.656	0.558	99.893

Table 20.2: Scenario A: fundamentalist-dominated market ($n_f = 0.8$, $n_c = 0.2$), converging toward $p_f = 100$

The price rises from \$95.00 toward the fundamental value of \$100 in shrinking steps, \$2.40, then \$1.66, then \$0.84, the damped convergence a stabilizing fundamentalist majority should produce: as price approaches p_f, the fundamentalist demand term that pulls it there mechanically weakens.

Scenario B changes only the population mix, to a market with no fundamentalists at all, $n_f = 0$, $n_c = 1$. With $n_f = 0$ the price update collapses to $p_{t+1} = p_t + \lambda\beta(p_t - p_{t-1})$, and since $\lambda\beta = 1.5 \times 1.0 = 1.5 > 1$,

Wulin.Suo@Queensu.ca

each period's price change is 1.5 times the previous period's change, an explosive recursion with no anchor to p_f at all:

$$p_1 = 95 + 1.5(95 - 93) = 95 + 3.00 = 98.00,$$
$$p_2 = 98 + 1.5(98 - 95) = 98 + 4.50 = 102.50,$$
$$p_3 = 102.5 + 1.5(102.5 - 98) = 102.5 + 6.75 = 109.25.$$

The price moves further from the fundamental value of \$100 every single period, \$3.00, then \$4.50, then \$6.75, a self-reinforcing bubble driven entirely by momentum-chasing, with the size of each successive move growing rather than shrinking. Nothing about the market mechanism changed between Scenario A and Scenario B; only the composition of the trading population did. This is precisely the kind of qualitative, regime-dependent behavior, stable in one population mix, explosive in another, that a single representative agent cannot produce, since a representative agent is, by construction, always the same agent.

Why: A General Stability Condition

Scenario A converged and Scenario B exploded, but the two scenarios also differed in n_c, which raises a natural question: exactly how much chartist weight tips the market from one regime to the other? The answer, worked out in full below, is the kind of population-level fact this chapter keeps returning to. It is written into neither individual behavioral rule: no fundamentalist's demand equation and no chartist's demand equation mentions a critical population share anywhere. Yet their combination produces a sharp, exactly calculable threshold that a representative-agent model could not even pose as a question, since such a model has no population mix to vary in the first place (Section 20.2.1). Intuitively, a chartist's demand today depends on how much the price just rose, but that very demand is itself part of what pushes the price up further tomorrow, feeding a fresh, and potentially larger, round of chartist demand the day after; whether that feedback loop fizzles out or feeds on itself depends only on how strong it is relative to the fundamentalist demand pulling the price the other way, the balance the threshold derived below makes precise. Substituting the two demand equations into the price-update rule turns the model into a linear recursion in the two most recent prices alone,

$$p_{t+1} = \lambda n_f \alpha p_f + (1 - \lambda n_f \alpha + \lambda n_c \beta)\, p_t - \lambda n_c \beta\, p_{t-1},$$

and the standard stability analysis for a recursion of this form (worked out in full, and confirmed against a parameter sweep, in the companion notebook) shows that, for every parameter combination used in this section, the market is stable if and only if

$$\lambda n_c \beta < 1.$$

Scenario A's coefficient was $\lambda n_c \beta = 1.5 \times 0.2 \times 1.0 = 0.30 < 1$, comfortably stable; Scenario B's, with no fundamentalists diluting the chartist share at all, was $\lambda n_c \beta = 1.5 \times 1.0 \times 1.0 = 1.50 > 1$, explosive. Solving $\lambda n_c \beta = 1$ for the population share gives the exact critical threshold,

$$n_c^* = \frac{1}{\lambda \beta} = \frac{1}{1.5 \times 1.0} \approx 0.667,$$

so this particular market tips from convergent to explosive once chartists exceed roughly two-thirds of the trading population, holding λ and β fixed at the values used throughout this section. The companion notebook confirms this threshold directly: at $n_c = 0.60$, just below n_c^*, the price oscillates around p_f with steadily shrinking amplitude, settling close to \$100 within about 60 periods, while at $n_c = 0.70$, just above n_c^*, the price oscillates with growing amplitude instead, 87.7 after only 15 periods and still widening. Convergence near the threshold is oscillatory rather than the smooth, monotonic approach Scenario A's more comfortably stable $n_c = 0.2$ produced, since the underlying recursion's roots are a complex-conjugate pair whenever n_c is this large; the market does not merely converge slower near the boundary, it converges differently, overshooting p_f and swinging back repeatedly before settling. This is the kind of sharp, threshold-dependent regime change, a market that behaves completely differently once one parameter crosses a critical value, that motivates the endogenous switching mechanism behind Section 20.3.1's stylized-facts simulation: real markets do not sit at a fixed n_c forever, and a population that drifts back and forth across a threshold like n_c^* is a population that alternates between the qualitatively different regimes worked out numerically above.

Wulin.Suo@Queensu.ca

20.3 Emergence and Systemic Risk

The point of these models is what they produce without being told to, including contagion nobody designed.

20.3.1 Emergent Stylized Facts: Fat Tails and Volatility Clustering from Simple Rules

Chapter 7's GARCH(1,1) model captures volatility clustering by directly imposing an autoregressive structure on the variance of returns: large moves are more likely to be followed by large moves because the model's own parameters say so. Heterogeneous agent models of the kind in Section 20.2.3 reproduce the same stylized fact, along with the excess kurtosis (fat tails) Chapter 7 discusses when introducing GARCH, without ever specifying a variance process at all. Suppose the population mix (n_f, n_c) is allowed to evolve over time, shifting toward chartists whenever chartist strategies have recently been more profitable, the kind of performance-based strategy switching Brock and Hommes formalize. The market then alternates endogenously between calm, fundamentalist-dominated periods resembling Scenario A above and volatile, chartist-dominated episodes resembling Scenario B. Clustered volatility and heavy-tailed returns emerge as a consequence of the switching dynamic itself, rather than as an assumption built into the model beforehand. This distinction matters practically: a GARCH model is a superb tool for forecasting volatility once its parameters are fit to data, exactly Chapter 7's use case, but it does not explain why volatility clusters in the first place, while an agent-based model of this kind offers one candidate mechanistic explanation, at the cost of being far harder to fit to any single historical dataset with confidence, a tension Section 20.4.1 returns to.

A Worked Example: Volatility Clustering from Endogenous Regime Switching

The companion notebook makes this concrete by extending the fundamentalist-chartist model to 500 simulated periods, with two bounded (non-explosive) regimes rather than Scenario B's deliberately unbounded pure-chartist case: a calm regime ($n_f = 0.85$, $n_c = 0.15$, small shocks) and a volatile regime ($n_f = 0.25$, $n_c = 0.75$, larger shocks). The simulation switches into the volatile regime whenever the most recent price move exceeds a fixed threshold, and reverts to the calm regime otherwise. The threshold is a simple proxy for the idea that a large recent move is the situation in which a chartist strategy would just have outperformed a fundamentalist one. Both regimes are individually stable: the price never diverges the way Scenario B's pure-chartist market did, ranging from \$90.61 to \$108.55 over the full 500 periods. But the two regimes' realized volatility differs sharply. The simulation spends 216 periods in the calm regime, with a return variance of 0.157, and 284 periods in the volatile regime, with a return variance of 1.121, a variance ratio of 7.14× between two regimes generated by the same underlying shock-generating process, differing only in which population of traders happens to be active. The full 500-period return series has a Pearson kurtosis of 5.23, well above the Gaussian benchmark of 3.0 (an excess kurtosis of 2.23), confirming fat tails, and the autocorrelation of squared returns, the standard volatility-clustering diagnostic Chapter 7 applies to real return series, is 0.761 at a lag of one period and remains a substantial 0.297 at a lag of five periods, both far from the value of zero i.i.d. returns would produce. None of these three statistics, the variance ratio, the excess kurtosis, or the squared-return autocorrelation, was a modeling target; all three emerged from a simulation whose only design choice was which population of simple, fixed behavioral rules was active in each period.

20.3.2 Agent-Based Models of Systemic Risk and Financial Contagion

The examples so far model a single market's price. A separate, and for regulators arguably more important, application of agent-based thinking models an entire financial system of interconnected institutions, extending Chapter 8's stress-testing framework from a single portfolio to a network. The underlying idea is intuitive before it is formal: if a bank cannot fully pay what it owes, every other bank counting on that payment is now short of cash too, and may in turn be unable to pay everyone counting on *it*, so a single institution's shortfall can pass through a network of obligations to institutions that never dealt with the original problem at all. Eisenberg and Noe (2001) formalize this chain reaction as a clearing vector: given a network of interbank obligations and each bank's external (non-interbank) assets, compute how much each

Wulin.Suo@Queensu.ca

bank actually pays, accounting for the fact that a bank which is not paid in full by others may not, in turn, be able to pay its own obligations in full.

A Worked Example: A Shock That Cascades Through the Network

Consider three banks connected in a simple chain: Bank A owes Bank B \$50, and Bank B owes Bank C \$40; Bank C has no obligations to anyone. Each bank also holds external assets: \$60 for A, \$10 for B, and \$5 for C. Figure 20.2 shows this network.

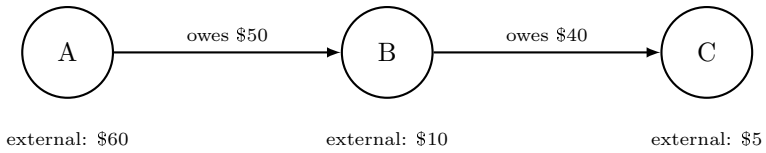


Figure 20.2: A three-bank interbank obligation network

In the baseline, no-shock case, every bank can pay its obligations in full: A's total resources are its \$60 of external assets (no one owes A anything), comfortably covering its \$50 obligation to B and leaving A with net worth $60 - 50 = \$10$. B's resources are its \$10 of external assets plus the \$50 it receives from A, \$60 total, covering its \$40 obligation to C and leaving B with net worth $60 - 40 = \$20$. C's resources are its \$5 of external assets plus the \$40 it receives from B, for a net worth of \$45. Total system-wide net worth is $10 + 20 + 45 = \$75$.

Now suppose an external shock, a bad loan or a trading loss unrelated to the interbank network, destroys \$45 of Bank A's external assets, leaving A with only \$15. Recomputing the clearing vector: A's resources are now just \$15 against its \$50 obligation, so A pays only $\min(15, 50) = \$15$ to B, a default with a 30% recovery rate, leaving A with net worth \$0. B's resources are its own \$10 of external assets plus only \$15 from A (not the \$50 it was owed), \$25 total, against its own \$40 obligation to C; B can pay only $\min(25, 40) = \$25$, so *B also defaults*, with a 62.5% recovery rate, even though B experienced no external shock of its own at all, its default is entirely a consequence of A's default. C's resources are its own \$5 plus only \$25 from B (not the \$40 it was owed), for a final net worth of \$30, down from \$45, a real loss even though C has no obligations to anyone and never itself defaults.

Table 20.3 summarizes the cascade. Total system-wide net worth falls from \$75 to $0 + 0 + 30 = \$30$, a system-wide loss of exactly \$45, precisely equal to the size of the original shock: nothing about the network creates or destroys value, it only redistributes a single institution's loss across every institution downstream of it, in a pattern that would be invisible to a stress test of Bank A's portfolio in isolation.

Bank	External (shocked)	Owed	Received	Paid	Net worth (baseline)	Net worth (shocked)
A	15	50	0	15	10	0
B	10	40	15	25	20	0
C	5	0	25	0	45	30

Table 20.3: The default cascade following a \$45 shock to Bank A's external assets (all figures in \$)

20.3.3 Case Vignette: The Bank of England's RAMSI Model

The contagion mechanism worked by hand in Section 20.3.2 is not merely a textbook simplification; it is a simplified version of machinery central banks actually use. The Bank of England's RAMSI (the Risk Assessment Model for Systemic Institutions) is a large-scale, top-down stress-testing model of the UK banking sector: top-down in the sense that the same model and the same assumptions are applied to every major bank, so the results can be compared across banks and added up to a view of the system as a whole. Work on it began before the 2008 financial crisis, whose Gaussian-copula CDO mispricing Chapter 9 examines in detail; a prototype was presented at central-bank conferences in 2007, and the model was used in the IMF's 2011 assessment of the UK financial sector. Each bank enters with its own balance sheet and income

Wulin.Suo@Queensu.ca

statement, projected forward under macroeconomic scenarios and linked to the other banks through interbank exposures, the same kind of network-level cascade Section 20.3.2's three-bank example illustrates, at a far larger and more realistic scale.

RAMSI is not a full agent-based model in this chapter's sense. Its banks do not optimize, and its own developers describe them as acting largely passively: each follows simple behavioral rules, such as a target capital ratio that governs how retained earnings are reinvested. What carries it beyond Section 20.3.2's clearing-vector calculation is a set of feedback effects layered on top of those rules. A bank whose fundamentals are projected to weaken pays more for funding and, past certain thresholds, is shut out of funding markets altogether, and banks perceived to resemble it face a higher chance of being shut out too. A bank in trouble may also sell assets, pushing down the prices of those assets and causing mark-to-market losses at every other bank holding them, a channel with no counterpart in the pure balance-sheet cascade of Section 20.3.2, since nothing there let one bank's response feed back into the market prices other banks' own balance sheets depend on. This distinction matters practically: a pure Eisenberg-Noe network captures what regulators call first-round, direct contagion (a defaulted counterparty's unpaid obligations), while fire-sale and funding feedbacks capture second-round, indirect contagion (one bank's distress damaging other, still-solvent banks through prices and funding markets), the kind of individually reasonable but collectively destabilizing dynamic Section 20.2.1's Schelling and Flash Crash discussions both illustrate. In the Bank's own prototype, these feedbacks turn jointly normal shocks into a bimodal distribution of system-wide outcomes, with a small second peak of contagious defaults in the tail, and no version of the three-bank chain or hub-and-spoke examples above, both purely first-round mechanisms, can reproduce that on its own.

Models of this kind are used to stress-test the UK banking system as a network rather than one institution at a time, directly extending the single-portfolio stress-testing framework of Chapter 8 to account for the kind of contagious, second-round loss the worked example above shows a purely institution-by-institution stress test would miss entirely: in that example, a stress test of Bank B alone, which received no direct shock, would have reported B as perfectly healthy, right up until it wasn't. Other central banks and the IMF have developed comparable network-based systemic-risk tools, some with genuinely agent-based behavioral components, and system-wide stress testing is now a standard, if still specialized, part of the macroprudential regulator's toolkit alongside the more familiar single-institution VaR and stress-testing methods Chapter 8 covers.

20.3.4 Agent-Based Models versus Multi-Agent Reinforcement Learning: Fixed Rules versus Learned Behavior

Chapter 19's discussion of multi-agent dynamics observes that a reinforcement-learning agent trained against a static, historical simulator implicitly assumes every other participant's behavior stays fixed, an assumption the Flash Crash's hot-potato effect shows can fail badly in practice. Agent-based models and multi-agent reinforcement learning (MARL) are best understood as two different responses to that same observation, rather than as competitors. An ABM specifies each agent's behavioral rule directly, fundamentalist, chartist, zero-intelligence, and asks what emerges from many such fixed rules interacting; a MARL system instead lets each agent's behavior itself be learned, typically through the same Q-learning or policy-gradient machinery Chapter 19 develops, adapting to the other agents' evolving behavior rather than following a rule fixed in advance. The ABM approach trades away realism about how real traders learn and adapt in exchange for transparency and tractability, every agent's rule in this chapter's examples is a single, interpretable equation, while MARL trades that transparency away for the possibility of agents that behave more like real, adaptive market participants, at the cost of the training instability and weak convergence guarantees Chapter 19 already flags as MARL's central open problem. A growing body of research combines the two directly, building ABMs in which some or all agents follow reinforcement-learning policies rather than fixed behavioral rules, using the ABM's simulated market as the kind of training environment Chapter 19's execution and market-making sections already rely on, an active research direction rather than a settled methodology. Table 20.4 summarizes the practical tradeoffs a modeler actually faces when choosing between the two.

Wulin.Suo@Queensu.ca

	Agent-based model	Multi-agent reinforcement learning
Agent behavior	Fixed, hand-specified rule	Learned via Chapter 19's Q-learning or policy-gradient methods
Interpretability	High: every rule is a single, readable equation	Low: behavior is implicit in a trained policy's weights
Adaptivity	None, unless the modeler adds explicit switching (Section 20.2.3)	Agents adapt continuously to each other's evolving behavior
Convergence guarantees	Not applicable; the model is simulated, not trained	Weak once multiple agents learn simultaneously (Chapter 19)
Computational cost	Low to moderate	High, and grows sharply with agent count
Typical role in this book	Exploring what mechanisms are capable of producing an observed pattern (Section 20.4.1)	Training a single production policy against a realistic simulated environment

Table 20.4: Agent-based models (fixed rules) versus multi-agent reinforcement learning (learned behavior)

A Worked Example: A Learning Seller in the Double Auction

To make the ABM-versus-MARL distinction concrete rather than only abstract, reconsider Section 20.2.2's double auction, but replace one zero-intelligence seller with a seller who learns, in the same trial-and-error spirit as Chapter 19's bandit examples. Each round, a new buyer arrives with a valuation drawn from $\{45, 50, 55, 60, 65\}$ with equal probability, facing a seller with cost \$40 choosing between two fixed ask levels: Arm H (ask \$58) or Arm Lo (ask \$47). A trade occurs whenever the ask is at or below the buyer's valuation, earning the seller a profit of ask minus cost; a rejected ask earns nothing. The true expected profit of each arm, unknown to the seller but computable directly from the valuation distribution, is

$$E[\text{profit}_H] = \Pr(v \geq 58) \times (58 - 40) = \tfrac{2}{5} \times 18 = 7.2,$$
$$E[\text{profit}_{Lo}] = \Pr(v \geq 47) \times (47 - 40) = \tfrac{4}{5} \times 7 = 5.6,$$

so Arm H, despite trading only half as often, is actually the better arm: its larger profit per trade more than compensates for its lower fill rate, a genuinely non-obvious tradeoff that neither arm's raw trade frequency reveals on its own.

The learning seller explores both arms twice (rounds 1 through 4, alternating H, Lo, H, Lo) and then plays greedily, choosing whichever arm has the higher running-average realized profit so far. Table 20.5 traces one representative 12-round history. After the four exploration rounds, Arm H's running average stands at \$18.00 against Arm Lo's \$7.00, so the seller plays Arm H for the remainder of the sequence, exactly matching the true optimal choice, and its running average, noisy at first, drifts down toward the true value of \$7.2 as more rounds accumulate.

This is the ABM-versus-MARL distinction from Table 20.4 made concrete within a single example: every other seller in Section 20.2.2 follows a fixed rule (draw a random ask, full stop), while this one adapts its behavior based on realized experience, the shift from a specified rule to a learned policy that separates the two columns of that table. It is also worth being honest about what a single 12-round history does and does not show. This particular valuation sequence happened to make both exploration draws for Arm H land on high-valuation buyers, letting the seller lock onto the correct arm immediately. A less fortunate exploration phase could just as easily have left the seller with a misleadingly low early estimate for the better arm, favoring the worse one for a while, the explore-exploit tension that makes reinforcement learning harder in practice than this one clean run suggests.

Wulin.Suo@Queensu.ca

Round	Buyer value	Arm played	Profit	Avg. profit, H	Avg. profit, Lo
1	60	H	18	18.00	–
2	65	Lo	7	18.00	7.00
3	60	H	18	18.00	7.00
4	60	Lo	7	18.00	7.00
5	65	H	18	18.00	7.00
6	65	H	18	18.00	7.00
7	50	H	0	14.40	7.00
8	50	H	0	12.00	7.00
9	65	H	18	12.86	7.00
10	60	H	18	13.50	7.00
11	65	H	18	14.00	7.00
12	50	H	0	12.60	7.00

Table 20.5: One 12-round history of the learning seller in the double auction

A Worked Example: Discrete-Choice Switching Replaces the Fixed Threshold

The learning seller above replaces one agent's fixed rule with something learned; it is worth asking the same question of Section 20.3.1's regime-switching market, where the population mix (n_f, n_c) was allowed to evolve, but only according to a threshold rule handed down by the modeler, switch to the volatile regime whenever the last price move exceeds a fixed cutoff, not according to anything any fundamentalist or chartist actually experienced. Brock and Hommes's own mechanism, named in passing in Section 20.2.1 and Section 20.3.1 and derived in full generality in this book's companion technical note on the Brock-Hommes model, replaces that external threshold with the comparison a real trader would make: at every period, each strategy's population share updates according to a discrete-choice (logit) rule driven by that strategy's own recently realized profit, so the switching Section 20.3.1 imposed from outside instead emerges from many individual agents each comparing two numbers and choosing, probabilistically, whichever strategy has recently paid off better.

Reusing Section 20.2.3's market exactly, $p_f = 100$, $\alpha = 0.3$, $\beta = 1.0$, $\lambda = 1.5$, $p_{-1} = 93$, $p_0 = 95$, define each strategy's realized profit at t as the price change just observed times the demand that strategy would have submitted one period earlier,

$$\pi_{f,t} = (p_t - p_{t-1})\, d^f_{t-1}, \qquad \pi_{c,t} = (p_t - p_{t-1})\, d^c_{t-1},$$

and update the fundamentalist population share by the binary logit rule

$$n_{f,t} = \frac{e^{\theta \pi_{f,t}}}{e^{\theta \pi_{f,t}} + e^{\theta \pi_{c,t}}}, \qquad n_{c,t} = 1 - n_{f,t},$$

where $\theta \geq 0$ is the intensity of choice (written θ here rather than the technical note's own β, to avoid clashing with this chapter's already-established chartist coefficient β): $\theta = 0$ freezes the population at whatever split it started with regardless of performance, while a large θ sends nearly the entire population to whichever strategy did even slightly better last period.

Starting from an uninformed $n_{f,0} = n_{c,0} = 0.5$, no fitness history yet exists to compare, and $\theta = 0.5$, Table 20.6 works the first three periods by hand.

The first period's fitness gap is already informative: with $\Delta p_1 = 2.625$, the chartist demand submitted the period before, $d^c_0 = 2.0$, earned a realized profit of $\pi_{c,1} = 2.625 \times 2.0 = 5.250$, against the fundamentalist's $\pi_{f,1} = 2.625 \times 1.5 = 3.938$; chartists were not merely more numerous, they were genuinely more profitable, since their demand happened to be better aligned with the up-move that materialized. The logit rule converts this into $n_{c,1} = 0.658$, already sitting at the very edge of the exact critical threshold $n^*_c \approx 0.667$ Section 20.2.3 derived analytically for this same λ and β. One further period of chartist outperformance, $\pi_{c,2} = 7.764$ against $\pi_{f,2} = 2.107$, pushes the population decisively past that threshold to $n_{c,2} = 0.944$, and the price increments confirm the market has genuinely crossed into the unstable regime: $\Delta p_1 = 2.625$,

Wulin.Suo@Queensu.ca

t	p_t	$n_{f,t}$	$n_{c,t}$	d_t^f	d_t^c	D_t	p_{t+1}
0	95.000	0.500	0.500	1.500	2.000	1.750	97.625
1	97.625	0.342	0.658	0.713	2.625	1.972	100.583
2	100.583	0.056	0.944	−0.175	2.958	2.783	104.757

Table 20.6: Discrete-choice switching ($\theta = 0.5$) replacing Section 20.3.1's fixed threshold. Each period's population shares are computed from the fitness realized over the immediately preceding period, then used to set that period's aggregate demand.

$\Delta p_2 = 2.958$, $\Delta p_3 = 4.174$, growing rather than shrinking, exactly Scenario B's explosive signature, but reached here with no threshold rule written into the model at all, only individual agents comparing two realized numbers each period and updating a choice probability accordingly.

This result is worth placing carefully within Table 20.4's ABM-versus-MARL contrast rather than folded unchanged into either column. It is not Chapter 19's Q-learning: no agent here maintains a state-action value table or bootstraps a return estimate the way the learning seller above does. It is closer to a population-level analogue of the same idea, an evolutionary or replicator dynamic in which strategies that recently paid off are simply used by more of the population next period. That distinction matters for interpretability: a single logit population share is still a single number a reader can audit by hand, exactly as this worked example just did, in a way a trained neural-network policy's weights are not, so discrete-choice switching sits closer to the transparent, fixed-rule end of Table 20.4's spectrum than the learning seller does, even though both replace a rule the modeler would otherwise have to impose with something the agents themselves determine. The companion technical note develops this same mechanism with a fully specified CARA-utility demand system rather than this chapter's simpler linear demand rules, and derives, in closed form, the equilibrium population share and its exact stability condition as a function of the intensity of choice, a genuine analogue of this section's n_c^*, but expressed as a threshold on θ rather than directly on the population share, since in that richer model the population share is itself an endogenous function of θ rather than a free parameter the modeler sets. Brock and Hommes's own result goes one step further still: beyond that critical intensity of choice, the market does not merely become unstable in the sense this section's numbers illustrate, its dynamics undergo a period-doubling bifurcation into deterministic chaos, price behavior that looks statistically indistinguishable from noise despite being generated by a fully deterministic, rational system, the technical note's own "rational route to randomness." A reader who wants that fully rigorous derivation, rather than this section's numerical illustration of the same underlying idea, should turn to the technical note directly.

20.4 Building and Validating a Model

An agent-based model that matches nothing is a simulation, not a model; this part covers the difference.

20.4.1 Calibrating and Validating Agent-Based Models

Chapter 6 validates a model by holding out data the model never saw during training and checking its predictions against it, a well-defined procedure precisely because the model produces a specific, checkable prediction for each held-out observation. Agent-based models are far harder to validate in this sense, because an ABM does not produce a single prediction to check against a single observation; it produces an entire simulated history, and the object of interest is usually a qualitative or statistical property of that history, does it produce fat tails, does it produce a plausible contagion cascade, not a point forecast. The standard approach, the method of simulated moments, instead chooses the model's free parameters (here, quantities like α, β, λ, and the population-switching rule governing n_f and n_c) to make the simulated data's statistical moments, its volatility, its kurtosis, its autocorrelation structure, match the same moments computed from real market data as closely as possible, rather than maximizing a likelihood the way Chapter 6's models typically do.

Wulin.Suo@Queensu.ca

This procedure carries a genuine risk that deserves the same skepticism Chapter 6 applies to overfitting: an ABM with enough free behavioral parameters can often be tuned to match almost any target set of stylized facts, which demonstrates that the model is flexible, not that its particular mechanism, fundamentalists and chartists switching by relative profitability, say, is the true explanation for why real markets exhibit those facts. A model that reproduces fat tails is not thereby confirmed; a different ABM, or even a different set of parameters within the same ABM, might reproduce the same fat tails through an entirely different mechanism. This is a substantive limitation, not a minor technicality, and is the central reason agent-based models are generally used in this book's spirit, as a tool for exploring what mechanisms are capable of producing an observed pattern, alongside real institutional case studies like the Flash Crash and the RAMSI vignette above, rather than as a forecasting tool to be validated the way Chapter 6's classifiers and Chapter 7's time-series models are.

Concretely: Matching a Target Moment

Section 20.3.1's regime-switching simulation happened to produce a Pearson kurtosis of 5.23 from a switching threshold of 0.25; nothing forced that particular value. Method-of-simulated-moments calibration treats the switching threshold (and any of the model's other free parameters) as a dial to search over, re-running the simulation at several threshold values and recording the resulting kurtosis, exactly as Table 20.7 does.

Switching threshold	Pearson kurtosis
0.15	3.82
0.20	3.91
0.25	5.23
0.30	6.62
0.35	11.64

Table 20.7: Simulated kurtosis as a function of the switching threshold (500 periods, otherwise identical to Section 20.3.1)

The direction is worth pausing on, because it is not the one intuition suggests first: kurtosis *rises* as the threshold rises, that is, as the volatile regime becomes *harder* to trigger, not easier. A low threshold puts the market into the volatile regime often, which raises overall variance but makes the two regimes blend into one another rather than standing sharply apart; a high threshold makes the volatile regime rare, so on the infrequent occasions it does trigger, it produces a small number of sharp outliers against an otherwise calm baseline, the mixture-of-a-common-calm-regime-and-a-rare-extreme-regime structure that drives kurtosis up. Suppose an analyst wanted the simulation to match a specific real asset's historically observed kurtosis of 6.0: Table 20.7 shows the threshold value that comes closest is 0.30, landing at 6.62, a small overshoot that a finer search between 0.25 and 0.30 would narrow further. Crucially, finding a threshold that approximately matches 6.0 would establish only that this particular mechanism, among what is likely a wide range of other threshold values, population-share assumptions, or entirely different ABM structures, is *capable* of producing that kurtosis, not that real traders actually switch strategies according to anything resembling this rule, and, as this reversed direction illustrates concretely, not that the calibrated parameter's economic interpretation, "how readily the market panics," behaves the way casual intuition about it would suggest. That gap, between matching a moment and confirming a mechanism, is precisely Section 20.4.1's central caution, restated here in terms of real, computed numbers rather than in the abstract.

20.4.2 Advanced Topics: Network Topology and the Limits of a Simple Chain

Section 20.3.2's three-bank example deliberately used the simplest possible network, a single chain, in which every bank has exactly one creditor and at most one debtor. Real interbank networks look nothing like a chain: they are typically core-periphery structures, a small number of large, heavily interconnected banks at the core, surrounded by many smaller banks each connected mainly to the core rather than to each other, and the shape of that network turns out to matter for how a shock propagates, not merely how large the shock is. Acemoglu, Ozdaglar, and Tahbaz-Salehi's widely cited "robust-yet-fragile" result formalizes an

Wulin.Suo@Queensu.ca

initially counterintuitive finding: for small shocks, a more densely connected network is more stable, since each bank's exposure to any single counterparty's distress is diluted across many other relationships, the diversification logic behind Chapter 12's portfolio theory. But past a certain shock size, that same dense interconnection reverses roles entirely, turning into a contagion channel that transmits a large shock further and faster than a sparser network would have, because a large loss concentrated at one interconnected core bank now has many more downstream counterparties to infect than it would in a sparse network.

A Worked Example: Chain versus Hub-and-Spoke

Consider replacing Section 20.3.2's chain with a hub-and-spoke network hit by the same \$45 shock, so the two topologies can be compared directly. Figure 20.3 sets them side by side. A hub bank H owes \$30 to each of three peripheral banks P_1, P_2, P_3 (\$90 owed in total), H holds \$100 of external assets, and each P_i holds \$5 of external assets and has no obligations of its own. In the baseline, H pays all three peripherals in full, leaving H with net worth $100 - 90 = \$10$ and each P_i with net worth $5 + 30 = \$35$, for a total system net worth of $10 + 3(35) = \$115$.

Now apply the same \$45 shock used against Bank A in Section 20.3.2, this time to H's external assets, reducing H's resources to \$55 against its \$90 total obligation. Eisenberg-Noe clearing requires H to pay each equally senior creditor the same pro-rata share of what it has: H can cover only $55/90 = 61.1\%$ of what it owes, so each P_i receives \$18.33 instead of the full \$30. H's net worth falls to $55 - 55 = \$0$, a default, exactly as Bank A's did in the chain. But now every peripheral bank absorbs a slice of the loss simultaneously rather than sequentially: each P_i ends with net worth $5 + 18.33 = \$23.33$, down from \$35, a 33.3% loss for every single one of the three peripherals, incurred in the same instant. Table 20.8 summarizes both networks side by side. Total system-wide loss is exactly \$45 in both cases, matching the shock size regardless of topology, a general property of this clearing mechanism with no bankruptcy costs. Topology does not change how much value a shock destroys in total. What it changes entirely is how that loss is distributed. The chain concentrates outright default onto two institutions in sequence (A, then B), with C absorbing a comparatively modest, once-removed spillover. The hub-and-spoke network instead concentrates outright default onto a single institution, the hub itself, while spreading a real but survivable loss simultaneously across every peripheral bank. None of the peripherals technically defaults, since none had any obligation to fail to meet, but all three are meaningfully weaker. Neither topology is simply "worse." Which one a regulator should worry about more depends on whether the greater concern is a small number of outright failures cascading through a chain of counterparties, or a single well-connected institution's distress passing a real loss simultaneously to everyone depending on it. That is the kind of topology-dependent judgment RAMSI-style network models are built to inform.

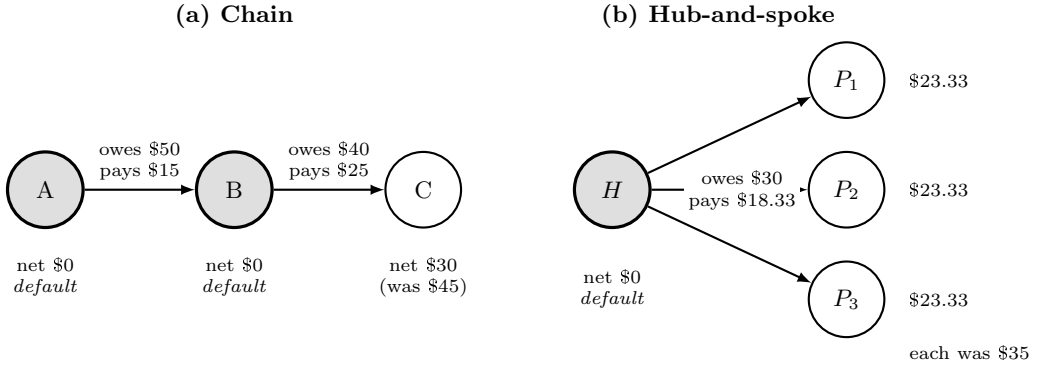


Figure 20.3: The same \$45 shock under two topologies. Shaded circles default outright; each arrow gives what is owed and what is actually paid after Eisenberg-Noe clearing. The total loss is \$45 either way, but the chain destroys two institutions one after another while the hub-and-spoke destroys one and weakens three at once.

Wulin.Suo@Queensu.ca

Network	Institutions defaulting outright	Total system loss	Loss distribution
Chain (Section 20.3.2)	2 of 3 (A, then B)	\$45	Concentrated, sequential
Hub-and-spoke (above)	1 of 4 (H only)	\$45	Diffuse, simultaneous

Table 20.8: The same \$45 shock, chain versus hub-and-spoke

A full treatment of how network topology governs the robust-yet-fragile transition would require simulating the clearing-vector calculation across many more network structures and shock sizes than the two compared here, the kind of large-scale, system-wide exercise real central-bank network models like RAMSI are built to run.

20.4.3 A Practical Workflow for Building an Agent-Based Model

Every example in this chapter, despite covering three quite different applications, market efficiency, price dynamics, and financial contagion, followed the same underlying workflow, worth stating explicitly as a template for building a new ABM from scratch, and, more fundamentally, as a concrete answer to what it actually means, step by step, to ask Table 20.1's question, "what pattern emerges from many interacting rules," rather than fitting a function to data or training a policy against an environment.

1. **Specify the agents and their behavioral rules.** Decide what types of agents populate the model (zero-intelligence traders, fundamentalists and chartists, banks in a network) and write down each type's rule as an explicit function of the current market state, exactly as Sections 20.2.2 through 20.3.2 did with a single equation or a short block of code per agent type.

2. **Specify the market mechanism.** Decide how individual agent actions aggregate into a new market state: a double-auction clearing rule, a price-impact equation, an Eisenberg-Noe clearing vector. This is the second box in Figure 20.1's simulation loop, and is usually the piece of the model most directly grounded in institutional fact rather than behavioral assumption.

3. **Choose free parameters and initial conditions.** Every worked example above had a handful of free parameters, α, β, λ, the population shares (n_f, n_c), a switching threshold, that are not derived from anything but simply set by the modeler. Report these explicitly and be prepared to defend each one, since Section 20.4.1's calibration critique applies to every one of them.

4. **Simulate forward and collect statistics.** Run the loop for enough periods that the statistic of interest, an efficiency percentage, a variance ratio, a kurtosis, a default count, stabilizes, and report it alongside the raw simulated series wherever possible, exactly as this chapter's companion notebook does for all four worked examples.

5. **Calibrate against real data if a real comparison is available.** Where a target real-world moment exists, Section 20.4.1's method of simulated moments searches over the free parameters from step 3 until simulated and real moments match as closely as possible.

6. **Interpret the result as a candidate mechanism, not a confirmed explanation.** A model that reproduces a target pattern has shown that its mechanism is *capable* of producing that pattern; Section 20.4.1's central caution, restated once more here because it is the single most important methodological point in this chapter, is that this is weaker evidence than a validated prediction from Chapter 6's train-test framework, and should be presented and used accordingly.

20.5 Challenges in Agent-Based Modeling for Finance

Agent-based models face several challenges that are worth naming explicitly, since they explain why ABMs remain a specialized tool rather than a default modeling choice throughout this book. Computational cost is real: simulating a large, heterogeneous population over many time steps, especially when agents themselves

Wulin.Suo@Queensu.ca

are learned via the reinforcement-learning methods of Section 20.3.4, can be far more expensive than fitting a single closed-form or statistical model to historical data. Every worked example in this chapter used only two to four agents specifically because a hand-checkable illustration has to; a research-grade financial ABM more typically simulates thousands to millions of agents, and, since every agent must be updated every period, cost grows at best linearly and often worse than linearly in both agent count and simulated horizon, a scaling concern this book's other methods, a single fitted regression or a single trained network regardless of dataset size, mostly do not share. General-purpose ABM frameworks such as Mesa (Python) and NetLogo exist specifically to manage this complexity, but building and validating a large-scale financial ABM remains a substantially larger engineering undertaking than fitting one of Chapter 6's models to a comparable dataset. Calibration and validation, discussed in Section 20.4.1, remain genuinely unresolved problems: the method of simulated moments can match target statistics without establishing that the underlying mechanism is correct, and there is no equivalent of Chapter 6's clean train-test split for an entire emergent history. Reproducibility across studies is weaker than in most of this book's other methods, since two research groups' ABMs of, nominally, the same phenomenon frequently differ in dozens of implementation details, agent count, order of updates, tie-breaking rules, that are rarely fully specified and can materially change the results. Finally, regulatory and industry acceptance lags behind the academic literature. RAMSI-style tools have earned a place in the macroprudential toolkit specifically because central banks can defend their assumptions in a way appropriate to their narrow, systemic-risk purpose. But agent-based models remain far from a routine part of the model validation and governance framework Chapter 15's three-lines-of-defense discipline and Chapter 18's explainability toolkit were built around. A rule-based simulation of thousands of interacting agents is simply a harder object to explain to a validator, or to a regulator, than a single fitted regression or classifier.

None of these challenges is a reason to avoid agent-based models; they are reasons to be precise about what a given ABM has and has not shown, the discipline Section 20.4.3's final step asks for. A model that reproduces the Flash Crash's hot-potato dynamics, or the fat tails of Section 20.3.1's regime-switching simulation, or RAMSI's contagion cascades, is a genuine, useful demonstration that a specific, named mechanism is *sufficient* to produce an observed pattern, valuable evidence in its own right, and often the only tractable way to reason about emergent phenomena at all. It is simply not, on its own, evidence that the mechanism is *necessary*, the one true explanation among the many candidate mechanisms that might produce the same pattern, and every application in this chapter should be read with that distinction in mind.

20.6 Key Takeaways

- Agent-based models simulate a population of heterogeneous, boundedly rational agents interacting repeatedly, and treat the aggregate pattern that emerges, efficiency, a bubble, a cascade, as the object of study, in contrast to this book's representative-agent equilibrium models and data-fitted statistical models.
- Market efficiency can emerge from a trading mechanism itself rather than from participant sophistication: zero-intelligence traders bidding at random within budget constraints, cleared through an ordinary double auction, recovered the fully efficient trade in this chapter's worked example.
- The same market mechanism can produce either smooth convergence to fundamental value or a self-reinforcing, explosive bubble, depending only on the population mix of fundamentalist and chartist traders, holding every other parameter fixed.
- Heterogeneous-agent models can reproduce fat tails and volatility clustering as an emergent consequence of strategy switching, rather than by directly imposing an autoregressive variance structure the way Chapter 7's GARCH models do: this chapter's 500-period simulation produced a 7.14× variance ratio between regimes, a Pearson kurtosis of 5.23 against a Gaussian benchmark of 3.0, and a squared-return autocorrelation of 0.761 at lag one, from nothing more than two fixed behavioral rules switching in and out of dominance.
- A clearing-vector approach to interbank obligations, the same network-contagion logic at the core of the Bank of England's top-down RAMSI stress-testing model, can show one institution's shock cascading

Wulin.Suo@Queensu.ca

into a second institution's default even when that second institution received no direct shock at all, a genuinely systemic risk a single-institution stress test cannot see.

- Agent-based models are far harder to calibrate and validate than this book's other methods, since matching a target set of statistical moments demonstrates flexibility, not that the model's particular mechanism is the true explanation for the observed pattern.

- Network topology shapes contagion independently of shock size: the "robust-yet-fragile" result shows dense interconnection dampens small shocks through diversification but can transmit large shocks further and faster than a sparser network would; the same \$45 shock produced two outright defaults in a chain network but only one, spread more diffusely, in a hub-and-spoke network of the same total size.

- The fundamentalist-chartist market's stability is governed by a single exact threshold, $n_c^* = 1/(\lambda\beta)$: below it the price converges, above it a bubble grows without bound, and near it convergence becomes oscillatory rather than smooth, a sharp regime change from one parameter crossing a critical value.

- Agent-based models answer a third kind of question this book's other two paradigms do not: not "what function best fits this data" (Chapter 6) or "what action sequence maximizes reward in this environment" (Chapter 19), but "what pattern emerges when many simple, interacting rules are simulated together."

- Even a calibrated parameter's economic interpretation can run against intuition: raising the fundamentalist-chartist model's switching threshold, making the volatile regime *harder* to trigger rather than easier, raised simulated kurtosis from 3.82 to 11.64 across a real parameter sweep, the opposite of the direction naive intuition suggests, underscoring why a calibrated ABM's internal mechanism should never be trusted just because its output moments look right.

- A fixed-rule ABM and a learning agent can occupy the exact same market: replacing one zero-intelligence seller with a simple trial-and-error learner that tracks realized profit converged on the true better strategy in this chapter's worked example, but a single unlucky exploration phase can just as easily lock a purely greedy learner onto the wrong one permanently, with no further exploration to self-correct it.

- A market's switching threshold can be crossed by assumption or by learning: Section 20.3.1's regime switch was a rule the modeler imposed from outside, while Section 20.3.4's discrete-choice mechanism let the population cross the identical $n_c^* \approx 0.667$ threshold on its own, purely because agents kept reallocating toward whichever strategy had just earned more, reaching $n_{c,2} = 0.944$ and an accelerating price after only two periods of chartist outperformance.

20.7 Suggested Exercises

1. Using the zero-intelligence double-auction setup of Section 20.2.2, suppose buyer 2's random bid is $b_2 = \$49$ instead of \$30, with every other draw unchanged. Recompute the clearing outcome and explain, in terms of total realized surplus, whether the market mechanism still selects the efficient allocation.

2. Section 20.2.3 derives the stability threshold $n_c^* = 1/(\lambda\beta)$ for $\lambda = 1.5$, $\beta = 1.0$. Recompute n_c^* for $\beta = 2.0$ instead, holding $\lambda = 1.5$ fixed, and use the companion notebook to confirm your answer by simulating the market just above and just below the new threshold.

3. Extend the three-bank contagion network of Section 20.3.2 by adding a fourth bank, D, to whom Bank C owes \$20, with D holding \$15 of external assets and no other obligations. Recompute the clearing vector under the same \$45 shock to Bank A's external assets, and report whether the cascade reaches D.

Wulin.Suo@Queensu.ca

4. Section 20.4.2's hub-and-spoke example used three peripheral banks. Redo the calculation with six peripheral banks instead, keeping the hub's total obligation at \$90 (so each peripheral is now owed \$15) and its external assets at \$100, under the same \$45 shock. Report each peripheral's percentage loss and compare it to the three-peripheral case's 33.3%, and explain the direction of the difference.

5. Section 20.4.1 argues that matching a target set of statistical moments does not confirm an ABM's underlying mechanism. Propose one additional form of evidence, beyond matching moments, that would make you more confident a particular agent-based mechanism is the right explanation for an observed market phenomenon.

6. Using the companion notebook's regime-switching simulation, lower the switching threshold from 0.25 to 0.15 and re-run the 500-period simulation. Report the new variance ratio and kurtosis, and explain, in terms of how often the market now enters the volatile regime, why the direction of the change makes sense.

7. Section 20.3.4's learning seller happened to explore into a lucky sequence. Recompute its running averages after exploration if rounds 1 through 4 instead see buyer values $50, 50, 50, 50$ (using the companion notebook to check your work), report which arm the seller would then play for the rest of the sequence, and explain why this outcome is a genuine failure of the explore-exploit process rather than merely bad luck that later self-corrects.

8. Using Section 20.3.4's discrete-choice market, recompute $n_{f,1}$, $n_{c,1}$, $n_{f,2}$, and $n_{c,2}$ with a lower intensity of choice, $\theta = 0.2$ instead of 0.5, holding every other parameter and the same $p_{-1} = 93$, $p_0 = 95$ fixed, and report the first period at which $n_{c,t}$ exceeds the critical threshold $n_c^* \approx 0.667$ derived in Section 20.2.3, comparing it to the $\theta = 0.5$ case worked in the text. Does halving the intensity of choice postpone the crossing, change its severity, or both?

9. **Challenge (real data).** Download the BIS locational banking statistics (the cross-border claims table, freely available at stats.bis.org with no registration required) for five or six reporting countries. (a) Treating each country's aggregate cross-border claims on the others as a simplified interbank exposure network, construct a liabilities matrix analogous to Section 20.3.2's three-bank example (using each country's claims on another as that other country's liability), and compute the Eisenberg-Noe clearing vector following an assumed shock that wipes out 10% of one country's external, non-interbank assets. (b) Compare the resulting loss cascade to this chapter's toy three-bank result, and discuss whether the real network's exposures look more chain-like or more hub-and-spoke, referencing Section 20.4.2's topology comparison. (c) Explain one specific way in which country-aggregated BIS data understates the fragility of the true institution-level network (for example, netting across many banks within a country, or omitting domestic interbank exposures entirely), and describe, referencing this chapter's RAMSI case vignette, what institution-level supervisory exposure data a central bank could use to close that gap.

20.8 Further Reading

- Schelling, T. C., "Dynamic Models of Segregation," *Journal of Mathematical Sociology*. The original checkerboard segregation model discussed in Section 20.2.1, the earliest and best-known agent-based model outside finance.

- Arthur, W. B., Holland, J. H., LeBaron, B., Palmer, R., and Tayler, P., "Asset Pricing Under Endogenous Expectations in an Artificial Stock Market," in *The Economy as an Evolving Complex System II*. The Santa Fe Institute Artificial Stock Market discussed in Section 20.2.1, widely regarded as the founding model of agent-based computational finance.

- Gode, D. K., and Sunder, S., "Allocative Efficiency of Markets with Zero-Intelligence Traders," *Journal of Political Economy*. The original zero-intelligence double-auction result worked through by hand in Section 20.2.2.

Wulin.Suo@Queensu.ca

- Brock, W. A., and Hommes, C. H., "A Rational Route to Randomness," *Econometrica*. The performance-based strategy-switching framework underlying Section 20.2.3's fundamentalist-chartist model.

- Lux, T., and Marchesi, M., "Scaling and Criticality in a Stochastic Multi-Agent Model of a Financial Market," *Nature*. An influential heterogeneous-agent model reproducing fat tails and volatility clustering, discussed in Section 20.3.1.

- Eisenberg, L., and Noe, T. H., "Systemic Risk in Financial Systems," *Management Science*. The clearing-vector framework for financial contagion worked through by hand in Section 20.3.2.

- Acemoglu, D., Ozdaglar, A., and Tahbaz-Salehi, A., "Systemic Risk and Stability in Financial Networks," *American Economic Review*. The "robust-yet-fragile" network-topology result discussed in Section 20.4.2.

- Zhang, K., Yang, Z., and Başar, T., "Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms," in *Handbook of Reinforcement Learning and Control*. A survey of the multi-agent reinforcement learning methods contrasted with fixed-rule agent-based models in Section 20.3.4, including the convergence difficulties that arise once multiple agents learn simultaneously.

- Bank of England, "RAMSI: A Top-Down Stress-Testing Model," Bank of England Financial Stability Paper No. 17, 2012. The real-world central-bank stress-testing model, top-down and network-based, discussed in the case vignette above.

- Axtell, R. L., and Farmer, J. D., "Agent-Based Modeling in Economics and Finance: Past, Present, and Future," *Journal of Economic Literature* 63(1): 197–287. A comprehensive, up-to-date survey of the field from two of its leading figures, and the best single starting point for a reader who wants to go beyond this chapter's introductory treatment.

- LeBaron, B., "Agent-Based Computational Finance," in *Handbook of Computational Economics*. An earlier survey of the agent-based finance literature specifically, including the calibration and validation challenges raised in Section 20.4.1.

- Farmer, J. D., and Foley, D., "The Economy Needs Agent-Based Modelling," *Nature*. An argument, from two leading practitioners, for agent-based approaches as a complement to equilibrium modeling in macroeconomics and finance.

Wulin.Suo@Queensu.ca

Chapter 21

Case Studies and Capstone Projects

21.1 Introduction

Every dataset used so far in this book, Cascade Home & Garden's satellite-derived car counts, Meridian Fabrication's financial statements, the ten-transaction fraud-scoring sample, was constructed for a single purpose: to make one method's mechanics checkable by hand, uncluttered by the messiness real data always carries. That was the right choice for teaching each method in isolation, but it leaves an honest gap this closing chapter exists to fill: a real financial dataset, of the kind a working data scientist actually downloads from a public repository such as the UCI Machine Learning Repository or Kaggle, arrives with undocumented category codes, an imbalanced target, correlated and redundant features, and no guarantee that the model that wins on one metric also wins on every other metric that matters.

This chapter works a single such dataset end to end, using nothing but tools this book has already built: feature engineering and a model bake-off spanning linear and tree-based methods, both from Chapter 6; real explainability tooling, Chapter 18's SHAP and permutation-importance methods, now run with the actual `shap` library rather than by hand; a cost-sensitive threshold choice extending Chapter 15's alert economics; and a real fairness audit extending Chapter 18's synthetic two-group example with genuine data. One topic here is genuinely new, calibration, and it is motivated directly by Chapter 9's own caveat that a credit score is only meaningful if the probability underneath it is accurate. The chapter then closes with governance considerations and a set of further capstone project ideas for the reader to pursue independently.

Two further, more advanced analyses push past what a first pass through this pipeline would typically include: a systematic hyperparameter search quantifying exactly how much is actually gained by tuning a model beyond a single, reasonable configuration, and an intersectional fairness cut that splits the test population two ways at once and uncovers a clear four-fifths-rule violation that this chapter's own single-attribute fairness audit, run first, entirely missed.

The dataset is the *Default of Credit Card Clients* dataset (Yeh and Lien, 2009), 30,000 Taiwanese credit card accounts with 23 raw features and a binary label, default on payment the following month, hosted on the UCI Machine Learning Repository and also widely distributed on Kaggle under the same name, the kind of source this chapter's title points to. Unlike every other dataset in this book, it was not built to illustrate anything in particular, and working through it surfaces problems no fictional dataset would: undocumented category codes that must be cleaned before any model can be fit, a real 22.1% default rate rather than a convenient round number, and, as this chapter's fairness audits show directly, a model that passes a single-attribute fairness check comfortably while failing an intersectional one, even though no protected attribute is among its inputs.

This chapter's structure mirrors the order a working data scientist actually follows on a new dataset, and is worth previewing before working through the details. It begins where any real project begins, inspecting the raw data closely enough to find its undocumented quirks (Section 21.2.1) and engineering features from raw monthly statements rather than starting from an already-clean feature table (Section 21.2.2). It then compares several model families on identical, held-out data (Section 21.3.1) and checks whether the winning model's probabilities can actually be trusted at face value (Section 21.3.2). From there it explains that model's behavior using real software rather than hand computation (Section 21.3.3), converts model output

Wulin.Suo@Queensu.ca

into an actual lending decision under an explicit cost assumption (Section 21.3.4), and audits that decision for fairness, first along one dimension (Section 21.3.5) and then, in Section 21.4.2, along two at once. A closing discussion of what deploying this pipeline in production would actually require (Section 21.4.1) is followed by an explicit, reusable template distilled from that same pipeline (Section 21.4.3), and then by a set of further capstone projects to apply the template to (Section 21.4.4). Those end the chapter, and the book, on the same note the introduction to Chapter 1 opened it: these tools are only as trustworthy as the discipline applied in using them.

21.2 The Data

The capstone begins where every real project does: with a dataset that is not what its documentation claims.

21.2.1 The Capstone Dataset and Its Imperfections

The raw dataset records, for each of 30,000 accounts, a credit limit (`LIMIT_BAL`), demographic fields (`SEX`, `EDUCATION`, `MARRIAGE`, `AGE`), six months of repayment status codes (`PAY_1` through `PAY_6`, where -1 indicates payment made duly and a positive integer indicates the number of months a payment is overdue), six months of billed amounts (`BILL_AMT1`–`BILL_AMT6`), six months of amounts actually paid (`PAY_AMT1`–`PAY_AMT6`), and the target, default on the following month's payment.

Loading the data immediately surfaces the first real-world imperfection the fictional datasets of earlier chapters never had: the data dictionary documents `EDUCATION` as taking values 1 (graduate school), 2 (university), 3 (high school), or 4 (other), yet the actual column also contains 14 rows coded 0, 280 rows coded 5, and 51 rows coded 6, none of which the documentation defines. `MARRIAGE` shows the same pattern: documented as 1 (married), 2 (single), or 3 (other), the column also contains 54 rows coded 0.

Neither anomaly is disclosed anywhere in the dataset's documentation, and a modeler who does not notice them will silently feed a machine learning model category codes with no defined meaning. Following the standard practice for this well-studied dataset, both sets of undocumented codes are folded into their respective "other" categories (0, 5, and 6 into `EDUCATION`'s existing category 4; 0 into `MARRIAGE`'s existing category 3) before any modeling begins, a cleaning step with no equivalent anywhere else in this book precisely because every other dataset was built clean by construction.

The repayment-status columns carry two further quirks of the same kind. First, the raw file has no `PAY_1` at all: the most recent month's status column is named `PAY_0`, while the billed and paid amounts run from `BILL_AMT1` and `PAY_AMT1`, so the companion notebook renames `PAY_0` to `PAY_1` to line the three six-month series up. Second, the dictionary defines the status codes as -1 (paid duly) and 1 through 9 (months overdue) only, yet the columns are dominated by two undocumented codes, -2 and 0, which together make up 66.9% of all monthly status entries and appear at least once for 25,939 of the 30,000 accounts. They are commonly read as "no balance to pay" and "revolving credit, minimum payment made," but nothing in the official documentation says so. This chapter leaves them as they are; since both rank below 1, neither counts as a delinquency in the `max_delinquency` feature defined below, which is a modeling decision a reader should notice rather than inherit silently.

Table 21.1 summarizes four representative raw fields before any cleaning or feature engineering, and is itself worth inspecting closely rather than skimming past, the habit Chapter 6 recommended before fitting any model to any dataset.

Field	Mean	Std. dev.	Min	Median	Max
`LIMIT_BAL` (NT$)	167,484	129,748	10,000	140,000	1,000,000
`AGE` (years)	35.5	9.2	21	34	79
`BILL_AMT1` (NT$)	51,223	73,636	−165,580	22,382	964,511
`PAY_AMT1` (NT$)	5,664	16,563	0	2,100	873,552

Table 21.1: Summary statistics for four raw fields (before cleaning)

`BILL_AMT1`'s negative minimum is yet another genuine data quirk beyond the code anomalies above,

Wulin.Suo@Queensu.ca

not documented anywhere in the dataset's description: a negative billed amount means the account was in credit, having overpaid the previous month, rather than owing anything at all, a legitimate state for a revolving credit account but one that a modeler unfamiliar with the data might mistake for a data-entry error and drop or clip without justification. `PAY_AMT1`'s heavy right skew, a median of just NT$2,100 against a mean of NT$5,664 and a maximum of NT$873,552, is the kind of distribution Section 21.2.2's engineered ratio features are designed to normalize before they reach a linear model, though the tree-based models in Section 21.3.1 are considerably less sensitive to this kind of skew than logistic regression is, since a tree splits on a feature's rank order rather than its raw scale.

Two further properties of the raw data are worth noting before any model is fit. The first is the class balance. A 22.1% default rate is considerably higher than a typical, real-world consumer credit-card portfolio, which more commonly runs in the low single digits, and the gap is a reminder that a published research dataset's class balance reflects however it happened to be sampled and whichever period it happened to be drawn from, not necessarily the balance any other institution's own portfolio will show. This data covers a six-month window in 2005, shortly before a Taiwanese consumer-credit crisis. A model built on it should therefore not be assumed to transfer its absolute predicted probabilities to a materially different portfolio without recalibration, even if its ranking of relatively risky against relatively safe accounts transfers reasonably well.

The second property is redundancy. Several of the raw features are highly correlated with each other by construction: the six `BILL_AMT` columns track a single account's revolving balance from one month to the next, so they move together closely. That matters directly for Section 21.3.3's explanation exercise, because an importance or attribution score assigned to any one of several mutually correlated features is, to some extent, arbitrary, a model being able to substitute one for another with little effect on its predictions. Chapter 6 raised this issue in the abstract when discussing multicollinearity in linear regression, and it carries over, in a different technical form, to tree-based models and their explanations alike.

21.2.2 From Raw Fields to Model-Ready Features

Chapter 6 introduced feature engineering in the abstract; this section performs it on genuinely raw fields for the first time. Five engineered features summarize the six months of repayment and billing history into a form a model can use more directly than six separate monthly columns each. `max_delinquency` is the worst (maximum) repayment-status code across all six months, a single number capturing whether an account was ever seriously overdue even if its most recent month looks clean. `avg_utilization` is the average, across the six months, of billed amount divided by credit limit, a standard credit-risk signal Chapter 9 introduced conceptually but never computed from raw statement data. `total_bill` and `total_paid` are the six-month sums of billed and paid amounts. And `pay_to_bill_ratio` is the ratio of the two, capturing how much of what an account owed it actually paid back over the window; it is clipped at 5 to bound the influence of a small number of accounts with an unusually large payment relative to a near-zero bill. These five, together with 20 of the 23 raw fields, give 25 features in total. The three raw fields left out are `SEX`, `MARRIAGE`, and `AGE`: taking U.S. fair-lending law as the reference point, as Chapter 18 did, the Equal Credit Opportunity Act and its Regulation B forbid basing a credit decision on sex or marital status and allow age only in narrow ways (for instance, as a variable in a demonstrably and statistically sound scoring system that never treats applicants aged 62 or older less favorably), so the cleanest choice is to keep all three out of the model and hold `SEX` and `AGE` aside purely as attributes for Sections 21.3.5 and 21.4.2's fairness audits.

The 30,000 accounts are split 80/20 into a training set of 24,000 and a held-out test set of 6,000, stratified by the default label so that both sets preserve the same 22.1% default rate, using a fixed random seed so the split (and every result in this chapter) is exactly reproducible from the companion notebook. Every model comparison, calibration check, explanation, threshold choice, and fairness audit in the rest of this chapter is computed on this same held-out test set, never on the data any model was fit to. Chapter 6 introduced that discipline as essential to detecting overfitting, and it matters more, not less, once a real dataset's quirks make it easy to convince oneself that a model has learned something it has only memorized.

One further discipline is worth naming explicitly because a real dataset makes it easy to violate by accident: every one of this section's engineered features, `max_delinquency` included, is computed from information available at the time each account's repayment history was recorded, not from anything that depends on knowing whether the account actually defaulted. This sounds obvious, but Chapter 17's point-

Wulin.Suo@Queensu.ca

in-time discipline for alternative data applies here in a more subtle form: a feature engineered carelessly from a dataset that also contains the outcome column, for instance a rolling average that accidentally incorporates a future month's billing data, can leak information about the label into the features and produce a model that looks excellent on a held-out test set drawn from the same leaking dataset, yet fails immediately once deployed on genuinely new applicants for whom that same leakage is not available. This dataset's structure happens to make such leakage easy to avoid, since every raw field precedes the single target month by construction, but the discipline of checking for it explicitly, not merely assuming a clean-looking dataset has none, is the habit a reader should carry into Section 21.4.4's further projects, several of which (the Lending Club dataset in particular) are considerably easier to leak by accident than this one.

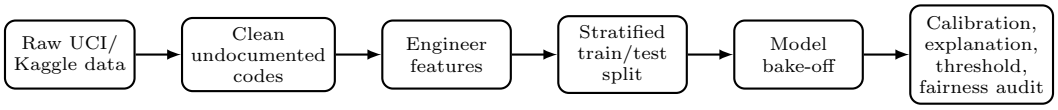


Figure 21.1: The end-to-end pipeline this chapter works through by hand

21.3 Modeling and Evaluation

This part runs a full bake-off, then asks the questions that decide whether any of it is usable.

21.3.1 A Model Bake-Off: Logistic Regression, Random Forests, and Gradient Boosting

Four models are fit on the training set and evaluated, once, on the untouched test set: a plain logistic regression (Chapter 6's workhorse, standardized features, no adjustment for class imbalance), a class-weighted logistic regression (Chapter 15's first remedy for imbalanced classification, applied here directly rather than to a stylized example), a random forest (300 trees, Chapter 6's bagging-based ensemble), and gradient-boosted trees via XGBoost (300 boosting rounds, Chapter 6's sequential, bias-reducing ensemble family, named but not fit by hand in earlier chapters).

Table 21.2 reports seven metrics for each.

Model	Accuracy	Precision	Recall	F_1	ROC-AUC	PR-AUC	Brier
Logistic (plain)	0.807	0.683	0.238	0.353	0.720	0.494	0.146
Logistic (balanced)	0.731	0.426	0.615	0.503	0.721	0.485	0.205
Random Forest	0.816	0.660	0.350	0.458	0.775	0.554	0.136
XGBoost	0.818	0.665	0.356	0.464	0.778	0.557	0.135

Table 21.2: Held-out test set performance for four models on the credit-default dataset

The plain logistic regression's accuracy (80.7%) looks respectable in isolation, the trap Chapter 15 warned against for a rare-event problem: with a 22.1% default rate, a model that is far too conservative about flagging defaults can still post high accuracy simply by defaulting, so to speak, to predicting the majority class, and this model's recall of only 23.8% shows it is missing more than three-quarters of the accounts that actually default at the standard 0.5 threshold. Class weighting flips the trade-off almost exactly as Chapter 15 described in the abstract: recall rises to 61.5% while precision falls from 68.3% to 42.6%, a substantially better F_1 (0.503 versus 0.353) but, notably, a distinctly worse Brier score (0.205 versus 0.146). That pairing is the first concrete evidence in this book that class weighting is not a free improvement. Reweighting the training loss to counter class imbalance also distorts the model's output away from a well-calibrated probability estimate, a real cost Chapter 15 never had occasion to quantify with an imbalance-sensitive metric like the Brier score, since its own worked example used a perfectly balanced confusion-matrix illustration rather than a held-out probability calibration check.

Wulin.Suo@Queensu.ca

The two tree ensembles dominate both discrimination metrics, ROC-AUC and PR-AUC, over either logistic regression. The random forest versus XGBoost comparison also gives Chapter 6's abstract bagging-versus-boosting distinction a concrete empirical instance for the first time: XGBoost's sequential, error-correcting boosting edges out the random forest's parallel, variance-reducing bagging on every metric in this table, though by a narrower margin (ROC-AUC 0.778 versus 0.775) than the gap either ensemble opens up over logistic regression. That pattern is consistent with a general finding, well documented in the applied machine learning literature, that gradient boosting and random forests are far closer competitors with each other on tabular data than either is with a linear model, even though boosting frequently comes out slightly ahead in practice.

This comparison is also a useful corrective to a common oversimplification, that a single logistic regression is always the "safe, interpretable" choice and a tree ensemble is always the "powerful, opaque" one. Both ensembles here outperform both logistic regressions on every ranking-based metric by a wide margin, roughly five to six points of ROC-AUC, while remaining explainable in the sense Chapter 18 formalized, using the real tooling Section 21.3.3 applies below, just not in the closed-form, coefficient-reading sense that made Chapter 9's linear credit-scoring model interpretable by construction.

A lender choosing between these four models is therefore not simply trading accuracy for interpretability along a single dial, as Chapter 6's abstract discussion sometimes suggests; it is trading a small amount of post-hoc explanation overhead (Section 21.3.3's SHAP computation) for a real, measurable gain in both discrimination and, as Section 21.3.2 shows next, calibration.

21.3.2 Is the Model's Probability Actually a Probability? Calibration in Practice

Chapter 9 noted, in passing, that a credit score is only meaningful to the extent that the default probability underlying it is itself well calibrated, not merely useful for ranking applicants from safest to riskiest; this section makes that check concrete for the first time.

Calibration asks whether, among all accounts a model assigns a predicted default probability of, say, 30%, roughly 30% actually default. Table 21.3 bins the test set into ten deciles of predicted probability for the plain logistic regression and for XGBoost, and compares each decile's average predicted probability to its actual observed default rate.

	Logistic (plain)		XGBoost	
Decile (low to high)	Mean predicted	Mean actual	Mean predicted	Mean actual
1	0.040	0.135	0.036	0.045
5	0.167	0.118	0.131	0.152
6	0.185	0.120	0.159	0.183
10	0.577	0.670	0.725	0.697

Table 21.3: Calibration by predicted-probability decile: plain logistic regression versus XGBoost

XGBoost's predicted and actual rates track closely, within three percentage points, in every decile shown (and, checking the full ten-bin table in the companion notebook, in every decile not shown here as well), consistent with its lower Brier score in Table 21.2. The plain logistic regression shows a more visible gap in its lowest decile, an average predicted probability of 4.0% against an actual default rate of 13.5%, more than three times higher, meaning the safest-looking accounts by this model's own ranking are considerably riskier in reality than the model's probabilities suggest, the kind of miscalibration a lender relying on this model's raw probability, rather than just its rank ordering, would want caught before deployment.

This is precisely why Section 21.3.4's cost-sensitive threshold choice and Section 21.3.5's fairness audit are both computed using XGBoost's probabilities rather than the plain logistic regression's, since both exercises depend on the model's predicted probability meaning what it claims to mean, not merely on the model's ability to rank risky accounts above safe ones.

Wulin.Suo@Queensu.ca

Fixing Miscalibration: Isotonic Recalibration

Identifying the plain logistic regression's calibration gap raises the natural next question: can it be corrected without discarding the underlying model and its accuracy? **Isotonic recalibration** fits a separate, nonparametric, monotonically increasing function that remaps a model's raw predicted probabilities onto better-calibrated ones. The remapping is learned from data the original model was fit on, but evaluated out-of-fold, which is Section 21.2.2's train/test discipline applied one level deeper, to the recalibration step itself. Fitting the remapping function directly on the test set, the way a careless analyst might, rather than by cross-validation on the training set alone, would leak the very data this recalibration is meant to be validated against.

Applied to the plain logistic regression, scikit-learn's `CalibratedClassifierCV` with isotonic regression, five-fold cross-validated entirely within the training set, improves the test-set Brier score from 0.1460 to 0.1419, a modest but genuine reduction, while leaving ROC-AUC essentially unchanged (0.720 before, 0.724 after). That is exactly what an isotonic remapping should do. Because the transformation is monotonic, it cannot change how the model ranks any two applicants relative to each other, only how well its stated probability matches reality at a given rank. The improvement is concentrated exactly where Table 21.3 found the original problem.

Recomputing the reliability table on the recalibrated probabilities for the lowest-probability bin,

$$\begin{aligned} \text{Recalibrated:} &\quad \text{predicted} = 9.8\%,\ \text{actual} = 12.3\%,\ \text{ratio} \approx 1.2, \\ \text{Original (uncalibrated):} &\quad \text{predicted} = 4.0\%,\ \text{actual} = 13.5\%,\ \text{ratio} > 3, \end{aligned}$$

This is a genuinely useful, general lesson beyond this one dataset: a model with strong discriminative power but poor calibration, the plain logistic regression's situation here, does not need to be replaced to become trustworthy as a probability estimate, only recalibrated, a considerably cheaper fix in practice than retraining an entirely different model family, and one that leaves every one of the model's original coefficients, and therefore its adverse-action interpretability, completely intact.

It is worth being equally clear about what recalibration does not fix: XGBoost's own Brier score (0.135) remains lower than the recalibrated logistic regression's (0.142) even after this correction, since XGBoost's advantage was never purely a calibration artifact to begin with, its higher ROC-AUC in Table 21.2 reflects genuinely superior discriminative power that a monotonic remapping of the logistic regression's probabilities cannot manufacture out of a linear model's more limited functional form.

Calibration matters beyond this chapter's own threshold and fairness exercises, too. A bank reporting expected credit losses under accounting standards such as IFRS 9 or the analogous U.S. CECL framework multiplies a probability of default directly into a provisioning calculation, Chapter 9's own EL = PD×LGD×EAD decomposition. A systematically miscalibrated probability, not merely a poorly ranked one, therefore translates directly into an over- or understated loss provision on the bank's own financial statements. That provision is exactly the kind of number Chapter 4 taught how to read and Chapter 9 taught how to model, and it is shown here to depend on a property, calibration, that a model's accuracy or even its ROC-AUC does not by itself guarantee.

Section 21.3.3 returns to this formula with real provisioning numbers, once this chapter's own low-risk and high-risk applicants have been introduced.

21.3.3 Explaining a Real, Nonlinear Model: SHAP and Permutation Importance

Chapter 18 computed an exact Shapley decomposition by hand for Chapter 9's credit-scoring model only because that model was linear in the log-odds; XGBoost is not, so the same exact closed form is unavailable here. The `shap` library's `TreeExplainer` algorithm computes exact Shapley values efficiently for tree ensembles specifically, without requiring the linear-model shortcut Chapter 18 relied on, by exploiting the tree structure directly rather than approximating with sampled perturbations the way Chapter 18's LIME example did.

Table 21.4 compares the resulting global feature ranking, mean absolute SHAP value across 1,000 held-out accounts, against Chapter 18's other global method, permutation importance (here, the drop in ROC-AUC when a feature is shuffled), computed on the full test set.

Wulin.Suo@Queensu.ca

Rank	Permutation importance (feature)	Mean |SHAP| (feature)
1	PAY_1 (0.0337)	max_delinquency (0.444)
2	max_delinquency (0.0318)	PAY_1 (0.339)
3	avg_utilization (0.0118)	avg_utilization (0.167)
4	BILL_AMT1 (0.0094)	BILL_AMT1 (0.130)
5	LIMIT_BAL (0.0057)	LIMIT_BAL (0.118)

Table 21.4: Top features by two global importance methods, XGBoost model

The two methods agree closely on which five features matter, the most recent repayment status, the worst repayment status over six months, utilization, the most recent bill, and credit limit, and even order ranks three through five identically, but disagree on the ranking of the top two: permutation importance ranks the single most recent month's repayment status (PAY_1) above the worst-ever status across six months, while SHAP ranks them the other way around. Neither ranking is simply wrong; they answer different questions, the caution Chapter 18 raised about permutation importance describing a feature's contribution to a specific performance metric on specific data, while SHAP describes each feature's average marginal contribution to individual predictions, and a modeler who reports only one of the two risks overstating a false precision about which single feature "matters most."

A single applicant's explanation makes this concrete at the individual level Section 21.3.1's aggregate table cannot. The first test-set applicant is assigned a predicted default probability of 13.07% by XGBoost.

SHAP's additive decomposition operates on the model's raw log-odds output rather than on the probability directly, exactly as Chapter 18's exact Shapley decomposition operated on log-odds for the linear credit-scoring model. This applicant's base value, the average log-odds prediction across the training population, is -1.272, and summing the individual SHAP contributions from every feature gives a total log-odds of -1.895. The sigmoid function converts that back to 13.07%, exactly matching the model's actual output. This verifies the same completeness property Chapter 18 checked by hand for a linear model, now confirmed for a genuinely nonlinear ensemble instead. The applicant's largest contributions are

$$\begin{aligned} \texttt{max_delinquency} = 0 \text{ (no serious delinquency)}: &\quad \text{SHAP} = -0.309, \\ \texttt{avg_utilization} = 12.1\% \text{ (low)}: &\quad \text{SHAP} = -0.215, \\ \text{Credit limit} = \text{NT\$50,000 (modest)}: &\quad \text{SHAP} = +0.175, \\ \text{Most-recent repayment status} = -1 \text{ (paid duly)}: &\quad \text{SHAP} = -0.151, \end{aligned}$$

the modest credit limit contributing positively to predicted risk since, in this dataset, lower limits are associated with higher-risk accounts.

Unlike Chapter 18's adverse-action reason codes, which followed directly and automatically from an exactly linear model's coefficients, producing this same kind of individualized explanation for a real deployed ensemble requires this extra step, running `TreeExplainer` against the fitted model, before a bank could issue a genuine, model-derived reason to an applicant under the Equal Credit Opportunity Act.

A contrasting, genuinely high-risk applicant from the same test sample makes the method's value clearer still. This second applicant is assigned a predicted default probability of 92.2%, and the same decomposition gives a base value of -1.272 and a summed log-odds of 2.470 (sigmoid of which is, again exactly, 92.2%). Here the dominant contributions run in the opposite direction from the low-risk applicant above:

$$\begin{aligned} \text{Most-recent repayment status} = 3 \text{ (three months overdue)}: &\quad \text{SHAP} = +1.009, \\ \text{Worst-ever repayment status} = 7 \text{ (severely delinquent)}: &\quad \text{SHAP} = +0.921, \\ \text{Credit limit} = \text{NT\$10,000 (the dataset's minimum)}: &\quad \text{SHAP} = +0.303, \end{aligned}$$

with pay_to_bill_ratio $= 0$ (repaid essentially nothing over six months) adding a further $+0.151$, and the most-recent-repayment contribution the single largest seen in either applicant's decomposition.

Placed side by side, the two applicants' explanations read almost as mirror images, no delinquency history and full repayment pushing one applicant's risk sharply down, recent and severe delinquency alongside near-zero repayment pushing the other's sharply up, the kind of concrete, individualized contrast an adverse-action

Wulin.Suo@Queensu.ca

notice or an internal credit-committee review would need, and exactly what a single aggregate feature-importance ranking like Table 21.4 cannot provide on its own.

A Worked Example: From Predicted Probability to a Provisioning Number

Section 21.3.2 promised to return to Chapter 9's EL = PD × LGD × EAD decomposition with real numbers; the two applicants just explained supply exactly what is needed. Each applicant's credit limit stands in for exposure at default, a standard simplifying assumption for a revolving credit-card line assumed fully drawn at the moment of default, and, absent any collateral information for an unsecured card balance, this example reuses Chapter 9's own illustrative LGD = 60%.

All amounts in this dataset are in New Taiwan dollars (NT$), not U.S. dollars; at the 2005 exchange rate of roughly NT$32 to the U.S. dollar, the low-risk applicant's NT$50,000 limit is worth only about US$1,550. For the low-risk applicant, PD = 13.07077% (the 13.07% reported earlier, carried to more decimals to reproduce the figure below exactly by hand) and EAD = NT$50,000 (this applicant's `LIMIT_BAL`), giving

$$\text{EL}_{\text{low-risk}} = 0.1307077 \times 0.60 \times \text{NT\$50,000} = \text{NT\$3,921.23},$$

while the high-risk applicant, with PD = 92.19913% (similarly carried to more decimals) and a much smaller EAD = NT$10,000 (a lower credit limit, itself consistent with this applicant's weaker profile), gives

$$\text{EL}_{\text{high-risk}} = 0.9219913 \times 0.60 \times \text{NT\$10,000} = \text{NT\$5,531.95}.$$

The high-risk applicant's expected loss is only about 1.41× the low-risk applicant's, a far smaller multiple than the two applicants' 7.05× gap in predicted default probability alone (92.20%/13.07%), because the high-risk applicant's credit limit, one-fifth of the other's, largely offsets an otherwise much larger provisioning number, the kind of interaction between the three separate inputs to expected loss that looking at PD in isolation would miss. A bank provisioning against its entire card portfolio would sum a version of this calculation across every account, so a systematic miscalibration of the kind Section 21.3.2 found and fixed in the plain logistic regression, understating risk in the model's lowest-probability decile, would understate the provisioning total specifically for the bank's safest-looking accounts, understating reserves in the segment a supervisor examining the bank's provisioning adequacy would be least likely to scrutinize closely.

`TreeExplainer`'s speed is itself worth noting: it computes exact Shapley values for a tree ensemble in time roughly proportional to the number of trees times the square of tree depth, rather than the exponential cost the general, model-agnostic Shapley formula Chapter 18 mentioned in passing would require, which is precisely why it is practical to run it here against a genuine, several-hundred-tree ensemble rather than only against the small, linear, two-feature toy models Chapter 18's hand computations were limited to. This matters operationally, not just theoretically: a bank issuing thousands of adverse-action notices a month needs an explanation method fast enough to run at that volume, and a sampling-based method like Chapter 18's LIME, however useful for a single, illustrative explanation, does not scale to that volume nearly as cleanly as an algorithm purpose-built for the specific model family being explained.

21.3.4 Choosing an Operating Threshold: A Real Cost-Sensitive Analysis

Every result so far used the default 0.5 classification threshold; Chapter 15's alert-economics section argued that the right threshold is a business decision balancing the cost of a missed default against the cost of denying a good customer, not a property of the model itself, and this section quantifies that trade-off directly rather than only in the abstract. Assigning an illustrative cost of $500 to a missed default (a false negative, an account that defaults but is approved) and $50 to wrongly denying a good customer (a false positive, a tenth of the missed-default cost, reflecting that a denied good customer typically costs a lender foregone interest income rather than a lost principal balance), the expected total cost across the test set is $\text{Cost}(t) = 500 \cdot FN(t) + 50 \cdot FP(t)$ at a given threshold t.

Table 21.5 sweeps this across a grid of thresholds using XGBoost's predicted probabilities.

The cost-minimizing threshold on this grid is $t = 0.10$, far below the naive 0.5 default, at a total expected cost of $209,750, versus $438,900 at the naive threshold, more than double. That value could have been

Wulin.Suo@Queensu.ca

Threshold	False negatives	False positives	Total cost
0.05	29	4,071	$218,050
0.10 (cost-minimizing)	129	2,905	$209,750
0.20	450	1,218	$285,900
0.35	667	510	$359,000
0.50 (naive default)	854	238	$438,900
0.60	918	179	$467,950

Table 21.5: Cost-sensitive threshold sweep, XGBoost model, held-out test set

predicted without any sweep. Denying an account with predicted default probability p is cheaper than approving it whenever $p \times \$500 > (1-p) \times \$50$, so for a calibrated model the cost-minimizing threshold is

$$t^* = \frac{c_{FP}}{c_{FP} + c_{FN}} = \frac{50}{50 + 500} = 0.091,$$

the same expected-cost comparison Chapter 15 used to derive its fraud-review threshold, adapted to a decision in which a correctly denied defaulter costs nothing. Only the ratio of the two costs matters, not their level or currency. The grid's empirical minimum at 0.10 sits right next to this analytic value (at exactly 0.091 the test-set cost is $214,100, the small difference being sampling noise in 6,000 accounts), which is itself a further check on Section 21.3.2's finding that XGBoost's probabilities are well calibrated.

The gap between $209,750 and $438,900 is itself the lesson: choosing a classification threshold by convention rather than by the institution's actual cost structure leaves real money on the table, considerably more here than the gap between any two models in Table 21.2, which is why Chapter 15 treated threshold choice as at least as consequential as model choice. The specific optimal threshold found here depends entirely on the assumed 10-to-1 cost ratio; a lender who instead estimated missed defaults and denied good customers as comparably costly would land on a threshold much closer to 0.5, underscoring that this is a genuine business judgment, not a number this chapter's methods can supply on their own.

The operating point this produces deserves an equally plain statement: at $t = 0.10$ the model approves only 31.6% of the test accounts, against 88.2% at the naive 0.5 threshold, and turns away 62.2% of the accounts that would not have defaulted. No real card lender operates anywhere near that strict a cutoff. The toy cost ratio is what drives it: charging a missed default only ten times a wrongly denied good customer, against a 22.1% base default rate, pushes the threshold below the probability most accounts are assigned, whereas a real lender's cost of denying a good customer includes the lifetime value of the relationship, and the cost of a missed default is proportional to the exposure rather than a flat amount, so a realistic cost model would place the threshold, and the approval rate, very differently.

This threshold sweep is, in effect, a business-cost-weighted walk along the same receiver operating characteristic curve Chapter 6 introduced in the abstract. Every threshold in Table 21.5 corresponds to exactly one point on that curve, trading a lower false-negative count for a higher false-positive count as the threshold falls. What this section adds is not a new statistical concept but a conversion of that familiar trade-off directly into dollars, so that the operating point is chosen by comparing two cost figures rather than by squinting at a curve and picking whichever point looks like a reasonable compromise.

A lender in practice would also want to stress-test this choice the way Chapter 9 stress-tested a credit portfolio: if a recession doubled the true cost of a missed default relative to a denied applicant, the cost-minimizing threshold would fall further still, flagging even more accounts as high-risk, the kind of scenario analysis a model-risk function should run before, not after, committing to a single operating threshold in production.

21.3.5 A Real Fairness Audit

Chapter 18's fairness audit used a synthetic eight-applicant dataset built to illustrate a point cleanly; this section runs the identical audit, the four-fifths rule and an error-rate comparison, on the actual test set, at

Wulin.Suo@Queensu.ca

the cost-minimizing threshold from Section 21.3.4, split by the dataset's `SEX` field. Table 21.6 reports the results.

Group	n	Approval rate	False-negative rate	False-positive rate
Male	2,402	30.9%	8.9%	62.5%
Female	3,598	32.1%	10.3%	62.0%

Table 21.6: A fairness audit by sex, XGBoost model at the cost-minimizing threshold ($t = 0.10$)

The adverse impact ratio, computed from the unrounded approval rates as 30.85%/32.13% = 0.960, clears the 80% four-fifths-rule threshold Chapter 18's fairness section formalized by a wide margin, and some gap is to be expected here, since male accounts in the test set actually default more often (23.4% against 21.3%). The error-rate comparison shows the same tension Chapter 18's synthetic example illustrated, now with genuine data: the group with the higher approval rate (female accounts) also has the higher false-negative rate, missing 10.3% of accounts that actually go on to default, versus 8.9% for male accounts, so the group nominally favored by the headline approval-rate statistic is simultaneously the group whose real default risk this specific threshold captures somewhat less completely. Neither the approval-rate comparison nor the error-rate comparison is "the" fairness result; both are properties of the same model and the same threshold, and, exactly as Chapter 18's fairness-impossibility result predicts, no adjustment to this one threshold alone can bring both gaps to zero simultaneously unless the two groups' underlying default rates happen to coincide.

It is worth being explicit about what this audit does, and does not, establish. Leaving the protected attributes out of the model mattered: the same XGBoost configuration refit with `SEX`, `MARRIAGE`, and `AGE` as inputs, which the companion notebook runs purely for contrast, approves 28.6% of male and 33.9% of female accounts at the same threshold, an adverse impact ratio of only 0.843, while scoring essentially no better (test ROC-AUC 0.779 against 0.778). But removing the attributes did not remove the difference. The model never sees `SEX`, and yet its decisions still differ measurably by sex, the pattern Chapter 18's Apple Card case vignette described in a real, consumer-facing product: "fairness through unawareness" narrows a disparity only as far as the remaining features fail to encode the omitted attribute. What this audit does not establish, on its own, is why the residual difference exists: whether it reflects the groups' genuinely different default rates or some other feature correlated with sex closely enough to reproduce the pattern. Income proxies and credit-limit assignment history are the leading suspects in this dataset, though testing that hypothesis rigorously would require the causal counterfactual-fairness framing Chapter 18's advanced topics introduced, not merely the correlational check performed here. Section 21.4.2 then shows directly that this single-attribute finding, reassuring as its 0.960 ratio looks in isolation, does not remotely tell the whole story once a second characteristic is added to the audit.

21.4 From Model to Project

A model is not a project; this part covers deployment, and the template for doing all of it again.

21.4.1 From Model to Deployment: Governance Considerations

Chapter 18's governance section organized model governance around SR 11-7's three pillars, conceptual soundness, ongoing monitoring, and outcomes analysis, and this pipeline maps onto all three concretely rather than abstractly.

Conceptual soundness is Section 21.3.1's model bake-off and Section 21.3.2's calibration check, documented well enough that an independent validator, Chapter 15's second line of defense, could reproduce every number in Table 21.2 from the companion notebook without needing to re-derive the modeling choices from scratch. Outcomes analysis is Section 21.3.5's fairness audit, which this chapter's own numbers show passing comfortably on one cut (0.960 by sex) and failing on another (Section 21.4.2's sex-by-age cut), so a one-time check at deployment is not sufficient; ongoing monitoring, using Chapter 18's population stability

Wulin.Suo@Queensu.ca

index against this exact feature set and a recurring rerun of both audits, is what would catch a shift in the applicant pool moving either ratio before an examiner does.

None of this replaces human judgment: Section 21.3.4's cost assumptions, the $500-to-$50 ratio driving the entire threshold choice, are a business decision no algorithm in this chapter can make on the institution's behalf, and choosing them is the kind of accountable, documented judgment call Chapter 15's three-lines-of-defense framework and the NIST AI RMF's "govern" function both exist to make explicit rather than implicit.

A concrete deployment checklist follows directly from this chapter's own sections, and is worth stating explicitly rather than leaving implicit. Before this model ever scores a real applicant, an institution would need Section 21.2.1's cleaning logic and Section 21.2.2's feature engineering encoded in a reproducible data pipeline rather than a one-off notebook cell. It would need Section 21.3.1's model selection and Section 21.4.2's hyperparameter search documented well enough that a validator could rerun them independently and obtain the same numbers reported here. It would need Section 21.3.2's calibration check and Section 21.3.4's cost assumptions re-approved on a recurring basis rather than fixed once at launch, since a model's calibration and an institution's true cost ratio both drift as the underlying applicant population and business environment change. And it would need Section 21.3.5 and Section 21.4.2's fairness audits, the intersectional cut included, rerun on every new cohort of applicants rather than merely at initial deployment, precisely because Section 21.4.2 showed directly that a single-attribute check passing today is no guarantee that a deeper cut would not fail tomorrow.

21.4.2 Advanced Topics

Two further analyses push this chapter's pipeline beyond what a first pass through it would typically include: a systematic search over XGBoost's hyperparameters, to see how much Section 21.3.1's single, reasonably chosen configuration actually left on the table, and a fairness audit that splits the test set two ways at once rather than one, revealing a disparity Section 21.3.5's single-attribute audit could not see at all.

Hyperparameter Tuning: Diminishing Returns in Practice

Section 21.3.1's XGBoost model used a single, manually chosen configuration (300 trees, maximum depth 4, learning rate 0.05). A randomized search over 30 candidate configurations varied tree depth (3 to 6), learning rate (0.01 to 0.1), the number of boosting rounds (100 to 500), the row and column subsampling fractions, and the minimum child weight controlling how conservatively a tree can split. Every candidate was evaluated by five-fold cross-validation on the training set alone, never touching the test set. That is exactly Section 21.2.2's train/test discipline applied one level deeper, since even a hyperparameter search must be validated on data the search itself never saw.

The best configuration found, depth 4, learning rate 0.1, 100 rounds, no row subsampling, 60% column subsampling, and a minimum child weight of 5, achieved a cross-validated ROC-AUC of 0.786, and, refit on the full training set and evaluated once on the held-out test set, a test ROC-AUC of 0.781, a PR-AUC of 0.554, and a Brier score of 0.135. Compared against Section 21.3.1's untuned XGBoost model (test ROC-AUC 0.778, PR-AUC 0.557, Brier 0.135), the result of an actual, reasonably thorough hyperparameter search is not even a uniform improvement: ROC-AUC rises by three thousandths, PR-AUC falls by three thousandths, and the Brier score does not move at the third decimal, all changes far smaller than the gap either tree ensemble opened up over logistic regression in Table 21.2.

This is a genuinely useful, if less exciting, finding. Once a model family is well chosen and its configuration is set to sensible, moderate values, as Section 21.3.1's untuned model already was, further hyperparameter search on a dataset of this size and feature set faces diminishing returns. The modest gap between the cross-validated score (0.786) and the held-out test score (0.781) carries its own small, honest reminder: a hyperparameter search's selected score is mildly optimistic, having been chosen, across 30 candidates, precisely because it happened to perform best on that particular cross-validation split.

Intersectional Fairness: What Aggregation Can Hide

Section 21.3.5's audit split the test set only by sex. Splitting it a second way at the same time, by sex and by an age cutoff at 30 years, reveals a pattern the single-attribute audit could not, shown in Table 21.7.

Wulin.Suo@Queensu.ca

Sex	Age group	n	Approval rate	False-negative rate
Male	Under 30	669	23.9%	4.7%
Male	30 and over	1,733	33.5%	10.5%
Female	Under 30	1,243	30.0%	10.1%
Female	30 and over	2,355	33.3%	10.4%

Table 21.7: An intersectional fairness cut: sex crossed with age group

The most disadvantaged subgroup by approval rate, young men under 30, is approved only 23.9% of the time, against 33.5% for the most favored subgroup, men 30 and over. That is an adverse impact ratio, from the unrounded rates, of 23.92%/33.53% = 0.713, a clear four-fifths-rule violation, and it stands in sharp contrast to Section 21.3.5's sex-only audit, which found a ratio of 0.960, comfortably on the right side of the same threshold. The two findings are not in tension; they are simply answers to different questions. Averaging young and older men together in the sex-only audit obscures the fact that young men, considered on their own, face a starkly lower approval rate than any other subgroup in this cross-tabulation, a disparity the single-attribute view cannot see because it never asks the question in the first place. Nor is this simply an age effect that a separate age audit would have caught at the same strength: among women the under-30 group is approved at 30.0% against 33.3%, a ratio of about 0.90, so the shortfall is concentrated where the two characteristics meet. It also arises in a model that, as Section 21.2.2 specified, never sees `AGE` at all, a second and sharper instance of Section 21.3.5's point that leaving an attribute out does not keep the features that correlate with it, shorter credit histories and lower assigned limits among the youngest borrowers, from carrying it back in; the most obvious such feature here is `LIMIT_BAL`, since the youngest borrowers, and young men above all, hold markedly lower credit limits (a median of NT$80,000 for men under 30 in the test set, against NT$150,000 for men 30 and over). This is precisely the failure mode fair-lending examiners test for directly under the doctrine of intersectional or subgroup analysis: a model can pass every single-attribute audit an institution runs and still produce a stark disparity for a subgroup defined by two or more characteristics at once, and the only way to find such a disparity is to look for it specifically, since Table 21.6's aggregate numbers give no indication whatsoever that it might exist.

A genuinely thorough fairness audit therefore cannot stop at Section 21.3.5's single-attribute cut, however clean its result looked; it must deliberately cross every combination of characteristics an institution has reason to worry about, a substantially larger testing burden than a single audit table might suggest, and one this dataset's own numbers show is not merely a theoretical concern.

21.4.3 A Template for Completing a Capstone Project

Section 21.4.4 below lists several real datasets a reader could build a capstone project on. This section is the template for actually doing so, an explicit, stage-by-stage checklist distilled directly from this chapter's own pipeline (Figure 21.1), so that a reader starting from any dataset on that list, or from one of their own choosing entirely, has a concrete standard to work toward rather than an open-ended instruction to "build a model." Each stage in Table 21.8 names the specific tool this chapter, or an earlier one, developed for it, along with a deliverable, a specific artifact, table, or number, that the stage should produce before the project moves on to the next. A project that cannot produce every deliverable in the table is not yet finished, whatever its headline accuracy number looks like.

A project report built around this template naturally falls into five sections, which a reader can use directly as a report or presentation outline. The **executive summary** states Stage 1's problem statement and the headline result in a single paragraph a non-technical reader could act on. **Data and methodology** covers Stages 2 and 3, written so that another analyst could reproduce the cleaning, the feature engineering, and the model choices exactly. **Results** presents Stages 3 and 4's tables together, so that a reader can judge discrimination and calibration side by side rather than being shown only whichever number happens to look most favorable. **Explainability and fairness** covers Stages 5 through 7, which belong together: an explanation of what drives the model's decisions is also evidence for or against a fairness concern, exactly as Section 21.3.5's discussion of `LIMIT_BAL` and Section 21.4.2's intersectional finding both showed directly. **Limitations and recommendations** carries Stage 8's governance memo, together with an explicit, named

Wulin.Suo@Queensu.ca

Stage	What this stage requires	Deliverable to produce and report
1. Problem framing and dataset selection	State the business decision the model will inform in one sentence (approve or deny a loan, flag or clear a transaction, and so on); confirm the chosen dataset has a genuine, non-leaking target label and enough rows to support a genuinely held-out test set.	A one-paragraph problem statement naming the target variable, its base rate, and the decision the model's output will feed into.
2. Data cleaning and feature engineering	Inspect every column's summary statistics and value counts before fitting anything, exactly Section 21.2.1's method; document any undocumented or anomalous codes found and how each was resolved; engineer at least one derived feature not present in the raw columns, and explicitly check it (and every other feature) for information that would not genuinely have been available at the moment a real prediction would be made, Section 21.2.2's leakage discipline.	A short data-quality memo listing every anomaly found and the cleaning decision made, plus the engineered feature list with a one-sentence leakage justification for each.
3. Train/test discipline and model bake-off	A stratified train/test split with a fixed, reported random seed; at least one intrinsically interpretable model (linear or logistic regression) and at least one tree ensemble, evaluated once, on the held-out test set alone, on accuracy, precision, recall, F_1, ROC-AUC, PR-AUC, and Brier score, Section 21.3.1's method.	A results table in the same format as Table 21.2, plus both sets' target base rates confirming the stratification held.
4. Calibration check	A reliability table (deciles of predicted probability against actual outcome rate) for at least the best-discriminating model, Section 21.3.2's method.	A calibration table in the format of Table 21.3, plus one sentence stating whether the model's raw probability, not merely its ranking, can be trusted at face value.
5. Explainability	A global importance ranking (permutation importance or mean absolute SHAP value) and at least one individual, local explanation for a specific prediction, Section 21.3.3's method.	A top-five global feature ranking and one specific case's local explanation, with its completeness check (SHAP values plus base value reproducing the model's actual output) shown explicitly.
6. Threshold and cost analysis	An explicit, stated assumption for the dollar cost of a false negative and a false positive, and the resulting cost-minimizing threshold found by sweeping a grid of candidate thresholds, Section 21.3.4's method.	The two assumed costs, the resulting threshold, and its total expected cost compared explicitly against the naive 0.5 threshold's cost.
7. Fairness audit	At least one single-attribute fairness cut (approval rate, the four-fifths adverse impact ratio, and a false-negative/false-positive rate comparison) and, wherever the data contains a second relevant characteristic, one intersectional cut crossing two attributes at once, Sections 21.3.5 and 21.4.2's method.	An approval-rate table and adverse impact ratio for at least one single-attribute cut, and, if attempted, one intersectional cut, with an explicit statement of whether either violates the four-fifths rule.
8. Governance write-up	A short memo addressing the three SR 11-7 pillars for this specific model: what conceptual soundness was checked, what ongoing monitoring (a population stability index or a re-audit cadence) is proposed, and what outcomes analysis would be needed before this model could be trusted with a real decision, Section 21.4.1's method.	A half-page governance memo naming a concrete monitoring metric, its planned check frequency, and who (which of Chapter 15's three lines of defense) would be responsible for acting on it.

Table 21.8: An eight-stage template for a capstone project, distilled from this chapter's own pipeline

Wulin.Suo@Queensu.ca

list of what the project did not check.

That last list is the one most often skipped, and it is the one that matters most. Section 21.4.2 showed concretely that an unchecked intersectional cut can hide a violation a report's own headline numbers give no hint of. A capstone report that does not name its own blind spots is not more trustworthy for having omitted them; it is merely less honest about the same limitation every model in this chapter has shared.

Two failure modes are worth naming explicitly, since they are the most common ways a project satisfies this template's letter while missing its point. The first is treating each stage as an isolated checkbox rather than letting later stages inform earlier ones: Section 21.3.2's finding that the plain logistic regression's safest decile was miscalibrated should change which model Stage 6 and Stage 7 are actually run on, not be reported and then quietly ignored in favor of whichever model had the best Stage 3 accuracy. The second is stopping the fairness audit (Stage 7) at the first cut that happens to pass, the mistake this chapter's own Section 21.3.5 would have made had Section 21.4.2 never been run at all; a reader whose single-attribute cut passes cleanly should treat that result as a reason to look harder for an intersectional disparity, not as a reason to stop looking.

21.4.4 Capstone Project Ideas

The pipeline built in this chapter generalizes directly to other publicly available financial datasets: clean, engineer features, split, bake off models, calibrate, explain, choose a threshold, audit for fairness. Each of the following is a genuine capstone project using a real, freely available dataset, several of them also distributed on Kaggle under the names given.

- **Credit card fraud detection.** The Kaggle/Worldline/ULB "Credit Card Fraud Detection" dataset (284,807 European card transactions, 492 confirmed frauds, features anonymized via PCA) is far more imbalanced than this chapter's 22% default rate, a genuinely severe 0.17% positive rate that makes Chapter 15's precision-recall framing, and this chapter's calibration and cost-sensitive threshold methods, considerably more consequential than they were here; a reader completing this project should expect the naive 0.5 threshold to be almost useless, and should find that Section 21.3.4's analytic rule $c_{FP}/(c_{FP} + c_{FN})$ still locates the cost-minimizing threshold only if the model's probabilities are genuinely calibrated, which is far harder to achieve and to check with 492 positives than with this chapter's 6,636.

- **Consumer lending at greater scale.** The Lending Club loan dataset (millions of originated peer-to-peer loans with real interest rates, loan grades, and repayment outcomes) extends this chapter's pipeline to a dataset large enough that model training time itself becomes a design constraint, and includes enough temporal history to test a model's stability across a full economic cycle, the "non-stationarity" concern Chapter 6 and Chapter 17 both raised only in the abstract.

- **An algorithmic trading backtest on real prices.** Historical daily prices for the S&P 500 constituents (freely available from sources such as Yahoo Finance or Stooq) let a reader rebuild Chapter 13's pairs-trading or Chapter 12's mean-variance framework on real, not fictional, return series, confronting transaction costs, survivorship bias (delisted constituents silently missing from many free datasets), and the gap between an in-sample backtest and genuine out-of-sample performance directly.

- **Financial sentiment at scale.** The Financial PhraseBank dataset (thousands of analyst-labeled sentences from financial news, by agreement level) or a Twitter financial-sentiment dataset let a reader rebuild Chapter 16's Loughran-McDonald scoring and Naive Bayes classifier on genuinely labeled data, and compare a hand-built lexicon approach against a fine-tuned transformer the way Chapter 16's chapter itself only described.

- **A second, independent fraud dataset for cross-dataset generalization.** The IEEE-CIS Fraud Detection dataset (a Kaggle competition dataset with several hundred anonymized features spanning transaction, identity, and device information) is large and high-dimensional enough to make Section 21.3.3's SHAP-based explanation genuinely necessary rather than merely illustrative, since no analyst could manually inspect several hundred features' relationships to a prediction the way this chapter's 25-feature model still just barely permits.

Wulin.Suo@Queensu.ca

- **A robo-advisor: automated, risk-based portfolio construction.** Using free historical price data for a diversified set of low-cost exchange-traded funds (the kind of instrument Chapter 3 introduced), a reader can rebuild the core of a robo-advisor, following Chapter 12's own worked treatment directly: a short risk-tolerance questionnaire mapped to a target asset allocation, the mean-variance optimizer (or its risk-parity variant) computing the actual portfolio weights on real rather than illustrative return data, periodic rebalancing back to those target weights as prices drift, and, for a genuinely realistic version, tax-loss harvesting logic and a fee-drag comparison against a comparable actively managed alternative, extending Chapter 12's stylized numbers to a real portfolio and, ideally, a second real case besides Schwab's to compare against. This project differs from every other one in this list in an instructive way: it is not a classification or regression problem at all, but an optimization and decision-rule problem, so Chapter 18's SHAP and permutation-importance tools do not directly apply, and the natural "explainability" question becomes instead whether the resulting portfolio weights are traceable back to the questionnaire's stated risk tolerance and Chapter 12's stated optimization objective, a different, allocation-based notion of interpretability worth contrasting explicitly with this chapter's classification-based one.
- **Derivative pricing and delta-hedging on real options data.** Historical options-chain data for a liquid underlying such as the S&P 500 ETF (available from sources including Yahoo Finance, CBOE historical data files, and options-focused Kaggle datasets) lets a reader confront Chapter 5's Black-Scholes-Merton and binomial models with actual market prices rather than an assumed volatility input: back out each option's implied volatility from its traded price, plot the resulting volatility smile or skew across strikes (a pattern Chapter 5's constant-volatility assumption cannot produce at all, making its own limitation directly visible), and then implement and backtest a discrete-time delta-hedging strategy, rebalancing a hedge position at fixed intervals and tracking the resulting hedging error against the theoretical, continuous-rebalancing ideal Chapter 5 derives. This project is a natural pair with Chapter 14's market-microstructure material, since a real hedging desk's rebalancing frequency is itself constrained by the transaction costs Chapter 14 modeled, so a reader can extend the backtest to show how hedging error and transaction cost trade off against each other as the rebalancing interval shrinks.
- **Portfolio risk management through a real historical crisis.** Real daily return data for a multi-asset portfolio (a mix of equity, bond, and commodity ETFs, again freely available from sources such as Yahoo Finance or Stooq) lets a reader rebuild Chapter 8's Value-at-Risk and Expected Shortfall machinery, parametric, historical-simulation, and Monte Carlo, on genuine returns rather than a small illustrative covariance matrix, and then backtest the resulting VaR estimates using Chapter 8's own exception-counting method across a window that spans a real historical stress period, the 2008 financial crisis or the 2020 pandemic crash, both of which are included in most free multi-year daily price histories. The point of choosing a window with a real crisis in it deliberately is to test the assumption every one of Chapter 8's VaR methods quietly makes, that recent historical volatility and correlation are a reasonable guide to near-term risk, against a period where that assumption visibly failed, correlations across asset classes that looked low in calm markets rose sharply during the crisis itself, a phenomenon a reader can verify directly by comparing the portfolio's rolling covariance matrix estimated inside versus outside the crisis window, before feeding either estimate into Chapter 12's mean-variance or risk-parity optimizer and comparing the resulting portfolios' actual, realized risk during the crisis itself.
- **A smaller-scale replication for a first project.** The Statlog German Credit dataset (1,000 applicants, 20 features), also mirrored on Kaggle as "German Credit Risk," is small enough to rerun this entire chapter's pipeline, cleaning, feature engineering, bake-off, calibration, SHAP, threshold choice, fairness audit, in an afternoon, making it a natural first capstone for a reader who wants to practice the full workflow once, end to end, before tackling one of the larger datasets above.

21.5 Discussion Questions

1. Table 21.2 shows the class-weighted logistic regression achieving the best F_1 score among the two logistic models but the worst Brier score of all four models. If you were advising a lender that needed

Wulin.Suo@Queensu.ca

to report a single default probability to a regulator for every applicant, which of the four models would you recommend, and would your answer change if the lender only needed to rank applicants from safest to riskiest rather than report a specific probability to anyone?

2. Section 21.2.1 cleaned two undocumented category codes by folding them into an existing "other" category. Discuss an alternative cleaning strategy (for instance, treating each undocumented code as missing data and imputing it, or as its own separate category) and what evidence in the data would help you decide which approach is more defensible.

3. Section 21.3.3 showed permutation importance and SHAP disagreeing on whether `PAY_1` or `max_delinquency` is more important. If a regulator asked your institution which single feature drives most of your credit model's decisions, how would you respond given this disagreement, and does the question itself presuppose something that may not be true of the model?

4. Section 21.3.4's cost-minimizing threshold depends entirely on the assumed \$500-to-\$50 cost ratio. Propose a concrete way an institution could estimate these two costs from its own historical data, and discuss whether the same cost ratio should apply uniformly to every applicant or could reasonably vary by loan size.

5. Section 21.3.5's adverse impact ratio (0.960) passes the four-fifths rule comfortably, yet the same model refit with `SEX`, `MARRIAGE`, and `AGE` as inputs scored 0.843, and Section 21.4.2's sex-by-age cut fails (0.713). Discuss what additional analysis, beyond recomputing these same ratios next quarter, an internal model risk function should perform to decide whether the passing result reflects a stable, defensible model, and how much of the remaining disparity it should attribute to proxies for the omitted attributes rather than to genuine differences in default risk.

6. Choose one of Section 21.4.4's capstone project ideas and walk through Table 21.8's eight stages for it explicitly, one sentence per stage naming the specific dataset column, model choice, or metric that stage would use, and flag any stage whose deliverable that project's dataset cannot actually support (for instance, a dataset with no protected-characteristic field at all, which would leave Stage 7 unable to produce a real fairness cut).

7. Section 21.4.4's robo-advisor and portfolio-risk-management projects are not classification problems and have no natural confusion matrix, ROC curve, or SHAP value. For either one, propose a concrete way to check whether the resulting system is behaving as intended, given that this chapter's usual evaluation toolkit does not directly apply, and identify which of Table 21.8's eight stages would need to be reinterpreted rather than dropped entirely.

8. Section 21.4.2's intersectional cut checked sex crossed with age. Propose a third characteristic from this dataset worth crossing with sex in a real fair-lending examination, justify your choice, and discuss what evidence would tell you whether it is worth checking before you actually run the numbers.

9. Section 21.4.3 names two failure modes: treating each template stage as an isolated checkbox, and stopping a fairness audit at the first cut that happens to pass. Describe a plausible way a well-intentioned project team could fall into each failure mode without realizing it, and propose one concrete process change (not a new statistical method) that would catch each one before a project is reported as finished.

21.6 Challenges in End-to-End Financial AI Projects

Several challenges recur across this chapter's pipeline and connect directly to themes from earlier chapters.

Real data is never clean by default. Section 21.2.1's undocumented category codes have no equivalent in any earlier chapter's fictional dataset, and a project that skips this step, rather than merely inheriting clean data, inherits whatever silent errors those undocumented codes introduce into every downstream model.

The best model by one metric is not always the best model by every metric. Table 21.2's class-weighted logistic regression illustrates this directly: best F_1, worst Brier score, among the four models

Wulin.Suo@Queensu.ca

compared, so "best model" is only a well-defined question once the deployment use case fixes which metric actually matters.

Different explanation methods can disagree, and both can be right. Section 21.3.3's permutation-importance-versus-SHAP disagreement on the top two features is not a bug in either method, but a direct consequence of the two methods answering genuinely different questions, the caution Chapter 18 raised in the abstract and this chapter now confirms empirically.

A threshold choice is a business decision with real financial consequences, not a modeling detail. Section 21.3.4's more than twofold cost difference between the naive and cost-optimal thresholds dwarfs the performance gap between any two models in this chapter's bake-off, a genuine, quantified instance of Chapter 15's more abstract alert-economics argument.

Removing a protected attribute is not the same as removing its effect. Leaving `SEX`, `MARRIAGE`, and `AGE` out of the model, as fair-lending law requires, raised Section 21.3.5's sex-only adverse impact ratio from 0.843 to 0.960 at almost no cost in accuracy, yet the model still treats the sexes measurably differently and, never having seen `AGE`, still approves young men at well under four-fifths of the rate of older men, which is why Section 21.4.1 treated ongoing outcome monitoring, not the choice of inputs alone, as the real governance obligation.

A single-attribute fairness audit can pass while an intersectional one fails. Section 21.4.2's sex-and-age cut found a clear four-fifths-rule violation (0.713) that Section 21.3.5's sex-only audit (0.960, passing) gave no hint of, meaning an institution that stops testing after the first, cleaner-looking result may never discover a disparity that genuinely exists for a subgroup it never explicitly checked.

Not every financial AI project is a classification problem, and explainability means something different for each. Section 21.4.4's robo-advisor project has no confusion matrix, no ROC curve, and no natural SHAP value at all, since it is an optimization problem, not a supervised-learning one, so "explaining" its output means tracing a set of portfolio weights back to a stated risk tolerance and a stated optimization objective, a genuinely different task from anything Section 21.3.3's methods were built for, and a reminder that this book's explainability toolkit, powerful as it is for classification and regression, is not a universal solvent for every kind of financial AI system.

An end-to-end pipeline is only as trustworthy as its reproducibility. Every number in this chapter came from a fixed random seed and a documented sequence of cleaning, feature engineering, and modeling choices, recorded in the companion notebook precisely so that Section 21.4.1's independent-validation obligation is actually satisfiable rather than merely asserted.

21.7 Key Takeaways

This chapter's dataset, the UCI/Kaggle Default of Credit Card Clients data, was the first in this book not built to illustrate anything, and working through it surfaced problems earlier chapters' clean, fictional datasets never could: undocumented category and status codes and a misnamed column to clean (Section 21.2.1), a real class imbalance whose remedy (class weighting) traded a better F_1 score for a visibly worse Brier score (Section 21.3.1), and a calibration gap in the plain logistic regression's safest-looking decile that a rank-based evaluation alone would never have surfaced (Section 21.3.2).

Running Chapter 18's explanation toolkit with real software, `shap`'s `TreeExplainer` and scikit-learn's permutation importance, on a genuinely nonlinear gradient-boosted model showed both the same completeness property Chapter 18 verified by hand for a linear model and a real disagreement between two legitimate importance rankings (Section 21.3.3).

A concrete, assumption-driven cost-sensitive threshold choice cut expected cost by more than half relative to the naive 0.5 default (Section 21.3.4), though only at an operating point far stricter than any real lender would accept, an artifact of the toy cost ratio that the analytic threshold $c_{FP}/(c_{FP} + c_{FN})$ makes transparent. The resulting model, built without `SEX`, `MARRIAGE`, or `AGE` as inputs, passed a sex-only fairness audit comfortably but still showed both a residual sex difference and the same approval-rate-versus-error-rate tension Chapter 18 demonstrated with synthetic data, making Section 21.4.1's point directly: leaving out a protected attribute is not fairness, and a single deployment-time check is not governance; ongoing monitoring is.

Section 21.4.2 pushed the pipeline one level deeper still, in two directions. A systematic hyperparameter

Wulin.Suo@Queensu.ca

search found no consistent gain at all over Section 21.3.1's single, reasonably chosen XGBoost configuration, a useful and unglamorous reminder that model-family choice and feature engineering matter far more than fine-tuning once a sensible default configuration is already in place. An intersectional cut by sex and age then uncovered a stark, four-fifths-rule-violating disparity for young male applicants that Section 21.3.5's own single-attribute audit, run first and reported as passing, gave no indication of whatsoever. That contrast is the sharpest illustration in this entire book of why a fairness audit's scope, and not merely its arithmetic, determines what it can find.

It is worth closing on how much of this book's twenty preceding chapters this one pipeline actually drew on, since that breadth is itself the point of a capstone chapter. Chapter 2's pandas and NumPy fundamentals ran every line of this chapter's code. Chapter 6's train/test discipline, its feature engineering, its bagging and boosting concepts, and its overfitting caution structured Sections 21.2.2 and 21.3.1 directly. Chapter 9's calibration caveat, stated once in passing there, became Section 21.3.2's central finding. Chapter 15's class-imbalance remedies and cost-sensitive threshold framing became Sections 21.3.1 and 21.3.4's empirical results. And Chapter 18's entire explainability and fairness toolkit, adapted from hand computation on a linear model to real software on a genuine ensemble, became Sections 21.3.3, 21.3.5, and 21.4.2.

A single real dataset, considerably messier and less cooperative than any this book constructed on purpose, was enough to put nearly every technical tool this book developed to genuine, simultaneous use. That is the case this closing chapter exists to make. None of these methods was ever meant to be practiced in isolation, and a real financial AI project rarely affords the luxury of using only one at a time.

Section 21.4.3 distilled that same pipeline into an explicit, eight-stage template: problem framing, data cleaning and feature engineering, model bake-off, calibration, explainability, threshold and cost analysis, fairness audit, and a governance write-up. Each stage names its own required deliverable, precisely so that a reader is never left guessing what "finished" means for a project like this one.

The chapter, and the book, close with Section 21.4.4's further capstone project ideas. Each is a real, freely available dataset, spanning fraud detection, consumer lending, algorithmic trading, financial sentiment, robo-advisory, derivatives hedging, and credit scoring at a different scale. They are meant for a reader now ready to run this same disciplined pipeline independently, on data this book did not hand-pick: clean, engineer, split, compare, calibrate, explain, price the threshold, and audit for fairness.

21.8 Suggested Exercises

1. Using Section 21.2.1's method, tabulate the default rate separately for accounts with an undocumented `EDUCATION` code (0, 5, or 6) before cleaning, and compare it to the default rate for documented codes 1 through 4, discussing whether folding the undocumented codes into category 4 seems reasonable given what you find.

2. Using Table 21.2, compute each model's precision-recall trade-off directly (precision divided by recall) and rank the four models by this ratio, then explain in your own words what a very high or very low ratio implies about a model's operating point.

3. Using Section 21.3.2's method, compute the calibration gap (mean predicted probability minus mean actual default rate) for every decile of the random forest's predictions, and state whether the random forest's calibration more closely resembles the plain logistic regression's or XGBoost's in Table 21.3.

4. Section 21.2.2 kept `AGE` out of the model. Refit the XGBoost model with `AGE` added back as a 26th feature, use Section 21.3.3's method to compute its mean absolute SHAP value and its rank among the 26 features, and rerun Table 21.7's audit. Discuss, in light of Chapter 18's discussion of proxy features correlated with protected characteristics, whether leaving `AGE` out actually removed age-related information from the model's decisions.

5. Using Section 21.3.4's method, recompute the cost-minimizing threshold if the cost of a missed default is revised from \$500 to \$1,000 (holding the \$50 false-positive cost fixed), and explain in words the direction you expect the optimal threshold to move before recomputing it.

Wulin.Suo@Queensu.ca

6. Using Section 21.3.5's method, recompute the adverse impact ratio and the false-negative-rate gap at the naive 0.5 threshold instead of the cost-minimizing 0.10 threshold, and discuss whether the choice of threshold changes which group appears more favored by each of the two statistics.

7. Using Section 21.4.4's description of the Kaggle credit card fraud dataset, explain why a 0.17% positive rate makes accuracy an even less useful headline metric than it was for this chapter's 22.1% default rate, referencing Chapter 15's original caution about accuracy under class imbalance.

8. Using Table 21.7's method, compute the adverse impact ratio for a third cut, sex crossed with `EDUCATION` (collapsed to graduate school versus all other categories), and state whether this cut reveals a four-fifths-rule violation as stark as the sex-and-age cut in the text.

9. Section 21.4.2 found no consistent improvement from hyperparameter tuning. Propose one plausible reason a much larger or higher-dimensional dataset (such as the IEEE-CIS fraud dataset in Section 21.4.4) might show a larger gain from the same kind of search, referencing the bias-variance discussion in Chapter 6.

10. Using Section 21.3.2's isotonic recalibration result, explain in your own words why the recalibrated logistic regression's ROC-AUC barely changed (0.720 to 0.724) even though its Brier score improved measurably, referencing the monotonicity property isotonic regression is constrained to satisfy.

11. Explain why fitting an isotonic recalibration function directly on the test set, rather than via cross-validation on the training set alone, would invalidate the resulting calibration check, referencing Section 21.2.2's train/test discipline.

21.9 Further Reading

- Yeh and Lien, "The Comparisons of Data Mining Techniques for the Predictive Accuracy of Probability of Default of Credit Card Clients," *Expert Systems with Applications*. The original paper introducing the dataset this chapter's case study is built on.
- UCI Machine Learning Repository, "Default of Credit Card Clients Data Set." The public hosting page for the dataset used throughout this chapter, including its full data dictionary.
- Chen and Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. The original paper describing the gradient-boosting implementation used in Section 21.3.1.
- Lundberg and Lee, "A Unified Approach to Interpreting Model Predictions," *Advances in Neural Information Processing Systems*. The SHAP paper underlying the `TreeExplainer` algorithm used in Section 21.3.3, previously cited in Chapter 18 for its linear-model application.
- Niculescu-Mizil and Caruana, "Predicting Good Probabilities with Supervised Learning," *Proceedings of the 22nd International Conference on Machine Learning*. A foundational study of which model families tend to produce well-calibrated probabilities, directly relevant to Section 21.3.2's findings.
- Provost and Fawcett, *Data Science for Business*. A practitioner-oriented treatment of the cost-sensitive threshold reasoning developed in Section 21.3.4.
- Dua and Graff (curators), "UCI Machine Learning Repository," University of California, Irvine. The broader repository this chapter's dataset is drawn from, and a starting point for several of Section 21.4.4's further project ideas.
- Buolamwini and Gebru, "Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification," *Proceedings of the Conference on Fairness, Accountability, and Transparency*. The landmark empirical demonstration that a system can pass single-attribute fairness checks while still showing a stark disparity for an intersectional subgroup, the same pattern Section 21.4.2 found directly in a lending context.

Wulin.Suo@Queensu.ca

Wulin.Suo@Queensu.ca

Appendix A

Datasets and APIs for Financial AI

A.1 Introduction

Every chapter in this book is built around a worked numerical example, and several of the later chapters, most notably Chapter 21's UCI credit-card default case study, are built around a real, publicly available dataset rather than a fictional one. This appendix collects, in one place, the data sources a reader is most likely to reach for when reproducing this book's examples with real data or extending them into an independent project: what each source covers, how it is typically licensed or priced, and the specific API or library used to pull it into Python.

This appendix is deliberately organized around *access mechanism* as much as content, since the practical difference between a dataset a reader can query in five minutes with a free API key and one that requires an institutional subscription is often the difference between a project that gets started and one that does not. Where this book's own companion notebooks already reach outside the book's fictional running examples for real data, chiefly Chapter 5's CBOE Volatility Index (VIX) history (Section A.2, downloaded live via the `yfinance` library) and Chapter 21's UCI "Default of Credit Card Clients" dataset (Section A.6), this appendix cross-references the relevant chapter directly.

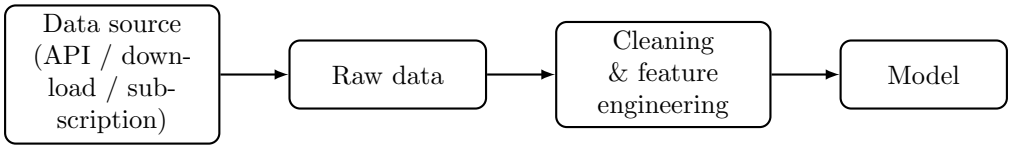


Figure A.1: Where the datasets and APIs in this appendix sit in a typical workflow: they supply the raw data, but the cleaning and feature-engineering steps introduced in Chapter 2's data-cleaning section and used throughout this book still have to be applied before the data reaches a model

A.2 Market Price and Trading Data

Table A.1 summarizes the most commonly used sources for equity, index, foreign exchange, and derivatives price data. Yahoo Finance, accessed through the `yfinance` Python library, is the source this book itself uses: Chapter 5's implied-volatility section downloads the actual daily history of the VIX this way, with no API key required. It is an unofficial wrapper around Yahoo Finance's public website rather than a documented, versioned API, which is precisely why it is free; the tradeoff, discussed further in Section A.13, is that it can change behavior or break without notice whenever Yahoo changes its own site.

Where CRSP and Compustat are US-centric, Refinitiv Datastream (a product of the London Stock Exchange Group since its 2021 acquisition of Refinitiv) is the closest institutional equivalent for international

Wulin.Suo@Queensu.ca

and cross-country work: decades of equity, index, bond, and FX price history across essentially every major market, not just the US, together with the Worldscope database of global company fundamentals bundled into the same platform. Like WRDS, it is licensed rather than freely available, typically through a university or employer subscription, and is accessed either through its legacy Excel add-in or through the Datastream Web Service (DSWS), a REST API with an official `DatastreamPy` Python client.

A.3 Options and Derivatives Data

Options data deserves its own treatment because it is structurally different from equity price data: a single underlying can have hundreds of simultaneously listed contracts across many strikes and expirations, and the quantity most useful for the work in Chapter 5 (pricing, the Greeks, implied volatility) and Chapter 8 (delta-normal and delta-gamma VaR for an options position) is not the raw quoted price but the implied volatility and Greeks computed from it, which not every source provides pre-computed.

OptionMetrics, accessed through WRDS as the IvyDB US database, is the main dataset academic options research is built on, occupying essentially the same role for options that CRSP occupies for equities (Section A.2): daily, exchange-sourced, precomputed implied volatilities and Greeks for every listed US equity and index option going back to 1996, standardized enough that a result computed on it is directly comparable across studies. It is not free, licensed only through an institutional WRDS subscription, which is precisely why Table A.2 also lists several lighter-weight alternatives, from a free but shallow current-chain snapshot to paid practitioner-oriented vendors, for a reader without WRDS access.

A.4 Fundamental and Financial Statement Data

The U.S. Securities and Exchange Commission's EDGAR system is the single most important free source of fundamental data for U.S. public companies, and it is directly relevant to Chapter 4's treatment of the income statement, balance sheet, and cash flow statement. Beyond the human-readable filings, the SEC exposes two machine-readable APIs at `data.sec.gov`: a full-text search API for locating specific language across filings (directly usable for Chapter 16's 10-K risk-factor classification example), and an XBRL "company facts" API that returns every structured financial figure, revenue, COGS, CFO, and so on, that a company has ever reported, tagged and time-stamped, for a given ticker or CIK (Central Index Key). Because these figures come directly from a company's own regulatory filings, they require no scraping or vendor licensing, only a descriptive `User-Agent` header identifying the requester, which the SEC requires of all automated EDGAR access.

A.5 Macroeconomic and Interest Rate Data

FRED, the Federal Reserve Bank of St. Louis's Federal Reserve Economic Data service, is the standard free source for the interest-rate and macroeconomic series this book uses conceptually in Chapter 5's term-structure discussion and Chapter 8's stress-testing scenarios. A free API key registered at `fred.stlouisfed.org` grants access to its REST API, and the community-maintained `fredapi` Python package wraps it in a single function call that returns a time-indexed `pandas` series, matching the data structures this book's notebooks already use throughout.

A.6 Credit, Lending, Insurance, and Consumer Finance Data

This is the category with the deepest overlap with this book's own worked examples. The UCI "Default of Credit Card Clients" dataset is already Chapter 21's own running case study, downloaded programmatically by that chapter's notebook rather than committed to this repository. The other datasets in Table A.5 are natural extensions for the capstone project ideas Chapter 21 closes with.

Wulin.Suo@Queensu.ca

Source	Access	API / library	Notes
Yahoo Finance	Free	`yfinance` (PyPI)	Equities, ETFs, indices, FX, crypto, options chains; used in Chapter 5 of this book (the VIX example), and a natural real-data upgrade for Chapter 2's synthetic AssetA/AssetB series; unofficial, can break without notice
Alpha Vantage	Free tier + paid	REST API, key required (`alphavantage.co`)	Equities, FX, crypto, and technical indicators; free tier is rate-limited to a handful of calls per minute
Nasdaq Data Link (formerly Quandl)	Free + paid tiers	REST API (`nasdaqdatalink.com`)	Broad marketplace of market, commodity, and economic datasets from many contributors
Polygon.io	Free limited tier + paid	REST and WebSocket API	Real-time and historical US equities, options, FX, and crypto, including tick-level data
Tiingo	Free tier + paid	REST and WebSocket API	End-of-day prices, fundamentals, and news
CBOE DataShop	Paid	Web portal, bulk download	Official source for VIX methodology data and listed options data
WRDS: CRSP, TAQ	Institutional subscription (typically via a university)	Web query tool, plus SAS/Python/R client	CRSP is the academic gold standard for US equity returns; TAQ provides tick-by-tick trade and quote data used in Chapter 14's microstructure examples
Refinitiv Datastream (LSEG)	Institutional subscription (typically via a university or employer)	Excel add-in, or the Datastream Web Service (DSWS) REST API via `DatastreamPy`	The standard source for international/cross-country equity, index, bond, and FX history, plus Worldscope global fundamentals in the same platform; fills the gap CRSP and Compustat leave outside the US

Table A.1: Market price and trading data sources

Wulin.Suo@Queensu.ca

Source	Access	API / library	Notes
Yahoo Finance	Free	`yfinance`'s `Ticker.option_chain()`	Current listed option chains (bid, ask, volume, open interest) for a given expiration; no deep history, and implied volatility is Yahoo's own estimate rather than a modeled surface
CBOE DataShop	Paid	Web portal, bulk download	Official historical end-of-day and intraday options quotes directly from the exchange that lists most US index and equity options
Polygon.io Options API	Free limited tier + paid	REST and WebSocket API	Historical and real-time options chains with server-computed Greeks and implied volatility
WRDS: OptionMetrics (IvyDB US)	Institutional subscription	Web query tool, plus SAS/Python/R client	The academic standard for US options research: daily implied volatility surfaces and Greeks, precomputed, going back to 1996; directly usable for Chapters 5 and 8
ORATS	Paid	REST API	Historical implied volatility surfaces, Greeks, and options analytics aimed at practitioners rather than academics
Deribit	Free, public endpoints	REST and WebSocket API, no key required for public data	The largest crypto options venue; full order book and historical settlement data for BTC and ETH options, useful for practicing the same Black-Scholes-Merton and Greeks machinery from Chapter 5 on an asset class with far higher realized volatility
CME DataMine	Paid	Bulk download / API	Historical futures and futures-options data (rates, commodities, equity index futures) directly from the exchange

Table A.2: Options and derivatives data sources

Wulin.Suo@Queensu.ca

Source	Access	API / library	Notes
SEC EDGAR	Free	REST API (`data.sec.gov`); full-text search and XBRL company-facts endpoints	Every US public company's own filings; ties directly to Chapters 4 and 16
Financial Modeling Prep	Free tier + paid	REST API	Pre-parsed income statements, balance sheets, cash flow statements, and ratios
WRDS: Compustat	Institutional subscription	Web query tool, plus SAS/Python/R client	Standardized, restated fundamentals used throughout academic finance research
Macrotrends, StockAnalysis.com	Free (web only)	No official API	Convenient for spot-checking a single company's numbers by hand; not suitable for programmatic bulk use

Table A.3: Fundamental and financial statement data sources

Source	Access	API / library	Notes
FRED	Free, key required	REST API (`fred.stlouisfed.org`); `fredapi` package	Treasury yields, GDP, inflation, unemployment; directly usable for Chapters 5 and 8
World Bank Open Data	Free	REST API (`worldbank.org`)	Global macro and development indicators across countries
IMF Data	Free	REST API	Global macro and financial-stability indicators
U.S. Bureau of Labor Statistics	Free, key optional	REST API (`bls.gov`)	US employment and price-inflation series

Table A.4: Macroeconomic and interest rate data sources

Wulin.Suo@Queensu.ca

Source	Access	API / library	Notes
UCI Default of Credit Card Clients	Free	Direct download via `urllib`	Chapter 21's own dataset; Taiwanese credit-card default data
UCI German Credit Data	Free	Direct download	Small, classic credit-scoring dataset, conceptually close to Chapter 6 and 9's examples
Lending Club loan data	Free (via Kaggle mirror)	Kaggle API	Loan-level peer-to-peer lending data; a Chapter 21 capstone idea
HMDA (Home Mortgage Disclosure Act) data	Free	REST API via the Consumer Financial Protection Bureau's data browser (`consumerfinance.gov`)	Mortgage application-level data widely used for fair-lending analysis; directly usable for Chapter 18's fairness audits
French Motor Third-Party Liability (`freMTPL2freq`)	Free	OpenML (`fetch_openml`) or the `CASdatasets` R package	The standard actuarial ratemaking benchmark: roughly 678,000 real motor policies with claim counts and rating features; Chapter 11's challenge exercise

Table A.5: Credit, lending, and consumer finance data sources

Wulin.Suo@Queensu.ca

A.7 Fraud, AML, and Regulatory Compliance Data

Genuinely labeled fraud data is scarce, since financial institutions rarely release real transaction-level fraud labels, which is why the two datasets in Table A.6 recur so often in both industry benchmarks and this book's own Chapter 15 and Chapter 21 capstone list. Operational-risk loss data proper is even scarcer: the industry's event-level loss databases (the ORX consortium foremost among them) are member-only, which is why Chapter 10's real-data challenge falls back on the public enforcement and complaint text sources in the same table, regulatory documents rich in the operational-loss and misconduct narratives that a bank's internal incident data contains.

A.8 Text, News, and Sentiment Data

Chapter 16's treatment of sentiment scoring rests on the Loughran-McDonald Master Dictionary, a word list purpose-built for financial text (ordinary sentiment dictionaries misclassify routine financial vocabulary, such as "liability" or "tax," as negative), freely downloadable from its maintainer's University of Notre Dame page. Table A.7 lists this and the other text sources most relevant to that chapter.

A.9 Alternative Data

Chapter 17's case vignette on satellite-derived retail and commodity signals (RS Metrics, UBS, and Orbital Insight's oil-storage monitoring) is representative of this entire category: the underlying vendor data is almost always enterprise-priced and licensed rather than available through a simple API key, since it is expensive to produce and its main customers are institutional investors, not students. Table A.8 separates the handful of alternative-data sources that are genuinely free or low-cost from the enterprise vendors that are not, so a reader does not spend time chasing a free tier that does not exist.

A.10 High-Frequency and Market Microstructure Data

Reconstructing an actual limit order book, the exercise Chapter 3 introduces by hand and Chapters 13 and 14 build trading strategies on top of, requires message-level data that most retail-facing APIs simply do not provide. LOBSTER, an academic data service built on top of NASDAQ's own historical ITCH data, is the most accessible source of genuine, reconstructed limit-order-book data: it offers a free academic tier (typically a handful of sample days for a limited set of tickers) alongside paid access to its full history.

A.11 Factor, Index, and Portfolio Data

No source is more central to Chapter 12's factor-model examples than the Ken French Data Library, maintained by Dartmouth's Tuck School of Business, which has published the Fama-French factor returns (market, size, value, and their later extensions) and a large set of industry-sorted portfolios as free, direct-download CSV and ZIP files for over two decades. It has no formal REST API, but its file layout and URLs have been stable for so long that the `pandas-datareader` package includes a dedicated reader for it.

A.12 Putting It Together: A Chapter-by-Chapter Dataset Map

Table A.11 condenses the sections above into a single reference: for each chapter of this book, which of the sources above is the most natural place to look for real data to extend that chapter's fictional examples. Chapter 21's own closing list of capstone project ideas already names several of these sources directly; this table extends that same mapping to every other chapter.

Wulin.Suo@Queensu.ca

Source	Access	API / library	Notes
IEEE-CIS Fraud Detection (Kaggle)	Free (Kaggle competition dataset)	Kaggle API	Real, anonymized e-commerce transaction fraud data; a Chapter 15/21 capstone idea
PaySim	Free	Kaggle API	Synthetic mobile-money transaction data modeled on a real provider's logs; well suited to practicing the structuring and layering detection introduced in Chapter 15
OFAC Sanctions Lists	Free	Bulk data files and a REST search API via the US Treasury	Sanctioned-party screening lists used in AML compliance workflows
CFPB Consumer Complaint Database	Free	REST API and CSV export via the Consumer Financial Protection Bureau (`consumerfinance.gov`)	Millions of consumer complaints about financial products, many with free-text narratives; the proxy Chapter 10's challenge solution uses for classifying operational-risk narratives by category
SEC litigation releases and administrative proceedings	Free	Bulk index files and full-text search via `sec.gov`	Enforcement-action texts rich in operational-loss and misconduct narratives; the other real-text source Chapter 10's challenge exercise suggests

Table A.6: Fraud, AML, and regulatory compliance data sources

Wulin.Suo@Queensu.ca

Source	Access	API / library	Notes
SEC EDGAR full-text search	Free	REST API (`data.sec.gov`)	Search across filing text; directly usable for Chapter 16's 10-K risk-factor classification example
Loughran-McDonald Master Dictionary	Free	Direct CSV/text download	The sentiment word list underlying Chapter 16's scoring examples
Financial PhraseBank	Free	Hugging Face `datasets` library, or direct download	Sentence-level, analyst-labeled financial news sentiment
GDELT Project	Free	REST API and BigQuery access	Global news event and tone database, updated continuously
Reddit API	Free, registered app required, rate-limited	`praw` (Python Reddit API Wrapper)	Social sentiment data, e.g., retail-investor forums
X (formerly Twitter) API	Paid tiers	REST and streaming API	Social sentiment; access has moved to paid tiers since 2023, unlike the largely free access assumed by older tutorials

Table A.7: Text, news, and sentiment data sources

Source	Access	API / library	Notes
Google Trends	Free, unofficial	`pytrends` (unofficial wrapper)	Search-interest series, often used as an attention or sentiment proxy
Orbital Insight, RS Metrics	Enterprise, paid	Vendor portal/API	Satellite-derived alternative data referenced in Chapter 17's case vignette; not accessible without a commercial contract
Similarweb, Sensor Tower	Paid	Vendor API	Web-traffic and app-usage panel data
MSCI ESG, Sustainalytics, Refinitiv ESG	Paid	Vendor API	ESG scores and underlying metrics

Table A.8: Alternative data sources

Wulin.Suo@Queensu.ca

Source	Access	API / library	Notes
LOBSTER	Free academic tier + paid	Web portal, bulk download (`lobsterdata.com`)	Reconstructed limit order book data for NASDAQ-listed stocks; directly usable for Chapters 3, 13, and 14
Databento	Pay-as-you-go	REST API, historical and live feeds	Modern, lower-cost alternative to legacy tick-data vendors for US equities, futures, and options
WRDS: NYSE TAQ	Institutional subscription	Web query tool, plus SAS/Python/R client	Tick-by-tick trade and quote data; the traditional academic source for Chapter 14-style microstructure research

Table A.9: High-frequency and market microstructure data sources

Source	Access	API / library	Notes
Ken French Data Library	Free	Direct CSV/ZIP download; also wrapped by `pandas-datareader`	Fama-French factor returns and industry portfolios; directly usable for Chapter 12
iShares, SPDR ETF holdings	Free	Daily CSV download from the issuer's own website	Actual ETF basket composition; usable for Chapter 14's ETF-basket arbitrage example
S&P Dow Jones Indices	Free (delayed) + paid (full)	Vendor portal	Index membership, methodology documents, and historical levels

Table A.10: Factor, index, and portfolio data sources

Wulin.Suo@Queensu.ca

Chapter	Most relevant data sources
1. Introduction	(conceptual; no dataset needed)
2. Python for Finance	Yahoo Finance / `yfinance`, for real practice alongside the chapter's synthetic AssetA/AssetB series
3. Financial Markets and Institutions	LOBSTER (limit order book reconstruction), Yahoo Finance
4. Financial Statements	SEC EDGAR (filings and XBRL company facts)
5. Valuation and Asset Pricing	FRED (yield curve), Yahoo Finance / `yfinance` (already used for the VIX in Section A.2), OptionMetrics or Deribit for real options chains (Section A.3)
6. Machine Learning Fundamentals	UCI German Credit Data, or any dataset above used generically for classification practice
7. Time Series Analysis and Forecasting	FRED (macro series), Yahoo Finance (price series)
8. Market Risk Management	Yahoo Finance and FRED, for a real-portfolio VaR backtest; OptionMetrics or Polygon.io for a real delta-normal/delta-gamma VaR options example (Section A.3)
9. Credit Risk Modeling	UCI Default of Credit Card Clients, Lending Club loan data, HMDA data
10. Operational Risk Modeling	CFPB Consumer Complaint Database, SEC litigation releases and administrative proceedings
11. Insurance and Actuarial Modeling	French Motor Third-Party Liability (`freMTPL2freq`, via OpenML or `CASdatasets`)
12. Portfolio Theory and Asset Allocation	Ken French Data Library, Yahoo Finance
13. Algorithmic Trading and Market Microstructure	LOBSTER, Yahoo Finance intraday data (where available)
14. High-Frequency Trading	LOBSTER, Databento, WRDS NYSE TAQ
15. Fraud Detection, Compliance, and Financial Crime	IEEE-CIS Fraud Detection, PaySim, OFAC sanctions lists
16. Natural Language Processing in Finance	SEC EDGAR full-text search, Loughran-McDonald dictionary, Financial PhraseBank, GDELT, Reddit API
17. Alternative Data and Deep Learning in Finance	Google Trends, satellite/alt-data vendors (enterprise), Similarweb
18. Explainability, Fairness, and Regulation	HMDA data, UCI Default of Credit Card Clients (as in Chapter 21's own capstone fairness audit)
19. Reinforcement Learning in Finance	Simulated market environments by design; LOBSTER or Yahoo Finance intraday data for a real extension of the chapter's execution and market-making agents
20. Agent-Based Models in Finance	Simulated agent populations by design; LOBSTER order-book data for a real zero-intelligence-trader comparison, or BIS locational banking statistics for a real interbank exposure network
21. Case Studies and Capstone Projects	The full list above; see Chapter 21's own closing section for specific project pairings

Table A.11: A chapter-by-chapter map from this book's topics to the datasets and APIs most relevant to them

Wulin.Suo@Queensu.ca

A.13 Practical Challenges in Sourcing Financial Data

Pulling real data into a project is rarely as simple as calling an API and moving on; several recurring problems are worth flagging explicitly.

Point-in-time and look-ahead bias. Most free APIs return a security's or company's data as it is known *today*, not as it was actually known on the date being studied. Fundamentals get restated after the fact, index membership changes, and a company's own name or ticker can change; naively joining "current" fundamentals to historical prices silently leaks future information into a backtest, the look-ahead-bias problem Chapter 17 raises directly.

Survivorship bias. A free API that only lists currently-traded tickers implicitly excludes every company that has since been delisted, gone bankrupt, or been acquired, which biases any historical universe built from it toward survivors. CRSP's paid, point-in-time data is the academic standard specifically because it avoids this.

Licensing and redistribution restrictions. Most vendor terms of service prohibit redistributing raw data even for non-commercial or educational use, which is why this book's own real-dataset chapters (Chapter 21, and this appendix's own Chapter 5 VIX example) download data programmatically at run time rather than committing it to the repository. The same discipline is worth following in any derivative project.

API and vendor instability. Unofficial APIs such as `yfinance` work by interacting with a public website rather than a documented, contractually stable interface, so they can change behavior or stop working without notice whenever the underlying site changes. Even official free tiers are periodically discontinued or replaced outright, as has happened repeatedly across the market-data-vendor landscape. Every specific vendor and endpoint named in this appendix should therefore be treated as illustrative rather than permanent: verify current availability and terms directly with the provider before building anything on top of one.

Rate limits and cost tiers. Free tiers are typically rate-limited (a fixed number of calls per minute or per day), which matters when a project needs to pull data for hundreds of tickers rather than one; budgeting for a paid tier, or batching and caching requests locally, is often unavoidable at scale.

A.14 Further Reading

- U.S. Securities and Exchange Commission, *EDGAR Application Programming Interfaces*, `sec.gov`.
- Federal Reserve Bank of St. Louis, *FRED API Documentation*, `fred.stlouisfed.org`.
- Kenneth R. French, *Data Library*, Tuck School of Business, Dartmouth College, `mba.tuck.dartmouth.edu`.
- LOBSTER, *Limit Order Book Data Documentation*, `lobsterdata.com`.
- Kaggle, *Datasets and Public API Documentation*, `kaggle.com`.
- UCI Machine Learning Repository, `archive.ics.uci.edu`.
- Loughran, T. and McDonald, B., "When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks," *Journal of Finance*, 2011 (the paper underlying the Loughran-McDonald Master Dictionary).

Wulin.Suo@Queensu.ca

Appendix B

Glossary of Finance Terms

This glossary is a quick-reference point, not a substitute for the chapter that actually develops a term. Each entry gives a one- or two-sentence working definition and points to the chapter (or chapters) where the concept is developed in full, usually with a worked numerical example; use the glossary to get unstuck on an unfamiliar term encountered before its formal introduction, then follow the pointer for the real treatment. Entries are alphabetized for lookup, since a reader hitting an unfamiliar term mid-chapter needs to find it quickly, not read the glossary front to back. This glossary deliberately does not redefine standard statistics and machine learning vocabulary (regression, neural network, gradient descent, and the like) that this book's intended reader already knows; it covers finance- and trading-specific jargon, regulatory terms, and a handful of quantitative-finance terms of art (e.g., "backtest," "look-ahead bias") that have a specific meaning in this context beyond their everyday use.

Adverse selection The risk that a market maker's counterparty knows something the market maker does not, so that trading against better-informed counterparties is systematically unprofitable; one of the three components Chapter 3 decomposes the bid-ask spread into.

Alpha The portion of an investment's return that is not explained by its exposure to broad market risk (beta); loosely, "skill" or "edge" rather than just being exposed to a rising market. Chapter 12 defines alpha formally as the intercept in a factor-model regression.

Alternative data Non-traditional data sources, such as satellite imagery, web traffic, or shipping data, used as an early proxy for economic activity that has not yet appeared in official statistics or financial statements. See Chapter 17.

AML (Anti-Money Laundering) The set of laws, regulations, and institutional controls designed to detect and prevent the disguising of criminal proceeds as legitimate funds. See Chapter 15.

Backtest Testing a trading strategy or model against historical data to estimate how it would have performed, a central tool of Chapter 13 that is also a common source of overly optimistic, overfit results if not done carefully.

Backtest overfitting The distortion that arises when a strategy is selected as the best of many configurations tested on the same history, so that its reported performance reflects the maximum of many noisy estimates rather than a genuine edge. Chapter 13 shows that searching a hundred worthless strategies over five years of daily data yields an expected best Sharpe ratio of about 1.12. The deflated Sharpe ratio corrects for it.

Balance sheet A financial statement showing what a company owns (assets), what it owes (liabilities), and the residual claim of its owners (equity) at a single point in time. See Chapter 4.

Basel Accords A series of international banking regulatory agreements (Basel I, II, III) setting minimum capital requirements and risk-management standards for banks, spanning market, credit, and operational risk. See Chapters 8, 9, and 10.

Wulin.Suo@Queensu.ca

Basis point (bp) One hundredth of one percentage point ($0.01\% = 0.0001$); the standard unit for quoting small differences in interest rates, spreads, and fees. See Chapter 3.

Beta A measure of how sensitive an asset's returns are to the overall market's returns; a beta of 1.5 means the asset tends to move 1.5% for every 1% move in the market. First used informally in Chapter 5, then estimated directly from data in Chapter 12.

Bid-ask spread The gap between the highest price a buyer is currently willing to pay (the bid) and the lowest price a seller is currently willing to accept (the ask); a direct, observable cost of trading and a proxy for an asset's liquidity. See Chapter 3.

Black-Scholes-Merton model A closed-form formula for pricing European call and put options as a function of the underlying's price, the strike price, time to maturity, volatility, and the risk-free rate, derived independently by Black and Scholes and by Merton in 1973; the model whose Greeks (delta, gamma, vega, theta) are the standard language for describing an option position's risk. See Chapter 5.

Bond A debt security in which the issuer (a company or government) promises to make periodic coupon payments and repay the bond's face value at maturity, in exchange for the money the bondholder lent upfront. See Chapter 5.

BSA (Bank Secrecy Act) The primary U.S. law requiring banks to maintain records and file reports (CTRs and SARs) that assist in detecting money laundering. See Chapter 15.

Call option A contract giving its holder the right, but not the obligation, to buy the underlying asset at a fixed strike price on or before a specified date; the holder profits if the underlying's price rises above the strike. See Chapter 5.

CAPM (Capital Asset Pricing Model) A model relating an asset's expected return to its beta and the market's expected excess return; the foundational single-factor model that later multi-factor models extend. See Chapter 12.

Cash flow statement A financial statement reconciling a company's cash balance from one period to the next, broken into operating, investing, and financing activities. See Chapter 4.

Catastrophe bond A bond whose investors lose principal if a specified catastrophe (a hurricane or earthquake meeting defined criteria) occurs, the proceeds going to the issuing insurer or reinsurer; a way of passing modeled tail risk directly to capital markets, paying a spread largely uncorrelated with ordinary market risk. See Chapter 11.

CDD / EDD (Customer / Enhanced Due Diligence) CDD is the baseline process of verifying a customer's identity and establishing their expected activity profile at onboarding; EDD applies heightened scrutiny to higher-risk customers, such as politically exposed persons. See Chapter 15.

CDS (Credit Default Swap) A derivative contract in which the protection buyer pays a periodic premium in exchange for a payout if a specified borrower defaults; effectively insurance against default. See Chapter 9.

CECL (Current Expected Credit Losses) The U.S. accounting standard requiring a lender to provision, from the day a loan is originated, against the credit losses expected over its entire remaining life rather than only the next twelve months. Its international counterpart is IFRS 9. See Chapter 9.

Chain ladder The classical actuarial method for estimating loss reserves: arrange cumulative claims by accident year and development age, estimate average development factors from completed history, and project each open year to its ultimate cost. See Chapter 11.

Circuit breaker An exchange rule that automatically halts trading in a security (or an entire market) after a sufficiently large price move, intended to give participants time to reassess rather than trade on a possible error or panic. See Chapter 14.

Wulin.Suo@Queensu.ca

Clearinghouse An intermediary that stands between the two counterparties to a trade after execution, guaranteeing settlement and reducing counterparty risk. See Chapter 3.

COGS (Cost of Goods Sold) The direct costs attributable to producing the goods or services a company sells, subtracted from revenue to compute gross profit. See Chapter 4.

Collateral Assets a borrower pledges to a lender to secure a loan, which the lender may seize if the borrower defaults; the basis for how loss given default depends on how well an exposure is secured. See Chapters 3 and 9.

Co-location Renting rack space physically inside (or adjacent to) an exchange's own data center specifically to minimize the network latency of sending and receiving orders. See Chapter 14.

Cointegration A long-run statistical relationship between two or more non-stationary time series whose particular linear combination is itself stationary; the basis for pairs-trading strategies. See Chapters 7 and 13.

Combined ratio An insurer's losses plus expenses divided by its premiums; below 100% the underwriting operation is profitable in its own right, and even slightly above 100% an insurer may profit overall from investment income on float. See Chapter 11.

Counterparty The other party to a financial transaction or contract; counterparty risk is the risk that this other party fails to honor its obligations. See Chapters 3 and 9.

Coupon The periodic interest payment a bond issuer promises to pay its bondholders, usually expressed as an annual percentage of the bond's face value. See Chapter 5.

Credibility theory The actuarial formula for blending an individual risk's own claims experience with its class average, weighting by $Z = n/(n + k)$; Bayesian shrinkage derived by actuaries decades before machine learning rederived it as regularization. See Chapter 11.

Credit rating A letter grade (e.g., AAA, BBB, CCC) assigned by a rating agency summarizing an issuer's or bond's creditworthiness. See Chapter 9.

Credit spread The extra yield a risky bond pays over a comparable risk-free bond, compensating the lender for default risk; wider spreads mean the market perceives more credit risk. See Chapters 6 and 9.

CTR (Currency Transaction Report) A report a U.S. financial institution must file for any cash transaction exceeding $10,000, regardless of whether the transaction looks suspicious. See Chapter 15.

Dark pool A private trading venue that does not publicly display its order book before a trade executes, used mainly by large institutional traders seeking to limit the market impact of a large order. See Chapter 3.

Default A borrower's failure to make a required payment on a debt obligation as promised; the event whose probability (PD), severity (LGD), and exposure (EAD) together determine expected credit loss. See Chapter 9.

Derivative A financial contract whose value is derived from the price of some other, underlying asset (a stock, a bond, a rate, a commodity), rather than having independent value of its own; options, futures, and CDS are all derivatives covered in this book. See Chapters 3 and 5.

Disparate impact A legal theory under which a facially neutral policy or model is nonetheless unlawful if it produces a substantially worse outcome for a protected group, regardless of intent; the four-fifths rule is the most common numerical test for it. See Chapter 18.

Diversification Combining assets whose returns are not perfectly correlated so that a portfolio's overall risk is less than the weighted average of its individual assets' risks. See Chapters 2 and 12.

Wulin.Suo@Queensu.ca

Dividend A cash (or stock) payment a company distributes to its shareholders, typically out of earnings. See Chapter 3.

Dividend Discount Model (DDM) A valuation method that prices a stock as the present value of all its expected future dividend payments. See Chapter 5.

Drawdown The decline in a portfolio's value from its most recent peak to a subsequent trough, usually expressed as a percentage; maximum drawdown is a common risk metric because it captures the pain of a losing streak in a way volatility alone does not. See Chapter 13.

Duration A bond's approximate percentage price sensitivity to a small change in interest rates; a duration of 5 means the bond's price falls by roughly 5% for every 1 percentage point rise in yields. See Chapter 5.

EAD (Exposure at Default) The estimated amount a lender is exposed to at the moment a borrower defaults, one of the three components (with PD and LGD) of expected loss. See Chapter 9.

EBITDA Earnings Before Interest, Taxes, Depreciation, and Amortization; a rough proxy for a company's operating cash-generating ability that strips out financing decisions, tax jurisdiction effects, and non-cash accounting charges. See Chapter 4.

ECOA (Equal Credit Opportunity Act) The U.S. law prohibiting credit discrimination on the basis of race, sex, and other protected characteristics, and requiring lenders to provide specific reasons when denying credit. See Chapter 18.

Efficient frontier The set of portfolios that offer the highest expected return for each level of risk (or, equivalently, the lowest risk for each level of expected return). See Chapter 12.

ESG (Environmental, Social, and Governance) A set of non-financial criteria used to evaluate a company's or portfolio's exposure to environmental, social, and governance-related risks and practices. See Chapter 12.

Exchange A regulated, centralized venue where buyers and sellers meet to trade standardized securities under a common set of rules. See Chapter 3.

Expected Loss The average loss a lender expects on a credit exposure, computed as the product of probability of default, loss given default, and exposure at default. See Chapter 9.

Expected Shortfall (ES) The average loss in the worst scenarios beyond the VaR threshold; unlike VaR, it answers "how bad, on average, is a bad day" rather than just "how bad is a day that is bad enough." Also called Conditional VaR (CVaR). See Chapter 8.

Face value (par value) The amount a bond's issuer promises to repay the bondholder at maturity, used as the basis for computing coupon payments. See Chapter 5.

Factor model A model explaining an asset's returns as a linear combination of exposures to a small number of common risk factors (market, size, value, momentum, and so on), extending CAPM's single-factor idea. See Chapter 12.

FATF (Financial Action Task Force) An intergovernmental body that sets international standards for combating money laundering and terrorist financing. See Chapter 15.

Fat-finger error A large, unintended order caused by human data-entry error (an extra zero, a misplaced decimal) rather than a deliberate trading decision; exchanges use price collars specifically to limit the damage such an error can cause. See Chapter 14.

Four-fifths rule A common (though not universally binding) threshold in U.S. discrimination law: if a protected group's selection or approval rate is less than 80% of the most-favored group's rate, the practice is presumed to have a disparate impact. See Chapter 18.

Wulin.Suo@Queensu.ca

Free Cash Flow (FCF) The cash a company generates after covering the capital expenditures needed to sustain its operations; the actual cash available to pay down debt, pay dividends, or reinvest. See Chapters 4 and 5.

FRTB (Fundamental Review of the Trading Book) A Basel Committee framework overhauling how banks must measure and hold capital against market risk, developed in response to weaknesses the 2008 financial crisis exposed in earlier VaR-based approaches. See Chapter 8.

Futures contract A standardized, exchange-traded agreement to buy or sell an asset at a predetermined price on a specified future date, marked to market daily. See Chapter 3.

GAAP (Generally Accepted Accounting Principles) The accounting standards used for financial reporting in the United States, the counterpart to the International Financial Reporting Standards used across much of the rest of the world. Differences between the two are one reason ratios pooled across firms in different jurisdictions may not be comparable. See Chapter 4.

GARCH Generalized Autoregressive Conditional Heteroskedasticity; a time-series model in which today's volatility depends on past volatility and past shocks, capturing the empirical fact that volatile periods tend to cluster together. See Chapter 7.

Greeks The sensitivities of an option's price to various underlying factors: delta (price), gamma (delta's own sensitivity to price), vega (volatility), and theta (time). See Chapter 5.

Gross margin Gross profit (revenue minus cost of goods sold) divided by revenue, expressed as a percentage. See Chapter 4.

Hedge A position taken specifically to offset the risk of an existing position, reducing (rather than eliminating) exposure to an adverse price move. See Chapters 5 and 8.

IFRS 9 The international accounting standard governing provisions for credit losses, which sorts every exposure into one of three stages and requires 12-month expected credit loss for performing exposures but *lifetime* expected loss once credit risk has increased significantly since origination. See Chapter 9.

Implied volatility The volatility level that, when plugged into an option-pricing model, reproduces the option's actual observed market price; a market-implied forecast of future volatility rather than a measure of past volatility. See Chapter 5.

Integration The final stage of money laundering, in which layered funds are reintroduced into the legitimate economy (e.g., through a real estate purchase) and become largely indistinguishable from legitimately earned money. See Chapter 15.

Inventory risk The risk a market maker bears that its current net position (long or short) moves against it before an offsetting trade arrives; one of the three components of the bid-ask spread and the central concern of inventory-based market-making models. See Chapters 3 and 13.

IPO (Initial Public Offering) A company's first sale of shares to the public, converting it from privately to publicly held. See Chapter 3.

Key risk indicator (KRI) A measurable quantity, such as staff overtime hours, system downtime, or employee turnover, tracked because it plausibly signals elevated operational-loss risk before any loss occurs; the input to predictive early-warning models of operational risk. See Chapter 10.

KYC (Know Your Customer) The due-diligence process of verifying a customer's identity and understanding their expected financial activity, the foundation of a bank's anti-money-laundering program. See Chapter 15.

Kyle's lambda A measure of price impact: the price change per unit of net order flow, estimated by regressing price changes on signed trading volume. See Chapters 13 and 14.

Wulin.Suo@Queensu.ca

Latency arbitrage A strategy that profits from being faster than other participants at reacting to new information or price changes, rather than from superior forecasting. See Chapter 14.

Layering The second stage of money laundering, in which placed funds are moved through a chain of transactions and accounts, often across institutions and jurisdictions, specifically to obscure their criminal origin. See Chapter 15.

Leverage The use of borrowed money to increase the potential return (and risk) of an investment or a company's balance sheet. See Chapters 4 and 8.

LGD (Loss Given Default) The fraction of an exposure a lender expects to lose if a borrower defaults, after accounting for any recovery (e.g., from collateral). See Chapter 9.

Limit order An order to buy or sell at a specified price or better, which rests in the order book until it either executes against an incoming order or is cancelled. See Chapter 3.

Liquidity How easily an asset can be bought or sold close to its current price without moving that price; a liquid market has narrow spreads and deep order books. See Chapter 3.

Long position Owning an asset outright, profiting if its price rises; the ordinary way of holding an investment, as opposed to a short position. See Chapter 3.

Look-ahead bias A backtesting error in which a strategy or model is (often inadvertently) given access to information that would not actually have been available at the time of the simulated decision, producing an unrealistically good result. See Chapter 17.

Loss distribution approach The classical actuarial method for quantifying operational risk: build the distribution of a year's total operational losses from a separately estimated frequency distribution (how many loss events occur) and severity distribution (how large each one is), typically by Monte Carlo simulation, then read capital off a far-tail quantile. See Chapter 10.

Margin Collateral a trader must post to open or maintain a leveraged position, protecting the counterparty (or exchange) against the trader's potential losses. See Chapter 3.

Margin call A broker's demand for additional collateral when a leveraged position's losses erode the trader's existing margin below a required maintenance level. See Chapter 3.

Market impact The extent to which placing a trade itself moves the price against the trader, distinct from the price movement that would have happened anyway. See Chapter 13.

Market maker A firm or algorithm that continuously quotes both a bid and an ask, profiting from the spread while bearing inventory risk. See Chapters 3 and 13.

Market order An order to buy or sell immediately at the best currently available price, trading certainty of execution for uncertainty of price. See Chapter 3.

Maturity The date on which a bond's principal is repaid, or an option or futures contract expires; time to maturity is a key input to bond pricing, duration, and option valuation. See Chapter 5.

Mean reversion The tendency of a price or spread to drift back toward its historical average after departing from it; the statistical basis for pairs trading and related strategies. See Chapters 7 and 13.

Merton model A structural credit-risk model treating a firm's equity as a call option on its assets, with default occurring if asset value falls below the face value of debt at maturity. See Chapter 9.

Model risk The risk that a model is wrong, misused, or misunderstood, and that a decision relying on it is therefore wrong even though the model appeared to function correctly; regulators treat it as an operational-risk discipline in its own right. See Chapters 8 and 10.

Wulin.Suo@Queensu.ca

NBBO (National Best Bid and Offer) The highest bid and the lowest offer available across all US equity exchanges, consolidated and published by the Securities Information Processor; Regulation NMS requires brokers to respect it when routing and executing orders. See Chapter 14.

Net Present Value (NPV) The present value of an investment's expected future cash flows minus its upfront cost; a positive NPV indicates the investment is expected to create value. See Chapter 5.

No-arbitrage The principle that two portfolios with identical future payoffs must have the same price today, since otherwise a riskless profit could be locked in; the foundation of most derivatives-pricing theory. See Chapters 3 and 5.

Notional (value) The total value a derivative contract controls or references, as distinct from the (usually much smaller) amount of capital actually posted to enter the position. See Chapter 5.

Operational risk The risk of loss resulting from inadequate or failed internal processes, people, and systems, or from external events, a rogue trader, a systems outage, a mis-sold product, as distinct from the market risk of price moves and the credit risk of counterparty default. See Chapter 10.

Option A derivative contract giving its holder the right, but not the obligation, to buy or sell an underlying asset at a fixed strike price by a specified date, in exchange for an upfront premium; see *call option* and *put option* for the two basic types. See Chapter 5.

Order book The list of all currently resting limit orders for a given security, organized by price level, showing the depth of buying and selling interest at each price. See Chapter 3.

Order flow The stream of incoming buy and sell orders a market or market maker observes; signed order flow (buys minus sells) is the input to price-impact measures such as Kyle's lambda. See Chapters 13 and 14.

OTC (Over-the-Counter) Trading conducted directly between two counterparties rather than on a centralized, regulated exchange. See Chapter 3.

Pairs trading A strategy that takes offsetting long and short positions in two historically related securities, betting that a temporary divergence between them will revert to its historical relationship. See Chapter 13.

PD (Probability of Default) The estimated likelihood that a borrower will fail to meet its debt obligations over a given time horizon. See Chapter 9.

PIN (Probability of Informed Trading) A market microstructure measure of the fraction of order flow estimated to come from traders with private information, as opposed to uninformed liquidity traders. See Chapter 14.

Placement The first stage of money laundering: introducing illicit cash into the financial system, for example through structured deposits. See Chapter 15.

Portfolio A collection of financial assets (stocks, bonds, cash, derivatives, or any combination) held together by an investor; this book's running two- and three-asset portfolios are the simplest possible example. See Chapters 2 and 12.

Present Value (PV) The value today of a future cash flow (or stream of cash flows), computed by discounting it back at an appropriate rate. See Chapter 5.

Prime broker A bank or securities firm providing a bundle of services (financing, securities lending, clearing) to institutional clients such as hedge funds. See Chapter 3.

Primary market The market in which securities are first sold by their issuer to investors (e.g., an IPO), as opposed to a secondary market where existing securities change hands between investors. See Chapter 3.

Wulin.Suo@Queensu.ca

Pure premium The expected loss cost of an insurance policy, expected claim frequency times expected claim severity, before loading for expenses and profit; the risk-based core of an insurance price. See Chapter 11.

Put option A contract giving its holder the right, but not the obligation, to sell the underlying asset at a fixed strike price on or before a specified date; the holder profits if the underlying's price falls below the strike. See Chapter 5.

Recovery rate The fraction of a defaulted exposure a lender ultimately recovers, typically through collateral or bankruptcy proceedings; loss given default is simply one minus the recovery rate. See Chapter 9.

Reinsurance Insurance purchased by insurers: contracts (often written in layers above an attachment point) that transfer part of an insurer's losses, especially correlated catastrophe losses that defeat ordinary pooling, to a reinsurer. See Chapter 11.

Repo (Repurchase Agreement) A short-term, collateralized loan structured as a sale of a security with an agreement to repurchase it later at a slightly higher price, the effective interest being the difference. See Chapter 3.

Retained earnings The cumulative portion of a company's net income that has been kept (not paid out as dividends) since the company's founding. See Chapter 4.

Risk-free rate The theoretical return on an investment with zero risk of default, typically proxied in practice by a short-term government bond yield; the baseline every risk premium (equity risk premium, credit spread) is measured against. See Chapters 5 and 12.

ROA (Return on Assets) Net income divided by total assets, measuring how efficiently a company generates profit from everything it owns. See Chapter 4.

ROE (Return on Equity) Net income divided by shareholders' equity, measuring the return generated on shareholders' invested capital. See Chapter 4.

ROIC (Return on Invested Capital) After-tax operating profit divided by the total capital (debt plus equity) invested in the business, often compared against WACC to judge whether a company is creating or destroying economic value. See Chapter 4.

Robo-advisor An automated investment-advisory service that builds and manages a portfolio for a client based on a short risk questionnaire, typically using mean-variance or similar optimization rather than a human advisor's judgment. See Chapter 12.

Rogue trading Unauthorized trading by an employee who conceals positions or losses from their institution's own controls, historically the source of several multi-billion-dollar operational losses (Barings in 1995, Société Générale in 2008, UBS in 2011) and the canonical example of internal fraud among the Basel operational-risk event types. See Chapter 10.

SAR (Suspicious Activity Report) A report a financial institution must file for any transaction pattern, regardless of size, that appears designed to evade reporting requirements or otherwise suggests criminal activity. See Chapter 15.

Sanctions screening Checking counterparties against government-maintained lists of prohibited individuals, entities, and countries; a binary legal-compliance check rather than a statistical risk model. See Chapter 15.

Scenario analysis Structured expert judgment estimating the frequency and severity of severe hypothetical events that have not occurred but plausibly could; in operational risk, the one data element able to speak to genuinely unprecedented tail events, and in market risk, a companion to statistical stress testing. See Chapters 8 and 10.

Wulin.Suo@Queensu.ca

Secondary market The market in which investors trade securities among themselves after their original issuance; most day-to-day trading (e.g., on a stock exchange) happens here. See Chapter 3.

Sharpe ratio A portfolio's average excess return (over the risk-free rate) divided by the standard deviation of that return; the standard measure of return earned per unit of risk taken. See Chapters 12 and 13.

Shell company An entity with no significant independent operations or assets, often used to obscure the true ownership or purpose of funds flowing through it. See Chapter 15.

Short position (short selling) Selling a borrowed asset now with the obligation to return an equivalent asset later, a bet that the price will fall. See Chapter 3.

SIP (Securities Information Processor) The consolidated feed that aggregates quotes from every US equity exchange into the single published NBBO; because it must collect and synchronize those quotes first, it is structurally slower than any individual exchange's own direct feed, which is what makes latency arbitrage against it possible. See Chapter 14.

Slippage The difference between the price a trader expected when deciding to trade and the price actually achieved once the order executes. See Chapter 13.

Stationarity A time series property in which the statistical characteristics (mean, variance, autocorrelation) do not change over time; many forecasting models require stationarity, or a transformation (such as differencing) to achieve it. See Chapter 7.

Stock (equity) A security representing fractional ownership of a company, entitling the holder to a proportional share of its residual value and (usually) a vote in corporate matters; also called equity, as opposed to debt. See Chapter 3.

Stress testing Evaluating how a portfolio, model, or institution would perform under an extreme but plausible adverse scenario, rather than relying solely on statistical measures estimated from typical historical data. See Chapter 8.

Strike price The fixed price at which an option holder may buy (call) or sell (put) the underlying asset. See Chapter 5.

Structuring (smurfing) Deliberately breaking a large cash transaction into several smaller ones, each below a regulatory reporting threshold, specifically to avoid triggering a report. See Chapter 15.

Survivorship bias The distortion introduced by analyzing only entities (companies, funds, strategies) that still exist today, silently excluding the failures that were removed along the way. See Chapters 1 and 13.

Systematic risk (and idiosyncratic risk) Systematic risk is the portion of an asset's risk driven by broad market-wide factors and cannot be diversified away; idiosyncratic risk is asset-specific and shrinks toward zero as a portfolio is diversified. This distinction is what CAPM's beta measures and diversification exploits. See Chapter 12.

10-K filing A U.S. public company's annual report filed with the SEC, containing audited financial statements alongside extensive qualitative disclosure, including a risk-factors section; a primary real-world source of the text financial NLP methods are applied to. See Chapter 16.

Telematics Driving-behavior data (mileage, speed, time of day, harsh braking) collected by a device or phone app and used to price usage-based auto insurance on observed behavior rather than demographic proxies. See Chapter 11.

Terminal value The estimated value of all cash flows beyond an explicit forecast horizon, typically the largest single component of a discounted-cash-flow valuation. See Chapter 5.

Wulin.Suo@Queensu.ca

Three lines of defense A governance framework dividing risk-management responsibility across business units (first line), independent risk and compliance functions (second line), and internal audit (third line). See Chapters 8 and 15.

Tick size The minimum allowed price increment a security can move by on an exchange. See Chapter 14.

Transition risk The financial impact on a company or portfolio of the policy, technology, and consumer-preference shifts associated with a transition to a lower-carbon economy, as distinct from physical climate risk. See Chapter 8.

TWAP / VWAP Time-Weighted and Volume-Weighted Average Price; two common execution benchmarks (and the algorithms designed to track them) that split a large order into smaller pieces over time, either evenly (TWAP) or in proportion to expected trading volume (VWAP). See Chapter 13.

Underlying (asset) The asset a derivative contract's value is based on, e.g., the stock an option gives the right to buy or sell. See Chapters 3 and 5.

Underwriting The process of evaluating and pricing the risk of extending credit or issuing insurance, historically performed by a human underwriter and increasingly informed by statistical scoring models. See Chapters 3, 9, and 11.

Unit root A property of a non-stationary time series in which shocks have a permanent rather than temporary effect; tested for directly (e.g., with the Augmented Dickey-Fuller test) before fitting many time-series models. See Chapter 7.

VaR (Value at Risk) A single dollar (or percentage) figure summarizing the loss a portfolio is not expected to exceed over a given time horizon at a given confidence level. See Chapter 8.

Volatility A statistical measure (typically the standard deviation of returns) of how much an asset's price fluctuates; higher volatility means larger, more frequent price swings. See Chapters 2 and 5.

Volatility clustering The empirical tendency for periods of high volatility to be followed by more high volatility, and calm periods to be followed by more calm, rather than volatility being independent from one period to the next. See Chapter 7.

WACC (Weighted Average Cost of Capital) A company's blended cost of financing, combining the cost of debt and the cost of equity in proportion to how much of each the company uses; the standard discount rate for valuing a firm's overall cash flows. See Chapter 4.

Walk-forward validation A time-series-appropriate alternative to a random train-test split, in which a model is repeatedly trained on data only up to a point in time and tested on the period immediately following, mimicking how the model would actually have been used in real time; essential for avoiding look-ahead bias in trading-signal backtests. See Chapters 7 and 13.

Working capital Current assets minus current liabilities; a measure of a company's short-term liquidity and ability to fund its day-to-day operations. See Chapter 4.

Yield The return an investor earns on a bond if held to maturity, accounting for both coupon payments and any difference between the purchase price and face value. See Chapter 5.

Yield curve A plot of yields against maturity for otherwise-comparable bonds (typically government bonds), whose shape (upward-sloping, flat, inverted) is widely watched as a signal about market expectations for future interest rates and growth. See Chapters 5 and 7.

Wulin.Suo@Queensu.ca

Appendix C

A Career Map: Roles, Chapters, and Preparation

This book's stated audience is a reader with quantitative training who wants a career in the financial industry, and the chapters have been building the technical vocabulary that industry runs on. What the chapters do not do, because no single chapter is the right place for it, is describe the industry's actual job landscape: which roles exist, what each one does all day, and which parts of this book matter most for each. This appendix closes that gap. It is a map, not a syllabus; job titles vary considerably across firms (the same work may be called "quantitative researcher" at a hedge fund, "strats" at an investment bank, and "data scientist" at a fintech), so the descriptions below are organized around what the work actually is rather than what any particular firm calls it.

One framing is worth internalizing before the table. Broadly, quantitative roles in finance divide into those that *make* risk (research and trading roles, judged on returns), those that *measure and police* risk (risk management, model validation, and compliance roles, judged on rigor and independence), and those that *build the machinery* both sides run on (engineering and data roles, judged on reliability and scale). All three groups hire people with this book's intended background, the mix of statistics, programming, and financial context, but they reward different temperaments, and the interview processes differ accordingly: the first group leans hardest on probability, brainteasers, and market intuition; the second on statistical rigor, regulatory awareness, and the ability to explain a model's limits; the third on software engineering and system design, with the finance tested more lightly.

C.1 Reading the Map

A few patterns in Table C.1 deserve comment. Chapters 2 and 6, Python and machine learning fundamentals, appear implicitly in every row; they are the price of admission everywhere and are omitted from rows only where a role's distinctive preparation lies elsewhere. Chapter 18, explainability and fairness, appears in every role that touches a regulated decision about a person, which is the pattern the chapter itself predicts: the closer a model sits to a consumer outcome, the more its explanation apparatus matters relative to its accuracy. And Chapter 21's capstone appears wherever the honest answer to "what should I build to demonstrate competence?" is a complete, end-to-end project rather than a familiarity with any single method; a worked capstone, cleaned data and calibration checks and fairness audit included, is the single most credible artifact a candidate without industry experience can bring to an interview, precisely because its eight-stage template mirrors how the work is actually structured inside a firm.

The risk-and-validation column of the industry deserves a special word, because candidates with strong ML backgrounds systematically underrate it. Model validation and risk roles are often easier to enter than front-office research, teach the firm's full model landscape faster than any other seat, are hired against a regulatory requirement rather than a profit forecast (making them notably more stable across market cycles), and increasingly involve genuinely hard technical work, challenging a deep-learning surveillance model requires understanding it at least as well as its builder did. A reader whose favorite chapters were 8

Wulin.Suo@Queensu.ca

Role	What the job actually is	Core chapters
Quantitative researcher (buy side)	Finding and testing predictive signals in market, fundamental, text, and alternative data; building and validating backtests	2, 5–7, 12, 13, 16, 17
Execution and e-trading quant	Designing the algorithms that execute orders and make markets: impact models, order routing, market-making logic	3, 13, 14, 19
Market risk quant	Measuring how much the firm can lose to market moves: VaR and ES models, stress testing, backtesting, limits	5, 7, 8
Credit risk modeler	Estimating who will default and what it costs: PD/LGD/EAD models, scorecards, portfolio credit models, capital	4, 6, 9, 18, 21
Actuary and insurance data scientist	Pricing, reserving, claims, and catastrophe modeling for insurers: GLMs and their machine-learning challengers inside a filed-rating regime	5, 6, 10, 11, 18
Model validator (second line)	Independently challenging every model above: testing assumptions, benchmarking, documentation, SR 11-7 discipline	6–11, 18
Operational-risk and surveillance analyst	Quantifying and detecting operational failures: loss models, anomaly detection, conduct and communications surveillance	6, 10, 15, 16
Fraud and financial-crime data scientist	Catching fraud, money laundering, and market abuse in transaction and network data, at extreme class imbalance	6, 14, 15, 17, 18
NLP and alternative-data researcher	Turning filings, calls, news, satellite and web data into investable signals with point-in-time discipline	13, 16, 17
Fintech and lending ML engineer	Building production credit, robo-advisory, and payments models end to end, with fairness and explainability constraints	2, 6, 9, 12, 18, 21
AI governance and compliance specialist	Setting and enforcing the rules AI systems must satisfy: model inventories, monitoring, fairness audits, regulatory response	10, 15, 16, 18

Table C.1: A map from common quantitative-finance roles to the chapters that prepare for them

Wulin.Suo@Queensu.ca

through 11 and 18 should take that as real information about where they would thrive.

C.2 Preparing, Concretely

Three concrete uses of this book map directly onto how candidates are actually evaluated. First, the worked examples are interview questions: computing a two-asset portfolio variance by hand, walking through a limit-order-book fill, explaining why the 99.9% operational-loss quantile dwarfs its mean, deriving a Shapley decomposition for a small linear model, and reasoning through Bayes' theorem at a realistic base rate are all standard fare, and every one of them is a worked example somewhere in Chapters 1 through 21. Second, the challenge exercises are portfolio projects: each one pairs a method from a chapter with a real, freely downloadable dataset, which is the scope of take-home assignments many firms now use in place of a second interview round. Third, the Further Reading lists are the canon: Hull for derivatives and risk, Bodie-Kane-Marcus for investments, Chan and Hilpisch for systematic trading, and the original papers behind XGBoost, SHAP, and the transformer are the references interviewers themselves learned from, and recognizing them signals fluency the way no self-description can.

The map's final message is the book's own: every role in Table C.1 is being reshaped by the same forces this book has traced chapter by chapter, and the durable advantage is not any single tool but the combination the intended reader has been building all along, enough statistics to be skeptical, enough programming to be self-sufficient, and enough financial context to know what the numbers mean and what happens when they are wrong.

Wulin.Suo@Queensu.ca

Wulin.Suo@Queensu.ca

Bibliography

Every chapter's own "Further Reading" section lists sources with a short note on how each connects to that chapter's material. This appendix consolidates all of them into a single alphabetized list, without the per-chapter annotations, as a stand-alone bibliography a reader can consult directly; several sources are cited in more than one chapter and appear here only once. Full author names, publishers, editions, and (for articles) volume/issue/page numbers were added by verifying each source rather than guessing; a handful of entries for which no single edition, page range, or exact date could be confirmed with confidence are given in the fullest form that could be verified, without an invented specific detail.

- Abbeel, P., and A. Y. Ng, "Apprenticeship Learning via Inverse Reinforcement Learning," *Proceedings of the Twenty-First International Conference on Machine Learning*, 2004, pp. 1–8.
- Acemoglu, D., A. Ozdaglar, and A. Tahbaz-Salehi, "Systemic Risk and Stability in Financial Networks," *American Economic Review*, vol. 105, no. 2, 2015, pp. 564–608.
- Adams, H., N. Zinsmeister, and D. Robinson, "Uniswap v2 Core," 2020. Available at: `https://uniswap.org/whitepaper.pdf`.
- Aldridge, I., *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*, 2nd ed., Wiley, Hoboken, NJ, 2013.
- Almgren, R., and N. Chriss, "Optimal Execution of Portfolio Transactions," *Journal of Risk*, vol. 3, no. 2, 2001, pp. 5–39.
- Ang, A., *Asset Management: A Systematic Approach to Factor Investing*, Oxford University Press, New York, 2014.
- Ansari, A. F., et al., "Chronos: Learning the Language of Time Series," *Transactions on Machine Learning Research*, 2024.
- Appel, G., *Technical Analysis: Power Tools for Active Investors*, Financial Times Prentice Hall, Upper Saddle River, NJ, 2005.
- Arthur, W. B., J. H. Holland, B. LeBaron, R. Palmer, and P. Tayler, "Asset Pricing Under Endogenous Expectations in an Artificial Stock Market," in *The Economy as an Evolving Complex System II*, W. B. Arthur, S. N. Durlauf, and D. A. Lane, eds., Addison-Wesley, Reading, MA, 1997, pp. 15–44.
- Avellaneda, M., and S. Stoikov, "High-Frequency Trading in a Limit Order Book," *Quantitative Finance*, vol. 8, no. 3, 2008, pp. 217–224.
- Axtell, R. L., and J. D. Farmer, "Agent-Based Modeling in Economics and Finance: Past, Present, and Future," *Journal of Economic Literature*, vol. 63, no. 1, 2025, pp. 197–287.
- Bailey, D. H., and M. López de Prado, "The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting, and Non-Normality," *Journal of Portfolio Management*, vol. 40, no. 5, 2014, pp. 94–107.
- Bank of England (O. Burrows, D. Learmonth, and J. McKeown), "RAMSI: A Top-Down Stress-Testing Model," Bank of England Financial Stability Paper No. 17, 2012.

Wulin.Suo@Queensu.ca

- Barocas, S., M. Hardt, and A. Narayanan, *Fairness and Machine Learning: Limitations and Opportunities*, MIT Press, Cambridge, MA, 2023.
- Basel Committee on Banking Supervision, *Basel III: A Global Regulatory Framework for More Resilient Banks and Banking Systems*, Bank for International Settlements, Basel, Switzerland, December 2010 (rev. June 2011).
- Basel Committee on Banking Supervision, *Minimum Capital Requirements for Market Risk*, Bank for International Settlements, Basel, Switzerland, January 2016 (rev. February 2019). The standard produced by the Fundamental Review of the Trading Book (FRTB), replacing Value at Risk with Expected Shortfall and introducing the sensitivities-based capital charge.
- Basel Committee on Banking Supervision, *Guidance on Credit Risk and Accounting for Expected Credit Losses*, Bank for International Settlements, Basel, 2015.
- Baydin, A. G., B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic Differentiation in Machine Learning: A Survey," *Journal of Machine Learning Research*, vol. 18, no. 153, 2018, pp. 1–43.
- Bellotti, T., and J. Crook, "Credit Scoring with Macroeconomic Variables Using Survival Analysis," *Journal of the Operational Research Society*, vol. 60, no. 12, 2009, pp. 1699–1707.
- Berg, F., J. F. Kölbel, and R. Rigobon, "Aggregate Confusion: The Divergence of ESG Ratings," *Review of Finance*, vol. 26, no. 6, 2022, pp. 1315–1344.
- Black, F., "Studies of Stock Price Volatility Changes," *Proceedings of the 1976 Meetings of the Business and Economics Statistics Section, American Statistical Association*, 1976, pp. 177–181.
- Black, F., and R. Litterman, "Global Portfolio Optimization," *Financial Analysts Journal*, vol. 48, no. 5, 1992, pp. 28–43.
- Board of Governors of the Federal Reserve System and Office of the Comptroller of the Currency, *Supervisory Guidance on Model Risk Management*, SR 11-7, April 4, 2011.
- Bodie, Z., A. Kane, and A. Marcus, *Investments*, 13th ed., McGraw Hill, New York, 2024.
- Bollinger, J. A., *Bollinger on Bollinger Bands*, McGraw-Hill, New York, 2001.
- Bolton, R. J., and D. J. Hand, "Statistical Fraud Detection: A Review," *Statistical Science*, vol. 17, no. 3, 2002, pp. 235–255.
- Brealey, R. A., S. C. Myers, F. Allen, and A. Edmans, *Principles of Corporate Finance*, 15th ed., McGraw Hill, New York, 2025.
- Britten-Jones, M., and A. Neuberger, "Option Prices, Implied Price Processes, and Stochastic Volatility," *Journal of Finance*, vol. 55, no. 2, 2000, pp. 839–866.
- Brock, W. A., and C. H. Hommes, "A Rational Route to Randomness," *Econometrica*, vol. 65, no. 5, 1997, pp. 1059–1095.
- Bruun & Hjejle, *Report on the Non-Resident Portfolio at Danske Bank's Estonian Branch*, law firm report commissioned by Danske Bank A/S, September 2018.
- Budish, E., P. Cramton, and J. Shim, "The High-Frequency Trading Arms Race: Frequent Batch Auctions as a Market Design Response," *Quarterly Journal of Economics*, vol. 130, no. 4, 2015, pp. 1547–1621.
- Buehler, H., L. Gonon, J. Teichmann, and B. Wood, "Deep Hedging," *Quantitative Finance*, vol. 19, no. 8, 2019, pp. 1271–1291.

Wulin.Suo@Queensu.ca

- Bühlmann, H., and A. Gisler, *A Course in Credibility Theory and its Applications*, Springer, Berlin, 2005.
- Buolamwini, J., and T. Gebru, "Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification," *Proceedings of Machine Learning Research*, vol. 81, 2018, pp. 77–91.
- Campbell, J. Y., A. W. Lo, and A. C. MacKinlay, *The Econometrics of Financial Markets*, Princeton University Press, Princeton, NJ, 1997.
- Campbell, J. Y., and S. B. Thompson, "Predicting Excess Stock Returns Out of Sample: Can Anything Beat the Historical Average?" *The Review of Financial Studies*, vol. 21, no. 4, 2008, pp. 1509–1531.
- Chan, E. P., *Algorithmic Trading: Winning Strategies and Their Rationale*, Wiley, Hoboken, NJ, 2013.
- Chan, E. P., *Machine Trading: Deploying Computer Algorithms to Conquer the Markets*, Wiley, Hoboken, NJ, 2017.
- Chawla, N. V., K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, 2002, pp. 321–357.
- Chen, T., and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- Chen, T., S. Kornblith, M. Norouzi, and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," *Proceedings of the 37th International Conference on Machine Learning*, PMLR vol. 119, 2020, pp. 1597–1607.
- Chopra, V. K., and W. T. Ziemba, "The Effect of Errors in Means, Variances, and Covariances on Optimal Portfolio Choice," *Journal of Portfolio Management*, vol. 19, no. 2, 1993, pp. 6–11.
- Chouldechova, A., "Fair Prediction with Disparate Impact: A Study of Bias in Recidivism Prediction Instruments," *Big Data*, vol. 5, no. 2, 2017, pp. 153–163.
- Consumer Financial Protection Bureau, "CFPB Announces First No-Action Letter to Upstart Network," press release, September 14, 2017.
- Counts, L., "How Hedge Funds Use Satellite Images to Beat Wall Street, and Main Street," UC Berkeley Haas Newsroom, May 28, 2019 (reporting on research co-authored by Panos N. Patatoukas and Zsolt Katona).
- Cristianini, N., and J. Shawe-Taylor, *An Introduction to Support Vector Machines and Other Kernel-based Learning Methods*, Cambridge University Press, Cambridge, UK, 2000.
- Crouhy, M., D. Galai, and R. Mark, *The Essentials of Risk Management*, 2nd ed., McGraw-Hill, New York, 2014.
- Cruz, M. G., G. W. Peters, and P. V. Shevchenko, *Fundamental Aspects of Operational Risk and Insurance Analytics: A Handbook of Operational Risk*, Wiley, Hoboken, NJ, 2015.
- D'Acunto, F., N. Prabhala, and A. G. Rossi, "The Promises and Pitfalls of Robo-Advising," *The Review of Financial Studies*, vol. 32, no. 5, 2019, pp. 1983–2020.
- Damodaran, A., *Investment Valuation: Tools and Techniques for Determining the Value of Any Asset*, 3rd ed., Wiley, Hoboken, NJ, 2012.
- Denuit, M., D. Hainaut, and J. Trufin, *Effective Statistical Learning Methods for Actuaries* (3 vols.), Springer, Cham, 2019–2020.
- Dickson, D. C. M., M. R. Hardy, and H. R. Waters, *Actuarial Mathematics for Life Contingent Risks*, 3rd ed., Cambridge University Press, Cambridge, UK, 2020.

Wulin.Suo@Queensu.ca

- Doshi-Velez, F., and B. Kim, "Towards a Rigorous Science of Interpretable Machine Learning," arXiv:1702.08608, 2017.
- Dosovitskiy, A., L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale," *International Conference on Learning Representations*, 2021.
- Dua, D., and C. Graff, *UCI Machine Learning Repository*, University of California, Irvine, School of Information and Computer Sciences, 2019.
- Dwork, C., M. Hardt, T. Pitassi, O. Reingold, and R. Zemel, "Fairness Through Awareness," *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, 2012, pp. 214–226.
- Eisenberg, L., and T. H. Noe, "Systemic Risk in Financial Systems," *Management Science*, vol. 47, no. 2, 2001, pp. 236–249.
- Engle, R. F., "Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation," *Econometrica*, vol. 50, no. 4, 1982, pp. 987–1007.
- Engle, R. F., and C. W. J. Granger, "Co-integration and Error Correction: Representation, Estimation, and Testing," *Econometrica*, vol. 55, no. 2, 1987, pp. 251–276.
- Fabozzi, F. J., *Fixed Income Analysis*, 2nd ed., CFA Institute Investment Series, Wiley, Hoboken, NJ, 2007.
- Fama, E. F., and K. R. French, "Common Risk Factors in the Returns on Stocks and Bonds," *Journal of Financial Economics*, vol. 33, no. 1, 1993, pp. 3–56.
- Farmer, J. D., and D. Foley, "The Economy Needs Agent-Based Modelling," *Nature*, vol. 460, 2009, pp. 685–686.
- Federal Reserve Bank of St. Louis, *FRED API Documentation*, fred.stlouisfed.org.
- Feng, C., and S. Fay, "An Empirical Investigation of Forward-Looking Retailer Performance Using Parking Lot Traffic Data Derived from Satellite Imagery," *Journal of Retailing*, vol. 98, no. 4, 2022, pp. 633–646.
- Financial Action Task Force (FATF), *International Standards on Combating Money Laundering and the Financing of Terrorism & Proliferation: The FATF Recommendations*, FATF, Paris, May 2012.
- Frees, E. W., *Regression Modeling with Actuarial and Financial Applications*, Cambridge University Press, Cambridge, UK, 2010.
- French, K. R., *Data Library*, Tuck School of Business, Dartmouth College, mba.tuck.dartmouth.edu.
- Giles, M., and P. Glasserman, "Smoking Adjoints: Fast Monte Carlo Greeks," *Risk*, vol. 19, no. 1, January 2006, pp. 88–92.
- Gode, D. K., and S. Sunder, "Allocative Efficiency of Markets with Zero-Intelligence Traders: Market as a Partial Substitute for Individual Rationality," *Journal of Political Economy*, vol. 101, no. 1, 1993, pp. 119–137.
- Goldburd, M., A. Khare, D. Tebaldi, and D. Modlin, *Generalized Linear Models for Insurance Rating*, 2nd ed., CAS Monograph No. 5, Casualty Actuarial Society, Arlington, VA, 2020.
- Goodfellow, I., Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, Cambridge, MA, 2016.
- Goodfellow, I., J. Shlens, and C. Szegedy, "Explaining and Harnessing Adversarial Examples," *International Conference on Learning Representations*, 2015.

Wulin.Suo@Queensu.ca

- Goodfellow, I., J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative Adversarial Networks," *Communications of the ACM*, vol. 63, no. 11, 2020, pp. 139–144.
- Grinold, R. C., and R. N. Kahn, *Active Portfolio Management: A Quantitative Approach for Producing Superior Returns and Controlling Risk*, 2nd ed., McGraw-Hill, New York, 2000.
- Grossi, P., and H. Kunreuther, eds., *Catastrophe Modeling: A New Approach to Managing Risk*, Springer, New York, 2005.
- Gu, S., B. Kelly, and D. Xiu, "Empirical Asset Pricing via Machine Learning," *The Review of Financial Studies*, vol. 33, no. 5, 2020, pp. 2223–2273.
- Guéant, O., C.-A. Lehalle, and J. Fernandez-Tapia, "Dealing with the Inventory Risk: A Solution to the Market Making Problem," *Mathematics and Financial Economics*, vol. 7, no. 4, 2013, pp. 477–507.
- Gupton, G. M., C. C. Finger, and M. Bhatia, *CreditMetrics—Technical Document*, J.P. Morgan & Co. Incorporated, New York, April 2, 1997.
- Hamilton, J. D., *Time Series Analysis*, Princeton University Press, Princeton, NJ, 1994.
- Hand, D. J., and W. E. Henley, "Statistical Classification Methods in Consumer Credit Scoring: A Review," *Journal of the Royal Statistical Society, Series A*, vol. 160, no. 3, 1997, pp. 523–541.
- Hansen, S., and M. McMahon, "Shocking Language: Understanding the Macroeconomic Effects of Central Bank Communication," *Journal of International Economics*, vol. 99, suppl. 1, 2016, pp. S114–S133.
- Harris, L., *Trading and Exchanges: Market Microstructure for Practitioners*, Oxford University Press, New York, 2003.
- Hastie, T., R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed., Springer, New York, 2009.
- He, K., X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *2016 IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- Hilpisch, Y., *Python for Algorithmic Trading: From Idea to Cloud Deployment*, O'Reilly Media, Sebastopol, CA, 2020.
- Hilpisch, Y., *Python for Finance: Mastering Data-Driven Finance*, 2nd ed., O'Reilly Media, Sebastopol, CA, 2018.
- Hinton, G. E., and R. R. Salakhutdinov, "Reducing the Dimensionality of Data with Neural Networks," *Science*, vol. 313, no. 5786, 2006, pp. 504–507.
- Ho, J., A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," *Advances in Neural Information Processing Systems 33*, 2020, pp. 6840–6851.
- Hochreiter, S., and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, 1997, pp. 1735–1780.
- Hull, J. C., *Options, Futures, and Other Derivatives*, 11th ed., Pearson, New York, 2022.
- Hull, J. C., *Risk Management and Financial Institutions*, 6th ed., Wiley, Hoboken, NJ, 2023.
- Hull, J. C., J. Chen, Z. Poulos, and J. Yuan, *Machine Learning in Business: An Introduction to the World of Data Science*, 4th ed., self-published, 2025.
- Hyndman, R. J., and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed., OTexts, Melbourne, Australia, 2021.

Wulin.Suo@Queensu.ca

- Institute of Internal Auditors, *The IIA's Three Lines Model: An Update of the Three Lines of Defense*, Institute of Internal Auditors, Lake Mary, FL, 2020.

- Jain, S., and B. C. Wallace, "Attention is not Explanation," *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 3543–3556.

- James, G., D. Witten, T. Hastie, R. Tibshirani, and J. Taylor, *An Introduction to Statistical Learning: with Applications in Python*, Springer, New York, 2023.

- Jorion, P., *Value at Risk: The New Benchmark for Managing Financial Risk*, 3rd ed., McGraw-Hill, New York, 2007.

- Jurafsky, D., and J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*, 3rd ed. (online draft), Stanford University.

- Kaggle, *Datasets and Public API Documentation*, kaggle.com.

- Katona, Z., M. O. Painter, P. N. Patatoukas, and J. Zeng, "On the Capital Market Consequences of Big Data: Evidence from Outer Space," *Journal of Financial and Quantitative Analysis*, vol. 60, no. 2, 2025, pp. 551–579.

- Kipf, T. N., and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," *International Conference on Learning Representations*, 2017.

- Kissell, R., *The Science of Algorithmic Trading and Portfolio Management*, Academic Press, Amsterdam, 2013.

- Klein, J. P., and M. L. Moeschberger, *Survival Analysis: Techniques for Censored and Truncated Data*, 2nd ed., Springer, New York, 2003.

- Kleinberg, J., S. Mullainathan, and M. Raghavan, "Inherent Trade-Offs in the Fair Determination of Risk Scores," *8th Innovations in Theoretical Computer Science Conference*, LIPIcs vol. 67, article 43, 2017.

- Kolanovic, M., and R. T. Krishnamachari, "Big Data and AI Strategies: Machine Learning and Alternative Data Approach to Investing," J.P. Morgan, Global Quantitative & Derivatives Strategy, May 2017.

- Kusner, M. J., J. R. Loftus, C. Russell, and R. Silva, "Counterfactual Fairness," *Advances in Neural Information Processing Systems 30*, 2017, pp. 4066–4076.

- Kyle, A. S., "Continuous Auctions and Insider Trading," *Econometrica*, vol. 53, no. 6, 1985, pp. 1315–1335.

- LeBaron, B., "Agent-Based Computational Finance," in *Handbook of Computational Economics, Volume 2*, L. Tesfatsion and K. L. Judd, eds., Elsevier, Amsterdam, 2006, pp. 1187–1233.

- LeCun, Y., Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, no. 7553, 2015, pp. 436–444.

- Ledoit, O., and M. Wolf, "Honey, I Shrunk the Sample Covariance Matrix," *Journal of Portfolio Management*, vol. 30, no. 4, 2004, pp. 110–119.

- Lewis, M., *Flash Boys: A Wall Street Revolt*, W.W. Norton & Company, New York, 2014.

- Lewis, P., E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems 33*, 2020, pp. 9459–9474.

Wulin.Suo@Queensu.ca

- Li, D. X., "On Default Correlation: A Copula Function Approach," *Journal of Fixed Income*, vol. 9, no. 4, 2000, pp. 43–54.
- Li, F., "The Information Content of Forward-Looking Statements in Corporate Filings—A Naïve Bayesian Machine Learning Approach," *Journal of Accounting Research*, vol. 48, no. 5, 2010, pp. 1049–1102.
- Litterman, R., and J. Scheinkman, "Common Factors Affecting Bond Returns," *Journal of Fixed Income*, vol. 1, no. 1, 1991, pp. 54–61.
- LOBSTER, *Limit Order Book Data Documentation*, lobsterdata.com.
- López de Prado, M., *Advances in Financial Machine Learning*, Wiley, Hoboken, NJ, 2018.
- Lou, Y., R. Caruana, and J. Gehrke, "Intelligible Models for Classification and Regression," *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2012, pp. 150–158.
- Loughran, T., and B. McDonald, "When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks," *Journal of Finance*, vol. 66, no. 1, 2011, pp. 35–65.
- Lundberg, S. M., and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.
- Lux, T., and M. Marchesi, "Scaling and Criticality in a Stochastic Multi-Agent Model of a Financial Market," *Nature*, vol. 397, 1999, pp. 498–500.
- Markowitz, H., "Portfolio Selection," *Journal of Finance*, vol. 7, no. 1, 1952, pp. 77–91.
- McDonald, R. L., *Derivatives Markets*, 3rd ed., Pearson, Boston, 2012.
- McKinney, W., *Python for Data Analysis*, 3rd ed., O'Reilly Media, Sebastopol, CA, 2022.
- McLean, R. D., and J. Pontiff, "Does Academic Research Destroy Stock Return Predictability?" *The Journal of Finance*, vol. 71, no. 1, 2016, pp. 5–32.
- McNeil, A. J., R. Frey, and P. Embrechts, *Quantitative Risk Management: Concepts, Techniques and Tools*, rev. ed., Princeton University Press, Princeton, NJ, 2015.
- Menkveld, A. J., "High-Frequency Trading and the New Market Makers," *Journal of Financial Markets*, vol. 16, no. 4, 2013, pp. 712–740.
- Merton, R. C., "On the Pricing of Corporate Debt: The Risk Structure of Interest Rates," *Journal of Finance*, vol. 29, no. 2, 1974, pp. 449–470.
- Michaud, R. O., and R. O. Michaud, *Efficient Asset Management: A Practical Guide to Stock Portfolio Optimization and Asset Allocation*, 2nd ed., Oxford University Press, New York, 2008.
- Mikolov, T., K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," arXiv:1301.3781, 2013.
- Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al., "Human-Level Control through Deep Reinforcement Learning," *Nature*, vol. 518, 2015, pp. 529–533.
- Molnar, C., *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*, 2nd ed., christophm.github.io/interpretable-ml-book, 2022.
- Morgan Stanley, "Key Milestone in Innovation Journey with OpenAI" and "Launch of AI @ Morgan Stanley Debrief" (press releases).

Wulin.Suo@Queensu.ca

- Murphy, K. P., *Machine Learning: A Probabilistic Perspective*, MIT Press, Cambridge, MA, 2012.
- National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, U.S. Department of Commerce, January 2023.
- Nevmyvaka, Y., Y. Feng, and M. Kearns, "Reinforcement Learning for Optimized Trade Execution," *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 673–680.
- New York State Department of Financial Services, *Report on Apple Card Investigation*, March 2021.
- Niculescu-Mizil, A., and R. Caruana, "Predicting Good Probabilities with Supervised Learning," *Proceedings of the 22nd International Conference on Machine Learning*, 2005, pp. 625–632.
- O'Hara, M., *Market Microstructure Theory*, Blackwell Publishers, Cambridge, MA, 1995.
- O'Hara, M., "High Frequency Market Microstructure," *Journal of Financial Economics*, vol. 116, no. 2, 2015, pp. 257–270.
- Ohlsson, E., and B. Johansson, *Non-Life Insurance Pricing with Generalized Linear Models*, Springer, Berlin, 2010.
- Ouyang, L., J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, "Training Language Models to Follow Instructions with Human Feedback," *Advances in Neural Information Processing Systems 35*, 2022, pp. 27730–27744.
- Palepu, K. G., P. M. Healy, and E. Peek, *Business Analysis and Valuation: IFRS Edition*, Cengage Learning EMEA.
- Pan, S. J., and Q. Yang, "A Survey on Transfer Learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, 2010, pp. 1345–1359.
- Patterson, S., *Dark Pools: The Rise of the Machine Traders and the Rigging of the U.S. Stock Market*, Crown Business, New York, 2012.
- Penman, S. H., *Financial Statement Analysis and Security Valuation*, 5th ed., McGraw-Hill, New York, 2013.
- Provost, F., and T. Fawcett, *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*, O'Reilly Media, Sebastopol, CA, 2013.
- Ribeiro, M. T., S. Singh, and C. Guestrin, "'Why Should I Trust You?': Explaining the Predictions of Any Classifier," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.
- Roll, R., "A Simple Implicit Measure of the Effective Bid-Ask Spread in an Efficient Market," *Journal of Finance*, vol. 39, no. 4, 1984, pp. 1127–1139.
- Rubinstein, M., "Implied Binomial Trees," *Journal of Finance*, vol. 49, no. 3, 1994, pp. 771–818.
- Salmon, F., "Recipe for Disaster: The Formula That Killed Wall Street," *Wired*, February 23, 2009.
- Saunders, A., and L. Allen, *Credit Risk Management In and Out of the Financial Crisis: New Approaches to Value at Risk and Other Paradigms*, 3rd ed., Wiley, Hoboken, NJ, 2010.
- Schelling, T. C., "Dynamic Models of Segregation," *Journal of Mathematical Sociology*, vol. 1, no. 2, 1971, pp. 143–186.
- Schulman, J., F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

Wulin.Suo@Queensu.ca

- Shapley, L. S., "A Value for n-Person Games," in *Contributions to the Theory of Games (AM-28), Volume II*, H. W. Kuhn and A. W. Tucker, eds., Annals of Mathematics Studies no. 28, Princeton University Press, Princeton, NJ, 1953, pp. 307–318.
- Sharpe, W. F., "Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk," *Journal of Finance*, vol. 19, no. 3, 1964, pp. 425–442.
- Shazeer, N., A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," *International Conference on Learning Representations*, 2017.
- Shevchenko, P. V., *Modelling Operational Risk Using Bayesian Inference*, Springer, Berlin, 2011.
- Shleifer, A., *Inefficient Markets: An Introduction to Behavioral Finance*, Oxford University Press, Oxford, 2000.
- Sims, C. A., "Macroeconomics and Reality," *Econometrica*, vol. 48, no. 1, 1980, pp. 1–48.
- Sundararajan, M., A. Taly, and Q. Yan, "Axiomatic Attribution for Deep Networks," *Proceedings of the 34th International Conference on Machine Learning*, PMLR vol. 70, 2017, pp. 3319–3328.
- Sutton, R. S., and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, Cambridge, MA, 2018.
- Tsay, R. S., *Analysis of Financial Time Series*, 3rd ed., Wiley, Hoboken, NJ, 2010.
- Turpin, M., J. Michael, E. Perez, and S. R. Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting," *Advances in Neural Information Processing Systems 36*, 2023.
- UCI Machine Learning Repository, *Default of Credit Card Clients Data Set*, University of California, Irvine.
- U.S. Financial Crimes Enforcement Network (FinCEN), *Bank Secrecy Act Regulations and Guidance*, U.S. Department of the Treasury.
- U.S. Securities and Exchange Commission, *A Plain English Handbook: How to Create Clear SEC Disclosure Documents*, August 1998.
- U.S. Securities and Exchange Commission, *EDGAR Application Programming Interfaces*, sec.gov.
- U.S. Securities and Exchange Commission, *Regulation NMS*, Release No. 34-51808, File No. S7-10-04, 2005.
- U.S. Securities and Exchange Commission, "Schwab Subsidiaries Misled Robo-Adviser Clients about Absence of Hidden Fees," press release 2022-104, June 13, 2022.
- U.S. Securities and Exchange Commission and Commodity Futures Trading Commission, *Findings Regarding the Market Events of May 6, 2010: Report of the Staffs of the CFTC and SEC to the Joint Advisory Committee on Emerging Regulatory Issues*, Washington, DC, September 30, 2010.
- Ustun, B., A. Spangher, and Y. Liu, "Actionable Recourse in Linear Classification," *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 2019, pp. 10–19.
- van der Aalst, W. M. P., *Process Mining: Data Science in Action*, 2nd ed., Springer, Berlin, 2016.
- VanderPlas, J., *Python Data Science Handbook: Essential Tools for Working with Data*, 2nd ed., O'Reilly Media, Sebastopol, CA, 2022.
- Vasicek, O. A., "The Distribution of Loan Portfolio Value," *Risk*, vol. 15, no. 12, 2002, pp. 160–162.

Wulin.Suo@Queensu.ca

- Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.
- Wei, J., X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Advances in Neural Information Processing Systems 35*, 2022, pp. 24824–24837.
- Wilder, J. W., Jr., *New Concepts in Technical Trading Systems*, Trend Research, Greensboro, NC, 1978.
- Wu, S., O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, and G. Mann, "BloombergGPT: A Large Language Model for Finance," arXiv:2303.17564, 2023.
- Wüthrich, M. V., and M. Merz, *Statistical Foundations of Actuarial Learning and its Applications*, Springer, Cham, 2023.
- Yao, S., J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," *International Conference on Learning Representations*, 2023.
- Yeh, I.-C., and C.-H. Lien, "The Comparisons of Data Mining Techniques for the Predictive Accuracy of Probability of Default of Credit Card Clients," *Expert Systems with Applications*, vol. 36, no. 2, 2009, pp. 2473–2480.
- Zhang, K., Z. Yang, and T. Başar, "Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms," in *Handbook of Reinforcement Learning and Control*, K. G. Vamvoudakis, Y. Wan, F. L. Lewis, and D. Cansever, eds., Springer, Cham, 2021, pp. 321–384.

Wulin.Suo@Queensu.ca

Index

60/40 Portfolio, 250

Absolute Price Oscillator (APO), 270
Accruals Ratio, 76
Actor-Critic, 400
actuarial science, 217
Adjoint algorithmic differentiation, 163
adverse impact ratio, 378, 446
adverse selection, 218, 351
agent-based models (ABMs), 2, 417
AI Winter, 4
AlexNet, 4
algorithmic recourse, 383
Almgren-Chriss model, 262, 391
AlphaGo, 4
alternative data, 463
Alternative Reference Rates (ARRs), 47
Alternative-Data Vendor Ecosystem, 351
Altman's Z-score, 74
Anomaly Detection in Transactions, 301
Apple Card gender-bias controversy, 372, 380, 446
ARIMA models, 136
AUC (Area Under the Curve), 111, 181
Authorized Push Payment (APP) Fraud, 307
Autocorrelation and the Correlogram, 131
autoencoder, 207, 305, 349
Avellaneda-Stoikov model, 257, 279, 393

backpropagation, 4, 122, 356, 383
backtest overfitting, 267
Backtesting Expected Shortfall, 158
Bag-of-Words and TF-IDF for Four Earnings-Call Sentences, 325
Balance Sheet, 64
Bank of England RAMSI model, 425
Bank Secrecy Act (BSA), 311
Basel Accords, 54, 69, 158
Basel III, 54
Basel operational-risk event types, 202
Basel Traffic-Light Backtesting Framework, 158
basis risk (parametric insurance), 227
Bayes' theorem, 17, 120, 282, 299
Behavioral Cloning, 400
Bellman equation, 395
Bid-ask bounce, 289
bid-ask spread, 48, 152, 243, 257, 281
binomial option pricing model, 89, 451
Black-Litterman model, 233, 242
Black-Scholes-Merton model, 3, 90, 92, 183, 367, 451
BloombergGPT, 341
Bollinger bands, 270
Box-Jenkins methodology, 136
Bradley-Terry model, 339
Brier score, 440
Brock-Hommes model, 422
Buhlmann credibility, 221

calibration, 19, 100, 117, 145, 157, 251, 274, 294, 379, 437
capital asset pricing model (CAPM), 85, 233, 266
Capstone Dataset and Its Imperfections, 438
careers in financial AI, 479
Cascade Home & Garden, 352, 437
Cash Conversion Cycle, 68
Cash Flow Statement, 67
Cash-flow underwriting, 193
catastrophe bond, 227
CBOE Volatility Index (VIX), 96, 272, 327
CECL, 187
central-bank communication, 326
Certainty Equivalent, 15
chain ladder, 224
Chain-of-Thought Prompting, 337
ChatGPT, 4
class imbalance, 304, 440
Clustering and Dimensionality Reduction, 121
Coherent Risk Measure, 155
cointegration, 108, 129
Cointegration and Pairs Trading, 137
Collateralized Debt Obligations (CDOs), 195
combined ratio, 219
commercial substance, 312
Common-Size Statements, 73
Component VaR, 154
Comprehensive Capital Analysis and Review (CCAR), 165
Concept Drift, 117
Conformal Prediction, 384
confusion matrix, 110, 299, 329, 386, 452
Convexity, 83

Wulin.Suo@Queensu.ca

convolutional neural network (CNN), 355
Correlation Stress Testing, 165
Cosine Similarity of Toy Embeddings, 330
counterfactual explanation, 371
covariance shrinkage, 236
Cox-Ross-Rubinstein (CRR) parametrization, 90
credibility theory, 221
credibility weighting, 203
Credit Assignment Problem, 412
Credit Conversion Factor (CCF), 178
Credit Default Swaps, 185
Credit Scorecard, 373
Credit Stress Testing, 179
CreditMetrics, 187
cross-validation, 114, 129, 447
Currency Transaction Report (CTR), 311
Current Expected Credit Losses (CECL), 442
Curse of Dimensionality, 112
Customer Due Diligence (CDD), 313

Danske Bank Estonia Case, 317
Danske Bank Estonia scandal, 317
Dark Pools, 259
Dartmouth Workshop, 4
Data Cleaning and Preparation, 32
Data Leakage, 117
Data Manipulation with pandas, 28
Debt Securities, 45
Deep Blue, 4
Deep Hedging, 406, 407
deep Q-network (DQN), 399
deflated Sharpe ratio, 268
Delta Hedging, 93, 168
Delta-Gamma Approximation, 161
Delta-Normal Approximation, 160
Derivatives, 46
Detecting Structuring, 311
Dickey-Fuller test, 130
differential privacy, 315
diffusion model, 323
discrete-time hazard model, 188
Diversification, 8
dividend discount model (DDM), 46, 86, 87
Dodd-Frank Act, 54, 286
DuPont identity, 71
DV01, 162
dynamic lapse, 226

EBITDA, 66
Economic capital, 159, 189
Economic Profit, 70
efficient frontier, 233, 262
efficient market hypothesis (EMH), 9
Eisenberg-Noe clearing model, 424
Embedding, 329
Engle-Granger two-step method, 137
Enhanced Due Diligence (EDD), 313
entitlement graph, 209
Environmental, Social, and Governance (ESG) Investing, 250
Epsilon-Greedy Exploration, 397
Equal Credit Opportunity Act (ECOA), 375, 443
Equalized Odds, 379, 388
Equity Securities, 46
ESG rating divergence, 251, 361
ETF Creation and Redemption, 283
EU Artificial Intelligence Act, 378
Event-Driven Sentiment Trading Signal, 327
Expected Shortfall, 151, 451
Expected Value of Perfect Information (EVPI), 17
explainable boosting machine (EBM), 182
exploration-exploitation trade-off, 394
Exposure at Default (EAD), 177

Fairness in Financial AI, 378
Fama-French three-factor model, 240
federated averaging, 315
federated learning, 315
Few-Shot Prompting, 336
Filtered Historical Simulation, 157
Financial Action Task Force (FATF), 311
Financial Data Ecosystem, 6
Flash Crash of 2010, 59, 274, 293
float (insurance), 219
FOMC statement analysis, 326
Forward Rate, 85
four-fifths rule, 378, 445
Fraud Scoring with Logistic Regression, 302
FRED (Federal Reserve Economic Data), 458
Free Cash Flow (FCF), 67
Free Cash Flow to Equity (FCFE), 75
Free Cash Flow to the Firm (FCFF), 75
Fundamental Review of the Trading Book (FRTB), 171

GARCH model, 41, 142, 153
Gaussian copula, 195
General Data Protection Regulation (GDPR), 378
generalized additive model (GAM), 182, 373
generalized linear model (GLM), 110, 220
generative adversarial network (GAN), 337
Gini coefficient, 181
Gini impurity, 119
global minimum-variance portfolio, 234
Glosten-Milgrom model, 279, 282, 403
Gode-Sunder zero-intelligence model, 420
Goodhart's law, 14, 208
Gordon growth model, 86

Wulin.Suo@Queensu.ca

GPT-3, 4
gradient boosting, 2, 120, 305, 441
Gradient boosting for credit scoring, 193
Gradient boosting for volatility forecasting, 166
gradient descent, 105, 362
Granger causality, 139
Granularity Adjustment, 191
graph neural network (GNN), 349
Greenwood's formula, 310

Hallucination, 334
Hazard Rate, 185
Herfindahl-Hirschman Index (HHI), 189
Hierarchical Risk Parity, 246
Holt's linear trend method, 136

Iceberg Orders, 259
IEEE-CIS Fraud Detection dataset, 463
IFRS 9, 187, 442
ImageNet, 4
Imitation Learning, 400
Implied Volatility, 94
Impulse Response Function, 139
in-degree and out-degree, 312
Income Statement, 66
Insider Trading Surveillance, 301
integrated gradients, 384
internal loss data, 202
Inverse Reinforcement Learning, 401
Isolation Forest, 305
Isotonic Recalibration, 442

jump-diffusion model, 100

K-means clustering, 121
K-nearest neighbors (KNN), 111, 180
Kalman filter, 129, 143, 282
Kaplan-Meier estimator, 299
Kaplan-Meier Survival of Flagged Accounts, 309
Kelly criterion, 16, 405
Ken French Data Library, 463
key risk indicators, 208
Knight Capital trading glitch, 274, 279, 316
Knight, Frank, 14
Knightian uncertainty, 14
Know-Your-Customer (KYC), 308
Kupiec's proportion-of-failures (POF) test, 158
Kyle model, 257, 281
Kyle's lambda, 279

Laplace smoothing, 120
large language model (LLM), 2, 106, 323, 384
Lasso Regression, 115
Latency Arbitrage and Stale Quotes, 281
Lee-Carter model, 225
Less discriminatory alternative, 194
Levenshtein distance, 313
Leverage and Solvency, 68
LIBOR, 47
LIME (Local Interpretable Model-agnostic Explanations), 372, 442
limit order book, 48
Liquidity Coverage Ratio (LCR), 160
Ljung-Box test, 134
LOBSTER, 463
logistic regression, 106, 180, 291, 299, 363, 371, 439
lognormal distribution, 204
London Whale (JPMorgan), 213
long short-term memory (LSTM), 357
Long-Term Capital Management (LTCM), 18, 58
longevity risk, 225
loss distribution approach, 204
Loss Given Default (LGD), 177
Loughran-McDonald dictionary, 325, 361, 463
Low-Rank Adaptation (LoRA), 339
LOXM, 408
Lux-Marchesi model, 422

maker-taker pricing model, 275
Market Efficiency, 9
market impact, 19, 50, 243, 259, 279, 327
Markov decision process (MDP), 391
Markowitz, Harry, 2, 234
Maximum drawdown, 155
Mean Absolute Percentage Error (MAPE), 144
Meridian Fabrication, 361
Meridian Fabrication Inc., 64, 81, 237, 437
Merton model, 184
method of simulated moments, 429
MiFID II, 54, 293
mixture of experts, 340
Mode Collapse, 338
Model Evaluation, 117
model risk, 211
model risk management, 13, 167, 246, 274, 292, 316, 339, 381
Monotonic constraints, 193
Monte Carlo simulation, 151, 451
Moody's KMV model, 184
moral hazard, 218
Morgan Stanley, 343
Moving-Average Crossover Strategy, 258
Multi-Agent Dynamics, 408
Multi-Agent Reinforcement Learning, 408
multi-armed bandit, 393
Multi-Modal Fusion via Cross-Attention, 363

Naive Bayes, 361
Naive Bayes classifier, 106, 323, 450

Wulin.Suo@Queensu.ca

National Best Bid and Offer (NBBO), 288
Net Stable Funding Ratio (NSFR), 160
News and Event-Driven Trading, 273
NIST AI Risk Management Framework, 382, 447

Object-Oriented Programming, 35
Off-Policy Evaluation, 409
Office of Foreign Assets Control (OFAC), 313
On-Chain Data, 350
operational risk, 201
OptionMetrics, 458
Orbital Insight, 365
Order flow imbalance (OFI), 290
Ordinary Least Squares (OLS), 107
overfitting, 19, 100, 105, 145, 172, 196, 236, 273, 305, 365, 439
Overfitting and Generalization, 113

Pairs-Trading Backtest, 265
parametric insurance, 227
Partial Autocorrelation Function (PACF), 136
Perceptron, 4
Poisson distribution, 204
Policy Gradient, 391, 400
population stability index (PSI), 371, 447
Portfolio Duration, 84
predictive process monitoring, 210
price optimization (insurance), 229
Principal Component Analysis (PCA), 121, 140
private credit, 45
private equity, 45
Probability of Default (PD), 177
Probability of Informed Trading (PIN), 283
process mining, 209
Prosecuting Spoofing, 286
Proximal Policy Optimization (PPO), 400
proxy model (Solvency II), 228
pure premium, 219
put-call parity, 92

Q-Learning, 391
Quant Quake (2007), 266

random forest, 2, 119, 440
rate on line, 227
Receiver Operating Characteristic (ROC) Curve, 111
recurrent neural network (RNN), 349
Refinitiv Datastream, 457
regime filter, 272
regret (bandit), 394
Regulation B, 375
Regulation NMS (Reg NMS), 54, 259, 288
Regulatory Capital, 189
Reinforcement Learning, 2, 391
Reinforcement Learning Framework, 392
reinforcement learning from human feedback (RLHF), 339
reinsurance, 226
Relative Strength Index (RSI), 270
Repurchase Agreement (Repo), 51
residual network (ResNet), 356
retrieval-augmented generation (RAG), 323
return predictability, 245
reverse optimization, 241
Reverse Stress Testing, 165
Ridge Regression, 115
Ridge regression for factor exposures, 246
risk adjustment, 225
Risk and Return, 6
Risk mitigation, 167
risk parity, 236
risk pooling, 218
Risk-Neutral Pricing, 90
ROC curve, 306, 440
rogue trading, 201
Roll's implied spread model, 50, 289
Root Mean Squared Error (RMSE), 144
RS Metrics, 365

Saliency Map, 383
Sanctions Screening, 313
Santa Fe Artificial Stock Market, 419
SARIMA (Seasonal ARIMA), 137
SARSA, 396
Satellite Imagery and Retailer Revenue, 352
scenario analysis, 203
Schelling segregation model, 419
Schwab Intelligent Portfolios, 249
Scorecard, 182
Scoring Two Earnings-Call Transcripts, 326
SEC EDGAR, 458
SEC Tick Size Pilot, 294
Secured Overnight Financing Rate (SOFR), 47
Securities and Exchange Commission (SEC), 458
Securities Information Processor (SIP), 288
Security Market Line, 238
segregation of duties, 209
selection bias, 267
self-attention, 323, 359, 384
Self-Attention for a Three-Token Sentence, 331
Self-Supervised Contrastive Pretraining, 364
Sensitivity-based risk limits, 161
Sequential Trade and Bayesian Learning, 282
SHAP, 375, 437
Shapley value, 372, 442
Sharpe ratio, 238, 267, 334
Short Selling and Margin Calls, 56
Sigmoid Function, 122
Sim-to-Real Gap, 409

Wulin.Suo@Queensu.ca

single-factor credit model, 190
SMOTE, 304, 337
softmax, 331
Solstice Robotics Inc., 72
Solvency II, 228
Spoofing, 286
square-root-of-time rule, 159
SR 11-7, 167, 347, 381, 446
stablecoin, 53
standard error, 268
Statistical Arbitrage Beyond Pairs, 266
Stop-loss, 169
stress testing, 58, 381
Structuring, 311
Supervised and Unsupervised Learning, 106
support vector machine (SVM), 2, 112
SupTech, 378
Suspicious Activity Report (SAR), 311
Swaps, 46
Synthetic Identity Fraud, 306
Systematic versus Idiosyncratic Risk, 237

Tangency Portfolio, 239
Tax-Loss Harvesting, 248
telematics, 223
Temporal-Difference Learning, 398
Term Structure of Interest Rates, 84
three lines of defense, 169, 317, 382, 449
Time Value of Money, 7
Time Value of Money, Revisited, 81
time-series foundation models, 146
tokenization (of assets), 53
transformer architecture, 4, 349, 372
tree-based models and ensembles, 119
TreeExplainer, 442
Triangular Arbitrage, 57
TWAP (time-weighted average price), 257
two-fund separation theorem, 239

UCI Machine Learning Repository, 458
Unit Testing, 39
Upstart Network, 372

Value at Risk (VaR), 151, 177, 252, 274, 292, 451
Value of information, 16
Vanishing-Gradient Problem, 356
Vasicek model, 190
vector autoregression (VAR), 129
Vision Transformer (ViT), 359
Visualization, 30
Volatility Clustering, 424
Volatility skew, 94
Volatility smile, 94
Volatility surface, 95
Volatility targeting, 168
Volatility-Adjusted VaR, 156
Volcker Rule, 54
VWAP (volume-weighted average price), 257

wash-sale rule, 248
Weight of Evidence (WOE), 182

XGBoost, 440

yfinance, 457
yield to maturity, 82

Zero-Shot Prompting, 336

Wulin.Suo@Queensu.ca